Signal Processing From the Ground Up

The Geometry of Frequency — From Sinusoids to a Wrist-Worn Gesture Controller

Self-Contained DSP for Audio, Images, and Motion

June 2026

Preface

There is a moment, when you first really understand the Fourier transform, that feels like a door opening in a wall you did not know was there. One day a sound is just a wiggly line on a screen, an unintelligible scribble of voltage against time. The next day you can *see* that the scribble is secretly a chord – a stack of pure tones, each with its own loudness and its own timing – and you can pick the chord apart note by note, push some notes down, lift others up, and put the sound back together changed. The wiggly line never changes. What changes is *you*. You have learned to look at the same object from a second direction, and from that direction its structure is suddenly obvious.

This book is about that second direction. It is about *frequency*: what it really is, why it is hiding inside every signal, how to measure it, how to manipulate it, and how to build hardware that does all of this in real time on a battery. The promise of the title – *The Geometry of Frequency* – is meant literally. We are going to treat signals as *vectors*, frequencies as *directions*, the Fourier transform as a *rotation of your coordinate system*, and filtering as *reshaping a shadow*. Almost everything that is usually presented as a wall of integrals and summations is, at heart, a picture. We will draw the picture first, every time, and only then write down the formula that makes the picture precise.

What this book is, and what it is not

This is a book *from the ground up*. That phrase is a promise about where we start, not a confession that we will stay shallow. We start from the ground – from a voltage wiggling on a wire, from a number arriving once every millisecond from a sensor – and we build, brick by visible brick, all the way up to spectrograms, digital filters, the FFT, sampling theory, and the signal-processing pipeline inside a real wearable device. By the end you should be able to read a research paper on gesture recognition, design a bandpass filter that actually does what you need, and write the C that runs it on a microcontroller.

It is *not* a formula reference. There are excellent ones already, and we will point to them. If all you want is the equation for the discrete Fourier transform, you can find it in thirty seconds; you do not need eight hundred pages for that. What eight hundred pages can buy you is *understanding* – the kind that lets you re-derive the equation when you have forgotten it, debug it when it misbehaves, and recognize the same idea wearing a different costume three chapters later. Rote memory is brittle. Geometry is not. If you understand that the Fourier transform projects a signal onto a family of rotating arrows, then the formula is just bookkeeping, and you will never be truly lost.

Focus

The single most important sentence in this book: **a signal is a vector, and frequency analysis is choosing a better basis to look at it in.** Almost every chapter is a variation on that theme. If a derivation ever feels like meaningless symbol-pushing, stop and ask: *what vector are we looking at, and what basis are we measuring it in?* The fog usually lifts.

Who you are

I am assuming you are smart, curious, and impatient with hand-waving – and also that you do *not* already know signal processing. You should be comfortable with the idea of a function, with basic trigonometry (sine and cosine), and with the notion that you can add and scale things. Calculus and linear algebra will appear, but I will reintroduce every piece I lean on, in the language of pictures, at the moment I need it. If you have seen an integral before, good; if it has gone hazy, the **Connection** boxes will pull you back to solid ground. You do not need to have touched a soldering iron, but I hope that by the end you will want to.

A word on prerequisites in the other direction: do not let *too much* prior exposure get in your way either. If you were taught Fourier analysis as a sequence of theorems to survive, you may have to unlearn the reflex of reaching for a formula before you have a picture. Be patient with the slow, visual openings of each chapter. They are doing the real work.

The through-line: a wrist gesture controller

A book that is only a tour of techniques is exhausting to read and easy to forget. So this book has a destination – a single concrete machine that we are secretly building the whole time. The machine is a **wrist- and forearm-worn gesture controller for playing instruments**. You wear it like a watch or a cuff; it watches how your hand and arm move and tense; and it emits **MIDI** – the universal language of electronic instruments – so that a flick of the wrist bends a note, a tap triggers a drum, a slow rotation sweeps a filter on a synthesizer. The dream is to play music with gesture, with no keyboard between you and the sound.

Why this goal? Because it touches almost everything in the book. To build it you must understand *sampling* (how fast to read the sensors), *filtering* (separating the gesture from the tremor and the noise), the *Fourier and short-time Fourier transforms* (what frequencies live in a motion, and how they change over time), *convolution and LTI systems* (the mathematics of every smoothing and shaping operation you will perform), *feature extraction and template matching* (turning a stream of numbers into the decision "that was a tap" by comparing it against patterns you recorded during a short calibration), and finally *embedded DSP* (making it all run in a few kilobytes on a chip that costs a couple of dollars). Each chapter, where it is natural, ends with a **Toward the wrist tracker** box that pulls the chapter's idea toward this build, so that the techniques never float free – they always have a job.

Toward the wrist tracker

Here is the machine we are building, named once so you can hold it in your head for eight hundred pages. A small, low-power board on the wrist and forearm carries an **IMU** (a high-rate accelerometer plus gyroscope), an array of **force-myography** (FMG) pads or force-sensitive resistors that feel the forearm tendons bulge as the fingers move, and a second high-rate accelerometer pressed to the skin to pick up the faint bio-acoustic taps of contact. Some versions add a surface-**EMG** channel for the muscles' electrical pulses. The wrist node stays deliberately thin: it samples, lightly filters, compresses, and streams the data over Bluetooth (BLE) to a **phone**, which does the heavy lifting – running a short guided calibration, building a per-user template for each finger movement, and conditioning the streams. A lightweight template match runs on the device itself for the low-latency live-play path, emitting MIDI to whatever instrument you like. By the capstone chapter you will have seen every block of this pipeline derived from first principles and built in code. We will not just admire it from a distance; we will build it.

Figure 1 sketches that split. The wrist node does as little as it can get away with – read, filter lightly, compress, send – so it stays small and runs for a long time on a coin cell. The phone, which has power and memory to spare, runs the calibration and the deeper analysis, while a small template matcher on the wrist keeps the live-play latency low.

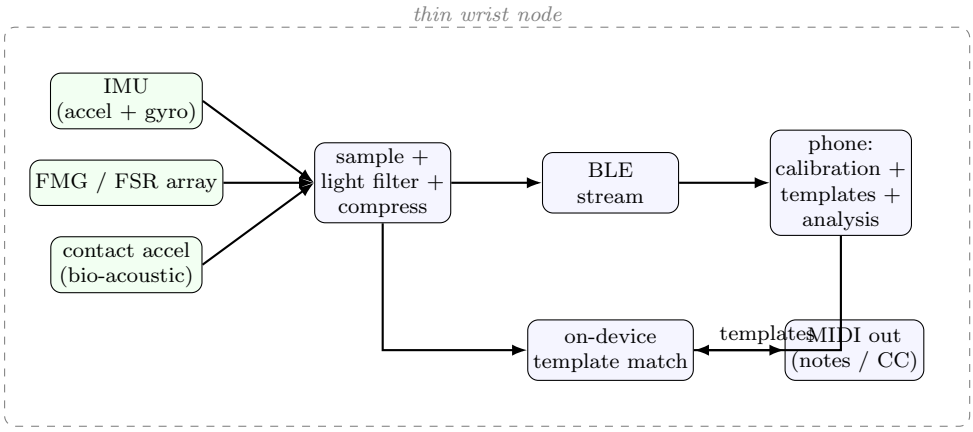


Figure 1: The capstone device, split across two computers. The wrist node senses three channels, conditions them lightly, and streams them over Bluetooth; the phone runs the per-finger calibration and the heavy offline analysis; a small template matcher on the wrist gives the live-play path its low latency and drives the MIDI output. No machine learning is involved – detection is a direct match against patterns you record during calibration.

A straight contract about sensors

Here is where I want to be completely straight with you, because the field is full of breathless marketing and it will save you months of frustration to know what is and is not possible early.

Which finger you moved is detectable from the wrist and forearm. When you flex a finger, the tendons that drive it slide and the muscles in your forearm bulge in a pattern that is specific to *that* finger. A band of force-myography (FMG) pads pressed against the forearm feels those bulges

directly; a surface-EMG electrode picks up the electrical pulses that command the same muscles. Both signals carry a usable per-finger signature. WristFlex demonstrated reliable detection of which finger pinched from a forearm FMG band; the Myo armband and the open MyoWare boards put sEMG finger sensing into many hands. So finger *movement* detection is not a fantasy – it is a solved-enough problem to build on, provided you do one thing.

That one thing is **per-finger calibration**. The tendon and muscle signatures differ from person to person and shift if the band rotates, so we do not ship a fixed model. We run a short guided enrollment: the app asks you to perform each finger movement a few times, the device records the FMG/EMG signature each time, and the phone builds a *template* (and a threshold) for each finger from *your* signals. Live detection is then a direct comparison – correlate the incoming signal against the stored templates and fire the nearest match. No machine learning, no opaque trained model: just your own recorded patterns and a distance measure. Calibration is the heart of the instrument.

Common trap

The most expensive mistake in wearable sensing is designing the *software* for capabilities the *hardware* cannot provide – but the opposite error is just as costly: under-reaching, and assuming the wrist can only see "the whole hand moved." It can see more than that, if you sense the right channel (tendons via FMG, muscle pulses via EMG) and calibrate per user. Throughout this book, whenever a technique brushes against the limits of what a sensor can physically measure, I will say so plainly, and point to the real published work so you can check me.

There is one limit I will not talk you out of, because physics is firm on it. **Continuous, per-joint finger angles cannot be recovered from a wrist IMU.** The bulk motion of the wrist simply does not contain the angle of each knuckle; the problem is underdetermined, and no clever software conjures information that never entered the device. If you need the actual joint angles – a full hand pose, every degree continuously – you need sensors on the fingers themselves: instrumented gloves or rings with their own IMUs, as in the finger-worn "Learning on the Rings" line of work. That is a clean upgrade path, and we will sketch it. But everything our instrument needs – knowing which finger moved, how hard, and when it tapped – lives comfortably on the doable side of that line.

The edge of our build over the single-modality lab papers is not a smarter algorithm; it is *multimodal fusion*. A high-rate IMU, an FMG/FSR array, and a second accelerometer used as a contact and bio-acoustic sensor (the trick behind ViBand) each see a different slice of the same gesture, and together they disambiguate what any one alone would miss – the way PLOS One (2020) combined IMU with acoustics, and MDPI Sensors (2023) counted gestures from a wrist IMU. Fuse those streams, calibrate per finger, and you have something more capable than the HR/steps/sleep wearables on the market today, which never attempt finger input at all. The result is something you can really wear and really play.

The colored boxes: a visual language

Throughout the book you will meet recurring colored boxes. They are not decoration; each one is a specific *kind* of thinking, and learning to read them is part of learning the book. Here is the legend, and you have already seen several of them in action above.

Why it works

A **Why it works** box justifies a technique. When I claim that some procedure – say, multiplying two spectra to convolve two signals – actually does what I say it does, this is where I show you the reason, usually geometric. If you only read one kind of box, read these: they are where understanding lives.

What is really happening

A **What is really happening** box strips away the notation and tells you, in plain words, the physical or geometric event behind the math. Formulas describe; these boxes explain. When a result feels like magic, this is where the magic is dismantled.

Build it (hardware)

A **Build it (hardware)** box is for hands. Wiring notes, sample-rate choices, an Arduino or Raspberry Pi code sketch, a gotcha about a particular sensor. This is where the abstractions touch metal. You can build the whole device from the contents of these boxes.

Connection

A **Connection** box links to the companion volumes – the ground-up books on *calculus*, *linear algebra*, and *probability*. Signal processing is where those three subjects meet: integrals become energy and averaging, inner products and orthogonality become frequency analysis, and randomness becomes noise. When an idea has a deeper home in one of those books, I will point you to it, so the four volumes reinforce rather than repeat each other.

You will also meet **Intuition**, **Formal**, **History**, **Common trap**, **Application**, **Try it yourself**, **Walking question**, **Why this matters**, and **Toward the wrist tracker** boxes. Their names say exactly what they hold. The *History* boxes in particular are worth your time: knowing that Fourier was nearly laughed out of the French Academy for claiming that any curve could be built from sines, or that the FFT was rediscovered because the Cold War needed to detect distant nuclear tests, makes the ideas stick in a way that no derivation can.

Intuition

Read the boxes in order the first time through, but on a second pass you can read a chapter *by box color*. Want only the intuition? Follow the green. Building hardware this weekend? Skim the orange **Build it** boxes. Reviewing for an exam? The gray **Formal** boxes are your statements of record. The colors are a navigation system, not just a mood.

How the book is organized

The book moves in waves, each wave building on the last.

- **Foundations.** What a signal is; continuous versus discrete time; and the central reframing that a signal is a *vector* in a space we can do geometry in. This is the conceptual bedrock; do not skip it even if it feels easy.

- **The geometry of frequency.** Sinusoids, phasors, and complex exponentials; what frequency really means; the Fourier series, the Fourier transform, the discrete Fourier transform, and the FFT; and the short-time Fourier transform that lets us watch frequencies change over time. This is the heart of the book.
- **Systems and filtering.** Linear time-invariant systems, convolution, impulse and frequency response, the z- and Laplace transforms, and how to design and apply real filters. This is the toolbox you will reach for constantly in the build.
- **The bridge to the real world.** Sampling and the Nyquist limit, quantization and converters, noise and random signals, spectral estimation, resampling, modulation, and a few advanced lenses (wavelets, the analytic signal). This is where clean theory meets messy reality.
- **Domains and the build.** Short tours of audio, images, and motion sensing; then a focused march through IMUs, sensing the hand, gesture detection by template matching and per-finger calibration, embedded DSP, and finally the **capstone**: assembling the wrist gesture instrument controller end to end.

The chapters are written to be read in order, but they are not a chain that snaps if you jump. If you arrive already knowing the Fourier transform and want filters, go straight to the systems chapters; the cross-references will send you back for anything you are missing.

Plots, code, and doing

This is a visual book and a hands-on book. Expect several figures in every chapter – waveforms, sums of sinusoids, magnitude and phase spectra, the rotating phasor on the unit circle, the sinc function, spectrograms, pole-zero plots, filter responses, the sampling and aliasing picture, convolution sliding past itself, two-dimensional image spectra, and the jagged traces of real IMU and EMG data. Where a short piece of Python or C makes an idea concrete, you will find it; signal processing is one of those subjects where ten lines of code can teach more than a page of prose.

To make the visual language tangible from the very first page, here is a signal and a glimpse of its frequency content side by side – the two views of the same object that this entire book is about.

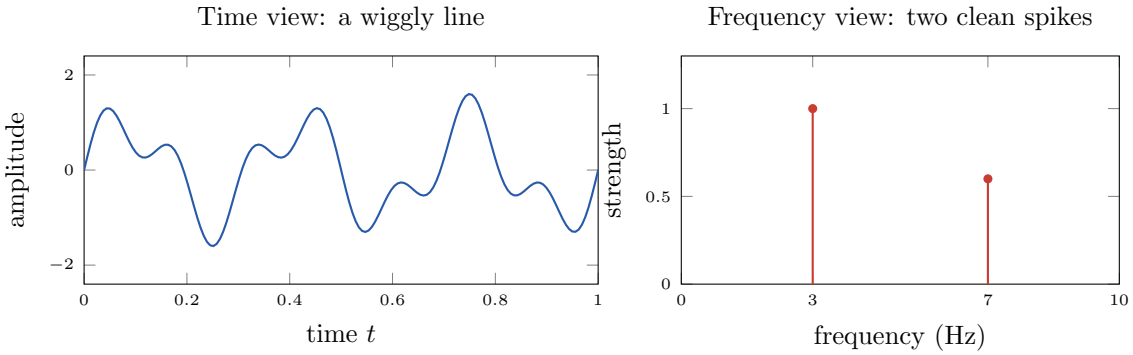


Figure 2: The whole book in one picture. On the left, a signal in the time domain: a single tangled curve. On the right, the *same* signal seen as the frequencies it is made of – here, a strong tone at 3 Hz and a weaker one at 7 Hz. Learning to move between these two pictures, in both directions, is the central skill of signal processing.

Figure 2 is the seed of everything. The left curve looks complicated; the right description is just two numbers and two heights. The Fourier transform is the machine that carries you from left to right, and the inverse transform carries you back. Most of the power in this field comes from doing your work in whichever of the two pictures makes it easy, then traveling to the other.

Try it yourself

You do not need any special hardware to start. If you have a computer with Python and NumPy, you can reproduce nearly every figure in this book and poke at it until it gives up its secrets. The **Try it yourself** boxes will suggest small experiments – generate a signal, transform it, change one number, watch what moves. Understanding in this subject is *earned at the keyboard*. Reading about the Fourier transform is like reading about swimming. Run the code.

To make the promise of Figure 2 concrete, look at how the two simple tones on the right actually *build* the tangled curve on the left. Each tone is a smooth, pure ripple; the signal is nothing more than their sum. Figure 3 shows the two ingredients separately and then stacked. Nothing here requires a formula – it is addition, done one column at a time – yet it is already the whole idea of the Fourier *series*: complicated things are sums of simple ripples.

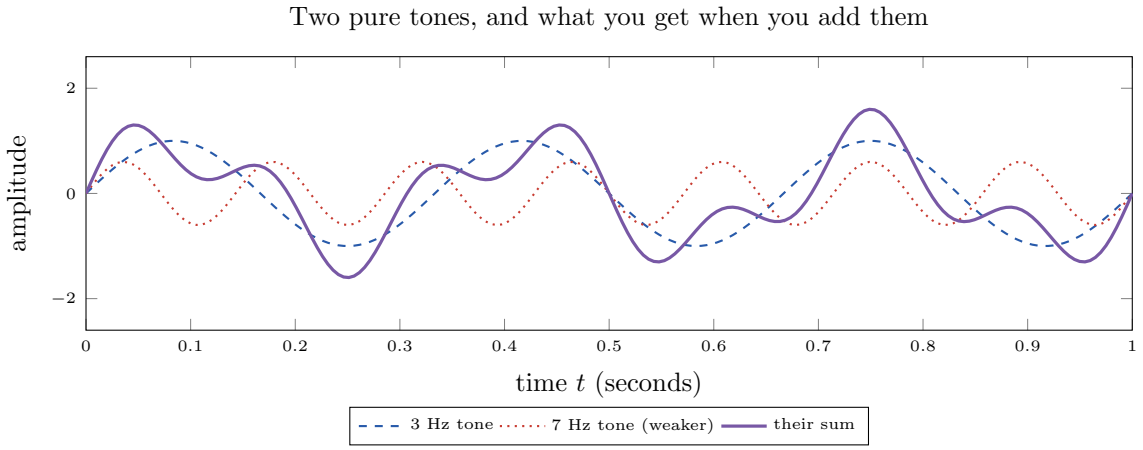


Figure 3: The same signal as in Figure 2, taken apart. The dashed curve is a clean 3 Hz sinusoid; the dotted curve is a weaker 7 Hz sinusoid; the solid purple curve is simply the two added together, point by point. The "wiggly line" was never mysterious – it was a recipe. Reading that recipe off the curve is what the Fourier transform does for us.

What is really happening

Look again at the solid curve in Figure 3 and try to *see* both tones at once: the slow, three-humps-per-second swell of the 3 Hz tone, with the faster 7 Hz ripple riding on top of it like texture on a wave. Once your eye learns to do this – to perceive a single curve as a stack of pure tones – you have begun to think in the frequency domain without any mathematics at all. The transform is just the instrument that makes this perception exact and reversible.

A second habit this book will build is reading a *spectrum* – the right-hand picture – as fluently as you read a waveform. A spectrum answers one question: *how much of each frequency is present?* Tall spike, lots of that frequency; no spike, none of it. Here is a worked reading of one, the kind we

will do constantly.

Example 0.1 (Reading a magnitude spectrum)**.** Suppose an accelerometer on the wrist, sampled while the hand performs a steady tremor plus a slow deliberate wave, produces the magnitude spectrum sketched in Figure 4. What can we say *just by looking*?

1. There is a tall spike near 1 Hz. A once-per-second oscillation is far too slow to be tremor; this is the *deliberate* hand wave. Its height tells us it dominates the motion's energy.
2. There is a second, shorter cluster near 6 Hz. Physiological hand tremor famously lives in roughly the 4 to 12 Hz band, so this bump is almost certainly involuntary tremor.
3. Everything above 10 Hz is essentially flat and small – the noise floor of the sensor, with no real signal sticking out of it.

The design consequence writes itself: to recover the intentional gesture and suppress the tremor, we want a filter that *keeps* the low-frequency spike and *rejects* the 6 Hz bump. We have not derived a single filter equation yet, and we already know what the filter must do, because we read it off the spectrum. That is the leverage this book is selling.

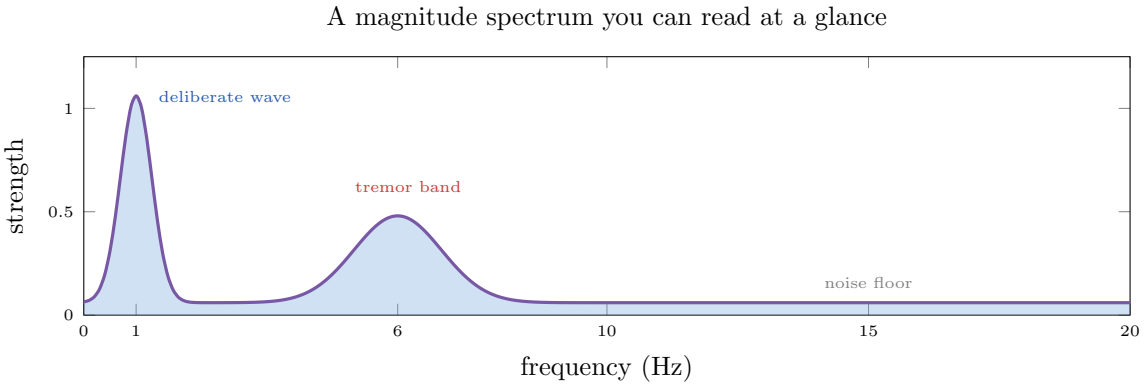


Figure 4: A made-up but realistic magnitude spectrum of wrist motion. A tall, narrow peak at low frequency is the intentional gesture; a broader, shorter bump in the 4–12 Hz region is involuntary tremor; the flat tail is sensor noise. Reading these three features – and deciding what to keep and what to throw away – is the everyday work of the signal processor.

A note on rigor

I have tried to make this book both intuitive and correct, and where those two goals pull apart I handle the tension openly rather than quietly fudging. When I simplify, I will tell you I am simplifying and where the full story lives. When a statement is true only under conditions, the conditions appear in a **Formal** box rather than being swept away. The geometry comes first because it is how human beings actually understand; the rigor comes second because it is what keeps the intuition from lying to us. You deserve both, and you will get both.

The same discipline governs the engineering. Real sensors are noisy, drift with temperature, and shift on the body. Real microcontrollers run out of cycles and memory. Real wireless links drop packets at the worst moment. I will not pretend these problems away to make the build sound easier than

it is. A device that works in the messy real world is worth more than a clean idea that only works in a slide deck, and getting there means respecting the mess.

Acknowledgment of the giants

History

The ideas in this book were paid for by extraordinary people. **Joseph Fourier** (1768–1830) claimed, to widespread disbelief, that any reasonable curve is a sum of sinusoids. **Harry Nyquist** and **Claude Shannon** told us exactly how fast we must sample to lose nothing. **James Cooley** and **John Tukey** gave us, in 1965, the fast Fourier transform that made real-time spectral analysis possible and quietly rewired the modern world. Generations of engineers and authors – Oppenheim and Schafer for their rigor, Steven Smith for his insistence that working engineers deserve clarity, and the visual explainers of our own era – shaped how these ideas can be taught. This book stands on their shoulders and tries to pass the view along.

Warm-up explorations

A preface is an odd place for exercises, so think of these less as homework and more as a way to calibrate your hands and eyes before the climb. None requires anything you have not already read on these pages; all of them reward five minutes at a keyboard or with a pencil. If they feel easy, good – they are meant to. If they feel slippery, that is exactly the muscle this book will train.

Exercise 0.1 (Add two tones by hand)**.** On graph paper (or in your head), sketch one full second of a 2 Hz sine wave and, beneath it, a 5 Hz sine wave at half the height. Now add them sample by sample at a few points – say every tenth of a second – and plot the sum. Compare your sketch to the spirit of Figure 3. Which tone sets the overall "shape," and which adds the "texture"?

Exercise 0.2 (Read a spectrum cold)**.** Cover the labels in Figure 4 and describe, in one sentence each, what the three features are and what a designer would want to do with them. Then uncover the labels and check yourself. The goal is to make spectrum-reading as automatic as reading a thermometer.

Exercise 0.3 (Name the box)**.** Flip back through this preface and, for each colored box you met, write a single phrase naming the *kind of thinking* it represents (justification, plain-language picture, hardware, limits, and so on). You are learning the book's navigation system; this is the map quiz.

Exercise 0.4 (State the sensor contract)**.** In your own words, and without looking, explain (a) why *which* finger moved *is* detectable from a forearm FMG or EMG band given per-finger calibration, and (b) why the *continuous joint angle* of each finger is *not* recoverable from a wrist IMU. Name a sensor that would close the second gap. If you can teach this to a friend, you have internalized the most important engineering lesson in the book before Chapter 1.

Exercise 0.5 (Run the swimmer's lap)**.** If you have Python and NumPy, generate the very signal of Figure 2 – `np.sin(2*np.pi*3*t) + 0.6*np.sin(2*np.pi*7*t)` on a fine time grid `t` – and plot it. Then call `np.fft.rfft` on it and plot the magnitude. You will not understand the FFT yet, and

that is fine; the point is to *see*, with your own data, the two spikes appear where the book promised they would. Keep this script. We will return to it.

How to begin

Turn the page. Read slowly at first; the early chapters plant seeds that flower much later, and time spent making the foundations *feel* obvious is repaid many times over. Keep a Python session open. When a box says *try it*, try it. When the geometry clicks – when you first *see* the rotating phasor, or watch a filter erase a hum from a recording, or read a tap straight off a wiggling accelerometer trace – you will know why this subject is worth the climb.

Welcome to the geometry of frequency. Let us go to the ground floor and start building.

Contents

Preface iii

I Signals and the Geometry of Functions 1

1 What a Signal Is 3

1.1 A signal is a function carrying information . . . 3
1.2 A gallery of real signals . . . 5
1.3 The dimensionality of the domain: 1D, 2D, and beyond . . . 6
1.4 Single channel versus multichannel . . . 6
1.5 Analog versus digital . . . 7
1.6 The five central questions of DSP . . . 10
1.7 A first clear-eyed look at the wrist controller . . . 11
1.8 Energy, power, and why we will care . . . 12
1.9 Looking ahead . . . 13
1.10 Exercises . . . 14

2 Continuous and Discrete Signals 15

2.1 Two Kinds of Time . . . 15
2.2 Sampling, as a First Idea . . . 17
2.3 The Small Zoo of Basic Signals . . . 18
2.3.1 The unit impulse . . . 18
2.3.2 The unit step . . . 18
2.3.3 The unit ramp . . . 19
2.3.4 The real exponential . . . 19
2.3.5 The sinusoid . . . 19

2.3.6 The complex exponential – a sneak preview . . . 20
2.4 Energy and Power . . . 20
2.5 Symmetry: Even and Odd . . . 21
2.6 Periodic and Aperiodic . . . 22
2.7 Transformations of the Independent Variable . . . 23
2.7.1 Time shift (delay and advance) . . . 23
2.7.2 Time reversal (reflection) . . . 23
2.7.3 Time scaling . . . 23
2.8 The Sifting Property: Building Signals from Impulses . . . 24
2.9 A Tiny Bit of Code . . . 26
2.10 Exercises . . . 27

3 The Geometry of Functions: Signals as Vectors 29
3.1 From arrows to signals . . . 29
3.2 What makes a vector space (and why signals form one) . . . 31
3.3 The inner product: one number that measures everything . . . 31
3.4 Norm, energy, and distance . . . 33
3.5 Orthogonality and orthonormal bases . . . 33
3.5.1 A worked basis change . . . 34
3.6 Projection: the geometry of "how much of this is in that" . . . 35
3.7 Sinusoids as an orthogonal basis: the seed of Fourier . . . 36
3.8 Function space and the leap to infinite dimensions . . . 38
3.9 A first numerical taste . . . 39
3.10 Toward the wrist tracker . . . 40
3.11 Summary . . . 42
3.12 Exercises . . . 42

4 Sinusoids, Phasors, and Complex Exponentials 45
4.1 What a sinusoid is, and the three numbers that pin it down . . . 45
4.2 Why cosine alone is awkward: the addition problem . . . 47
4.3 Rotation on the unit circle, and where cosine comes from . . . 47
4.4 Euler's formula: rotation written as one number . . . 48
4.5 Phasors: freezing the spin . . . 49

4.6 Two counter-rotating arrows: the negative frequency . . . 51
4.7 Complex exponentials are the eigenfunctions of LTI systems . . . 52
4.8 Discrete-time complex exponentials (a first look) . . . 53
4.9 A worked round trip in code . . . 55
4.10 Toward the wrist tracker . . . 56
4.11 History and a note on notation . . . 57
Summary . . . 58
4.12 Exercises . . . 58

II Frequency and the Fourier Family 61

5 The Idea of Frequency 63
5.1 What we are really after . . . 63
5.2 Frequency as a rate of repetition . . . 64
5.2.1 Ordinary frequency versus angular frequency . . . 64
5.3 The atoms: pure sinusoids and their three numbers . . . 65
5.4 Why decompose into frequencies at all? . . . 66
5.5 Building a waveform from sinusoids: the gentle path . . . 66
5.6 The spectrum: a signal seen edge-on . . . 68
5.7 Harmonics and timbre: why a violin is not a flute . . . 68
5.8 Beats: two close frequencies . . . 69
5.9 Complex exponentials and the meaning of negative frequency . . . 70
5.10 From a few lines to a continuous spectrum . . . 72
5.11 A worked example end to end . . . 73
5.12 Frequency for the wrist controller . . . 74
5.13 What to carry forward . . . 75
5.14 Exercises . . . 76

6 Fourier Series 77
6.1 The Big Picture: A Periodic Signal Is a Recipe . . . 77
6.1.1 The three forms you will meet . . . 78
6.2 Computing Coefficients Is Projection . . . 78
6.2.1 The harmonics are orthogonal . . . 79

6.2.2 Projection gives the coefficients . . . 80
6.3 The Complex-Exponential Form . . . 81
6.3.1 Translating between the forms . . . 82
6.4 Worked Example: The Square Wave . . . 82
6.5 Convergence and the Gibbs Phenomenon . . . 84
6.6 Worked Example: The Triangle Wave . . . 85
6.7 The Line Spectrum: What a Periodic Signal Looks Like in Frequency . . . 86
6.8 Energy and Parseval's Theorem . . . 87
6.9 Computing Coefficients in Code . . . 88
6.10 A Note on Real Periodic Signals from Sensors . . . 89
6.11 Summary . . . 90
6.12 Exercises . . . 90

7 The Fourier Transform 93

7.1 From series to transform: letting the period explode . . . 93
7.2 The continuous-time Fourier transform . . . 95
7.3 Magnitude and phase: reading a spectrum . . . 96
7.4 A worked transform: the rectangle and the sinc . . . 97
7.5 More essential transforms . . . 99
7.5.1 The impulse and the constant . . . 99
7.5.2 The complex exponential and the cosine . . . 99
7.5.3 The one-sided decaying exponential . . . 100
7.5.4 The Gaussian: the shape that transforms to itself . . . 100
7.6 The uncertainty principle: you cannot be sharp in both worlds . . . 101
7.7 Parseval and the conservation of energy . . . 103
7.8 A computational sanity check . . . 104
7.9 What to carry forward . . . 105
7.10 Exercises . . . 105

8 The Discrete Fourier Transform 107

8.1 From the continuous transform to a finite sum . . . 107
8.2 The twiddle factor and the roots of unity . . . 109
8.3 The DFT as a matrix . . . 109

8.4 Why it inverts: orthogonality of the rows . . . 110
8.5 Reading the output: bins, resolution, and units . . . 111
8.6 Two worked DFTs by hand . . . 113
8.7 The hidden assumption: the DFT is circular . . . 114
8.8 Spectral leakage . . . 115
8.9 Parseval: energy is preserved . . . 116
8.10 Cost, and a look ahead to the FFT . . . 117
8.11 Exercises . . . 119

9 The Fast Fourier Transform 121
9.1 Recalling the DFT, and counting its cost . . . 121
9.2 Where the redundancy hides . . . 123
9.3 Divide and conquer: split by even and odd . . . 124
9.4 The butterfly . . . 125
9.5 Bit-reversal: why the inputs come in scrambled . . . 127
9.6 It computes the same numbers – a small proof and a check . . . 128
9.7 Using a real FFT: reading the output . . . 129
9.8 Zero-padding and the difference between resolution and interpolation . . . 130
9.9 A short NumPy example . . . 131
9.10 The inverse FFT comes for free . . . 132
9.11 Toward the wrist tracker . . . 133
9.12 Practical notes and gotchas . . . 134
9.13 Exercises . . . 135

10 The Short-Time Fourier Transform and Spectrograms 137
10.1 Why the plain Fourier transform is not enough . . . 137
10.2 The idea: look through a moving window . . . 139
10.2.1 Two ways to read the same equation . . . 139
10.3 Windows: the shape of the porthole . . . 140
10.4 The spectrogram: a picture of the STFT . . . 141
10.5 The central tradeoff: time versus frequency resolution . . . 143
10.5.1 Wideband versus narrowband . . . 144
10.6 From integral to algorithm: the discrete STFT . . . 145

10.6.1 Overlap and the hop size . . . 145
10.6.2 Inverting the STFT: overlap-add . . . 147
10.7 Worked example: reading numbers off the discrete STFT . . . 147
10.8 Applications: speech, music, motion . . . 148
10.9 Toward the wrist tracker . . . 148
10.10 Summary . . . 150
10.11 Exercises . . . 150

11 Properties of the Fourier Transform and the Convolution Theorem 153
11.1 The conventions we will use . . . 154
11.2 Linearity: the transform is a linear map . . . 154
11.3 Time shift: delay becomes a phase ramp . . . 155
11.4 Frequency shift (modulation): multiply by a phasor, slide the spectrum . . . 156
11.4.1 Real modulation and the cosine . . . 156
11.5 Scaling: stretch in time, squeeze in frequency . . . 157
11.6 Duality: the transform is almost its own inverse . . . 158
11.7 A small table, now understood not memorized . . . 159
11.8 Convolution, recalled geometrically . . . 160
11.9 The convolution theorem . . . 161
11.9.1 The dual: multiplication in time is convolution in frequency . . . 162
11.10 Fast filtering: the convolution theorem at work . . . 163
11.11 Correlation: convolution's close cousin . . . 164
11.12 Worked examples . . . 165
11.13 Exercises . . . 166

III Systems and Filtering 169

12 Linear Time-Invariant Systems 171
12.1 Systems: the input–output view . . . 171
12.2 Linearity: scaling and superposition . . . 172
12.3 Time invariance: the system has no clock . . . 173
12.4 Putting them together: LTI . . . 174
12.5 The impulse: a single tap . . . 175

12.6 The impulse response and the convolution sum . . . 176
12.7 FIR and IIR: finite versus infinite memory . . . 178
12.8 Causality and stability . . . 178
12.9 Difference equations: how systems are really run . . . 180
12.10 Complex exponentials are the eigenfunctions of LTI systems . . . 181
12.11 Block diagrams: drawing systems . . . 184
12.12 Why this matters for the wrist controller . . . 185
12.13 A continuous-time footnote . . . 186
12.14 Summary . . . 186
12.15 Exercises . . . 187

13 Convolution **189**

13.1 The one idea: build any signal out of impulses . . . 189
13.2 From impulse response to convolution . . . 190
13.3 Flip, shift, multiply, sum — the moving picture . . . 192
13.4 Worked example: the moving average . . . 194
13.5 Worked example: a digital echo . . . 195
13.6 Properties — and why each one is geometrically obvious . . . 195
13.7 Convolution is polynomial (and integer) multiplication . . . 197
13.8 Edges and the cost of convolution . . . 197
13.9 A glance ahead: 2D convolution for images . . . 199
13.10 Exercises . . . 200

14 Impulse and Frequency Response **201**

14.1 The one experiment that defines everything . . . 201
14.1.1 It is literally the DTFT of h . . . 202
14.2 Magnitude and phase: what the complex number means . . . 203
14.2.1 Decibels: why gain is plotted on a log scale . . . 204
14.3 A first worked example: the two-tap averager . . . 204
14.3.1 The mirror image: a two-tap difference . . . 205
14.4 Group delay: the phase slope you can feel . . . 206
14.5 Reading a frequency response like a map . . . 207
14.6 How a system reshapes a spectrum . . . 208

14.7 Resonance: when a system rings . . . 209
14.8 Putting numbers in: a worked FIR low-pass . . . 211
14.9 The two views, side by side . . . 212
14.10Summary . . . 214
14.11Exercises . . . 214

15 The Z-Transform and the Laplace Transform 217

15.1 Why Fourier is not enough . . . 217
15.2 From rotating to spiraling: defining the z-transform . . . 218
15.2.1 A worked transform: the right-sided exponential . . . 219
15.3 The region of convergence, geometrically . . . 219
15.4 Poles, zeros, and the rational picture . . . 220
15.4.1 Difference equation → transfer function, mechanically . . . 221
15.5 Stability and causality, read off the plane . . . 222
15.6 The frequency response is a walk around the unit circle . . . 223
15.7 The Laplace transform: the continuous twin . . . 226
15.7.1 Stability in the s-plane . . . 226
15.8 Transfer functions and block diagrams . . . 227
15.9 Inverting the transform: partial fractions . . . 228
15.10Toward the wrist tracker . . . 229
15.11Summary . . . 230
15.12Exercises . . . 230

16 Designing Filters 233

16.1 What a filter is supposed to do . . . 233
16.1.1 Why the perfect mask is impossible . . . 234
16.2 Two families: FIR and IIR . . . 235
16.2.1 The tradeoffs, plainly . . . 236
16.3 Specifying a filter before you design it . . . 237
16.4 Linear phase: the FIR superpower . . . 238
16.5 FIR design by the window method . . . 239
16.5.1 Truncation, and the ripples it causes . . . 239
16.5.2 Why a window helps, and which one . . . 240

16.5.3 Worked design: a low-pass with the Hamming window . . . 241
16.6 IIR design: borrow from the analog masters . . . 243
16.6.1 The Butterworth prototype: maximally flat . . . 244
16.6.2 The bilinear transform: warping analog into digital . . . 245
16.6.3 Worked design: a Butterworth low-pass in three lines . . . 246
16.7 FIR versus IIR, decided by the spectrum plot . . . 247
16.8 Design checklist . . . 249
16.9 Exercises . . . 249

17 Practical Filtering **251**
17.1 Streaming: the signal that never ends . . . 252
17.1.1 What a causal filter is allowed to know . . . 252
17.2 Sample-by-sample versus block processing . . . 253
17.3 Fast convolution: filtering with the FFT . . . 254
17.3.1 The wrap-around problem . . . 254
17.3.2 Overlap-add . . . 255
17.3.3 Overlap-save . . . 256
17.4 Implementing the filter: direct forms . . . 257
17.4.1 Direct Form I and II . . . 257
17.4.2 Cascade of biquads: the practical default . . . 258
17.5 Fixed-point and numerical issues . . . 259
17.5.1 What fixed point is . . . 260
17.5.2 The three things that bite . . . 260
17.6 Latency: the clock you cannot cheat . . . 261
17.6.1 Group delay: how late the filter makes the signal . . . 262
17.6.2 IIR buys low latency with phase distortion . . . 262
17.6.3 The full latency budget . . . 263
17.6.4 Zero-phase filtering — and why you cannot use it live . . . 264
17.7 Moving average: the workhorse smoother . . . 264
17.7.1 The running-sum trick: $O(1)$ per sample . . . 265
17.7.2 The exponential moving average (one-pole IIR) . . . 266
17.8 Median filter: killing spikes without smearing edges . . . 266
17.9 Putting a real sensor pipeline together . . . 268

17.10Summary . . . 269
17.11Exercises . . . 269

IV Sampling and Real Signals 271

18 Sampling and the Nyquist Theorem 273

18.1 What sampling actually does . . . 273
18.1.1 Sampling as multiplication by an impulse train . . . 274
18.2 The spectrum-replication view . . . 274
18.3 Band-limiting, Nyquist, and the Sampling Theorem . . . 275
18.4 Aliasing: when the copies collide . . . 276
18.4.1 Folding: where the alias actually lands . . . 277
18.5 Anti-alias filtering: stopping the lie before it happens . . . 279
18.6 Reconstruction: rebuilding the continuous signal . . . 280
18.7 Choosing a sample rate in practice . . . 282
18.8 Bandpass sampling (a useful surprise) . . . 284
18.9 Exercises . . . 285

19 Quantization, ADCs, and DACs 287

19.1 The vertical grid: from a value to a code . . . 287
19.2 Quantization error as noise . . . 289
19.3 Signal-to-noise ratio and the 6 dB-per-bit rule . . . 291
19.4 Seeing and hearing quantization . . . 292
19.5 How a real ADC works . . . 294
19.5.1 Successive approximation (SAR): binary search in hardware . . . 294
19.5.2 Sigma-delta: trade speed for resolution by oversampling . . . 295
19.6 Dithering: adding noise to get resolution . . . 297
19.7 From bits back to the world: the DAC . . . 298
19.8 Reading a real sensor through an ADC . . . 298
19.9 Summary . . . 301
19.10Exercises . . . 301

20 Noise, SNR, and Random Signals 303

20.1 Signal plus noise: the basic picture . . . 303
20.1.1 We cannot write down the noise, so we describe its habits . . . 304
20.2 Mean, power, and variance of a random signal . . . 305
20.2.1 Stationarity and ergodicity: when one recording is enough . . . 305
20.3 Autocorrelation: how a signal relates to its own past . . . 306
20.4 Power spectral density: where the power lives in frequency . . . 307
20.5 White noise and colored noise . . . 308
20.5.1 Coloring noise is just filtering it . . . 310
20.6 Signal-to-noise ratio (SNR) . . . 310
20.7 Averaging to beat down noise . . . 311
20.7.1 Two flavors: ensemble averaging vs. smoothing . . . 312
20.8 Matched filtering: the optimal detector . . . 313
20.9 Sensor noise models for the wrist tracker . . . 314
20.9.1 IMU noise: white rate noise and the random walk it integrates to . . . 315
20.9.2 EMG noise: where the real signal lives, and what fights it . . . 316
20.10 What to carry forward . . . 318
20.11 Exercises . . . 318

21 Spectral Estimation **321**

21.1 What "the spectrum" even means for a random signal . . . 321
21.1.1 The two difficulties . . . 322
21.2 The periodogram: the obvious estimator . . . 323
21.2.1 It is (nearly) unbiased — the good news . . . 323
21.2.2 Its variance does not go away — the disaster . . . 324
21.3 The cure: average independent looks (Bartlett) . . . 325
21.4 Windowing and leakage, revisited for estimation . . . 326
21.4.1 Leakage effect one: bias and the smeared truth . . . 327
21.4.2 Leakage effect two: tapering for the average . . . 327
21.5 Welch's method: overlap, window, average . . . 327
21.5.1 Welch in code . . . 329
21.6 Resolution versus variance, made quantitative . . . 330
21.7 Worked example: choosing parameters end to end . . . 331
21.8 Spectral analysis of real audio and motion signals . . . 332

21.9 Beyond the periodogram: a brief map . . . 334
21.10Exercises . . . 335

V Multirate, Modulation, and Time-Frequency 337

22 Decimation, Interpolation, and Resampling 339

22.1 What it means to change a sample rate . . . 339
22.2 Decimation: the geometry of throwing samples away . . . 340
22.2.1 What downsampling does to the spectrum . . . 340
22.2.2 The fix: filter, then decimate . . . 341
22.2.3 You only need to compute the outputs you keep . . . 342
22.3 Interpolation: the geometry of inventing samples . . . 342
22.3.1 Step 1: zero-stuffing (the expander) . . . 342
22.3.2 What zero-stuffing does to the spectrum . . . 343
22.3.3 Step 2: the anti-imaging (interpolation) filter . . . 343
22.3.4 Interpolation kernels you can picture . . . 344
22.4 Rational resampling: changing rate by L/M . . . 344
22.5 Polyphase: doing it without the waste . . . 345
22.5.1 The idea in one picture . . . 345
22.5.2 The decomposition, written out . . . 346
22.5.3 A minimal polyphase interpolator in code . . . 347
22.6 Arbitrary and time-varying rates . . . 347
22.7 Why this matters for fusing sensors at different rates . . . 348
22.8 Putting the pieces together . . . 350
22.9 Exercises . . . 350

23 Modulation and I/Q Signals 353

23.1 The one idea: multiplying by a sinusoid shifts the spectrum . . . 353
23.2 Amplitude modulation: writing on the height of a carrier . . . 355
23.3 Frequency modulation: writing on the rotation rate . . . 356
23.4 The analytic signal: a preview . . . 358
23.5 I/Q: the complex baseband . . . 359
23.5.1 Reconstructing the real signal from I/Q . . . 360

23.6 Why I/Q is everywhere . . . 360
23.7 Demodulation: getting the message back . . . 362
23.7.1 Coherent (synchronous) demodulation . . . 363
23.7.2 Envelope detection (noncoherent) . . . 363
23.7.3 FM demodulation . . . 363
23.8 A worked numeric example . . . 364
23.9 Summary . . . 365
23.10Exercises . . . 365

24 Wavelets and Multiresolution 369

24.1 The tyranny of a single window . . . 369
24.2 One little wave, stretched and shifted . . . 371
24.3 The continuous wavelet transform . . . 372
24.4 From continuous to discrete: the dyadic grid . . . 372
24.5 The scaling function: where the leftovers go . . . 373
24.6 Haar: the wavelet you can do by hand . . . 374
24.7 The filter-bank view: it is just filter, downsample, repeat . . . 376
24.8 Wavelets for compression and denoising . . . 377
24.9 Transients, onsets, and the wrist controller . . . 378
24.10Exercises . . . 380

25 The Hilbert Transform and the Analytic Signal 383

25.1 The shadow problem: recovering the arrow from its projection . . . 383
25.2 Defining the Hilbert transform by what it does to frequencies . . . 384
25.2.1 The time-domain face: convolution with $1/(\pi t)$ . . . 386
25.3 The analytic signal: stapling the shadow to its partner . . . 386
25.4 Instantaneous amplitude, phase, and frequency . . . 388
25.4.1 A worked numeric example: a chirp . . . 388
25.4.2 A jump-free instantaneous-frequency formula . . . 389
25.5 Computing it on a real (sampled) signal . . . 390
25.6 Why this matters: demodulation and envelope following . . . 392
25.7 Connection back to I/Q and to projection . . . 393
25.8 Exercises . . . 394

VI Audio Signal Processing **395**

26 Digital Audio Fundamentals **397**

26.1 From a pressure wave to a number . . . 397
26.2 What the ear demands: the hearing range . . . 399
26.3 Choosing the sample rate . . . 400
26.4 Bit depth: how finely we measure each sample . . . 402
26.5 Clipping, full scale, and headroom . . . 403
26.6 Loudness, decibels, and dBFS . . . 405
26.7 The audio spectrum and octaves . . . 406
26.8 Build it: record and play on a Raspberry Pi . . . 407
26.9 Channels, interleaving, and file formats . . . 408
26.10 Real limits, and the bridge to the wrist controller . . . 409
Exercises . . . 410

27 Audio Features **413**

27.1 What a feature is, geometrically . . . 413
27.2 Short-time energy: how loud, right now . . . 414
27.3 Zero-crossing rate: how often it flips sign . . . 416
27.4 Spectral shape: centroid, spread, rolloff . . . 417
27.5 Spectral flux: how fast the spectrum is changing . . . 418
27.6 The mel scale: hearing as the ear does . . . 419
27.7 MFCCs, derived step by step . . . 421
27.8 Chroma: folding the spectrum onto the twelve pitch classes . . . 423
27.9 Pitch detection: finding the fundamental period . . . 424
27.10 Onset detection: when did something happen . . . 426
27.11 Putting it together: a feature pipeline on a clip . . . 427
27.12 The same features on motion signals . . . 428
27.13 Summary . . . 429
27.14 Exercises . . . 430

28 Audio Effects and Synthesis **433**

28.1 The three knobs: amplitude, spectrum, time . . . 433
28.2 Amplitude effects: gain, tremolo, and distortion . . . 434

28.2.1 Gain and tremolo . . . 434
28.2.2 Distortion: bending the line . . . 435
28.3 Spectral effects: EQ as everyday filtering . . . 436
28.4 Delay: the seed of echo, reverb, flanging, and chorus . . . 437
28.4.1 Single echo and the feedforward comb filter . . . 437
28.4.2 Repeating echoes and the feedback comb . . . 438
28.4.3 The all-pass filter: delay without coloration . . . 439
28.4.4 Reverb: stacking combs and all-passes . . . 439
28.4.5 Flanger and chorus: modulating the delay length . . . 440
28.5 Synthesis I: additive — building timbre by stacking sinusoids . . . 440
28.6 Synthesis II: subtractive — carving timbre out of a rich source . . . 441
28.7 Synthesis III: FM — rich timbre from a wobbling phase . . . 442
28.8 Synthesis IV: wavetable — play back one stored cycle . . . 444
28.9 The envelope: shaping a sound in time (ADSR) . . . 445
28.10 Putting it together: a complete synth voice . . . 446
28.11 Toward the wrist tracker: gesture as a control signal . . . 447
28.12 Summary . . . 448
28.13 Exercises . . . 448

29 Music and Instrument Signals 451

29.1 A note is not a frequency – it is a spectrum with a shape . . . 451
29.2 The envelope: attack, decay, and why a piano is not an organ . . . 453
29.3 Inharmonicity: when the ladder bends . . . 455
29.4 Pitch and tuning: from frequency to note number . . . 455
29.5 Polyphony: more than one note at once . . . 457
29.6 MIDI: music as a stream of bytes . . . 458
29.7 Latency: the unforgiving budget of a playable instrument . . . 460
29.8 Setting the musical target for the capstone . . . 463
29.9 Exercises . . . 464

VII Image Signal Processing 465

30 Images as Two-Dimensional Signals 467

30.1 From one axis to two . . . 467
30.2 What a pixel really is . . . 468
30.2.1 Color: three signals stacked . . . 469
30.3 Sampling and resolution in two dimensions . . . 470
30.4 Brightness and contrast . . . 472
30.5 The histogram: a signal's value distribution . . . 472
30.5.1 Histogram stretching and equalization . . . 473
30.6 Point operations: one pixel at a time . . . 475
30.6.1 Gamma: the most important nonlinear point operation . . . 476
30.6.2 Thresholding . . . 476
30.7 The jump from 1-D to 2-D, made precise . . . 477
30.8 A worked example . . . 478
30.9 Summary . . . 480
30.10Exercises . . . 480

31 The 2D Fourier Transform and Image Filtering 483
31.1 Two-dimensional waves: the new basis . . . 483
31.2 The 2D DFT . . . 485
31.2.1 The transform separates into rows then columns . . . 485
31.3 Reading a 2D spectrum . . . 486
31.4 2D convolution . . . 487
31.4.1 The convolution theorem, in 2D . . . 488
31.5 Low-pass filtering: blurring . . . 489
31.5.1 The box blur, and why it is mediocre . . . 489
31.5.2 The Gaussian blur . . . 489
31.6 Separable filters: the same result, far cheaper . . . 490
31.7 High-pass filtering: edges and sharpening . . . 492
31.7.1 The Laplacian: a second-derivative edge finder . . . 492
31.7.2 Unsharp masking: sharpening . . . 492
31.8 Filtering in the frequency domain . . . 493
31.9 A worked numeric example . . . 495
31.10Toward the wrist tracker . . . 496
31.11Summary . . . 496

Exercises . . . 497

32 Edges, Features, and the 2D Convolution 499

32.1 An edge is a place where brightness changes fast . . . 499

32.1.1 Discrete derivatives: differences between neighbors . . . 500

32.2 The 2D convolution, concretely . . . 501

32.2.1 Boundaries, separability, and cost . . . 502

32.3 From differences to the Sobel operator . . . 503

32.4 Canny: turning a gradient into clean edges . . . 504

32.5 Blurring versus differentiating: two faces of one operation . . . 505

32.6 Beyond edges: corners and blobs . . . 507

32.6.1 Corners: the structure tensor . . . 507

32.6.2 Blobs: the Laplacian of Gaussian across scale . . . 508

32.7 The 2D convolution as the heart of a CNN . . . 508

32.7.1 Learned filters rediscover the classics . . . 509

32.8 Carrying the idea to the wrist . . . 510

32.9 Summary . . . 511

32.10Exercises . . . 512

33 Image Transforms and Compression 515

33.1 Why a transform enables compression . . . 515

33.2 From DFT to DCT . . . 516

33.2.1 The DCT basis functions, drawn . . . 518

33.3 Energy compaction: the property that matters . . . 519

33.4 The 2D DCT . . . 520

33.5 JPEG, end to end . . . 521

33.5.1 Stage 1: colour transform and subsampling . . . 521

33.5.2 Stage 2: block DCT . . . 521

33.5.3 Stage 3: quantization – where the bits are lost . . . 522

33.5.4 Stage 4: zigzag, run-length, and entropy coding . . . 522

33.5.5 Decoding and the artifacts . . . 524

33.6 A word on wavelets: JPEG2000 . . . 524

33.7 Code: a DCT-based compressor in a few lines . . . 525

33.8 Toward the wrist tracker . . . 527
33.9 Exercises . . . 528

VIII Sensing Motion and the Hand 531

34 Inertial Sensors: Accelerometers, Gyroscopes, Magnetometers 533

34.1 Three sensors, three different physical questions . . . 533
34.2 The accelerometer: a mass on springs . . . 534
34.2.1 The mechanical picture . . . 534
34.2.2 How MEMS makes it tiny . . . 535
34.2.3 Tilt sensing: getting angles from gravity . . . 536
34.3 The gyroscope: rotation through the Coriolis effect . . . 536
34.4 The magnetometer: a chip-scale compass . . . 538
34.5 Units, ranges, and what the raw numbers mean . . . 538
34.5.1 Noise, bias, and drift – the three error families . . . 539
34.6 Reading an IMU over I2C . . . 540
34.7 What wrist motion actually looks like . . . 541
34.8 What the wrist can and cannot read . . . 542
34.9 A worked end-to-end example . . . 545
34.10Summary . . . 545
34.11Exercises . . . 546

35 Calibration and Sensor Models 549

35.1 Why a raw reading is not the truth . . . 549
35.2 The standard sensor error model . . . 550
35.2.1 Bias: a finger on the scale . . . 550
35.2.2 Scale factor: the wrong gain . . . 551
35.2.3 Misalignment and cross-axis coupling . . . 551
35.2.4 Temperature drift: the model breathes . . . 552
35.2.5 Random walk and the noise terms . . . 553
35.3 The big idea: a still sensor should trace a sphere . . . 554
35.4 The ellipsoid fit . . . 555
35.4.1 The simplest version: the bounding box . . . 556

35.4.2 Magnetometer: hard iron and soft iron . . . 556
35.5 Gyroscope bias estimation . . . 558
35.5.1 Why gyro bias is the cardinal sin . . . 558
35.5.2 Estimating the bias: average while still . . . 558
35.6 Allan variance: reading a sensor's whole noise story . . . 559
35.6.1 What it measures . . . 559
35.7 Why calibration must come before fusion . . . 561
35.8 Worked example: separating bias from drift . . . 563
35.9 Exercises . . . 564

36 Orientation Estimation and Sensor Fusion 565
36.1 What orientation is, geometrically . . . 566
36.1.1 Euler angles: three turns in a row . . . 566
36.1.2 The gimbal-lock trap . . . 567
36.1.3 Rotation matrices . . . 567
36.1.4 Quaternions: the practitioner's choice . . . 568
36.2 Why each sensor is not enough alone . . . 569
36.2.1 The gyroscope: smooth but drifting . . . 569
36.2.2 Accelerometer and magnetometer: noisy but absolute . . . 570
36.3 The complementary filter . . . 571
36.4 The Kalman filter as optimal fusion . . . 573
36.5 The Madgwick and Mahony filters . . . 574
36.6 Getting a stable wrist orientation . . . 576
36.7 Exercises . . . 578

37 Sensing the Hand from the Wrist: EMG, Force Myography, and Bio-Acoustics 581
37.1 Why the wrist is a hard place to listen from . . . 582
37.1.1 Degrees of freedom: what you are trying to estimate . . . 582
37.2 The IMU ceiling: what motion sensing alone can and cannot do . . . 583
37.3 Surface EMG: tapping the muscles' command lines . . . 584
37.3.1 The signal and its spectrum . . . 584
37.3.2 What EMG achieves – and why calibration is the whole game . . . 586
37.4 Force myography: sensing the bulge, not the voltage . . . 588

37.4.1 The signal: slow, large, robust . . . 588
37.5 Bio-acoustics: listening to the body's mechanical hum . . . 590
37.6 The decision table . . . 591
37.6.1 A decision flow for the builder . . . 592
37.7 Combining modalities: the realistic capstone . . . 593
37.8 Exercises . . . 594

38 Gesture and Activity Recognition **597**
38.1 The recognition pipeline, end to end . . . 597
38.2 Windowing and segmentation . . . 599
38.2.1 How long should a window be? . . . 599
38.2.2 Event segmentation: finding where a gesture is . . . 600
38.3 Time-domain features . . . 601
38.4 Frequency-domain features . . . 603
38.5 A worked example: four wrist gestures . . . 604
38.6 Classification . . . 605
38.7 Template matching and dynamic time warping . . . 607
38.8 Evaluating a recognizer . . . 608
38.8.1 Accuracy and its lies . . . 608
38.8.2 The confusion matrix . . . 609
38.8.3 The cardinal sin: leaky evaluation . . . 609
38.8.4 Cross-subject generalization: the number that hurts . . . 610
38.9 What the wrist can and cannot sense . . . 611
38.10 Putting it together: a reference recognizer . . . 613
38.11 Summary . . . 614
38.12 Exercises . . . 614

39 Machine Learning on Signals **617**
39.1 Classification as carving up feature space . . . 617
39.2 Logistic regression: a soft hyperplane . . . 619
39.2.1 Training: maximize the probability of the data . . . 620
39.2.2 Regularization: the first guard against overfitting . . . 620
39.3 Support vector machines: the widest street . . . 620

39.3.1 The kernel trick: curved boundaries for free . . . 621
39.4 Random forests: many simple questions . . . 622
39.5 When to hand-craft features and when to learn them . . . 623
39.6 Small neural networks . . . 623
39.6.1 Softmax for several gestures . . . 623
39.7 1-D convolutional networks on raw windows . . . 624
39.8 Evaluating without fooling yourself: train, validation, test . . . 625
39.8.1 Cross-validation when data is scarce . . . 626
39.9 The cross-subject gap (the one that bites) . . . 626
39.10 Beyond accuracy: confusion and thresholds . . . 627
39.11 On-device inference on a microcontroller . . . 627
39.12 Data collection and labeling for a gesture set . . . 629
39.13 Exercises . . . 630

IX Building It: Embedded DSP 633

40 Embedded Signal Processing on Arduino and Raspberry Pi 635
40.1 The shape of the problem: streaming, not batch . . . 636
40.2 Getting samples in: timers, ADCs, and the enemy called jitter . . . 636
40.2.1 Where samples come from . . . 636
40.2.2 What jitter does to a spectrum . . . 638
40.3 Ring buffers: a finite window onto an infinite stream . . . 639
40.4 Numbers on a small chip: fixed-point vs floating-point . . . 640
40.4.1 What floating-point costs . . . 640
40.4.2 What fixed-point is . . . 640
40.4.3 A practical rule of thumb . . . 641
40.5 Running real DSP: filters and FFTs on an MCU vs a Pi . . . 642
40.5.1 FIR and IIR filters on the MCU . . . 642
40.5.2 The FFT: CMSIS-DSP on the MCU . . . 642
40.5.3 The Pi side: NumPy and SciPy . . . 643
40.6 Getting answers out: serial, BLE, and MIDI . . . 644
40.7 Power and real-time budgets . . . 645

40.7.1 The real-time budget, made concrete . . . 645
40.7.2 The power budget . . . 645
40.8 Arduino vs Raspberry Pi for the wrist controller . . . 646
40.9 The capstone: a buildable finger-gesture controller . . . 647
40.9.1 Bill of materials and wiring . . . 648
40.9.2 The pipeline, channel by channel . . . 648
40.9.3 The per-finger calibration routine – the heart of it . . . 649
40.9.4 Live template matching . . . 650
40.9.5 Latency budget . . . 650
40.9.6 MIDI mapping . . . 651
40.9.7 Failure modes . . . 651
40.9.8 The upgrade path . . . 651
40.10Exercises . . . 652

41 Capstone: A Wrist-Worn Gesture Controller for Instruments 655
41.1 What the wrist reveals, and the one thing it hides . . . 655
41.2 The pipeline, in one picture . . . 657
41.2.1 Where the work runs: thin wrist node, phone does the heavy lifting . . . 658
41.3 Choosing the hand sensors . . . 659
41.4 Sampling: how fast, and why . . . 660
41.5 Conditioning and de-noising . . . 661
41.6 Calibration: the heart of the instrument . . . 662
41.6.1 Step one: per-channel baseline and scale . . . 662
41.6.2 Step two: per-finger enrollment and templates . . . 662
41.6.3 Step three: IMU calibration . . . 663
41.7 Windowing and feature extraction . . . 663
41.7.1 Why a small feature set beats raw samples . . . 664
41.8 Template matching: detection without a learned model . . . 665
41.8.1 Smoothing the decision: from frame labels to finger events . . . 666
41.9 Sensor fusion: the IMU as the expressive axis . . . 667
41.10Latency budgeting . . . 668
41.11MIDI mapping . . . 668
41.12Bill of materials . . . 670

41.13Accuracy, difficulty, and failure modes . . . 670
41.14Upgrade path . . . 671
41.15Exercises . . . 672

X Mathematical Background **675**

42 Appendix: Complex Numbers, Linear Algebra, and Calculus for DSP **677**

42.1 Why this toolkit, and how the pieces fit . . . 677
42.2 Complex numbers and Euler's formula . . . 678
42.2.1 A number that is also a point . . . 678
42.2.2 Arithmetic: addition slides, multiplication rotates . . . 678
42.2.3 Polar form: magnitude and angle . . . 679
42.2.4 Euler's formula: the heart of the matter . . . 680
42.2.5 Conjugate, magnitude squared, and division . . . 681
42.3 Vectors, matrices, inner products, bases, and eigenvectors . . . 682
42.3.1 A signal is a vector . . . 682
42.3.2 Inner product: how much two signals agree . . . 682
42.3.3 Orthogonality and orthonormal bases . . . 683
42.3.4 Matrices: linear maps on signals . . . 684
42.3.5 Eigenvectors: the directions a system leaves alone . . . 684
42.4 The calculus DSP needs . . . 685
42.4.1 The integral as a continuous sum . . . 686
42.4.2 The few rules we actually use . . . 686
42.4.3 The delta function: an infinitely thin spike . . . 687
42.5 Just enough probability for noise . . . 688
42.5.1 Random variables, mean, and variance . . . 688
42.5.2 The Gaussian, and white noise . . . 688
42.5.3 Signal-to-noise ratio . . . 689
42.6 Exercises . . . 690

Part I

Signals and the Geometry of Functions

Chapter 1

What a Signal Is

Before we touch a single Fourier integral, before we sample anything or design a filter, we need to agree on what we are even looking at. The whole edifice of signal processing rests on one deceptively simple object: the *signal*. Get a clear, geometric picture of what a signal is, and everything that follows – frequency, the Fourier transform, filtering, sampling – becomes a story about *shapes* and *measurements* rather than a wall of formulas. Get a muddy picture, and you will spend the rest of the book pushing symbols around without ever seeing what they mean.

This chapter builds that picture from the ground up. We will see that a signal is, at heart, just a function: a rule that assigns a value to each instant of time (or each point in space). We will look at real signals you can hear, see, and wear on your wrist. We will draw the great dividing lines of the field – one dimension versus two, single channel versus many, analog versus digital – and we will state the five central questions that the rest of the book exists to answer. And throughout, we will keep one concrete project in view: a wrist- or forearm-worn **gesture controller** that turns the motion of your hand into music.

1.1 A signal is a function carrying information

Pluck a guitar string. The string vibrates; it shoves the air; the air pressure at your eardrum rises and falls hundreds of times a second. If you could write down the air pressure at every instant, you would have a number for each moment in time – a *function of time*. That function *is* the sound. Nothing is lost in describing the sound this way; the wiggling number contains everything your ear receives.

Definition 1.1 (Signal, working definition)**.** A **signal** is a function x that maps an independent variable (such as time t or position) to a value $x(t)$, where the variation of that value carries *information*.

Two words in that definition deserve a second look. The first is *function*. A signal is not the wire it travels on, not the sensor that produced it, not the file it is stored in. It is the mathematical object: an assignment of a value to each point of some domain. When we write

$$x : \mathbb{R} \to \mathbb{R}, \qquad t \mapsto x(t),$$

we mean exactly the air-pressure-versus-time idea, abstracted so we can do mathematics to it.

The second word is *information*. A constant function $x(t) = 5$ for all t is a perfectly good function, but as a signal it is dull: it never changes, so it tells you nothing beyond its one value. Information lives in *variation* – in how the signal differs from instant to instant. This is why the entire field is obsessed with *change*, and why frequency (the rate of a particular kind of change) will turn out to be the master concept of the book.

Intuition

Think of a signal as a recording of "what was the value, moment by moment." A thermometer you read once an hour produces a signal: temperature versus time. A microphone produces a signal: pressure versus time. A photograph produces a signal: brightness versus position. In every case the same skeleton appears – an independent variable (the input) and a dependent value (the output) that wiggles in a way that means something.

The graph is the signal

The most important habit you can build right now is to *see the graph*. Whenever someone says "a signal," picture its plot: the horizontal axis is the independent variable, the vertical axis is the value. Figure 1.1 shows the simplest interesting signal – a pure tone, a single sinusoid. Stare at it. The height at each t is the value $x(t)$; the wiggling is the information; the rate of wiggling is the frequency.

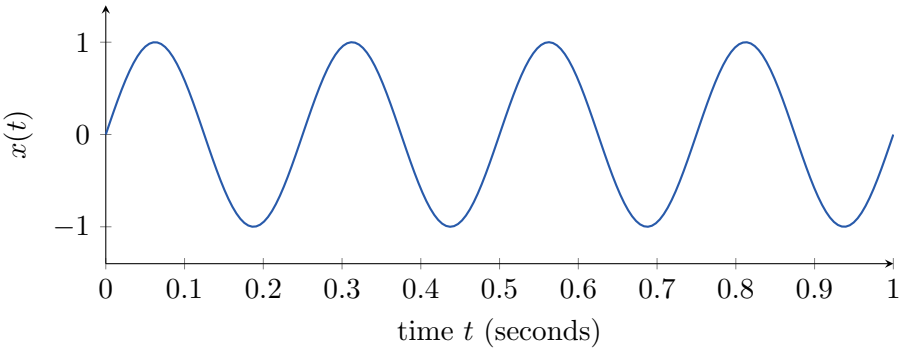


Figure 1.1: A pure tone: $x(t) = \sin(2\pi f t)$ with $f = 4$ Hz. The value at each instant is the height of the curve; the four full up-and-down cycles per second *are* its frequency.

Why it works

Why insist on the graph instead of the formula $x(t) = \sin(2\pi f t)$? Because most real signals have no tidy formula at all – speech, an accelerometer trace during a wrist flick, a heartbeat. They exist only as data: a long list of measured values. The graph is the one representation that works for *every* signal, formula or not. Geometry is the common language; the algebra is a convenience that happens to apply to special cases.

1.2 A gallery of real signals

Definitions are cheap; intuition comes from examples. Here is a small gallery of signals you will meet again and again in this book. Notice that they are wildly different physically – some you hear, some you see, some you feel – yet they are all the same kind of mathematical object.

Audio. Sound is variation in air pressure over time. A microphone converts that pressure into a voltage; the signal is voltage (proportional to pressure) versus time. Human hearing runs from roughly 20 Hz to 20 kHz, which already tells you something quantitative: the fastest meaningful wiggles in music happen tens of thousands of times per second.

Images. A grayscale photograph is brightness as a function of *position*: $x(u, v)$ where (u, v) names a pixel. Here the independent variable is two-dimensional space, not time. The "information" is the spatial pattern – edges, textures, shapes.

Accelerometer traces. Strap a tiny accelerometer to your wrist and it reports acceleration along three axes, dozens or hundreds of times per second. Wave your hand and you get three wiggling curves. This is exactly the raw material of our gesture controller, so we will return to it constantly.

Biosignals. Your body is a noisy broadcaster. The electrocardiogram (ECG) is the heart's electrical activity versus time. The electromyogram (EMG) is the electrical activity of muscles – and, crucially for finger-level sensing, the forearm muscles that move your fingers sit right under a wristband. We will lean hard on this fact in Chapter 37.

Figure 1.2 sketches three of these side by side so you can feel both their diversity and their shared skeleton.

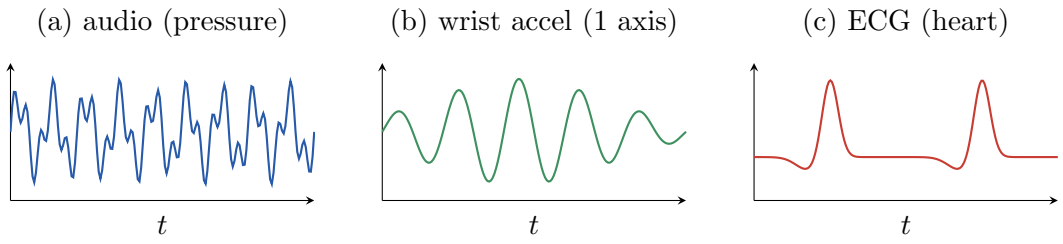


Figure 1.2: Three signals, one skeleton. (a) A bit of audio (a sum of two tones). (b) One axis of a wrist accelerometer during a single flick – a burst that grows and fades. (c) A stylized ECG with two heartbeats. Different physics, same object: a value versus time.

Toward the wrist tracker

Our running goal is a **wrist/forearm gesture controller** that outputs MIDI – in plain terms, a band you wear that watches your hand and plays an instrument. Panel (b) above is the heart of it: every gesture you make becomes a little story written in acceleration (and rotation and muscle activity) over time. The entire job of the controller is to take those wiggling curves and decide, in real time, *what did the hand just do?* – then emit the right

musical note or control change. Every chapter of this book is a tool for reading panel (b) correctly. Keep that image in mind: we are learning to read the handwriting of motion.

1.3 The dimensionality of the domain: 1D, 2D, and beyond

Signals are sorted first by the shape of their *input*, the independent variable.

One-dimensional (1D). The input is a single number – almost always time. Audio, ECG, a single accelerometer axis: all 1D. The graph is a curve. This is where we will spend most of the book, because the ideas are cleanest here and generalize cleanly.

Two-dimensional (2D). The input is a pair, typically a position (u, v). An image is the canonical 2D signal: $x(u, v)$ is the brightness at pixel (u, v). The "graph" is now a surface, or more usefully a picture. Everything we learn about 1D frequency will reappear in 2D as *spatial* frequency – fine textures are high spatial frequency, broad gradients are low. Chapters 30–33 are devoted to this.

Higher dimensions. Video is a 3D signal $x(u, v, t)$: brightness as a function of two space coordinates and time. A volumetric medical scan is $x(u, v, w)$ over three space coordinates. The machinery extends, but the intuition you build in 1D and 2D carries you the rest of the way.

Focus

Do not confuse the dimension of the *domain* (the input) with the dimension of the *value* (the output). A grayscale image is a 2D-domain, 1D-value signal: two-number input, one-number brightness out. A color image is 2D-domain, 3D-value (red, green, blue). Time is 1D no matter how many sensors you attach – a hundred microphones still listen along the single axis of time. The next section is about that output side: how many values come out at each instant.

1.4 Single channel versus multichannel

So far each signal has produced *one* value at each instant. But many real systems produce several at once. A stereo recording gives two values per instant (left ear, right ear). A three-axis accelerometer gives three. A modern wrist band might stream nine numbers per instant: three of acceleration, three of rotation rate (gyroscope), three of magnetic field (magnetometer).

We call each of these parallel streams a **channel**. A *multichannel* signal is best thought of as a *vector-valued* function of time:

$$\mathbf{x}(t) = \big(x_1(t),\, x_2(t),\, \ldots,\, x_C(t)\big),$$

where C is the number of channels. At each instant the signal is a point in $\mathbb{R}^C$; as time runs, that point traces a path. For the nine-axis wrist band, $\mathbf{x}(t)$ is a moving point in nine-dimensional space, and a "gesture" is a recognizable shape of that path.

Figure 1.3 shows the idea for a two-channel signal. On the left the two channels are drawn the usual way, each as its own curve against time. On the right the same data is drawn as a single path – channel 1 on the horizontal axis, channel 2 on the vertical – with time hidden inside the motion along the curve. A smooth circular wave of the hand produces a loop; that loop *is* the gesture, seen as a shape in channel space.

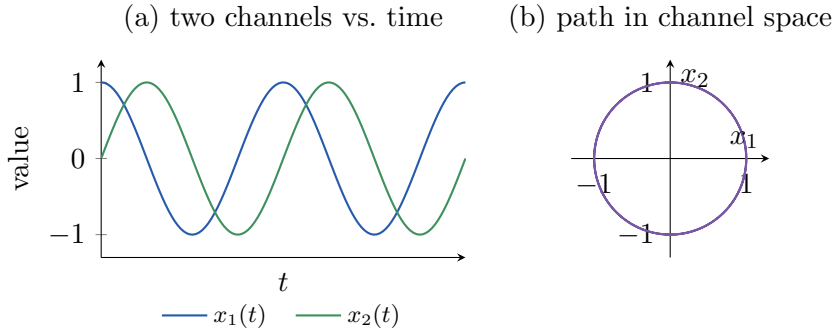


Figure 1.3: Two views of one two-channel signal. (a) Each channel as its own curve against time. (b) The same signal as a point $(x_1(t), x_2(t))$ tracing a path in channel space; here a quarter of the run has gone by before the loop closes. A faster wave runs around the same loop more times in the same window. Exercise 1.10 asks you to sketch such a path yourself.

Intuition

A single-channel signal is a bead sliding along a wire (one number, up and down). A three-channel accelerometer is a bead flying around inside a room (three numbers, a 3D trajectory). The richer the channel count, the more room the bead has to move – and the more ways a gesture can distinguish itself from every other gesture. Recognition gets easier with more channels, up to a point.

Connection

Treating a multichannel signal as a vector-valued function is your first taste of a theme that runs through the companion *Linear Algebra From the Ground Up*: *signals are vectors*. In Chapter 3 we push this much further and treat an *entire single-channel recording* – all its samples at once – as one vector in a high-dimensional space. Inner products, orthogonality, and projection, which you may know from that book as abstract operations, will turn out to be exactly how the Fourier transform measures frequency. Hold the thought; it is the spine of the whole subject.

1.5 Analog versus digital

Here is the deepest dividing line in the field, and the one most often misunderstood. It is really *two* independent distinctions wearing one name.

Continuous versus discrete in time

A real microphone produces a voltage that exists at *every* instant – there is a value at $t = 0.1$ s, at $t = 0.10001$ s, at every real number you can name. This is a **continuous-time** signal: its domain is the real line, or an interval of it. We write $x(t)$ with a parenthesis to flag it.

A computer cannot store a value for every one of the uncountably many instants. So we **sample**: we look only at evenly spaced moments $t = 0, T_s, 2T_s, 3T_s, \ldots$, where T_s is the *sampling period*, and record the value at each. The result is a **discrete-time** signal – a sequence of numbers indexed by an integer n. We write $x[n]$ with a square bracket to flag it; here $x[n]$ means "the value of the underlying signal at time nT_s."

Formal

Notation we will keep for the rest of the book:

- Round brackets $x(t)$, $t \in \mathbb{R}$: *continuous-time*. The domain is a continuum.
- Square brackets $x[n]$, $n \in \mathbb{Z}$: *discrete-time*. The domain is the integers.

The sampling rate is $f_s = 1/T_s$, in samples per second (often written Hz). A CD samples audio at $f_s = 44100$ Hz; a typical wrist IMU might run at $f_s = 100$ to 1600 Hz.

Continuous versus discrete in value

The second distinction is about the *value*, not the time. A continuous-time voltage can sit at *any* real value: $0.7314\ldots$ volts, exactly. But to store a number a computer must round it to one of finitely many levels – this is **quantization**. A signal whose values are restricted to a finite set of levels is *quantized*.

Putting both together:

- **Analog** signal: continuous in *both* time and value. The native output of microphones, accelerometer sensing elements, electrodes.

- **Digital** signal: discrete in *both* time and value – sampled *and* quantized. This is what lives in a file, a register, an array.

The device that crosses from analog to digital is the **analog-to-digital converter** (ADC); the device that crosses back is the **digital-to-analog converter** (DAC). Figure 1.4 shows the two acts – sampling and quantization – happening to a smooth analog signal.

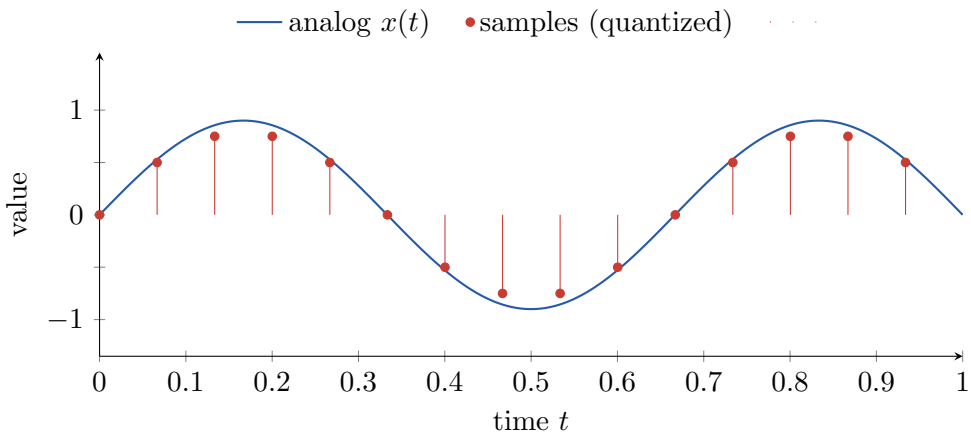


Figure 1.4: From analog to digital. The smooth blue curve is the continuous-time, continuous-value analog signal. Sampling keeps it only at evenly spaced instants (the dots); quantization snaps each sample to the nearest level on a fixed value grid (here multiples of 0.25). The dots are the digital signal $x[n]$.

Common trap

"Digital is just analog but on a computer" – this is the single most common misconception, and it hides two real losses. Sampling can throw away fast wiggles that fall between the sample instants (*aliasing*, Chapter 18). Quantization adds a small rounding error to every value (*quantization noise*, Chapter 19). Both are usually manageable, but they are real, and pretending the digital signal is identical to the analog one will eventually bite you. We will make peace with both losses rigorously later; for now, just know they exist.

History

The word "signal" predates electronics by centuries – think of smoke signals or naval flags, discrete symbols sent across distance. The *continuous* electrical signal arrived with the telegraph and especially the telephone (1870s onward), which had to carry the continuously varying pressure of speech down a wire. The *digital* signal as we know it crystallized in the 1920s–1940s at Bell Labs, where Harry Nyquist and later Claude Shannon worked out precisely how many samples per second you need to capture a continuous signal without loss. That result – the sampling theorem – is the bridge this whole book is built on, and we will earn it carefully in Chapter 18.

Connection

The continuous/discrete split is exactly the boundary between integration and summation that you met in *Calculus From the Ground Up*. Almost every formula in this book comes in two versions: a continuous one with an integral $\int x(t)\,\mathrm{d}t$ and a discrete one with a sum $\sum_n x[n]$. They are the same idea – "accumulate the signal" – viewed through the two lenses of the calculus book. When a discrete formula looks strange, recover the intuition by asking what its continuous integral cousin is doing.

1.6 The five central questions of DSP

Why is there a whole field – and a whole book – about these wiggling functions? Because once you have a signal, there are exactly five things you ever want to do with it. Every technique you will learn is an answer to one of these.

1. **Represent.** How do we store and describe a signal accurately and efficiently? How many samples? How many bits per sample? Can we describe it by its frequencies instead of its samples? (Sampling, quantization, Fourier representations.)
2. **Transform.** How do we re-express the signal in a form that reveals what we care about? The Fourier transform is the giant here: it rewrites a signal-in-time as a signal-in-frequency, turning "what does it look like moment to moment" into "what frequencies is it made of." (Chapters 6–11, 15.)
3. **Filter.** How do we keep the parts we want and remove the parts we do not – the hum, the drift, the high-frequency hiss, the gravity baseline in an accelerometer? (LTI systems, convolution, filter design, Chapters 12–17.)
4. **Analyze.** How do we measure properties of the signal – its dominant frequency, its energy, how it changes over time, how two channels relate? (Spectral estimation, spectrograms, statistics of noise, Chapters 10, 20–21.)
5. **Recognize.** How do we decide *what the signal means*? Is this a heartbeat or an artifact? Is this gesture a flick or a twist? This is where signal processing hands off to pattern recognition and machine learning. (Chapters 38–39.)

These five are not a random list; they line up into a pipeline, and our controller walks the whole length of it on every gesture. Figure 1.5 draws that pipeline so you can carry the picture into the chapters that fill in each stage.



Figure 1.5: The five central questions as one pipeline. The first four stages progressively refine the signal – raw samples, a frequency picture, a cleaned version, a few measured numbers – and the last stage makes a decision. The bottom row shows what each stage does inside the wrist controller.

What is really happening

Notice the arc. The first four questions progressively *refine* the signal – from raw samples, to a frequency picture, to a cleaned-up version, to a handful of measured numbers. The fifth question makes a *decision*. Our wrist controller walks this exact path on every gesture: represent the IMU stream as samples; transform a window of it to see its frequency content; filter out gravity and jitter; analyze for features like burst energy and dominant frequency; and finally recognize which gesture occurred and emit a MIDI message. The book is ordered, roughly, to march you down this pipeline.

1.7 A first clear-eyed look at the wrist controller

It is tempting – and you will see it claimed in marketing – that a wristband can read your individual finger movements from motion alone. It is worth being precise about this from page one, because being clear about what a sensor can and cannot do is the difference between an engineer and a salesperson.

Toward the wrist tracker

What a wrist-worn IMU can and cannot do. An inertial measurement unit (IMU) on the wrist measures the *motion of the wrist* – its linear acceleration and its rotation rate. From that it can reliably detect *coarse, whole-hand gestures*: a flick, a twist, a tap, a shake, raising or lowering the forearm, a circular wave. These are discrete, distinguishable motions of the limb, and they are plenty to drive an expressive instrument.

What a wrist IMU *cannot* do is recover the continuous joint angles of your individual fingers. Curling your index finger while holding your wrist still produces almost no signal at the wrist; the IMU is, quite literally, in the wrong place to see it. Recovering finger-level activity from a wristband requires a *different physical channel*: electromyography (EMG) or force myography (FMG), which sense the forearm muscles and tendons that drive the fingers and run right under a wristband. Even those give muscle *activation*, not clean joint angles; continuous finger *pose* realistically needs sensors on the fingers themselves (instrumented gloves or rings). We will treat EMG and FMG with the same care in Chapter 37 and cite the real research there. For now, set your expectations correctly: *the IMU gives us gestures of the hand as a whole; finger-level detail needs help.*

This is not a disappointment – it is a design constraint, and good engineering lives inside constraints. A flick, a twist, a tap, and a hold, mapped to musical actions, already make a real instrument. Knowing the limit up front keeps us from chasing impossible signal processing later.

Build it (hardware)

You can start collecting the very signals this book is about with almost no hardware. A $5–$15 IMU breakout board (for example an MPU-6050, which combines a three-axis accelerometer and three-axis gyroscope) wired to an Arduino over I^2C will stream six channels of motion. The sketch below reads the accelerometer and prints three numbers per line at a fixed rate – a multichannel, discrete-time signal you can capture and plot. Do not worry about the details yet; just notice that the output is exactly the object of Definition 1.1: values versus time, sample by sample.

Listing 1.1: Streaming a 3-axis accelerometer as a discrete-time, multichannel signal (Arduino + MPU-6050).

```
#include <Wire.h>
const int MPU = 0x68;          // I2C address of the IMU
const int FS_HZ = 100;         // sampling rate: 100 samples/sec
const unsigned long DT_US = 1000000UL / FS_HZ;

void setup() {
  Serial.begin(115200);
  Wire.begin();
```

```
  Wire.beginTransmission(MPU);
  Wire.write(0x6B); Wire.write(0); // wake the sensor
  Wire.endTransmission(true);
}

void loop() {
  static unsigned long next = 0;
  unsigned long now = micros();
  if (now < next) return;        // hold a steady sampling period T_s
  next = now + DT_US;

  Wire.beginTransmission(MPU);
  Wire.write(0x3B);              // first accel register
  Wire.endTransmission(false);
  Wire.requestFrom(MPU, 6, true);
  int16_t ax = Wire.read() << 8 | Wire.read();
  int16_t ay = Wire.read() << 8 | Wire.read();
  int16_t az = Wire.read() << 8 | Wire.read();

  // One row = one sample x[n] = (ax, ay, az): three channels at instant n.
  Serial.print(ax); Serial.print(',');
  Serial.print(ay); Serial.print(',');
  Serial.println(az);
}
```

Try it yourself

Run the sketch (or grab any logged IMU dataset) and plot the three columns against sample index. Then *flick your wrist* and look at the trace. You will see a burst: the curves leap, swing, and settle, much like Figure 1.2(b). Try a slow twist and a sharp tap, and compare their shapes. You are already doing the first job of the whole field – looking at a signal and asking what its shape means. Everything technical in this book is in service of answering that question reliably and automatically.

1.8 Energy, power, and why we will care

One last pair of ideas will recur so often that it belongs in this opening chapter: how "large" a signal is. There are two natural measures, and which one applies depends on whether the signal eventually dies away or runs forever.

The **energy** of a signal is the running total of its value squared:

$$E = \int_{-\infty}^{\infty} |x(t)|^2 \, \mathrm{d}t \qquad \text{(continuous)}, \qquad E = \sum_{n=-\infty}^{\infty} |x[n]|^2 \qquad \text{(discrete)}.$$

Squaring does two jobs at once: it makes negative dips count as much as positive peaks (a signal that swings down hard is not "small"), and it weights big excursions heavily. A transient like a single wrist flick has finite energy – it starts, happens, and stops.

A signal that never stops – a pure tone running forever, or the steady hum of mains power – has *infinite* energy, so energy is the wrong yardstick for it. For these we use **power**, the energy per unit

time, i.e. the average of $|x|^2$:

$$P = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} |x(t)|^2 \, \mathrm{d}t.$$

Intuition

Energy is "how much total commotion was there, start to finish" – the right question for a burst that begins and ends, like a gesture. Power is "how much commotion per second, on average" – the right question for something ongoing, like background vibration or a sustained tone. A flick has energy; a tremor has power.

Connection

The expression $\sum |x[n]|^2$ is the squared *length* of the signal-as-vector from the linear-algebra book, and $\int |x(t)|^2 \mathrm{d}t$ is its continuous analogue. So "energy" is just "length squared," and the famous Parseval/Plancherel theorem of Chapter 11 will say something beautifully geometric: a signal's energy is the same whether you measure it in time or in frequency, because the Fourier transform is a rotation that preserves length. You are collecting the pieces of that picture already.

Why this matters

Step back and admire the unification. Sound, images, heartbeats, and the motion of your hand are, to a wildly different set of human senses, utterly unlike one another. Yet we have just put them all in one frame: each is a function from a domain (time or space) to a value, carrying information in its variation, with a length we call energy. That one frame is what lets a single toolkit – the Fourier transform, filtering, sampling – serve audio engineers, radiologists, cardiologists, and instrument-builders alike. Seeing the common skeleton under different flesh is the first real act of signal-processing thinking.

1.9 Looking ahead

We now have the vocabulary the rest of the book assumes. A signal is a function carrying information in its variation (Section 1.1). Its domain may be 1D time, 2D space, or more (Section 1.3); its value may be a single number or a vector of channels (Section 1.4); and it may be analog (continuous in time and value) or digital (sampled and quantized), Section 1.5. Everything we do answers one of five questions – represent, transform, filter, analyze, recognize (Section 1.6) – and our concrete target throughout is a wrist gesture controller whose real limits we have already stated (Section 1.7).

The very next chapter sharpens the continuous-versus-discrete distinction into precise mathematical objects we can compute with. Then, in Chapter 3, we take the leap that powers the entire subject: we stop thinking of a signal as a wiggling curve and start thinking of it as a single *vector* in a vast space – the geometric idea from which frequency and the Fourier transform will emerge as naturally as shadows from a light.

1.10 Exercises

Exercise 1.1. For each of the following, state (i) the dimension of the domain, (ii) the number of channels, and (iii) whether it is most naturally modeled as analog or digital: a vinyl record groove; a stereo MP3 file; a black-and-white scanned document; the raw output of a three-axis gyroscope; a color video clip; the reading on a digital kitchen scale once per second.

Exercise 1.2. Explain, in your own words and without formulas, why a constant signal $x(t) = c$ carries no information, while $x(t) = \sin(2\pi f t)$ does. What is the smallest change you could make to the constant signal so that it begins to carry information?

Exercise 1.3. A wrist IMU is sampled at $f_s = 200$ Hz for 3 seconds while you perform one flick. (a) How many samples are recorded per channel? (b) If it has 6 channels (accel + gyro) stored as 16-bit integers, how many bytes does the recording occupy? (c) What is the sampling period T_s in milliseconds?

Exercise 1.4. Sketch (by hand or with code) a discrete-time signal $x[n]$ for $n = 0, 1, \ldots, 15$ that you would expect from a single sharp tap on a table, recorded by one accelerometer axis. Describe in words the shape you drew and why.

Exercise 1.5. Quantization snaps each sample to the nearest level on a value grid. If an ADC has 8 bits and covers the range $[-2g, +2g]$ (where g is the acceleration of gravity), how many levels are there, and what is the spacing between adjacent levels (the resolution) in units of g? Repeat for a 12-bit ADC over the same range and comment on the improvement.

Exercise 1.6. Classify each task below under exactly one of the five central questions (represent, transform, filter, analyze, recognize): (a) removing the constant gravity component from an accelerometer axis; (b) deciding whether a gesture was a flick or a twist; (c) choosing how many samples per second to store; (d) computing the dominant frequency of a tremor; (e) re-expressing a chord as the set of frequencies it contains.

Exercise 1.7. Consider the finite discrete signal $x[n] = \{2, -1, 3, 0, -2\}$ for $n = 0, \ldots, 4$ and zero elsewhere. Compute its energy $E = \sum_n |x[n]|^2$. Then form $y[n] = 2x[n]$ and compute its energy. By what factor did the energy change, and why?

Exercise 1.8. A pure tone $x(t) = A\sin(2\pi f t)$ runs forever. Argue that its energy is infinite but its power is finite. (You do not need to evaluate the integral exactly; reason about the average of $\sin^2$ over a full cycle, which is $1/2$.) What is the power in terms of A?

Exercise 1.9. A marketing brochure claims a wrist band "tracks every finger in 3D using its built-in motion sensor." Using the discussion in Section 1.7, write two or three sentences explaining what part of this claim is physically plausible and what part is not, and name the kind of sensor that would actually be needed for the implausible part.

Exercise 1.10. A multichannel signal can be viewed as a moving point in $\mathbb{R}^C$. For a two-axis sensor ($C = 2$) recording a smooth circular wave of the hand, sketch the path traced by the point $(x_1(t), x_2(t))$ in the plane (not versus time – the two axes plotted against *each other*). What shape do you expect, and what would a faster wave change about it?

Exercise 1.11. (Open-ended.) Record or download a short IMU trace of yourself making three different gestures. Plot all channels. Without doing any math, list three *visual* features that distinguish the gestures (for example: which axis spikes first, how long the burst lasts, whether it oscillates). These hand-found features are exactly what an automatic recognizer will later learn – you are sketching its job by eye.

Chapter 2

Continuous and Discrete Signals

In the previous chapter we agreed that a signal is just a rule that assigns a value to a moment of time – a function. That single sentence hides a fork in the road, and almost everything else in this book lives on one branch or the other of that fork. On one side: *continuous-time* signals, the smooth voltages and pressures and accelerations of the physical world, defined at every instant. On the other: *discrete-time* signals, the lists of numbers that a microcontroller actually stores and processes, defined only at a clock's tick.

The gesture controller we are building lives on *both* sides at once. Your wrist moves through real, continuous space. An accelerometer feels that motion as a continuous voltage. But the chip that decides "that was a flick, send a MIDI note" only ever sees a stream of integers arriving 200 times a second. The whole art of this book is learning to think clearly across that boundary – to know what survives the crossing and what is lost forever. This chapter builds the two worlds side by side, gives you the small zoo of basic signals you will meet again in every later chapter, and introduces the discrete impulse, the humble spike that will turn out to be the secret engine of convolution and of every filter you ever write.

2.1 Two Kinds of Time

Picture an audio recording. The microphone's diaphragm is pushed back and forth by air pressure; at every instant – not just at chosen moments, but at *every* real-numbered instant – there is a definite pressure. Call it $x(t)$. The independent variable t ranges over the whole real line $\mathbb{R}$ (or some interval of it). There is a value at $t = 1$ second, at $t = 1.5$, at $t = 1.500001$, at $t = \pi/7$. Between any two instants there are infinitely many more. This is a **continuous-time** signal.

Now picture the same sound after it has gone through the analog-to-digital converter in your laptop. The converter looks at the pressure, writes down a number, waits a fixed interval, looks again, writes down another number, and so on – 44,100 times per second for CD audio. What you are left with is a *sequence*: $x[0], x[1], x[2], \ldots$ There is no such thing as "$x[1.5]$". The signal is only defined at integer indices. This is a **discrete-time** signal.

Focus

The bracket is not decoration. In this book, and in most of the field, we write parentheses for continuous time, $x(t)$ with $t \in \mathbb{R}$, and square brackets for discrete time, $x[n]$ with $n \in \mathbb{Z}$. When you see $x[n]$, the variable n is an *integer index*: it counts samples, not seconds. The conversion between the index n and a real time is $t = nT_s$, where T_s is the time between samples.

Definition 2.1 (Continuous- and discrete-time signals)**.** A *continuous-time* signal is a function $x : \mathbb{R} \to \mathbb{R}$ (or $\mathbb{R} \to \mathbb{C}$), written $x(t)$, defined for every real t in its domain. A *discrete-time* signal is a function $x : \mathbb{Z} \to \mathbb{R}$ (or $\mathbb{Z} \to \mathbb{C}$), written $x[n]$, defined only for integer n. We call $x[n]$ a *sequence*.

A discrete-time signal is best drawn not as a curve but as a *stem plot*: a dot at each integer n, with a thin stalk connecting it to the axis, to remind you that there is nothing in between. Figure 2.1 shows the same underlying sinusoid in both worlds.

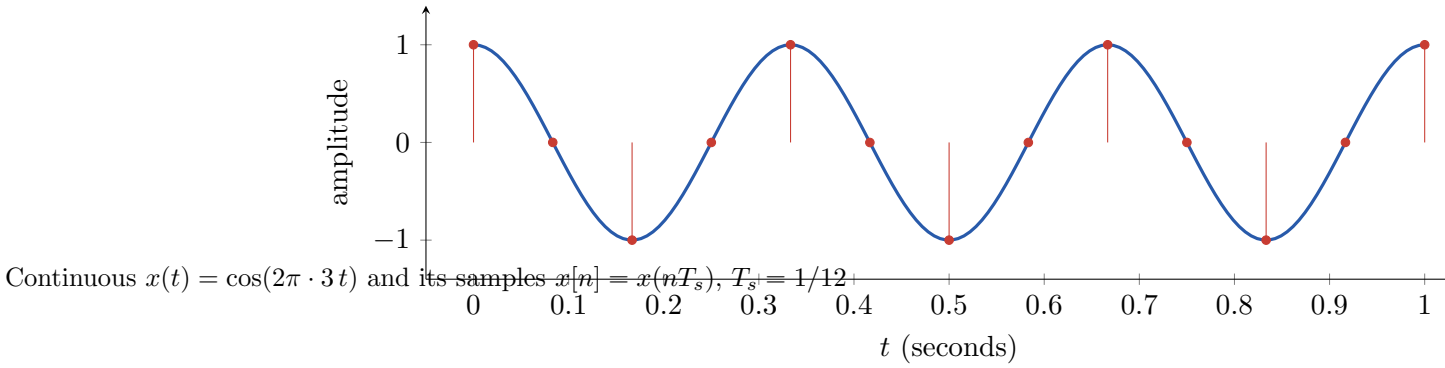


Figure 2.1: The continuous signal (blue curve) exists at every instant. The discrete signal (red stems) is the list of values read off at the sampling instants $t = nT_s$. The discrete signal knows nothing about the curve between its dots.

Intuition

A continuous signal is a *landscape*; a discrete signal is a *list of altitudes measured along a hike*. From the list alone you cannot say what the ground did between your steps – maybe it was smooth, maybe there was a hole you stepped over. Sampling throws away the in-between, and a huge fraction of signal processing is about knowing when that is safe (the sampling theorem) and when it bites you (aliasing).

Why we even bother with the continuous world

You might object: computers are digital, so why not study only sequences? Because the physics is continuous. The wrist moves smoothly; light, sound, and EMG voltages are continuous; the laws that govern them (Newton, Maxwell) are differential equations in continuous time. If you only ever think in samples you will be forever surprised by phenomena – aliasing, the need for anti-alias filters, the meaning of "frequency" above half the sample rate – that only make sense when you remember

the continuous truth that the samples came from. We keep both worlds in view precisely because the interesting bugs live at the border.

Connection

This fork mirrors a fork in the companion calculus book: the integral $\int x(t)\,\mathrm{d}t$ for continuous accumulation versus the sum $\sum_n x[n]$ for discrete accumulation. Almost every continuous-time formula in this book has a discrete twin in which $\int \to \sum$ and $\mathrm{d}t \to 1$ (or T_s). Energy, convolution, the Fourier transform – all come in these two flavors, and seeing them as the same idea in two costumes will save you enormous memorization.

2.2 Sampling, as a First Idea

We will devote a whole later chapter to sampling done right. But the basic move is so central that we introduce it now, in its simplest form, so the rest of this chapter can speak both languages.

Ideal sampling means: pick a fixed spacing T_s (the *sampling period*, in seconds), and define the sequence

$$x[n] \;=\; x(nT_s), \qquad n \in \mathbb{Z}.$$

The reciprocal $f_s = 1/T_s$ is the *sampling rate* or *sampling frequency*, measured in samples per second, or hertz (Hz). If $f_s = 200$ Hz then $T_s = 5$ ms, and $x[n]$ is the value of the underlying signal $5n$ milliseconds after we started.

Example 2.1 (From seconds to indices)**.** An accelerometer is sampled at $f_s = 200$ Hz. You want the sample nearest to $t = 0.31$ s. Then $n = tf_s = 0.31 \times 200 = 62$. Conversely, sample $n = 150$ corresponds to $t = n/f_s = 150/200 = 0.75$ s. Get comfortable sliding between t and n; you will do it constantly.

Common trap

Sampling does *not* mean "rounding the values." Sampling discretizes *time*; rounding the values to a finite set of levels is a separate step called *quantization* (its own chapter). A signal can be discrete in time but have perfectly real-valued (infinitely fine) amplitudes – that is exactly the $x[n] \in \mathbb{R}$ of Definition 2.1. Real ADCs do both at once, but they are different operations and confusing them causes real bugs.

Toward the wrist tracker

The wrist tracker's MPU-6050 IMU is, in effect, an array of tiny springs whose deflection becomes a continuous voltage. The chip's built-in ADC samples that voltage, typically at something between 100 Hz and 1 kHz, and hands your microcontroller a sequence $a_x[n], a_y[n], a_z[n]$ of acceleration samples. Every gesture-detection idea later in the book operates on these sequences. Choosing f_s is your first real design decision: too low and a fast flick of the wrist slips between samples (aliasing, lost gestures); too high and you waste power and flood the CPU. Human hand motion has very little energy above about 20 Hz, so an f_s of a few hundred hertz is plenty – but we will only be able to *justify* that number once we have the sampling theorem.

2.3 The Small Zoo of Basic Signals

A handful of elementary signals appear over and over. They are the "primary colors" of signal processing: more complicated signals are built from them, and the behavior of a whole system can often be summarized by how it responds to one of them. We introduce each in both continuous and discrete form.

2.3.1 The unit impulse

In discrete time the impulse is the simplest signal imaginable: it is 1 at the origin and 0 everywhere else.

$$\delta[n] \;=\; \begin{cases} 1, & n = 0, \\ 0, & n \neq 0. \end{cases}$$

This is the *Kronecker delta* and there is nothing subtle about it – a single spike, as in the left panel of Figure 2.2. Do not let its plainness fool you; by the end of this chapter (and decisively in the convolution chapter) it will turn out that *every* discrete signal is a sum of shifted, scaled impulses, and that a linear system is completely described by its response to this one spike.

The continuous-time impulse $\delta(t)$ – the *Dirac delta* – is a stranger beast. We would like a signal that is "zero everywhere except at $t = 0$, yet has total area 1." No ordinary function does this: a single point has zero width, so zero area. The Dirac delta is defined not by its values but by what it does inside an integral, through the *sifting* property

$$\int_{-\infty}^{\infty} x(t)\,\delta(t)\,\mathrm{d}t \;=\; x(0),$$

and more generally $\int x(t)\,\delta(t - t_0)\,\mathrm{d}t = x(t_0)$. Think of $\delta(t)$ as the limit of a tall narrow rectangle of width ε and height $1/\varepsilon$ as $\varepsilon \to 0$: the area stays 1 while the spike grows infinitely thin and tall. We draw it as an upward arrow of "height 1" (really, of *area* 1).

> **Formal**
>
> Strictly, $\delta(t)$ is not a function but a *distribution* (generalized function): a rule that eats a smooth test function $x(t)$ and returns the number $x(0)$. Engineers manipulate it as if it were a function and almost never get into trouble, because every legitimate use ends with δ inside an integral, where the sifting property tells you the answer. The discrete $\delta[n]$ needs none of this care – it is a plain, finite sequence that is 1 at zero and 0 elsewhere.

2.3.2 The unit step

The unit step "turns on" at the origin:

$$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases} \qquad\qquad u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0. \end{cases}$$

The step models switching something on: applying a constant force at $t = 0$, closing a circuit, the onset of a sustained note. It also lets you *window* a signal: $x[n]\,u[n]$ is $x[n]$ for $n \geq 0$ and zero before, which is how you describe a signal that "starts" at a definite time.

Step and impulse are intimately related. In discrete time, the step is the running sum of impulses, and the impulse is the first difference of the step:

$$u[n] = \sum_{k=-\infty}^{n} \delta[k], \qquad \delta[n] = u[n] - u[n-1].$$

In continuous time the same pair appears with calculus instead of sums: $u(t) = \int_{-\infty}^{t} \delta(\tau)\,\mathrm{d}\tau$ and, formally, $\delta(t) = \dfrac{\mathrm{d}}{\mathrm{d}t}u(t)$ – the derivative of a jump is an impulse.

Intuition

Summation/integration and differencing/differentiation are inverse operations, so δ and u form a derivative/antiderivative pair. This is your first glimpse of a recurring theme: the continuous and discrete worlds run the same play with $\int \leftrightarrow \sum$ and $\frac{\mathrm{d}}{\mathrm{d}t} \leftrightarrow$ (first difference).

2.3.3 The unit ramp

The ramp is the running sum (or integral) of the step: it rises linearly once it switches on.

$$r[n] = n\,u[n], \qquad r(t) = t\,u(t).$$

Concretely, $r[n] = \ldots, 0, 0, 0, 1, 2, 3, \ldots$ starting its climb at $n = 0$. The trio impulse $\rightarrow$ step $\rightarrow$ ramp is each the running accumulation of the one before it; differencing walks back the other way. The ramp shows up whenever a quantity grows at a constant rate after some onset – a position under constant velocity, or a phase that increases linearly (which we will soon see *is* a constant-frequency sinusoid).

2.3.4 The real exponential

$$x(t) = e^{at} \quad (a \in \mathbb{R}), \qquad x[n] = \alpha^{n} \quad (\alpha \in \mathbb{R}).$$

For the continuous form, $a < 0$ gives exponential *decay* (the discharge of a capacitor, the die-away of a struck string), $a > 0$ gives growth, and $a = 0$ gives a constant. For the discrete form α^n: $|\alpha| < 1$ decays, $|\alpha| > 1$ grows, and a *negative* α makes the sign alternate each sample – a flavor of behavior with no continuous-time analog among real exponentials, and a hint that the fastest wiggle a sequence can have is sign-flipping every sample. The link between the two forms is $\alpha = e^{aT_s}$ when $x[n]$ samples e^{at}.

2.3.5 The sinusoid

The star of the whole book:

$$x(t) = A\cos(2\pi f t + \phi) = A\cos(\omega t + \phi),$$

with *amplitude* A, *frequency* f in hertz (so $\omega = 2\pi f$ in radians/second is the *angular* frequency), and *phase* ϕ in radians. Its discrete cousin is

$$x[n] = A\cos(\hat{\omega}\, n + \phi), \qquad \hat{\omega} = \omega T_s = 2\pi f / f_s,$$

where $\hat{\omega}$ is the *normalized* (or digital) angular frequency, in radians *per sample*. The sinusoid matters above all others because, as later chapters will show, it is the one signal shape that passes through any linear time-invariant system unchanged except in amplitude and phase. That is what makes "frequency" the natural language for such systems.

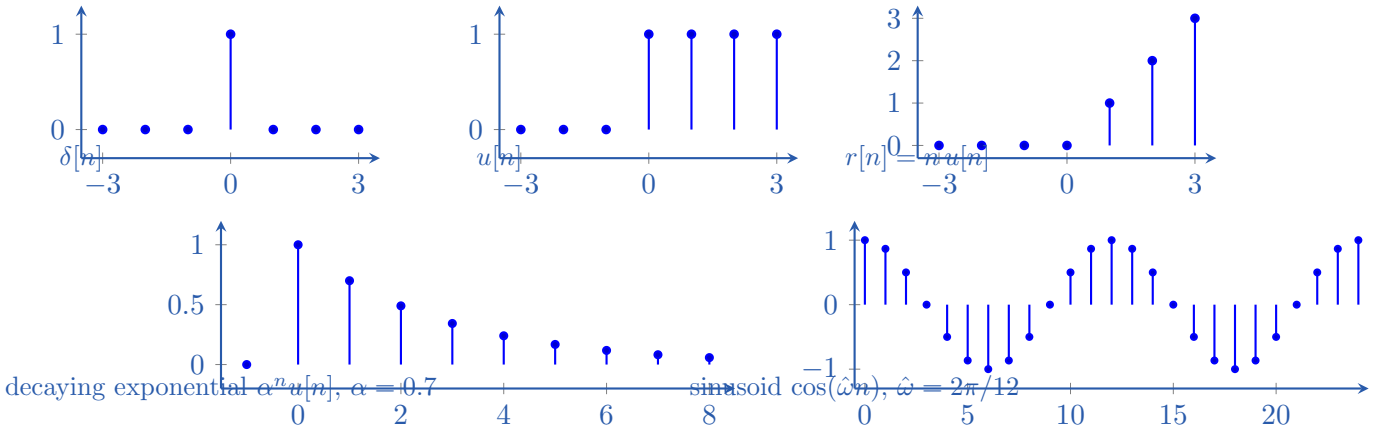


Figure 2.2: The basic discrete-time signals: impulse, step, ramp (top), and the decaying exponential and a sampled sinusoid (bottom). Every signal we analyze later is, in some sense, built from these.

2.3.6 The complex exponential – a sneak preview

The single most important signal in the book is one we cannot yet fully justify:

$$x(t) = e^{\mathrm{j}\omega t} = \cos(\omega t) + \mathrm{j}\sin(\omega t).$$

Geometrically (later chapters make this vivid) $e^{\mathrm{j}\omega t}$ is a point of unit length rotating around the origin of the complex plane at angular speed ω: its real part traces a cosine, its imaginary part a sine. It bundles a cosine and a sine into one rotating arrow, and it is the true "atom" of frequency. We mention it here only so the cast of characters is complete; phasors and rotation get the full geometric treatment in the chapter on sinusoids and phasors.

2.4 Energy and Power

How "big" is a signal? Two measures dominate, and choosing between them is really a question about whether your signal is a transient burst or an endless hum.

The instantaneous power of a signal is $|x(t)|^2$ (we use $|\cdot|^2$ so the definitions work for complex signals too). Integrating that over all time gives the **total energy**:

$$E = \int_{-\infty}^{\infty} |x(t)|^2 \,\mathrm{d}t, \qquad E = \sum_{n=-\infty}^{\infty} |x[n]|^2.$$

A signal with finite, nonzero energy is an *energy signal* – think of a single clap, a flick of the wrist, anything that starts, happens, and dies away.

But many useful signals never die away: a pure tone, a power-line hum, a constant. Their energy is infinite, so energy is useless for comparing them. For these we use average power – energy per unit time, in the limit:

$$P = \lim_{T\to\infty} \frac{1}{2T}\int_{-T}^{T} |x(t)|^2\,\mathrm{d}t, \qquad P = \lim_{N\to\infty} \frac{1}{2N+1}\sum_{n=-N}^{N} |x[n]|^2.$$

A signal with finite, nonzero average power is a *power signal*. The two categories are mutually exclusive: an energy signal has $P = 0$ (its finite energy, spread over infinite time, averages to nothing), and a power signal has $E = \infty$.

Example 2.2 (Power of a sinusoid)**.** Take $x(t) = A\cos(\omega t + \phi)$. Using $\cos^2\theta = \frac{1}{2}(1 + \cos 2\theta)$, the $\cos 2\theta$ term averages to zero over any whole number of periods, leaving

$$P = \frac{1}{T_0}\int_0^{T_0} A^2\cos^2(\omega t + \phi)\,\mathrm{d}t = \frac{A^2}{2}.$$

So the average power of a sinusoid is half its amplitude squared, regardless of frequency or phase. The corresponding root-mean-square (RMS) amplitude is $A/\sqrt{2}$ – the number your multimeter reports for an AC voltage. Mains electricity quoted as "230 V" is an RMS figure; its peak is $230\sqrt{2} \approx 325$ V.

Example 2.3 (Energy of a decaying exponential)**.** $x[n] = \alpha^n u[n]$ with $|\alpha| < 1$. Then

$$E = \sum_{n=0}^{\infty} |\alpha|^{2n} = \frac{1}{1 - |\alpha|^2},$$

a finite number – it is an energy signal. For $\alpha = 0.7$, $E = 1/(1 - 0.49) \approx 1.96$.

> **Connection**
>
> Energy is just the squared length (norm) of a signal viewed as a vector: $E = \|x\|^2 = \langle x, x\rangle$. The next chapter makes this literal – signals *are* vectors, energy *is* squared length, and Parseval's theorem (later) will say that this length is the same whether you measure it in the time domain or the frequency domain. The link to the probability book is just as direct: average power is the mean square, and for a zero-mean random signal it equals the variance $\mathrm{Var}(x)$, the foundation of signal-to-noise ratio.

2.5 Symmetry: Even and Odd

A signal is **even** if it is its own mirror image about $t = 0$, and **odd** if mirroring flips its sign:

$$\text{even:}\quad x(-t) = x(t), \qquad\qquad \text{odd:}\quad x(-t) = -x(t).$$

Cosine is even ($\cos(-\theta) = \cos\theta$); sine is odd ($\sin(-\theta) = -\sin\theta$); the impulse is even; the ramp is neither. An odd signal must pass through the origin ($x(0) = -x(0) \Rightarrow x(0) = 0$).

The reason we care is a small miracle: *every* signal splits uniquely into an even part plus an odd part,

$$x_e(t) = \tfrac{1}{2}\big(x(t) + x(-t)\big), \qquad x_o(t) = \tfrac{1}{2}\big(x(t) - x(-t)\big), \qquad x = x_e + x_o.$$

(Check: $x_e + x_o = x$, and x_e is even while x_o is odd.) This is not idle bookkeeping. The Fourier transform treats even and odd parts very differently – even parts produce real (cosine) spectra, odd parts produce imaginary (sine) spectra – so the even/odd split is secretly the cosine/sine split, which is secretly the real/imaginary split of the spectrum. Symmetry in time is symmetry in frequency, and exploiting it is how the real-input FFT runs roughly twice as fast.

Example 2.4 (Splitting a one-sided exponential)**.** Let $x[n] = \alpha^n u[n]$. Its mirror is $x[-n] = \alpha^{-n} u[-n]$. The even part is $x_e[n] = \frac{1}{2}(\alpha^n u[n] + \alpha^{-n} u[-n])$ – a two-sided symmetric decay – and the odd part is the antisymmetric remainder; at $n = 0$, $x_o[0] = 0$ and $x_e[0] = 1$, recovering $x[0] = 1$.

2.6 Periodic and Aperiodic

A continuous signal is **periodic** if it repeats after some shift T_0,

$$x(t + T_0) = x(t) \quad \text{for all } t,$$

and the smallest such $T_0 > 0$ is the *fundamental period*; its reciprocal $f_0 = 1/T_0$ is the fundamental frequency. A discrete signal is periodic if $x[n + N] = x[n]$ for some integer period N.

Here the two worlds genuinely diverge, and the difference trips up beginners. A continuous sinusoid $\cos(\omega t)$ is *always* periodic. A discrete sinusoid $\cos(\hat{\omega} n)$ is periodic *only if* $\hat{\omega}/2\pi$ is a rational number. The reason: we need $\hat{\omega} N = 2\pi k$ for integers N, k, i.e. $\hat{\omega}/2\pi = k/N$ must be rational. If $\hat{\omega}/2\pi$ is irrational, the samples never line up to repeat, even though they came from a perfectly periodic continuous signal.

Example 2.5 (A discrete sinusoid that is *not* periodic)**.** $\cos(n)$ – that is, $\hat{\omega} = 1$ radian/sample – has $\hat{\omega}/2\pi = 1/(2\pi)$, which is irrational. So the sequence $\cos(0), \cos(1), \cos(2), \ldots$ never exactly repeats: $\cos(n)$ is an aperiodic discrete signal. By contrast $\cos(2\pi n/12)$ has $\hat{\omega}/2\pi = 1/12$, rational, so it repeats every $N = 12$ samples (visible in Figure 2.2).

Walking question

You sample a 50 Hz tone at $f_s = 200$ Hz. Is the resulting sequence periodic, and if so, what is its period in samples?

Work it. $\hat{\omega} = 2\pi f/f_s = 2\pi(50)/200 = 2\pi/4$, so $\hat{\omega}/2\pi = 1/4$: rational, period $N = 4$ samples. Indeed four samples of a 50 Hz tone at 200 Hz span exactly one cycle ($4 \times 5\,\text{ms} = 20\,\text{ms} = 1/50\,\text{s}$). Now try 60 Hz at 200 Hz: $\hat{\omega}/2\pi = 60/200 = 3/10$, period $N = 10$.

Common trap

A real recorded signal is never truly periodic – it started at some finite time and will stop. "Periodic" is an idealization we apply to a steady middle stretch. This matters for the DFT later: it implicitly assumes the finite chunk you fed it repeats forever, and the mismatch at the seam causes *spectral leakage*, fixed by windowing. File that away; it is one of the most common surprises in practical spectrum analysis.

2.7 Transformations of the Independent Variable

Many operations reshape a signal by doing something to its *time axis* rather than its values. Three are fundamental: shifting, reflecting, and scaling. Getting the direction of each right is a classic source of sign errors, so we go slowly.

2.7.1 Time shift (delay and advance)

Replacing t by $t - t_0$ *delays* the signal by t_0:

$$y(t) = x(t - t_0).$$

It looks backwards but is right: the value y takes at time t_0 is the value x took at time 0, so the whole signal has moved t_0 to the right (later). A positive t_0 is a delay; a negative t_0 is an advance (the signal moves left, earlier). In discrete time, $y[n] = x[n - n_0]$ shifts the sequence right by n_0 samples. The one-sample delay $x[n - 1]$ – "yesterday's value" – is the workhorse of every digital filter; in hardware it is literally one memory cell.

Intuition

Why does "$-t_0$" move things *right*? Track one feature, say a peak that x has at $t = 0$. In $y(t) = x(t - t_0)$, that peak now occurs where the argument $t - t_0 = 0$, i.e. at $t = t_0$. The feature relocated from 0 to t_0: rightward by t_0. Subtracting inside the argument adds to the time of every feature.

2.7.2 Time reversal (reflection)

$y(t) = x(-t)$ flips the signal left-to-right about $t = 0$ – running the tape backwards. We already used it to define even/odd. In discrete time, $y[n] = x[-n]$. Reflection is the operation hiding inside correlation and inside convolution (where one of the two signals is flipped before sliding), so it is worth being fluent with.

2.7.3 Time scaling

$y(t) = x(at)$ with $a > 1$ *compresses* the signal toward the origin (it plays faster, finishes sooner); $0 < a < 1$ *stretches* it (plays slower). Again it can feel backwards: $a = 2$ *shrinks* the time axis because a feature at x-time t_0 now appears at y-time $t_0/2$. A negative a scales *and* reflects.

Time scaling is where continuous and discrete part ways most sharply. In continuous time you can scale by *any* real factor. In discrete time, $x[an]$ only makes sense for integer an. Integer compression ($x[2n]$ keeps every other sample – this is *downsampling*/decimation) and integer expansion (insert zeros between samples – *upsampling*) are genuinely different operations from continuous scaling, with their own subtleties: they can create or destroy frequency content. The chapter on resampling treats them properly.

Example 2.6 (Reading off a compound transform)**.** Sketch $y(t) = x(2t - 4)$ given $x(t)$. Mixing up the order of operations is exactly the trap, so use a fixed recipe: rewrite the argument in the canonical form $a(t - t_d)$. Here $2t - 4 = 2(t - 2)$, so $a = 2$ and $t_d = 2$. The recipe is shift-then-scale: take $x(t)$,

shift right by 2 to get $x(t-2)$, then compress the time axis by 2 to get $x(2(t-2)) = x(2t-4)$. Verify with one landmark: $y(2)$ should equal $x(0)$, and indeed $2 \cdot 2 - 4 = 0$. ✓

Common trap

When a transform mixes shift and scale, do not eyeball it. Rewrite the argument in the canonical form $a(t - t_d)$, read off the scale a and the shift t_d, and apply shift-then-scale. Then verify with a single landmark point (where the argument is zero). Two minutes of this beats an hour debugging a flipped filter.

2.8 The Sifting Property: Building Signals from Impulses

We close with the idea that quietly powers the rest of the book. Look again at the discrete impulse. Shift it: $\delta[n-k]$ is a spike sitting at $n = k$. Scale that spike by a number c: $c\,\delta[n-k]$ is a spike of height c at position k. Now add up a whole lineup of such spikes, one for each integer k, with the height of the k-th spike chosen to be the actual signal value $x[k]$:

$$x[n] \;=\; \sum_{k=-\infty}^{\infty} x[k]\,\delta[n-k]\,.$$

Read it carefully. On the right, for any particular n, every term vanishes except the one with $k = n$ (because $\delta[n-k] = 0$ unless $k = n$), and that surviving term is $x[n] \cdot 1 = x[n]$. So the equation is true – but it is true in a profound way. It says **any** discrete signal is a sum of shifted, scaled impulses. The impulse is a *basis* for all signals: a complete set of building blocks, one per time slot. Figure 2.3 shows the decomposition for a short sequence.

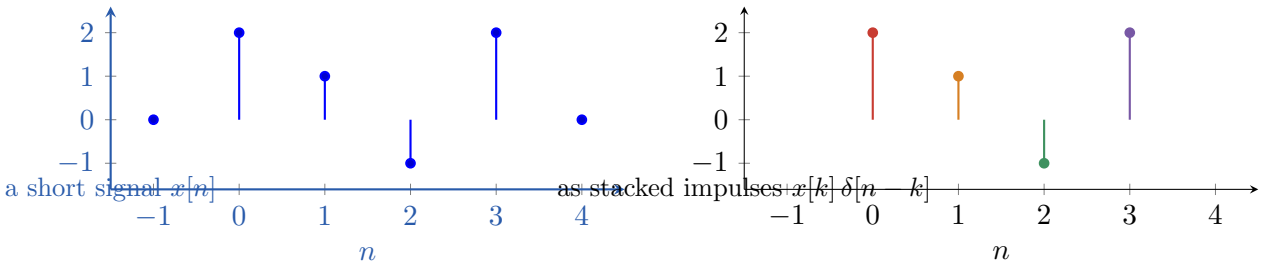


Figure 2.3: The sifting decomposition. The sequence on the left (blue) is the sum of four scaled, shifted impulses (right): height 2 at $n = 0$, height 1 at $n = 1$, height -1 at $n = 2$, height 2 at $n = 3$. Each impulse carries exactly one sample value, sitting in its own time slot.

What is really happening

The sifting sum decomposes a signal by *location in time*: it says "x is this much at slot 0, plus this much at slot 1, plus ...," each "this much" carried by its own impulse. That sounds almost too obvious to be useful. The payoff comes from *linearity and time-invariance*: if you know how a system responds to a single impulse $\delta[n]$ – call that response $h[n]$ – then by linearity its response to $x[k]\delta[n-k]$ is $x[k]\,h[n-k]$, and by superposition its response to the

whole sum is

$$y[n] = \sum_k x[k]\, h[n-k].$$

That sum is *convolution*, and it means a single experiment – poke the system with one spike, record $h[n]$ – tells you its output for *every* possible input. This is the central result of linear systems, and the sifting property is the seed it grows from.

The continuous version replaces the sum with an integral and the Kronecker delta with the Dirac delta:

$$x(t) = \int_{-\infty}^{\infty} x(\tau)\, \delta(t-\tau)\, \mathrm{d}\tau,$$

which is exactly the sifting property of Section 2.3.1 read as "x is a continuum of infinitesimal impulses, one at each instant τ, weighted by $x(\tau)$."

Why it works

Why phrase the obvious fact "$x[n]$ equals $x[n]$" as an impulse sum at all? Because it converts the question "what does a system do to an arbitrary signal?" into the far smaller question "what does it do to one impulse?" The trick is to write a complicated input as a combination of simple inputs you already understand, push each simple piece through, and add up the results. We will run this exact play twice more in this book: once with impulses (giving convolution and the time-domain view) and once with sinusoids/complex exponentials (giving the Fourier transform and the frequency-domain view). Two choices of building block, two whole theories. The impulse and the sinusoid are the two great alphabets of signals.

Toward the wrist tracker

For the wrist tracker, the impulse-response idea is how we will reason about the filters that clean up the IMU stream. The hand motion we care about is slow (a few hertz); sensor noise and physiological tremor are faster. A smoothing filter is fully described by its impulse response $h[n]$ – literally, "what comes out if you feed in a single spike." Once we have $h[n]$ for our chosen smoother, convolution tells us what it does to a real gesture, and the Fourier view (later) tells us *which* frequencies it keeps or kills. The plain little $\delta[n]$ of this chapter is the test signal we will poke every filter with.

Why this matters

Step back and admire the shape of the chapter. We split time into continuous and discrete; built a small zoo of canonical signals; learned to measure size (energy, power), to classify by symmetry and periodicity, and to reshape the time axis. Then, with the sifting property, we found that one of those zoo animals – the impulse – is secretly a complete *language*: every signal is a sentence written in impulses. That a single spike could generate everything is the kind of leverage that makes signal processing beautiful, and it is the thread we pull on for the next several chapters.

2.9 A Tiny Bit of Code

Everything above is concrete enough to type. Here is much of the chapter in NumPy: build the basic sequences, sample a sinusoid, compute energy and power, and verify the even/odd split.

```
import numpy as np

fs   = 200.0                 # sampling rate, Hz
Ts   = 1.0 / fs
n    = np.arange(0, 200)     # sample indices 0..199 (one second)
t    = n * Ts                # the real time of each sample

# basic signals as sequences
delta = (n == 0).astype(float)          # unit impulse
step  = (n >= 0).astype(float)          # unit step (all ones here)
ramp  = n * step                        # unit ramp
expo  = 0.97 ** n                       # decaying exponential
sine  = np.cos(2*np.pi*5*t)             # 5 Hz sinusoid, sampled

# energy and (average) power of the sinusoid
E = np.sum(np.abs(sine)**2)             # finite-window energy
P = np.mean(np.abs(sine)**2)            # approx A^2/2 = 0.5 for A=1
print(f"energy={E:.2f}  power={P:.3f}  (expect ~0.5)")

# even/odd split of a one-sided exponential
m   = np.arange(-50, 51)
x   = np.where(m >= 0, 0.97**m, 0.0)
xe  = 0.5 * (x + x[::-1])               # even part
xo  = 0.5 * (x - x[::-1])               # odd part
assert np.allclose(xe + xo, x)          # the split reconstructs x
```

Try it yourself

Run the snippet, then change one thing at a time. (1) Set `sine = np.cos(2*np.pi*190*t)` – a 190 Hz tone sampled at 200 Hz – and *plot* it with `matplotlib`. It will look like a slow 10 Hz wave, not a fast one: your first encounter with aliasing, the ghost that haunts under-sampling. (2) Change the decay base from 0.97 to 1.01 and watch `expo` blow up – the boundary $|\alpha| = 1$ between energy and power signals is razor-thin. (3) Build a delayed impulse `np.roll(delta, 5)` and confirm it is $\delta[n-5]$.

History

The continuous impulse is named for Paul Dirac, who introduced it in his 1930s work on quantum mechanics as a convenient "function" that was zero everywhere but integrated to one. Mathematicians were uneasy – no such function exists – until Laurent Schwartz put it on rigorous footing in the late 1940s with the theory of *distributions*, for which he won the Fields Medal in 1950. The discrete Kronecker delta is older and tamer, named for Leopold Kronecker in the nineteenth century. The lesson for an engineer: a tool can be indispensable in practice decades before the mathematics that justifies it catches up.

2.10 Exercises

Exercise 2.1. A signal is sampled at $f_s = 256$ Hz. (a) What is T_s? (b) Which sample index n is closest to $t = 1.0$ s? (c) What real time does sample $n = 400$ correspond to?

Exercise 2.2. Classify each as an energy signal, a power signal, or neither, and justify: (a) $x[n] = (0.5)^n u[n]$; (b) $x(t) = 3\cos(20\pi t)$; (c) $x[n] = u[n]$ (the unit step); (d) $x(t) = t\,u(t)$ (the ramp).

Exercise 2.3. Compute the total energy of $x[n] = (0.8)^n u[n]$. Then compute the energy of the *delayed* version $x[n - 10]$. Does delaying a signal change its energy? Explain in one sentence why this had to be true.

Exercise 2.4. Let $x[n] = \{\,\underline{2}, 3, -1\,\}$ for $n = 0, 1, 2$ (underline marks $n = 0$), and zero elsewhere. Write $x[n]$ explicitly as a sum of shifted, scaled impulses $\sum_k x[k]\,\delta[n - k]$.

Exercise 2.5. Find the even and odd parts of $x[n] = \{\,\underline{1}, 2, 3\,\}$ for $n = 0, 1, 2$ (zero elsewhere). Verify that x_e is even, x_o is odd, and $x_e + x_o = x$ at every index.

Exercise 2.6. For each discrete sinusoid, state whether it is periodic, and if so give the fundamental period N in samples: (a) $\cos(2\pi n/8)$; (b) $\cos(0.3\pi n)$; (c) $\cos(3n)$; (d) $\cos(2\pi n/5) + \cos(2\pi n/3)$ (think about the least common multiple of the two periods).

Exercise 2.7. A continuous sinusoid $\cos(2\pi f t)$ is sampled at f_s. Show that the resulting sequence is periodic in n if and only if f/f_s is rational, and that the period is then $N = f_s/\gcd(f, f_s)$ when f, f_s are integers. Test it on $f = 60$, $f_s = 200$.

Exercise 2.8. Given an arbitrary $x(t)$, sketch (in words or rough pictures) the result of each transform, and identify the canonical shift/scale: (a) $x(t + 3)$; (b) $x(-t)$; (c) $x(3t)$; (d) $x(-2t + 6)$. For (d), give the landmark check at the point where the argument is zero.

Exercise 2.9. Using only the relations $\delta[n] = u[n] - u[n - 1]$ and $u[n] = \sum_{k \le n} \delta[k]$, prove that the ramp satisfies $r[n] - r[n - 1] = u[n - 1]$. (Hint: write the ramp as a running sum of the step and difference it.)

Exercise 2.10. *Toward the wrist tracker.* Suppose your IMU streams acceleration at $f_s = 200$ Hz, and a typical wrist "flick" gesture lasts about 150 ms. (a) How many samples does one flick span? (b) If you instead sampled at only $f_s = 20$ Hz, how many samples would the flick span, and why might that be too few to recognize the gesture reliably? (c) Argue qualitatively, using the energy idea, why a brief flick is an "energy signal" while the steady hum of holding your arm still resembles a "power signal" plus a constant offset (gravity).

Exercise 2.11. A digital one-sample-delay element outputs $y[n] = x[n - 1]$. Its impulse response is $h[n] = \delta[n - 1]$. Using the convolution sum $y[n] = \sum_k x[k]\,h[n - k]$ from the sifting discussion, confirm that feeding in an arbitrary $x[n]$ indeed produces $x[n - 1]$. This is the smallest possible nontrivial filter – and you just verified it from first principles.

Chapter 3

The Geometry of Functions: Signals as Vectors

Everything in this book hangs on one idea, and it is a geometric one. A signal – a sound, an image, a stream of acceleration readings from a sensor on your wrist – is a *vector*. Not a metaphorical vector, not a vector "in spirit," but a genuine element of a vector space, with all the machinery of arrows, lengths, angles, and projections that you already know from two and three dimensions. Once you see this, the Fourier transform stops being a frightening integral and becomes something almost obvious: it is the act of measuring the components of a signal along a particular set of perpendicular directions. The directions happen to be pure tones.

This chapter builds that picture from the ground up. We will start with arrows in the plane, watch them grow extra coordinates until they become signals, and discover that the same three operations – adding, scaling, and taking inner products – carry over completely. Along the way we will meet the concepts that organize the rest of the book: *energy* (a length), *correlation* (an angle), *orthogonality* (perpendicularity), and *orthonormal bases* (perpendicular rulers you measure with). At the end, the sinusoids will step forward as the most useful ruler of all, and the seed of Fourier analysis will be planted.

Focus

The single sentence to carry out of this chapter: **analyzing a signal means projecting it onto a basis, and a Fourier transform is exactly that, where the basis vectors are sinusoids.** Everything else – norms, inner products, orthogonality – is the apparatus that makes "project onto" precise.

3.1 From arrows to signals

Start with the most familiar vector of all: an arrow in the plane, written as a pair of numbers $\mathbf{v} = (v_1, v_2)$. You know three things you can do with it.

1. **Add** two vectors: $(a_1, a_2) + (b_1, b_2) = (a_1 + b_1,\ a_2 + b_2)$. Geometrically, lay them tip-to-tail.
2. **Scale** a vector by a number: $c\,(v_1, v_2) = (cv_1,\ cv_2)$. This stretches or flips the arrow without

changing the line it lies on.

3. **Measure** with the dot product: $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2$. This one number secretly contains both lengths and the angle between them, through $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta$.

Now do something that looks like cheating but is not. Add a third coordinate, getting points in $\mathbb{R}^3$; nothing breaks – adding, scaling, and the dot product $a_1 b_1 + a_2 b_2 + a_3 b_3$ all still work. Add a fourth, a tenth, a thousandth. The pictures in your head get harder to draw, but the *algebra* never changes. A vector in $\mathbb{R}^N$ is just a list of N numbers, and the three operations are component-by-component sums, component-by-component scalings, and a dot product that sums N products.

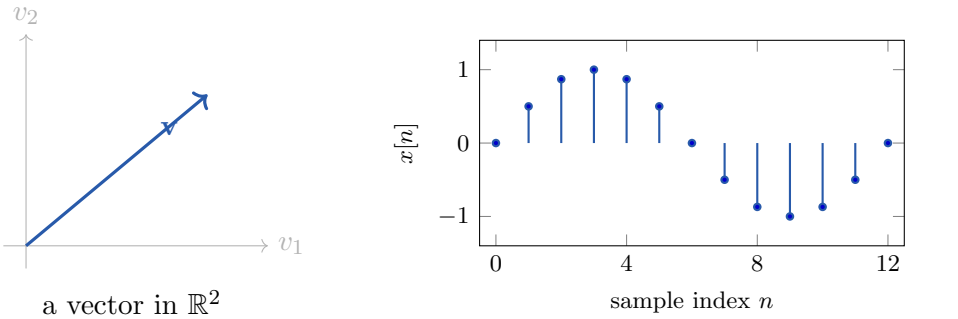
Here is the move that turns linear algebra into signal processing. A discrete signal sampled at N instants *is* a list of N numbers. The reading you take from a wrist accelerometer at times $t_0, t_1, \ldots, t_{N-1}$ is the vector

$$\mathbf{x} = \big(x[0],\, x[1],\, \ldots,\, x[N-1]\big) \in \mathbb{R}^N.$$

A one-second snippet of audio at 44,100 samples per second is a single vector living in $\mathbb{R}^{44100}$. The index that we usually call "time" is playing exactly the role that the index $1, 2, 3$ played for the arrow's coordinates. There is no difference in kind, only in dimension.

Intuition

Stop thinking of a waveform as a wiggly curve and start thinking of it as *one point in a very high-dimensional space*. The whole one-second audio clip is a single dot somewhere in $\mathbb{R}^{44100}$. Two clips that "sound similar" are two dots that are close together. Silence is the origin. Making a sound twice as loud is scaling its vector. Mixing two sounds is vector addition. The geometry you will spend this book learning is the geometry of that space.



a vector in $\mathbb{R}^2$ — the same idea with 13 coordinates

Figure 3.1: Left: a vector in $\mathbb{R}^2$ has two coordinates. Right: a length-13 signal has thirteen coordinates, $x[0], \ldots, x[12]$. A signal is just a vector with a lot of components; the index n plays the role of the coordinate label.

Connection

This chapter is the bridge between the companion *Linear Algebra From the Ground Up* and all the frequency-domain ideas to come. If you have read that book, every term here – vector space, inner product, norm, orthonormal basis, projection – is the same object you already met, now with the components indexed by *time* instead of by $1, 2, 3$. The payoff is that the

deepest theorem of finite-dimensional linear algebra (every vector is a unique combination of an orthonormal basis) becomes, almost word for word, the Fourier representation of a signal.

3.2 What makes a vector space (and why signals form one)

We have been waving our hands at "vector space." Let us pin it down, because the abstraction is exactly what lets the same theorems serve arrows and audio at once.

Definition 3.1 (Vector space, informally)**.** A (real) vector space is a set of objects you can *add* and *scale by real numbers*, such that the obvious rules hold: addition is commutative and associative, there is a zero element, every element has a negative, scaling distributes over addition, and $1 \cdot \mathbf{x} = \mathbf{x}$. Any set with these two operations behaving sensibly is a vector space, and its elements are called vectors – whatever they happen to look like.

The phrase "whatever they happen to look like" is the whole point. The definition never mentions arrows. So let us check signals against it. Take the set of all real sequences of length N. Adding two of them samplewise gives another length-N sequence. Scaling one by a constant gives another. The all-zeros sequence is the zero vector. Negatives, associativity, distributivity – all hold, because they hold component by component. So:

Proposition 3.1. *The set of length-N real signals, with samplewise addition and scalar multiplication, is the vector space $\mathbb{R}^N$.*

Nothing here required N to be finite in spirit. The set of all infinite sequences $\{x[n]\}_{n\in\mathbb{Z}}$, and even the set of all functions $x(t)$ of a continuous variable, satisfy Definition 3.1 just as well: you can add two functions, $(x+y)(t) = x(t)+y(t)$, and scale a function, $(cx)(t) = c\,x(t)$. Continuous-time signals form a vector space too – an infinite-dimensional one. We will mostly reason in $\mathbb{R}^N$ (or its complex cousin $\mathbb{C}^N$) because the hardware we build will always sample, and a finite list of numbers is what a microcontroller actually holds. But keep in mind that the continuous case is the same animal with infinitely many coordinates.

Why it works

Why insist on the abstract definition instead of just saying "a signal is a list of numbers"? Because the theorems we want – the Pythagorean theorem, the existence of orthonormal bases, the fact that projection minimizes error – are proved *only* from the vector-space axioms plus an inner product. By verifying that signals satisfy the axioms, we inherit every one of those theorems for free, without re-deriving anything. The abstraction is not decoration; it is the mechanism of reuse.

3.3 The inner product: one number that measures everything

The dot product was the workhorse of plane geometry, because it encoded both length and angle. We need its generalization to signals. For two real length-N signals $\mathbf{x}$ and $\mathbf{y}$, define

Definition 3.2 (Inner product of signals)**.**

$$\langle \mathbf{x}, \mathbf{y} \rangle \;=\; \sum_{n=0}^{N-1} x[n]\, y[n].$$

For complex signals (which we will need the moment sinusoids become complex exponentials) we conjugate the second argument:

$$\langle \mathbf{x}, \mathbf{y} \rangle \;=\; \sum_{n=0}^{N-1} x[n]\, \overline{y[n]}.$$

This is the dot product, summed over time instead of over three spatial axes. The conjugate in the complex case looks like a technicality but it is essential: it guarantees that $\langle \mathbf{x}, \mathbf{x} \rangle$ is a real, nonnegative number (a length squared had better not be imaginary). Watch what falls out of this single definition.

It gives a length (energy). Setting $\mathbf{y} = \mathbf{x}$,

$$\langle \mathbf{x}, \mathbf{x} \rangle = \sum_{n} |x[n]|^2 \;=\; \|\mathbf{x}\|^2.$$

The right side is the *energy* of the signal, and its square root is the norm, the length of the vector. A loud sound is a long vector; a quiet sound is a short one. This is not a pun: the physical energy delivered by a voltage $x(t)$ into a unit resistor really is $\int x(t)^2\, \mathrm{d}t$, the continuous analog of $\sum x[n]^2$.

It gives an angle (correlation). Just as in the plane,

$$\langle \mathbf{x}, \mathbf{y} \rangle = \|\mathbf{x}\|\, \|\mathbf{y}\| \cos\theta, \qquad \text{so} \qquad \cos\theta = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\|\, \|\mathbf{y}\|}.$$

The cosine of the "angle between two signals" is exactly what statisticians call the (normalized) *correlation* between them. Two signals that point the same way ($\theta = 0$) are perfectly correlated; two that point oppositely are anti-correlated; and two at right angles ($\langle \mathbf{x}, \mathbf{y} \rangle = 0$) are *uncorrelated* – they share nothing.

What is really happening

When you "compute the correlation" of a candidate signal with a template – say, matching a recorded swipe gesture against a stored prototype – you are literally measuring the cosine of the angle between two vectors in a several-hundred-dimensional space. A big positive inner product means the angle is small means "these point the same way" means "this looks like that." Correlation detectors, matched filters, and (soon) the Fourier transform are all the same geometric act: take an inner product and see how much one vector leans along another.

Example 3.1 (Inner products of short signals)**.** Let $\mathbf{x} = (1, 2, 3, 4)$ and $\mathbf{y} = (4, 3, 2, 1)$, both in $\mathbb{R}^4$.

$$\langle \mathbf{x}, \mathbf{y} \rangle = 1 \cdot 4 + 2 \cdot 3 + 3 \cdot 2 + 4 \cdot 1 = 20,$$

$$\|\mathbf{x}\| = \sqrt{1 + 4 + 9 + 16} = \sqrt{30}, \quad \|\mathbf{y}\| = \sqrt{30}, \quad \cos\theta = \frac{20}{30} \approx 0.667,$$

so $\theta \approx 48°$. The two signals are similar but not identical – one rises, the other falls, yet both have the same energy and a substantial overlap. Now try $\mathbf{u} = (1, -1, 1, -1)$ and $\mathbf{w} = (1, 1, 1, 1)$: $\langle \mathbf{u}, \mathbf{w} \rangle = 1 - 1 + 1 - 1 = 0$. These two are *orthogonal*: a fast alternation and a constant share nothing. Hold onto that example; it is a baby Fourier basis.

Common trap

The inner product is *not* elementwise multiplication, and it is *not* the product of the two signals. It is the single number you get after multiplying samplewise *and then summing*. Forgetting the sum (leaving a whole signal where a scalar should be) is the most common beginner slip, and it makes "project onto" meaningless. Inner product in, one number out.

3.4 Norm, energy, and distance

Because the norm came from an inner product, it behaves like a true length and gives us a notion of *distance* between signals:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\| = \sqrt{\sum_n |x[n] - y[n]|^2}.$$

This is just the Euclidean (Pythagorean) distance, now in N dimensions. It answers "how different are these two signals?" with a single number, and it is the quantity every denoiser, compressor, and machine-learning loss is secretly trying to make small. When we say a reconstructed signal is "close to" the original, we mean this distance is tiny.

The crown jewel that the inner product buys us is the Pythagorean theorem, and it is worth stating because the entire idea of spectral energy rests on it.

Theorem 3.1 (Pythagoras for signals)**.** *If* $\mathbf{x}$ *and* $\mathbf{y}$ *are orthogonal,* $\langle \mathbf{x}, \mathbf{y} \rangle = 0$*, then*

$$\|\mathbf{x} + \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2.$$

Proof. Expand using bilinearity of the inner product: $\|\mathbf{x}+\mathbf{y}\|^2 = \langle \mathbf{x}+\mathbf{y}, \mathbf{x}+\mathbf{y} \rangle = \|\mathbf{x}\|^2 + 2\langle \mathbf{x}, \mathbf{y} \rangle + \|\mathbf{y}\|^2$, and the middle term vanishes. □

Why this matters

Theorem 3.1 is why we can speak of "the energy in the 440 Hz component" as a separate, additive thing. If a signal decomposes into orthogonal pieces – and sinusoids of different frequencies, we will see, are orthogonal – then the total energy is the *sum* of the energies of the pieces. No cross terms, no interference in the bookkeeping. This is Parseval's theorem in embryo, and it is the reason a spectrum can be read straight off as "how much energy lives at each frequency."

3.5 Orthogonality and orthonormal bases

In the plane, the directions $\mathbf{e}_1 = (1, 0)$ and $\mathbf{e}_2 = (0, 1)$ are special: they are perpendicular and unit length, and *any* vector is a combination of them whose coefficients you read off trivially, $\mathbf{v} =$

$v_1\mathbf{e}_1 + v_2\mathbf{e}_2$. The coefficient v_1 is nothing but the projection of $\mathbf{v}$ onto $\mathbf{e}_1$, i.e. $\langle \mathbf{v}, \mathbf{e}_1 \rangle$. We want this same convenience for signals, with a basis we get to choose.

Definition 3.3 (Orthonormal set)**.** A set of vectors $\{\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{N-1}\}$ is *orthonormal* if

$$\langle \mathbf{b}_k, \mathbf{b}_m \rangle = \delta_{km} = \begin{cases} 1 & k = m \\ 0 & k \neq m, \end{cases}$$

that is, each vector has unit length and any two distinct vectors are orthogonal. If the set also spans the whole space (here, N such vectors in an N-dimensional space), it is an *orthonormal basis*.

The magic of an orthonormal basis is the *analysis formula*. Suppose $\mathbf{x} = \sum_m c_m \mathbf{b}_m$ for some unknown coefficients c_m. Take the inner product of both sides with one particular basis vector $\mathbf{b}_k$:

$$\langle \mathbf{x}, \mathbf{b}_k \rangle = \Big\langle \sum_m c_m \mathbf{b}_m, \ \mathbf{b}_k \Big\rangle = \sum_m c_m \langle \mathbf{b}_m, \mathbf{b}_k \rangle = \sum_m c_m \, \delta_{mk} = c_k.$$

Every term in the sum collapses except the one that matches, because the basis is orthonormal. So the coefficient is recovered by a single inner product:

Theorem 3.2 (Analysis and synthesis)**.** *For an orthonormal basis* $\{\mathbf{b}_k\}$*, every signal* $\mathbf{x}$ *can be written*

$$\underbrace{c_k = \langle \mathbf{x}, \mathbf{b}_k \rangle}_{\textit{analysis: extract a coefficient}}, \qquad \underbrace{\mathbf{x} = \sum_{k=0}^{N-1} c_k \, \mathbf{b}_k}_{\textit{synthesis: rebuild the signal}}.$$

What is really happening

Look at what just happened. To find "how much of direction $\mathbf{b}_k$ is in my signal," you take *one inner product*. To rebuild the signal, you add up each direction weighted by how much of it is present. **Analysis is a pile of inner products; synthesis is a weighted sum.** The Fourier transform will be precisely this, with the $\mathbf{b}_k$ chosen to be sinusoids. Everything else in the next several chapters is detail on that one sentence.

Why it works

Why does orthonormality matter so much – why not use any old basis? With a non-orthogonal basis you *can* still write $\mathbf{x} = \sum_m c_m \mathbf{b}_m$, but to find the coefficients you must solve a coupled $N \times N$ linear system, because the inner-product trick above leaves cross terms $\langle \mathbf{b}_m, \mathbf{b}_k \rangle \neq 0$. Orthonormality *diagonalizes* that system: each coefficient is computed independently, by its own inner product, with no interference from the others. That independence is what makes the Fast Fourier Transform fast and what makes a spectrum interpretable channel by channel.

3.5.1 A worked basis change

Let us make Theorem 3.2 concrete in $\mathbb{R}^2$ before unleashing it on signals. Take the rotated orthonormal basis

$$\mathbf{b}_0 = \tfrac{1}{\sqrt{2}}(1, 1), \qquad \mathbf{b}_1 = \tfrac{1}{\sqrt{2}}(1, -1).$$

Check: $\|\mathbf{b}_0\|^2 = \frac{1}{2}(1+1) = 1$, likewise $\mathbf{b}_1$, and $\langle \mathbf{b}_0, \mathbf{b}_1 \rangle = \frac{1}{2}(1-1) = 0$. Orthonormal. Now decompose $\mathbf{x} = (3, 1)$:

$$c_0 = \langle \mathbf{x}, \mathbf{b}_0 \rangle = \tfrac{1}{\sqrt{2}}(3+1) = \tfrac{4}{\sqrt{2}} = 2\sqrt{2}, \qquad c_1 = \langle \mathbf{x}, \mathbf{b}_1 \rangle = \tfrac{1}{\sqrt{2}}(3-1) = \sqrt{2}.$$

Reconstruct: $c_0\mathbf{b}_0 + c_1\mathbf{b}_1 = 2\sqrt{2} \cdot \frac{1}{\sqrt{2}}(1,1) + \sqrt{2} \cdot \frac{1}{\sqrt{2}}(1,-1) = (2,2) + (1,-1) = (3,1) = \mathbf{x}$. It rebuilds perfectly. And notice energy is preserved: $\|\mathbf{x}\|^2 = 9 + 1 = 10$, while $c_0^2 + c_1^2 = 8 + 2 = 10$. The coefficients carry exactly the same energy as the samples; we have only rotated our point of view. (That is exactly the basis of Example 3.1, scaled to unit length – the "average" direction $\mathbf{b}_0$ and the "difference" direction $\mathbf{b}_1$. It is the smallest possible Fourier transform, and also the simplest wavelet.)

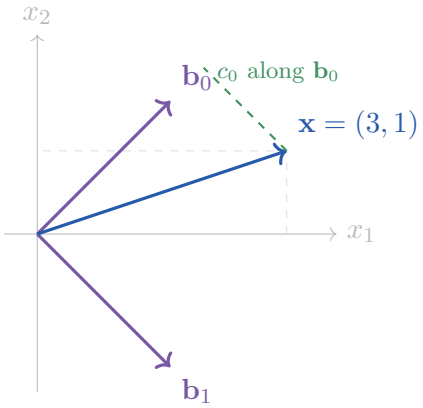


Figure 3.2: Changing basis is rotating your rulers. The same point $\mathbf{x} = (3, 1)$ has coordinates $(3, 1)$ in the standard basis and $(2\sqrt{2}, \sqrt{2})$ in the rotated orthonormal basis $\{\mathbf{b}_0, \mathbf{b}_1\}$. The dashed green line is the projection that yields c_0. The point did not move; only our description of it did.

3.6 Projection: the geometry of "how much of this is in that"

The coefficient $c_k = \langle \mathbf{x}, \mathbf{b}_k \rangle$ is a *projection*. It deserves its own section because projection is also the right way to think about approximation, and approximation is what real systems do when they cannot keep every coefficient.

The projection of $\mathbf{x}$ onto a unit vector $\mathbf{b}$ is the vector $\langle \mathbf{x}, \mathbf{b} \rangle \mathbf{b}$: it is the shadow $\mathbf{x}$ casts on the line through $\mathbf{b}$. The leftover, $\mathbf{x} - \langle \mathbf{x}, \mathbf{b} \rangle \mathbf{b}$, is orthogonal to $\mathbf{b}$ – it is the part of $\mathbf{x}$ that the direction $\mathbf{b}$ simply cannot see. This splits any signal into "the part along $\mathbf{b}$" plus "the part perpendicular to $\mathbf{b}$," and by Theorem 3.1 their energies add.

Theorem 3.3 (Projection minimizes error)**.** *Among all scalar multiples $\alpha\mathbf{b}$ of a unit vector $\mathbf{b}$, the one closest to $\mathbf{x}$ (smallest $\|\mathbf{x} - \alpha\mathbf{b}\|$) is $\alpha = \langle \mathbf{x}, \mathbf{b} \rangle$. More generally, the best approximation of $\mathbf{x}$ using only the directions $\{\mathbf{b}_k : k \in S\}$ is obtained by keeping their projection coefficients $c_k = \langle \mathbf{x}, \mathbf{b}_k \rangle$ and dropping the rest.*

Why it works

This is the entire theoretical basis of lossy compression and of bandlimiting. When MP3 throws away inaudible frequency components, or when a low-pass filter discards high-frequency directions, it is *projecting* the signal onto a smaller set of basis vectors. Theorem 3.3 guarantees this projection is the closest possible approximation within those kept directions – you cannot do better with the same budget of components. Keeping the largest coefficients and discarding the smallest is provably the energy-optimal thing to do in an orthonormal basis.

Example 3.2 (Approximating a ramp with a few basis vectors)**.** Take the ramp $\mathbf{x} = (1, 2, 3, 4)$ from Example 3.1 and the orthonormal "average" direction $\mathbf{b}_0 = \frac{1}{2}(1, 1, 1, 1)$ (length 1, since $4 \cdot (\frac{1}{2})^2 = 1$). Its coefficient is $c_0 = \langle \mathbf{x}, \mathbf{b}_0 \rangle = \frac{1}{2}(1 + 2 + 3 + 4) = 5$. The projection is $c_0\mathbf{b}_0 = 5 \cdot \frac{1}{2}(1, 1, 1, 1) = (2.5, 2.5, 2.5, 2.5)$ – the best constant approximation of the ramp is its average, 2.5, exactly as intuition demands. The residual $(-1.5, -0.5, 0.5, 1.5)$ is the part of the ramp that a constant cannot represent, and it is orthogonal to $\mathbf{b}_0$ (its entries sum to zero). To capture the residual we would need a basis vector that tilts – a low "frequency."

Walking question

Before reading on, predict: if you projected that same ramp onto a single one-cycle cosine instead of the constant, would you get a large or small coefficient? (Hint: the ramp is mostly a steady rise plus a gentle bow; a cosine over one window also rises then falls. They overlap, but not perfectly.) We will return to this when the sinusoidal basis arrives – the point is that *each* basis direction answers a different question about the same signal.

3.7 Sinusoids as an orthogonal basis: the seed of Fourier

We now choose our basis. Of all the orthonormal rulers we could lay down in signal space, one set is supremely useful: the *sinusoids*. The reasons are deep (they are the eigenvectors of every time-invariant system, a fact Chapter 12 develops), but the first reason we can verify right now, with nothing but the inner product: **sinusoids of different frequencies are orthogonal.**

Consider the discrete signals built from a fundamental frequency that fits a whole number of cycles into N samples:

$$\mathbf{c}_k[n] = \cos\!\left(\tfrac{2\pi kn}{N}\right), \qquad \mathbf{s}_k[n] = \sin\!\left(\tfrac{2\pi kn}{N}\right), \qquad n = 0, \dots, N-1.$$

The integer k counts how many full cycles the sinusoid completes across the window. The claim is that for $k \neq m$ (within range),

$$\langle \mathbf{c}_k, \mathbf{c}_m \rangle = \sum_{n=0}^{N-1} \cos\!\left(\tfrac{2\pi kn}{N}\right) \cos\!\left(\tfrac{2\pi mn}{N}\right) = 0,$$

and similarly cosines are orthogonal to sines, and distinct sines to each other. The cleanest way to prove this – and the reason complex exponentials will take over the whole book – is to package cosine and sine into a single rotating complex exponential $\mathbf{e}_k[n] = e^{\mathrm{j}2\pi kn/N} = \cos(2\pi kn/N) + \mathrm{j}\sin(2\pi kn/N)$. Then

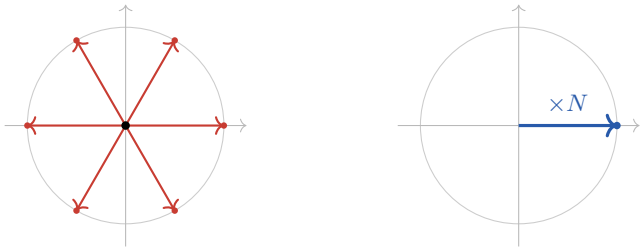
Theorem 3.4 (Orthogonality of the discrete complex exponentials)**.** *For $k, m \in \{0, 1, \ldots, N-1\}$,*

$$\langle \mathbf{e}_k, \mathbf{e}_m \rangle = \sum_{n=0}^{N-1} e^{\mathrm{j}2\pi kn/N} \overline{e^{\mathrm{j}2\pi mn/N}} = \sum_{n=0}^{N-1} e^{\mathrm{j}2\pi(k-m)n/N} = \begin{cases} N & k = m, \\ 0 & k \neq m. \end{cases}$$

Proof. If $k = m$ each term is $e^0 = 1$ and the sum is N. If $k \neq m$, write $r = e^{\mathrm{j}2\pi(k-m)/N}$, a root of unity with $r \neq 1$. The sum is a finite geometric series $\sum_{n=0}^{N-1} r^n = \dfrac{r^N - 1}{r - 1}$. But $r^N = e^{\mathrm{j}2\pi(k-m)} = 1$, so the numerator is zero while the denominator is not. The sum vanishes. □

What is really happening

That geometric-series argument is the mathematical heart of all of Fourier analysis, and it is purely geometric: summing N equally spaced points around the unit circle gives the centroid of a regular polygon, which sits at the center – zero – unless all the points coincide at 1 (the case $k = m$). The phasors cancel by symmetry. *Different frequencies cancel; the matching frequency reinforces.* That cancel-versus-reinforce is exactly how a Fourier transform "picks out" the frequency present in a signal.



$k \neq m$: sum $= 0$ $\qquad$ $k = m$: sum $= N$

Figure 3.3: The geometric series of Theorem 3.4 as phasors on the unit circle. When $k \neq m$ the terms $e^{\mathrm{j}2\pi(k-m)n/N}$ are equally spaced points around the circle (here $N = 6$); their average is the polygon's centroid, the origin, so they cancel. When $k = m$ every term sits at angle 0 and they reinforce to give N. Cancel versus reinforce is how the transform picks out a frequency.

Figure 3.3 shows the two cases side by side. So the N complex exponentials $\{\mathbf{e}_0, \ldots, \mathbf{e}_{N-1}\}$ are mutually orthogonal, each with norm $\sqrt{N}$. Dividing by $\sqrt{N}$ makes them orthonormal: $\mathbf{b}_k = \frac{1}{\sqrt{N}}\mathbf{e}_k$. By Theorem 3.2, any signal expands as a sum of these, and the coefficients are inner products. That expansion *is* the Discrete Fourier Transform. We have, in effect, already written it down:

$$X[k] = \langle \mathbf{x}, \mathbf{e}_k \rangle = \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}2\pi kn/N}, \qquad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{\mathrm{j}2\pi kn/N}.$$

The analysis equation on the left extracts "how much of frequency k is in $\mathbf{x}$"; the synthesis equation on the right rebuilds the signal from its frequency content. The factor $1/N$ is the bookkeeping for our basis having norm $\sqrt{N}$ instead of 1. Chapter 8 unpacks all of this, but the geometry is settled here and now: **the DFT is a change of basis to the orthogonal sinusoids, computed by projection.**

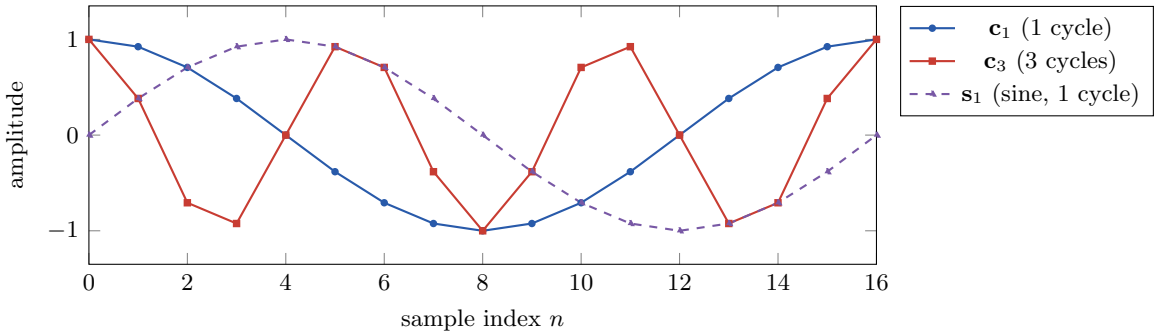


Figure 3.4: Three members of the orthogonal sinusoidal basis for $N = 16$: a one-cycle cosine, a three-cycle cosine, and a one-cycle sine. Each pair has inner product zero – summing the samplewise products over the window gives exactly 0. These mutually perpendicular "directions" are the rulers the Fourier transform measures along.

History

Joseph Fourier announced in 1807, to a skeptical Paris Academy that included Lagrange and Laplace, that *any* periodic function could be written as a sum of sines and cosines. Lagrange objected that it could not possibly represent functions with corners. Fourier was essentially right and Lagrange essentially wrong, but the fight forced mathematicians to make "sum of infinitely many basis vectors" rigorous – a century-long project that produced the modern theory of function spaces, convergence, and the space L^2 we sketch below. The geometry was clear to Fourier; the analysis took the nineteenth century to finish.

3.8 Function space and the leap to infinite dimensions

Everything above lived in $\mathbb{R}^N$ or $\mathbb{C}^N$ – finitely many coordinates. What happens for a continuous-time signal $x(t)$, which has a value at every one of uncountably many instants? The same geometry survives if we make two natural substitutions: sums become integrals, and the index n becomes the continuous variable t. The inner product of two signals on an interval $[a, b]$ is

$$\langle x,\, y\rangle = \int_a^b x(t)\,\overline{y(t)}\,\mathrm{d}t, \qquad \|x\|^2 = \int_a^b |x(t)|^2\,\mathrm{d}t.$$

A function is "in the space" precisely when this energy integral is finite. That space – square-integrable functions with the integral inner product – is called L^2, and it is the natural home of finite-energy signals. It is infinite-dimensional, but it keeps the two properties we cared most about: an inner product (hence lengths, angles, orthogonality) and *completeness* (Cauchy sequences of signals converge to a signal in the space). A space with those properties is a *Hilbert space*, and L^2 is the prototype.

Intuition

Think of L^2 as "$\mathbb{R}^\infty$ done carefully." Each instant t contributes a coordinate, so a continuous signal is a point with infinitely many coordinates. The integral $\int x\,\overline{y}\,\mathrm{d}t$ is the natural limit of the finite dot product $\sum x[n]\overline{y[n]}$ as the samples get dense. Almost every geometric statement

you trust in $\mathbb{R}^N$ – Pythagoras, projection minimizes error, an orthonormal basis lets you read coefficients by inner products – holds in L^2. The one new subtlety is convergence: an infinite sum of basis vectors must be shown to actually settle down to the signal, which is the technical content of "Fourier series converge."

Connection

In *Calculus From the Ground Up* the integral was area under a curve and the limit of Riemann sums. Here it returns wearing a new hat: $\int x\,\overline{y}\,dt$ is an *inner product*, the continuous limit of a dot product. The same expression you learned to compute as an area is the machine that measures the angle between two signals. And in *Probability From the Ground Up*, the expectation $\mathbb{E}[XY]$ of two zero-mean random signals is *also* an inner product – which is why "uncorrelated" (probability) and "orthogonal" (geometry) are the same word. The three books are describing one structure from three doors.

Formal

One caution: not every function lives in L^2. A pure infinite sinusoid $\cos(2\pi ft)$ has infinite energy over all of $\mathbb{R}$ ($\int \cos^2 = \infty$), so strictly it is not an L^2 function – which is exactly why the plain Fourier transform of a sinusoid is a (Dirac) spike rather than an ordinary function, and why in practice we always analyze a *windowed*, finite chunk that *is* square-integrable. The finite, sampled vectors a microcontroller actually stores never run into this; they always live safely in $\mathbb{C}^N$. We flag the subtlety so the integrals later do not surprise you, not because it will trouble the hardware.

3.9 A first numerical taste

The whole chapter is computable in a few lines. The snippet below builds two short signals, measures their inner product, energy, and the angle between them, then projects one signal onto the constant direction – exactly Example 3.2. Run it and watch the geometry turn into numbers.

Listing 3.1: Signals as vectors: inner product, energy, angle, projection.

```
import numpy as np

x = np.array([1., 2., 3., 4.])
y = np.array([4., 3., 2., 1.])

ip     = np.dot(x, y)                 # inner product  -> 20.0
energy = np.dot(x, x)                 # ||x||^2        -> 30.0
cos_th = ip / (np.linalg.norm(x) * np.linalg.norm(y))
print("inner product :", ip)
print("energy of x   :", energy)
print("cos(angle)    :", round(cos_th, 4), "-> angle deg:",
      round(np.degrees(np.arccos(cos_th)), 1))

# Project x onto the unit "average" direction b0 = (1,1,1,1)/2
b0 = np.ones(4) / 2.0                 # ||b0|| = 1
c0 = np.dot(x, b0)                    # coefficient    -> 5.0
```

```
proj = c0 * b0                          # best constant approx -> 2.5 each
residual = x - proj
print("c0, projection:", c0, proj)
print("residual.b0   :", round(np.dot(residual, b0), 12))  # ~0
```

The next snippet verifies Theorem 3.4 numerically: it forms the matrix of all pairwise inner products of the complex-exponential basis and confirms it is N times the identity – the basis really is orthogonal, by computer as well as by hand.

Listing 3.2: The sinusoidal basis is orthogonal: inner products vanish off the diagonal.

```
import numpy as np
N = 16
n = np.arange(N)
# basis vectors e_k[n] = exp(j 2 pi k n / N), stacked as columns
E = np.array([np.exp(1j*2*np.pi*k*n/N) for k in range(N)])   # E[k] is e_k
G = E.conj() @ E.T                                          # Gram matrix <e_k, e_m>
print(np.round(G.real, 6))   # = N on the diagonal, 0 elsewhere
print("max off-diagonal magnitude:", np.max(np.abs(G - N*np.eye(N))))
```

Try it yourself

Change N to any value and rerun the second snippet; the off-diagonal magnitude stays at the level of floating-point dust ($\sim 10^{-13}$), confirming exact orthogonality. Then replace the complex exponentials with *non*-harmonic frequencies (say $k + 0.5$ cycles per window) and watch the Gram matrix fill up with nonzero off-diagonal terms – the orthogonality is special to frequencies that complete a whole number of cycles in the window. That single observation is the reason "spectral leakage" exists and why we will later apply windows; you are seeing the seed of it here.

3.10 Toward the wrist tracker

Toward the wrist tracker

Here is how this chapter pays off for the gesture controller. The IMU on your wrist streams acceleration and angular velocity, sampled at, say, 200 Hz. A half-second of one axis is a vector in $\mathbb{R}^{100}$. Recognizing a gesture – a flick, a twist, a tap – comes down to two geometric operations you now own:

- **Correlation with templates.** Store a prototype vector for each gesture (Figure 3.5). A live window is classified by which template it has the largest (normalized) inner product with – the smallest angle. That is a nearest-direction classifier, nothing more than the cosine of Section 3.3.
- **Projection onto a frequency basis.** Many gestures differ in their *rhythm* and *tempo*, not their raw shape. Projecting the window onto the sinusoidal basis (Section 3.7) – i.e. taking its DFT – turns "how fast does the wrist oscillate" into a handful of coefficients that are far more stable to classify than the raw samples. The energy in a few frequency bins is a compact, rotation-of-viewpoint summary of the motion.

The same inner product does double duty: against time-domain templates it measures similarity of shape; against sinusoids it measures frequency content. We will build both pipelines in later chapters; this chapter gave you the vector space they live in.

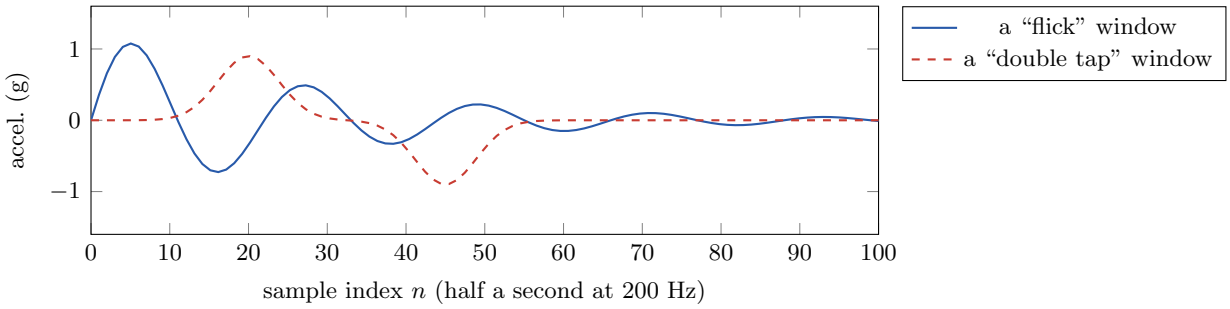


Figure 3.5: Two half-second windows of a single accelerometer axis, each a vector in $\mathbb{R}^{100}$. A flick is a brief damped oscillation; a double tap is a pair of sharp bursts. Classifying a live window is choosing which stored template it has the smallest angle to – a single normalized inner product per template.

Build it (hardware)

A concrete starting sketch for an Arduino-class board reading an IMU: keep a circular buffer of the last N samples per axis, and every few milliseconds compute the inner product of that buffer with each stored gesture template (a multiply-accumulate loop – the one primitive every DSP chip is built around). Normalize by the buffer's norm so loudness/intensity does not dominate. In fixed-point C the core is:

Listing 3.3: Inner-product (matched-filter) core for an MCU.

```
// returns sum_{i} buf[i]*tmpl[i]  (a single inner product / projection)
int32_t inner_product(const int16_t *buf, const int16_t *tmpl, int N) {
    int32_t acc = 0;
    for (int i = 0; i < N; ++i)
        acc += (int32_t)buf[i] * (int32_t)tmpl[i];   // multiply-accumulate
    return acc;
}
```

That loop is literally Definition 3.2 in silicon. Everything fancier – the FFT, FIR filtering, matched detection – is built from millions of these.

Common trap

What the wrist can actually sense. A single wrist-worn IMU measures the motion of the *wrist*, full stop. No amount of geometry recovers individual finger joint angles from it: the information simply is not in the signal. An IMU can reliably distinguish *coarse, discrete* gestures (a flick, a rotation, a double-tap, broad hand poses with characteristic motion), but continuous finger pose is out of reach. Finger-level discrimination needs a different sensor that sees muscle activity – surface EMG or force myography (FMG) on the forearm – and truly continuous individual-finger tracking needs sensors *on the fingers* (instrumented gloves

or rings). We will cover EMG and FMG in their own chapters. For now: signals-as-vectors is the right framework, but choose a feature your sensor can actually observe, or the most elegant inner product in the world will be projecting onto noise.

3.11 Summary

The arc of this chapter, in one breath: a signal is a vector; the inner product gives it a length (energy), an angle to other signals (correlation), and a notion of perpendicularity (orthogonality); an orthonormal basis lets you read off coordinates by single inner products (analysis) and rebuild the signal by a weighted sum (synthesis); projection onto a chosen basis is the best approximation within that basis; and the sinusoids form an orthogonal basis whose analysis-and-synthesis is the Fourier transform. The rest of the book is the consequences of those facts. Next we look hard at the basis vectors themselves – sinusoids, phasors, and complex exponentials – because understanding the ruler deeply is what makes the measurements meaningful.

3.12 Exercises

Exercise 3.1. Let $\mathbf{x} = (2, -1, 0, 3)$ and $\mathbf{y} = (1, 1, 1, 1)$ in $\mathbb{R}^4$. Compute $\langle \mathbf{x}, \mathbf{y} \rangle$, $\|\mathbf{x}\|$, $\|\mathbf{y}\|$, and the angle θ between them. Is $\mathbf{x}$ closer to being parallel or perpendicular to the constant signal $\mathbf{y}$? What does $\langle \mathbf{x}, \mathbf{y} \rangle$ tell you about the *average* of $\mathbf{x}$?

Exercise 3.2. Show directly from Definition 3.2 that the inner product is linear in its first argument: $\langle a\mathbf{u} + b\mathbf{v}, \mathbf{w} \rangle = a\langle \mathbf{u}, \mathbf{w} \rangle + b\langle \mathbf{v}, \mathbf{w} \rangle$ for real scalars a, b. Where in the proof of Theorem 3.1 was this used?

Exercise 3.3. Verify that $\mathbf{u} = (1, -1, 1, -1)$ and $\mathbf{w} = (1, 1, 1, 1)$ are orthogonal, and that together with $\mathbf{p} = (1, 1, -1, -1)$ and $\mathbf{q} = (1, -1, -1, 1)$ they form an orthogonal set of four vectors in $\mathbb{R}^4$. Normalize them to an orthonormal basis. (This is the 4-point Walsh–Hadamard basis – a "square-wave" cousin of the Fourier basis.)

Exercise 3.4. Using the orthonormal basis from the previous exercise, find the four coefficients $c_k = \langle \mathbf{x}, \mathbf{b}_k \rangle$ for $\mathbf{x} = (4, 2, 2, 0)$, then reconstruct $\mathbf{x}$ from them to check. Confirm Parseval: $\|\mathbf{x}\|^2 = \sum_k c_k^2$.

Exercise 3.5. Prove the Cauchy–Schwarz inequality $|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\| \, \|\mathbf{y}\|$ for real signals, by considering $\|\mathbf{x} - t\mathbf{y}\|^2 \geq 0$ as a quadratic in t and requiring its discriminant to be nonpositive. Why does this guarantee that the "angle" defined by $\cos\theta = \langle \mathbf{x}, \mathbf{y} \rangle / (\|\mathbf{x}\|\|\mathbf{y}\|)$ is always a real angle (i.e. $|\cos\theta| \leq 1$)?

Exercise 3.6. For $N = 4$, write out the four complex-exponential basis vectors $\mathbf{e}_k[n] = e^{\mathrm{j}2\pi kn/4}$ as explicit 4-tuples (their entries are among $\{1, \mathrm{j}, -1, -\mathrm{j}\}$). Verify by direct summation that $\langle \mathbf{e}_1, \mathbf{e}_3 \rangle = 0$ but $\langle \mathbf{e}_1, \mathbf{e}_1 \rangle = 4$, matching Theorem 3.4.

Exercise 3.7. Take the ramp $\mathbf{x} = (1, 2, 3, 4)$. Its best constant approximation was $(2.5, 2.5, 2.5, 2.5)$ (Example 3.2). Compute the energy of the residual and confirm that $\|\mathbf{x}\|^2 = \|\text{projection}\|^2 + \|\text{residual}\|^2$, an instance of Theorem 3.1. What fraction of the signal's energy is captured by the constant term alone?

Exercise 3.8. Using the geometric-series identity, evaluate $\sum_{n=0}^{N-1} e^{\mathrm{j}2\pi n/N}$ for $N = 5$ in two ways: (a) symbolically, showing it is exactly 0; (b) by plotting the five phasors on the unit circle and arguing geometrically that they sum to the center. Connect this to the "different frequencies cancel" claim of Section 3.7.

Exercise 3.9. (Continuous case.) On the interval $[0, 1]$, show that $\cos(2\pi t)$ and $\cos(4\pi t)$ are orthogonal under the integral inner product $\int_0^1 x(t)y(t)\,\mathrm{d}t$, and compute the norm of each. Why is $\int_0^1 \cos^2(2\pi t)\,\mathrm{d}t = \frac{1}{2}$ rather than 1, and what unit-norm version of the cosine would you use as a basis vector?

Exercise 3.10. (Sensor limits.) Suppose you record one accelerometer axis during a "wrist flick" and during a "finger snap." Explain, in terms of which information lives in the IMU signal vector, why a matched filter might distinguish the flick from rest but cannot tell you *which finger* snapped. What additional sensor (and what kind of signal vector) would carry the missing information? Cite the distinction between IMU, EMG/FMG, and on-finger sensing.

Exercise 3.11. (Computational.) Modify the second code listing to build the real sinusoidal basis (cosines and sines for $k = 0, \ldots, N/2$) for $N = 8$ and form its Gram matrix. Identify which pairs are orthogonal and which basis vectors are missing or redundant (think about $k = 0$ and $k = N/2$). How many real basis vectors do you need to span all of $\mathbb{R}^8$, and does that match the count of independent DFT coefficients?

Chapter 4

Sinusoids, Phasors, and Complex Exponentials

If frequency is the soul of signal processing, the sinusoid is its body. Almost everything we will do in this book – the Fourier series, the Fourier transform, filters, the frequency response of a system, sampling, modulation – is built on a single, beautifully simple object: a wave that goes up and down forever at one steady rate. In this chapter we are going to look at that object until we can *see* it from three directions at once: as a wiggle in time, as a shadow of something spinning, and as a single complex number that carries everything we need to know. The third view, the spinning one, is the secret weapon of the whole field. By the end of the chapter the formula $e^{\mathrm{j}\omega t}$ should feel less like a magic incantation and more like the most natural way to describe rotation that anyone ever invented.

4.1 What a sinusoid is, and the three numbers that pin it down

Start with the most familiar curve in mathematics, the cosine. In its bare form $\cos(t)$ it oscillates between $+1$ and -1, completing one full cycle every 2π units of t. A real signal in the world – a sound pressure, a voltage on a wire, the side-to-side sway of your forearm – is rarely so plain. To describe an actual oscillation we need to stretch it, speed it up, and slide it sideways. Those three operations correspond to exactly three numbers.

Definition 4.1 (Continuous-time sinusoid)**.** A (real) continuous-time sinusoid is any signal of the form

$$x(t) \;=\; A\cos(\omega t + \phi),$$

where $A \geq 0$ is the *amplitude*, ω is the *angular frequency* in radians per second, and ϕ is the *phase* in radians. The ordinary frequency is $f = \omega/(2\pi)$ in hertz (cycles per second), and the *period* is $T = 1/f = 2\pi/\omega$ seconds.

Let us take the three numbers one at a time, because each one has a clean geometric meaning that will pay off later.

Amplitude A is a vertical scaling. It is how far the wave reaches above and below its center line; the peak-to-peak swing is $2A$. Double A and you double the loudness of a tone, the brightness of a

flicker, the size of a sway. Amplitude has the same physical units as the signal itself – volts, pascals, g of acceleration.

Angular frequency ω is a horizontal scaling: it controls how fast the argument of the cosine runs. Because cosine repeats every time its argument advances by 2π, the signal completes a full cycle whenever ωt grows by 2π, i.e. every $T = 2\pi/\omega$ seconds. Bigger ω means the cycles are squeezed closer together in time – a higher pitch, a faster wiggle.

Phase ϕ is a horizontal shift. Writing $A\cos(\omega t + \phi) = A\cos\big(\omega(t + \phi/\omega)\big)$ shows that adding ϕ slides the whole wave earlier in time by ϕ/ω seconds. Phase tells you *where in its cycle* the wave is at the instant $t = 0$. A cosine has $\phi = 0$ (it starts at its peak); a sine is a cosine with $\phi = -\pi/2$, because $\sin(\omega t) = \cos(\omega t - \pi/2)$.

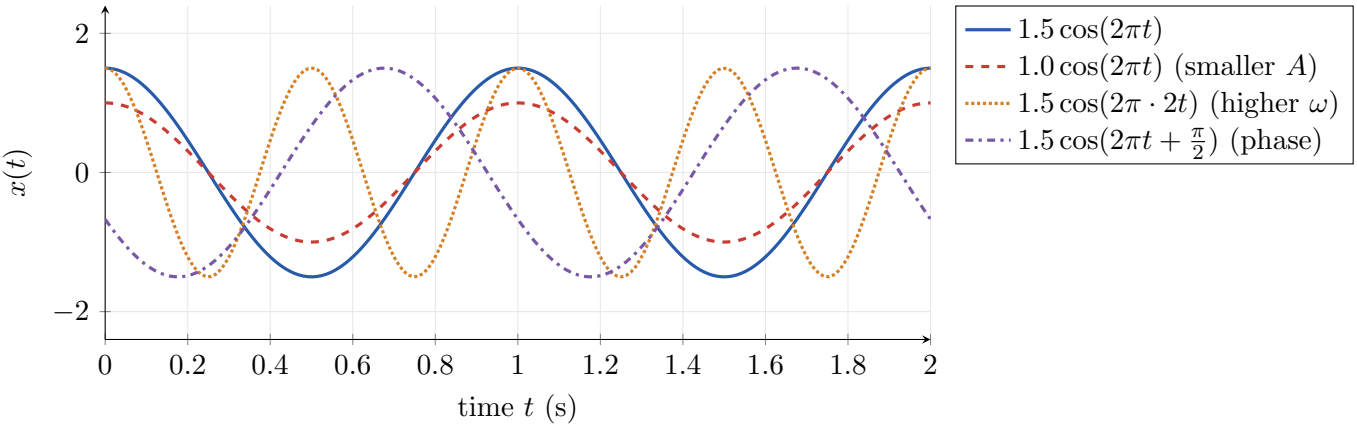


Figure 4.1: The three knobs. Amplitude scales the height, angular frequency squeezes the cycles together, and phase slides the wave sideways. Each curve changes exactly one parameter from the solid blue reference.

Common trap

Do not confuse f (hertz, cycles per second) with ω (radians per second). They differ by the factor 2π that converts one full turn into one cycle: $\omega = 2\pi f$. Mixing them up is the single most common unit error in DSP – a filter that should cut at 50 Hz ends up cutting at $50/(2\pi) \approx 8$ Hz, or at $50 \cdot 2\pi \approx 314$ Hz. Always write the units, and when in doubt ask: "does one cycle make my argument advance by 2π?"

Intuition

A sinusoid is the *only* waveform that keeps its shape under every operation we care about. Differentiate it, integrate it, delay it, run it through any linear time-invariant system, and out comes another sinusoid of the *same frequency* – only A and ϕ change. Square waves, triangle waves, your own EMG envelope: none of them have this property. That single fact is why the entire theory of frequency is built out of sinusoids and nothing else. We will make it precise in Section 4.7.

4.2 Why cosine alone is awkward: the addition problem

Here is a small task that will motivate everything that follows. Suppose two sources produce sinusoids of the *same* frequency ω but different amplitudes and phases, and they add together (two speakers, two reflections of a radio wave, two muscle fibers firing). What is the sum?

$$A_1 \cos(\omega t + \phi_1) + A_2 \cos(\omega t + \phi_2) \; = \, ?$$

We *know* the answer must again be a sinusoid of frequency ω, because adding two same-frequency cosines cannot manufacture a new rate of oscillation. So the result is $A_3 \cos(\omega t + \phi_3)$ for some new amplitude and phase. But finding A_3 and ϕ_3 with trig identities is genuinely painful. Expand each term with the angle-addition formula $\cos(\omega t + \phi) = \cos\phi \cos\omega t - \sin\phi \sin\omega t$, collect the $\cos\omega t$ and $\sin\omega t$ pieces, and then re-bundle them back into a single cosine. It works, but it is bookkeeping with no insight, and it gets worse with three or four terms.

Walking question

Before reading on, try the special case $\cos(\omega t) + \cos(\omega t + \pi/2)$ by hand with trig. You should get $\sqrt{2}\,\cos(\omega t - \pi/4)$. Notice how much algebra that took for two unit-amplitude terms. Now imagine summing the hundred sinusoids that make up a violin note. We need a better tool.

The better tool is to stop thinking of a sinusoid as a wiggle and start thinking of it as the shadow of something turning. That shift is the whole game.

4.3 Rotation on the unit circle, and where cosine comes from

Picture a point moving counterclockwise around a circle of radius A at a steady angular speed ω. At time t its angle (measured from the positive horizontal axis) is $\omega t + \phi$, where ϕ is wherever it started at $t = 0$. Its coordinates are, by the definition of sine and cosine,

$$\big(A\cos(\omega t + \phi)\,,\; A\sin(\omega t + \phi) \big).$$

Look at the first coordinate. *The horizontal position of a steadily rotating point is exactly our sinusoid.* A cosine is not fundamentally a wiggly curve at all; it is the *shadow* that a rotating point casts on the horizontal axis. The vertical shadow is a sine. The wiggle in time that we plotted in Figure 4.1 is what you get when you unroll that circular motion against a moving time axis.

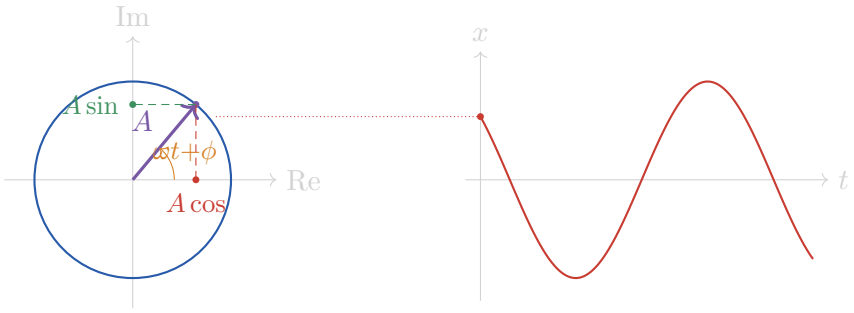


Figure 4.2: Left: a vector of length A rotates counterclockwise; at time t its angle is $\omega t + \phi$. Its projection onto the real axis (red) is $A\cos(\omega t + \phi)$; onto the imaginary axis (green) is $A\sin(\omega t + \phi)$. Right: unrolling the red projection against time gives the cosine wave. The wiggle is the shadow of the spin.

What is really happening

When you watch a cosine oscillate, what is *really* happening is that something is going around in a circle at constant speed and you are only allowed to see one coordinate of it. The "slowing down" near the peaks and the "speeding through" near zero – the non-uniform pace that makes the cosine curve look like it has personality – is an illusion of the projection. The rotation itself is perfectly uniform. All the apparent acceleration lives in the geometry of the shadow, not in the motion.

This reframing already explains the period. The rotating point returns to its start after one full turn, 2π radians, which at speed ω takes $T = 2\pi/\omega$ seconds. It explains amplitude: the radius of the circle. And it explains phase: the starting angle. Three numbers, three pieces of one circle. The only thing missing is a way to write "a rotating vector" as a single algebraic object so we can compute with it. For that we need the complex plane.

4.4 Euler's formula: rotation written as one number

A point in the plane with coordinates (a, b) can be packaged as a single complex number $z = a + \mathrm{j}b$, where j is the imaginary unit with $\mathrm{j}^2 = -1$. (Engineers write j rather than i, since i is reserved for electric current.) Addition of complex numbers is just vector addition of the points. The magic is in *multiplication*: multiplying by a complex number of magnitude 1 and angle θ rotates the plane by θ. So complex multiplication *is* rotation – which is exactly the operation we needed for sinusoids.

The bridge between the additive picture (coordinates) and the multiplicative picture (rotation) is Euler's formula, the most important single equation in this book.

Theorem 4.1 (Euler's formula)**.** *For any real angle θ,*

$$e^{\mathrm{j}\theta} \;=\; \cos\theta + \mathrm{j}\sin\theta.$$

Consequently $e^{\mathrm{j}\theta}$ is the point on the unit circle at angle θ, and $|e^{\mathrm{j}\theta}| = \sqrt{\cos^2\theta + \sin^2\theta} = 1$.

Why it works

Why should the exponential function, born from compound interest and growth, have anything to do with circles? Because both are defined by the same differential law, just with a twist of j. The real exponential satisfies $\frac{\mathrm{d}}{\mathrm{d}t}e^t = e^t$: its rate of change points the *same* direction as its value, so it grows. Now consider $u(t) = e^{\mathrm{j}t}$. Differentiating, $u'(t) = \mathrm{j}\,e^{\mathrm{j}t} = \mathrm{j}\,u(t)$. Multiplying by j is a 90° rotation, so the velocity of the point $u(t)$ is always exactly perpendicular to its position vector. A velocity forever perpendicular to the radius, with constant speed, is precisely the definition of uniform circular motion. The exponential whose rate is rotated by a quarter turn cannot grow outward; it can only *circle*. That is Euler's formula, read as a sentence about motion.

Focus

A second, algebraic, route. Write down the Taylor series of e^x and substitute $x = \mathrm{j}\theta$:

$$e^{\mathrm{j}\theta} = \sum_{n=0}^{\infty} \frac{(\mathrm{j}\theta)^n}{n!} = \underbrace{\left(1 - \tfrac{\theta^2}{2!} + \tfrac{\theta^4}{4!} - \cdots\right)}_{\cos\theta} + \mathrm{j}\underbrace{\left(\theta - \tfrac{\theta^3}{3!} + \tfrac{\theta^5}{5!} - \cdots\right)}_{\sin\theta},$$

using $\mathrm{j}^2 = -1$, $\mathrm{j}^3 = -\mathrm{j}$, $\mathrm{j}^4 = 1$ to sort the powers into real and imaginary buckets. The even powers reassemble the cosine series; the odd powers, each carrying one factor of j, reassemble the sine series. The companion calculus book derives these three series; here we just admire how cleanly they interlock.

Now feed time into the angle. A point rotating at angular speed ω, starting at angle ϕ on a circle of radius A, is the single complex-valued signal

$$z(t) \;=\; A\,e^{\mathrm{j}(\omega t+\phi)} \;=\; \underbrace{Ae^{\mathrm{j}\phi}}_{\text{constant}}\; e^{\mathrm{j}\omega t}.$$

This is the **complex exponential** (also called a complex sinusoid). Everything we wanted is in here. Its real part is our cosine, its imaginary part is our sine:

$$\operatorname{Re} z(t) = A\cos(\omega t + \phi), \qquad \operatorname{Im} z(t) = A\sin(\omega t + \phi).$$

And it factors beautifully: the time-dependence $e^{\mathrm{j}\omega t}$ (the spinning) separates from a constant $Ae^{\mathrm{j}\phi}$ that carries the amplitude and the starting angle. That constant has a name.

4.5 Phasors: freezing the spin

When every sinusoid in a problem shares the same frequency ω, the factor $e^{\mathrm{j}\omega t}$ is common to all of them – they all spin together, in lockstep. So we can mentally ride along with the rotation, "stop the clock," and keep only the part that distinguishes one sinusoid from another: its amplitude and phase. That frozen complex number is the phasor.

Definition 4.2 (Phasor)**.** The *phasor* of the sinusoid $A\cos(\omega t + \phi)$ is the complex number

$$X \;=\; A\,e^{\mathrm{j}\phi} \;=\; A\cos\phi + \mathrm{j}A\sin\phi.$$

The sinusoid is recovered as $x(t) = \operatorname{Re}\big(X\,e^{\mathrm{j}\omega t}\big)$. The magnitude $|X| = A$ is the amplitude; the angle $\arg X = \phi$ is the phase.

A phasor is an arrow in the complex plane. Its length is the amplitude, its direction is the phase. To get the actual signal back, you set this arrow spinning at ω and watch its real-axis shadow. Same-frequency sinusoids become *static* arrows, and – here is the payoff – **adding sinusoids becomes adding arrows.**

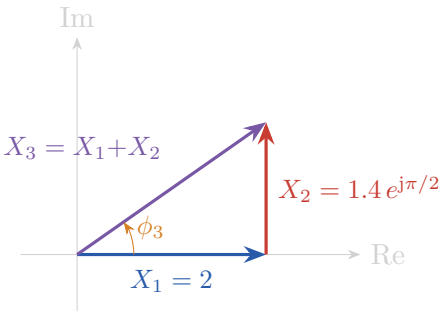


Figure 4.3: Adding two same-frequency sinusoids is tip-to-tail vector addition of their phasors. Here $2\cos\omega t + 1.4\cos(\omega t + \frac{\pi}{2})$ has phasor sum $X_3 = 2 + 1.4\text{j}$, so the result is $|X_3|\cos(\omega t + \phi_3)$ with $|X_3| = \sqrt{2^2 + 1.4^2} \approx 2.44$ and $\phi_3 = \arctan(1.4/2) \approx 0.61$ rad.

This dissolves the awkward addition problem of Section 4.2. Because Re is linear,

$$A_1\cos(\omega t + \phi_1) + A_2\cos(\omega t + \phi_2) = \text{Re}\Big[(X_1 + X_2)\,e^{\text{j}\omega t}\Big],$$

so we just add the phasors $X_1 + X_2 = X_3$ as complex numbers and read off the new amplitude and phase as $|X_3|$ and $\arg X_3$. No trig identities. Adding waves has become adding arrows.

Example 4.1 (The walking-question sum, finally easy)**.** Recompute $\cos(\omega t) + \cos(\omega t + \frac{\pi}{2})$. The phasors are $X_1 = 1\,e^{\text{j}0} = 1$ and $X_2 = 1\,e^{\text{j}\pi/2} = \text{j}$. Their sum is $X_3 = 1 + \text{j}$, which has magnitude $|1 + \text{j}| = \sqrt{2}$ and angle $\arg(1 + \text{j}) = \pi/4$. Therefore

$$\cos(\omega t) + \cos\big(\omega t + \tfrac{\pi}{2}\big) = \sqrt{2}\,\cos\big(\omega t + \tfrac{\pi}{4}\big).$$

Two lines, and we even see *why*: a horizontal unit arrow plus a vertical unit arrow makes a diagonal arrow of length $\sqrt{2}$ at 45°.

Example 4.2 (Three-way sum)**.** Find $3\cos(\omega t) + 2\cos(\omega t - \frac{2\pi}{3}) + 2\cos(\omega t + \frac{2\pi}{3})$. Phasors: $X_1 = 3$, $X_2 = 2e^{-\text{j}2\pi/3} = 2(-\frac{1}{2} - \text{j}\frac{\sqrt{3}}{2}) = -1 - \text{j}\sqrt{3}$, and $X_3 = 2e^{\text{j}2\pi/3} = -1 + \text{j}\sqrt{3}$. Summing: $X_2 + X_3 = -2$ (the imaginary parts cancel), so $X_1 + X_2 + X_3 = 3 - 2 = 1$. The result is simply $1 \cdot \cos(\omega t)$. The two side terms, symmetric about the real axis, partially cancel the main term. Try that with trig identities and you will appreciate the arrows.

Why it works

Why are we allowed to add the phasors and ignore the spinning $e^{\text{j}\omega t}$? Because all the terms share it. Factor it out: $\sum_k \text{Re}(X_k e^{\text{j}\omega t}) = \text{Re}\big((\sum_k X_k)\,e^{\text{j}\omega t}\big)$. The common rotation is applied once, at the end, to the summed arrow. This only works when the frequencies match – if two sinusoids spin at different rates, their arrows drift relative to each other and there is no single frozen picture. Different frequencies do not add to a single sinusoid; that is the doorway to the Fourier series.

Connection

The phasor sum is an inner-product / vector statement in disguise. In the companion linear-algebra book, a complex number $a+\mathrm{j}b$ is the vector (a, b), complex addition is vector addition, and $|\cdot|$ is the Euclidean norm. Everything in this section is two-dimensional vector geometry wearing a complex-number costume. Later, when we project signals onto sinusoids in the Fourier transform, the inner product $\langle x, y\rangle$ from that book will be the engine – and the complex exponential will be the family of vectors we project onto.

4.6 Two counter-rotating arrows: the negative frequency

We defined the phasor by taking the real part of a single rotating arrow. There is a second, perfectly symmetric, way to recover a real cosine, and it will matter enormously once we reach the Fourier transform. Start from Euler's formula written both ways:

$$e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta, \qquad e^{-\mathrm{j}\theta} = \cos\theta - \mathrm{j}\sin\theta.$$

Add them and the imaginary parts cancel; subtract them and the real parts cancel. Solving,

$$\boxed{\cos\theta = \frac{e^{\mathrm{j}\theta} + e^{-\mathrm{j}\theta}}{2}, \qquad \sin\theta = \frac{e^{\mathrm{j}\theta} - e^{-\mathrm{j}\theta}}{2\mathrm{j}}.}$$

Setting $\theta = \omega t$, a real cosine is the average of two arrows: one of length $\frac{1}{2}$ spinning counterclockwise at $+\omega$, and one of length $\frac{1}{2}$ spinning clockwise at $-\omega$. At every instant their vertical (imaginary) components are mirror images and cancel, leaving a purely real, horizontal sum that oscillates as $\cos\omega t$.

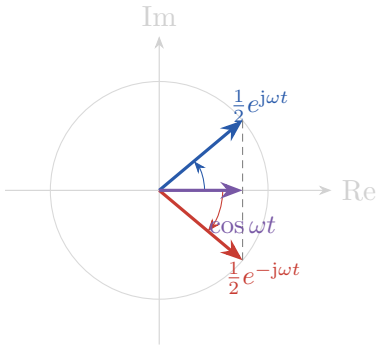


Figure 4.4: A real cosine as two counter-rotating half-arrows. The blue one spins at $+\omega$, the red at $-\omega$; their imaginary parts always cancel and their real parts add, tracing $\cos\omega t$ on the horizontal axis. This is the origin of "negative frequency."

Intuition

"Negative frequency" is not mystical and it is not less than zero in any physical sense. It is just *the other direction of rotation.* Counterclockwise is positive, clockwise is negative. A real-valued signal, having no imaginary part to keep track of a preferred sense of spin, must always contain equal and opposite amounts of both – which is why the spectrum of a real

signal is symmetric about zero. When we get to the Fourier transform you will see those mirrored peaks at $+\omega$ and $-\omega$ and now you will know they are the two arrows of every real cosine.

Common trap

A single complex exponential $e^{j\omega t}$ has only *one* frequency, $+\omega$, and its spectrum is a single peak with nothing at $-\omega$. It is the real cosine, not the complex exponential, that needs both. Quadrature receivers and the analytic signal of Chapter 25 exploit exactly this: by carrying both a real and an imaginary channel you can represent a one-sided spectrum and distinguish $+\omega$ from $-\omega$ – something a single real wire physically cannot do.

4.7 Complex exponentials are the eigenfunctions of LTI systems

We promised in Section 4.1 that sinusoids are special because they survive linear time-invariant (LTI) systems intact. Now we can state *why*, and the reason is the deepest single idea in the chapter – it is the reason the Fourier transform exists at all.

A system is *linear* if scaling and adding inputs scales and adds outputs, and *time-invariant* if delaying the input merely delays the output by the same amount (the system's behavior does not depend on *when* you poke it). Almost every device we model – a filter, a resonant cavity, an RC circuit, the mechanical response of soft tissue – is LTI to good approximation. Feed such a system the complex exponential $x(t) = e^{j\omega t}$. The claim is that the output is the *same* exponential, only scaled by a complex number that depends on ω:

$$e^{j\omega t} \xrightarrow{\text{LTI system } \mathcal{H}} H(\omega)\, e^{j\omega t}.$$

Why it works

Here is the argument in one breath. Let $y(t) = \mathcal{H}\{e^{j\omega t}\}$ be the output. Time-invariance says that delaying the input by τ delays the output by τ: $\mathcal{H}\{e^{j\omega(t-\tau)}\} = y(t-\tau)$. But $e^{j\omega(t-\tau)} = e^{-j\omega\tau}\, e^{j\omega t}$, and the scalar $e^{-j\omega\tau}$ pulls out by linearity, so the left side is $e^{-j\omega\tau}\, y(t)$. Therefore $y(t-\tau) = e^{-j\omega\tau}\, y(t)$ for all τ. A function that, when shifted by any τ, comes back as itself times $e^{-j\omega\tau}$ must be a constant multiple of $e^{j\omega t}$. Writing that constant as $H(\omega)$ gives $y(t) = H(\omega)e^{j\omega t}$. The exponential is preserved; only its complex amplitude is multiplied.

In linear-algebra language: the complex exponential is an *eigenfunction* of every LTI system, and $H(\omega)$ is the corresponding *eigenvalue*. The system does not bend or reshape the exponential – it can only stretch it (change amplitude, $|H(\omega)|$) and rotate it (change phase, $\arg H(\omega)$).

What is really happening

This is what is *really* happening when audio engineers speak of a filter's "frequency response." The filter is a linear map, and exponentials are its eigenvectors. An eigenvector, fed in, comes out merely scaled. So if you describe an arbitrary signal as a combination of exponentials – which is exactly what the Fourier transform does – then the filter's effect is trivial: multiply each exponential component by its own number $H(\omega)$. A hard problem ("what does this

circuit do to my complicated waveform?") becomes a list of independent multiplications. The entire reason we decompose signals into sinusoids is that sinusoids are the coordinate system in which every LTI system is diagonal.

A single eigenvalue $H(\omega)$ is one complex number, but as ω ranges over all frequencies it traces two real curves: a gain $|H(\omega)|$ and a phase $\arg H(\omega)$. Together they are the system's *frequency response*. Figure 4.5 shows the pair for a simple one-pole low-pass, $H(\omega) = 1/(1 + \mathrm{j}\omega/\omega_c)$: low frequencies pass at near-unit gain and near-zero phase, high frequencies are attenuated and delayed. Every LTI system you will meet is described by one such gain curve and one such phase curve.

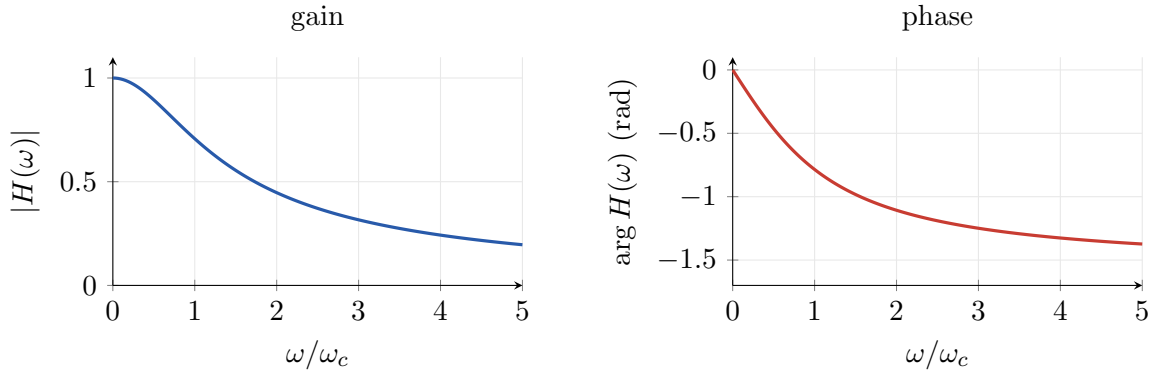


Figure 4.5: Frequency response of a one-pole low-pass $H(\omega) = 1/(1 + \mathrm{j}\omega/\omega_c)$. Left: the gain $|H|$ falls from 1 toward 0, reaching $1/\sqrt{2}$ at $\omega = \omega_c$. Right: the phase $\arg H = -\arctan(\omega/\omega_c)$ swings from 0 down toward $-\pi/2$. Feeding in $e^{\mathrm{j}\omega t}$ returns the same exponential scaled by this one complex number.

For a real input cosine the same logic applies through the two counter-rotating arrows of Section 4.6: the $+\omega$ arrow is scaled by $H(\omega)$ and the $-\omega$ arrow by $H(-\omega)$, and the result reassembles into

$$\cos(\omega t) \xrightarrow{\mathcal{H}} |H(\omega)| \cos\big(\omega t + \arg H(\omega)\big).$$

Same frequency in, same frequency out – amplitude scaled by the gain $|H(\omega)|$, time shifted by the phase $\arg H(\omega)$. That is the promise from the start of the chapter, now proved.

Connection

"Eigenfunction," "eigenvalue," "diagonal in this basis": these are the words of the companion linear-algebra book, applied to vectors that happen to be functions. An LTI system is a linear operator on the infinite-dimensional space of signals; the Fourier transform is the change of basis that diagonalizes it. Chapters 12–14 build the convolution machinery that computes $H(\omega)$ from a system's impulse response.

4.8 Discrete-time complex exponentials (a first look)

Real hardware samples. Your microcontroller does not see $\cos(\omega t)$ for continuous t; it sees a sequence of numbers $x[n] = A\cos(\Omega n + \phi)$ taken every T_s seconds, where n is an integer sample index and

$\Omega = \omega T_s$ is the *discrete* angular frequency in radians *per sample*. The complex version is $x[n] = Ae^{j\phi}e^{j\Omega n}$ – a point that hops around the unit circle by Ω radians at each tick instead of sweeping continuously.

Two surprises arrive the moment we discretize, and both come straight from the geometry of hopping around a circle. We mention them now and develop them fully in Chapters 5 and 18.

Periodicity in frequency. Because $e^{j2\pi n} = 1$ for every integer n, increasing Ω by 2π changes nothing: $e^{j(\Omega+2\pi)n} = e^{j\Omega n}$. Two discrete frequencies that differ by 2π produce *identical* sample sequences. A continuous sinusoid has a distinct identity at every frequency from 0 to ∞; a discrete one repeats every 2π. All distinct discrete frequencies live in a single window of width 2π, usually taken as $(-\pi, \pi]$.

Aliasing. Two different continuous tones can land on the same samples. If Ω and $\Omega + 2\pi$ give the same hops, then a fast continuous wave can masquerade as a slow one once sampled. The highest distinct frequency is $\Omega = \pi$, half a turn per sample – the Nyquist limit, which Chapter 18 turns into the famous sampling theorem.

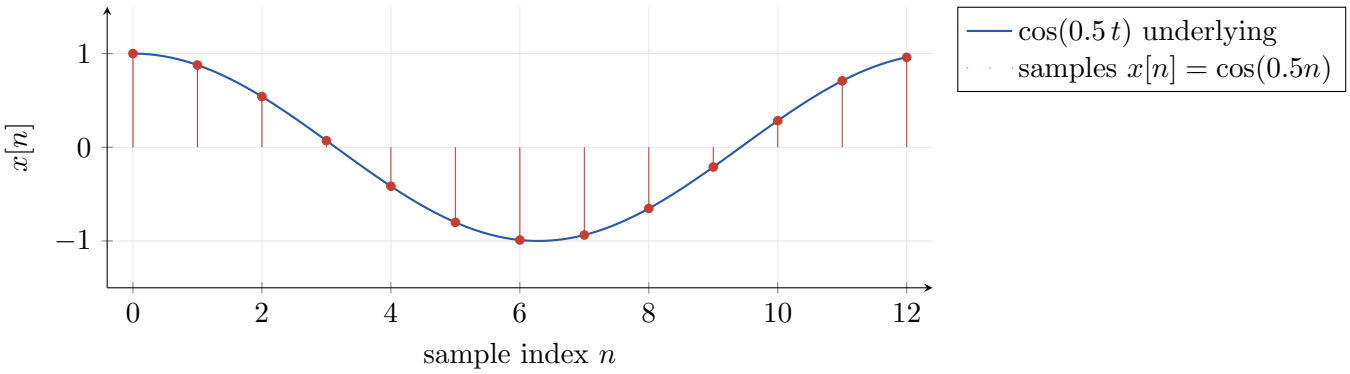


Figure 4.6: A discrete-time sinusoid is samples of the rotating point taken at integer ticks. Here $\Omega = 0.5$ rad/sample. Increasing Ω by 2π would leave every red dot exactly where it is – the source of aliasing and the periodicity of discrete frequency.

Figure 4.7 makes the masquerade concrete. A slow tone and a much faster one, chosen so their frequencies differ by exactly one full turn per sample, pass through the very same integer samples. Looking only at the dots, you cannot tell which wave produced them.

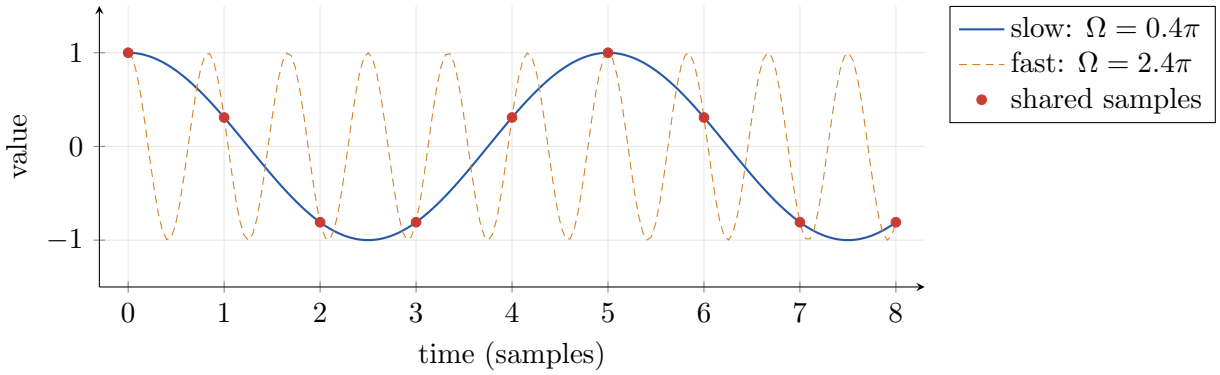


Figure 4.7: Aliasing. The slow tone ($\Omega = 0.4\pi$) and the fast tone ($\Omega = 2.4\pi$) differ by exactly 2π per sample, so they agree at every integer tick (red dots). From the samples alone the two are indistinguishable; the faster one is aliased onto the slower.

4.9 A worked round trip in code

Let us verify the phasor identity numerically – it is reassuring to watch the arrows do the work. The snippet adds two same-frequency sinusoids directly and via their phasors, and confirms the two agree.

```python
import numpy as np

w  = 2*np.pi*5.0          # 5 Hz, angular
t  = np.linspace(0, 1, 2000)

# two sinusoids of the SAME frequency
A1, p1 = 2.0,  0.0
A2, p2 = 1.4,  np.pi/2

x_direct = A1*np.cos(w*t + p1) + A2*np.cos(w*t + p2)

# phasor sum: add complex amplitudes, then read magnitude and angle
X = A1*np.exp(1j*p1) + A2*np.exp(1j*p2)
A3, p3 = np.abs(X), np.angle(X)
x_phasor = A3*np.cos(w*t + p3)

print(f"A3 = {A3:.4f}, phi3 = {p3:.4f} rad")      # -> 2.4413, 0.6107
print("max error:", np.max(np.abs(x_direct - x_phasor)))  # ~1e-15
```

Listing 4.1: Adding sinusoids: time-domain vs. phasor.

The maximum difference is at the level of floating-point round-off: the two computations are the same operation seen from two sides. The phasor route did the hard part – finding A_3 and ϕ_3 – with a single complex addition.

Try it yourself

Modify the snippet to add *three* sinusoids and reproduce Example 4.2 (expect $A_3 = 1$, $\phi_3 = 0$). Then deliberately break the assumption: give the second sinusoid a *different* frequency (6 Hz instead of 5). Plot the sum. You will see it is no longer a clean sinusoid – it beats and wanders – because the phasor trick requires a common ω. That failure is the whole motivation for Fourier analysis: when frequencies differ, you cannot collapse to one arrow; you must keep them all.

4.10 Toward the wrist tracker

Toward the wrist tracker

Why does a chapter on sinusoids matter for a wrist-worn gesture controller that will output MIDI? Because nearly every signal the device handles is best understood as a superposition of rotating phasors.

Tremor and motion. When the forearm oscillates – a deliberate shake, a vibrato gesture, or involuntary physiological tremor near 8–12 Hz – the accelerometer and gyroscope axes each read an approximate sinusoid. The *same* physical oscillation appears on the x, y, and z axes with different amplitudes and *phases*, exactly the same-frequency-different-phase situation of Section 4.5. Combining the axes is phasor addition; the phase differences between axes encode the *direction* of the motion in space.

The frequency response is your filter. The eigenfunction property of Section 4.7 is the reason we can design a digital filter to keep the slow, intentional gesture band and reject the fast tremor band: each frequency component is multiplied by its own gain $|H(\omega)|$, independently. We will lean on this constantly.

A MIDI vibrato. If you map the *frequency* of a detected wrist oscillation to a MIDI modulation-wheel value and its *amplitude* to depth, a performer can "dial in" vibrato by shaking faster or wider. Estimating that single frequency and amplitude is, at heart, finding the dominant phasor in the signal – the simplest possible Fourier analysis.

Toward the wrist tracker

A real limit, stated plainly. A wrist- or forearm-worn IMU measures the motion of the *whole limb*. It can sense gross orientation, swings, taps, and oscillations like the tremor or vibrato above. It *cannot*, by itself, recover the continuous joint angles of individual fingers: bending only the index finger produces almost no signal at the wrist. Finger-level intent requires a different physics – surface EMG or force myography (FMG) reading the muscle activity in the forearm (Chapter 37), and continuous finger *pose* requires sensors on the fingers themselves, such as instrumented rings. Treat the IMU as a detector of coarse, often discrete, whole-hand gestures, and reach for EMG/FMG when you need to know what the fingers are doing. We will keep returning to this boundary so that nothing we build over-promises. This framing is well supported by the wearable-sensing literature (surface-EMG gesture studies, and the known limitations of inertial-only hand tracking).

Build it (hardware)

A concrete first experiment. Strap an MPU-6050 IMU to your forearm, sample the accelerometer at, say, 200 Hz, and shake your hand back and forth at a steady rate. Then estimate the oscillation frequency from the zero-crossings of one axis – the rate at which the cosine's shadow crosses its center line is twice the frequency, since each cycle has two crossings.

```
// ax[]: N recent accelerometer samples, mean-removed; fs = sample rate (Hz)
int crossings = 0;
for (int n = 1; n < N; n++) {
    if ((ax[n-1] <= 0.0f && ax[n] > 0.0f) ||   // upward crossing
        (ax[n-1] >= 0.0f && ax[n] < 0.0f))     // downward crossing
        crossings++;
}
float freq_hz = (0.5f * crossings) * (fs / (float)N);  // cycles per second
```

Listing 4.2: Crude zero-crossing frequency estimate on a microcontroller.

This is the cheapest possible frequency estimator and it works for a clean single tone. It is fragile against noise and multiple tones, which is precisely why we will build the Fourier transform: to read *all* the frequencies at once, robustly. But feeling a number come out that tracks your shaking is a good first taste of turning motion into a control signal.

4.11 History and a note on notation

History

Leonhard Euler published the identity $e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta$ in 1748 in his *Introductio in analysin infinitorum*, though related results were in the air (Roger Cotes had a logarithmic form in 1714). The special case $e^{\mathrm{j}\pi} + 1 = 0$ – linking e, j, π, 1, and 0 in one line – is routinely voted the most beautiful equation in mathematics. The *phasor* as an engineering tool came later, popularized by Charles Proteus Steinmetz in the 1890s to tame alternating-current circuit analysis: by freezing the 50/60 Hz spin into static complex numbers, he turned differential equations into ordinary algebra and made the AC power grid designable. Every time you add two arrows instead of wrestling trig identities, you are using Steinmetz's trick.

Reading note

A notation caution for newcomers crossing between fields. Mathematicians and physicists write the imaginary unit as i and often the complex exponential as $e^{-i\omega t}$ (note the minus sign); electrical engineers write j and frequently $e^{+\mathrm{j}\omega t}$. The sign convention only flips the sense of rotation and the sign of phase; nothing physical changes, but you must pick one convention and hold it. This book uses j and the $e^{+\mathrm{j}\omega t}$ convention throughout.

Why this matters

Step back and notice what we have bought. We started with three real numbers (A, ω, ϕ) and a stubborn trig-identity problem. By recognizing the cosine as the shadow of a rotation, we compressed amplitude-and-phase into a *single* complex number, turned adding waves into adding arrows, and – almost for free – discovered that these same complex exponentials are the eigenfunctions that make every linear system simple. One geometric idea, rotation, unified algebra, signals, and systems. That unification is why the rest of the book can be as clean as it is.

Summary

- A real sinusoid $A\cos(\omega t + \phi)$ is fixed by three numbers: amplitude (vertical scale), angular frequency (how fast, $\omega = 2\pi f$), and phase (sideways shift / starting angle).
- A cosine is the horizontal shadow of a point rotating at constant speed; a sine is the vertical shadow. The wiggle's apparent acceleration is an artifact of projection.
- Euler's formula $e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta$ makes a rotating vector a single number. The complex exponential $Ae^{\mathrm{j}\phi}e^{\mathrm{j}\omega t}$ packages all three parameters; the constant $X = Ae^{\mathrm{j}\phi}$ is the *phasor*.
- Same-frequency sinusoids add by adding phasors tip-to-tail, replacing trig identities with vector geometry.
- A real cosine is two counter-rotating half-arrows at $\pm\omega$; this is what "negative frequency" means and why real spectra are symmetric.
- Complex exponentials are the eigenfunctions of every LTI system: in, $H(\omega)$ times the same exponential, out. This is the reason we decompose signals into sinusoids at all.
- Discrete-time exponentials hop around the circle; frequency becomes periodic with period 2π, foreshadowing aliasing and Nyquist.

4.12 Exercises

Exercise 4.1 (Units and conversions)**.** A signal completes 440 cycles per second (the musical note A4). Give its frequency f, angular frequency ω, and period T. If you sample it at $f_s = 8000$ Hz, what is the discrete angular frequency $\Omega = \omega/f_s$ in radians per sample? Is it below the Nyquist limit π?

Exercise 4.2 (Cosine, sine, and phase)**.** Express $\sin(\omega t)$, $-\cos(\omega t)$, and $-\sin(\omega t)$ each in the form $\cos(\omega t + \phi)$ with $\phi \in (-\pi, \pi]$. Sketch the four phasors 1, and the three you found, as arrows in the complex plane and confirm they sit at 0, $-\pi/2$, π, $\pi/2$.

Exercise 4.3 (Phasor addition by hand)**.** Add $3\cos(\omega t) + 4\cos(\omega t - \frac{\pi}{2})$ using phasors. Find the resultant amplitude and phase, and write the answer as a single cosine. (Hint: the phasors are 3 and $-4\mathrm{j}$; the geometry is a 3–4–5 right triangle.)

Exercise 4.4 (When cancellation is total)**.** For what phase difference $\Delta\phi = \phi_2 - \phi_1$ do two *equal-amplitude*, same-frequency sinusoids cancel exactly to zero? Explain with the arrow picture, and

give the amplitude of the sum as a function of $\Delta\phi$ in general. (You should find the sum amplitude is $2A|\cos(\Delta\phi/2)|$.)

Exercise 4.5 (Euler both ways)**.** Using $\cos\theta = (e^{\mathrm{j}\theta} + e^{-\mathrm{j}\theta})/2$, derive the identity $\cos^2\theta = \frac{1}{2} + \frac{1}{2}\cos(2\theta)$ by multiplying out the exponentials. Notice you never needed to remember a half-angle formula – the exponentials did it for you.

Exercise 4.6 (Product to sum)**.** Show that $\cos(\omega_1 t)\cos(\omega_2 t) = \frac{1}{2}\cos((\omega_1-\omega_2)t) + \frac{1}{2}\cos((\omega_1+\omega_2)t)$ by writing each cosine with Euler's formula and expanding. This "beating" of two close frequencies is the basis of amplitude modulation (Chapter 23) and of the amplitude wobble you saw in the lab exercise.

Exercise 4.7 (The eigenfunction argument)**.** Suppose an LTI system has output $y(t) = \mathcal{H}\{e^{\mathrm{j}\omega t}\}$. Using only linearity and time-invariance, fill in each step of the derivation in Section 4.7 that $y(t) = H(\omega)e^{\mathrm{j}\omega t}$. Where exactly is time-invariance used, and where is linearity?

Exercise 4.8 (Frequency response of a delay)**.** A pure delay system outputs $y(t) = x(t-\tau)$. Show it is LTI, and by feeding in $x(t) = e^{\mathrm{j}\omega t}$ find its eigenvalue $H(\omega)$. Confirm $|H(\omega)| = 1$ (a delay does not change amplitude) and $\arg H(\omega) = -\omega\tau$ (it adds a phase proportional to frequency – "linear phase").

Exercise 4.9 (Discrete periodicity)**.** Show that the discrete sequences $x_1[n] = \cos(0.3\pi\, n)$ and $x_2[n] = \cos(2.3\pi\, n)$ are identical for all integers n. Which one corresponds to a frequency inside the fundamental band $(-\pi, \pi]$? What does this say about a continuous tone that, when sampled, produces these numbers?

Exercise 4.10 (Three-axis tremor, conceptual)**.** A forearm tremor at $f = 9$ Hz appears on the three accelerometer axes as $a_x = 0.20\cos(\omega t)$, $a_y = 0.15\cos(\omega t - 1.2)$, $a_z = 0.05\cos(\omega t + 0.4)$ (all in g, with $\omega = 2\pi \cdot 9$). Using phasors, find the amplitude and phase of the combined projection $a_x + a_y + a_z$. Then explain in one sentence why the *phase differences* between axes carry information about the direction of the shaking that the amplitudes alone do not.

Exercise 4.11 (Build reflection)**.** The zero-crossing estimator in the build box reports half the crossing count times f_s/N. Explain why a single noisy sample near a zero crossing can cause it to badly over-count, and propose one simple fix (e.g. a small hysteresis band or a low-pass pre-filter). Connect your answer to why we will prefer Fourier-based frequency estimation later in the book.

Part II

Frequency and the Fourier Family

Chapter 5

The Idea of Frequency

5.1 What we are really after

We have spent the previous chapters building two ideas with our bare hands. First, a signal is a function: a number that changes as time goes by, or as you move across a sensor. Second, a sinusoid is the most special function there is, because it is the shape that survives being differentiated, integrated, delayed, and pushed through any linear time-invariant system without changing its *shape* – only its size and its timing. In this chapter we put those two ideas together and ask the question that organizes the whole rest of the book:

Focus

What does it mean to say a signal "contains" a frequency? And why is it so powerful to describe a signal not by its values moment to moment, but by the mixture of frequencies it is built from?

This is not a small question. The answer – that *any* reasonable signal can be written as a sum of pure sinusoids, and that this sum is unique – is the Fourier idea, and it took a century of argument to make precise. We are not going to prove it yet. We are going to *see* it. By the end of the chapter you will be able to look at a wiggly waveform and have a real intuition for what its spectrum must look like, and look at a spectrum and imagine the waveform. That two-way fluency is the single most useful skill in all of signal processing.

Toward the wrist tracker

The wrist controller we are building toward lives or dies on this idea. A finger tap, a wrist flick, a tremor, the 60 Hz hum the wall socket induces in the EMG leads, the steady gravity vector the accelerometer reports – each of these shows up as energy at a particular *frequency* or band of frequencies. "Is the user tapping rhythmically at 4 Hz, or is that just arm sway at 0.5 Hz?" is a question about the spectrum, not about any single sample. Learning to think in frequency is learning the native language of the device.

5.2 Frequency as a rate of repetition

Start with the plainest possible meaning. A signal is *periodic* with period T if it repeats exactly after T seconds:

$$x(t+T) = x(t) \quad \text{for all } t.$$

The smallest such T is *the* period. The *frequency* is just how many times that repetition happens per second:

$$f = \frac{1}{T} \qquad \text{[cycles per second, or hertz, Hz].}$$

A sinusoid that completes one full cycle every 0.25 s repeats 4 times a second, so $f = 4$ Hz. Double the frequency and the pattern is squeezed to half the width in time; halve the frequency and it stretches to twice the width. That inverse relationship between time and frequency – squeeze one, stretch the other – is a theme we will meet again and again, all the way to the uncertainty principle in Chapter 10.

Intuition

Frequency is a *rate*. It answers "how fast does this pattern recur?" Low frequency means slow, lazy, broad wiggles. High frequency means fast, tight, needle-like wiggles. Everything else in this chapter is an elaboration of that one sentence, made precise enough to compute with.

5.2.1 Ordinary frequency versus angular frequency

There are two units for the same physical rate, and confusing them is the single most common bookkeeping error in the field. A pure cosine can be written either way:

$$x(t) = \cos(2\pi f t) = \cos(\omega t), \qquad \omega = 2\pi f.$$

- **Ordinary frequency** f is measured in hertz: cycles per second. It is what a tuner, an oscilloscope, or a datasheet quotes. A 440 Hz tuning fork has $f = 440$.
- **Angular frequency** ω is measured in radians per second. One full cycle is 2π radians of phase, so $\omega = 2\pi f$. It is what the math wants, because the argument of cos and of $e^{j\theta}$ is an *angle*.

The reason both survive is that hardware counts cycles and mathematics counts angles, and there are 2π radians in a cycle. Keep a mental dictionary: whenever you see a bare ω, divide by 2π in your head to get the hertz you can feel. Table 5.1 collects the three names.

Common trap

The phrase "a frequency of ω" is ambiguous unless units are stated. Worse, the discrete world (Chapter 6 onward) uses yet a third creature: the normalized *digital* frequency in radians per *sample*, often also called ω, where π corresponds to half the sampling rate. Always ask: per second or per sample? Hertz or radians? When in doubt, write $2\pi f t$ in full – it is never wrong.

Name	Symbol	Units	Relation
Period	T	seconds	$T = 1/f$
Ordinary freq.	f	hertz (cycles/s)	$f = 1/T$
Angular freq.	ω	radians/second	$\omega = 2\pi f$

Table 5.1: The three names for one rate. Memorize $\omega = 2\pi f$ and $f = 1/T$.

5.3 The atoms: pure sinusoids and their three numbers

A general single-frequency real signal needs exactly three numbers to pin down:

$$x(t) = A\cos(2\pi f t + \phi).$$

- A – the *amplitude*: how tall the wave is, how loud, how strong.
- f – the *frequency*: how fast it repeats.
- ϕ – the *phase*: where in its cycle it is at $t = 0$, i.e. a sideways shift in time. A phase of $-\pi/2$ turns a cosine into a sine.

Three numbers, total. That economy is the whole point. A raw recorded waveform might be ten thousand samples; if it happens to be a single sinusoid, those ten thousand numbers carry only *three* numbers' worth of information. Frequency-domain thinking is, at bottom, the search for that hidden economy in every signal. Figure 5.1 shows two such atoms side by side.

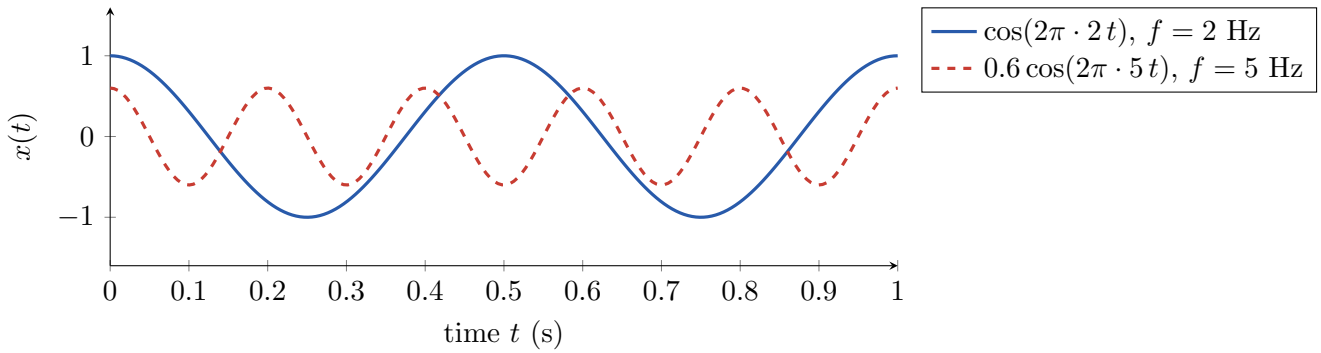


Figure 5.1: Two pure sinusoids over one second. The blue wave completes 2 cycles (period 0.5 s); the red wave completes 5 cycles (period 0.2 s) and is shorter. Higher frequency = tighter wiggle; smaller amplitude = shorter wave.

Connection

In the linear-algebra companion you met orthonormal bases: a handful of perpendicular unit vectors so that every vector is a unique weighted sum of them, and the weights are inner products. Keep that picture warm. The Fourier claim is exactly that the sinusoids form such a basis for the space of signals, with the inner product $\langle x, y\rangle = \int x(t)\,\overline{y(t)}\,dt$. "How much of frequency f does x contain?" will literally be an inner product. We are sneaking up on that.

5.4 Why decompose into frequencies at all?

Before the machinery, the motivation. Why is breaking a signal into sinusoids worth the trouble, rather than, say, breaking it into a sum of little spikes or boxes?

Why it works

Reason 1: sinusoids are the eigenfunctions of every LTI system. Push a sinusoid through any linear time-invariant system – a filter, a cable, a room, a mechanical linkage – and out comes *the same frequency*, only rescaled in amplitude and shifted in phase. No new frequencies are ever created. We will prove this in Chapter 12, but the consequence is decisive: if you describe the input as a sum of sinusoids, the system acts on each one *independently and trivially* (multiply by a complex number). A horrendous integral (convolution) becomes elementary multiplication. Spikes and boxes do not have this property; sinusoids alone do.

Why it works

Reason 2: it matches how the world generates and senses signals. A plucked string vibrates at a fundamental and its harmonics. A motor hums at its rotation rate and multiples. The cochlea in your ear is literally a bank of frequency-tuned hairs – you *hear* in the frequency domain. AC mains radiate at 50 or 60 Hz. When the physics speaks in frequencies, analysis in frequencies is not a trick; it is reading the world in its own handwriting.

Why it works

Reason 3: it separates things that overlap in time but not in frequency. A drum hit and a flute note can happen at the same instant – hopelessly tangled in the time waveform – yet sit in completely different frequency bands, trivially separable once you look at the spectrum. Most of practical DSP, including pulling a finger-tap transient out of slow arm motion, is exactly this kind of separation.

5.5 Building a waveform from sinusoids: the gentle path

Here is the experiment that, more than any equation, makes the Fourier idea click. We will build a non-sinusoidal shape – a square wave – by adding sinusoids, one at a time, and watch the target emerge.

Take odd harmonics of a fundamental f_0, each with amplitude $1/k$:

$$s_N(t) = \frac{4}{\pi} \sum_{\substack{k=1 \\ k \text{ odd}}}^{N} \frac{1}{k} \sin(2\pi k f_0 t) = \frac{4}{\pi}\left(\sin \omega_0 t + \tfrac{1}{3} \sin 3\omega_0 t + \tfrac{1}{5} \sin 5\omega_0 t + \cdots\right).$$

One term is a lonely sine. Two terms already has the rough plateau-and-edge look of a square. Many terms and it is unmistakably a square wave with sharp risers, as Figure 5.2 shows.

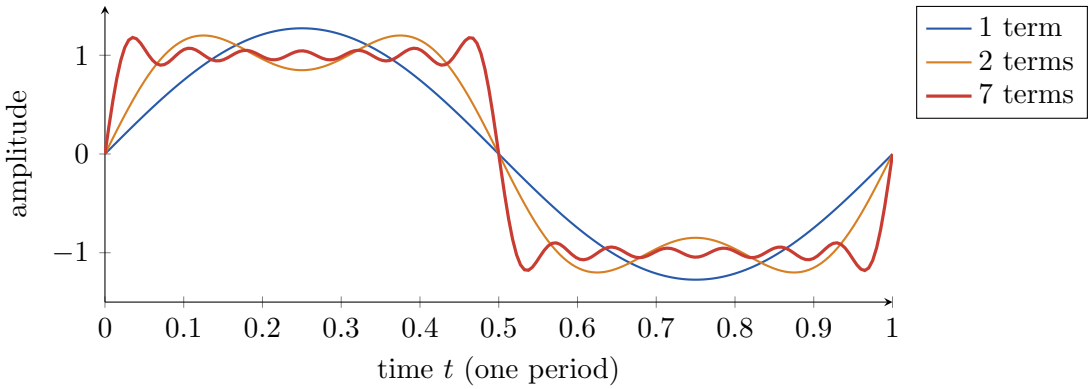


Figure 5.2: A square wave emerging from odd harmonics. One sine is a poor square; add the third and fifth and seventh harmonics (here up to the 13th) and the flat tops and steep edges appear. The little overshoot at the jumps never goes away – that is the Gibbs phenomenon, revisited in Chapter 6.

What is really happening

Watch what each new term *does*. The fundamental sets the overall up-down rhythm. Each higher harmonic is a faster, smaller correction that sharpens the corners and flattens the tops. The *slow* sinusoids carry the gross shape; the *fast* ones carry the fine detail and the edges. This is the deepest single intuition about spectra: **low frequencies are gross shape, high frequencies are sharp detail and edges.** A blurry image is one with the high frequencies removed; a muffled voice is one with the highs rolled off; a signal with sharp transients must have substantial high-frequency content. You can now reason about a spectrum without computing a single integral.

Try it yourself

Type this and drag the slider in your head from 1 term to 50. The corners get crisper; the overshoot at the edge stubbornly stays about 9% tall. That stubborn ear on the jump is telling you something real about representing a discontinuity with smooth waves.

```
import numpy as np, matplotlib.pyplot as plt
t = np.linspace(0, 1, 1000, endpoint=False)
def square_approx(N):
    s = np.zeros_like(t)
    for k in range(1, N+1, 2):          # odd harmonics only
        s += (1/k) * np.sin(2*np.pi*k*t)
    return (4/np.pi) * s
for N in (1, 3, 7, 49):
    plt.plot(t, square_approx(N), label=f"{(N+1)//2} terms")
plt.legend(); plt.xlabel("t"); plt.show()
```

5.6 The spectrum: a signal seen edge-on

If a signal is a sum of sinusoids, then to specify it you need only list, for each frequency present, its amplitude and its phase. That list *is* the spectrum.

Definition 5.1 (Spectrum, informal)**.** The *spectrum* of a signal is the description of which frequencies it contains and in what amount. For each frequency f we record an *amplitude* (or magnitude) $A(f)$ – how much of that frequency is present – and a *phase* $\phi(f)$ – the timing of that component. Plotting A against f gives the *magnitude spectrum*; plotting ϕ against f gives the *phase spectrum*.

Two ways to draw the very same square wave, in Figure 5.3:

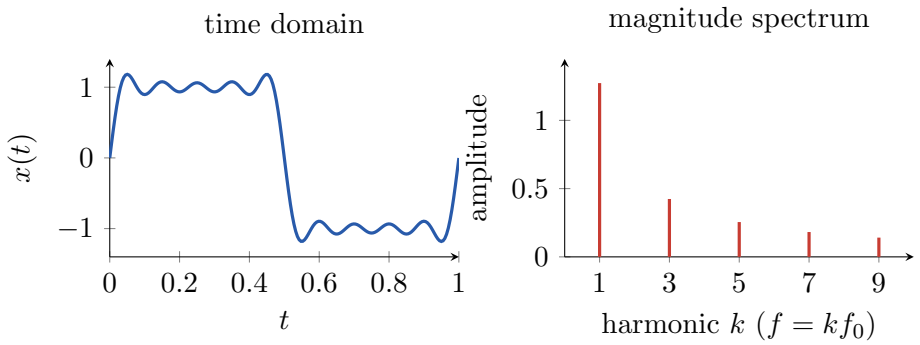


Figure 5.3: The same signal, two views. Left: the wiggle in time. Right: its line spectrum – a stem at each odd harmonic whose height is $\frac{4}{\pi k}$. The spectrum is the signal "seen edge-on": all the time-domain complexity collapses to a handful of dots.

Why this matters

Notice the economy. The left plot needs hundreds of points to draw; the right plot needs five dots. Same information. The spectrum is not an approximation or a summary – for a periodic signal it is a *complete, lossless* re-encoding. This is the quiet miracle that makes MP3, JPEG, radio, and the FFT possible: most real signals are sparse or simple in frequency even when they look hopeless in time.

5.7 Harmonics and timbre: why a violin is not a flute

When a frequency-f_0 tone and its integer multiples $2f_0, 3f_0, \ldots$ all appear together, f_0 is the *fundamental* and the multiples are *harmonics* (or overtones). The ear assigns the *pitch* from the fundamental, but the *relative amplitudes of the harmonics* are what make a trumpet sound like a trumpet and a clarinet like a clarinet, even on the same note. That recipe of harmonic strengths is the *timbre*, sketched for two cases in Figure 5.4.

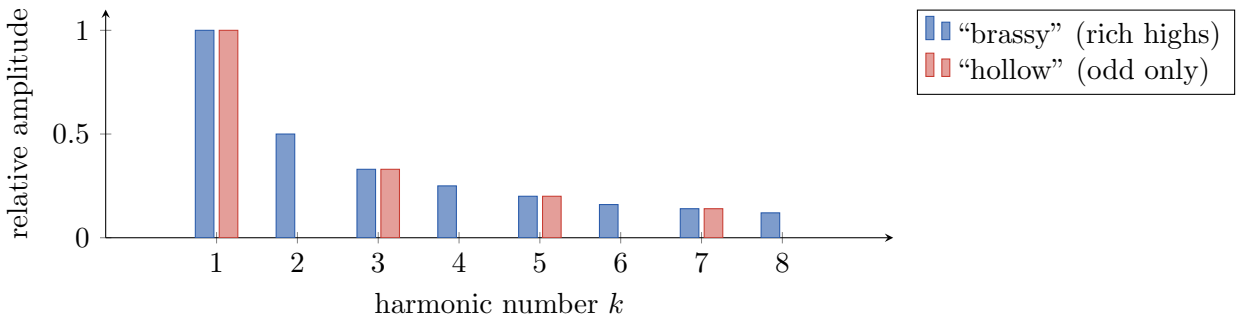


Figure 5.4: Two timbres on the *same* fundamental. Left bars: a full harmonic series, energy in every multiple – bright and brassy. The odd-only series (like an idealized clarinet's lower register) sounds hollow and woody. Same pitch, different spectrum, different instrument.

Application

This is why a synthesizer can imitate instruments by *additive synthesis*: choose the fundamental for the pitch, then dial in a harmonic recipe for the timbre. It is also why audio compression works – the ear barely notices a weak harmonic sitting next to a strong one (masking), so the encoder can throw the weak one away. Both tricks are pure spectrum thinking.

Toward the wrist tracker

For the wrist controller, "timbre" has a motion analogue. A deliberate finger tap and an accidental table bump can have similar peak acceleration in time, but their *harmonic content* differs: a crisp tap is a sharp transient (broadband, lots of highs), while a soft bump is smoother (energy concentrated low). Looking at the shape of the spectrum – not just the peak height – is one of the cheapest ways to tell a real gesture from noise. We will turn this into real features in Chapter 38.

5.8 Beats: two close frequencies

Add two sinusoids of nearly equal frequency and something striking happens: a slow throbbing in loudness. This is *beating*, and the algebra is a one-line revelation. With $f_1 = f_0 - \frac{\Delta}{2}$ and $f_2 = f_0 + \frac{\Delta}{2}$,

$$\cos(2\pi f_1 t) + \cos(2\pi f_2 t) = \underbrace{2\cos(\pi \Delta\, t)}_{\text{slow envelope}}\ \underbrace{\cos(2\pi f_0 t)}_{\text{fast carrier}}.$$

The sum equals a fast tone at the *average* frequency f_0, multiplied by a slow envelope that wobbles at the *difference* frequency Δ. You hear the average pitch swelling and fading Δ times a second. Figure 5.5 plots the result.

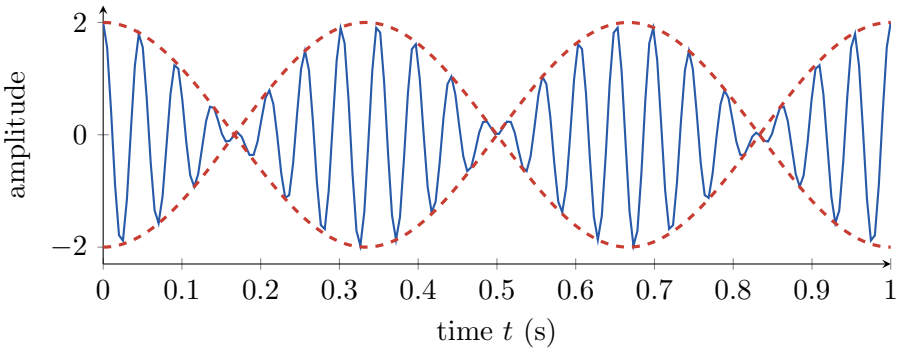


Figure 5.5: Beats: $\cos(2\pi \cdot 20t) + \cos(2\pi \cdot 23t)$ in blue. The dashed red envelope wobbles at the difference $\Delta = 3$ Hz, so the loudness swells three times a second while the underlying pitch sits at the average 21.5 Hz.

History

Piano tuners exploit this directly. Strike a note against a reference; if you hear beats, the two are slightly detuned, and the beat *rate equals the frequency error*. Tune until the beating slows to a crawl and stops – now $\Delta = 0$ and the pitches match. The ear, with no instrument, resolves a fraction of a hertz. It is one of the oldest practical uses of the difference-frequency identity, predating any notion of a Fourier transform by centuries.

Common trap

Beating is *not* a new frequency. The spectrum of the beating signal has exactly two lines, at 20 and 23 Hz – nothing at 3 Hz and nothing at 21.5 Hz. The throb at 3 Hz is what your *loudness perception* (a nonlinear thing) makes of two pure tones. The lesson: an envelope you see in time need not be a line in the spectrum. Time-domain appearance and frequency-domain content are different views, and reading one off the other takes care.

5.9 Complex exponentials and the meaning of negative frequency

Real cosines are clumsy to compute with because the angle-sum identities multiply like rabbits. The fix, met in Chapter 4, is Euler's formula:

$$e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta.$$

A complex exponential $e^{\mathrm{j}2\pi ft}$ is a point on the unit circle in the complex plane, spinning counterclockwise at f revolutions per second. Its real part traces a cosine, its imaginary part a sine. One spinning arrow – a *phasor* – carries the cosine and sine together. This is not a notational convenience; it is the cleaner truth, because the rotation *is* the frequency, made visible as motion. Figure 5.6 shows the two senses of rotation side by side.

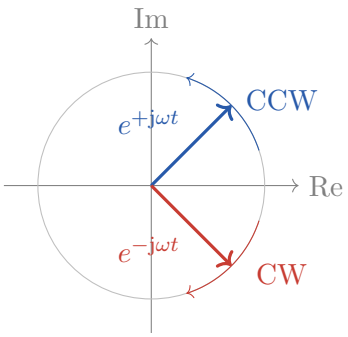


Figure 5.6: Two phasors at $t > 0$. The blue arrow $e^{+j\omega t}$ spins counterclockwise – positive frequency. The red arrow $e^{-j\omega t}$ spins clockwise – negative frequency. Same speed, opposite sense of rotation.

So what on earth is a *negative* frequency? Geometrically it is the cleanest thing in the world: **negative frequency just means the phasor spins the other way.** Counterclockwise is positive; clockwise is negative. There is nothing mysterious or unphysical about turning the other direction.

Why it works

Why negative frequencies are forced on us. Rearrange Euler's formula and you get the identity at the heart of all real-signal spectra:

$$\cos(2\pi ft) = \tfrac{1}{2}e^{j2\pi ft} + \tfrac{1}{2}e^{-j2\pi ft}.$$

A real cosine is the *average of two counter-rotating phasors*, one at $+f$ and one at $-f$. Picture them: two arrows spinning in opposite directions. Their vertical (imaginary) parts always cancel, so the sum is purely real; their horizontal parts reinforce, tracing the cosine. To build a real wave from complex atoms you *must* use both senses of rotation. That is why the spectrum of any real signal is symmetric: every component at $+f$ has a mirror twin at $-f$. Negative frequency is not a fiction; it is the bookkeeping that lets one elegant complex framework describe real signals.

What is really happening

The cancellation is worth seeing slowly. At each instant the $+f$ phasor is at angle $+\omega t$ and the $-f$ phasor at angle $-\omega t$. They are mirror images across the real axis. Add two mirror-image arrows: the up-down parts kill each other, the left-right parts double. The result slides back and forth along the real axis – a cosine. Frequency $+f$ and frequency $-f$ are the two "hands" that together clap out a real oscillation.

Focus

For *real* signals, negative frequencies are redundant: knowing the $+f$ half tells you the $-f$ half by symmetry, so many textbooks (and tuners and audio plots) quietly fold them away and show only $f \geq 0$. But for *complex* signals – the I/Q data of a radio, or the analytic signal we build in Chapter 25 – positive and negative frequencies are genuinely different and independent. A single phasor spinning one way has energy at exactly one frequency, positive or negative, with no twin. That asymmetry is the whole basis of how a radio tells an upper

sideband from a lower one.

Connection

The probability companion will lean on this same complex-exponential atom: the *characteristic function* of a random variable is $\mathbb{E}[e^{j\omega X}]$, the expected value of a phasor whose spin rate is set by the data. It is the Fourier transform of a probability density, and convolving two densities (adding two independent random variables) becomes multiplying their characteristic functions – the very same "convolution becomes multiplication" magic that makes frequency thinking pay off here. One idea, two books.

5.10 From a few lines to a continuous spectrum

So far every example was *periodic*, and its spectrum was a discrete set of lines at the fundamental and its harmonics. What about a signal that does not repeat – a single finger tap, a click, a one-shot transient?

The bridge is to imagine stretching the period T toward infinity. The harmonics sit at multiples of $f_0 = 1/T$; as T grows, those allowed frequencies crowd closer and closer together (spacing $1/T \to 0$). In the limit the discrete comb of lines fuses into a continuous curve. A non-periodic signal has a *continuous* spectrum: energy spread over a range of frequencies rather than parked on a few exact ones. That limit is the Fourier *transform*, the subject of Chapter 7; the line-spectrum case is the Fourier *series* of Chapter 6. Figure 5.7 contrasts the two.

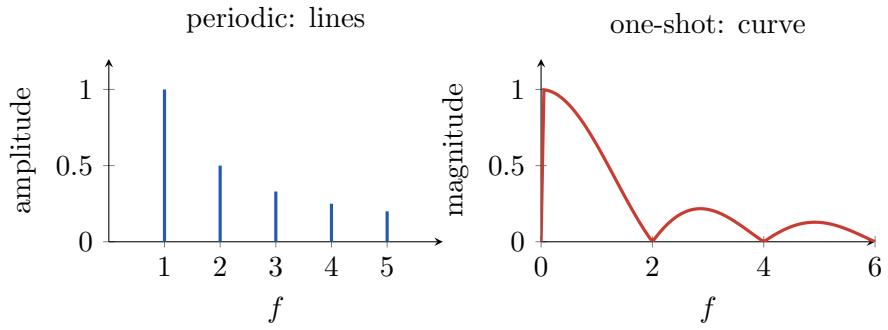


Figure 5.7: Left: a periodic signal has a discrete line spectrum (energy only at harmonics). Right: a non-repeating pulse has a continuous spectrum (a smooth curve – here a sinc-like magnitude). Stretching the period to infinity merges the lines into the curve.

Intuition

A short, sharp event must be *broadband*: to localize energy tightly in time you need many frequencies cooperating, just as it took many harmonics to make the square wave's sharp edge. A long, pure tone is *narrowband*: one frequency, spread over a long time. Brief in time $\Leftrightarrow$ wide in frequency. This trade-off is the seed of the time–frequency uncertainty principle (Chapter 10) and the reason a drum (brief) sounds "noisy" while an organ pipe (sustained) sounds "pure".

5.11 A worked example end to end

Example 5.1 (Reading a signal's spectrum by inspection)**.** Consider

$$x(t) = 3 + 2\cos(2\pi \cdot 10\,t) + \cos(2\pi \cdot 60\,t - \tfrac{\pi}{2}).$$

Describe its spectrum without any integral.

The constant 3. A constant is a cosine of frequency zero ($\cos(0) = 1$), amplitude 3. This is the *DC component* – the signal's average value. It is a line at $f = 0$ of height 3.

The $2\cos(2\pi \cdot 10t)$ term. Amplitude 2, frequency 10 Hz, phase 0. A line at 10 Hz of height 2 (and, by symmetry, a twin at -10 Hz if we use the two-sided complex picture).

The $\cos(2\pi \cdot 60t - \pi/2)$ term. Amplitude 1, frequency 60 Hz, phase $-\pi/2$ (which makes it a *sine*, $\sin(2\pi \cdot 60t)$). A line at 60 Hz of height 1, carrying a phase of $-\pi/2$.

Spectrum (one-sided magnitude): spikes at $0, 10, 60$ Hz of heights $3, 2, 1$. Nothing anywhere else. We read off the complete frequency content of a signal that, drawn in time, looks like a complicated wiggle – and we never integrated. That is the fluency this chapter is for. Figure 5.8 draws the three lines.

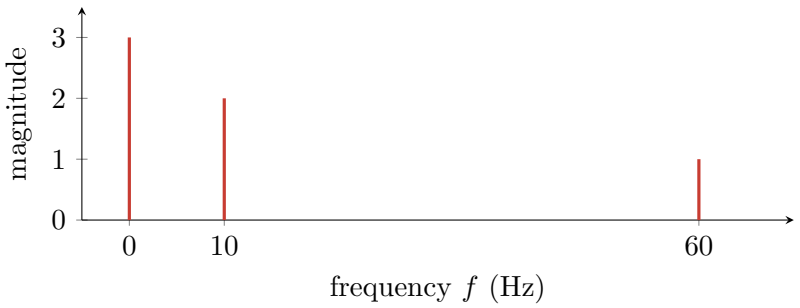


Figure 5.8: One-sided magnitude spectrum of $x(t) = 3 + 2\cos(2\pi \cdot 10\,t) + \cos(2\pi \cdot 60\,t - \pi/2)$. A DC line of height 3, the 10 Hz tone of height 2, and the 60 Hz tone of height 1 – read off by inspection.

Walking question

Suppose the 60 Hz term were unwanted hum from the wall socket contaminating a sensor. Knowing it lives *only* at 60 Hz, how would you remove it without harming the 10 Hz signal you care about? (Answer foreshadowed: a filter that passes 10 Hz and rejects 60 Hz – a notch or low-pass. The entire reason filtering is possible is that the interference and the signal occupy different frequencies. We build such filters in Chapter 16.)

5.12 Frequency for the wrist controller

Toward the wrist tracker

Let us anchor every idea above in the device. Typical frequency bands you will meet once the IMU and EMG are strapped on:

- **Gravity / posture**: essentially DC up to ~ 0.5 Hz on the accelerometer. Slow, low frequency – the gross orientation of the forearm.
- **Deliberate gestures**: roughly 1–8 Hz. A comfortable finger-tap or wrist-flick rhythm lives here. This is the band we actually want.
- **Tremor**: a physiological hand tremor sits around 8–12 Hz – close enough to fast gestures that telling them apart is a real spectral problem.
- **Muscle activity (EMG)**: surface EMG energy spreads broadly, roughly 20–450 Hz, which is exactly why EMG needs a much higher sampling rate than the IMU.
- **Mains hum**: a sharp interfering line at 50 or 60 Hz (and its harmonics), right in the middle of the EMG band – a notch filter's classic target.

Every design choice – sampling rate, filter cutoffs, which features to compute – flows from where in frequency the things you care about and the things you do not care about happen to sit. Figure 5.9 lays the bands out on one axis.

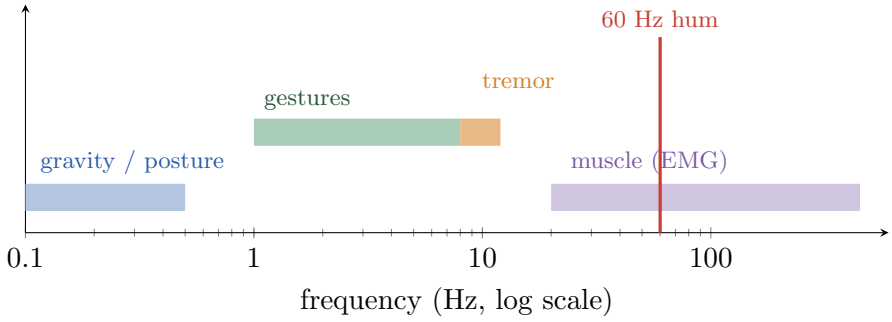


Figure 5.9: Where the wrist controller's signals live in frequency. Slow posture sits near DC; deliberate gestures occupy 1–8 Hz with tremor pressing right against them at 8–12 Hz; surface EMG spreads broadly to a few hundred hertz, with the mains hum a sharp line at 60 Hz cutting through it.

Build it (hardware)

A first hands-on frequency check, before any fancy transform. Sample the accelerometer steadily, then estimate the rate of a repetitive gesture by counting zero-crossings of the (mean-removed) signal: a sinusoid at f Hz crosses zero $2f$ times per second.

```
import numpy as np
def gesture_rate_hz(x, fs):
    x = x - np.mean(x)                      # remove DC (gravity/offset)
    crossings = np.sum((x[:-1] < 0) & (x[1:] >= 0))  # rising zero-crossings
```

```
    duration = len(x) / fs
    return crossings / duration              # one crossing per cycle
# fs = 100 Hz IMU stream; x = one window of one axis
```

Crude, fooled by noise, and blind to anything but the dominant rhythm – but it runs in microcontroller-friendly integer-ish math and gives a usable tap-tempo estimate before you have an FFT. We graduate to the real spectrum in Chapter 8.

Common trap

What a wrist IMU can and cannot do. It is tempting to imagine the spectrum of a wrist accelerometer revealing individual finger movements. It cannot. A wrist-mounted IMU measures the gross translation and rotation of the *wrist*; the tiny, mechanically isolated motions of individual finger joints barely couple into it. The IMU can tell "a tap happened" or "the wrist rolled", i.e. coarse, discrete gestures – it cannot recover continuous, per-finger joint angles. Distinguishing which finger tapped, or tracking finger pose, needs *other physics*: surface EMG or force myography reading the forearm muscles that drive the fingers (Chapter 37), or sensors actually on the fingers (instrumented rings/gloves). Frequency analysis sharpens whatever the sensor can sense; it cannot conjure information the sensor never captured. We will keep saying this, because overclaiming here is the most common way these projects fail.

5.13 What to carry forward

Pause

- Frequency is a *rate of repetition*. Three names for it: period T, ordinary frequency $f = 1/T$ in Hz, angular frequency $\omega = 2\pi f$ in rad/s.
- A pure sinusoid is fixed by three numbers: amplitude, frequency, phase.
- Signals can be *built* from sinusoids. Low frequencies carry gross shape; high frequencies carry sharp detail and edges.
- The *spectrum* lists how much of each frequency is present (magnitude) and its timing (phase). For periodic signals it is a few lines; for one-shot signals it is a continuous curve.
- Harmonics and their relative strengths are *timbre*; beats are two close frequencies producing a slow loudness throb (and are *not* a new spectral line).
- A complex exponential is a spinning phasor; *negative frequency* just means spinning the other way. Real signals need both $+f$ and $-f$ twins, which is why real spectra are symmetric.
- Brief in time $\Leftrightarrow$ broad in frequency.

Next chapter we make the periodic case exact: the Fourier series, where the harmonic amplitudes stop being guesses and become inner products you can compute.

5.14 Exercises

Exercise 5.1. A signal repeats every 40 ms. Give its period in seconds, its ordinary frequency in hertz, and its angular frequency in radians per second.

Exercise 5.2. Write $5\cos(120\pi t + \pi/3)$ in the form $A\cos(2\pi f t + \phi)$, identifying A, f (Hz), and ϕ. Is 120π an angular frequency or an ordinary one? How do you know?

Exercise 5.3. Using $\cos\theta = \frac{1}{2}(e^{\mathrm{j}\theta} + e^{-\mathrm{j}\theta})$, write $x(t) = 4\cos(2\pi \cdot 50\, t)$ as a sum of two complex exponentials. At which two frequencies (including sign) does it have energy, and what is the magnitude at each?

Exercise 5.4. Two tuning forks sound together and you count 5 loudness swells per second. One fork is known to be exactly 440 Hz. What are the two possible frequencies of the other fork? Explain why two answers are possible from the beat rate alone.

Exercise 5.5. Sketch (by reasoning, not computing) the magnitude spectrum of $x(t) = 1 + \cos(2\pi \cdot 5t) + \frac{1}{2}\sin(2\pi \cdot 5t)$. Hint: combine the two 5 Hz terms into a single cosine with a phase, then list the lines.

Exercise 5.6. A square wave of fundamental $f_0 = 100$ Hz is built from odd harmonics with amplitudes $\propto 1/k$. List the frequencies and relative amplitudes of its first four nonzero spectral lines.

Exercise 5.7. Explain in one or two sentences each: (a) why a perfectly sharp step edge in a signal requires arbitrarily high frequencies; (b) why a pure, long-sustained tone occupies essentially a single frequency.

Exercise 5.8. A real signal has spectral lines at ± 30 Hz and ± 90 Hz. Argue from symmetry that the signal is real, and write a plausible time-domain expression that produces exactly these lines.

Exercise 5.9. The zero-crossing rate estimator in the build box reports 8 rising crossings in a 2-second accelerometer window after mean removal. What gesture frequency does it estimate? Name two situations (think noise and harmonics) where this estimate would be wrong, and say in which direction.

Exercise 5.10. You record forearm EMG and see a strong, narrow 60 Hz line plus broadband energy from 20 to 300 Hz. Which part is the muscle signal and which is interference? Roughly where in frequency would you place a filter to suppress the interference while keeping most of the muscle energy, and what kind of filter is it?

Exercise 5.11. (Conceptual.) A friend claims that because they can clearly see a slow throbbing envelope at 3 Hz when two 1 kHz-ish tones beat together, "there must be a 3 Hz component in the signal." Use the spectrum of the beating signal to explain exactly what is and is not present at 3 Hz.

Exercise 5.12. (Toward the build.) Argue, using the limits discussed in this chapter, why no amount of clever frequency analysis of a single *wrist* accelerometer can recover which individual finger tapped. What additional sensing would you add, and what physical quantity would it measure?

Chapter 6

Fourier Series

There is a claim so audacious that when Joseph Fourier first made it, the leading mathematicians of Paris simply refused to believe it. The claim is this: *any* periodic signal – no matter how jagged, how square-cornered, how spiky – can be built by adding together pure sine waves whose frequencies are whole-number multiples of one fundamental frequency. A square wave, with its perfectly vertical edges, is secretly a sum of smooth, rounded sinusoids. A sawtooth ramp is too. The buzz of a violin string, the vowel in a sung note, the periodic flexing of a tendon as you tap a finger – all of them decompose into a stack of sinusoids.

This chapter is where the whole book turns from *describing* signals to *decomposing* them. In the previous chapters we learned to see a signal as a vector (Chapter 3) and a sinusoid as a rotating phasor (Chapter 4), and we built an intuition for what frequency means (Chapter 5). Now we cash all of that in. The Fourier series is nothing more – and nothing less – than *the coordinates of a periodic signal in a basis made of sinusoids*. Every idea here is geometry: projection onto basis vectors, orthogonality, and the Pythagorean theorem. The integrals are just inner products wearing a calculus costume.

6.1 The Big Picture: A Periodic Signal Is a Recipe

Let us fix vocabulary. A signal $x(t)$ is *periodic* with period T if $x(t+T) = x(t)$ for all t. The smallest such T is the *fundamental period*; its reciprocal $f_0 = 1/T$ is the *fundamental frequency* (in Hz), and $\omega_0 = 2\pi/T$ is the fundamental *angular* frequency (in radians per second). A signal that repeats every T seconds repeats f_0 times per second.

Intuition

Think of a periodic signal as a single *tile* of length T stamped over and over along the time axis. Because the tile repeats forever, all the information lives in one period. The Fourier series will tell us how to reproduce that one tile by mixing pure sinusoids – and once the tile is right, the infinite repetition takes care of itself.

The sinusoids we are allowed to use are special: their frequencies must be *harmonically related*, meaning each is an integer multiple of ω_0. The k-th *harmonic* is $\cos(k\omega_0 t)$ or $\sin(k\omega_0 t)$. Why integer multiples? Because those are exactly the sinusoids that themselves repeat every T seconds:

if $\omega = k\omega_0$ then $\cos(\omega(t+T)) = \cos(k\omega_0 t + 2\pi k) = \cos(k\omega_0 t)$. A sinusoid at $2.5\,\omega_0$ would *not* line up with the tile – after one period T it has done two and a half cycles and starts the next tile out of phase. Only harmonics fit perfectly inside the tile, edge to edge.

Why it works

Harmonics are the building blocks that share the signal's period. If you add together any collection of T-periodic functions, the sum is still T-periodic. So a sum of harmonics is automatically a candidate for representing a T-periodic signal. The deep claim of the Fourier series is the converse: these harmonics are not just *a* family of T-periodic functions, they are a *complete* family – rich enough to build any (reasonable) T-periodic signal at all.

6.1.1 The three forms you will meet

The Fourier series comes dressed in three equivalent costumes. It helps to see all three at the start so you are never surprised:

- **Sine–cosine (trigonometric) form.** A sum $\frac{a_0}{2} + \sum_{k\geq 1}\big(a_k \cos k\omega_0 t + b_k \sin k\omega_0 t\big)$. The most concrete; great for real signals and hand computation.
- **Amplitude–phase form.** A sum $C_0 + \sum_{k\geq 1} C_k \cos(k\omega_0 t - \phi_k)$. Each harmonic is one cosine with its own amplitude C_k and phase shift ϕ_k. This is how an engineer *hears* it: "how much of harmonic k, and shifted by how much?"
- **Complex-exponential form.** A sum $\sum_{k=-\infty}^{\infty} c_k\, e^{\mathrm{j}k\omega_0 t}$. The cleanest, most symmetric form – the one that generalizes to the Fourier transform. The c_k are complex numbers (rotating phasors). This will become our home.

They carry exactly the same information; we will translate freely between them.

6.2 Computing Coefficients Is Projection

Here is the single most important idea in the chapter, and it is pure linear algebra. In Chapter 3 we learned that the natural inner product for signals on an interval is the integral of their product. For real T-periodic signals we define

$$\langle x,\, y\rangle \;=\; \int_0^T x(t)\, y(t)\, \mathrm{d}t, \tag{6.1}$$

and over a single period this is a genuine inner product: it is linear, symmetric, and $\langle x, x\rangle = \int_0^T x(t)^2\, \mathrm{d}t \geq 0$, zero only for the zero signal. (The integral can be taken over *any* full period, e.g. $-T/2$ to $T/2$; the result is the same because everything is T-periodic.) With an inner product comes geometry: length, angle, and – crucially – *orthogonality*.

Connection

This is the inner product from the linear-algebra companion, made continuous. There, $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i u_i v_i$ sums a product over coordinates. Here we sum (integrate) a product over time. Time t plays the role of the index i; the signal $x(t)$ is a vector with one component for every instant. Every theorem about orthonormal bases – projection, Pythagoras, Parseval – transfers verbatim.

6.2.1 The harmonics are orthogonal

Everything works because the harmonic sinusoids are mutually *orthogonal* on one period: the inner product of two different harmonics is zero. Let us verify the key one. For integers $m, n \geq 1$,

$$\int_0^T \cos(m\omega_0 t)\cos(n\omega_0 t)\,\mathrm{d}t = \begin{cases} 0, & m \neq n, \\ T/2, & m = n. \end{cases} \tag{6.2}$$

Why it works

The cleanest way to see Eq. (6.2) is geometric, not trigonometric. The product-to-sum identity gives $\cos A \cos B = \frac{1}{2}[\cos(A-B) + \cos(A+B)]$, so the integrand is half a cosine at frequency $(m-n)\omega_0$ plus half a cosine at $(m+n)\omega_0$. *Integrating a sinusoid over a whole number of its own periods gives zero* – you sweep equally through positive and negative values and they cancel. When $m \neq n$, both $(m-n)$ and $(m+n)$ are nonzero integers, so both pieces complete whole cycles over $[0, T]$ and vanish. When $m = n$, the first piece is $\cos(0) = 1$, a constant whose integral is T, giving $\frac{1}{2}T$; the second still vanishes. That single fact – "a sinusoid integrated over whole cycles averages to zero" – is the engine behind the entire Fourier machinery.

By identical arguments, $\int_0^T \sin(m\omega_0 t)\sin(n\omega_0 t)\,\mathrm{d}t = \frac{T}{2}\delta_{mn}$ and, beautifully, *every* sine is orthogonal to *every* cosine: $\int_0^T \sin(m\omega_0 t)\cos(n\omega_0 t)\,\mathrm{d}t = 0$ for all m, n. The sines and cosines together form an orthogonal basis for T-periodic signals – a coordinate system.

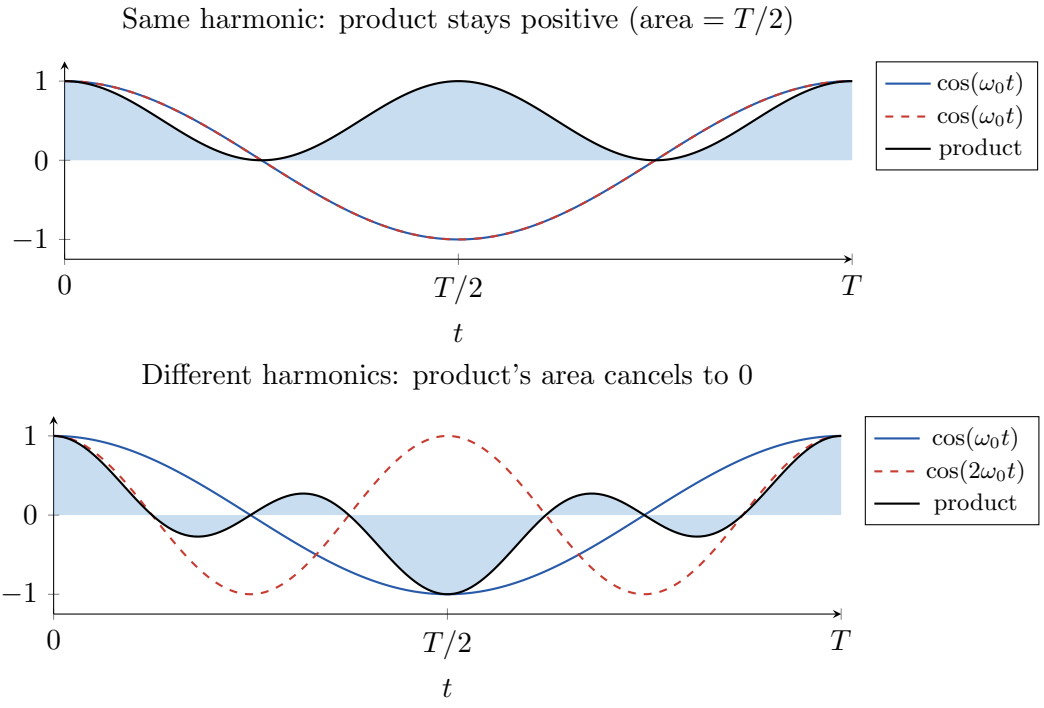


Figure 6.1: Orthogonality, seen as area. Top: a harmonic times itself is always nonnegative, so its area over a period is positive ($= T/2$). Bottom: two different harmonics multiply to a wave whose positive and negative lobes have equal area, so the integral is exactly zero. This cancellation is why projecting onto one harmonic does not pick up any of the others.

6.2.2 Projection gives the coefficients

Once you have an orthogonal basis, finding coordinates is automatic. If we believe $x(t) = \frac{a_0}{2} + \sum_{k\geq 1}(a_k \cos k\omega_0 t + b_k \sin k\omega_0 t)$, then to extract a particular coefficient – say a_n – we take the inner product of both sides with $\cos n\omega_0 t$. On the right, orthogonality annihilates every term except the one we want, which survives with its $T/2$ self-energy:

$$\langle x,\, \cos n\omega_0 t\rangle = a_n \cdot \frac{T}{2} \quad \Longrightarrow \quad a_n = \frac{2}{T}\int_0^T x(t)\cos(n\omega_0 t)\,\mathrm{d}t. \tag{6.3}$$

Identically, $b_n = \frac{2}{T}\int_0^T x(t)\sin(n\omega_0 t)\,\mathrm{d}t$, and the constant term is $\frac{a_0}{2} = \frac{1}{T}\int_0^T x(t)\,\mathrm{d}t$, the plain *average* of the signal (its DC level). Notice we wrote the constant as $a_0/2$ precisely so that the formula (6.3) for a_n works at $n = 0$ too – a tidy bookkeeping trick.

What is really happening

A Fourier coefficient is a **correlation score**. The integral $\int_0^T x(t)\cos(n\omega_0 t)\,\mathrm{d}t$ asks: "as I slide along one period, does x tend to be large exactly where $\cos n\omega_0 t$ is large?" If yes, the product is mostly positive and the integral is big – the signal contains a lot of harmonic n. If x has no resemblance to that harmonic, the product is positive about as often as negative and the score is near zero. The Fourier series runs this comparison against *every* harmonic

at once, and the scores are the coordinates. Decomposition is just "how much does the signal look like each pure tone?"

Formal

Projection formula, abstractly. For an orthogonal family $\{\phi_k\}$ with $\langle \phi_k, \phi_\ell \rangle = \|\phi_k\|^2 \delta_{k\ell}$, the best representation of x is $x = \sum_k \alpha_k \phi_k$ with $\alpha_k = \langle x, \phi_k \rangle / \|\phi_k\|^2$. Equation (6.3) is exactly this with $\phi_n = \cos n\omega_0 t$ and $\|\phi_n\|^2 = T/2$. The $2/T$ is nothing but $1/\|\phi_n\|^2$. No magic – just normalization.

6.3 The Complex-Exponential Form

The sine–cosine form is correct but clumsy: two families of coefficients, special cases at $k = 0$, and trig identities everywhere. Euler's formula $e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta$ lets us fuse the sines and cosines into a single elegant object. Recall from Chapter 4 that a cosine is the average of two counter-rotating phasors:

$$\cos(k\omega_0 t) = \tfrac{1}{2}\big(e^{\mathrm{j}k\omega_0 t} + e^{-\mathrm{j}k\omega_0 t}\big), \qquad \sin(k\omega_0 t) = \tfrac{1}{2\mathrm{j}}\big(e^{\mathrm{j}k\omega_0 t} - e^{-\mathrm{j}k\omega_0 t}\big). \tag{6.4}$$

Substituting these into the trigonometric series and collecting terms by the phasor $e^{\mathrm{j}k\omega_0 t}$ regroups everything into one clean sum that runs over *all* integers, positive and negative:

$$\boxed{x(t) = \sum_{k=-\infty}^{\infty} c_k\, e^{\mathrm{j}k\omega_0 t}, \qquad c_k = \frac{1}{T}\int_0^T x(t)\, e^{-\mathrm{j}k\omega_0 t}\,\mathrm{d}t.} \tag{6.5}$$

The synthesis equation (left) *builds* the signal from phasors; the analysis equation (right) *extracts* each phasor's complex weight. Note the sign flip: to *measure* the component rotating at $+k\omega_0$, we multiply by the *counter*-rotating $e^{-\mathrm{j}k\omega_0 t}$. This freezes that harmonic into a constant whose average survives, while every other harmonic keeps spinning and averages to zero – the same orthogonality trick as before, now for complex exponentials: $\frac{1}{T}\int_0^T e^{\mathrm{j}m\omega_0 t} e^{-\mathrm{j}n\omega_0 t}\mathrm{d}t = \delta_{mn}$.

Why it works

Why negative frequencies appear. A real cosine is built from *two* phasors spinning in opposite directions; their imaginary parts cancel, leaving a real wave on the axis. To represent a real signal with single phasors, we therefore need both $+k\omega_0$ and $-k\omega_0$. The negative-frequency term is not mystical – it is the clockwise-spinning partner that keeps the sum real. Concretely, for a real $x(t)$ the coefficients come in conjugate pairs: $c_{-k} = \overline{c_k}$. That single symmetry is what guarantees the imaginary parts cancel out at every instant (Figure 6.2).

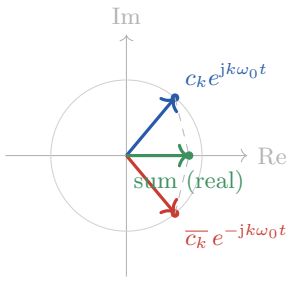


Figure 6.2: Why negative frequencies keep a real signal real. The harmonic at $+k\omega_0$ (blue, spinning counterclockwise) and its conjugate partner at $-k\omega_0$ (red, spinning clockwise) are mirror images across the real axis at every instant. Their imaginary parts are equal and opposite, so they cancel, and the sum (green) always lands on the real axis. A real harmonic is this matched pair, never a lone phasor.

6.3.1 Translating between the forms

The dictionary is short and worth memorizing. For $k \geq 1$,

$$c_k = \tfrac{1}{2}(a_k - \mathrm{j}b_k), \qquad c_{-k} = \tfrac{1}{2}(a_k + \mathrm{j}b_k), \qquad c_0 = \tfrac{a_0}{2}, \tag{6.6}$$

and going the other way, $a_k = 2\,\mathrm{Re}\,c_k$ and $b_k = -2\,\mathrm{Im}\,c_k$. The amplitude–phase form falls right out of the polar form of c_k: writing $c_k = \tfrac{1}{2}C_k e^{-\mathrm{j}\phi_k}$, the two conjugate phasors at $\pm k\omega_0$ recombine into $C_k \cos(k\omega_0 t - \phi_k)$. So

$$C_k = 2|c_k| = \sqrt{a_k^2 + b_k^2}, \qquad \phi_k = \arg(a_k + \mathrm{j}b_k) = \mathrm{atan2}(b_k, a_k). \tag{6.7}$$

C_k is how loud harmonic k is; ϕ_k is where in its cycle it starts. The *magnitude* spectrum $\{C_k\}$ is what a spectrum analyzer plots; the *phase* spectrum $\{\phi_k\}$ tells you the alignment – often ignored by ear, but essential for reconstructing the actual waveform.

Common trap

There are several incompatible normalization conventions in the wild. Some books put $1/T$ on the synthesis side, some split it as $1/\sqrt{T}$ on both, some integrate over $[-\pi, \pi]$. They all describe the same physics but the numbers differ. The convention here – $1/T$ on *analysis* (6.5), none on synthesis – is the most common in engineering and the one that makes c_0 equal the signal average. When you read a formula elsewhere, *check where the* $1/T$ *lives* before trusting a coefficient value.

6.4 Worked Example: The Square Wave

Let us actually compute something. Take the odd square wave of period T that equals $+1$ on the first half-period and -1 on the second: $x(t) = +1$ for $0 < t < T/2$ and $x(t) = -1$ for $T/2 < t < T$. It is the canonical hard test: perfectly flat plateaus joined by infinitely steep jumps. Can smooth sinusoids really build those vertical cliffs?

Symmetry first – it does half the work. The signal is *odd* ($x(-t) = -x(t)$ about the midpoint of each plateau). An odd signal has *no* cosine content and no DC: $a_k = 0$ for all k, and $a_0 = 0$.

Only sines survive. This is not a coincidence – cosines are even, sines are odd, and projecting an odd signal onto an even basis function gives zero. Symmetry lets you discard half the integrals before lifting a pen.

Intuition

Always check symmetry before computing. **Even** signals ($x(-t) = x(t)$, mirror- symmetric) are pure cosine: $b_k = 0$. **Odd** signals ($x(-t) = -x(t)$, point- symmetric) are pure sine: $a_k = 0$. **Half-wave symmetric** signals (the second half is the negated first half, as here) have *only odd-numbered harmonics*: $c_k = 0$ for even k. The square wave is both odd and half-wave symmetric, so only odd sine harmonics remain. Symmetry is free information.

Computing the surviving sine coefficients with $b_k = \frac{2}{T}\int_0^T x(t)\sin(k\omega_0 t)\mathrm{d}t$, split into the two halves and integrate:

$$\begin{aligned} b_k &= \frac{2}{T}\left[\int_0^{T/2}\sin(k\omega_0 t)\,\mathrm{d}t - \int_{T/2}^{T}\sin(k\omega_0 t)\,\mathrm{d}t\right] = \frac{2}{k\pi}\left(1-\cos k\pi\right) \\ &= \begin{cases} \dfrac{4}{k\pi}, & k \text{ odd}, \\ 0, & k \text{ even}. \end{cases} \end{aligned} \tag{6.8}$$

So the square wave's Fourier series is

$$x(t) = \frac{4}{\pi}\left(\sin\omega_0 t + \frac{1}{3}\sin 3\omega_0 t + \frac{1}{5}\sin 5\omega_0 t + \frac{1}{7}\sin 7\omega_0 t + \cdots\right) = \frac{4}{\pi}\sum_{k\,\text{odd}}\frac{\sin k\omega_0 t}{k}. \tag{6.9}$$

The amplitudes fall off like $1/k$ – slowly. That slow decay is the price of the sharp edges, and it is the whole reason a square wave is hard to reproduce and sounds "bright" and buzzy: it is packed with strong high harmonics.

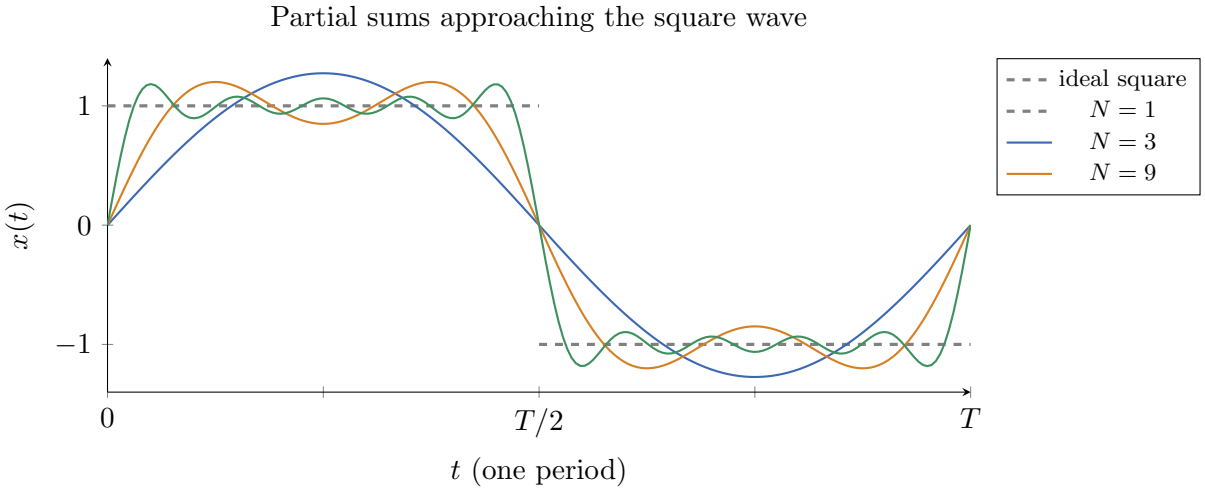


Figure 6.3: Building a square wave from odd harmonics. Even one harmonic captures the gross shape; adding the third and fifth flattens the plateaus and steepens the edges. As more harmonics join, the sum hugs the ideal square everywhere except at the jumps – where a stubborn overshoot refuses to go away. That overshoot is the Gibbs phenomenon, next.

Example 6.1 (Numbers you can check)**.** At the midpoint of the first plateau, $t = T/4$, every odd harmonic $\sin(k\omega_0 T/4) = \sin(k\pi/2)$ equals $+1, -1, +1, \ldots$ for $k = 1, 3, 5, \ldots$ So the series there is $\frac{4}{\pi}(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots)$. The parenthesized sum is the Leibniz series for $\pi/4$, so the total is $\frac{4}{\pi} \cdot \frac{\pi}{4} = 1$ – exactly the height of the plateau. Fourier's formula quietly hands us $\pi/4 = 1 - \frac{1}{3} + \frac{1}{5} - \cdots$ as a bonus.

6.5 Convergence and the Gibbs Phenomenon

How can a sum of *continuous* sinusoids equal a *discontinuous* square wave? The answer is that it does, but not in the naive pointwise sense everywhere. Three facts settle the matter.

1. **At every point where x is continuous**, the partial sums converge to $x(t)$. Smooth stretches are reproduced exactly in the limit.

2. **At a jump**, the series converges to the *midpoint* of the jump – the average of the left and right limits. For our square wave at $t = 0$ and $t = T/2$ the series gives 0, splitting the difference between $+1$ and -1. The series cannot pick a side, so it sits politely in the middle.

3. **Near a jump, the partial sums overshoot** by a fixed percentage that never shrinks. This is the Gibbs phenomenon.

What is really happening

The overshoot in Figure 6.4 does not go away as you add harmonics – it just gets *narrower*. Each finite partial sum overshoots the jump by about 8.95% of the jump height (the limiting figure is $\frac{1}{\pi}\int_0^\pi \frac{\sin u}{u} du - \frac{1}{2} \approx 0.0895$), forever. What *does* shrink is the *width* of the wiggle: it scales like $1/N$ and squeezes toward the discontinuity. So the energy of the error goes to zero (the area under the wiggle vanishes) even though the peak height of the error does not. Convergence in energy, not in worst-case height. This is exactly why a perfect "brick-wall" digital filter rings, and why an abrupt edit in audio clicks.

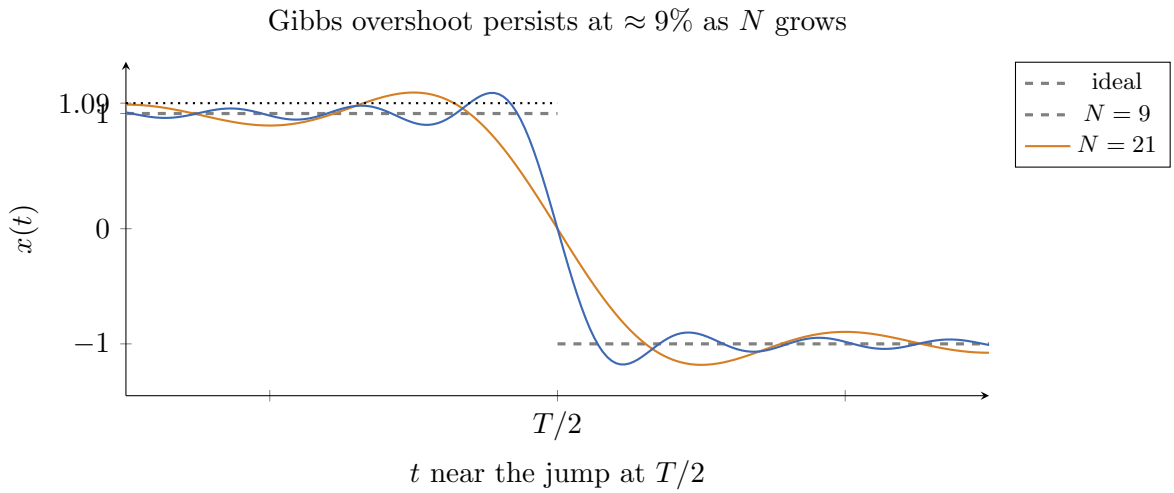


Figure 6.4: Zoom on the jump. Going from $N = 9$ to $N = 21$ harmonics squeezes the overshoot closer to the edge and adds faster ripples, but the peak still reaches roughly 1.09 – about 9% past the true value of 1. More harmonics narrow the error; they do not lower its peak.

History

The overshoot is named for the physicist J. Willard Gibbs, who explained it in letters to *Nature* in 1898–1899. But he was not first: the English mathematician Henry Wilbraham had described and computed it in 1848, fifty years earlier, only to be entirely forgotten. The effect surfaced again when Albert Michelson (of Michelson–Morley fame) built a mechanical *harmonic analyzer* that summed up to 80 sinusoids, and was puzzled to see persistent spikes at the discontinuities of a square wave. He suspected a flaw in his beautiful brass machine. The machine was perfect; mathematics itself was overshooting.

Why it works

When does the series converge nicely, and how fast? The smoother the signal, the faster its coefficients die out and the better the convergence. A useful rule of thumb: if a signal and its first $(p-1)$ derivatives are continuous but the p-th has jumps, the coefficients decay like $1/k^{p+1}$. A square wave (jump in the function itself, $p = 0$) decays like $1/k$. A triangle wave (continuous, but its derivative jumps, $p = 1$) decays like $1/k^2$ – which is why it has no Gibbs overshoot and converges visibly faster. A perfectly smooth signal has coefficients that plummet faster than any power of k. **Sharp corners cost high harmonics.**

6.6 Worked Example: The Triangle Wave

To feel the smoothness-versus-decay rule, compute the triangle wave: the continuous, even, peak-to-peak ± 1 wave that ramps up then down. It is the integral of the square wave, and integration smooths – so we expect faster decay. Carrying out the projection (it is even, so only cosines survive) gives only odd harmonics again, but now falling off as $1/k^2$:

$$x_{\triangle}(t) = \frac{8}{\pi^2}\left(\cos\omega_0 t - \frac{1}{9}\cos 3\omega_0 t + \frac{1}{25}\cos 5\omega_0 t - \cdots\right) = \frac{8}{\pi^2}\sum_{k\,\text{odd}} \frac{(-1)^{(k-1)/2}}{k^2}\cos k\omega_0 t. \tag{6.10}$$

Compare term by term with the square wave (6.9): same odd harmonics, but $1/k^2$ instead of $1/k$. The third harmonic of the square is 1/3 of the fundamental; for the triangle it is only 1/9. The high harmonics are crushed, so just three terms already trace a near-perfect triangle – and there is no overshoot, because the triangle has no jumps for Gibbs to exploit.

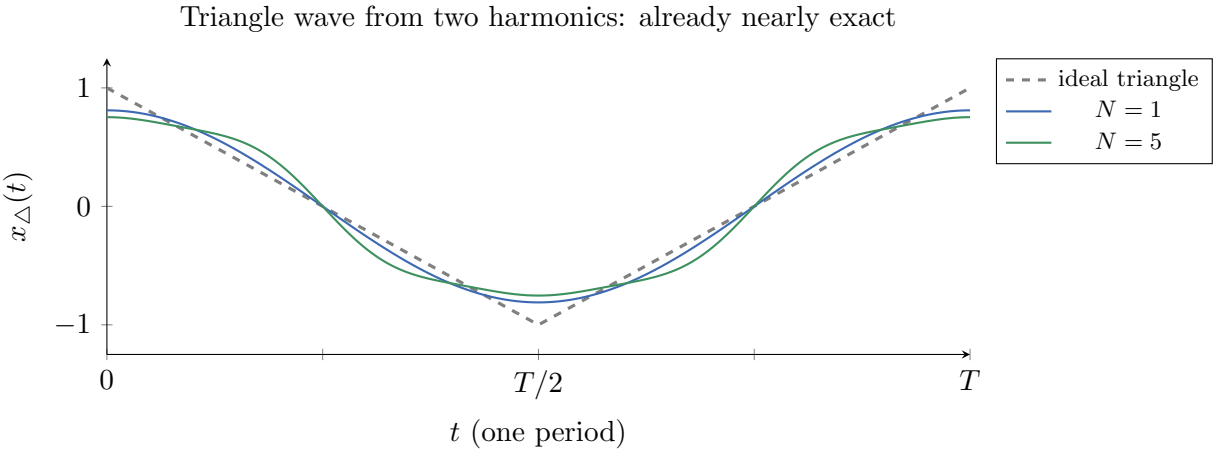


Figure 6.5: The triangle wave converges far faster than the square. With coefficients dropping as $1/k^2$, just the fundamental plus two more odd harmonics ($N = 5$) hugs the ideal ramp with no visible overshoot. Compare Figure 6.3, where the square wave still shows pronounced ripple at the same harmonic count.

Connection

This is the calculus companion's "integration is smoothing" lesson, read in frequency. The triangle is the running integral of the square; integrating multiplies the k-th coefficient by $1/(\mathrm{j}k\omega_0)$ – an extra factor of $1/k$. Differentiating does the reverse, multiplying by $\mathrm{j}k\omega_0$ and *amplifying* high harmonics. That is precisely why differentiators are noisy and integrators are calming: the operations reshape the spectrum by a factor of k.

6.7 The Line Spectrum: What a Periodic Signal Looks Like in Frequency

Plot the coefficient magnitudes $|c_k|$ (or C_k) against frequency $k\omega_0$ and you get the *spectrum* of the periodic signal. Because only the harmonic frequencies $0, \pm\omega_0, \pm 2\omega_0, \ldots$ carry any weight, the spectrum is not a smooth curve – it is a set of isolated *spikes*, one per harmonic, with nothing in between. This is a **line spectrum**, and it is the visual signature of periodicity.

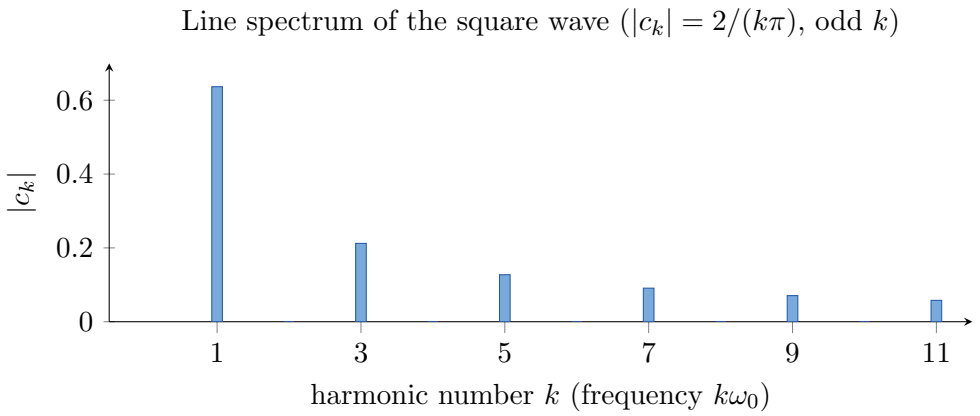


Figure 6.6: The square wave's one-sided line spectrum. Energy sits *only* at odd harmonics; even harmonics are exactly absent (half-wave symmetry). The heights fall as $1/k$ – the slow decay that makes the wave so rich in highs. A periodic signal lives on a discrete comb of frequencies, never in between.

Why this matters

The line spectrum is one of the most important pictures in all of signal processing. **Periodic in time $\Leftrightarrow$ discrete (line) in frequency.** A signal that repeats forever in time uses only a sparse, countable set of frequencies. In the next chapter we will see the mirror image: a signal that does *not* repeat – a one-shot pulse – spreads its energy across a *continuous* band of frequencies. The Fourier transform fills in the gaps between the lines as the period T grows to infinity and the harmonics $\omega_0 = 2\pi/T$ crowd together into a continuum. Hold this duality; it organizes everything that follows.

Application

Why a violin and a flute playing the same note sound different. Both produce a periodic pressure wave at, say, 440 Hz, so both have a line spectrum on the same comb of harmonics $(440, 880, 1320, \ldots$ Hz). What differs is the *heights* of the lines – the relative strength of each harmonic. A flute is nearly a pure fundamental with weak upper harmonics; a violin's bowed string is rich, closer to a sawtooth, with strong highs. This recipe of harmonic amplitudes is *timbre*. Pitch is which comb; timbre is the shape of the line spectrum on it. Synthesizers exploit this directly: additive synthesis literally adds harmonics with chosen C_k to sculpt a tone.

6.8 Energy and Parseval's Theorem

We have a vector and an orthogonal basis, so the Pythagorean theorem must apply – and it does, gloriously. The average power of the signal over one period equals the sum of the powers of its individual harmonics. In complex-exponential form this is *Parseval's theorem for series*:

$$\boxed{\frac{1}{T}\int_0^T |x(t)|^2\,\mathrm{d}t \;=\; \sum_{k=-\infty}^{\infty} |c_k|^2.} \tag{6.11}$$

In trigonometric/amplitude form the same statement reads $\frac{1}{T}\int_0^T x(t)^2\mathrm{d}t = \left(\frac{a_0}{2}\right)^2 + \frac{1}{2}\sum_{k\geq 1}(a_k^2+b_k^2) = C_0^2 + \frac{1}{2}\sum_{k\geq 1} C_k^2$.

Why it works

Parseval is just Pythagoras in the function space. The left side is $\|x\|^2/T$, the squared length of the signal vector (its average power). The right side adds up the squared lengths of its components along each orthogonal basis direction. Because the basis is orthogonal, there are no cross terms – the energy of the whole is the sum of the energies of the parts, exactly as $\|\mathbf{v}\|^2 = \sum_i v_i^2$ in ordinary geometry. The harmonics do not "share" energy; each owns a private slice, and they add.

Example 6.2 (Power of the square wave, two ways)**.** Directly: the square wave is ± 1 everywhere, so $\frac{1}{T}\int_0^T x^2\mathrm{d}t = 1$. Via Parseval with $c_k = 2/(k\pi)$ for odd k (counting both $\pm k$): $\sum_k |c_k|^2 = 2\sum_{k\,\text{odd}>0}\left(\frac{2}{k\pi}\right)^2 = \frac{8}{\pi^2}\sum_{k\,\text{odd}}\frac{1}{k^2} = \frac{8}{\pi^2}\cdot\frac{\pi^2}{8} = 1$. The two answers agree, and as a side effect we have proved $\sum_{k\,\text{odd}} 1/k^2 = \pi^2/8$. Parseval is a power accountant that never loses a watt.

Connection

In the noise and random-signals chapter we will compute power spectral density – where total power is again distributed across frequency, but now for *random* signals where the spectrum is continuous and statistical. Parseval is the deterministic ancestor of that idea: energy conservation across the time/frequency divide. The same accounting governs both.

6.9 Computing Coefficients in Code

For signals you only know as samples (the usual case with real sensors), you approximate the analysis integral (6.5) by a sum over samples – which is, foreshadowing hard, exactly the discrete Fourier transform of Chapters 8–9. Here is the direct, unoptimized version, useful for building intuition before the FFT arrives:

Listing 6.1: Fourier coefficients of one period by direct integration.

```
import numpy as np

def fourier_coeffs(x, T, K):
    """Complex FS coefficients c_k for k = -K..K of one period.
    x: samples over [0,T), evenly spaced. Returns dict k -> c_k."""
    N = len(x)
    t = np.arange(N) * (T / N)            # sample times in [0, T)
    w0 = 2 * np.pi / T
    c = {}
    for k in range(-K, K + 1):
        # average of x(t) * exp(-j k w0 t)  ~=  (1/T) * integral
```

```
        c[k] = np.mean(x * np.exp(-1j * k * w0 * t))
    return c

# Square wave, one period
N = 1000
x = np.where(np.arange(N) < N // 2, 1.0, -1.0)
c = fourier_coeffs(x, T=1.0, K=7)
print("c_1  =", c[1])          # ~ -0.6366j   -> b_1 = 4/pi
print("|c_3| =", abs(c[3]))    # ~ 0.2122     -> 2/(3 pi)
```

Try it yourself

Run the snippet, then *resynthesize*: build `x_hat = sum(c[k]*np.exp(1j*k*w0*t) for k in c)` and overlay `x_hat.real` on the original. Watch the Gibbs ears appear at the jumps. Now sweep `K` from 1 to 50 and plot the maximum overshoot versus `K`: it stalls near 9% and refuses to fall, no matter how high you push `K`. You have just measured Gibbs with your own hands.

6.10 A Note on Real Periodic Signals from Sensors

So far our square and triangle waves were mathematically perfect. Real periodic signals – a heartbeat, a footstep cadence, the cyclic motion of a wrist during a repeated gesture – are only *approximately* periodic, and that gap matters in practice.

Toward the wrist tracker

Imagine the wrist-worn controller and a player tapping a steady beat. The accelerometer sees a roughly periodic bump train. The Fourier series suggests a tempting move: the fundamental ω_0 of that bump train *is* the tap rate, and its harmonics describe the *shape* of each tap (a sharp tap is harmonic-rich; a soft one is nearly a single sinusoid). So a few low-order coefficients could give us both *tempo* and a crude *articulation* feature to map to MIDI velocity.

But be clear-eyed about two limits. (1) Real motion is only *quasi*-periodic: the player drifts in tempo, so a strict series over a fixed T smears – we will need the short-time / windowed view of Chapter 10, not an idealized series. (2) Far more important: a wrist IMU senses the *gross motion of the whole forearm*, not individual fingers. No Fourier analysis of an accelerometer can recover *which* finger moved or a continuous finger-joint angle – that information is simply not present in the wrist's acceleration. Detecting individual finger activity needs electromyography (EMG) or force myography (FMG) over the forearm muscles; continuous finger *pose* needs sensors on the fingers themselves (e.g. instrumented rings). The Fourier series is a superb tool for the rhythmic and harmonic *structure* of whatever the sensor genuinely sees – and a disciplined builder is clear about exactly what that is.

Build it (hardware)

A first hands-on experiment on real hardware. Sample one axis of an MPU-6050 accelerometer at, say, 200 Hz while you tap a finger on a table at a steady rate. Record a few seconds, then:

- Estimate the tap period T (peak-to-peak spacing in the accelerometer trace).
- Window exactly one period and run the `fourier_coeffs` routine above.

- Inspect $|c_1|, |c_2|, |c_3|$: a crisp tap shows strong upper harmonics; a gentle one collapses toward the fundamental.

Map ω_0 to MIDI clock (tempo) and an upper-harmonic ratio like $|c_2|/|c_1|$ to note velocity. This is a real, if coarse, gesture-to-MIDI path that uses only what the IMU genuinely measures. Resist the urge to claim finger identity from it.

6.11 Summary

Pause

- A T-periodic signal is a sum of harmonically related sinusoids – frequencies $k\omega_0$, $\omega_0 = 2\pi/T$. These harmonics form an *orthogonal basis*.
- A Fourier coefficient is a *projection* (inner product) of the signal onto one harmonic: a correlation score for "how much of this pure tone is present."
- The complex form $x(t) = \sum_k c_k e^{\mathrm{j}k\omega_0 t}$, $c_k = \frac{1}{T}\int_0^T x e^{-\mathrm{j}k\omega_0 t}\mathrm{d}t$, is the cleanest; for real signals $c_{-k} = \overline{c_k}$.
- Sharp features (jumps, corners) demand strong high harmonics; smoother signals have faster-decaying coefficients ($1/k$ for a square, $1/k^2$ for a triangle).
- At a jump the series hits the midpoint and overshoots by $\approx 9\%$ (Gibbs); the overshoot narrows but never shrinks in height.
- The spectrum of a periodic signal is a *line spectrum* – discrete spikes on a harmonic comb. Periodic in time $\Leftrightarrow$ discrete in frequency.
- Parseval: average power equals the sum of squared coefficient magnitudes – Pythagoras in function space.

6.12 Exercises

Exercise 6.1. For the period-T signal $x(t) = A$ for $0 < t < T/4$ and 0 elsewhere in $[0, T)$ (a duty-cycle-$\frac{1}{4}$ pulse train), compute c_0 and a general c_k. Show c_0 is the time average $A/4$, and that $|c_k|$ follows a |sinc|-like envelope. (This previews the connection between rectangular pulses and the sinc that dominates the next chapter.)

Exercise 6.2. A signal has only the coefficients $c_0 = 0$, $c_{\pm 1} = \frac{1}{2}$, $c_{\pm 2} = \frac{1}{4}\mathrm{j}$, all others zero, with $\omega_0 = 2\pi$. Write $x(t)$ as a sum of real sinusoids (amplitude–phase form). Confirm it is real by checking $c_{-k} = \overline{c_k}$.

Exercise 6.3. Prove the cross-orthogonality $\int_0^T \sin(m\omega_0 t)\cos(n\omega_0 t)\,\mathrm{d}t = 0$ for all integers $m, n \geq 1$. (Hint: use $\sin A \cos B = \frac{1}{2}[\sin(A+B) + \sin(A-B)]$ and the whole-cycle averaging argument.)

Exercise 6.4. The full-wave rectified cosine $x(t) = |\cos(\omega_0 t/2)|$ has period $T = \pi/\omega_0$ times two... more carefully: $|\cos t|$ has period π. Compute its Fourier coefficients and show all harmonics are present but decay like $1/k^2$. Where does its DC value $2/\pi$ come from physically? (This is exactly the ripple a rectifier feeds a power-supply filter.)

Exercise 6.5. Using the square-wave series (6.9), evaluate it at $t = T/4$ to re-derive the Leibniz formula $\pi/4 = 1 - \frac{1}{3} + \frac{1}{5} - \cdots$. Then evaluate the *triangle*-wave series (6.10) at its peak and identify the numeric series you obtain.

Exercise 6.6. Sketch (or compute) the line spectrum of $x(t) = 2 + 3\cos(\omega_0 t) - \sin(2\omega_0 t)$. Give $c_0, c_{\pm 1}, c_{\pm 2}$ and the magnitudes $|c_k|$. Verify Parseval directly by integrating x^2 over one period and comparing to $\sum_k |c_k|^2$.

Exercise 6.7. A sawtooth wave $x(t) = t/T$ for $0 \le t < T$ (then repeating) has $c_k = \mathrm{j}/(2\pi k)$ for $k \neq 0$ and $c_0 = \frac{1}{2}$. (a) Verify c_0. (b) Confirm the $1/k$ decay and explain, via the smoothness rule, why a sawtooth and a square wave share the same decay rate. (c) Predict whether the sawtooth shows Gibbs overshoot, and where.

Exercise 6.8. Modify the `fourier_coeffs` listing to also *resynthesize* the signal from K harmonics and return the maximum reconstruction error. Run it on the square wave for $K = 5, 15, 45$. Report the peak overshoot in each case and confirm it approaches roughly 9% rather than zero.

Exercise 6.9. (Smoothness vs. decay.) Construct a continuous, piecewise-linear period-T signal of your choice whose first derivative is discontinuous. Predict its coefficient decay rate from the smoothness rule, compute the first six coefficients numerically, and check the prediction by fitting $\log|c_k|$ versus $\log k$.

Exercise 6.10. (Toward the build.) You record a quasi-periodic accelerometer tap train and find the fundamental is not constant – the player speeds up over four seconds. Explain in one paragraph why a single Fourier series over the whole record is the wrong tool, and what you would do instead (name the chapter-10 technique). Then state plainly what a wrist IMU *cannot* tell you about individual fingers, and which sensor you would add to get that information.

Chapter 7

The Fourier Transform

In the last chapter we learned to take a *periodic* signal and break it into a discrete ladder of harmonics: a sum of sinusoids whose frequencies are integer multiples of one fundamental. That is the Fourier *series*. But almost nothing in the real world is exactly periodic. A hand clap, a footstep, a single twitch of a muscle, the chirp of a bird, the pulse you send down a wire – these are *transient* events. They happen once and are gone. They have no period at all.

The Fourier transform is what the Fourier series becomes when we let the period grow without bound. The discrete ladder of harmonics fuses into a continuous *spectrum*. Instead of asking "how much of harmonic number 7 is in this signal?" we ask "how much of *every* frequency, continuously, is in this signal?" The answer is itself a function – a function of frequency rather than of time. Learning to read that function, to see a time-domain wiggle and its frequency-domain fingerprint as two faces of one object, is the central skill of all signal processing.

This chapter builds that bridge geometrically. We will watch the harmonics crowd together as the period stretches, define the continuous-time Fourier transform (CTFT) and its inverse, learn to read magnitude and phase, compute the handful of transforms that you will reach for again and again (especially the rectangle and its sinc), meet the uncertainty principle that ties time-width to frequency-width, and close the loop with Parseval's energy theorem. Throughout, the through-line of the book – a wrist-worn gesture controller that emits MIDI – gives us a reason to care: a gesture is a transient, and the transform is how we will look at it.

7.1 From series to transform: letting the period explode

Recall the complex (exponential) form of the Fourier series for a signal $x(t)$ that repeats every T seconds. With fundamental angular frequency $\omega_0 = 2\pi/T$,

$$x(t) = \sum_{k=-\infty}^{\infty} c_k \, e^{\mathrm{j}k\omega_0 t}, \qquad c_k = \frac{1}{T}\int_{-T/2}^{T/2} x(t)\, e^{-\mathrm{j}k\omega_0 t}\,\mathrm{d}t. \tag{7.1}$$

Each coefficient c_k is a complex number telling us the amplitude and phase of the harmonic sitting at frequency $k\omega_0$. The harmonics live on a ladder whose rungs are spaced $\omega_0 = 2\pi/T$ apart. A short period T means widely spaced rungs; a long period means a fine ladder.

Now imagine a signal that is really just a single isolated pulse – say a smooth bump of muscle activity

lasting a fraction of a second – but which we *pretend* repeats with some huge period T. As we slide T to larger and larger values, two things happen at once.

- The rungs of the ladder, spaced $\omega_0 = 2\pi/T$, move closer together. In the limit $T \to \infty$ they merge into a *continuum* of frequencies.
- Each coefficient c_k, which carries a factor $1/T$, shrinks toward zero. The signal is spread out over a longer and longer period, so any single harmonic carries vanishingly little of it.

The trick is to track the right quantity so that nothing collapses to zero. Define

$$X(\mathrm{j}\omega) \;=\; T\,c_k\Big|_{k\omega_0=\omega} \;=\; \int_{-T/2}^{T/2} x(t)\,e^{-\mathrm{j}\omega t}\,\mathrm{d}t. \tag{7.2}$$

We multiplied c_k by T to kill the troublesome $1/T$, and we relabeled the discrete index $k\omega_0$ as a continuous variable ω. The quantity $X(\mathrm{j}\omega)$ is the *envelope* that the scaled coefficients trace out. It does not collapse as $T \to \infty$; it converges to a perfectly good function of ω.

Intuition

Think of the spectrum as a row of vertical bars, one per harmonic, with height $|c_k|$. As T grows, the bars get shorter (height $\sim 1/T$) but also more numerous and densely packed. If instead you plot $T|c_k| = |X(\mathrm{j}\omega)|$, the bars settle onto a fixed *curve*. The Fourier transform is that curve: the smooth silhouette the harmonics approach as the period explodes.

To recover $x(t)$ we take the synthesis sum (7.1) and substitute $c_k = X(\mathrm{j}k\omega_0)/T = X(\mathrm{j}k\omega_0)\,\omega_0/(2\pi)$:

$$x(t) = \sum_{k=-\infty}^{\infty} \frac{\omega_0}{2\pi}\,X(\mathrm{j}k\omega_0)\,e^{\mathrm{j}k\omega_0 t}. \tag{7.3}$$

This is a Riemann sum. The samples are spaced $\Delta\omega = \omega_0$ apart, the summand is $\frac{1}{2\pi}X(\mathrm{j}\omega)e^{\mathrm{j}\omega t}\,\Delta\omega$, and as $\omega_0 \to 0$ the sum becomes an integral. We have arrived:

$$x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(\mathrm{j}\omega)\,e^{\mathrm{j}\omega t}\,\mathrm{d}\omega. \tag{7.4}$$

Harmonic bars settling onto the transform envelope $|X(\mathrm{j}\omega)|$

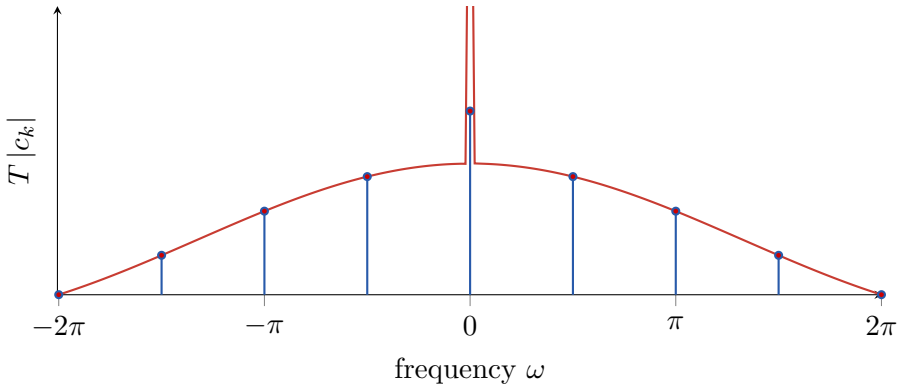


Figure 7.1: As the assumed period T grows, the harmonics (blue stems) crowd together and the scaled coefficients $T|c_k|$ trace the continuous envelope $|X(\mathrm{j}\omega)|$ (red). The Fourier transform is the limiting silhouette of the line spectrum.

History

Joseph Fourier (1768–1830) introduced his trigonometric series in 1807 while studying heat flow, to the dismay of Lagrange and Laplace, who doubted that arbitrary functions could be built from smooth sinusoids. The passage from series to the integral transform was developed through the nineteenth century (Cauchy, Poisson, Dirichlet) and made fully rigorous in the twentieth with the theory of distributions (Schwartz) and the L^2 theory (Plancherel). The engineer's habit of writing $X(\mathrm{j}\omega)$ – with the imaginary unit baked into the argument – comes from viewing the transform as the Laplace transform evaluated on the imaginary axis $s = \mathrm{j}\omega$, a connection we revisit in the chapter on the Laplace and z transforms.

7.2 The continuous-time Fourier transform

Definition 7.1 (Fourier transform pair)**.** For a signal $x(t)$ the *Fourier transform* (analysis equation) and its *inverse* (synthesis equation) are

$$\boxed{X(\mathrm{j}\omega) = \int_{-\infty}^{\infty} x(t)\, e^{-\mathrm{j}\omega t}\, \mathrm{d}t} \qquad \boxed{x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\mathrm{j}\omega)\, e^{\mathrm{j}\omega t}\, \mathrm{d}\omega} \tag{7.5}$$

We write $x(t) \overset{\mathcal{F}}{\longleftrightarrow} X(\mathrm{j}\omega)$, or $X = \mathcal{F}\{x\}$ and $x = \mathcal{F}^{-1}\{X\}$.

The two formulas are nearly mirror images. The forward transform multiplies by $e^{-\mathrm{j}\omega t}$ and integrates over time; the inverse multiplies by $e^{+\mathrm{j}\omega t}$ and integrates over frequency, with a $1/(2\pi)$ out front. That lone factor of 2π is the only asymmetry, and it is the price of measuring frequency in radians per second. (Some authors measure in cycles per second, $f = \omega/2\pi$; then both integrals carry $e^{\mp \mathrm{j}2\pi f t}$ and the 2π disappears. We will note this convention where it matters.)

Why it works

Why does $\int x(t)\, e^{-\mathrm{j}\omega t}\, \mathrm{d}t$ measure "how much frequency ω is in x"? Because $e^{-\mathrm{j}\omega t}$ is a unit-length complex phasor spinning at rate $-\omega$. Multiplying $x(t)$ by it *de-rotates* any component of x that spins at $+\omega$, freezing it into a constant; every other component keeps spinning. Integrating then averages: the frozen component survives and accumulates, while the still-spinning components average to nearly zero over the whole line. The integral is a *correlation* of x with a test phasor – a projection of the signal onto the sinusoid of frequency ω. It is exactly the inner product $\langle x,\, e^{\mathrm{j}\omega t} \rangle$ from the linear-algebra view of signals, now taken over the whole real line.

What is really happening

The inverse transform (7.4) is a recipe: to rebuild $x(t)$, take a phasor $e^{\mathrm{j}\omega t}$ at *every* frequency, weight it by the complex number $X(\mathrm{j}\omega)\, \mathrm{d}\omega/(2\pi)$, and add them all up. A transient signal is therefore an infinite blend of eternal, never-decaying sinusoids whose phases are arranged so that they interfere constructively in one small region of time and destructively everywhere else. The whole drama of a pulse is hidden in the *phase* agreement of infinitely many sine waves.

When does the integral exist? A clean sufficient condition is absolute integrability:

$$\int_{-\infty}^{\infty} |x(t)|\,\mathrm{d}t < \infty \quad \Longrightarrow \quad X(\mathrm{j}\omega) \text{ exists, bounded and continuous.} \tag{7.6}$$

This covers every finite-energy pulse you will actually sample from a sensor. Many useful idealizations – a pure cosine, a constant, a step – are *not* absolutely integrable, yet they have perfectly usable transforms once we allow Dirac deltas. We treat those as limits and as distributions; the engineering payoff is enormous and the bookkeeping is light.

Connection

This is the same projection idea as the inner product in the linear-algebra companion, pushed to a continuous "basis." There, a vector $\mathbf{v}$ is reconstructed as $\sum_k \langle \mathbf{v}, \mathbf{e}_k \rangle \mathbf{e}_k$ over an orthonormal basis $\{\mathbf{e}_k\}$. Here the "basis vectors" are the phasors $e^{\mathrm{j}\omega t}$, indexed by a continuous label ω, and the sum becomes the integral (7.4). The catch is that $e^{\mathrm{j}\omega t}$ has infinite energy, so it is not a true element of the space L^2; making the analogy airtight needs the theory of tempered distributions. The geometry, though, is exactly the finite-dimensional one you already know.

7.3 Magnitude and phase: reading a spectrum

$X(\mathrm{j}\omega)$ is complex, so at each frequency it carries two numbers. Write it in polar form:

$$X(\mathrm{j}\omega) = |X(\mathrm{j}\omega)|\, e^{\mathrm{j}\angle X(\mathrm{j}\omega)}. \tag{7.7}$$

The *magnitude spectrum* $|X(\mathrm{j}\omega)|$ says how strongly frequency ω is present. The *phase spectrum* $\angle X(\mathrm{j}\omega)$ says *where* that sinusoid sits in time – its alignment. Magnitude tells you the ingredients; phase tells you when each ingredient arrives.

Common trap

Beginners stare only at the magnitude spectrum and ignore phase. For *recognizing* a steady tone, magnitude is often enough. But phase is what localizes events in time. Two signals can share an identical magnitude spectrum yet sound or look completely different: a sharp drum hit and a smeared whoosh can have the same magnitude spectrum and opposite phase behavior. If you ever scramble the phases of a recording while keeping magnitudes, you destroy all transients and are left with noise-like mush. Phase is not optional.

Real signals have conjugate-symmetric spectra. If $x(t)$ is real (every physical sensor reading is), then

$$X(-\mathrm{j}\omega) = \overline{X(\mathrm{j}\omega)}. \tag{7.8}$$

This single fact has two consequences you will use constantly: the magnitude spectrum is *even* ($|X(-\mathrm{j}\omega)| = |X(\mathrm{j}\omega)|$) and the phase spectrum is *odd*. So the negative-frequency half of a real signal's spectrum is just a mirror of the positive half – it carries no new information, only the bookkeeping needed to keep the synthesis real. This is why, in practice, we plot only $\omega \geq 0$.

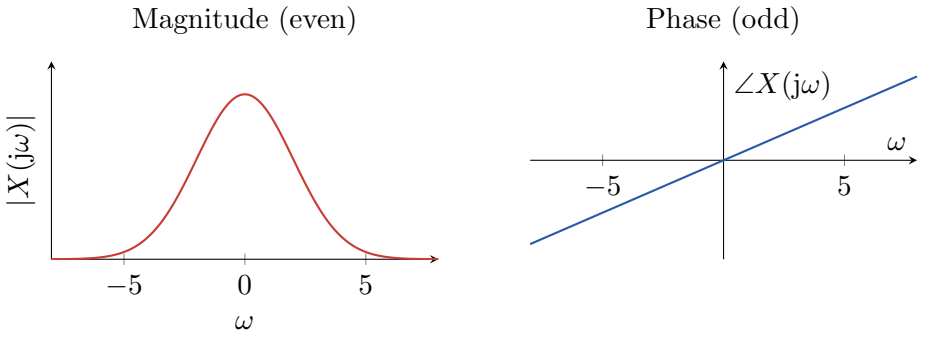


Figure 7.2: For any real signal the magnitude spectrum is an even function and the phase spectrum is odd. A linear phase like the one shown ($\angle X = -\omega t_0$ up to sign) corresponds to a pure time shift.

Intuition

A *linear* phase, $\angle X(j\omega) = -\omega t_0$, means "every frequency is delayed by the same amount t_0." Geometrically, delaying a signal in time rotates each phasor in proportion to its frequency: fast phasors must rotate more to keep step. A delay therefore shows up as a phase ramp, never as a change in magnitude. We prove this as the time-shift property in the properties chapter, but the picture is worth keeping now: *shifting in time tilts the phase, it never touches the magnitude.*

7.4 A worked transform: the rectangle and the sinc

The single most important transform pair in the whole subject is the rectangle and the sinc. Master this one and half of sampling theory, filter design, and windowing will feel inevitable.

Let the unit-height rectangular pulse of total width τ be

$$x(t) = \operatorname{rect}\left(\frac{t}{\tau}\right) = \begin{cases} 1, & |t| < \tau/2, \\ 0, & |t| > \tau/2. \end{cases} \tag{7.9}$$

Its transform is a direct integral:

$$\begin{aligned} X(j\omega) &= \int_{-\tau/2}^{\tau/2} e^{-j\omega t}\,dt = \left[\frac{e^{-j\omega t}}{-j\omega}\right]_{-\tau/2}^{\tau/2} = \frac{e^{j\omega\tau/2} - e^{-j\omega\tau/2}}{j\omega} \\ &= \frac{2\sin(\omega\tau/2)}{\omega} = \tau\,\frac{\sin(\omega\tau/2)}{\omega\tau/2} = \tau\,\operatorname{sinc}\left(\frac{\omega\tau}{2}\right), \end{aligned} \tag{7.10}$$

using the unnormalized convention $\operatorname{sinc}(u) = \sin(u)/u$ with $\operatorname{sinc}(0) = 1$. So

$$\boxed{\operatorname{rect}\left(\frac{t}{\tau}\right) \xleftrightarrow{\mathcal{F}} \tau\,\operatorname{sinc}\left(\frac{\omega\tau}{2}\right).} \tag{7.11}$$

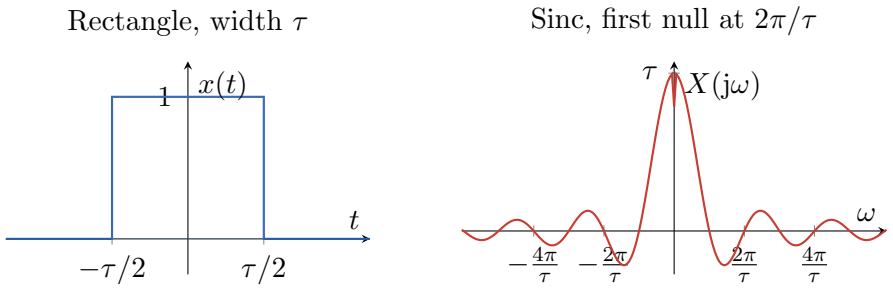


Figure 7.3: The fundamental pair. A rectangle of width τ (here $\tau = 2$) transforms to $\tau\,\mathrm{sinc}(\omega\tau/2)$. The first zero crossings sit at $\omega = \pm 2\pi/\tau$, so a *narrower* pulse gives a *wider* sinc. The central peak has height τ, the area of the pulse.

Two features of (7.10) deserve a moment.

- **The DC value is the area.** At $\omega = 0$, $X(\mathrm{j}0) = \tau = \int x\,\mathrm{d}t$. This is general: $X(\mathrm{j}0)$ always equals the total area of the signal. Frequency zero asks "what is the net DC content?" and the answer is the integral.
- **Nulls at multiples of $2\pi/\tau$.** The sinc crosses zero whenever $\omega\tau/2 = m\pi$, i.e. $\omega = 2\pi m/\tau$. A short pulse pushes those nulls outward: its energy is smeared across a broad band. A long pulse pulls them inward: its energy is concentrated near DC. This reciprocity is the seed of the uncertainty principle in Section 7.6.

Example 7.1 (A 20 ms gate)**.** Suppose a gesture detector marks a window of activity as a rectangular gate of width $\tau = 20\,\mathrm{ms} = 0.02\,\mathrm{s}$. The first spectral null of that gate sits at $\omega = 2\pi/\tau = 2\pi/0.02 \approx 314\,\mathrm{rad/s}$, i.e. $f = 1/\tau = 50\,\mathrm{Hz}$. So merely *turning a measurement on and off* over 20 ms injects spectral content spread out to roughly 50 Hz and beyond – a fact we must respect when we later filter the gated signal, lest the gate's own spectrum masquerade as real motion.

Why it works

Why a sinc and not some other curve? Because a rectangle is the indicator of an interval, and the transform of an indicator is the integral of a complex phasor over that interval. Integrating $e^{-\mathrm{j}\omega t}$ over a symmetric interval gives a difference of two phasors $e^{\pm\mathrm{j}\omega\tau/2}$, and the difference of two unit phasors equidistant from the real axis is purely imaginary and equal to $2\mathrm{j}\sin(\cdot)$. Divide by the $\mathrm{j}\omega$ from the integration and you are left with $2\sin(\omega\tau/2)/\omega$. The sinc is the fingerprint of *sharp edges*: any hard on/off boundary in time produces sinc-like ripples in frequency.

Duality: it works the other way too. Because the analysis and synthesis integrals differ only by a sign and a 2π, the pair runs in reverse. A *sinc in time* transforms to a *rectangle in frequency*:

$$\frac{\sin(\Omega t)}{\pi t} \;\overset{\mathcal{F}}{\longleftrightarrow}\; \mathrm{rect}\!\left(\frac{\omega}{2\Omega}\right) = \begin{cases} 1, & |\omega| < \Omega, \\ 0, & |\omega| > \Omega. \end{cases} \tag{7.12}$$

This is the ideal low-pass filter: a brick wall in frequency that passes everything below Ω and blocks everything above. Its time-domain shape is an infinitely long, two-sided sinc – which is why a *perfect*

brick-wall filter is non-causal and must be approximated in practice. We will build that story in the filtering chapters; for now, notice that duality hands us a second pair for free.

Why this matters

There is something almost magical in the rect/sinc duality. The crispest possible thing in time (a hard-edged box) is the most spread-out thing in frequency (an endless ripple), and vice versa. The Fourier transform turns sharpness into spread and spread into sharpness, like a lens that swaps near and far. Once you feel this, you can predict the qualitative shape of a spectrum just by looking at how abrupt a signal is.

7.5 More essential transforms

A small dictionary of pairs covers most of practice. Each one rewards a geometric reading.

7.5.1 The impulse and the constant

The Dirac delta $\delta(t)$ is the idealized instantaneous spike with unit area. By the sifting property,

$$\mathcal{F}\{\delta(t)\} = \int_{-\infty}^{\infty} \delta(t)\, e^{-\mathrm{j}\omega t}\, \mathrm{d}t = e^{-\mathrm{j}\omega\cdot 0} = 1. \tag{7.13}$$

An impulse contains *every* frequency equally and in phase – a perfectly flat, white spectrum. This is why striking a system with a sharp impulse and listening to the response reveals the system's behavior at all frequencies at once (the impulse response, central to LTI systems). Running the pair backward by duality gives the transform of a constant:

$$\mathcal{F}\{1\} = 2\pi\, \delta(\omega). \tag{7.14}$$

A constant (pure DC) is all area at frequency zero and nothing elsewhere – an impulse in the frequency domain.

7.5.2 The complex exponential and the cosine

Shifting the constant's spectrum sideways gives the eternal phasor:

$$\mathcal{F}\{e^{\mathrm{j}\omega_0 t}\} = 2\pi\, \delta(\omega - \omega_0). \tag{7.15}$$

A single never-ending sinusoid is a single spectral line – an impulse at its own frequency. Real cosines and sines are sums of two phasors, so they give two lines:

$$\mathcal{F}\{\cos(\omega_0 t)\} = \pi\, \delta(\omega - \omega_0) + \pi\, \delta(\omega + \omega_0), \qquad \mathcal{F}\{\sin(\omega_0 t)\} = \frac{\pi}{\mathrm{j}}\big[\delta(\omega - \omega_0) - \delta(\omega + \omega_0)\big]. \tag{7.16}$$

The pair of impulses at $\pm\omega_0$ is exactly the conjugate symmetry of (7.8) for a real signal. Cosine's two impulses are real and equal; sine's are imaginary and opposite, encoding the 90° phase difference between them (Figure 7.4).

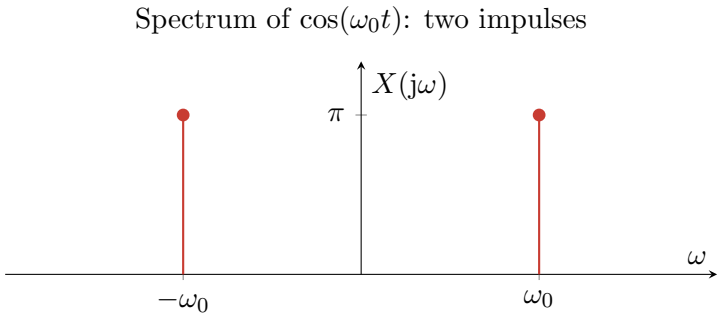


Figure 7.4: A never-ending cosine has a line spectrum: two impulses of weight π at $\omega = \pm\omega_0$ (arrows drawn as stems, since a true impulse has infinite height and finite area). Sine differs only in that its two impulses are imaginary and of opposite sign.

7.5.3 The one-sided decaying exponential

A causal decay $x(t) = e^{-at}\,u(t)$ with $a > 0$ (here u is the unit step) is the prototype of a first-order system settling toward rest:

$$X(\mathrm{j}\omega) = \int_0^\infty e^{-at} e^{-\mathrm{j}\omega t}\,\mathrm{d}t = \frac{1}{a + \mathrm{j}\omega}, \qquad |X(\mathrm{j}\omega)| = \frac{1}{\sqrt{a^2 + \omega^2}}. \tag{7.17}$$

The magnitude falls off as $1/\omega$ at high frequency – a gentle low-pass shape. The parameter a sets the *corner*: at $\omega = a$ the magnitude has dropped by $1/\sqrt{2}$. Fast decay (large a, short time constant) means a wide spectrum; slow decay means a narrow one. The reciprocity strikes again (Figure 7.5).

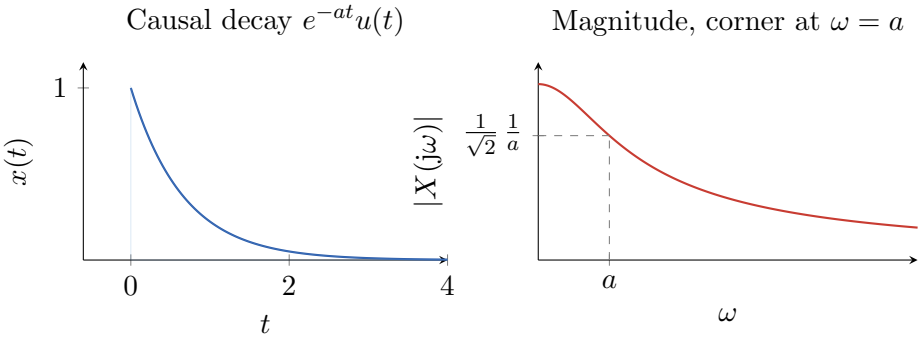


Figure 7.5: The one-sided decaying exponential and its magnitude spectrum $1/\sqrt{a^2 + \omega^2}$. The response is flat near DC (height $1/a$) and rolls off as $1/\omega$; at the corner $\omega = a$ it has fallen to $1/\sqrt{2}$ of its peak. A faster decay (larger a) pushes the corner higher and widens the band.

7.5.4 The Gaussian: the shape that transforms to itself

The Gaussian pulse $x(t) = e^{-t^2/(2\sigma^2)}$ has the remarkable property that its transform is again a Gaussian:

$$e^{-t^2/(2\sigma^2)} \;\xleftrightarrow{\mathcal{F}}\; \sigma\sqrt{2\pi}\; e^{-\sigma^2\omega^2/2}. \tag{7.18}$$

A wide time-Gaussian (large σ) becomes a narrow frequency-Gaussian (width $1/\sigma$), and conversely. No edges, no ripples – the Gaussian is the smoothest possible bump and pays for that smoothness

with the smoothest possible spectrum. It is also the unique shape that *simultaneously minimizes* the time-frequency spread, as we will see next, which is why Gaussian windows are prized in spectral analysis.

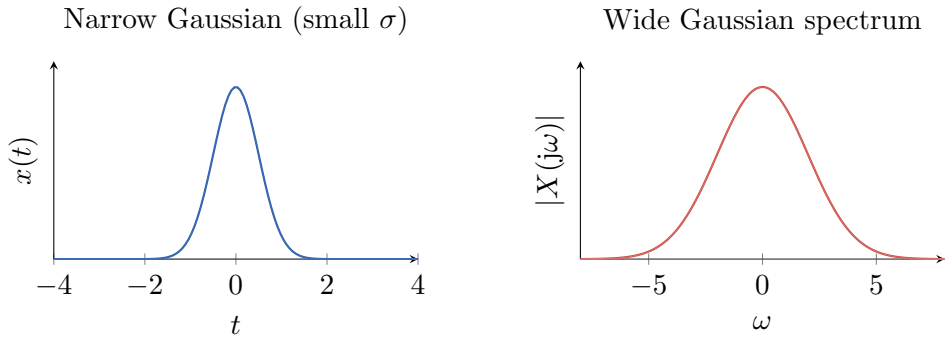


Figure 7.6: A Gaussian transforms to a Gaussian. Squeeze it in time and it spreads in frequency by the reciprocal amount; their widths multiply to a constant.

Table 7.1: A working dictionary of Fourier transform pairs (radian-frequency convention).

Name	$x(t)$	$X(\mathrm{j}\omega)$
Impulse	$\delta(t)$	1
Constant	1	$2\pi\,\delta(\omega)$
Phasor	$e^{\mathrm{j}\omega_0 t}$	$2\pi\,\delta(\omega-\omega_0)$
Cosine	$\cos(\omega_0 t)$	$\pi[\delta(\omega-\omega_0)+\delta(\omega+\omega_0)]$
Rectangle	$\mathrm{rect}(t/\tau)$	$\tau\ \mathrm{sinc}(\omega\tau/2)$
Sinc	$\dfrac{\sin(\Omega t)}{\pi t}$	$\mathrm{rect}(\omega/2\Omega)$
Causal decay	$e^{-at}u(t),\ a>0$	$\dfrac{1}{a+\mathrm{j}\omega}$
Gaussian	$e^{-t^2/(2\sigma^2)}$	$\sigma\sqrt{2\pi}\,e^{-\sigma^2\omega^2/2}$

7.6 The uncertainty principle: you cannot be sharp in both worlds

Every pair we have met whispers the same law. Narrow in time means broad in frequency. The rectangle's sinc nulls march outward as τ shrinks; the decaying exponential's corner climbs as the decay quickens; the Gaussian's frequency width is exactly $1/\sigma$. This is not a coincidence of examples. It is a theorem.

Define the effective *duration* and *bandwidth* of a finite-energy signal as the standard deviations of its energy density in each domain. With energy $E=\int|x(t)|^2\mathrm{d}t=\frac{1}{2\pi}\int|X(\mathrm{j}\omega)|^2\mathrm{d}\omega$ (Parseval, Section 7.7), and assuming the signal centered at $t=0$ and $\omega=0$,

$$(\Delta t)^2=\frac{1}{E}\int t^2|x(t)|^2\,\mathrm{d}t,\qquad (\Delta\omega)^2=\frac{1}{2\pi E}\int\omega^2|X(\mathrm{j}\omega)|^2\,\mathrm{d}\omega. \tag{7.19}$$

Theorem 7.1 (Time-bandwidth uncertainty)**.** *For any finite-energy signal,*

$$\Delta t\cdot\Delta\omega\ \geq\ \tfrac{1}{2}, \tag{7.20}$$

with equality only *for a Gaussian pulse.*

The proof is a Cauchy–Schwarz argument applied to $t\,x(t)$ and $x'(t)$, using integration by parts and the differentiation property $\mathcal{F}\{x'(t)\} = \mathrm{j}\omega\, X(\mathrm{j}\omega)$; we sketch it in the exercises. The takeaway needs no algebra.

Intuition

To pin down *when* something happens you must look at a short slice of time – but a short slice cannot contain many cycles of any single sinusoid, so its frequency is blurry. To pin down *what frequency* a tone is, you must watch it for many cycles – but watching for a long time blurs out exactly *when* it occurred. Time-precision and frequency-precision are antagonists. You can trade one for the other, but their product can never beat the floor $\frac{1}{2}$.

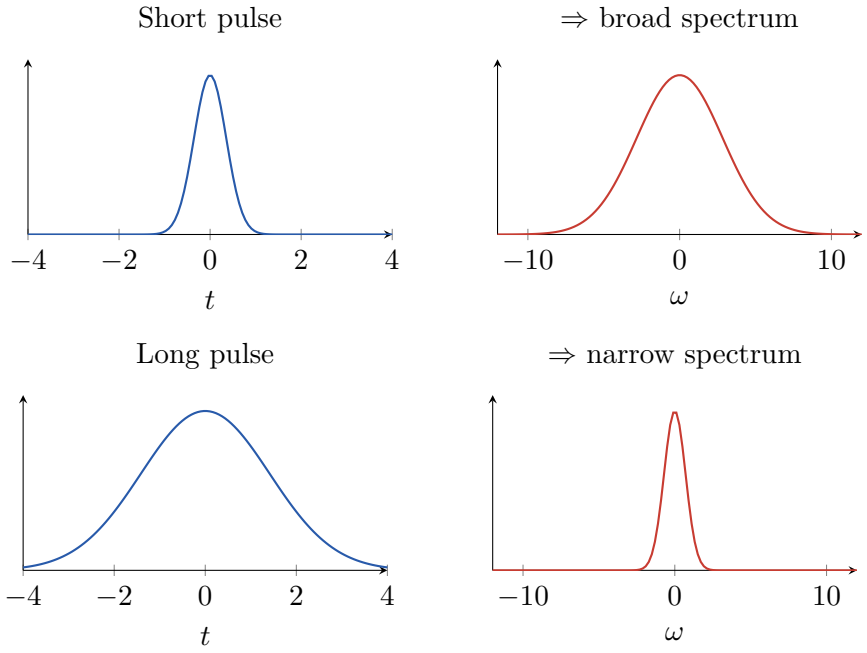


Figure 7.7: The uncertainty tradeoff in pictures. Squeezing the pulse in time (top) forces the spectrum to spread; stretching it (bottom) sharpens the spectrum. Their widths cannot both be small.

Application

This is the governing constraint of every spectrogram. To see a fast event you use a short analysis window and accept coarse frequency resolution; to resolve two close tones you use a long window and accept smeared timing. There is no setting that gives both. The short-time Fourier transform chapter is, in a sense, one long meditation on choosing the least-bad point on this tradeoff for the job at hand.

Toward the wrist tracker

The uncertainty principle is the first hard physical limit on our wrist controller, and it is worth stating bluntly. Suppose we want the instrument to respond to a flick of the wrist within 20 ms so it feels instantaneous to a player. By Theorem 7.1, a 20 ms analysis window can resolve frequency only to roughly $\Delta f \sim 1/(2\pi \cdot 0.02) \approx 8\,\text{Hz}$ at the theoretical floor – in practice, with a real tapering window, more like 50–80 Hz. That is plenty to tell a sharp tap from a slow sweep, but it is *not* enough to separate two motions that differ only by a few Hz in their tremor spectrum. Low latency and fine frequency discrimination pull in opposite directions; the controller's gesture set must be designed to live within that budget rather than fight it. We return to this when we lay out the capstone.

7.7 Parseval and the conservation of energy

The transform does not just relocate a signal's information; it preserves its *energy*. This is the continuous analogue of the fact that a rotation does not change the length of a vector – the Fourier transform is (up to the 2π) an isometry.

Theorem 7.2 (Parseval / Rayleigh energy theorem)**.** *For finite-energy signals,*

$$\int_{-\infty}^{\infty} |x(t)|^2 \,\mathrm{d}t = \frac{1}{2\pi} \int_{-\infty}^{\infty} |X(\mathrm{j}\omega)|^2 \,\mathrm{d}\omega. \tag{7.21}$$

More generally the inner product is preserved: $\int x(t)\overline{y(t)}\,\mathrm{d}t = \frac{1}{2\pi}\int X(\mathrm{j}\omega)\overline{Y(\mathrm{j}\omega)}\,\mathrm{d}\omega$.

Why it works

The general identity follows by writing one signal through the inverse transform and swapping the order of integration:

$$\int x(t)\overline{y(t)}\,\mathrm{d}t = \int x(t)\left[\frac{1}{2\pi}\int \overline{Y(\mathrm{j}\omega)}\, e^{-\mathrm{j}\omega t}\,\mathrm{d}\omega\right]\mathrm{d}t = \frac{1}{2\pi}\int \overline{Y(\mathrm{j}\omega)}\left[\int x(t)e^{-\mathrm{j}\omega t}\mathrm{d}t\right]\mathrm{d}\omega,$$

and the inner bracket is exactly $X(\mathrm{j}\omega)$. Setting $y = x$ gives the energy form. In the language of the linear-algebra companion, the transform preserves inner products, so it preserves lengths and angles: it is a unitary map (after rescaling by $1/\sqrt{2\pi}$).

The quantity $|X(\mathrm{j}\omega)|^2$ is the *energy spectral density*: integrate it over a band of frequencies and you get the energy the signal carries in that band. This is how you answer questions like "what fraction of this twitch's energy lives below 30 Hz?" – you integrate the density there and divide by the total.

Example 7.2 (Energy of a rectangular pulse, both ways)**.** The pulse $\mathrm{rect}(t/\tau)$ has time-domain energy $\int_{-\tau/2}^{\tau/2} 1^2\,\mathrm{d}t = \tau$. Parseval insists the frequency side agrees:

$$\frac{1}{2\pi}\int_{-\infty}^{\infty}\left[\tau\,\mathrm{sinc}\!\left(\tfrac{\omega\tau}{2}\right)\right]^2\mathrm{d}\omega = \frac{\tau^2}{2\pi}\cdot\frac{2\pi}{\tau} = \tau,$$

using the standard integral $\int_{-\infty}^{\infty}\mathrm{sinc}^2(\omega\tau/2)\,\mathrm{d}\omega = 2\pi/\tau$. The numbers match. The energy of a pulse is the same whether you count it in time or sum the squared magnitudes of its frequency content.

Connection

Parseval is the integral cousin of the Pythagorean theorem. In a finite-dimensional inner product space, $\|\mathbf{v}\|^2 = \sum_k |\langle \mathbf{v}, \mathbf{e}_k \rangle|^2$ over an orthonormal basis. Here the "coordinates" of x are its transform values $X(\mathrm{j}\omega)$, the sum becomes an integral, and the $1/(2\pi)$ is the normalization that makes the phasor "basis" behave like an orthonormal one. Energy in time equals energy in frequency for the same reason that a vector's length does not depend on which orthonormal axes you measure it along.

7.8 A computational sanity check

We have not yet built the discrete machinery (that is the next chapter), but it is worth seeing that the rect/sinc pair is not an abstraction. The following snippet samples a rectangle densely, takes a crude numerical transform, and confirms the sinc shape and the first-null location.

Listing 7.1: Numerically confirming rect to sinc.

```
import numpy as np

fs   = 2000.0            # samples per second
t    = np.arange(-1.0, 1.0, 1/fs)
tau  = 0.1               # 100 ms rectangle
x    = (np.abs(t) < tau/2).astype(float)

# crude continuous-transform estimate: Riemann sum of x(t) e^{-j w t} dt
w    = 2*np.pi*np.linspace(-200, 200, 4001)        # rad/s grid
X    = (x[None, :] * np.exp(-1j*np.outer(w, t))).sum(axis=1) / fs

print("first null (predicted) =", 2*np.pi/tau, "rad/s")   # ~ 62.8
print("peak |X| vs tau:", np.abs(X).max(), "vs", tau)     # both ~ 0.1
```

The printed peak magnitude lands on $\tau = 0.1$ (the pulse area, as predicted by the DC rule), and the first null appears at $\omega = 2\pi/\tau \approx 62.8\,\mathrm{rad/s}$. The Riemann sum is exactly the analysis integral (7.5) discretized; the next chapter makes this precise with the discrete Fourier transform and the FFT.

Build it (hardware)

On a microcontroller you will never evaluate the continuous integral; you will sample the sensor at some rate f_s and run a discrete transform. But the continuous picture is still your design oracle. Before writing a line of firmware, sketch the *expected* continuous spectrum of the motion you care about: a wrist flick is roughly a single bump a few tens of milliseconds wide, so by the rect/sinc and Gaussian reasoning its energy lives mostly below $\sim 20\,\mathrm{Hz}$. That sketch tells you that a sampling rate of, say, $200\,\mathrm{Hz}$ is comfortably more than enough for the gross motion, and it tells you where to put the corner of your anti-alias and smoothing filters. The continuous transform is the map; the discrete transform is the vehicle.

Toward the wrist tracker

A word about what the spectrum can and cannot tell us. A single wrist-mounted IMU (accelerometer plus gyroscope) measures the gross motion of the forearm and hand as a rigid-ish body. Its Fourier spectrum cleanly separates a deliberate flick from hand tremor (a few Hz) from electrical hum, and that is genuinely useful for triggering discrete gestures. What it *cannot* do, no matter how cleverly you transform it, is recover the angle of an individual finger joint: those motions are largely invisible to a sensor strapped to the wrist. Continuous finger pose needs sensors *on* the fingers (instrumented rings or gloves); finger-level intent without rings needs surface EMG or force myography (FMG) on the forearm, which sense muscle activity rather than bulk motion. The Fourier transform is the right lens for all of these signals – but it cannot conjure information the sensor never captured. We keep this limit front and center as the build progresses; the chapters on EMG/FMG and on gesture recognition spell out the precise boundaries.

7.9 What to carry forward

Pause

The Fourier transform is the Fourier series with the period sent to infinity: the discrete harmonic ladder becomes a continuous spectrum $X(\mathrm{j}\omega)$. The forward transform projects $x(t)$ onto each eternal phasor $e^{\mathrm{j}\omega t}$ (an inner product over the whole line); the inverse reassembles x from those projections. Magnitude says how much of each frequency is present, phase says when it arrives, and for real signals the spectrum is conjugate-symmetric so only $\omega \geq 0$ carries new information. The rect/sinc pair encodes the master lesson – sharpness in one domain is spread in the other – which hardens into the uncertainty principle $\Delta t\, \Delta\omega \geq \frac{1}{2}$, saturated only by the Gaussian. Parseval guarantees energy is conserved across the two domains. Next we discretize all of this into the DFT, the version that actually runs on hardware.

7.10 Exercises

Exercise 7.1 (Area and DC value)**.** Without computing a full transform, state $X(\mathrm{j}0)$ for (a) a triangular pulse of base 2τ and peak height 1, and (b) $e^{-at}u(t)$ with $a > 0$. In each case explain why $X(\mathrm{j}0)$ equals the time-domain area.

Exercise 7.2 (Scaling and the reciprocity of widths)**.** Using the rect/sinc pair, find the transform of $\mathrm{rect}(t/(2\tau))$ and of $\mathrm{rect}(2t/\tau)$. Where is the first null in each? Confirm that doubling the pulse width halves the null frequency, and halving the width doubles it.

Exercise 7.3 (Time shift tilts the phase)**.** Starting from Definition 7.1, show that $\mathcal{F}\{x(t-t_0)\} = e^{-\mathrm{j}\omega t_0}X(\mathrm{j}\omega)$. Conclude that a pure delay leaves $|X(\mathrm{j}\omega)|$ unchanged and adds the linear phase $-\omega t_0$. Relate this to Figure 7.2.

Exercise 7.4 (Cosine as two lines)**.** Write $\cos(\omega_0 t) = \frac{1}{2}e^{\mathrm{j}\omega_0 t} + \frac{1}{2}e^{-\mathrm{j}\omega_0 t}$ and use $\mathcal{F}\{e^{\mathrm{j}\omega_0 t}\} = 2\pi\delta(\omega-\omega_0)$ to derive (7.16). Then do the same for $\sin(\omega_0 t)$ and explain why its impulses are imaginary while cosine's are real.

Exercise 7.5 (Decaying exponential, magnitude and phase)**.** For $x(t) = e^{-at}u(t)$, $a > 0$, you found $X(\mathrm{j}\omega) = 1/(a + \mathrm{j}\omega)$. Sketch $|X(\mathrm{j}\omega)|$ and $\angle X(\mathrm{j}\omega)$ for $\omega \geq 0$. At what ω does the magnitude fall to $1/\sqrt{2}$ of its DC value, and what is the phase there?

Exercise 7.6 (Parseval as a tool)**.** Use Parseval's theorem to evaluate $\int_{-\infty}^{\infty} \dfrac{\sin^2(\Omega t)}{(\pi t)^2}\,\mathrm{d}t$ by recognizing the integrand as $|x(t)|^2$ for the ideal low-pass sinc of (7.12). (Hint: its transform is a rectangle of height 1 and width 2Ω.)

Exercise 7.7 (The Gaussian saturates uncertainty)**.** For $x(t) = e^{-t^2/(2\sigma^2)}$ with transform (7.18), compute Δt and $\Delta\omega$ (the energy-density standard deviations). Show $\Delta t = \sigma/\sqrt{2}$, $\Delta\omega = 1/(\sigma\sqrt{2})$, and hence $\Delta t\,\Delta\omega = \tfrac{1}{2}$ exactly.

Exercise 7.8 (Sketch the uncertainty proof)**.** Fill in the Cauchy–Schwarz argument for Theorem 7.1: apply the inequality $|\int t\,x(t)\,\overline{x'(t)}\,\mathrm{d}t|^2 \leq \big(\int t^2|x|^2\big)\big(\int |x'|^2\big)$, use integration by parts on the left to get $-\tfrac{1}{2}\int |x|^2\,\mathrm{d}t$, and use $\int |x'(t)|^2\mathrm{d}t = \tfrac{1}{2\pi}\int \omega^2 |X(\mathrm{j}\omega)|^2\mathrm{d}\omega$ on the right. Assemble these into $\Delta t\,\Delta\omega \geq \tfrac{1}{2}$.

Exercise 7.9 (Numerical experiment)**.** Adapt Listing 7.1 to a Gaussian $x(t) = e^{-t^2/(2\sigma^2)}$. Verify numerically that the transform is again a Gaussian and that as you halve σ the spectral width doubles. Plot the product of the measured time and frequency widths and confirm it stays near the floor $\tfrac{1}{2}$.

Exercise 7.10 (Gating a real signal)**.** A 50 Hz cosine is observed only through a rectangular window of width $\tau = 40$ ms. Using the convolution-in-frequency idea (multiplication in time corresponds to convolution of spectra), describe qualitatively what happens to the two impulses of (7.16): where do they end up, and how wide does each become? What does this say about how cleanly you can read a tone's frequency from a short observation? Connect your answer to Example 7.1.

Chapter 8

The Discrete Fourier Transform

So far our Fourier story has lived in two slightly idealized worlds. In the Fourier series we took a signal that repeats forever and broke it into a *discrete* list of harmonics. In the Fourier transform we took a signal of finite energy that lives on the whole continuous line and broke it into a *continuous* smear of frequencies. Both are beautiful. Neither is what a computer can actually hold.

A computer holds a finite list of numbers. Your microphone gives you, say, 1024 samples. Your IMU gives you a block of 256 accelerometer readings. There is no "$t \to \infty$", there is no integral over all of $\mathbb{R}$, there is just a finite array. The Discrete Fourier Transform (the DFT) is the Fourier idea rebuilt *exactly* for that situation: finitely many samples in, finitely many frequency numbers out, no approximation hidden anywhere, computable with a finite number of multiplies and adds.

This is the version of Fourier you will actually run. Every spectrum analyzer, every audio plugin, every gesture-recognition front end ultimately calls a DFT (usually through its fast cousin, the FFT, which is the subject of the next chapter). So we are going to build it slowly and geometrically, see it as a matrix, understand exactly what each output number *means*, and meet the two things that surprise everyone the first time: the DFT secretly thinks your signal is periodic, and that secret causes *spectral leakage*.

Focus

The whole chapter in one sentence: the DFT takes N samples and reports how much of each of N specially chosen rotating phasors is hiding inside them — and it does this by taking inner products, which is just "projection" from the linear algebra book wearing a complex-valued coat.

8.1 From the continuous transform to a finite sum

Recall the (continuous-time) Fourier transform of a signal $x(t)$:

$$X(f) \;=\; \int_{-\infty}^{\infty} x(t)\, e^{-\mathrm{j}2\pi f t}\, \mathrm{d}t.$$

Geometrically this is one giant inner product: it slides a complex exponential $e^{-\mathrm{j}2\pi f t}$ (a unit phasor spinning at frequency f) against $x(t)$ and asks "how much do you two overlap?". We are going to discretize this idea in two steps, and watch each idealization fall away.

Step 1: we only have samples. Suppose we sampled $x(t)$ every T_s seconds, giving the sequence $x[n] = x(nT_s)$ for $n = 0, 1, \ldots, N-1$. The integral becomes a sum (this is just a Riemann sum, the bridge the calculus book makes precise):

$$X(f) \;\approx\; \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}2\pi f n T_s}\, T_s.$$

We will drop the constant T_s out front for now (it is just an overall scale and we will track scaling carefully later). What remains depends on f only through the product fT_s, so define the *normalized* angular frequency $\omega = 2\pi f T_s$ (radians per sample). Then

$$X(\omega) \;=\; \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}\omega n}.$$

This is the Discrete-Time Fourier Transform (DTFT). It eats a finite (or infinite) sequence and produces a function of a *continuous* variable ω. Better — but still not finite output. A computer cannot store a function of a continuous knob.

Step 2: we only want finitely many frequencies. Here is the clean move. We have N input numbers. Linear algebra tells us N numbers carry exactly N degrees of freedom, so we should ask for exactly N frequency numbers back — no more (that would be redundant) and no fewer (that would lose information). Which N frequencies? We pick them *evenly spaced* around the full circle of possible $\omega \in [0, 2\pi)$:

$$\omega_k \;=\; \frac{2\pi k}{N}, \qquad k = 0, 1, \ldots, N-1.$$

Sampling the DTFT at exactly these N points gives the DFT:

$$\boxed{X[k] \;=\; \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}2\pi k n/N}, \qquad k = 0, \ldots, N-1.}$$

Intuition

Read the formula as a question repeated N times. For each k we cook up a phasor that completes *exactly k full turns* over the N samples, spin it backwards against the data, add everything up, and the size of the result tells us how strongly that particular "k-turns-per-block" rhythm is present. $k = 0$ is no turning at all (the average / DC level). $k = 1$ is one slow turn across the whole block. $k = N - 1$ is the fastest turn the block can resolve.

History

The idea of sampling frequencies as well as time is old — Gauss used essentially the FFT algorithm in 1805 to interpolate the orbit of the asteroid Pallas, predating Fourier's own 1807 memoir. But the DFT as a workhorse only exploded after Cooley and Tukey published their fast algorithm in 1965, making an N-point transform cost about $N \log_2 N$ operations instead of N^2. Suddenly real-time spectra were cheap, and digital signal processing as an engineering discipline was born. We will earn that speedup ourselves in the next chapter; here we build the slow, fully-understood version first.

8.2 The twiddle factor and the roots of unity

The repeated quantity $e^{-\mathrm{j}2\pi/N}$ shows up everywhere, so it gets a name. Define the *twiddle factor*

$$W_N \;=\; e^{-\mathrm{j}2\pi/N}.$$

Then the DFT and its inverse (which we will justify in Section 8.4) read compactly:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad x[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]\, W_N^{-kn}.$$

Geometrically W_N is the point you land on after rotating clockwise by one N-th of a full circle. Its powers $W_N^0, W_N^1, \ldots, W_N^{N-1}$ are the Nth *roots of unity*: N points spread perfectly evenly around the unit circle. That even spreading is the secret engine of everything that follows.

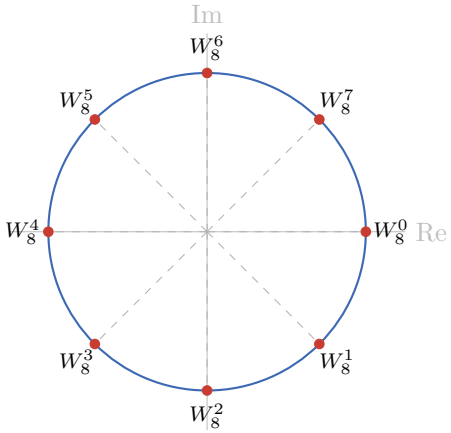


Figure 8.1: The eight roots of unity $W_8^k = e^{-\mathrm{j}2\pi k/8}$ for $N = 8$. The DFT "measures" a signal against phasors that sit at these evenly spaced angles. Note the clockwise numbering, because of the minus sign in the exponent.

Why it works

Why *evenly spaced* phasors and not some other set? Because evenly spaced roots of unity are *orthogonal*. Two of these sampled phasors, viewed as vectors in $\mathbb{C}^N$, have zero inner product unless they are the same one. That is precisely the property that made the Fourier series coefficients independent in the earlier chapter, and it is what lets us invert the DFT cleanly. Orthogonality is not a lucky accident; it is the reason this particular grid of frequencies is the right grid. We prove it in Section 8.4.

8.3 The DFT as a matrix

Here is the cleanest way to *see* the DFT, and the place where the linear algebra book pays off directly. The DFT is a linear map from $\mathbb{C}^N$ to $\mathbb{C}^N$, so it *is* a matrix. Stack the inputs into a vector $\mathbf{x} = (x[0], \ldots, x[N-1])^\top$ and the outputs into $\mathbf{X} = (X[0], \ldots, X[N-1])^\top$. Then

$$\mathbf{X} = \mathbf{F}_N\, \mathbf{x}, \qquad (\mathbf{F}_N)_{kn} = W_N^{kn} = e^{-\mathrm{j}2\pi kn/N}.$$

Row k of $\mathbf{F}_N$ is the conjugated phasor for frequency bin k, sampled at the N time points. Computing $X[k]$ is taking the dot product of that row with your data. *The DFT is matrix-vector multiplication*, full stop.

Let us write out $\mathbf{F}_4$ completely. With $N = 4$, $W_4 = e^{-\mathrm{j}\pi/2} = -\mathrm{j}$, so its powers cycle $W_4^0 = 1,\ W_4^1 = -\mathrm{j},\ W_4^2 = -1,\ W_4^3 = \mathrm{j}$, then repeat. The exponent is kn mod 4:

$$\mathbf{F}_4 = \begin{pmatrix} W^0 & W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 & W^3 \\ W^0 & W^2 & W^4 & W^6 \\ W^0 & W^3 & W^6 & W^9 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -\mathrm{j} & -1 & \mathrm{j} \\ 1 & -1 & 1 & -1 \\ 1 & \mathrm{j} & -1 & -\mathrm{j} \end{pmatrix}.$$

Intuition

Look at the rows as "test rhythms". Row 0 is all ones: it just sums the data, measuring the constant (DC) part. Row 2 alternates $1, -1, 1, -1$: it measures the fastest wiggle, the part that flips sign every sample (Nyquist). Rows 1 and 3 carry the genuine in-between rotations, and they are complex conjugates of each other — a fact that will become the symmetry of real-signal spectra.

Connection

This matrix view ties three books together. (1) Linear algebra: $\mathbf{F}_N$ is a change of basis. The standard basis describes a signal by "how much at each time sample"; $\mathbf{F}_N$ rewrites it as "how much at each frequency". Same vector, new coordinates. (2) The rows are orthogonal (next section), so $\mathbf{F}_N$ is — up to a scale — a *unitary* matrix, the complex cousin of a rotation. The DFT does not stretch or distort the signal's geometry, it just rotates your viewpoint. (3) Calculus: each entry is a sampled integral kernel, the Riemann-sum ghost of $e^{-\mathrm{j}2\pi ft}$.

8.4 Why it inverts: orthogonality of the rows

The single fact that makes the DFT useful is that you can get your signal *back*. Take the columns (equivalently rows, since $\mathbf{F}_N$ is symmetric) and check their inner products. Consider two frequency vectors $\mathbf{w}_k$ and $\mathbf{w}_m$ with entries $(\mathbf{w}_k)_n = W_N^{-kn}$ (the inverse phasors). Their Hermitian inner product is

$$\langle \mathbf{w}_k,\, \mathbf{w}_m \rangle = \sum_{n=0}^{N-1} W_N^{-kn}\, \overline{W_N^{-mn}} = \sum_{n=0}^{N-1} e^{\mathrm{j}2\pi(k-m)n/N}.$$

This is a finite geometric series in the ratio $r = e^{\mathrm{j}2\pi(k-m)/N}$. Two cases:

- If $k = m$, every term is $e^0 = 1$, so the sum is N.
- If $k \neq m$ (with $0 \leq k, m < N$), then $r \neq 1$ but $r^N = e^{\mathrm{j}2\pi(k-m)} = 1$, so $\sum_{n=0}^{N-1} r^n = \frac{r^N - 1}{r-1} = \frac{1-1}{r-1} = 0$.

Compactly,

$$\sum_{n=0}^{N-1} e^{\mathrm{j}2\pi(k-m)n/N} = N\,\delta[k-m],$$

where δ is 1 at zero and 0 otherwise. The frequency vectors are mutually orthogonal, each with squared length N.

Why it works

That single identity *is* the inverse DFT. Multiply the synthesis formula $x[n] = \frac{1}{N}\sum_k X[k]W_N^{-kn}$ through: substitute the analysis formula for $X[k]$, swap the order of summation, and the inner sum collapses to $N\,\delta$ by the identity above, leaving exactly $x[n]$. The factor $1/N$ is just the bookkeeping that undoes the squared-length N. Try this manipulation yourself in Exercise 8.1; it is three lines and it demystifies the whole transform.

In matrix language the same fact reads $\mathbf{F}_N^*\mathbf{F}_N = N\,\mathbf{I}$, where $\mathbf{F}_N^*$ is the conjugate transpose. Hence

$$\mathbf{F}_N^{-1} = \tfrac{1}{N}\,\mathbf{F}_N^*,$$

which is why the inverse DFT is just the forward DFT with the sign of the exponent flipped and a $1/N$ scale. Forward and inverse are nearly the same machine.

Common trap

Where does the $1/N$ go? Conventions differ and this bites everyone. Three common scalings: (a) $1/N$ on the inverse only (what we use, the default in NumPy's `fft`/`ifft` and MATLAB); (b) $1/\sqrt{N}$ on *both*, making $\mathbf{F}_N/\sqrt{N}$ exactly unitary and Parseval's theorem prettiest; (c) $1/N$ on the forward only (common in some physics texts). They differ only by an overall constant. Always check your library's convention before trusting absolute magnitudes, especially when you compute power.

8.5 Reading the output: bins, resolution, and units

You ran an N-point DFT on data sampled at rate $f_s = 1/T_s$. You got back N complex numbers $X[0], \ldots, X[N-1]$. What do they *mean* physically?

Each output is a bin. Output $X[k]$ corresponds to the analog frequency

$$f_k = \frac{k}{N}\,f_s = \frac{k f_s}{N} \quad \text{Hz} \qquad (k = 0, \ldots, N-1).$$

The bins are equally spaced by

$$\Delta f = \frac{f_s}{N} = \frac{1}{NT_s} = \frac{1}{\text{record length in seconds}}.$$

This Δf is the *frequency resolution*: the smallest frequency difference the transform can place in separate bins.

What is really happening

The resolution formula $\Delta f = 1/(NT_s)$ says something profound and a little sobering: **resolution is set by how long you watched, not by how fast you sampled**. NT_s is the total duration of your record. Watch for 1 second and your bins are 1 Hz apart, no matter whether

you sampled at 1 kHz or 1 MHz. To tell two close tones apart you must observe longer. Sampling faster only buys you a higher *top* frequency, not finer spacing. This is a baby version of the uncertainty principle: you cannot be simultaneously sharp in time (short record) and sharp in frequency (fine bins).

Magnitude and phase. Each $X[k]$ is complex; write $X[k] = |X[k]|\, e^{\mathrm{j}\angle X[k]}$. The magnitude $|X[k]|$ says *how much* of that frequency is present; the phase $\angle X[k]$ says *where* that sinusoid sits in time (its alignment). Most spectrum plots show magnitude (or power $|X[k]|^2$) because the eye reads "which frequencies are loud" most easily, but phase carries the timing and is essential for reconstruction — throw it away and you cannot get the signal back.

The upper half is a mirror (for real signals). If $x[n]$ is real — as all physical sensor data is — then

$$X[N-k] = \overline{X[k]}.$$

So the second half of the spectrum is the complex conjugate of the first half read backwards. It carries no new information. The genuinely distinct frequencies live in bins 0 through $N/2$. Bins above $N/2$ are the "negative frequencies" folded around; bin k for $k > N/2$ really represents $-(N-k)$ cycles, i.e. the same physical frequency rotating the other way.

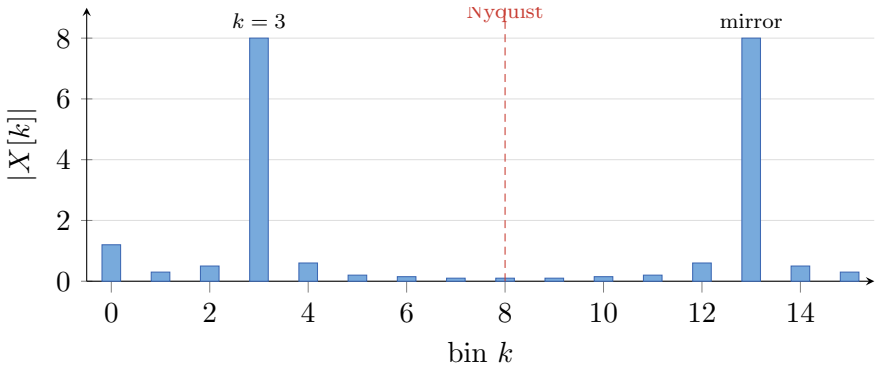


Figure 8.2: Magnitude spectrum of a real $N = 16$ signal containing a single tone at 3 cycles per block (plus a small DC offset). The peak at $k = 3$ is mirrored at $k = 13 = N - 3$. Everything to the right of the Nyquist bin $k = 8$ is redundant for real data. With $f_s = 16$ Hz this peak sits at $f_3 = 3 \cdot 16/16 = 3$ Hz.

Example 8.1 (Putting numbers on the axes)**.** You record an IMU axis at $f_s = 200$ Hz and collect $N = 256$ samples.

- Record length: $NT_s = 256/200 = 1.28$ s.
- Bin spacing: $\Delta f = 200/256 \approx 0.781$ Hz.
- Bin k sits at $f_k = 0.781\, k$ Hz. Bin 5 is at 3.9 Hz.
- Nyquist (bin 128) is at 100 Hz; bins 129–255 mirror bins 127–1.
- A 4 Hz hand tremor lands near bin $4 \,/\, 0.781 \approx 5.1$ — between bins 5 and 6, which is exactly the situation that causes leakage (Section 8.8).

8.6 Two worked DFTs by hand

Nothing builds intuition like grinding a small transform by hand. We do two.

Example 8.2 (A 4-point DFT of a real signal)**.** Let $x = (1, 2, 3, 4)$. Using $\mathbf{F}_4$ from Section 8.3,

$$\mathbf{X} = \mathbf{F}_4\mathbf{x} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -\mathrm{j} & -1 & \mathrm{j} \\ 1 & -1 & 1 & -1 \\ 1 & \mathrm{j} & -1 & -\mathrm{j} \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}.$$

Row by row:

$$\begin{aligned} X[0] &= 1 + 2 + 3 + 4 = 10, \\ X[1] &= 1 - 2\mathrm{j} - 3 + 4\mathrm{j} = -2 + 2\mathrm{j}, \\ X[2] &= 1 - 2 + 3 - 4 = -2, \\ X[3] &= 1 + 2\mathrm{j} - 3 - 4\mathrm{j} = -2 - 2\mathrm{j}. \end{aligned}$$

So $\mathbf{X} = (10,\ -2 + 2\mathrm{j},\ -2,\ -2 - 2\mathrm{j})$. Sanity checks: $X[0] = 10$ is the sum (four times the average 2.5). And $X[3] = \overline{X[1]}$, exactly the real-signal mirror symmetry $X[N-k] = \overline{X[k]}$. The magnitudes are $(10,\ 2\sqrt{2},\ 2,\ 2\sqrt{2})$.

Example 8.3 (Recovering a pure tone exactly)**.** Now take $N = 8$ and the sampled cosine $x[n] = \cos(2\pi \cdot 2 \cdot n/8) = \cos(\pi n/2)$, which completes exactly 2 cycles in the block. The samples are

$$x = (1,\ 0,\ -1,\ 0,\ 1,\ 0,\ -1,\ 0).$$

Because the tone fits a *whole* number of cycles in the window, the DFT is as clean as it ever gets. Using Euler, $\cos\theta = \frac{1}{2}e^{\mathrm{j}\theta} + \frac{1}{2}e^{-\mathrm{j}\theta}$, so the signal is half a phasor at $+2$ cycles and half a phasor at -2 cycles. The DFT therefore puts energy in exactly two bins:

$$X[2] = \tfrac{N}{2} = 4, \qquad X[6] = \tfrac{N}{2} = 4,$$

and $X[k] = 0$ for all other k. (Bin $6 = N - 2$ is the mirror, the -2-cycle component.) Verify $X[2]$ directly:

$$X[2] = \sum_{n=0}^{7} x[n]\, e^{-\mathrm{j}2\pi \cdot 2n/8} = \sum_{n} x[n]\, e^{-\mathrm{j}\pi n/2} = \sum_{n} x[n]\,(\cos\tfrac{\pi n}{2} - \mathrm{j}\sin\tfrac{\pi n}{2}).$$

The sin part hits zero wherever $x[n] \neq 0$, and the cos part reproduces $x[n]$ itself, so $X[2] = \sum_n x[n]^2 = 1 + 1 + 1 + 1 = 4$. A tone that fits the window perfectly produces a single sharp spike. Remember that phrase "fits the window perfectly" — the moment it does not, leakage appears.

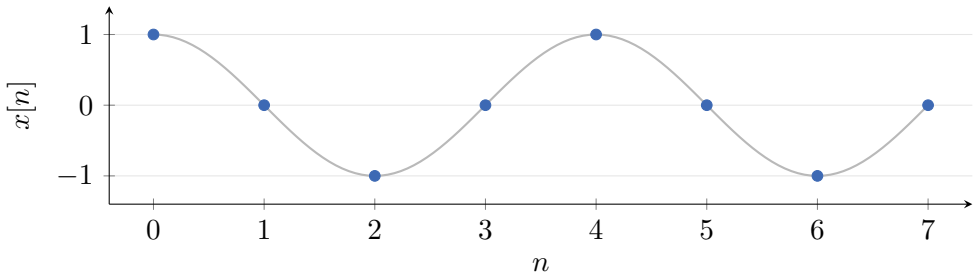


Figure 8.3: The eight samples of Example 8.3: $x[n] = \cos(\pi n/2)$, exactly 2 cycles across the $N = 8$ block (samples as dots, underlying cosine in gray). Because a whole number of cycles fits the window, the periodic extension closes without a seam and the whole tone lands in bins 2 and 6.

Try it yourself

Reproduce both examples in three lines of NumPy and confirm by eye:

```
import numpy as np
print(np.fft.fft([1,2,3,4]))              # -> [10, -2+2j, -2, -2-2j]
n = np.arange(8)
print(np.round(np.fft.fft(np.cos(np.pi*n/2)).real, 3))  # spike at 2 and 6
```

Then break it on purpose: change the tone to 2.5 cycles per block (`np.cos(2*np.pi*2.5*n/8)`) and watch the single spike smear across many bins. That smear is the subject of the next section.

8.7 The hidden assumption: the DFT is circular

Here is the most important conceptual point in the chapter, and the one most likely to surprise you. The DFT does not see your N samples as a finite snippet sitting in silence. It sees them as *one period of an infinitely repeating signal*. The block wraps around: sample N would be sample 0 again.

Why? Look at the basis phasors $e^{-\mathrm{j}2\pi kn/N}$. Every one of them satisfies $e^{-\mathrm{j}2\pi k(n+N)/N} = e^{-\mathrm{j}2\pi kn/N}$ — they are all periodic with period N. If your toolbox is built entirely from period-N pieces, then *anything* you build is period-N. The DFT literally cannot represent a signal that is not periodic in N; it represents the periodic extension of your data whether you wanted that or not.

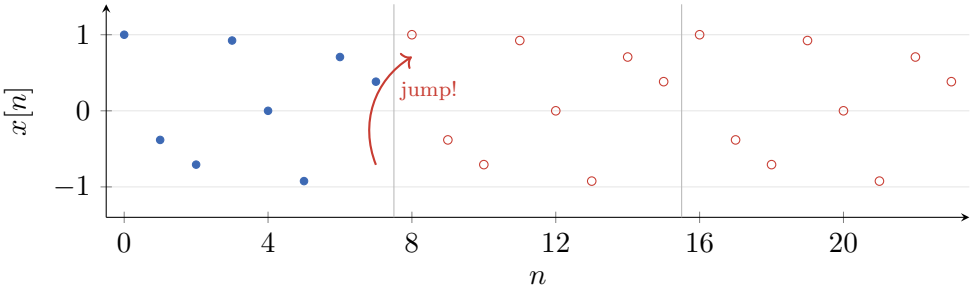


Figure 8.4: The DFT's view of a 2.5-cycle tone in an 8-sample window: it tiles the block end to end (filled dots = your data, open dots = the implied repeats). Since 2.5 cycles do not close smoothly, a discontinuity appears at every seam (red arrow). The DFT must spend energy across many bins to manufacture that corner — that is leakage.

What is really happening

This circularity is not a bug, it is the exact mathematical content of the DFT. Everything "circular" in DSP flows from it: circular convolution (the DFT multiplies spectra, which in time means wrap-around convolution, not the linear convolution you usually want), circular shifts, the mirror symmetry of real spectra, and leakage. Hold onto the mental image of your data *taped into a ring*. Many later confusions evaporate the moment you remember the DFT lives on a circle, not a line.

8.8 Spectral leakage

Now we can explain the smear you saw in the lab box. When a tone completes a *whole* number of cycles in the window (Example 8.3), the periodic extension is seamless: the ring closes perfectly, the tone is genuinely one of the basis phasors, and all its energy lands in one bin. This is called an *on-bin* or *coherent* frequency.

But a real 4 Hz tremor sampled into a 1.28 s window (Example 8.1) sits at bin 5.1 — *between* bins. Its periodic extension has a jump at every seam (Figure 8.4). A jump is sharp, and sharp features need many frequencies to build, so the DFT spreads the tone's energy across *all* the bins, strongest near 5 and 6 but with a slowly decaying skirt everywhere. That spreading is **spectral leakage**: energy "leaks" out of the true frequency into neighboring bins.

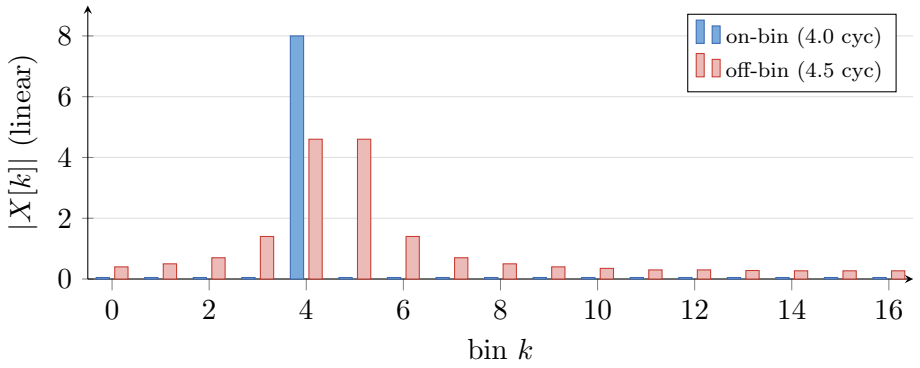


Figure 8.5: On-bin versus off-bin tones, $N = 32$ (first half shown). The on-bin tone (blue, exactly 4 cycles per block) lands in a single bin. The off-bin tone (red, 4.5 cycles) splits between bins 4 and 5 and leaks a decaying skirt across every other bin. The peak is also lower — energy that scattered is energy not in the main bin (this height loss is called *scalloping*).

Why it works

There is a sharper way to see leakage that connects straight to the continuous transform of Chapter 7. Taking only N samples is the same as multiplying an infinitely long signal by a *rectangular window* (1 inside the block, 0 outside). Multiplication in time is convolution in frequency. The Fourier transform of a rectangle is a sinc, so the true line spectrum of a pure tone gets *convolved with a sinc*. When the tone is on-bin, the DFT samples that sinc exactly at its zero crossings except for the one peak, so you see a clean spike. When the tone is off-bin, the DFT samples the sinc *between* its zeros and you see the whole ringing shape. Leakage is the sinc's side-lobes, sampled. We will make this precise when we study windowing and the convolution theorem.

Taming leakage: windowing (a preview)

The rectangular window's side-lobes fall off slowly (only ~ 13 dB down from the peak). The fix is to multiply your N samples by a smooth *window* function $w[n]$ that tapers gently to zero at both ends *before* taking the DFT. A tapered block has no jump at the seam, so the periodic extension is smooth, so far less energy leaks. The price: the smooth taper widens the main lobe, so you trade

resolution for lower leakage. You cannot escape the trade-off, only choose where to sit on it.

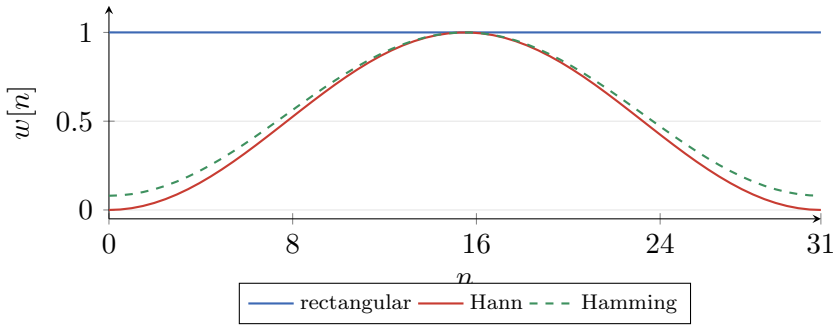


Figure 8.6: Three windows over $N = 32$ samples. The rectangular window (do nothing) keeps full amplitude but jumps at the edges; the Hann and Hamming windows taper to (near) zero, smoothing the seam and suppressing leakage at the cost of a wider main lobe. Chapter 21 develops the full menu and their trade-offs.

Common trap

Do not confuse *zero-padding* with better resolution. Appending zeros to your block before the DFT (say from $N = 256$ to $N = 1024$) gives you more bins, so the spectrum looks smoother and you can read peak locations more finely. But it adds *no new information*: the underlying record length NT_s that sets Δf did not change, you just interpolated the same DTFT more densely. Zero-padding *interpolates*; only observing longer truly *resolves*.

8.9 Parseval: energy is preserved

Because $\mathbf{F}_N/\sqrt{N}$ is unitary, the DFT preserves length (energy) up to the scale factor. Concretely,

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

The total energy in the time samples equals the total energy across the bins (divided by N). This is the discrete Parseval / Rayleigh identity, and it is the same statement as "a rotation does not change a vector's length", now in $\mathbb{C}^N$.

Example 8.4 (Check Parseval on the 4-point example)**.** From Example 8.2, the time energy is $1^2+2^2+3^2+4^2 = 30$. The spectral energy is $|10|^2+|-2+2\mathrm{j}|^2+|-2|^2+|-2-2\mathrm{j}|^2 = 100+8+4+8 = 120$, and $120/N = 120/4 = 30$. They match.

Connection

Parseval is why we can compute signal *power* in the frequency domain — crucial later for spectral estimation and for the noise/SNR chapter. If a sensor adds white noise of variance σ^2 per sample, that energy spreads evenly across all N bins, so each bin carries an expected σ^2 of noise power while a coherent tone piles all its energy into one bin. That contrast — noise flat, signal concentrated — is the entire reason the DFT is such an effective detector. We will

quantify the resulting "processing gain" in the probability / random-signals chapter.

8.10 Cost, and a look ahead to the FFT

The matrix $\mathbf{F}_N$ is $N \times N$ and dense, so computing $\mathbf{X} = \mathbf{F}_N\mathbf{x}$ directly costs about N^2 complex multiply-adds. For $N = 1024$ that is roughly a million operations *per transform*; for the real-time audio rate of dozens of transforms per second it quickly becomes a problem on a small microcontroller.

The next chapter shows that $\mathbf{F}_N$ has enormous hidden structure — the twiddle factors repeat in a recursive pattern — which lets us factor the matrix into a product of very sparse matrices and slash the cost to $\sim N \log_2 N$. That is the Fast Fourier Transform. Everything you understood here about bins, leakage, and circularity carries over unchanged; the FFT computes *exactly* the same numbers, only faster.

Toward the wrist tracker

For the wrist gesture controller, the DFT is the front-end that turns raw motion into features. A practical pipeline: sample the IMU at $f_s = 200\,\text{Hz}$, buffer $N = 256$ samples (1.28 s), apply a Hann window, take the DFT of each axis, and read off band energies — e.g. how much power sits in 1–5 Hz (deliberate gesture motion) versus 8–12 Hz (physiological tremor) versus the near-DC bin (gravity / orientation). Those band energies are compact, robust features for a gesture classifier (Figure 8.7).

The limits, stated plainly. A wrist-worn IMU alone senses the *gross* motion of the whole forearm and hand. Its spectrum can reliably separate coarse, discrete gestures (a flick, a twist, a tap, "holding still") and tell rhythmic motion from static pose. It *cannot* recover individual finger joint angles — the fingers barely move the wrist, so their signal is essentially absent from a wrist IMU. Continuous finger pose needs sensors on the fingers (instrumented rings or gloves); finger-level *activation* needs forearm EMG or force myography (FMG), which read muscle electrical or mechanical activity. The DFT will be just as central there — EMG band power is a standard feature — but be clear-eyed about what each sensor can and cannot deliver. Promising "per-finger MIDI" from a lone wrist IMU is a promise the physics will not keep.

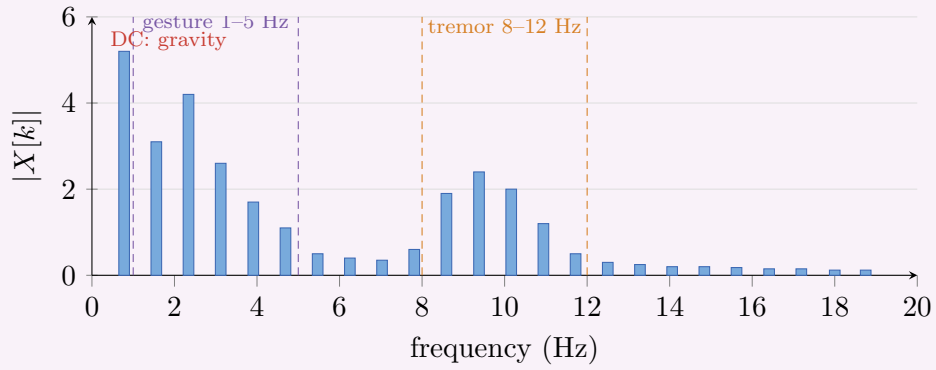


Figure 8.7: Sketch of a windowed wrist-IMU magnitude spectrum during a slow gesture ($f_s = 200\,\text{Hz}$, $N = 256$, so $\Delta f \approx 0.78\,\text{Hz}$). Summing $|X[k]|^2$ over the 1–5 Hz band captures deliberate motion; the 8–12 Hz band tracks physiological tremor; the near-DC bin carries the gravity vector. These three band energies make compact gesture features.

Build it (hardware)

A minimal fixed-block DFT loop in C, suitable as a reference before you drop in a real FFT library (CMSIS-DSP on Cortex-M, or KissFFT on a Pi). Keep N a power of two so swapping in an FFT later is painless.

```c
#include <math.h>
#define N 256
// in[] = windowed real samples; out_mag[] = magnitude of bins 0..N/2
void dft_mag(const float in[N], float out_mag[N/2 + 1]) {
    for (int k = 0; k <= N/2; ++k) {      // real signal: half the bins suffice
        float re = 0.f, im = 0.f;
        for (int n = 0; n < N; ++n) {
            float ang = -2.f * (float)M_PI * k * n / N;
            re += in[n] * cosf(ang);
            im += in[n] * sinf(ang);
        }
        out_mag[k] = sqrtf(re*re + im*im);
    }
}
```

This is $O(N^2)$ and recomputes cosines in the inner loop — fine for understanding and for tiny N, far too slow for $N = 256$ at 200 Hz on an 8-bit AVR. Precompute a twiddle table, or just use an FFT. The point of writing it once by hand is so the FFT is never a black box.

Why this matters

Step back and admire what just happened. We started with an integral over all of time against a continuum of frequencies — an object no computer can touch — and by sampling *both* time and frequency we landed on a single finite matrix that is, up to scale, a rotation of $\mathbb{C}^N$. The same orthogonality that powered Fourier series, the same projection idea from linear algebra, the same time/frequency uncertainty from calculus, all reappear in finite, computable form.

The DFT is where the grand Fourier theory becomes something you can run on a chip strapped to your wrist. That is a remarkable amount of unity in one $N \times N$ array.

8.11 Exercises

Exercise 8.1. Prove the inverse DFT. Start from $y[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]e^{\mathrm{j}2\pi kn/N}$, substitute $X[k] = \sum_m x[m]e^{-\mathrm{j}2\pi km/N}$, swap the order of the two sums, and use the orthogonality identity $\sum_k e^{\mathrm{j}2\pi k(n-m)/N} = N\,\delta[n-m]$ to show $y[n] = x[n]$. Identify exactly where the factor $1/N$ is consumed.

Exercise 8.2. Compute the 4-point DFT of $x = (1, 0, 0, 0)$ (a unit impulse) and of $x = (1, 1, 1, 1)$ (a constant). Explain in one sentence each why one spectrum is flat and the other is a single spike. What general rule about "narrow in time vs. broad in frequency" do these two illustrate?

Exercise 8.3. By hand, find the 8-point DFT of $x[n] = \cos(2\pi \cdot 3n/8)$. Which two bins are nonzero and what are their values? Then do the same for $x[n] = \sin(2\pi \cdot 3n/8)$ and explain why the nonzero values are now imaginary, connecting the result to the phase of the tone.

Exercise 8.4. You sample at $f_s = 1000\,\mathrm{Hz}$ and want to distinguish two tones 2 Hz apart. (a) What is the minimum record length in seconds? (b) The minimum number of samples N? (c) If instead you only had $N = 256$ samples at this rate, what is your bin spacing, and could you separate the two tones? Explain using $\Delta f = 1/(NT_s)$.

Exercise 8.5. A tone of 4.5 cycles per $N = 16$ block is fed to a DFT. (a) Sketch the periodic extension and mark the discontinuity at the seam. (b) Explain why no single bin captures it. (c) Will multiplying by a Hann window before the DFT increase or decrease the leaked side-lobes, and what does it cost you? Answer in terms of the seam discontinuity.

Exercise 8.6. Confirm Parseval's identity for the 8-point tone $x = (1, 0, -1, 0, 1, 0, -1, 0)$ of Example 8.3. Compute $\sum_n x[n]^2$ directly, compute $\frac{1}{N}\sum_k |X[k]|^2$ from the two nonzero bins $X[2] = X[6] = 4$, and show they agree.

Exercise 8.7. Show that for any real input $x[n]$, the DFT satisfies $X[N-k] = \overline{X[k]}$. (Hint: write out $X[N-k]$, use $e^{-\mathrm{j}2\pi(N-k)n/N} = e^{\mathrm{j}2\pi kn/N}$, and recognize the conjugate.) Conclude that an N-point DFT of real data has only $N/2 + 1$ independent complex values, and check that the real-number count balances.

Exercise 8.8. Zero-padding. Take $x = (1, 2, 3, 4)$ and append four zeros to make an 8-point input. Without grinding all eight sums, argue what zero-padding does to the *bin spacing* and to the *frequency resolution* (are they the same thing?). Then in NumPy compare `np.fft.fft(x)` with `np.fft.fft(x, 8)` and describe how the second interpolates the first.

Exercise 8.9. Write out the $\mathbf{F}_8$ matrix's row $k = 4$ explicitly and explain why it is the "Nyquist row". What time-domain signal does it perfectly match, and why is it the fastest oscillation an 8-sample block can represent?

Exercise 8.10. (IMU mini-project, conceptual.) You buffer $N = 256$ samples of one accelerometer axis at $f_s = 200\,\mathrm{Hz}$ and window with a Hann window. (a) Give the formula for the center frequency of bin k. (b) Which bins cover the 1–5 Hz "gesture" band? (c) Explain in two or three sentences why summing $|X[k]|^2$ over that band is a more robust gesture feature than any single bin's magnitude. (d) State one thing this wrist-IMU spectrum *cannot* tell you about the hand, and which sensor you would add to get it.

Exercise 8.11. (Matrix structure.) Verify numerically that $\mathbf{F}_4^*\mathbf{F}_4 = 4\mathbf{I}$ by computing two off-diagonal inner products of rows and showing they vanish, plus one diagonal inner product showing it equals 4. Relate this to why $\mathbf{F}_4^{-1} = \frac{1}{4}\mathbf{F}_4^*$.

Exercise 8.12. (Leakage experiment.) In NumPy, build $x[n] = \cos(2\pi f_0 n/f_s)$ for $f_s = 256$, $N = 256$, and sweep f_0 from 10.0 to 10.5 Hz in steps of 0.1. For each, plot $|X[k]|$ near bin 10. Describe how the single spike turns into a two-bin split and a skirt as f_0 moves off-bin, and confirm the peak height drops (scalloping loss).

Chapter 9

The Fast Fourier Transform

In the last chapter we built the Discrete Fourier Transform (DFT): the machine that takes N samples of a signal and tells you, for each of N frequencies, how much of that frequency is present and in what phase. Geometrically, the DFT projects your signal vector onto N rotating phasors and reads off the coordinates. It is the right idea. The problem is that, done the obvious way, it is *slow* – so slow that for a long time people simply did not compute spectra of real signals at all.

Then, in 1965, two researchers published a four-page note describing an algorithm that computed the very same numbers, exactly, but vastly faster. It did not approximate the DFT; it reorganized the arithmetic so that enormous amounts of redundant work collapsed. Spectra that would have taken a day took a second. Whole fields – radar, seismology, medical imaging, digital audio, wireless – were rebuilt on top of it. The algorithm is the *Fast Fourier Transform* (FFT), and it is one of the most consequential pieces of arithmetic ever written down.

This chapter is about *why* the FFT is fast and *how* it works, built from the geometry up. We will see exactly where the wasted work in the naive DFT hides, watch divide-and-conquer expose it, draw the famous butterfly, understand the strange bit-reversal shuffle, and then – crucially – learn to actually *use* an FFT: zero-padding, reading the output of a real-valued FFT, and what the numbers mean. By the end you will both understand the FFT and be able to call one correctly, which matters because our wrist gesture controller will run an FFT on a tiny microcontroller hundreds of times per second.

9.1 Recalling the DFT, and counting its cost

Let $x[0], x[1], \ldots, x[N-1]$ be our N samples. The DFT produces N complex numbers $X[0], \ldots, X[N-1]$ by

$$X[k] \;=\; \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}\,2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1. \tag{9.1}$$

Each $X[k]$ asks: "how much does the signal align with the phasor that completes exactly k whole turns over the window?" The factor $e^{-\mathrm{j}2\pi kn/N}$ is that phasor, sampled at instant n.

That exponential appears so often that it earns a name. Define the **twiddle factor**

$$W_N \;=\; e^{-\mathrm{j}\,2\pi/N}, \qquad \text{so that} \qquad X[k] \;=\; \sum_{n=0}^{N-1} x[n]\, W_N^{kn}. \tag{9.2}$$

W_N is a single small clockwise rotation: it is the Nth root of unity sitting just below the positive real axis on the unit circle. All the phasors in the DFT are just powers of this one number. That is the secret the FFT will exploit, but first let us see the cost of ignoring it.

Intuition

The DFT is a matrix-vector product. Stack the outputs into a vector $\mathbf{X}$ and the inputs into $\mathbf{x}$; then $\mathbf{X} = \mathbf{F}\,\mathbf{x}$, where the $N \times N$ matrix $\mathbf{F}$ has entries $F_{kn} = W_N^{kn}$. Every row of $\mathbf{F}$ is one phasor; multiplying by $\mathbf{F}$ measures the overlap of $\mathbf{x}$ with each. A dense $N \times N$ matrix times a vector takes about N^2 multiply-add operations. That single sentence is the entire performance problem.

Focus

The naive DFT is $O(N^2)$. Each of the N outputs $X[k]$ is a sum over N inputs, so it needs N complex multiplications and $N-1$ complex additions. With N outputs that is N^2 complex multiplies in total. Double N and the work *quadruples*.

To feel why $O(N^2)$ hurts, put numbers on it. A complex multiply is a handful of real multiplies and adds; call the whole DFT "N^2 operations" for scale.

N	N^2 (naive)	$N \log_2 N$ (FFT)	speedup
64	4,096	384	$\approx 11\times$
1,024	1,048,576	10,240	$\approx 102\times$
65,536	$\approx 4.3 \times 10^9$	$\approx 1.0 \times 10^6$	$\approx 4096\times$
10^6	10^{12}	$\approx 2 \times 10^7$	$\approx 50000\times$

A one-million-point spectrum is 10^{12} operations the naive way – minutes to hours on a real machine – versus 2×10^7 with the FFT, a small fraction of a second. The gap is not a constant factor you can shrug off; it grows without bound. Figure 9.1 shows the two curves pulling apart.

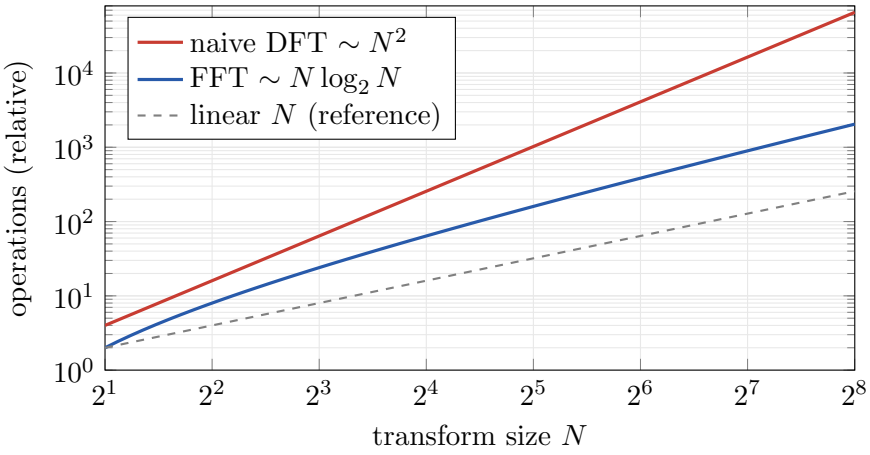


Figure 9.1: Operation count versus N on log–log axes. The naive DFT (red) climbs as N^2; the FFT (blue) hugs the nearly linear $N \log_2 N$ curve. The vertical gap *between* them keeps widening, which is why the FFT is not a minor optimization but a change in what is computable.

Connection

This is exactly the divide-and-conquer story from the algorithms companion. Sorting went from $O(N^2)$ (insertion/selection sort) to $O(N \log N)$ (mergesort) by splitting the problem in half, solving the halves, and stitching the results together cheaply. The FFT is the same recursion applied to the DFT: split, recurse, combine. If you have met the Master Theorem, the FFT satisfies $T(N) = 2\,T(N/2) + O(N)$, whose solution is $O(N \log N)$. The whole intellectual content of this chapter is finding the cheap "combine" step.

9.2 Where the redundancy hides

Before we can remove wasted work we have to *see* it. The naive DFT recomputes the same phasor values over and over and never reuses partial sums between different outputs. Two structural facts about W_N are the key.

Why it works

Two symmetries of the twiddle factor. Because $W_N = e^{-\mathrm{j}2\pi/N}$ lives on the unit circle:

- *Periodicity:* $W_N^{k+N} = W_N^k$. Going one extra full turn returns you to the same point. So there are really only N distinct phasor values, even though the index kn in (9.2) ranges up to $(N-1)^2$.
- *Half-turn (the crucial one):* $W_N^{k+N/2} = -W_N^k$, because half a revolution is a sign flip: $e^{-\mathrm{j}\pi} = -1$. And $W_N^2 = W_{N/2}$: skipping every other point on an N-circle lands you exactly on an $(N/2)$-circle.

These tiny identities are the entire engine of the FFT. The first says values repeat; the second says a size-N problem secretly *contains* a size-$N/2$ problem. Figure 9.2 shows the half-turn symmetry on the unit circle.

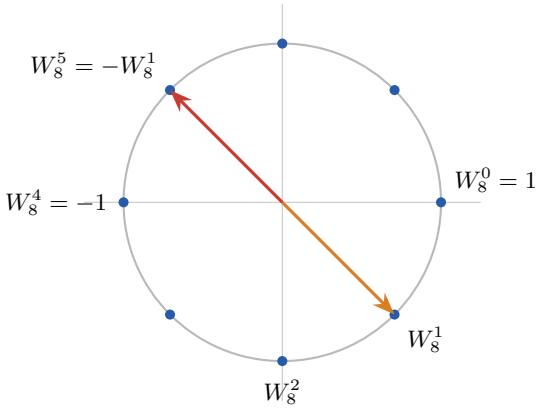


Figure 9.2: The eight twiddle factors $W_8^k = e^{-\mathrm{j}2\pi k/8}$ are equally spaced on the unit circle, stepping clockwise. A power and its half-turn partner sit diametrically opposite: $W_8^5 = -W_8^1$ (orange and red). That single sign flip is the half-turn identity $W_N^{k+N/2} = -W_N^k$ that lets one twiddle multiply produce two outputs in the butterfly.

The naive computation throws all this away. When it computes $X[k]$ and then $X[k+N/2]$, it grinds through two full independent sums even though, term by term, their phasors differ only by a sign. Multiply that wasted symmetry across all N outputs and you have found the factor that the FFT recovers.

9.3 Divide and conquer: split by even and odd

Assume N is a power of two (we will return to other sizes). The Cooley–Tukey idea is to split the *input* samples into two interleaved halves – the even-indexed samples and the odd-indexed samples – compute a DFT of each half, and then combine. Let

$$e[m] = x[2m], \qquad o[m] = x[2m+1], \qquad m = 0, \ldots, \tfrac{N}{2} - 1. \tag{9.3}$$

Now split the sum in (9.2) into its even and odd terms:

$$\begin{aligned} X[k] &= \sum_{n \text{ even}} x[n]\, W_N^{kn} \;+\; \sum_{n \text{ odd}} x[n]\, W_N^{kn} \\ &= \sum_{m=0}^{N/2-1} x[2m]\, W_N^{k(2m)} \;+\; \sum_{m=0}^{N/2-1} x[2m+1]\, W_N^{k(2m+1)}. \end{aligned} \tag{9.4}$$

Look at the exponents. In the even sum, $W_N^{k(2m)} = (W_N^2)^{km} = W_{N/2}^{km}$ – the size-N twiddle became a size-$N/2$ twiddle, exactly by the identity $W_N^2 = W_{N/2}$. In the odd sum, pull one factor of W_N^k out front: $W_N^{k(2m+1)} = W_N^k\, W_{N/2}^{km}$. Substituting,

$$X[k] \;=\; \underbrace{\sum_{m=0}^{N/2-1} e[m]\, W_{N/2}^{km}}_{E[k]} \;+\; W_N^k \underbrace{\sum_{m=0}^{N/2-1} o[m]\, W_{N/2}^{km}}_{O[k]}. \tag{9.5}$$

What is really happening

Read (9.5) slowly: $E[k]$ is *the DFT of the even samples* and $O[k]$ is *the DFT of the odd samples*, each a transform of length $N/2$. We have rewritten one size-N DFT in terms of two size-$N/2$ DFTs plus a single twiddle multiply W_N^k on the odd half. That is the divide-and-conquer split: a big projection expressed as two small projections, glued by a rotation.

There is a catch and it is a gift. $E[k]$ and $O[k]$ are defined for $k = 0, \ldots, N/2 - 1$ only, because they are length-$N/2$ transforms. But $X[k]$ needs k all the way to $N - 1$. How do we get the top half of the output from half-length transforms? The half-turn identity. For the top half, write $k + N/2$ with k in the lower range:

$$X[k] = E[k] + W_N^k \, O[k], \tag{9.6}$$

$$X[k + N/2] = E[k] - W_N^k \, O[k], \qquad k = 0, \ldots, \tfrac{N}{2} - 1. \tag{9.7}$$

The minus sign in (9.7) is the half-turn $W_N^{k+N/2} = -W_N^k$ doing its work; and E, O are periodic with period $N/2$, so $E[k + N/2] = E[k]$ and likewise for O. *Two outputs come out of one twiddle multiply.* The product $W_N^k \, O[k]$ is computed once and then added and subtracted. Half the multiplications just vanished – and that is only the top level of the recursion.

Walking question

Why does splitting the input "by even and odd" work so cleanly, while splitting it into "first half / second half" does not? Because the DFT couples each output to every input through the *product* kn in the exponent. Striding by 2 in n turns W_N^2 into $W_{N/2}$ – a clean change of scale on the unit circle. Splitting by contiguous blocks does not produce that scaling; it produces a phase shift, which leads to a different (and also valid) factorization called decimation-in-frequency. Even/odd is decimation-in-time, the version drawn most often.

9.4 The butterfly

Equations (9.6)–(9.7) describe a tiny two-input, two-output computation that is the atom of every FFT. It is called a *butterfly* because of how the data-flow diagram looks. You feed it $E[k]$ and $O[k]$; you multiply the odd one by the twiddle W_N^k; then you form a sum and a difference.

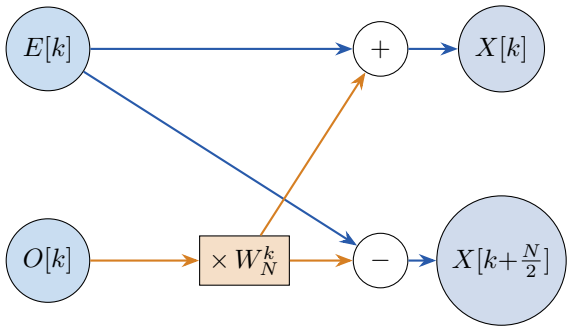


Figure 9.3: The radix-2 decimation-in-time butterfly. The odd input is rotated by the twiddle W_N^k; the result is added to the even input to make the lower output and subtracted to make the upper output. One complex multiply, two complex adds, two outputs. The whole FFT is thousands of these wired together.

Focus

Cost of one butterfly: exactly one complex multiply ($W_N^k\, O[k]$) and two complex adds. There are $N/2$ butterflies at each stage and $\log_2 N$ stages, giving $\frac{N}{2}\log_2 N$ complex multiplies overall – the $O(N \log N)$ promise made concrete.

Stacking the stages

We do not stop after one split. Each half-length DFT is itself a DFT of a power-of-two size, so we split it again, and again, until the transforms have length 1. A length-1 DFT is the identity – a single sample is its own spectrum, since the sum in (9.1) has one term and the phasor is $W_1^0 = 1$. So the recursion bottoms out for free.

Counting the levels: from N down to 1 by repeated halving takes $\log_2 N$ steps. Each level does $O(N)$ work in butterflies. That is the recurrence $T(N) = 2T(N/2) + O(N)$ once more, total $O(N \log N)$. Figure 9.4 shows the data flow for $N = 8$: three stages of butterflies turning eight inputs into eight outputs.

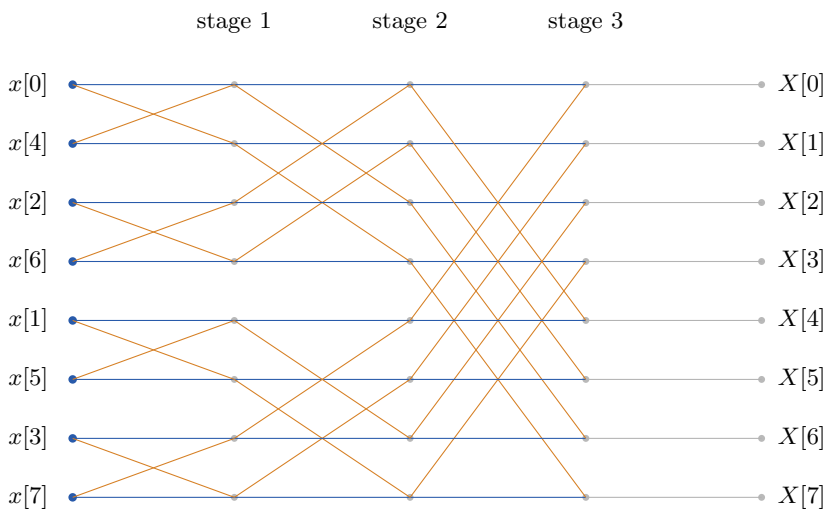


Figure 9.4: An 8-point radix-2 FFT, decimation-in-time. Inputs enter on the left in *bit-reversed* order $(0, 4, 2, 6, 1, 5, 3, 7)$; three stages of butterflies (orange = cross term carrying a twiddle, blue = straight term) produce the outputs in natural order on the right. Each vertical pair of crossing lines is one butterfly from Figure 9.3.

9.5 Bit-reversal: why the inputs come in scrambled

You may have noticed the inputs in Figure 9.4 are not in order; they are $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$. This is not a typo, and it is not optional. It falls directly out of the even/odd splitting, and understanding it removes all the mystery.

Track what the recursion does to the indices. The first split sends even indices $\{0, 2, 4, 6\}$ to one group and odd $\{1, 3, 5, 7\}$ to the other. Within "even" we split again by even/odd *of the half-index*: $\{0, 4\}$ and $\{2, 6\}$. One more split isolates each element. Follow any index through these choices and you are reading its binary digits from the least significant bit upward:

index	binary	bits reversed	new position
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

What is really happening

Bit-reversal in one sentence: repeatedly sorting by "even or odd" is the same as sorting by the bits of the index from least-significant to most-significant, and that is the same as

reversing the bits. Sample $x[n]$ must start in the slot whose index is n with its binary digits reversed. For $n = 1 = 001_2$ the reverse is $100_2 = 4$, so $x[1]$ goes to slot 4 – exactly what the diagram shows.

This gives the standard *in-place* FFT a clean shape: first permute the array by bit-reversal, then run $\log_2 N$ stages of butterflies that read and write the same array. No extra storage for the data, and the outputs emerge in natural frequency order. (Some libraries instead take natural-order input and produce bit-reversed output; the permutation has to live somewhere.)

Common trap

Bit-reversal only produces a clean reshuffle when N is a power of two. For other sizes, N is factored differently (a length-12 transform might split as 3×4), the "digit-reversal" is in a mixed radix, and the butterflies are larger. Good libraries (FFTW, the FFT in NumPy/SciPy, KissFFT, CMSIS-DSP) handle all this; you almost never write bit-reversal yourself. But knowing it exists explains why FFT speed and even *accuracy* can depend sharply on whether N is 1024 versus 1023 or 1000.

History

The 1965 paper by James **Cooley** (IBM) and John **Tukey** (Princeton/Bell Labs), "An algorithm for the machine calculation of complex Fourier series," set off the explosion – partly because it arrived just as digital computers became widespread, and partly, in the Cold War context, because fast spectra mattered for detecting nuclear tests from seismic data. But the core idea is far older. Carl Friedrich **Gauss** wrote down an equivalent doubling trick around **1805** – before Fourier's own 1807 work on the series that bear his name – to interpolate the orbits of the asteroids Pallas and Juno. Gauss's note was in neglected Latin and unpublished in his lifetime; the method was reinvented in pieces by Runge, Danielson, Lanczos and others over the next 150 years. The lesson the field took to heart: the FFT was "the most important numerical algorithm of our lifetime" (Gilbert Strang), and it had been hiding in plain sight for a century and a half. See Cooley & Tukey, *Math. Comp.* **19** (1965), and Heideman, Johnson & Burrus, "Gauss and the history of the FFT" (1984).

9.6 It computes the same numbers – a small proof and a check

The FFT is *exact*: up to floating-point rounding it returns the identical $X[k]$ as (9.1). We already proved the inductive step – (9.5) and the butterfly (9.6)–(9.7) are algebraic identities, not approximations – and the base case (length-1 DFT is the identity) is immediate. By induction on $\log_2 N$, the recursion computes the DFT for every power-of-two N. Let us verify by hand on the smallest interesting case.

Example 9.1 (A 4-point FFT by hand)**.** Take $x = (1, 2, 0, 1)$, so $N = 4$ and $W_4 = e^{-\mathrm{j}\pi/2} = -\mathrm{j}$. The powers we need are $W_4^0 = 1$, $W_4^1 = -\mathrm{j}$.

Split. Even samples $e = (x[0], x[2]) = (1, 0)$; odd samples $o = (x[1], x[3]) = (2, 1)$.

Two length-2 DFTs (a length-2 DFT is just sum and difference, since $W_2 = -1$):

$$E = (\, e[0] + e[1],\ e[0] - e[1]\,) = (1,\ 1), \qquad O = (\, o[0] + o[1],\ o[0] - o[1]\,) = (3,\ 1).$$

Combine with butterflies, using $W_4^0 = 1$, $W_4^1 = -\mathrm{j}$, for $k = 0, 1$:

$$X[0] = E[0] + W_4^0 O[0] = 1 + 3 = 4, \qquad X[2] = E[0] - W_4^0 O[0] = 1 - 3 = -2,$$
$$X[1] = E[1] + W_4^1 O[1] = 1 + (-\mathrm{j})(1) = 1 - \mathrm{j}, \qquad X[3] = E[1] - W_4^1 O[1] = 1 + \mathrm{j}.$$

So $X = (4,\ 1-\mathrm{j},\ -2,\ 1+\mathrm{j})$. Check against the direct sum: $X[0] = \sum x[n] = 4$; good. And $X[3] = \overline{X[1]}$ as it must for a real input (next section). Plugging into (9.1) directly gives the same four numbers, with a fraction of the arithmetic of the brute-force $4^2 = 16$ multiplies.

9.7 Using a real FFT: reading the output

Knowing how the FFT works is half the battle; reading its output correctly is the other half, and it trips up nearly everyone at first. Suppose you sample a real signal at rate f_s (samples per second) and take an N-point FFT. Here is what the N output numbers mean.

Focus

Frequency of bin k. Output $X[k]$ measures the component at frequency

$$f_k \;=\; k\,\frac{f_s}{N} \quad \text{Hz}, \qquad k = 0, 1, \ldots, N-1. \tag{9.8}$$

The spacing between bins is $\Delta f = f_s/N$, the *frequency resolution*. Bin 0 is DC (the average / zero frequency); bin 1 is one cycle per window; and so on.

Common trap

The top half is negative frequencies. For $k > N/2$, bin k represents the frequency $f_k - f_s$, i.e. a *negative* frequency $-(N-k)f_s/N$. Equivalently, $X[N-k] = \overline{X[k]}$ for a real input – the spectrum is conjugate-symmetric, so the upper half is a mirror image of the lower half carrying no new information. Bin $N/2$ is the Nyquist frequency $f_s/2$. If you plot all N magnitudes you will see a mirror; only bins 0 through $N/2$ are independent for real data.

Because of that symmetry, real-input FFTs have a dedicated, cheaper routine (`numpy.fft.rfft`) that returns only the $\lfloor N/2 \rfloor + 1$ non-redundant bins, $k = 0 \ldots N/2$. Use it for sensor data – it is faster and removes the temptation to double-count energy. Figure 9.5 shows a worked magnitude spectrum.

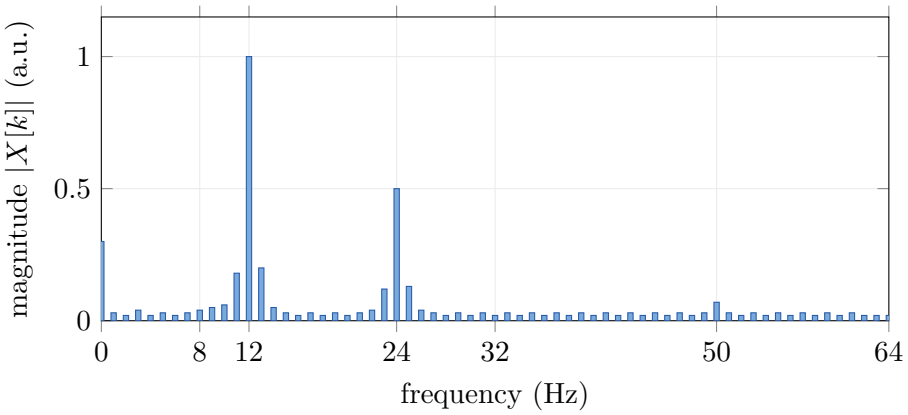


Figure 9.5: Magnitude spectrum (`rfft`) of a 1 s record sampled at $f_s = 128$ Hz, $N = 128$, so each bin is 1 Hz wide. Two real tones appear as peaks at 12 Hz (amplitude 1) and 24 Hz (amplitude 0.5); the bar at 0 Hz is a DC offset, and the small clutter is broadband noise. Only bins 0 to $N/2 = 64$ are shown because the rest are the conjugate mirror.

Amplitude calibration

A raw FFT bin is *not* the amplitude of the sinusoid. For a real cosine of amplitude A that lands exactly on bin k (with $0 < k < N/2$), the FFT puts magnitude $A\,N/2$ in bin k (and the same in its mirror $N - k$). To recover A from a one-sided spectrum, scale by $2/N$:

$$A \;\approx\; \frac{2}{N}\,\big|X[k]\big| \quad (0 < k < N/2), \qquad A_{\mathrm{DC}} = \frac{1}{N}\,|X[0]|, \qquad A_{\mathrm{Nyq}} = \frac{1}{N}\,|X[N/2]|. \tag{9.9}$$

The DC and Nyquist bins use $1/N$ rather than $2/N$ because they have no separate mirror partner. If you forget the $2/N$ you will report amplitudes off by a factor of $N/2$ – the single most common FFT-reading mistake.

9.8 Zero-padding and the difference between resolution and interpolation

A frequent move is *zero-padding*: appending zeros to your signal before the FFT, usually to reach a power-of-two length or to get more output points. It is useful, but it is widely misunderstood, so let us be precise about what it does and does not buy.

Why it works

Zero-padding x from length N to length $M > N$ does **not** add information – the zeros carry none. What it does is evaluate the *same* underlying continuous-frequency spectrum (the DTFT of your finite record) at more, finely spaced points. The true frequency resolution – your ability to *separate* two nearby tones – is fixed by the duration of the real (nonzero) data, $\Delta f \approx f_s/N$. Zero-padding gives a smoother, denser plot (interpolation), not sharper peaks. Two tones unresolved at length N stay unresolved no matter how many zeros you add.

Intuition

Think of the finite record as the true infinite spectrum convolved with a sinc-shaped window response whose width is set by the record length. Zero-padding samples that fixed sinc shape more densely. You see the shape better – handy for locating a peak precisely or for making a pretty curve – but you cannot make the sinc itself narrower by padding. To narrow it you must collect more *real* data (longer N) or trade resolution for a cleaner window (Chapter on windowing).

Common trap

Two real, distinct uses of zero-padding, often confused:

1. *Spectral interpolation / nicer plots:* pad to $2N$, $4N$, ... to draw a denser magnitude curve or to pin down a peak frequency between bins.
2. *Linear convolution via the FFT:* to convolve two sequences correctly with the FFT you must zero-pad *both* to at least the sum of their lengths minus one, otherwise the circular convolution the DFT computes will wrap around and corrupt the result. (We return to this in the convolution chapter.)

Reaching a power-of-two length for speed is a happy by-product of (1), not a third purpose.

9.9 A short NumPy example

Talk is cheap; here is a complete, correct script that synthesizes two tones, transforms them, and reads amplitudes back using the calibration of (9.9).

Listing 9.1: Computing and reading a real FFT.

```
import numpy as np

fs = 128.0          # sampling rate, Hz
N  = 128            # number of samples (power of two -> fast)
t  = np.arange(N) / fs

# Signal: 1.0 amplitude at 12 Hz + 0.5 amplitude at 24 Hz + DC offset
x = 0.3 + 1.0*np.cos(2*np.pi*12*t) + 0.5*np.cos(2*np.pi*24*t)

X     = np.fft.rfft(x)                  # only non-redundant bins: 0..N/2
freqs = np.fft.rfftfreq(N, d=1/fs)      # frequency of each bin, in Hz

# Amplitude calibration: 2/N for interior bins, 1/N for DC and Nyquist
amp = (2.0/N) * np.abs(X)
amp[0] /= 2.0                           # DC bin has no mirror
if N % 2 == 0:
    amp[-1] /= 2.0                      # Nyquist bin has no mirror

# Report the strongest few bins
peaks = np.argsort(amp)[::-1][:3]
for k in sorted(peaks):
    print(f"{freqs[k]:6.1f} Hz   amplitude ~ {amp[k]:.3f}")

```

```
# Verify the FFT matches the brute-force DFT exactly (to rounding)
n = np.arange(N)
F = np.exp(-2j*np.pi*np.outer(n, n)/N)      # the DFT matrix
X_full = np.fft.fft(x)
assert np.allclose(X_full, F @ x)           # same numbers, ~N^2 vs N log N work
print("FFT matches direct DFT:", np.allclose(X_full, F @ x))
```

Running it prints frequencies near 0.0, 12.0, and 24.0 Hz with amplitudes near 0.3, 1.0, and 0.5 – the very signal we put in – and confirms the FFT equals the dense DFT matrix product. Two small but vital habits live in that listing: use `rfftfreq` so you never hand-compute bin frequencies, and never trust a bin magnitude as an amplitude until you have applied the $2/N$ scale.

Try it yourself

Modify the listing to time both methods on $N = 4096$:

```
import time
x = np.random.randn(4096)
n = np.arange(4096); F = np.exp(-2j*np.pi*np.outer(n,n)/4096)
t0=time.perf_counter(); _=F@x;            t_dft=time.perf_counter()-t0
t0=time.perf_counter(); _=np.fft.fft(x);  t_fft=time.perf_counter()-t0
print(f"DFT {t_dft*1e3:.1f} ms   FFT {t_fft*1e3:.3f} ms   "
      f"speedup {t_dft/t_fft:.0f}x")
```

You should see the FFT beat the matrix multiply by two to three orders of magnitude – and that is against NumPy's heavily optimized BLAS, which makes the matrix multiply far faster than a textbook triple loop would be. Against naive C the gap is larger still.

9.10 The inverse FFT comes for free

The inverse DFT,

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, W_N^{-kn}, \tag{9.10}$$

differs from the forward transform only by a sign in the exponent and a $1/N$ scale. So the *same* butterfly machinery computes it – conjugate the inputs, run a forward FFT, conjugate the outputs, and divide by N; or simply use the library's `ifft`. The practical consequence is large: any operation easy in the frequency domain (filtering, convolution, correlation) becomes a round trip – FFT, multiply, inverse FFT – each leg $O(N \log N)$. This "transform, operate, transform back" pattern is the workhorse of the rest of the book.

Application

Fast convolution. Filtering a length-N signal with a length-M kernel costs $O(NM)$ by direct convolution. Pad both to $N + M - 1$, FFT each, multiply the spectra pointwise, and inverse-FFT: total $O((N + M) \log(N + M))$. For long filters this is dramatically faster, and it is how audio reverb plugins convolve a signal with a multi-second impulse response in real time. The convolution theorem (later chapter) is what licenses the pointwise multiply.

9.11 Toward the wrist tracker

Toward the wrist tracker

Our gesture controller will carry an inertial measurement unit (IMU: accelerometer plus gyroscope) on the forearm, and we want to react to motion in real time – ideally with latency low enough to feel instant while playing. A tremor, a tap, a wrist flick, the rhythmic shake of a shaker gesture: these have signatures in the *frequency* content of the IMU streams, not just their raw values. The plan is to buffer a short window of samples, FFT it, and read features off the spectrum (dominant frequency, band energies).

Make it concrete. Sample the IMU at $f_s = 200$ Hz and use a window of $N = 256$ samples. Then:

- Window duration $= N/f_s = 256/200 = 1.28$ s of history per FFT.
- Bin spacing $\Delta f = f_s/N = 200/256 \approx 0.78$ Hz – fine enough to tell a 3 Hz tremor from a 5 Hz one.
- Useful band: 0 to Nyquist $= f_s/2 = 100$ Hz, far above human limb motion (mostly under 20 Hz), so we have generous headroom and can even downsample.
- Cost: $\frac{N}{2}\log_2 N = 128 \times 8 = 1024$ butterflies – a few thousand floating-point operations, trivial for an FFT a few hundred times per second even on a Cortex-M microcontroller. The naive DFT at $N^2 = 65536$ would be far heavier and might miss the real-time budget.

If 1.28 s of latency is too sluggish (it is, for percussive triggers), shrink the window or slide it with overlap; that is the short-time Fourier transform of the next chapter. The FFT is what makes any of this affordable on a wrist-worn battery device.

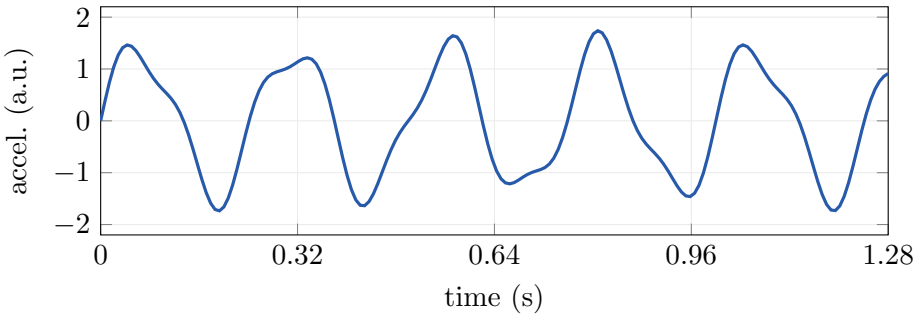


Figure 9.6: A simulated forearm accelerometer trace during a rhythmic shake: a strong 4 Hz oscillation with a weaker 9 Hz wobble, over one $N = 256$ window at $f_s = 200$ Hz (1.28 s). The FFT of this block turns the tangle of wiggles into two clean peaks, from which the controller reads the dominant gesture rate.

The frequency picture is what makes this tractable: Figure 9.6 shows a raw IMU trace whose rhythm is obvious to a spectrum but awkward to read off the time series.

Toward the wrist tracker

Hard limits – read this twice. An FFT of a wrist or forearm IMU tells you about the *gross motion of the whole limb*: is it shaking, at what rate, how vigorously, and coarse posture changes. It *cannot* recover individual finger joint angles. The kinematic information about finger flexion simply is not present in the arm-mounted accelerometer/gyro signal in any reliable form; no transform conjures information the sensor never captured. To sense fingers you need a different modality: surface electromyography (sEMG) or force myography (FMG) over the forearm muscles can discriminate a handful of discrete finger/grip gestures (commercial armbands such as the former Myo, and the research behind Meta's wrist sEMG work, demonstrate this), and *continuous* finger pose realistically needs sensors on the fingers themselves (instrumented gloves or rings). So: FFT on the IMU is excellent for rhythm, taps, flicks, and activity state – treat finger-level control as a separate, harder problem requiring EMG/FMG or finger-mounted sensing, and say so plainly in any demo.

Build it (hardware)

On an Arduino-class board, a fixed-point or float radix-2 FFT is cheap if you respect a few rules.

- **Pick a power-of-two window** (128, 256, 512). Non-power-of-two sizes drag in slower mixed-radix code or padding.
- **Use a vendor library, not your own bit-reversal.** On ARM Cortex-M, ARM's `CMSIS-DSP` provides `arm_rfft_fast_f32` – a tuned real FFT with the twiddle tables baked in. On an ESP32, the ESP-DSP library has equivalents.
- **Remove DC first.** Subtract the running mean of the accelerometer before the FFT, or bin 0 will dominate and swamp the motion bands you care about.
- **Window the block** (Hann) before transforming to reduce spectral leakage from the abrupt block edges; this matters more than zero-padding for clean peaks.
- **Budget the memory.** A 256-point float FFT needs roughly 256 complex words of scratch plus a twiddle table – a few kilobytes, fine on most M-class parts.

A practical loop: fill a 256-sample ring buffer at 200 Hz, every 64 new samples copy it out, remove mean, apply Hann, `arm_rfft_fast_f32`, take magnitude, then classify from band energies. That overlap gives a fresh spectrum every $64/200 = 0.32$ s while keeping the 0.78 Hz resolution of a 256-point window.

9.12 Practical notes and gotchas

A short field guide to the mistakes people actually make.

- **Spectral leakage.** A tone whose frequency falls *between* bins does not sit in one bin; its energy smears across neighbors (the sinc skirts of the window). Apply a window function (Hann, Hamming, Blackman) before the FFT to suppress the side lobes – at the cost of slightly wider main lobes. This is a resolution-versus-leakage trade, covered fully in the windowing chapter.
- **Power spectrum scaling.** For energy/power, square the magnitude and apply the right

normalization for your window; do not compare raw $|X[k]|^2$ across different window functions or lengths without renormalizing.

- **Real vs. complex transform.** For real sensor data use `rfft`; it is about twice as fast and returns exactly the non-redundant half. Only use the full complex `fft` when your data is genuinely complex (e.g. IQ radio samples).
- **In-place and ordering.** If you call a low-level FFT, know whether it expects or returns bit-reversed order, and whether it overwrites the input. High-level libraries hide this; embedded ones often do not.
- **Numerical accuracy.** The FFT is not only faster than the naive DFT, it is usually *more accurate*: $O(\log N)$ rounding steps accumulate per output instead of $O(N)$ in the long naive sum. Speed and accuracy point the same way here.

Pause

The whole chapter in five beats: (1) the DFT is an $N \times N$ matrix multiply, $O(N^2)$. (2) The twiddle factor's half-turn symmetry means a size-N DFT contains two size-$N/2$ DFTs. (3) Splitting input by even/odd and recombining with a butterfly turns that into $O(N \log N)$. (4) The recursion's index shuffle is bit-reversal. (5) To *use* an FFT: bin k sits at kf_s/N Hz, the top half mirrors the bottom for real data, scale by $2/N$ for amplitude, and zero-padding interpolates the plot without adding resolution.

9.13 Exercises

Exercise 9.1. For $N = 4096$, compute the exact number of complex multiplications used by (a) the naive DFT (N^2) and (b) the radix-2 FFT ($\frac{N}{2}\log_2 N$). What is the ratio? If one complex multiply takes 5 ns, estimate the wall-clock time of each.

Exercise 9.2. Compute the 4-point DFT of $x = (0, 1, 2, 3)$ by hand using the radix-2 split of Section 9.3 (find E, O, then the butterflies). Verify $X[0]$ equals the sum of the samples and check conjugate symmetry $X[3] = \overline{X[1]}$.

Exercise 9.3. Write out the bit-reversal permutation for $N = 16$ (a 4-bit index). Which input index ends up in slot 9? Confirm your answer by reversing the bits of $9 = 1001_2$.

Exercise 9.4. Prove that if $x[n]$ is real then $X[N-k] = \overline{X[k]}$. Start from the DFT definition (9.1) and use $W_N^{(N-k)n} = \overline{W_N^{kn}}$. What does this say about how many of the N outputs are independent?

Exercise 9.5. You sample at $f_s = 8$ kHz and take a 1024-point FFT. (a) What frequency does bin 200 represent? (b) What is the bin spacing Δf? (c) Which bin is closest to a 1 kHz tone, and what is the residual error in Hz?

Exercise 9.6. A real cosine of amplitude 3.0 at exactly 50 Hz is sampled at $f_s = 200$ Hz, $N = 200$. (a) Which bin does it land in? (b) What magnitude $|X[k]|$ do you expect in that bin before calibration? (c) Apply (9.9) to recover the amplitude.

Exercise 9.7. You have $N = 100$ samples of a signal sampled at $f_s = 1000$ Hz containing two tones at 210 Hz and 215 Hz. (a) What is Δf? Are the tones resolved? (b) If you zero-pad to 1000 points, does the FFT now separate them into two distinct peaks? Explain in terms of resolution versus interpolation. (c) What *would* let you resolve them?

Exercise 9.8. The FFT satisfies $T(N) = 2T(N/2) + cN$ for some constant c (the cN is the butterfly combine step). Solve this recurrence by unrolling it $\log_2 N$ times and show the solution is $T(N) = cN \log_2 N + T(1) \cdot N$. Identify which term dominates for large N.

Exercise 9.9. Adapt the listing in Section 9.9 to a noisy signal: add `0.2*np.random.randn(N)` to `x`. Does the peak detection still find the two tones? Now reduce the tone amplitudes until a peak disappears into the noise floor. What does this tell you about detecting weak gestures in a noisy IMU stream?

Exercise 9.10. For the wrist tracker, you want at least 0.5 Hz frequency resolution and you sample the IMU at $f_s = 200$ Hz. (a) What is the minimum window length N (round up to a power of two)? (b) How many seconds of latency does that window impose? (c) If that latency is too high for triggering a drum hit, name two changes (from this chapter and the next) that reduce perceived latency, and state what each costs.

Exercise 9.11. In two or three sentences, explain to a teammate why running an FFT on the forearm IMU *cannot* give per-finger joint angles, and what sensing modality you would add if the application genuinely needed finger-level control. Be specific about the limitation, not vague.

Chapter 10

The Short-Time Fourier Transform and Spectrograms

The Fourier transform is a beautiful machine, but it has a blind spot, and that blind spot is *time*. Hand it a recording of someone whistling a rising tone, or a snippet of speech, or the wrist motion of a guitarist strumming, and it will tell you precisely *which* frequencies are present across the whole recording. What it will not tell you is *when* each frequency happened. It flattens the entire history of the signal into one verdict. For a steady, unchanging signal that verdict is the whole truth. For anything that *changes* – which is to say, for almost every interesting signal in the real world – it is a smear.

This chapter is about fixing that. The fix is almost embarrassingly simple in concept: instead of analysing the whole signal at once, chop it into short pieces and Fourier-analyse each piece. The result is the **Short-Time Fourier Transform** (STFT), and when we draw its magnitude as a picture we get a **spectrogram** – arguably the single most useful visualization in all of signal processing. By the end you will be able to read a spectrogram the way a musician reads a score, you will understand the iron law that trades time resolution against frequency resolution, and you will see why the spectrogram is exactly the right lens for the changing, gesture-driven signals at the heart of our wrist controller.

10.1 Why the plain Fourier transform is not enough

Recall what the Fourier transform of a signal $x(t)$ does. It computes, for every frequency f, the inner product of x with a complex sinusoid that spins at that frequency:

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-\mathrm{j}2\pi f t}\, \mathrm{d}t = \langle x,\, e^{\mathrm{j}2\pi f t} \rangle. \tag{10.1}$$

The integral runs over *all* time, from the beginning of the universe to the end of it. Every instant of the signal contributes to every frequency bin with equal weight. That is exactly what we want when the signal is *stationary* – when its frequency content does not change – because then "which frequencies are present" is a well-posed question with a single answer.

Intuition

Think of (10.1) as asking the whole signal one yes/no-ish question per frequency: "On the whole, how much do you look like a sine wave wiggling at f cycles per second?" If the signal is a pure tone the answer is sharp. But if the signal was a low tone for the first second and a high tone for the second second, the transform reports *both* tones, with no hint that they took turns. It heard the whole concert and told you the set list – but not the running order.

Here is the trap in concrete form. Consider two signals over $0 \le t < 2$ seconds. Signal A is a 5 Hz tone for the first second followed by a 15 Hz tone for the second second. Signal B is a 15 Hz tone first, then a 5 Hz tone. These are obviously different signals – if they were sounds you would hear one rising and the other falling. Yet their magnitude spectra $|X(f)|$ are nearly identical: both contain "5 Hz" and "15 Hz" in roughly equal amounts. All the information distinguishing them is hidden in the *phase*, tangled across the whole frequency axis in a way no human can read.

Common trap

"The magnitude spectrum tells me what is in my signal." It tells you what is in your signal *integrated over the entire record*. Two completely different time-stories can produce the same magnitude spectrum. If your signal evolves – speech, music, a gesture, an engine spinning up – the global spectrum is the wrong tool, and no amount of zero-padding or finer frequency resolution will fix it. You need to localize in time.

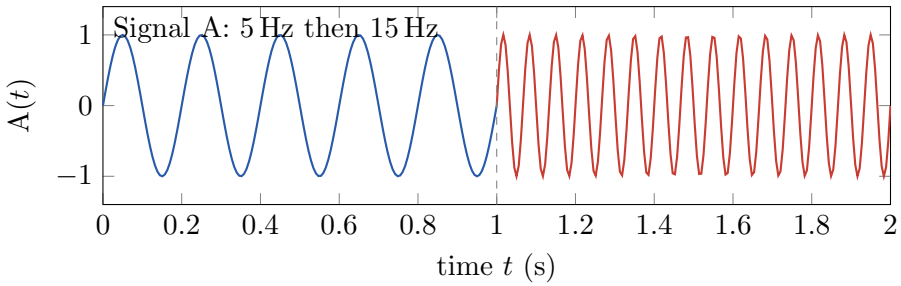


Figure 10.1: A non-stationary signal: a low tone for one second, then a high tone. Its global magnitude spectrum would show both 5 and 15 Hz with no clue that they happened in sequence. Reversing the order leaves the magnitude spectrum almost unchanged.

What is really happening

What is really happening is a mismatch of *basis* and *signal*. The Fourier basis $\{e^{j2\pi ft}\}$ consists of functions that are spread out over all time – each one is an eternal, never-decaying oscillation. To represent something local in time (like "a burst of 15 Hz around $t = 1.5$ s") in such a basis, you must add up a huge number of these eternal sinusoids whose tails cancel everywhere except the burst. The information about *when* survives only in the delicate phase relationships among many bins. The STFT changes the basis to one made of *localized* pieces, so that "when" becomes an axis you can read directly.

10.2 The idea: look through a moving window

The cure follows from the diagnosis. If the problem is that the integral sees all of time at once, then restrict its view. Before transforming, multiply the signal by a **window** $w(t)$ – a short bump that is near zero everywhere except in a small neighbourhood. The product $x(t)\,w(t-\tau)$ is a copy of the signal as seen through a porthole centred at time τ: nonzero only near τ, zero elsewhere. Fourier-transform *that*, and you learn the frequency content of the signal *around time* τ. Then slide the window to a new τ and repeat.

Definition 10.1 (Short-Time Fourier Transform)**.** Given a signal $x(t)$ and a window $w(t)$, the (continuous) STFT is

$$X(\tau, f) \;=\; \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, e^{-\mathrm{j}2\pi f t}\, \mathrm{d}t. \tag{10.2}$$

It is a function of *two* variables: the time position τ of the window, and the frequency f. For each fixed τ it is just an ordinary Fourier transform of the windowed slice $x(t)\,w(t-\tau)$.

Compare (10.2) with (10.1): the only new ingredient is the factor $w(t-\tau)$. That single factor turns a one-dimensional function of frequency into a two-dimensional function of time *and* frequency. We have traded a sharp but timeless answer for a slightly blurry but time-resolved one.

Why it works

Why does multiplying by a window before transforming localize in time? Because the integral (10.2) can only "hear" the signal where the window is nonzero. Outside the porthole, $w(t-\tau) = 0$ kills the integrand regardless of what x is doing there. So $X(\tau, f)$ is built *entirely* from the slice of x near τ. Slide τ and you get a fresh local spectrum for each position – a movie of the spectrum over time, rather than a single snapshot.

Intuition

A spotlight in a dark theatre. The plain Fourier transform turns on the house lights and asks "what is on this stage, overall?" The STFT keeps the house dark and walks a narrow spotlight (w) across the stage, asking at each position "what is *here*, right now?" A wide spotlight sees a lot of the stage at once (poor time localization) but lets your eyes adjust to read fine detail (good frequency detail). A pinpoint spotlight tells you exactly *where* something is, but you can barely make out what it is. That tension is the whole story of this chapter.

10.2.1 Two ways to read the same equation

There is a subtlety worth pinning down, because it explains a phase convention you will meet in real libraries. Equation (10.2) multiplies by $e^{-\mathrm{j}2\pi f t}$ using *absolute* time t. An equivalent and very common alternative slides the complex exponential along with the window:

$$\tilde{X}(\tau, f) \;=\; \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, e^{-\mathrm{j}2\pi f (t-\tau)}\, \mathrm{d}t \;=\; e^{\mathrm{j}2\pi f \tau}\, X(\tau, f). \tag{10.3}$$

The two differ only by the phase factor $e^{\mathrm{j}2\pi f\tau}$. Since we will spend almost all our time looking at the *magnitude* $|X(\tau, f)|$, and $|e^{\mathrm{j}2\pi f\tau}| = 1$, the choice does not affect any spectrogram we draw. It

matters only when you reconstruct the signal or interpret phase. Keep it in your back pocket and move on.

Connection

The STFT is, at heart, an *inner product* – the same notion from the linear algebra companion. Fix τ and f; then $X(\tau, f) = \langle x,\ g_{\tau,f} \rangle$ where $g_{\tau,f}(t) = w(t-\tau)\, e^{\mathrm{j}2\pi f t}$ is a windowed sinusoid called a *time-frequency atom* or *Gabor atom*. Each atom is a little wave-packet parked at time τ and tuned to frequency f. The STFT measures how much of each atom lives in your signal. This is exactly the projection-onto-a-basis view of Fourier analysis, now with a basis whose members are local in both time and frequency. The wavelet transform of Chapter 24 is the same idea with atoms that stretch and shrink instead of just sliding.

10.3 Windows: the shape of the porthole

We cannot avoid windowing – even doing nothing is a window. If you simply grab N consecutive samples and transform them, you have implicitly multiplied the infinite signal by a **rectangular window** rect that is 1 on those samples and 0 elsewhere. The question is never *whether* to window, only *which shape*.

The shape matters because of what happens at the *edges*. A rectangular window slams the signal to zero abruptly. In the frequency domain, that abrupt cut convolves the true spectrum with the transform of the rectangle – a sinc function with tall *side lobes* (Chapter 7). Those side lobes leak energy from a strong tone into neighbouring frequencies, a phenomenon called **spectral leakage**. A tapered window that eases the signal gently down to zero at both ends has much smaller side lobes, at the cost of a slightly wider central lobe.

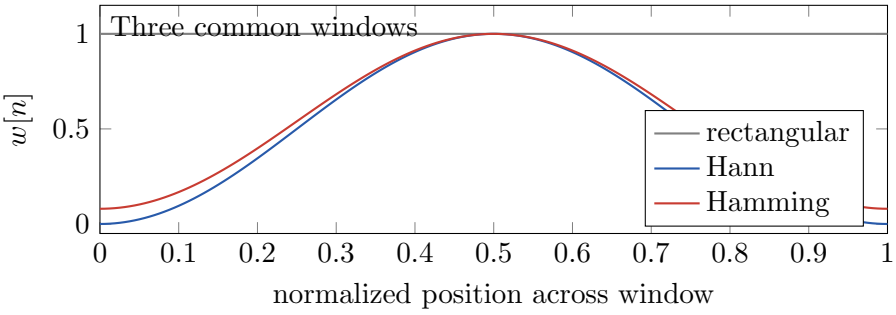


Figure 10.2: The rectangular window has hard edges (large leakage, narrow main lobe). The Hann and Hamming windows taper smoothly to (or near) zero, trading a wider main lobe for dramatically lower side lobes.

The two tapered windows in Figure 10.2 are workhorses. For a window of length N (samples $n = 0, \ldots, N-1$):

$$\text{Hann:} \quad w[n] = \tfrac{1}{2}\left(1 - \cos\tfrac{2\pi n}{N-1}\right), \tag{10.4}$$

$$\text{Hamming:} \quad w[n] = 0.54 - 0.46\cos\tfrac{2\pi n}{N-1}. \tag{10.5}$$

The Hann window goes all the way to zero at both ends; the Hamming window stops just shy of zero, which cancels the nearest side lobe even better but leaves a small pedestal far out. For everyday spectrograms the Hann window is a fine default and the one we will assume unless noted.

Focus

A window does two jobs at once, and it is worth holding both in mind:

1. **It localizes in time.** The window's *width* sets how big a slice of signal each spectrum sees.
2. **It shapes the frequency response.** The window's *taper* sets how cleanly that slice's spectrum is rendered – how much leakage there is.

Width controls the time/frequency tradeoff (next section); taper controls leakage within whatever resolution the width allows.

Connection

Multiplication in time is *convolution* in frequency – the convolution theorem of Chapter 11. The windowed slice $x(t)w(t-\tau)$ therefore has spectrum $X(f) * W(f)$ (up to a phase from the shift). Each true spectral line gets replaced by a blurred copy of the window's own spectrum $W(f)$. Choosing a window is choosing the shape of that blur. This is the same machinery that governs FIR filter design in Chapter 16, where windows tame the ripples of a truncated ideal filter.

10.4 The spectrogram: a picture of the STFT

The STFT $X(\tau, f)$ is a complex-valued function of two real variables. We cannot easily look at a complex 2-D surface, so we throw away the phase and look at the squared magnitude. That image is the spectrogram.

Definition 10.2 (Spectrogram)**.** The spectrogram of x is the squared magnitude of its STFT,

$$S(\tau, f) \;=\; |X(\tau, f)|^2, \tag{10.6}$$

displayed as an image with time τ on the horizontal axis, frequency f on the vertical axis, and S encoded as brightness or colour. Because spectral energy spans many orders of magnitude, it is almost always shown in decibels, $10 \log_{10} S$.

Focus

How to read a spectrogram.

- **Horizontal axis = time.** Left is earlier, right is later. Read it like a piano roll or a music score scrolling past.
- **Vertical axis = frequency.** Bottom is low (bass, slow), top is high (treble, fast).
- **Colour/brightness = energy.** Bright means strong; dark means little or no energy at that time and frequency.
- **Horizontal bright lines** = sustained tones (a held note, a hum, a steady rotation).

- **Vertical bright streaks** = sudden broadband events (a click, a drum hit, a footstep, a tap of the finger).
- **Sloping lines** = glides – a rising line is a frequency sweeping up, like a whistle going up or a motor spinning faster.
- **Stacks of evenly spaced lines** = a harmonic tone (a voiced vowel, a plucked string); the spacing is the fundamental.

The schematic in Figure 10.3 shows the spectrogram of our running example – the rising signal of Figure 10.1 – with the time-ordering now plainly visible. The plain Fourier transform could never show this.

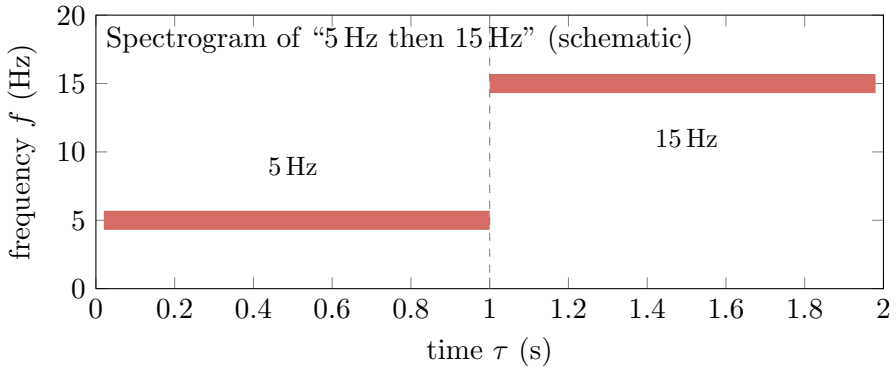


Figure 10.3: The spectrogram of the signal in Figure 10.1. The low tone occupies the first second, the high tone the second second – exactly the "running order" the global Fourier transform discarded. Each bright patch has nonzero height (frequency spread) and width (time spread) set by the window; an ideal point would require a window that is both infinitely short and infinitely long.

Application

The voiceprint. The spectrogram was born in wartime speech research at Bell Labs in the 1940s (the "sound spectrograph"), and it remains the native representation of speech. A vowel shows up as a stack of harmonics shaped by broad bright bands called *formants* – resonances of the vocal tract – that slide as the mouth changes shape. Consonants such as /s/ and /f/ appear as high-frequency noise fills; plosives like /p/ and /t/ as a brief silence then a vertical burst. Trained readers can literally read speech off a spectrogram. Every modern speech recognizer still starts by computing something very close to a spectrogram (the mel-spectrogram of Chapter 27).

History

The sound spectrograph was developed at Bell Telephone Laboratories during World War II and declassified afterward; the 1947 book *Visible Speech* by Potter, Kopp, and Green popularized reading speech from these images and even explored teaching the deaf to speak by watching their own spectrograms in near real time. The mathematics was older – Dennis Gabor's 1946 paper *Theory of Communication* introduced the windowed transform with a Gaussian window and the "logon" (what we now call a time-frequency atom), and proved the

uncertainty limit we meet in the next section. The spectrogram is thus the visible face of Gabor's analysis.

10.5 The central tradeoff: time versus frequency resolution

Here is the iron law of the STFT, and it is not a defect of any particular algorithm – it is a law of nature about waves. *You cannot be arbitrarily precise in time and in frequency at the same time.* The window length forces a choice.

Think about it physically. To measure a frequency precisely you must watch an oscillation for many cycles – a single glimpse cannot tell a 100 Hz tone from a 101 Hz tone, because in a few milliseconds they look identical. Resolving 1 Hz of difference requires watching for about 1 second, because only then have the two tones drifted a full cycle out of step. But watching for a full second means you have *no idea when* within that second the tone occurred – you have blurred time by a second to sharpen frequency to a hertz.

Why it works

Why is this unavoidable? Because frequency *is* a statement about behaviour over an interval of time. "This signal has frequency f" is shorthand for "this signal repeats every $1/f$ seconds, for a while." If you only look for an instant, the phrase "for a while" has no meaning, so neither does a precise frequency. The shorter your window, the fewer cycles fit inside it, and the fuzzier the frequency you can name. Conversely, the longer the window, the more time you have smeared together, and the vaguer your sense of *when*. The two resolutions are reciprocals of one window length; tightening one necessarily loosens the other.

We can make this quantitative. If the window spans a duration T (in seconds), then:

$$\Delta t \approx T \qquad \text{and} \qquad \Delta f \approx \frac{1}{T}, \qquad \text{so} \qquad \Delta t \cdot \Delta f \approx 1. \tag{10.7}$$

The product of the time spread and the frequency spread is roughly constant. You can shuttle resolution back and forth between the axes by changing T, but you cannot shrink the product below a fixed floor. Figure 10.4 draws this as a tiling of the time-frequency plane: each window length reshapes the tiles but holds their area fixed. With a Gaussian window the floor is exactly $\Delta t\,\Delta f \geq \frac{1}{4\pi}$ – this is the *Gabor–Heisenberg uncertainty* relation, the same inequality (up to constants) that governs position and momentum in quantum mechanics, for the same mathematical reason.

Formal

Define the spreads of a unit-energy window w by its second moments about its centre,

$$(\Delta t)^2 = \int t^2\,|w(t)|^2\,\mathrm{d}t, \qquad (\Delta f)^2 = \int f^2\,|W(f)|^2\,\mathrm{d}f.$$

Then for *any* window,

$$\Delta t \cdot \Delta f \;\geq\; \frac{1}{4\pi},$$

with equality *only* for a Gaussian $w(t) = e^{-at^2}$. No window beats the Gaussian; no window escapes the bound. This is why the Gaussian-windowed STFT (the *Gabor transform*) is the

theoretically optimal trade and why "you can't have both" is a theorem, not an excuse.

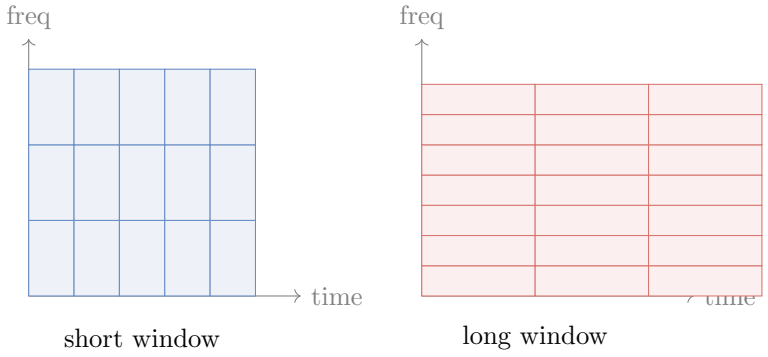


Figure 10.4: The time-frequency plane tiled by STFT atoms. Every tile has the same *area* ($\Delta t\,\Delta f \approx 1$), so a short window gives tall, thin tiles (good time resolution, coarse frequency) and a long window gives short, wide tiles (good frequency resolution, coarse time). Changing the window length reshapes the tiles but never shrinks their area.

10.5.1 Wideband versus narrowband

The same signal looks qualitatively different depending on which way you spend the tradeoff. The two regimes even have names borrowed from speech analysis:

- **Narrowband spectrogram** (long window, e.g. 30–50 ms of speech): fine *frequency* resolution, coarse time. Individual harmonics of a voiced sound resolve into separate horizontal lines, but fast events smear. Good for "what notes / which pitch."
- **Wideband spectrogram** (short window, e.g. 3–6 ms of speech): fine *time* resolution, coarse frequency. Harmonics blur together into broad formant bands, but each glottal pulse shows as a crisp vertical striation. Good for "what is happening when / timing of events."

Figure 10.5 contrasts the two for a single tone that jumps in frequency. Neither view is "correct"; they answer different questions.

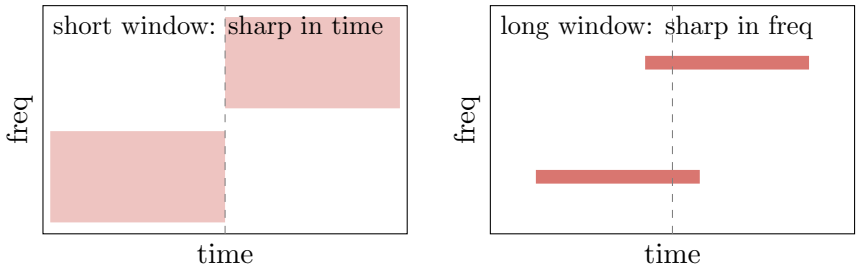


Figure 10.5: The same tone-jump (low then high), analysed two ways. Short window (left): the jump time is crisp but each tone is a fat vertical smear in frequency. Long window (right): each tone is a thin horizontal band in frequency, but the transition time is blurred and the two patches even overlap in time. The window length picks your poison.

Common trap

"I will just zero-pad my short window to get more frequency bins." Zero-padding gives you more closely spaced *samples* of the spectrum – a smoother-looking curve – but it does *not* improve frequency *resolution*. Two tones that were unresolvable in a 6 ms window stay unresolvable no matter how many zeros you append; you are interpolating, not informing. True resolution comes only from a longer *real* window, i.e. more genuine signal. Resolution is set by how long you looked, not by how finely you grid the answer.

Walking question

You want to resolve two musical notes a semitone apart near middle A (440 Hz and about 466 Hz, a gap of ≈ 26 Hz). What window length do you need? From (10.7), $\Delta f \approx 1/T$, so to get $\Delta f \leq 26$ Hz you need $T \gtrsim 1/26 \approx 38$ ms. At a 44.1 kHz sample rate that is about $0.038 \times 44100 \approx 1700$ samples, so a window of $N = 2048$ is a sensible choice. Now ask the opposite: with a 38 ms window, how well can you time a note onset? Only to about ± 38 ms – noticeably loose for fast passages. That is the tradeoff in a single concrete breath.

10.6 From integral to algorithm: the discrete STFT

Real signals are sampled, and real spectra are computed with the FFT (Chapter 9). The discrete STFT replaces the integral of (10.2) with a sum over a length-N window, hops the window forward by H samples (the **hop size**), and takes an N-point DFT of each frame:

$$X[m,k] \;=\; \sum_{n=0}^{N-1} x[\,mH+n\,]\; w[n]\; e^{-\mathrm{j}2\pi kn/N}, \qquad k = 0,\ldots,N-1. \tag{10.8}$$

Here m indexes the *frame* (window position, $\tau = mH/f_s$) and k indexes the *frequency bin* ($f = kf_s/N$, where f_s is the sample rate). Each column of the spectrogram is one frame's FFT; you build the image column by column as you slide along.

Focus

The four knobs of a discrete STFT, and what each one buys you:

Knob	Symbol	Controls
window length	N	time/frequency resolution tradeoff
window shape	$w[n]$	leakage / side-lobe level
hop size	H	overlap, smoothness, redundancy, cost
FFT length	$\geq N$	bin spacing (display, not resolution)

10.6.1 Overlap and the hop size

If you set the hop equal to the window length ($H = N$), the frames sit side by side with no overlap. That is cheap but risky: a tapered window suppresses the signal near each frame's edges, so an event landing on a window boundary is *seen weakly by both neighbours* and can nearly vanish. The cure

is **overlap**: advance the window by less than its length, so successive frames share samples. A hop of $H = N/2$ gives 50% overlap; $H = N/4$ gives 75%. Overlap makes the spectrogram smoother in time, ensures every sample is covered where some window is near its peak, and is required for clean reconstruction.

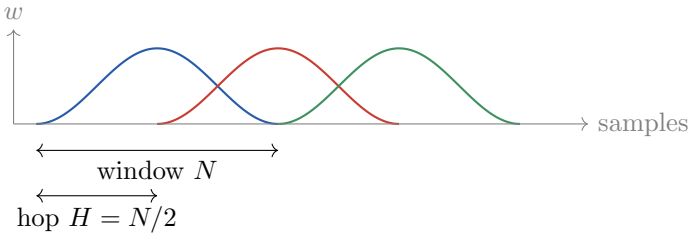


Figure 10.6: Three Hann windows at 50% overlap ($H = N/2$). Where one window dips, its neighbour is rising, so every sample is well covered. With careful window/hop choices the overlapping windows even sum to a constant – the constant-overlap-add property that lets you reconstruct the signal exactly.

Why it works

Why overlap rather than just a longer window? Overlap does *not* change the time/frequency resolution – that is still set by N alone. What overlap buys is *temporal sampling* of the time-frequency plane. Think of each frame as a snapshot; with no overlap you photograph the scene every N samples and may miss fast action between shots. With overlap you photograph more often, catching transitions and giving a fluid, alias-free movie of the spectrum over time. It costs more computation (more FFTs) but no resolution penalty.

Try it yourself

Compute a spectrogram yourself with a few lines of NumPy and watch the knobs bite. Try a chirp first – a tone whose frequency rises linearly – because its spectrogram should be a clean diagonal line, and any blur or staircase you see is the tradeoff made visible.

```
import numpy as np
import matplotlib.pyplot as plt

fs = 8000                          # sample rate (Hz)
t  = np.arange(0, 2.0, 1/fs)       # 2 seconds
# linear chirp: instantaneous freq goes 200 -> 1800 Hz
x  = np.sin(2*np.pi*(200*t + (800/2)*t**2))

N, H = 256, 64                     # window length, hop (75% overlap)
w    = np.hanning(N)
frames = range(0, len(x)-N, H)
S = np.array([np.abs(np.fft.rfft(x[i:i+N]*w))**2 for i in frames]).T
S_db = 10*np.log10(S + 1e-12)

plt.imshow(S_db, origin='lower', aspect='auto',
           extent=[0, 2, 0, fs/2])
plt.xlabel('time (s)'); plt.ylabel('frequency (Hz)')
```

```
plt.show()
# Now rerun with N=1024, H=256. The diagonal gets thinner
# vertically (better freq) but you see less precisely *where*
# along it the energy is -- the tradeoff, in one experiment.
```

10.6.2 Inverting the STFT: overlap-add

Because the STFT is redundant (with overlap we store more numbers than we started with), we can usually throw the signal away and rebuild it from $X[m, k]$. The recipe is **overlap-add**: inverse-FFT each frame back to a short time slice, multiply by a synthesis window, and add the slices back into place with the same hop. If the analysis and synthesis windows satisfy the *constant overlap-add* (COLA) condition – their shifted-by-H copies sum to a constant – the reconstruction is exact. This invertibility is what makes the STFT a processing tool, not just a viewer: modify $X[m, k]$ (filter, denoise, pitch-shift, time-stretch) and resynthesize. It is the engine behind the phase vocoder and countless audio effects in Chapter 28.

10.7 Worked example: reading numbers off the discrete STFT

Example 10.1 (Grids and resolution)**.** Let $f_s = 1000\,\text{Hz}$ and take a window of $N = 200$ samples with hop $H = 100$ (50% overlap). What are the time and frequency grids, and what resolution do we get?

Frequency grid. Bin k sits at $f_k = kf_s/N = k \cdot 1000/200 = 5k$ Hz. So bins are spaced 5 Hz apart: $0, 5, 10, \ldots$ up to the Nyquist frequency of 500 Hz at $k = 100$.

Frequency resolution. The window spans $T = N/f_s = 200/1000 = 0.2\,\text{s}$, so by (10.7) $\Delta f \approx 1/T = 5\,\text{Hz}$. (With a Hann window the true main-lobe width is roughly $2/T = 10\,\text{Hz}$, a bit coarser than the bin spacing – a reminder that bin spacing and resolution are not the same thing.)

Time grid. Frame m is centred near $\tau_m = (mH + N/2)/f_s = (100m + 100)/1000\,\text{s}$, i.e. frames step every $H/f_s = 0.1\,\text{s}$.

Time resolution. Each frame blurs time by about $T = 0.2\,\text{s}$. So an event shorter than 0.2 s cannot be timed more precisely than that, even though the frame *spacing* is a finer 0.1 s – again, sampling density (0.1 s) is not resolution (0.2 s). To time events to 20 ms you would shorten the window to $N \approx 20$, accepting a coarse $\Delta f \approx 50\,\text{Hz}$ in return.

10.8 Applications: speech, music, motion

Application

Music transcription and the piano roll. A spectrogram of a melody is already most of the way to sheet music: each note is a horizontal segment whose height is its pitch and whose length is its duration; chords are vertical stacks; vibrato is a wavy line. Automatic music transcription, the Shazam audio fingerprint (which hashes constellations of spectrogram peaks), and pitch-correction plug-ins all live in the STFT domain. A long window resolves close pitches; a short window pins down fast rhythmic onsets – so transcribers often run *both* and combine.

Application

Machinery and biomedical monitoring. A bearing beginning to fail injects a rising whine and periodic clicks into a machine's vibration; on a spectrogram this is a new sloping line plus regularly spaced vertical streaks, visible long before a human would hear it. In an EEG, a seizure announces itself as a sudden broadband brightening; sleep stages have characteristic spectral signatures (spindles, slow waves) that clinicians read straight off the spectrogram. The common thread: *the interesting information is in how the spectrum changes*, which is precisely what the spectrogram makes visible.

10.9 Toward the wrist tracker

Toward the wrist tracker

Gesture signals are *non-stationary by definition* – a gesture is a change over time, so a single global spectrum of a recording session is almost useless. This is exactly the setting the STFT was built for, and the spectrogram is our primary lens on wrist-worn sensor data.

IMU (accelerometer + gyroscope). Picture the forearm during a tremolo strum or a rhythmic tap. The accelerometer and gyro signals are oscillatory with a frequency set by the motion's tempo, riding on slowly drifting baselines from gravity and arm reorientation. A spectrogram of the gyro magnitude turns a strum at 6 strokes per second into a bright horizontal line at $\approx 6\,\text{Hz}$ that *appears when the strumming starts and vanishes when it stops*, and a single sharp tap into a clean vertical broadband streak (Figure 10.7). Reading tempo, detecting onset and offset, and separating "holding still" from "moving rhythmically" all become trivial in this view, when they were tangled in the raw trace.

Sensible numbers. Hand/wrist motion energy lives roughly in 0–20 Hz (voluntary motion is mostly below 5 Hz; physiological tremor sits around 8–12 Hz). Sample the IMU at $f_s = 100$–$200\,\text{Hz}$ so Nyquist (Chapter 18) comfortably clears the band. A window of $T \approx 0.5\,\text{s}$ ($N = 64$ at 128 Hz) gives $\Delta f \approx 2\,\text{Hz}$ – enough to tell tremor from voluntary tapping – while still updating fast enough to feel responsive. Use 50–75% overlap so the controller reacts within one hop, $\sim 100\,\text{ms}$, instead of one whole window.

The limits, stated plainly. A wrist-mounted IMU senses the motion of the *forearm*, full stop. Its spectrogram can reveal *coarse, discrete* gestures – rhythmic versus still, the tempo

of a repetitive motion, a flick, a twist, a tap – because those are gross forearm dynamics. It *cannot* recover which individual finger moved or by how much: finger flexions barely perturb the wrist's inertial signal, and there is no frequency band that magically encodes finger identity. For finger-level events you need a sensor that listens to the muscles or tendons – surface EMG or force myography on the forearm (Chapter 37) – and for continuous finger joint angles you need sensors *on the fingers* (instrumented rings or gloves). The STFT is a superb tool for what the IMU can actually see; it cannot conjure information the sensor never captured.

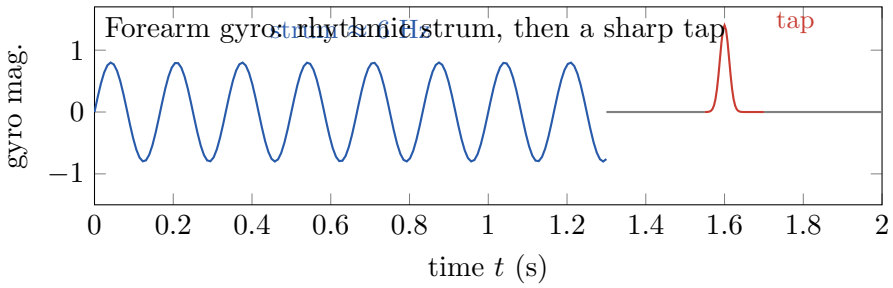


Figure 10.7: A schematic forearm-gyro trace. The rhythmic strum is a sustained oscillation (a horizontal line on the spectrogram near 6 Hz); the tap is a brief impulse (a vertical broadband streak). The spectrogram separates these two events that share the same raw amplitude range.

Build it (hardware)

A streaming spectrogram on a microcontroller. You do not need to store the whole recording – the STFT is naturally incremental, one frame at a time. Keep a circular buffer of the last N IMU samples; every H samples, copy the buffer, multiply by a precomputed Hann window, run a fixed-point FFT (CMSIS-DSP's `arm_rfft_q15` on a Cortex-M, or KissFFT on an ESP32), and you have one spectrogram column.

```
// fires every H new samples (e.g. H=32 at fs=128 Hz)
void on_hop(int16_t *ring, int N) {
    int16_t frame[N];
    for (int n = 0; n < N; n++)
        frame[n] = mul_q15(ring[n], hann_lut[n]); // windowing
    arm_rfft_q15(&S, frame, spectrum);            // in-place FFT
    for (int k = 0; k < N/2; k++)
        mag[k] = mag_q15(spectrum[2*k], spectrum[2*k+1]);
    classify(mag);   // is there 6 Hz energy? a broadband spike?
}
```

With $N = 64$ this is a few thousand multiply-accumulates per hop – trivial even for an 80 MHz part – so a battery wrist unit can stream spectrogram features in real time within a tight power budget. Precompute the Hann window into a lookup table; never call `cos` in the hot loop.

Why this matters

The deep idea of this chapter is that *representation is everything*. The raw IMU trace and its spectrogram contain the same numbers, related by a linear, invertible transform – no

information is created. Yet "is the wrist strumming, and how fast?" is nearly invisible in one representation and screaming obvious in the other. Good signal processing is, over and over, the art of rotating your data into the basis where the answer is easy to see. The STFT rotates a stream of samples into a time-frequency picture, and suddenly motion, music, and speech all become things you can simply *look at*.

10.10 Summary

- The global Fourier transform tells you *what* frequencies are present but not *when*; two signals with different time-orderings can share a magnitude spectrum.
- The **STFT** fixes this by transforming short windowed slices, producing $X(\tau, f)$, a function of both time and frequency (Def. 10.1). Each value is an inner product of the signal with a localized time-frequency atom.
- The **spectrogram** $|X(\tau, f)|^2$ (usually in dB) is the image of the STFT: time across, frequency up, energy as brightness. Horizontal lines are tones, vertical streaks are transients, slopes are glides.
- **Windows** (Hann, Hamming, ...) localize in time and control spectral leakage; the rectangular window is the no-taper default with the worst leakage.
- The **time/frequency tradeoff** is a law: $\Delta t\, \Delta f \approx 1$. Long windows resolve frequency (narrowband); short windows resolve time (wideband). Zero-padding refines display, not resolution.
- **Overlap** (small hop H) samples the time-frequency plane more densely and enables exact overlap-add reconstruction, without changing resolution.
- For the wrist controller, the spectrogram of IMU data exposes rhythmic motion, tempo, onsets, and tremor – but an IMU alone sees only coarse forearm motion, not individual fingers.

10.11 Exercises

Exercise 10.1. Two signals last 2 s: signal A is 100 Hz then 300 Hz (1 s each); signal B is 300 Hz then 100 Hz. Argue why their global magnitude spectra are nearly identical, and sketch the two spectrograms to show how they differ. What feature of the spectrogram encodes the ordering?

Exercise 10.2. You sample at $f_s = 2000$ Hz and want to distinguish two tones 8 Hz apart. (a) What minimum window duration T do you need? (b) How many samples N is that? (c) With that window, to what precision (in ms) can you locate a sudden onset? Comment on whether this window is good for timing fast events.

Exercise 10.3. A colleague computes a 64-sample STFT frame, zero-pads it to 1024 points, and claims the resulting smooth spectrum gives "16 times better frequency resolution." Explain precisely what zero-padding does and does not improve, and state what they would actually need to change to resolve closer tones.

Exercise 10.4. With window length $N = 512$ at $f_s = 16$ kHz, compute the time between frames for hops of (a) $H = 512$, (b) $H = 256$, (c) $H = 128$. For each, give the overlap percentage and the

spectrogram's frame rate (columns per second). Which would you choose to capture drum onsets, and why?

Exercise 10.5. Consider a single 50 Hz tone sampled at 1000 Hz over a 256-sample window. Describe how the spectrogram column differs when you use a rectangular versus a Hann window. Which shows more leakage into neighbouring bins, and which has the wider main lobe? State the tradeoff in one sentence.

Exercise 10.6. A linear chirp sweeps from 100 Hz to 900 Hz over 1 s. (a) Describe the ideal spectrogram. (b) If you use a very long window, what artifact appears along the diagonal and why? (c) If you use a very short window, what artifact appears instead? Relate both to $\Delta t\,\Delta f \approx 1$.

Exercise 10.7. A forearm IMU is sampled at 128 Hz while the user strums rhythmically at about 5 strokes per second, occasionally tapping the table sharply. (a) On a spectrogram of the gyro magnitude, what does the steady strumming look like? (b) What does a single tap look like? (c) Choose a window length N (a power of two) that resolves the 5 Hz strumming from a 10 Hz tremor, and justify it with the resolution formula.

Exercise 10.8. Your teammate proposes using the wrist-IMU spectrogram to detect *which* finger the user pressed, reasoning that "each finger must vibrate at its own frequency." Explain, citing the physics of what an IMU measures, why this will not work, and name the sensor modalities (and where they must be placed) that *can* recover finger-level information.

Exercise 10.9. Explain the constant-overlap-add (COLA) condition in your own words. Why is it required for exact reconstruction from a modified STFT? Give one example of an audio effect that is implemented by modifying STFT frames and resynthesizing with overlap-add.

Exercise 10.10. The Gabor (Gaussian-windowed) STFT achieves the theoretical minimum $\Delta t\,\Delta f = 1/(4\pi)$. (a) Why might you still prefer a Hann window in practice, even though it is not optimal? (b) In what sense is "optimal" here a statement about a *single* atom rather than about reading a whole spectrogram?

Chapter 11

Properties of the Fourier Transform and the Convolution Theorem

In the last few chapters we built the Fourier transform from the ground up: a signal is a vector, sinusoids are an orthogonal basis, and the transform is nothing more than the list of inner products of the signal with each pure frequency. That single picture – *projection onto rotating phasors* – is the whole story. Everything in this chapter is a *consequence* of it.

The danger with this material is that it is usually taught as a table to be memorized: "time shift becomes a phase ramp, scaling flips the width, convolution becomes multiplication." Memorized, the table is forgettable and joyless. Understood, every line of it is a short geometric sentence about what happens to a bundle of spinning vectors when you nudge, stretch, or smear the signal. We are going to *see* each property, then prove it in two or three lines, then use it.

And one property – the convolution theorem – is so important that the rest of digital signal processing leans on it. It is the reason filtering is fast, the reason the FFT is worth having, the reason a microcontroller on your wrist can do in microseconds what looks like a heavy integral. We will treat it as the centerpiece it is.

Focus

By the end of this chapter you should be able to look at a manipulation of a signal (delay it, speed it up, multiply it by a cosine, slide one signal across another) and *predict* what happens to its spectrum without computing a single integral – and you should understand, in your bones, why convolution in time is multiplication in frequency.

11.1 The conventions we will use

Fourier conventions are a minefield of factors of 2π. We fix one and stick to it. For a continuous signal $x(t)$ we write the transform in terms of ordinary frequency f (in hertz):

$$X(f) = \mathcal{F}\{x\}(f) = \int_{-\infty}^{\infty} x(t)\, e^{-\mathrm{j}2\pi f t}\, \mathrm{d}t, \tag{11.1}$$

$$x(t) = \int_{-\infty}^{\infty} X(f)\, e^{+\mathrm{j}2\pi f t}\, \mathrm{d}f. \tag{11.2}$$

This is the cleanest convention: no stray 2π out front, and the forward and inverse transforms are symmetric. We will occasionally switch to angular frequency $\omega = 2\pi f$ when it makes a formula prettier, and we will note when we do. We write the transform-pair relationship as

$$x(t) \;\longleftrightarrow\; X(f).$$

For discrete signals $x[n]$ the discrete-time Fourier transform (DTFT) and the DFT have the same properties with $\int$ replaced by $\sum$ and a periodic frequency axis; we will state the discrete versions where they differ in a way that bites you in practice (the DFT's shifts are *circular*, which matters enormously and we will not let you forget it).

Intuition

Hold one mental image throughout: $X(f)$ is a *recipe of phasors*. The number $X(f)$ is a complex amplitude – a length and a starting angle – telling you how much of the unit-speed rotation $e^{\mathrm{j}2\pi f t}$ to add in, and at what phase, to reconstruct $x(t)$. Every property below is the answer to "what does this operation do to all those lengths and starting angles at once?"

11.2 Linearity: the transform is a linear map

The first property is the one we already used to *define* the transform, so it almost does not feel like a property. If a and b are constants,

$$a\,x(t) + b\,y(t) \;\longleftrightarrow\; a\,X(f) + b\,Y(f).$$

The proof is one line: the integral in (11.1) is linear, so it passes through sums and scalars. Done.

Linearity is not exciting, but it is load-bearing. It is what lets us *build signals out of pieces and reason about each piece separately.* When you see a recorded forearm tremor as "a slow drift plus a 10 Hz physiological tremor plus broadband sensor noise," linearity is the permission slip that says the spectrum of the sum is the sum of the spectra – you can find and remove the drift in the frequency domain without disturbing the tremor.

Why it works

Linearity is exactly the statement that the Fourier transform is a *change of basis* (Chapter on signals as vectors). A change of basis is a linear map by construction: rewriting a vector in new coordinates cannot care whether you scaled it or added two vectors before or after

rewriting. Every linear-algebra fact about coordinate changes – and there are many – is now available to you for free.

Connection

In the linear-algebra companion, a linear map between finite-dimensional spaces is a matrix. The DFT is literally a matrix: $X = Fx$ where $F_{kn} = e^{-\mathrm{j}2\pi kn/N}$. Linearity of the transform is just the statement $F(ax + by) = aFx + bFy$, which is true of *every* matrix. The Fourier matrix happens to be a beautiful one (unitary up to scale, with rows that are the orthogonal phasor basis), but linearity asks nothing special of it.

11.3 Time shift: delay becomes a phase ramp

Now the geometry starts to pay off. Suppose we delay the signal by t_0 seconds, forming $x(t - t_0)$. What happens to its spectrum?

$$\begin{aligned}
\mathcal{F}\{x(t-t_0)\}(f) &= \int_{-\infty}^{\infty} x(t-t_0)\, e^{-\mathrm{j}2\pi ft}\, \mathrm{d}t \\
&= \int_{-\infty}^{\infty} x(u)\, e^{-\mathrm{j}2\pi f(u+t_0)}\, \mathrm{d}u \qquad (u = t - t_0) \\
&= e^{-\mathrm{j}2\pi ft_0} \int_{-\infty}^{\infty} x(u)\, e^{-\mathrm{j}2\pi fu}\, \mathrm{d}u \\
&= e^{-\mathrm{j}2\pi ft_0}\, X(f). \qquad (11.3)
\end{aligned}$$

So

$$x(t - t_0) \;\longleftrightarrow\; e^{-\mathrm{j}2\pi ft_0}\, X(f).$$

The *magnitude* of the spectrum is completely unchanged: $|e^{-\mathrm{j}2\pi ft_0}| = 1$. Only the phase changes, and it changes by a *linear ramp* in frequency: $-2\pi ft_0$.

What is really happening

Why a *ramp*, and why proportional to f? Each frequency component is a phasor spinning at f revolutions per second. To delay the whole signal by t_0, you must rewind every phasor to where it was t_0 seconds ago. A fast phasor (large f) covered more angle in t_0 seconds than a slow one, so it has to be rewound by more angle. The angle to rewind is exactly (rev/s) $\times$ (s) $\times\, 2\pi = 2\pi ft_0$. Faster phasors get more phase shift – and "proportional to f" is the definition of a straight line through the origin. A pure delay is a *linear phase* shift. That phrase will return, with force, when we design filters that must not smear the timing of a gesture.

Common trap

A constant delay gives *linear* phase. The reverse is the lesson designers care about (Figure 11.1): if a system's phase response is *not* a straight line versus frequency, then different

frequencies are delayed by different amounts, and a sharp event (a tap, a finger flick) gets smeared in time. Linear phase = pure delay = no waveform distortion. Nonlinear phase = frequency-dependent delay = smearing. This single fact is why "linear-phase FIR filters" are prized in gesture and audio work.

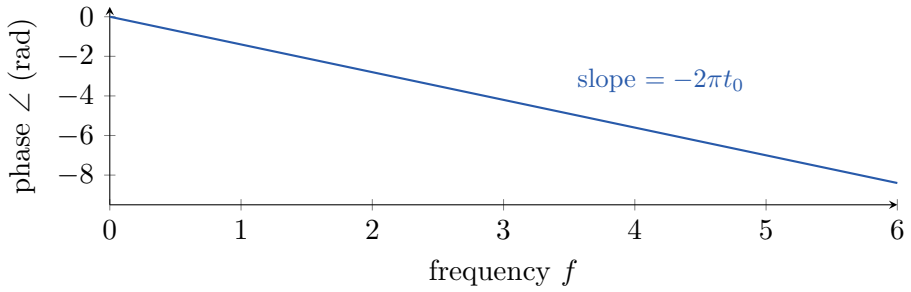


Figure 11.1: Time shift adds a phase ramp. A pure delay t_0 multiplies the spectrum by $e^{-\mathrm{j}2\pi f t_0}$, leaving magnitude untouched and tilting the phase along a straight line of slope $-2\pi t_0$. A larger delay tilts the line more steeply.

11.4 Frequency shift (modulation): multiply by a phasor, slide the spectrum

Time shift had a mirror image, and Fourier symmetry guarantees it. Multiply the signal by a complex phasor $e^{\mathrm{j}2\pi f_0 t}$:

$$\begin{aligned}\mathcal{F}\{e^{\mathrm{j}2\pi f_0 t}\,x(t)\}(f) &= \int_{-\infty}^{\infty} x(t)\,e^{\mathrm{j}2\pi f_0 t}\,e^{-\mathrm{j}2\pi f t}\,\mathrm{d}t \\ &= \int_{-\infty}^{\infty} x(t)\,e^{-\mathrm{j}2\pi (f-f_0) t}\,\mathrm{d}t = X(f-f_0). \end{aligned} \tag{11.4}$$

Multiplying by a phasor in time *slides the entire spectrum* up by f_0:

$$e^{\mathrm{j}2\pi f_0 t}\,x(t) \;\longleftrightarrow\; X(f-f_0).$$

This is the exact dual of the time-shift property – delay in time is a phase ramp in frequency; a phase ramp in time (which is what $e^{\mathrm{j}2\pi f_0 t}$ is) is a shift in frequency. The two formulas are the same statement viewed from the two sides of the transform.

11.4.1 Real modulation and the cosine

Multiplying by a real cosine is the practical case. Using $\cos(2\pi f_0 t) = \frac{1}{2}(e^{\mathrm{j}2\pi f_0 t} + e^{-\mathrm{j}2\pi f_0 t})$ and linearity:

$$\cos(2\pi f_0 t)\,x(t) \;\longleftrightarrow\; \tfrac{1}{2}X(f-f_0) + \tfrac{1}{2}X(f+f_0).$$

The spectrum is *copied* – one copy slid up to $+f_0$, one slid down to $-f_0$, each at half height. This is amplitude modulation, and the picture (Figure 11.2) is worth memorizing.

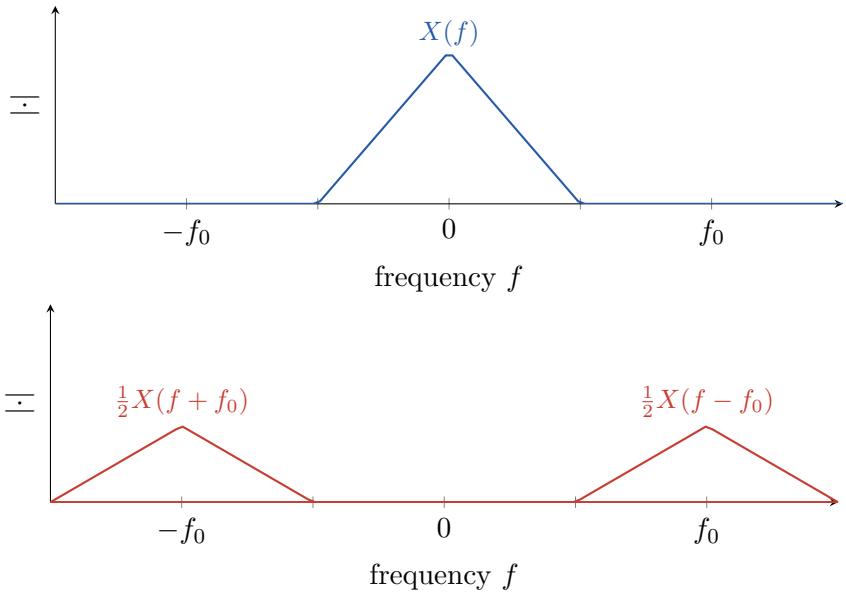


Figure 11.2: Modulation by a cosine. Top: the baseband spectrum $X(f)$ of a slowly varying signal. Bottom: after multiplying by $\cos(2\pi f_0 t)$, the spectrum is copied to $\pm f_0$ at half height. The shape is preserved; it is only *relocated*.

Application

This is how AM radio works, how an SDR tunes a station, and how I/Q demodulation pulls a signal down to baseband (Chapter on modulation and I/Q). It is also how a sensor's spectrum gets corrupted: if a motor or a switching power supply injects a tone at f_0 into your wrist IMU's supply rail, any real fluctuation $x(t)$ riding on that rail appears in your data as a copy of $X(f)$ *moved up to* f_0. Recognizing modulation in a spectrum – "two symmetric copies of the same shape" – is a debugging superpower.

11.5 Scaling: stretch in time, squeeze in frequency

Speed the signal up or slow it down by a factor $a > 0$, forming $x(at)$.

$$\begin{aligned}\mathcal{F}\{x(at)\}(f) &= \int_{-\infty}^{\infty} x(at)\, e^{-\mathrm{j}2\pi f t}\,\mathrm{d}t \\ &= \frac{1}{a}\int_{-\infty}^{\infty} x(u)\, e^{-\mathrm{j}2\pi (f/a) u}\,\mathrm{d}u \qquad (u = at) \\ &= \frac{1}{a}\, X\!\left(\frac{f}{a}\right). \end{aligned} \tag{11.5}$$

For general real $a \neq 0$ (allowing time reversal),

$$x(at) \;\longleftrightarrow\; \frac{1}{|a|}\, X\!\left(\frac{f}{a}\right).$$

If $a > 1$ you compressed the signal in time ($x(at)$ happens faster), and the spectrum stretched out

and shrank in height. If $a < 1$ you stretched the signal out in time, and the spectrum narrowed and grew taller (Figure 11.3). Time and frequency pull in opposite directions.

Intuition

This is the mathematical heart of the uncertainty principle, and you can feel it without any equations. Clap your hands: a sharp, brief sound (narrow in time) has a broad, hissy spectrum. Hum a long steady note (wide in time): its spectrum is a sharp spike. You cannot be brief in time *and* sharp in frequency at the same time. The scaling theorem is the exact accounting: width-in-time times width-in-frequency is a constant. Squeeze one, the other bulges.

Toward the wrist tracker

The scaling law sets a hard expectation for the wrist controller. Suppose you want to distinguish a slow wrist roll from a fast finger flick. The flick is *brief* in time, so by scaling it is *broad* in frequency – its energy spreads across many bins. The slow roll is *long* in time and so *narrow* in frequency. This is good news and a warning at once. Good: the two gestures live in different frequency regions and are separable. Warning: to resolve a brief flick you need a short analysis window, which by the very same law gives you *coarse* frequency resolution. You cannot have fine time *and* fine frequency resolution simultaneously; the spectrogram chapter is the art of choosing the trade-off per gesture.



Figure 11.3: Scaling. Compressing a Gaussian in time (left, dashed, $x(2t)$) makes its spectrum wider and shorter (right, dashed). Narrow in time forces broad in frequency.

11.6 Duality: the transform is almost its own inverse

Look again at the forward and inverse transforms, (11.1) and (11.2). They differ only in the sign of the exponent. This near-symmetry has a beautiful consequence: *any* transform pair, read backwards, is (almost) another transform pair. Precisely, if $x(t) \longleftrightarrow X(f)$, then

$$X(t) \longleftrightarrow x(-f).$$

That is, take the function that was your *spectrum*, feed it in as a signal, and its spectrum is your original signal, flipped. (If you use the angular convention a 2π appears; in the hertz convention (11.1) the duality is this clean.)

Duality is a labor-saving device of the first order: every pair you ever derive secretly gives you a second one for free.

Example 11.1 (The rect and the sinc, both directions)**.** The rectangular pulse of width T, $\mathrm{rect}(t/T)$, has transform

$$\mathrm{rect}(t/T) \;\longleftrightarrow\; T\,\mathrm{sinc}(fT),$$

where $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$. A box in time is a sinc in frequency. Now apply duality: a sinc in *time* must be a box in *frequency*. And indeed it is – a perfect sinc-shaped signal has a perfectly flat, brick-wall lowpass spectrum. This is exactly the ideal lowpass filter, and the reason an ideal filter is non-causal (the sinc extends to $t = -\infty$): you get it for free from duality, no second integral required.

Why it works

Why does duality hold at all? Because the only difference between analysis and synthesis – between measuring how much of each phasor is present and adding the phasors back up – is which way the phasors spin ($e^{-\mathrm{j}}$ vs. $e^{+\mathrm{j}}$). Flip the spin and analysis becomes synthesis. So computing the spectrum of a spectrum just runs synthesis with the spin flipped, which reconstructs the original signal, reversed. The symmetry of the formulas is not a coincidence; it reflects that projecting onto a basis and reassembling from that basis are the same operation in opposite directions.

11.7 A small table, now understood not memorized

We can now collect the properties. Read each line as a sentence about phasors, not as a fact to drill.

Property	**Time**	**Frequency**
Linearity	$a\,x(t) + b\,y(t)$	$a\,X(f) + b\,Y(f)$
Time shift	$x(t - t_0)$	$e^{-\mathrm{j}2\pi f t_0}\,X(f)$
Freq. shift	$e^{\mathrm{j}2\pi f_0 t}\,x(t)$	$X(f - f_0)$
Scaling	$x(at)$	$\frac{1}{\lvert a\rvert}X(f/a)$
Duality	$X(t)$	$x(-f)$
Conjugation	$x^*(t)$	$X^*(-f)$
Convolution	$(x * y)(t)$	$X(f)\,Y(f)$
Multiplication	$x(t)\,y(t)$	$(X * Y)(f)$

The conjugation line is worth a word, because it explains a feature you have already seen in every spectrum of a real signal. If $x(t)$ is real, then $x^*(t) = x(t)$, so $X^*(-f) = X(f)$: the spectrum has *conjugate symmetry*. Its magnitude is even and its phase is odd. That is why we usually plot only the positive-frequency half of a real signal's spectrum – the negative half is its mirror image and carries no new information.

The last two lines are the convolution theorem and its dual. They deserve a whole section, and the rest of the chapter.

History

Joseph Fourier (1768–1830) introduced his series in the 1807 manuscript *On the Propagation of Heat in Solid Bodies*, claiming that *any* function could be written as a sum of sines and cosines. The committee judging it – Lagrange, Laplace, Legendre – was skeptical to the point of hostility; Lagrange in particular doubted that sinusoids could represent a function with corners. They were wrong about the sweeping claim's limits but right that it needed care, and the controversy drove a century of analysis (Dirichlet, Riemann, Lebesgue) that pinned down exactly when the series converges. The properties in this chapter are the practical residue of that long argument: once you trust the representation, manipulating signals becomes algebra on their spectra.

11.8 Convolution, recalled geometrically

We will treat convolution in depth in its own chapter, but we need its picture now. The convolution of x and h is

$$(x * h)(t) = \int_{-\infty}^{\infty} x(\tau)\, h(t-\tau)\, \mathrm{d}\tau.$$

The mechanical recipe: flip h, slide it to position t, multiply it point-by-point against x, and add up (integrate) the overlap (Figure 11.4). The result at t is "how much of x is seen through a flipped-and-shifted copy of h."

What is really happening

Forget the flip for a moment and think of what convolution is *for*. h is a little shape – an impulse response, a smear, a blur kernel. Convolving x with h stamps a copy of h at every point of x, scaled by the value of x there, and adds all the stamps up. A spike in x becomes a copy of h; a smooth ramp in x becomes a smeared ramp. Convolution is *smearing x with the shape h*. Filtering, blurring, echo, averaging, the response of any linear time-invariant system – all of it is one signal smeared by another.

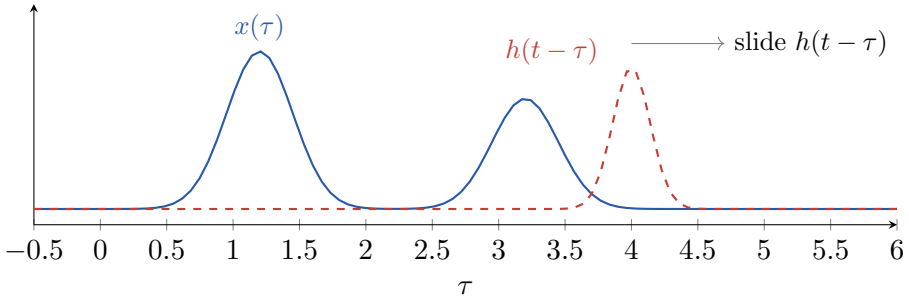


Figure 11.4: The sliding picture of convolution. The flipped kernel $h(t-\tau)$ (red, dashed) slides along $x(\tau)$ (blue). At each position t the output is the area of the product – the overlap. Convolution smears x by the shape of h.

Computing this overlap integral at every t is expensive. For two length-N discrete signals it costs on the order of N^2 multiply-adds: for each of N output positions you sum N products. For a one-second window of IMU data at a modest rate this is already thousands of operations *per output sample*. The convolution theorem is about to make that cost collapse.

11.9 The convolution theorem

Theorem 11.1 (Convolution theorem)**.** *If $x(t) \longleftrightarrow X(f)$ and $h(t) \longleftrightarrow H(f)$, then*

$$(x * h)(t) \;\longleftrightarrow\; X(f)\,H(f).$$

Convolution in the time domain is pointwise multiplication *in the frequency domain.*

Proof. Take the transform of the convolution and swap the order of integration (both integrals are over all of $\mathbb{R}$, so this is legal when the signals are well-behaved, e.g. absolutely integrable):

$$\begin{aligned}\mathcal{F}\{x * h\}(f) &= \int_{-\infty}^{\infty}\left[\int_{-\infty}^{\infty} x(\tau)\,h(t-\tau)\,\mathrm{d}\tau\right] e^{-\mathrm{j}2\pi f t}\,\mathrm{d}t \\ &= \int_{-\infty}^{\infty} x(\tau)\left[\int_{-\infty}^{\infty} h(t-\tau)\,e^{-\mathrm{j}2\pi f t}\,\mathrm{d}t\right]\mathrm{d}\tau. \qquad (11.6)\end{aligned}$$

The inner integral is the transform of a *shifted* h, which by the time-shift property (11.3) is $e^{-\mathrm{j}2\pi f\tau}H(f)$. Substitute:

$$\begin{aligned}&= \int_{-\infty}^{\infty} x(\tau)\,e^{-\mathrm{j}2\pi f\tau}\,H(f)\,\mathrm{d}\tau \\ &= H(f)\int_{-\infty}^{\infty} x(\tau)\,e^{-\mathrm{j}2\pi f\tau}\,\mathrm{d}\tau = H(f)\,X(f). \qquad (11.7)\end{aligned}$$

□

Notice the proof is *built out of the time-shift property*. That is no accident, and it points straight at the intuition.

Why it works

Why convolution is multiplication – the eigenfunction view. Feed a single pure phasor $e^{\mathrm{j}2\pi f t}$ into a convolution with h:

$$(e^{\mathrm{j}2\pi f\cdot} * h)(t) = \int h(\tau)\,e^{\mathrm{j}2\pi f(t-\tau)}\,\mathrm{d}\tau = e^{\mathrm{j}2\pi f t}\underbrace{\int h(\tau)\,e^{-\mathrm{j}2\pi f\tau}\,\mathrm{d}\tau}_{H(f)} = H(f)\,e^{\mathrm{j}2\pi f t}.$$

The phasor comes out *unchanged in frequency*, only *scaled* by the single complex number $H(f)$. Phasors are the *eigenfunctions* of convolution, and $H(f)$ is the eigenvalue at frequency f. Now recall what the Fourier transform does: it writes x as a sum (integral) of exactly these phasors. Convolution acts on each phasor independently, multiplying it by its own eigenvalue $H(f)$. So in the phasor coordinates – the frequency domain – convolution is just "scale each coordinate by $H(f)$," which is pointwise multiplication. The tangle of the overlap integral untangles completely in the basis that diagonalizes it.

Connection

This is the same phenomenon as diagonalizing a matrix in linear algebra. A convolution is a linear, shift-invariant operator; the phasors $e^{j2\pi ft}$ are its eigenvectors; $H(f)$ is the spectrum of eigenvalues. The Fourier transform is the change of basis that diagonalizes *every* shift-invariant operator at once. In the DFT, this is literal: a circular convolution is multiplication by a circulant matrix, and *every* circulant matrix is diagonalized by the DFT matrix F. The convolution theorem is the eigenvalue decomposition of circulant matrices, written in signal-processing clothing.

Why it works

Why this is the most useful fact in DSP. Three reasons, each enormous.

1. *It turns design into shaping.* Designing a filter *is* choosing $H(f)$ – a curve that is near 1 where you want to keep signal and near 0 where you want to kill it. Convolution then realizes that curve. You think in the picture you actually care about (which frequencies survive) instead of in the opaque time-domain kernel.

2. *It makes filtering fast.* The overlap integral costs $O(N^2)$. Multiplication in frequency costs $O(N)$. With the FFT to get in and out of the frequency domain at $O(N \log N)$, the whole filtering job drops from $O(N^2)$ to $O(N \log N)$. For a length-1024 block that is roughly a hundredfold speedup; for longer ones, far more.

3. *It unifies the field.* Filtering, correlation, matched detection, deconvolution (dividing by $H(f)$ to undo a known smear), the response of any LTI system, even multiplication of polynomials and big integers – all are convolutions, and all become multiplications. One theorem, the whole toolbox.

11.9.1 The dual: multiplication in time is convolution in frequency

By duality (Section 11.6), the convolution theorem has a twin:

$$x(t)\,y(t) \;\longleftrightarrow\; (X * Y)(f).$$

Multiplying two signals in time *convolves* their spectra. We already met a special case: modulation by a cosine (Section 11.4) multiplied $x(t)$ by $\cos(2\pi f_0 t)$, and the result was the spectrum copied to $\pm f_0$. That is exactly convolution of $X(f)$ with the two spikes that make up the cosine's spectrum. The whole modulation picture is one instance of the dual theorem.

Common trap

This dual is also where *windowing* comes from, and it is the single most common source of confusion in spectral work. Whenever you analyze a finite chunk of a signal you have implicitly multiplied it by a rectangular window in time. By the dual theorem, your computed spectrum is the true spectrum *convolved* with the window's transform – a sinc. That convolution smears every sharp spectral line into a sinc shape with ringing side-lobes ("spectral leakage"). You did not measure the true spectrum; you measured it smeared by the window. Choosing a gentler window (Hann, Hamming) is choosing a narrower, lower-side-lobe smearing kernel. The spectrogram and spectral-estimation chapters live and die by this.

11.10 Fast filtering: the convolution theorem at work

Let us turn the theorem into an algorithm. To filter x with kernel h:

1. Transform: $X = \mathcal{F}\{x\}$, $H = \mathcal{F}\{h\}$ (in practice, FFT).
2. Multiply pointwise: $Y[k] = X[k]\,H[k]$.
3. Inverse transform: $y = \mathcal{F}^{-1}\{Y\}$.

Listing 11.1: Fast filtering via the convolution theorem.

```python
import numpy as np

def fast_filter(x, h):
    """Filter signal x with kernel h using the convolution theorem.
    Returns the full linear convolution (length len(x)+len(h)-1)."""
    n = len(x) + len(h) - 1
    nfft = 1 << (n - 1).bit_length()       # next power of two -> fast FFT
    X = np.fft.rfft(x, nfft)               # real FFT, O(n log n)
    H = np.fft.rfft(h, nfft)
    y = np.fft.irfft(X * H, nfft)[:n]      # multiply in freq, then invert
    return y

# sanity check against direct (slow) convolution
x = np.random.randn(2000)
h = np.array([0.2, 0.2, 0.2, 0.2, 0.2])    # 5-point moving average
assert np.allclose(fast_filter(x, h), np.convolve(x, h))
```

Common trap

Circular vs. linear convolution – the trap that gets everyone once. The DFT does *not* compute the ordinary (linear) convolution. It computes *circular* convolution: it treats the signal as one period of a periodic signal, so the kernel that runs off the right edge wraps around and corrupts the left edge. The cure is zero-padding. Pad both signals out to length at least $\text{len}(x) + \text{len}(h) - 1$ before the FFT, and the wrap-around lands only in the padding, where it does no harm. The code above does exactly this – the line `nfft` chooses a length large enough that the circular result equals the linear one. Forget the pad and you will see mysterious garbage at the start of your filtered signal.

Intuition

Is the FFT route always faster? No – there is a crossover. For a very short kernel (say a 5-tap average) direct convolution is cheaper per output sample than two FFTs and an inverse. The frequency route wins decisively once the kernel gets long (tens to hundreds of taps) or you must filter a long stream. For streaming, the standard trick is *overlap-add* (or overlap-save): chop the long input into blocks, FFT-filter each block, and stitch the overlapping tails back together. That keeps the $O(N \log N)$ advantage while bounding latency and memory – exactly what you need on an embedded target.

Build it (hardware)

On a microcontroller at the wrist you rarely FFT-filter the raw IMU stream; sample rates are low (say 100–500 Hz) and kernels are short, so a direct FIR loop wins and has predictable, jitter-free timing. Where the frequency route earns its keep is in *feature extraction*: every analysis frame you take a real FFT of a short window (CMSIS-DSP's `arm_rfft_fast_f32` on a Cortex-M4F does a 256-point real FFT in well under a millisecond) and read band energies straight off $|X[k]|^2$. Rule of thumb for the build: *filter* the time stream with a short FIR (cheap, low latency), and use the *FFT* to *describe* each window for the classifier. Two different jobs, two different tools, both justified by this one chapter.

11.11 Correlation: convolution's close cousin

Correlation answers a different but related question: *how much does x look like y when y is slid to lag τ?* The cross-correlation is

$$r_{xy}(\tau) = \int_{-\infty}^{\infty} x^*(t)\, y(t+\tau)\, \mathrm{d}t.$$

It is convolution *without the flip* (with a conjugate on x). The flip is precisely the difference: convolution flips one signal, correlation does not. So $r_{xy}(\tau) = (x^*(-\cdot) * y)(\tau)$, and the convolution theorem immediately gives the *correlation theorem*:

$$r_{xy}(\tau) \;\longleftrightarrow\; X^*(f)\, Y(f).$$

Cross-correlate in time = multiply by the conjugate spectrum in frequency. As with filtering, this makes correlation fast: FFT both signals, multiply one by the conjugate of the other, inverse FFT.

Intuition

Why a conjugate and not a plain product? Because correlation measures *alignment*. In the frequency domain, $X^*(f)\,Y(f)$ has magnitude $|X||Y|$ (how much energy both share at frequency f) and phase $\angle Y - \angle X$ (how far out of step they are at f). When x and y are the same signal shifted by τ_0, the time-shift property says their phases differ by exactly the ramp $-2\pi f\tau_0$; multiplying by the conjugate *cancels* their common shape and leaves precisely that ramp, whose inverse transform is a spike at τ_0. Correlation finds the lag by letting all frequencies vote, and the conjugate is what makes the votes line up.

The special case $y = x$ gives the *autocorrelation* $r_{xx}(\tau)$, whose transform is $X^*(f)X(f) = |X(f)|^2$ – the *power spectrum*. This is the Wiener–Khinchin theorem in miniature: the autocorrelation and the power spectrum are a Fourier pair. It is the backbone of pitch detection, of spectral estimation, and of measuring how "rhythmic" a noisy signal is.

Connection

For a random signal, the autocorrelation $r_{xx}(\tau) = \mathbb{E}[x(t)\,x(t+\tau)]$ is an *expectation* (probability companion), and its Fourier transform is the power spectral density. This is the bridge between this chapter's deterministic algebra and the statistics of noise: the convolution/correlation machinery carries over verbatim to random signals, with integrals reinterpreted as

expectations. We build this out in the chapter on noise and random signals.

Application

Matched filtering. To detect a known template $s(t)$ buried in noise, correlate the incoming signal against s. The output peaks at the lag where the template best aligns – and among all linear filters this maximizes the signal-to-noise ratio at the detection instant (provably optimal for white noise). GPS receivers correlate against known pseudo-random codes; radar correlates the echo against the transmitted chirp; and a gesture recognizer can correlate a live IMU window against a stored "tap" template to flag candidate taps. By the correlation theorem, every one of these is a multiply-by-conjugate in the frequency domain, hence FFT-fast.

Toward the wrist tracker

For the wrist controller, correlation is how you turn "does this window contain gesture g?" into a number. Record a clean prototype of each coarse gesture (double-tap, wrist twist, snap) and, frame by frame, compute the normalized cross-correlation peak between the live window and each prototype. A high, well-localized peak is a detection; its lag tells you *when*. This is robust and cheap, and it degrades gracefully – exactly the property you want on a device that must never fire a wrong note in the middle of a performance.

Know its ceiling, though. Template correlation on a wrist-worn IMU detects *coarse, whole-hand* motions well. It cannot recover which individual finger moved or by how much: the IMU sits proximal to the fingers and simply does not see independent finger joint angles. Distinguishing finger-level events needs electromyography (EMG) or force myography (FMG) sensing the forearm muscles, and *continuous* finger pose needs sensors on the fingers themselves (rings, gloves). Correlation makes the gestures the IMU *can* see reliable; it does not conjure information the sensor never captured.

11.12 Worked examples

Example 11.2 (Delaying a pulse)**.** A rectangular pulse $p(t) = \text{rect}(t/T)$ has spectrum $P(f) = T\,\text{sinc}(fT)$, which is real (the pulse is centered at 0, hence even). Now delay it by $t_0 = T/2$ so it starts at 0 and ends at T. By the time-shift property the new spectrum is

$$e^{-\text{j}2\pi f(T/2)}\,T\,\text{sinc}(fT).$$

The *magnitude* $|T\,\text{sinc}(fT)|$ did not move at all – delaying a signal never changes which frequencies are present or how strong they are. What changed is the phase, which now carries a $-\pi fT$ ramp encoding "this pulse arrives $T/2$ seconds late." All timing information in a signal lives in the *phase* of its spectrum; the magnitude is timing-blind.

Example 11.3 (Convolving two boxes gives a triangle)**.** Convolve a width-T box with itself. In time, sliding one box across the other makes the overlap grow linearly, peak at full overlap, then shrink – a *triangle* of width $2T$. Now check it through the convolution theorem. Each box transforms to $T\,\text{sinc}(fT)$, so the convolution transforms to $[T\,\text{sinc}(fT)]^2 = T^2\,\text{sinc}^2(fT)$. And indeed the known transform of a triangle is sinc^2. Two ways to the same place: slide-and-overlap in time, or square

the sinc in frequency. The frequency route also explains a feature you might not have predicted – because $\text{sinc}^2 \geq 0$ everywhere, the triangle's spectrum has *no sign changes*, unlike the box's sinc, which dips negative. Squaring in frequency lifted those negative lobes up. This is exactly why a triangular (Bartlett) window leaks less than a rectangular one: its spectral side-lobes are the *square* of the rectangle's, hence much smaller.

Example 11.4 (Smoothing as lowpass filtering)**.** A 5-point moving average has kernel $h[n] = \frac{1}{5}(\delta[n] + \delta[n-1] + \cdots + \delta[n-4])$. Its DTFT is a sum of phasors – a Dirichlet kernel – whose magnitude is

$$|H(f)| = \frac{1}{5}\left|\frac{\sin(5\pi f)}{\sin(\pi f)}\right|,$$

with f in cycles per sample. It equals 1 at $f = 0$ (DC passes untouched, since the weights sum to 1) and falls off toward higher f, with nulls where $5\pi f$ is a nonzero multiple of π (Figure 11.5). So convolving with this kernel *multiplies* the input spectrum by a lowpass curve: slow trends survive, fast wiggles are attenuated, and any frequency landing exactly on a null is annihilated. "Averaging smooths the data" and "averaging is a lowpass filter" are the same sentence read in the two domains – and the convolution theorem is the dictionary between them. This is precisely the smoother you would run on a jittery wrist-IMU axis before extracting features.

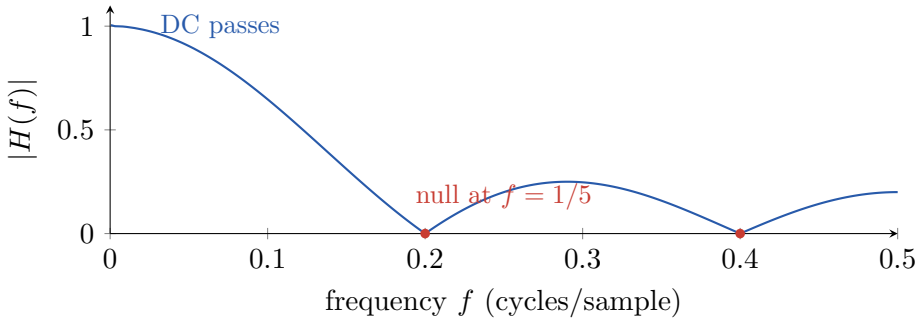


Figure 11.5: Magnitude response of the 5-point moving average, $|H(f)| = \frac{1}{5}|\sin(5\pi f)/\sin(\pi f)|$. It equals 1 at DC, falls off toward higher frequencies, and has nulls at $f = 1/5$ and $f = 2/5$ cycles/sample. Averaging is a lowpass filter.

Walking question

Suppose you compute the cross-correlation of two recordings of the *same* gesture taken a fraction of a second apart, and the correlation peak sits at lag $\tau = +12$ samples. What does that tell you, and what would you read off the *phase* of $X^*(f)Y(f)$ to confirm it? (Answer: the second recording lags the first by 12 samples; the phase should be the linear ramp $-2\pi f \cdot 12$ across frequency – timing lives in phase slope, and a clean straight ramp is the signature of a pure delay between two otherwise identical signals.)

11.13 Exercises

Exercise 11.1. Using only the time-shift and linearity properties, find the Fourier transform of $x(t) = \delta(t-2) + \delta(t+2)$. Show that the result is real and express it as a single cosine in f. Explain geometrically why a symmetric pair of impulses gives a purely real (cosine) spectrum.

Exercise 11.2. A signal $x(t)$ has transform $X(f)$. Write the transform of $y(t) = x(t)\cos(2\pi f_0 t)\cos(2\pi f_0 t)$ in terms of X. (Hint: apply the modulation property twice, or use a double-angle identity first.) How many copies of X appear, and at what frequencies and heights?

Exercise 11.3. Prove the scaling property for $a < 0$ directly from the definition, being careful with the limits of integration when you substitute $u = at$, and confirm that the $1/|a|$ (not $1/a$) factor is correct. What does $a = -1$ do to a signal and to its spectrum?

Exercise 11.4. Starting from the pair $\mathrm{rect}(t/T) \leftrightarrow T\,\mathrm{sinc}(fT)$, use duality to write the transform of $\mathrm{sinc}(t/B)$. Sketch its magnitude. Why does this make sinc the ideal lowpass filter, and why can such a filter never run in real time?

Exercise 11.5. Let h be a 3-point kernel $[1, -2, 1]$. Compute $H(f)$ (DTFT) and identify the filter type from its magnitude. Where are its zeros, and what does the kernel do to a constant (DC) input and to a slowly varying ramp? Relate your answer to the second derivative.

Exercise 11.6. You filter a length-1000 signal with a length-50 kernel using the FFT, but you set the FFT length to exactly 1000 instead of zero-padding. Describe precisely what corruption appears in the output and where. How long must the FFT be to avoid it, and why?

Exercise 11.7. Show that the autocorrelation of any real signal is even, $r_{xx}(-\tau) = r_{xx}(\tau)$, and is maximized at $\tau = 0$. Then show, using the correlation theorem, that its Fourier transform $|X(f)|^2$ is real and nonnegative. Why must a valid power spectrum be nonnegative?

Exercise 11.8. Estimate the operation counts for filtering a length-N signal with a length-M kernel (a) directly and (b) via FFT (one N-point FFT each for signal and kernel, N complex multiplies, one inverse FFT). For $N = 4096$, find the kernel length M at which the FFT method becomes cheaper. State your assumptions.

Exercise 11.9. A wrist IMU axis is contaminated by a 50 Hz tone from nearby mains wiring, riding on top of the gesture signal. In the frequency domain, what is the cleanest way to remove it, and what time-domain operation does that correspond to? What artifact does an abrupt "notch" (setting one bin to zero) create in the time domain, and how does the convolution theorem predict it?

Exercise 11.10. Two microphones (or two IMUs) record the same event with an unknown relative delay. Explain step by step how to recover the delay using one FFT per channel, a conjugate multiply, and one inverse FFT, and how to read the delay off the result. What goes wrong if the two recordings are not actually shifted copies of each other but merely similar?

Exercise 11.11. (Open-ended.) Argue both sides of the claim "the convolution theorem is the most useful fact in DSP." List three concrete tasks it makes possible or fast, and one situation where reaching for it is the wrong move (where direct time-domain processing is simpler or faster). Tie your wrong-move example to the wrist controller.

Part III

Systems and Filtering

Chapter 12

Linear Time-Invariant Systems

So far we have spent our energy on *signals* – objects that carry information through time. A microphone trace, an accelerometer stream, the EMG crackle from a forearm muscle: these are all signals. But signals rarely sit still. We feed them through things. A microphone signal passes through a tone control. An accelerometer reading passes through a smoothing filter before we trust it. The raw EMG from your wrist passes through a rectifier and an envelope detector before any gesture is recognized.

Each of those "things" is a *system*: a box that takes a signal in and produces a signal out. This chapter is about a very special, very common, and almost magically well-behaved class of systems – the **linear time-invariant** (LTI) systems. The whole edifice of classical signal processing – filters, the frequency response, convolution, the transfer function – rests on two innocent-looking assumptions. We will see that those two assumptions are so powerful that they pin a system down *completely*: tell me how an LTI system reacts to a single tap, and I can tell you how it reacts to *any* input whatsoever.

Focus

A **system** is a map T that turns an input signal x into an output signal $y = T\{x\}$. We will write this as

$$x[n] \longrightarrow \boxed{T} \longrightarrow y[n]$$

for discrete time, or $x(t) \to y(t)$ for continuous time. The chapter's central claim: if T is linear and time-invariant, then it is fully described by one signal – its response to an impulse – and it acts on every sinusoid simply by scaling and phase-shifting it.

We will work mostly in discrete time, because that is what runs on the microcontroller strapped to the wrist. Everything has a continuous-time twin; we point those out as we go.

12.1 Systems: the input–output view

Forget, for a moment, what is inside the box. A system is defined entirely by its behavior: which output it assigns to each input. This is the *black box* or *input–output* view, and it is the right altitude from which to start.

Example 12.1 (Some systems, with their rules)**.** Each of these maps an input sequence $x[n]$ to an output sequence $y[n]$.

- **Amplifier:** $y[n] = 3\,x[n]$. Scale by a constant.
- **Delay:** $y[n] = x[n-2]$. Shift two samples to the right.
- **Two-point averager:** $y[n] = \frac{1}{2}\big(x[n] + x[n-1]\big)$.
- **Accumulator:** $y[n] = \sum_{k=-\infty}^{n} x[k]$ (a running sum).
- **Squarer:** $y[n] = x[n]^2$.
- **Rectifier:** $y[n] = |x[n]|$ (the heart of EMG envelope detection).
- **Time-warp:** $y[n] = x[2n]$ (decimation / downsampling).

These look superficially similar – each is "some formula for y in terms of x." But they split sharply into two camps once we ask the two questions that organize this entire chapter:

1. Is the system **linear**? (Does it respect scaling and adding?)
2. Is it **time-invariant**? (Does it treat all moments alike?)

The amplifier, delay, and averager pass both tests. The squarer and rectifier fail linearity. The time-warp fails time-invariance. By the end of the chapter you will see exactly why those failures matter – and why the systems that pass both tests are the ones we can analyze with Fourier methods.

Intuition

Think of a system as a *machine that reshapes signals*. A filter that removes hiss reshapes the signal by deleting its high-frequency content. A reverb reshapes by smearing energy forward in time. The two LTI properties are exactly the conditions under which this reshaping can be understood, frequency by frequency, without the frequencies interfering with one another.

12.2 Linearity: scaling and superposition

The first property is the one you already know from linear algebra, transplanted to signals. Recall (from the companion linear-algebra book) that a map T is *linear* when it commutes with the two vector-space operations: scaling and addition.

Definition 12.1 (Linearity)**.** A system T is **linear** if for all inputs x_1, x_2 and all scalars a, b (real or complex),

$$T\{\,a\,x_1[n] + b\,x_2[n]\,\} \;=\; a\,T\{x_1[n]\} + b\,T\{x_2[n]\}.$$

This single equation bundles two ideas:

- **Homogeneity (scaling):** doubling the input doubles the output. $T\{a\,x\} = a\,T\{x\}$.
- **Additivity (superposition):** the response to a sum is the sum of the responses. $T\{x_1 + x_2\} = T\{x_1\} + T\{x_2\}$.

Connection

In the linear-algebra book a linear map satisfied $T(a\mathbf{u}+b\mathbf{v}) = aT\mathbf{u}+bT\mathbf{v}$ and was represented by a *matrix*. A discrete signal *is* a vector (Chapter 3 treated $x[n]$ as a vector indexed by n), and a linear system *is* a (possibly infinite) matrix acting on it: $y = \mathbf{H}\,x$. Time-invariance, coming next, is the extra condition that makes this matrix have a beautiful repeating structure – and that structure is what convolution is.

Why it works

Why obsess over superposition? Because it is the engine of *decompose and reassemble*. If I can write a complicated input as a sum of simple pieces,

$$x = \sum_k c_k\,\phi_k,$$

and I know what the system does to each simple piece ϕ_k, then linearity hands me the answer for free:

$$y = T\{x\} = \sum_k c_k\,T\{\phi_k\}.$$

The entire Fourier program is exactly this: choose the ϕ_k to be sinusoids, because (we will prove) the system treats each sinusoid in the most trivial possible way. Linearity is the permission slip that lets us add the answers back up.

Testing for linearity

To *prove* a system linear you must verify the definition in general. To *disprove* it, a single counterexample suffices.

Example 12.2 (The amplifier is linear)**.** $T\{x\}[n] = 3x[n]$. Then $T\{ax_1 + bx_2\}[n] = 3(ax_1[n] + bx_2[n]) = a\,(3x_1[n]) + b\,(3x_2[n]) = a\,T\{x_1\}[n] + b\,T\{x_2\}[n]$. Linear. ✓

Example 12.3 (The squarer is not linear)**.** $T\{x\}[n] = x[n]^2$. Try scaling: $T\{2x\}[n] = (2x[n])^2 = 4x[n]^2$, but $2\,T\{x\}[n] = 2x[n]^2$. Since $4x[n]^2 \neq 2x[n]^2$ in general, homogeneity fails. The squarer is nonlinear.

Common trap

"Linear" does *not* mean "the graph is a straight line," and it does *not* mean "contains no constant term." Consider $y[n] = x[n] + 5$. Its input–output graph is a straight line, yet it is *not* linear: $T\{0\} = 5 \neq 0$, but every linear system must send the zero signal to the zero signal (set $a = b = 0$ in Definition 12.1). Such "line plus offset" maps are called *affine*, not linear. The offset breaks superposition: $T\{x_1+x_2\} = x_1+x_2+5$, while $T\{x_1\}+T\{x_2\} = x_1+x_2+10$.

12.3 Time invariance: the system has no clock

The second property says the system does not care *when* you run the experiment. Shift the input in time, and the output shifts by the same amount, unchanged in shape.

Definition 12.2 (Time invariance)**.** A system T is **time-invariant** if, whenever $y[n] = T\{x[n]\}$, then for every integer shift n_0,

$$T\{\, x[n - n_0] \,\} \;=\; y[n - n_0].$$

In words: delaying the input by n_0 delays the output by n_0 and does nothing else.

Intuition

A time-invariant system has *no internal clock and no calendar*. It does not know whether it is Monday or whether the signal arrived early or late. Run the same input today or tomorrow and you get the same output, just slid along the time axis to match. A microphone preamp is time-invariant. A system whose gain ramps up over the first second after power-on is *not* – it knows how long it has been running.

Testing for time invariance

The recipe is mechanical and worth doing carefully, because the most common student error lives right here.

Walking question

Recipe. To test time invariance, compute two things and compare:

1. **Shift-then-process.** Let $x_{\text{sh}}[n] = x[n - n_0]$, push it through the system, get $T\{x_{\text{sh}}\}[n]$.
2. **Process-then-shift.** Take the original output $y[n] = T\{x\}[n]$ and shift it: $y[n - n_0]$.

If the two agree *for all* x and all n_0, the system is time-invariant. The trap is in step 1: wherever an n appears in the system's formula *attached to* x, you must replace $x[\,\cdot\,]$ by the shifted input, but any *free* n (an explicit dependence on the current time) stays put – and that mismatch is exactly what breaks time invariance.

Example 12.4 (The delay is time-invariant)**.** $T\{x\}[n] = x[n - 2]$, so $y[n] = x[n - 2]$. *Shift-then-process:* feeding $x[n - n_0]$ gives output $x[(n - 2) - n_0] = x[n - 2 - n_0]$. *Process-then-shift:* $y[n - n_0] = x[(n - n_0) - 2] = x[n - 2 - n_0]$. They match. Time-invariant. ✓

Example 12.5 (The downsampler is not time-invariant)**.** $T\{x\}[n] = x[2n]$, so $y[n] = x[2n]$. *Shift-then-process:* feed $x[n - n_0]$; the output is $x[2n - n_0]$. *Process-then-shift:* $y[n - n_0] = x[2(n - n_0)] = x[2n - 2n_0]$. These differ ($x[2n - n_0]$ vs. $x[2n - 2n_0]$) for, e.g., $n_0 = 1$. So downsampling is *not* time-invariant: a one-sample input delay does not produce a one-sample output delay. (This is why resampling, Chapter 22, needs its own careful theory.)

Example 12.6 (A multiplier by n is not time-invariant)**.** $T\{x\}[n] = n\,x[n]$. Here the free n out front is the culprit. Shift-then-process: $n\,x[n - n_0]$. Process-then-shift: $(n - n_0)\,x[n - n_0]$. The factors n and $(n - n_0)$ disagree – the system weights later samples more heavily, so it manifestly has a clock.

12.4 Putting them together: LTI

Definition 12.3 (LTI system)**.** A system is **LTI** if it is both linear (Definition 12.1) and time-invariant (Definition 12.2).

Of the zoo in Example 12.1: the amplifier, delay, two-point averager, and accumulator are LTI; the squarer and rectifier fail linearity; the downsampler fails time invariance. The averager and accumulator deserve a second look because they are the prototypes of the two great families of filters we build later (FIR and IIR).

Example 12.7 (The two-point averager is LTI)**.** $y[n] = \frac{1}{2}(x[n] + x[n-1])$. Linearity follows because it is a sum of two linear pieces (scaled, delayed copies of x), and any sum of linear operations is linear. Time invariance follows because each piece – a scaling and a fixed delay – is time-invariant, and the sum of time-invariant operations is time-invariant. This "sum of scaled delays" template is the giveaway: any such combination is automatically LTI, and *every* LTI system turns out to be exactly a (possibly infinite) sum of scaled delays. That is the punchline of the next two sections.

12.5 The impulse: a single tap

To unlock LTI systems we need one special signal: the **unit impulse** (also called the unit sample, or Kronecker delta in discrete time),

$$\delta[n] = \begin{cases} 1, & n = 0, \\ 0, & n \neq 0. \end{cases}$$

It is a single spike of height 1 at time zero and silence everywhere else – the cleanest possible "tap" you can give a system.

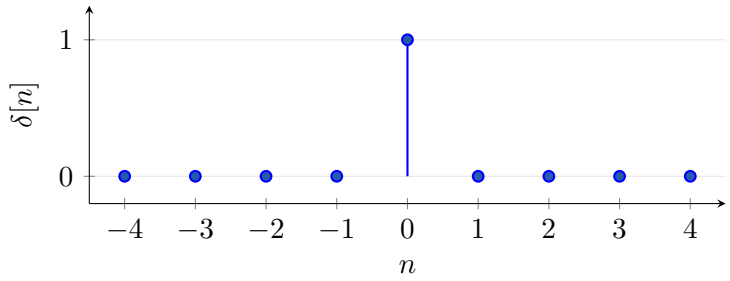


Figure 12.1: The unit impulse $\delta[n]$: a single sample of height one at $n = 0$.

The reason the impulse is special is a *sifting* identity that says any signal is a sum of shifted, scaled impulses.

Formal

The sifting (sampling) representation. Every discrete signal can be written as

$$x[n] \;=\; \sum_{k=-\infty}^{\infty} x[k]\,\delta[n-k].$$

Why this is obviously true: fix an n. On the right-hand side, the term $\delta[n-k]$ is zero unless $k = n$, in which case it equals 1. So the entire sum collapses to the single survivor $x[n]\cdot 1 = x[n]$. The sum just re-reads the signal back to you, one sample at a time.

That equation looks like bookkeeping, but read it as a *decomposition*: the signal x is a weighted superposition of building blocks $\delta[n-k]$, where the weight on the block sitting at time k is precisely the sample value $x[k]$. We have decomposed x into impulses. We are now exactly one application of linearity away from the whole theory.

History

The continuous-time cousin of $\delta[n]$ is the *Dirac delta* $\delta(t)$, introduced by the physicist Paul Dirac in the late 1920s as an idealized "unit mass concentrated at a point." Mathematicians were uneasy – it is not a function in the ordinary sense – until Laurent Schwartz put it on rigorous footing with the *theory of distributions* in the 1940s. The discrete $\delta[n]$ has none of these difficulties: it is an ordinary, finite-valued sequence. One more reason we like living in discrete time on the microcontroller.

12.6 The impulse response and the convolution sum

Now feed the impulse into an LTI system and watch what comes out. Give it a name.

Definition 12.4 (Impulse response)**.** The **impulse response** of a system T is its output when the input is the unit impulse:

$$h[n] \;=\; T\{\delta[n]\}.$$

The impulse response is a signal – the system's "signature," the shape of its reaction to a single tap. A bell's impulse response is its ringing decay. A room's impulse response is its reverberation (clap once, record what comes back). Here is the theorem that makes $h[n]$ the most important object in the chapter.

Theorem 12.1 (An LTI system is its impulse response)**.** *If T is LTI with impulse response $h[n] = T\{\delta[n]\}$, then for* every *input $x[n]$ the output is the* ***convolution***

$$y[n] \;=\; (x * h)[n] \;=\; \sum_{k=-\infty}^{\infty} x[k]\, h[n-k].$$

Why it works

The proof is just our two properties, in order. Start from the sifting representation and apply the system:

$$y[n] = T\{x[n]\} = T\left\{\sum_{k} x[k]\,\delta[n-k]\right\}.$$

Step 1 – linearity. The system commutes with the (weighted) sum. The weights $x[k]$ are constants as far as n is concerned, so they pull out:

$$y[n] = \sum_{k} x[k]\; T\{\delta[n-k]\}.$$

Step 2 – time invariance. The system's response to the *shifted* impulse $\delta[n-k]$ is just the

shifted impulse response, $T\{\delta[n-k]\} = h[n-k]$. Substitute:

$$y[n] = \sum_k x[k]\, h[n-k] = (x * h)[n]. \qquad \blacksquare$$

That is the whole story. Linearity let us break the input into impulses and add up the answers; time invariance told us the answer to each shifted impulse is a shifted copy of one master response h.

What is really happening

What is really happening: an LTI system *stamps a copy of its impulse response at every input sample*, scaled by that sample's value, and adds all the stamped copies up. Picture a drummer hitting a ringing cymbal. Each hit at time k with strength $x[k]$ launches one cymbal-ring $x[k]\, h[n-k]$. The sound you hear at time n is the sum of all the still-ringing decays from every earlier hit. Convolution is "overlapping echoes, added."

This is a remarkable amount of leverage. A system can be infinitely complicated as a rule, but if it is LTI then *one experiment* – measure $h[n]$ – predicts its response to every conceivable input. We measure h, we store it, and convolution does the rest.

Convolution as stamped, scaled, added echoes

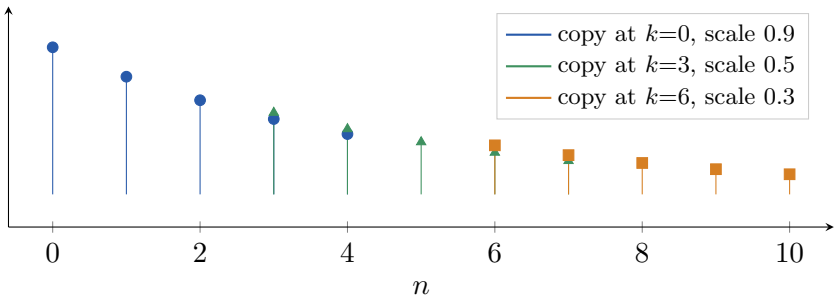


Figure 12.2: Each input sample launches a scaled, shifted copy of the impulse response $h[n] = 0.8^n u[n]$. The output is their pointwise sum. Three input samples are shown ($x[0]$=0.9, $x[3]$=0.5, $x[6]$=0.3); a real signal launches one such echo per sample.

Reading note

Convolution gets a whole chapter of its own (Chapter 13): the flip-and-slide picture, its commutativity ($x * h = h * x$), and fast algorithms. Here we only need the *fact* that LTI output = input convolved with h. We will lean on it immediately.

Example 12.8 (Convolving by hand)**.** Let the impulse response be the two-point averager, $h[n] = \frac{1}{2}\delta[n] + \frac{1}{2}\delta[n-1]$, i.e. $h = (\frac{1}{2}, \frac{1}{2})$ at $n = 0, 1$. Let the input be the short pulse $x = (1, 3, 2)$ at $n = 0, 1, 2$. Then

$$y[n] = \sum_k x[k]\, h[n-k] = \tfrac{1}{2}x[n] + \tfrac{1}{2}x[n-1].$$

March n forward:

$$y[0] = \tfrac{1}{2}(1) + \tfrac{1}{2}(0) = 0.5,$$
$$y[1] = \tfrac{1}{2}(3) + \tfrac{1}{2}(1) = 2.0,$$
$$y[2] = \tfrac{1}{2}(2) + \tfrac{1}{2}(3) = 2.5,$$
$$y[3] = \tfrac{1}{2}(0) + \tfrac{1}{2}(2) = 1.0.$$

So $y = (0.5, 2.0, 2.5, 1.0)$. Note the output is one sample *longer* than the input: convolving a length-L signal with a length-M impulse response gives length $L + M - 1$. The averaging visibly smooths the jumpy input.

12.7 FIR and IIR: finite versus infinite memory

The *length* of $h[n]$ tells you how long the system remembers a tap.

- If $h[n]$ has only finitely many nonzero samples, the system has **Finite Impulse Response (FIR)**. The averager $h = (\frac{1}{2}, \frac{1}{2})$ is FIR: a tap is forgotten after one sample.
- If $h[n]$ rings forever, the system has **Infinite Impulse Response (IIR)**. The accumulator's impulse response is the unit step $h[n] = u[n] = 1$ for $n \geq 0$ – it never forgets.

FIR systems are computed directly as a finite convolution sum. IIR systems are usually *not* computed from h (which is infinite!) but from a compact recursive rule – a difference equation, the subject of Section 12.9.

12.8 Causality and stability

Two further properties decide whether a system can be *built* and whether it can be *trusted*. They translate into clean conditions on $h[n]$.

Causality: no peeking at the future

Definition 12.5 (Causality)**.** A system is **causal** if the output at time n depends only on the input at times n and earlier – never on future inputs. For an LTI system this is equivalent to

$$h[n] = 0 \quad \text{for all } n < 0.$$

Why it works

Why does causality reduce to $h[n] = 0$ for $n < 0$? In the convolution sum $y[n] = \sum_k x[k]\, h[n-k]$, a future input $x[k]$ with $k > n$ contributes only through $h[n-k]$ where $n - k < 0$. If h is zero on negative indices, those future terms vanish, so $y[n]$ uses only present and past inputs. Conversely, if $h[n_*] \neq 0$ for some $n_* < 0$, then poking the input at a future time changes a present output – non-causal.

Any system that must run in *real time* – the firmware on the wrist band reacting to motion as it happens – *must* be causal; it physically cannot read samples that have not arrived. Non-causal

systems are not useless, though: when you process a recorded file offline, "the future" is already on disk, and symmetric (non-causal) filters with zero phase distortion become available. We exploit exactly this in Chapter 17.

Common trap

A symmetric smoother like $y[n] = \frac{1}{3}(x[n-1]+x[n]+x[n+1])$ is *not* causal – it reads $x[n+1]$, one sample into the future. You can still implement it in real time, but only by accepting a one-sample *delay*: compute and emit $y[n]$ once $x[n+1]$ is in hand, i.e. output the value labeled n at clock tick $n+1$. Linear-phase filters buy their pristine phase response with exactly this kind of bulk latency. On a gesture controller that latency is part of your end-to-end "time from motion to MIDI" budget.

Stability: bounded in, bounded out (BIBO)

Definition 12.6 (BIBO stability)**.** A system is **BIBO stable** if every bounded input produces a bounded output: whenever $|x[n]| \le B_x < \infty$ for all n, there exists $B_y < \infty$ with $|y[n]| \le B_y$ for all n.

For an LTI system there is a beautifully simple test in terms of h.

Theorem 12.2 (Stability condition)**.** *An LTI system is BIBO stable if and only if its impulse response is* absolutely summable*:*

$$\sum_{n=-\infty}^{\infty} |h[n]| \;<\; \infty.$$

Why it works

($\Leftarrow$ sufficiency) Suppose $S = \sum_n |h[n]| < \infty$ and $|x[n]| \le B_x$. Bound the output directly:

$$|y[n]| = \left|\sum_k h[k]\, x[n-k]\right| \le \sum_k |h[k]|\,|x[n-k]| \le B_x \sum_k |h[k]| = B_x\, S < \infty.$$

So the output is bounded by $B_x S$ for every n – stable.
($\Rightarrow$ necessity) If $S = \infty$, we construct a bounded input that blows up. Pick the bounded input $x[n] = \operatorname{sign} h[-n]$ (which satisfies $|x[n]| \le 1$). Then at $n = 0$, $y[0] = \sum_k h[k]\, x[-k] = \sum_k h[k] \operatorname{sign} h[k] = \sum_k |h[k]| = \infty$. A bounded input produced an unbounded output – unstable.

Example 12.9 (Which of our systems are stable?)**.**

- Averager $h = (\frac{1}{2}, \frac{1}{2})$: $\sum|h| = 1 < \infty$. Stable.
- Accumulator $h[n] = u[n]$: $\sum_{n\ge 0} 1 = \infty$. *Unstable* – feed it a constant $x[n] = 1$ and the running sum grows without bound. (True of any pure integrator; in practice we tame it with feedback or leakage.)
- Decaying echo $h[n] = a^n u[n]$ with $|a| < 1$: geometric series $\sum_{n\ge 0} |a|^n = \frac{1}{1-|a|} < \infty$. Stable. With $|a| \ge 1$ it diverges – unstable.

Connection

The geometric-series threshold $|a| < 1$ is not a coincidence. In Chapter 15 we will see that a is a *pole* of the system's z-transform, and the stability condition "$|a| < 1$" becomes the elegant geometric statement *all poles lie strictly inside the unit circle*. Stability, which we just met as a summability inequality, will turn into a picture you can read at a glance from a pole–zero plot.

12.9 Difference equations: how systems are really run

Convolution describes *what* an LTI system does. But you rarely *run* an IIR system by convolving with its infinite h. Instead you run a **linear constant-coefficient difference equation** (LCCDE): a finite recipe relating a few output samples to a few input samples.

Definition 12.7 (LCCDE)**.** A linear constant-coefficient difference equation has the form

$$\sum_{i=0}^{N} a_i \, y[n-i] \;=\; \sum_{j=0}^{M} b_j \, x[n-j],$$

with constant coefficients a_i, b_j and $a_0 \neq 0$. Solving for the current output gives the directly runnable recursion

$$y[n] = \frac{1}{a_0}\left(\sum_{j=0}^{M} b_j \, x[n-j] - \sum_{i=1}^{N} a_i \, y[n-i]\right).$$

The right-hand side is a *recipe you can type into a loop*: to get the next output, take a weighted blend of recent inputs (the b_j part, the *feedforward* or FIR part) and subtract a weighted blend of recent outputs (the a_i part, the *feedback* or recursive part). The feedback is what makes the impulse response infinite from a finite rule.

Example 12.10 (A one-pole smoother: leaky integrator)**.** Take $N = 1$, $M = 0$, $a_0 = 1$, $a_1 = -a$, $b_0 = (1 - a)$:

$$y[n] = (1-a)\, x[n] + a\, y[n-1], \qquad 0 < a < 1.$$

This is the workhorse exponential smoother (the digital first-order low-pass). Its impulse response: feed $x = \delta$, assume rest ($y[n] = 0$ for $n < 0$):

$$y[0] = (1-a), \quad y[1] = a(1-a), \quad y[2] = a^2(1-a), \; \ldots \;\Rightarrow\; h[n] = (1-a)\, a^n u[n].$$

A one-line recursion produces an *infinite* geometric impulse response (Figure 12.3). That is the power – and the danger – of feedback: it is stable here only because $|a| < 1$ makes the geometric series converge (cf. Theorem 12.2).

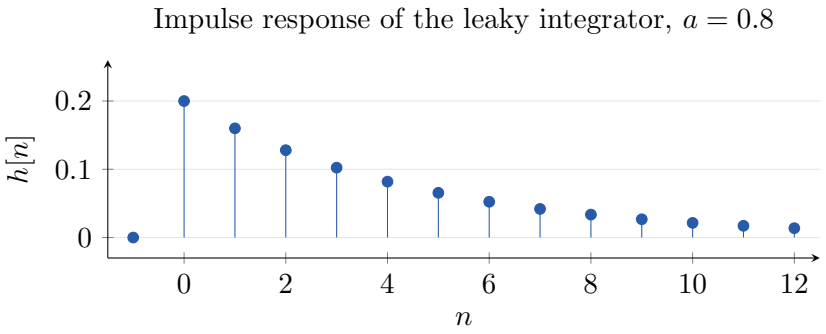


Figure 12.3: The one-line recursion $y[n] = (1-a)x[n] + a\,y[n-1]$ produces the geometric impulse response $h[n] = (1-a)a^n u[n]$ (here $a = 0.8$, so $h[0] = 0.2$). It decays but never reaches zero – an infinite impulse response from finite memory.

Listing 12.1: The leaky integrator in a loop – the cheapest useful real-time filter.

```
def leaky_integrator(x, a):
    y = 0.0
    out = []
    for xn in x:                  # one multiply-add per sample
        y = (1 - a) * xn + a * y # blend new input with smoothed history
        out.append(y)
    return out
# a near 1.0 -> heavy smoothing (long memory); a near 0 -> barely smooths.
```

Focus

FIR vs. IIR, from the difference equation. If there is no feedback ($N = 0$: no $y[n-i]$ terms on the right), the system is FIR – the output is a finite window of inputs and h is simply the list of b_j coefficients. If there *is* feedback ($N \geq 1$), the system is generally IIR. FIR is always stable (a finite sum of bounded terms) and can have exactly linear phase; IIR is cheaper (fewer taps for the same sharpness) but can go unstable and distorts phase. We will weigh this trade-off carefully in Chapter 16.

12.10 Complex exponentials are the eigenfunctions of LTI systems

We now reach the most beautiful fact in the chapter, the one that connects LTI systems to everything Fourier. Feed an LTI system not an impulse but an everlasting complex exponential, $x[n] = e^{\mathrm{j}\omega n}$, and ask what comes out.

Theorem 12.3 (Eigenfunction property)**.** *Let T be LTI with impulse response $h[n]$. For the input $x[n] = e^{\mathrm{j}\omega n}$,*

$$y[n] \;=\; H(e^{\mathrm{j}\omega})\,e^{\mathrm{j}\omega n}, \qquad \textit{where} \qquad H(e^{\mathrm{j}\omega}) \;=\; \sum_{k=-\infty}^{\infty} h[k]\,e^{-\mathrm{j}\omega k}.$$

The output is the same *complex exponential, merely multiplied by the complex number $H(e^{\mathrm{j}\omega})$.*

Why it works

Plug $x[n] = e^{\mathrm{j}\omega n}$ into the convolution sum and factor:

$$y[n] = \sum_k h[k]\, x[n-k] = \sum_k h[k]\, e^{\mathrm{j}\omega(n-k)} = e^{\mathrm{j}\omega n} \underbrace{\sum_k h[k]\, e^{-\mathrm{j}\omega k}}_{H(e^{\mathrm{j}\omega})}.$$

The shift-invariance of the exponential is the whole trick: $e^{\mathrm{j}\omega(n-k)} = e^{\mathrm{j}\omega n}\, e^{-\mathrm{j}\omega k}$ splits into the part that depends on n (pulled out front) and the part that depends on k (summed against h). The k-sum no longer depends on n, so it is just a constant scaling.

What is really happening

This is exactly the eigenvector idea from linear algebra. A matrix $\mathbf{A}$ usually rotates and stretches a vector into a new direction; but along an *eigenvector* it merely scales, $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Here the "matrix" is the LTI system T, the "eigenvectors" are the complex exponentials $e^{\mathrm{j}\omega n}$, and the "eigenvalues" are the complex numbers $H(e^{\mathrm{j}\omega})$. *Every LTI system is diagonal in the basis of complex exponentials.* That is the deep reason the Fourier transform is the natural language of filtering: it changes coordinates into the basis where every LTI system is just a list of scalings.

Connection

The function $H(e^{\mathrm{j}\omega}) = \sum_k h[k]e^{-\mathrm{j}\omega k}$ is precisely the **discrete-time Fourier transform** of the impulse response (Chapter 7). So two pictures of the same system meet here: the *time-domain* picture is the impulse response $h[n]$, and the *frequency-domain* picture is its DTFT $H(e^{\mathrm{j}\omega})$. Convolution in time (Theorem 12.1) becomes *multiplication* in frequency, $Y(e^{\mathrm{j}\omega}) = H(e^{\mathrm{j}\omega})\, X(e^{\mathrm{j}\omega})$ – the convolution theorem you met for signals now reread as a statement about systems.

The frequency response: magnitude and phase

The eigenvalue $H(e^{\mathrm{j}\omega})$ is the **frequency response**. Being complex, it carries two real pieces of information at each frequency ω:

$$H(e^{\mathrm{j}\omega}) = |H(e^{\mathrm{j}\omega})|\, e^{\mathrm{j}\angle H(e^{\mathrm{j}\omega})}.$$

- $|H(e^{\mathrm{j}\omega})|$ – the **magnitude response** – says how much the system amplifies (> 1) or attenuates (< 1) a pure tone at frequency ω.
- $\angle H(e^{\mathrm{j}\omega})$ – the **phase response** – says how much the system delays (shifts) that tone.

Because real sinusoids are sums of conjugate exponentials, $\cos(\omega n) = \frac{1}{2}(e^{\mathrm{j}\omega n} + e^{-\mathrm{j}\omega n})$, the eigenfunction property gives the practical rule: a real LTI system turns the input $\cos(\omega n)$ into

$$y[n] = |H(e^{\mathrm{j}\omega})|\, \cos\big(\omega n + \angle H(e^{\mathrm{j}\omega})\big).$$

A sinusoid in gives the same-frequency sinusoid out, rescaled in amplitude and shifted in phase – never a new frequency. This "no new frequencies" property is the signature of LTI systems and the reason nonlinear systems (the squarer, the rectifier) are so different: they manufacture harmonics.

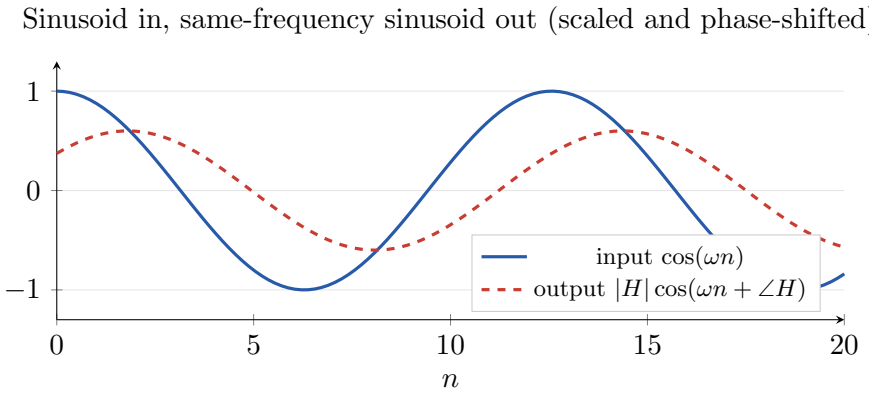


Figure 12.4: An LTI system maps a pure tone to a tone at the *same* frequency, multiplied by the magnitude $|H(e^{j\omega})|$ (here a gain of 0.6) and slid by the phase $\angle H(e^{j\omega})$ (here a lag of about one sample). The shape and frequency are untouched.

Example 12.11 (Frequency response of the two-point averager)**.** For $h = (\frac{1}{2}, \frac{1}{2})$,

$$H(e^{j\omega}) = \tfrac{1}{2} + \tfrac{1}{2}e^{-j\omega} = e^{-j\omega/2}\cos(\omega/2),$$

where the last step factors out the average phase. Hence $|H(e^{j\omega})| = |\cos(\omega/2)|$ and the phase is the pure linear delay $-\omega/2$. At DC ($\omega = 0$) the magnitude is 1 – constants pass untouched. At the highest digital frequency ($\omega = \pi$, the fastest alternation $\ldots, +1, -1, +1, \ldots$) the magnitude is $\cos(\pi/2) = 0$ – the averager kills it completely. So the two-point averager is a (gentle) low-pass filter, exactly as intuition demanded.

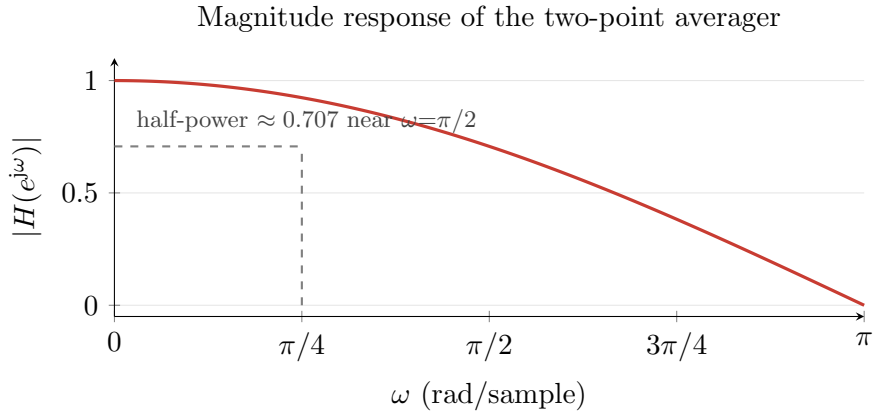


Figure 12.5: The averager passes low frequencies (gain 1 at $\omega = 0$) and rejects the fastest alternation (gain 0 at $\omega = \pi$): a simple low-pass.

The mirror image is the *first-difference* system $h = (1, -1)$, the simplest edge detector. Its magnitude response $|H(e^{j\omega})| = 2|\sin(\omega/2)|$ is zero at DC and largest at $\omega = \pi$: a high-pass that ignores slow trends and reacts to fast change (Figure 12.6). You will derive it in the exercises.

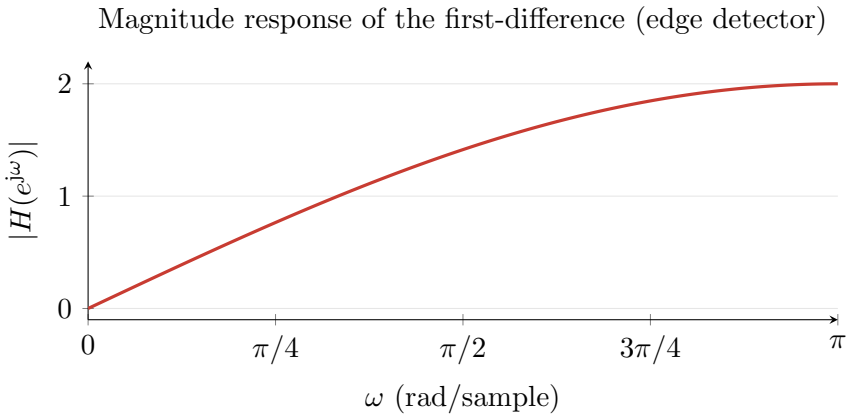


Figure 12.6: The first difference $h = (1, -1)$ rejects DC (gain 0 at $\omega = 0$) and peaks at the fastest alternation (gain 2 at $\omega = \pi$): a high-pass, the complement of the averager.

Why this matters

Step back and admire the unification. **One** object, the impulse response $h[n]$, simultaneously: (i) generates every output by convolution; (ii) decides causality by where it is nonzero; (iii) decides stability by whether it is absolutely summable; (iv) *is* the system's frequency response once you take its Fourier transform. Time-domain shape and frequency-domain behavior are two readings of a single page. Mastering LTI systems means becoming fluent in flipping between those two readings at will.

12.11 Block diagrams: drawing systems

Engineers rarely write difference equations on the workbench; they *draw* them. A block diagram is a wiring picture built from three primitives:

- an **adder** (a circle with +) that sums incoming signals;
- a **gain** (a triangle or labeled arrow) that scales by a constant;
- a **unit delay** (a box labeled z^{-1}) that holds a sample for one clock tick – this is the hardware register that gives the system its memory.

The label z^{-1} is borrowed from the z-transform (Chapter 15), where delaying by one sample corresponds to multiplying by z^{-1}. For now read it simply as "delay by one."

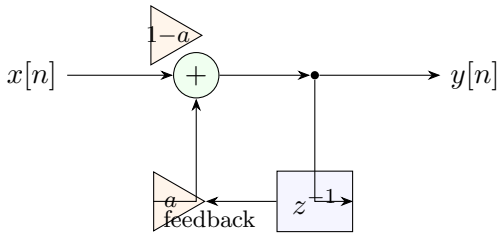


Figure 12.7: Block diagram of the leaky integrator $y[n] = (1-a)x[n] + a\,y[n-1]$. The z^{-1} block stores the previous output; the loop back through gain a is the feedback that makes the impulse response infinite.

Walking question

Read a block diagram back into an equation. Trace from the output adder backward. In Figure 12.7 the adder produces $y[n]$ by summing two paths: the input scaled by $(1-a)$, and the previous output (the z^{-1} box turns $y[n]$ into $y[n-1]$) scaled by a. Writing that sum gives $y[n] = (1-a)x[n] + a\,y[n-1]$ – back to Example 12.10. Diagram, equation, impulse response, and frequency response are four faces of one system; fluency means moving among them without friction.

12.12 Why this matters for the wrist controller

Toward the wrist tracker

Almost every processing block in the gesture controller is, by design, LTI – and we keep it that way on purpose, because LTI is the part we can reason about with the frequency response.

The LTI parts. The IMU gives accelerometer and gyroscope streams riddled with high-frequency vibration and electrical noise. The first thing we do is *low-pass* each axis – often with the very leaky integrator of Example 12.10, one multiply-add per sample, trivially cheap on an 8-bit micro. Removing the gravity component to isolate dynamic motion is a *high-pass*; isolating a forearm-tremor band is a *band-pass*. All three are LTI filters, fully described by their impulse responses, fully predictable from their frequency responses.

The deliberately non-LTI parts. Gesture recognition itself is *not* LTI, and that is the point. To turn EMG into a "pinch detected" event we *rectify* ($y = |x|$, nonlinear – Example 12.3's cousin) and then threshold (also nonlinear). Nonlinearity is exactly what lets us collapse a continuous waveform into a discrete decision. The clean architecture is: *LTI filtering first* (where the math is friendly), *nonlinear decision last* (where we actually want new behavior, like emitting a MIDI note-on).

Causality is non-negotiable here. The controller must react as motion happens; it cannot read future samples. Every filter in the live path must be causal ($h[n] = 0$ for $n < 0$), and every sample of group delay it adds is latency between your gesture and the sound. Budget it.

Common trap

What the sensors can and cannot tell you. It is tempting to imagine an LTI filter bank that "decodes" individual finger joint angles from a single wrist IMU. It cannot, and no amount of filtering changes that. A wrist-mounted IMU measures the motion of the *wrist*; it has no observability of independent finger flexion when the wrist is still. The most an IMU-only band can do reliably is detect *coarse, discrete* gestures – a flick, a tap, a wrist rotation, a shake. Recovering finger-level activity requires sensing closer to the muscles or fingers: surface EMG or force-myography (FMG) on the forearm for discrete finger gestures, or instrumented rings / data gloves for continuous finger pose. This is consistent with the published gesture-sensing literature (e.g. Myo-style forearm EMG armbands for discrete finger gestures, and FMG cuffs for grip states). LTI filtering cleans up whatever the sensor can observe; it cannot manufacture information the sensor never captured.

Application

The same LTI toolkit reappears everywhere in this book and in the wild: *audio* – a graphic equalizer is a parallel bank of LTI band-pass filters, and a convolution reverb literally convolves your signal with the measured impulse response of a concert hall; *radio* – the channel between transmitter and receiver is modeled as an LTI system whose impulse response is the multipath profile; *biomedical* – the 50/60 Hz mains hum in an ECG is removed by an LTI notch filter; *seismology* – the earth's response to a distant quake is the convolution of the source with the LTI transfer function of the rock between. Learn LTI once, recognize it forever.

12.13 A continuous-time footnote

Everything above has a continuous-time mirror. A continuous LTI system is described by its impulse response $h(t) = T\{\delta(t)\}$, the output is the convolution *integral*

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau)\, h(t-\tau)\, \mathrm{d}\tau,$$

the eigenfunctions are $e^{\mathrm{j}\Omega t}$ with eigenvalue $H(\mathrm{j}\Omega) = \int h(t)e^{-\mathrm{j}\Omega t}\mathrm{d}t$ (the continuous Fourier transform of h), causality is $h(t) = 0$ for $t < 0$, and BIBO stability is $\int |h(t)|\mathrm{d}t < \infty$. The analog smoothing RC circuit in front of your ADC is a continuous-time first-order low-pass; its discrete cousin is the leaky integrator. Same geometry, integrals instead of sums.

12.14 Summary

Pause

The chapter in one breath.

- A *system* maps input signals to output signals.
- *Linear* = respects scaling and addition (superposition); *time-invariant* = no clock, shifting the input just shifts the output. *LTI* = both.

- Any signal is a sum of scaled, shifted impulses (sifting). Apply linearity then time invariance and you get: *LTI output = input convolved with the impulse response* $h[n]$ (Theorem 12.1). One measured h predicts all behavior.
- *Causal* $\Leftrightarrow h[n] = 0$ for $n < 0$ (required for real time). *BIBO stable* $\Leftrightarrow \sum_n |h[n]| < \infty$.
- FIR (finite h, no feedback, always stable, linear phase possible) vs. IIR (infinite h, feedback, cheaper, can be unstable). Both run as difference equations.
- Complex exponentials are the *eigenfunctions* of LTI systems; the eigenvalue is the *frequency response* $H(e^{\mathrm{j}\omega})$, the DTFT of h. A sinusoid in gives the same-frequency sinusoid out, scaled by $|H|$ and phase-shifted by $\angle H$ – never a new frequency.
- Block diagrams (+, gain, z^{-1}) are the engineer's drawing of the same difference equation.

12.15 Exercises

Exercise 12.1 (Classify the zoo)**.** For each system in Example 12.1, state whether it is (a) linear and (b) time-invariant, with a one-line justification or counterexample for each. Which are LTI?

Exercise 12.2 (Affine is not linear)**.** Show that $y[n] = 2x[n] - 1$ violates Definition 12.1 by exhibiting two signals x_1, x_2 for which $T\{x_1 + x_2\} \neq T\{x_1\} + T\{x_2\}$. Then explain in one sentence why *any* nonzero constant term ruins linearity.

Exercise 12.3 (Time-variant detector)**.** Consider $y[n] = x[n]\cos(\omega_0 n)$ (amplitude modulation by a fixed carrier). Prove it is linear but *not* time-invariant. Where does the free n enter?

Exercise 12.4 (Convolve by hand)**.** Let $h = (1, -1)$ at $n = 0, 1$ (the first-difference / edge detector) and $x = (2, 2, 5, 5, 5)$ at $n = 0, \ldots, 4$. Compute $y = x * h$ by marching n forward as in Example 12.8. Where is the output largest, and why does that make sense for an edge detector?

Exercise 12.5 (Impulse response of a difference equation)**.** A system obeys $y[n] = x[n] - x[n-3]$. Find $h[n]$ by feeding $x = \delta$. Is it FIR or IIR? Causal? Stable (check $\sum |h[n]|$)?

Exercise 12.6 (Leaky-integrator impulse response)**.** For $y[n] = (1-a)x[n] + a\,y[n-1]$ with $a = 0.9$, compute $h[0], h[1], h[2], h[3]$ numerically and confirm they follow $h[n] = (1-a)a^n$. What is $\sum_{n \geq 0} h[n]$, and why does that value mean "unity DC gain"?

Exercise 12.7 (Stability boundary)**.** For $h[n] = a^n u[n]$, the system is BIBO stable iff $|a| < 1$. (a) Evaluate $\sum_n |h[n]|$ for $a = 0.5$ and for $a = 1.5$. (b) For $a = 1$ exactly (the accumulator restricted to a single pole on the boundary), is the system stable? Give a bounded input whose output is unbounded.

Exercise 12.8 (Frequency response from h)**.** For the first-difference system $h = (1, -1)$, show that $H(e^{\mathrm{j}\omega}) = 1 - e^{-\mathrm{j}\omega} = 2\mathrm{j}\, e^{-\mathrm{j}\omega/2} \sin(\omega/2)$. Sketch $|H(e^{\mathrm{j}\omega})| = 2|\sin(\omega/2)|$ over $0 \leq \omega \leq \pi$. Is it low-pass or high-pass? Connect this to its role as an edge detector.

Exercise 12.9 (Sinusoid in, sinusoid out)**.** The two-point averager (Example 12.11) has $H(e^{\mathrm{j}\omega}) = e^{-\mathrm{j}\omega/2}\cos(\omega/2)$. If the input is $x[n] = \cos(\frac{\pi}{2} n)$, write the exact output $y[n]$ including its amplitude and phase shift. By how many samples is the tone delayed?

Exercise 12.10 (Cascade of LTI systems)**.** Two LTI systems with impulse responses h_1 and h_2 are connected in series (output of the first feeds the second). Using Theorem 12.1 twice, show the overall impulse response is $h_1 * h_2$, and argue from $Y = H_2(H_1 X)$ that the overall frequency response is the *product* $H_1(e^{\mathrm{j}\omega})H_2(e^{\mathrm{j}\omega})$. Why does this prove that the *order* of two LTI blocks does not matter?

Exercise 12.11 (Nonlinearity makes harmonics)**.** Feed $x[n] = \cos(\omega_0 n)$ into the squarer $y[n] = x[n]^2$. Using $\cos^2\theta = \frac{1}{2} + \frac{1}{2}\cos(2\theta)$, show the output contains a DC term and a tone at $2\omega_0$ – a frequency *not present* in the input. Contrast this with the "no new frequencies" guarantee for LTI systems, and explain why this property is precisely what makes a rectifier useful for turning an oscillating EMG burst into a detectable envelope.

Exercise 12.12 (Draw and read a block diagram)**.** Draw the block diagram (using adders, gains, and z^{-1} blocks) for the FIR filter $y[n] = 0.25x[n] + 0.5x[n-1] + 0.25x[n-2]$. How many delay elements does it need? Then read your diagram back into the difference equation to check it. Is this filter's phase linear? (Hint: look at the symmetry of its coefficients.)

Chapter 13

Convolution

In the previous chapter we made a single, far-reaching discovery: if a system is *linear* and *time-invariant* (an LTI system), then its entire behavior is locked down by one curve — the way it responds to a single sharp poke, the impulse response. Everything the system will ever do to any input, however complicated, is already encoded in that one response.

This chapter is about cashing in that promise. We are going to derive the exact machine that turns "input plus impulse response" into "output." That machine is **convolution**. It is, without exaggeration, the central operation of signal processing. Filtering is convolution. Echo and reverb are convolution. Blurring and sharpening images is convolution. Smoothing a noisy accelerometer trace is convolution. The Fourier transform's most famous theorem is a statement about convolution. And when we eventually smooth, debounce, and feature-extract the signals coming off a wrist sensor, every one of those steps is a convolution in disguise.

Our promise for this chapter: you will not memorize the convolution formula. You will *see* it. We build it from the geometry of decomposing a signal into impulses, watch it run as a flip-shift-multiply-sum animation rendered in stills, prove its properties from that picture, and run it on real examples — a moving-average smoother and a digital echo — in both the continuous and discrete worlds.

13.1 The one idea: build any signal out of impulses

Everything flows from a deceptively simple observation. Take a discrete-time signal $x[n]$ — just a list of numbers, one per sample index. Here is a way to write it that looks like cheating but is the key to the whole subject:

$$x[n] \;=\; \sum_{k=-\infty}^{\infty} x[k]\,\delta[n-k]. \tag{13.1}$$

Read this slowly. The unit impulse $\delta[n]$ is 1 at $n = 0$ and 0 everywhere else. So $\delta[n-k]$ is a single spike sitting at position k. Multiply that spike by the number $x[k]$, and you get a spike of height $x[k]$ at position k. Now add up one such scaled spike for every k. What do you get back? Exactly the original signal. At index n, only the $k = n$ term in the sum survives (every other $\delta[n-k]$ is zero there), and it contributes $x[n]$. The equation is a tautology — but a *generative* one.

Intuition

A signal is a stack of impulses. Picture $x[n]$ as a bar chart. Each bar is one impulse: it lives at a particular time, and it has a particular height. The whole signal is nothing more than that collection of weighted, shifted spikes, laid side by side. We have not changed the signal at all — we have only chosen to *see* it as a sum of elementary pieces. Choosing the right elementary pieces is the recurring trick of this entire book. With sinusoids we got the Fourier transform. With impulses we are about to get convolution.

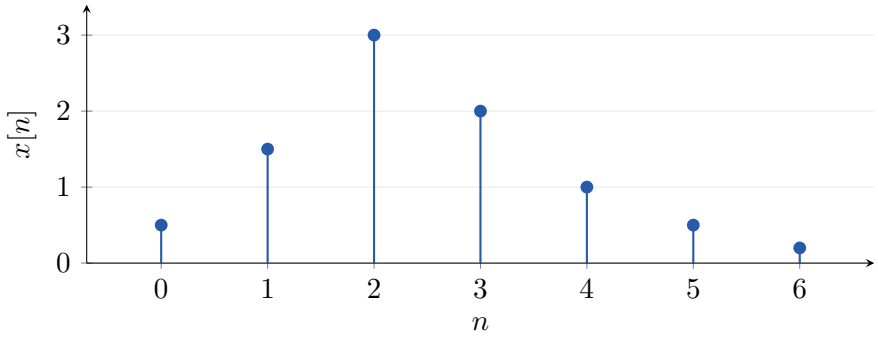


Figure 13.1: A discrete signal is a row of impulses. Each stem is one term $x[k]\,\delta[n-k]$ of Eq. (13.1): a spike at position k with height $x[k]$. The signal *is* the collection.

Why bother rewriting a signal as a sum of impulses? Because of what an LTI system does with a sum of impulses. This is where linearity and time-invariance earn their keep.

13.2 From impulse response to convolution

Let $T\{\cdot\}$ denote our LTI system: feed it an input, it produces an output. Define its **impulse response** as the output when the input is a single unit impulse at the origin:

$$h[n] \;:=\; T\{\delta[n]\}. \tag{13.2}$$

This is the one curve from the previous chapter. Now watch the system handle an arbitrary input $x[n]$, using the two properties in turn.

Step 1 — time-invariance. If a poke at time 0 produces $h[n]$, then a poke at time k produces the same shape, just shifted to start at k:

$$T\{\delta[n-k]\} \;=\; h[n-k].$$

The system does not care *when* you poke it; it only cares about the delay between poke and observation.

Step 2 — linearity. The input is a sum of scaled, shifted impulses, Eq. (13.1). A linear system lets us push T through the sum and through the scalars: the response to a sum is the sum of responses,

and the response to a scaled input is the scaled response. So

$$\begin{aligned} y[n] \;=\; T\{x[n]\} &= T\left\{\sum_k x[k]\,\delta[n-k]\right\} \\ &= \sum_k x[k]\,T\{\delta[n-k]\} \qquad \text{(linearity)} \\ &= \sum_k x[k]\,h[n-k]. \qquad \text{(time-invariance)} \end{aligned} \tag{13.3}$$

That last line *is* the convolution sum. We did not invent it; we were forced into it. Once you accept that a signal is a stack of impulses, and that the system responds to each impulse with a shifted copy of h, the only thing left to do is add up all those copies, each weighted by how strong its impulse was. That sum is convolution.

What is really happening

Convolution is a superposition of echoes. Every input sample $x[k]$ throws a scaled copy of the impulse response h into the future (and present), landing at time k. At any output time n, you stand still and collect the contributions from *all* those copies that happen to reach n. A copy launched at time k contributes $x[k]\,h[n-k]$ — its launch strength $x[k]$ times the impulse response evaluated at the elapsed time $n-k$. The output is the running total of overlapping echoes. Hold onto this "superposition of shifted, scaled copies" picture; it is the truest mental model of convolution.

Definition 13.1 (Discrete convolution)**.** The convolution of two discrete-time signals x and h is the signal

$$(x * h)[n] \;=\; \sum_{k=-\infty}^{\infty} x[k]\,h[n-k]. \tag{13.4}$$

We also write $y = x * h$. The output of an LTI system is the convolution of its input with its impulse response: $y = x * h$.

The continuous-time story is identical, with the sum becoming an integral. A continuous signal is a "sum" of infinitely many infinitely thin impulses, the sifting property of the Dirac delta:

$$x(t) \;=\; \int_{-\infty}^{\infty} x(\tau)\,\delta(t-\tau)\,\mathrm{d}\tau. \tag{13.5}$$

Run the same two-step argument — time-invariance turns $\delta(t-\tau)$ into $h(t-\tau)$, linearity pushes T inside the integral — and you land on the continuous convolution.

Definition 13.2 (Continuous convolution)**.**

$$(x * h)(t) \;=\; \int_{-\infty}^{\infty} x(\tau)\,h(t-\tau)\,\mathrm{d}\tau. \tag{13.6}$$

Connection

Equation (13.1) is the impulse-basis cousin of writing a vector in coordinates. In the linear-algebra companion, a vector $\mathbf{x} = \sum_k x_k \, \mathbf{e}_k$ is a weighted sum of standard basis vectors $\mathbf{e}_k$. Here the "basis vectors" are shifted impulses $\delta[n-k]$, and $x[k]$ are the coordinates. An LTI system is a linear map; once you know what it does to each basis element ($\delta[n-k] \mapsto h[n-k]$), linearity fixes what it does to everything. Convolution is just that linear map written out in the impulse basis. The companion's inner-product view returns later: each output sample $y[n]$ is the inner product of the input with a flipped, shifted copy of h.

13.3 Flip, shift, multiply, sum — the moving picture

Definition 13.1 has a $h[n-k]$ in it, and that minus sign in front of k is the whole visual drama of convolution. Let us decode it as a recipe you can run by hand. To compute the single output value $y[n]$ at one chosen index n:

1. **Flip.** Take $h[k]$ and reflect it about $k = 0$ to get $h[-k]$. The impulse response is now running backwards.

2. **Shift.** Slide the flipped response right by n to get $h[n-k]$. It is now anchored at the output time n.

3. **Multiply.** For every k, multiply $x[k]$ by the overlapping value $h[n-k]$.

4. **Sum.** Add up all those products. The total is $y[n]$.

Then move to the next output index, shift the flipped response one more step, and repeat. The flipped response *slides across* the input; at each position you take the running dot product of the overlap. That sliding-overlap motion is convolution made visible.

Why it works

Why flip? The minus sign is not arbitrary, and it is not the same as correlation (which uses $h[n+k]$ and does *not* flip). It comes straight from the echo picture. An impulse launched at time k has been traveling for $n-k$ samples by the time we observe at n. Early launches (k small) have been traveling longest, so they show us the *late* part of h; recent launches show us the early part of h. When you line the responses up against the input axis k, that "oldest launch sees latest h" bookkeeping reverses the order of h — it flips. The flip is causality and elapsed-time bookkeeping made geometric.

Let us run the machine on a tiny example we can check by hand, then draw two frames of the slide.

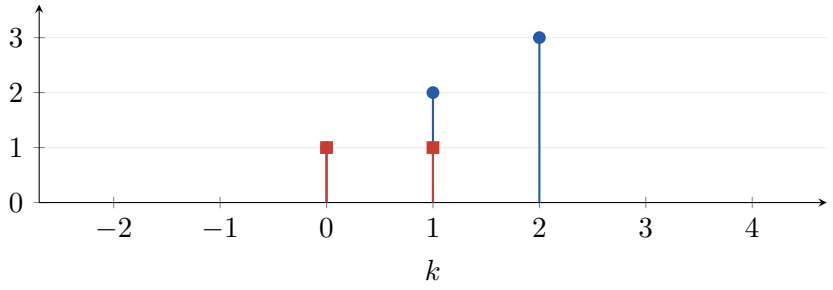
Example 13.1 (Convolving two short sequences)**.** Let $x = [\,1,\ 2,\ 3\,]$ at indices $n = 0, 1, 2$, and let $h = [\,1,\ 1\,]$ at indices $n = 0, 1$ (a two-tap summer). The output has length $3 + 2 - 1 = 4$. Using

$y[n] = \sum_k x[k]\,h[n-k]$:

$$\begin{aligned} y[0] &= x[0]h[0] = 1\cdot 1 = 1,\\ y[1] &= x[0]h[1] + x[1]h[0] = 1\cdot 1 + 2\cdot 1 = 3,\\ y[2] &= x[1]h[1] + x[2]h[0] = 2\cdot 1 + 3\cdot 1 = 5,\\ y[3] &= x[2]h[1] = 3\cdot 1 = 3. \end{aligned}$$

So $x * h = [1,\ 3,\ 5,\ 3]$. Notice the output is longer than either input: convolution *spreads* a signal out. Notice too that the same numbers appear if you treat it as polynomial multiplication: $(1+2z+3z^2)(1+z) = 1+3z+5z^2+3z^3$. That is not a coincidence; we return to it in Section 13.7.

Frame 1: computing $y[1]$ — flipped h anchored at $n = 1$



Frame 2: computing $y[2]$ — flipped h slid one step right

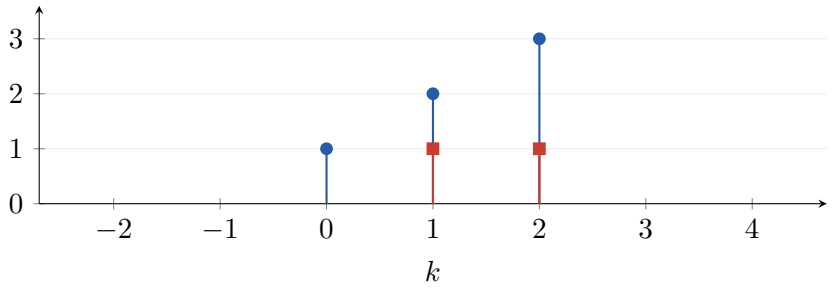


Figure 13.2: Two stills of the slide for Example 13.1. Blue stems are the input $x[k]$; red squares are the flipped-and-shifted response $h[n-k]$. To get $y[n]$, multiply the overlapping blue and red values and add. Frame 1 ($n = 1$): overlap at $k = 0, 1$ gives $1{+}2 = 3$. Frame 2 ($n = 2$): overlap at $k = 1, 2$ gives $2{+}3 = 5$. Sliding the red pattern one notch right and re-summing produces the whole output.

Walking question

Before reading on, compute $y[0]$ and $y[3]$ for Example 13.1 by sliding the red pattern in Figure 13.2 all the way left (so it overlaps only $x[0]$) and all the way right (only $x[2]$). You should get 1 and 3. When does the slide start producing nonzero output, and when does it stop? The answer — overlap begins when the leading edge of h first touches x and ends when its trailing edge leaves — is exactly why the output length is $\text{len}(x) + \text{len}(h) - 1$.

13.4 Worked example: the moving average

The most common smoother on earth is the moving average, and it is a convolution. An M-point moving average replaces each sample with the average of itself and its $M-1$ predecessors:

$$y[n] \;=\; \frac{1}{M}\sum_{m=0}^{M-1} x[n-m] \;=\; (x * h)[n], \qquad h[n] = \begin{cases} 1/M, & 0 \le n \le M-1, \\ 0, & \text{otherwise.} \end{cases} \tag{13.7}$$

The impulse response is a *rectangle* of height $1/M$ and width M. Flip a rectangle and it looks the same; sliding it across the input and summing the overlap is literally "average everything under the box." That is why the moving average is sometimes called a boxcar filter.

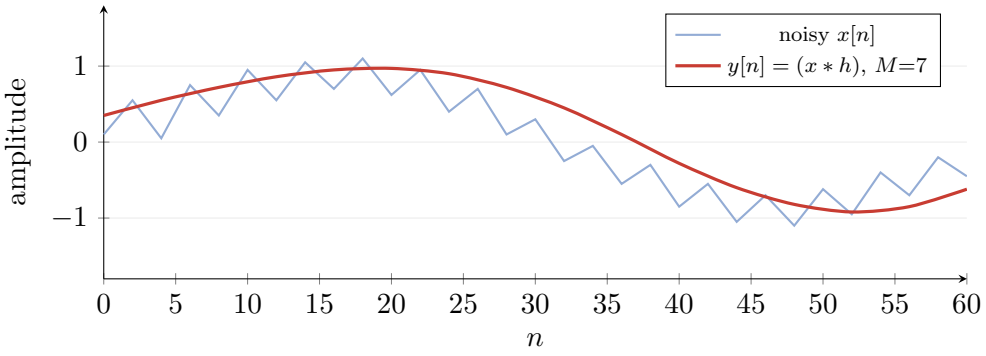


Figure 13.3: A moving average is convolution with a rectangle. The jagged blue trace is a noisy signal; the smooth red trace is the output of convolving it with a width-M boxcar. Averaging neighbors cancels uncorrelated wiggles while leaving the slow trend intact. (Traces drawn schematically to show the effect.)

Common trap

The moving average is a low-pass filter, but a mediocre one. Its frequency response is the sinc-shaped magnitude $|H(\omega)| = |\mathrm{sinc}(M\omega/2)/\,\mathrm{sinc}(\omega/2)|$ (we derive this kind of thing in the next chapter). The trap: that response has big *sidelobes* and even passes through zero at certain frequencies while letting others leak. So a boxcar smooths, but it also smears and ripples in the frequency domain. When you need real attenuation, reach for a designed filter, not a raw moving average. The moving average wins on one axis only: it is the cheapest possible low-pass, and with a running sum it costs one add and one subtract per sample regardless of M.

Connection

Why does averaging kill noise? Lean on the probability companion. If each sample is signal plus independent zero-mean noise of variance σ^2, averaging M of them leaves the signal (roughly) intact but shrinks the noise variance to σ^2/M — the variance of a mean of independent variables. So the noise standard deviation drops by $\sqrt{M}$. Convolution with a length-M boxcar buys you a $\sqrt{M}$ improvement in signal-to-noise, at the cost of blurring features narrower than M samples. That tradeoff — noise rejection versus time resolution — is the eternal tension of smoothing.

13.5 Worked example: a digital echo

Echo is the purest demonstration of the superposition-of-copies picture. A single delayed-and-attenuated repeat is the system

$$y[n] \;=\; x[n] + a\,x[n-D], \tag{13.8}$$

where D is the delay in samples and $0 < a < 1$ is how much quieter the echo is. What impulse response produces this? Feed in $\delta[n]$: the output is $\delta[n] + a\,\delta[n-D]$, so

$$h[n] = \delta[n] + a\,\delta[n-D] = \{\,1 \text{ at } n{=}0, \quad a \text{ at } n{=}D\,\}.$$

Two spikes: the direct sound and one echo. Convolving any music $x[n]$ with this two-spike h lays down a copy of the whole song at full volume, plus a second copy delayed by D samples and scaled by a. That is exactly what an echo *is* — a delayed, fainter copy summed with the original. The echo picture of convolution from Section 13.2 was not a metaphor; for this h it is literal.

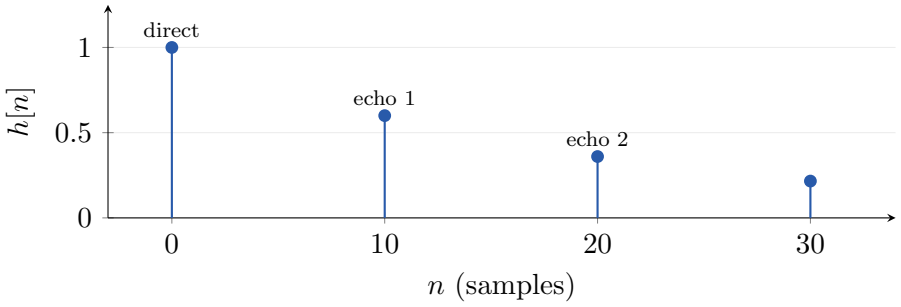


Figure 13.4: Impulse response of a feedback echo (here $D = 10$, $a = 0.6$, with the output fed back so each echo spawns the next). Each spike is one delayed, fainter copy. Convolving a sound with this comb of spikes stamps that pattern of repeats onto every moment of the sound. A single-tap echo would show just the first two spikes; a reverb's impulse response is a dense, decaying cloud of thousands.

Application

Real reverb is convolution with a measured room. To capture how a cathedral or a recording studio sounds, engineers play a known test signal (a starter pistol, a balloon pop, or a swept sine) and record the result; deconvolving the input out leaves the room's impulse response — a dense, exponentially decaying spray of reflections lasting seconds. A *convolution reverb* plugin then convolves your dry recording with that captured impulse response, transplanting the cathedral onto your voice. The same idea, run in reverse, lets acousticians diagnose flutter echoes and resonances in a hall. Convolution is how you wear a room like a costume.

13.6 Properties — and why each one is geometrically obvious

Convolution obeys a handful of algebraic laws. We could grind them out by manipulating sums, but the point of this book is to *see* them. Each falls out of either the echo picture or the slide picture.

Commutativity: $x * h = h * x$. Substitute $m = n - k$ in the sum; the roles of input and impulse response swap. Geometrically: it does not matter whether you flip-and-slide h across x or x across

h — the set of overlapping products is the same. Practically, this is liberating: "filter the signal" and "stamp the filter's pattern with the signal" compute the same thing.

Associativity: $(x * h_1) * h_2 = x * (h_1 * h_2)$. Cascading two LTI systems is itself an LTI system, and its impulse response is the convolution of the two individual responses (Figure 13.5). So a chain of filters can be collapsed into one equivalent filter — precompute $h_1 * h_2$ once and convolve with it forever after.

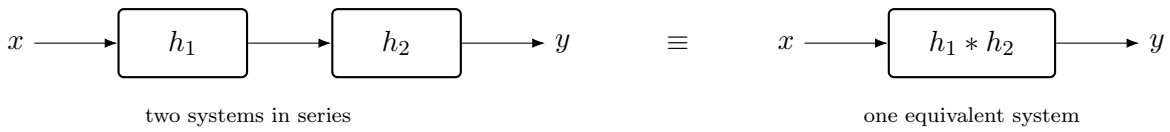


Figure 13.5: Associativity as a wiring fact. Cascading an h_1 filter into an h_2 filter is identical to a single filter whose impulse response is $h_1 * h_2$. You can precompute the combined response once and convolve with it forever after.

Distributivity over addition: $x * (h_1 + h_2) = x * h_1 + x * h_2$. Two LTI systems wired in parallel (same input, outputs summed) equal one system whose impulse response is the sum of theirs. This is just linearity again.

Identity: $x * \delta = x$. Convolving with a single unit spike at the origin returns the signal unchanged. Of course — δ is the "do-nothing" impulse response, the wire that passes its input straight through. And $x * \delta[n - D] = x[n - D]$: convolving with a shifted spike is a pure delay.

Shift: if $y = x * h$, then $x[n - D] * h = y[n - D]$. Delay the input, delay the output by the same amount. This is just time-invariance, restated.

Formal

Commutativity, made precise. Starting from $(x * h)[n] = \sum_k x[k]\, h[n - k]$, let $m = n - k$, so $k = n - m$ and the sum over all k becomes a sum over all m:

$$(x * h)[n] = \sum_m x[n - m]\, h[m] = \sum_m h[m]\, x[n - m] = (h * x)[n].$$

The same substitution works for the continuous integral. Every other property admits an equally short proof; try associativity as an exercise by writing the double sum and reordering.

Why this matters

Step back and admire the structure. Under convolution, signals form a commutative algebra: there is an addition, a commutative and associative "multiplication" $(*)$, it distributes over addition, and there is a multiplicative identity (δ). It is, structurally, exactly like ordinary number multiplication — and as Section 13.7 shows, that resemblance is not superficial. Convolution *is* multiplication, viewed in the right coordinates. The Fourier transform is the change of coordinates that makes the disguise fall away.

13.7 Convolution is polynomial (and integer) multiplication

Example 13.1 dropped a hint: convolving the coefficient lists $[1, 2, 3]$ and $[1, 1]$ gave $[1, 3, 5, 3]$, the coefficients of $(1 + 2z + 3z^2)(1 + z)$. This is a theorem, not a fluke. Encode a sequence $x[n]$ as the coefficients of a polynomial (or a power series), $X(z) = \sum_n x[n]\, z^n$. Then multiplying two such polynomials,

$$X(z)\,H(z) = \Big(\sum_k x[k]z^k\Big)\Big(\sum_m h[m]z^m\Big) = \sum_n \Big(\underbrace{\sum_k x[k]\,h[n-k]}_{(x*h)[n]}\Big)z^n,$$

collects, into the coefficient of z^n, exactly the convolution sum. The $z^k \cdot z^m = z^n$ bookkeeping $(k + m = n)$ is the same as the $n - k$ index in convolution.

Why it works

This is why the z-transform turns convolution into plain multiplication: $\mathcal{Z}\{x*h\} = X(z)\,H(z)$. It is also why the Fourier transform does the same with $z = e^{j\omega}$ — the convolution theorem says $\mathcal{F}\{x * h\} = X \cdot H$. Convolution in the time domain, multiplication in the transform domain: one operation seen in two coordinate systems. The transform is the magic decoder ring that straightens convolution's flip-and-slide into a pointwise product.

And the same logic explains why grade-school long multiplication of integers *is* convolution-with-carries: multiplying 123×11 produces the digit-convolution $[1, 3, 5, 3]$ before you carry. Convolution is one of the oldest operations you know, wearing a new uniform.

History

The word "convolution" (from Latin *convolvere*, to roll together) is younger than the operation. Integrals of the form $\int f(\tau)g(t-\tau)\,d\tau$ appear in the work of d'Alembert, Euler, Laplace, and Poisson in the 1700s and 1800s — in solutions of differential equations and in probability, where the density of a sum of two independent random variables is the convolution of their densities. The German term *Faltung* ("folding," capturing the flip) was standard well into the 20th century before the English "convolution" won out. The flip-and-slide picture, and the realization that it is *the* operation of linear systems, crystallized with the rise of LTI systems theory and communications engineering in the mid-1900s.

13.8 Edges and the cost of convolution

Two practical facts bite the moment you convolve real, finite signals.

Length and edges. Convolving a length-N signal with a length-M impulse response gives a length-$(N + M - 1)$ result. The extra $M - 1$ samples are the "tail" where the impulse response is sliding off the end. Near both edges the response only partially overlaps the data, so those output samples are based on fewer terms — they are less trustworthy. In practice you choose a boundary policy: zero-pad (assume the signal is 0 outside, the default and the source of those tails), repeat the edge value, reflect the signal, or wrap it around (circular convolution, which the DFT performs). For a causal real-time filter you keep only the first N outputs and accept a startup transient.

Cost. Direct convolution costs about $N \cdot M$ multiply-adds: for each of N outputs you sum M products. When both signals are long this is brutal — $O(N^2)$. The convolution theorem rescues us: transform both signals with the FFT ($O(N \log N)$), multiply pointwise, inverse-transform. For long filters this "fast convolution" is dramatically cheaper, and it is how serious reverb and large-kernel image filters are actually computed.

Listing 13.1: Discrete convolution three ways: by hand, with NumPy, and the FFT route.

```python
import numpy as np

def conv_direct(x, h):
    N, M = len(x), len(h)
    y = np.zeros(N + M - 1)
    for n in range(N + M - 1):
        for k in range(N):              # flip-shift-multiply-sum
            m = n - k
            if 0 <= m < M:
                y[n] += x[k] * h[m]
    return y

x = np.array([1., 2., 3.])
h = np.array([1., 1.])
print(conv_direct(x, h))                # [1. 3. 5. 3.]
print(np.convolve(x, h))                # [1. 3. 5. 3.]  -- same

# Fast convolution via the FFT (zero-pad to avoid circular wrap):
L = len(x) + len(h) - 1
Y = np.fft.ifft(np.fft.fft(x, L) * np.fft.fft(h, L)).real
print(np.round(Y, 6))                   # [1. 3. 5. 3.]  -- same again
```

Build it (hardware)

On an Arduino-class microcontroller you will convolve with *short* impulse responses — a 5-to-15-tap smoother on each IMU axis, say — and you will do it sample-by-sample in real time, never storing the whole signal. Keep the last M samples in a small *circular buffer* and compute one output per new sample:

Listing 13.2: Streaming FIR convolution (length-M) in C.

```c
#define M 7
float h[M] = {1./M,1./M,1./M,1./M,1./M,1./M,1./M};  // boxcar smoother
float buf[M] = {0};      // circular buffer of recent inputs
int   pos = 0;

float fir_step(float x_new) {
    buf[pos] = x_new;                // overwrite oldest sample
    float y = 0.0f;
    int idx = pos;
    for (int m = 0; m < M; m++) {    // dot product with the taps
        y += h[m] * buf[idx];
        idx = (idx == 0) ? M-1 : idx-1;
    }
    pos = (pos + 1) % M;
    return y;
```

```
}
```

This is $O(M)$ per sample and uses $O(M)$ memory — trivial for M in the single digits at IMU sample rates of a few hundred hertz. For a boxcar specifically you can do even better with a running sum (one add, one subtract), but the FIR form above generalizes to any designed filter.

Toward the wrist tracker

Convolution is the first real signal-processing step in the wrist tracker. Raw accelerometer and gyroscope streams are noisy and jittery; before you can detect a gesture you smooth each axis with a short low-pass convolution (a boxcar or, better, a designed FIR) to kill sensor hash and hand tremor. Tap and flick detection then convolves the smoothed signal with a short *matched* kernel shaped like the gesture's acceleration bump — a sliding template match, which (Section 13.3) is convolution with a flipped template. Even debouncing a contact event is a tiny convolution that spreads and thresholds the pulse.

Be clear about what this buys you. Smoothing and template-convolution on a wrist IMU detect *coarse, discrete* motions — a flick, a tap, a wrist roll, a double-tap. They do *not* recover continuous individual finger joint angles; the wrist simply does not move enough, distinctly enough, when only a finger curls. Per-finger sensing needs EMG or force myography on the forearm, and continuous finger *pose* needs sensors on the fingers themselves (rings or a glove). Those signals, too, get convolved — EMG is band-pass filtered and its envelope is extracted by convolution — but the sensor, not the math, sets the ceiling on what gestures you can read. Convolution cleans and matches the signal you have; it cannot conjure information the sensor never captured.

13.9 A glance ahead: 2D convolution for images

Everything here generalizes to two dimensions with a second sum (or integral). For an image $x[m, n]$ and a small kernel $h[i, j]$,

$$(x * h)[m, n] \;=\; \sum_{i} \sum_{j} x[i, j]\, h[m - i,\, n - j]. \tag{13.9}$$

The recipe is the same: flip the kernel (now in both axes), slide it across the image, and at each position take the sum of the overlapping products. A 3×3 averaging kernel blurs; a kernel with a positive center and negative surround sharpens or finds edges. We devote a later chapter to it, because image filtering, edge detection, and the convolutional layers of neural networks are all this one operation in 2D. The intuition you built in 1D — a sliding, weighted, flipped overlap — transfers wholesale; only the bookkeeping gains an index.

Pause

The whole chapter in one breath: a signal is a stack of impulses; an LTI system answers each impulse with a shifted, scaled copy of its impulse response; the output is the superposition of all those copies; written as a formula that superposition is the convolution sum $\sum_k x[k]h[n-k]$; computed by hand it is flip, shift, multiply, sum; it is commutative, associative, distributive,

with δ as identity; it is secretly polynomial multiplication, which is why a transform turns it into a pointwise product; and it is the workhorse behind filtering, echo, smoothing, and — in 2D — image processing.

13.10 Exercises

Exercise 13.1. By hand, compute $x * h$ for $x = [\,2,\ 0,\ -1,\ 3\,]$ (indices 0–3) and $h = [\,1,\ 2\,]$ (indices 0–1). State the length of the result and verify it by multiplying the polynomials $2 - z^2 + 3z^3$ and $1 + 2z$.

Exercise 13.2. A three-point moving average has $h = [\,1/3,\ 1/3,\ 1/3\,]$. Convolve it with the unit step $x[n] = 1$ for $n \geq 0$ (and 0 before). Sketch the output for $n = -1$ through $n = 4$ and describe the startup transient. How many samples until the output reaches its steady value?

Exercise 13.3. Prove associativity of convolution, $(x * h_1) * h_2 = x * (h_1 * h_2)$, by writing the left side as a double sum and reindexing. Then interpret the result as a statement about cascaded LTI systems.

Exercise 13.4. A single-echo system has $h[n] = \delta[n] + 0.5\,\delta[n-4]$. (a) Write the difference equation relating $y[n]$ to $x[n]$. (b) Convolve h with itself. What system does $h * h$ represent, and how many echoes does it have? (c) In general, what does convolving an echo with itself k times do?

Exercise 13.5. Show that $x * \delta[n - D] = x[n - D]$ directly from the convolution sum. Explain in one sentence why this means "convolving with a shifted spike is a pure delay."

Exercise 13.6. The flip in convolution is what distinguishes it from cross-correlation, $r[n] = \sum_k x[k]\,h[k + n]$ (no flip). (a) For h symmetric, $h[n] = h[-n]$, show convolution and correlation give the same answer. (b) Give a short, non-symmetric h where they differ, and compute both for some input. (c) Which one is the right tool for sliding-template gesture matching, and why does the flip matter there?

Exercise 13.7. You convolve a length-1000 signal with a length-64 filter. (a) How many multiply-adds does direct convolution take, roughly? (b) Estimate the cost of the FFT route ($\sim 3 \cdot L \log_2 L$ for three transforms of length L, plus L multiplies, with $L = 1024$). (c) At what filter length does the FFT route start to win, qualitatively?

Exercise 13.8. Using the connection to the probability companion, suppose each sample of a flat signal is corrupted by independent noise of standard deviation $\sigma = 0.2$. You smooth with an M-point moving average. (a) What is the noise standard deviation of the output? (b) How large must M be to cut the noise to 0.05? (c) What feature width (in samples) do you lose by choosing that M, and how is this the noise-versus-resolution tradeoff?

Exercise 13.9. Write (in pseudocode or Python) a streaming convolution that processes one sample at a time with a length-M FIR filter using a circular buffer, matching the C sketch in this chapter. Test it against `numpy.convolve` on a random input and confirm the outputs agree (allowing for the edge/startup samples).

Exercise 13.10. Take the 2×2 image $X = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$ and the 2×2 kernel $H = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$. Compute the full 2D convolution $X * H$ using Eq. (13.9) (remember to flip H in both axes). What size is the result, and which output entry corresponds to the kernel fully overlapping the image?

Chapter 14

Impulse and Frequency Response

In the last two chapters we learned that a linear, time-invariant (LTI) system is completely described by a single sequence: its response to a unit impulse. We called that sequence the *impulse response* $h[n]$, and we saw that the output of the system is the convolution of the input with h. That is a remarkable fact, but on its own it can feel a little opaque. If I hand you a list of forty numbers and tell you "this is your filter," can you tell me what it *does* to a sound? To a tremor in your wrist? To a 60 Hz hum on a sensor wire?

The answer is yes, and the key is to look at the same system through a second window. The impulse response lives in *time*: it tells you how the system spreads a single poke out over the following samples. The *frequency response* lives in *frequency*: it tells you, for each pure tone you could feed in, how much that tone is amplified or attenuated and how much it is delayed. These two descriptions are not rivals. They are the same object seen from two sides, joined by the Fourier transform. Learning to flip between them fluently — "what does this $h[n]$ do to a 100 Hz sine?" and "what $h[n]$ gives me *that* curve?" — is most of what it means to think like a filter designer.

This chapter builds the frequency response geometrically: a filter is a machine that takes a complex exponential in and gives the *same* complex exponential back out, only rescaled by a complex number. That single number, as the input frequency sweeps, traces out the whole story.

14.1 The one experiment that defines everything

Let us run the simplest possible test. Feed an LTI system a single pure complex exponential that has existed for all time,

$$x[n] = e^{\mathrm{j}\omega n}, \qquad n \in \mathbb{Z},$$

where ω is the angular frequency in radians per sample. Because the system is LTI, its output is the convolution of x with the impulse response $h[n]$:

$$y[n] = (h * x)[n] = \sum_{k=-\infty}^{\infty} h[k]\, x[n-k] = \sum_{k=-\infty}^{\infty} h[k]\, e^{\mathrm{j}\omega(n-k)}.$$

Now pull the part that does not depend on k out of the sum:

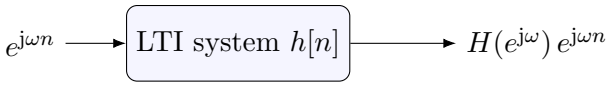
$$y[n] = \underbrace{e^{\mathrm{j}\omega n}}_{\text{the input}} \cdot \underbrace{\sum_{k=-\infty}^{\infty} h[k]\, e^{-\mathrm{j}\omega k}}_{\text{a number}}.$$

Stare at that for a moment. The output is the *same* complex exponential $e^{\mathrm{j}\omega n}$ we put in, multiplied by a constant that does not depend on n at all. The constant is built only from h and the chosen frequency ω. We give it a name:

$$H(e^{\mathrm{j}\omega}) \;=\; \sum_{k=-\infty}^{\infty} h[k]\, e^{-\mathrm{j}\omega k}. \tag{14.1}$$

So the experiment yields (Figure 14.1)

$$x[n] = e^{\mathrm{j}\omega n} \;\longrightarrow\; y[n] = H(e^{\mathrm{j}\omega})\, e^{\mathrm{j}\omega n}. \tag{14.2}$$



same frequency out,
scaled and phase-shifted by one number

Figure 14.1: The defining experiment. A pure complex exponential is the one input an LTI system cannot reshape: it returns the same exponential, multiplied by the single complex number $H(e^{\mathrm{j}\omega})$.

What is really happening

Complex exponentials are the *eigenfunctions* of every LTI system. Feed in $e^{\mathrm{j}\omega n}$ and the system cannot change its shape — it is still a pure rotation at frequency ω. All the system is allowed to do is scale it and spin its starting phase, and both of those are captured by the single complex number $H(e^{\mathrm{j}\omega})$. The function $H(e^{\mathrm{j}\omega})$, swept over all ω, is the *frequency response*, and it is just the eigenvalue attached to each frequency.

Connection

This is exactly the eigenvalue/eigenvector story from the linear algebra book, lifted to infinite dimensions. Convolution by h is a linear operator on the space of sequences. The complex exponentials $e^{\mathrm{j}\omega n}$ are its eigenvectors, and $H(e^{\mathrm{j}\omega})$ is the eigenvalue belonging to the eigenvector of frequency ω. Filtering is "diagonal" in the frequency basis — it just multiplies each frequency component by its own number. That is the whole reason the Fourier transform is so central: it is the change of basis that diagonalizes every LTI system at once.

14.1.1 It is literally the DTFT of h

Look again at Equation (14.1). It is the discrete-time Fourier transform (DTFT) of the impulse response, from Chapter 7. So we can state the central identity of this chapter in one line:

Formal

The frequency response is the DTFT of the impulse response:

$$H(e^{\mathrm{j}\omega}) = \mathcal{F}\{h[n]\} = \sum_{n=-\infty}^{\infty} h[n]\, e^{-\mathrm{j}\omega n}.$$

Conversely h is the inverse DTFT of H,

$$h[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} H(e^{\mathrm{j}\omega})\, e^{\mathrm{j}\omega n}\, \mathrm{d}\omega.$$

Time and frequency descriptions of an LTI system carry exactly the same information; the DTFT is the dictionary that translates one into the other.

The notation $H(e^{\mathrm{j}\omega})$ looks heavy, but it is deliberate. It reminds you that H is really a function of the point $e^{\mathrm{j}\omega}$ on the unit circle of the complex plane. As ω runs from $-\pi$ to π, the point $e^{\mathrm{j}\omega}$ walks once around that circle. Frequency response is what you see as you stroll around the unit circle. (In Chapter 15 the z-transform will let z roam the whole plane; the frequency response is the slice of the z-transform taken right on the circle $|z| = 1$.)

Why it works

Why does the convolution theorem fall out for free here? Because Equation (14.2) says each eigen-input is scaled by $H(e^{\mathrm{j}\omega})$. An arbitrary input $x[n]$ is, by the inverse DTFT, a sum (integral) of such eigen-inputs, one per frequency, with weights $X(e^{\mathrm{j}\omega})$. The system scales each one by its eigenvalue, so the output weights are $Y(e^{\mathrm{j}\omega}) = H(e^{\mathrm{j}\omega})\, X(e^{\mathrm{j}\omega})$. Convolution in time becomes *multiplication* in frequency. We did the work in Chapter 11; the eigenfunction picture is why it had to be true.

Focus

Two facts to carry through the rest of the book:

1. $H(e^{\mathrm{j}\omega})$ is periodic in ω with period 2π. A discrete-time system only "knows" frequencies on one trip around the unit circle; everything else is an alias.
2. If $h[n]$ is real (as it is for every filter we will actually build), then $H(e^{-\mathrm{j}\omega}) = \overline{H(e^{\mathrm{j}\omega})}$. The magnitude is even and the phase is odd, so we usually plot only $0 \le \omega \le \pi$.

14.2 Magnitude and phase: what the complex number means

$H(e^{\mathrm{j}\omega})$ is complex, so like any complex number it has a magnitude and an angle:

$$H(e^{\mathrm{j}\omega}) = |H(e^{\mathrm{j}\omega})|\, e^{\mathrm{j}\,\angle H(e^{\mathrm{j}\omega})}.$$

Put this back into the eigen-relation (14.2) and write the input as a real cosine, $x[n] = \cos(\omega n)$. By linearity and the fact that $\cos(\omega n) = \frac{1}{2}(e^{\mathrm{j}\omega n} + e^{-\mathrm{j}\omega n})$, the steady-state output of a real system is

$$x[n] = \cos(\omega n) \quad \longrightarrow \quad y[n] = |H(e^{\mathrm{j}\omega})|\, \cos\big(\omega n + \angle H(e^{\mathrm{j}\omega})\big). \tag{14.3}$$

So the two parts of the complex number have crisp physical meanings.

- **Magnitude response** $|H(e^{j\omega})|$: the *gain* at frequency ω. A value of 2 doubles a sine of that frequency; 0.1 knocks it down to a tenth; 0 kills it outright. This is the curve people usually mean when they say "the filter's response."
- **Phase response** $\angle H(e^{j\omega})$: the *phase shift* the filter adds at frequency ω. A shift of $-\pi/2$ turns a cosine into a sine; in time it slides that sine later (a delay).

Intuition

A cosine goes in, a cosine of the *same frequency* comes out — never a different frequency. An LTI system cannot create new tones; it can only turn the volume knob (magnitude) and slide the start time (phase) of each tone already present. That is the entire grammar of linear filtering. Distortion that generates harmonics — the growl of an overdriven guitar amp — is exactly the signature of *non*-linearity.

14.2.1 Decibels: why gain is plotted on a log scale

Magnitude responses span enormous ranges — a good filter might pass a band at gain 1 and reject another at gain 0.001. To see both at once we use decibels:

$$|H|_{\text{dB}} = 20 \log_{10} |H(e^{j\omega})|.$$

A gain of 1 is 0 dB; gain $\frac{1}{\sqrt{2}} \approx 0.707$ is -3 dB; gain 0.1 is -20 dB; gain 0.001 is -60 dB. The factor 20 (not 10) is because power goes as amplitude squared and $10 \log_{10}(|H|^2) = 20 \log_{10}|H|$. The famous "$-3$ dB point" marks where a filter has dropped to half *power*, and it is the conventional edge of a passband.

Common trap

Do not confuse the unit-circle frequency ω (radians per sample, range $-\pi$ to π) with an ordinary frequency in hertz. If the sampling rate is f_s, then $\omega = 2\pi f / f_s$. The top of the digital band, $\omega = \pi$, is the Nyquist frequency $f_s/2$. We will hammer this in Chapter 18; for now, just remember that "$\omega = \pi$" means "the fastest wiggle a sampled signal can represent: flipping sign every sample."

14.3 A first worked example: the two-tap averager

Nothing makes this concrete like the smallest real filter. Take the two-point moving average,

$$y[n] = \tfrac{1}{2}\, x[n] + \tfrac{1}{2}\, x[n-1].$$

Its impulse response is read straight off the coefficients: poke it with $\delta[n]$ and you get $h[0] = \frac{1}{2}$, $h[1] = \frac{1}{2}$, all else zero. Plug into Equation (14.1):

$$H(e^{j\omega}) = \tfrac{1}{2} + \tfrac{1}{2} e^{-j\omega}.$$

Now do the standard trick: factor out half the total phase to expose a clean cosine. Pull out $e^{-\mathrm{j}\omega/2}$:

$$H(e^{\mathrm{j}\omega}) = e^{-\mathrm{j}\omega/2}\left(\tfrac{1}{2}e^{\mathrm{j}\omega/2} + \tfrac{1}{2}e^{-\mathrm{j}\omega/2}\right) = e^{-\mathrm{j}\omega/2}\cos\left(\tfrac{\omega}{2}\right).$$

This little form is a gift, because it hands us magnitude and phase separately:

$$|H(e^{\mathrm{j}\omega})| = \left|\cos\left(\tfrac{\omega}{2}\right)\right|, \qquad \angle H(e^{\mathrm{j}\omega}) = -\tfrac{\omega}{2}.$$

Read it. At DC ($\omega = 0$) the gain is $\cos 0 = 1$: a constant passes untouched, which makes sense, the average of a constant is itself. At Nyquist ($\omega = \pi$) the gain is $\cos(\pi/2) = 0$: the alternating sequence $\ldots, 1, -1, 1, -1, \ldots$ averages adjacent samples to zero and is annihilated. In between, the gain falls smoothly. This is a (gentle) *low-pass* filter: it favors slow wiggles and suppresses fast ones. And the phase is a perfectly straight line, $-\omega/2$. We will see in a moment that a straight-line phase is the best kind to have.

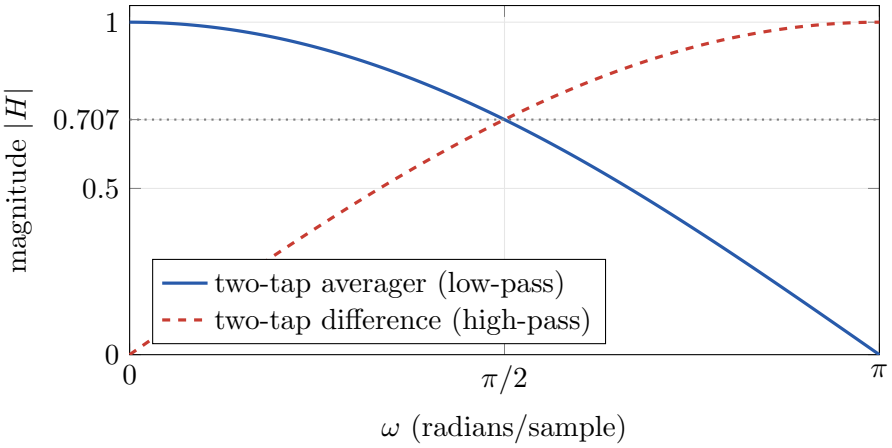


Figure 14.2: Magnitude responses of the two simplest filters. The averager $h = [\frac{1}{2}, \frac{1}{2}]$ passes low frequencies and nulls Nyquist; the difference $h = [\frac{1}{2}, -\frac{1}{2}]$ does the opposite. The dotted line marks the -3 dB (0.707) half-power level.

14.3.1 The mirror image: a two-tap difference

Flip one sign: $y[n] = \frac{1}{2}x[n] - \frac{1}{2}x[n-1]$, so $h = [\frac{1}{2}, -\frac{1}{2}]$. The same algebra gives

$$H(e^{\mathrm{j}\omega}) = \tfrac{1}{2} - \tfrac{1}{2}e^{-\mathrm{j}\omega} = \mathrm{j}\, e^{-\mathrm{j}\omega/2}\sin\left(\tfrac{\omega}{2}\right), \qquad |H(e^{\mathrm{j}\omega})| = \left|\sin\left(\tfrac{\omega}{2}\right)\right|.$$

Now the gain is 0 at DC and rises to 1 at Nyquist — a *high-pass* filter (the dashed curve in Figure 14.2). The intuition is just as clean: a difference asks "how much did the signal change between samples?" A constant does not change, so it is removed; a fast alternation changes maximally, so it survives. Subtraction is differentiation in disguise, and differentiation emphasizes high frequencies.

Example 14.1 (Predicting the output by hand)**.** Send $x[n] = \cos(\frac{\pi}{2}n)$ through the averager. Here $\omega = \pi/2$, so $|H| = \cos(\pi/4) = 0.707$ and $\angle H = -\pi/4$. By Equation (14.3) the steady-state output is

$$y[n] = 0.707\cos\left(\tfrac{\pi}{2}n - \tfrac{\pi}{4}\right).$$

Check it directly: $x = \ldots, 1, 0, -1, 0, 1, \ldots$ and $y[n] = \frac{1}{2}(x[n]+x[n-1]) = \ldots, 0.5, 0.5, -0.5, -0.5, \ldots$, which is indeed a cosine of amplitude 0.707 shifted by an eighth of a period. The frequency response told us the answer without grinding through the convolution.

14.4 Group delay: the phase slope you can feel

Magnitude is intuitive; phase takes more care. The raw phase $\angle H(e^{\mathrm{j}\omega})$ is hard to read because a delay shows up as a phase that grows with frequency, and because phase wraps around every 2π. The quantity that turns phase into something physical is the *group delay*:

$$\tau_g(\omega) = -\frac{\mathrm{d}}{\mathrm{d}\omega}\angle H(e^{\mathrm{j}\omega}). \tag{14.4}$$

Group delay measures, in samples, how long the filter holds back the *envelope* of a narrowband signal centered at ω. If you send in a short pulse whose energy sits near frequency ω, the bump comes out $\tau_g(\omega)$ samples later.

Why it works

Why the *derivative* of phase, and why does it act on the envelope? Picture a narrowband packet: a fast carrier $e^{\mathrm{j}\omega_0 n}$ multiplied by a slow envelope. The carrier is shifted by the phase itself (*phase delay*, $-\angle H/\omega$). But the envelope is built from a cluster of nearby frequencies, and what matters to them is how the phase *changes* from one to the next — its local slope. A constant slope means every component in the cluster lines up after the same time shift, so the envelope reappears intact, just later. That common time shift is exactly $-\mathrm{d}(\angle H)/\mathrm{d}\omega$.

Intuition

Linear phase = constant group delay = no waveform distortion. If $\angle H(e^{\mathrm{j}\omega}) = -\alpha\omega$ for some constant α, then $\tau_g = \alpha$ for *every* frequency. All frequencies are delayed by the same α samples, so the output is just the input shifted in time (and rescaled in amplitude band by band). The shape of the signal is preserved.

If instead the slope varies with ω, different frequencies arrive at different times. Bass lags treble, the waveform smears, and a crisp impulse turns into a ringing blur. That smearing is *phase distortion*, and it is inaudible in some applications and ruinous in others.

Our two-tap averager had $\angle H = -\omega/2$, a perfectly straight line, so $\tau_g = \frac{1}{2}$ sample at every frequency. It is a linear-phase filter — the whole signal is delayed by half a sample and nothing is smeared. This is no accident: any FIR filter whose taps are symmetric (or antisymmetric) about their center has exactly linear phase, a property we will exploit heavily in Chapter 16.

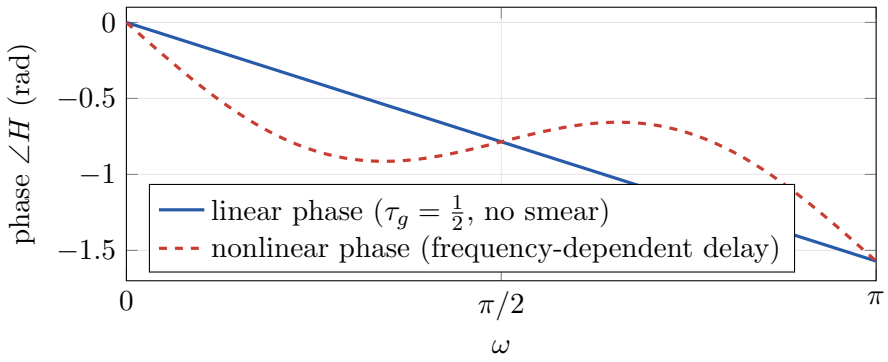


Figure 14.3: A straight phase line delays all frequencies equally and preserves waveform shape. A curved phase line delays them differently, smearing transients even when the magnitude response is identical.

Common trap

Two filters can have *identical* magnitude responses and behave completely differently, because their phases differ. "It has a flat passband" tells you nothing about whether it will ring or smear. Always check the phase (or group delay) when transient shape matters — and it almost always matters for the short, sharp motions of a gesture.

14.5 Reading a frequency response like a map

Hand an engineer a magnitude plot and they read it the way you read a face. Here is the vocabulary.

- **Passband:** the range of ω where the gain is near its maximum (ideally near 0 dB). These frequencies survive.
- **Stopband:** the range where the gain is small (deeply negative dB). These frequencies are rejected.
- **Transition band:** the slope between them. A steep transition is "sharp"; a gentle one is "soft." Sharpness costs taps (longer h) and costs delay.
- **Cutoff frequency** ω_c: by convention the -3 dB point, where the gain has fallen to 0.707 (half power).
- **Ripple:** small wiggles in the passband or stopband. Some filter designs trade passband flatness for a steeper transition; the wiggles are the price.
- **Notch / null:** a frequency where the gain is exactly (or nearly) zero — a surgical strike on one tone, like 60 Hz mains hum.
- **Resonant peak:** a frequency where the gain spikes *above* the passband — the system rings there. More on this in Section 14.7.

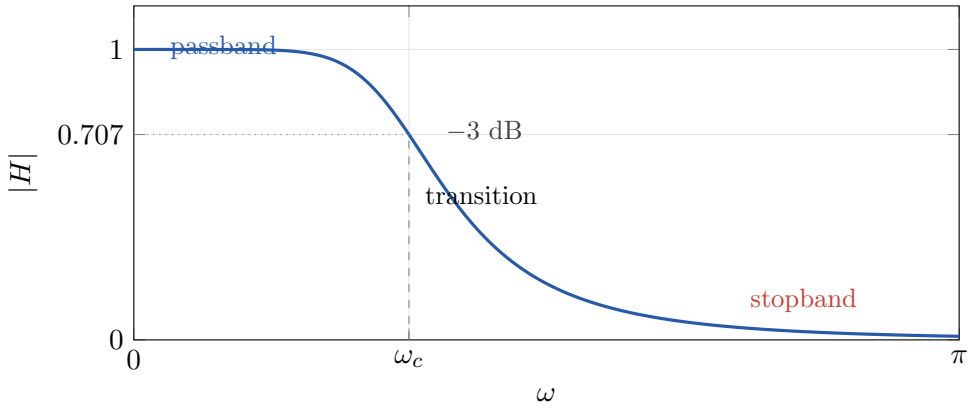


Figure 14.4: Anatomy of a low-pass magnitude response. The cutoff ω_c is read at the -3 dB (half-power, 0.707) crossing.

Walking question

Here is a magnitude curve that is ≈ 1 from $\omega = 0$ up to $\omega = 0.4\pi$, then drops fast to near 0 and stays there. What does this filter do to a recording of speech sampled at 8 kHz?
The Nyquist frequency is 4 kHz, mapped to $\omega = \pi$. So $\omega = 0.4\pi$ is $0.4 \times 4 = 1.6$ kHz. The filter passes everything below about 1.6 kHz and removes everything above. Speech keeps its vowels and low formants but loses the crisp "s" and "f" sounds, which live higher up. It will sound muffled, as if heard through a wall. You just read a filter without computing a single convolution.

14.6 How a system reshapes a spectrum

We now have everything we need to picture filtering end to end. By the convolution theorem, in the frequency domain the system simply multiplies the input spectrum by its frequency response, point by point:

$$Y(e^{j\omega}) = H(e^{j\omega})\, X(e^{j\omega}). \tag{14.5}$$

And magnitudes multiply while phases add:

$$|Y| = |H| \cdot |X|, \qquad \angle Y = \angle H + \angle X.$$

What is really happening

Filtering is *spectral shaping*. Lay the input spectrum $|X|$ over the filter's gain curve $|H|$. Where the curve is high, those frequencies of the input pass through loud; where it dips, they are turned down; where it hits zero, they vanish. The output spectrum is the input spectrum "painted" by the filter's gain. Nothing new appears; the existing colors are just re-weighted.

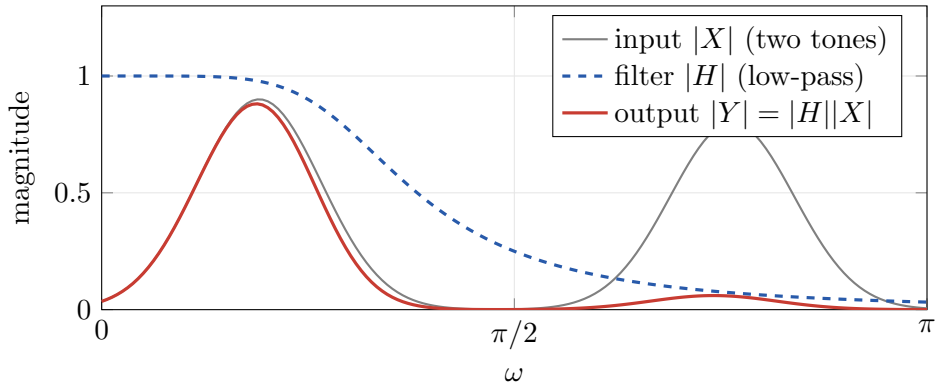


Figure 14.5: An input with a low-frequency tone and a high-frequency tone, sent through a low-pass filter. The low bump survives nearly intact; the high bump is crushed where $|H|$ is small. Output magnitude is the pointwise product.

Example 14.2 (Killing 60 Hz hum)**.** A sensor sampled at $f_s = 1000$ Hz picks up mains interference at 60 Hz. In digital terms that tone sits at $\omega_0 = 2\pi \cdot 60/1000 = 0.377$ rad. To remove it we want a frequency response that is ≈ 1 everywhere except a deep null at $\pm\omega_0$. A two-zero "notch" does it: place a transmission zero exactly at $e^{\pm \mathrm{j}\omega_0}$ so that $H(e^{\mathrm{j}\omega_0}) = 0$. We will build the $h[n]$ for this from its zeros in Chapter 15, but the point now is that we specify the filter by drawing the response we want — a flat line with one surgical notch — and then work backward to the coefficients.

14.7 Resonance: when a system rings

So far our filters only attenuated. But many systems amplify a narrow band of frequencies dramatically — a wine glass, a guitar body, a tuned circuit, a mass on a spring. In the frequency response this shows up as a tall, narrow peak. In the time domain it shows up as *ringing*: poke the system once and it keeps oscillating for a while, decaying slowly. These are two views of the same thing.

The simplest resonator is a single-pole feedback filter,

$$y[n] = x[n] + r\, e^{\mathrm{j}\omega_0}\, y[n-1] \quad \text{(complex)}, \qquad 0 < r < 1, \tag{14.6}$$

or, in real form using a conjugate pair of poles,

$$y[n] = x[n] + 2r\cos(\omega_0)\, y[n-1] - r^2\, y[n-2].$$

Its impulse response is a decaying sinusoid, $h[n] \approx r^n \cos(\omega_0 n)$ for $n \geq 0$: an exponentially fading ring at frequency ω_0. The closer r creeps toward 1, the slower the decay and — this is the duality — the *taller and narrower* the spectral peak.

Intuition

A resonance is feedback that nearly — but not quite — sustains itself at one frequency. Energy fed in near ω_0 gets recycled around the feedback loop many times before it leaks away, building up a large response. The number of times it recycles is roughly the *quality factor* Q: high Q means a tall sharp peak in frequency and a long ring in time; low Q means a broad bump that dies quickly. "Sharp in frequency" and "long in time" are the same statement, the uncertainty

principle wearing a filter's clothes.

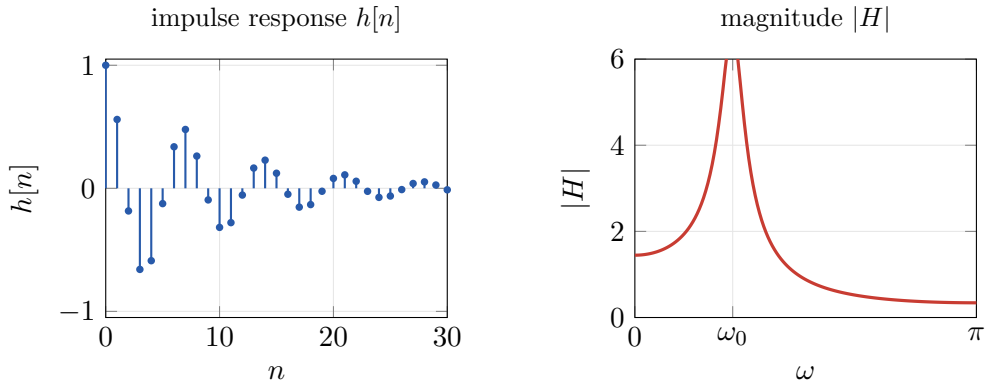


Figure 14.6: A resonator with poles at radius $r = 0.9$ and angle $\omega_0 = 0.9$ rad. Left: the impulse response is a decaying ring. Right: the matching magnitude response has a tall narrow peak at ω_0. Long ring $\Leftrightarrow$ sharp peak.

Why it works

Where does the peak come from, geometrically? Write the frequency response as a ratio of distances in the complex plane (Chapter 15 makes this precise). The denominator contains a factor $|e^{j\omega} - p|$, the distance from the unit-circle point $e^{j\omega}$ to the pole $p = r\,e^{j\omega_0}$. As ω sweeps past ω_0, the unit-circle point glides closest to the pole, that distance shrinks, and the response — being one over the distance — shoots up. The nearer the pole sits to the circle ($r \to 1$), the smaller the minimum distance, and the higher the spike. Resonance is just "the unit circle passing close to a pole" (Figure 14.7).

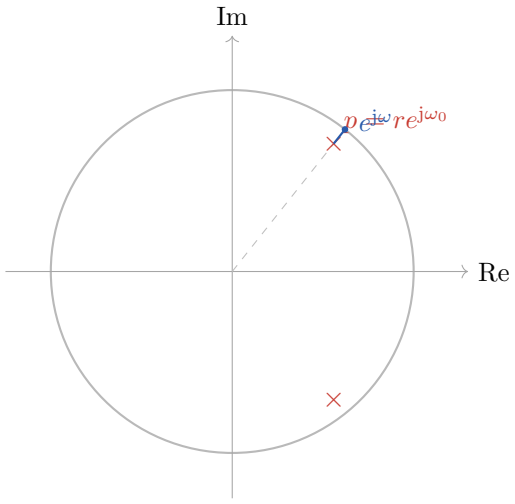


Figure 14.7: Pole-zero view of the resonator. The conjugate poles (×) sit just inside the unit circle at radius $r = 0.9$ and angle $\pm\omega_0$. As the test point $e^{j\omega}$ slides around the circle to ω_0, its distance to the pole (blue segment) reaches a minimum, so the response — one over that distance — peaks. Push the pole toward the circle and the peak grows taller and narrower.

Application

Resonance is everywhere in real instruments. The body of an acoustic guitar has strong resonances that color its tone; a vocal tract has resonances (formants) that distinguish "ah" from "ee"; a snare drum has a tuned ring. When we model or synthesize these sounds (Chapter 28–29), we build banks of resonators and set each pole's angle (pitch) and radius (decay time). Conversely, to *analyze* a sound we look for spectral peaks and infer the resonances that produced them.

14.8 Putting numbers in: a worked FIR low-pass

Let us compute a frequency response we cannot do in our heads, to see the machinery turn. Take a length-5 averaging filter,

$$h = \tfrac{1}{5}\,[\,1,\ 1,\ 1,\ 1,\ 1\,], \qquad h[0..4] = 0.2.$$

By Equation (14.1),

$$H(e^{\mathrm{j}\omega}) = \tfrac{1}{5}\sum_{n=0}^{4} e^{-\mathrm{j}\omega n}.$$

This is a finite geometric series. Summing it and using the Dirichlet-kernel trick (factor out the middle phase) gives the clean closed form

$$H(e^{\mathrm{j}\omega}) = e^{-\mathrm{j}2\omega}\,\frac{1}{5}\,\frac{\sin(5\omega/2)}{\sin(\omega/2)}.$$

The magnitude is the (aliased) sinc shape $|H| = \tfrac{1}{5}|\sin(5\omega/2)/\sin(\omega/2)|$, and the phase is the straight line -2ω (so $\tau_g = 2$ samples — the center of a 5-tap symmetric filter, exactly as the linear-phase rule predicts). It is low-pass, with a main lobe near DC and decaying side lobes — the longer the average, the narrower the main lobe and the lower the cutoff.

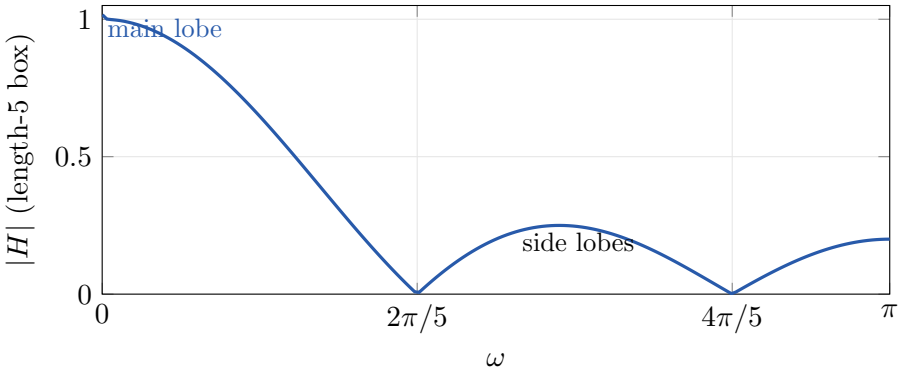


Figure 14.8: Magnitude response of the length-5 box-average filter. Note the nulls at $\omega = 2\pi/5$ and $4\pi/5$ (where $\sin(5\omega/2) = 0$) and the leaky side lobes — the price of an abrupt, rectangular impulse response.

Common trap

Those side lobes are why a plain moving average is a *poor* low-pass filter: its stopband attenuation is only about -13 dB at the first side lobe, so it leaks high-frequency content. The cure is to taper the taps smoothly to zero instead of chopping them off square — this is windowing, the subject of Chapter 16. The square impulse response causes the leaky spectrum; a smoother h buys a cleaner H.

Try it yourself

Compute and plot frequency responses yourself. The direct, no-magic way is the DTFT sum from Equation (14.1); in practice we sample it densely with an FFT (zero-padded), which is what `freqz` does.

```
import numpy as np
import matplotlib.pyplot as plt

def freq_response(h, N=1024):
    w = np.linspace(0, np.pi, N)
    # H(e^{jw}) = sum_n h[n] e^{-j w n}, evaluated at each w
    n = np.arange(len(h))
    H = h @ np.exp(-1j * np.outer(n, w))   # shape (N,)
    return w, H

h = np.ones(5) / 5.0                       # length-5 box average
w, H = freq_response(h)

plt.subplot(2,1,1)
plt.plot(w, 20*np.log10(np.abs(H) + 1e-9))  # magnitude in dB
plt.ylabel("|H| (dB)"); plt.ylim(-60, 5)

plt.subplot(2,1,2)
plt.plot(w, np.unwrap(np.angle(H)))         # unwrap before reading slope
plt.xlabel("omega (rad/sample)"); plt.ylabel("phase (rad)")
plt.show()

# group delay = -d(phase)/d(omega), as a finite difference
phase = np.unwrap(np.angle(H))
gd = -np.gradient(phase, w)
print("group delay near DC:", gd[1])        # ~ 2 samples
```

Swap in `h = np.array([0.5, 0.5])` or `[0.5, -0.5]` and confirm the curves in Figure 14.2. Then try a longer box and watch the cutoff move down. Always `unwrap` the phase before differentiating, or the 2π jumps will wreck your group delay.

14.9 The two views, side by side

It is worth collecting the dictionary in one place, because fluency *is* the skill.

Time domain: impulse response $h[n]$	**Frequency domain: response** $H(e^{\mathrm{j}\omega})$
What the system does to one poke	What it does to each pure tone
Output via convolution $y = h * x$	Output via multiplication $Y = HX$
Length of h = memory / latency	Sharpness of transitions
Symmetric taps	Linear phase, constant group delay
Long, slowly decaying ring	Tall, narrow resonant peak
Abrupt (boxy) h	Leaky side lobes in $\lvert H\rvert$
Delay by d samples: $h[n] = \delta[n-d]$	Pure phase ramp $e^{-\mathrm{j}\omega d}$, flat gain

Why this matters

The deepest idea in this chapter is that *a filter is one object with two faces.* Designers think in frequency ("flat to 1 kHz, then 40 dB down") and implement in time (a list of taps run through a convolution loop on a chip). Every property you care about — latency, ringing, sharpness, distortion — can be read from either face once you know how the DTFT maps between them. The rest of the book is, in a sense, just getting good at reading both faces at once.

Toward the wrist tracker

For the wrist gesture controller, the frequency response is the lens through which we decide what to keep and what to throw away in the raw IMU and muscle signals.

Where the signals live. Deliberate hand and wrist motion has energy mostly below about 5 Hz; physiological tremor sits near 8–12 Hz; mains hum is at 50/60 Hz; and surface-EMG bursts from forearm muscles carry useful energy roughly 20–450 Hz. These bands barely overlap, which is a gift: we can separate them with the right magnitude responses. A low-pass on the accelerometer keeps the gesture motion and rejects tremor and hum; a band-pass on the EMG keeps the muscle bursts and removes both DC drift and high-frequency junk.

Why phase matters here. Gestures are short, sharp *transients* — a flick, a tap, a clench. A filter with nasty nonlinear phase will smear that transient in time, blurring exactly the onset we need for low-latency, musical triggering. So we favor linear-phase (symmetric FIR) filters near the signal path, accepting their fixed delay because at least that delay is the *same* for every frequency and therefore predictable.

The hard limit. A frequency response only *re-weights* tones; it cannot conjure information the sensor never captured. A wrist-mounted IMU alone measures the gross motion of the whole forearm/wrist; no choice of $h[n]$ turns it into a sensor of individual finger-joint angles — that signal simply is not present in the IMU stream. Detecting which finger moved needs forearm EMG or force myography (FMG); reconstructing continuous finger pose needs sensors *on the fingers.* Filtering cleans up what the sensor saw. It never adds a sense the sensor does not have.

Build it (hardware)

On an Arduino or Pi reading an IMU at, say, $f_s = 200$ Hz, a cheap and effective anti-tremor low-pass is a length-M moving average plus a small linear-phase trim, or a single-pole IIR smoother:

```
// One-pole low-pass: y[n] = a*x[n] + (1-a)*y[n-1]
// Cutoff (rad/sample) ~ a for small a; choose a from f_c, f_s:
//   a = 1 - exp(-2*pi*f_c/f_s)
float a = 1.0f - expf(-2.0f * 3.14159265f * 3.0f / 200.0f); // ~3 Hz cutoff
static float y = 0.0f;
float filt(float x) { y += a * (x - y); return y; }
```

This costs one multiply and one add per sample — trivial on an 8-bit MCU. Its frequency response is $H(e^{\mathrm{j}\omega}) = a/(1-(1-a)e^{-\mathrm{j}\omega})$: a gentle low-pass with a single pole at $1-a$. It is *not* linear-phase (its group delay rises near DC), so when transient timing is critical, prefer a short symmetric FIR. Measure the actual response by feeding the firmware a recorded chirp and running the `freq_response` code above on the output.

14.10 Summary

- Feeding an LTI system a complex exponential $e^{\mathrm{j}\omega n}$ returns the same exponential scaled by a complex number $H(e^{\mathrm{j}\omega})$. Complex exponentials are the eigenfunctions of LTI systems; H collects the eigenvalues.
- $H(e^{\mathrm{j}\omega})$ is the DTFT of the impulse response $h[n]$. Time and frequency descriptions hold identical information.
- $|H(e^{\mathrm{j}\omega})|$ is the gain at each frequency; $\angle H(e^{\mathrm{j}\omega})$ is the phase shift. A cosine in comes out a cosine of the same frequency, rescaled and phase-shifted.
- Group delay $\tau_g = -\mathrm{d}(\angle H)/\mathrm{d}\omega$ is the envelope delay. Linear phase $\Rightarrow$ constant group delay $\Rightarrow$ no waveform smearing.
- Filtering multiplies spectra: $Y = HX$. The filter "paints" the input spectrum with its gain curve; it cannot create new frequencies.
- A pole near the unit circle gives resonance: a tall narrow peak in frequency and a long decaying ring in time — two faces of one system.
- Abrupt impulse responses give leaky spectra; smooth (windowed) ones give clean spectra. Symmetric taps give linear phase.

14.11 Exercises

Exercise 14.1. For the difference filter $h = [\frac{1}{2}, -\frac{1}{2}]$, the magnitude response is $|\sin(\omega/2)|$. (a) Find its value at $\omega = 0$, $\pi/2$, and π. (b) At what ω has the gain reached 0.707 (the -3 dB point)? (c) Send $x[n] = \cos(\pi n)$ (the fastest sequence) through it and verify the output amplitude matches $|H(e^{\mathrm{j}\pi})|$.

Exercise 14.2. A pure delay of d samples has impulse response $h[n] = \delta[n - d]$. Compute $H(e^{\mathrm{j}\omega})$. Show that its magnitude is 1 at every frequency and its group delay is exactly d at every frequency. Explain in words why a pure delay is the ideal linear-phase "filter" that changes nothing but timing.

Exercise 14.3. A three-tap symmetric filter is $h = [\frac{1}{4}, \frac{1}{2}, \frac{1}{4}]$. (a) Show that $H(e^{\mathrm{j}\omega}) = e^{-\mathrm{j}\omega}\big(\frac{1}{2} + \frac{1}{2}\cos\omega\big)$. (b) Read off the magnitude and phase. (c) What is the group delay? (d) Is this low-pass or high-pass, and what is its gain at DC and at Nyquist?

Exercise 14.4. Two filters have the same magnitude response but different phase: filter A is $h_A = [1, 0, 0]$ delayed appropriately, and filter B reverses the taps of some asymmetric h. Construct a concrete length-3 example where $|H_A| = |H_B|$ for all ω but the filters delay a transient differently. (Hint: compare $h = [1, 0.5, 0]$ with its time-reversal.)

Exercise 14.5. You sample a sensor at $f_s = 500$ Hz and want to remove a 50 Hz hum. (a) What digital frequency ω_0 (in radians/sample) does 50 Hz map to? (b) Sketch the magnitude response you would want (flat, with a notch at $\pm\omega_0$). (c) Why must the notch be placed at $\pm\omega_0$ and not just $+\omega_0$ for a real-valued filter?

Exercise 14.6. For the single-pole low-pass $y[n] = a\,x[n] + (1 - a)\,y[n - 1]$ with $0 < a < 1$: (a) Find $h[n]$ (the impulse response) in closed form. (b) Find $H(e^{\mathrm{j}\omega})$ and show $|H(e^{\mathrm{j}0})| = 1$. (c) Explain how the cutoff moves as $a \to 0$ versus $a \to 1$, and connect this to the length of the impulse response.

Exercise 14.7. A resonator has poles at radius r and angle ω_0. (a) As $r \to 1$, describe what happens to the height and width of the spectral peak and to the decay time of the impulse response. (b) The half-power bandwidth of the peak is roughly $\Delta\omega \approx 2(1 - r)$ for r near 1. If $r = 0.99$, estimate the bandwidth and the number of samples the ring takes to decay to $1/e$. Comment on how "sharp in frequency" and "long in time" are linked.

Exercise 14.8. Using the `freq_response` code in the chapter, plot the magnitude (in dB) of the length-5, length-11, and length-21 box-average filters on the same axes. (a) How does the cutoff frequency change with length? (b) Does the first side-lobe level (about -13 dB) change with length? (c) Explain why making the filter longer sharpens the transition but does not fix the leaky side lobes — and name the chapter that does fix them.

Exercise 14.9. A wrist IMU streams accelerometer data at 200 Hz. Deliberate gesture motion is below ~ 5 Hz; tremor is near ~ 10 Hz. (a) Choose a low-pass cutoff in hertz that keeps gestures but suppresses tremor, and convert it to ω_c. (b) The one-pole smoother in the build box has rising group delay near DC. Explain, in terms of musical triggering latency, why this could be a problem and when a symmetric FIR would be the better choice. (c) State plainly one thing this filtered IMU signal still cannot tell you about the hand, and what sensor you would add to get it.

Exercise 14.10. (Conceptual.) Explain, without equations, why an LTI filter can never turn a single input sine into two output sines of different frequencies. Then give a real-world example of a *non*-linear device that does create new frequencies, and say how you would recognize its signature in a spectrum.

Chapter 15

The Z-Transform and the Laplace Transform

So far the Fourier transform has been our universal lens: feed it a signal, get back "how much of each frequency is inside." That lens is wonderful, but it has a blind spot. The Fourier transform only ever looks at signals that *neither grow nor decay* – pure oscillations of constant amplitude. The instant a signal blows up, or a filter teeters on the edge of stability, the Fourier integral (or sum) refuses to converge and the lens goes dark.

This chapter widens the lens. We replace the pure oscillation $e^{\mathrm{j}\omega t}$ with a *spiral* e^{st} that is allowed to grow or shrink as it turns. The continuous version of this idea is the **Laplace transform**; the discrete version is the **z-transform**. Both are built from exactly one new move: let the frequency variable become *complex*. Everything geometric you already know about the complex plane – magnitude as radius, angle as phase, multiplication as rotate-and-scale – comes flooding back, and out of it falls a stunningly compact picture of every linear time-invariant system: a handful of dots on a plane, the **poles and zeros**.

Focus

The one idea of this chapter: *a frequency that is allowed to grow or decay is just a complex number*. Plot where a system's response blows up (poles) and where it vanishes (zeros), and you can read off stability, transient behavior, and the entire frequency response by walking around a circle. The Fourier transform is the special case where you only ever stand exactly on that circle.

15.1 Why Fourier is not enough

Recall the discrete-time Fourier transform (DTFT) of a sequence $x[n]$:

$$X(e^{\mathrm{j}\omega}) = \sum_{n=-\infty}^{\infty} x[n]\, e^{-\mathrm{j}\omega n}.$$

The kernel $e^{-\mathrm{j}\omega n}$ has magnitude exactly 1 for every n. It rotates but never shrinks. So for this sum to converge, $x[n]$ itself must eventually shrink: the sequence has to be *absolutely summable*, $\sum_n |x[n]| < \infty$.

That rules out a lot of signals we genuinely care about. The unit step $u[n]$ (all ones for $n \geq 0$) is not summable. The growing geometric sequence $2^n u[n]$ certainly is not. An unstable filter's impulse response is not. Yet these are exactly the signals whose behavior we most need to understand – often *because* they misbehave.

Intuition

The DTFT tests your signal against rotating unit-vectors. If your signal grows, no amount of rotating-without-shrinking can "catch up" to it, and the sum diverges. The fix is almost obvious once you see the problem: give the test vectors permission to grow or shrink too. Multiply the kernel by a real radius r^{-n}. If r is big enough, it tames any growth in $x[n]$ before summing. That single extra knob *is* the z-transform.

15.2 From rotating to spiraling: defining the z-transform

Take the DTFT and replace the unit-magnitude kernel $e^{\mathrm{j}\omega}$ with a general complex number $z = re^{\mathrm{j}\omega}$, where $r = |z|$ is now free to be anything positive.

Definition 15.1 (The z-transform)**.** For a discrete-time signal $x[n]$, the (bilateral) z-transform is

$$X(z) \;=\; \mathcal{Z}\{x[n]\} \;=\; \sum_{n=-\infty}^{\infty} x[n]\, z^{-n}, \qquad z \in \mathbb{C}.$$

The set of complex z for which this sum converges is the *region of convergence* (ROC).

Write $z^{-n} = r^{-n} e^{-\mathrm{j}\omega n}$ and the connection to Fourier is naked:

$$X(re^{\mathrm{j}\omega}) \;=\; \sum_{n} \big(x[n]\, r^{-n}\big)\, e^{-\mathrm{j}\omega n} \;=\; \text{DTFT of } \big(x[n]\, r^{-n}\big).$$

So the z-transform is nothing but *the Fourier transform of an exponentially reweighted version of your signal*. Choose the radius r that flattens out your signal's growth, and the Fourier machinery converges again.

What is really happening

Geometrically, the kernel $z^{-n} = (1/r)^n e^{-\mathrm{j}\omega n}$ is a **spiral**: as n marches forward it both rotates (the $e^{-\mathrm{j}\omega n}$ part) and scales (the r^{-n} part). The DTFT used a circle (constant radius); the z-transform uses a whole family of spirals, one for every radius. Standing exactly on the unit circle $r = 1$ returns you to the DTFT. The z-plane is just the complex plane with this spiral interpretation pinned to it.

Connection

This is the complex-numbers chapter cashing in. A complex number $z = re^{\mathrm{j}\omega}$ is a radius and an angle; raising it to a power, z^{-n}, multiplies the angles and the radii. "Multiplication is rotate-and-scale" is the entire reason the z-transform works. If r and ω still feel like separate gadgets, re-read the polar-form section: the z-plane will not click until $re^{\mathrm{j}\omega}$ feels like a single object.

15.2.1 A worked transform: the right-sided exponential

The most important example in the whole subject. Let $x[n] = a^n u[n]$ – a geometric sequence switched on at $n = 0$, with a a (possibly complex) constant. Then

$$X(z) = \sum_{n=0}^{\infty} a^n z^{-n} = \sum_{n=0}^{\infty} \left(\frac{a}{z}\right)^n.$$

This is a geometric series. It converges exactly when $|a/z| < 1$, i.e. $|z| > |a|$, and sums to

$$\boxed{X(z) = \frac{1}{1 - az^{-1}} = \frac{z}{z - a}, \qquad \text{ROC: } |z| > |a|.}$$

Look at what happened. The infinite signal collapsed to a tidy ratio of polynomials in z. There is a value of z that makes the denominator vanish ($z = a$): the function explodes there. That special point is a **pole**. And the numerator vanishes at $z = 0$: a **zero**. The whole signal is now encoded in two dots and an ROC.

Example 15.1 (Two transforms, same formula, different ROC)**.** Now try the *left-sided* signal $y[n] = -a^n u[-n-1]$ (nonzero only for $n \leq -1$). A short calculation gives

$$Y(z) = -\sum_{n=-\infty}^{-1} a^n z^{-n} = \frac{z}{z - a}, \qquad \text{ROC: } |z| < |a|.$$

The algebraic expression is *identical* to the right-sided case, but the ROC is the inside of the circle, not the outside. This is the crucial lesson: **the formula $X(z)$ alone does not determine the signal – you must also state the ROC**. Same poles, same zeros, opposite signals. The ROC is what tells them apart.

Common trap

Beginners memorize "$a^n u[n] \leftrightarrow z/(z-a)$" and quietly drop the ROC. Then they are baffled when an inverse transform is ambiguous or a stability test gives nonsense. The pole locations are the skeleton; the ROC is which side of each circle you live on. Always carry the ROC.

15.3 The region of convergence, geometrically

The ROC is always a set defined purely by the radius $|z|$ – never by the angle. Why? Because convergence of $\sum_n x[n] r^{-n} e^{-\mathrm{j}\omega n}$ depends on whether $x[n] r^{-n}$ is summable, and that magnitude test does not involve ω at all. Geometrically: **the ROC is a union of full circles centered at the origin – it is always an annulus** (a ring), possibly with inner radius 0 or outer radius ∞.

The shape of the ring follows directly from the "handedness" of the signal:

- **Right-sided** (causal-ish, $x[n] = 0$ for $n < N$): ROC is the *outside* of a circle, $|z| > r_{\max}$, reaching out to infinity. Growth at large n is what sets the inner radius.
- **Left-sided** (anti-causal): ROC is the *inside* of a circle, $|z| < r_{\min}$.

- **Two-sided**: ROC is a genuine annulus $r_1 < |z| < r_2$ sandwiched between an inner and outer circle (and may be empty, meaning no z-transform exists).
- **Finite-length**: ROC is the entire plane, possibly minus $z = 0$ and/or $z = \infty$.

A bedrock rule: **the ROC never contains a pole**. Poles are exactly where $X(z)$ blows up, so the sum cannot converge there. The poles are therefore the "fence posts" between which the ROC ring is stretched.

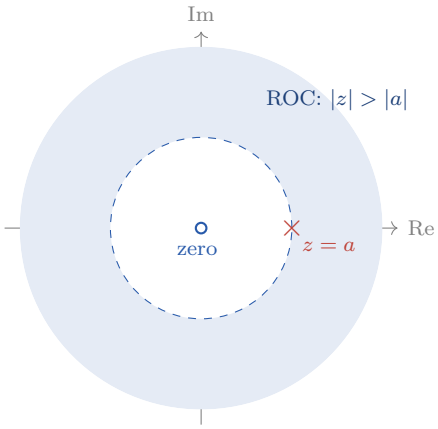


Figure 15.1: Pole-zero plot and ROC for $x[n] = a^n u[n]$ with $|a| = 0.8$. A pole ($\times$) sits at $z = a$, a zero ($\circ$) at the origin. The ROC (shaded) is everything *outside* the pole circle. Because $|a| < 1$, the ROC contains the unit circle, so the DTFT exists and the signal is stable.

Why it works

Why is "ROC contains the unit circle" the same as "the DTFT exists"? Because the unit circle *is* the DTFT, by construction (set $r = 1$ in $z = re^{j\omega}$). If $r = 1$ lies inside the convergence ring, then $\sum_n x[n]e^{-j\omega n}$ converges – which is precisely the statement that the Fourier transform of $x[n]$ exists. So the single geometric question "does the shaded ring cross the unit circle?" answers "is this signal Fourier-analyzable / stable?"

15.4 Poles, zeros, and the rational picture

For any system built from delays, adds, and constant multiplies – which is to say, *every* linear difference equation – the z-transform of the impulse response is a ratio of polynomials:

$$H(z) = \frac{B(z)}{A(z)} = \frac{b_0 + b_1 z^{-1} + \cdots + b_M z^{-M}}{1 + a_1 z^{-1} + \cdots + a_N z^{-N}}.$$

Factor numerator and denominator:

$$H(z) = G\,\frac{\prod_{k=1}^{M}(z - z_k)}{\prod_{k=1}^{N}(z - p_k)}.$$

The roots z_k of the numerator are the **zeros** (where H vanishes); the roots p_k of the denominator are the **poles** (where H blows up); G is an overall gain. A pole-zero plot draws zeros as $\circ$ and poles

as $\times$ on the z-plane. That small picture, plus the ROC, *is* the system – it contains everything the difference equation does.

Focus

Poles control what the system does on its own (the natural/transient response, resonances, stability). **Zeros control what the system rejects** (notches, nulls, the shaping of the response). Read poles for "how it rings," zeros for "what it kills."

15.4.1 Difference equation → transfer function, mechanically

The z-transform turns the awkward operation of *time delay* into plain multiplication, via the delay property:

$$x[n-k] \;\xleftrightarrow{\mathcal{Z}}\; z^{-k}\,X(z).$$

So z^{-1} is literally "the delay-by-one-sample operator." This single fact converts any difference equation into algebra.

Example 15.2 (A one-pole lowpass)**.** Consider the smoothing filter $y[n] = \alpha\, y[n-1] + (1-\alpha)\, x[n]$ with $0 < \alpha < 1$ (an exponential moving average). Transform both sides using the delay property:

$$Y(z) = \alpha z^{-1} Y(z) + (1-\alpha)X(z) \;\Longrightarrow\; H(z) = \frac{Y(z)}{X(z)} = \frac{1-\alpha}{1-\alpha z^{-1}} = \frac{(1-\alpha)\,z}{z-\alpha}.$$

One pole at $z = \alpha$, one zero at $z = 0$. Since $0 < \alpha < 1$, the pole is inside the unit circle, so a causal (right-sided) implementation has ROC $|z| > \alpha$ which contains the unit circle: the filter is stable. The closer α creeps toward 1, the closer the pole creeps to the unit circle at $z = 1$ (DC), and the more sharply the filter emphasizes low frequencies – and the longer it rings. We will see this "ringing" is exactly the pole pulling on the frequency response.

Connection

Notice the impulse response of that filter is $h[n] = (1-\alpha)\alpha^n u[n]$ – a decaying exponential – and its transform is the right-sided $z/(z-\alpha)$ form from Section 15.2. *The pole location α is the base of the exponential.* Pole near the origin ⇒ fast decay; pole near the unit circle ⇒ slow decay; pole on or outside it ⇒ no decay or blow-up. The pole's radius is the system's memory.

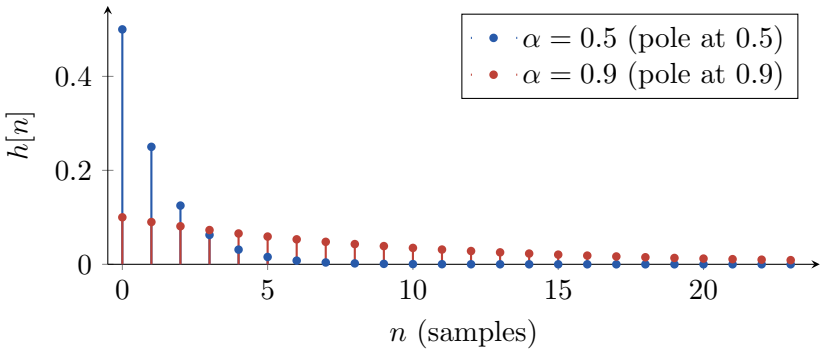


Figure 15.2: Impulse response $h[n] = (1-\alpha)\alpha^n u[n]$ of the one-pole lowpass for two pole radii. The pole at $\alpha = 0.5$ (blue) sits closer to the origin and decays fast; the pole at $\alpha = 0.9$ (red) hugs the unit circle and decays slowly, so the filter holds a longer memory and rings longer. The pole's radius is exactly the per-sample decay factor.

15.5 Stability and causality, read off the plane

This is where the pole-zero plot earns its keep. Two questions dominate filter design: *is it stable?* and *is it causal (can I run it forward in real time)?* Both are answered by where the poles sit relative to the unit circle, together with the ROC.

Formal

Stability (BIBO). A causal LTI system is stable if and only if all its poles lie *strictly inside* the unit circle, $|p_k| < 1$. Equivalently: the ROC includes the unit circle.

Causality. A system is causal if and only if its ROC is the exterior of the outermost pole (and $H(z)$ is proper).

Both at once. A causal, stable system has all poles inside the unit circle, and the ROC is the region outside the outermost pole, which therefore necessarily contains the unit circle.

Intuition

Why "inside the unit circle = stable"? A pole at radius $|p|$ contributes a term p^n to the impulse response. If $|p| < 1$, $p^n \to 0$: the natural response dies out, energy leaks away, the system settles. If $|p| > 1$, p^n grows without bound: the slightest input rings forever louder – unstable. If $|p| = 1$, p^n neither grows nor dies (a pure undamped oscillation, like an ideal resonator or integrator): marginally stable, living right on the knife's edge. The unit circle is literally the boundary between decay and growth.

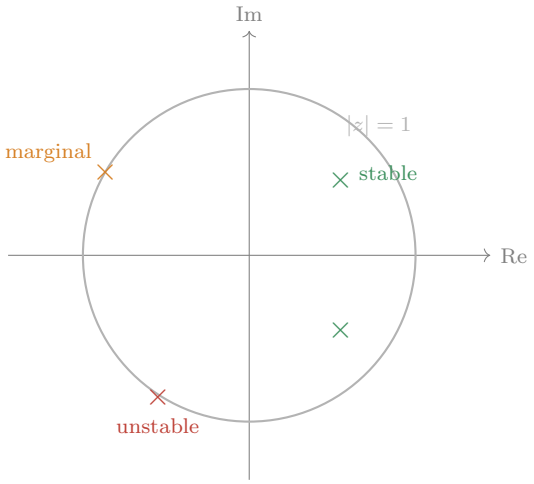


Figure 15.3: Stability at a glance. Poles inside the unit circle (green) give a decaying, stable response. A pole exactly on the circle (orange) is marginally stable – an undamped oscillation. A pole outside (red) means the response grows without bound: unstable. For a *causal* system you read this directly; zeros may sit anywhere without affecting stability.

Common trap

Zeros do *not* affect stability. You can scatter zeros anywhere – inside, on, or outside the unit circle – and a causal system stays stable as long as the poles behave. (Zeros outside the circle do affect *phase*: such a filter is "non-minimum-phase," a topic for the filter-design chapter.) Only poles vote on stability.

15.6 The frequency response is a walk around the unit circle

Here is the most beautiful payoff of the whole chapter, and the reason DSP engineers sketch pole-zero plots on napkins. The frequency response is just the z-transform evaluated on the unit circle:

$$H(e^{j\omega}) = H(z)\big|_{z=e^{j\omega}} = G\,\frac{\prod_k (e^{j\omega} - z_k)}{\prod_k (e^{j\omega} - p_k)}.$$

Each factor like $(e^{j\omega} - p_k)$ is a *vector in the complex plane*: it points from the pole p_k to the point $e^{j\omega}$ on the unit circle. Its magnitude is the *distance* from the pole to that point. So:

$$|H(e^{j\omega})| = |G|\,\frac{\prod_k (\text{distance from } e^{j\omega} \text{ to zero } z_k)}{\prod_k (\text{distance from } e^{j\omega} \text{ to pole } p_k)}.$$

What is really happening

Imagine walking a pin once around the unit circle as ω goes from 0 to 2π. At each step, stretch a rubber band from the pin to every pole and every zero. The gain is (product of zero-distances) divided by (product of pole-distances).

When you pass near a pole, that pole-distance gets tiny, you are dividing by a small

number, and the gain *spikes* – a resonance/peak.

When you pass near a zero, that zero-distance gets tiny, you multiply by a small number, and the gain *dips* – a notch (an exact null if the zero is right on the circle).

The entire frequency response is choreographed by how close your circular walk strays to each dot. That is the whole game.

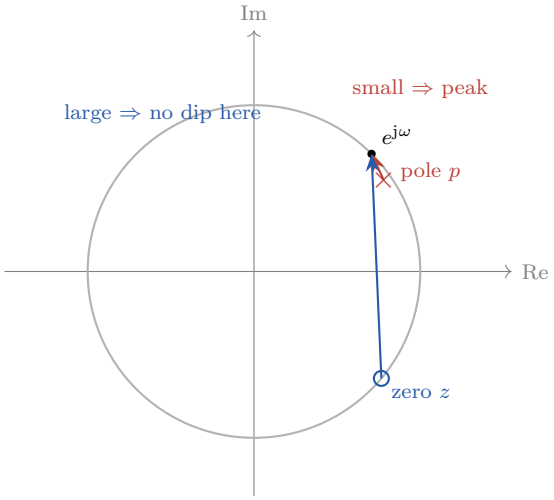


Figure 15.4: The frequency response as distances. As ω sweeps, the evaluation point $e^{j\omega}$ slides around the unit circle. The gain at each ω is the product of distances to the zeros divided by the product of distances to the poles. Near the pole p the pole-distance shrinks and the gain peaks; near the zero z (here on the circle) the gain would null out.

Example 15.3 (A resonator from a single pole pair)**.** Put a pair of complex-conjugate poles at $p = re^{\pm j\omega_0}$ with r just under 1, say $r = 0.95$, $\omega_0 = \pi/4$. The transfer function is

$$H(z) = \frac{1}{(1 - re^{j\omega_0}z^{-1})(1 - re^{-j\omega_0}z^{-1})} = \frac{1}{1 - 2r\cos\omega_0\, z^{-1} + r^2 z^{-2}}.$$

The frequency response peaks sharply at $\omega = \omega_0$, because that is where the circular walk passes closest to the poles. The peak's sharpness (its Q) is set by $1 - r$: poles hugging the circle give a tall, narrow resonance; poles further in give a broad, gentle hump. We have designed a tunable bandpass filter from two dots.

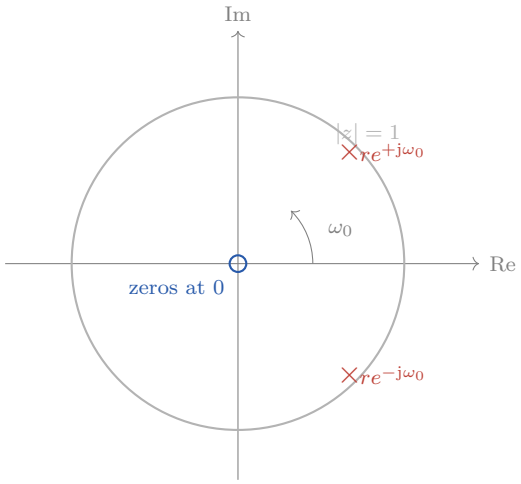


Figure 15.5: Pole-zero plot of the two-pole resonator with $r = 0.95$, $\omega_0 = \pi/4$. The conjugate poles sit just inside the unit circle at angle $\pm\omega_0$; the double zero lies at the origin. A circular walk passes nearest the poles at $\omega = \omega_0$, which is where the magnitude response of Figure 15.6 spikes.

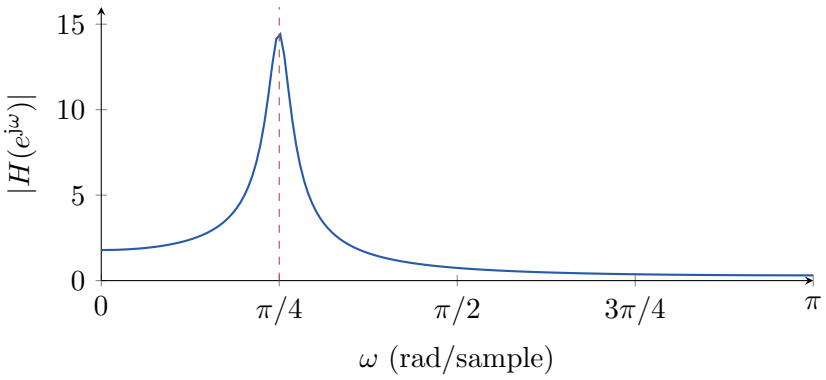


Figure 15.6: Magnitude response of the two-pole resonator with $r = 0.95$, $\omega_0 = \pi/4$ (dashed line). The peak sits exactly where the walk around the unit circle passes nearest the poles. Push r toward 1 and this peak grows taller and narrower.

Why it works

Why complex-*conjugate* pairs? A real difference equation has real coefficients, and the roots of a real polynomial come in conjugate pairs. Geometrically, a conjugate pair sits symmetrically above and below the real axis, so their two resonances land at $+\omega_0$ and $-\omega_0$ – which is exactly the symmetry a real signal's spectrum already has. A lone complex pole would describe a complex-valued filter; pairing it up keeps everything real. This is the complex-numbers chapter's "conjugates undo imaginary parts" showing up as hardware reality.

15.7 The Laplace transform: the continuous twin

Everything above has a continuous-time mirror image. For continuous signals we cannot use z^{-n} (there is no integer n), so we use the continuous spiral e^{-st} where $s = \sigma + \mathrm{j}\omega$ is a complex frequency: σ is a growth/decay rate (nepers per second) and ω is the ordinary angular frequency.

Definition 15.2 (The Laplace transform)**.** For a continuous-time signal $x(t)$, the (bilateral) Laplace transform is

$$X(s) = \mathcal{L}\{x(t)\} = \int_{-\infty}^{\infty} x(t)\, e^{-st}\, \mathrm{d}t, \qquad s = \sigma + \mathrm{j}\omega \in \mathbb{C},$$

with an ROC consisting of the σ for which the integral converges.

Set $\sigma = 0$ (i.e. $s = \mathrm{j}\omega$, evaluate on the imaginary axis) and you recover the continuous-time Fourier transform $X(\mathrm{j}\omega) = \int x(t)e^{-\mathrm{j}\omega t}\mathrm{d}t$. The $e^{-\sigma t}$ factor is the continuous analog of the r^{-n} taming factor: it weights the signal so the integral can converge even when $x(t)$ grows.

Focus

The dictionary between the two worlds:

	Continuous (Laplace)	**Discrete (z)**
complex frequency	$s = \sigma + \mathrm{j}\omega$	$z = re^{\mathrm{j}\omega}$
Fourier lives on	imaginary axis $\sigma = 0$	unit circle $r = 1$
stable region	left half-plane $\mathrm{Re}(s) < 0$	inside circle $\lvert z\rvert < 1$
"do nothing" point	$s = 0$	$z = 1$
derivative / delay	$\frac{\mathrm{d}}{\mathrm{d}t} \leftrightarrow s$	delay $\leftrightarrow z^{-1}$

What is really happening

The single most useful fact about Laplace: *differentiation becomes multiplication by s.* If $x(t) \leftrightarrow X(s)$ then $x'(t) \leftrightarrow sX(s)$ (zero initial conditions). That is why Laplace eats differential equations alive – a circuit's integro-differential equations turn into ordinary algebra in s, you solve, then transform back. The z-transform does the same trick for difference equations with z^{-1} as the delay.

15.7.1 Stability in the s-plane

The mapping of stability is just as clean, but the boundary changes shape. In the z-plane the dividing line is the unit circle; in the s-plane it is the **imaginary axis**. A pole at $s = \sigma + \mathrm{j}\omega$ contributes $e^{\sigma t}e^{\mathrm{j}\omega t}$ to the response: if $\sigma < 0$ it decays (stable), if $\sigma > 0$ it grows (unstable), if $\sigma = 0$ it oscillates forever (marginal). So for a causal continuous system:

$$\text{stable} \iff \text{all poles in the left half-plane } \mathrm{Re}(s_k) < 0.$$

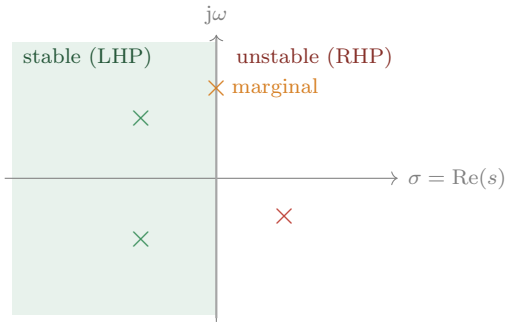


Figure 15.7: The s-plane stability picture. Poles in the left half-plane (green region) give decaying, stable behavior; poles in the right half-plane grow; poles on the imaginary axis oscillate undamped. Compare Figure 15.3: the unit circle has been "unrolled" into the imaginary axis, its interior into the left half-plane.

Why it works

Why does the inside of the circle map to the left half-plane? Because sampling links the two by $z = e^{sT}$, where T is the sample period. Write $s = \sigma + \mathrm{j}\omega$:

$$z = e^{\sigma T} e^{\mathrm{j}\omega T} \;\Rightarrow\; |z| = e^{\sigma T}.$$

So $\sigma < 0$ (left half-plane) gives $|z| < 1$ (inside the circle), $\sigma = 0$ (the imaginary axis) gives $|z| = 1$ (the unit circle), and $\sigma > 0$ gives $|z| > 1$. The exponential e^{sT} literally wraps the s-plane's vertical strips around the z-plane's circles. This is the seed of digital filter design: take an analog filter you trust in the s-plane and map it into the z-plane.

History

Pierre-Simon Laplace developed his transform in the 1780s as a tool for probability and celestial mechanics – long before electronics. Oliver Heaviside, the self-taught Victorian telegraph engineer, turned it into an *operational calculus* in the 1880s–90s, boldly treating $\frac{\mathrm{d}}{\mathrm{d}t}$ as an algebraic symbol p (essentially s) to solve circuit equations. Mathematicians were scandalized by his lack of rigor; engineers were delighted by his results. "Shall I refuse my dinner because I do not fully understand the process of digestion?" he retorted. The rigor was supplied later; the z-transform arrived in the 1950s–60s as sampled-data control and digital computers made discrete time the natural setting – the same Cooley–Tukey era that gave us the FFT.

15.8 Transfer functions and block diagrams

A **transfer function** $H(z)$ (or $H(s)$) is the z- (or Laplace-) transform of a system's impulse response. Equivalently, it is the ratio output/input in the transform domain: $Y(z) = H(z)X(z)$. The convolution in time becomes multiplication in the transform domain – this is the convolution theorem from the Fourier chapter, now extended off the circle.

This multiplicative structure is why transfer functions are the lingua franca of system design. Cascade two systems and their transfer functions *multiply*: $H(z) = H_1(z)H_2(z)$. Put them in parallel and

they *add*: $H(z) = H_1(z) + H_2(z)$. Wrap one in a feedback loop with H_2 and you get the feedback formula $H = H_1/(1 + H_1H_2)$. Complex systems are built like LEGO from these three rules.

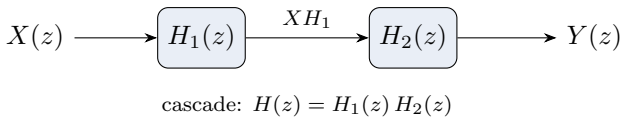


Figure 15.8: A cascade of two systems. In the transform domain their transfer functions simply multiply, and their pole-zero plots overlay (the combined system has the union of all poles and all zeros). This is why filter design so often proceeds one second-order "biquad" section at a time.

Application

Analog circuits. An RC lowpass has transfer function $H(s) = 1/(1 + sRC)$: one pole at $s = -1/RC$ on the negative real axis, deep in the stable left half-plane. The pole's distance from the origin sets the cutoff frequency $\omega_c = 1/RC$. An RLC circuit adds a conjugate pole pair and can resonate – exactly the continuous cousin of Example 15.3. Every analog filter you will ever meet is a pattern of dots in the s-plane.

15.9 Inverting the transform: partial fractions

To get back to the time domain you usually do not compute a contour integral – you do **partial fractions**, breaking $H(z)$ into a sum of one-pole pieces whose inverses you already know.

Example 15.4 (Inverse by partial fractions)**.** Take

$$H(z) = \frac{1}{(1 - \frac{1}{2}z^{-1})(1 - \frac{1}{3}z^{-1})}.$$

Expand into partial fractions in z^{-1}:

$$H(z) = \frac{A}{1 - \frac{1}{2}z^{-1}} + \frac{B}{1 - \frac{1}{3}z^{-1}}, \quad A = 3,\ B = -2.$$

Assuming a causal (right-sided) system, each term inverts via the standard pair $\frac{1}{1-az^{-1}} \leftrightarrow a^n u[n]$:

$$h[n] = 3\left(\tfrac{1}{2}\right)^n u[n] - 2\left(\tfrac{1}{3}\right)^n u[n].$$

Both poles $(\frac{1}{2}, \frac{1}{3})$ are inside the unit circle, so $h[n] \to 0$: stable, as the pole positions promised. The decay is dominated by the pole nearest the unit circle $(\frac{1}{2})$, the slowest-decaying mode.

Common trap

The same $H(z)$ with a different ROC inverts to a different $h[n]$ (left-sided terms use $-a^n u[-n-1]$). Always pin the ROC before inverting, and choose the term form (right- vs left-sided) to match it. Forgetting this is the number-one inverse-transform mistake.

15.10 Toward the wrist tracker

Toward the wrist tracker

Every filter on the wrist controller – the lowpass that removes hand tremor from the accelerometer, the highpass that strips gravity to leave dynamic motion, the bandpass that isolates a tap's frequency band, the smoother on the EMG envelope – is a little pole-zero plot living in a microcontroller. When you write

```
y += alpha*(x - y);
```

on the Arduino, you have placed a single pole at $z = 1 - \alpha$ (an exponential smoother, Example 15.2). Choosing α *is* choosing where that pole sits between the origin (snappy, noisy) and the unit circle at $z = 1$ (smooth, sluggish, laggy). The pole-zero picture turns "how laggy should my gesture detector feel?" into a single geometric dial.

The stability rule matters in hardware specifically. An IIR (recursive) filter with a pole accidentally pushed outside the unit circle by a coefficient typo, or by fixed-point rounding on an 8-bit MCU, will *self-oscillate* – your gesture signal explodes into garbage with no input at all. "Keep the poles inside the circle" is not abstract; it is the difference between a filter that works and a sensor that screams.

Build it (hardware)

A practical second-order "biquad" bandpass to isolate a finger-tap transient (ω_0 tuned to the tap's dominant band, r near 0.9–0.97 for selectivity), realized directly from the two-pole resonator of Example 15.3:

```
// One biquad bandpass section, Direct Form I, float math.
// Poles at r*e^{+-j w0}; a zero pair shapes the passband.
float b0, b1, b2, a1, a2;   // precomputed from r, w0
float x1=0, x2=0, y1=0, y2=0;

float biquad(float x) {
    float y = b0*x + b1*x1 + b2*x2 - a1*y1 - a2*y2;
    x2 = x1; x1 = x;        // shift input history
    y2 = y1; y1 = y;        // shift output history
    return y;
}
// Coeffs for a resonator bandpass:
//   a1 = -2*r*cos(w0);  a2 = r*r;        // denominator -> the two poles
//   b0 = 1 - r;  b1 = 0;  b2 = -(1-r);   // zeros at z=+-1: DC/Nyquist nulls
```

Sanity check on-device: with $r < 1$ the recursion must settle to zero after an impulse. If it rings forever or grows, a coefficient is wrong and a pole has escaped the circle. On fixed-point hardware, leave headroom: keep $r \leq 0.97$ so rounding cannot nudge $|p|$ past 1.

Common trap

The hard sensor limit. None of this transform machinery conjures information the sensor never captured. A wrist-worn IMU senses the forearm's gross motion and orientation; no

amount of clever pole placement recovers *individual finger joint angles* from it – that signal simply is not in the accelerometer stream. Distinguishing which finger tapped needs forearm *EMG* or *force-myography* (muscle/tendon signals), and *continuous* finger pose needs sensors on the fingers themselves (e.g. instrumented rings). The z-transform is how you clean and shape whatever the sensor genuinely sees; it is not a substitute for the right sensor.

15.11 Summary

- The z-transform $X(z) = \sum_n x[n]z^{-n}$ generalizes the DTFT by letting the frequency variable be a complex number $z = re^{\mathrm{j}\omega}$ – a *spiral* kernel that can grow or shrink. It is the DTFT of the reweighted signal $x[n]r^{-n}$.
- The **region of convergence** is always an annulus (ring) and never contains a pole. Right-sided signals give outside-the-circle ROCs, left-sided give inside. The formula plus the ROC together determine the signal.
- A system's transfer function is a ratio of polynomials; its **poles** ($\times$) and **zeros** ($\circ$) on the z-plane, plus the ROC, encode it completely. Poles set resonance and stability; zeros set notches.
- **Stability + causality** $\iff$ all poles strictly inside the unit circle (ROC contains the unit circle). The unit circle is the boundary between decay and growth.
- The **frequency response** is the z-transform on the unit circle; its magnitude is the product of zero-distances over pole-distances as you walk the circle – peaks near poles, dips near zeros.
- The **Laplace transform** is the continuous analog (e^{-st} kernel, $s = \sigma + \mathrm{j}\omega$). Fourier lives on the imaginary axis; stability is the left half-plane. Sampling links the two via $z = e^{sT}$, wrapping the LHP onto the disk.

15.12 Exercises

Exercise 15.1. Compute the z-transform and ROC of $x[n] = \left(\frac{3}{4}\right)^n u[n]$. Mark its pole and zero on a sketch of the z-plane and state whether the DTFT exists.

Exercise 15.2. The sequence $x[n] = \left(\frac{3}{4}\right)^n u[n] + 2^n u[n]$ is a sum of a decaying and a growing term. Find $X(z)$, locate both poles, and determine the ROC. Does the DTFT exist? Explain geometrically using the unit circle.

Exercise 15.3. Two systems have the *same* pole-zero diagram (a single pole at $z = 0.6$, a zero at the origin) but different ROCs. Write the two possible impulse responses and say which one is stable and which is causal.

Exercise 15.4. A difference equation reads $y[n] = 0.9\,y[n-1] - 0.81\,y[n-2] + x[n]$. Find $H(z)$, its two poles (in polar form $re^{\pm \mathrm{j}\omega_0}$), and predict the frequency ω_0 at which the magnitude response peaks. Is the system stable?

Exercise 15.5. For the one-pole lowpass $H(z) = (1-\alpha)/(1-\alpha z^{-1})$, compute $|H(e^{\mathrm{j}\omega})|$ at $\omega = 0$ and $\omega = \pi$, and show the ratio (the DC-to-Nyquist attenuation) grows as $\alpha \to 1$. Interpret using the pole-to-circle distance.

Exercise 15.6. Using partial fractions, find the causal inverse z-transform of $H(z) = \dfrac{1}{(1-\frac{1}{4}z^{-1})(1+\frac{1}{2}z^{-1})}$. Confirm $h[n] \to 0$ and identify the slowest-decaying mode.

Exercise 15.7. Show that a real-coefficient filter must have poles and zeros that are either real or come in complex-conjugate pairs. (Hint: take the complex conjugate of the polynomial equation $A(p) = 0$ and use that the coefficients are real.)

Exercise 15.8. An analog RC lowpass has $H(s) = 1/(1 + sRC)$. Find its pole in the s-plane, its cutoff ω_c, and the magnitude $|H(\mathrm{j}\omega)|$. Sketch the pole and explain why the system is unconditionally stable.

Exercise 15.9. Using the sampling relation $z = e^{sT}$ with $T = 1$ ms, map the analog pole $s = -200 + \mathrm{j}2\pi(1000)$ into the z-plane. Compute $|z|$ and the angle; is the resulting discrete pole inside the unit circle? What discrete-time resonant frequency (in rad/sample) does it correspond to?

Exercise 15.10. Place two zeros exactly on the unit circle at $z = e^{\pm \mathrm{j}\pi/2}$ and no poles (other than at the origin). Write $H(z)$, identify what frequency it nulls, and explain in one sentence how you would use this as a notch to kill a single interfering tone in an EMG recording (e.g. mains pickup after appropriate frequency scaling).

Exercise 15.11. (Build/reflection.) On an 8-bit microcontroller you implement $y[n] = 2r\cos(\omega_0)\, y[n-1] - r^2 y[n-2] + x[n]$ with $r = 0.999$. Explain why this is risky in fixed-point arithmetic, what failure you would observe, and how moving the pole inward (smaller r) trades resonance sharpness for safety. Relate your answer to the wrist controller's tap-detection filter.

Chapter 16

Designing Filters

Up to now we have mostly *analyzed* signals: we took something the world handed us — a chord, an accelerometer trace, an image — and asked what frequencies were hiding inside it. A filter does the opposite. A filter is a machine we *design on purpose* to keep some frequencies and throw others away. It is the first piece of signal processing where you are no longer a spectator; you are an engineer with a target in mind.

And you already own all the parts. In the chapter on LTI systems we learned that any linear, time-invariant box is completely described by its impulse response $h[n]$, that running a signal through it is convolution $y = x * h$, and — the magic key — that convolution in time is *multiplication* in frequency:

$$Y(\omega) \;=\; X(\omega)\,H(\omega).$$

That single equation *is* the theory of filtering. The factor $H(\omega)$, the frequency response, is a knob you get to draw: a curve that says "multiply this frequency by 1, that one by 0, this one by 0.5." Designing a filter means choosing an $H(\omega)$ shaped the way you want, and then finding a real, finite, runnable $h[n]$ (or a difference equation) that produces it. This chapter is about how to do that choosing well.

Focus

A filter multiplies each frequency of the input by a number you chose in advance. The wish-list curve is $H(\omega)$. "Filter design" is the craft of turning that wish into a short list of coefficients a chip can actually multiply and add — trading away a little of the wish to make it cheap, stable, and causal.

16.1 What a filter is supposed to do

Forget formulas for a second and think in pictures. A signal's spectrum is a landscape: energy piled up at some frequencies, flat at others. A filter is a *mask* you lay over that landscape. Where the mask is 1 the energy passes untouched; where the mask is 0 the energy is erased; in between it is scaled. Four mask shapes cover almost everything you will ever want.

- **Low-pass**: keep low frequencies, kill high ones. The mask is a plateau near $\omega = 0$ that drops to zero. This is the workhorse — smoothing, anti-aliasing, removing hiss, taming sensor jitter.

- **High-pass**: keep high frequencies, kill low ones. The mask is zero near DC and rises to a plateau. Use it to remove a slow drift or a DC offset while keeping the wiggles.
- **Band-pass**: keep a middle band, kill everything above and below. A plateau floating between two cliffs. Radios tune with these; so does an EMG front end that only wants the 20–450 Hz muscle band.
- **Band-stop** (notch): the inverse — kill one narrow band, keep the rest. The classic job is murdering 50/60 Hz mains hum without touching the signal around it.

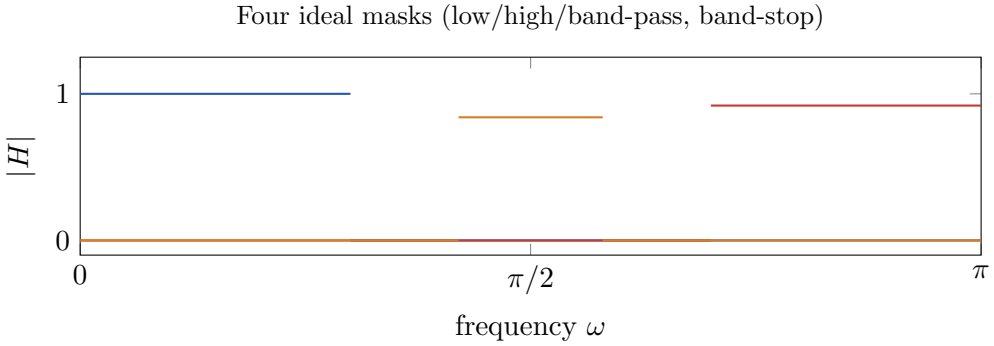


Figure 16.1: Idealized magnitude responses. Blue: low-pass. Red: high-pass (drawn slightly low so the lines do not overlap). Orange: band-pass. A band-stop is just "1 minus a band-pass".

Intuition

A high-pass is a low-pass turned inside out, and a band-stop is a band-pass turned inside out. Spectrally, "keep the rest" means subtract from the all-pass mask $H = 1$: if $H_{\text{LP}}(\omega)$ keeps the lows, then $1 - H_{\text{LP}}(\omega)$ keeps the highs. In time that is $\delta[n] - h_{\text{LP}}[n]$. So if you can build one good low-pass, you can build all four. We will lean on this constantly.

Connection

The masking picture is the convolution theorem from the Fourier chapter wearing work clothes. Multiplying spectra ($Y = XH$) corresponds to convolving in time ($y = x * h$). Everything in this chapter is just: "what time-domain $h[n]$ multiplies the spectrum by the shape I want?" That is an inverse-Fourier question, and the linear-algebra book's view of the Fourier transform as an orthogonal change of basis is exactly why the answer is unique.

16.1.1 Why the perfect mask is impossible

The masks in Figure 16.1 have vertical cliffs: pass at full strength up to a frequency, then *instantly* zero. That perfect "brick wall" is the dream. Here is the hard truth that organizes the whole field: you cannot build it with a finite, real filter, and you cannot even get arbitrarily close without paying.

The reason is the uncertainty relationship between time and frequency. A brick wall in frequency is a rect function. Its inverse Fourier transform is a sinc — and the sinc is infinitely long and two-sided.

Why it works

An ideal low-pass with cutoff ω_c has frequency response $H(\omega) = \mathrm{rect}(\omega/2\omega_c)$ (one inside the band, zero outside). Inverse-transforming gives the impulse response

$$h[n] \;=\; \frac{\omega_c}{\pi}\,\mathrm{sinc}\!\left(\frac{\omega_c\, n}{\pi}\right) \;=\; \frac{\sin(\omega_c n)}{\pi n}.$$

Look at what this demands. It is nonzero for *all* n, from $-\infty$ to $+\infty$. To compute even one output sample you would need every past *and every future* input sample, weighted by a slowly decaying ripple. That filter is non-causal (needs the future) and infinitely long (needs infinite memory and infinite multiplies). Real hardware has neither. Every practical design is a negotiation: how do we chop, shift, and smooth that ideal sinc into something finite and causal while keeping the mask close to what we wanted?

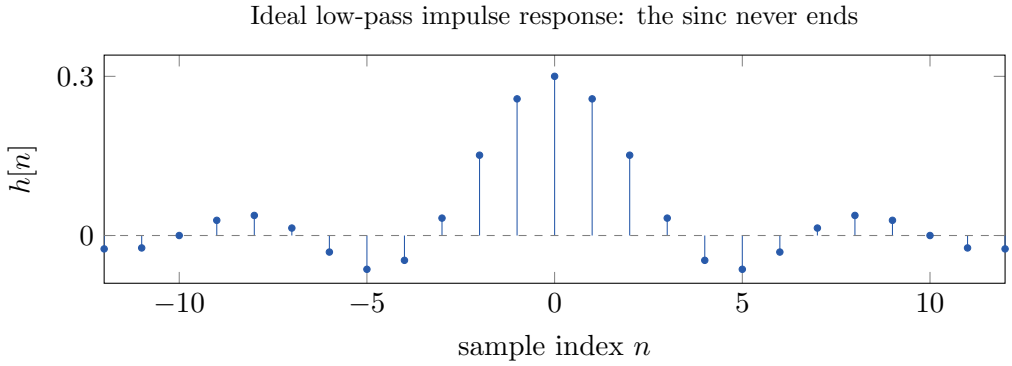


Figure 16.2: The impulse response of the ideal low-pass ($\omega_c = 0.3\pi$) is a sampled sinc. It is symmetric about $n = 0$, decays only as $1/n$, and goes on forever in both directions. Truncating and shifting it is the entire "window method" below.

16.2 Two families: FIR and IIR

Every digital filter falls into one of two families, distinguished by one question: *does the impulse response ever truly end?*

A **Finite Impulse Response** (FIR) filter has an $h[n]$ that is nonzero for only finitely many taps. You compute each output as a plain weighted sum of recent inputs:

$$y[n] \;=\; \sum_{k=0}^{M} b_k\, x[n-k].$$

There is no feedback. Kick it with an impulse and the output rings for exactly $M+1$ samples, then is dead silent forever. The coefficients b_k *are* the impulse response: $h[k] = b_k$.

An **Infinite Impulse Response** (IIR) filter feeds its own past outputs back in:

$$y[n] \;=\; \sum_{k=0}^{M} b_k\, x[n-k] \;-\; \sum_{k=1}^{P} a_k\, y[n-k].$$

Because the output depends on earlier outputs, a single impulse can echo around the feedback loop forever (decaying, if the filter is stable). A two-tap IIR can produce an impulse response that never exactly reaches zero — hence "infinite".

Intuition

FIR is a *moving window* that only ever looks at the input: a tapped delay line you weight and sum. IIR is a *resonator*: it has internal state that keeps ringing, like a struck bell or an RC circuit relaxing. The bell's tone outlives the strike — that is feedback, that is the "infinite" tail.

What is really happening

Both filters are still just convolution, $y = x * h$. The difference is *how* you compute that convolution. FIR computes it directly, multiplying out a finite list of taps. IIR computes the *same kind of* convolution implicitly through a recurrence: a short feedback formula that, if you unrolled it forever, would reproduce an infinitely long $h[n]$. IIR is a clever shorthand — it buys a long, sharp impulse response with very few coefficients, and the bill for that bargain is the risk of instability and nonlinear phase.

16.2.1 The tradeoffs, plainly

Neither family is "better"; they trade different things. Here is the ledger.

	FIR	**IIR**
Sharpness per coefficient	Low — needs many taps for a steep cutoff	High — a few coefficients give a very steep cutoff
Stability	Always stable (no feedback)	Can blow up if poorly designed or quantized
Phase	Can be *exactly* linear (constant delay)	Nonlinear phase; warps waveform shape
Memory / compute	More (a long delay line)	Less
Design intuition	Direct: shape the taps	Indirect: place poles and zeros
Analog heritage	None really	Mirrors classic analog filters (RC, LC, Butterworth)

Common trap

The seductive trap is to reach for IIR *because* it is cheap, then discover your nicely-shaped signal has its peaks shifted relative to its valleys. IIR filters delay different frequencies by different amounts (nonlinear phase). For audio loudness that is usually fine — the ear is fairly deaf to phase. For anything where the *shape* of the waveform matters — an ECG QRS

complex, a gesture's sharp accelerometer spike, lining up two channels — that smearing is a real bug. When shape matters, prefer FIR.

Toward the wrist tracker

For the wrist gesture controller these tradeoffs are not academic. We will run a low-pass on the accelerometer to separate gravity (slow tilt) from motion (fast), a band-pass on any EMG/FMG channel to isolate the muscle band, and a notch to kill mains hum. On a small microcontroller, compute and memory are scarce, which pulls toward IIR. But a *tap* or *flick* gesture is recognized by the precise timing and shape of an acceleration spike, and IIR's phase smearing can blur two near-simultaneous taps into one. So a typical wrist build mixes both: cheap IIR biquads for the EMG band-pass and the hum notch (where only the envelope matters), and a short linear-phase FIR on the accelerometer (where spike timing must stay aligned across axes). Choosing per channel, with the table above in hand, is the real engineering.

16.3 Specifying a filter before you design it

You cannot hit a target you have not drawn. Before any design method, you write a *specification*: a tolerance template the magnitude response must live inside. Because the brick wall is impossible (Section 16.1.1), the spec deliberately leaves slack. Five numbers describe a low-pass spec.

- **Passband edge** ω_p: up to here, keep the signal.
- **Stopband edge** ω_s: beyond here, kill the signal.
- **Transition band** (ω_p, ω_s): the ramp in between, where we make no promises. Its *width* is what costs you taps. Narrow transition = expensive filter.
- **Passband ripple** δ_p: how much the gain is allowed to wobble around 1 in the passband (often quoted in dB).
- **Stopband attenuation** δ_s: how far down the leftover must be pushed in the stopband (e.g. $-60\,\text{dB}$ means leakage no larger than 0.001).

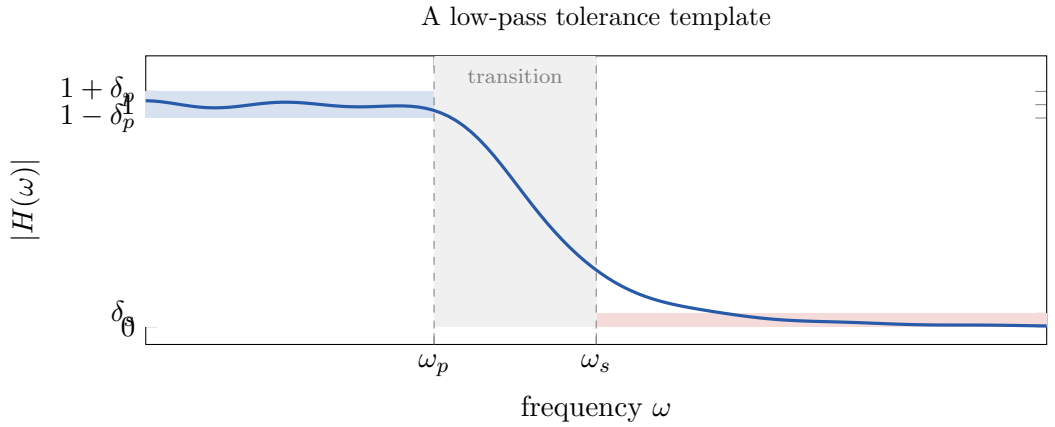


Figure 16.3: The spec is a corridor, not a curve. The response (blue) must stay inside the passband ripple band (top left), may do anything in the gray transition, and must stay under the stopband ceiling δ_s (bottom right). Tightening any of δ_p, δ_s, or the transition width raises the filter's order.

Formal

Magnitudes are usually quoted in decibels, $|H|_{\text{dB}} = 20\log_{10}|H|$. Handy landmarks: a gain of 1 is 0 dB; the half-power point $|H| = 1/\sqrt{2}$ is -3 dB (this defines the conventional "cutoff" for analog-style filters); $|H| = 0.1$ is -20 dB; $|H| = 0.001$ is -60 dB. Passband ripple of 0.1 dB means the gain stays within $\pm 1.2\%$ of unity. The decibel scale is used because attenuation requirements span many orders of magnitude and our perception of "how much is left" is roughly logarithmic.

16.4 Linear phase: the FIR superpower

Before designing anything, understand the one property that makes FIR worth its extra taps: it can have *exactly linear phase*, which means *constant delay for every frequency* — and that means it does not distort waveform shape.

What is phase doing? The frequency response is complex, $H(\omega) = |H(\omega)|\, e^{\mathrm{j}\phi(\omega)}$. The magnitude $|H|$ says how much each frequency is scaled; the phase $\phi(\omega)$ says how much each frequency is *delayed*. A sinusoid at frequency ω passing through the filter comes out shifted by $\phi(\omega)$ radians, i.e. delayed in time by $-\phi(\omega)/\omega$ samples. This is the *group delay*.

Why it works

If $\phi(\omega) = -\alpha\,\omega$ is a straight line through the origin, then the delay $-\phi(\omega)/\omega = \alpha$ is the *same constant* for every frequency. Every component of the signal is held back by exactly α samples, so they all stay in step. The waveform comes out delayed but otherwise *unchanged in shape*. If instead $\phi(\omega)$ bends, low and high components are delayed by different amounts, they slide out of alignment, and a crisp spike spreads into a lopsided smear — even when the magnitude response is perfect. Constant delay = no shape distortion. That is the prize.

Here is the key fact: an FIR filter has exactly linear phase *whenever its taps are symmetric* (or

antisymmetric) about their center, $h[k] = h[M-k]$. Symmetry is a geometric condition you can guarantee by construction. The ideal sinc in Figure 16.2 is symmetric about $n = 0$; if we keep that symmetry while truncating and just shift the whole thing right so it starts at $n = 0$ (making it causal), linear phase is preserved automatically. The shift is exactly the constant group delay $\alpha = M/2$ samples.

Intuition

Symmetric taps mean the filter treats "a bit before the center" and "a bit after the center" identically. There is no front-to-back lopsidedness to make one frequency lag another. The filter's only effect on timing is a bulk delay equal to half its length. IIR filters, built from feedback, cannot generally be symmetric in this way, which is the deep reason they have nonlinear phase.

Connection

This is the inner-product / symmetry idea from the linear algebra book. A real-symmetric tap vector projects onto cosines and sines in a balanced way; the imaginary parts of the response arrange themselves into a pure linear phase term $e^{-\mathrm{j}\alpha\omega}$ that factors out, leaving a purely real "zero-phase" gain. Symmetry in the coefficient vector becomes linearity in the phase.

16.5 FIR design by the window method

Now we actually build something. The window method is the most intuitive FIR design technique, and it falls right out of everything above:

1. Start from the ideal filter's impulse response (the infinite sinc).
2. **Truncate** it to $M+1$ taps — chop off the tails.
3. **Shift** it to start at $n = 0$ so the filter is causal.
4. Multiply the kept taps by a smooth **window** to tame the damage done by the chop.

Steps 1–3 alone already give a working filter. Let us see why step 4 is needed.

16.5.1 Truncation, and the ripples it causes

Chopping the sinc to a finite length is, in the frequency domain, multiplying the ideal rect mask by something not nice at all. Truncating $h[n]$ to a window of length N is the same as *multiplying* the infinite ideal $h[n]$ by a rectangular window $w[n]$ (one inside the kept range, zero outside). By the convolution theorem, multiplication in time is convolution in frequency:

$$H_{\text{trunc}}(\omega) \;=\; H_{\text{ideal}}(\omega) * W(\omega).$$

The spectrum of a rectangular window is a sinc-shaped curve with a tall main lobe and slowly-decaying side lobes. Convolving the clean brick wall with that smears the cliff into a ramp and — worse — sprinkles *ringing ripples* on both sides of the edge. This overshoot near the cliff is the

Gibbs phenomenon, and it is stubborn: making the filter longer makes the ripples narrower but *not* shorter. The first overshoot stays near 9% no matter how many taps you add.

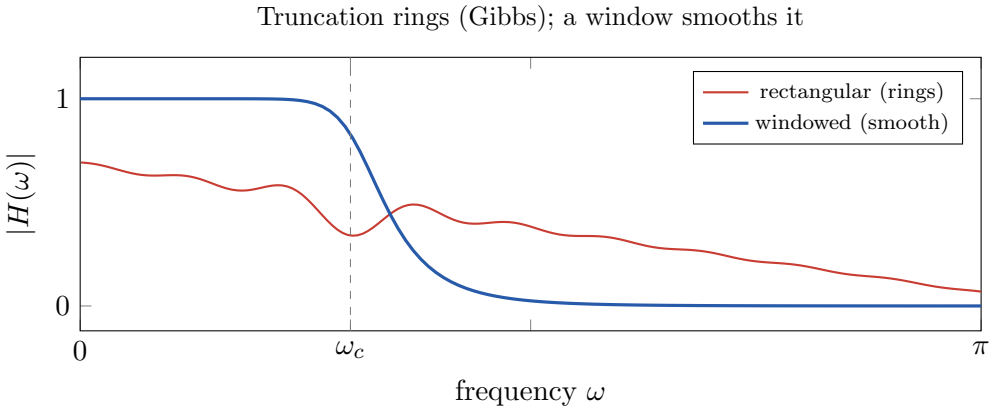


Figure 16.4: Schematic of the window method's effect near the cutoff. A bare rectangular truncation (red) overshoots and rings around the edge. A tapered window (blue) trades a slightly wider transition for a much cleaner, ripple-free stopband. The plotted curves are illustrative shapes, not an exact computation.

16.5.2 Why a window helps, and which one

The cure is to stop chopping abruptly. Instead of multiplying by a hard rectangle, multiply the ideal taps by a window $w[n]$ that *tapers smoothly* to zero at the ends. A gentler taper has a spectrum with much lower side lobes, so the smearing it convolves into the response has far weaker ripples.

Why it works

Everything hinges on one tradeoff inside the window's own spectrum: *main-lobe width versus side-lobe height.* A wide, smooth taper (low side lobes) gives a clean stopband — deep attenuation, tiny ripples — but its fat main lobe smears the cliff into a wide, gentle transition band. A narrow taper (the rectangle is the extreme) gives the sharpest possible transition but tall side lobes and ugly ringing. *You are buying stopband cleanliness with transition width.* Choosing a window is choosing where on that line to sit.

A few standard windows, from sharpest-but-leakiest to smoothest:

Window	**Peak side lobe**	**Character**
Rectangular	-13 dB	sharpest transition, worst stopband (rings)
Hann	-31 dB	gentle all-rounder, easy default
Hamming	-41 dB	like Hann but flatter stopband near the edge
Blackman	-58 dB	very clean stopband, wider transition
Kaiser (β)	tunable	one knob β slides the whole tradeoff

The Kaiser window deserves a note: its single parameter β lets you dial exactly between "sharp" and "clean", and Kaiser gave simple formulas that take your spec (desired stopband attenuation and transition width) and spit out both β *and* the number of taps you need. That makes it the practical default when you have a real spec to hit.

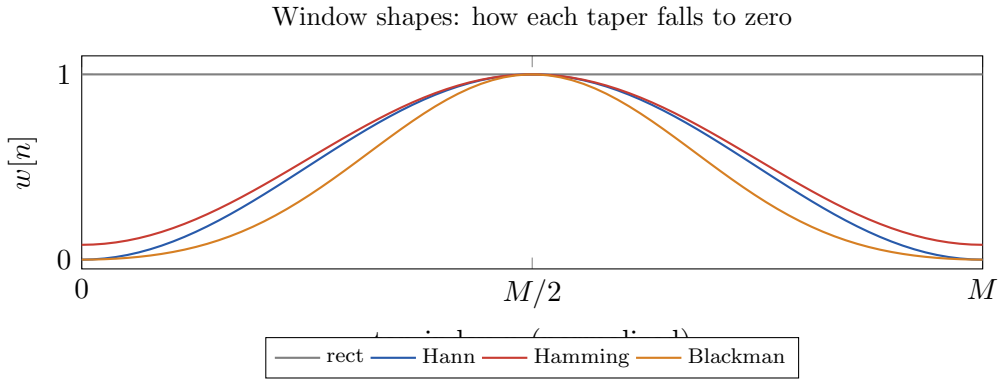


Figure 16.5: The same length, four tapers. The rectangle keeps every tap at full weight then drops to zero at the ends — that cliff is what rings. Hann, Hamming, and Blackman fade the end taps out, each a little more gently, buying lower side lobes at the cost of a wider main lobe.

16.5.3 Worked design: a low-pass with the Hamming window

Let us design a real filter end to end. **Spec**: sampling rate $f_s = 1000\,\text{Hz}$; pass everything below about 100 Hz; suppress everything above about 200 Hz by at least 40 dB. (This is exactly the kind of accelerometer low-pass we will want.)

Step 1 — normalize. Digital filters live in normalized frequency $\omega = 2\pi f/f_s$ (radians/sample), or in cycles/sample f/f_s. The natural cutoff to aim the sinc at is the *middle* of the transition band, $f_c = 150\,\text{Hz}$, so $\omega_c = 2\pi(150/1000) = 0.3\pi$.

Step 2 — choose length from the window. A Hamming window gives about 53 dB stopband attenuation (plenty for our 40 dB) and a transition width of roughly $\Delta\omega \approx 6.6\pi/M$ in normalized terms. We want the transition to fit inside 100–200 Hz, i.e. $\Delta\omega = 2\pi(100/1000) = 0.2\pi$. Solving $0.2\pi = 6.6\pi/M$ gives $M \approx 33$, so take $M = 34$ (an even M gives an odd number of taps $M+1 = 35$, which centers a tap exactly at the middle — convenient for a clean integer delay).

Step 3 — the ideal taps, truncated and shifted. Center the sinc at $\alpha = M/2 = 17$ and evaluate:

$$h_{\text{ideal}}[n] \;=\; \frac{\sin\big(\omega_c(n-\alpha)\big)}{\pi(n-\alpha)}, \qquad n = 0, 1, \ldots, 34,$$

with the removable point $h_{\text{ideal}}[\alpha] = \omega_c/\pi$.

Step 4 — apply the window. Multiply tap by tap:

$$h[n] \;=\; h_{\text{ideal}}[n] \cdot w[n], \qquad w[n] = 0.54 - 0.46\cos\!\left(\frac{2\pi n}{M}\right).$$

That is the filter. Thirty-five symmetric coefficients, exactly linear phase (constant delay of 17 samples = 17 ms here), guaranteed stable. In code:

```python
import numpy as np

fs   = 1000.0
fc   = 150.0                  # cutoff at middle of transition band
M    = 34                     # filter order -> M+1 = 35 taps
wc   = 2*np.pi*fc/fs          # = 0.3*pi rad/sample
n    = np.arange(M+1)
a    = M/2                    # center / group delay (samples)

# ideal low-pass sinc, centered and shifted to be causal
x = wc*(n - a)
h_ideal = np.where(x == 0, wc/np.pi, np.sin(x)/(np.pi*(n - a)))

# Hamming window, then taper the ideal taps
w = 0.54 - 0.46*np.cos(2*np.pi*n/M)
h = h_ideal * w
h /= h.sum()                  # normalize DC gain to exactly 1

# frequency response, to check against the spec
W   = np.linspace(0, np.pi, 512)
H   = np.array([np.sum(h*np.exp(-1j*om*n)) for om in W])
mag_dB = 20*np.log10(np.abs(H) + 1e-12)
```

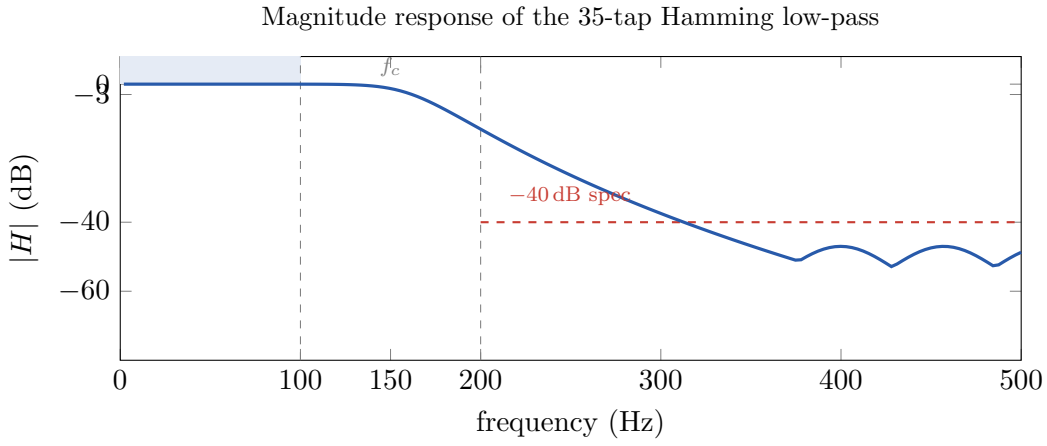


Figure 16.6: The designed filter (modeled curve). It is flat through the 100 Hz passband, falls through the transition, and sits well below the −40 dB spec line (dashed red) by 200 Hz, with a stopband floor near −53 dB set by the Hamming window. Spec met.

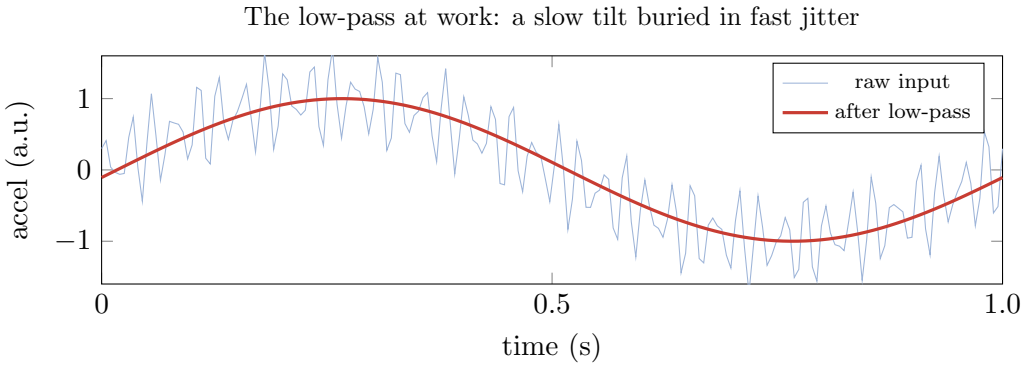


Figure 16.7: What the 35-tap low-pass does to a signal shaped like the accelerometer problem: a slow 1 Hz tilt (the part we want) drowning in 40–55 Hz jitter. The filter passes the slow component nearly untouched and erases the fast jitter, delaying everything by the constant 17-sample group delay. This is gravity separated from vibration.

Walking question

Suppose the verification plot showed the stopband only reaching −35 dB — one nudge short of spec. What is the cheapest fix? Not more taps (Hamming's floor is fixed by the *window*, not the length; more taps only narrows the transition). The right move is to switch to a window with deeper side lobes — Blackman (−58 dB floor) — and then, since Blackman's transition is wider, bump the length back up to keep the 100–200 Hz transition. This is the window method's whole logic in one decision: *window sets the floor, length sets the transition.*

Common trap

A subtle bookkeeping bug: after windowing, the DC gain $\sum_k h[k]$ is usually close to 1 but not exactly 1, so a constant input comes out slightly scaled. The one-line fix is to normalize, `h /= h.sum()` (for a low-pass). Forget it and your gravity estimate on the accelerometer drifts by a percent or two — small, but it accumulates in an orientation estimate.

16.6 IIR design: borrow from the analog masters

FIR is built directly in the digital world. IIR design takes a different, historically older route: *steal a great analog filter and convert it to digital.* Analog engineers spent a century perfecting smooth, efficient frequency-response shapes out of resistors, capacitors, and inductors. Those shapes are described by their *poles and zeros* in the s-plane (the Laplace domain of the previous chapter), and we can transplant them into the digital z-plane.

16.6.1 The Butterworth prototype: maximally flat

The most beloved analog prototype is the **Butterworth** low-pass. Its defining property is geometric and easy to picture: it is *maximally flat* in the passband. Its magnitude-squared response is

$$|H(\mathrm{j}\Omega)|^2 \;=\; \frac{1}{1+(\Omega/\Omega_c)^{2N}},$$

where N is the filter order and Ω_c the cutoff. Read that formula geometrically. At $\Omega = 0$ the response is exactly 1, and the first $2N-1$ derivatives are zero — the curve leaves DC as flat as a table. At $\Omega = \Omega_c$ it is always $1/\sqrt{2}$ ($-3\,\mathrm{dB}$), regardless of N. Far past cutoff it rolls off at $20N\,\mathrm{dB}$ per decade — doubling N doubles the steepness.

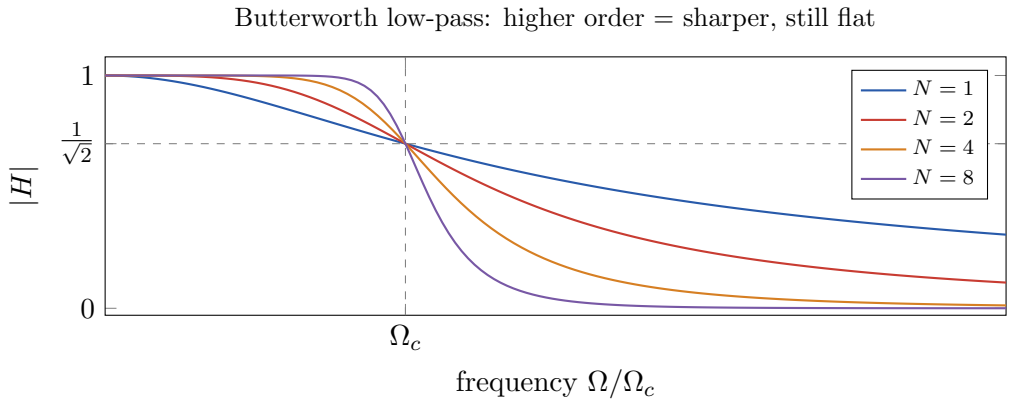


Figure 16.8: Butterworth magnitude for several orders. Every curve passes through $-3\,\mathrm{dB}$ at Ω_c and is monotonic (no ripple). Higher order squares off the corner. This single shape, with order chosen to meet a spec, covers a huge fraction of real IIR designs.

Intuition

The poles of an order-N Butterworth sit on a semicircle in the left half of the s-plane, perfectly evenly spaced (Figure 16.9). That geometric tidiness — points equally spread on a circle, exactly like the roots of unity from the DFT chapter — is *why* the passband is so flat: the contributions of the poles balance to cancel every low-order wiggle. Other prototypes move those poles onto an ellipse (Chebyshev: steeper, but ripple appears) or add zeros (elliptic: steepest of all, ripple in both bands). Butterworth is the "no ripple anywhere" choice.

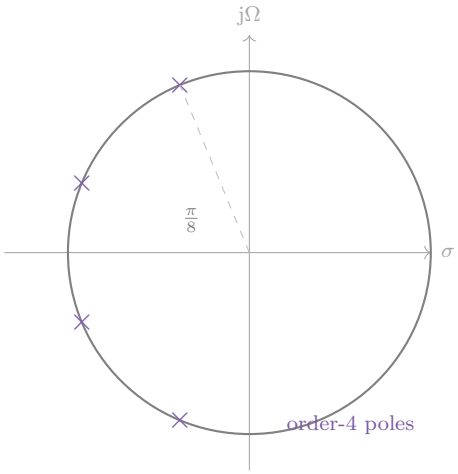


Figure 16.9: The four poles of an order-4 Butterworth low-pass in the analog s-plane. They sit on a circle of radius Ω_c, evenly spaced and confined to the stable left half-plane. Equal spacing on the circle (the same geometry as roots of unity) is what makes the passband maximally flat. Higher orders add more poles around the same semicircle.

History

Stephen Butterworth, a British engineer, described this filter in a 1930 paper titled simply "On the theory of filter amplifiers." He wanted an amplifier whose gain was as flat as possible across the passband for radio work, and he showed the all-poles-on-a-circle design achieved it. He reportedly noted that the mathematics, while elegant, required solving high-degree equations "which can only be done by hard work" — a reminder that before computers, even choosing a filter order was labor. Today `scipy.signal.butter(N, Wc)` returns the coefficients in microseconds.

16.6.2 The bilinear transform: warping analog into digital

We have a gorgeous analog design $H_a(s)$. How do we make it digital, i.e. find an $H(z)$ with the same shape? The bridge is the **bilinear transform**:

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}}.$$

Substituting this into $H_a(s)$ turns the continuous transfer function into a discrete one with the same general response. Why this particular substitution? Recall from the last chapter that the analog s-plane's imaginary axis (real frequencies) must map onto the digital z-plane's unit circle (digital frequencies), and the stable left-half s-plane must map *inside* the unit circle (so stable analog filters become stable digital filters). The bilinear transform is the simplest algebraic map that does exactly this.

What is really happening

The bilinear transform takes the *entire* infinite analog frequency axis (Ω from 0 to ∞) and squashes it onto the *finite* digital range (ω from 0 to π). Infinite into finite means the mapping

must compress as you go up: it is the warp

$$\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right).$$

Near $\omega = 0$ this is nearly linear ($\Omega \approx \omega/T$) — digital and analog frequencies agree. But as $\omega \to \pi$, the tan blows up, so high digital frequencies get pulled from ever-higher analog frequencies. This *frequency warping* is the one quirk of the method: a Butterworth corner you placed at a particular analog frequency lands at a slightly different digital frequency than you would naively expect.

Why it works

The cure for warping is to "pre-warp": before designing the analog prototype, deliberately distort the target cutoff by the *inverse* of the warp, so that after the bilinear transform squashes it, it lands exactly where you wanted. If you want a digital cutoff at ω_c, design the analog prototype for $\Omega_c = \frac{2}{T}\tan(\omega_c/2)$. The warp and the pre-warp cancel at the cutoff, and your -3 dB point ends up precisely on target. Every library's `butter` does this for you automatically; knowing it exists explains why the band edges still come out right despite the nonlinear map.

16.6.3 Worked design: a Butterworth low-pass in three lines

Same spec target as before — a low-pass on a 1000 Hz stream with cutoff near 150 Hz — but now IIR.

```
from scipy.signal import butter, sosfilt
import numpy as np

fs = 1000.0
fc = 150.0                      # -3 dB cutoff (Hz)
N  = 4                          # order: ~24 dB/octave rolloff

# design: 'sos' = cascade of second-order sections (numerically robust)
sos = butter(N, fc/(fs/2), btype='low', output='sos')

# apply to a signal x
# y = sosfilt(sos, x)           # causal IIR filtering
```

Four coefficients-worth of work (two biquad sections) gives a rolloff that an FIR would need dozens of taps to match. That is the IIR bargain: sharpness for almost nothing. The bills are the two we already named — nonlinear phase, and a stability you must respect.

Build it (hardware)

Run IIR as biquads, never as one big polynomial. A high-order IIR written as a single long difference equation is a numerical minefield on a fixed-point/16-bit micro: the coefficients span a huge dynamic range and tiny rounding errors can push a pole outside the unit circle, turning your stable filter into an oscillator. The fix is universal: factor the filter into a *cascade*

of second-order sections ("biquads"), each a tiny 2-pole/2-zero stage. SciPy's `output='sos'` gives you exactly this. On an Arduino-class chip a biquad is five multiplies and four adds per sample — trivial — and the cascade stays stable. This is the standard way audio codecs, EQs, and our wrist controller's EMG band-pass are actually run.

Common trap

Stability is not optional with IIR. An FIR is unconditionally stable; an IIR is stable only if every pole sits strictly inside the unit circle. Quantizing coefficients, or designing too high an order with too sharp a cutoff, can nudge a pole onto or outside the circle, and the filter *self-oscillates* or blows up — there is no input that fixes it. Always verify the designed poles' radii (< 1), keep the order modest, and run as biquads. When in doubt, an FIR cannot do this to you.

16.7 FIR versus IIR, decided by the spectrum plot

Put the two designs side by side and the choice becomes concrete. For the *same* -3 dB cutoff, the order-4 Butterworth and the 35-tap Hamming FIR have comparable magnitude rolloff — but their phase behavior is night and day.

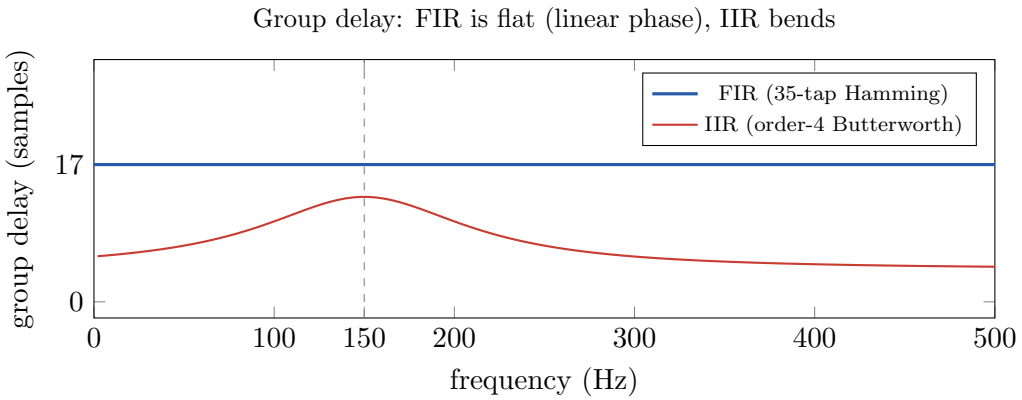


Figure 16.10: The deciding picture. The FIR delays every frequency by exactly 17 samples (flat blue line) — waveform shape preserved. The IIR's delay (red) peaks near the cutoff: frequencies there lag the others, smearing shape. Same magnitude job, very different effect on timing.

Why this matters

This is why "which filter?" is never a one-size answer. If you only care *how loud* each frequency ends up — audio tone shaping, an envelope, a power estimate — IIR's cheap sharpness wins and nobody hears the phase. If you care *when* things happen — aligning sensor channels, preserving the shape of a transient, measuring a delay — FIR's flat group delay is worth every extra tap. The spectrum plot tells you the magnitude story; the group-delay plot tells you the timing story; a good engineer reads both before committing.

Application

The mains-hum notch. A microphone or a biopotential amplifier picks up a sharp 50 or 60 Hz tone from the power grid. The ideal cure is a band-stop so narrow it removes only the hum and leaves the music or muscle signal intact. This is the textbook IIR job: a single biquad with a complex-conjugate pair of zeros right on the unit circle at the hum frequency (forcing the gain to exactly zero there) and a matching pole pair just inside the circle (so the notch is razor thin), as in Figure 16.11. Two coefficient-pairs, a notch you could not begin to match with a short FIR. The pole radius sets the notch width; push it toward 1 for a narrower notch — but not *onto* the circle, or you lose stability.

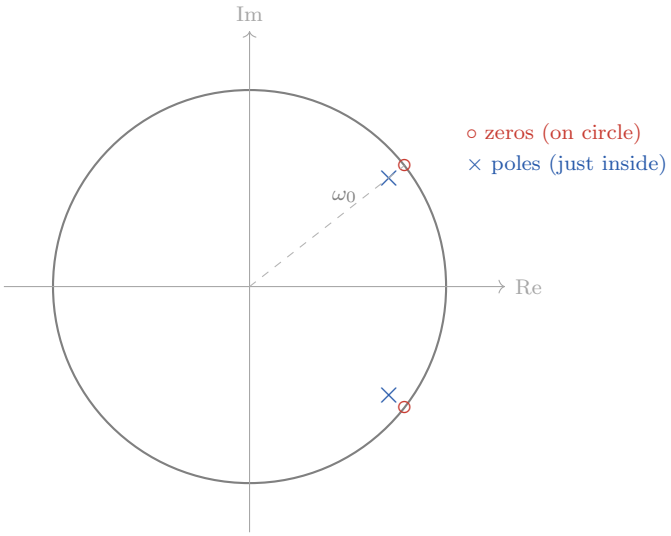


Figure 16.11: Pole-zero plot of a mains-hum notch biquad. A conjugate pair of zeros sits exactly *on* the unit circle at the hum angle $\omega_0 = 2\pi f_0/f_s$, forcing the gain to zero there. A conjugate pole pair sits at the same angle but just inside the circle (radius near 0.9); the closer the poles creep toward the circle, the narrower the notch — but a pole *on* the circle would make the filter unstable.

Toward the wrist tracker

The wrist controller's filter stack, decided.

- *Accelerometer, separate gravity from motion*: a low-pass with cutoff around 1–3 Hz isolates the slow gravity/tilt component; subtract it to get the fast motion (Figure 16.7 shows this separation in action). Channels must stay time-aligned across x, y, z so a flick spike looks simultaneous — use a short **linear-phase FIR** so all three axes share one constant delay.
- *EMG/FMG muscle band*: band-pass roughly 20–450 Hz to grab muscle activity, then rectify and low-pass to get an envelope. Phase does not matter for an envelope, so use cheap **IIR biquads**.
- *Mains hum*: a single **IIR notch** biquad at 50 or 60 Hz.

And the limit, stated plainly: this stack cleans up and bands the signals, but no amount of filtering lets a *wrist-mounted IMU alone* recover individual finger joint angles. Filtering

separates gravity from motion and isolates frequency bands; it cannot manufacture information the sensor never captured. Coarse, discrete wrist and forearm gestures — yes. Continuous per-finger pose needs sensors on the fingers (rings) or richer forearm sensing (multi-channel EMG/FMG), as we discuss in the hand-sensing chapter. A filter is a frequency-shaped scalpel, not a source of new degrees of freedom.

16.8 Design checklist

The whole chapter compresses to a procedure you can run every time.

1. **Write the spec**: band type; passband and stopband edges; allowed ripple δ_p; required attenuation δ_s. Draw the tolerance corridor.
2. **Decide phase**: does waveform *shape*/timing matter? Yes → lean FIR (linear phase). No, only magnitude → IIR is fine and cheaper.
3. **If FIR**: pick a window from your stopband attenuation (window sets the floor), then a length from your transition width (length sets the steepness); build the windowed sinc; normalize the DC gain.
4. **If IIR**: pick a prototype (Butterworth for no ripple), choose the order from your rolloff/attenuation need, design via the bilinear transform (the library pre-warps for you), and emit second-order sections.
5. **Verify**: plot the magnitude against the corridor *and* the group delay. For IIR, check every pole radius < 1.
6. **Implement**: FIR as a tapped sum; IIR as cascaded biquads. Confirm on real data, on the real hardware, at the real sample rate.

Pause

If only one sentence survives: *a filter is a frequency mask you design, FIR shapes that mask with a windowed sinc and gives you exact linear phase at the cost of taps, IIR borrows an analog prototype through the bilinear transform and gives you cheap sharpness at the cost of phase and a stability you must watch.* Everything else is detail in service of hitting a tolerance corridor.

16.9 Exercises

Exercise 16.1. A signal is sampled at $f_s = 8000\,\text{Hz}$. You want a low-pass that passes audio up to $3000\,\text{Hz}$ and strongly rejects everything above $3800\,\text{Hz}$. Give the passband edge, the stopband edge, and the transition width, all in (a) Hz, (b) cycles per sample, and (c) radians per sample. Why is $\omega = \pi$ the highest meaningful frequency here, and what Hz does it correspond to?

Exercise 16.2. The ideal low-pass impulse response is $h[n] = \sin(\omega_c n)/(\pi n)$. (a) Use l'Hopital or the small-angle limit to show $h[0] = \omega_c/\pi$. (b) Explain in words why $h[n]$ being nonzero for all

n makes the ideal filter both non-causal and infinitely long. (c) What single operation makes a truncated version causal, and how many samples of delay does it cost?

Exercise 16.3. You truncate the ideal sinc to 21 taps with a rectangular window and find the first stopband ripple reaches -21 dB. A colleague says "just use 201 taps, the ripple will vanish." Explain why they are wrong (name the phenomenon), what 201 taps *does* improve, and what actually lowers that first ripple.

Exercise 16.4. Show that a symmetric FIR, $h[k] = h[M-k]$, has linear phase. (Hint: group the k and $M-k$ terms in $H(\omega) = \sum_k h[k]e^{-\mathrm{j}\omega k}$, factor out $e^{-\mathrm{j}\omega M/2}$, and show what remains is purely real.) State the constant group delay in samples.

Exercise 16.5. Design, on paper, a 15-tap Hamming-windowed low-pass with normalized cutoff $\omega_c = 0.25\pi$. Write the formula for each tap $h[n]$ ($n = 0, \ldots, 14$), identify the center tap and its value, and state the filter's group delay. You do not need to compute all the numbers — show the setup and compute $h[7]$ (the center) and $h[6]$ exactly.

Exercise 16.6. For a Butterworth low-pass, $|H(\mathrm{j}\Omega)|^2 = 1/(1 + (\Omega/\Omega_c)^{2N})$. (a) Verify the gain at $\Omega = \Omega_c$ is $1/\sqrt{2}$ for every order N. (b) Show the asymptotic rolloff far above cutoff is $20N$ dB per decade of frequency. (c) What order N is needed to reach -60 dB attenuation one decade above the cutoff?

Exercise 16.7. The bilinear transform warps frequencies by $\Omega = \frac{2}{T}\tan(\omega/2)$. For $T = 1$, you want a digital cutoff at $\omega_c = 0.4\pi$. (a) What analog cutoff Ω_c should you pre-warp to? (b) By what percentage does it differ from the naive choice $\Omega_c = \omega_c$? (c) Explain why ignoring pre-warping matters more for high cutoffs than low ones.

Exercise 16.8. You implement an order-8 Butterworth as a single 9-coefficient difference equation in 16-bit fixed point and the output slowly grows without bound even for a tiny input. (a) What has most likely happened to at least one pole? (b) Why does a cascade of four biquads avoid this? (c) Could this failure ever happen to an FIR filter? Justify.

Exercise 16.9. For the wrist controller, you must low-pass the three accelerometer axes to extract gravity. A teammate proposes an order-2 IIR Butterworth (cheap) on each axis. (a) What artifact will this introduce that could misalign the three axes during a fast flick? (b) Propose an FIR alternative and state what you trade to get the fix. (c) For the separate EMG envelope path, argue why the same teammate's IIR choice is actually the right call there.

Exercise 16.10. Design a 60 Hz notch (band-stop) as a single biquad for $f_s = 1000$ Hz. (a) At what angle on the unit circle do you place the zeros, and why exactly on the circle? (b) Where do you place the poles to make the notch narrow, and what constrains how close to the circle they may go? (c) Sketch the resulting magnitude response and mark the notch frequency and the -3 dB notch width.

Exercise 16.11. (Open-ended build.) On an Arduino or Raspberry Pi, implement the 35-tap FIR low-pass from Section 16.5.3 as a circular buffer of the last 35 samples. (a) Estimate the multiply-accumulate count per output sample. (b) At $f_s = 1000$ Hz, what fraction of the CPU does that consume on your board? (c) Now implement the order-4 IIR as two biquads and compare the per-sample cost. Which would you ship, and on which channel?

Chapter 17

Practical Filtering

In the last chapter we designed filters: we chose a cutoff, picked a window or a Butterworth shape, and ended up holding a set of coefficients. That is the easy part. A filter on paper is an idealization — an infinitely precise impulse response, applied to an infinitely long signal, with infinite numerical accuracy, in zero time. The moment you want to run it on a stream of samples arriving in real time from an accelerometer strapped to a moving wrist, every one of those idealizations breaks, and a new set of questions appears that the textbook filter-design chapter never asked:

- The signal never ends. How do I filter *now*, with only the samples I have seen so far, before the next one arrives?
- Samples come one at a time, but the FFT wants a whole block. Do I filter sample-by-sample, or buffer up a block and transform it? What is the cost of each?
- A long FIR filter convolved the slow way costs N multiplies per output sample. Can the FFT make that cheaper, even on a live stream?
- My multiplies are not exact — they round. On a microcontroller they may not even be floating point. When does that rounding bite?
- Every filter delays the signal. For a musical instrument you are *playing*, that delay is felt in the fingertips. How much can I afford, and where does it come from?
- Real sensor data has spikes, dropouts, and drift that no linear filter handles gracefully. What actually cleans it up?

This chapter is about turning a designed filter into a thing that runs. It is the most concrete chapter in the book, and one of the most important for the wrist controller, because a gesture instrument lives or dies on *latency* and on *robust de-noising*. We will build everything from the same geometric pictures we have been using — convolution as a sliding overlap, the spectrum as a set of frequency knobs — and then get our hands dirty with buffers, ring indices, fixed-point scaling, and the humble moving average that does ninety percent of the real work in a sensor pipeline.

17.1 Streaming: the signal that never ends

When we wrote convolution in Chapter 13 we treated the whole signal as a vector sitting in memory. That is *offline* or *batch* processing: you have a recording, you process it, you save the result. Plenty of DSP is like that — cleaning up a finished audio file, analyzing a stored IMU log. But a live instrument is the opposite. Samples arrive forever, one (or a few) at a time, and you must emit output continuously, keeping up with the input rate and never falling behind.

Definition 17.1 (Real-time / streaming processing)**.** A processing system is *real-time* if it produces each output within a fixed, bounded time of receiving the input that determines it, and *keeps up* indefinitely: the average time to process one sample must be less than the sample period $T_s = 1/f_s$. A *streaming* algorithm is one that can run on an unbounded input using bounded memory, holding only a finite window of past samples (its *state*).

The two words are not the same. "Streaming" is about memory: you cannot store the whole signal, so you keep only what you need. "Real-time" is about a deadline: you must finish before the next sample, on average, with a worst case you can bound. A filter can be streaming but too slow for real time (it keeps up on a fast PC but not on a tiny microcontroller), and offline processing can be wonderfully fast without being real-time at all.

Intuition

Picture the signal as a paper tape being pulled past a reading head at f_s characters per second. Offline processing is reading the whole tape spread out on a table. Streaming is reading it through a slot: you only ever see a short window, and once a character scrolls past the slot it is gone unless you copied it. The filter's memory is exactly the part of the tape currently inside the slot — and a good filter design keeps that slot short.

17.1.1 What a causal filter is allowed to know

A filter that runs on a stream can only use samples that have already arrived. That is the *causality* constraint from Chapter 12, and now it is not a mathematical nicety — it is physics. At sample time n you may use $x[n], x[n-1], x[n-2], \ldots$ and past outputs $y[n-1], y[n-2], \ldots$, but you cannot use $x[n+1]$, because it does not exist yet.

This immediately rules out the symmetric, zero-delay filters we like to draw. The ideal low-pass filter has a sinc impulse response that stretches infinitely in *both* directions; half of it is in the future. A symmetric linear-phase FIR of length N is centered at tap $(N-1)/2$, so to compute the output "at the center" you need samples up to $(N-1)/2$ ahead. In a stream you cannot peek ahead, so you simply wait: you emit the centered result $(N-1)/2$ samples *late*. That delay is the price of causality, and we will track it obsessively in Section 17.6.

Focus

Three knobs control everything in this chapter, and they trade off against each other:

- **Quality** — how sharp the cutoff, how deep the stopband.
- **Latency** — how long after a sample arrives you can act on it.
- **Cost** — multiplies, adds, and memory per sample.

You cannot maximize all three. A sharp linear-phase FIR is high quality but high latency; a short IIR is low latency and cheap but has phase distortion. The art of practical filtering is picking the right corner of this triangle for the job.

17.2 Sample-by-sample versus block processing

There are two fundamental ways to structure a streaming filter, and the choice colors the whole implementation.

Sample-by-sample. You process exactly one input sample at a time, producing one output sample, updating a little bundle of internal state in between. This is how you naturally implement a difference equation, an IIR filter, a one-pole smoother, a PLL. The code reads like the math:

```
// Direct-Form I FIR, one sample in -> one sample out.
// h[0..N-1] are the taps; z[] is a ring buffer of the last N inputs.
float fir_step(float x, const float *h, float *z, int *pos, int N) {
    z[*pos] = x;                // store newest input
    float acc = 0.0f;
    int k = *pos;
    for (int i = 0; i < N; i++) {
        acc += h[i] * z[k];     // newest tap times newest sample, etc.
        k = (k == 0) ? N - 1 : k - 1;   // walk backward in time
    }
    *pos = (*pos + 1) % N;      // advance ring head
    return acc;
}
```

The virtues are minimal latency (you can emit output the instant a sample arrives) and simple, predictable timing. The cost is that long FIR filters do N multiply-accumulates (MACs) per sample, every sample, forever.

Block (frame) processing. You collect L samples into a buffer, then process the whole block at once. This is mandatory for anything built on the FFT — spectrograms (Chapter 10), fast convolution (below), spectral subtraction — because the FFT operates on a block. It is also how audio hardware actually delivers data: a sound card or an I2S peripheral hands you a buffer of (say) 64 or 128 samples per interrupt, not one sample per interrupt, because per-sample interrupts would drown the CPU.

What is really happening

Block processing buys efficiency by amortizing fixed costs — the FFT's $O(L \log L)$, function-call overhead, cache warm-up — across many samples. But it spends *latency* to do it: you cannot start processing a block until its last sample has arrived, so the first sample of a block waits one full block time, $L\,T_s$, before anything happens. Bigger blocks are more efficient and more laggy. This single tradeoff is why a guitar amp plugin set to a 512-sample buffer at 48 kHz feels rubbery (about 11 ms of buffering latency before any processing) while 32

samples feels immediate but burns CPU.

	Sample-by-sample	**Block**
Latency	minimal (filter delay only)	block delay + filter delay
FFT-based work	not possible	natural
Per-sample cost	fixed, every sample	amortized, bursty
Code shape	difference equation	buffer then transform
Typical use	IIR, smoothers, control	fast convolution, spectral

Table 17.1: The two streaming structures. Most real systems mix them: a sample-by-sample pre-filter feeding a block-based analysis stage.

Toward the wrist tracker

For the wrist controller, the IMU runs slowly — typically f_s between 100 and 1000 Hz — so per-sample MACs are cheap and we will favor *sample-by-sample* processing on the motion path to keep latency near zero. The EMG/FMG path, if present, samples faster (1–2 kHz) and may want short blocks for feature extraction. The lesson to carry forward: keep the *control* path (the part that decides "a note happened now") sample-by-sample and short; push any heavy spectral analysis onto a block-based path that is allowed to lag, and use it only for slow decisions like "which gesture mode are we in."

17.3 Fast convolution: filtering with the FFT

Suppose your filter is genuinely long — an FIR with $N = 1024$ taps, say a finely-shaped equalizer or a reverberation impulse response. Direct convolution costs N MACs per output sample. Can we do better? Yes, and the lever is the convolution theorem from Chapter 11: convolution in time is multiplication in frequency.

$$y[n] = (x * h)[n] \quad \Longleftrightarrow \quad Y(e^{\mathrm{j}\omega}) = X(e^{\mathrm{j}\omega})\, H(e^{\mathrm{j}\omega}). \tag{17.1}$$

So we could FFT the signal, multiply by the FFT of h, and inverse-FFT. The cost of an FFT of length M is $O(M \log M)$, so multiplying two spectra and transforming back is far cheaper than N multiplies per sample when N is large. There is just one catch, and it is a sharp one.

17.3.1 The wrap-around problem

The DFT does not implement ordinary (linear) convolution. It implements *circular* convolution: it treats both sequences as if they were periodic, wrapped around a circle of length M. When you multiply two length-M DFTs and invert, the output is

$$y_{\text{circ}}[n] = \sum_{k=0}^{M-1} x[k]\, h[\,(n-k) \bmod M\,]. \tag{17.2}$$

The mod$M$ is the villain. The tail of h that should have spilled past the end of the block instead wraps around and corrupts the *beginning*, a defect called *time-domain aliasing*. The fix is classic: give the convolution enough room so the tail has somewhere to go that is not the start.

Why it works

Linear convolution of a length-L signal block with a length-N filter produces an output of length $L + N - 1$. If we run the DFT at a length $M \geq L + N - 1$ — by zero-padding both inputs out to M — then the wrapped copies of the result land entirely in the zero-padding region and never overlap the genuine output. Circular convolution at length $M \geq L + N - 1$ *equals* linear convolution. That single inequality is the whole secret of fast convolution.

17.3.2 Overlap-add

A stream is too long to transform in one shot, so we chop it into consecutive blocks of length L and filter each block separately. Each block, convolved with the length-N filter, grows to length $L+N-1$ — so consecutive outputs *overlap* by $N - 1$ samples. Overlap-add says: just add the overlapping tails back together.

$$x[n] = \sum_r x_r[n], \qquad y[n] = \sum_r (x_r * h)[n], \tag{17.3}$$

where x_r is the r-th block of L input samples (zeros elsewhere). Because convolution is linear, the convolution of the sum is the sum of the convolutions, so adding the overlapping output pieces reconstructs the exact linear convolution of the whole stream.

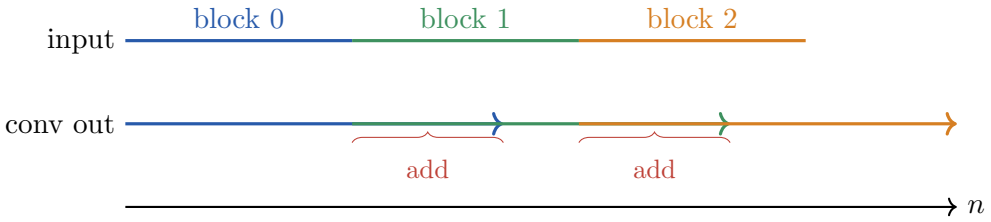


Figure 17.1: Overlap-add with block length $L = 3$ and filter length $N = 3$ (so each convolved block has length $L + N - 1 = 5$). Consecutive output pieces overlap by $N - 1 = 2$ samples; summing the overlaps reconstructs the exact linear convolution.

The recipe, per block:

1. Take the next L input samples; zero-pad to length $M \geq L + N - 1$ (use a power of two for the FFT).
2. Multiply its FFT by $H = \mathcal{F}\{h\}$ (computed *once*, ahead of time).
3. Inverse-FFT to get $L + N - 1$ output samples.
4. Add the first $N - 1$ of them to the saved tail from the previous block; emit; save this block's last $N - 1$ samples as the new tail.

```python
import numpy as np

def overlap_add(x, h, L):
    N = len(h)
    M = 1 << int(np.ceil(np.log2(L + N - 1)))   # FFT length, power of 2
    H = np.fft.rfft(h, M)                         # precompute once
    y = np.zeros(len(x) + N - 1)
    for i in range(0, len(x), L):
        blk = x[i:i+L]
        Y = np.fft.irfft(np.fft.rfft(blk, M) * H, M)  # = linear conv of blk*h
        y[i:i+len(Y)] += Y                            # overlap and ADD
    return y[:len(x) + N - 1]
```

17.3.3 Overlap-save

Overlap-save attacks the same problem from the other side. Instead of zero-padding the input and adding tails, it deliberately *allows* the circular wrap-around to corrupt part of each block, then simply throws the corrupted part away.

You feed the FFT blocks of length M that *overlap the input* by $N-1$ samples (each block reuses the last $N-1$ samples of the previous one). After circular convolution at length M, the first $N-1$ output samples are garbage (wrapped) and the remaining $M-N+1$ are correct linear-convolution outputs. Save the good ones, discard the rest. No addition of tails is needed — hence the older name "overlap-discard."

	Overlap-add	**Overlap-save**
Input blocks	contiguous, no overlap	overlap by $N-1$
Per block	zero-pad input	no zero-pad of input
Combine step	add overlapping output tails	discard first $N-1$ outputs
Bookkeeping	save and add a tail buffer	save last $N-1$ input samples
Arithmetic	identical FFT count	identical FFT count

Table 17.2: Overlap-add versus overlap-save. They compute the same answer with the same number of FFTs; they differ only in how they stitch blocks together. Pick whichever bookkeeping you find less error-prone.

Common trap

The single most common fast-convolution bug is choosing M too small. If $M < L+N-1$ the wrap-around eats into real output and you get a quiet, hard-to-spot distortion — often a faint echo or a click at block boundaries. Always assert $M \geq L+N-1$ in code. The second most common bug is forgetting that $H = \mathcal{F}\{h\}$ must be computed at the *same* FFT length M as the blocks; reuse a precomputed H, but recompute it if you change M.

Why it works

When is fast convolution actually worth it? Direct FIR costs about N real MACs per output sample. Overlap-add costs roughly two length-M FFTs and M complex multiplies per block of $L \approx M$ outputs, i.e. on the order of $c \log_2 M$ operations per output sample for a small constant c. The crossover is around $N \approx 60$–100 taps on a typical CPU: below that, direct convolution wins (and has lower latency and zero block delay); above it, FFT convolution pulls steadily ahead, and for N in the thousands it is dramatically faster. For the wrist tracker's short smoothing filters (N of a handful to a few dozen), *direct* convolution is both faster and lower latency — save fast convolution for long audio impulse responses.

17.4 Implementing the filter: direct forms

Set the FFT aside and look at how a filter's difference equation actually becomes a data-flow of delays, multiplies, and adds. The general LTI difference equation from Chapter 15 is

$$y[n] = \sum_{i=0}^{N} b_i \, x[n-i] \; - \sum_{k=1}^{M} a_k \, y[n-k], \tag{17.4}$$

with feedforward (numerator) coefficients b_i and feedback (denominator) coefficients a_k. An FIR filter is the special case with no feedback ($a_k = 0$); an IIR filter has feedback. There are several ways to wire this up, all algebraically equal but *numerically different*.

17.4.1 Direct Form I and II

Direct Form I implements Equation (17.4) literally: one chain of delays for the past inputs, another for the past outputs, multiply by the b's and a's, sum. It needs $N + M$ delay registers.

Direct Form II notices that the two delay chains carry redundant information and merges them, halving the delay storage to $\max(N, M)$. It computes an intermediate "state" signal $w[n]$:

$$w[n] = x[n] - \sum_{k=1}^{M} a_k \, w[n-k], \tag{17.5}$$

$$y[n] = \sum_{i=0}^{N} b_i \, w[n-i]. \tag{17.6}$$

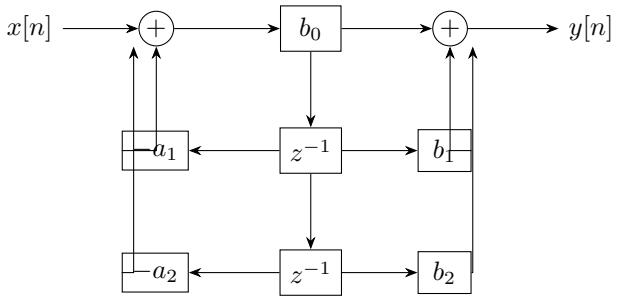


Figure 17.2: Direct Form II of a second-order section. A single delay line (z^{-1} blocks) holds the shared state $w[n]$; the feedback taps $-a_k$ enter the left adder and the feedforward taps b_i enter the right adder. Half the delays of Direct Form I.

Common trap

Direct Form II saves memory but is *worse* numerically for high-order IIR filters: the shared state $w[n]$ can have a much larger dynamic range than the input or output, so it overflows (in fixed point) or loses precision (in floating point) more readily. The practical rule, below, sidesteps this entirely.

17.4.2 Cascade of biquads: the practical default

Never implement a high-order IIR filter as one big Direct Form. The reason is the same one that makes a high-degree polynomial wretched to evaluate from its coefficients: the roots (poles and zeros) become wildly sensitive to tiny coefficient errors, and a filter that was stable on paper can ring or blow up once its coefficients are rounded to 16 or 24 bits.

The cure is to factor the transfer function into a *cascade* of second-order sections ("biquads"):

$$H(z) = g \prod_{s=1}^{S} \frac{b_{0,s} + b_{1,s}\, z^{-1} + b_{2,s}\, z^{-2}}{1 + a_{1,s}\, z^{-1} + a_{2,s}\, z^{-2}}. \tag{17.7}$$

Each second-order section holds just one pole pair and one zero pair, so its coefficients localize the action and stay well-conditioned. You run the sections one after another, the output of each feeding the next.

Why it works

Why second order and not first? Real filter poles and zeros come in *complex-conjugate pairs* (because $h[n]$ is real). A complex pole at $re^{\mathrm{j}\theta}$ and its conjugate $re^{-\mathrm{j}\theta}$ combine into one real second-order section with $a_1 = -2r\cos\theta$, $a_2 = r^2$. Keeping each conjugate pair together in its own section is what keeps the coefficients real and the arithmetic small. This is also why "biquad" (bi-quadratic: quadratic over quadratic) is the universal currency of audio EQ, sensor filters, and DSP chips — every parametric EQ band you have ever turned is one biquad.

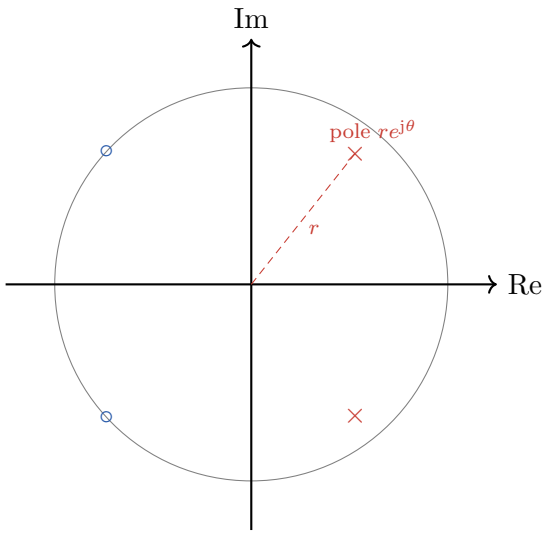


Figure 17.3: Pole-zero plot of one biquad. The complex-conjugate pole pair (red $\times$) at $re^{\pm j\theta}$ gives the real denominator $1 + a_1 z^{-1} + a_2 z^{-2}$ with $a_1 = -2r\cos\theta$, $a_2 = r^2$; the conjugate zero pair (blue $\circ$) gives the numerator. Poles inside the unit circle keep the section stable; the closer they sit to the circle, the sharper and more fixed-point-sensitive the resonance.

```c
// One biquad, Transposed Direct Form II (good numerics, 2 state vars).
typedef struct { float b0,b1,b2,a1,a2; float z1,z2; } biquad;

float biquad_step(biquad *q, float x) {
    float y = q->b0 * x + q->z1;
    q->z1   = q->b1 * x - q->a1 * y + q->z2;
    q->z2   = q->b2 * x - q->a2 * y;
    return y;
}
// A high-order filter is just an array of these run in sequence.
float cascade_step(biquad *qs, int S, float x) {
    for (int s = 0; s < S; s++) x = biquad_step(&qs[s], x);
    return x;
}
```

The *transposed* Direct Form II shown here is the workhorse: it has the same low storage as Direct Form II but much better numerical behavior, because its two state variables stay closer to the signal's natural range. It is what you will find inside almost every audio library's biquad.

17.5 Fixed-point and numerical issues

On a desktop you filter in 32- or 64-bit floating point and rarely think about arithmetic error. On the small microcontroller that will live on the wrist strap, you may have no floating-point unit at all, or a slow one. Then you filter in *fixed point*: integers interpreted as fractions, with an implied binary point. Understanding what goes wrong there is the difference between a filter that works on the bench and one that hisses, clips, or drifts in the field.

17.5.1 What fixed point is

A fixed-point format $Qm.f$ stores a number as an integer scaled by 2^{-f}: the integer X represents the real value $X \cdot 2^{-f}$, with m integer bits and f fractional bits. For example $Q1.15$ uses a 16-bit signed integer to represent values in $[-1, 1)$ with a resolution of $2^{-15} \approx 3 \times 10^{-5}$. Adding two $Q1.15$ numbers is just integer addition; *multiplying* two of them gives a $Q2.30$ result that you must shift back down by 15 bits to return to $Q1.15$.

Connection

This is exactly the quantization story of Chapter 19, applied not to the input sample but to every internal coefficient and intermediate result. Each rounding injects a tiny error that, statistically, behaves like added noise (Chapter 20): uniform on $\pm\frac{1}{2}$ LSB, with variance $\Delta^2/12$ where $\Delta = 2^{-f}$ is the quantization step. A filter is a chain of such injections, and the question is whether they accumulate or wash out.

17.5.2 The three things that bite

1. Coefficient quantization. Rounding the b_i, a_k to fixed point moves the poles and zeros. For a sharp, high-order filter the poles sit close together and close to the unit circle, where small coefficient nudges cause big pole movements — exactly the ill-conditioning that the biquad cascade fixes. A direct-form 8th-order filter quantized to 16 bits can have its poles shoved outside the unit circle and *go unstable*; the same filter as four biquads stays put.

2. Roundoff (product) noise. Every multiply produces more bits than fit, so the result is rounded, adding noise. In an IIR filter this noise circulates through the feedback and can be amplified near resonant peaks. The defense is to keep enough fractional bits and to scale section gains so signals use the available range without overflowing.

3. Overflow and limit cycles. If an intermediate sum exceeds the representable range it *wraps* (in two's-complement) from the largest positive value to the largest negative — a catastrophic, full-scale glitch. Worse, the feedback in an IIR filter can sustain a small self-perpetuating oscillation called a *limit cycle*: with a constant or zero input, the rounded output settles into a tiny periodic wiggle instead of going to rest.

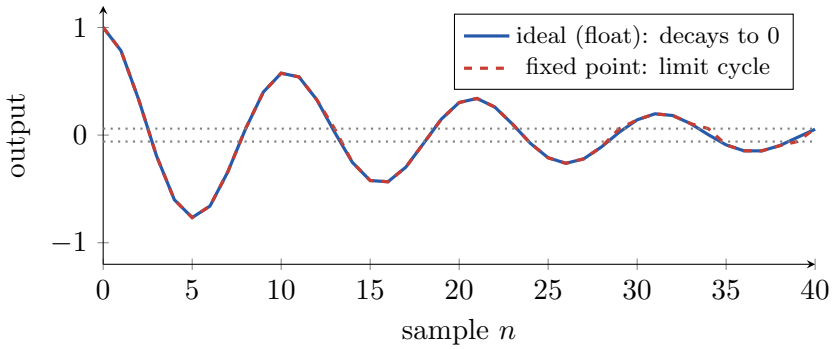


Figure 17.4: A limit cycle in an IIR filter. In floating point the resonator's ringing decays smoothly to zero (solid). In fixed point, rounding can trap the output in a small sustained oscillation (dashed) that never dies — audible as a faint idle tone, or as a sensor reading that refuses to settle.

Focus

Practical fixed-point survival kit:

- Implement IIR filters as cascaded *biquads*, never one big direct form.
- Use *transposed Direct Form II* sections.
- Accumulate products in a wider register (e.g. 32- or 64-bit accumulator for 16-bit data) and round only once, at the end.
- *Scale* each section so internal signals fill the range without overflowing; saturate (clamp) rather than wrap on overflow.
- For FIR filters, prefer fixed point freely — with no feedback they cannot have limit cycles or go unstable, only add bounded roundoff noise.

Common trap

Saturating arithmetic (clamp to the max/min on overflow) turns a catastrophic wrap-around glitch into a graceful, soft distortion. Most DSP instruction sets have saturating add/multiply for exactly this reason. If your platform wraps by default, add an explicit clamp on the accumulator before storing the output. One missing clamp is the classic cause of an occasional loud "pop" in an otherwise clean audio path.

17.6 Latency: the clock you cannot cheat

For a musical controller, latency is the single most important number in the whole system. A performer feels the gap between moving and hearing the result in their fingertips, and beyond a threshold the instrument stops feeling like part of the body. So let us account for every microsecond a filter costs.

17.6.1 Group delay: how late the filter makes the signal

A filter's frequency response $H(e^{\mathrm{j}\omega}) = |H|\, e^{\mathrm{j}\phi(\omega)}$ has a phase $\phi(\omega)$. The *group delay* is the negative slope of that phase,

$$\tau_g(\omega) = -\frac{\mathrm{d}\phi(\omega)}{\mathrm{d}\omega}, \tag{17.8}$$

measured in samples. It is, intuitively, how many samples late the envelope of a narrow band of frequencies near ω comes out of the filter. (We met this in Chapter 14; here it becomes a budget line.)

What is really happening

For a *symmetric, linear-phase FIR* of length N, the phase is a straight line and the group delay is the *same at every frequency*:

$$\tau_g = \frac{N-1}{2} \text{ samples.}$$

Every frequency is delayed equally, so the waveform shape is preserved — but it is preserved *late*, by half the filter length. A 101-tap linear-phase low-pass at $f_s = 200$ Hz delays everything by 50 samples, i.e. $50/200 = 0.25$ s — a quarter second! That is unplayable. Linear phase is gorgeous for offline analysis and ruinous for live control.

Intuition

Linear-phase FIR is symmetric, so its "center of mass" is in the middle. To emit the centered output you must wait until the middle tap lines up with the newest sample — which means waiting $(N-1)/2$ samples after the event you care about. The sharper the filter (the more taps), the longer the wait. There is no trick around it: **a sharp linear-phase filter is inherently laggy.**

17.6.2 IIR buys low latency with phase distortion

An IIR filter reaches a given sharpness with far fewer coefficients than an FIR, and its group delay can be a handful of samples even for a steep response. The price is *nonlinear phase*: different frequencies are delayed by different amounts, so transients smear in shape. For control signals (we mostly care *when* a gesture happens, not the pristine shape of the waveform) this trade is usually worth it: an IIR low-pass with τ_g of a few samples beats a linear-phase FIR with τ_g of fifty.

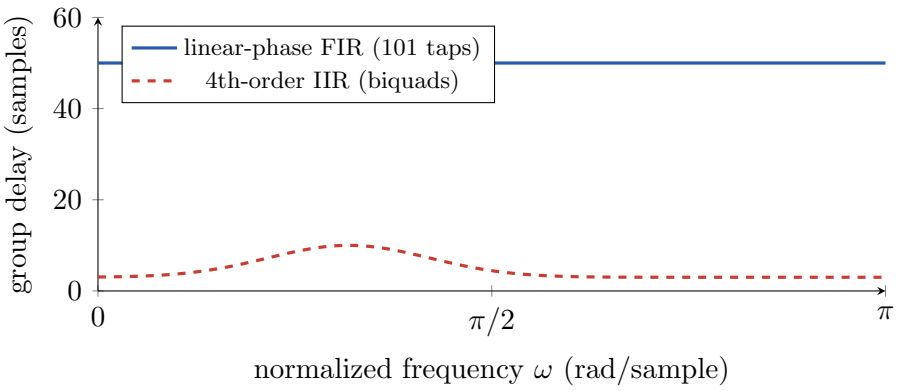


Figure 17.5: Group delay versus frequency for two filters of comparable sharpness. The FIR delays every frequency by a flat 50 samples (linear phase). The IIR delays by only a few samples but unevenly, peaking near its cutoff (nonlinear phase). For live control the IIR's low, if uneven, delay usually wins.

17.6.3 The full latency budget

End-to-end latency for the wrist controller is a sum, and the filter is only one term:

$$\tau_{\text{total}} = \underbrace{\tau_{\text{sensor}}}_{\text{ADC + internal LPF}} + \underbrace{\tau_{\text{block}}}_{L\,T_s \text{ if buffered}} + \underbrace{\tau_{\text{filter}}}_{\tau_g \cdot T_s} + \underbrace{\tau_{\text{compute}}}_{\text{DSP time}} + \underbrace{\tau_{\text{link}}}_{\text{MIDI/BLE}} + \underbrace{\tau_{\text{synth}}}_{\text{sound engine}} . \tag{17.9}$$

Toward the wrist tracker

Musicians reliably notice action-to-sound latency above roughly 10 ms, and 20–30 ms feels sluggish for percussive playing. Budget accordingly:

- **Sensor LPF:** MEMS IMUs have a built-in anti-alias/low-pass with its own group delay — read the datasheet; a low bandwidth setting can cost several ms. Choose a bandwidth that just covers your gesture frequencies (motion energy is mostly below 20 Hz).
- **Blocking:** go sample-by-sample on the control path; do not buffer the gesture detector.
- **Filter:** use short IIR smoothers (τ_g of a few samples). Avoid long linear-phase FIRs on the live path. At $f_s = 200$ Hz, each sample is 5 ms, so even a 5-sample group delay is 25 ms — so *sample faster* ($f_s \geq 500$ Hz) on the control path to shrink the per-sample time.
- **Link:** classic serial MIDI adds about 1 ms per message; Bluetooth LE MIDI can add 10–30 ms or more and is jittery — prefer USB or wired for serious playing.

The takeaway: filtering latency is one of several terms, and the wireless link often dwarfs it. Design the whole chain, not just the filter.

History

The 10 ms figure is not folklore. Studies of musicians (e.g. work by Wessel and Wright at Berkeley's CNMAT, and later latency-perception studies) consistently find that performers detect and are disturbed by action-to-sound delays in the 10 ms range, with skilled percus-

sionists more sensitive still. It is the same order as the acoustic delay of standing a couple of meters from your own amplifier — a delay real musicians already learn to hate.

17.6.4 Zero-phase filtering — and why you cannot use it live

There is a beloved offline trick: filter the signal forward, then filter the result *backward* with the same filter (`filtfilt` in SciPy/MATLAB). The backward pass cancels the forward pass's phase exactly, giving *zero* net phase distortion and zero net delay — and it squares the magnitude response, sharpening it.

Common trap

Zero-phase filtering requires running the filter backward in time, which means you must have the *entire signal* already. It is fundamentally non-causal and **cannot be used in real time**. It is perfect for cleaning a stored IMU log offline, useless on the live wrist stream. Do not let a clean offline result fool you into thinking the same filter will work live — the live version will have all the delay back, plus phase distortion.

17.7 Moving average: the workhorse smoother

Almost every real sensor pipeline starts with a moving average, because it is the cheapest possible low-pass filter and it is exactly the right tool for knocking down broadband noise on a slowly-varying signal. The length-L moving average is

$$y[n] = \frac{1}{L}\sum_{k=0}^{L-1} x[n-k]. \tag{17.10}$$

It is an FIR filter whose L taps are all $1/L$ — a rectangular impulse response. We computed its frequency response in Chapter 14; its magnitude is the *aliased sinc* (Dirichlet kernel):

$$|H(e^{\mathrm{j}\omega})| = \frac{1}{L}\left|\frac{\sin(L\omega/2)}{\sin(\omega/2)}\right|. \tag{17.11}$$

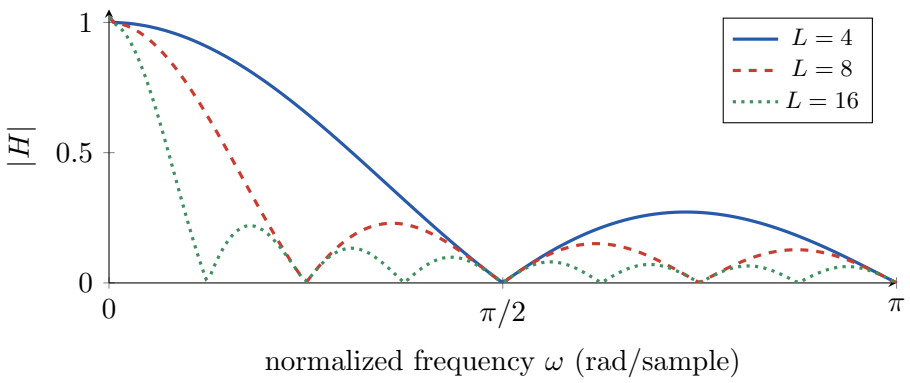


Figure 17.6: Magnitude response of the length-L moving average. Longer windows push the first null lower (more smoothing) but the stopband lobes stay high (≈ -13 dB) — the moving average is a gentle low-pass with leaky stopband, not a sharp filter. The nulls sit at $\omega = 2\pi m/L$.

What is really happening

The moving average is a *terrible* frequency-selective filter — its first sidelobe is only 13 dB down, so it lets plenty of high frequency leak through. But it is a *wonderful* smoother of broadband noise, because averaging L independent noisy samples reduces the noise standard deviation by $\sqrt{L}$ (Chapter 20) while passing slow signal almost untouched. When your goal is "make this jittery sensor reading calmer," not "isolate a precise frequency band," the moving average is usually the right and cheapest answer.

17.7.1 The running-sum trick: $O(1)$ per sample

The naive moving average does L adds per output. But a moving *sum* changes by only two terms each step: add the new sample, subtract the one that just fell off the window. So you can compute it in *constant* time regardless of L:

$$S[n] = S[n-1] + x[n] - x[n-L], \qquad y[n] = S[n]/L. \tag{17.12}$$

```c
// O(1) moving average via running sum. ring[] holds the last L samples.
typedef struct { float ring[64]; int L, pos; float sum; } movavg;

float movavg_step(movavg *m, float x) {
    m->sum += x - m->ring[m->pos];  // add new, drop oldest
    m->ring[m->pos] = x;
    m->pos = (m->pos + 1) % m->L;
    return m->sum / m->L;
}
```

Common trap

The running-sum recurrence is numerically fragile in fixed point or with a constant DC offset: rounding errors and bias accumulate in S forever, since the two correction terms never quite

cancel in finite precision. Over millions of samples S can drift. Use a wide integer accumulator (or periodically recompute the sum from scratch), and prefer it only where the per-sample savings matter. For short L (a handful of taps) just sum directly — it is robust and the cost is trivial.

17.7.2 The exponential moving average (one-pole IIR)

The moving average has a hard, finite window. Its cousin the *exponential* moving average (EMA) weights the past with a smooth exponential decay and is the simplest IIR filter there is:

$$y[n] = \alpha\, x[n] + (1-\alpha)\, y[n-1], \qquad 0 < \alpha < 1. \tag{17.13}$$

One multiply, one add, one state variable. Smaller α means heavier smoothing (a longer effective memory). Its single pole sits at $z = 1-\alpha$, its time constant is roughly $\tau \approx 1/\alpha$ samples, and its -3 dB cutoff is approximately $\omega_c \approx \alpha$ for small α.

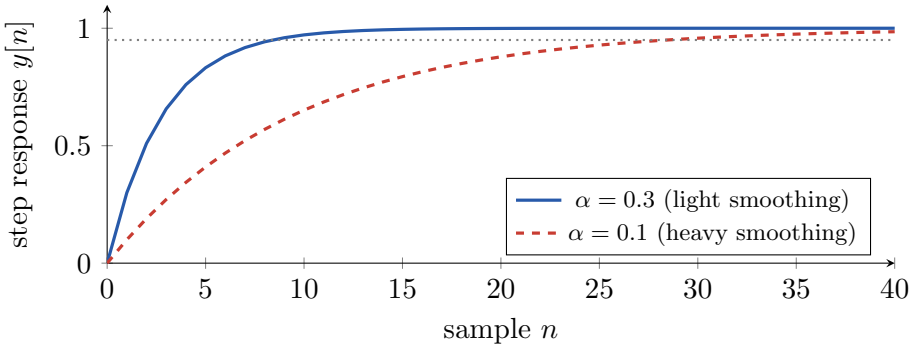


Figure 17.7: Step response of the exponential moving average for two values of α. The output rises toward the new level as $1-(1-\alpha)^n$; smaller α smooths harder but lags more, taking longer to reach the 95% line (dotted). The effective memory is about $1/\alpha$ samples.

Focus

Moving average vs. EMA — which to reach for.

- *Moving average:* flat weighting, sharper transition for the same memory, exact known group delay $(L-1)/2$, but needs an L-sample buffer. Good when you want a defined window and linear phase.
- *EMA:* one state variable, no buffer, trivially cheap, instantly tunable by one knob α. Group delay is roughly $(1-\alpha)/\alpha$ samples. Good when memory is tight and you want a smooth, gently-decaying response — the default for embedded sensor smoothing.

17.8 Median filter: killing spikes without smearing edges

Linear filters — moving average, EMA, any FIR/IIR — share a fatal weakness against *impulsive* noise. A single wild sample (a sensor glitch, an electrical spike, a dropped I2C read returning garbage)

gets *spread* by a linear filter: the moving average smears one bad value across L outputs instead of removing it. Worse, it pulls the average toward the outlier.

The *median* filter replaces each sample with the median of a sliding window of L samples (odd L):

$$y[n] = \text{median}\big(x[n-k],\ \ldots,\ x[n+k]\big), \qquad L = 2k+1. \tag{17.14}$$

The median is a *nonlinear* operation — it is not a convolution, has no frequency response, and does not obey superposition. But it has a magical property: it is immune to outliers. A single spike, no matter how huge, cannot be the median of a window in which it is a minority, so it is simply discarded — while genuine edges and steps in the signal are preserved sharply, not blurred.

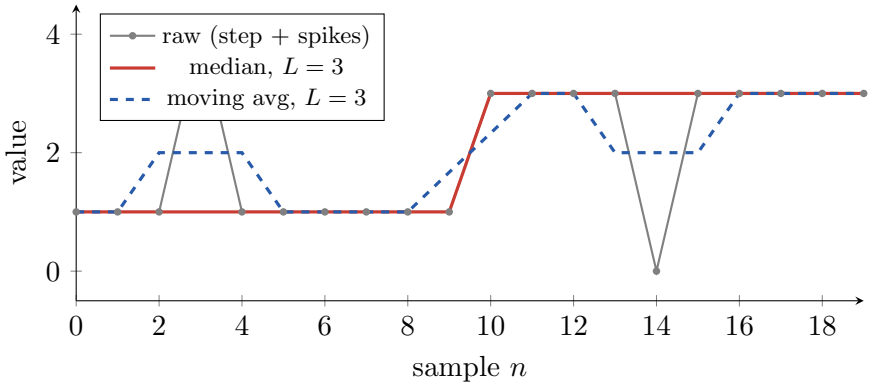


Figure 17.8: Median versus moving average on a step contaminated by two spikes (at $n = 3$ and $n = 14$). The median (solid red) removes both spikes completely and keeps the step edge razor-sharp. The moving average (dashed blue) smears each spike into its neighbors and rounds off the edge.

Why it works

Why does the median preserve edges while the mean blurs them? At a step, the window straddling the edge contains some "low" samples and some "high" samples. The mean returns an intermediate value — a ramp across the edge. The median returns whichever side is the majority — it stays low until the window is more than half past the edge, then jumps. The output is a clean step, just shifted by the window center. Same reason it kills spikes: a spike is a minority of one in its window, so the median never selects it.

```
import numpy as np

def median_filter(x, L):        # L odd
    k = L // 2
    xp = np.pad(x, k, mode='edge')
    return np.array([np.median(xp[i:i+L]) for i in range(len(x))])
```

Common trap

The median filter is more expensive (sorting a window is $O(L \log L)$ naively, or $O(L)$ with a maintained sorted structure) and *nonlinear*: cascading two median filters is not the same as one, and you cannot reason about it with a frequency response. It also rounds off thin peaks

narrower than k samples — if a genuine narrow feature is real signal, the median will eat it just like a spike. Use it specifically against impulsive noise, not as a general smoother.

Application

Median filtering is everywhere outliers appear: removing salt-and-pepper noise from images (a 2D median, Chapter 32), de-glitching GPS fixes, cleaning ECG baselines of motion artifacts, and — for us — rejecting the occasional corrupt sample from a flaky I2C/SPI sensor read. A common, cheap, and very effective combination is a short *median* (length 3 or 5) to kill spikes, followed by a moving average or EMA to smooth the residual broadband jitter.

17.9 Putting a real sensor pipeline together

We now have all the pieces. A robust, low-latency de-noising front end for a streaming sensor is almost always the same small stack, in this order:

1. **De-glitch** with a short median (length 3 or 5) to discard impulsive spikes and dropped-read garbage *before* they can contaminate any averaging.

2. **Smooth** with an EMA or short moving average to knock down the broadband jitter that remains, choosing the cutoff to just cover your gesture frequencies (motion energy lives below ~ 20 Hz).

3. **Remove drift / DC** with a gentle high-pass (a one-pole DC blocker $y[n] = x[n] - x[n-1] + \rho\, y[n-1]$, ρ near 1) if you only care about *changes*, e.g. taps and motion onsets, not absolute orientation.

```c
// Streaming sensor front end: median(3) -> EMA -> DC block.
// State persists across calls; one sample in, one out; minimal latency.
#include <math.h>
typedef struct { float m0,m1,m2; float ema; float xp,yp; } pipe;

static float median3(float a, float b, float c){
    float hi = fmaxf(a,fmaxf(b,c)), lo = fminf(a,fminf(b,c));
    return a + b + c - hi - lo;            // the middle value
}
float pipe_step(pipe *p, float x, float alpha, float rho){
    p->m2 = p->m1; p->m1 = p->m0; p->m0 = x;     // shift median window
    float md = median3(p->m0, p->m1, p->m2);     // 1) de-glitch
    p->ema  = alpha*md + (1.0f-alpha)*p->ema;    // 2) smooth (EMA)
    float dc = p->ema - p->xp + rho*p->yp;       // 3) DC block (high-pass)
    p->xp = p->ema; p->yp = dc;
    return dc;
}
```

Toward the wrist tracker

This three-stage front end is exactly what runs first on every channel of the wrist tracker's IMU — three accelerometer axes and three gyro axes. It is sample-by-sample (no block latency), costs a handful of operations per sample, has no large group delay, and is robust to the glitchy reads you *will* get from a sensor bus on a moving, sweaty wrist. Everything downstream — orientation fusion (Chapter 36), gesture features (Chapter 38) — assumes its input has already been cleaned by something like this.

And the caveat from the start of the book still holds: this pipeline makes the IMU's *coarse motion* signal clean and responsive, but no amount of filtering conjures information the sensor never captured. A wrist IMU can give you crisp, low-latency detection of *discrete* gestures (a flick, a twist, a tap) and continuous *wrist* orientation — it cannot recover individual finger joint angles. For finger-level control you need EMG/FMG on the forearm or sensors on the fingers (Chapter 37). Clean signals, and claims that match what the sensor can actually measure.

17.10 Summary

- **Streaming** means bounded memory on an endless signal; **real-time** means meeting a per-sample deadline. Causality forbids peeking ahead, so every filter delays.
- **Sample-by-sample** processing minimizes latency and suits IIR and smoothers; **block** processing enables FFT-based work but adds one block of delay.
- **Fast convolution** (overlap-add / overlap-save) makes long FIR filtering cheap via the FFT, provided $M \geq L + N - 1$ so circular convolution equals linear. It wins above $\sim$ 60–100 taps; below that, direct convolution is faster and lower latency.
- Implement IIR filters as **cascaded transposed-Direct-Form-II biquads**, never one big direct form, to stay stable and accurate — especially in **fixed point**, where coefficient quantization, roundoff noise, overflow, and limit cycles are the dangers.
- **Latency** is a budget. Linear-phase FIR delays by $(N-1)/2$ samples; short IIR delays by a few; zero-phase `filtfilt` is offline only. For a musical controller, keep total action-to-sound delay near 10 ms.
- For real sensor data, the practical de-noising stack is a short **median** to kill spikes, then a **moving average / EMA** to smooth, optionally a **DC blocker** to remove drift — cheap, robust, and low-latency.

17.11 Exercises

Exercise 17.1. A streaming filter runs on a microcontroller at $f_s = 500$ Hz. Each output sample takes 1.2 ms of compute. Is this real-time? What is the maximum FIR length you could afford if each tap costs 2 μs and you must also leave 30% of the sample period free for other work?

Exercise 17.2. You filter a stream with an FIR of length $N = 200$ using overlap-add with input blocks of length $L = 256$. (a) What is the minimum FFT length M? Round up to a power of two.

(b) How many samples of the overlapping output tail must be saved and added between consecutive blocks? (c) What is the block-induced latency in samples, and in milliseconds at $f_s = 48$ kHz?

Exercise 17.3. Show by direct expansion that circular convolution of a length-3 signal $x = [x_0, x_1, x_2]$ with a length-3 filter $h = [h_0, h_1, h_2]$ at DFT length $M = 4$ does *not* equal their linear convolution, but at $M = 5$ it does. Identify exactly which output sample is corrupted at $M = 4$ and why.

Exercise 17.4. A length-101 symmetric linear-phase low-pass FIR runs at $f_s = 200$ Hz. (a) What is its group delay in samples and in milliseconds? (b) If a musician needs total latency under 10 ms and this filter alone exceeds it, name two concrete changes that would bring the filter's contribution under 5 ms, and state the cost of each.

Exercise 17.5. An exponential moving average uses $\alpha = 0.1$. (a) Where is its pole? (b) Estimate its time constant in samples and its approximate -3 dB cutoff in rad/sample. (c) At $f_s = 200$ Hz, what is that cutoff in Hz? (d) Roughly how many samples does it take for a step input to reach 95% of its final value?

Exercise 17.6. Implement the running-sum moving average of Equation (17.12) in any language and feed it a constant input of 1.0 for 10^7 samples using `float32` arithmetic. Compare the final output to the exact value 1.0. Explain the discrepancy in terms of accumulated roundoff, and show that a `float64` or integer accumulator fixes it.

Exercise 17.7. Take the signal in Figure 17.8 (a step from 1 to 3 at $n = 10$ with a spike of value 4 at $n = 3$ and a spike of value 0 at $n = 14$). By hand, compute the output of (a) a length-3 median filter and (b) a length-3 moving average at samples $n = 3, 4, 13, 14, 15$. Confirm the median removes the spikes and the average smears them.

Exercise 17.8. A second-order IIR resonator has poles at $re^{\pm \mathrm{j}\theta}$ with $r = 0.99$, $\theta = 0.5$. (a) Write the real biquad denominator coefficients a_1, a_2. (b) Explain why this filter is especially prone to fixed-point trouble (limit cycles, coefficient sensitivity), referencing the pole radius. (c) What single implementation choice most reduces the risk?

Exercise 17.9. Explain in one paragraph each why (a) you cannot use SciPy's `filtfilt` zero-phase filtering on the live wrist-controller stream, and (b) a moving average is a poor choice when you need to isolate a narrow frequency band but an excellent choice when you simply want to reduce broadband sensor noise.

Exercise 17.10. Design (on paper) the de-noising front end for the wrist controller's gyroscope channel. The gyro is sampled at $f_s = 500$ Hz, occasionally returns a single garbage sample from a bad SPI read, and carries broadband noise plus a slow temperature-dependent bias drift. Specify each stage (type, length or α), justify your choices against latency and the gesture bandwidth (< 20 Hz), and estimate the total group delay of your chain in milliseconds.

Exercise 17.11. (Harder.) The transposed Direct Form II biquad in the C listing of Section 17.4.2 uses two state variables z_1, z_2. Trace through the update equations for an impulse input $x = [1, 0, 0, \dots]$ with coefficients $b_0 = 1$, $b_1 = b_2 = 0$, $a_1 = -0.9$, $a_2 = 0$ (a one-pole filter dressed as a biquad). Confirm that the output is the expected geometric decay $y[n] = 0.9^n$, and state the pole location.

Part IV

Sampling and Real Signals

Chapter 18

Sampling and the Nyquist Theorem

Every digital instrument we will ever build – a microphone feeding a laptop, an accelerometer feeding a microcontroller, the wrist tracker we are working toward – faces the same first, unavoidable decision. The world is continuous: voltages, pressures, accelerations all vary smoothly in time. A computer, by contrast, can only ever hold a *list of numbers*. Somewhere between the analog world and the digital one, a continuous wiggle must be turned into a finite stream of samples.

The astonishing fact at the heart of this chapter is that, under one clearly stated condition, *nothing is lost*. A signal can be chopped into a discrete sequence of instantaneous readings and then perfectly rebuilt, with no approximation, into the exact continuous waveform you started with. This is the Sampling Theorem, and the condition is the Nyquist criterion. When the condition is violated, something specific and visual goes wrong – the signal does not merely degrade, it *lies* to you, pretending to be a different signal entirely. That lie is aliasing, and learning to see it geometrically is the real payoff of this chapter.

18.1 What sampling actually does

Start with a continuous-time signal $x(t)$, a real number for every real time t. To sample it, we pick a fixed spacing T_s (the *sampling period*) and record the value of x at the instants $t = nT_s$ for integer n:

$$x[n] \;=\; x(nT_s), \qquad n \in \mathbb{Z}.$$

The reciprocal $f_s = 1/T_s$ is the *sampling rate*, measured in samples per second (Hz). If we sample at $f_s = 1000\,\text{Hz}$ we take one reading every $T_s = 1\,\text{ms}$. That is the entire operation: read the signal on a regularly spaced grid of times and throw everything in between away.

Intuition

Picture the continuous signal as a smooth pencil line drawn on graph paper, and sampling as standing only on the vertical grid lines and noting where the pencil line crosses each one. You end up with a row of dots. The deep question of this chapter is: given only the dots, can you redraw the pencil line uniquely? Sometimes yes (Nyquist satisfied), sometimes no (aliasing) – and the boundary between the two cases is sharp.

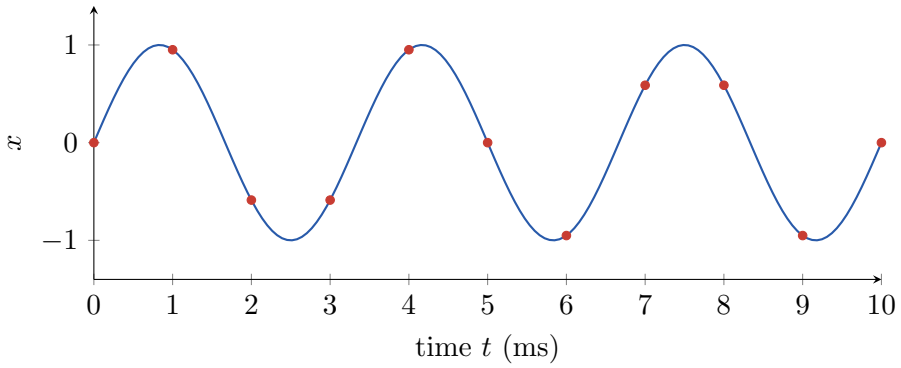


Figure 18.1: A continuous signal (blue) sampled once per millisecond ($f_s = 1\,\text{kHz}$ on this 1 ms-per-unit axis). The red dots are all the computer ever sees; the blue curve is the truth we hope to recover.

18.1.1 Sampling as multiplication by an impulse train

To analyze sampling we need a clean mathematical model of "read on a grid." The cleanest one is multiplication by a *Dirac comb* (also called an impulse train),

$$\delta_{T_s}(t) \;=\; \sum_{n=-\infty}^{\infty} \delta(t - nT_s),$$

a row of infinitely thin, infinitely tall spikes spaced T_s apart, each with unit area. Multiplying our signal by the comb produces the *sampled signal*

$$x_s(t) \;=\; x(t)\,\delta_{T_s}(t) \;=\; \sum_{n=-\infty}^{\infty} x(nT_s)\,\delta(t - nT_s),$$

which is a train of impulses whose areas are the sample values. This idealization keeps the samples living on the continuous-time axis, so we can take its Fourier transform and compare it to the spectrum of the original. That comparison is the whole game, and it produces the spectrum-replication picture that makes everything else obvious.

Formal

The Dirac comb is not an ordinary function; it is a distribution, justified rigorously by how it behaves inside integrals: $\int f(t)\,\delta(t-a)\,\mathrm{d}t = f(a)$. We use it here only as a bookkeeping device. Real hardware does not produce infinite spikes; it produces finite numbers held for a finite time. Section 18.6 returns to what physical converters actually do.

18.2 The spectrum-replication view

Here is where geometry takes over from algebra. The Fourier transform of the Dirac comb is itself a Dirac comb: a comb with spacing T_s in time transforms into a comb with spacing $f_s = 1/T_s$ in frequency. And we already know (from the convolution theorem of the properties chapter) that multiplication in time is convolution in frequency. Putting these two facts together gives the single

most important equation about sampling:

$$X_s(f) \;=\; f_s \sum_{k=-\infty}^{\infty} X(f - k f_s)\,.$$

In words: **sampling replicates the spectrum.** Take the original spectrum $X(f)$, make infinitely many copies of it, shift the copies so they sit centered at every integer multiple of the sampling rate f_s, and add them all up. That sum is the spectrum of the sampled signal.

What is really happening

This is what is *really* happening when you sample. You did not just "lose the in-between values." In the frequency domain you stamped out a periodic pattern: the original spectrum, copied endlessly along the frequency axis at spacing f_s. A discrete-time signal always has a periodic spectrum, and the period is exactly the sampling rate. Every weird sampling phenomenon – aliasing, the Nyquist limit, the need for anti-alias filters – is just a statement about whether those copies stay out of each other's way or crash into each other.

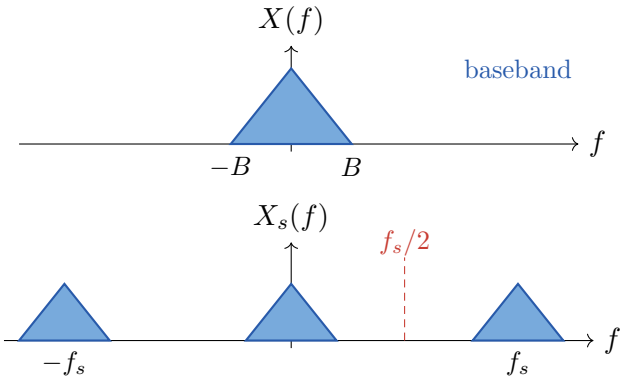


Figure 18.2: Top: the original spectrum is band-limited to $|f| \le B$. Bottom: sampling copies it at every multiple of f_s. Here $f_s > 2B$, so the copies have a clear gap between them; the red line marks the Nyquist frequency $f_s/2$, which falls inside the gap. Recovery is possible.

18.3 Band-limiting, Nyquist, and the Sampling Theorem

The replication picture instantly tells us when sampling is lossless. Suppose the original signal is *band-limited*: its spectrum is exactly zero above some highest frequency B, so all of its energy lives in $|f| \le B$. Each shifted copy of the spectrum is then a lump of width $2B$ centered at kf_s. Two neighboring lumps, centered at 0 and f_s, will avoid overlapping precisely when the right edge of the first ($+B$) sits below the left edge of the second ($f_s - B$):

$$B \;<\; f_s - B \qquad \Longleftrightarrow \qquad f_s \;>\; 2B.$$

Theorem 18.1 (Nyquist–Shannon Sampling Theorem)**.** *If a continuous-time signal $x(t)$ contains no frequency components at or above B (i.e. $X(f) = 0$ for $|f| \ge B$), then $x(t)$ is completely determined by its samples $x(nT_s)$ taken at a uniform rate $f_s = 1/T_s > 2B$. The original signal can be reconstructed*

exactly by

$$x(t) \;=\; \sum_{n=-\infty}^{\infty} x(nT_s)\,\operatorname{sinc}\!\left(\frac{t-nT_s}{T_s}\right), \qquad \operatorname{sinc}(u)=\frac{\sin(\pi u)}{\pi u}.$$

Two pieces of vocabulary deserve careful separation, because beginners constantly mix them up.

- The **Nyquist rate** is a property of the *signal*: it is $2B$, the minimum sampling rate that will work for a signal whose highest frequency is B.
- The **Nyquist frequency** (or folding frequency) is a property of the *sampler*: it is $f_s/2$, the highest frequency a sampler running at f_s can represent without ambiguity.

The sampling theorem is then a single sentence: sample faster than twice the highest frequency present, equivalently keep the highest frequency below your Nyquist frequency, and you lose nothing.

Why it works

Why exactly twice, and not, say, three times? Because a single sinusoid of frequency B has spectral lines at $+B$ and $-B$. Sampling copies those lines around every multiple of f_s. The copy of the $-B$ line shifted up by f_s lands at $f_s - B$. For that intruder to stay above your real $+B$ line you need $f_s - B > B$, i.e. $f_s > 2B$. The factor of two is the unavoidable price of frequency having both a positive and a negative half: every real signal's spectrum is symmetric, so a copy always comes marching in from the right toward your positive frequencies, and you must outrun it by a factor of two.

History

The result is named for Harry Nyquist (Bell Labs, 1928), who established the $2B$ rate in the context of telegraph signaling, and Claude Shannon, who in his 1949 paper *Communication in the Presence of Noise* stated and proved the reconstruction form we use today. The Russian engineer Vladimir Kotelnikov published an equivalent theorem in 1933, and E. T. Whittaker had studied the interpolation series in 1915, which is why you sometimes see it called the Whittaker–Kotelnikov–Shannon theorem. Shannon himself was characteristically modest, calling it "common knowledge in the communication art."

Connection

The reconstruction formula in Theorem 18.1 is an *orthogonal expansion*, exactly like the Fourier series you met earlier. The shifted sincs $\operatorname{sinc}((t-nT_s)/T_s)$ form an orthogonal basis for the space of band-limited signals, and the sample values $x(nT_s)$ are the coordinates of x in that basis. "A band-limited signal is determined by its samples" is the signal- processing way of saying "a vector is determined by its coordinates" – the same inner- product geometry from the linear-algebra companion, now playing out in function space.

18.4 Aliasing: when the copies collide

Now violate the condition. Let the signal contain a frequency B that is *above* the Nyquist frequency, i.e. $B > f_s/2$, equivalently $f_s < 2B$. In the replication picture the lumps are now too wide for

their spacing, and neighboring copies overlap. Where they overlap they add together, and the sum is hopelessly scrambled: there is no way, looking only at the overlapped region, to know how much came from the original lump and how much from its neighbor. That overlap is *aliasing*.

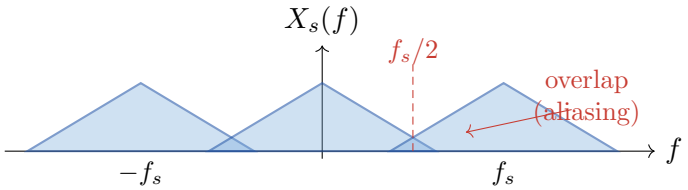


Figure 18.3: Now $f_s < 2B$: each spectral lump is wider than the spacing f_s, so the copies overlap. Energy that belonged at high frequency reappears *folded* back below the Nyquist frequency $f_s/2$, masquerading as a lower frequency.

18.4.1 Folding: where the alias actually lands

The most useful single fact about aliasing is the formula for where a too-high frequency *appears* to be. A pure tone at frequency f_0, sampled at f_s, is indistinguishable from a tone at the *apparent* (alias) frequency

$$f_{\text{alias}} \;=\; \left| \, f_0 - f_s \,\operatorname{round}(f_0/f_s) \, \right|,$$

which always lands in $[0, f_s/2]$. The picture is a sheet of paper folded back and forth like an accordion at every multiple of $f_s/2$: frequencies above Nyquist do not vanish, they reflect ("fold") down into the legal band.

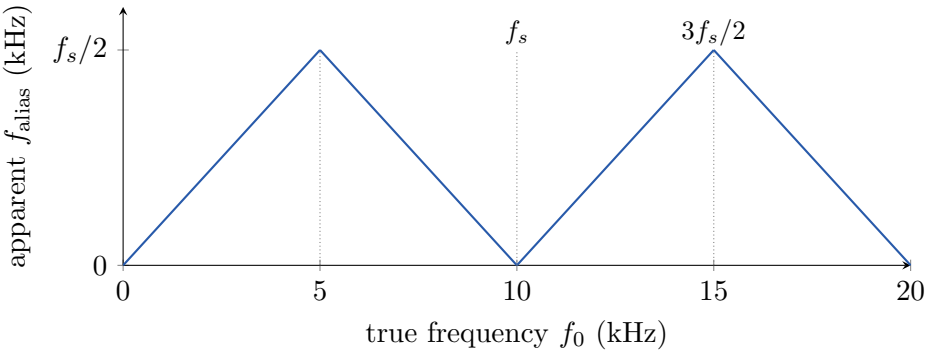


Figure 18.4: The folding map for $f_s = 10\,\text{kHz}$. The apparent frequency is a triangle wave that reflects at every multiple of the Nyquist frequency $f_s/2 = 5\,\text{kHz}$. A true tone at 7 kHz reads back as 3 kHz, exactly as Example 18.1 finds.

Example 18.1 (A tone that lies)**.** Sample a 7 kHz pure tone at $f_s = 10\,\text{kHz}$. The Nyquist frequency is 5 kHz, and $7 > 5$, so we expect trouble. Folding: the nearest multiple of f_s to 7 is $10 \cdot \text{round}(0.7) = 10$, so $f_{\text{alias}} = |7 - 10| = 3\,\text{kHz}$. The samples of the 7 kHz tone are *numerically identical* to the samples of a genuine 3 kHz tone – there is no test, no algorithm, nothing in the data that can tell them apart. The 7 kHz tone has stolen the identity of a 3 kHz tone.

Example 18.2 (Two cosines, one set of samples)**.** Let $f_s = 8\,\text{kHz}$. Consider $\cos(2\pi \cdot 1\,t)$ and

$\cos(2\pi \cdot 9\,t)$ (kHz). At sample times $t = n/8$ ms,

$$\cos\Big(2\pi \cdot 9 \cdot \tfrac{n}{8}\Big) = \cos\Big(2\pi \cdot 1 \cdot \tfrac{n}{8} + 2\pi n\Big) = \cos\Big(2\pi \cdot 1 \cdot \tfrac{n}{8}\Big),$$

because $2\pi n$ is a whole number of turns. The 9 kHz cosine and the 1 kHz cosine produce *exactly* the same samples. This is aliasing reduced to one line of trigonometry: any two frequencies that differ by a multiple of f_s are indistinguishable once sampled.

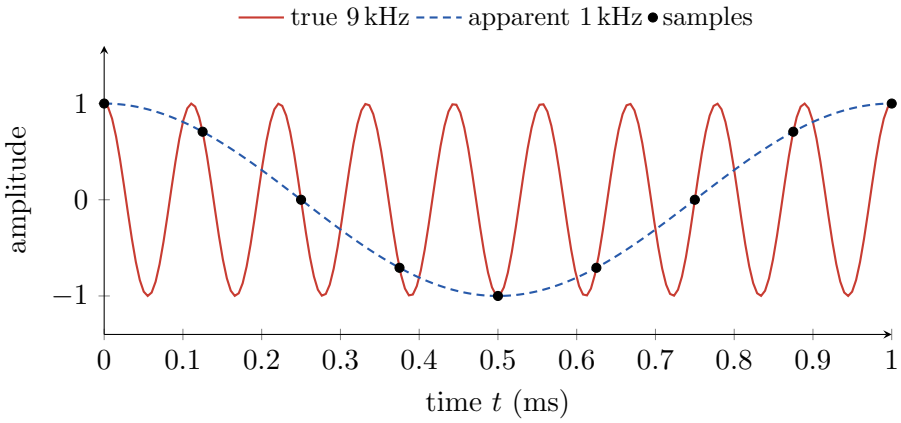


Figure 18.5: The wagon-wheel effect in one picture. Both curves pass through every black sample dot, so the sampler at $f_s = 8$ kHz cannot tell the true 9 kHz tone (solid red) from the slow 1 kHz alias (dashed blue). The fast wiggles between samples are simply invisible to the sampler.

What is really happening

The famous *wagon-wheel effect* in old films – spoked wheels that seem to spin backward or stand still as the stagecoach speeds up – is aliasing in pure form. A movie camera samples the world at about 24 frames per second. When the wheel's spoke- passing frequency climbs past 12 Hz (the camera's Nyquist frequency), each frame catches the spokes having rotated *almost* a full spoke-spacing, so they appear to have crept backward. The wheel is spinning fast; the sampled wheel lies and spins slow. Your eyes are being shown a perfect alias.

Common trap

"My signal only has energy up to 1 kHz, so $f_s = 2$ kHz is fine." Almost never. The theorem requires *strictly* greater than $2B$, and it requires the spectrum to be *exactly* zero at and above B. Real signals have tails, and real anti-alias filters cannot cut infinitely sharply. In practice you sample at 2.2× to 2.5× the highest frequency of interest (audio CD: 44.1 kHz for a 20 kHz audio band) to leave the filter room to roll off. Sampling at exactly the Nyquist rate is a theorem about idealized signals, not engineering advice.

Common trap

Sampling exactly *at* a frequency you care about is its own trap. Sample $\sin(2\pi \cdot 5\,t)$ at $f_s = 10$ kHz and you can hit it precisely at its zero crossings: every sample is 0, and the tone vanishes completely. A frequency sitting exactly on the Nyquist line is ambiguous in phase

and amplitude. This is the deeper reason the theorem says $f_s > 2B$ and not $f_s \geq 2B$.

18.5 Anti-alias filtering: stopping the lie before it happens

Here is the crucial asymmetry: **aliasing is irreversible**. Once two frequency copies have overlapped and added, the sum is a single number per frequency bin, and no amount of clever processing can unmix them. You cannot remove aliasing in software after sampling. Therefore the defense must be installed *before* the sampler, in the analog domain.

That defense is the *anti-alias filter*: an analog low-pass filter placed immediately ahead of the analog-to-digital converter, designed to remove all energy at and above the Nyquist frequency $f_s/2$ before sampling ever takes place. If nothing remains above $f_s/2$, there is nothing to fold down, and the replicated copies cannot overlap. The filter manufactures the band-limited condition that the theorem demands.

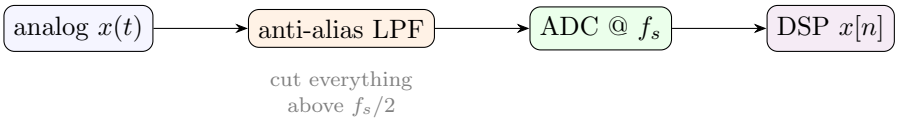


Figure 18.6: The acquisition chain. The anti-alias low-pass filter must sit in the analog domain, ahead of the converter – after the ADC it is far too late.

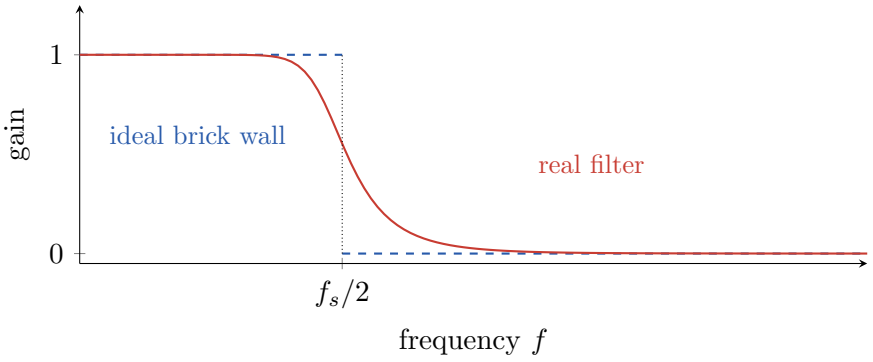


Figure 18.7: An ideal anti-alias filter is a brick wall (dashed): perfect passband, instant cutoff at $f_s/2$. Real analog filters (solid) roll off gradually, which is exactly why we oversample – to give the skirt room to reach the stopband before the Nyquist line.

Why it works

Why must the anti-alias filter be analog? Because aliasing happens *during* sampling, the instant the continuous signal is read on the grid. By the time you have digital samples in memory, the high frequencies have already folded down and are sitting inside your data wearing the disguise of low frequencies. A digital filter applied afterward sees only the disguised result and removes nothing genuine. The only moment to act is while the signal is still continuous and the high frequencies are still where they belong.

Application

Oversampling converters (the sigma-delta ADCs in essentially every modern audio chip, phone, and many IMUs) exploit this cleverly. They sample at a very high rate – often 64× or 256× the final rate – so a gentle, cheap analog filter suffices, then apply a sharp *digital* low-pass filter and discard most samples (decimation; see the resampling chapter). Pushing the brick-wall job into the digital domain, where filters can be made arbitrarily sharp and perfectly linear-phase, is one of the quiet triumphs of modern DSP. The lesson of this chapter is not violated: the initial analog filter still runs ahead of the high-rate sampler.

18.6 Reconstruction: rebuilding the continuous signal

Sampling's mirror image is reconstruction: turning the sample sequence back into a continuous waveform. The replication picture makes the recipe obvious. The spectrum of the sampled signal is the original baseband lump plus all its copies. To recover the original, simply *delete every copy and keep the one at baseband* – that is, apply an ideal low-pass filter with cutoff $f_s/2$. In the frequency domain reconstruction is a single multiplication by a rectangle.

What does that rectangle look like back in the time domain? The inverse Fourier transform of an ideal low-pass rectangle is a sinc. So filtering the impulse train with an ideal low-pass is the same as replacing each impulse with a sinc and adding them up, which is precisely the interpolation series of Theorem 18.1:

$$x(t) \;=\; \sum_{n} x[n]\,\operatorname{sinc}\!\left(\frac{t - nT_s}{T_s}\right).$$

Intuition

The sinc is the perfect interpolating shape. It equals 1 at its own center and exactly 0 at every other sample instant ($t = mT_s$, $m \neq 0$). So in the sum, at each sample time $t = nT_s$ only that sample's sinc contributes and all the others vanish – the reconstruction passes exactly through every data point. Between the points, the overlapping sinc tails conspire to draw in the smoothest possible band-limited curve that threads all the dots. Linear interpolation "connects the dots" with straight lines; sinc interpolation connects them with the unique band-limited curve, which is the genuine original.

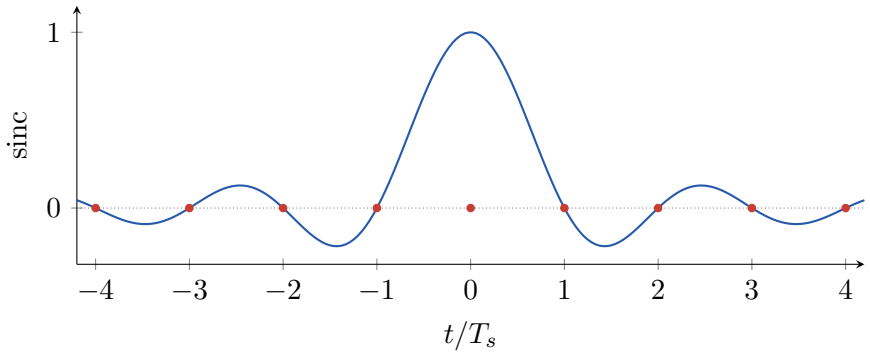


Figure 18.8: The reconstruction kernel $\mathrm{sinc}(t/T_s)$. It is 1 at the origin and zero at every other integer multiple of T_s (red dots). Centering one such curve on each sample, scaling by the sample value, and summing gives back the exact continuous signal.

Common trap

The ideal sinc reconstruction is non-causal and infinitely long – it needs samples from the infinite past and future, so no real device implements it exactly. Practical DACs use a *zero-order hold*: each sample is held flat until the next, producing a staircase. The staircase has a mild built-in low-pass droop (a sinc in frequency, this time from holding rather than from interpolating), which is corrected by a gentle analog *reconstruction filter* after the DAC. Same physics as the input chain, run in reverse.

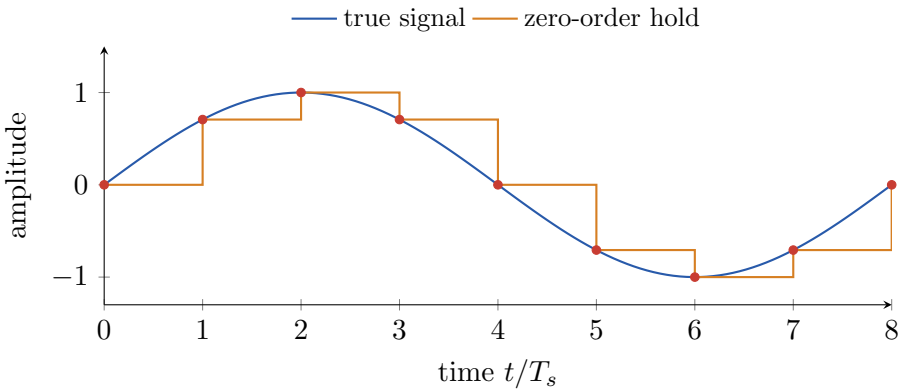


Figure 18.9: A real DAC reconstructs with a zero-order hold (orange staircase): each sample value is held flat until the next. The staircase tracks the true band-limited curve (blue) but introduces a step-shaped error and the mild $|\mathrm{sinc}(fT_s)|$ droop corrected by the analog reconstruction filter.

Example 18.3 (Sinc interpolation by hand)**.** Suppose $T_s = 1$ and the only nonzero samples are $x[0] = 2$ and $x[1] = -1$, with all other samples zero. The reconstructed value halfway between them, at $t = 0.5$, is

$$x(0.5) = 2\,\mathrm{sinc}(0.5) + (-1)\,\mathrm{sinc}(-0.5) = 2 \cdot \frac{2}{\pi} - 1 \cdot \frac{2}{\pi} = \frac{2}{\pi} \approx 0.637.$$

Note this is *not* the linear-interpolation midpoint $(2+(-1))/2 = 0.5$. The band-limited reconstruction lands somewhere different because it is the smoothest curve consistent with *all* the (here zero) samples, not just the two neighbors.

18.7 Choosing a sample rate in practice

The theorem tells you the floor; engineering tells you where to actually stand. The procedure is always the same three steps.

1. Decide the highest frequency B you genuinely need to capture.
2. Choose f_s comfortably above $2B$ – typically $2.2B$ to $2.5B$ – to leave the anti-alias filter room to roll off.
3. Install an analog low-pass anti-alias filter with cutoff near B and adequate attenuation by $f_s/2$.

Table 18.1: Typical highest frequencies of interest and the sample rates chosen for them.

Domain	Useful band B	Common f_s	Why
Telephone speech	3.4 kHz	8 kHz	intelligibility, not fidelity
Music (CD)	20 kHz	44.1 kHz	edge of hearing + filter margin
Pro audio	20 kHz	48/96 kHz	headroom, easier filtering
IMU (human motion)	5–25 Hz	100–200 Hz	gesture dynamics + margin
Surface EMG	$\sim$ 500 Hz	1–2 kHz	muscle spectral content

Toward the wrist tracker

For the wrist gesture controller, sample-rate choice splits cleanly by sensor.

The IMU (accelerometer + gyroscope). Voluntary human limb motion – a flick of the wrist, a tap, a roll of the forearm – has almost all of its energy below about 10–20 Hz; even sharp impacts rarely carry useful structure past $\sim$ 50 Hz. By the theorem we would need only $\sim$ 100 Hz, but we sample the IMU at 100–200 Hz anyway. The extra rate is not for the motion bandwidth; it is for low latency (a MIDI note must fire within a few milliseconds of the gesture) and for robust feature extraction. Sampling an IMU at 1 kHz to chase "finger detail" is wasted power and storage – there is no finger detail in the wrist's motion to alias in the first place.

The EMG / FMG channels. If we add surface EMG to sense muscle activation, the story changes completely: useful sEMG energy extends to roughly 400–500 Hz, so we must sample at 1–2 kHz and, critically, place a real analog anti-alias filter ahead of that ADC. Skipping it lets mains-hum harmonics and high-frequency muscle and motion noise fold straight down into the band where our gesture features live – silently corrupting the very signal we are trying to classify.

The hard limit. No amount of sample rate turns a wrist IMU into a finger tracker. Sampling faster cannot create information the sensor never captured: a device on the forearm sees the gross motion of the limb, not the individual flexion of each finger joint. Continuous per-finger pose needs sensors *on the fingers* (rings, gloves) or muscle-level sensing (EMG/FMG) with a trained classifier – and even then yields coarse, discrete finger states, not smooth joint angles. The Nyquist theorem governs how finely we read each channel; it says nothing about what the channel can see.

Build it (hardware)

On a typical MCU (Arduino-class or a Pi Pico), the safest pattern is a hardware timer-driven, fixed-rate sample loop – never sampling "as fast as the loop happens to run," which produces jitter that smears your spectrum.

```
// Fixed 200 Hz IMU sampling driven by a hardware timer ISR.
// Period = 1/200 s = 5000 us. Keep the ISR tiny: read and stash.
volatile int16_t ax, ay, az, gx, gy, gz;
volatile bool    ready = false;

void onTimer() {                          // fires every 5000 us
  imu_read(&ax,&ay,&az,&gx,&gy,&gz);      // one register burst
  ready = true;                           // hand off to the main loop
}

void loop() {
  if (ready) {
    ready = false;
    process_sample(ax,ay,az,gx,gy,gz); // filtering, features, MIDI
  }
}
```

Three field-tested rules:

- Drive the rate from a timer, not from `delay()` or loop timing – jitter is sampling on an irregular grid, and it injects broadband noise.
- Configure the IMU's *own* on-chip low-pass (its DLPF/ODR registers) as your anti-alias filter; set its bandwidth below $f_s/2$. Most MEMS IMUs include this – use it.
- Match the ODR (output data rate) to your timer rate. If the chip outputs at 1 kHz but you read at 200 Hz without its internal filter engaged, you alias on-chip before the data ever reaches you.

Try it yourself

A five-minute aliasing demo on any laptop. Generate a tone that sweeps upward in frequency and listen as it folds back down after crossing Nyquist:

```
import numpy as np, sounddevice as sd
fs = 8000                              # deliberately low rate
t  = np.arange(0, 4, 1/fs)
f  = np.linspace(0, 7000, t.size)      # sweep 0 -> 7 kHz (past 4 kHz Nyquist!)
x  = np.sin(2*np.pi*f*t/2)             # chirp (factor 2: instantaneous freq = f)
sd.play(0.2*x, fs); sd.wait()
```

You will hear the pitch rise to 4 kHz, then – impossibly – come back *down* again as the true frequency keeps climbing. That descending ghost is the alias of the ascending real tone, folding at the Nyquist frequency. Now raise `fs` to 44100 and the fold vanishes: the sweep climbs cleanly the whole way.

18.8 Bandpass sampling (a useful surprise)

The theorem is usually stated for baseband signals (energy from 0 to B), giving the familiar $f_s > 2B$. But the deeper requirement is only that the spectral copies not overlap – and if a signal's energy lives in a narrow band *far* from zero, the copies can be made to interleave neatly even at a rate far below 2×(highest frequency). This is *bandpass* or *undersampling*, and it is how a radio can digitize a 100 MHz signal of, say, 1 MHz bandwidth with an ADC running at only a few megahertz.

Intuition

The number that matters is the signal's *bandwidth*, not its highest frequency. A 1 MHz-wide lump centered at 100 MHz still only needs its copies spaced so they do not collide, which can be arranged with f_s a little above 2 MHz, provided f_s is chosen so the lump lands cleanly in one of the replication slots. Sampling deliberately aliases the band down to a convenient low frequency, on purpose and losslessly. It is aliasing turned from foe into tool – which only makes sense once you truly see sampling as spectrum replication.

Why this matters

Step back and notice the unity. One picture – copies of the spectrum, stamped out at spacing f_s – explains the Nyquist rate (copies must not touch), aliasing (copies overlapping), the wagon wheel (a copy posing as the original), anti-alias filtering (narrow the lump so copies fit), reconstruction (keep one copy, delete the rest), and bandpass sampling (slot a distant lump into a gap). Master that single image and the entire subject of sampling collapses into one geometric idea you can draw on a napkin.

Summary

- Sampling reads $x(t)$ on a uniform grid $t = nT_s$; in frequency it *replicates* the spectrum at every multiple of $f_s = 1/T_s$.
- **Sampling theorem:** if $X(f) = 0$ for $|f| \geq B$ and $f_s > 2B$, the samples determine $x(t)$ exactly. $2B$ is the Nyquist *rate*; $f_s/2$ is the Nyquist *frequency*.
- **Aliasing** is spectral copies overlapping; a too-high tone f_0 folds to an apparent frequency in $[0, f_s/2]$ and becomes indistinguishable from it. It is irreversible.
- **Anti-alias filtering** must be analog and precede the ADC; it band-limits the signal so the copies cannot overlap.
- **Reconstruction** keeps the baseband copy via an ideal low-pass, equivalent to summing shifted, scaled sincs through the data points.
- In practice, oversample (2.2–2.5×). For the wrist tracker: $\sim$ 100–200 Hz for the IMU, 1–2 kHz with a real anti-alias filter for EMG – and no sample rate ever conjures finger detail the sensor cannot see.

18.9 Exercises

Exercise 18.1. A signal contains frequencies up to 4.5 kHz. (a) What is the Nyquist rate? (b) You sample at 9 kHz exactly – is this safe in practice, and why or why not? (c) Suggest a practical sample rate and justify it.

Exercise 18.2. A 13 kHz pure tone is sampled at 10 kHz. Using the folding formula, find the apparent (alias) frequency. Then verify your answer by writing the sampled cosine and reducing its argument modulo 2π.

Exercise 18.3. Two sinusoids of frequency f_1 and f_2 produce identical samples at rate f_s. Show that f_1 and f_2 must satisfy $f_2 = f_1 + kf_s$ or $f_2 = -f_1 + kf_s$ for some integer k. Interpret the second family in terms of the folding picture.

Exercise 18.4. Sketch, by hand, the replicated spectrum $X_s(f)$ for a triangular baseband spectrum of half-width $B = 3$ kHz when (a) $f_s = 10$ kHz and (b) $f_s = 5$ kHz. In which case do the copies overlap, and over what frequency interval?

Exercise 18.5. Explain in one or two sentences why an anti-alias filter implemented as a digital FIR filter applied to the samples cannot prevent aliasing, no matter how sharp it is made.

Exercise 18.6. The wagon wheel of a stagecoach has 12 evenly spaced spokes and is filmed at 24 fps. (a) At what wheel rotation rate (rev/s) does the wheel first appear *stationary*? (b) Just below that rate, which way does the wheel appear to turn?

Exercise 18.7. Using the sinc interpolation series with $T_s = 1$, reconstruct the value at $t = 0.25$ for a signal whose only nonzero samples are $x[0] = 1$ and $x[2] = 1$. Compare with linear interpolation between the nearest samples.

Exercise 18.8. A zero-order-hold DAC outputs each sample held for T_s. Its frequency response has magnitude $|\mathrm{sinc}(fT_s)|$. Compute the gain droop (in dB) at $f = f_s/4$ relative to DC, and explain why a reconstruction filter is needed.

Exercise 18.9 (Toward the wrist tracker)**.** You are logging a wrist IMU for gesture recognition. The fastest deliberate motion you care about is a wrist flick with energy up to ~ 15 Hz. (a) What is the minimum sample rate? (b) Why might you still choose 200 Hz? (c) Your IMU streams at a fixed internal rate of 1 kHz but you only read every fifth sample in software, with the chip's on-board low-pass disabled. Describe precisely what goes wrong and how to fix it.

Exercise 18.10 (Bandpass sampling)**.** A radio signal occupies 99.5 to 100.5 MHz (bandwidth 1 MHz). Explain why an ADC running at only a few MHz could, in principle, capture it without loss, and state what property of f_s must be arranged. What real-world impairment makes this harder than the clean theory suggests?

Chapter 19

Quantization, ADCs, and DACs

In the last chapter we cut time into a grid. Sampling answered the question *when* do we look at a signal, and Nyquist told us how often we must look so that nothing is lost. But there is a second grid hiding inside every digital system, and it is just as important and far less discussed: the grid on the *vertical* axis. A computer cannot store the real number $0.7182818\ldots$ It stores a finite pattern of bits. So when the analog-to-digital converter (the ADC) measures a voltage, it must not only pick a moment in time – it must also snap the measured value to the nearest rung on a ladder of allowed values. That snapping is **quantization**, and it is the subject of this chapter.

Sampling is about the horizontal grid; quantization is about the vertical grid. Together they turn a smooth analog wiggle into a finite list of integers – the only thing a microcontroller, a CPU, or a DSP chip can actually hold. Going the other way, a digital-to-analog converter (the DAC) reads integers back out and rebuilds a voltage you can send to a speaker or a motor. This chapter builds the geometric picture of that vertical grid, derives the famous "about 6 dB per bit" rule from scratch, opens up the two ADC architectures you will actually meet (successive-approximation and sigma-delta), and shows why oversampling and *adding noise on purpose* (dithering) can buy you resolution that the raw bit count seems to forbid. We end at the bench, reading a real sensor through the ADC on an Arduino and a Raspberry Pi.

19.1 The vertical grid: from a value to a code

Picture the ADC's job geometrically. The input is a voltage x somewhere in an allowed range, say from 0 to a reference voltage V_{ref}. The output is one of a finite set of integer *codes*: $0, 1, 2, \ldots, 2^N - 1$ for an N-bit converter. The converter draws a ladder of evenly spaced horizontal lines across the input range and reports which rung your voltage is nearest. The spacing between rungs is the single most important number in the whole chapter:

$$\Delta \;=\; \frac{V_{\text{ref}}}{2^N}.$$

We call Δ the **quantization step** or the value of one **LSB** (least significant bit) – the smallest change in voltage that can possibly change the output code.

Intuition

Sampling chops a continuous curve into a row of vertical pins (one per sample instant). Quantization then files the top of each pin down to the nearest line on a piece of ruled paper. After both operations the signal lives entirely on a finite lattice: finitely many time slots across, finitely many value-rungs up. A digital signal is a set of points on graph paper, and nothing in between exists.

Figure 19.1 shows the transfer characteristic – output code versus input voltage – for a tiny 3-bit converter (eight rungs). It is a staircase. Inside each tread the output does not change at all; cross a riser and the code jumps by one. The flat treads are exactly where information is being thrown away: every input voltage in a tread is reported by the same code.

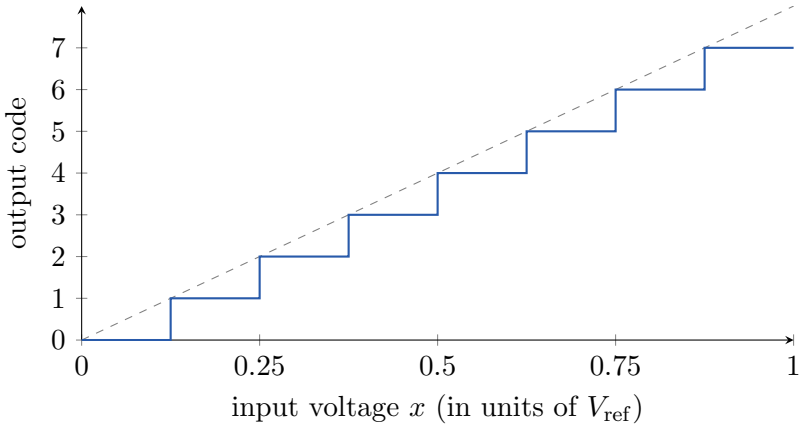


Figure 19.1: The quantizer transfer characteristic for a 3-bit converter (8 codes). The dashed line is the ideal continuous response; the blue staircase is what you actually get. Each flat tread has width $\Delta = V_{\text{ref}}/8$, and any input within a tread maps to the same code.

Definition 19.1 (Uniform quantizer)**.** A **uniform quantizer** with step Δ maps a real input x to the nearest multiple of Δ:

$$Q(x) \;=\; \Delta \cdot \operatorname{round}\!\left(\frac{x}{\Delta}\right).$$

The integer $\operatorname{round}(x/\Delta)$, clipped to the available code range, is the digital code stored in memory.

Bit depth. The number N is the **bit depth** or resolution. Each extra bit doubles the number of rungs and halves Δ. The vocabulary you will hear:

- **8-bit**: 256 levels. Old samplers, simple sensors, control loops where you only need coarse values.
- **10/12-bit**: the ADCs built into microcontrollers (Arduino Uno is 10-bit, many Cortex-M parts are 12-bit). Plenty for an IMU axis or a slider.
- **16-bit**: CD-quality audio. 65,536 levels.
- **24-bit**: studio audio and high-end sigma-delta sensor front-ends.

Common trap

Bit depth and sample rate are independent axes and people constantly confuse them. Sample rate (Chapter 18) is the horizontal grid spacing and is governed by Nyquist; it decides the highest *frequency* you can represent. Bit depth is the vertical grid spacing; it decides the smallest *amplitude change* you can represent, which sets the noise floor. A 1 MHz, 4-bit converter and a 1 kHz, 24-bit converter are good at completely different things. Always ask both "how fast?" and "how fine?"

Mid-rise versus mid-tread, and clipping

There are two ways to position the staircase. A *mid-tread* quantizer has a flat tread centered on zero, so small inputs map to code zero – handy because true silence reads as exactly zero. A *mid-rise* quantizer has a riser at zero, so it never outputs an exact zero but treats positive and negative symmetrically. Audio converters are usually mid-tread. The difference is one half-LSB of offset and rarely matters in practice, but it explains why your DC reading might sit on 0 or might dither between -1 and $+1$ codes.

What always matters is **clipping**. The ladder has a top and a bottom. Push the input above V_{ref} (or below ground) and the code saturates at its maximum (or minimum) and *stays there*. Geometrically the staircase goes flat forever. Clipping is not graceful rounding – it is a hard wall that flattens the peaks of your waveform into plateaus, generating a spray of harmonics. Sampling errors can be undone by a filter; clipping destroys information irreversibly. Set your input gain so peaks land just below full scale, never above.

Connection

The round-to-nearest rule is exactly the projection idea from the linear-algebra companion, applied to a one-dimensional lattice. $Q(x)$ is the closest point of the set $\Delta\mathbb{Z}$ to x, i.e. the orthogonal projection of x onto a discrete grid. Quantization error is the residual $x - Q(x)$, the part of x that the grid cannot represent – the same "what is left over after projection" object you met with Fourier coefficients, now in the crudest possible space.

19.2 Quantization error as noise

The interesting quantity is not $Q(x)$ but the **error** it makes:

$$e \;=\; Q(x) - x.$$

Because Q rounds to the nearest rung, and the rungs are spaced Δ apart, the error can never exceed half a step in magnitude:

$$-\tfrac{\Delta}{2} \;\le\; e \;\le\; \tfrac{\Delta}{2}.$$

That bound is exact and unconditional. The error is the vertical distance from each pin-top down (or up) to the nearest ruled line, and that distance is at most half the line spacing.

Here is the move that makes quantization tractable. For a busy signal – music, an accelerometer trace, anything that swings across many codes from sample to sample – the error e behaves like a

random number, uniformly spread across the interval $[-\Delta/2, \Delta/2]$, and roughly independent from one sample to the next. We do not need this to be exactly true; we need it to be a good model, and for active signals it is excellent. Under that model, quantization is just a weak, broadband **noise** added to the clean signal.

Why it works

Why is treating a *deterministic* rounding rule as random legitimate? Because the signal sweeps across the treads in a way that is, from the error's point of view, effectively unpredictable. If x moves by a fraction of an LSB that has no simple relation to Δ, the fractional part $x/\Delta \bmod 1$ tumbles through $[0, 1)$ and visits all positions about equally. The error is a deterministic function of an effectively-uniform phase, so it inherits a uniform distribution. This is the same reason that a fast-spinning analog phase looks like a random angle. The model fails only when the signal is too quiet or too slow to stir the phase – which is precisely the case dithering will fix in Section 19.6.

The power of the noise

Model e as uniform on $[-\Delta/2, \Delta/2]$. Its mean is zero, and its power is its variance – a textbook integral from the probability companion:

$$\sigma_e^2 \;=\; \mathbb{E}[e^2] \;=\; \frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e^2 \,\mathrm{d}e \;=\; \frac{\Delta^2}{12}.$$

That little $\Delta^2/12$ is one of the most useful numbers in all of DSP. It says the quantization noise power depends only on the step size, and shrinks as the *square* of the step. Halve Δ (add one bit) and you cut the noise power by a factor of four.

Connection

$\sigma_e^2 = \Delta^2/12$ is just the variance of a uniform distribution, derived in the probability book. The $1/12$ is not magic: $\mathrm{Var}(U)$ for U uniform on an interval of width w is $w^2/12$, and here $w = \Delta$. Quantization is your first concrete meeting with a *random signal* – a noise process whose statistics you can compute exactly. Chapter 20 picks up this thread and treats noise in earnest.

What is really happening

Visualize what $\Delta^2/12$ means on the graph. After quantizing, your reconstructed waveform is the true waveform plus a thin, grassy fuzz of height about $\pm\Delta/2$. That fuzz is spread fairly evenly across *all* frequencies (it is broadband, "white"). It is not a distortion of the shape so much as a fine hiss riding on top. Crucially, its total power is fixed by Δ alone – it does not care how loud or quiet your signal is. That single fact drives the entire SNR story below: a loud signal stands far above a fixed hiss; a quiet one drowns in it.

19.3 Signal-to-noise ratio and the 6 dB-per-bit rule

Now we can answer the question every spec sheet brags about: how good is an N-bit converter? We compare the power of the largest signal it can carry against the power of the quantization hiss. That ratio, in decibels, is the **signal-to-noise ratio** (SNR).

Take a full-scale sinusoid – the loudest signal that fits without clipping. Its peak-to-peak swing fills the whole range $2^N\Delta$, so its amplitude is $A = 2^N\Delta/2 = 2^{N-1}\Delta$. A sinusoid of amplitude A has power

$$P_{\text{sig}} \;=\; \frac{A^2}{2} \;=\; \frac{(2^{N-1}\Delta)^2}{2} \;=\; \frac{2^{2N}\,\Delta^2}{8}.$$

The noise power is $\sigma_e^2 = \Delta^2/12$. Their ratio is

$$\text{SNR} \;=\; \frac{P_{\text{sig}}}{\sigma_e^2} \;=\; \frac{2^{2N}\Delta^2/8}{\Delta^2/12} \;=\; \frac{12}{8}\,2^{2N} \;=\; \frac{3}{2}\,2^{2N}.$$

The Δ^2 cancels – a beautiful and important fact. The SNR of an ideal converter does not depend on the reference voltage or the step size; it depends only on the number of bits. Now convert to decibels:

$$\text{SNR}_{\text{dB}} \;=\; 10\log_{10}\!\left(\frac{3}{2}\,2^{2N}\right) \;=\; 10\log_{10}\tfrac{3}{2} \;+\; 2N\cdot 10\log_{10}2.$$

Plug in $10\log_{10}2 \approx 3.0103$ and $10\log_{10}\frac{3}{2} \approx 1.76$:

$$\boxed{\text{SNR}_{\text{dB}} \;\approx\; 6.02\,N \;+\; 1.76\ \text{dB}.}$$

Focus

The 6 dB-per-bit rule. Each additional bit of resolution buys you about 6.02 dB of SNR – roughly a factor of two in amplitude, four in power. The +1.76 dB is the bonus from using a full-scale sinusoid (a constant that depends on the signal's crest factor; it changes if the signal is not a sinusoid, but the $6.02\,N$ slope is universal).

Example 19.1 (Putting numbers on it)**.**

- **8-bit:** $6.02(8) + 1.76 \approx 49.8$ dB. Audibly hissy for music, fine for a control signal.
- **12-bit** (typical MCU ADC): $6.02(12) + 1.76 \approx 74$ dB. Excellent for an IMU axis or a slider.
- **16-bit** (CD): $6.02(16) + 1.76 \approx 98$ dB. The hiss sits about 98 dB below a full-scale tone – below the threshold of hearing in a normal room.
- **24-bit:** ≈ 146 dB in theory. In practice the analog electronics (thermal noise in resistors, the reference, the op-amps) hit a wall around 20–21 bits of *effective* resolution; the last bits are buried in real noise. This is why we speak of ENOB.

Definition 19.2 (ENOB)**.** The **effective number of bits** of a real converter is obtained by measuring its actual SNR (including all real-world noise and distortion) and inverting the formula:

$$\text{ENOB} \;=\; \frac{\text{SNR}_{\text{dB}} - 1.76}{6.02}.$$

A converter labelled "16-bit" with a measured SNR of 90 dB has ENOB ≈ 14.7 bits. The marketing number is the size of the register; ENOB is what you actually get.

Figure 19.2 plots the ideal line. It is dead straight, slope 6.02 dB/bit – one of the cleanest "more is better" curves in engineering.

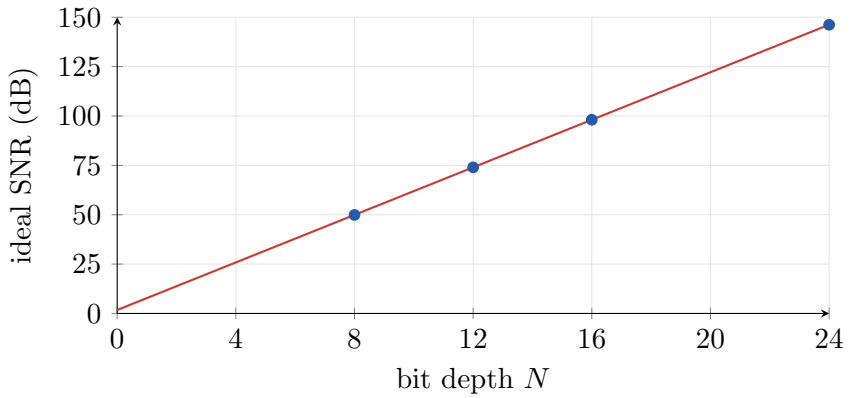


Figure 19.2: Ideal SNR versus bit depth, SNR $\approx 6.02N + 1.76$ dB. Dots mark the common depths from Example 19.1. Real converters fall below this line by their noise and distortion; the vertical gap is what ENOB captures.

Walking question

If quantization noise power is fixed at $\Delta^2/12$ regardless of the signal, what happens to the SNR of a *quiet* signal – say one using only the bottom few codes? Its P_{sig} plummets while the noise stays put, so SNR collapses. This is why recording engineers chase a hot input level, and why a 16-bit recording made at -40 dB has the effective fidelity of a roughly 9-bit recording. Use the whole ladder.

19.4 Seeing and hearing quantization

Let us make the hiss visible. Figure 19.3 quantizes a sine wave to a brutal 3 bits so the staircase is obvious. The smooth curve is the original; the stepped curve is what you store; the gap between them, sample by sample, is the error e – and you can see it is a little sawtooth-like wiggle bounded by $\pm\Delta/2$.

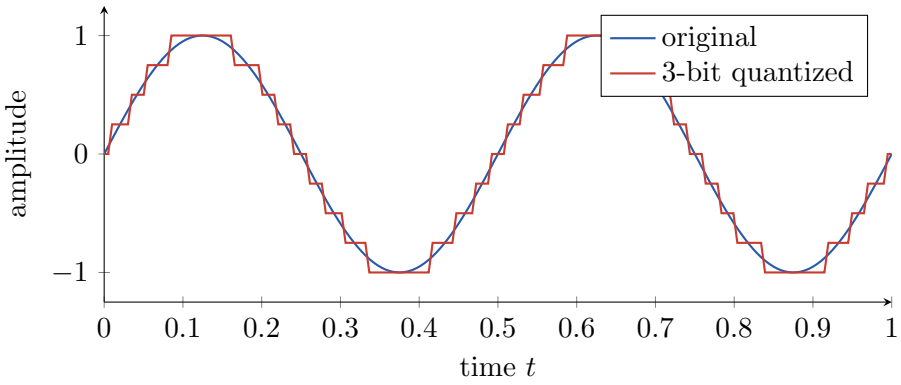


Figure 19.3: A sine wave quantized to 3 bits (step $\Delta = 0.25$ on a $[-1, 1]$ range). The red staircase is the stored signal; the vertical gaps to the blue curve are the quantization error, each no larger than $\Delta/2 = 0.125$.

The error itself is worth a plot of its own. Figure 19.4 draws $e = Q(x) - x$ for that same 3-bit sine. It is a jagged little signal that lives entirely inside the band $[-\Delta/2, \Delta/2]$ and never escapes it – the bound from Section 19.2 made visible. For a busy signal this trace looks like grass; that is the white-noise model at work.

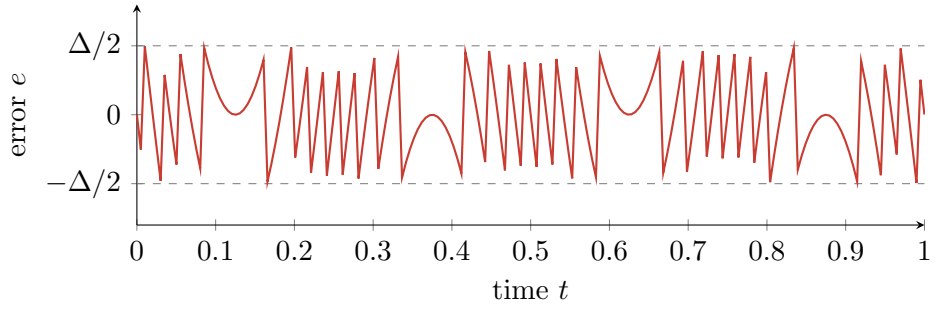


Figure 19.4: The quantization error $e = Q(x) - x$ for the 3-bit sine of Figure 19.3. It stays strictly between the dashed bounds $\pm\Delta/2$ and, for an active signal, scatters like broadband noise.

Try it yourself

Quantize a tone yourself and *listen* to the error. In Python:

```
import numpy as np
fs = 44100
t  = np.arange(fs) / fs           # 1 second
x  = 0.9 * np.sin(2*np.pi*440*t)  # full-ish-scale A4

def quantize(x, bits):
    levels = 2**bits
    step   = 2.0 / levels         # x in [-1, 1]
    return step * np.round(x / step)

for b in (16, 8, 4, 2):
    xq  = quantize(x, b)
```

```
    err = xq - x
    snr = 10*np.log10(np.mean(x**2) / np.mean(err**2))
    print(f"{b:2d}-bit  measured SNR = {snr:6.2f} dB")
```

You will see the measured SNR track $6.02b + 1.76$ closely for the higher bit depths, and the error audibly turns from gentle hiss (high bits) into gritty, signal-correlated distortion (2–4 bits). That correlation – the error "following" the tone – is the failure of the random model at low bit depth, and the reason dither exists.

19.5 How a real ADC works

So far the quantizer has been an abstract staircase. How does silicon actually find the nearest rung? There are several architectures; two dominate the world you will build in, and they sit at opposite ends of a speed/resolution trade-off.

19.5.1 Successive approximation (SAR): binary search in hardware

The successive-approximation register (SAR) ADC is the workhorse inside almost every microcontroller. Its idea is a binary search, and it is genuinely elegant.

The converter holds an internal DAC and a comparator. To convert an unknown input x, it asks a sequence of yes/no questions, one per bit, from the most significant bit down:

1. Guess the MSB is 1: set the trial DAC output to half of full scale, $V_{\text{ref}}/2$. Ask the comparator: is x above this?
2. If yes, keep that bit as 1; if no, clear it to 0. Either way, move to the next bit and add (or not) the next halving, $V_{\text{ref}}/4$.
3. Repeat down to the LSB. After N comparisons the register holds the code.

Each step halves the remaining uncertainty – exactly binary search on the value axis. An N-bit SAR takes N comparator decisions, so its conversion time grows only *linearly* with resolution. That makes SAR fast, low-power, and cheap, which is why it is everywhere from 10-bit to 18-bit and from kHz to several MHz sample rates.

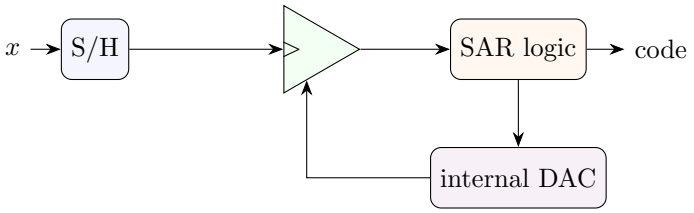


Figure 19.5: A SAR ADC. A sample-and-hold (S/H) freezes the input; the SAR logic toggles bits of an internal DAC and uses the comparator to do a binary search, settling on the code in N steps.

Why it works

Why the sample-and-hold matters: the binary search takes N comparisons during which the input must not move, or the early (big) decisions and the late (small) decisions would be about *different* voltages and the answer would be garbage. The S/H grabs the value at one instant and holds it steady on a capacitor while the search runs. This is the hardware that realizes the "sample at an instant" idealization of Chapter 18 – a real, physical freezing of the signal.

Example 19.2 (A SAR binary search by hand)**.** A 4-bit SAR with $V_{\text{ref}} = 8$ V converts $x = 5.3$ V. With a half-LSB offset added so the search rounds to nearest, the trial weights are $4, 2, 1, 0.5$ V.

- Bit 3: trial 4.0. Is $5.3 > 4.0$? Yes $\rightarrow$ bit $= 1$, accumulator $= 4.0$.
- Bit 2: trial $4.0 + 2.0 = 6.0$. Is $5.3 > 6.0$? No $\rightarrow$ bit $= 0$, accumulator stays 4.0.
- Bit 1: trial $4.0 + 1.0 = 5.0$. Is $5.3 > 5.0$? Yes $\rightarrow$ bit $= 1$, accumulator $= 5.0$.
- Bit 0: trial $5.0 + 0.5 = 5.5$. Is $5.3 > 5.5$? No $\rightarrow$ bit $= 0$, accumulator stays 5.0.

Code $= 1010_2 = 10$, representing 5.0 V. The true value was 5.3 V; the residual error is -0.3 V. Note $\Delta = 0.5$ V here, so $\Delta/2 = 0.25$ V; the 0.3 V residual exceeds it because the simple truncating search lands on the rung *below*. A real SAR adds the half-LSB offset to its comparator threshold so the worst-case error becomes the promised $\Delta/2$. A small but instructive subtlety.

19.5.2 Sigma-delta: trade speed for resolution by oversampling

The other architecture you will meet – in audio codecs, in 24-bit sensor front-ends, in the ADCs on precision IMU and EMG chips – is the **sigma-delta** ($\Sigma\Delta$) converter. It is almost the philosophical opposite of SAR. Where SAR resolves a value finely in one shot, sigma- delta uses a crude quantizer (often just *one bit*) running absurdly fast, then does the real work in arithmetic.

The trick has three ingredients, and each is worth seeing on its own.

1. Oversampling spreads the noise out. Suppose you sample far faster than Nyquist requires – by an **oversampling ratio** $\text{OSR} = f_s/(2f_{\max})$. The total quantization noise power is still $\Delta^2/12$, but now it is spread (white) across a much wider band, from 0 to $f_s/2$. The fraction that falls in your actual signal band $[0, f_{\max}]$ is only $1/\text{OSR}$ of the total. After you digitally low-pass filter and discard everything above $f_{\max}$, the in-band noise power drops by OSR:

$$\sigma^2_{\text{in-band}} = \frac{\Delta^2/12}{\text{OSR}}.$$

In decibels that is a gain of $10\log_{10}\text{OSR}$. Every quadrupling of the sample rate (OSR $\times$ 4) wins ≈ 6 dB – one bit of effective resolution – *for free*, purely by sampling faster and filtering.

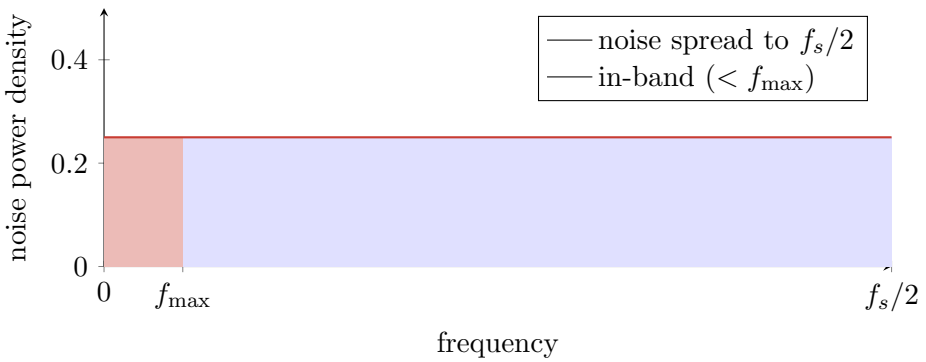


Figure 19.6: Oversampling. The same total quantization noise (area under the floor) is spread across a much wider band, so the slice that lands in the signal band $[0, f_{\max}]$ (red) is small. A digital filter keeps the red part and discards the rest.

2. Noise shaping pushes the noise uphill. Sigma-delta does much better than plain oversampling by *shaping* the noise spectrum so it slopes steeply upward with frequency, piling almost all of the quantization noise into the high band that the filter will throw away. A feedback loop integrates the difference (the "delta") between input and the quantizer's output and feeds it back. The loop forces the average output to track the input – the low frequencies come through cleanly – while the inevitable error is shoved up to high frequencies. A first- order loop wins 9 dB per octave of OSR instead of 3; higher-order loops win even more. This is why a 1-bit modulator at a few MHz can deliver 20+ effective bits in the audio band.

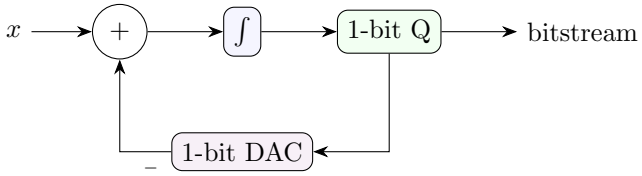


Figure 19.7: A first-order sigma-delta modulator. The integrator accumulates the error between input and the fed-back 1-bit estimate; the feedback drives the running average of the output to equal the input, while pushing quantization noise to high frequency where a decimation filter removes it.

3. Decimation turns the fast bitstream into clean samples. The modulator emits a torrent of single bits at the high oversampled rate. A digital low-pass *decimation* filter averages and downsamples this stream to the final output rate, producing high-resolution multi-bit samples in the signal band. (Decimation gets its own treatment in Chapter 22.) The net effect: the analog part is trivial (a comparator and an integrator), and the heavy lifting is done by cheap digital arithmetic – exactly the trade modern chips love.

Focus

SAR vs. sigma-delta in one breath. SAR: one sample, N fast comparisons, moderate resolution, easy to multiplex across many channels, low latency – ideal for control loops, IMUs, general MCU sensing. Sigma-delta: oversample hard, shape the noise, decimate – very high resolution, built-in anti-alias filtering, but higher latency and best for one continuous channel

– ideal for audio and precision instrumentation.

19.6 Dithering: adding noise to get resolution

Return to the failure mode we keep flagging: when the signal is small or slowly varying, the quantization error stops looking random. It becomes *correlated* with the signal, and correlated error is not benign hiss – it is **distortion**, harmonics that the ear and the spectrum analyzer both find ugly. The cure is counterintuitive and beautiful: **add a small amount of random noise before quantizing.**

Intuition

Imagine a quiet sine wave whose entire swing fits between two rungs of the ladder. Without help, the quantizer reports a flat line at the nearer rung – the tone vanishes into a stair-step and its information is lost. Now sprinkle in a little noise of about one LSB. On each sample the noisy value sometimes pushes up to the next rung and sometimes down. The *individual* outputs are wrong, but their *running average* tracks the true sub-LSB value. You have traded a coherent, audible error for incoherent hiss – and incoherent hiss is far easier to ignore and can be averaged or filtered away.

Why it works

Dither works because averaging recovers information that a single quantized sample cannot hold. A value sitting 0.3 LSB above a rung, dithered, will round up about 30% of the time and down 70% of the time; the long-run mean of the codes is exactly 0.3 LSB above the rung. The dither *linearizes* the quantizer on average: it converts a deterministic, signal-dependent rounding into an unbiased estimator whose error is independent of the signal. You pay a small, fixed increase in noise floor and in return you banish the distortion harmonics entirely. For well-chosen dither (triangular PDF, two LSB peak-to-peak) the residual error is not only decorrelated but its variance becomes independent of the signal level – the gold standard.

This is not a hack confined to audio. It is standard in every high-quality ADC and DAC, in image processing (the "dithered" gradients that hide banding in 8-bit images), and in any place a coarse grid would otherwise create visible or audible structure. The deep idea – *break correlated error into uncorrelated noise, then average* – recurs throughout signal processing.

Application

Audio mastering. When a 24-bit studio mix is reduced to 16-bit for a CD, engineers add TPDF (triangular probability density) dither, often with a spectral shape that pushes the added noise into frequencies where the ear is least sensitive ("noise-shaped dither"). The result is that quiet reverb tails fade smoothly into silence instead of chopping off in audible steps, even though only 16 bits are stored.

19.7 From bits back to the world: the DAC

A digital-to-analog converter runs the staircase in reverse: given a code, produce a voltage. The simplest mental model is a bank of switches and a resistor network (an "R-2R ladder") that adds up weighted contributions of each bit. But the geometry that matters is what comes *out*.

A DAC does not emit the ideal samples (infinitely thin pins). It holds each sample's voltage constant until the next one arrives – a **zero-order hold** (ZOH). The output is a staircase in time, not a smooth curve. That staircase is the samples convolved with a rectangular pulse one sample wide, and convolution with a rectangle in time means multiplication by a *sinc* in frequency:

$$H_{\text{ZOH}}(f) \;=\; T_s\,\operatorname{sinc}(fT_s)\,e^{-\mathrm{j}\pi fT_s}, \qquad |H_{\text{ZOH}}(f)| = T_s\,|\operatorname{sinc}(fT_s)|.$$

Two consequences follow, and both have fixes.

1. **High-frequency droop.** The sinc magnitude sags as f rises, reaching about -3.9 dB at the band edge $f = f_s/2$. Your highest wanted frequencies come out slightly quiet. Fix: a gentle digital or analog *de-emphasis* that boosts the top of the band by the inverse sinc.
2. **Spectral images.** The staircase contains copies (images) of your spectrum around every multiple of f_s. Fix: an analog **reconstruction filter** (low-pass) after the DAC that smooths the steps and removes the images – the mirror image of the anti-alias filter before the ADC.

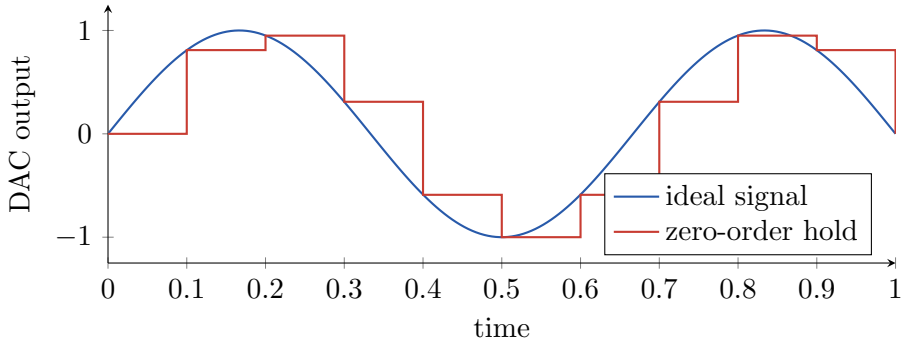


Figure 19.8: A DAC's zero-order-hold output (red) holds each sample until the next. The steps contain high-frequency images and a slight sinc droop; an analog reconstruction filter smooths them back toward the ideal curve (blue).

Connection

The ZOH droop is the sinc from Chapter 18 wearing a different hat. There, sampling in time put a sinc-shaped interpolation kernel between samples; here, holding in time multiplies the spectrum by a sinc. Same rectangle, same transform pair rect $\leftrightarrow$ sinc – just applied on the way out instead of the way in. Whenever you see a sinc, look for a rectangle nearby.

19.8 Reading a real sensor through an ADC

Enough theory; let us read a voltage. Everything above turns into a handful of practical numbers the moment you touch hardware.

Build it (hardware)

Arduino Uno (ATmega328P): 10-bit SAR ADC. The Uno has one SAR converter multiplexed across six pins. Key facts:

- Resolution $N = 10 \Rightarrow 1024$ codes. With the default $V_{\text{ref}} = 5$ V, $\Delta = 5/1024 \approx 4.9$ mV per code. You cannot resolve a change smaller than that.
- `analogRead()` returns $0 \ldots 1023$. Convert to volts with `v = raw * 5.0 / 1024.0`.
- Default conversion time is about 100 μs (∼9600 reads/s), set by the ADC clock prescaler. Plenty for an IMU at 100–1000 Hz; far too slow for audio.
- For quiet measurements use the internal 1.1 V reference (`analogReference(INTERNAL)`): now $\Delta \approx 1.07$ mV, about four times finer, at the cost of a smaller input range.

```
// Read a flex sensor / EMG envelope on A0, oversample to fake extra bits.
void setup() { Serial.begin(115200); }

void loop() {
  long acc = 0;
  const int OS = 16;              // average 16 reads
  for (int i = 0; i < OS; i++) acc += analogRead(A0);
  // OSR=16 -> +2 effective bits IF the signal jitters by >= 1 LSB
  float code12 = acc / 4.0;       // 10-bit*16 -> ~12-bit scale
  float volts  = code12 * (5.0 / 4096.0);
  Serial.println(volts, 4);
  delay(5);                       // ~200 Hz output
}
```

The averaging only buys real resolution if the input (plus its natural noise) wiggles by at least an LSB between reads – that is dithering happening for free. On a dead-quiet, perfectly stable voltage, averaging 16 identical reads gives you back the same number and no extra bits.

Build it (hardware)

Raspberry Pi: no analog pins at all. The Pi's GPIO is digital only. To read an analog sensor you add an external ADC over SPI or I^2C:

- **MCP3008**: 8-channel, 10-bit SAR, SPI, up to ∼200 kS/s. Cheap and ubiquitous; the standard "give my Pi an ADC" chip.
- **ADS1115**: 4-channel, 16-bit sigma-delta, I^2C, with a programmable-gain amplifier. Slower (up to 860 S/s) but far finer – a good match for an EMG envelope or any low-frequency, high-dynamic-range signal.

```
# Raspberry Pi + ADS1115 (16-bit sigma-delta) over I2C
import board, busio
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn

i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1115(i2c)
ads.gain = 1                      # +/-4.096 V full scale
```

```
chan = AnalogIn(ads, ADS.P0)      # single-ended on A0

while True:
    print(f"{chan.voltage:7.4f} V  (raw {chan.value})")
```

With ±4.096 V full scale and 16 bits, $\Delta \approx 8.192/65536 \approx 125\,\mu$V – about 40 times finer than the Uno's default. The sigma-delta front-end also gives you built-in anti-alias filtering, which matters for the low, clean signals (EMG/FMG envelopes) we want.

Figure 19.9 sketches why the converter choice bites here. An EMG envelope is a small, slow bump riding on a steady baseline; if the ADC's Δ is comparable to the bump's height, the staircase swallows the feature whole. A finer step traces the same bump smoothly.

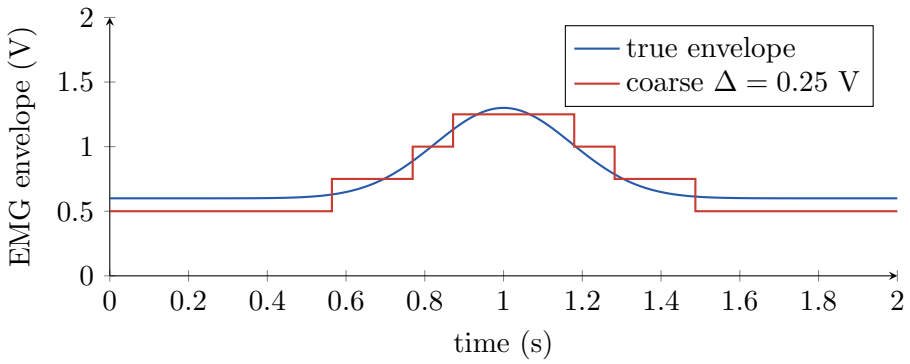


Figure 19.9: A slow EMG-style envelope (blue) and the same signal read by a converter whose step Δ is a large fraction of the feature height (red staircase). Coarse quantization smears the activation into a few plateaus; a 16-bit front-end with a small Δ keeps the shape.

Toward the wrist tracker

For the wrist controller, match the converter to the signal. The **IMU** (accelerometer + gyro, Chapter 34) already contains its own high-quality sigma-delta ADCs and hands you clean multi-bit samples over I^{2}C/SPI – you never touch its quantizer, but its ENOB and noise density (specified in $\mu g/\sqrt{\text{Hz}}$) set the floor on how small a wrist tilt you can detect. The **muscle signals** that hint at finger intent (EMG/FMG, Chapter 37) are tiny, low-frequency, and ride on a large slow baseline – a textbook case for a 16-bit sigma-delta front-end like the ADS1115 with a PGA, where dithering and oversampling genuinely buy you resolution. And keep a clear eye on what bits can and cannot do: a finer ADC reads the forearm's coarse muscle activity more precisely, but *no* bit depth lets a wristband recover continuous individual finger-joint angles – that information is not present in the signal the wristband sees (Chapter 37 makes this limit precise). Quantization sets how cleanly you capture what is there; it cannot conjure what is not.

History

The $\Delta^2/12$ noise model and the additive-noise view of quantization were put on firm footing by Bernard Widrow in the 1950s and 60s; his quantization theorem gives precise conditions

(roughly, a band-limited error spectrum) under which the uniform-noise model is *exactly* right. Dither has an even more colorful origin: WWII-era mechanical analog computers worked more accurately while vibrating in flying bombers than sitting still on the ground – the vibration was unintentional dither, shaking the gears past their stiction. Engineers later added motors to vibrate the computers on purpose. Sigma-delta modulation was introduced by Inose, Yasuda, and Murata in 1962 and became the dominant audio-ADC architecture once cheap, fast digital filtering made the decimation stage practical.

19.9 Summary

Quantization is the vertical twin of sampling. An N-bit ADC snaps each sample to the nearest of 2^N rungs spaced $\Delta = V_{\text{ref}}/2^N$ apart; the error is bounded by $\pm\Delta/2$ and, for busy signals, behaves like uniform noise of power $\Delta^2/12$. Comparing that fixed noise to a full-scale sinusoid gives the master rule SNR $\approx 6.02N + 1.76$ dB – about a bit per 6 dB – and inverting it on a real converter gives its ENOB. SAR converters do a fast hardware binary search for moderate-resolution, low-latency, multi-channel sensing; sigma-delta converters oversample a crude quantizer, shape the noise out of band, and decimate, reaching very high resolution for one continuous channel. Oversampling alone buys ≈ 3 dB per doubling of f_s; noise shaping does much better; and *adding* noise (dither) converts ugly signal-correlated distortion into benign hiss you can average away. On the way back out, a DAC holds each sample (zero-order hold), which droops the top of the band and creates spectral images that a reconstruction filter cleans up. At the bench, the Arduino gives you a 10-bit SAR for free; the Pi needs an external ADC, where a 16-bit sigma-delta part is the right tool for small, slow biosignals – always remembering that resolution captures what the signal contains and never more.

19.10 Exercises

Exercise 19.1. An ADC has $V_{\text{ref}} = 3.3$ V and $N = 12$ bits. (a) Compute the LSB size Δ in millivolts. (b) What is the maximum quantization error in millivolts? (c) What is the ideal SNR in dB for a full-scale sinusoid?

Exercise 19.2. Derive the quantization noise power $\sigma_e^2 = \Delta^2/12$ from scratch by integrating e^2 against a uniform density on $[-\Delta/2, \Delta/2]$. Then state in one sentence why the answer is independent of where the signal sits on the ladder.

Exercise 19.3. You record audio at 16 bits but, through a gain mistake, your loudest peaks only reach 1/8 of full scale. (a) By how many dB has your signal power dropped relative to full scale? (b) Estimate your effective number of bits. (c) State the practical lesson.

Exercise 19.4. A SAR ADC has $N = 8$ bits and $V_{\text{ref}} = 4.096$ V (so $\Delta = 16$ mV). Trace the binary search by hand for an input of 2.7 V: list each trial voltage, each comparator decision, and the final 8-bit code. What voltage does that code represent, and what is the residual error?

Exercise 19.5. You oversample a signal by OSR $= 64$ (no noise shaping) and then ideally low-pass filter to the signal band. (a) How many dB of in-band SNR improvement do you gain? (b) How many effective bits is that? (c) Why does this require the quantization error to be uncorrelated across samples, and what tool guarantees that?

Exercise 19.6. Explain, in your own words and with a small numeric example, how dithering lets the long-run *average* of quantized codes represent a value that lies strictly between two rungs. Use a true value of 0.4 LSB above a rung and predict the fraction of samples that round up.

Exercise 19.7. A DAC runs at $f_s = 48$ kHz with a zero-order hold. (a) Write the magnitude of the ZOH frequency response. (b) Compute the droop in dB at $f = 20$ kHz relative to DC. (c) Describe the two analog problems the ZOH creates and the filter that fixes each.

Exercise 19.8. Modify the Arduino sketch in Section 19.8 to switch to the internal 1.1 V reference. (a) What is the new Δ? (b) Under what condition on your sensor's output range is this a good idea, and when would it clip? (c) Why does oversampling-and-averaging fail to add bits on a perfectly stable, noise-free input, and what is the one-sentence connection to dither?

Exercise 19.9. A "16-bit" ADC datasheet quotes a measured SNR of 91 dB. (a) Compute its ENOB. (b) The same part quoted with a $4\times$ slower (more averaged) output mode reports 97 dB; compute the new ENOB and explain physically where the extra bits came from. (c) Why can no amount of averaging push the ENOB past the analog noise floor of the front-end?

Exercise 19.10. For the wrist controller, you must choose an ADC for an EMG envelope signal that spans roughly 0–2 V, changes slowly (under 50 Hz of useful bandwidth), and must resolve features as small as a few hundred microvolts. (a) Would you pick a 10-bit SAR at 200 kS/s or a 16-bit sigma-delta at 860 S/s? Justify using Δ and the bandwidth. (b) State one thing this ADC choice still *cannot* tell you about the user's individual fingers, and why.

Chapter 20

Noise, SNR, and Random Signals

Up to this chapter we have treated signals as if they were exact. A sinusoid was *the* sinusoid; a sampled waveform was a clean list of numbers we could push through a filter and trust. Reality is messier. Plug a microphone in and listen to "silence" — you will hear a hiss. Wire an accelerometer to a microcontroller, hold it perfectly still, and the readings still jitter in the last few bits. Press two electrodes to your forearm to pick up the tiny voltages of a muscle and you will find the muscle's signal half-buried under hum, drift, and crackle. Every real measurement arrives as

$$\text{what you wanted} \;+\; \text{what you did not.}$$

That second term is *noise*, and learning to think about it clearly is not a sad footnote to signal processing — it is half the subject. The reason the Fourier transform, filtering, and averaging are worth so much is precisely that they let us pull a wanted signal out of an unwanted swamp.

The trouble is that noise is, by its nature, not a fixed waveform. You cannot write down "the" noise the way you write down a sine. Run the same experiment twice and you get two different hiss patterns. So we need a new kind of object: a *random signal*, a signal we describe not by its exact values but by its *statistics* — how big it tends to be, how fast it wiggles, how its value now relates to its value a moment ago. This is where the companion probability book pays off directly. Expectation, variance, and correlation, which there were abstract operations on random variables, become here concrete, audible, plottable properties of signals. By the end of the chapter you will be able to say what white noise *is*, why averaging cleans up a measurement and exactly how much, why a matched filter is the optimal detector, and what kinds of noise a wrist-worn IMU and forearm EMG actually fight.

20.1 Signal plus noise: the basic picture

The workhorse model for almost everything in this chapter is the simplest one imaginable. The measured signal $x[n]$ is the true signal $s[n]$ we care about, plus an additive noise term $w[n]$:

$$x[n] \;=\; s[n] \;+\; w[n]. \tag{20.1}$$

"Additive" means the noise just adds on top; it does not, say, multiply the signal or warp time. "Additive" is not always exactly right — quantization error from Chapter 19 is signal-dependent, and some sensor noise scales with the reading — but it is right often enough, and simple enough,

that it is the place to start. Two more assumptions usually ride along: the noise has *zero mean* (it is just as likely to push the reading up as down), and it is *independent* of the signal (the muscle twitching does not change the thermal hiss in the wire).

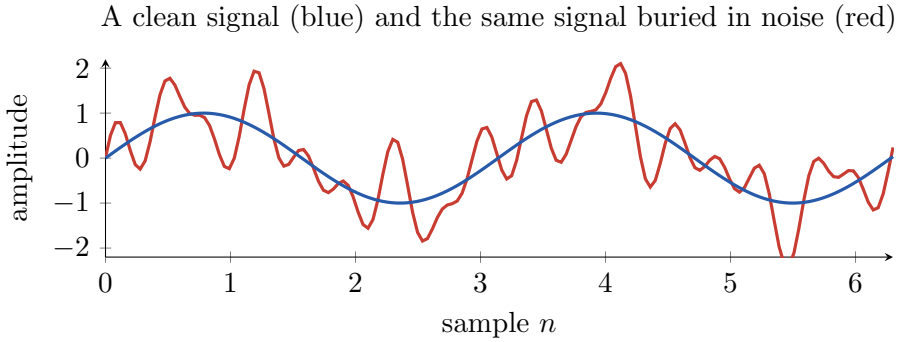


Figure 20.1: The same underlying slow tone s shown clean (blue) and after a noisy channel adds a hash of high-frequency junk (red). The eye still finds the slow swell; that is a hint that frequency separation will help us.

Intuition

Why is additive zero-mean noise such a friendly enemy? Because both of its defining features point to a cure. "Additive" means the noise rides *alongside* the signal rather than mangling it, so anything linear we do to x — a filter, a sum, an average — acts on s and w separately. "Zero mean" means the noise has nowhere to hide on average: pile up enough independent copies and the positive and negative excursions cancel, while the signal, repeating itself, reinforces. Almost every noise-fighting trick in this chapter is one of those two observations cashed in.

20.1.1 We cannot write down the noise, so we describe its habits

Here is the conceptual jump. Suppose I record the noise on my EMG amplifier for one second, twice in a row, with the muscle relaxed. The two recordings are different number-for-number — yet they "look the same": same typical loudness, same buzzy texture, same lack of any slow trend. What is stable across runs is not the values but their *statistics*. A random signal (the proper name is a *stochastic process*) is best thought of as a whole *ensemble* of possible waveforms, one of which nature hands you on any given day. We describe the ensemble, not the single draw.

Connection

This is exactly the random-variable viewpoint from the probability book, with a clock attached. There, a random variable W had a distribution; we summarized it by its mean $\mathbb{E}[W]$ and variance $\mathrm{Var}(W)$. Here we have a random variable $w[n]$ *at every time index* n, an entire indexed family of them. The new ingredient — the thing time buys us — is that we can also ask how $w[n]$ relates to $w[m]$ at a *different* time. That relationship, the autocorrelation, is the genuinely new statistic of this chapter, and it is what the power spectrum will turn out to be.

20.2 Mean, power, and variance of a random signal

Two numbers describe "where a random signal sits" and "how much it wiggles." The *mean* is the ensemble average at each time,

$$\mu_x[n] \;=\; \mathbb{E}\big[x[n]\big], \tag{20.2}$$

where the expectation $\mathbb{E}[\cdot]$ averages over the ensemble — imagine running the experiment infinitely many times and averaging the value you get at index n. For our additive model with zero-mean noise, $\mu_x[n] = s[n]$: the mean *is* the clean signal. That is already a profound statement. It says that if we could only average over enough independent runs, the noise would melt away and leave s behind. Section 20.7 makes that quantitative.

The second number is *average power*. For a deterministic discrete signal, power is the mean of the squared values. For a random signal it is the *expected* squared value,

$$P_x \;=\; \mathbb{E}\big[x[n]^2\big]. \tag{20.3}$$

The names come from circuits: a voltage v across a $1\,\Omega$ resistor dissipates v^2 watts, so $\mathbb{E}[v^2]$ is literally average power. We carry the word over to every kind of signal. When the mean is zero, power and *variance* coincide,

$$\mathrm{Var}(x[n]) = \mathbb{E}\big[(x[n]-\mu_x[n])^2\big] = \mathbb{E}[x[n]^2] = P_x,$$

and we usually write the noise power as $\sigma_w^2 = \mathbb{E}[w[n]^2]$. Because the signal and noise are independent and the noise is zero-mean, their powers *add*:

$$P_x \;=\; \underbrace{\mathbb{E}[s[n]^2]}_{P_s} \;+\; \underbrace{\mathbb{E}[w[n]^2]}_{\sigma_w^2}. \tag{20.4}$$

The cross term $2\,\mathbb{E}[s[n]\,w[n]]$ vanishes because s is fixed and w has zero mean (or, if s is itself random, because they are independent). Powers adding, not amplitudes, is the single most useful bookkeeping fact about noise.

Why it works

Why do *powers* add rather than amplitudes? Geometry. Think of the signal and the noise as two vectors. Independence with zero mean makes them *orthogonal* on average: their inner product $\mathbb{E}[s\,w]$ is zero. And for orthogonal vectors the Pythagorean theorem holds — the squared length of the sum is the sum of the squared lengths. "Power" is squared length, so $P_x = P_s + \sigma_w^2$ is just Pythagoras in the space of random signals. This is the same orthogonality that ran through the linear-algebra book; noise simply gives it a probabilistic flavor.

20.2.1 Stationarity and ergodicity: when one recording is enough

There is a practical worry lurking. The mean and power above are *ensemble* averages — averages over many parallel universes of the experiment — but in the lab you usually have *one* recording. When can a *time* average over your single record stand in for the ensemble average you cannot perform?

Two assumptions rescue us. A process is (wide-sense) *stationary* if its statistics do not drift with time: the mean is a constant μ_x, and the power and correlations depend only on time *differences*,

not on absolute time. A stationary process is *ergodic* if, in addition, a long enough time average along one record converges to the ensemble average. For ergodic stationary noise,

$$\frac{1}{N}\sum_{n=0}^{N-1} x[n] \xrightarrow[N\to\infty]{} \mu_x, \qquad \frac{1}{N}\sum_{n=0}^{N-1} x[n]^2 \xrightarrow[N\to\infty]{} P_x.$$

Common trap

Ergodicity is an assumption, not a guarantee, and real sensors violate it constantly. An IMU warming up drifts (its mean moves — not stationary). EMG during a contraction is louder than at rest (its power changes with time — not stationary). Treating a non-stationary record as if a single long time average described it is a classic way to compute a confident, wrong number. The safe move is to work on short windows over which stationarity is plausible — which is exactly why the short-time Fourier transform of Chapter 10 exists.

20.3 Autocorrelation: how a signal relates to its own past

Mean and power tell you the size of a random signal but nothing about its *texture*. Two noises can have identical power yet feel completely different: one a smooth low rumble, the other a sharp hiss. The difference is in how each sample relates to its neighbors. A rumble changes slowly, so $x[n]$ and $x[n+1]$ are nearly equal — strongly correlated. A hiss jumps around, so neighbors are nearly unrelated. The statistic that captures this is the *autocorrelation*: the correlation of the signal with a time-shifted copy of itself.

For a zero-mean wide-sense-stationary process, define the autocorrelation at lag k as

$$R_x[k] \;=\; \mathbb{E}\big[\, x[n]\, x[n+k] \,\big]. \tag{20.5}$$

It depends only on the lag k, not on n (that is what stationarity buys). A few properties read straight off the definition and are worth committing to intuition:

- $R_x[0] = \mathbb{E}[x[n]^2] = P_x$. *The autocorrelation at zero lag is the power.* A signal is always perfectly correlated with itself.
- $R_x[-k] = R_x[k]$: the autocorrelation is symmetric. Looking back and looking forward by the same amount give the same correlation.
- $|R_x[k]| \le R_x[0]$: the peak is always at lag zero. Nothing correlates with the signal better than the signal itself.
- How fast $R_x[k]$ *decays* as k grows measures how quickly the signal forgets its past. Slow decay $\Rightarrow$ smooth, low-frequency signal; fast decay $\Rightarrow$ rough, high-frequency signal.

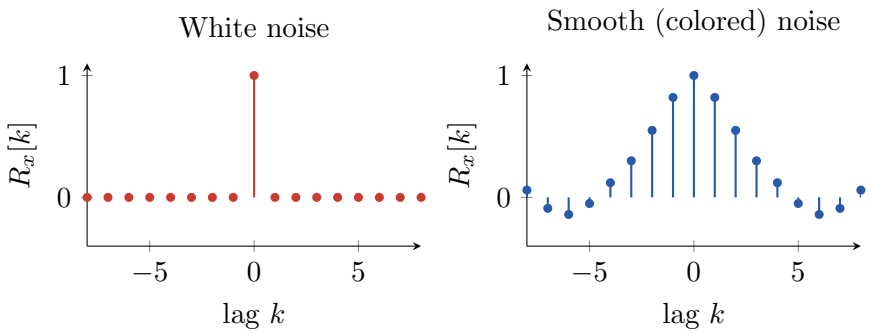


Figure 20.2: Autocorrelation is the fingerprint of texture. White noise (left) is a single spike at lag 0: each sample is uncorrelated with every other. Smooth colored noise (right) has a broad hump — neighboring samples are similar, so the signal remembers its past for several lags.

What is really happening

Autocorrelation is just an inner product with a sliding copy of yourself. Recall from Chapter 3 that the inner product $\langle a, b\rangle$ measures how much two vectors point the same way. $R_x[k]$ is $\langle x[n], x[n+k]\rangle$ averaged over the ensemble: it asks "after I slide the signal by k, how much does it still line up with the original?" At $k = 0$ it lines up perfectly (that is the power). As the shift grows, a rough signal falls out of step quickly and a smooth one stays aligned. The whole curve is a self-portrait of the signal's memory.

Example 20.1 (Autocorrelation of a noisy sinusoid)**.** Let $x[n] = A\cos(\omega_0 n + \phi) + w[n]$, where ϕ is a random phase uniform on $[0, 2\pi)$ and w is independent zero-mean white noise with power σ_w^2. Using $\mathbb{E}[\cos\alpha\cos\beta] = \frac{1}{2}\cos(\alpha - \beta)$ after averaging over ϕ,

$$R_x[k] \;=\; \frac{A^2}{2}\cos(\omega_0 k) \;+\; \sigma_w^2\,\delta[k].$$

The sinusoid contributes a cosine in the lag k that *never dies* — a periodic signal remembers its phase forever — while the noise contributes a single spike at $k = 0$. This is the key to finding a periodic signal in noise: at nonzero lags the noise has averaged away and only the cosine survives. Pitch detectors and heart-rate monitors exploit exactly this.

Try it yourself

Generate `x = np.sin(0.3*np.arange(2000)) + np.random.randn(2000)` and plot `np.correlate(x, x, 'full')` near the center. Even though the sinusoid is invisible in the raw trace (the noise is as big as the signal), the autocorrelation shows clean ripples at the signal's period. You are watching the noise spike collapse to lag 0 while the periodic memory stands proud.

20.4 Power spectral density: where the power lives in frequency

We now have two ways to describe a random signal's structure: its autocorrelation $R_x[k]$ in the lag domain, and — waiting in the wings — its spectrum in the frequency domain. The bridge between

them is one of the most important results in all of signal processing, and it is exactly the kind of time/frequency duality this book keeps returning to.

The *power spectral density* (PSD) $S_x(e^{\mathrm{j}\omega})$ of a stationary random signal tells you how much of the signal's power sits at each frequency. You cannot take an ordinary Fourier transform of a noise waveform — it never settles down, so the transform does not converge. But you *can* transform its autocorrelation, which is a nice decaying deterministic sequence. That is the content of the Wiener–Khinchin theorem.

Formal

Wiener–Khinchin theorem. For a wide-sense-stationary process, the power spectral density is the discrete-time Fourier transform of the autocorrelation:

$$S_x(e^{\mathrm{j}\omega}) \;=\; \sum_{k=-\infty}^{\infty} R_x[k]\, e^{-\mathrm{j}\omega k}, \qquad R_x[k] \;=\; \frac{1}{2\pi}\int_{-\pi}^{\pi} S_x(e^{\mathrm{j}\omega})\, e^{\mathrm{j}\omega k}\,\mathrm{d}\omega. \tag{20.6}$$

Setting $k = 0$ in the inverse relation gives the total power as the area under the PSD:

$$P_x = R_x[0] = \frac{1}{2\pi}\int_{-\pi}^{\pi} S_x(e^{\mathrm{j}\omega})\,\mathrm{d}\omega.$$

Why it works

Why should the spectrum of power be the transform of the autocorrelation, of all things? Think about what each frequency component contributes. A signal that oscillates strongly at frequency ω_0 has an autocorrelation that itself oscillates at ω_0 (Example 20.1). The Fourier transform's whole job is to detect "how much oscillation at each frequency" is present in a sequence. Apply it to $R_x[k]$ and it reports how much oscillation the signal's memory contains at each ω — which is precisely how the power is distributed across frequency. The transform of the self-similarity is the spectrum of the energy. Wiener and Khinchin made this airtight, but the picture is that simple.

Connection

The probability book defined the variance of a sum of independent contributions as the sum of variances. The PSD is that statement made spectral: it splits the total variance (power) into a density across frequency, $\mathrm{Var} = \int S_x\,\mathrm{d}f$. Each frequency band is an independent "bucket" of variance, and they add. When later you read that a filter "shapes the noise spectrum," it means it reweights those buckets before they are added back up.

20.5 White noise and colored noise

We can now define the two most important species of noise by the shape of their spectra — and the names are not metaphors but a direct analogy with light.

White noise has a *flat* PSD: equal power at every frequency, just as white light contains every color of the visible spectrum equally. Flat in frequency means, by Wiener–Khinchin, an autocorrelation

that is a single spike,

$$S_w(e^{j\omega}) = \sigma_w^2 \ \text{(constant)} \quad \Longleftrightarrow \quad R_w[k] = \sigma_w^2\,\delta[k].$$

A spike at lag 0 and zero elsewhere says: every sample is uncorrelated with every other sample. White noise has no memory at all. It is the most "random" a signal can be, and it is the standard idealized model for thermal (Johnson) noise in resistors and shot noise in electronics.

Common trap

"White" says nothing about the *distribution* of the values — only that samples are uncorrelated. White Gaussian noise (samples drawn from a bell curve) is the famous case, but you can have white noise whose samples are ± 1 coin flips. Conversely, "Gaussian" says nothing about correlation; you can have Gaussian noise that is heavily colored. Whiteness is about *the spectrum*; Gaussianity is about *the histogram*. Keep the two axes separate.

Colored noise has a PSD that is not flat: its power is concentrated in some bands more than others, so successive samples *are* correlated. "Coloring" white noise just means filtering it (Section 20.5.1). Common colors, named again after light:

- *Pink noise* ($S(f) \propto 1/f$): more power at low frequencies, falling 3 dB per octave. Ubiquitous in nature — it is the spectrum of the rumble of traffic, the flicker noise in electronics, and slow sensor drift.
- *Brown / red noise* ($S(f) \propto 1/f^2$): even more low-frequency dominance; the spectrum of a random walk. This is what integrating white noise produces — and integrating a gyroscope's white rate noise is *exactly* how orientation estimates drift.
- *Blue / violet noise*: power rising with frequency, the spectrum you get from *differentiating* a signal. Numerical differentiation of a noisy sensor amplifies high-frequency noise for this reason.

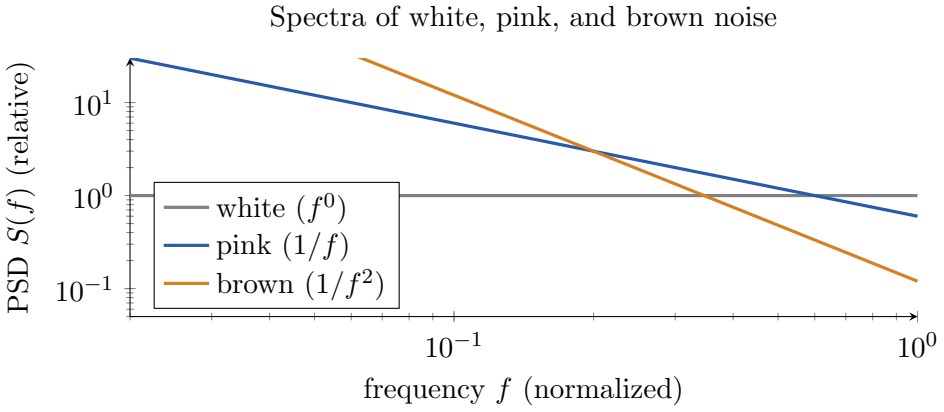


Figure 20.3: Noise "colors" on log-log axes. White is flat. Pink falls as $1/f$ (a straight line of slope -1 here). Brown falls as $1/f^2$ (slope -2), piling almost all of its power into the lowest frequencies — which is why random-walk drift dominates the long-term behavior of integrated sensors.

History

The "white" metaphor traces to the recognition, in the 1920s, that thermal agitation in a conductor produces a voltage with an essentially flat spectrum — John B. Johnson measured it at Bell Labs in 1928 and Harry Nyquist explained it that same year from thermodynamics, giving the formula $S_v = 4k_BTR$ for the noise power spectral density of a resistor R at temperature T. Norbert Wiener (1930) and Aleksandr Khinchin (1934) independently established the autocorrelation–spectrum link that lets us even speak of the "spectrum" of a signal that never repeats. Together these results turned noise from a nuisance into a quantity you can compute, predict, and design against.

20.5.1 Coloring noise is just filtering it

Pass white noise with power σ_w^2 through an LTI filter with frequency response $H(e^{\mathrm{j}\omega})$. What comes out? Because filtering multiplies each frequency component by H, and power is amplitude squared, the output PSD is the input PSD scaled by $|H|^2$:

$$S_y(e^{\mathrm{j}\omega}) \;=\; |H(e^{\mathrm{j}\omega})|^2\, S_w(e^{\mathrm{j}\omega}) \;=\; |H(e^{\mathrm{j}\omega})|^2\, \sigma_w^2. \tag{20.7}$$

This single formula is the engine behind every "colored" noise: choose the filter shape and you choose the color. A one-pole low-pass filter applied to white noise yields pink-ish noise; an integrator ($H \approx 1/(1-z^{-1})$) yields brown noise; a differencer yields blue noise. It also tells you exactly how a filter changes the noise power: integrate $|H|^2\sigma_w^2$ over frequency. A filter that passes only a tenth of the band lets through only a tenth of white-noise power — the basis of using a band-pass filter to clean up a narrowband measurement.

Example 20.2 (Noise out of a moving-average filter)**.** The simplest smoother, the L-point moving average $y[n] = \frac{1}{L}\sum_{i=0}^{L-1} x[n-i]$, fed white noise of power σ_w^2, produces output power

$$\sigma_y^2 = \frac{1}{L^2}\sum_{i=0}^{L-1}\sigma_w^2 = \frac{\sigma_w^2}{L}.$$

The noise power drops by a factor of L (the amplitude by $\sqrt{L}$). The filter has thrown away the high-frequency part of the white spectrum, keeping only a low-pass slice. We are about to see this same $1/L$ law from the averaging side — they are the same fact.

20.6 Signal-to-noise ratio (SNR)

We need one number to say "how clean is this measurement?" That number is the *signal-to-noise ratio*, the ratio of signal power to noise power:

$$\mathrm{SNR} \;=\; \frac{P_s}{\sigma_w^2} \;=\; \frac{\mathbb{E}[s[n]^2]}{\mathbb{E}[w[n]^2]}. \tag{20.8}$$

Because powers can span enormous ranges, SNR is almost always quoted in *decibels*:

$$\mathrm{SNR}_{\mathrm{dB}} \;=\; 10\log_{10}\frac{P_s}{\sigma_w^2} \;=\; 20\log_{10}\frac{A_s}{A_w}, \tag{20.9}$$

where A_s, A_w are RMS amplitudes (the factor changes from 10 to 20 because power is amplitude squared, and log of a square is twice the log). A few anchor points worth memorizing: SNR = 1 means 0 dB (signal and noise equally loud); every factor of 10 in power is +10 dB; every doubling of *amplitude* is about +6 dB; a factor of 2 in power is about +3 dB.

Intuition

Decibels feel fussy until you notice they turn the two operations you care about into addition. Cascading two stages *multiplies* their power gains, which *adds* their dB values. And the human ear and eye both respond roughly logarithmically, so dB also matches perception: a steady +10 dB feels like "a notch louder" whether you start from a whisper or a shout. SNR in dB is the natural currency for talking about how much headroom a measurement has.

Example 20.3 (Reading SNR off a recording)**.** Suppose a relaxed-muscle EMG baseline has RMS 0.01 mV and a contraction produces RMS 0.20 mV. The signal (the part above baseline) sits at roughly 0.20 mV against 0.01 mV of noise, so

$$\mathrm{SNR}_{\mathrm{dB}} = 20\log_{10}\frac{0.20}{0.01} = 20\log_{10}(20) \approx 26 \text{ dB}.$$

That is a comfortably detectable contraction. At 0 dB (noise as loud as signal) you would struggle; below that, you need the averaging and filtering of the next sections, or a better electrode.

Common trap

SNR is meaningless until you say *over what bandwidth* and *for what signal*. White noise has power spread over the whole band, so the noise that actually hurts you is only the part inside your signal's band. Quoting an SNR without naming the band (or the filter that defines it) is like quoting a speed without a direction. The right SNR for detection is computed *after* the processing that isolates the signal — which is precisely why filtering and matched filtering raise the SNR you get to use.

20.7 Averaging to beat down noise

Here is the most-used noise weapon in all of measurement, and it follows directly from "powers add" plus "the mean is the clean signal." Suppose you can record the *same* signal $s[n]$ many times — M repetitions, each with independent noise:

$$x_m[n] = s[n] + w_m[n], \qquad m = 1, \dots, M.$$

Average the recordings sample by sample:

$$\bar{x}[n] = \frac{1}{M}\sum_{m=1}^{M} x_m[n] = s[n] + \frac{1}{M}\sum_{m=1}^{M} w_m[n].$$

The signal term is untouched — it is identical in every run, so averaging M copies of $s[n]$ just gives $s[n]$ back. The noise term is an average of M *independent* zero-mean noises. Its power is

$$\mathrm{Var}\left(\frac{1}{M}\sum_{m} w_m[n]\right) = \frac{1}{M^2}\sum_{m}\mathrm{Var}(w_m[n]) = \frac{1}{M^2}\cdot M\sigma_w^2 = \frac{\sigma_w^2}{M}.$$

The noise power falls by a factor of M; the noise *amplitude* (its RMS) falls by $\sqrt{M}$. And so:

$$\mathrm{SNR}_{\bar{x}} = M \cdot \mathrm{SNR}_x, \qquad \mathrm{SNR}_{\mathrm{dB}} \text{ improves by } 10\log_{10} M. \tag{20.10}$$

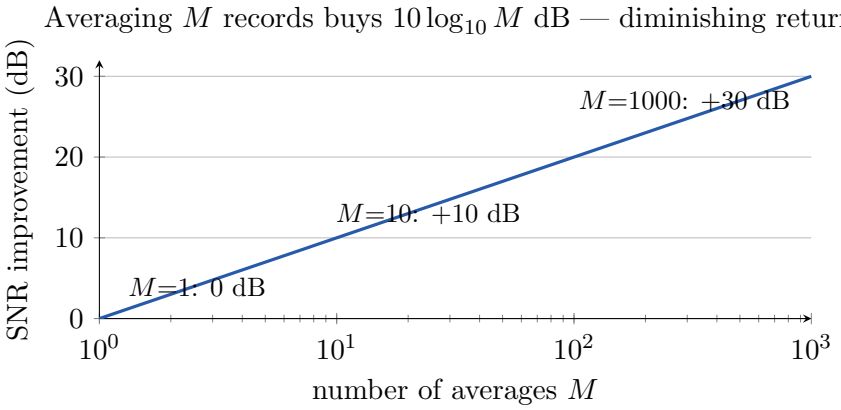


Figure 20.4: The $\sqrt{M}$ law has a sting in its tail. The first ten averages buy a full 10 dB, but to gain the *next* 10 dB you need ten times as many averages again. Quadrupling your averaging time only halves the noise. Beyond a point, a quieter sensor beats more averaging.

What is really happening

Why does the signal survive averaging while the noise dies? Because they are *coherent* and *incoherent*, respectively. The signal is identical in every run, so its copies add "in phase" — M copies of amplitude A give amplitude MA, i.e. power grows as M^2. The noise is independent run to run, so its copies add "out of phase" like a random walk — M steps give a typical distance $\sqrt{M}$, i.e. power grows only as M. Divide both by M (the averaging) and the signal sits at constant amplitude while the noise shrinks as $1/\sqrt{M}$. Coherent gain beats incoherent gain; that gap *is* the SNR improvement.

Common trap

Averaging only works when the signal is genuinely repeatable and the runs can be *aligned*. If the signal jitters in time between runs (different trigger delays), naive averaging smears and attenuates it — you average away the very thing you wanted. This is why evoked-potential and radar systems trigger so carefully, and why gesture pipelines time-align repetitions before averaging templates. Also: the $1/M$ law assumes *independent* noise. Correlated noise (a shared 50/60 Hz hum on every run) does not average down at all — it is coherent too. Averaging fights random noise, not interference.

20.7.1 Two flavors: ensemble averaging vs. smoothing

The averaging above was *across repetitions* (ensemble averaging). When you cannot repeat the experiment, you can instead average *across time* within one record — a moving average, which is just a low-pass filter. The σ_w^2/L result of Example 20.2 is the same $1/M$ law in disguise, with $M = L$ neighboring samples playing the role of repetitions. The price is different though: ensemble averaging

keeps the signal intact, while time smoothing also low-passes the signal, blurring any fast features it had. There is no free lunch — you trade noise for bandwidth.

Example 20.4 (How long to average?)**.** You measure a static accelerometer offset buried in white noise of RMS 5 mg and you want the estimate good to 0.5 mg RMS. You need the noise to fall by a factor of 10 in amplitude, hence by 100 in power, so $M = 100$ samples. At 100 Hz output rate that is one second of stillness. Want 0.05 mg? That is $\times 100$ in amplitude, $M = 10{,}000$ samples, 100 seconds — and by then the bias has probably drifted (non-stationarity), so more averaging stops helping. The math tells you when to stop chasing it.

20.8 Matched filtering: the optimal detector

Averaging assumed you could repeat the experiment. Often you cannot — you get one noisy record and must decide *whether and where* a known signal shape appears in it: a radar pulse echoing back, a sync word in a data packet, a particular flick of the wrist in an IMU stream. The optimal tool is the *matched filter*, and its logic is pure geometry.

Suppose the known signal shape (the "template") is $s[n]$, of length L, and your record is $x[n] = s[n - n_0] + w[n]$ with white noise — the template is hiding at unknown position n_0. To test position m, slide the template there and take the inner product with the data:

$$y[m] \;=\; \sum_{n} s[n-m]\, x[n] \;=\; (x * \tilde{s})[m], \tag{20.11}$$

which is a correlation — equivalently a convolution with the *time-reversed* template $\tilde{s}[n] = s[-n]$ (Figure 20.6). The output $y[m]$ peaks sharply when the template lines up with the buried copy.

Why it works

Why is correlating with the template the *best* possible thing to do? Cast it as vectors. The data over a window is the signal vector $\mathbf{s}$ plus a noise vector $\mathbf{w}$. A linear detector is an inner product $\langle \mathbf{h}, \mathbf{x} \rangle$ with some weight vector $\mathbf{h}$. The signal part of the output is $\langle \mathbf{h}, \mathbf{s} \rangle$ and the noise part has power $\sigma_w^2 \, \|\mathbf{h}\|^2$ (white noise). We want to maximize output SNR, $\langle \mathbf{h}, \mathbf{s} \rangle^2 / (\sigma_w^2 \|\mathbf{h}\|^2)$. By the Cauchy–Schwarz inequality, $\langle \mathbf{h}, \mathbf{s} \rangle^2 \le \|\mathbf{h}\|^2 \|\mathbf{s}\|^2$, with equality *only when* $\mathbf{h}$ *is parallel to* $\mathbf{s}$. So the optimal weights *are* the signal itself: $\mathbf{h} = \mathbf{s}$. The matched filter wins because it points the detector exactly along the signal direction, collecting all of the signal's energy while seeing noise from only that one direction.

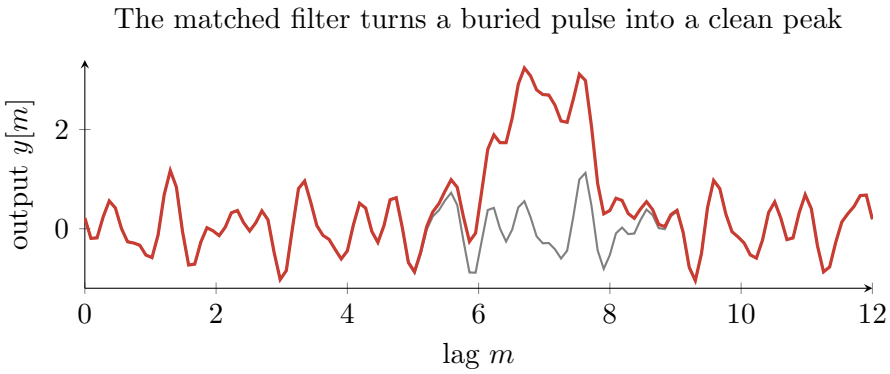


Figure 20.5: Output of a matched filter sliding along a noisy record (gray: the noise-only baseline). Where the template aligns with the hidden copy (here near $m = 7$) the correlation collects all the signal energy into one sharp peak (red) that stands far above the noise floor. Detection becomes "is there a peak above threshold?"

What is really happening

The peak height of a matched filter is the signal's *energy* $\sum_n s[n]^2$, while the noise at the output has power $\sigma_w^2 \sum_n s[n]^2$. So the output SNR is the total signal *energy* divided by the noise PSD — it depends only on how much energy the pulse carries, *not* on its shape. A long quiet chirp and a short loud spike with the same energy detect equally well. That is the deep reason radar uses long coded pulses: spread the energy out in time to stay within power limits, then let the matched filter squeeze it back into a sharp peak. The technique is called *pulse compression*.

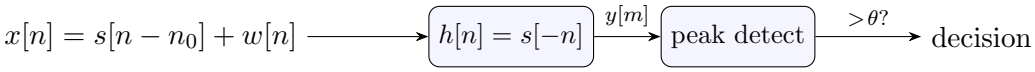


Figure 20.6: The matched-filter detector as a block diagram. Convolving the noisy record with the time-reversed template $h[n] = s[-n]$ correlates the data against the known shape; the output $y[m]$ is then compared to a threshold θ. Detection reduces to "did the correlation peak clear the threshold, and where?"

Connection

The matched filter is the convolution of Chapter 13 and the inner-product geometry of Chapter 3 meeting probability. Correlation *is* convolution with a reversed template; the Cauchy–Schwarz optimality *is* the orthogonality geometry of inner-product spaces; and "maximize SNR" *is* the expectation/variance machinery of the probability book. Three threads of this series tie together in one estimator.

20.9 Sensor noise models for the wrist tracker

Everything above becomes concrete the moment you wire up a real sensor. Here is a plain inventory of what an IMU and an EMG front end throw at you, in the language of this chapter.

20.9.1 IMU noise: white rate noise and the random walk it integrates to

A MEMS gyroscope and accelerometer each report a true value plus several noise terms. The two that matter most:

- **Noise density (white noise).** The instantaneous output has a roughly white, zero-mean fluctuation. Datasheets quote it as a *density*: e.g. a gyro at $0.01\,^\circ/\mathrm{s}/\sqrt{\mathrm{Hz}}$. The $\sqrt{\mathrm{Hz}}$ is Wiener–Khinchin in disguise: white power is spread per unit bandwidth, so the RMS you actually see is the density times $\sqrt{\text{bandwidth}}$. Halve your bandwidth (filter harder) and the white-noise RMS drops by $\sqrt{2}$.

- **Bias instability and random walk (colored noise).** The zero point wanders slowly. Integrating the gyro's white rate noise to get angle produces *angle random walk* — brown noise, $S(f) \propto 1/f^2$ — whose error grows as $\sqrt{t}$. A slowly drifting bias on top adds an even-lower-frequency wander. This is the unavoidable reason a gyro-only orientation estimate drifts and must be corrected by the accelerometer and magnetometer (the sensor fusion of Chapter 36).

Build it (hardware)

The standard tool for reading these terms off real data is the *Allan deviation*: log the IMU at rest for a few hours, then plot the deviation of the running average versus averaging time τ on log-log axes. A slope of $-1/2$ at short τ is the white noise (more averaging helps, the $1/\sqrt{M}$ law); the *minimum* of the curve is the bias instability (the floor where more averaging stops helping); a $+1/2$ slope at long τ is the random walk (drift dominating). The τ at the minimum tells you the longest you should ever average before recalibrating. A short Python sketch:

```
import numpy as np
def allan_dev(x, fs, taus):
    # x: stationary sensor record; fs: sample rate (Hz)
    out = []
    for tau in taus:
        m = int(tau * fs)                       # samples per averaging bin
        k = len(x) // m
        means = x[:k*m].reshape(k, m).mean(1)   # bin averages
        d = np.diff(means)
        out.append(np.sqrt(0.5 * np.mean(d**2)))  # Allan deviation
    return np.array(out)
```

Sample at the highest rate the sensor supports, then low-pass and decimate in software: averaging in the digital domain is free SNR, whereas noise picked up before the ADC is gone for good.

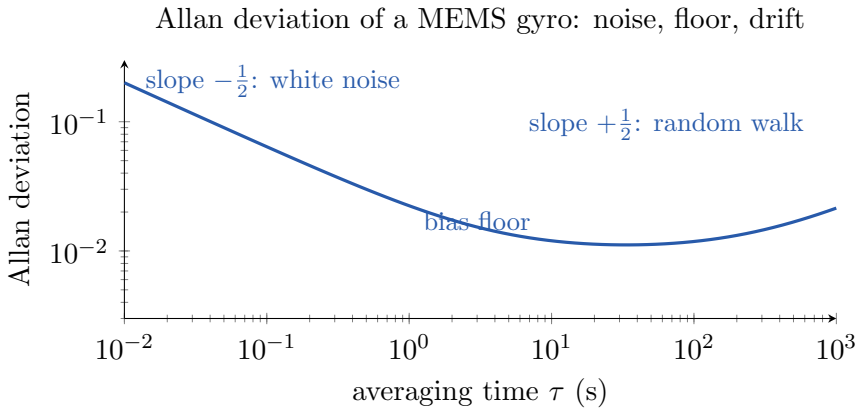


Figure 20.7: The classic Allan-deviation "bathtub." Short averaging is dominated by white noise that averages down; a flat minimum marks the bias-instability floor; long averaging is dominated by random-walk drift that averages *up*. The dip locates the sweet-spot averaging time and the irreducible noise floor of the part.

20.9.2 EMG noise: where the real signal lives, and what fights it

Surface EMG — the muscle voltages we will use to sense the hand — is a far harsher noise problem than the IMU, because the wanted signal is microvolts to low millivolts and the contaminants are large and structured (Figure 20.8):

- **Powerline interference** at 50/60 Hz and its harmonics, coupled capacitively from mains wiring. This is *not* random noise — it is a coherent sinusoid, so averaging does not touch it. You fight it with a notch filter and good differential electrodes.

- **Baseline wander / motion artifact**: low-frequency (< 10 Hz) drift from electrode-skin impedance changes as you move — colored, low-frequency, large. A high-pass filter around 20 Hz removes most of it (at the cost of the lowest EMG content).

- **Thermal and amplifier noise**: broadband, roughly white, set by the electrode impedance and the front-end op-amp. This *does* average down and sets the ultimate floor.

The useful EMG energy sits roughly in the 20–450 Hz band, so the standard front end is a band-pass ($\approx$ 20–450 Hz) plus a 50/60 Hz notch — literally Equation (20.7) put to work, shaping $|H|^2$ to pass the signal band and crush the interference bands.

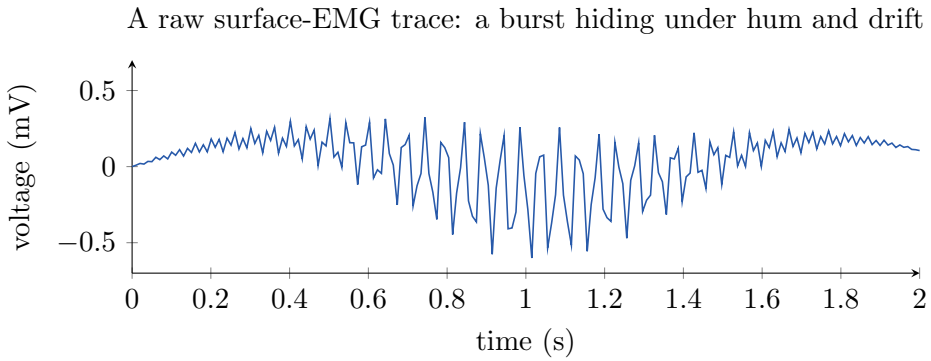


Figure 20.8: One second into the record the muscle fires (the fast burst near $t = 1$ s), but the raw trace is dominated by slow baseline wander and steady 50 Hz hum. A high-pass kills the wander, a notch kills the hum, and a band-pass keeps the burst — the front end of Equation (20.7) at work.

Toward the wrist tracker

For the wrist controller, this chapter sets the rules of the game. The IMU gives us clean, high-SNR *coarse* motion — a flick, a tilt, a tap — because those gestures live well above the noise floor and we can low-pass away the white jitter cheaply. But keep in view the limit stated back in the preface: an IMU on the wrist cannot recover individual finger joint angles. Finger intent shows up in the forearm *muscles*, and reading it means EMG (or force myography), with all the powerline, motion-artifact, and microvolt-floor problems above. So the noise budget drives the whole architecture: use the IMU for robust discrete gestures where SNR is generous, and reserve EMG — with aggressive band-pass + notch filtering and short-window RMS features — for the finger-level distinctions, accepting that even then we get coarse muscle-activation classes, not continuous joint angles. Continuous finger pose needs sensors *on* the fingers (instrumented rings/gloves). Designing the gesture set to live where the SNR is good is the single most important decision in the build.

Application

The same toolkit recurs across fields under different names. In audio, dithering deliberately adds white noise to decorrelate quantization error (Chapter 19); in astronomy, "stacking" frames is ensemble averaging that pulls faint galaxies out of sensor noise; in MRI and EEG, signal averaging over repeated stimuli recovers evoked responses at SNRs far below 0 dB; in communications, the matched filter is the heart of every receiver. Learn the four moves of this chapter — model the noise, measure its spectrum, average what is repeatable, filter/match what is not — and you can clean up a signal in any domain.

20.10 What to carry forward

Pause

The compressed version of this chapter:

- Real measurements are $x = s + w$. We describe w by statistics, not values: a random signal is an *ensemble*.
- Mean is the clean signal; *power* (= variance for zero mean) is the size of the wiggle. Independent signal and noise have *powers that add* — Pythagoras for random signals.
- *Autocorrelation* $R_x[k]$ measures memory; its Fourier transform is the *power spectral density* (Wiener–Khinchin). White noise = flat PSD = spike autocorrelation; colored noise = shaped PSD = filtered white.
- $SNR = P_s/\sigma_w^2$, quoted in dB; always say over what band.
- *Averaging* M repeatable records improves SNR by $10\log_{10} M$ dB ($1/\sqrt{M}$ in amplitude) — but only for independent noise and aligned signals.
- The *matched filter* (correlate with the known template) maximizes output SNR; its gain is the signal energy over the noise PSD, by Cauchy–Schwarz.
- IMU noise is mostly white rate noise plus random-walk drift (Allan deviation reveals the split); EMG fights powerline hum, motion artifact, and a microvolt floor. Know the limits: IMU → coarse gestures only; finger intent needs EMG/FMG; continuous finger pose needs finger-mounted sensors.

20.11 Exercises

Exercise 20.1. A signal has RMS amplitude 2.0 and is corrupted by independent zero-mean noise of RMS 0.5. (a) What is the SNR as a plain ratio? (b) In dB? (c) If you average 16 independent recordings, what is the new noise RMS and the new SNR in dB?

Exercise 20.2. Prove that for independent, zero-mean noise added to a deterministic signal, the cross term $\mathbb{E}[s[n]\,w[n]]$ in the power $\mathbb{E}[(s+w)^2]$ is zero, and hence $P_x = P_s + \sigma_w^2$. State exactly which assumption each step uses.

Exercise 20.3. White noise of power $\sigma_w^2 = 4$ is passed through a filter with $|H(e^{\mathrm{j}\omega})|^2 = 2$ for $|\omega| < \pi/4$ and 0 elsewhere. Using the area-under-the-PSD formula, compute the output power. (Hint: the input PSD is flat at σ_w^2; integrate $|H|^2\sigma_w^2$ over $[-\pi, \pi]$ and divide by 2π.)

Exercise 20.4. Show that the autocorrelation of white noise, $R_w[k] = \sigma_w^2\delta[k]$, has a *flat* PSD by plugging it into the Wiener–Khinchin sum (20.6). Then do the reverse: given a flat PSD $S(e^{\mathrm{j}\omega}) = c$, recover $R_w[k]$ via the inverse transform.

Exercise 20.5. A random walk is $y[n] = y[n-1] + w[n]$ with white w. (a) Show y is the output of an integrator $H(z) = 1/(1 - z^{-1})$ driven by white noise. (b) Argue from $|H(e^{\mathrm{j}\omega})|^2 = 1/|1 - e^{-\mathrm{j}\omega}|^2$ that its PSD behaves like $1/\omega^2$ near $\omega = 0$ — i.e. it is brown noise. (c) Connect this to why integrating gyro noise gives drift that grows as $\sqrt{t}$.

Exercise 20.6. You want to detect a known length-64 template buried in white noise. (a) Write the matched-filter output as a correlation. (b) If the template has unit energy and the noise PSD is σ_w^2, what is the output peak SNR? (c) If you halve the template's amplitude everywhere, by how many dB does the peak SNR drop?

Exercise 20.7. Explain, using the coherent/incoherent picture, why averaging M recordings does *not* reduce 50 Hz powerline interference that is present (in phase) on every recording, even though it reduces the thermal noise by $\sqrt{M}$. What operation *would* remove the powerline component, and why?

Exercise 20.8. A gyro datasheet lists noise density $0.014\,^\circ/\mathrm{s}/\sqrt{\mathrm{Hz}}$. (a) If you run it with a 50 Hz output bandwidth, estimate the white-noise RMS in $^\circ/\mathrm{s}$. (b) You apply a moving average of $L = 10$ samples at 200 Hz sampling. By what factor does the white-noise RMS drop, and what new effective bandwidth does that correspond to? (c) Name one noise term that this averaging will *not* help with.

Exercise 20.9. On an Allan-deviation log-log plot, identify which physical noise term produces each of: a $-1/2$ slope at small τ, a flat minimum, and a $+1/2$ slope at large τ. Explain in one sentence each why averaging helps, plateaus, then hurts.

Exercise 20.10. (Design.) You are choosing a gesture set for the wrist controller. Given that the IMU delivers high SNR for coarse motion but cannot resolve finger joints, and EMG delivers only coarse muscle-activation classes against heavy interference, propose a 4-gesture vocabulary and justify each choice by where its SNR is favorable. State explicitly one gesture you would *not* attempt with this hardware and why.

Chapter 21

Spectral Estimation

Up to now we have treated the spectrum as something we simply *compute*. Feed N samples to a DFT, out comes $X[k]$, done. That picture holds only for one very special kind of signal: a clean, deterministic, perfectly periodic waveform that we happened to capture for a whole number of periods. Real signals are not like that. Your microphone picks up a singer plus room rumble plus thermal hiss. Your wrist IMU picks up a gesture plus muscle tremor plus the bus you are riding on. These signals are partly *random*, they go on far longer than any block you can grab, and the part you care about is buried in the part you do not.

When a signal is random, its DFT is random too. Run the transform on one block, get one answer; run it on the next block, get a visibly different answer — same underlying process, different numbers. So a single DFT is not "the spectrum" of a random signal. It is one noisy *estimate* of an underlying quantity, the *power spectral density* (PSD), which tells us how the signal's power is distributed across frequency *on average*. The whole craft of this chapter is turning noisy DFTs into trustworthy estimates of that average, and understanding the price every method charges.

Focus

The chapter in one sentence: the squared DFT magnitude (the *periodogram*) is an unbiased but hopelessly *noisy* estimate of the power spectrum — its scatter does not shrink as you collect more data — and the fix is to *average* many short periodograms, trading frequency resolution for a calmer, more reliable curve. Everything else (windows, Welch, resolution-versus-variance) is detail on that one trade.

21.1 What "the spectrum" even means for a random signal

Take a deterministic finite-energy signal $x(t)$. Its Fourier transform $X(f)$ exists, and $|X(f)|^2$ is the *energy spectral density*: integrate it over a band and you get the energy in that band (Parseval, from the transform chapter). Beautiful — but a random signal that hums along forever has *infinite* energy and *finite power*, so $X(f)$ does not even exist as an ordinary function. We need a power-based replacement.

Intuition

Energy is the total; power is energy per unit time. A pure tone playing forever delivers infinite total energy but a perfectly steady, finite power. For signals that never end, "power per unit frequency" is the right currency, and the power spectral density $S_x(f)$ is the ledger: $S_x(f)\,\mathrm{d}f$ is the average power carried by the sliver of frequencies between f and $f + \mathrm{d}f$.

Formally, take a *wide-sense stationary* random signal — one whose statistics do not drift with time, so its mean is constant and its autocorrelation

$$r_x[m] \;=\; \mathbb{E}\big[x[n]\,x[n+m]\big]$$

depends only on the lag m, not on n (this is the random-signal language from the noise chapter). The power spectral density is the DTFT of that autocorrelation:

$$\boxed{S_x(\omega) \;=\; \sum_{m=-\infty}^{\infty} r_x[m]\,e^{-\mathrm{j}\omega m}.}$$

This is the *Wiener–Khinchin theorem*, and it is the cornerstone of the whole subject: the spectrum of a random signal is not the transform of the signal (which is random and ill-defined) but the transform of its *autocorrelation* (which is a stable, deterministic function).

Why it works

Why autocorrelation? Because $r_x[m]$ is exactly the deterministic skeleton that survives the randomness. Any single realization of the signal is a wild, unrepeatable squiggle, but the *average* of $x[n]x[n+m]$ over all realizations is a fixed number for each lag. Wiener–Khinchin says: throw away the unrepeatable squiggle, keep its correlation structure, transform that, and you get the one spectral object that actually means something. The power at frequency ω is governed by how strongly the signal correlates with time-shifted copies of itself.

Connection

This is the inner-product idea from the linear-algebra book wearing a probabilistic coat. $r_x[m] = \mathbb{E}[x[n]x[n+m]]$ is a *statistical* inner product between the signal and a shifted copy of itself — the expectation plays the role of the sum in $\langle x,\, y\rangle$. And the noise chapter's notion that white noise has a flat spectrum is just Wiener–Khinchin applied to $r_x[m] = \sigma^2\delta[m]$: the transform of a single spike at lag 0 is a constant, so white noise spreads its power equally over all frequencies. That is literally what "white" means.

21.1.1 The two difficulties

We have a clean target, $S_x(\omega)$. Now the bad news, stated plainly so we can spend the rest of the chapter fighting it.

1. **We never see the expectation.** $r_x[m]$ is an average over infinitely many imaginary repetitions of the experiment. We have *one* finite recording. We must estimate $r_x[m]$ (and hence S_x) from a single chunk of data.

2. **We have finitely many samples.** The sum in Wiener–Khinchin runs over all lags $m \in \mathbb{Z}$; we can only form a finite number of them, and the high lags are estimated from very few sample pairs, so they are themselves garbage.

Estimating $S_x(\omega)$ from a finite, noisy record is *spectral estimation*. There is no perfect answer — only well-understood trade-offs. We will meet the most important estimator first, watch it fail in an instructive way, and then repair it.

21.2 The periodogram: the obvious estimator

The most natural thing in the world: take your N samples, transform them, square the magnitude, normalize. That is the *periodogram*.

$$\hat{S}_x^{\text{per}}(\omega_k) \;=\; \frac{1}{N}\left|\sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}\omega_k n}\right|^2 \;=\; \frac{1}{N}\,|X[k]|^2.$$

The $1/N$ converts "energy of the block" into "power" (energy per sample). The name comes from Arthur Schuster, who in the 1890s used it to hunt for hidden periodicities in sunspot and weather records — he was literally looking for *periods*, hence "periodogram".

What is really happening

Here is what the periodogram secretly is. Squaring the DFT magnitude is the same as transforming the signal's *own* autocorrelation estimated from the block. In symbols, $\frac{1}{N}|X[k]|^2$ is the DFT of $\hat{r}_x[m] = \frac{1}{N}\sum_n x[n]x[n+m]$, the *sample* autocorrelation. So the periodogram is exactly "Wiener–Khinchin, but with the true $r_x[m]$ replaced by the one we could measure from a single block". That substitution is the original sin behind every problem that follows.

History

Schuster (1898) coined "periodogram" to find the supposed period of terrestrial magnetism and sunspots. For decades people plotted raw periodograms and saw spurious "peaks" everywhere, then argued about which were real. The mystery — why clean data gave such jagged, irreproducible spectra — was only fully resolved when the statistics of the estimator were understood in the mid-20th century (Bartlett, Tukey, Blackman, Welch). The periodogram's wild variance, which we are about to quantify, sank many a confident claim of a hidden cycle.

21.2.1 It is (nearly) unbiased — the good news

On average, the periodogram points at the right answer. For a long enough block,

$$\mathbb{E}\big[\hat{S}_x^{\text{per}}(\omega)\big] \;\approx\; S_x(\omega).$$

More precisely the expected periodogram is the true PSD *smeared* by the window's spectrum (we will see exactly how in Section 21.4); as $N \to \infty$ that smear narrows and the bias vanishes. So if you could average infinitely many periodograms, you would converge to the truth. Good.

21.2.2 Its variance does not go away — the disaster

Now the catastrophe. For Gaussian data the periodogram at each frequency behaves like the true value times a random multiplier with mean 1 and standard deviation ≈ 1:

$$\hat{S}_x^{\text{per}}(\omega) \;\approx\; S_x(\omega) \cdot \tfrac{1}{2}\chi_2^2, \qquad \operatorname{Var}\big[\hat{S}_x^{\text{per}}(\omega)\big] \;\approx\; S_x(\omega)^2.$$

Stare at that variance. It equals the *square of the very thing we are trying to measure*, and — this is the killer — it **does not depend on** N. Collect ten times more data and the periodogram does not get smoother. It just gets *finer* (more frequency bins), each bin still rattling around its true value with a standard deviation as big as the value itself.

Common trap

"More samples $\Rightarrow$ better spectrum" is false for the raw periodogram. More samples buy you more frequency *bins*, not less *scatter* per bin. A 1-million-point periodogram of white noise is a million-point hairy mess whose spikes are exactly as tall, relative to the mean, as a 256-point one. This is the single most common misconception in practical spectral analysis. The estimator is *not consistent*: it never settles down.

Why it works

Why does the noise refuse to average out? Because each DFT bin is a sum of N terms, and the central limit theorem makes the real and imaginary parts of $X[k]$ roughly Gaussian. Their squared sum is a chi-squared with 2 degrees of freedom — *regardless of* N. Adding more samples adds more bins, each still built from its own two Gaussian draws. You are not pooling information about a given frequency; you are spreading a fixed amount of per-frequency information ever more thinly. To actually reduce the scatter at a frequency you must combine *several independent looks* at that same frequency. That is the entire idea of the next section.

Let us see the disaster and its cure in one picture.

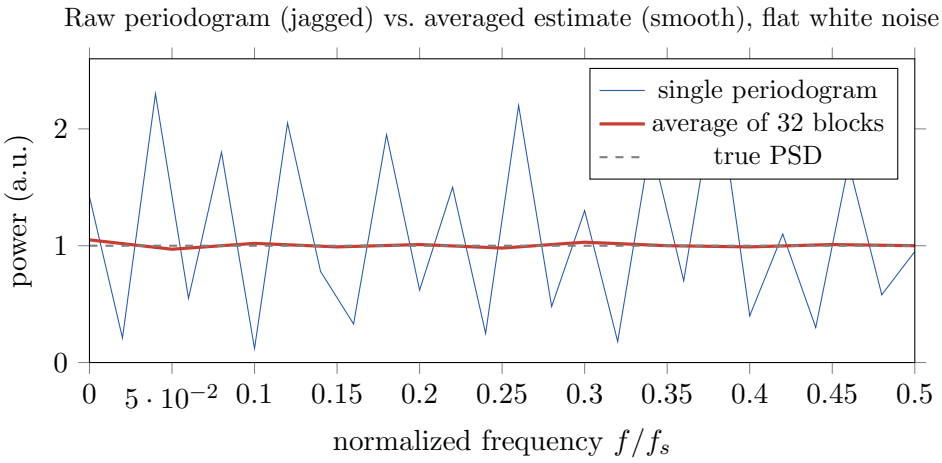


Figure 21.1: White noise has a flat true PSD (dashed). A single periodogram (thin blue) swings between near zero and more than double the truth at every bin, and adding samples will not calm it. Averaging 32 independent periodograms (thick red) hugs the true value: the scatter has shrunk by roughly $\sqrt{32}$.

21.3 The cure: average independent looks (Bartlett)

The variance lesson points straight at the fix. If one periodogram has standard deviation as big as the signal, average K *independent* periodograms and the standard deviation drops by $\sqrt{K}$ — the bread-and-butter law of averaging from the probability book. Bartlett (1948) made this concrete: chop the record into K non-overlapping segments of length L (so $N = KL$), periodogram each, and average them frequency by frequency.

$$\hat{S}_x^{\text{Bart}}(\omega_k) \;=\; \frac{1}{K}\sum_{i=0}^{K-1}\frac{1}{L}\left|\sum_{n=0}^{L-1} x_i[n]\, e^{-\mathrm{j}\omega_k n}\right|^2,$$

where $x_i[n] = x[iL + n]$ is the ith segment. The variance falls like

$$\operatorname{Var}\big[\hat{S}_x^{\text{Bart}}(\omega)\big] \;\approx\; \frac{1}{K}\, S_x(\omega)^2.$$

Intuition

Think of each segment as one photograph of a fidgety subject. One photo is blurry with motion; thirty photos averaged reveal the steady pose underneath. Bartlett's method takes K snapshots of the spectrum and averages them. The fidget (variance) washes out; the pose (the true PSD) emerges. But there is no free lunch, and the price is hiding in the segment length L.

Here is the trade, stated with painful clarity. You have a fixed budget of N samples. You must split it as $N = KL$.

- **More segments** K (each shorter) $\Rightarrow$ smoother estimate, lower variance.

- **Longer segments** L (fewer of them) $\Rightarrow$ finer frequency resolution, because a length-L DFT can distinguish frequencies only $\approx f_s/L$ apart.

You cannot have both. With N fixed, pushing K up pushes L down and vice versa. This is the *resolution–variance trade-off*, and it is the central tension of all spectral estimation. We will give it its own section (Section 21.6) once we have the last ingredient, windowing.

Example 21.1 (Reading the trade-off off real numbers)**.** Suppose $f_s = 1000$ Hz and you collect $N = 8192$ samples (about 8.2 s). Two extreme plans:

- One block, $L = 8192$: resolution $\Delta f = f_s/L \approx 0.12$ Hz — exquisite — but $K = 1$, so the curve is the full hairy periodogram.
- $K = 64$ blocks of $L = 128$: resolution $\Delta f = f_s/L \approx 7.8$ Hz — coarse — but variance down by $64\times$, standard deviation down by $8\times$. A clean, readable curve that cannot separate two tones closer than ~ 8 Hz.

A sensible middle, $K = 16$, $L = 512$: $\Delta f \approx 2$ Hz and a $4\times$ drop in standard deviation. Which to pick depends entirely on the question you are asking of the data.

21.4 Windowing and leakage, revisited for estimation

We met spectral leakage in the DFT chapter: chopping a signal to a finite block multiplies it by a rectangular window, and multiplication in time is convolution in frequency, so every true spectral line gets convolved with the window's transform — a sinc-like shape with a tall main lobe and a long train of side lobes. For *estimation* this matters in two distinct ways, and keeping them separate is the key to using windows well.

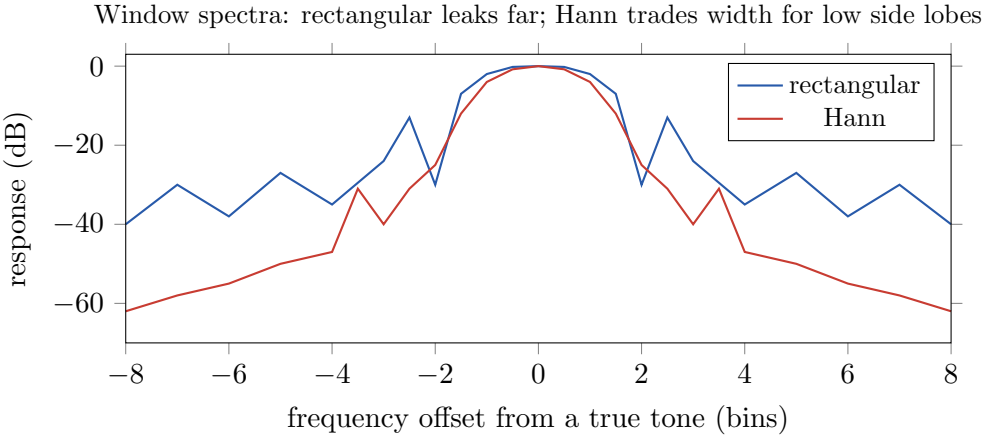


Figure 21.2: The fundamental window trade. The rectangular window has the narrowest main lobe (best resolution) but side lobes only -13 dB down that fall off slowly — a weak tone can be drowned by a strong neighbor's leakage. The Hann window roughly doubles the main-lobe width (worse resolution) but pushes side lobes to -31 dB and beyond, so it hides weak components far less. There is no window that wins on both.

21.4.1 Leakage effect one: bias and the smeared truth

The expected periodogram is not the true PSD — it is the true PSD convolved with the window's *power* spectrum $|W(\omega)|^2$:

$$\mathbb{E}\big[\hat{S}_x^{\text{per}}(\omega)\big] \;=\; \frac{1}{N}\,\big(S_x * |W|^2\big)(\omega).$$

Two consequences. The *main lobe* blurs nearby detail together (a resolution loss — you cannot resolve two tones inside one main-lobe width). The *side lobes* let power from strong components splatter into far-away bins where they do not belong (a dynamic-range loss — a quiet tone next to a loud one disappears under the loud one's skirts). Choosing a window is choosing how to split your pain between these two.

21.4.2 Leakage effect two: tapering for the average

When we average overlapping segments (next section), the window does a second job: it tapers each segment smoothly to zero at its ends. This matters because a rectangular cut leaves jagged discontinuities at the segment edges, which dump energy into the side lobes *and* make overlapping segments share a lot of the same edge artifacts. A smooth taper (Hann, Hamming, Blackman) softens the edges so overlapping segments are closer to statistically independent, which is what makes the averaging actually deliver its variance reduction.

Focus

Two windows you will reach for ninety percent of the time:

- **Hann** ($w[n] = 0.5 - 0.5\cos(2\pi n/(L-1))$): the safe default. Good side-lobe suppression (-31 dB), fast roll-off, plays beautifully with 50% overlap. Use it unless you have a reason not to.
- **Rectangular** (no window): only when you have captured exactly an integer number of periods of a periodic signal, or when you genuinely need the sharpest possible resolution and the dynamic range is small.

Reach for **Blackman–Harris** (side lobes below -90 dB) when a tiny tone sits next to a huge one, and accept its fat main lobe.

Connection

This is the same convolution theorem the LTI and Fourier chapters hammered: multiply in time $\Rightarrow$ convolve in frequency. Windowing is nothing but applying that theorem deliberately. The window's transform *is* the smearing kernel, and the entire art of window design is sculpting a kernel with a narrow main lobe and small side lobes — a goal the uncertainty principle forbids you from achieving perfectly, exactly as it forbids a signal from being both time-limited and band-limited.

21.5 Welch's method: overlap, window, average

Welch (1967) combined both ideas — windowing and averaging — and added one more trick: *overlapping* the segments. His method is the default spectral estimator in essentially every toolbox

(`scipy.signal.welch`, MATLAB's `pwelch`), so it deserves a careful walk-through.

Walking question

Why overlap the segments at all? Bartlett's non-overlapping segments waste the data that the window tapers to zero near each segment's edges — those samples barely contribute. By sliding the window forward by only half a segment (50% overlap), the samples that were down-weighted as one segment's edge become the well-weighted center of the next. You get *more* segments out of the same N samples, hence more variance reduction, almost for free. The catch: overlapping segments are not fully independent, so K overlapping segments reduce variance by somewhat less than a factor of K — but still more than the smaller number of non-overlapping segments would.

The recipe, in full (Figure 21.3 shows the flow):

1. Choose a segment length L (sets resolution $\approx f_s/L$) and a window $w[n]$ of length L (Hann is the usual pick).
2. Slide the window across the data in steps of $L/2$ (50% overlap), producing K overlapping segments $x_i[n] = x[iL/2 + n]$.
3. Window and transform each: $X_i[k] = \sum_{n=0}^{L-1} w[n]\, x_i[n]\, e^{-\mathrm{j}2\pi kn/L}$.
4. Form each segment's *modified periodogram* and average:

$$\hat{S}_x^{\text{Welch}}(\omega_k) \;=\; \frac{1}{K}\sum_{i=0}^{K-1} \frac{1}{LU}\,|X_i[k]|^2, \qquad U = \frac{1}{L}\sum_{n=0}^{L-1} w[n]^2.$$

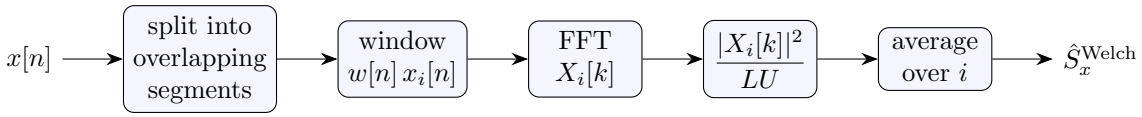


Figure 21.3: Welch's pipeline: the record is cut into overlapping segments, each is tapered by the window, transformed, turned into a calibrated modified periodogram, and the periodograms are averaged bin by bin. The averaging step is where the variance reduction lives.

The factor U is the window's mean-square value, and dividing by it restores the correct *power* calibration — without it, a tapering window would make every estimate read low because it throws away signal energy at the segment edges. Getting this normalization right is the difference between a PSD you can read absolute power off of and one that is merely the right shape.

Common trap

Two normalization booby-traps. (1) Forgetting the $1/U$ window correction: your PSD comes out scaled down (by about 2.7× for a Hann window) and your power readings are simply wrong. (2) Mixing up the *power-spectral-density* normalization (divide by f_s too, units of power/Hz, for noise-like signals) with the *amplitude/power-spectrum* normalization (units of power, for discrete tones). For a sine wave you want the latter so the peak reads the tone's actual power; for broadband noise you want the former so the level is independent of your

DFT length. `scipy.signal.welch` exposes this as `scaling='density'` vs. `'spectrum'` — choose deliberately.

21.5.1 Welch in code

Listing 21.1: Welch's method by hand, then with SciPy. The two agree.

```python
import numpy as np

def welch_psd(x, fs, L=256, overlap=0.5, window='hann'):
    x = np.asarray(x, float)
    step = int(L * (1 - overlap))           # hop between segments
    w = np.hanning(L) if window == 'hann' else np.ones(L)
    U = np.mean(w**2)                        # window power, for calibration
    segs = [x[i:i+L] for i in range(0, len(x) - L + 1, step)]
    psd = np.zeros(L // 2 + 1)
    for s in segs:
        X = np.fft.rfft(w * s)
        psd += (np.abs(X)**2) / (L * U)
    psd /= len(segs)
    psd[1:-1] *= 2                           # fold negative-freq power onto positive
    psd /= fs                                # 'density' scaling: power per Hz
    f = np.fft.rfftfreq(L, 1/fs)
    return f, psd

# Demo: 50 Hz tone in white noise, sampled at 1 kHz
fs = 1000
n  = np.arange(8192)
x  = np.sin(2*np.pi*50*n/fs) + 0.5*np.random.randn(len(n))
f, P = welch_psd(x, fs, L=256)

# Compare against the library (should match closely)
from scipy.signal import welch
f2, P2 = welch(x, fs=fs, nperseg=256, window='hann', scaling='density')
print(np.allclose(P, P2, rtol=1e-6))   # True
```

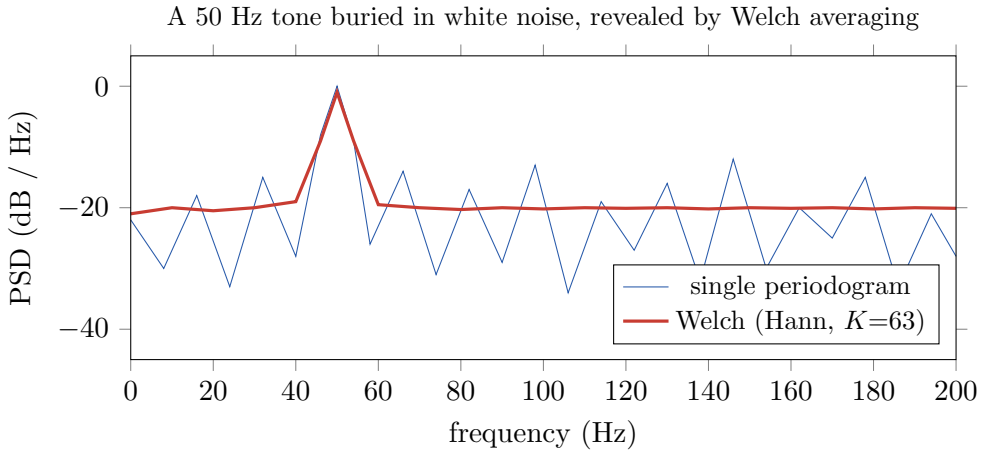


Figure 21.4: The payoff. A single periodogram (thin blue) has a noise floor so jagged that the 50 Hz tone is only barely the tallest spike. Welch averaging (thick red) flattens the floor to a clean, readable −20 dB and the tone stands 19 dB proud — the same data, made legible by trading resolution for variance reduction.

History

Peter Welch published his method in a two-page IEEE paper in 1967, the same era as the Cooley–Tukey FFT. The timing was no accident: cheap FFTs made it practical to compute hundreds of segment transforms, and Welch's contribution was the right *statistical* packaging — modified periodograms, overlap, averaging — to turn that cheap compute into a low-variance estimate. Sixty years on, when you call `pwelch` or `scipy.signal.welch`, you are running essentially his 1967 recipe.

21.6 Resolution versus variance, made quantitative

Now we can state the central trade-off with numbers rather than hand-waving. Fix your total record length N and sampling rate f_s. The two quantities you care about pull in opposite directions through the segment length L.

Frequency resolution. A length-L windowed DFT cannot distinguish two tones closer than roughly the window's main-lobe width:

$$\Delta f \;\approx\; C_w \cdot \frac{f_s}{L},$$

where C_w is a window-dependent constant: $C_w \approx 0.89$ for rectangular, ≈ 1.5 for Hann, ≈ 1.8 for Blackman. Larger $L \Rightarrow$ finer Δf. Good resolution wants *long* segments.

Variance. With 50% overlap you get about $K \approx 2N/L$ segments, and the estimate's variance falls roughly as $1/K$:

$$\operatorname{Var}\big[\hat{S}_x(\omega)\big] \;\approx\; \frac{S_x(\omega)^2}{K} \;\approx\; \frac{L}{2N}\, S_x(\omega)^2.$$

Smaller $L \Rightarrow$ more segments $\Rightarrow$ lower variance. A smooth estimate wants *short* segments.

Why this matters

There it is, in two formulas pointing in opposite directions. Resolution improves as L grows; variance improves as L shrinks. With N fixed you slide a single knob, L, between "sharp but noisy" and "smooth but blurry". There is no setting that is best for everything — the right L is dictated by your question. Are you trying to *separate* two close tones? Push L up. Are you trying to *measure the level* of a broadband floor? Push L down. This single trade-off, once internalized, demystifies ninety percent of practical spectral analysis.

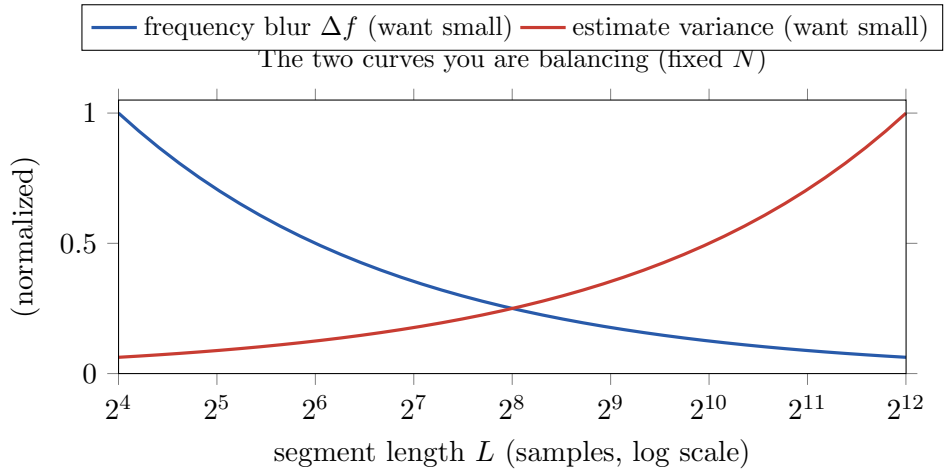


Figure 21.5: Two opposing pulls as the segment length L varies with the total record N held fixed. Frequency blur (blue) shrinks as L grows — long segments resolve fine detail. Estimate variance (red) grows with L — long segments mean fewer of them to average. The good operating point sits where the curves cross for *your* priorities, not at either extreme.

Remark 21.1 (The equivalent-degrees-of-freedom view). Statisticians summarize the smoothness of a Welch estimate by its *equivalent degrees of freedom*, $\nu \approx 2K$ (each segment contributing a χ^2_2). The fractional standard error of the estimate is $\approx \sqrt{2/\nu} = 1/\sqrt{K}$. So "how many segments" and "how reliable is each PSD value" are the same question: $K = 1$ gives a 100% standard error (the raw periodogram's hopeless scatter); $K = 100$ gives 10%. This number, not the pretty curve, is what you cite when someone asks "is that peak real?".

21.7 Worked example: choosing parameters end to end

Example 21.2 (Separating two close tones in noise)**.** You record $N = 4096$ samples at $f_s = 2000$ Hz. You suspect two tones near 300 Hz and 312 Hz, 12 Hz apart, plus broadband noise. Design a Welch analysis.

Resolution requirement. To separate tones 12 Hz apart with a Hann window ($C_w \approx 1.5$) we need $\Delta f \le 12$ Hz, i.e.

$$1.5\,\frac{f_s}{L} \le 12 \;\Rightarrow\; L \ge 1.5 \cdot \frac{2000}{12} \approx 250.$$

Round up to a power of two: $L = 512$ (giving $\Delta f \approx 5.9$ Hz, a comfortable margin).

How many segments? With 50% overlap and step 256:

$$K \approx \frac{N-L}{L/2} + 1 = \frac{4096-512}{256} + 1 = 15.$$

So $\nu \approx 30$ degrees of freedom and a fractional standard error of $1/\sqrt{15} \approx 26\%$ — enough to see the two peaks clearly and trust they are real, though the noise floor will still look slightly rough.

If you needed a calmer floor, drop to $L = 256$: now $\Delta f \approx 11.7$ Hz — right at the edge of resolving the two tones, risky — but $K \approx 31$ and standard error $\approx 18\%$. The tension is explicit: $L = 512$ to *see* the two tones, $L = 256$ to *measure the floor*. With only 4096 samples you cannot do both superbly; collecting more data is the only real escape, and it buys you more segments (lower variance) at whatever resolution you have already fixed.

Try it yourself

Generate exactly this signal and watch the trade-off with your own eyes:

```
import numpy as np; from scipy.signal import welch
fs, N = 2000, 4096
n = np.arange(N)
x = (np.sin(2*np.pi*300*n/fs) + np.sin(2*np.pi*312*n/fs)
     + 0.7*np.random.randn(N))
for L in (128, 256, 512, 1024):
    f, P = welch(x, fs=fs, nperseg=L, window='hann')
    # zoom the plot to 280..330 Hz and watch the twin peaks merge as L drops
    print(L, "resolves two peaks:", L >= 256)
```

At $L = 128$ the two tones fuse into one blob; at $L = 512$ they split cleanly but the floor is rougher; at $L = 1024$ the split is sharp yet you have only ≈ 7 segments, so the floor rattles. Feel the knob.

21.8 Spectral analysis of real audio and motion signals

Application

Audio. A 4-second guitar note at $f_s = 44.1$ kHz is 176,400 samples. Welch with $L = 4096$ and Hann gives $\Delta f \approx 16$ Hz — fine enough to separate the fundamental (~ 82 Hz for low E) from its harmonics at 164, 246 Hz, etc. — and roughly $K \approx 85$ segments for a smooth, trustworthy harmonic profile. That PSD is the raw material for pitch detection, timbre fingerprinting, and the audio-feature pipelines of later chapters. For *music*, though, you often want the *spectrogram* (short-time Fourier transform) instead of a single averaged PSD, because the spectrum changes over the note's attack, sustain, and decay — Welch deliberately averages that time variation away, which is exactly wrong when the time variation is the point.

Application

Motion / IMU. A wrist IMU sampled at $f_s = 100$ Hz captures human motion that lives almost entirely below 20 Hz: gait is 1–3 Hz, voluntary arm gestures 0.5–5 Hz, physiological

tremor 4–12 Hz. To resolve a 5 Hz tremor from a 3 Hz gesture (2 Hz apart) with a Hann window you need $L \geq 1.5 \cdot 100/2 = 75$, say $L = 128$ ($\Delta f \approx 0.8$ Hz, ~ 1.3 s per segment). A 20-second recording then yields $K \approx 30$ segments — a clean PSD of the motion's rhythmic content. The accelerometer PSD typically shows a strong low-frequency hump (intentional motion) and, in some subjects, a distinct tremor peak; the gyro PSD complements it for rotational components.

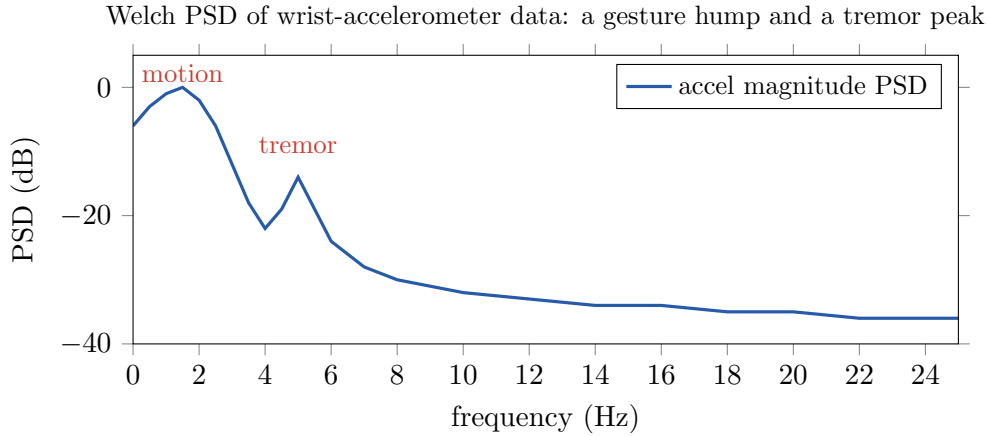


Figure 21.6: A representative Welch PSD of wrist-worn accelerometer data during a slow arm gesture. A broad low-frequency hump near 1–2 Hz is the intentional motion; a smaller distinct peak near 5 Hz is physiological tremor. The averaging is what makes the tremor peak stand out cleanly from the noise floor instead of being one of dozens of random spikes.

Toward the wrist tracker

For the wrist instrument controller, the PSD is a *feature extractor*, not a display. A short Welch (or simpler: the band-power of a single windowed DFT) over a sliding ~ 1 s window gives you a handful of numbers — power in 0–3 Hz (deliberate motion), 4–12 Hz (tremor), and higher bands (taps, impacts) — that feed the gesture classifier downstream. These band powers are cheap, robust to the exact gesture timing, and far more stable than raw samples. Be clear about what they can and cannot do. A wrist IMU's spectrum can tell you *that* the arm is moving rhythmically and roughly how fast, and it can flag tremor or taps. It **cannot** recover individual finger joint angles — the spectral signature of finger motion barely reaches the wrist. Continuous finger pose needs sensors on the fingers (instrumented rings) or muscle sensing (EMG / force myography), whose own spectra we estimate with exactly these same Welch tools in the hand-sensing chapter. The PSD is a strong coarse-gesture and rhythm detector; it is not a finger tracker, and any product claim otherwise is false.

Build it (hardware)

On an Arduino-class board you usually cannot afford a full Welch with overlapping buffers in RAM. The lightweight field version: keep one circular buffer of $L = 128$ or 256 samples, apply a precomputed Hann window (store it as a constant array), run a fixed-point or CMSIS-DSP FFT (`arm_rfft_fast_f32`), square the bins, and *exponentially average* the resulting

periodogram into a running estimate

$$\hat{S} \leftarrow \alpha\,\hat{S} + (1-\alpha)\,|X|^2, \qquad \alpha \approx 0.9.$$

This is "Welch on a budget": the exponential average plays the role of averaging $K \approx 1/(1-\alpha) \approx 10$ segments, smoothing the estimate without ever storing more than one segment at a time. Then sum bins into your 3–4 band powers and ship those to the classifier. A Cortex-M4 at 80 MHz does a 256-point real FFT in well under a millisecond, leaving plenty of headroom at 100 Hz IMU rates.

21.9 Beyond the periodogram: a brief map

Welch is the workhorse, but it is not the only tool, and it is worth knowing what lies past it so you can recognize when you have outgrown it.

- **Multitaper (Thomson).** Instead of one window, apply several specially designed orthogonal windows (discrete prolate spheroidal sequences) to the *same* segment and average their periodograms. This squeezes more independent looks out of a *short* record than Welch can — the best general-purpose choice when data is precious (short EEG epochs, brief gestures). The cost is more computation and a fixed resolution–bandwidth product you set up front.

- **Parametric / AR methods (Yule–Walker, Burg).** Assume the signal is the output of an all-pole filter driven by white noise, fit the filter coefficients, and read the spectrum off the model. These give very sharp peaks from short records — excellent when you *know* the signal is a few resonances (speech formants, a couple of vibration modes) — but they hallucinate structure if the model order is wrong, so they demand more faith in your assumptions than the assumption-free periodogram family.

- **The periodogram family stays the safe default.** When you do not have strong prior knowledge, Welch (or multitaper for short records) makes the fewest assumptions and fails the most gracefully. Reach for parametric methods only when you can defend the model.

Pause

Pause and consolidate. The arc of this chapter: a random signal's spectrum is the PSD, the transform of its autocorrelation (Wiener–Khinchin). The periodogram estimates it but is unbiased-yet-hopelessly-noisy, with variance that ignores how much data you have. The cure is averaging independent looks (Bartlett), improved by windowing and overlap (Welch), governed everywhere by the resolution-versus-variance trade-off you tune with the segment length. Windows sculpt the smearing kernel; the trade-off picks how to split your pain between sharpness and smoothness. That is the whole subject, and it is enough to do real, trustworthy spectral analysis of audio and motion.

21.10 Exercises

Exercise 21.1. The variance that won't die. Generate $N = 1024$ samples of white Gaussian noise and plot its periodogram. Now do the same for $N = 16384$. Confirm by eye (and by computing the ratio of standard deviation to mean across bins) that the relative scatter is the *same* in both, even though the second has 16× more bins. Explain in one sentence why this is the expected result, not a bug.

Exercise 21.2. Averaging law. For white noise, compute Welch estimates with $K = 1, 4, 16, 64$ non-overlapping segments (adjust N so L stays fixed). Measure the standard deviation of the estimate across frequency for each K and verify it falls like $1/\sqrt{K}$. By how much would you expect $K = 256$ to improve over $K = 1$?

Exercise 21.3. Window dynamic range. Build a signal with a strong tone at 100 Hz (amplitude 1) and a weak tone at 130 Hz (amplitude 0.01, i.e. −40 dB), in light noise, at $f_s = 1000$ Hz. Estimate the PSD with a rectangular window and with a Blackman–Harris window at the same L. Which window lets you see the weak tone, and why? Relate your answer to side-lobe level.

Exercise 21.4. Resolution requirement. You must separate tones at 440 Hz and 445 Hz ($f_s = 8000$ Hz) using a Hann window ($C_w \approx 1.5$). What is the minimum segment length L? If you have $N = 8000$ samples (1 s) and use 50% overlap, how many segments do you get, and what is the resulting fractional standard error?

Exercise 21.5. Normalization sanity check. Take a pure 1 V-amplitude sinusoid (no noise) and compute its Welch PSD with `scaling='spectrum'`. The peak should read the tone's power, 0.5 V^2 ($= A^2/2$). Verify you get this with the $1/U$ window correction in place, and show that omitting the correction makes the peak read low by the window's loss factor.

Exercise 21.6. Wiener–Khinchin by hand. For white noise with $r_x[m] = \sigma^2\delta[m]$, show directly from the Wiener–Khinchin sum that $S_x(\omega) = \sigma^2$ for all ω (a flat spectrum). Then for a decaying exponential autocorrelation $r_x[m] = \sigma^2 a^{|m|}$ with $0 < a < 1$, compute $S_x(\omega)$ and sketch its shape. What kind of signal has this "red" (low-frequency-heavy) spectrum?

Exercise 21.7. Overlap payoff. Estimate the PSD of a fixed record with Bartlett (non-overlapping) and with Welch (50% overlap), using the same L and a Hann window. Welch produces more segments from the same data — count them. Does the estimate variance fall by the full ratio of segment counts? Explain why overlapping segments are not fully independent and what that does to the realized variance reduction.

Exercise 21.8. IMU band powers. Record (or simulate) 20 s of 100 Hz accelerometer data containing a 2 Hz arm-swing and a 6 Hz tremor. Compute a Welch PSD with $L = 128$, then integrate it over the bands 0–3 Hz and 4–12 Hz to get two band-power features. Show how those two numbers change between "arm swinging" and "arm still but tremoring", and argue why these features are more robust for a gesture classifier than the raw samples.

Exercise 21.9. The wrong tool. You analyze a guitar note's full 4 s with a single Welch PSD and get a clean harmonic series, but your pitch tracker still fails during the note's attack. Explain what Welch averaged away, and which tool from an earlier chapter you should use instead when the spectrum changes over time.

Exercise 21.10. Truthful limits. A teammate proposes selling the wrist controller as a device that "reads individual finger movements from the IMU spectrum". Using the physics of where finger motion shows up and what a wrist sensor can observe, write a short, specific rebuttal, and propose what sensing modality would actually be required for continuous finger pose.

Part V

Multirate, Modulation, and Time-Frequency

Chapter 22

Decimation, Interpolation, and Resampling

A sample rate is a promise. When an analog-to-digital converter hands you a stream of numbers and says "these arrived at 48,000 per second," it has fixed a clock that every later interpretation depends on. But the world rarely cooperates by giving you exactly the rate you want. Your accelerometer streams at 104 Hz, your gyroscope at 208 Hz, your microphone at 44,100 Hz, and the gesture model you trained expects everything resampled to a tidy 100 Hz grid. Somewhere, somebody has to change a sample rate.

Changing a sample rate sounds like bookkeeping — keep every other sample, or squeeze new samples in between — and if you do it that naively you will silently corrupt your data. This chapter is about doing it correctly, and more importantly about *seeing* why the naive versions fail. Everything here is a direct consequence of two ideas you already own: the spectrum of a sampled signal is periodic (the sampling chapter), and filtering is shaping that spectrum (the filter chapters). Decimation, interpolation, and rational resampling are just those two ideas applied with care.

22.1 What it means to change a sample rate

Let us be precise about the object we are manipulating. A discrete signal $x[n]$ is a sequence of numbers. By itself it carries no time information at all — $x[0], x[1], x[2], \ldots$ is just a list. The sample rate f_s (in samples per second, Hz) is the extra fact that tells us each index n corresponds to physical time $t = n/f_s = n\,T$, where $T = 1/f_s$ is the sampling period.

To "change the sample rate" means: produce a *new* sequence $y[m]$, sampled at a new rate f_s', that represents the *same underlying continuous signal* as $x[n]$ did. The phrase "same underlying continuous signal" is the whole game. We are not free to invent numbers; we are obligated to land on the values that the original analog waveform would have produced had we sampled it at the new rate in the first place.

Intuition

Think of $x[n]$ as marks you painted on a moving conveyor belt at regular spacing. Decimation is stepping back so the marks look closer together (fewer marks per meter of belt); interpolation

is painting extra marks in between. In both cases the belt — the real signal — never moved. You are only changing how densely you describe it. The trap is that the marks already *hide* information about what happened between them, and a careless redescription can make the belt appear to carry a pattern that was never there.

Three operations cover essentially all cases:

- **Decimation** (downsampling): lower the rate by an integer factor M, so $f_s' = f_s/M$. Fewer samples.
- **Interpolation** (upsampling): raise the rate by an integer factor L, so $f_s' = L\, f_s$. More samples.
- **Rational resampling**: change the rate by a ratio L/M, e.g. $44100 \to 48000$ is $\times 160/147$. Combine the two.

A truly arbitrary (irrational) ratio is handled by approximating it with a rational one, or by computing samples on demand with a continuous-time interpolation kernel; we will meet both.

Focus

The single most important rule of this chapter, stated up front so you can hold it while we build the reasons:

Before you throw samples away, remove the frequencies that the lower rate cannot represent. After you add samples, remove the spectral images that the upsampling created.

Anti-alias on the way down, anti-image on the way up. Everything else is detail.

22.2 Decimation: the geometry of throwing samples away

Start with the most innocent-looking operation imaginable. To halve the sample rate, keep every second sample and discard the rest:

$$y[m] = x[mM], \qquad m \in \mathbb{Z}. \tag{22.1}$$

For $M = 2$: $y = (x[0], x[2], x[4], x[6], \dots)$. This is called *downsampling* or *compressing*, and the block-diagram symbol is a downward arrow with the factor: $\downarrow M$.

Geometrically nothing could be simpler — we are reading the same list with a bigger stride. So why is there a chapter? Because the spectrum does something violent.

22.2.1 What downsampling does to the spectrum

Recall the central fact about sampling: a signal sampled at rate f_s has a spectrum that repeats every f_s. The discrete-time Fourier transform (DTFT) is periodic with period 2π in normalized frequency $\omega = 2\pi f/f_s$. When we downsample by M, the new rate is f_s/M, so the new spectrum must be periodic with the *smaller* period f_s/M. The only way to take a spectrum periodic on f_s and make

it periodic on f_s/M is to overlay M shifted copies on top of each other. Formally,

$$Y(e^{\mathrm{j}\omega}) = \frac{1}{M}\sum_{k=0}^{M-1} X\left(e^{\mathrm{j}(\omega-2\pi k)/M}\right). \tag{22.2}$$

Read this slowly. The downsampled spectrum is the original spectrum *stretched* by M (the ω/M), *scaled* in height by $1/M$, and then *summed over M shifted versions* (the $2\pi k$ shifts). The stretch is benign. The sum is where the danger lives: those M copies can land on top of one another. When they overlap, frequencies that were distinct in x add together in y and become inseparable. That overlap is *aliasing*, the exact same phenomenon as undersampling an analog signal — here it is induced in the digital domain by the act of decimation.

Why it works

Why must the copies overlap unless we prevent it? After stretching by M, the content that originally lived in $[0, \pi]$ now occupies $[0, M\pi]$ — it is M times too wide to fit in one period of length 2π. The replication then folds the excess back in. The only way to avoid the fold is to make sure the original content was narrow enough *before* stretching that, even after stretching by M, it still fits inside a single 2π window. That means the original signal must occupy only $|\omega| < \pi/M$, i.e. frequencies below $f_s/(2M)$ — precisely the new Nyquist limit.

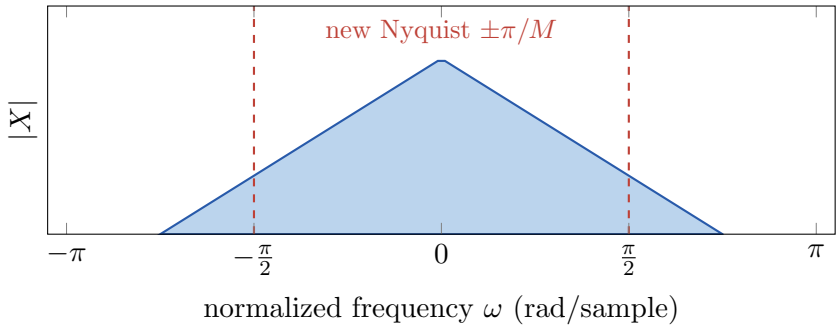


Figure 22.1: Before decimating by $M = 2$. The new Nyquist limit after $\downarrow 2$ is $\pm\pi/M = \pm\pi/2$ (dashed). The signal (blue) spills past it out to $|\omega| = 3\pi/4$; that excess content beyond the dashed lines will fold back and alias unless a lowpass removes it first.

22.2.2 The fix: filter, then decimate

The remedy is forced on us by the geometry. Before stride-reading, run $x[n]$ through a lowpass filter that removes everything above the new Nyquist frequency $f_s/(2M)$, i.e. above $\omega_c = \pi/M$. With the offending frequencies gone, the shifted copies in (22.2) no longer overlap, and the decimation is information-preserving for the band we kept. The combined operation — lowpass then $\downarrow M$ — is what most engineers actually mean by "decimation."

$$x[n] \longrightarrow \boxed{\text{LPF, cutoff } \pi/M} \longrightarrow \boxed{\downarrow M} \longrightarrow y[m]. \tag{22.3}$$

Common trap

The classic beginner bug: "I'll just take every other sample, it's faster." If the signal has *any* energy above the new Nyquist limit — and almost all real signals do, because of wideband noise alone — that energy folds down and contaminates your low band permanently. There is no later filter that can remove it, because after aliasing the noise and the signal share the same frequency bins. The damage is done at the moment of decimation. Filter first.

Example 22.1 (Decimating an IMU stream from 208 to 52 Hz)**.** An ICM-20948 gyroscope is configured for 208 Hz output, but our gesture model wants 52 Hz, so $M = 4$. The new Nyquist is 26 Hz. Vibration from a tapping finger or a nearby motor can easily put energy at, say, 40 Hz. If we naively keep every 4th sample, that 40 Hz tremor aliases: it lands at $|40 - 52| = 12$ Hz — right in the band where deliberate wrist motions live (a few Hz to ~ 15 Hz). It would masquerade as a real gesture. A lowpass at 26 Hz before decimating kills the tremor before it can fold.

What is really happening

What is really happening when we "decimate" is that we admit our new, coarser clock is incapable of describing fast wiggles, so we deliberately blur the signal down to wiggles it *can* describe, and only then re-sample it on the coarse grid. Decimation is deliberate forgetting: we choose what to lose, rather than letting aliasing scramble it unpredictably.

22.2.3 You only need to compute the outputs you keep

Here is the first efficiency insight, and it foreshadows polyphase. In the chain (22.3) the filter computes an output for *every* input sample, but the $\downarrow M$ immediately discards $M - 1$ of every M of them. That is wasteful: we are doing the full convolution at the high rate only to throw most of it away. Since the filter output at the discarded indices is never used, we can simply *not compute it*. If the lowpass has N taps, the direct chain costs N multiplies per input sample; computing only the kept outputs costs N multiplies per *output* sample, i.e. N/M per input sample — an M-fold saving. We make this rigorous with polyphase decomposition in Section 22.5.

22.3 Interpolation: the geometry of inventing samples

Now go the other way. We want more samples than we have. The correct version proceeds in two steps that mirror decimation in reverse.

22.3.1 Step 1: zero-stuffing (the expander)

To raise the rate by L, insert $L - 1$ zeros after each sample:

$$v[m] = \begin{cases} x[m/L], & m = 0, \pm L, \pm 2L, \dots \\ 0, & \text{otherwise.} \end{cases} \tag{22.4}$$

The block symbol is $\uparrow L$. For $L = 3$ and $x = (a, b, c)$ we get $v = (a, 0, 0, b, 0, 0, c, 0, 0)$. We have not changed any existing value and we have made the sequence L times longer, so it now lives on a grid that is L times finer. So far so harmless — but spectrally something curious happens.

22.3.2 What zero-stuffing does to the spectrum

Zero-stuffing does *not* change the spectrum's shape at all; it merely relabels the frequency axis. Because we packed the same content into L times as many samples, the new (higher) rate compresses the frequency axis, and the original spectrum, which lived once per 2π, now appears L times within one new period:

$$V(e^{j\omega}) = X(e^{j\omega L}). \tag{22.5}$$

The DTFT of v is just the DTFT of x compressed by L — which means L copies of the baseband spectrum now appear in $[-\pi, \pi]$. The extra $L-1$ copies are called *spectral images*. They are not noise; they are exact replicas of the signal sitting at higher frequencies, created purely by the act of inserting zeros.

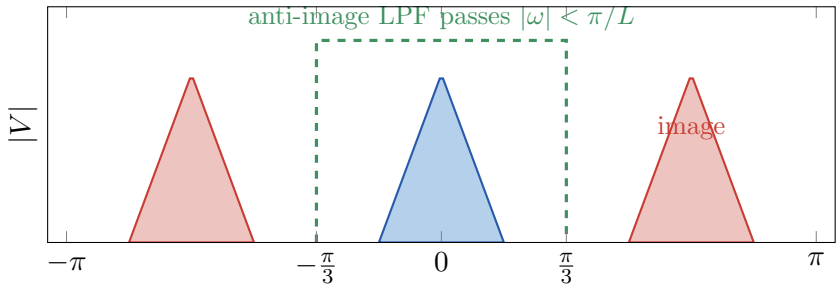


Figure 22.2: After $\uparrow L$ with $L = 3$. The baseband copy (blue) is the signal we want; the red bumps are spectral images created by zero-stuffing. The interpolation lowpass (green) keeps only $|\omega| < \pi/L$ and removes the images, smoothly filling in the zeros.

22.3.3 Step 2: the anti-imaging (interpolation) filter

The zeros we inserted are placeholders; we want the filter to replace them with sensible interpolated values. In the frequency domain that is exactly the same as removing the images: keep only the baseband copy $|\omega| < \pi/L$ and zero out the rest with a lowpass filter of cutoff π/L. So:

$$x[n] \longrightarrow \boxed{\uparrow L} \longrightarrow \boxed{\text{LPF, cutoff } \pi/L,\ \text{gain } L} \longrightarrow y[m]. \tag{22.6}$$

The gain L compensates for the energy spreading: a unit-height passband would make the output L times too quiet, because only $1/L$ of the samples carry all the energy. With gain L the interpolated samples have the right amplitude.

Why it works

Why is "remove the images" the same as "fill in the zeros sensibly"? In the time domain, convolving the zero-stuffed sequence with the filter's impulse response h places a copy of h, scaled by $x[k]$, at every original sample position kL, and sums them. The zeros are filled by the overlapping skirts of those kernels. If h is an ideal lowpass (a sinc), the result is the unique band-limited interpolation — the same curve the Whittaker–Shannon formula gives. "Kill the images" and "draw the smooth band-limited curve through the samples" are literally the same operation viewed from two domains.

22.3.4 Interpolation kernels you can picture

The choice of h is a choice of how you believe the signal behaves between samples. Three kernels, in increasing quality and cost:

- **Zero-order hold (nearest)**: h is a rectangle of width L. Every inserted sample copies its left neighbor. Cheap, blocky; its frequency response is a wide sinc that passes a lot of image energy. Staircase output.
- **Linear interpolation**: h is a triangle of width $2L$. Inserted samples sit on straight lines between originals. Better; frequency response is sinc^2, so images are attenuated more. Still rolls off the top of the passband.
- **Bandlimited (windowed sinc)**: h approximates the ideal brick-wall lowpass. Sharpest image rejection, flattest passband, highest cost. This is what high-quality audio resamplers use.

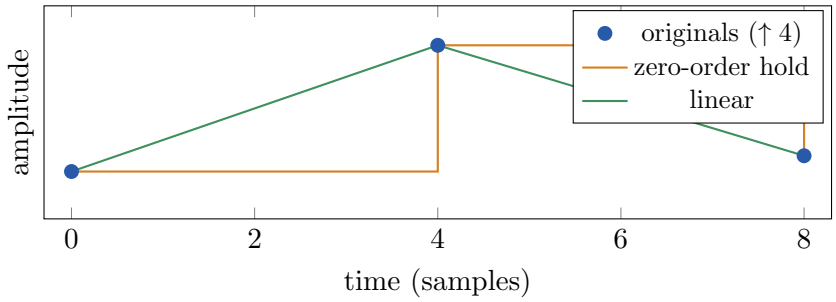


Figure 22.3: Three originals (dots) upsampled by $L = 4$. Zero-order hold (orange) gives a staircase; linear (green) gives straight segments. A windowed-sinc filter would give a smooth band-limited curve through the same dots.

Connection

The interpolation filter *is* the discrete analogue of the Whittaker–Shannon reconstruction integral from the sampling chapter, and that integral is itself a projection: it expresses the band-limited signal in the orthogonal basis of shifted sincs (linear algebra book, on orthonormal bases). Choosing a cheaper kernel is choosing a non-ideal, non-orthogonal basis — you trade reconstruction error for arithmetic. Every resampler is a statement about which basis you can afford.

22.4 Rational resampling: changing rate by L/M

Most real rate conversions are neither pure integers up nor down. To go from 44.1 kHz to 48 kHz you multiply by $48000/44100 = 160/147$. The recipe is to do both, *in the right order*: upsample by L, then downsample by M, with a single lowpass in the middle that serves *both* as anti-image (for the $\uparrow L$) and anti-alias (for the $\downarrow M$):

$$x \;\to\; \boxed{\uparrow L} \;\to\; \boxed{\text{LPF, cutoff } \min\!\left(\tfrac{\pi}{L}, \tfrac{\pi}{M}\right),\ \text{gain } L} \;\to\; \boxed{\downarrow M} \;\to\; y. \tag{22.7}$$

Focus

Always upsample first. If you downsample before upsampling, the $\downarrow M$ may have already discarded frequencies that the final rate *could* have kept, and no later upsampling recovers them. Going up first, then filtering, then down preserves the maximum bandwidth the output rate can support. The single shared filter must protect both edges, so its cutoff is the *lower* of the two limits, $\min(\pi/L, \pi/M)$.

Example 22.2 (44.1 kHz to 48 kHz)**.** Reduce 48000/44100: dividing by gcd = 300 gives 160/147. So $L = 160$, $M = 147$. We conceptually upsample by 160 (to 7.056 MHz!), lowpass at the lower of $\pi/160$ and $\pi/147$, namely $\pi/160$, then downsample by 147. We never actually materialize the 7 MHz stream — polyphase (next section) lets us compute only the 48000 outputs we keep, directly from the 44100 inputs.

Example 22.3 (A messy ratio: 104 → 100 Hz)**.** An accelerometer at 104 Hz, model wants 100 Hz. Ratio 100/104 = 25/26, so $L = 25$, $M = 26$. Tiny rate change, but the same machinery applies: upsample by 25, lowpass at $\min(\pi/25, \pi/26) = \pi/26$, downsample by 26. Because L and M are close, the filter does very little reshaping and a modest design suffices. Contrast this with sloppy alternatives: dropping one sample every 26 (sample-and-hold decimation) introduces a periodic glitch at 4 Hz — squarely inside the gesture band — whereas the proper filter spreads the tiny correction smoothly.

Common trap

Reduce the ratio before choosing L and M. Using 48000/44100 unreduced ($L = 48000$, $M = 44100$) asks for a 2.1 GHz intermediate rate and a filter with millions of taps for no benefit. The reduced form 160/147 gives the identical result with a tractable design. Always divide by $\gcd(L, M)$ first.

22.5 Polyphase: doing it without the waste

The chains above are correct but, taken literally, absurd: in Example 22.2 they build a 7 MHz signal that is 99% zeros, filter all of it, then throw away 146 of every 147 outputs. Polyphase decomposition is the algebra that removes every wasted multiply. The result is not an approximation — it computes *exactly* the same output, just without touching zeros or computing samples we discard.

22.5.1 The idea in one picture

Consider interpolation by L. The zero-stuffed signal v is mostly zeros, and when we convolve it with the filter h, every multiply by one of those zeros contributes nothing. At any given output index, only the taps of h that happen to land on the nonzero (original) samples do real work. As the output index advances through one period of L, a *different* subset of taps lights up each time. So instead of one length-N filter run at the high rate, we have L smaller *subfilters* (the polyphase components), each run at the low rate, and we interleave their outputs.

Intuition

Picture the filter's N taps as a comb, and the nonzero samples as teeth spaced L apart. Slide the comb past the teeth. At each shift, only the comb positions sitting exactly on a tooth matter; the rest hover over zeros. There are exactly L distinct alignments before the pattern repeats. Each alignment *is* a polyphase subfilter. Polyphase is simply the observation that you should evaluate each alignment separately and skip the empty positions.

22.5.2 The decomposition, written out

Split the filter's impulse response into L interleaved phases. The ρ-th polyphase component is

$$h_\rho[n] = h[nL + \rho], \qquad \rho = 0, 1, \ldots, L-1. \tag{22.8}$$

That is, h_0 takes taps $h[0], h[L], h[2L], \ldots$; h_1 takes $h[1], h[L+1], \ldots$; and so on. Together the L subfilters contain every tap of h exactly once. For interpolation, output phase ρ is produced by filtering the *original* (not zero-stuffed) signal with h_ρ:

$$y[mL + \rho] = \sum_k x[k]\, h_\rho[m - k] = (x * h_\rho)[m]. \tag{22.9}$$

Each of the L subfilters has about N/L taps and runs at the input rate, so the total work is $\sim N$ multiplies per input sample — the same as one full filter run, but with the $\times L$ rate inflation completely removed. The zeros are never multiplied because they were never created.

For decimation by M the dual statement holds: split the *input* into M phases, filter each by a subfilter, and *sum* the results to form each kept output. Again you compute only outputs you keep, at $\sim N/M$ multiplies per input sample.

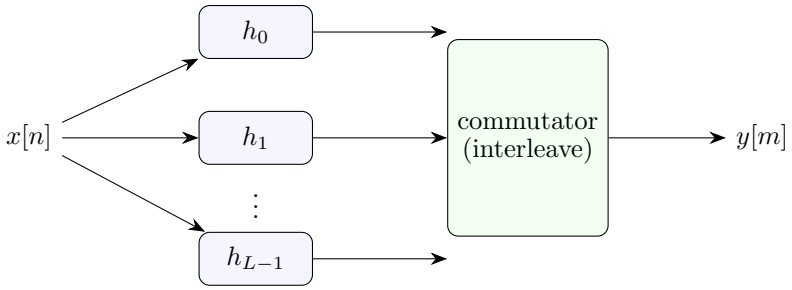


Figure 22.4: Polyphase interpolator. The input drives L short subfilters $h_0, \ldots, h_{L-1}$ in parallel at the low rate; a commutator interleaves their outputs to form the high-rate stream. No zeros are ever multiplied.

Why it works

Why is this exact and not an approximation? Because zero-stuffing followed by convolution is a *linear* operation, and (22.9) is just that same linear map re-grouped. Multiplying by zero and adding the result is algebraically removable without changing the sum. Polyphase reorders the arithmetic so that the removable operations are never performed. The Noble identities (which let you move a filter through a $\downarrow M$ or $\uparrow L$ block) are the formal license for

this reordering; the payoff is an order-M or order-L speedup with bit-for-bit identical output.

Example 22.4 (Tap count saved)**.** A 48-tap interpolation filter at $L = 4$ run the naive way costs 48 multiplies for *each* high-rate output, i.e. 192 per input sample. Polyphase splits it into four 12-tap subfilters; producing all four output phases costs $4 \times 12 = 48$ multiplies per input sample — a $4\times$ saving, exactly the upsampling factor.

22.5.3 A minimal polyphase interpolator in code

```python
import numpy as np

def polyphase_upsample(x, h, L):
    """Upsample x by integer L using prototype lowpass h (length ~ N*L)."""
    h = np.asarray(h, float)
    pad = (-len(h)) % L                  # pad so length is a multiple of L
    hh = np.concatenate([h, np.zeros(pad)])
    phases = hh.reshape(-1, L).T         # phases[rho] = h[rho], h[rho+L], ...
    outs = [np.convolve(x, phases[rho]) for rho in range(L)]
    y = np.empty(L * len(outs[0]))
    for rho in range(L):
        y[rho::L] = outs[rho]            # interleave: y[mL+rho] = outs[rho][m]
    return L * y                         # gain L compensates energy spreading
```

Listing 22.1: Polyphase upsample by L; no zeros are multiplied.

For everyday work you would not hand-roll this; SciPy's `resample_poly` does rational L/M resampling with an internally designed polyphase filter, and `scipy.signal.decimate` handles integer-M decimation with the anti-alias filter built in. The code above exists so you can *see* that the library is doing nothing magical — just the interleave-and-skip we drew.

22.6 Arbitrary and time-varying rates

Sometimes the ratio is not nicely rational, or it drifts: two sensors with independent crystal oscillators are never *exactly* at their nominal rates, so a stream nominally at 100 Hz might really be 100.012 Hz and wandering with temperature. Two practical tools:

- **Farrow / fractional-delay interpolation.** Fit a low-order polynomial (often cubic) through a few neighboring samples and evaluate it at the desired fractional position $\mu \in [0, 1)$ between input samples. The same coefficient structure works for *any* μ, so the output instant can be placed anywhere — ideal when the resampling phase advances by a non-integer step each output. This is the workhorse of software-defined radio symbol timing recovery.
- **Rational approximation.** Pick L/M close enough to the true ratio that the residual error is below your tolerance, then use polyphase. For 100.012/100 a ratio like 25003/25000 is overkill-accurate; a coarser one is usually fine.

Try it yourself

Generate a 7 Hz sinusoid sampled at 200 Hz. (1) Decimate to 50 Hz the wrong way (`x[::4]`) and the right way (`scipy.signal.decimate(x,4)`); overlay their spectra. (2) Add a 90 Hz tone before decimating and watch where it lands in each case — the naive version aliases it to $90 - 4 \cdot 25 = -10$ Hz, i.e. a phantom 10 Hz line that the proper filter removes. Seeing the phantom appear is the whole lesson of this chapter in one plot.

22.7 Why this matters for fusing sensors at different rates

Toward the wrist tracker

The wrist controller does not have one signal — it has a small zoo of them, each on its own clock. A typical build streams:

- accelerometer + gyroscope (IMU) at, say, 208 Hz;
- magnetometer at a slow 50 Hz (mag ODRs are usually low);
- if present, an EMG or force-myography (FMG) front end at 1000+ Hz, because muscle activity carries energy up to several hundred Hz.

The sensor-fusion filter (orientation estimator) and the gesture classifier both want a *single* synchronous frame rate — often 100 Hz, a comfortable grid for wrist motion whose deliberate content sits below ~ 15 Hz. So every stream must be resampled onto that common 100 Hz grid before fusion. The IMU is *decimated* (anti-alias at 50 Hz, then down); the magnetometer is *interpolated* up; the EMG is heavily decimated, but only *after* you have extracted its high-frequency features (envelope, RMS, band powers), because those features are what you actually fuse at 100 Hz — you do not feed raw 1 kHz EMG into a 100 Hz model and expect its fast content to survive.

Get the resampling wrong and the fusion silently degrades: an un-anti-aliased IMU decimation folds motor/tremor vibration into the gesture band and the classifier learns to fire on it; a magnetometer interpolated with a staircase hold injects step glitches that the orientation filter reads as sudden rotations. The boring, unglamorous resampling stage is where a lot of "my gesture detector is twitchy" bugs actually live.

Common trap

Resampling moves samples onto a common rate; it does *not* align them in absolute time. Two sensors started at different instants, or read over a bus with different latencies, are time-*offset* even at the same rate. You must also estimate and correct that offset (a fixed delay, or a slowly varying one if the clocks drift) — typically by cross-correlating a shared motion event, or by timestamping every sample at acquisition. Same-rate is necessary but not sufficient for "synchronized."

Why this matters

Be clear-eyed about what resampling can and cannot buy you at the wrist. Resampling fixes *rate* mismatches; it adds no information. A wrist-worn IMU sampled and resampled to any rate still cannot recover continuous individual finger-joint angles — the wrist simply does not move enough, per-finger, for the IMU to see it. The IMU gives you coarse wrist orientation and discrete gross gestures. Finger-level discrimination needs EMG or FMG on the forearm (and even then it is classification of a handful of states, not a continuous joint readout); true continuous finger pose needs sensors on the fingers themselves, such as instrumented rings. Clean resampling makes whatever signal you *do* have usable across sensors; it cannot conjure a degree of freedom the sensor never measured.

Build it (hardware)

A pragmatic on-device pipeline for a Cortex-M class MCU (e.g. the nRF52840 or an ESP32 reading the IMU over SPI):

1. Read each sensor in its own ISR/queue at its native ODR; timestamp every sample from a single monotonic clock.
2. Per stream, run a short anti-alias FIR (a 16–32-tap polyphase decimator is plenty for IMU 208 $\rightarrow$ 104 $\rightarrow$ 52 in stages — cascading two $\downarrow 2$ stages is cheaper than one $\downarrow 4$, because each stage's filter only has to be sharp relative to its own band).
3. Resample slow streams up to the common grid with linear interpolation if CPU is tight; the magnetometer changes slowly enough that linear is usually fine. Reserve windowed-sinc for anything whose passband you care about near Nyquist.
4. Emit one fused frame per output tick; feed that to the orientation filter and the classifier.

Cascaded half-band filters are the standard trick: a half-band FIR has nearly half its taps equal to zero, so a $\downarrow 2$ half-band stage is astonishingly cheap, and several in series give large decimation factors at minimal cost. CIC (cascaded integrator-comb) filters are an even cheaper, multiplier-free option for very large integer factors — common in the decimation stage of sigma-delta ADCs — at the price of a droopy passband you must later compensate.

Application

Rational resampling is everywhere once you look:

- **Audio.** Every time a 44.1 kHz CD track plays through a 48 kHz sound card, a 160/147 polyphase resampler runs. Sample-rate converters are a whole product category precisely because doing it transparently is hard.
- **Software radio.** A wideband receiver samples at tens of MHz, then decimates hard to bring one narrow channel down to a manageable baseband rate — multi-stage polyphase/CIC decimation is the front end of essentially every SDR.
- **Biomedical.** ECG/EEG recorded at one rate are routinely resampled to a study's standard rate so records from different machines can be compared; done carelessly it shifts QRS timing.
- **Motion / wearables.** Exactly our case: aligning IMU, mag, and EMG/FMG onto one

grid for fusion.

History

The polyphase viewpoint and the multirate "Noble identities" were crystallized in the 1970s–80s, with Maurice Bellanger's work on polyphase networks and the later synthesis in P. P. Vaidyanathan's *Multirate Systems and Filter Banks* (1993), which became the standard reference. The same multirate algebra underlies filter banks and, soon after, wavelets (the chapter ahead) — decimation and interpolation are the atoms from which multiresolution analysis is built. What looks like a humble rate-changing trick is the same machinery that powers JPEG- 2000 and MP3 subband coding.

22.8 Putting the pieces together

Strip away the algebra and the chapter is two sentences. To go *down* in rate, first lowpass to the new Nyquist so nothing folds, then keep every M-th sample. To go *up*, insert $L-1$ zeros and then lowpass to erase the images the zeros created. Rational changes do both with one shared filter, upsampling first; polyphase makes all of it cheap by never multiplying a zero or computing a sample you will discard. And in a real multi-sensor build, this is the quiet stage that decides whether your fused signal is clean or full of phantoms.

22.9 Exercises

Exercise 22.1. A signal occupies $|\omega| < 0.6\pi$. You decimate by $M = 2$ without filtering. Sketch the resulting spectrum from (22.2) and identify the band of frequencies that becomes corrupted by aliasing. What cutoff would the anti-alias filter have needed?

Exercise 22.2. A tone at $f_0 = 4300$ Hz is sampled at $f_s = 16$ kHz, then decimated by $M = 4$ *without* an anti-alias filter. To what frequency does it alias in the 4 kHz output? Now repeat with a proper anti-alias filter and state what happens to the tone instead.

Exercise 22.3. You upsample $x = (1, -1)$ by $L = 3$ using zero-order hold, then by $L = 3$ using linear interpolation. Write out the two output sequences (ignore filter delay). Which one has more high-frequency image energy, and why does the time-domain shape tell you so?

Exercise 22.4. Show from (22.5) that zero-stuffing alone (no filter) leaves the *values* at the original sample positions unchanged. Why, then, is the interpolation filter still necessary — what is wrong with the in-between zeros?

Exercise 22.5. Reduce the resampling ratios for the following and give (L, M): (a) $96000 \to 44100$; (b) $1000 \to 100$; (c) $208 \to 100$. For (c), state the cutoff $\min(\pi/L, \pi/M)$ of the shared filter.

Exercise 22.6. A 64-tap interpolation filter is used at $L = 8$. Count the multiplies per input sample for (a) the naive zero-stuff-then-filter implementation and (b) the polyphase implementation. What is the ratio, and how does it relate to L?

Exercise 22.7. Explain, using (22.2) and (22.5), why rational resampling must upsample *before* downsampling. Construct a concrete L/M and a signal where doing it in the wrong order loses recoverable bandwidth.

Exercise 22.8. A half-band lowpass filter has $h[n] = 0$ for all even n except the center tap. Argue why this makes a $\downarrow 2$ polyphase decimator especially cheap, and estimate the multiply savings versus a general filter of the same length.

Exercise 22.9. (Wrist tracker.) Your IMU runs at 208 Hz and your magnetometer at 50 Hz; both must reach a common 100 Hz grid. Give the resampling operation for each stream (factors and filter type), and explain which stream is at risk of aliasing and which is at risk of imaging.

Exercise 22.10. (What resampling cannot buy.) A teammate proposes resampling the wrist IMU to 1000 Hz so the model can "read individual finger movements." Explain in two or three sentences why the higher rate cannot recover finger-joint angles, and what sensor change would actually be required.

Exercise 22.11. (Code.) Modify the polyphase listing to perform decimation by M instead of interpolation: split the *input* into M phases, filter each by the corresponding h_ρ, and sum. Verify on a test signal that your output matches `scipy.signal.decimate` up to filter transients.

Exercise 22.12. Two nominally-100 Hz sensors actually run at 100.00 and 100.05 Hz. Over a 60-second capture, how many samples do they drift apart? Describe a resampling strategy (rational approximation or fractional-delay) that keeps them aligned, and state what additional information you need beyond the sample values.

Chapter 23

Modulation and I/Q Signals

So far every signal we have studied has lived at its "natural" place on the frequency axis. A 50 Hz hum sits at 50 Hz. A muscle twitch smears its energy across a few tens of hertz near zero. Speech crowds into the band below a few kilohertz. The Fourier transform told us *where* a signal lives; filtering let us keep or discard pieces of that real estate. This chapter is about the verb that acts on frequency itself: *modulation* – the deliberate act of *moving* a signal's spectrum from one place to another, and later moving it back.

That single idea – "multiply by a sinusoid and the spectrum slides" – is the beating heart of all radio, of every Wi-Fi packet and every FM station, of radar and sonar and ultrasound, of the lock-in amplifier on a lab bench, and, as we will see, of the very accelerometer and gyroscope chips that will sit on our wrist tracker. It is also the doorway to the single most important data structure in modern signal processing: the *I/Q* (in-phase / quadrature) pair, a complex number per sample that carries both the amplitude and the phase of a signal relative to a reference oscillator. Learn to see I/Q geometrically – as a little arrow spinning in the complex plane, slowed down until it crawls – and an enormous amount of "hard" engineering becomes obvious.

We build the chapter from the modulation property of the Fourier transform, watch spectra shift in pictures, then assemble amplitude modulation (AM) and frequency modulation (FM) as two ways of writing information onto a carrier. We meet the analytic signal as a preview (the next chapter, on the Hilbert transform, makes it rigorous), define the complex baseband and the I/Q representation, explain why I/Q is literally everywhere in radio and sensing hardware, and finish with the basic recipes for getting the message back out: coherent and envelope demodulation.

23.1 The one idea: multiplying by a sinusoid shifts the spectrum

Recall the modulation (frequency-shift) property of the Fourier transform, the dual of the time-shift property. If $x(t) \overset{\mathcal{F}}{\longleftrightarrow} X(\mathrm{j}\omega)$, then multiplying x by a complex exponential shifts its whole spectrum:

$$e^{\mathrm{j}\omega_c t}\, x(t) \;\overset{\mathcal{F}}{\longleftrightarrow}\; X\big(\mathrm{j}(\omega-\omega_c)\big). \tag{23.1}$$

The proof is one line – substitute into the transform integral and absorb the exponential into the kernel:

$$\int_{-\infty}^{\infty} \big[e^{\mathrm{j}\omega_c t}x(t)\big]\, e^{-\mathrm{j}\omega t}\,\mathrm{d}t = \int_{-\infty}^{\infty} x(t)\, e^{-\mathrm{j}(\omega-\omega_c)t}\,\mathrm{d}t = X\big(\mathrm{j}(\omega-\omega_c)\big). \tag{23.2}$$

Multiplying in time by $e^{\mathrm{j}\omega_c t}$ *translates* the spectrum to the right by ω_c. That is the whole game.

Intuition

Frequency is rotation rate (Chapter on sinusoids and phasors). The signal $x(t)$ is a recipe of little spinning phasors at various rates. Multiplying every one of them by $e^{\mathrm{j}\omega_c t}$ – itself a phasor spinning at rate ω_c – speeds each one up by exactly ω_c. A phasor that was spinning at ω now spins at $\omega + \omega_c$. So the entire collection of frequencies slides up the axis by ω_c, rigidly, without changing shape. *Modulation is just adding a constant to everybody's rotation rate.*

Real signals do not come with a single complex exponential; they come with cosines. A cosine is two phasors, one spinning each way, $\cos(\omega_c t) = \frac{1}{2}e^{\mathrm{j}\omega_c t} + \frac{1}{2}e^{-\mathrm{j}\omega_c t}$. So multiplying by a real cosine makes *two* shifted copies, one up and one down:

$$\cos(\omega_c t)\, x(t) \;\xleftrightarrow{\mathcal{F}}\; \tfrac{1}{2}X\big(\mathrm{j}(\omega-\omega_c)\big) + \tfrac{1}{2}X\big(\mathrm{j}(\omega+\omega_c)\big). \tag{23.3}$$

This is the most-used equation in all of communications. A baseband message $x(t)$ crowded around $\omega = 0$ becomes, after multiplication by a carrier cosine, two half-height replicas centered at $\pm\omega_c$. The picture is worth a thousand derivations.

Multiplying by $\cos(\omega_c t)$: the baseband triangle splits into two half-height copies

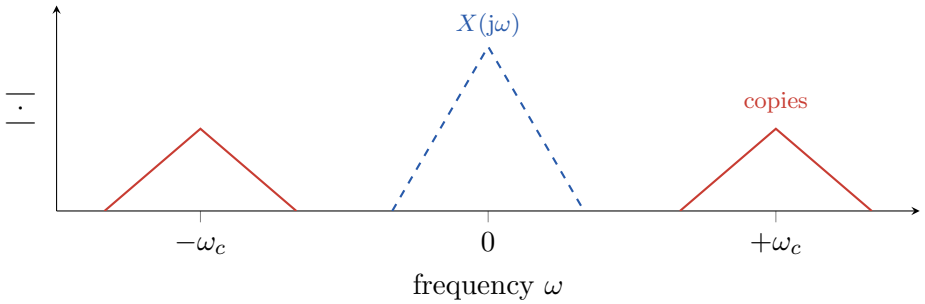


Figure 23.1: The modulation property in pictures. A baseband spectrum $X(\mathrm{j}\omega)$ (blue, dashed) multiplied by a carrier cosine produces two half-height, frequency-shifted replicas at $\pm\omega_c$ (red). Demodulation will be the act of sliding one copy back to the origin.

Why it works

Why does multiplication in time shift in frequency, while *convolution* in time multiplies in frequency? They are the same theorem read two ways. Multiplying $x(t)$ by $\cos\omega_c t$ in time is *convolving* $X(\mathrm{j}\omega)$ with the spectrum of the cosine, which is a pair of impulses at $\pm\omega_c$. Convolving any spectrum with an impulse at ω_c simply copies it, shifted, to ω_c. Two impulses, two copies. Everything in this chapter is the convolution theorem wearing different clothes.

Connection

The frequency-shift property is the Fourier dual of the time-shift property from the properties chapter. Both come from the same algebraic fact: $e^{\mathrm{j}at}$ is an *eigen-thing* of shifting. In the linear-algebra companion this is the statement that the Fourier basis diagonalizes the shift operator; here it tells us that a pure tone is the natural "carrier" for moving information

around the frequency axis.

23.2 Amplitude modulation: writing on the height of a carrier

The oldest way to send a message on a radio wave is to let the message control the *strength* of a fast oscillation. Pick a carrier frequency ω_c far above anything in your message $m(t)$ (think ω_c at 1 MHz, m at audio rates below 5 kHz). *Double-sideband suppressed-carrier* AM is the bare product

$$s_{\mathrm{DSB}}(t) = m(t)\,\cos(\omega_c t), \tag{23.4}$$

whose spectrum, by (23.3), is two copies of $M(\mathrm{j}\omega)$ at $\pm\omega_c$. It is spectrally efficient but a receiver cannot find the carrier phase from the signal alone, because when $m(t)$ goes negative the carrier flips phase by 180°.

Classic broadcast AM fixes this by adding a constant so the envelope never crosses zero:

$$s_{\mathrm{AM}}(t) = A\big[1 + \mu\, m(t)\big]\cos(\omega_c t), \qquad |m(t)| \le 1, \quad 0 < \mu \le 1. \tag{23.5}$$

Here μ is the *modulation index*. As long as $1 + \mu m(t) > 0$, the slow *envelope* $A[1+\mu m(t)]$ traces the message exactly, riding on the fast carrier. A receiver can then recover m by the crudest possible circuit – a diode and a capacitor that follow the peaks. That is why AM radio was buildable in 1920 with a crystal and a coil.

AM: a fast carrier whose height is shaped by the slow message envelope

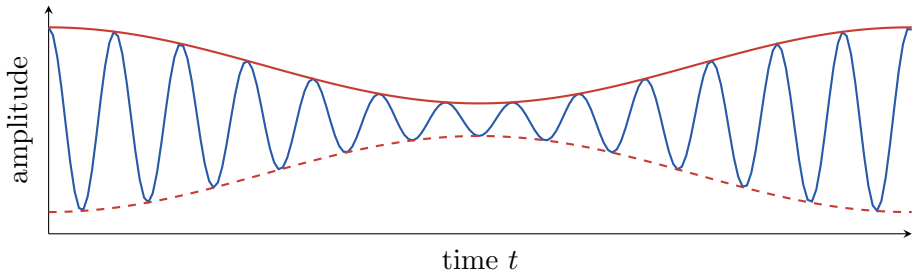


Figure 23.2: Amplitude modulation with $\mu = 0.7$. The blue waveform is the transmitted $s_{\mathrm{AM}}(t)$; the red curve is the envelope $A[1+\mu m(t)]$ that a diode detector follows. Because the envelope stays positive, the message is read off directly.

Example 23.1 (Sidebands of a single tone)**.** Let the message be a pure tone $m(t) = \cos(\omega_m t)$ with $\mu = 1$. Then

$$s_{\mathrm{AM}}(t) = A\cos(\omega_c t) + \tfrac{A}{2}\cos\big((\omega_c + \omega_m)t\big) + \tfrac{A}{2}\cos\big((\omega_c - \omega_m)t\big).$$

The spectrum is three pairs of lines: the *carrier* at $\pm\omega_c$ plus an *upper sideband* at $\pm(\omega_c + \omega_m)$ and a *lower sideband* at $\pm(\omega_c - \omega_m)$. All the *information* lives in the sidebands; the carrier line carries none of it (it is a constant) yet consumes most of the power. That waste is exactly what DSB-SC and single-sideband (SSB) schemes attack by removing the carrier, and one sideband too.

Common trap

Overmodulation. If $\mu > 1$, the quantity $1 + \mu m(t)$ dips below zero and the envelope "folds." A simple envelope detector then reproduces $|1 + \mu m(t)|$, a rectified, badly distorted version of the message. Keeping $\mu \leq 1$ is not a stylistic choice; it is the condition under which envelope detection is even valid. The same trap reappears in sensing: an amplitude-modulated vibration read by a peak-follower must not flip sign, or the recovered envelope is garbage.

Application

Where AM-style shifting hides. Beyond radio, the multiply-by-a-carrier trick is the basis of the *lock-in amplifier*: to measure a tiny signal buried in noise, you deliberately modulate it at a known frequency ω_c (chop a light beam, drive a bridge), then multiply the noisy measurement by $\cos \omega_c t$ and low-pass filter. Only energy that was sitting at ω_c survives the trip back to baseband; broadband noise is shifted but mostly filtered away. Ultrasound imaging, optical heterodyne detection, and capacitive touch sensing all run on the same principle.

23.3 Frequency modulation: writing on the rotation rate

AM writes the message onto the carrier's *height*. FM writes it onto the carrier's *rate of rotation*. This is the natural move once you have internalized that frequency *is* rotation rate: instead of nudging the length of the spinning arrow, nudge how fast it spins.

The clean way to say "how fast it is spinning right now" is the *instantaneous frequency*. Write any narrowband signal as $s(t) = A \cos \theta(t)$ with a time-varying phase $\theta(t)$. The instantaneous angular frequency is the rate the phase advances:

$$\omega_i(t) \;=\; \frac{\mathrm{d}\theta}{\mathrm{d}t}. \tag{23.6}$$

For an ordinary tone $\theta(t) = \omega_c t$ and $\omega_i = \omega_c$, a constant – the arrow spins at a steady rate. FM makes the rate wobble in step with the message:

$$\omega_i(t) = \omega_c + k_f\, m(t) \quad \Longrightarrow \quad s_{\mathrm{FM}}(t) = A \cos\!\Big(\omega_c t + k_f \int_0^t m(\tau)\, \mathrm{d}\tau\Big). \tag{23.7}$$

The integral appears because phase is the accumulation of frequency; this is the same "angle is the running total of angular velocity" relation from the calculus companion. The constant k_f sets how many radians-per-second of frequency swing you get per unit of message.

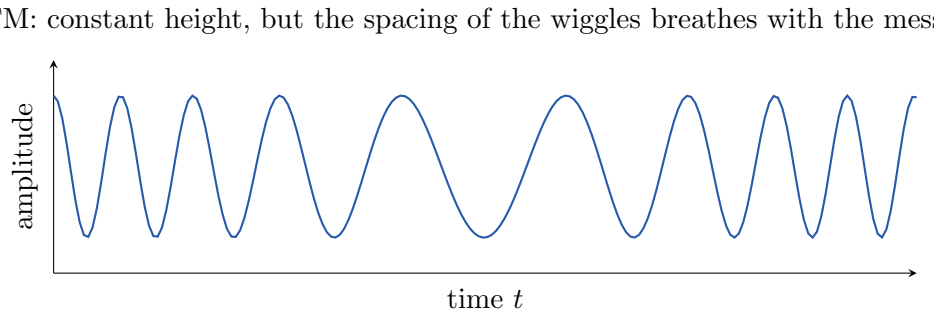


Figure 23.3: Frequency modulation. The amplitude is rock-steady; the *information* is in the changing spacing of the zero crossings – the wiggles bunch up where the instantaneous frequency is high and spread out where it is low.

The spectrum of FM is genuinely more complicated than AM. Even a single-tone message $m(t) = \cos \omega_m t$ produces *infinitely many* sidebands, spaced ω_m apart, whose heights are Bessel functions $J_n(\beta)$ of the modulation index $\beta = k_f A_m/\omega_m$. The useful, memorable summary is *Carson's rule*: the practical bandwidth of an FM signal is

$$B \;\approx\; 2(\Delta f + f_m), \tag{23.8}$$

where Δf is the peak frequency deviation and f_m the highest message frequency. Wider deviation buys more noise immunity at the cost of more bandwidth – the trade FM broadcasting makes deliberately, which is why FM sounds cleaner than AM but eats more spectrum.

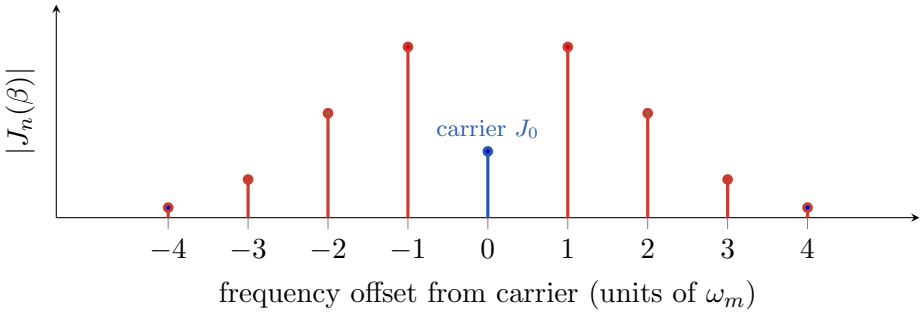


Figure 23.4: The line spectrum of a single-tone FM signal with modulation index $\beta = 2$. Sidebands sit at multiples of ω_m from the carrier, and their heights are the Bessel values $J_n(\beta)$. The carrier line (J_0) is not even the tallest – power has spilled into the sidebands. Carson's rule (23.8) captures where the comb has effectively died out.

History

Edwin Armstrong demonstrated wideband FM in 1933, against the prevailing belief (voiced by no less than the mathematician John Carson) that frequency modulation could only *narrow* bandwidth, never improve fidelity. Armstrong's insight was the opposite of intuition: by *spending* bandwidth on large frequency deviation, FM trades it for a dramatic gain in signal-to-noise ratio, because noise mostly perturbs amplitude, and an FM receiver throws amplitude

away. The legal and corporate battles that followed were as bitter as the engineering was brilliant.

Why it works

Why FM resists noise. Most noise adds to the *amplitude* of a received wave; it nudges the arrow's length far more than its rotation rate. An FM receiver begins by hard-limiting – clipping the signal to constant amplitude – which discards those length perturbations entirely, keeping only the timing of the zero crossings, where the message lives. AM has no such escape: its message *is* the amplitude, so amplitude noise lands directly on the recovered signal. This is the deep reason a wrist sensor that encodes information in *timing* (zero-crossing rate, frequency) can be more robust than one that encodes it in *level*.

23.4 The analytic signal: a preview

Both AM and FM forced us to talk about "the envelope" and "the instantaneous frequency" of a real signal. But a real cosine is ambiguous: $A\cos(\omega t)$ is two phasors spinning in opposite directions, and "the" rotation rate of a counter-rotating pair is not well defined. To make envelope and instantaneous frequency precise, we keep *only one* of the two phasors. That one-sided object is the *analytic signal*.

Given a real signal $s(t)$, its analytic signal is

$$s_a(t) = s(t) + \mathrm{j}\,\hat{s}(t), \tag{23.9}$$

where $\hat{s}(t)$ is the *Hilbert transform* of s – the next chapter is devoted to it. Geometrically, the Hilbert transform supplies the missing imaginary part so that the negative-frequency phasor cancels and only the positive-frequency one remains. The test case makes this vivid: for $s(t) = \cos\omega_c t$ we have $\hat{s}(t) = \sin\omega_c t$, so

$$s_a(t) = \cos\omega_c t + \mathrm{j}\sin\omega_c t = e^{\mathrm{j}\omega_c t}, \tag{23.10}$$

a single clean phasor. In the frequency domain, forming s_a deletes the negative- frequency half of the spectrum and doubles the positive half.

Once we have the single arrow $s_a(t) = a(t)\,e^{\mathrm{j}\phi(t)}$, the envelope and instantaneous frequency we used informally above become exact:

$$a(t) = |s_a(t)| \quad \text{(envelope)}, \qquad \omega_i(t) = \frac{\mathrm{d}}{\mathrm{d}t}\arg s_a(t) \quad \text{(instantaneous frequency)}. \tag{23.11}$$

What is really happening

The analytic signal is the bridge from real, two-sided, ambiguous waveforms to a single unambiguous spinning arrow. Everything we are about to call "I/Q" is just the analytic signal after we have slid it down to baseband: stop the fast spin, and what remains is a slowly drifting complex number whose length is the envelope and whose angle is the phase. The Hilbert chapter proves the cancellation; here we simply use the result.

23.5 I/Q: the complex baseband

Here is the central construction of the chapter. Take a real bandpass signal – a carrier at ω_c carrying some message – and multiply it by *both* a cosine and a sine at the carrier frequency. The two products are the *in-phase* (I) and *quadrature* (Q) components:

$$I(t) = \mathrm{LPF}\{\, s(t) \cdot 2\cos\omega_c t \,\}, \qquad Q(t) = \mathrm{LPF}\{\, s(t) \cdot (-2\sin\omega_c t) \,\}, \tag{23.12}$$

where LPF is a low-pass filter that throws away the copy that landed up at $2\omega_c$. "Quadrature" just means 90° apart: cosine and sine are the same wave a quarter cycle out of step. Bundle the two real numbers into one complex number, the *complex baseband* or *I/Q signal*:

$$z(t) = I(t) + \mathrm{j}\, Q(t). \tag{23.13}$$

To see what z is, write the bandpass signal in its general narrowband form, $s(t) = a(t)\cos\big(\omega_c t + \phi(t)\big)$. Expanding and discarding the $2\omega_c$ terms with the LPF gives the clean result

$$I(t) = a(t)\cos\phi(t), \qquad Q(t) = a(t)\sin\phi(t), \qquad \Longrightarrow \qquad z(t) = a(t)\, e^{\mathrm{j}\phi(t)}. \tag{23.14}$$

So $z(t)$ is exactly the analytic signal with the carrier spin removed: its *magnitude* is the envelope and its *angle* is the message phase. The single complex number per sample carries both pieces of information that a single real number cannot.

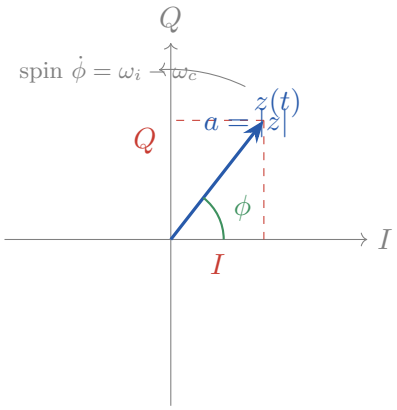


Figure 23.5: The I/Q sample as an arrow in the complex plane. Its horizontal projection is I, its vertical projection is Q, its length is the envelope $a(t)$, and its angle is the phase $\phi(t)$. After the carrier is removed, the arrow drifts slowly; its residual spin rate is the message's instantaneous frequency offset from the carrier.

Intuition

Imagine standing on a carousel turning at exactly the carrier rate ω_c, watching the received phasor. In that rotating frame the fast carrier spin is cancelled, and the phasor appears nearly still – drifting only as the *message* changes it. The I/Q mixer in (23.12) is precisely the act of stepping onto that carousel. $\cos\omega_c t$ measures the part of the arrow pointing "forward" along your motion, $\sin\omega_c t$ the part pointing "sideways." Together they pin down the arrow's tip in the rotating frame, which is exactly $z(t)$.

Formal

The two carriers $\cos\omega_c t$ and $\sin\omega_c t$ are *orthogonal* over any interval that is an integer number of carrier periods: $\int_0^T \cos(\omega_c t)\sin(\omega_c t)\,\mathrm{d}t = 0$. That orthogonality is why I and Q are independent channels – projecting onto one tells you nothing about the other. In the inner-product language of the linear-algebra companion, $\{\cos\omega_c t, \sin\omega_c t\}$ is an orthogonal basis for the two-dimensional space of sinusoids at ω_c, and (I, Q) are simply the coordinates of s in that basis. This is the same geometry as the real/imaginary split of a complex number.

23.5.1 Reconstructing the real signal from I/Q

The construction is reversible. Given the complex baseband $z = I + \mathrm{j}Q$, you rebuild the real passband signal by multiplying back up by the carrier and taking the real part:

$$s(t) = \Re\{z(t)\, e^{\mathrm{j}\omega_c t}\} = I(t)\cos\omega_c t - Q(t)\sin\omega_c t. \tag{23.15}$$

The two-multiplier structure on the right is the *quadrature modulator* (or "I/Q upconverter"), and the pair (mixer down, mixer up) is the backbone of every software- defined radio transmitter and receiver. Different choices of how you fill $I(t)$ and $Q(t)$ give every modulation scheme at once: put the message in $|z|$ and you have AM; put it in $\arg z$ and you have FM/PM; put discrete points in the (I, Q) plane and you have the digital constellations (QPSK, 16-QAM) behind Wi-Fi and LTE.

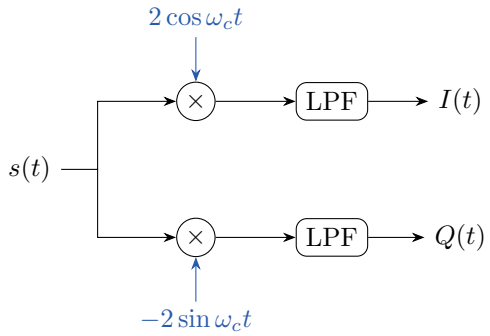


Figure 23.6: The I/Q downconverter (quadrature demodulator). The incoming real signal is multiplied by a cosine and a sine from the local oscillator; each product is low-pass filtered to drop the image at $2\omega_c$, yielding the baseband pair $I(t), Q(t)$. Run the diagram backward, summing instead of splitting, to get the upconverter of (23.15).

23.6 Why I/Q is everywhere

It is worth stating plainly why this complex-baseband representation took over radio and sensing, because the reasons are practical, not aesthetic.

1. Negative frequencies become real. A real signal's spectrum is symmetric: the negative-frequency half is a mirror of the positive half and carries no new information. A *complex* baseband signal has an *asymmetric* spectrum – it can have energy at $+3\,\mathrm{kHz}$ and none at $-3\,\mathrm{kHz}$. That asymmetry is exactly what lets I/Q distinguish a tone *above* the local oscillator from one *below* it.

A single real mixer cannot tell those apart (both fold to the same baseband frequency); the I/Q pair can, because the sign of Q relative to I encodes which side of the carrier the energy came from.

2. Half the sampling rate. A real bandpass signal of bandwidth B centered at a high ω_c would need a fast ADC to sample directly. After I/Q downconversion the information sits in two slow baseband channels, each centered on zero. Complex sampling at rate f_s covers a full bandwidth f_s, versus only $f_s/2$ for real sampling, so the (I, Q) pair captures the same band at half the per-channel rate of a direct real ADC. The hardware shrinks dramatically.

3. One representation for every modulation. As noted, AM, FM, PM, and all digital constellations are just trajectories of the same arrow $z(t)$ in the (I, Q) plane. A software-defined radio implements them all by changing arithmetic on $I + \mathrm{j}Q$, not by changing circuits.

4. The math is the analytic signal. I/Q *is* the analytic signal shifted to baseband, so all the clean facts – envelope is $|z|$, instantaneous frequency is the derivative of $\arg z$ – apply directly, with no $2\omega_c$ junk to filter mentally.

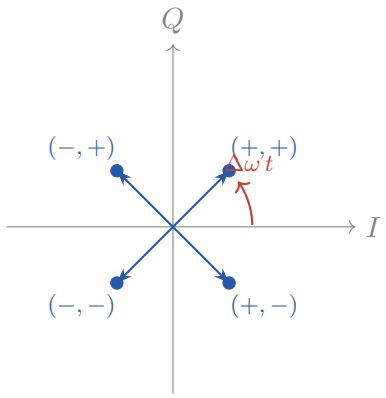


Figure 23.7: A QPSK constellation: each transmitted symbol is one of four fixed points in the (I, Q) plane, $(\pm 1, \pm 1)/\sqrt{2}$. A receiver local-oscillator frequency offset $\Delta\omega$ makes the whole constellation rotate at rate $\Delta\omega$ (red arc) – the same spinning that fades coherent DSB-SC in and out, seen as a picture.

Application

I/Q in sensing, not just radio. Doppler radar and ultrasound velocimetry read target *velocity* from the slow rotation rate of $z(t)$: a moving target shifts the echo's carrier, so the baseband arrow spins at the Doppler frequency, and the *sign* of its spin (only visible with I/Q) tells you approaching versus receding. The same trick powers 60 GHz gesture radars (e.g. Google's Soli) that watch a hand's micro-motions as patterns in the I/Q time series. MRI scanners record the spin signal directly as quadrature I/Q. Even a humble capacitive touch controller demodulates an excitation tone into I and Q to separate "how much capacitance" (in phase) from "how much loss" (quadrature).

Toward the wrist tracker

Our wrist controller does not transmit radio, but I/Q thinking pays off twice. First, several MEMS sensor front ends are *themselves* I/Q systems under the hood: a MEMS gyroscope

drives a proving mass at a fixed resonance and reads Coriolis-induced motion by *synchronous (I/Q) demodulation* against that drive tone – the very lock-in scheme of (23.12). Understanding I/Q demystifies why a gyro has a "drive frequency" in its datasheet and why temperature shifts of that frequency cause bias drift (Chapter on calibration).
Second, we can borrow the technique deliberately. Suppose we attach a small piezo or electrode that injects a steady tone into the forearm and listen for how a clench changes it (a bioimpedance or force-myography idea). Demodulating that tone into I and Q gives us a clean, slowly varying two-number feature – magnitude and phase of the tissue's response – far easier to threshold into a gesture than the raw high-frequency waveform. *The limit to keep in mind:* this yields a coarse "how hard is the wrist flexing" scalar, not individual finger joint angles. Continuous per-finger pose still requires sensors on the fingers (instrumented rings) or rich multi-channel EMG/FMG arrays; a single wrist tone, however cleverly demodulated, cannot recover what it never measured.

Build it (hardware)

Software I/Q demodulation in a few lines. If you sample a real signal at f_s and know the carrier f_c, you can do the I/Q split entirely in code – no analog mixer required. Multiply by the complex carrier and low-pass filter:

Listing 23.1: Numerical I/Q downconversion of a real bandpass signal.

```
import numpy as np
from scipy.signal import butter, filtfilt

def iq_demod(s, fs, fc, bw):
    n  = np.arange(len(s))
    lo = np.exp(-1j * 2*np.pi * fc * n / fs)   # carousel: spin backward at fc
    z  = s * lo                                # shift carrier copy down to 0 Hz
    b, a = butter(4, bw / (fs/2))              # low-pass to drop the 2*fc image
    I = filtfilt(b, a, z.real)
    Q = filtfilt(b, a, z.imag)
    return I + 1j*Q                            # complex baseband

# envelope and instantaneous frequency from the I/Q arrow:
# env   = np.abs(z)
# instf = np.diff(np.unwrap(np.angle(z))) * fs / (2*np.pi) + fc
```

On a microcontroller you would replace `filtfilt` with a small causal IIR or a moving average, and precompute the sine/cosine tables. The structure is identical: spin backward by the carrier, then smooth.

23.7 Demodulation: getting the message back

Demodulation is just (23.3) run in reverse: multiply the received signal by a carrier again to slide one of the sidebands back to the origin, then low-pass filter to keep it and discard the copy that flew up to $2\omega_c$. There are two flavors, distinguished by whether the receiver knows the carrier's phase.

23.7.1 Coherent (synchronous) demodulation

If the receiver has a local oscillator *locked* to the carrier's frequency and phase, recovery is exact. For DSB-SC, $s(t) = m(t)\cos\omega_c t$, multiply again:

$$s(t) \cdot 2\cos\omega_c t = 2m(t)\cos^2\omega_c t = m(t) + m(t)\cos 2\omega_c t. \tag{23.16}$$

The first term is the message back at baseband; the second is a copy at $2\omega_c$ that the low-pass filter removes. This is the I/Q downconverter of (23.12) with $\phi = 0$, and it is the gold standard: coherent detection is what gives digital radios their noise performance, and it is mandatory for any phase-bearing scheme (FM, PM, QAM).

Common trap

Phase error costs you. If the local oscillator is off the carrier phase by a fixed $\Delta\phi$, coherent DSB-SC recovers $m(t)\cos\Delta\phi$ – attenuated, and dead ($\cos 90° = 0$) if you are a quarter cycle off. A *frequency* error $\Delta\omega$ is worse: the recovered message gets multiplied by the slowly rotating $\cos(\Delta\omega\, t)$, fading in and out. In I/Q terms, a frequency offset makes the whole constellation *spin*. Real receivers spend real effort on a carrier-recovery loop (a phase-locked loop, or a digital equivalent) to drive $\Delta\phi, \Delta\omega \to 0$.

23.7.2 Envelope detection (noncoherent)

For full AM (23.5), where the message rides in a strictly positive envelope, you do not need the carrier phase at all. Just follow the peaks: rectify (take $|\cdot|$), then low-pass to smooth out the carrier ripple. The output tracks $A[1 + \mu m(t)]$, and a DC-blocking capacitor strips the constant to leave $m(t)$. This is the diode-and-capacitor detector of the crystal radio, and the geometric statement of "follow the peaks" is simply "read off $|z(t)|$," the length of the I/Q arrow.

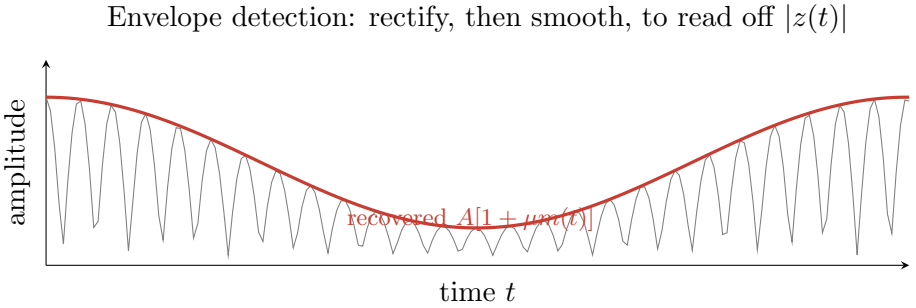


Figure 23.8: Noncoherent envelope detection. The gray trace is the rectified AM waveform; the smoothing filter rides its peaks to recover the envelope (red), which is the message plus a constant. No carrier phase knowledge is needed – only that the envelope never crossed zero.

23.7.3 FM demodulation

To demodulate FM, recover the instantaneous frequency, which by (23.11) is the derivative of the phase of the analytic (I/Q) signal. The cleanest digital recipe is: downconvert to I/Q, then differen-

tiate the unwrapped angle,

$$\hat{m}(t) \;\propto\; \omega_i(t) - \omega_c \;=\; \frac{\mathrm{d}}{\mathrm{d}t}\arg z(t). \tag{23.17}$$

A neat trick avoids the explicit arctangent: since $\frac{\mathrm{d}}{\mathrm{d}t}\arg z = \Im\{\dot{z}\,\bar{z}\}/|z|^2$, one differentiates z, multiplies by the conjugate, and takes the imaginary part. Hardware FM detectors approximate the same derivative with a slope filter or a phase-locked loop. Either way, the message is read from *how fast the I/Q arrow is spinning*, exactly the picture in Figure 23.5.

Connection

The "differentiate the angle" recipe leans on the chain rule and on the fact (calculus companion) that $\frac{\mathrm{d}}{\mathrm{d}t}\arg z = \Im(\dot{z}/z)$. The "multiply by the conjugate" form is the same expression cleared of its denominator's phase – a small piece of complex algebra that saves an expensive arctan and an unwrap on an embedded chip. It reappears in pitch trackers and in phase-vocoder time stretching in the audio chapters.

23.8 A worked numeric example

Let us push numbers through to make the spectra concrete. Take a message tone at $f_m = 1\,\mathrm{kHz}$, $m(t) = \cos(2\pi \cdot 1000\,t)$, and a carrier at $f_c = 20\,\mathrm{kHz}$.

DSB-SC. The product $s(t) = m(t)\cos(2\pi f_c t)$ has lines at $f_c \pm f_m = 19\,\mathrm{kHz}$ and $21\,\mathrm{kHz}$ (and their negatives). No energy at $20\,\mathrm{kHz}$: the carrier is suppressed. Bandwidth occupied: $2f_m = 2\,\mathrm{kHz}$.

Full AM, $\mu = 0.5$. $s(t) = A[1 + 0.5\cos(2\pi f_m t)]\cos(2\pi f_c t)$ adds a carrier line at $20\,\mathrm{kHz}$ of relative height 1 plus sidebands at 19 and $21\,\mathrm{kHz}$ of relative height 0.25 each. The fraction of power in the (informative) sidebands is $\frac{\mu^2/2}{1+\mu^2/2} = \frac{0.125}{1.125} \approx 11\%$ – a vivid reminder of why suppressing the carrier is attractive.

I/Q sanity check. Downconvert the DSB-SC signal by multiplying by $e^{-\mathrm{j}2\pi f_c t}$ and low-pass filtering. The $21\,\mathrm{kHz}$ line moves to $+1\,\mathrm{kHz}$, the $19\,\mathrm{kHz}$ line to $-1\,\mathrm{kHz}$; the negative-frequency images move to -41 and $-39\,\mathrm{kHz}$ and are filtered away. The surviving baseband $z(t) = \frac{1}{2}e^{\mathrm{j}2\pi f_m t} + \frac{1}{2}e^{-\mathrm{j}2\pi f_m t} = \cos(2\pi f_m t) = m(t)$ – real, because DSB-SC had a symmetric spectrum about the carrier. Had we sent only the upper sideband (SSB), z would be the single complex tone $\frac{1}{2}e^{\mathrm{j}2\pi f_m t}$, asymmetric, and the imaginary part of z would be nonzero – the I/Q pair *seeing* the one-sidedness that a real mixer would miss.

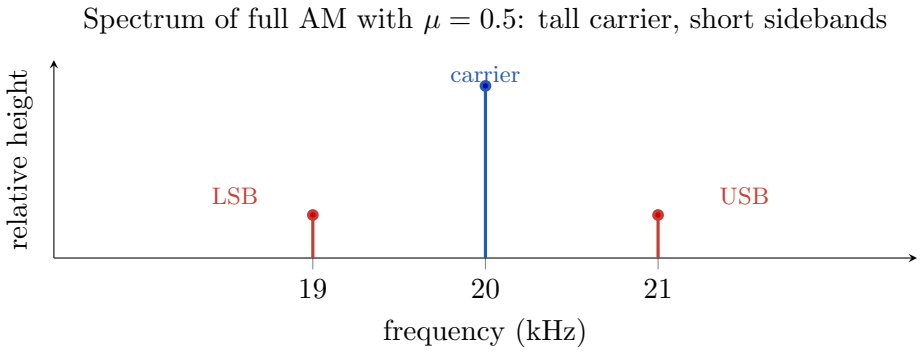


Figure 23.9: Positive-frequency line spectrum of the worked full-AM example. The carrier at 20 kHz dwarfs the two sidebands at 19 and 21 kHz, where all the information lives – the power inefficiency that motivates suppressed-carrier and single-sideband schemes.

Try it yourself

Generate the full-AM signal of the worked example at a sample rate of 200 kHz, take its FFT, and confirm the three lines at $19, 20, 21$ kHz with the 4:1 height ratio between carrier and sidebands. Then I/Q-downconvert with `iq_demod` from Listing 23.1 and verify that $|z(t)|$ reproduces $1 + 0.5\cos(2\pi \cdot 1000\, t)$. Finally, change μ to 1.5 and watch envelope detection fail where the envelope folds – the overmodulation trap, seen rather than merely warned about.

23.9 Summary

Modulation is the single idea that multiplying by a sinusoid slides a spectrum along the frequency axis – a corollary of the convolution theorem. AM writes the message onto the carrier's height (easy to detect, power-inefficient, vulnerable to amplitude noise); FM writes it onto the carrier's rotation rate (constant amplitude, wider bandwidth by Carson's rule, robust to amplitude noise). The analytic signal removes the ambiguity of real cosines by keeping one phasor, making "envelope" and "instantaneous frequency" precise. Sliding the analytic signal down to baseband gives the I/Q pair $z = I + \mathrm{j}Q$, a single complex number per sample whose length is the envelope and whose angle is the phase. I/Q is everywhere – it distinguishes positive from negative frequencies, halves sampling rates, unifies every modulation scheme, and underlies sensors from Doppler radar to the MEMS gyro on our own wrist. Demodulation is just modulation in reverse: coherent detection when phase is known, envelope detection when it is not, and angle- differentiation for FM. The next chapter makes the analytic signal rigorous by building the Hilbert transform that supplies its imaginary half.

23.10 Exercises

Exercise 23.1 (The shift, by hand)**.** Starting from the analysis integral, prove the modulation property (23.3) for $\cos \omega_c t$ by writing the cosine as $\frac{1}{2}(e^{\mathrm{j}\omega_c t} + e^{-\mathrm{j}\omega_c t})$ and applying (23.1) to each term. Identify which copy is the upper sideband and which is the lower.

Exercise 23.2 (Sidebands of two tones)**.** A message $m(t) = \cos(2\pi \cdot 800\, t) + \frac{1}{2}\cos(2\pi \cdot 2000\, t)$ amplitude- modulates a 50 kHz carrier in full-AM form with $\mu = 0.6$. List every spectral line (in

kHz) on the positive-frequency axis and its relative height. What total bandwidth does the signal occupy?

Exercise 23.3 (Power in the sidebands)**.** For full AM with a single sinusoidal message and modulation index μ, show that the fraction of total transmitted power carried by the two sidebands is $\dfrac{\mu^2/2}{1+\mu^2/2}$. Evaluate it at $\mu = 0.5$ and $\mu = 1$, and comment on why broadcasters nonetheless keep μ below 1.

Exercise 23.4 (Instantaneous frequency)**.** A signal is $s(t) = \cos(\omega_c t + \beta \sin \omega_m t)$. Compute its instantaneous frequency $\omega_i(t)$ using (23.6). What are the maximum and minimum instantaneous frequencies, and how do they relate to the peak deviation Δf in Carson's rule (23.8)?

Exercise 23.5 (Carson's rule)**.** An FM broadcast station has peak frequency deviation $\Delta f = 75\,\text{kHz}$ and a highest audio frequency $f_m = 15\,\text{kHz}$. Estimate the occupied bandwidth from Carson's rule (23.8). How does this compare to the 200 kHz channel spacing used in FM broadcasting, and why is the guard band reasonable?

Exercise 23.6 (I/Q of a known signal)**.** Take $s(t) = \cos\big((\omega_c + \omega_m)t\big)$ – a single upper-sideband tone. Apply the I/Q downconversion (23.12) (multiply by $2\cos\omega_c t$ and $-2\sin\omega_c t$, low-pass) and show that $z(t) = e^{\mathrm{j}\omega_m t}$. Now repeat for a *lower*-sideband tone $\cos((\omega_c - \omega_m)t)$ and show $z(t) = e^{-\mathrm{j}\omega_m t}$. Explain how the *sign* of z's rotation distinguishes the two, and why a single real mixer cannot.

Exercise 23.7 (Phase-error penalty)**.** A coherent DSB-SC receiver multiplies $s(t) = m(t)\cos\omega_c t$ by $2\cos(\omega_c t + \Delta\phi)$ and low-pass filters. Show that the recovered baseband is $m(t)\cos\Delta\phi$. At what $\Delta\phi$ does the output vanish, and what does this imply about the need for carrier recovery? Then redo the calculation with a frequency error $\Delta\omega$ and describe the result.

Exercise 23.8 (Envelope versus coherent)**.** Explain in two or three sentences why an envelope detector works for full AM ($\mu \le 1$) but fails for DSB-SC. Then explain why DSB-SC *can* be recovered coherently. Tie both answers to the geometry of the I/Q arrow $z(t)$ and whether its length alone, or its full position, carries the message.

Exercise 23.9 (FM discriminator without arctan)**.** Verify the identity $\dfrac{\mathrm{d}}{\mathrm{d}t}\arg z(t) = \dfrac{\Im\{\dot z(t)\,\overline{z(t)}\}}{|z(t)|^2}$ for a complex signal $z(t) = a(t)e^{\mathrm{j}\phi(t)}$ with $a(t) > 0$. Explain why the numerator alone (without dividing by $|z|^2$) is an adequate FM discriminator when the envelope is constant, and why this is attractive on a microcontroller.

Exercise 23.10 (Numerical experiment)**.** Using Listing 23.1, synthesize an FM signal whose message is a 200 Hz sine, carrier 10 kHz, deviation 1 kHz, sampled at 100 kHz. Downconvert to I/Q, recover the instantaneous frequency by differentiating the unwrapped angle, and plot it against the true message. Where does the recovered message degrade, and what filter bandwidth in `iq_demod` gives the cleanest result?

Exercise 23.11 (Lock-in / synchronous detection)**.** A tiny sensor signal $\varepsilon\cos(\omega_c t + \phi)$ is buried in broadband noise. You multiply the noisy measurement by $\cos\omega_c t$ and by $\sin\omega_c t$ and low-pass filter each. Show that the two outputs recover $\frac{\varepsilon}{2}\cos\phi$ and $\frac{\varepsilon}{2}\sin\phi$, so that magnitude and phase of the buried tone are obtained while out-of-band noise is rejected. Relate this to the MEMS-gyro readout described in the "Toward the wrist tracker" box.

Exercise 23.12 (Constellation thinking)**.** In QPSK, the transmitter sets (I, Q) to one of the four points $(\pm 1, \pm 1)/\sqrt{2}$ once per symbol (Figure 23.7). Sketch these on the (I, Q) plane. If the receiver's

local oscillator has a slow frequency offset $\Delta\omega$, describe what happens to the received constellation over time, and explain why this is the same phenomenon as the fading $\cos(\Delta\omega\, t)$ term in coherent DSB-SC demodulation.

Chapter 24

Wavelets and Multiresolution

The spectrogram was a triumph: by chopping a signal into short windows we finally saw *when* each frequency happened. But the spectrogram made one decision and froze it. We picked a window length, and that single choice fixed our time resolution and our frequency resolution *everywhere*, for *every* frequency, for the whole signal. A short window let us pin down the timing of a quick click but blurred the pitch of a slow hum. A long window resolved the hum beautifully but smeared the click across half a second. There was no escape: one window, one compromise, applied uniformly to a world that is anything but uniform.

Wavelets are what you get when you stop accepting that compromise. The core idea is so natural it feels inevitable once you see it: *use short windows to look at high frequencies and long windows to look at low frequencies.* Fast things deserve a fast look; slow things deserve a patient one. This is **multiresolution** – analysing a signal at many scales at once, each scale with the resolution appropriate to it. By the end of this chapter you will see wavelets geometrically (as projections onto a family of stretched, shifted little waves), you will work the Haar wavelet out by hand on real numbers, you will understand the filter-bank machine that makes the discrete wavelet transform run in linear time, and you will see why all of this is exactly the right lens for catching the sharp *onset* of a gesture on a wrist sensor – a job the Fourier transform is structurally bad at.

24.1 The tyranny of a single window

Let us be precise about what went wrong with the Short-Time Fourier Transform (STFT). The STFT slides a fixed window $w(t)$ of some length T_w along the signal and takes a Fourier transform inside each placement:

$$\mathrm{STFT}\{x\}(\tau, f) = \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, e^{-\mathrm{j}2\pi f t}\, \mathrm{d}t. \tag{24.1}$$

The window has a width in time, call it Δt, and because of the uncertainty principle its Fourier transform has a width in frequency Δf, with

$$\Delta t \cdot \Delta f \;\geq\; \frac{1}{4\pi}. \tag{24.2}$$

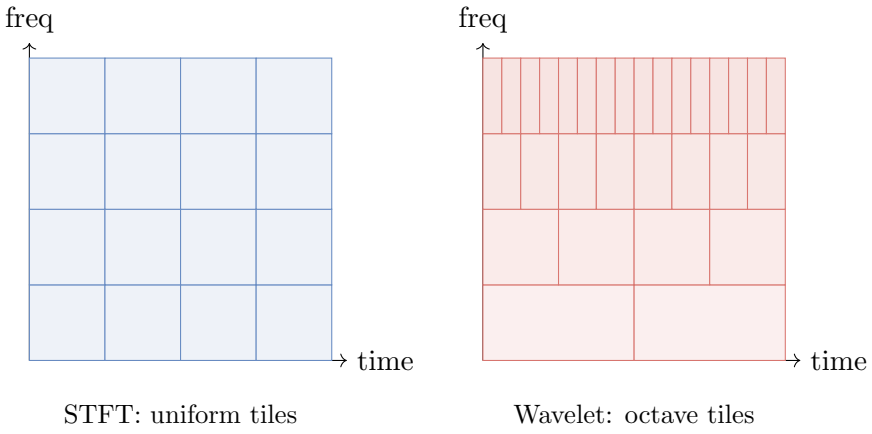
You cannot make both small. Worse, in the STFT the *same* Δt and the same Δf apply to the lowest frequency bin and the highest frequency bin alike. The time-frequency plane gets tiled by identical

rectangles, all the same shape (Figure 24.1, left).

Intuition

Picture the time-frequency plane as a sheet of graph paper you are trying to cover with tiles, where each tile is one "measurement" with some width in time and some height in frequency. The uncertainty principle says every tile must have at least a certain *area*. The STFT forces every tile to be the same *shape* – a fixed rectangle. Wavelets keep the area but vary the shape: tall-and-thin tiles up high (good timing, coarse pitch), short-and-wide tiles down low (good pitch, coarse timing). Same total cost, far better fit to how real signals are organized.

Why is the wavelet tiling the *right* one and not just *a* one? Because of how the physical world packs information. A high-frequency event – a tap, a plosive in speech, the snap when your wrist flicks – is necessarily brief; it is made of fast wiggles and it is over quickly, so you want fine *time* resolution there and you can afford coarse frequency resolution (there is not much room between "high" and "very high" that matters). A low-frequency component – the slow drift of your forearm, the fundamental of a bass note – is by its nature spread out in time, so fine *frequency* resolution is what pays off and coarse time resolution costs you nothing. Constant *relative* bandwidth, $\Delta f/f = \text{const}$, is the natural law, and it is exactly what musical octaves, the cochlea in your ear, and wavelets all use.



STFT: uniform tiles Wavelet: octave tiles

Figure 24.1: Tiling the time-frequency plane. The STFT (left) uses identical rectangles everywhere – one resolution for all frequencies. The wavelet transform (right) uses short-in-time, tall-in-frequency tiles at high frequencies and the reverse at low frequencies. Every tile has the same area (the uncertainty minimum); only the shape changes.

Connection

The "same area, different shape" picture is the time-frequency uncertainty principle from the Fourier-transform chapter, which is itself the Cauchy-Schwarz inequality from the linear-algebra companion in disguise. A signal and its transform cannot both be arbitrarily concentrated, because concentration is an inner-product statement and inner products obey Cauchy-Schwarz. Wavelets do not beat the principle – nothing does – they just *allocate* the unavoidable blur intelligently.

24.2 One little wave, stretched and shifted

The Fourier transform compares your signal against pure sinusoids $e^{\mathrm{j}2\pi ft}$ – infinite, eternal waves of a single frequency. That is why it has no sense of time: its yardsticks have no location. A wavelet replaces the eternal sinusoid with a *little wave*: a function $\psi(t)$ that wiggles for a while and then dies away to nothing (Figure 24.2). It is local. It has a position.

From this single *mother wavelet* ψ we manufacture an entire family by two operations: **scaling** (stretching or squishing it in time) and **translation** (sliding it along):

$$\psi_{a,b}(t) \;=\; \frac{1}{\sqrt{a}}\,\psi\!\left(\frac{t-b}{a}\right), \qquad a>0,\; b\in\mathbb{R}. \tag{24.3}$$

Here b is *where* we look and a is the *scale* – the stretch factor. Large a stretches ψ out, so it wiggles slowly: a low-frequency probe. Small a squishes it, so it wiggles fast: a high-frequency probe. The factor $1/\sqrt{a}$ out front is a normalization that keeps every member of the family at the same energy, $\|\psi_{a,b}\| = \|\psi\|$, so that comparisons across scales are fair.

What is really happening

Look at what (24.3) really does. As you make the probe wider (bigger a) to chase lower frequencies, it automatically gets *longer in time* – it covers more of the signal. As you make it narrower (smaller a) to chase higher frequencies, it automatically gets *shorter in time*. The window length is no longer a knob you set by hand; it is welded to the frequency you are asking about. That single equation *is* the multiresolution idea. Everything else in this chapter is consequences and bookkeeping.

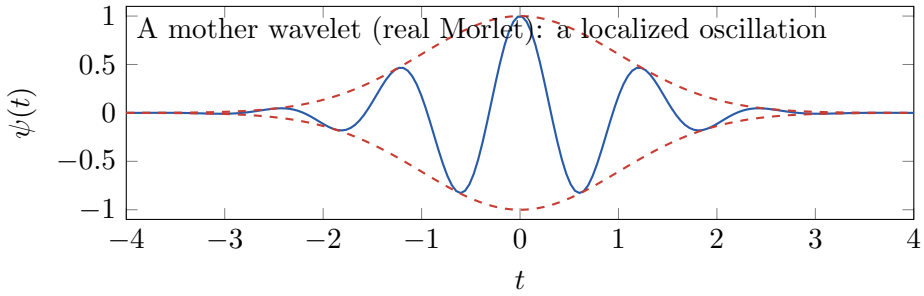


Figure 24.2: A mother wavelet $\psi(t) = \cos(5t)\,e^{-t^2/2}$: an oscillation multiplied by a decaying envelope (dashed). Unlike a Fourier sinusoid it has a definite location and finite reach. Stretching it (large a) makes it slower and longer; squishing it (small a) makes it faster and shorter.

Two properties make a function a legal mother wavelet, and both are intuitive. First, it must have *zero average*, $\int \psi(t)\,\mathrm{d}t = 0$ – it wiggles up as much as down, which is what "wave" means and which makes it blind to constant offsets. Second, it must be localized (decay to zero), which is what "little" means. Together these are summarized by the *admissibility condition* $\int_0^\infty |\Psi(f)|^2/f\,\mathrm{d}f < \infty$, where Ψ is the Fourier transform of ψ; the zero-mean requirement is exactly what keeps this integral finite at $f = 0$. A zero-mean, localized bump: that is all a wavelet is.

24.3 The continuous wavelet transform

Now we do the only thing we ever do in this book to measure "how much of a template is in a signal": we take an inner product. The **continuous wavelet transform** (CWT) of a signal $x(t)$ is its inner product with every member of the wavelet family:

$$W_x(a,b) \;=\; \langle x,\, \psi_{a,b}\rangle \;=\; \frac{1}{\sqrt{a}}\int_{-\infty}^{\infty} x(t)\,\psi^*\!\left(\frac{t-b}{a}\right)\mathrm{d}t. \tag{24.4}$$

The output $W_x(a,b)$ is a function of two variables – scale a and position b – and it answers a wonderfully concrete question: "how strongly does the signal, near time b, resemble a wiggle of scale a?" Plot $|W_x(a,b)|$ as an image with b across and a up the side and you have a **scalogram**, the wavelet cousin of the spectrogram.

Why it works

Why is this an inner product and not something fancier? Because the question "how much does x look like the template $\psi_{a,b}$?" is, geometrically, "what is the component of x along the direction $\psi_{a,b}$?" – and the component of a vector along a unit direction is exactly the inner product. The CWT is nothing but *projection*, the same operation behind the Fourier transform, behind least-squares, behind the dot products of your linear-algebra book. We have only changed the set of directions we project onto: from eternal sinusoids to localized, multi-scale wavelets. The machinery is identical; the yardsticks are better suited to changing signals.

Intuition

A useful mental animation: fix a scale a and slide the wavelet across the signal (vary b). You are doing a correlation – a convolution with a flipped wavelet – and you light up wherever the signal locally matches that wiggle. Now change a and do it again with a stretched wavelet, lighting up where the signal has slower structure. Stack all these slide-passes and you get the scalogram: fast structure sits high (small a), slow structure sits low (large a), and each lights up at the *time* it occurs.

Application

Scalograms shine on transients. A scratch on an old recording, a heartbeat's QRS complex in an ECG, a crack in the acoustic emission of a stressed beam, the chirp of a gravitational-wave inspiral (LIGO genuinely used wavelet-family methods in its analysis pipelines): all are short, sharp, broadband events that a fixed-window spectrogram either blurs in time or misses in frequency. The CWT catches them because its high-frequency probes are short enough to pin the moment of the event while its low-frequency probes still resolve the slow background.

24.4 From continuous to discrete: the dyadic grid

The CWT computes $W_x(a,b)$ for *every* real a and b. That is hugely redundant – neighbouring scales and positions overlap enormously, so we are storing the same information many times over. For computation and for compression we want a discrete, non-redundant sample of the (a,b) plane that

still captures everything. The inspired choice is the **dyadic grid**: sample scale in powers of two and sample position proportionally to scale.

$$a = 2^j, \qquad b = k\,2^j, \qquad j, k \in \mathbb{Z}. \tag{24.5}$$

Each step in j doubles the scale (one octave). At a coarser scale (larger j) the wavelet is wider, so we slide it in correspondingly larger steps – we do not need to sample position finely when the probe itself is broad. The discretized wavelets are

$$\psi_{j,k}(t) = 2^{-j/2}\,\psi\big(2^{-j}t - k\big)\,. \tag{24.6}$$

The miracle – and it is genuinely a miracle the first time you meet it – is that for the right choice of ψ this discrete, octave-spaced family is not just non-redundant but a complete *orthonormal basis* for all finite-energy signals:

$$\langle \psi_{j,k},\, \psi_{j',k'} \rangle = \delta_{jj'}\,\delta_{kk'}. \tag{24.7}$$

Every signal is a unique sum $x = \sum_{j,k} \langle x,\, \psi_{j,k}\rangle\, \psi_{j,k}$, with the wavelet coefficients $d_{j,k} = \langle x,\, \psi_{j,k}\rangle$ playing exactly the role the Fourier coefficients played – except each one is local in both time and scale. This is the **discrete wavelet transform** (DWT).

Common trap

Do not confuse the DWT with "the CWT sampled on a grid." Sampling the CWT generally gives a redundant frame, not a basis. The DWT is a separate, sharper object: a genuine orthonormal expansion that uses exactly N coefficients to represent N samples, with a perfect inverse and an $O(N)$ algorithm. You get that only for special, carefully engineered wavelets – Haar, Daubechies, and their kin – not for an arbitrary localized bump.

24.5 The scaling function: where the leftovers go

There is a loose end. Wavelets have zero mean – they are deliberately blind to the slow, near-constant trend of a signal. If we only ever project onto wavelets, we can never reconstruct the average value or the lowest, slowest drift. We need a partner that captures the leftovers: the **scaling function** $\varphi(t)$ (also called the *father wavelet*), a localized bump with *nonzero* mean that represents the coarse, smoothed version of the signal.

The two functions are bound together by two *refinement equations*, the deep structural heart of the whole subject. They say that the scaling function at one scale is a weighted combination of *shifted copies of itself at the next finer scale*:

$$\varphi(t) = \sqrt{2}\sum_{n} h[n]\,\varphi(2t - n), \tag{24.8}$$

$$\psi(t) = \sqrt{2}\sum_{n} g[n]\,\varphi(2t - n). \tag{24.9}$$

The coefficients $h[n]$ and $g[n]$ are short sequences of numbers – a low-pass filter h and a high-pass filter g – and astonishingly *they are all you need*. Give me the filter taps $h[n]$ (with $g[n]$ derived from them by a simple alternating-sign-and-reverse rule, $g[n] = (-1)^n h[1-n]$) and I can generate the scaling function, the wavelet, and the entire transform. The continuous functions φ and ψ are, in a real sense, just elaborate ways of drawing a picture of a handful of filter taps.

What is really happening

The refinement equation (24.8) is self-similarity made into an equation: "I am built out of smaller copies of myself." That is why wavelets are deeply related to fractals, and why a smooth-looking Daubechies scaling function is actually generated by an infinitely-recursive subdivision of a few numbers. The practical payoff is enormous: because the whole transform is encoded in short filters, we never have to handle the continuous functions at all on a computer. We just filter and downsample. That realization – continuous multiresolution equals discrete filtering – is what turned wavelets from a mathematical curiosity into a workhorse.

24.6 Haar: the wavelet you can do by hand

Abstractions deserve a concrete witness. The oldest and simplest wavelet, the **Haar wavelet** (Alfred Haar, 1909 – two decades before the word "wavelet" existed), is one you can compute with on paper. Its scaling function is a single unit box, and its wavelet is a box that goes up then down:

$$\varphi(t) = \begin{cases} 1 & 0 \le t < 1 \\ 0 & \text{else} \end{cases} \qquad \psi(t) = \begin{cases} +1 & 0 \le t < \tfrac{1}{2} \\ -1 & \tfrac{1}{2} \le t < 1 \\ 0 & \text{else.} \end{cases} \tag{24.10}$$

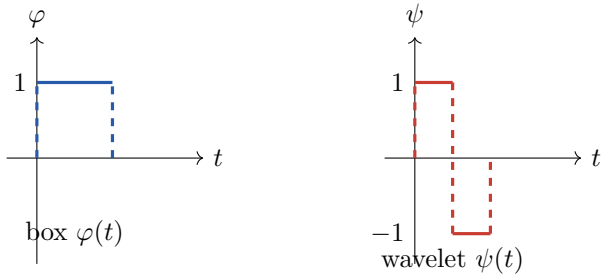


Figure 24.3: The Haar scaling function (a box, left) and the Haar wavelet (up-then-down, right). The box captures a local average; the up-then-down captures a local difference. Everything in multiresolution is, at heart, averages and differences.

What do these compute? Take a signal and a single Haar scaling function over an interval: the inner product is (up to a constant) the *average* of the signal on that interval. Take the Haar wavelet: the inner product is the *difference* between the average of the first half and the average of the second half – a local *detail*. Average and difference. That is the entire Haar transform, and it is something you already understand intuitively.

The Haar filter taps are correspondingly the simplest possible. From (24.8), the box $\varphi(t)$ equals the sum of two half-width boxes, $\varphi(t) = \varphi(2t) + \varphi(2t-1)$, which matched against (24.8) gives

$$h = \left[\tfrac{1}{\sqrt{2}},\ \tfrac{1}{\sqrt{2}}\right], \qquad g = \left[\tfrac{1}{\sqrt{2}},\ -\tfrac{1}{\sqrt{2}}\right]. \tag{24.11}$$

The low-pass filter h is "add the pair and rescale" (an average); the high-pass filter g is "subtract the pair and rescale" (a difference). The $1/\sqrt{2}$ keeps energy constant.

Example 24.1 (One step of the Haar DWT by hand)**.** Take the eight-sample signal

$$x = [\,4,\ 6,\ 10,\ 12,\ 10,\ 6,\ 4,\ 2\,].$$

Process it in adjacent pairs. For each pair (x_{2i}, x_{2i+1}) compute a scaled average $s_i = (x_{2i}+x_{2i+1})/\sqrt{2}$ and a scaled difference $d_i = (x_{2i} - x_{2i+1})/\sqrt{2}$.

Averages (the coarse approximation):

$$s = \tfrac{1}{\sqrt{2}}[\,4{+}6,\ 10{+}12,\ 10{+}6,\ 4{+}2\,] = \tfrac{1}{\sqrt{2}}[\,10,\ 22,\ 16,\ 6\,] \approx [\,7.07,\ 15.56,\ 11.31,\ 4.24\,].$$

Differences (the detail coefficients):

$$d = \tfrac{1}{\sqrt{2}}[\,4{-}6,\ 10{-}12,\ 10{-}6,\ 4{-}2\,] = \tfrac{1}{\sqrt{2}}[\,-2,\ -2,\ 4,\ 2\,] \approx [\,-1.41,\ -1.41,\ 2.83,\ 1.41\,].$$

We have turned eight numbers into four averages and four details – still eight numbers, no information lost (the transform is invertible: recover each pair from $x_{2i} = (s_i + d_i)/\sqrt{2}$, $x_{2i+1} = (s_i - d_i)/\sqrt{2}$). Notice the details are *small* where the signal is smooth and *large* where it changes fast. That smallness is the seed of compression.

Example 24.2 (The full multiresolution pyramid)**.** The point of the DWT is that we do not stop after one step – we recurse on the *averages*, leaving the details aside. Continuing Example 24.1 with $s \approx [7.07, 15.56, 11.31, 4.24]$:

Level 2 averages:

$$\tfrac{1}{\sqrt{2}}[7.07{+}15.56,\ 11.31{+}4.24] \approx [16.0,\ 11.0], \qquad \tfrac{1}{\sqrt{2}}[7.07{-}15.56,\ 11.31{-}4.24] \approx [-6.0,\ 5.0]$$

(the second pair being the level-2 details).

Level 3: average $\tfrac{1}{\sqrt{2}}[16.0{+}11.0] \approx 19.09$; detail $\tfrac{1}{\sqrt{2}}[16.0{-}11.0] \approx 3.54$.

The final transform stacks one coarsest average with all the details from every level:

$$[\underbrace{19.09}_{\text{coarse}},\ \underbrace{3.54}_{\text{lvl 3}},\ \underbrace{-6.0,\ 5.0}_{\text{lvl 2}},\ \underbrace{-1.41,\ -1.41,\ 2.83,\ 1.41}_{\text{lvl 1}}\,].$$

Eight inputs, eight outputs, organized from coarsest to finest. The single coarse number is (a scaled version of) the overall average; everything else is detail at ever-finer time scales. *This* is multiresolution: the signal decomposed into a coarse sketch plus a hierarchy of corrections.

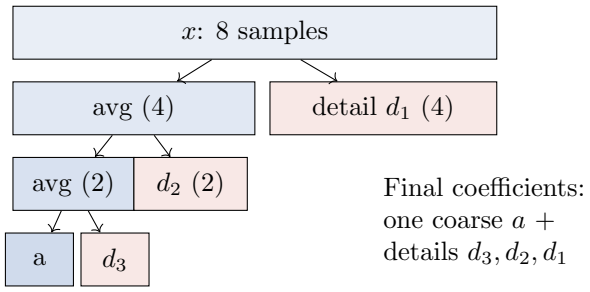


Figure 24.4: The multiresolution pyramid. Each level low-passes (averages) and high-passes (differences) the previous approximation, then keeps the details and recurses only on the average. The details accumulate; the approximation shrinks by half each level. This cascade is the discrete wavelet transform.

24.7 The filter-bank view: it is just filter, downsample, repeat

Examples 24.1 and 24.2 quietly revealed the algorithm. Each level does three things:

1. convolve the current approximation with the low-pass filter h (averaging),
2. convolve it with the high-pass filter g (differencing),
3. **downsample** both outputs by 2 (keep every other sample – we only needed one average and one difference per pair).

Keep the high-pass output as that level's detail; feed the low-pass output into the next level. This is **Mallat's fast wavelet transform** (Stephane Mallat and Yves Meyer, 1989), and it is the engine that made wavelets practical.

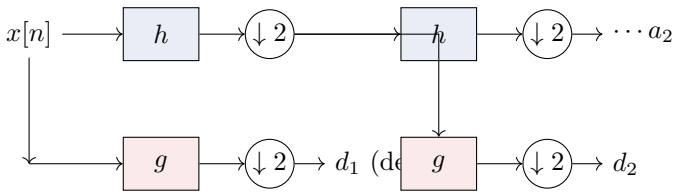


Figure 24.5: Mallat's filter bank. Low-pass (h) and high-pass (g) filtering followed by downsampling by 2 splits a signal into a coarse approximation and a detail band. The approximation feeds the next stage, doubling the analysed scale each time. Reconstruction (the synthesis bank) runs this in reverse with upsampling and matched filters.

Why it works

Why does downsampling not throw information away? After low-pass filtering, the approximation occupies only the bottom half of the frequency band; by the sampling theorem it is fully described at half the sample rate, so keeping every other sample is lossless. Likewise the detail occupies the top half and survives its own downsampling. The two half-rate streams together carry exactly the information of the full-rate input – this is the *perfect reconstruction* property, and it is why a careful filter-bank design (the filters must satisfy the *quadrature mirror* conditions) loses nothing. Filter, split, downsample: no redundancy, no loss.

The cost is delightful. Level 1 filters N samples; level 2 filters $N/2$; level 3 filters $N/4$; and so on. The total work is $N + N/2 + N/4 + \cdots < 2N$, so the entire DWT is $O(N)$ – even faster than the FFT's $O(N \log N)$. A multi-scale, time-localized analysis that runs in linear time is a remarkable thing to have on a small microcontroller.

History

The threads came together slowly. Haar built his orthonormal step-function basis in 1909. In the early 1980s the geophysicist Jean Morlet, analysing seismic echoes that the STFT handled poorly, introduced scaled-and-shifted "wavelets of constant shape" and worked with the physicist Alex Grossmann to put the CWT on firm footing. Yves Meyer found smooth orthonormal wavelets in 1985; Stephane Mallat connected wavelets to the filter banks engineers already used for subband coding, yielding the fast transform (1989); and Ingrid Daubechies (1988)

constructed families of compactly supported orthonormal wavelets with tunable smoothness – the workhorses used in JPEG 2000 and countless applications today. Pure math, geophysics, and signal-processing engineering converging on the same object is a sign you have found something real.

Try it yourself

PyWavelets makes the whole pyramid one call. Try it on a signal with a transient:

```
import numpy as np, pywt
# smooth ramp with a sudden spike at n=300 (a "gesture onset")
n = np.arange(512)
x = 0.5*np.sin(2*np.pi*3*n/512)
x[300:305] += 4.0                          # the transient
coeffs = pywt.wavedec(x, 'db4', level=5)   # multilevel DWT
for j, c in enumerate(coeffs):
    print(f"band {j}: {len(c):4d} coeffs, max |d| = {np.abs(c).max():.2f}")
# the FINEST detail band lights up sharply right at the spike -- and
# its index pinpoints WHEN. The smooth sine barely touches the details.
xr = pywt.waverec(coeffs, 'db4')           # perfect reconstruction
print("reconstruction error:", np.max(np.abs(x - xr)))
```

Swap `'db4'` for `'haar'` to feel the difference: Haar reacts instantly but blockily; Daubechies-4 is smoother and localizes a clean spike with fewer ringing artifacts.

24.8 Wavelets for compression and denoising

Look again at the transformed signal in Example 24.2: most of the detail coefficients are small. This is the universal experience with real signals – images, audio, sensor traces – because real signals are mostly smooth with occasional sharp features. The DWT *concentrates the energy* into a few large coefficients (the coarse trend and the details at genuine edges) and leaves a sea of near-zero coefficients elsewhere. This is called **sparsity**, and it is the foundation of both compression and denoising.

Compression: keep the largest coefficients, throw away (zero out) the tiny ones, and store only the survivors. Reconstruct from those; the error is tiny because you discarded only small numbers. This is precisely how **JPEG 2000** compresses images – a wavelet transform, then quantize and keep the significant coefficients – and it visibly beats old block-DCT JPEG at high compression, with graceful blur instead of ugly blocky artifacts (the FBI adopted a wavelet scheme for its fingerprint database for exactly this reason).

Denoising: white noise spreads its energy *evenly* across all wavelet coefficients, so it shows up as a uniform low-level fuzz, while the real signal lives in a few large coefficients. *Thresholding* – zeroing every coefficient below some level, then inverting the transform – erases the fuzz and keeps the signal. Donoho and Johnstone's *wavelet shrinkage* (early 1990s) made this rigorous and near-optimal. Crucially, unlike a simple low-pass filter, wavelet thresholding removes noise *without* smearing the sharp edges, because edges survive as a few large detail coefficients that sail over the threshold.

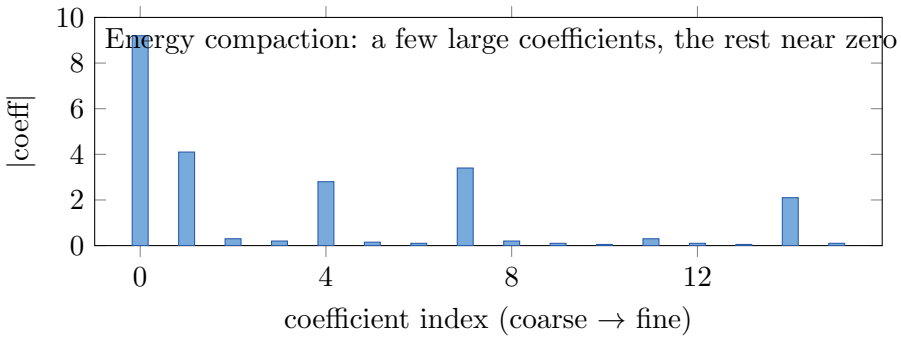


Figure 24.6: A typical wavelet coefficient magnitude profile for a piecewise-smooth signal. A handful of coefficients (the coarse trend, and details sitting on real edges) carry almost all the energy; the rest are nearly zero. Keeping the big ones compresses; thresholding away the small ones denoises.

Connection

Sparsity ties wavelets to the probability and statistics companion. "Most coefficients are tiny" is a statement about a heavy-tailed coefficient distribution; choosing a threshold is a hypothesis test (signal vs. noise) per coefficient; and the optimal threshold $\sigma\sqrt{2\ln N}$ for N samples of noise of standard deviation σ comes straight from the expected maximum of N Gaussian draws. Denoising is estimation, and estimation is probability.

24.9 Transients, onsets, and the wrist controller

We arrive at why wavelets belong in this book. The gestures that drive our instrument controller are defined by their *transitions*: the instant a flick begins, the snap of a tap, the moment a hold is released. These are *onsets* – short, sharp, broadband events sitting on top of slow background motion. They are precisely the signals the Fourier transform handles worst and the wavelet transform handles best.

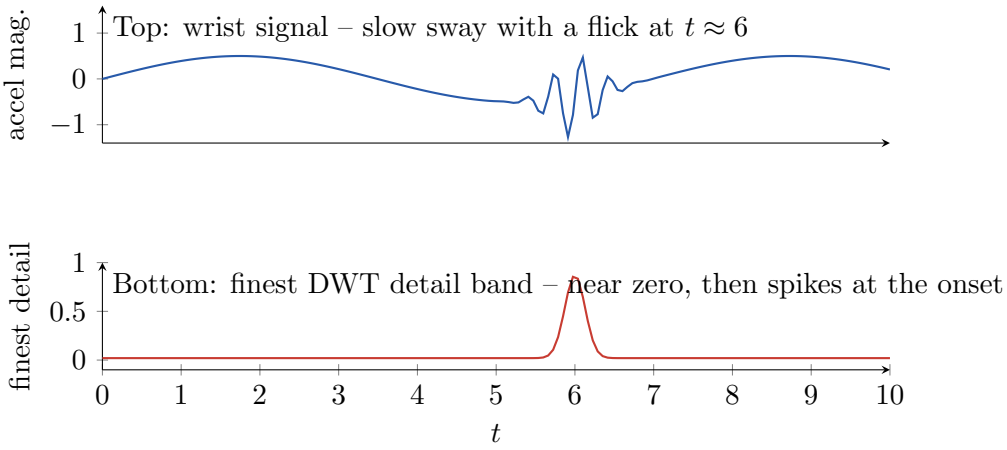


Figure 24.7: Why the DWT is a natural onset detector. The slow forearm sway lives in the coarse bands and barely touches the finest detail band, which stays near zero (bottom) until the flick begins – then it spikes, and the position of the spike pinpoints *when*. Thresholding this band gives a crisp gesture trigger.

Toward the wrist tracker

On a wrist-worn IMU, a deliberate flick produces a brief, high-frequency burst in the accelerometer and gyro – riding on the slow, low-frequency sway of the whole forearm. Run a few-level DWT on the incoming stream and watch the *finest detail band*: it stays near zero during smooth motion and *spikes* the instant a flick starts, with the coefficient's position telling you *when* down to a handful of samples (Figure 24.7). That is an onset detector with built-in time resolution, far crisper than waiting for an STFT frame to fill. A simple, effective gesture trigger: threshold the energy in the finest one or two detail bands; when it exceeds a baseline, fire a MIDI note-on. The coarse approximation band, meanwhile, gives you the slow posture/tilt for free in the same single transform – onset *and* pose from one $O(N)$ pass.

Build it (hardware)

A Haar (or short Daubechies) DWT is cheap enough to run live on an Arduino-class MCU at IMU rates (say 200 Hz). You do not even need the textbook pyramid: a *lifting* implementation of Haar is just averages and differences in place, no extra buffers. Sketch of the onset stage:

```
// one Haar lifting step over a small ring buffer of accel magnitude
// (low-pass = average, high-pass = difference), in place
for (int i = 0; i < N; i += 2) {
    float avg  = 0.70710678f * (buf[i] + buf[i+1]);
    float diff = 0.70710678f * (buf[i] - buf[i+1]);
    approx[i/2] = avg;       // feeds next level (slow posture)
    detail[i/2] = diff;      // finest detail = onset energy
}
// onset trigger: short-term energy of finest detail vs. baseline
float e = 0;
for (int k = 0; k < N/2; k++) e += detail[k]*detail[k];
if (e > GATE * baseline) midi_note_on();   // fire the gesture
baseline = 0.98f*baseline + 0.02f*e;        // slow adaptive floor
```

Tune `GATE` on real captures of your own flicks; the adaptive baseline keeps it working as the resting noise floor drifts.

Common trap

A reality check, repeated because it matters. A wrist IMU detecting a sharp onset tells you that *something abrupt happened* and roughly its intensity and timing. It does *not* tell you which finger moved or by how much. The DWT is a brilliant *onset and energy* detector; it does not conjure information the sensor never captured. Distinguishing individual fingers needs the muscle-level signals of EMG or force myography on the forearm; continuous finger *pose* needs sensors on the fingers themselves (rings or gloves). Wavelets sharpen *when* and *how hard*; they cannot upgrade a wrist IMU into a finger tracker.

Why this matters

Step back and see the arc of this part of the book. The Fourier transform gave us frequency but lost time. The STFT bought back time at a fixed exchange rate. Wavelets refused the fixed rate and let the exchange depend on the scale – fast looks for fast things, slow looks for slow things – and in doing so matched the way information is actually organized in the physical world, the way octaves are organized in music, and the way your own cochlea is organized. That a single idea – "stretch and shift one little wave" – yields an orthonormal basis, a linear-time algorithm, the best image codec, near-optimal denoising, and a clean gesture trigger is the kind of unreasonable effectiveness that makes signal processing beautiful.

24.10 Exercises

Exercise 24.1. Sketch the time-frequency tiling for an STFT with a 64-sample window and for a 4-level dyadic wavelet transform of the same 256-sample signal. For each, state the time resolution and frequency resolution in the lowest and highest analyzed bands. Explain in one sentence why the wavelet tiling is better suited to a signal that is a slow hum with an occasional sharp click.

Exercise 24.2. Compute the full Haar DWT (all levels) of $x = [2, 4, 4, 6, 8, 8, 6, 4]$ by hand, using the scaled average/difference rule from Example 24.1. Report the final coefficient vector (coarsest average followed by details from coarse to fine). Verify it has the same number of entries as x.

Exercise 24.3. Take your level-1 averages and details from the previous exercise and reconstruct the original eight samples using $x_{2i} = (s_i + d_i)/\sqrt{2}$ and $x_{2i+1} = (s_i - d_i)/\sqrt{2}$. Confirm you recover x exactly. What does this demonstrate about the DWT as a linear map?

Exercise 24.4. Show numerically (using your answer to Exercise 24.2) that the sum of squares of the wavelet coefficients equals the sum of squares of the original samples. Which property of the Haar transform guarantees this, and what is the name of the analogous Fourier result?

Exercise 24.5. The Haar wavelet ψ has integral zero, but the scaling function φ does not. Compute both integrals from (24.10). Explain why a signal's overall average is carried entirely by the coarsest scaling coefficient and never by any wavelet coefficient.

Exercise 24.6. Verify that the Haar filters in (24.11) satisfy $\sum_n h[n]^2 = 1$ (unit energy) and $\sum_n h[n]\,g[n] = 0$ (orthogonality of the low- and high-pass channels). Why are these two conditions exactly what a perfect-reconstruction, orthonormal filter bank needs?

Exercise 24.7. A signal has $N = 2^{20}$ samples. Counting one multiply-add per filter tap, give the approximate total operation count for a full Haar DWT and compare it to an FFT of the same signal ($\approx N \log_2 N$). By what factor is the DWT cheaper? Why does the geometric series $N + N/2 + N/4 + \cdots$ matter to your answer?

Exercise 24.8. You add white Gaussian noise of standard deviation $\sigma = 0.1$ to a signal of $N = 1024$ samples and take its DWT. Using the universal threshold $\lambda = \sigma\sqrt{2\ln N}$, compute λ. Describe the denoising procedure (threshold then invert) in three steps, and explain why this removes noise without blurring a sharp edge in the signal, unlike a low-pass filter.

Exercise 24.9. Using PyWavelets, generate a 3 Hz sine of length 512 with a 5-sample spike added at index 200. Run `pywt.wavedec` with `'db2'` at 4 levels. Plot each detail band stacked. Identify which band and which coefficient index pinpoint the spike, and confirm the spike's location matches index 200 after accounting for the downsampling at each level.

Exercise 24.10. Design (on paper) a wrist-IMU gesture-onset detector based on the finest Haar detail band, as in the *Build it* box. Specify: the input signal you would feed it (which IMU channel(s) and any preprocessing), how you would set the threshold from a calibration recording, and one false-trigger failure mode (e.g. bumping the table). State plainly one thing this detector *cannot* tell you about the hand, and what sensor you would add to get it.

Exercise 24.11. The wavelet transform has constant *relative* bandwidth ($\Delta f/f$ fixed), while the STFT has constant *absolute* bandwidth (Δf fixed). Compute the absolute bandwidth of a wavelet band centered at 10 Hz and one centered at 160 Hz if $\Delta f/f = 0.3$. Explain why this matches the way musical pitch and human hearing are organized, and why an STFT with a single window cannot reproduce it.

Chapter 25

The Hilbert Transform and the Analytic Signal

Look at a single note from a guitar string. The waveform wiggles up and down thousands of times a second, but riding on top of those wiggles is a slower, gentler shape: the note swells when you pluck it and then fades away. That slow shape is the *envelope*. Your ear barely hears the individual wiggles at all – it hears the *pitch* (how fast the wiggles go) and the *loudness* (how big the envelope is), and it tracks both as they change in time.

We already have a tool for asking "what frequencies are present?" – the Fourier transform. But the Fourier transform answers that question for the whole signal at once, or, with the short-time trick of Chapter 10, for a little window at a time. It does not hand us, sample by sample, a clean answer to two very physical questions:

- *How big is the signal right now?* – its **instantaneous amplitude**, also called the envelope.
- *How fast is it oscillating right now?* – its **instantaneous frequency**.

The Hilbert transform is the machine that lets us answer both, point by point in time. It does this by quietly completing a real signal into a *complex* one – the **analytic signal** – whose magnitude is the envelope and whose angle, unwound, gives the phase whose rate of change is the instantaneous frequency. The whole chapter is really one geometric idea: a real cosine is the shadow of a rotating arrow (a phasor), and if we can recover the arrow from its shadow, we can read off the arrow's length (amplitude) and how fast it is turning (frequency) at every instant.

This connects directly to the I/Q signals of Chapter 23 and to the AM/FM demodulation that underlies radio. It also gives us a robust envelope follower – exactly the kind of thing we will want when the wrist tracker has to decide "is a muscle burst happening right now, and how strong is it?" We will also be clear about what envelopes can and cannot tell us about the hand.

25.1 The shadow problem: recovering the arrow from its projection

Recall the picture that has run through this whole book. A cosine

$$x(t) = A\cos(\omega t + \phi) \tag{25.1}$$

is the real part of a rotating complex phasor:

$$z(t) = A\,e^{\mathrm{j}(\omega t+\phi)} = A\cos(\omega t+\phi) + \mathrm{j}\,A\sin(\omega t+\phi), \qquad x(t) = \mathrm{Re}\{z(t)\}. \tag{25.2}$$

The phasor is an arrow of length A spinning at ω radians per second. The signal $x(t)$ we actually measure – a voltage, a microphone sample, an accelerometer reading – is just the *shadow* of that arrow on the real axis (Figure 25.1).

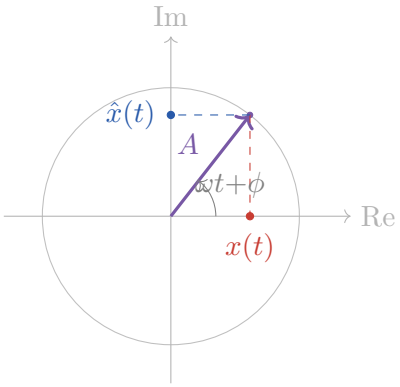


Figure 25.1: A real signal $x(t) = A\cos(\omega t+\phi)$ is the projection (shadow) of a rotating phasor onto the real axis. The *missing* coordinate is the projection onto the imaginary axis, $A\sin(\omega t+\phi)$. The Hilbert transform manufactures exactly that missing coordinate, $\hat{x}(t)$, so we can rebuild the whole arrow.

Here is the problem. Given only the shadow $x(t)$, can we recover the arrow $z(t)$? A shadow loses information: many different arrows cast the same shadow at one instant. What saves us is that the arrow is *turning steadily*. If we know the shadow for all time, the missing vertical coordinate is determined – it is the same wave, shifted a quarter turn. For our single cosine, the vertical coordinate is $A\sin(\omega t+\phi)$, which is just $A\cos(\omega t+\phi-\pi/2)$: the cosine delayed by 90°.

Intuition

A 90° phase shift is the whole secret. "Sine is cosine shifted by a quarter period." If we had a black box that took *any* signal and shifted every frequency component by exactly −90° – not by a fixed time, but by a fixed *angle* – then feeding it a cosine would give us a sine, and we could staple the two together into the complex arrow. That black box is the Hilbert transform.

Why "a fixed angle, not a fixed time"? Delaying by a fixed *time* τ shifts a component at frequency ω by $\omega\tau$ radians – more phase shift for higher frequencies. We want the *same* quarter-turn for every component, regardless of its frequency. That is a strange, frequency-dependent delay (a quarter of a long period is many milliseconds; a quarter of a short period is microseconds). No simple time delay can do it, which is why the Hilbert transform is its own special operator.

25.2 Defining the Hilbert transform by what it does to frequencies

The cleanest way to define the Hilbert transform is in the frequency domain, by stating exactly the phase rotation we just asked for. Let $X(\mathrm{j}\omega)$ be the Fourier transform of $x(t)$. The Hilbert transform

$\hat{x}(t) = \mathcal{H}\{x\}(t)$ is the signal whose spectrum is

$$\widehat{X}(\mathrm{j}\omega) = -\mathrm{j}\,\mathrm{sgn}(\omega)\,X(\mathrm{j}\omega), \qquad \mathrm{sgn}(\omega) = \begin{cases} +1, & \omega > 0, \\ 0, & \omega = 0, \\ -1, & \omega < 0. \end{cases} \tag{25.3}$$

That is the entire definition. Multiplying by $-\mathrm{j}$ is a rotation by -90° in the complex plane; multiplying by $+\mathrm{j}$ is $+90^\circ$. So Equation (25.3) says:

- **positive** frequencies get rotated by -90° (multiplied by $-\mathrm{j}$),
- **negative** frequencies get rotated by $+90^\circ$ (multiplied by $+\mathrm{j}$),
- the DC term ($\omega = 0$) is killed.

Figure 25.2 plots this phase response: flat at $-\pi/2$ for positive frequencies, flat at $+\pi/2$ for negative ones.

Why it works

Why opposite signs for positive and negative frequencies? Because a real cosine is built from *two* counter-rotating phasors, $\cos\theta = \frac{1}{2}e^{\mathrm{j}\theta} + \frac{1}{2}e^{-\mathrm{j}\theta}$. The $+\omega$ phasor spins counterclockwise; the $-\omega$ phasor spins clockwise. To turn cos into sin we must push the counterclockwise one back by 90° and the clockwise one forward by 90° – they have to be rotated in *opposite* angular senses so that their real parts add up to a sine instead of cancelling. The factor $-\mathrm{j}\,\mathrm{sgn}(\omega)$ encodes precisely that. Notice its magnitude is 1 everywhere (except DC): the Hilbert transform changes only phase, never the amount of any frequency. It is an all-pass 90° phase shifter.

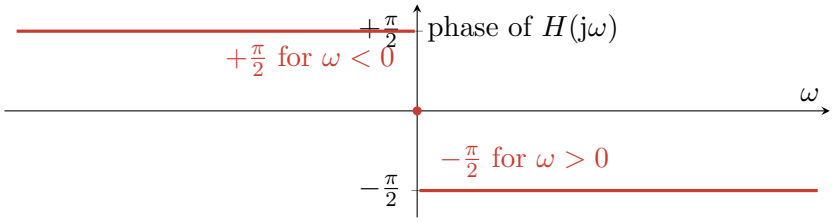


Figure 25.2: Phase response of the Hilbert transform, $H(\mathrm{j}\omega) = -\mathrm{j}\,\mathrm{sgn}(\omega)$. The magnitude is 1 at every frequency (an all-pass), and the phase is a flat $\mp 90^\circ$: positive frequencies rotate by $-\pi/2$, negative ones by $+\pi/2$, with DC killed. The same quarter-turn at every frequency is what no fixed time delay can achieve.

Let us check it on a cosine. The spectrum of $A\cos(\omega_0 t)$ is two spikes, $\pi A\,[\delta(\omega-\omega_0)+\delta(\omega+\omega_0)]$. Apply Equation (25.3): the spike at $+\omega_0$ is multiplied by $-\mathrm{j}$, the spike at $-\omega_0$ by $+\mathrm{j}$. Inverse-transforming, that is exactly the spectrum of $A\sin(\omega_0 t)$. So

$$\mathcal{H}\{\cos(\omega_0 t)\} = \sin(\omega_0 t), \qquad \mathcal{H}\{\sin(\omega_0 t)\} = -\cos(\omega_0 t) \quad (\omega_0 > 0). \tag{25.4}$$

Cosine becomes sine; sine becomes minus-cosine. Apply $\mathcal{H}$ twice and you have rotated everything by 180°, i.e. negated the signal: $\mathcal{H}^2 = -I$ (on signals with no DC). This is the operator analogue of $\mathrm{j}^2 = -1$, and it is the first sign that $\mathcal{H}$ is the right tool for building complex signals out of real ones.

25.2.1 The time-domain face: convolution with $1/(\pi t)$

Multiplication in frequency is convolution in time (the convolution theorem, Chapter 11). The inverse Fourier transform of the multiplier $-\mathrm{j}\,\mathrm{sgn}(\omega)$ is the function $1/(\pi t)$. Therefore

$$\hat{x}(t) = (h_{\mathcal{H}} * x)(t) = \frac{1}{\pi}\,\mathrm{p.v.}\!\int_{-\infty}^{\infty} \frac{x(\tau)}{t-\tau}\,\mathrm{d}\tau, \qquad h_{\mathcal{H}}(t) = \frac{1}{\pi t}. \tag{25.5}$$

The "p.v." is the Cauchy *principal value*: the kernel $1/(\pi t)$ blows up at $t = 0$, and we take the symmetric limit around the singularity so the infinity cancels itself. Geometrically, Equation (25.5) says $\hat{x}(t)$ is a weighted average of the whole signal, with weight $1/(t-\tau)$ – nearby samples pull hardest, with opposite signs on the two sides of t. It is fundamentally *nonlocal*: to know $\hat{x}$ at one instant in principle requires the signal at all instants. That is the price of demanding the same phase shift at every frequency.

History

David Hilbert (1862–1943) studied this transform around 1905 in the context of integral equations and what are now called Riemann–Hilbert problems – pure mathematics, nothing to do with radios. The engineering payoff came in the 1940s. Dennis Gabor (of holography fame) introduced the *analytic signal* in his 1946 paper "Theory of Communication," explicitly to give a precise meaning to the instantaneous amplitude and frequency of a signal. The mathematician's quarter-turn became the engineer's envelope detector.

25.3 The analytic signal: stapling the shadow to its partner

Now we assemble the complex arrow. Given a real signal $x(t)$, define the **analytic signal**

$$x_a(t) = x(t) + \mathrm{j}\,\hat{x}(t). \tag{25.6}$$

The real part is the original signal (our shadow); the imaginary part is its Hilbert transform (the manufactured partner). For a single cosine this gives exactly the phasor we started with:

$$x_a(t) = \cos(\omega_0 t) + \mathrm{j}\sin(\omega_0 t) = e^{\mathrm{j}\omega_0 t}. \tag{25.7}$$

We have recovered the rotating arrow from its shadow.

There is a clean frequency-domain way to see what just happened. The spectrum of the analytic signal is

$$X_a(\mathrm{j}\omega) = X(\mathrm{j}\omega) + \mathrm{j}\big(-\mathrm{j}\,\mathrm{sgn}(\omega)\big)X(\mathrm{j}\omega) = \big(1+\mathrm{sgn}(\omega)\big)X(\mathrm{j}\omega) = \begin{cases} 2X(\mathrm{j}\omega), & \omega > 0, \\ X(0), & \omega = 0, \\ 0, & \omega < 0. \end{cases} \tag{25.8}$$

The analytic signal is the original signal with its **negative-frequency half deleted** and its positive half doubled (Figure 25.3).

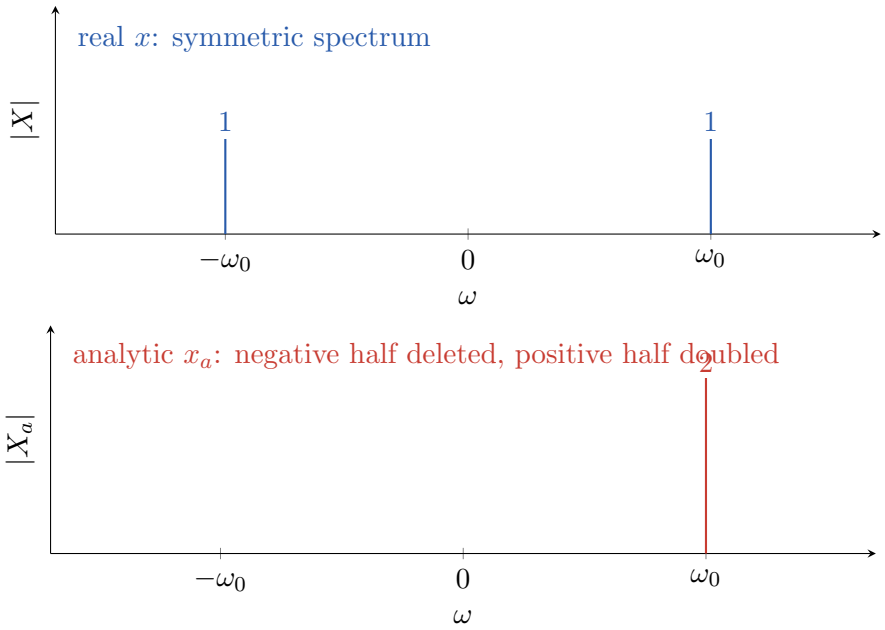


Figure 25.3: Top: a real cosine has two equal spectral spikes at $\pm\omega_0$. Bottom: its analytic signal keeps only the positive-frequency spike, doubled. A one-sided spectrum is what makes the time signal a clean rotating phasor instead of two counter-rotating ones.

What is really happening

This is the deep reason the analytic signal is so useful. A real signal is forced to have a *symmetric* spectrum (Hermitian: $X(-\mathrm{j}\omega) = \overline{X(\mathrm{j}\omega)}$), because it is the sum of counter-rotating phasor pairs. Those negative frequencies carry no new information – they are just mirror images – but they are what make a real cosine "stand still and breathe" rather than rotate. By throwing the mirror image away, the analytic signal becomes a signal that purely *rotates*. And a rotating thing has an unambiguous length (the envelope) and an unambiguous rate of turn (the instantaneous frequency). You cannot ask "how fast is $\cos(\omega t)$ rotating?" – it does not rotate, it oscillates. You *can* ask how fast $e^{\mathrm{j}\omega t}$ rotates. The Hilbert transform converts the unanswerable question into the answerable one.

Connection

This is the same one-sided-spectrum object you met as the *complex baseband* or I/Q signal in Chapter 23. There, a receiver mixed a real bandpass signal down with two local oscillators 90° apart to produce in-phase (I) and quadrature (Q) channels; the complex signal $I + \mathrm{j}Q$ is exactly an analytic signal (shifted to baseband). The Hilbert transform is the way to build $I + \mathrm{j}Q$ from a single real channel by computation rather than by a second mixer. Linear-algebra reader: deleting negative frequencies is an orthogonal projection onto the subspace of positive-frequency exponentials – the analytic-signal map is a projection followed by a doubling.

25.4 Instantaneous amplitude, phase, and frequency

Once we have the analytic signal in polar form,

$$x_a(t) = a(t)\, e^{\mathrm{j}\theta(t)}, \tag{25.9}$$

the two physical quantities fall right out of the geometry of the arrow:

$$a(t) = |x_a(t)| = \sqrt{x(t)^2 + \hat{x}(t)^2} \qquad \textbf{instantaneous amplitude} \text{ (envelope)}, \tag{25.10}$$

$$\theta(t) = \arg x_a(t) = \operatorname{atan2}\big(\hat{x}(t),\, x(t)\big) \qquad \textbf{instantaneous phase}, \tag{25.11}$$

$$\omega_i(t) = \frac{\mathrm{d}\theta}{\mathrm{d}t} \qquad \textbf{instantaneous (angular) frequency}. \tag{25.12}$$

The envelope $a(t)$ is the length of the arrow – how big the oscillation is right now, independent of where it happens to be in its cycle. The instantaneous frequency $\omega_i(t)$ is how fast the arrow is turning right now. Dividing by 2π gives the instantaneous frequency in hertz, $f_i(t) = \omega_i(t)/(2\pi)$.

Example 25.1 (Amplitude-modulated tone)**.** Let a slowly varying envelope $a(t) > 0$ multiply a fast carrier:

$$x(t) = a(t)\cos(\omega_c t), \qquad \omega_c \text{ large}.$$

If $a(t)$ varies slowly compared with the carrier (its spectrum sits near DC, well below ω_c), then the Hilbert transform passes the slow factor through almost untouched and only shifts the carrier: $\hat{x}(t) \approx a(t)\sin(\omega_c t)$. Then

$$x_a(t) \approx a(t)\cos(\omega_c t) + \mathrm{j}\, a(t)\sin(\omega_c t) = a(t)\, e^{\mathrm{j}\omega_c t},$$

so the recovered envelope is $|x_a(t)| = a(t)$ exactly, and the instantaneous frequency is $\mathrm{d}\theta/\mathrm{d}t = \omega_c$. The Hilbert transform has *demodulated* the AM signal: it pulled the message $a(t)$ off the carrier without any tuning, filtering, or knowledge of ω_c. This is the workhorse use of the analytic signal.

Common trap

The clean separation in Example 25.1 requires the **Bedrosian condition**: the envelope's spectrum and the carrier's spectrum must not overlap (the envelope must be *low*-pass and the carrier *high*-pass, with a gap between them). If the "envelope" wiggles as fast as the carrier, $\mathcal{H}\{a(t)\cos\omega_c t\} \neq a(t)\sin\omega_c t$, and the recovered $a(t) = |x_a(t)|$ is *not* the factor you wrote down. Worse, instantaneous frequency can then go *negative* or spike wildly – a sign that "one envelope and one frequency" is no longer a sensible description of the signal. The analytic signal always gives you *an* answer; it is meaningful only when the signal really is a single narrow-band component (one arrow, slowly changing). For multi-component signals, decompose first (band-pass filter, or empirical mode decomposition) and take one component at a time.

25.4.1 A worked numeric example: a chirp

Take a *linear chirp* – a tone whose frequency sweeps upward – with a Gaussian envelope:

$$x(t) = a(t)\cos\big(\phi(t)\big), \qquad \phi(t) = \omega_0 t + \tfrac{1}{2}\beta t^2, \qquad a(t) = e^{-(t/\sigma)^2}. \tag{25.13}$$

The true instantaneous frequency is, by definition of ϕ,

$$\omega_i(t) = \frac{\mathrm{d}\phi}{\mathrm{d}t} = \omega_0 + \beta t,$$

a straight line rising at rate β. Provided the envelope is slow and the sweep is gentle (Bedrosian again), the analytic signal recovers $a(t) = |x_a(t)| = e^{-(t/\sigma)^2}$ and $\omega_i(t) = \omega_0 + \beta t$ to good approximation. Figure 25.4 shows the chirp with the Hilbert envelope hugging its peaks, and the recovered instantaneous frequency tracking the rising line.

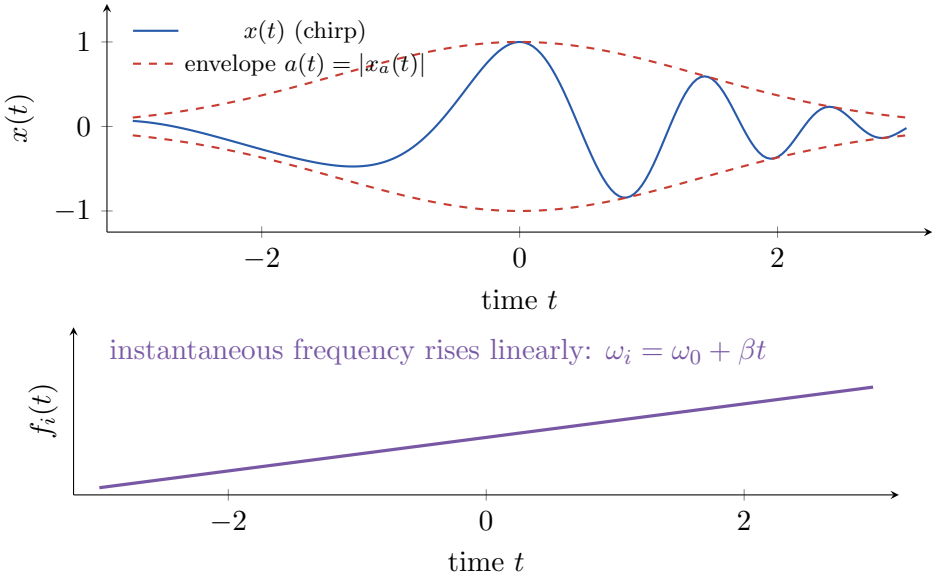


Figure 25.4: A Gaussian-windowed linear chirp. Top: the real signal (blue) with its Hilbert envelope (red dashed) tracing the amplitude. Bottom: the recovered instantaneous frequency rises as a straight line – the analytic signal reads off both "how loud" and "how high" at every instant, something a single Fourier transform cannot do.

Walking question

Why differentiate the *unwrapped* phase, and not just take atan2 and differentiate? Because atan2 returns an angle in $(-\pi, \pi]$ and jumps by 2π every time the arrow crosses the negative real axis. Differentiating across a jump gives a giant spurious spike. We must *unwrap* – add multiples of 2π to keep the phase continuous – before differentiating. Equivalently, there is a jump-free formula that never forms the angle at all (next section). Think of it as an odometer: you do not read instantaneous speed by watching the trip-meter wrap from 999.9 to 000.0; you track total distance and differentiate that.

25.4.2 A jump-free instantaneous-frequency formula

Differentiating $\theta = \arg x_a = \mathrm{Im}\{\ln x_a\}$ directly gives a tidy expression that needs no unwrapping. Using $\mathrm{d}\theta/\mathrm{d}t = \mathrm{Im}\{\dot{x}_a/x_a\}$ and writing $x_a = x + \mathrm{j}\hat{x}$,

$$\omega_i(t) = \mathrm{Im}\left\{\frac{\dot{x}_a(t)}{x_a(t)}\right\} = \frac{x(t)\,\dot{\hat{x}}(t) - \hat{x}(t)\,\dot{x}(t)}{x(t)^2 + \hat{x}(t)^2}. \tag{25.14}$$

The numerator is the cross product of the arrow with its velocity; the denominator is the arrow's squared length. This is just the formula for angular velocity of a moving point in the plane – $\omega = (\mathbf{r} \times \dot{\mathbf{r}})/\|\mathbf{r}\|^2$ – specialized to our arrow. No atan2, no wrapping. It is the recommended way to compute instantaneous frequency numerically.

25.5 Computing it on a real (sampled) signal

In practice $x(t)$ is a finite list of samples, and we want a fast, finite Hilbert transform. The frequency-domain definition Equation (25.3) turns directly into an FFT recipe, which is exactly what `scipy.signal.hilbert` does – note that, confusingly, that function returns the *analytic signal* x_a, not $\hat{x}$ alone.

The recipe is just Equation (25.8) on the DFT: take the FFT, zero the negative-frequency bins, double the positive ones, leave DC (and Nyquist) alone, and inverse-FFT.

Listing 25.1: Analytic signal, envelope, and instantaneous frequency via the FFT.

```python
import numpy as np

def analytic_signal(x):
    """Return x_a = x + j*Hilbert(x) for a real 1-D array (length N)."""
    N = len(x)
    X = np.fft.fft(x)
    h = np.zeros(N)
    h[0] = 1                        # DC: keep as is
    if N % 2 == 0:
        h[N//2] = 1                 # Nyquist bin (even N): keep
        h[1:N//2] = 2               # positive freqs: double
    else:
        h[1:(N+1)//2] = 2           # positive freqs: double
    return np.fft.ifft(X * h)       # negative freqs already zeroed by h

# --- demo: envelope + instantaneous frequency of a chirp ---
fs   = 2000.0                       # sample rate, Hz
t    = np.arange(0, 1.0, 1/fs)
x    = np.exp(-((t-0.5)/0.2)**2) * np.cos(2*np.pi*(50*t + 60*t**2))

xa   = analytic_signal(x)
env  = np.abs(xa)                                   # instantaneous amplitude
phi  = np.unwrap(np.angle(xa))                      # continuous phase
f_i  = np.diff(phi) / (2*np.pi) * fs                # instantaneous freq, Hz
print("freq sweeps from %.0f to %.0f Hz" % (f_i[0], f_i[-1]))
```

Common trap

Two practical pitfalls. (1) **Edge effects**: the FFT treats the signal as periodic, and the nonlocal Hilbert kernel smears the wrap-around discontinuity inward. The envelope and instantaneous frequency are unreliable near the two ends – window the signal, or discard a transient region at each edge. (2) **Differentiation amplifies noise**: ω_i involves a derivative, so even small wiggles in the phase become large spikes in frequency. Smooth the phase (or the

analytic signal) before differentiating, and remember that instantaneous frequency is meaningless wherever the envelope is near zero (the arrow's length is tiny, so its angle is undefined and jumps around).

For *streaming* or embedded use you do not want a whole-signal FFT. Instead you approximate the 90° shifter with an FIR filter. The ideal impulse response, sampled, is

$$h[n] = \begin{cases} \dfrac{2}{\pi n}, & n \text{ odd}, \\ 0, & n \text{ even}, \end{cases} \tag{25.15}$$

windowed to a finite length (Figure 25.5). Notice every even tap is zero, which halves the multiplications. A practical odd-length, linear-phase FIR Hilbert filter delays the signal by (taps $-$ 1)/2 samples, so you must delay the real path by the same amount before combining the two into x_a. We design these exactly as the band-pass filters of Chapter 16, with the Hilbert (Type III/IV) constraint.

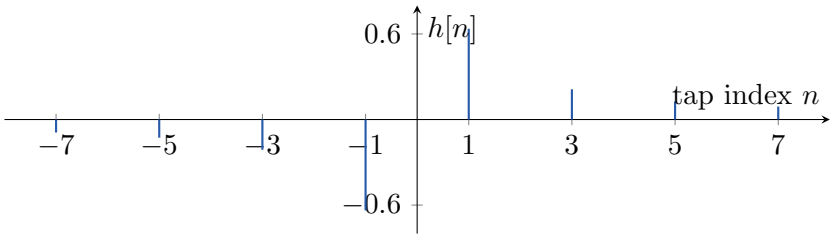


Figure 25.5: The ideal sampled Hilbert impulse response $h[n] = 2/(\pi n)$ for odd n, zero for even n (only the odd taps are drawn; the even taps sit at zero). It is antisymmetric about $n = 0$ and decays slowly as $1/n$, which is why truncation needs a window. Half the taps being zero is a free speedup.

Build it (hardware)

On an Arduino-class MCU, a full FFT per sample is overkill for envelope following. Two cheap options:

(a) Short FIR Hilbert pair. Store an odd-length window of recent samples; convolve with a precomputed 31- or 63-tap Hilbert kernel (Equation (25.15)) for $\hat{x}$, and a matched delay for x; then env $= \sqrt{x^2 + \hat{x}^2}$. With every other tap zero, a 63-tap filter is only about 32 multiply-adds per sample – fine at audio rates on an ARM Cortex-M.

(b) Rectify-and-smooth. For pure *envelope* following (not instantaneous frequency), you often do not need the Hilbert transform at all: take $|x|$ (or x^2) and low-pass filter it with a one-pole leaky integrator, `env += alpha*(fabs(x) - env);`. This is the classic analog-style envelope detector. It is biased (a full-wave-rectified sine has mean $2A/\pi$, not A) and ripples at twice the carrier, but it is one multiply per sample. Use the Hilbert envelope when you need accuracy or the instantaneous frequency; use rectify-and-smooth when you need cheapness.

25.6 Why this matters: demodulation and envelope following

Application

AM radio is literally Example 25.1: a slow audio message $a(t)$ rides on a fast carrier, and the receiver recovers $a(t) = |x_a(t)|$. Old radios did this with a diode (rectify-and-smooth); a software-defined radio does it with the Hilbert transform or with an I/Q mix.
FM radio and vibrato live in the *phase*: the message is encoded as $\omega_i(t) = \omega_c + k\,m(t)$, so FM demodulation *is* computing instantaneous frequency, Equation (25.14). The same math measures the pitch wobble (vibrato) of a sung note or the frequency glide of a bird call.
Audio and music. The envelope is the "A" in an ADSR amplitude contour; envelope followers drive compressors, auto-wah, sidechain ducking, and onset detectors. The instantaneous frequency sharpens pitch estimates between FFT bins (the basis of the phase-vocoder used for time-stretching and pitch-shifting, Chapter 28).
Biomedical. The Hilbert transform is the standard tool for the *amplitude envelope* and *phase* of neural oscillations (e.g. the power in a band, or phase–amplitude coupling in EEG/LFP), and for the envelope of the QRS complex in ECG.

Toward the wrist tracker

For the wrist tracker, the analytic signal is our *burst detector*. A surface-EMG or FMG channel during a muscle contraction looks like a noisy oscillation that swells and fades – exactly an amplitude-modulated signal. The Hilbert envelope $|x_a(t)|$ gives a smooth, sign-independent "muscle activity level" without the half-cycle ripple of plain rectification, which is what we threshold to decide *a gesture is happening now* and to measure *how hard* (useful for mapping contraction strength to MIDI velocity or to a continuous controller).
For the IMU, the same trick turns a band-passed accelerometer axis into a tremor or oscillation envelope – handy for detecting a periodic gesture (a shake, a tap train) and reading its strength and rate via instantaneous frequency.
Know the limits. An envelope is a *scalar amplitude*, not a hand pose. A wrist IMU's Hilbert envelope can tell you "the forearm is oscillating, fairly hard, at about 5 Hz," but it cannot recover which finger moved – a single IMU has no access to individual finger joints (Chapter 34). Even with EMG, the Hilbert envelope gives you *when* and *how strong*, not a continuous joint angle; finger-level discrimination needs multiple EMG/FMG channels and a classifier (Chapters 37–38), and *continuous* finger pose needs sensors on the fingers themselves. Treat the envelope as a trigger-and-intensity signal, not as a pose sensor.

Try it yourself

Run the code in Listing 25.1 on three signals and watch the envelope and instantaneous frequency: (1) a pure 440 Hz tone – the envelope should be flat and the frequency a flat line at 440; (2) the Gaussian chirp – envelope a bump, frequency a ramp; (3) a recording of yourself clenching your fist with a cheap EMG board (or, lacking one, a band-passed accelerometer axis while you tap a table) – the envelope should rise during activity. Then deliberately break the Bedrosian condition: multiply two tones of similar frequency and watch the "envelope" beat and the instantaneous frequency dip negative. That misbehaviour is the trap of Section 25.4 made visible.

25.7 Connection back to I/Q and to projection

We close the loop with Chapter 23. There, a complex baseband signal $I(t) + \mathrm{j}Q(t)$ came from *two* physical mixers fed by oscillators 90° apart. Here we built the same kind of object – a one-sided, purely rotating complex signal – from a *single* real channel by computation. The two viewpoints are one thing:

- **Hardware I/Q:** a 90° phase split done with analog circuitry, at the carrier, before sampling.
- **Hilbert / analytic signal:** the same 90° split done with a digital filter, after sampling.

Both deliver a complex signal whose magnitude is the envelope and whose phase rate is the instantaneous frequency. Choosing between them is an engineering trade: hardware I/Q needs two matched analog paths but halves the required sample rate; the Hilbert approach needs only one ADC channel but must sample fast enough to capture the carrier and then do the filtering in software.

Formal

Summary of the operator. $\mathcal{H}$ is the LTI all-pass system with frequency response $H(\mathrm{j}\omega) = -\mathrm{j}\,\mathrm{sgn}(\omega)$ (unit magnitude, $\mp 90°$ phase) and impulse response $1/(\pi t)$. It satisfies $\mathcal{H}^2 = -I$ (off DC), commutes with time shifts and derivatives, and is energy-preserving (Parseval): $\|\hat{x}\| = \|x\|$. The analytic-signal map $x \mapsto x_a = x + \mathrm{j}\mathcal{H}x$ has the one-sided spectrum of Equation (25.8); its modulus is the envelope and its phase derivative the instantaneous frequency. These are well-defined and physically meaningful precisely when x is a single narrow-band component (Bedrosian).

Recap

- A real signal is the shadow of a rotating phasor; the missing coordinate is the signal shifted by 90° at *every* frequency.
- The **Hilbert transform** supplies that coordinate: in frequency it multiplies by $-\mathrm{j}\,\mathrm{sgn}(\omega)$ (a $\mp 90°$ all-pass rotation); in time it convolves with $1/(\pi t)$.
- The **analytic signal** $x_a = x + \mathrm{j}\hat{x}$ has a one-sided spectrum (negative frequencies deleted, positive doubled) and so purely rotates.
- Its modulus is the **envelope** (instantaneous amplitude); the derivative of its unwrapped phase is the **instantaneous frequency** (Equation (25.14) avoids unwrapping).
- These give parameter-free AM demodulation, FM demodulation, envelope following, and pitch tracking – but only for single narrow-band components (Bedrosian); for multi-component signals, separate components first.
- It is the computational twin of hardware I/Q, and for the wrist tracker it is a robust *when/how-strong* burst detector – never a finger-pose sensor.

25.8 Exercises

Exercise 25.1. Using only Equation (25.3), compute $\mathcal{H}\{\cos(\omega_0 t)\}$ and $\mathcal{H}\{\sin(\omega_0 t)\}$ for $\omega_0 > 0$, and confirm Equation (25.4). Then show $\mathcal{H}^2\{\cos(\omega_0 t)\} = -\cos(\omega_0 t)$, illustrating $\mathcal{H}^2 = -I$.

Exercise 25.2. What is $\mathcal{H}\{c\}$ for a constant signal $x(t) = c$? Use the frequency-domain definition and explain physically why the answer must be zero. What does this imply about the envelope of $x(t) = c + \cos(\omega_0 t)$ computed via the analytic signal – is it 1, $c + 1$, or something time-varying? (Hint: the DC term is real and unrotated; sketch the arrow.)

Exercise 25.3. For $x(t) = (1 + m\cos\omega_m t)\cos\omega_c t$ with $\omega_m \ll \omega_c$ and $0 < m < 1$ (standard AM), write down the analytic signal under the Bedrosian approximation and show the recovered envelope is $1 + m\cos\omega_m t$. What goes wrong if $m > 1$ (overmodulation)?

Exercise 25.4. Starting from $\theta = \mathrm{Im}\{\ln x_a\}$ and $x_a = x + \mathrm{j}\hat{x}$, derive the jump-free instantaneous-frequency formula Equation (25.14). Verify it on $x(t) = \cos(\omega_0 t)$ (for which $\hat{x} = \sin\omega_0 t$) and confirm it returns the constant ω_0.

Exercise 25.5. Prove that the Hilbert transform preserves energy, $\|\hat{x}\| = \|x\|$, using Parseval's theorem and the fact that $|-\mathrm{j}\,\mathrm{sgn}(\omega)| = 1$ for $\omega \neq 0$. What is the one signal class for which energy is *not* preserved, and why?

Exercise 25.6. Show that, for a signal with no DC, $x(t)$ and $\hat{x}(t)$ are orthogonal: $\int x(t)\hat{x}(t)\,\mathrm{d}t = 0$. (Hint: write the inner product in the frequency domain with Parseval; the multiplier $-\mathrm{j}\,\mathrm{sgn}(\omega)$ makes the integrand odd.) Interpret geometrically in terms of the rotating arrow.

Exercise 25.7. The ideal sampled Hilbert FIR has taps $h[n] = 2/(\pi n)$ for odd n and 0 for even n. (a) Why are the even taps zero? Relate it to the symmetry of the kernel $1/(\pi t)$. (b) Truncate to $|n| \le 7$ and apply a Hamming window; compute the resulting taps. (c) Why must you delay the real signal path by $(\text{length} - 1)/2$ samples before forming $x_a = x_{\text{delayed}} + \mathrm{j}(h * x)$?

Exercise 25.8. Run Listing 25.1. (a) Plot the envelope and the instantaneous frequency for the chirp; confirm the frequency sweeps linearly. (b) Add Gaussian noise of standard deviation 0.05 to x and recompute – describe what happens to the instantaneous frequency and explain why. (c) Smooth the analytic signal (or the phase) before differentiating and show the improvement.

Exercise 25.9. Construct a two-tone signal $x(t) = \cos(\omega_1 t) + \cos(\omega_2 t)$ with ω_1, ω_2 close. Compute its analytic-signal envelope analytically and show it is $2|\cos\big(\frac{\omega_2 - \omega_1}{2}t\big)|$ (a beat). Explain why this is a *correct* envelope yet a *misleading* description if you imagined "one tone with slowly varying amplitude," and connect this to the Bedrosian warning.

Exercise 25.10. (Toward the wrist tracker.) You have a 1 kHz surface-EMG channel. Design an envelope follower two ways: (i) Hilbert envelope via a 63-tap FIR; (ii) rectify-and-smooth with a one-pole filter at 10 Hz cutoff. For each, state the latency, the multiply-adds per sample, and one accuracy advantage. Then explain, in two sentences, why neither envelope can tell you *which* finger moved, and what additional sensing would be required.

Part VI

Audio Signal Processing

Chapter 26

Digital Audio Fundamentals

We have spent the book so far building general machinery: signals as vectors, frequency as rotation, the Fourier transform as projection, sampling and quantization as the bridge between the continuous world and the computer. Now we point all of it at one of the richest and most familiar signals there is – **sound**. This chapter is where the abstractions earn their keep. By the end you will know, concretely and without hand-waving, how the pressure wave at your eardrum becomes a list of integers in a file, why those integers are sampled 44,100 times a second and stored in 16 bits, what "loud" really means, what clipping is and how to avoid it, and how to record and play a sound on a Raspberry Pi.

This is also the launchpad for the next several chapters. Once we agree on what a digital audio signal *is*, we can extract features from it (Chapter 27), shape it with effects and synthesize it from scratch (Chapter 28), and analyze the signals of real instruments (Chapter 29). And although our through-line project is a *gesture* controller, audio is both its eventual *output* (it will emit MIDI that drives a synthesizer) and a possible *input* (a contact microphone on the wrist hears tendon snaps and taps). So sound matters to us in two directions, and it pays to understand it from the ground up.

26.1 From a pressure wave to a number

Sound is a mechanical wave: a region of slightly compressed air chases a region of slightly rarefied air outward from a vibrating source. What reaches your ear – or a microphone – is the *air pressure at one point* rising and falling over time. Strip away everything else and a sound is exactly the object from Chapter 1: a function $p(t)$ giving pressure (really, the tiny deviation from atmospheric pressure) at each instant.

A microphone is a transducer: it converts that pressure variation into a proportional voltage $v(t)$. A dynamic mic moves a coil in a magnetic field; a condenser mic varies the gap of a charged capacitor; a MEMS mic (the kind inside your phone, and the kind you will solder to a Pi) flexes a micromachined membrane over a tiny back-plate. The physics differs, but the job is identical: pressure in, voltage out, ideally with the voltage a clean scaled copy of the pressure. From the signal-processing point of view we do not care which mic; we care that we now have a continuous-time electrical signal $v(t)$ that *is* the sound.

Intuition

Picture the whole chain as a relay of copies of the same wiggle. Vibrating object → air pressure → membrane motion → voltage → (after the steps below) a list of numbers → (on playback) voltage → speaker cone motion → air pressure → your eardrum. Every arrow is supposed to preserve the *shape* of the wiggle. Digital audio is the art of turning the wiggle into integers and back without your ear noticing the round trip.

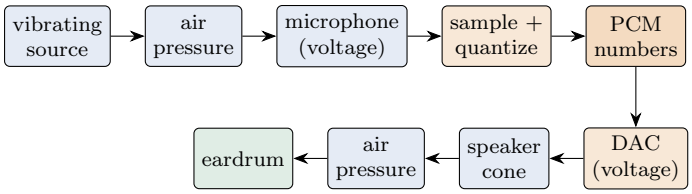


Figure 26.1: The audio chain, top row capture and bottom row playback. Every arrow should preserve the shape of the wiggle; the two orange stages (sample plus quantize on the way in, reconstruct on the way out) are where the continuous world meets the integers of this chapter.

Figure 26.1 lays out that relay end to end. To get from the continuous voltage $v(t)$ to numbers we apply the two operations developed in Chapters 18 and 19, now in their audio incarnation:

1. **Sampling** (discretize *time*): measure $v(t)$ at regular instants $t = nT_s$, producing a sequence $v[n] = v(nT_s)$. The spacing T_s is the sampling period; its reciprocal $f_s = 1/T_s$ is the *sample rate*.

2. **Quantization** (discretize *value*): snap each measured voltage to the nearest level on a finite grid, so it fits in a fixed number of bits. The number of bits is the *bit depth*.

The result – a stream of quantized samples taken at a fixed rate – is called **pulse-code modulation**, PCM. It is the bedrock representation underneath WAV files, audio CDs, and the buffers your sound card hands you. Everything fancier (MP3, AAC, Opus) is a clever way of storing *fewer* bits that decode *back* to PCM before they reach a speaker. We will build PCM from the geometry and treat compression as a later refinement.

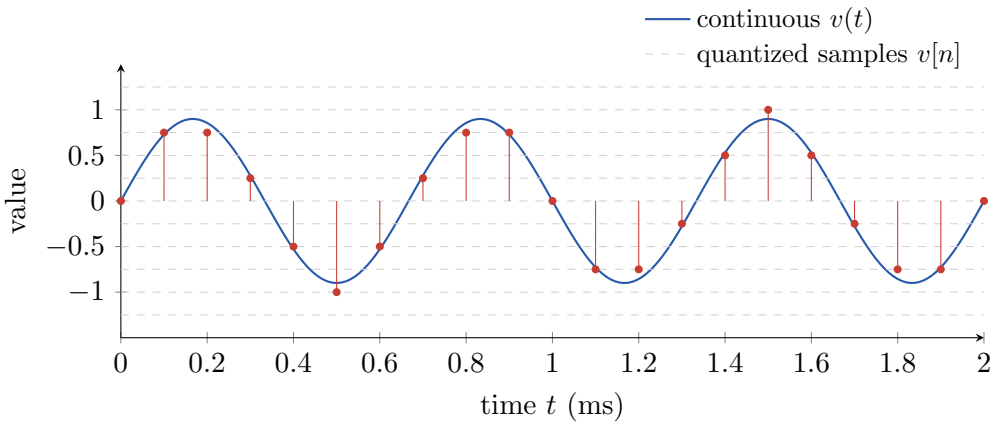


Figure 26.2: The two discretizations of PCM. Time is sampled (the dots occur only at multiples of T_s); value is quantized (the dots land only on the dashed level grid). The smooth blue curve is the original continuous signal; the red stems are what the computer actually stores.

What is really happening

Notice that two completely different things are happening in Figure 26.2, and it is worth keeping them mentally separate forever. *Sampling* is about the horizontal axis: we only know the signal at certain instants. *Quantization* is about the vertical axis: even at those instants we only record a rounded value. Sampling errors show up as aliasing (Chapter 18); quantization errors show up as a faint noise floor (Chapter 19). Audio quality is the project of making both errors small enough that the ear cannot hear them.

26.2 What the ear demands: the hearing range

Audio is the one branch of signal processing where the specification is written by biology. We do not need to capture sound perfectly; we need to capture it well enough to fool a particular organ. So before choosing numbers we ask: what can a human ear actually detect?

Frequency range. A healthy young ear responds to vibrations from roughly 20 Hz to 20,000 Hz (20 kHz). Below 20 Hz you feel rumble more than hear pitch; above 20 kHz (ultrasound) the cochlea simply does not respond. With age the upper limit retreats – many adults top out near 15–16 kHz. This three-decade span (a factor of 1000 in frequency) is the single most important fact in audio engineering, because it sets how fast we must sample.

Amplitude range. The ear's dynamic range is staggering. The quietest audible sound pressure (the *threshold of hearing*) and the loudest tolerable sound (the *threshold of pain*) differ by a factor of about 10^6 in pressure amplitude – a power ratio of about 10^{12}. No one wants to write "the pressure was 0.0000063 pascals," so loudness is measured logarithmically, in **decibels** (dB). We will make decibels precise in Section 26.6; for now just register that the ear hears *ratios*, not absolute amounts, which is exactly why a logarithmic scale fits it.

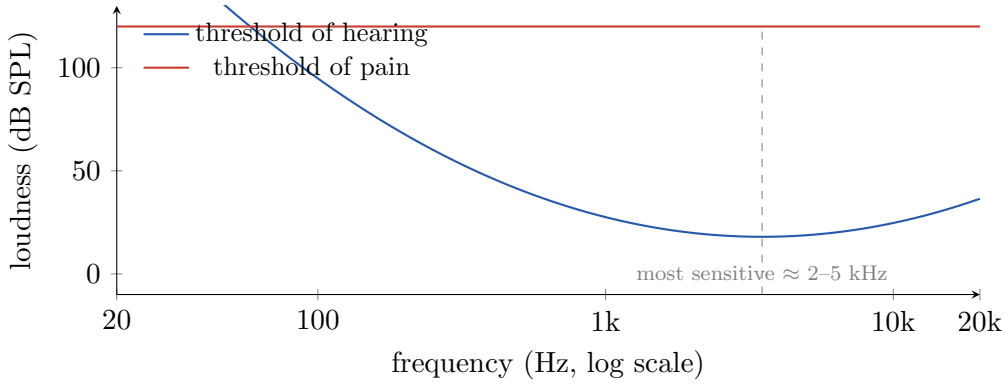


Figure 26.3: The audible field (schematic). Sound is heard only between the two curves. The ear is most sensitive around 2–5 kHz (where speech consonants and a crying baby live), and far less sensitive at the extremes of the range. The vertical gap is the available dynamic range at each frequency.

Connection

That the ear hears on a logarithmic scale in *both* axes – octaves in frequency (next section) and decibels in amplitude – is the same Weber–Fechner pattern you met in the probability and calculus companions, where perception tracks the *logarithm* of a stimulus. The recurring lesson: when a quantity spans many orders of magnitude and we care about relative change, work in log. We did it for the Bode plots of Chapter 16; we do it again here.

26.3 Choosing the sample rate

The Nyquist–Shannon theorem (Chapter 18) is blunt: to capture all frequencies up to $f_{\max}$ without aliasing, you must sample at more than twice $f_{\max}$. Set $f_{\max} = 20$ kHz, double it, and you get 40 kHz as the bare minimum sample rate for full-range audio. Real systems push a little above the minimum to leave room for a non-ideal anti-aliasing filter (which cannot drop from passband to stopband infinitely fast). That margin is exactly why the standard rates sit a bit over 40 kHz.

Why it works

Why 44,100 Hz of all numbers? History, not acoustics. Early digital audio masters were stored on video tape (the only affordable medium with enough bandwidth in the late 1970s), using a PCM adaptor that packed audio samples into the lines of a video frame. The arithmetic of NTSC and PAL video lines – so many usable lines per field, so many fields per second, so many samples per line – happened to land on 44,100 samples per second for both formats. Sony and Philips adopted it for the Compact Disc, and the world inherited it. It comfortably exceeds the 40 kHz Nyquist floor, giving the anti-alias filter a 2 kHz transition band above 20 kHz.

History

The deeper root is older still. In 1928 Harry Nyquist, at Bell Labs, studied how many distinct pulses a telegraph channel of a given bandwidth could carry; in 1949 Claude Shannon turned the idea into the sampling theorem we use today. Their result – that a bandlimited signal is *exactly* reconstructible from samples taken faster than twice its highest frequency – is the reason digital audio can be transparent at all. Without it, "sampling" would be a lossy guess; with it, sampling above Nyquist loses *nothing* (before quantization). Every time you hear a CD, you are hearing Shannon's theorem in action.

The common rates you will meet:

Rate	Nyquist limit	Where it is used
8,000 Hz	4 kHz	Telephone speech; intelligible but dull (no high consonant detail).
16,000 Hz	8 kHz	"Wideband" speech, voice assistants, many speech models.
22,050 Hz	$\approx$ 11 kHz	Half of CD rate; older games, low-bandwidth music.
44,100 Hz	$\approx$ 22 kHz	Audio CD; the default for music distribution.
48,000 Hz	24 kHz	Pro audio, video, film, and most computer sound hardware.
96,000 Hz	48 kHz	High-resolution recording and studio production.

Common trap

"More is better" is a trap. Sampling music at 192 kHz does not make it sound better to a human ear – there is nothing audible above 20 kHz to capture, so you are storing ultrasonic noise at four times the file size. Higher rates earn their keep *during production* (they ease filter design and reduce artifacts when pitch-shifting or applying nonlinear effects), then you render down. For a final delivery file, 44.1 or 48 kHz is transparent. The same restraint applies to our IMU and EMG work later: sample fast enough to clear Nyquist for the band you care about, plus a margin, and not absurdly faster.

A quick reality check by hand

Example 26.1 (Will it alias?)**.** You record at $f_s = 8000$ Hz (telephone rate) and someone whistles a pure tone at 6000 Hz with no anti-aliasing filter. What happens? The Nyquist frequency is $f_s/2 = 4000$ Hz. A 6000 Hz tone exceeds it, so it folds: the apparent frequency is

$$f_{\text{alias}} = |f_s - f_{\text{in}}| = |8000 - 6000| = 2000 \text{ Hz}.$$

The whistle is recorded as a 2 kHz tone – a genuinely wrong pitch. This is why every real ADC front end has an analog low-pass (anti-alias) filter before the sampler, cutting everything above $f_s/2$. The filter cannot be perfect, which is the whole reason we sampled a bit above the 40 kHz floor in the first place.

26.4 Bit depth: how finely we measure each sample

Sampling fixes *when* we measure; bit depth fixes *how precisely*. With b bits per sample we have 2^b available levels to represent the voltage. Audio almost always uses *signed* integers so that 0 sits in the middle and the wave swings positive and negative: 8-bit audio runs $-128 \dots 127$, 16-bit runs $-32768 \dots 32767$, and so on.

The key consequence is the **signal-to-quantization-noise ratio**. Rounding each sample to the nearest level introduces an error of at most half a level. Treating that error as roughly uniform noise (Chapter 19), the resulting SNR for a full-scale signal works out to the famous rule of thumb

$$\mathrm{SNR}_{\mathrm{dB}} \approx 6.02\, b + 1.76.$$

Every bit you add buys about 6 dB of dynamic range (Figure 26.4). This single line explains every standard bit depth at a glance:

Bits b	Levels 2^b	SNR $\approx 6b{+}1.76$ dB	Character
8	256	$\approx$ 50 dB	Audibly hissy; old games, telephony.
12	4096	$\approx$ 74 dB	Decent; many low-cost ADCs and DACs.
16	65,536	$\approx$ 98 dB	CD quality; transparent for playback.
24	16.7M	$\approx$ 146 dB	Studio recording; huge headroom margin.

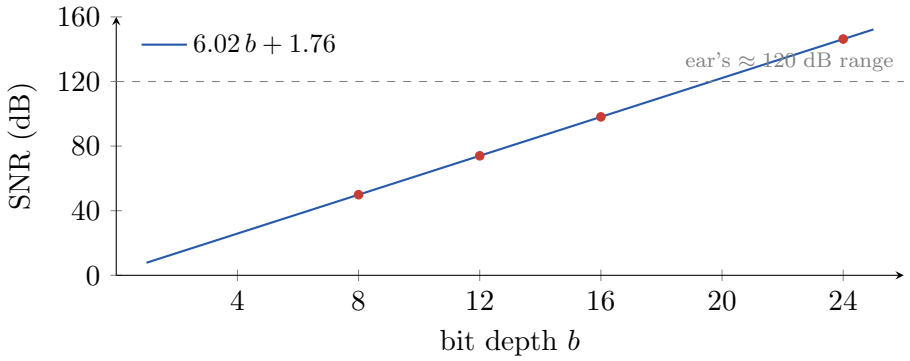


Figure 26.4: Quantization SNR is a straight line in bit depth, about 6 dB per bit. The dots mark the standard depths from the table; 16 bits already clears the ear's quiet-room-to-loud-passage gap, and 24 bits sits far above the ear's full dynamic range (dashed).

Why it works

Why is 16 bits "enough" for a finished recording when the ear's dynamic range is about 120 dB? Because you never need to span the entire range *in one recording at one volume setting*. 16 bits give $\approx$ 98 dB of usable range – more than the difference between a quiet room and a loud passage of music – and dithering (adding a whisper of noise before quantizing) smears the residual quantization error into an inaudible, hiss-like floor instead of audible distortion.

Studios record at 24 bits not because the ear hears 146 dB, but because extra headroom means you can set levels conservatively, never clip, and still have noise far below audibility after mixing.

Connection

The $6.02b + 1.76$ formula is not an audio fact; it is a *quantization* fact derived once in Chapter 19 from a uniform-noise model. Here we are simply reading it through audio glasses. This is the payoff of building general machinery first: a single derivation about rounding error now tells us why CDs are 16-bit and why a cheap 8-bit sound chip hisses.

Integer versus floating point

Inside a digital audio *workstation*, samples are usually carried as 32-bit floating-point numbers in the range $[-1.0, +1.0]$, not integers. Float has two virtues for *processing*: its dynamic range is enormous (so intermediate sums never overflow), and crucially, *values above* 1.0 *are not destroyed* – a signal can momentarily exceed full scale inside a processing chain and be brought back down before the final integer conversion. Integers, by contrast, have no room above the top level; exceed it and the sample is permanently clamped. That clamping is clipping, the subject of the next section. The conversion between the two worlds is a simple scale: float $1.0 \leftrightarrow$ int 32767 for 16-bit.

26.5 Clipping, full scale, and headroom

Every digital format has a hard ceiling: the largest value a sample can hold. We call it **full scale** (often written 0 dBFS, "decibels relative to full scale"). When the incoming signal tries to swing past full scale, the sample cannot follow – it gets pinned at the maximum. The rounded tops and bottoms of the waveform are **clipping**.

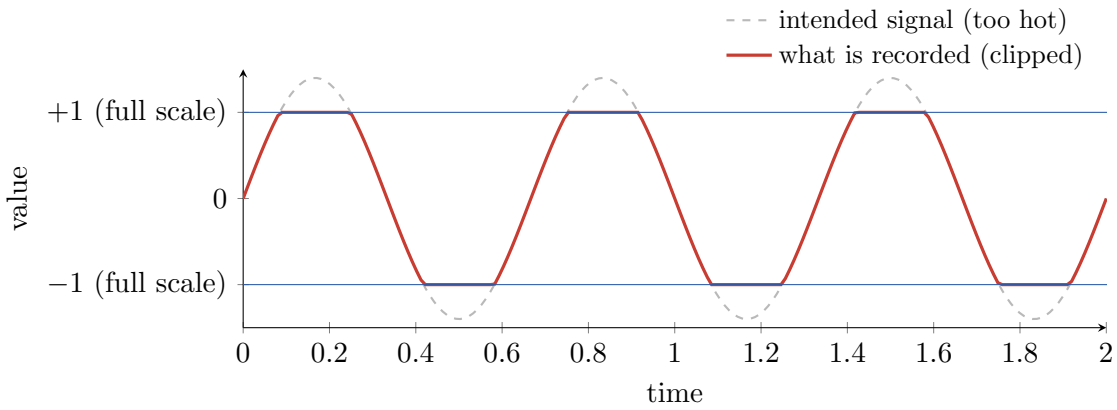


Figure 26.5: Clipping. The intended sine (gray, dashed) swings beyond full scale, so its peaks are flattened against the ±1 ceiling (red). The flat tops are new sharp corners in the waveform, and sharp corners mean new high-frequency content – harmonics that were not in the original.

What is really happening

What does clipping *do* to the sound, geometrically? Flattening the peaks turns a smooth sinusoid into something closer to a square wave. From the Fourier series (Chapter 6) you know a square wave is a sine plus odd harmonics at $3f, 5f, 7f, \ldots$ (Figure 26.6). So clipping *manufactures harmonics that were never played.* Mild clipping adds a gritty edge (guitar distortion pedals do this deliberately and gently); hard clipping turns clean audio into a harsh buzz. Crucially, those new harmonics can exceed Nyquist and then alias back as inharmonic, metallic tones – clipping and aliasing conspire to make digital overload sound especially nasty.

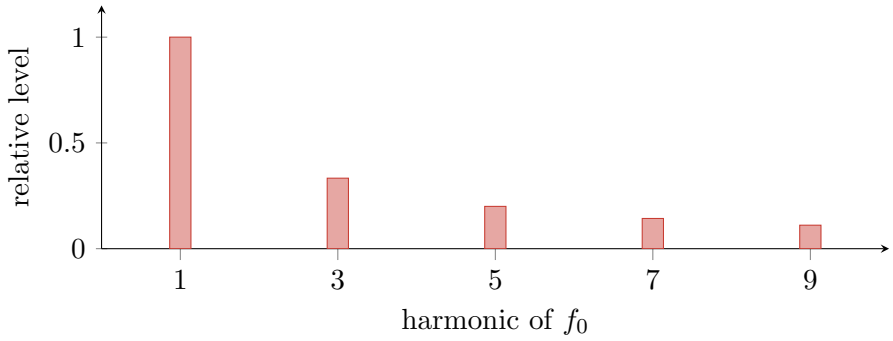


Figure 26.6: Spectrum of a hard-clipped sine, which approaches a square wave: the fundamental plus odd harmonics at $3f_0, 5f_0, 7f_0, \ldots$ with amplitudes $1/3, 1/5, 1/7, \ldots$. None of these were in the original tone; clipping built them, and the high ones can fold back below Nyquist as aliasing.

Headroom is the safety gap you deliberately leave between your typical signal peaks and full scale, measured in dB. If your loudest peaks hit -6 dBFS, you have 6 dB of headroom: a sudden transient twice as loud as expected still fits. Recording engineers chase a balance – too little headroom risks clipping on an unexpected peak; too much wastes dynamic range and pushes the quiet parts toward the quantization floor. A common practical target is peaks around -12 to -6 dBFS while tracking, leaving comfortable margin.

Example 26.2 (How much headroom is that?)**.** You set your recording level so that the loudest expected peak reaches half of full scale, i.e. amplitude 0.5 on a $[-1, 1]$ scale. How many dB of headroom is that? Headroom is the ratio of full scale to peak, in dB:

$$\text{headroom} = 20\log_{10}\left(\frac{1.0}{0.5}\right) = 20\log_{10}(2) \approx 6.0 \text{ dB}.$$

So "half of full scale" is exactly 6 dB of headroom – another appearance of the "factor of two is 6 dB" rule that also governs bit depth. A transient up to twice your expected peak will just touch full scale without clipping.

Common trap

Clipping is irreversible. Once peaks are flattened, the lost information is *gone*; no plugin can truly un-clip, only disguise it. This is the opposite of being a little too quiet, which you can simply amplify later (at the cost of also lifting the noise floor). When in doubt, record a touch quieter. The asymmetry – recoverable quietness versus unrecoverable clipping – is why

seasoned engineers leave generous headroom.

26.6 Loudness, decibels, and dBFS

We have used decibels three times already; now let us pin them down. A decibel is a way of expressing a *ratio* of two quantities on a logarithmic scale. For *power* (or energy, or intensity):

$$L_{\mathrm{dB}} = 10 \log_{10}\!\left(\frac{P}{P_{\mathrm{ref}}}\right).$$

For *amplitude* (voltage, pressure, sample value) – which is the square root of power – the factor doubles to compensate for the square:

$$L_{\mathrm{dB}} = 20 \log_{10}\!\left(\frac{A}{A_{\mathrm{ref}}}\right).$$

The reference A_{ref} is what makes the number meaningful. Two references dominate audio:

- **dB SPL** (sound pressure level): $A_{\mathrm{ref}} = 20\,\mu\mathrm{Pa}$, the nominal threshold of hearing. This measures sound *in the air*. 0 dB SPL is barely audible; conversation is ≈ 60 dB; a rock concert ≈ 110 dB; pain ≈ 120–130 dB.
- **dBFS** (decibels relative to full scale): A_{ref} is the maximum sample value. By construction 0 dBFS is full scale and every real digital signal sits *below* it, so dBFS values are negative (e.g. -12 dBFS).

Intuition

The two "10" and "20" formulas are not two different definitions – they are the same definition seen through power versus amplitude. Power goes as amplitude squared, and $\log(A^2) = 2 \log A$, so the factor 10 on power becomes 20 on amplitude. Memorize one anchor and you can rebuild the rest: *a factor of* 2 *in amplitude is* ≈ 6 *dB; a factor of* 10 *in amplitude is exactly* 20 *dB; a factor of* 10 *in power is exactly* 10 *dB.*

Example 26.3 (RMS and a sine's level)**.** "How loud is a signal" is usually answered with its *RMS* (root-mean-square) value, not its peak, because RMS tracks delivered power and correlates with perceived loudness. For a sine of amplitude A,

$$A_{\mathrm{rms}} = \sqrt{\frac{1}{T}\int_0^T \big(A \sin(2\pi f t)\big)^2 \,\mathrm{d}t} = \frac{A}{\sqrt{2}} \approx 0.707\,A,$$

because the average of $\sin^2$ over a cycle is $1/2$. So a full-scale sine ($A = 1$) has $A_{\mathrm{rms}} \approx 0.707$, i.e. its RMS level is $20 \log_{10}(0.707) \approx -3$ dBFS even though its *peak* is 0 dBFS. This 3 dB gap between peak and RMS for a sine – its "crest factor" – is why a meter showing peaks and a meter showing loudness never agree, and why you must say *which* you mean.

Application

Modern streaming services (Spotify, YouTube, Apple Music) "loudness-normalize" every track to a common target measured in **LUFS** (loudness units relative to full scale), a refinement of dBFS that weights frequencies the way the ear does and integrates over time. This is why a quiet jazz track and a loud pop track play back at similar perceived volume without you touching the knob. It also quietly ended the "loudness war" – the decades-long arms race of squashing dynamics to sound louder than the next track – because over-compressed masters now simply get turned *down* to hit the same LUFS target, gaining nothing.

26.7 The audio spectrum and octaves

Because the ear hears frequency logarithmically, the natural unit of pitch is not the hertz but the **octave**: a doubling of frequency. Two notes an octave apart (f and $2f$) sound "like the same note, higher" – the most consonant interval there is, because every harmonic of the upper note coincides with a harmonic of the lower. The entire audible range, 20 Hz to 20 kHz, spans about 10 octaves ($2^{10} = 1024 \approx 1000$), which is why we drew the audiogram on a log axis.

Western music subdivides each octave into 12 equal-ratio steps (the equal-tempered semitone), so the frequency ratio of one semitone is

$$r = 2^{1/12} \approx 1.0595.$$

Standard tuning pins one note: A4 (the A above middle C) = 440 Hz. Every other note follows by multiplying or dividing by r:

$$f(\text{note}) = 440 \cdot 2^{\,n/12},$$

where n is the number of semitones above ($n > 0$) or below ($n < 0$) A4. This is the exact map we will need to *emit* when the wrist controller plays an instrument: a detected gesture chooses a MIDI note number, and MIDI note m corresponds to $f = 440 \cdot 2^{(m-69)/12}$ Hz.

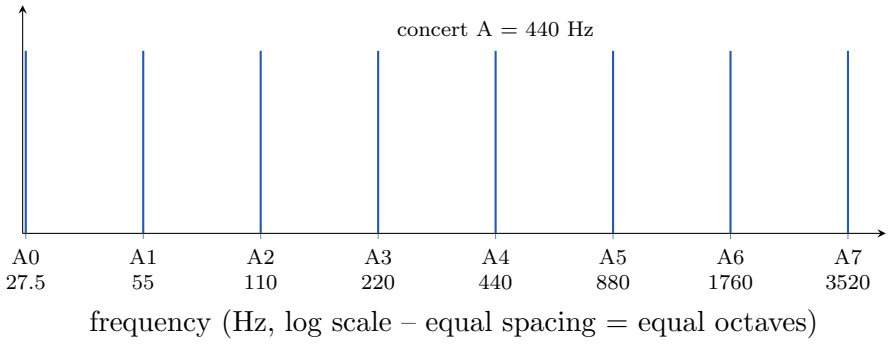


Figure 26.7: Successive A notes, each an octave (a doubling) above the last. On a logarithmic axis the octaves are *evenly spaced* – the same visual distance for every doubling – which is exactly how the ear perceives them.

Example 26.4 (From note to frequency)**.** What frequency is middle C (C4)? Middle C is 9 semitones below A4, so $n = -9$:

$$f = 440 \cdot 2^{-9/12} = 440 \cdot 2^{-0.75} \approx 440 \cdot 0.5946 \approx 261.6 \text{ Hz}.$$

And its MIDI note number is 60, since A4 is MIDI 69 and $69 - 9 = 60$. You can sanity-check with the MIDI formula: $440 \cdot 2^{(60-69)/12} = 440 \cdot 2^{-9/12} \approx 261.6$ Hz. The two formulas agree because they are the same equation written with different "zero" notes.

Rough map of the audio band. It helps to carry a mental partition of the spectrum: *sub-bass* (20–60 Hz, felt more than heard), *bass* (60–250 Hz, fundamental of most instruments), *midrange* (250 Hz–4 kHz, where speech and melody live and the ear is most sensitive), *presence* (4–8 kHz, consonant clarity and "edge"), and *brilliance* (8–20 kHz, air and cymbal shimmer). Equalizers, the tone controls of audio, simply boost or cut these regions; we will build the filters that do it in Chapter 28.

26.8 Build it: record and play on a Raspberry Pi

Enough theory – let us make a Pi capture real PCM and play it back, so the numbers above stop being abstract. A Raspberry Pi has *no* analog audio input of its own, so for recording you add a small USB sound card or an I2S MEMS microphone; its onboard 3.5 mm jack (or HDMI) can play *out*. The software stack is ALSA underneath, usually wrapped by friendlier tools.

Build it (hardware)

Hardware. A $5 USB audio dongle (record + play) is the painless choice and shows up as an ALSA card. For a tidier, all-digital path use an I2S MEMS mic such as the INMP441 or SPH0645, wired to the Pi's I2S pins (BCLK, LRCLK, DOUT, plus 3V3 and GND) and enabled via a device-tree overlay in `/boot/config.txt`. I2S skips analog wiring entirely: the mic does its own ADC and streams PCM straight to the Pi.

Find your devices.

```
arecord -l      # list capture (input) devices -> note card,device numbers
aplay   -l      # list playback (output) devices
```

Record and play raw PCM.

```
# Record 5 s, 44.1 kHz, 16-bit signed little-endian, mono, from card 1 dev 0
arecord -D plughw:1,0 -f S16_LE -r 44100 -c 1 -d 5 take1.wav
# Play it back through the default output
aplay take1.wav
```

The flags *are* this chapter: `-r 44100` is the sample rate, `S16_LE` is the 16-bit signed PCM bit depth, `-c 1` is one (mono) channel.

Once the bytes are on disk you can treat them as the integer signal we have been discussing. In Python, `numpy` and `soundfile` (or the stdlib `wave` module) read a WAV file straight into an array of samples:

```
import numpy as np, soundfile as sf

x, fs = sf.read("take1.wav")          # x in [-1, 1] floats, fs = 44100
print("samples:", x.shape[0], " rate:", fs, " duration:", x.shape[0]/fs, "s")

peak = np.max(np.abs(x))              # peak amplitude, 0..1
```

```
rms  = np.sqrt(np.mean(x**2))          # RMS amplitude
dbfs_peak = 20*np.log10(peak + 1e-12)
dbfs_rms  = 20*np.log10(rms  + 1e-12)
print(f"peak {dbfs_peak:6.1f} dBFS   rms {dbfs_rms:6.1f} dBFS   "
      f"headroom {-dbfs_peak:4.1f} dB")

clipped = np.mean(np.abs(x) > 0.999)  # fraction of samples pinned at full scale
print(f"clipped fraction: {clipped:.4%}")
```

Run that on a recording where you spoke too loudly and the "clipped fraction" jumps off zero – you have measured the very effect of Figure 26.5. Run it on a quiet recording and the headroom number balloons, confirming Example 26.2's arithmetic with your own voice.

Try it yourself

Record the same phrase three times: once whispering, once at normal volume, once shouting into the mic. Print peak dBFS, RMS dBFS, and clipped fraction for each. Predict before you run: which run clips? Which has the most headroom? Then plot the waveforms and confirm by eye that the shout's peaks are flat-topped while the whisper rides far below full scale. You have just experienced headroom, clipping, and dynamic range as data, not vocabulary.

Build it (hardware)

Generate a tone to test playback (no mic needed). This writes a clean A4 sine at −6 dBFS so you can verify your output chain and hear the difference between clean and clipped:

```
import numpy as np, soundfile as sf
fs, dur, f0 = 44100, 2.0, 440.0
t = np.arange(int(fs*dur)) / fs
clean = 0.5 * np.sin(2*np.pi*f0*t)                # peak 0.5 -> -6 dBFS headroom
hot   = np.clip(1.4*np.sin(2*np.pi*f0*t), -1, 1) # deliberately clipped
sf.write("clean_a4.wav", clean, fs, subtype="PCM_16")
sf.write("clipped_a4.wav", hot,  fs, subtype="PCM_16")
```

Play both through `aplay`. The clipped one buzzes – those are the manufactured harmonics from the "what is really happening" box, now in your ears.

26.9 Channels, interleaving, and file formats

Real audio is rarely a single stream. **Stereo** carries two channels (left, right); surround formats carry more. In a PCM buffer the samples are usually *interleaved*: L, R, L, R, … in time order, so frame n holds all channels for instant n back-to-back. A two-channel, 16-bit, 44.1 kHz stream therefore consumes

$$44100 \times 2 \times 2 = 176{,}400 \text{ bytes per second} \approx 10.1 \text{ MB/min},$$

which is exactly the uncompressed bitrate of an audio CD. That number is why lossy codecs exist.

Focus

The format zoo, demystified.

- **WAV / AIFF** – raw PCM in a thin container. Lossless, large, universally readable. Our default for analysis because the samples are *exactly* what we recorded.
- **FLAC / ALAC** – *lossless* compression: smaller files, bit-for-bit identical PCM on decode. Good for archiving.
- **MP3 / AAC / Opus** – *lossy* compression: discards detail the ear is unlikely to notice (Chapter 33's psychoacoustic masking idea, applied to sound). Small files, but the PCM you get back is *not* the PCM you put in. Never analyze fine spectral detail from a heavily compressed file.

Example 26.5 (Sizing a recording)**.** How big is a 3-minute stereo song at CD quality, uncompressed? Bytes per second is 44100×2 ch $\times 2$ bytes $= 176{,}400$. Over 180 s that is $176{,}400 \times 180 = 31{,}752{,}000$ bytes ≈ 30.3 MB. The same song as a 256 kbit/s AAC file is $256{,}000 \times 180/8 \approx 5.76$ MB – about a fifth the size for near-transparent quality. The factor-of-five-ish saving is the everyday payoff of lossy coding.

26.10 Real limits, and the bridge to the wrist controller

Toward the wrist tracker

Where does digital audio meet our gesture controller? In two solid places, and one tempting trap.

Output (solid). The controller's job is to *make music*: a recognized gesture maps to a MIDI note, and the note maps to a frequency by the equal-tempered formula of Section 26.7, which a synth turns into PCM. Every concept here – sample rate, bit depth, headroom, dBFS – governs that output path. When we build the synthesizer in Chapter 28 we will be writing exactly the PCM streams this chapter defined.

Input (real but limited). A *contact* microphone or piezo pressed against the wrist can pick up the faint acoustic "clicks" of tendons and the thud of a finger tap – a field called *bioacoustic* sensing (Chapter 37). It is genuinely useful for detecting *events* (a tap happened; which of a few coarse gestures) using the audio pipeline of this chapter.

The trap. What a wrist microphone *cannot* do is recover continuous, individual finger *joint angles*. The acoustic signal is a smeared, overlapping mixture; it does not encode per-finger pose. Continuous finger tracking needs sensors *on the fingers* (rings, gloves) or, for muscle-level inference, EMG/FMG on the forearm – and even those give limited, calibrated estimates, not perfect joint angles. Treat wrist audio as an *event detector*, never as a finger-pose sensor.

Why this matters

The quiet triumph of this chapter is that one organ's limits – 20 Hz to 20 kHz, a 120 dB dynamic range, logarithmic in both axes – dictate almost every engineering number in digital audio. Sample just above twice 20 kHz; use enough bits that quantization noise sinks below the threshold of hearing; measure loudness in decibels because the ear does. Specs are not

arbitrary; they are the ear, written in arithmetic. Hold onto that habit – letting the *receiver* define the requirements – and it will guide every sensor choice in the chapters ahead, audio or motion alike.

We now have a firm grip on what a digital audio signal *is*: PCM samples at a chosen rate and bit depth, bounded by full scale, measured in decibels, carrying frequencies the ear can place on a log scale of octaves. The next chapter turns these samples into *features* – energy, zero crossings, spectral centroid, MFCCs – the compact descriptions that let a machine (and our gesture controller) tell one sound from another.

Exercises

Exercise 26.1. A mono recording is made at $f_s = 48{,}000$ Hz, 24-bit, lasting 90 seconds. (a) How many samples are stored? (b) How many bytes (assume 3 bytes per 24-bit sample)? (c) What is the Nyquist frequency, and is that enough to capture the full audible range?

Exercise 26.2. Using $\mathrm{SNR}_{\mathrm{dB}} \approx 6.02\,b + 1.76$, find the approximate SNR for $b = 10$, 14, and 20 bits. How many extra bits would you need to gain 30 dB of dynamic range over a 16-bit recording, and why is that number what it is?

Exercise 26.3. A tone of 19 kHz is recorded at $f_s = 32{,}000$ Hz with the anti-aliasing filter accidentally switched off. (a) What is the Nyquist frequency? (b) Does the tone alias, and if so, to what apparent frequency? (c) Is the aliased tone audible?

Exercise 26.4. You record a signal whose loudest peak reaches amplitude 0.25 on a $[-1, 1]$ scale. (a) What is the peak level in dBFS? (b) How much headroom (in dB) do you have? (c) If you later amplify the file by 9 dB, what is the new peak amplitude, and have you clipped?

Exercise 26.5. Show, using the $20\log_{10}$ definition, that doubling a signal's amplitude raises its level by about 6 dB and that multiplying its amplitude by 10 raises its level by exactly 20 dB. Then explain in one sentence why the power-based formula uses $10\log_{10}$ instead.

Exercise 26.6. (a) Compute the frequency of the note E5 (E above the treble staff), which is 7 semitones above A4. (b) Compute the frequency of A2, three octaves below A4. (c) What MIDI note number does each correspond to, given A4 is MIDI 69?

Exercise 26.7. A square wave at fundamental $f_0 = 5$ kHz is recorded at $f_s = 44{,}100$ Hz. Its harmonics fall at $3f_0, 5f_0, 7f_0, \ldots$. (a) List the harmonic frequencies up to 30 kHz. (b) Which of them exceed the Nyquist frequency and will alias? (c) Compute the aliased frequency of the first one that folds. (This is why clipping – which makes square-like waves – can sound metallic.)

Exercise 26.8. A full-scale sine has peak 0 dBFS. (a) Using $A_{\mathrm{rms}} = A/\sqrt{2}$, what is its RMS level in dBFS? (b) A full-scale square wave has $A_{\mathrm{rms}} = A$ (it is always at $\pm A$). What is *its* RMS level in dBFS? (c) Explain why, at the same peak level, a square wave sounds louder than a sine.

Exercise 26.9. Estimate the uncompressed storage for one hour of 5.1 surround sound (6 channels) at 48 kHz, 24-bit. Express the answer in gigabytes. Then estimate the same hour as a 1.5 Mbit/s lossy file and give the compression ratio.

Exercise 26.10. (Build.) Record a 5-second clip with `arecord` at 44.1 kHz/16-bit, then run the Python snippet from Section 26.8. Report peak dBFS, RMS dBFS, and clipped fraction. Re-record while speaking much louder and report the same three numbers. Which changed most, and does the clipped fraction behave the way Section 26.5 predicts?

Exercise 26.11. (Open-ended.) A product page advertises a wristband that "listens to your hand with a tiny microphone and reconstructs each finger's position in real time." Using Section 26.10, write three sentences separating the plausible claim from the implausible one, and name the sensor type that would actually be required for continuous finger-pose tracking.

Chapter 27

Audio Features

A spectrogram is a gorgeous picture, but a picture is not a number. If you want a machine to *decide* something – "is this speech or silence?", "is this a plucked string or a struck drum?", "did a note just start?", "which note is it?" – you need to boil the rich, high-dimensional waveform down to a handful of numbers that capture what matters and throw away what does not. Those numbers are **features**.

A feature is a measurement. The waveform of one second of CD audio is 44100 numbers; that is far too much to feed a classifier directly, and almost all of it is redundant or irrelevant for the question you actually care about. A feature is a function that takes that wall of samples and returns something small and meaningful – the energy, the brightness, the pitch, the rate of change. Good features are to a signal what a good summary is to a book: lossy on purpose, keeping the gist.

This chapter builds the classic audio features from the ground up, geometrically. We will see short-time energy and zero-crossing rate (the two cheapest features in all of audio), the spectral shape descriptors (centroid, spread, rolloff, flux), the mel scale and the MFCCs derived step by careful step, chroma for harmony, pitch detection by autocorrelation and the YIN refinement, and onset detection for rhythm. Throughout, keep one fact in mind: **every one of these features is a function of either the waveform or its spectrum, and almost every one of them applies, unchanged, to the motion signals from a wrist sensor.** Energy, zero-crossing rate, spectral centroid, and flux are exactly the features a gesture recognizer computes on accelerometer data. Learn them once here, on audio where your ears can check the answer, and you get them for free on the IMU.

27.1 What a feature is, geometrically

A frame of audio is a vector. Take N consecutive samples $\mathbf{x} = (x_0, x_1, \ldots, x_{N-1})$ – a typical frame is 20–40 ms, so N is a few hundred to a couple thousand samples. This frame lives in $\mathbb{R}^N$. A feature is a map $\mathbb{R}^N \to \mathbb{R}$ (or to a small vector) that collapses those N dimensions down to one or a few.

Intuition

Think of the frame as a single point in a very high-dimensional space. A feature is a *measuring instrument* you point at that point. Energy measures how far the point is from the origin. Zero-crossing rate measures how jagged the path through the samples is. The spectral centroid measures, after rotating into the Fourier basis, where the "center of mass" of the energy sits along the frequency axis. Each feature is a different ruler, and the art is choosing rulers that line up with the distinction you care about.

Because audio changes over time, we almost never compute a feature on the whole signal. We compute it *per frame*, sliding a window along the signal exactly as we did for the STFT in Chapter 10. The result is a **feature contour**: a new, much shorter, much slower signal – one number (or a few) per frame, perhaps every 10 ms. We have traded 44100 samples per second for, say, 100 feature vectors per second. That is the whole game: turn an audio stream into a low-rate stream of descriptors that a downstream stage – a threshold, a state machine, a classifier – can act on.

Focus

The pipeline for the entire chapter is always the same three steps:

1. **Frame.** Slice the signal into overlapping windows (e.g. 25 ms frames, hopping 10 ms).
2. **Window.** Multiply each frame by a taper (Hann, Hamming) to avoid edge artifacts – see Chapter 10.
3. **Measure.** Apply a feature function to each frame, producing one number (or vector) per frame.

Different features differ only in step 3. Get the framing right once and you can swap features freely.

Connection

A feature map $\mathbb{R}^N \to \mathbb{R}$ is exactly the kind of object the linear-algebra book calls a *functional*. The linear ones (like "the DC value" or any single DFT bin) are inner products with a fixed vector; the nonlinear ones (energy is a quadratic form, ZCR involves sign changes) are not, but the geometric picture – projecting a high-dimensional point down to something measurable – is the same. And when a feature is an average over a frame, it is an *expectation* in disguise, which is why the probability book's tools for means and variances apply directly to feature contours.

27.2 Short-time energy: how loud, right now

The simplest question you can ask of a frame is: how much is going on? The answer is its **energy**, the squared length of the frame vector:

$$E[m] = \sum_{n=0}^{N-1} \big(x[mH + n]\, w[n]\big)^2 = \|\mathbf{x}_m\|^2, \tag{27.1}$$

where m indexes the frame, H is the hop size, and $w[n]$ is the window. This is nothing but $\|\cdot\|^2$ from Chapter 3, applied per frame. Silence sits near the origin ($E \approx 0$); a loud passage sits far out.

Two cosmetic variants are common. The **root-mean-square (RMS)** amplitude divides by N and takes a square root, $\mathrm{RMS}[m] = \sqrt{E[m]/N}$, which puts it back in the same units as the samples (it is the standard deviation of the frame when the mean is zero). And because loudness is perceived roughly logarithmically, we usually report energy in **decibels**, $10 \log_{10}(E[m] + \varepsilon)$, with a tiny ε to avoid $\log 0$.

Why it works

Why square, rather than sum the absolute values? Squaring makes energy a smooth, differentiable quadratic form $\mathbf{x}^\top \mathbf{x}$, it matches the physical notion of energy (power is amplitude squared), and – crucially – it connects to the spectrum through Parseval's theorem: the energy you compute in time (27.1) equals (up to a constant) the sum of squared magnitudes of the DFT. The same number, measured in two bases. That is why energy is the bridge feature between the time-domain features of this section and the spectral features of the next.

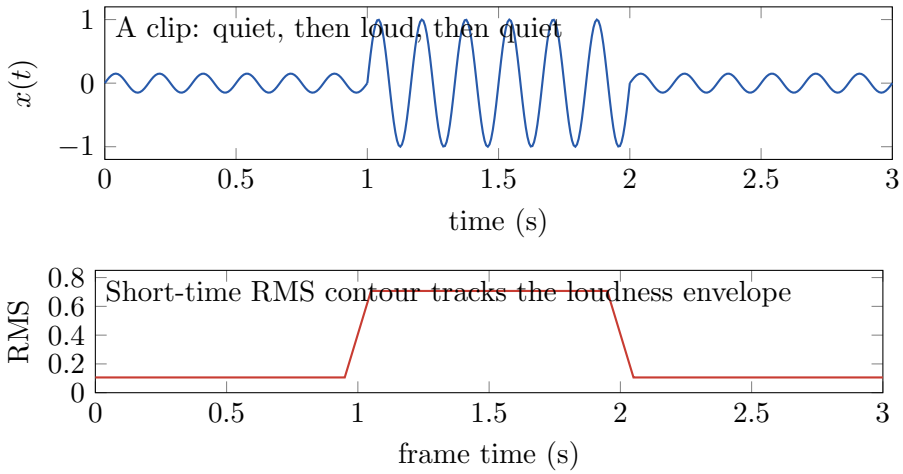


Figure 27.1: Top: a waveform that is quiet, then loud, then quiet. Bottom: the short-time RMS contour, one value per frame, recovers the loudness envelope – a slow, low-rate signal you can threshold to find "where the sound is."

Example 27.1 (Energy of a pure tone)**.** Take a frame of a sinusoid $x[n] = A \sin(2\pi f n / f_s)$ with no window. Over an integer number of cycles, $\sum \sin^2 = N/2$, so $E = A^2 N/2$ and $\mathrm{RMS} = A/\sqrt{2}$. That $\sqrt{2}$ is exactly the peak-to-RMS ratio of a sine you may know from AC circuits: a sine of amplitude 1 has RMS 0.707. The RMS contour of Figure 27.1 jumps from $0.15/\sqrt{2} \approx 0.106$ to $1/\sqrt{2} \approx 0.707$, which is precisely what the plot shows.

Common trap

Energy is not loudness. Human loudness perception is frequency-dependent (a 60 Hz tone and a 3 kHz tone of equal energy do *not* sound equally loud) and compressive. Raw RMS is a fine *relative* measure for finding where the action is, but do not call it loudness without an equal-loudness weighting (A-weighting) and a logarithmic scale. The same caution applies to motion: the acceleration "energy" of a flick and a slow wave can be similar while they feel completely different.

27.3 Zero-crossing rate: how often it flips sign

The second-cheapest feature counts how often the waveform crosses zero within a frame:

$$\mathrm{ZCR}[m] = \frac{1}{2N} \sum_{n=1}^{N-1} \big| \operatorname{sgn}(x[mH + n]) - \operatorname{sgn}(x[mH + n - 1]) \big|. \tag{27.2}$$

Each term is 1 when consecutive samples have opposite signs and 0 otherwise; dividing by $2N$ gives a rate per sample (multiply by f_s for crossings per second). It costs only comparisons and additions – no multiplies, no FFT – which is why early speech chips and the cheapest microcontrollers leaned on it heavily.

Intuition

Zero-crossing rate is a crude pitch-and-noisiness meter rolled into one. A pure low tone crosses zero twice per cycle, so its ZCR is small and proportional to frequency. White noise wiggles constantly, so its ZCR is high. A periodic, low-pitched sound (a vowel, a bowed cello) has low ZCR; a noisy, high-frequency sound (the hiss of /s/, a cymbal, a snare) has high ZCR. So ZCR is the classic **voiced/unvoiced** discriminator in speech, and more generally a tonal-versus-noisy detector.

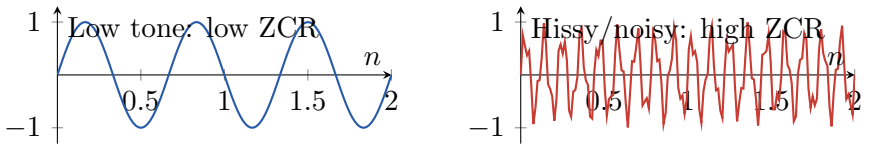


Figure 27.2: Zero-crossing rate distinguishes tonal from noisy. The slow tone (left) flips sign rarely; the high-frequency, hissy signal (right) flips constantly.

Common trap

ZCR is fooled by DC offset and by noise near zero. A small constant bias can stop a quiet signal from ever crossing zero (artificially low ZCR), while broadband noise sitting on top of a clean tone adds spurious crossings each time the sum dithers across zero (artificially high ZCR). Always remove the mean (high-pass / DC-block) before computing ZCR, and consider a small hysteresis band around zero so that tiny wiggles do not register as crossings.

Toward the wrist tracker

Short-time energy and ZCR are the first two features your wrist controller will ever compute, and it will compute them on accelerometer magnitude, not audio. Energy of the acceleration signal tells you *how vigorous* a motion is – the difference between resting and gesturing – and is the natural trigger to wake the recognizer from a low-power idle. ZCR of a gyroscope axis tells you how *oscillatory* the motion is – a tremor or a fast shake has high ZCR; a smooth sweep has low ZCR. Both cost a few additions per sample, which is exactly what a battery-powered microcontroller on your forearm can afford to run continuously. The caveat worth stating: these coarse features detect *that* a gesture happened and roughly how energetic it was; they do not, on their own, recover which fingers moved. That finer question waits for Chapter 37.

27.4 Spectral shape: centroid, spread, rolloff

Now cross into the frequency domain. Take the magnitude spectrum of a frame, $|X[k]|$ for bins $k = 0, \dots, N/2$ at frequencies $f_k = k f_s / N$. Treat the *energy per bin*, $|X[k]|^2$ (or sometimes $|X[k]|$), as a distribution of "mass" along the frequency axis. Once you see the spectrum as a mass distribution, the language of statistics describes its shape exactly.

Spectral centroid: the center of mass (brightness)

The **spectral centroid** is the energy-weighted mean frequency:

$$\text{centroid}[m] = \frac{\sum_k f_k \, |X_m[k]|}{\sum_k |X_m[k]|}. \tag{27.3}$$

This is precisely the formula for the center of mass of point masses $|X[k]|$ at positions f_k – or, in probability language, the *expected frequency* $\mathbb{E}[f]$ under the distribution $p_k = |X[k]| / \sum_j |X[j]|$. A spectrum with most of its energy down low (a bass note, a vowel) has a low centroid; one with energy up high (a cymbal, the hiss of /s/) has a high centroid. The centroid correlates remarkably well with the perceptual notion of **brightness**.

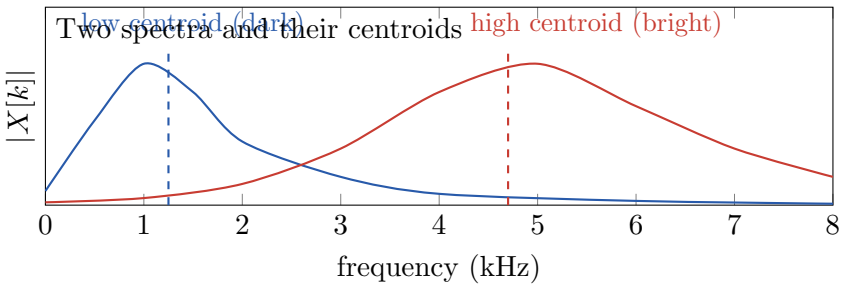


Figure 27.3: The spectral centroid is the balance point of the magnitude spectrum. The blue spectrum (energy low) balances around 1.25 kHz; the red spectrum (energy high) balances around 4.7 kHz and sounds brighter.

Spectral spread, skewness, flatness

If the centroid is the mean, the **spectral spread** (or bandwidth) is the standard deviation – how wide the energy is smeared around the centroid:

$$\text{spread}[m] = \sqrt{\frac{\sum_k (f_k - \text{centroid}[m])^2 \, |X_m[k]|}{\sum_k |X_m[k]|}}. \tag{27.4}$$

A pure tone has near-zero spread; broadband noise has huge spread. Carrying the moment-by-moment analogy further gives spectral *skewness* (lopsidedness) and *kurtosis* (peakiness), all borrowed verbatim from the moments of a probability distribution.

A close cousin is **spectral flatness**, the ratio of the geometric mean to the arithmetic mean of the spectrum:

$$\text{flatness}[m] = \frac{\left(\prod_k |X_m[k]|^2\right)^{1/K}}{\frac{1}{K}\sum_k |X_m[k]|^2}. \tag{27.5}$$

Flatness is near 1 for white noise (a flat spectrum: geometric mean equals arithmetic mean) and near 0 for a pure tone (energy in one bin makes the geometric mean collapse). It is the standard **tonal-versus-noisy** measure, the spectral-domain partner of ZCR, and it is what audio codecs use to decide how aggressively they can quantize a band.

Spectral rolloff: the frequency below which most energy lives

The **spectral rolloff** $R[m]$ is the frequency below which a fixed fraction κ (commonly 0.85 or 0.95) of the total spectral energy is contained:

$$R[m] = \min\Big\{ f_K : \sum_{k \le K} |X_m[k]|^2 \ge \kappa \sum_{k} |X_m[k]|^2 \Big\}. \tag{27.6}$$

In probability terms this is just the *κ-quantile* of the spectral energy distribution. Rolloff, like centroid, tracks brightness and the voiced/unvoiced distinction, but it is more robust to a few stray high-frequency bins because it asks about cumulative energy rather than a mean.

Example 27.2 (Centroid of a two-tone frame)**.** Suppose a frame's spectrum has just two significant lines: amplitude 3 at 200 Hz and amplitude 1 at 1000 Hz. The centroid is

$$\frac{3 \cdot 200 + 1 \cdot 1000}{3 + 1} = \frac{1600}{4} = 400 \text{ Hz}.$$

The strong low line pulls the balance point down toward it – exactly as a heavier mass pulls a center of gravity. If you added high-frequency noise the centroid would climb, which is the basis for using it as a brightness/noise detector.

27.5 Spectral flux: how fast the spectrum is changing

Everything so far described a *single* frame. **Spectral flux** measures the *change* between consecutive frames – how different this frame's spectrum is from the last one:

$$\text{flux}[m] = \sum_{k} \big(|X_m[k]| - |X_{m-1}[k]| \big)_+^2, \tag{27.7}$$

where $(\cdot)_+ = \max(\cdot, 0)$ keeps only the bins that *grew* (half-wave rectification). Geometrically, flux is (a rectified version of) the squared Euclidean distance between two successive spectrum vectors: it is large when the sound's frequency content suddenly shifts and near zero when the sound is steady.

Why it works

Why keep only the increases? Because we usually use flux to find *onsets* – moments when new energy appears (a note begins, a consonant bursts, a drum is struck). When a note *ends*, energy in its bins *decreases*; that is a release, not an onset, and we do not want it to trigger. Half-wave rectification keeps only rising energy, so flux peaks at the births of sounds, not their deaths. This makes spectral flux the workhorse of onset detection in Section 27.10.

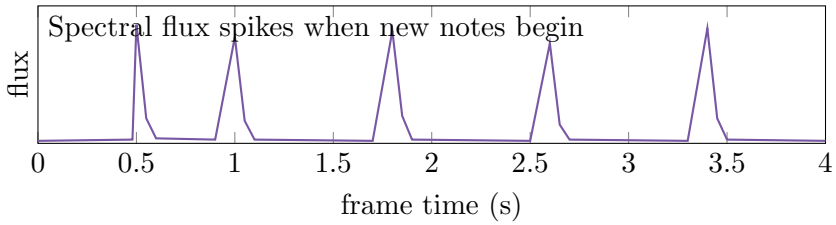


Figure 27.4: Spectral flux over a clip of five separated notes. Each note onset produces a sharp spike; the steady-state and silence between notes produce almost no flux. Picking the peaks of this curve detects onsets.

Connection

Flux is a discrete time-derivative of the spectrum, the same operation the calculus book studies as a difference quotient, applied here to a vector-valued signal (the whole spectrum at each frame). Many features in this chapter are "derivative-like": flux is the derivative of the spectrum, onset strength is the derivative of energy, and the famous *delta* and *delta-delta* MFCCs are literally first and second derivatives of the cepstral features over time. A recurring theme: *what* something is comes from one frame; *events* come from how it changes between frames.

27.6 The mel scale: hearing as the ear does

So far our spectral features used a *linear* frequency axis: bin k sits at $f_k = kf_s/N$, equally spaced. But human hearing is not linear. We are exquisitely sensitive to changes among low frequencies and progressively coarser up high. The musical interval from 100 to 200 Hz (one octave) sounds like the same "distance" as 1000 to 2000 Hz, even though the second spans ten times as many hertz. Pitch perception is roughly *logarithmic*.

The **mel scale** is an empirical warping of the frequency axis built to make equal perceptual distances correspond to equal numeric distances. A common formula is

$$m(f) = 2595 \log_{10}\!\left(1 + \frac{f}{700}\right), \tag{27.8}$$

which is nearly linear below about 1 kHz and logarithmic above it – matching the ear's behavior. "Mel" comes from "melody"; the scale was anchored so that 1000 mel equals a 1000 Hz tone at a reference loudness.

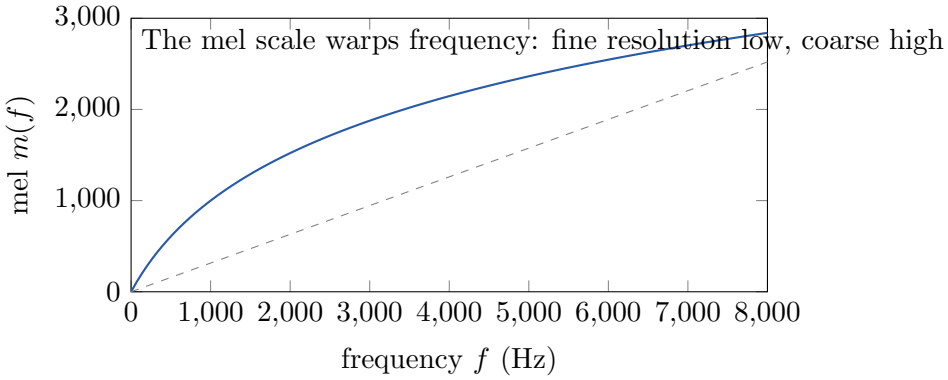


Figure 27.5: The mel mapping (blue) versus a straight line (gray). Equal steps on the vertical mel axis correspond to ever-wider hertz steps as frequency rises, which is why a mel-spaced filterbank places many narrow filters down low and a few wide ones up high.

The mel filterbank

We do not resample the spectrum onto the mel axis directly; instead we pool the linear spectrum into a small number of overlapping **triangular filters** spaced evenly on the mel axis. Each filter is a triangle in frequency: it rises from zero at the previous center, peaks at its own center, and falls to zero at the next center. Because the centers are equally spaced *in mel*, the triangles are narrow at low frequency and wide at high frequency. We compute, for each of (typically) 26–40 filters, the weighted sum of spectral energy falling under its triangle. The output is the **mel spectrum**: a short vector that summarizes the spectral envelope the way the ear's frequency analysis roughly does.

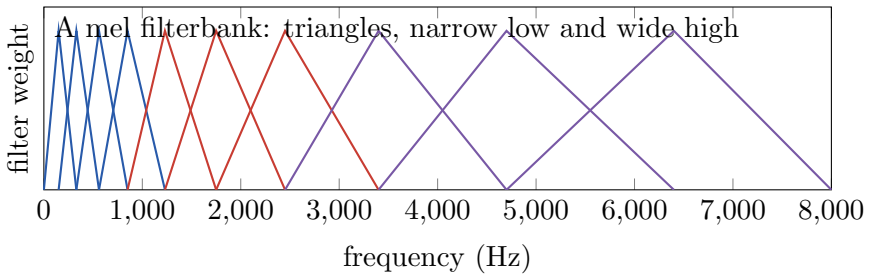


Figure 27.6: A mel filterbank. Filter centers are equally spaced on the mel axis, so in hertz the triangles bunch up at low frequency (fine resolution where the ear cares) and stretch out at high frequency (coarse resolution where it does not).

History

The mel scale grows out of 1930s–1940s psychoacoustics, notably the work of Stevens, Volkmann, and Newman (1937), who asked listeners to adjust tones until one pitch felt "twice as high" as another and built a perceptual scale from the answers. Decades later, when engineers needed a front-end for automatic speech recognition that mimicked the ear, this old perceptual scale was exactly the warping they reached for – a lovely case of mid-century perception experiments quietly becoming the bedrock of modern speech technology.

27.7 MFCCs, derived step by step

Mel-frequency cepstral coefficients (MFCCs) are the single most famous audio feature – the backbone of speech recognition for decades and still a strong baseline for any audio classification task. They look intimidating as a formula but each step has a clear geometric purpose. Let us build them from the mel spectrum we just computed.

Focus

The MFCC recipe, in seven steps:

1. **Frame and window** the signal (Hann/Hamming, ∼25 ms).
2. **FFT**; take the magnitude (or power) spectrum $|X[k]|^2$.
3. **Mel filterbank**: pool into ∼26–40 mel-band energies.
4. **Logarithm**: take log of each band energy.
5. **DCT**: apply a discrete cosine transform across the bands.
6. **Keep the low coefficients** (typically the first 12–13).
7. **(Optional) deltas**: append first/second time-derivatives.

Steps 1–3 we have already met. The magic is in steps 4 and 5; here is why each one is there.

Step 4: the logarithm separates source from filter

Speech (and most pitched instruments) is well modeled as a *source* – the buzzing vocal folds or vibrating string, rich in harmonics – passed through a *filter* – the resonances of the vocal tract or instrument body that shape which harmonics are loud. In the frequency domain these *multiply*: the observed spectrum is (source spectrum) × (filter response). Multiplication is hard to separate; but $\log(ab) = \log a + \log b$ turns the product into a *sum*. After the log, the slowly varying filter envelope and the rapidly varying harmonic source sit side by side as additive components, ready to be pried apart.

What is really happening

This is the central trick of the *cepstrum* (an anagram of "spectrum"). Think of the log-spectrum as itself a kind of signal, but one whose "time" axis is frequency. The vocal-tract filter is a *slowly* varying shape across frequency – a few broad bumps (the formants). The harmonic source is a *fast* ripple across frequency – a comb of evenly spaced harmonics. "Slow" and "fast" variation across the frequency axis means *low* and *high* "frequency" of this new signal. So a Fourier-like transform of the log-spectrum sorts filter (slow, low) from source (fast, high). Keeping the low coefficients keeps the filter shape – the timbre – and discards the pitch. That is exactly what you want for recognizing *what* was said regardless of *what pitch* it was said at.

Step 5: the DCT decorrelates and compresses

To Fourier-analyze the log-mel spectrum we use the **discrete cosine transform** (DCT) rather than the full DFT. The DCT expresses the M-vector of log-mel energies as a sum of slow-to-fast cosine shapes:

$$c[i] = \sum_{j=0}^{M-1} \log\big(S[j]\big)\, \cos\Big[\frac{\pi i}{M}\Big(j + \tfrac{1}{2}\Big)\Big], \qquad i = 0, 1, \ldots, M-1, \tag{27.9}$$

where $S[j]$ is the j-th mel band energy. The coefficients $c[i]$ are the MFCCs. We keep the low-order ones ($i = 1, \ldots, 12$ or so), which describe the broad, smooth shape of the spectral envelope, and drop the high-order ones, which capture the fine harmonic ripple (pitch) we want to ignore.

Why it works

Why the DCT and not the DFT? Three reasons, all geometric. First, the log-mel spectrum is real, and the DCT is the natural real-input cousin of the DFT (it assumes an even extension, so it needs only cosines – no imaginary part, no phase to worry about). Second, the DCT is famously good at *energy compaction*: for the smooth, correlated log-mel envelopes that speech produces, it packs almost all the information into the first few coefficients, so 12 numbers suffice. Third – and this is the deep reason – for signals with the kind of correlation structure speech has, the DCT basis is very nearly the Karhunen–Loève (eigenvector) basis, the optimal one that *decorrelates* the components. Decorrelated features are exactly what simple classifiers (which assume diagonal covariance) want. The DCT is doing a free, fixed, fast approximation to PCA.

What the coefficients mean

$c[0]$ is the average of the log-mel energies – essentially overall log-energy (loudness), and often dropped or replaced by the frame energy. $c[1]$ measures the overall *tilt* of the spectrum (energy concentrated low versus high – a brightness/balance axis, close cousin of the centroid). $c[2]$ measures whether energy is concentrated in the middle versus the edges, and so on: each higher coefficient corresponds to a faster cosine ripple across the mel bands, describing ever-finer detail of the spectral envelope shape.

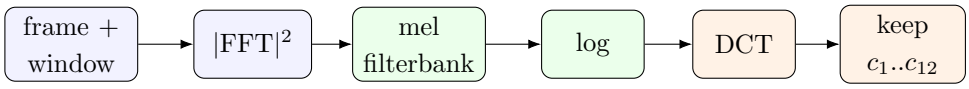


Figure 27.7: The MFCC pipeline. Source–filter separation happens at the log step; the DCT then decorrelates and compresses the smooth envelope into a dozen numbers.

Example 27.3 (Why the log matters: a worked sketch)**.** Imagine two recordings of the same vowel /a/, one sung at 110 Hz and one at 220 Hz. The vocal-tract filter (the formant shape that makes it an /a/) is identical; only the harmonic comb differs. Before the log, doubling the source amplitude or changing its harmonic spacing scrambles the spectrum multiplicatively. After log, the formant envelope is the same additive "baseline" in both, and the harmonic combs differ only in their fast ripple – which the DCT shoves into high coefficients you discard. The surviving low MFCCs are nearly identical for both pitches: the feature has become *pitch-invariant*, recognizing the /a/-ness. That invariance is the entire reason MFCCs dominated speech recognition.

Common trap

MFCCs deliberately throw away pitch. That is a feature for speech recognition and a *bug* for tasks where pitch is the point – melody transcription, key detection, singer identification. For those, use chroma (next section) or a pitch tracker. Choosing MFCCs for a melody task is one of the most common beginner mistakes in music information retrieval.

Deltas: motion in the cepstrum

A single MFCC vector describes one 25 ms snapshot, but speech and gesture are about *trajectories*. The **delta** coefficients approximate the time derivative of each MFCC,

$$\Delta c[i, m] \approx \frac{\sum_{\theta=1}^{\Theta} \theta \, (c[i, m+\theta] - c[i, m-\theta])}{2 \sum_{\theta=1}^{\Theta} \theta^2}, \tag{27.10}$$

a regression slope over a few neighboring frames, and the **delta-delta** coefficients do it again for the second derivative. Appending deltas typically triples the feature dimension ($13 \to 39$) and captures *how the spectrum is moving* – which, as we keep seeing, is where the events live.

27.8 Chroma: folding the spectrum onto the twelve pitch classes

For music we often want the opposite of MFCCs: we want to keep harmony and pitch class while ignoring timbre and octave. The **chroma** (or pitch-class profile) feature does this by folding the spectrum onto the twelve notes of the chromatic scale – C, C#, D, …, B – summing the energy of all octaves of each pitch class into a single bin.

The construction uses the same logarithmic ear-fact as the mel scale, but applied musically: an octave is a doubling of frequency, and Western music splits each octave into twelve equal ratios of $2^{1/12}$ (the equal-tempered semitone). To build chroma, map each spectral bin to its nearest musical pitch, then to that pitch's class modulo 12 (collapsing octaves), and accumulate energy into a 12-dimensional vector:

$$\text{chroma}[p] = \sum_{k \,:\, \text{class}(f_k) = p} |X[k]|^2, \qquad p = 0, \ldots, 11. \tag{27.11}$$

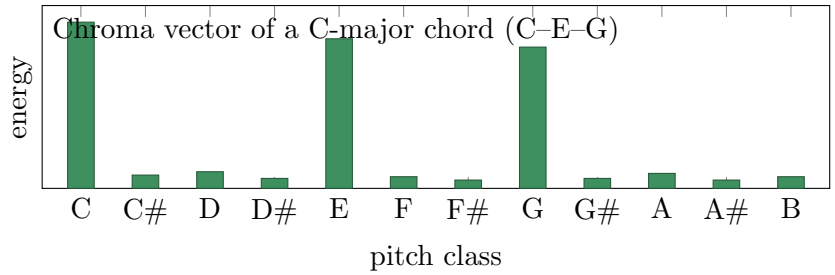


Figure 27.8: Chroma of a C-major chord: energy piles up on C, E, and G regardless of which octaves the notes are played in or what instrument plays them. The small nonzero values on other classes are leakage from harmonics and windowing.

Intuition

Chroma is the musical inverse of MFCCs. MFCCs keep the spectral *envelope* (timbre) and discard pitch; chroma keeps the *pitch class* and discards both timbre and octave. A C-major chord on a piano and on a guitar have very different MFCCs (different timbres) but nearly identical chroma (same notes). That octave- and timbre-invariance is exactly what you want for chord recognition, key estimation, and matching cover versions of a song.

Application

Chroma underlies practical music systems: chord-recognition apps that print guitar tabs from audio, key-detection in DJ software that matches songs in compatible keys, and audio-to-score alignment that follows a live performer through a known piece. Comparing the sequence of chroma vectors between two recordings (via dynamic time warping) is also how "cover song" and audio-fingerprint-adjacent systems decide that two performances are the same underlying song.

27.9 Pitch detection: finding the fundamental period

Pitch is not the same as a single feature value – it is an estimate of the **fundamental frequency** f_0, the rate at which a (quasi-)periodic signal repeats. A vowel, a bowed string, a sung note: all repeat a basic waveform shape many times per frame. Pitch detection asks: what is the period of that repetition?

Common trap

"Just take the FFT and find the biggest peak." This fails constantly. For many instruments and voices the fundamental is *weaker* than its harmonics, or even physically absent (the "missing fundamental" – a telephone removes everything below 300 Hz, yet you still hear a 150 Hz male voice at its true pitch because your brain infers f_0 from the harmonic *spacing*). The strongest spectral peak is often the second or third harmonic. Robust pitch detection looks at *periodicity*, not at peak height.

Autocorrelation: how well a signal matches a shifted copy of itself

The cleanest way to find a period is to slide the signal against a copy of itself and see at which shift it lines up best. That is the **autocorrelation**:

$$r[\tau] = \sum_{n} x[n]\, x[n+\tau]. \tag{27.12}$$

At lag $\tau = 0$ the signal perfectly overlaps itself, giving the maximum (that is just the energy). As τ grows, $r[\tau]$ dips and then, when τ equals one period T_0, the signal lines up with itself again and $r[\tau]$ peaks once more. The first strong peak after lag zero sits at $\tau = T_0$, and $f_0 = f_s/T_0$.

Why it works

Autocorrelation is an inner product (Chapter 3) between the signal and a time-shifted version of itself: $r[\tau] = \langle x[n],\, x[n+\tau] \rangle$. An inner product measures alignment – how much two vectors point the same way. When the shift equals the period, the shifted copy is nearly identical to the original, so their alignment (inner product) is large. Pitch detection by autocorrelation is therefore literally "at what shift does the signal most resemble itself?" – a purely geometric question.

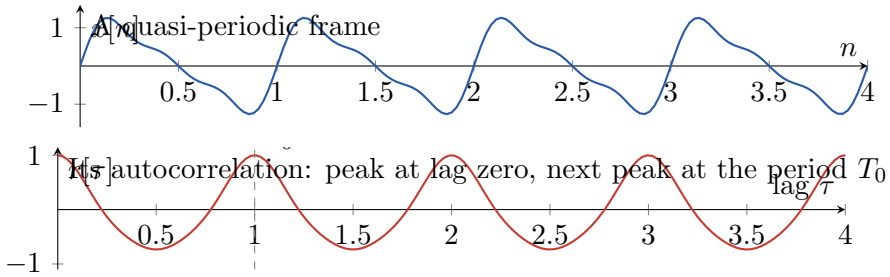


Figure 27.9: Top: a periodic frame. Bottom: its autocorrelation peaks at lag 0 and again at the period T_0. Locating that first non-trivial peak gives the period, hence $f_0 = f_s/T_0$.

Connection

By the convolution theorem (Chapter 11), autocorrelation can be computed *fast* as $r = \mathrm{IFFT}\big(|X[k]|^2\big)$: the inverse transform of the power spectrum. So pitch detection in time (autocorrelation) and pitch analysis in frequency (the power spectrum) are two views of the same object – the Wiener–Khinchin theorem made practical. This also reveals why "peak in spectrum" and "peak in autocorrelation" are related but not identical: the autocorrelation folds all the harmonics back onto the single period, which is exactly why it sees the true f_0 even when the fundamental bin is weak.

YIN: fixing autocorrelation's mistakes

Plain autocorrelation has two well-known failure modes: it can pick lag zero or a too-short lag (*octave-too-high* errors, since a half-period also correlates somewhat), and amplitude changes within the frame bias the peaks. The **YIN** algorithm (de Cheveigné and Kawahara, 2002) is a celebrated set of refinements that makes period detection far more reliable. Its key ideas:

1. **Difference function instead of correlation.** Use the squared difference $d[\tau] = \sum_n (x[n] - x[n+\tau])^2$, which is *zero* when the shifted copy matches (a valley to find) rather than a peak. This is more stable than correlation under amplitude variation.

2. **Cumulative mean normalization.** Divide $d[\tau]$ by its running average up to τ. This deliberately makes lag zero *not* a winner and suppresses the spurious sub-period dips that cause octave errors.

3. **Absolute threshold.** Take the first valley that drops below a fixed threshold (e.g. 0.1), rather than the global minimum – this prefers the true (longest, lowest-pitch) period over its

harmonics.

4. **Parabolic interpolation.** Fit a parabola around the chosen valley to estimate the period to sub-sample precision, beating the resolution of the integer lags.

YIN and its descendants (pYIN, CREPE in the neural era) are what real pitch trackers use. The lesson is general: the naive idea (autocorrelation) is correct in spirit, and the engineering is in defeating its predictable biases – octave errors, amplitude sensitivity, and discrete resolution.

Example 27.4 (From lag to frequency)**.** At $f_s = 16{,}000\,\text{Hz}$, suppose the difference function's first deep valley sits at lag $\tau = 145$ samples, and parabolic interpolation refines it to $\tau = 145.3$. Then

$$f_0 = \frac{f_s}{\tau} = \frac{16000}{145.3} \approx 110.1 \text{ Hz},$$

an A2 – the open A string of a bass guitar. Note how the interpolation matters: the integer lag 145 would give 110.34 Hz and lag 146 would give 109.6 Hz, a difference you can hear. Sub-sample interpolation is not a luxury for pitch.

27.10 Onset detection: when did something happen

Rhythm is about *events in time*: the instants when notes begin, drums hit, syllables start. An **onset** is the start of a new sonic event. Detecting onsets turns continuous audio into a sequence of time stamps – the raw material for beat tracking, tempo estimation, automatic transcription, and rhythm games.

The standard recipe builds an **onset strength signal** (also called a detection function), one value per frame that is large at onsets and small otherwise, then picks its peaks. The building blocks are features we already have:

- **Energy difference.** The simplest detector: the positive first difference of the (log) energy contour. Works for percussive, sharply-attacked sounds (drums, plucks) where energy jumps suddenly.

- **Spectral flux.** Section 27.5, the half-wave-rectified change in the magnitude spectrum. Catches onsets that change *timbre* or pitch without a big energy jump (a legato note change, a bowed attack) – the most widely used general-purpose onset feature.

- **Phase deviation.** For soft, blurry onsets, the *phase* of each bin advances predictably during steady tones; an onset disrupts that predictability. Phase-based and complex-domain detectors combine magnitude flux with phase irregularity for the hardest cases.

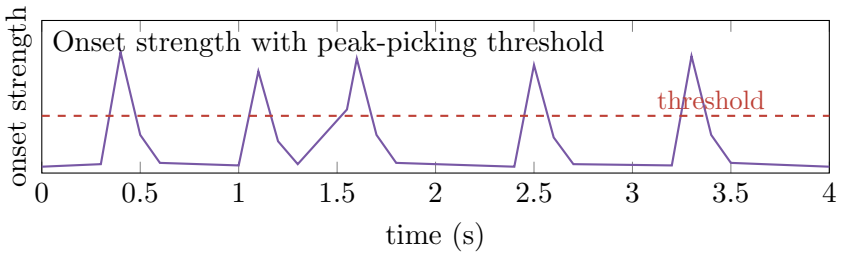


Figure 27.10: An onset-strength contour. Peaks above an adaptive threshold (red) are declared onsets. The threshold usually tracks a local moving average so that quiet and loud passages are handled fairly.

Peak picking

Turning the strength curve into discrete onsets needs care, because the curve is noisy and onsets come in clusters. Practical peak picking: (1) smooth the strength signal lightly; (2) require a peak to exceed a local moving-average threshold by a margin, so loud and quiet sections are treated fairly; (3) enforce a minimum spacing between onsets (a refractory period of a few tens of milliseconds) so a single attack is not counted several times. These three steps – smooth, adaptive-threshold, refractory – recur in every event detector, audio or not.

History

The 2000s saw a friendly arms race over onset detection driven by the MIREX evaluation campaigns, where research groups competed on shared annotated audio. Spectral flux, complex-domain, and phase-based detectors were systematically compared; the broad finding – that spectral flux is a remarkably strong, simple baseline and that the hard cases are soft onsets in expressive music – still shapes how onset detectors are built today, even neural ones, which often just learn a better detection function over the same framed spectra.

27.11 Putting it together: a feature pipeline on a clip

The features in this chapter are rarely used alone; a real system computes several per frame and stacks them into a feature vector, then feeds the stream to a classifier or a state machine. Here is a compact, correct sketch of the front end in Python/NumPy.

Listing 27.1: Framing plus a handful of features per frame.

```
import numpy as np

def frames(x, N=1024, H=512):
    # generator of windowed frames
    w = np.hanning(N)
    for m in range(0, len(x) - N, H):
        yield x[m:m+N] * w

def features(frame, fs):
    # time-domain
```

```
    energy = np.sum(frame**2)
    rms    = np.sqrt(energy / len(frame))
    sgn    = np.sign(frame)
    zcr    = np.mean(np.abs(np.diff(sgn))) / 2.0
    # spectral
    X  = np.abs(np.fft.rfft(frame))
    fk = np.fft.rfftfreq(len(frame), 1.0 / fs)
    Xs = X.sum() + 1e-12
    centroid = np.sum(fk * X) / Xs
    spread   = np.sqrt(np.sum((fk - centroid)**2 * X) / Xs)
    cumE     = np.cumsum(X**2)
    rolloff  = fk[np.searchsorted(cumE, 0.85 * cumE[-1])]
    return dict(rms=rms, zcr=zcr, centroid=centroid,
                spread=spread, rolloff=rolloff)

def spectral_flux(prev_mag, mag):
    d = mag - prev_mag
    return np.sum(np.maximum(d, 0.0)**2)   # half-wave rectified
```

Walking question

Run this front end on a clip and you get five contours (RMS, ZCR, centroid, spread, rolloff) plus a flux contour, each at the frame rate. Look at them together: during a vowel, RMS is high, ZCR is low, centroid is low; during an /s/, RMS is moderate, ZCR is high, centroid is high. During silence everything drops. You have, in a few dozen lines, built a representation in which speech-versus- silence and voiced-versus-unvoiced are nearly linearly separable – the kind of input a tiny classifier can handle. Ask yourself: which single feature best separates your particular clip's classes? That is the feature-selection question every applied project must answer.

Build it (hardware)

On a microcontroller you will not run an FFT every frame for free, so budget deliberately. The cheap tier – RMS energy and ZCR – needs no FFT and runs comfortably at full sample rate on an Arduino-class chip; use it as an always-on gate. The spectral tier – centroid, rolloff, flux – needs one real FFT per frame; a 256- or 512-point fixed-point FFT (e.g. CMSIS-DSP's `arm_rfft`) on a Cortex-M4 at 80–120 MHz fits easily in a 10 ms hop. The expensive tier – a full mel filterbank plus DCT for MFCCs – is feasible on an M4/M7 but is where you start watching the clock and the battery. A common embedded pattern: gate on energy, and only run the FFT/MFCC stage when the gate is open. This same tiered budgeting is exactly how the wrist controller will manage its own feature extraction.

27.12 The same features on motion signals

Here is the payoff promised at the start. A tri-axial accelerometer streams three signals $a_x[n], a_y[n], a_z[n]$ at, say, 100–200 Hz; a gyroscope adds three more. These are signals exactly like audio, only slower and lower-dimensional, and the features of this chapter transfer with almost no change:

- **Energy / RMS** of the acceleration magnitude $\sqrt{a_x^2 + a_y^2 + a_z^2}$ measures motion intensity – resting versus gesturing – and is the standard activity gate.
- **Zero-crossing rate** of a band-passed axis measures how oscillatory a motion is, distinguishing a shake or tremor from a smooth sweep.
- **Spectral centroid and rolloff** of a short motion FFT capture whether the motion energy is slow (a deliberate wave) or fast (a flick or vibration).
- **Spectral flux** marks the *onset* of a gesture, the motion analogue of a note attack – the trigger that says "a gesture is starting, begin classifying."
- **MFCC-style features** (a small mel-like filterbank plus DCT on the motion spectrum) are used in human-activity-recognition research and work surprisingly well for distinguishing walking, typing, and gesturing.

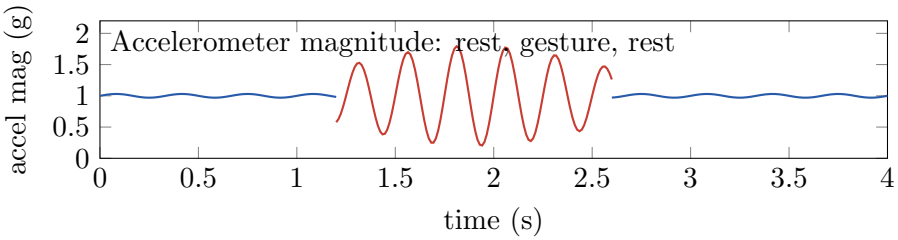


Figure 27.11: A motion trace with a gesture in the middle. Short-time energy rises in the gesture region (intensity gate); spectral flux spikes at its start (onset); centroid reports how fast the motion is. The very same code from Section 27.11 produces these contours.

Toward the wrist tracker

For the wrist controller, the plan is now concrete. An always-on energy gate on accelerometer magnitude wakes the system; a spectral-flux onset detector marks the gesture's start; and a short feature vector (energy, ZCR, centroid, a few mel-DCT coefficients, plus their deltas) computed over the gesture window feeds the classifier that decides *which* gesture it was, which is then mapped to a MIDI message. Every block in that sentence is a feature from this chapter. **The limit, stated plainly:** this pipeline recognizes coarse, discrete wrist and forearm gestures – a flick, a twist, a tap, a hold. A wrist-worn IMU *cannot* recover continuous individual finger joint angles; the wrist simply does not move enough, distinctly enough, when only a finger moves. Detecting which finger pressed, or tracking continuous finger pose, needs sensors that see the muscles or the fingers themselves – surface EMG or force myography on the forearm for discrete finger events, or instrumented rings for continuous pose. We confront those sensors and their own features in Chapter 37. Audio-style features get you robust *gesture* control; they do not get you a glove.

27.13 Summary

A feature collapses a high-dimensional frame to a few meaningful numbers, computed frame by frame to produce a low-rate contour. The cheapest features are time-domain: *energy/RMS* (squared

length, the loudness envelope) and *zero-crossing rate* (sign flips, a tonal-versus-noisy and voiced-versus- unvoiced meter). Treating the magnitude spectrum as a mass distribution yields the shape features: *centroid* (mean frequency, brightness), *spread* (standard deviation), *rolloff* (a quantile), and *flatness* (tonal-versus-noisy). *Spectral flux* measures change between frames and drives onset detection.

The *mel scale* warps frequency to match the ear's logarithmic pitch sensitivity; pooling the spectrum through a mel filterbank, taking the log (to separate source from filter), and applying a DCT (to decorrelate and compress) yields the *MFCCs*, the timbre-describing, pitch-invariant feature that dominated speech recognition. Their opposite, *chroma*, folds the spectrum onto twelve pitch classes to keep harmony while discarding timbre and octave. *Pitch* is found by periodicity – autocorrelation (an inner product of the signal with a shifted copy of itself), refined by *YIN* to beat octave errors and reach sub-sample precision. *Onsets* come from peak-picking a detection function built from energy difference, spectral flux, or phase deviation.

Every one of these features is a function of the waveform or its spectrum, and nearly all of them apply unchanged to motion signals, which is why this chapter is the feature toolbox for the wrist controller – with the standing caveat that IMU features yield coarse gestures, not finger pose.

27.14 Exercises

Exercise 27.1 (RMS and dB)**.** A frame of $N = 1024$ samples holds a sine of amplitude 0.5 over an integer number of cycles. Compute its energy, RMS, and energy in dB ($10 \log_{10} E$). Then a second frame holds the same sine plus white noise of RMS 0.05; estimate the new RMS (assume signal and noise are uncorrelated, so energies add) and the change in dB. Is the dB change something a listener would notice?

Exercise 27.2 (ZCR of a tone)**.** Show that a pure sinusoid of frequency f sampled at f_s has an expected zero-crossing rate of $2f/f_s$ crossings per sample. Use this to "read off" the frequency of a clean tone from its ZCR. Then explain, with a sketch, why this estimate breaks down when a second, higher-frequency tone or noise is added.

Exercise 27.3 (Centroid arithmetic)**.** A frame's spectrum has lines of magnitude 4 at 250 Hz, 2 at 500 Hz, and 1 at 2000 Hz. Compute the spectral centroid and the spectral spread. Now double the 2000 Hz line to magnitude 2 and recompute both. By how much, and in which direction, did each move? Interpret in terms of brightness.

Exercise 27.4 (Rolloff as a quantile)**.** Given the same three-line spectrum as Exercise 27.3 (use magnitudes $4, 2, 1$ as energies for simplicity), compute the 0.85 spectral rolloff frequency. Explain why rolloff can be more robust than centroid when a single loud high-frequency bin appears, and give an example spectrum where centroid moves a lot but rolloff barely moves.

Exercise 27.5 (Why the log in MFCCs)**.** Two frames have spectra $A[k]$ and $2A[k]$ (the second is the first scaled by a constant gain, as if recorded louder). Show that after the log step their mel-band vectors differ only by a constant offset, and therefore – because the DCT of a constant lands entirely in $c[0]$ – their MFCCs $c[1], c[2], \ldots$ are *identical*. What practical invariance does this give the feature?

Exercise 27.6 (DCT energy compaction)**.** Take a length-8 vector that is a smooth ramp $v_j = j$ for $j = 0, \ldots, 7$. Compute its DCT (by hand or with a few lines of code) and report how much of the total energy lands in the first two coefficients versus the rest. Repeat for a vector of random values. Explain what this demonstrates about why MFCCs keep only the low-order coefficients.

Exercise 27.7 (Autocorrelation period)**.** A signal is sampled at $f_s = 8000\,\text{Hz}$. Its autocorrelation has its first strong peak after lag zero at $\tau = 100$ samples. What is the estimated f_0? Now suppose a spurious peak also appears at $\tau = 50$; which note is that, and why might a naive "first peak" rule pick it? Which YIN step is designed to prevent this error?

Exercise 27.8 (The missing fundamental)**.** A periodic tone has $f_0 = 200\,\text{Hz}$ but the recording high-passes away everything below $400\,\text{Hz}$, leaving only the harmonics at $400, 600, 800, \ldots$ Hz. Explain why an FFT-peak pitch detector would report the wrong answer, while an autocorrelation detector still finds $f_0 = 200\,\text{Hz}$. What is the lag of the relevant autocorrelation peak at $f_s = 16000\,\text{Hz}$?

Exercise 27.9 (Onset detection function)**.** Explain why spectral flux uses half-wave rectification (keeping only increases) rather than the full difference. Construct a short example – a note that grows then fades – and sketch both the full-difference and the half-wave-rectified flux, showing how the rectified version peaks at the attack only.

Exercise 27.10 (Peak picking pitfalls)**.** You apply a fixed global threshold to an onset-strength curve and find it misses soft onsets in quiet passages while double-counting loud drum hits. Propose two fixes (one about the threshold, one about timing), and explain how each addresses a specific failure. Relate your answer to the smooth / adaptive-threshold / refractory recipe.

Exercise 27.11 (Transfer to motion)**.** You record $100\,\text{Hz}$ accelerometer magnitude during ten repetitions of a wrist flick separated by rest. Describe which features from this chapter you would compute and how you would combine an energy gate with a spectral-flux onset detector to segment the ten flicks. Then state plainly what this pipeline *cannot* tell you about the hand, and which sensor you would add to recover finger-level information.

Exercise 27.12 (Feature separability)**.** For each pair below, name the single feature from this chapter that best separates them and justify it in one sentence: (a) silence vs. speech; (b) a vowel vs. the consonant /s/; (c) a flute note vs. a cymbal crash; (d) a C-major chord vs. an A-minor chord; (e) a sustained note vs. the instant a new note begins.

Chapter 28

Audio Effects and Synthesis

So far we have spent our energy *analyzing* sound: pulling it apart into frequencies, smoothing it, measuring its features. This chapter flips the arrow. We are going to *make* and *shape* sound. An echo, a chorus, a fuzzy guitar tone, the breathy attack of a flute, the metallic bite of an FM bell — every one of these is a signal-processing recipe, and once you can read the recipe you can hear the math.

The wonderful thing is that you already own all the tools. An audio effect is almost always a filter, a delay, or a nonlinearity — and we have studied filters and delays at length. A synthesizer is almost always a small bank of oscillators run through a filter and an amplitude envelope — and an oscillator is just the rotating phasor of the early chapters sampled in time. So this chapter is less about new theory and more about *seeing* how a handful of familiar pieces, wired together, produce the entire palette of electronic and processed sound.

Our plan: first effects, organized by the operation underneath them — amplitude (gain, distortion), spectrum (EQ), and delay (echo, reverb, flanger, chorus). Then synthesis, organized by how the timbre is built — adding sinusoids up (additive), carving them away from a rich source (subtractive), wobbling one oscillator with another (FM), or reading a stored cycle (wavetable). Throughout we keep the geometry in view: a timbre is a shape in the frequency domain, an envelope is a shape in the time domain, and an effect is a transformation of one or both. Short code sketches make every idea concrete, and a closing goalbox connects the chapter to driving a software instrument from the wrist controller.

28.1 The three knobs: amplitude, spectrum, time

Strip away the marketing names on a guitar pedal or a plugin and you find only three things being manipulated.

- **Amplitude** — how loud each sample is. Scaling it by a constant is gain. Scaling it by a slowly varying curve is tremolo or an envelope. Passing it through a *nonlinear* curve is distortion.
- **Spectrum** — how energy is distributed across frequency. Reshaping it is equalization (EQ) and, more generally, filtering.
- **Time** — when each sample is heard. Copying the signal to a later time and adding it back is

delay, and delay is the seed of echo, reverb, flanging, and chorus.

Intuition

Almost every effect is a combination of these three, plus one twist: *modulation*, where a slow control signal (a low-frequency oscillator, an envelope, or a sensor reading from your wrist) drives one of the three knobs over time. A static filter is an EQ; a filter whose cutoff sweeps is a wah pedal. A fixed delay is an echo; a delay whose length wobbles is a flanger. The *movement* is what makes an effect feel alive, and that movement is exactly where a gesture controller will plug in.

Two of these knobs we can already turn cold. Gain is multiplication by a scalar g: $y[n] = g\,x[n]$. EQ is the LTI filtering of the earlier chapters: choose a frequency response $H(e^{\mathrm{j}\omega})$ and convolve. The two genuinely new ideas in this chapter are *nonlinearity* (distortion, and the heart of FM synthesis) and *structured delay* (the comb and all-pass building blocks of spatial effects). We take them in turn.

28.2 Amplitude effects: gain, tremolo, and distortion

28.2.1 Gain and tremolo

The simplest effect imaginable is a volume knob:

$$y[n] \;=\; g\,x[n]. \tag{28.1}$$

This is linear and time-invariant, so it changes nothing about the spectrum's *shape* — every frequency is scaled by the same g. Make g vary slowly in time and you get **tremolo**:

$$y[n] \;=\; \big(1 + d\,\cos(2\pi f_m n / f_s)\big)\,x[n], \tag{28.2}$$

with modulation depth $d \in [0, 1]$ and a modulation rate f_m of a few hertz. Now g is time-*varying*, so this is no longer LTI; in the spectrum it splits each tone into sidebands at $\pm f_m$ (amplitude modulation, the same mechanism we met in the modulation chapter). At a few hertz you hear it as a pulsing in loudness rather than as new pitches (Figure 28.1).

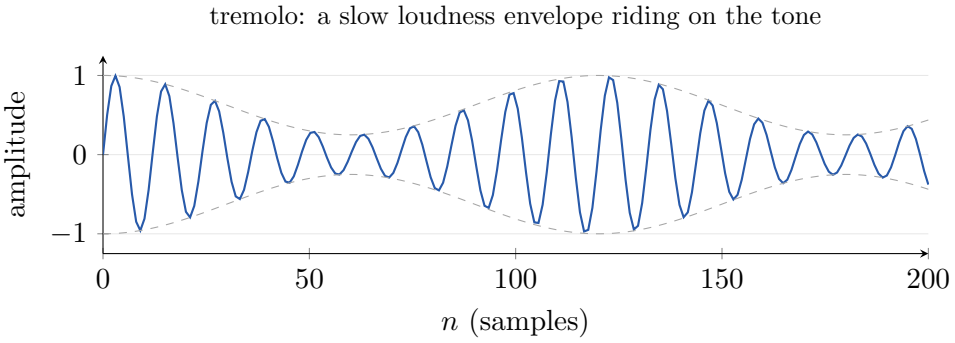


Figure 28.1: Tremolo: the audio-rate tone (solid) is scaled by a slow loudness curve (dashed envelope), Eq. (28.2). The envelope rises and falls at f_m a few times per second, so you hear a pulsing volume rather than a change in pitch.

28.2.2 Distortion: bending the line

Gain multiplies by a number. **Distortion** sends each sample through a *curved* function $f(\cdot)$, applied sample by sample:

$$y[n] \;=\; f\big(x[n]\big). \tag{28.3}$$

This is a *memoryless nonlinearity* (also called waveshaping): the output at time n depends only on the input at time n, but not linearly. That single word "nonlinear" is doing enormous work. A nonlinear function manufactures *new frequencies* that were not in the input.

Why it works

Why does bending the amplitude curve create new frequencies? Feed a pure tone $x[n] = \cos(\omega n)$ into a polynomial nonlinearity, say $f(x) = x^2$. Then

$$y[n] = \cos^2(\omega n) = \tfrac{1}{2} + \tfrac{1}{2}\cos(2\omega n).$$

A DC term and a tone at *twice* the frequency appeared — harmonics that the input never contained. Cube it and you get a tone at 3ω as well. Any nonlinear f can be expanded as a power series $f(x) = a_0 + a_1 x + a_2 x^2 + \cdots$, and the x^k term generates energy up to the k-th harmonic. Bending the line sprays the spectrum upward into harmonics. *That harmonic spray is the sound of distortion*: a clean sine becomes a buzzy, rich tone.

The most important shape in practice is a **saturating** curve — one that is nearly straight for small inputs but flattens out for large ones, like an amplifier running out of headroom. Two common choices:

$$\text{soft clip (tanh):} \quad f(x) = \tanh(k\,x), \tag{28.4}$$

$$\text{hard clip:} \quad f(x) = \max\big(-1, \min(1, k\,x)\big). \tag{28.5}$$

The gain k is the "drive." Small k stays in the straight middle of the curve and barely changes the sound; large k pins the waveform against the flat top and bottom, squaring it off.

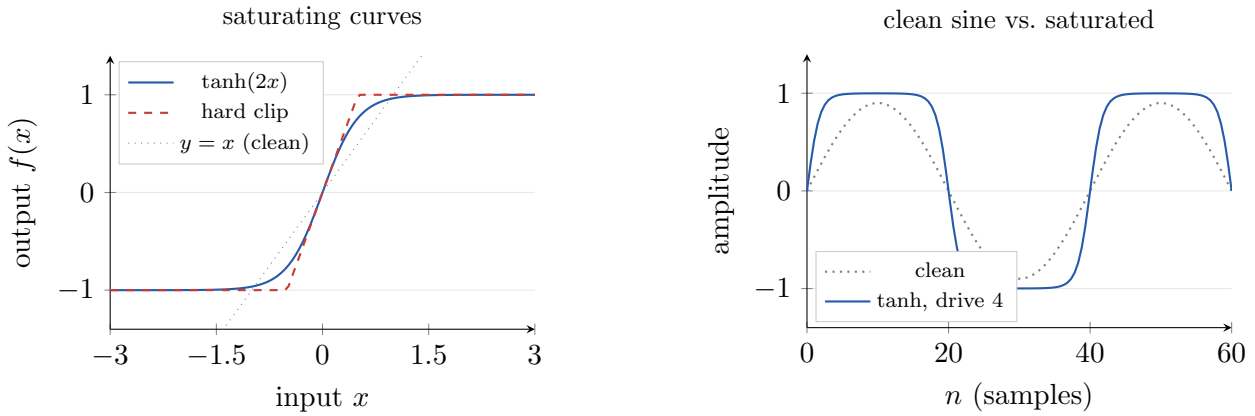


Figure 28.2: Left: saturating transfer curves. Near the origin they follow $y = x$ (clean); far out they flatten, clipping the peaks. Right: a sine pushed through tanh with high drive — its rounded top and bottom flatten toward a square wave, which is rich in odd harmonics.

What is really happening

What is really happening when you "turn up the gain" on a distortion pedal is not that it gets louder — the saturating curve caps the output amplitude. What changes is the *spectrum*. As drive increases, the waveform morphs from a sine toward a square, and the spectrum fills in with harmonics: $3f, 5f, 7f, \ldots$ for a symmetric (odd) curve like tanh, and even harmonics too if the curve is asymmetric (which is why tube amps, slightly asymmetric, sound "warmer" than symmetric transistor clipping). Distortion is a spectral effect dressed up as an amplitude effect.

Common trap

Distortion aliases. Because waveshaping creates harmonics far above the input frequency, those harmonics can easily exceed the Nyquist limit $f_s/2$ and fold back as inharmonic, dissonant aliases. A clean-sounding software distortion almost always *oversamples*: upsample by 4×–8×, apply the nonlinearity where there is room for the new harmonics, low-pass filter, then decimate back. Skip this and a high note run through heavy distortion turns to gravel.

Listing 28.1: Soft-clip distortion with a drive knob.

```
import numpy as np

def distort(x, drive=4.0):
    # tanh soft-clip; normalize so quiet passages are unchanged
    return np.tanh(drive * x) / np.tanh(drive)

# (Production code would oversample around this line to tame aliasing.)
```

28.3 Spectral effects: EQ as everyday filtering

An **equalizer** is just a filter (or a bank of filters) you adjust by ear. There is no new mathematics here — it is the frequency-response shaping of the filtering chapters — but the vocabulary is worth pinning down because it is how musicians and engineers talk about spectrum.

- **Shelving filters** lift or cut everything above (high shelf) or below (low shelf) a corner frequency by a fixed amount. Think of the "bass" and "treble" knobs on a stereo.
- **Peaking / bell filters** boost or cut a band centered on a chosen frequency, with a width set by the quality factor Q. A graphic EQ is a row of fixed bells; a parametric EQ lets you move each bell's center, gain, and Q.
- **High-pass / low-pass** remove rumble below or hiss above a cutoff.

These are almost universally built as **biquads** — second-order IIR sections, Equation (28.6) — because two poles and two zeros are exactly enough to place one resonant bump or notch, and you cascade several to build a complicated curve.

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]. \tag{28.6}$$

The well-known "audio EQ cookbook" formulas give the five coefficients $\{b_0, b_1, b_2, a_1, a_2\}$ for each EQ type directly from (f_c, Q, gain); in practice you compute them once whenever a knob moves and run the recurrence per sample.

Connection

A biquad's two poles are a complex-conjugate pair sitting at radius r and angle $\theta = 2\pi f_c/f_s$ inside the unit circle, as the z-transform chapter showed. The closer r creeps to 1, the sharper and taller the resonant peak in the frequency response — the same pole-near-the-circle geometry that governs every resonator, from a plucked string to a vowel formant in the human voice. EQ, synthesis filters, and acoustic resonance are one piece of geometry wearing three hats.

28.4 Delay: the seed of echo, reverb, flanging, and chorus

Now the genuinely spatial effects. They all grow from one trivial operation: a **delay line**, a buffer that hands back what you put in D samples ago, i.e. $x[n - D]$. The delay length in seconds is $\tau = D/f_s$. What you do with the delayed copy — and whether you feed it back — determines the whole family.

28.4.1 Single echo and the feedforward comb filter

Add one delayed, attenuated copy to the original:

$$y[n] = x[n] + g\,x[n - D], \qquad 0 < g < 1. \tag{28.7}$$

With τ in the tens or hundreds of milliseconds you hear a discrete **echo** (a slap-back). With τ of only a few milliseconds your ear cannot separate the two copies in time — instead you hear a change in *timbre*. Why? Look at the frequency response. The impulse response is $h[n] = \delta[n] + g\,\delta[n - D]$, so

$$H(e^{\mathrm{j}\omega}) = 1 + g\,e^{-\mathrm{j}\omega D}, \qquad |H(e^{\mathrm{j}\omega})|^2 = 1 + g^2 + 2g\cos(\omega D). \tag{28.8}$$

The magnitude rises and falls cosinusoidally with frequency: peaks where the delayed copy arrives in phase (ωD a multiple of 2π) and dips where it arrives out of phase. Plotted against frequency it looks like the teeth of a comb — hence **comb filter**. The teeth sit at multiples of $f_s/D = 1/\tau$ hertz.

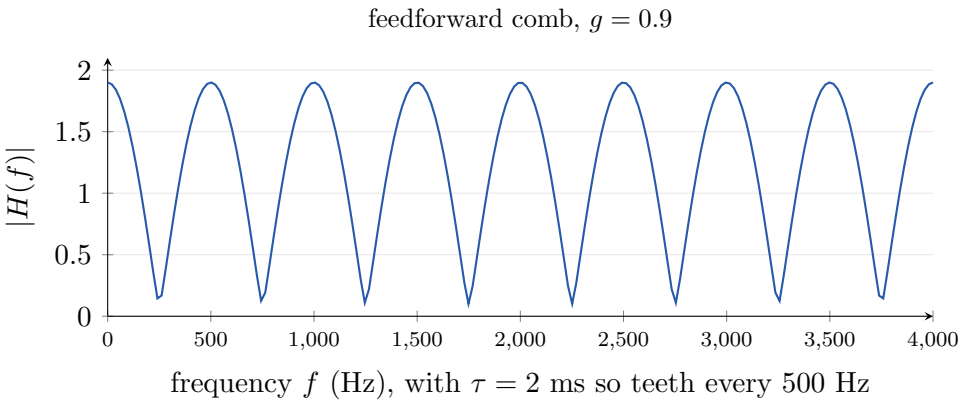


Figure 28.3: The magnitude response of a single-echo (feedforward comb) filter, Eq. (28.8). Evenly spaced peaks and notches — the comb teeth — spaced $1/\tau$ apart. A short delay is therefore a timbral effect, not a rhythmic one: it colors the spectrum rather than producing an audible repeat.

28.4.2 Repeating echoes and the feedback comb

A real echo in a canyon repeats and fades. To get that, feed the output back into the delay:

$$y[n] = x[n] + g\,y[n - D]. \tag{28.9}$$

Now each output sample re-enters the delay line, so a single input impulse comes back at $D, 2D, 3D, \ldots$ with amplitudes $g, g^2, g^3, \ldots$ — a geometric train of decaying repeats. This is an IIR filter; its impulse response is infinite (it never exactly reaches zero), which is exactly what we want for a decaying tail. Stability demands $|g| < 1$, or the echoes would grow without bound and the system would scream.

Walking question

How long does the feedback echo take to fade? Each pass multiplies by g, so after m repeats the level is g^m. The standard "RT60" measure asks when the sound has dropped 60 dB, i.e. to 10^{-3} of its start. Solve $g^m = 10^{-3}$: $m = -3/\log_{10} g$ repeats, taking $T_{60} = m\,\tau$ seconds. For $g = 0.7$ and $\tau = 0.1$ s: $m \approx 19$ repeats, $T_{60} \approx 1.9$ s. The decay time is set jointly by the feedback gain and the delay length — the two knobs on a delay pedal.

Listing 28.2: Feedback echo with a delay buffer.

```python
import numpy as np

def echo(x, fs, delay_s=0.25, g=0.6):
    D = int(delay_s * fs)
    y = np.zeros(len(x) + D)
    y[:len(x)] = x
    for n in range(D, len(y)):           # y[n] = x[n] + g*y[n-D]
        y[n] += g * y[n - D]
    return y
```

28.4.3 The all-pass filter: delay without coloration

The comb filters above change the magnitude spectrum — they put audible teeth in it. But for natural-sounding reverb we want to smear the sound in time *without* stamping a comb-shaped color on it. The tool for that is the **all-pass filter**: it passes every frequency at full, equal magnitude (hence the name) while scrambling the *phase*, and thus the timing.

$$y[n] = -g\,x[n] + x[n-D] + g\,y[n-D], \qquad |H(e^{\mathrm{j}\omega})| = 1 \text{ for all } \omega. \tag{28.10}$$

Why it works

How can a filter delay things yet leave *every* magnitude untouched? Its transfer function has the special form $H(z) = \dfrac{-g + z^{-D}}{1 - g\,z^{-D}}$: the numerator coefficients are the denominator coefficients reversed. Each pole at radius r is mirrored by a zero at radius $1/r$ along the same ray. On the unit circle the contributions of every mirror pair cancel exactly in magnitude — so $|H| = 1$ everywhere — but their *phases* add, producing a frequency-dependent delay. The all-pass is pure time-smear with zero spectral coloring, which is precisely the ingredient a reverberator needs to build density without ringing.

28.4.4 Reverb: stacking combs and all-passes

Reverberation is what a room does: thousands of reflections off walls, floor, and ceiling arrive in a dense, decaying wash. We cannot afford to compute thousands of literal echoes, so the classic **Schroeder reverberator** (1962) fakes the statistics of a room cheaply:

- Several *feedback comb* filters in parallel, each with a different (mutually prime) delay, build up a dense set of decaying echoes. Coprime delays keep their teeth from lining up into an audible pitch.
- A few *all-pass* filters in series then smear that into a smooth tail without adding coloration, multiplying the echo density at each stage.

A pre-delay (a plain delay before the network) sets the perceived room size, and a low-pass filter in each feedback loop makes high frequencies decay faster than lows — which is what real air and soft furnishings do, and what makes a reverb sound like a room rather than a tin can.

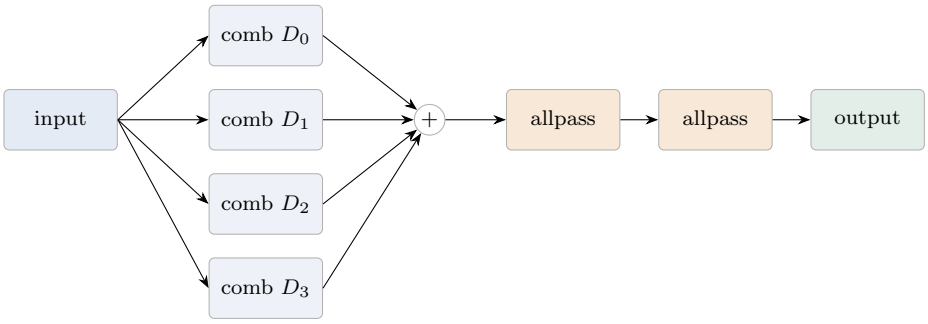


Figure 28.4: A Schroeder reverberator: parallel feedback combs (coprime delays) build echo density, then series all-passes smear the result into a smooth, uncolored tail. Add a low-pass in each comb's feedback path for realistic high-frequency decay.

28.4.5 Flanger and chorus: modulating the delay length

Take the short-delay feedforward comb of Eq. (28.7) and *slowly sweep the delay* D with a low-frequency oscillator (LFO). As D changes, the comb teeth slide up and down the spectrum. That sweeping comb is a **flanger** — the jet-plane "whoosh" — with delays around 1–10 ms and usually some feedback to deepen the notches.

Chorus is the same idea with a longer base delay (15–35 ms), a deeper, slower, often randomized LFO, and little or no feedback. The longer delay puts the comb teeth so close together that you do not hear them as coloration; instead, the constantly shifting delay slightly detunes the copy (Doppler-style pitch wobble), and mixing it with the original mimics several players slightly out of tune and out of time — the lush "ensemble" sound.

Effect	Base delay	Feedback?	What you hear
Slap echo	50–150 ms	light	one discrete repeat
Delay/echo	100–800 ms	yes	rhythmic repeats, fading
Reverb	dense spread	combs + all-pass	a room/space
Flanger	1–10 ms, swept	yes	sweeping "whoosh"
Chorus	15–35 ms, swept	little	thick "ensemble"

Table 28.1: The delay-based effects are one circuit — a delay line plus optional feedback plus optional modulation — tuned to different delay ranges.

Common trap

Sweeping a delay means reading the buffer at a *fractional* sample position $n - D(n)$, where $D(n)$ is not an integer. Rounding to the nearest sample produces audible "zipper" clicks. Use linear (or higher-order) interpolation between the two neighboring samples in the delay buffer. This fractional-delay read is the secret sauce that makes flangers and choruses sound smooth.

28.5 Synthesis I: additive — building timbre by stacking sinusoids

Now we turn from shaping existing sound to creating it. The most direct route follows straight from Fourier: *any* periodic tone is a sum of harmonics, so to build a tone, add up sinusoids.

$$y[n] = \sum_{k=1}^{K} A_k(n)\, \sin\!\big(2\pi k f_0 n / f_s + \phi_k\big). \tag{28.11}$$

The fundamental f_0 sets the pitch; the relative amplitudes A_k set the **timbre** — the "color" that distinguishes a flute from a violin at the same pitch. Make the $A_k(n)$ change over time and you capture how a real instrument's spectrum evolves (the bright attack of a plucked string mellowing as upper harmonics die first).

Intuition

Additive synthesis is the inverse Fourier series used as an instrument. You are literally drawing the spectrum bar by bar and letting the synth play it back. It can in principle reproduce *any*

harmonic sound — it is the most general periodic-tone synthesizer there is. Its drawback is cost and control: a rich tone may need dozens of partials, each with its own time-varying amplitude curve, so you are steering dozens of knobs at once. The other synthesis methods are largely clever shortcuts that get a rich, evolving spectrum from far fewer controls.

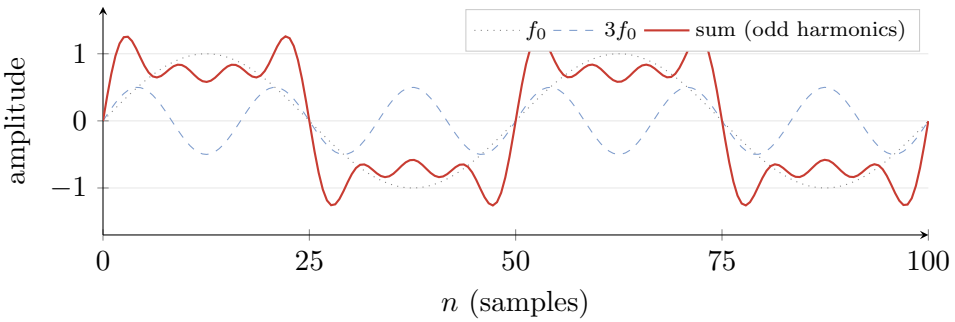


Figure 28.5: Additive synthesis of a square-ish tone: the fundamental plus odd harmonics with amplitudes $1, \frac{1}{3}, \frac{1}{5}, \frac{1}{7}, \ldots$ As more partials are added the sum approaches a square wave. Choosing the harmonic amplitudes is choosing the timbre.

28.6 Synthesis II: subtractive — carving timbre out of a rich source

The dominant approach in classic analog synthesizers reverses the additive strategy. Instead of building a spectrum up from nothing, start with a harmonically *rich* waveform and *carve away* what you do not want with a filter. The signal chain is the iconic synth voice:

$$\boxed{\text{oscillator}} \rightarrow \boxed{\text{filter}} \rightarrow \boxed{\text{amplifier}}$$

- **Oscillator (VCO).** A waveform rich in harmonics: a sawtooth (all harmonics, $A_k \propto 1/k$), a square/pulse (odd harmonics; pulse width changes the recipe), or a triangle (odd harmonics, fast roll-off). The sawtooth, with its full harmonic comb, is the workhorse.
- **Filter (VCF).** A resonant low-pass whose *cutoff* decides how much of the harmonic richness survives, and whose *resonance* (a pole pushed near the unit circle) adds a vocal-like peak at the cutoff. Sweep the cutoff and you get the classic "wah"/filter-sweep sound.
- **Amplifier (VCA).** Applies the loudness envelope (next section).

What is really happening

The reason subtractive synthesis feels so expressive is that a single knob — the filter cutoff — moves a huge amount of spectral energy at once. Opening the filter as a note begins, then closing it as the note sustains, mimics exactly how real plucked and struck instruments behave: bright at the attack, mellow as it rings. One time-varying cutoff stands in for dozens of individual harmonic envelopes that additive synthesis would force you to draw by hand. Subtractive synthesis is additive synthesis with the harmonic envelopes *ganged together* under one control.

Common trap

A naive sawtooth or square generated by $((nf_0/f_s) \bmod 1)$ has perfect, sharp corners — and therefore harmonics extending infinitely high, far past Nyquist. Sampled directly, those fold back as aliasing and the oscillator sounds gritty and detuned on high notes. Real synths use *band-limited* oscillators (BLEP/BLIT methods, or band-limited wavetables) that round the corners just enough to keep every harmonic below $f_s/2$.

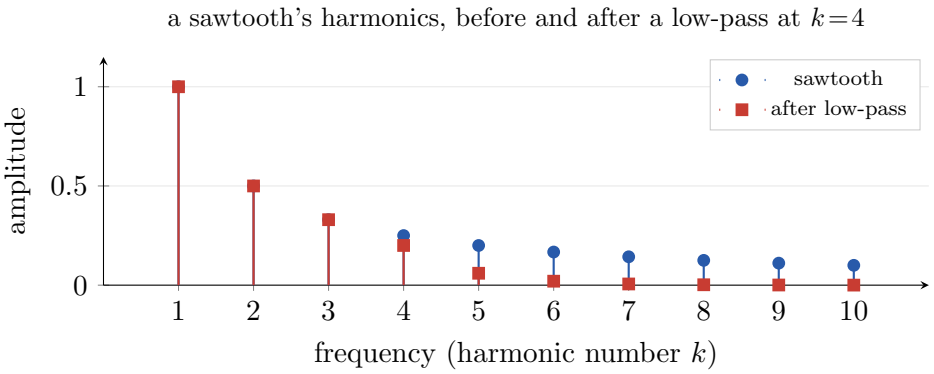


Figure 28.6: Subtractive synthesis in the spectrum. Blue: the sawtooth's full harmonic series, $A_k = 1/k$. Red: after a resonant low-pass with cutoff near the 4th harmonic — harmonics above the cutoff are carved away (with a small resonant bump right at the cutoff). Sweeping the cutoff slides the red "cliff" up and down.

28.7 Synthesis III: FM — rich timbre from a wobbling phase

Here is where nonlinearity returns in spectacular fashion. **Frequency modulation** synthesis, discovered by John Chowning at Stanford in the late 1960s, gets startlingly complex, evolving spectra from just *two* oscillators by using one to wobble the other's frequency:

$$y(t) = \sin\Big(2\pi f_c t + I \sin(2\pi f_m t)\Big). \tag{28.12}$$

The *carrier* at f_c has its instantaneous phase shoved back and forth by the *modulator* at f_m. The single most important number is the **modulation index** I: it controls how violently the phase wobbles, and therefore how rich the spectrum becomes.

History

Chowning stumbled onto FM in 1967 while experimenting with extreme vibrato: as he pushed the vibrato rate up into the audio range, the wobble stopped sounding like vibrato and started sounding like a completely new *timbre*. He recognized that very fast frequency modulation generates audible sidebands. Stanford licensed the technique to Yamaha, whose 1983 DX7 — built entirely on FM — went on to define the sound of 1980s pop and became one of the best-selling synthesizers ever made. A research curiosity about vibrato became a worldwide sonic signature.

Why it works

Why does wobbling the phase create a whole comb of sidebands? Expanding Eq. (28.12) with the Jacobi–Anger identity gives

$$y(t) = \sum_{k=-\infty}^{\infty} J_k(I)\, \sin\big(2\pi(f_c + k f_m)t\big),$$

where J_k are Bessel functions. The spectrum is a *ladder* of components at $f_c, f_c \pm f_m, f_c \pm 2f_m, \ldots$, with strengths $J_k(I)$ (Figure 28.8). When I is small only $k = 0, \pm 1$ matter and you hear a nearly pure tone. As I grows, the energy spreads out to higher $|k|$ — the timbre brightens and fills in. *One knob, I, sweeps from a sine to a brassy blare.* That is why a single FM "operator pair" driven by an envelope on I can imitate brass, bells, and electric pianos so vividly.

Why this matters

The ratio f_m/f_c decides whether FM sounds musical or clangorous. When the ratio is a simple integer (1:1, 2:1, 3:2), the sidebands $f_c + k f_m$ land on exact harmonics of a common fundamental, and you hear a clear pitch with a rich but harmonic timbre. When the ratio is irrational or complex, the sidebands fall between harmonics, giving the inharmonic, metallic, bell-like tones FM is famous for. Two oscillators and two numbers — I and the ratio — span an enormous expressive range. This efficiency is exactly why FM ruled 1980s digital synthesizers, where memory and compute were precious.

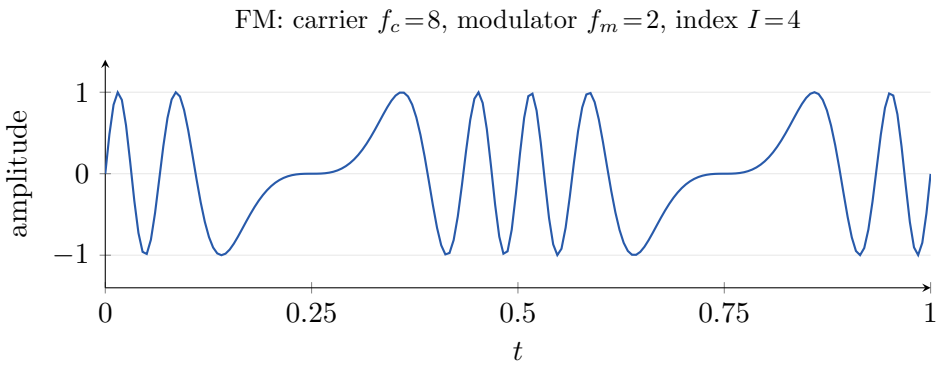


Figure 28.7: An FM waveform. Notice how the carrier's local frequency speeds up and slows down — the cycles bunch and stretch — as the modulator pushes its phase. In the spectrum this becomes a ladder of sidebands whose richness is set by the modulation index I.

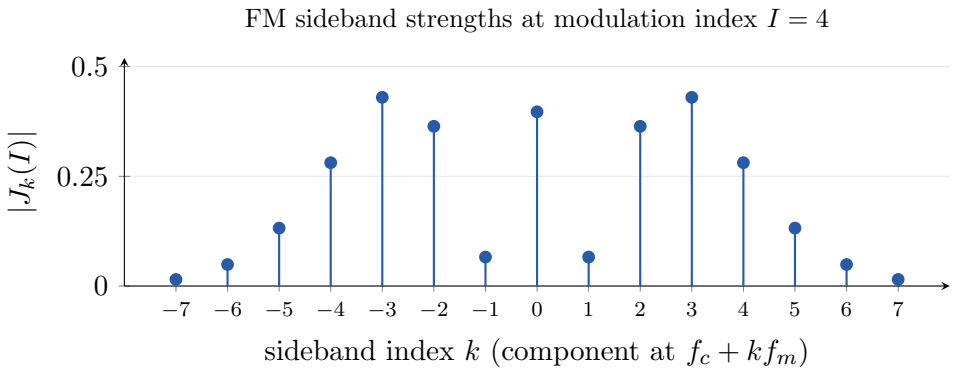


Figure 28.8: The FM spectrum for index $I = 4$: component k sits at $f_c + kf_m$ with strength $|J_k(I)|$, the Bessel function from the whybox. Notice the carrier ($k = 0$) is no longer dominant — energy has spread out into the sidebands. As I grows the ladder reaches farther out and the tone brightens; as $I \to 0$ only $k = 0, \pm 1$ survive and the tone returns to a near-pure sine.

Listing 28.3: Two-operator FM with an envelope on the index.

```
import numpy as np

def fm_tone(fc, fm, dur, fs=44100, I0=5.0, tau=0.4):
    t = np.arange(int(dur*fs)) / fs
    I = I0 * np.exp(-t / tau)            # index decays -> bright attack, mellow tail
    return np.sin(2*np.pi*fc*t + I*np.sin(2*np.pi*fm*t))

bell = fm_tone(fc=440, fm=440*1.41, dur=2.0)   # near-irrational ratio -> metallic
```

28.8 Synthesis IV: wavetable — play back one stored cycle

The cheapest way to get an arbitrary timbre is to *store* one period of the desired waveform in a table and read it out in a loop. A **phase accumulator** steps an index through the table; how fast it steps sets the pitch.

$$\phi[n] = \big(\phi[n-1] + \Delta\big) \bmod N, \qquad \Delta = \frac{f_0\, N}{f_s}, \qquad y[n] = \text{table}\big[\phi[n]\big]. \tag{28.13}$$

Here N is the table length and Δ the phase increment. A larger increment skips through the table faster, raising the pitch. Because Δ is generally fractional, you again interpolate between table entries.

Intuition

A wavetable oscillator is just additive synthesis *precomputed* and frozen: sum your harmonics once, store the resulting cycle, then replay it cheaply forever. You trade the live control of additive synthesis for almost no runtime cost. "Wavetable synthesis" in modern soft-synths (Serum, Massive) extends this by storing *many* tables and smoothly crossfading between them as a note evolves — morphing the timbre in time, which is exactly the evolving spectrum that additive synthesis offered, recovered through a stack of snapshots.

Common trap

A single stored cycle has a fixed harmonic content, so transposing it far upward will push its upper harmonics past Nyquist and alias. Production wavetable synths keep a set of band-limited versions of each table (*mip-maps*), one per octave, and pick the version whose highest harmonic still fits below $f_s/2$ for the note being played.

28.9 The envelope: shaping a sound in time (ADSR)

An oscillator alone drones forever at constant loudness — nothing like a real note, which is born, swells, settles, and dies. The **amplitude envelope** is the time-domain shape multiplied onto the sound, and the universal model for it is **ADSR**:

- **Attack** — time to rise from silence to peak after the note starts (a key press, or a wrist flick).
- **Decay** — time to fall from the peak to the sustain level.
- **Sustain** — the held *level* (not a time) maintained while the note is held.
- **Release** — time to fall back to silence after the note ends.

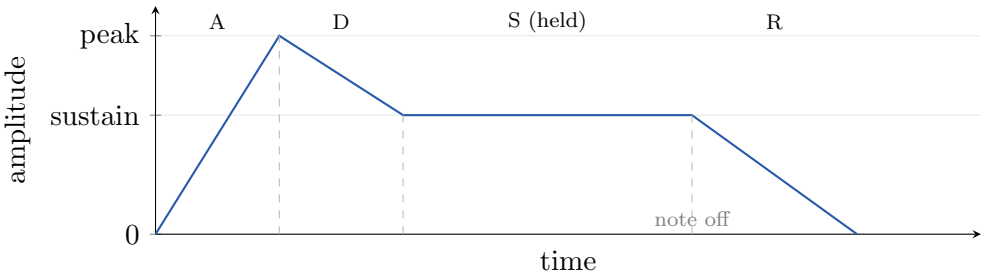


Figure 28.9: The ADSR envelope. Attack and decay fire when the note starts; the sustain level holds while the key is down; release begins when the note is let go. Multiplying the oscillator by this curve turns a drone into a note.

What is really happening

The envelope is most of what your ear uses to name an instrument. Classic experiments show that if you splice off the attack transient of a recorded piano, trumpet, and violin and play only their steady sustains, listeners struggle to tell them apart — the *attack* carries the identity. A fast, percussive attack with no sustain says "plucked" or "struck"; a slow attack with a long sustain says "bowed" or "blown." This is why the same oscillator can become a plucked bass or a swelling pad purely by changing the envelope. Timbre lives in the spectrum *and* its evolution; the envelope governs the evolution.

Envelopes do not have to drive loudness only. Route an ADSR (or a second one) to the filter cutoff and you get the bright-attack-then-mellow behavior of subtractive synthesis; route it to the FM modulation index and you get the brassy swell of an FM trumpet. The envelope is a general-purpose, one-shot control signal — and that makes it the natural place for a gesture to take over.

Listing 28.4: A linear ADSR envelope generator.

```python
import numpy as np

def adsr(fs, a, d, s_level, s_time, r):
    seg = lambda t, lo, hi: np.linspace(lo, hi, int(t*fs), endpoint=False)
    return np.concatenate([
        seg(a, 0.0, 1.0),               # attack
        seg(d, 1.0, s_level),           # decay
        seg(s_time, s_level, s_level),  # sustain (held)
        seg(r, s_level, 0.0),           # release
    ])

# voice = oscillator(...) * adsr(44100, 0.01, 0.1, 0.6, 0.5, 0.3)
```

28.10 Putting it together: a complete synth voice

We can now read the block diagram of essentially any monophonic synthesizer voice as a sentence in the language of this book.

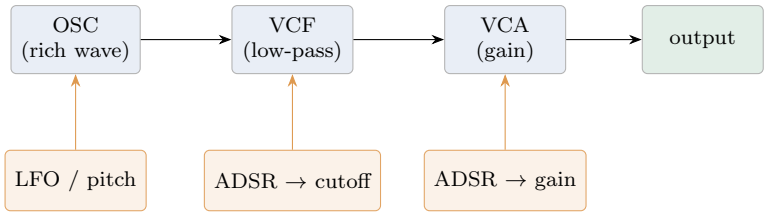


Figure 28.10: A canonical subtractive voice. Blue: the audio path (oscillator → filter → amplifier). Orange: the control signals — envelopes and an LFO — that modulate the audio path over time. Every electronic instrument is a variation on this skeleton; the gesture controller's job is to supply or replace the orange control signals.

Application

Why this is the whole ballgame. A drum machine is this voice with a noise oscillator and a very short envelope. A pad is this voice with a slow attack and long release. An organ is several oscillators (additive) with a near-square envelope. A DX-style electric piano is the FM voice of Section 28.7 with a decaying index envelope. Stack a delay, a chorus, and a reverb after the VCA and you have a finished, recordable sound. There is nothing in an expensive synthesizer that is not a tuned, polished arrangement of the boxes in this chapter.

28.11 Toward the wrist tracker: gesture as a control signal

Toward the wrist tracker

Everything in this chapter splits cleanly into two streams: the *audio path* (oscillators, filters, delays — the blue boxes) and the *control path* (envelopes, LFOs, knobs — the orange boxes). The audio path runs at the sample rate, 44.1 kHz, far faster than any human motion. The control path runs slowly, tens to a few hundred updates per second — and that is exactly the rate at which a wrist sensor produces useful information.

So the design is clean: **let the wrist drive the control path**, and let a software synth (or a hardware one over MIDI) own the audio path. Concretely:

- A coarse, discrete *gesture* (a flick, a fist, a wrist-tilt held) is a reliable trigger — map it to MIDI *note-on/note-off*, the moment the ADSR fires.
- A continuous, smoothly varying *orientation* (wrist pitch or roll from the fused IMU of the orientation chapters) is a reliable knob — map it to a MIDI *continuous controller*: filter cutoff, FM index, vibrato depth, or volume. Slow-moving, low-dimensional, exactly what the control path wants.

This is why the synthesis architecture matters to the build: it tells us we do *not* need to recover anything fast or finely detailed to make expressive music. A handful of clean, slow control streams modulating a well-designed voice is already a playable instrument.

Common trap

Know what the wrist can supply. A wrist-worn IMU measures the *orientation and motion of the whole forearm/wrist*, not the angle of each finger. It cannot recover continuous individual finger-joint angles — the fingers are mechanically downstream of the wrist and largely invisible to it. What it gives reliably is (a) coarse discrete gestures (flick, twist, tap, clench-versus-relax via the accompanying jolt) and (b) smooth wrist/forearm orientation. Individual-finger control needs sensors that see the muscles or the fingers themselves — surface EMG or force myography on the forearm for finger *activation*, or instrumented rings for finger *pose* (the hand-sensing chapter goes into this). Designing the instrument around triggers and slow knobs is not a compromise forced by laziness; it is matching the controller to what the physics of the sensor can actually deliver.

Build it (hardware)

A minimal end-to-end test, before any custom hardware: run a small synth on a laptop or Raspberry Pi (e.g. a SuperCollider, Pure Data, or a 50-line NumPy voice), and feed it MIDI from a microcontroller on the wrist.

Firmware side (Arduino-style pseudocode):

Listing 28.5: Wrist control to MIDI: trigger + one continuous knob.

```
// Runs at ~100 Hz. imu gives fused pitch (radians) and accel-based jerk.
float pitch = imu_pitch();          // smooth wrist tilt: a "knob"
float jerk  = imu_accel_jerk();     // sudden motion: a "trigger"

// Continuous controller: map wrist tilt [-pi/4, pi/4] -> CC 74 (filter cutoff)
```

```
int cc = clamp((int)((pitch + 0.785f) / 1.571f * 127.0f), 0, 127);
midiControlChange(0, 74, cc);        // channel 0, CC #74

// Discrete trigger: a sharp flick fires a note (with simple debounce)
if (jerk > THRESH && (now - last_note) > 120 /*ms*/) {
    midiNoteOn(0, 60, 100);          // middle C, velocity 100
    last_note = now;  note_active = true;
} else if (note_active && jerk < REL_THRESH) {
    midiNoteOff(0, 60, 0);  note_active = false;
}
```

Notes from experience: smooth the IMU-derived knob with a one-pole low-pass or the MIDI value will jitter and the filter will buzz; debounce the trigger (a single flick produces a burst of high-acceleration samples); and send CC updates only when the value actually changes, to avoid flooding the MIDI bus.

28.12 Summary

Audio effects and synthesis are not a new theory — they are the filters, delays, and oscillators of the earlier chapters, arranged with intent.

- Every effect turns one of three knobs: **amplitude** (gain, tremolo, distortion), **spectrum** (EQ = filtering), or **time** (delay). Movement comes from *modulating* a knob.
- **Distortion** is a memoryless nonlinearity; bending the amplitude curve sprays the spectrum into harmonics. Watch out for aliasing.
- **Delay** seeds the spatial effects. A short delay is a *comb filter* (timbral); feedback makes it a decaying echo; *all-pass* filters smear time without coloring the spectrum; stacking combs and all-passes makes *reverb*; modulating a short delay makes *flanger* and *chorus*.
- **Synthesis** builds timbre four ways: *additive* (sum sinusoids), *subtractive* (filter a rich source), *FM* (wobble a phase for rich sidebands from two oscillators), and *wavetable* (replay a stored cycle).
- The **ADSR envelope** shapes a sound in time and carries much of an instrument's identity, especially in its attack.
- For the wrist controller, the split between a fast *audio path* and a slow *control path* is the design key: drive the control path with discrete gesture triggers (note-on/off) and smooth orientation knobs (continuous controllers), matched to what a wrist IMU can actually sense.

28.13 Exercises

Exercise 28.1. A pure tone $x[n] = \cos(\omega_0 n)$ is fed through the cubic waveshaper $f(x) = x - \frac{1}{3}x^3$. Using $\cos^3\theta = \frac{3}{4}\cos\theta + \frac{1}{4}\cos 3\theta$, find the frequencies present in the output and their amplitudes. Which new frequency appears, and is it a harmonic of ω_0?

Exercise 28.2. A feedforward comb filter has $h[n] = \delta[n] + g\,\delta[n-D]$ with $f_s = 48$ kHz and $D = 24$ samples. (a) What is the delay τ in milliseconds? (b) At what frequencies (in Hz) are the comb's first three peaks? (c) If $g = 0.95$, what are the maximum and minimum values of $|H(e^{\mathrm{j}\omega})|$?

Exercise 28.3. For the feedback echo $y[n] = x[n] + g\,y[n-D]$ with $\tau = 300$ ms, choose g so the echoes decay by 60 dB in about 3 seconds. (Use $g^m = 10^{-3}$ with $m = T_{60}/\tau$.) Then state the danger if g is set to exactly 1.0.

Exercise 28.4. Verify the all-pass magnitude claim for $D = 1$: $H(z) = \dfrac{-g + z^{-1}}{1 - gz^{-1}}$. Substitute $z = e^{\mathrm{j}\omega}$ and show algebraically that $|H(e^{\mathrm{j}\omega})| = 1$ for all ω (numerator and denominator have equal magnitude). What does this tell you about why all-pass filters are used to build reverb tails?

Exercise 28.5. You want to additively synthesize a sawtooth at $f_0 = 200$ Hz, sampled at $f_s = 44.1$ kHz, using harmonics $A_k = 1/k$. What is the largest harmonic index k you may include before exceeding Nyquist? What goes wrong audibly if you include harmonics beyond that?

Exercise 28.6. An FM voice has $f_c = 300$ Hz and modulator $f_m = 300$ Hz (ratio 1:1). (a) List the frequencies of the spectral components for $k = -2, \ldots, 2$. (b) Are they all harmonics of a common fundamental? (c) Now set $f_m = 300\sqrt{2}$ Hz; describe qualitatively how the timbre changes and why.

Exercise 28.7. A wavetable of length $N = 2048$ is played at $f_s = 44.1$ kHz. (a) What phase increment Δ produces $f_0 = 440$ Hz? (b) Since Δ is not an integer, what must you do when reading the table, and what artifact appears if you simply round to the nearest index?

Exercise 28.8. Sketch (or compute in code) two ADSR envelopes: one for a plucked string and one for a bowed pad. Give rough numbers (in ms and sustain level) for each of A, D, S, R, and justify each choice in terms of how the real instrument behaves.

Exercise 28.9. Design a mapping from a wrist IMU to a subtractive synth voice. Specify: which sensor quantity triggers note-on/note-off, which quantity drives the filter cutoff (a MIDI continuous controller), and how you would smooth and debounce each. Then state explicitly one musical gesture this scheme *cannot* support, and which additional sensor (EMG, FMG, or a finger ring) would be needed to support it.

Exercise 28.10. (Code) Implement a chorus effect: a delay line of base length 25 ms whose read position is modulated by a 0.5 Hz sine of depth ± 5 ms, mixed 50/50 with the dry signal, using linear interpolation for the fractional delay. Run it on a sustained synth note and on a drum loop. Describe how the effect differs on the two sources and why (think about what a slowly varying delay does to a steady tone versus to sharp transients).

Exercise 28.11. A tremolo is $y[n] = (1 + d\cos(2\pi f_m n/f_s))\,x[n]$ applied to a tone $x[n] = \cos(2\pi f_0 n/f_s)$. Use a product-to-sum identity to show the output contains components at f_0 and $f_0 \pm f_m$. Why, at $f_m \approx 5$ Hz, do you perceive a single pulsing tone rather than three separate pitches?

Exercise 28.12. (Concept) Explain in two or three sentences why distortion, FM synthesis, and the harmonics of a clipped oscillator are all manifestations of the *same* underlying phenomenon, and contrast that with EQ and echo, which share a different underlying phenomenon. Name the two phenomena.

Chapter 29

Music and Instrument Signals

We have spent many chapters learning to take signals apart – into sinusoids, into spectra, into features. This chapter turns that machinery on one of the richest signal families humans have ever made: the sounds of musical instruments, and the control language we use to drive them. There are two reasons to care. The first is that music is the single best playground for everything we have built. A plucked string is a living demonstration of harmonics, decay, and timbre; a struck drum is a lesson in inharmonicity; a chord is polyphony you can hear. The second reason is the whole point of this book: at the end of it we want to build a *gesture controller* – something worn on the wrist and forearm that turns motion of the hand into music. To build that, we first have to know exactly what "music" means as a signal and as a stream of control messages, and we have to reckon with the brutal timing budget a playable instrument imposes. By the end of this chapter you should be able to look at a note and see four things at once: its pitch as a frequency, its timbre as a spectrum, its life as an amplitude envelope, and its representation as a handful of bytes called MIDI. That last picture is the target our controller will aim at.

29.1 A note is not a frequency – it is a spectrum with a shape

Ask someone what the note A above middle C "is" and they will say 440 Hz. That is true and it is almost useless. A pure 440 Hz sinusoid sounds like a hearing test, not like a violin, an oboe, or a piano playing A. Every real instrument playing that note produces energy at 440 Hz *and* at a whole ladder of higher frequencies, and it does so with an amplitude that rises and falls over the life of the note. The pitch is one number; the actual signal is a spectrum with a time-varying shape. The job of this section is to pull those two ideas – the ladder (harmonics) and the shape over time (the envelope) – apart so we can study each on its own, and then put them back together.

Intuition

A single instrument note is, to a first approximation, a sum of sinusoids whose frequencies are *integer multiples* of one fundamental, each with its own amplitude, the whole thing multiplied by a slow envelope that swells and fades. Pitch lives in the spacing of the ladder. Timbre – why a trumpet and a clarinet at the same pitch sound different – lives in the *relative heights* of the rungs. Attack and decay – why a piano sounds nothing like an organ – live in the envelope. Get these three apart in your mind and the rest of the chapter is bookkeeping.

Why integer multiples? It is not a coincidence or a convention; it falls straight out of the physics of the thing that vibrates. A string clamped at both ends, or an air column in a pipe, can only support standing waves that "fit" the boundary – a half wavelength, a full wavelength, three halves, and so on. Those allowed modes have frequencies $f_0, 2f_0, 3f_0, \ldots$, an arithmetic ladder built on the lowest mode f_0. We met exactly this structure in the Fourier series: any periodic signal of period $T_0 = 1/f_0$ is a sum of sinusoids at f_0 and its integer multiples. A sustained instrument note is (nearly) periodic, so its spectrum is (nearly) a comb of spikes at the harmonics.

Connection

This is the Fourier series wearing a costume. In the language of Section 29.1 a note is periodic with fundamental f_0; the Fourier-series coefficients c_k are exactly the complex amplitudes of the k-th harmonic. "Timbre" is just the magnitude profile $|c_k|$ versus k. Everything we proved about Fourier series – orthogonality of the harmonics, Parseval's energy identity – applies verbatim to the steady part of a musical tone.

Naming the ladder

Some vocabulary, because the words trip people up. The *fundamental* is f_0, the lowest mode, and it is the one your ear reports as "the pitch." The *harmonics* are the whole ladder $f_0, 2f_0, 3f_0, \ldots$; the k-th harmonic sits at kf_0. *Overtones* are all the modes *above* the fundamental, so the first overtone is the second harmonic. (Yes, the off-by-one is a needless historical cruelty. We will say "k-th harmonic" and avoid "overtone" to stay safe.)

Definition 29.1 (Harmonic series of a tone)**.** A periodic tone of fundamental frequency f_0 has spectral energy concentrated at the frequencies $f_k = kf_0$ for $k = 1, 2, 3, \ldots$. The complex amplitude c_k of the k-th harmonic sets both its loudness ($|c_k|$) and its phase ($\arg c_k$). The collection $\{|c_k|\}$, viewed as a shape over k, is the tone's *harmonic spectrum* or, loosely, its *timbre*.

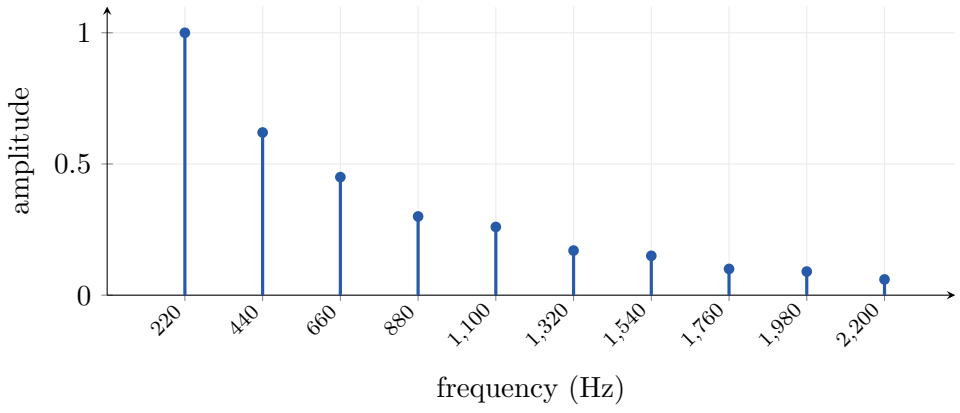


Figure 29.1: The harmonic comb of a sustained tone at $f_0 = 220$ Hz (the note A3). Energy appears only at integer multiples of the fundamental; the falling heights are this instrument's timbre. Change the spacing and you change the pitch; change the heights and you change which instrument it sounds like.

Timbre is the shape of the comb

Two notes at the same pitch have the same comb *spacing*; what differs is the *profile* of heights. A few archetypes worth carrying in your head:

- A **flute** or a gently bowed string near the bridge can be almost a pure fundamental with a couple of weak harmonics – a "round," soft sound.
- A **clarinet** famously suppresses the even harmonics (its cylindrical bore closed at one end behaves like the odd-harmonic series), so the $3f_0, 5f_0, 7f_0$ rungs dominate. That hollow, woody quality is the even harmonics being missing.
- A **trumpet** or a bowed string near the bridge is rich in high harmonics – many tall rungs far up the ladder – which we hear as "bright" or "buzzy."
- A **sawtooth** (the staple of analog synths) contains *every* harmonic with amplitude $1/k$; a **square** wave contains only odd harmonics, also $1/k$. These are not instruments but they are the reference shapes synthesis is built from.

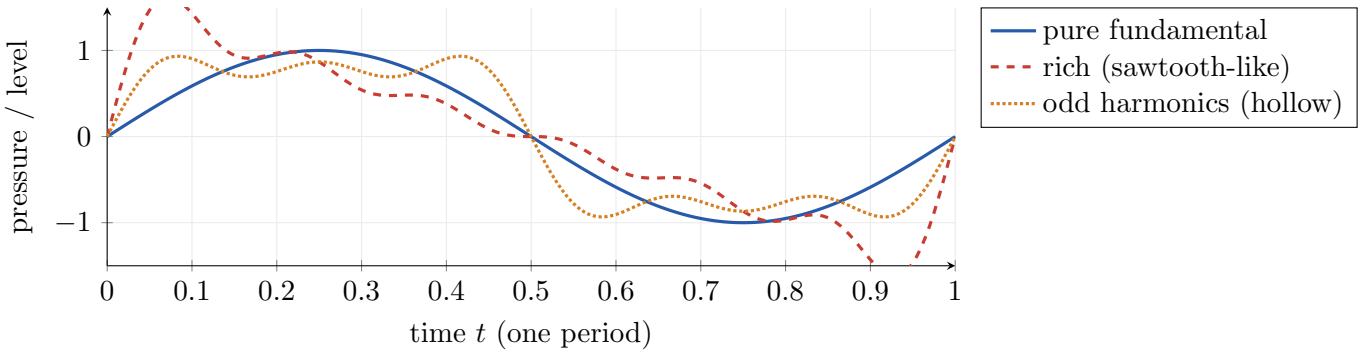


Figure 29.2: Same pitch (same period), three timbres. The waveform shape in time is just a different way of reading the harmonic profile: adding more, taller high harmonics sharpens the corners of the wave, which the ear hears as a brighter tone.

Why it works

Why does "more high harmonics" sound "brighter"? Sharp features in time – corners, jumps, fast wiggles – require high-frequency content to build, exactly as a square wave needs its endless odd harmonics to make its vertical edges. The ear's cochlea is itself a bank of frequency-tuned filters (Chapter 26), so it literally measures how much energy sits high up the ladder. A spectrum weighted toward the top excites the high-frequency channels more, and the brain labels that "bright." Timbre perception is, to a striking degree, just the ear reading the shape of the comb.

29.2 The envelope: attack, decay, and why a piano is not an organ

A sustained spectrum is only half the story, and for percussive instruments it is the smaller half. Pluck a string and the sound is loudest at the very start and then fades; bow a string and you can

hold it forever; strike a piano key and you get a sharp onset followed by a long, slow decay that never quite holds steady. That overall loudness-versus-time curve is the *amplitude envelope*, and it carries an astonishing amount of identity. The classic demonstration: record a piano note, cut off the first tenth of a second – the *attack transient* – and play the rest. It no longer sounds like a piano at all; people often guess organ or flute. The steady harmonics were barely touched. We removed only the beginning, and the instrument's identity went with it.

History

This was demonstrated systematically by Jean-Claude Risset and Max Mathews at Bell Labs in the 1960s, working on some of the first computer-synthesized instrument tones. They found that to make a convincing trumpet you could not just get the steady harmonics right; you had to reproduce how each harmonic *rises at a different rate* during the attack – the high harmonics often arrive late and decay first. Their work, and Mathews's MUSIC programs, are the direct ancestors of every software synthesizer and the reason we model notes as "spectrum times envelope, with the spectrum itself drifting" rather than as a fixed tone switched on and off.

The ADSR model

The workhorse description of an envelope, borrowed from synthesizers, breaks the note into four phases, the *ADSR* model:

- **Attack** – the rapid rise from silence to peak level when the note begins. For a plucked or struck note this is milliseconds; for a slowly bowed note it can be hundreds of milliseconds. This is the *attack transient*, and it is where most of the instrument's fingerprint lives.
- **Decay** – the drop from the peak down to a steady level after the initial onset.
- **Sustain** – the held level while the note is kept active (a key held down, a bow kept moving). Note this is a *level*, not a time; a piano has essentially no true sustain, its sound decaying throughout.
- **Release** – the fade to silence after the note is let go (key up).

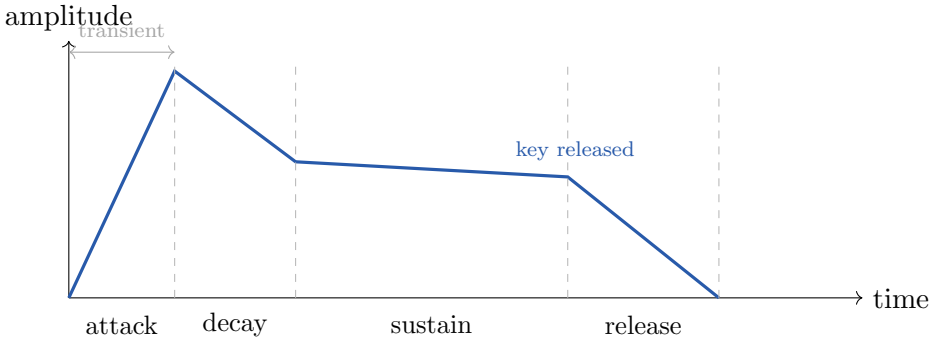


Figure 29.3: The ADSR amplitude envelope. The note's identity is concentrated in the steep attack transient at the left; the sustain is a held level, not a duration, and the release begins when the player lets go. A piano has a tall attack and essentially no flat sustain (the whole tail is decay); an organ is nearly all sustain with negligible attack and release.

What is really happening

What is really happening when you "hear the attack"? Two things at once. First, the broadband *transient* – the click of the hammer, the scrape of the bow catching, the chiff of air – is a short, sharp event, and short sharp events are spectrally wide (the time–bandwidth trade-off again). Second, during the attack the harmonics are not yet in their steady balance: they ramp up at different rates and the pitch itself may be momentarily unstable. The ear is exquisitely tuned to these first tens of milliseconds because in the natural world the onset of a sound is where the information about "what made it" is densest. This is also exactly why a controller that lags the onset feels dead, a point we return to in Section 29.7.

29.3 Inharmonicity: when the ladder bends

The integer-multiple story is an idealization. Real strings have stiffness, so their higher modes are pulled slightly *sharp* of the perfect ladder: the k-th partial of a real piano string sits not at kf_0 but at roughly

$$f_k \;=\; k f_0 \sqrt{1 + B\,k^2},$$

where B is a small *inharmonicity coefficient* that grows with string stiffness and is larger for the short, thick strings of the bass. This is why a piano is tuned with "stretched" octaves: the top notes are tuned a touch sharp and the bottom notes a touch flat so that the actual partials of one note line up with the actual partials of the next. A perfectly mathematically tuned piano sounds out of tune.

Common trap

Do not assume a pitch detector that hunts for perfectly integer-spaced peaks will work on real instruments. Piano, marimba, and especially bells violate the integer ladder badly. A bell's partials can be wildly inharmonic; the "pitch" you hear is sometimes a *missing fundamental* that the brain infers from the spacing of partials that are not even harmonic. Pitch is a perceptual construct, not just a peak in a spectrum.

Application

Drums and cymbals abandon the ladder entirely. A circular drumhead's modes are set by the zeros of Bessel functions, not by integers, so the partials are inharmonic and you perceive a sense of pitch only weakly (more for tom and timpani, almost none for a snare). A cymbal is closer to filtered noise than to a tone. This matters for the controller: *percussive* gestures are far easier to map than *pitched* ones, because a drum hit only needs an onset and a velocity, not a stable frequency. The first instrument our wrist controller will convincingly play is a drum kit.

29.4 Pitch and tuning: from frequency to note number

To turn "a frequency" into "a note" we need the map between hertz and the names musicians use. The modern Western system is *twelve-tone equal temperament*: the octave (a doubling of frequency) is

divided into twelve equal steps called *semitones*, each a frequency ratio of $2^{1/12} \approx 1.0595$. Everything is anchored to a reference, conventionally A4 = 440 Hz.

Definition 29.2 (Equal-tempered frequency)**.** With A4 = 440 Hz as the anchor, the frequency n semitones away is

$$f \;=\; 440 \cdot 2^{\,n/12} \quad \text{Hz}.$$

Conversely, the number of semitones above A4 of a frequency f is $n = 12\,\log_2(f/440)$, and fractional n measures how far the note is mistuned. One hundredth of a semitone is a *cent*, so the mistuning in cents is $1200\,\log_2(f/f_{\text{target}})$.

Example 29.1 (How far apart are adjacent notes?)**.** One semitone up from 440 Hz is $440 \cdot 2^{1/12} \approx 466.2$ Hz, a gap of about 26 Hz. But one semitone up from 110 Hz (A2) is $110 \cdot 2^{1/12} \approx 116.5$ Hz, a gap of only 6.5 Hz. The *ratio* is constant ($2^{1/12}$); the *absolute* gap shrinks fourfold every octave you go down. A pitch detector therefore needs much finer frequency resolution in the bass than in the treble – one of the reasons short DFT windows struggle with low notes (Chapter 10), and a reason the constant-Q transform, which spaces its bins logarithmically, is a natural fit for music.

Connection

The logarithm in Definition 29.2 is why music is a logarithmic phenomenon and why the right frequency axis for music is $\log f$. On a log axis the twelve semitones are *evenly* spaced and every octave is the same width, as in Figure 29.4. This is precisely the constant-Q / wavelet philosophy of Chapter 24: constant *ratio* bandwidth rather than constant *absolute* bandwidth. The ear does the same thing – pitch perception is roughly logarithmic in frequency.

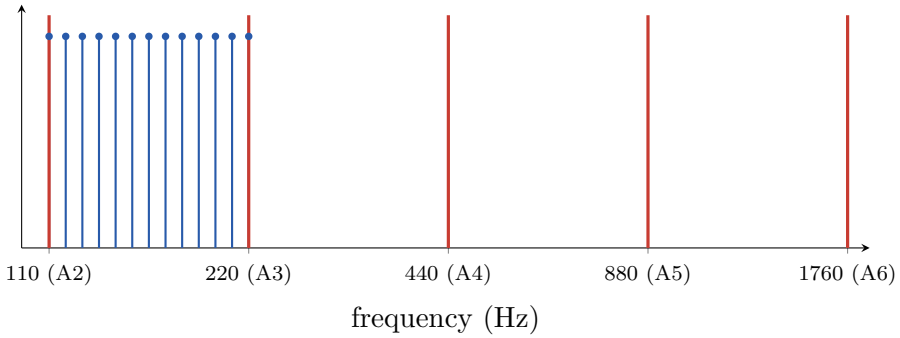


Figure 29.4: The equal-tempered octave A2–A3 on a $\log_2$ frequency axis. The blue comb marks the twelve semitones from 110 to 220 Hz; on this axis they are *evenly* spaced. The taller red marks are successive A's (110, 220, 440, 880, 1760 Hz), and every octave A2–A3–A4–A5 spans the same horizontal width even though the absolute hertz gaps double each octave. This even spacing is why a log axis is the natural one for music.

Finding f_0 from a recorded note is its own small art, because of the missing-fundamental problem and the inharmonicity above. Two families of method dominate.

Spectral. Take a DFT, find the harmonic comb, and infer f_0 from the spacing rather than from any single peak (a *harmonic product spectrum* multiplies downsampled copies of the magnitude spectrum so the true f_0 reinforces across harmonics). Robust to a weak or missing fundamental; limited in the bass by the DFT's resolution.

Time-domain autocorrelation. A periodic signal of period T_0 correlates strongly with itself shifted by T_0. So compute the autocorrelation and find the first strong peak away from zero lag; its lag is T_0 and $f_0 = 1/T_0$. The widely used *YIN* algorithm is a refined version using a difference function instead of a raw correlation to avoid octave errors. This works even when the fundamental is weak, because periodicity in time survives a missing fundamental.

Listing 29.1: Autocorrelation pitch estimate (the idea, not production code).

```python
import numpy as np

def estimate_f0(x, fs, fmin=50, fmax=1000):
    x = x - np.mean(x)
    corr = np.correlate(x, x, mode='full')[len(x)-1:]   # nonneg lags
    lo, hi = int(fs/fmax), int(fs/fmin)                  # lag search window
    lag = lo + int(np.argmax(corr[lo:hi]))              # first strong peak
    return fs / lag                                      # f0 in Hz
```

Walking question

Suppose you sample at $f_s = 8000$ Hz and the autocorrelation peak lands at lag 182 samples. What note is it? The period is $182/8000 = 22.75$ ms, so $f_0 = 8000/182 \approx 43.96$ Hz. Then $n = 12\log_2(43.96/440) \approx -39.9$ semitones below A4, i.e. essentially F1. Notice the resolution problem: the next available integer lag, 181, gives 44.2 Hz – already 10 cents away. In the bass, single-sample lag steps are coarse. Real pitch trackers interpolate between lags (parabolic fit) to do better.

29.5 Polyphony: more than one note at once

So far, one note. Real music is mostly *polyphonic*: chords on a piano, several strings in an orchestra, a singer over a guitar. In the time domain this is simply addition – the air pressure is the sum of every source – so the spectrum is the sum of every comb. That sounds easy and it is, going *forward*: to synthesize a chord you add the notes. Going *backward* – given a mixed recording, recover which notes are playing – is one of the genuinely hard problems in audio, called *multi-pitch estimation* or automatic music transcription.

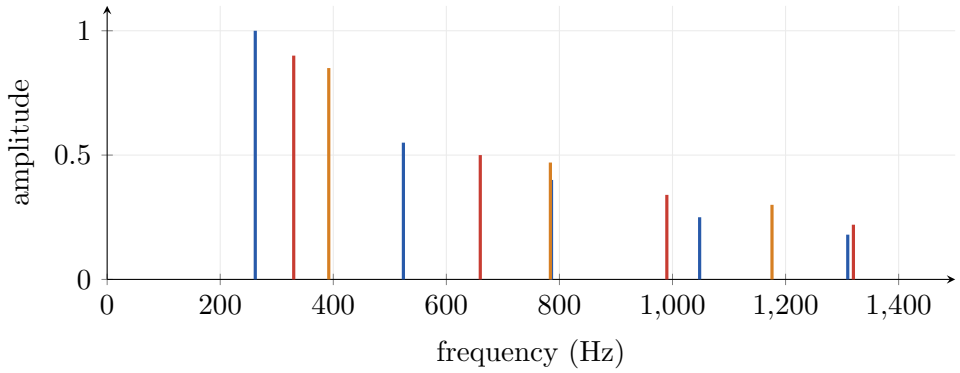


Figure 29.5: Three overlapping harmonic combs – a C major chord (C4, E4, G4). The combined spectrum is just the union of the three; a transcription algorithm must work backward to separate them. Note the near-collision around 784–786 Hz: the third harmonic of C4 and the second harmonic of G4 almost coincide (C and G are a fifth apart), which is exactly *why* that interval sounds consonant – and exactly why the two notes are hard to tell apart automatically.

Why it works

Why is multi-pitch hard when summing is so easy? Because harmonics *collide*. Consonant intervals are consonant precisely because their harmonics overlap: an octave shares every other harmonic, a fifth shares one in three. When two notes share spectral energy, no amount of looking at the magnitude spectrum can cleanly say how much belongs to each. The problem is ill-posed in the same way as "two numbers sum to 10, what are they?" Solving it requires priors – about how many notes are likely, about instrument timbres, about musical structure – which is why modern transcription leans on machine learning (Chapter 39). For our controller we sidestep this entirely: we *generate* polyphony from gestures rather than trying to *hear* it.

29.6 MIDI: music as a stream of bytes

Everything above describes music as a *signal* – pressure versus time. But there is a second, utterly different representation that our controller will actually produce: *MIDI*, the Musical Instrument Digital Interface, a standard from 1983 for sending performance *events* between devices. MIDI does not carry audio. It carries instructions: "press this key this hard now," "release it," "bend the pitch," "move this knob." A separate *synthesizer* or *sampler* turns those instructions into sound. This split – events here, sound there – is the single most important idea for the capstone, because a wrist sensor can plausibly produce *events* even when it could never produce convincing *audio*.

History

MIDI was hammered out in 1982–83 by Dave Smith (Sequential Circuits) and Ikutaro Kakehashi (Roland), deliberately as an open, royalty-free standard so synthesizers from rival makers could talk to each other. It was designed for the slow microcontrollers and cheap serial hardware of the time – 31,250 baud, one direction, a handful of bytes per event. That frugality is why it has survived four decades almost unchanged and why it runs happily on the tiny processors we will put on a wrist. MIDI 2.0 (2020) finally adds higher resolution and bidirectionality,

but classic MIDI 1.0 is still what you reach for first.

Anatomy of a message

A classic MIDI message is one *status* byte followed by one or two *data* bytes. The status byte has its high bit set (≥ 128) and encodes both the message type and a 4-bit channel (0–15, so up to 16 instruments on one wire). Data bytes have their high bit clear, so each carries 7 bits, a value 0–127. The handful you must know:

- **Note On** – status `0x90 | channel`, then *note number* (0–127) and *velocity* (0–127). Start a note. Velocity is how hard/fast the key was struck – usually mapped to loudness and often to brightness too.
- **Note Off** – status `0x80 | channel`, then note number and release velocity. Stop a note. (A Note On with velocity 0 is, by convention, also a Note Off – a trick that lets a stream of running Note Ons turn notes off cheaply.)
- **Control Change (CC)** – status `0xB0 | channel`, then a *controller number* (0–127) and a *value* (0–127). The general-purpose continuous knob: CC 1 is the modulation wheel, CC 7 is channel volume, CC 11 is expression, CC 64 is the sustain pedal.
- **Pitch Bend** – status `0xE0 | channel`, then two data bytes forming a 14-bit value (0–16383, center 8192). A high-resolution continuous slide in pitch.
- **Program Change** – status `0xC0 | channel`, then one byte: choose which instrument "patch" this channel plays.

Definition 29.3 (MIDI note number)**.** MIDI numbers the equal-tempered keyboard so that A4 = 440 Hz is note 69, and each semitone is one unit. The frequency of MIDI note m is

$$f(m) \;=\; 440 \cdot 2^{(m-69)/12} \quad \text{Hz},$$

and the nearest MIDI note to a measured frequency f is $m = \operatorname{round}\big(69 + 12\log_2(f/440)\big)$. Middle C (C4) is note 60; the standard 88-key piano spans notes 21 (A0) to 108 (C8).

Example 29.2 (Bytes for a note)**.** To play middle C at medium-strong velocity on channel 1, then release it, you send six bytes:

$$\underbrace{\texttt{0x90}}_{\text{Note On, ch 1}}\ \underbrace{\texttt{0x3C}}_{60=\text{C4}}\ \underbrace{\texttt{0x64}}_{100=\text{velocity}} \qquad \text{then later} \qquad \underbrace{\texttt{0x80}}_{\text{Note Off}}\ \underbrace{\texttt{0x3C}}_{60}\ \underbrace{\texttt{0x00}}_{\text{rel. vel.}}.$$

Three bytes to start a note, three to stop it. At the MIDI baud rate each byte takes about 0.32 ms, so a Note On is on the wire in roughly 1 ms – a number we will need in Section 29.7.

Common trap

MIDI is *not* sound and a MIDI note number is *not* a frequency in the analog sense. Note 60 is exactly C4 – you cannot send "a little sharp" with a note number alone; for continuous pitch you must add Pitch Bend (14-bit) on top. Likewise, everything in classic MIDI is quantized to 7 bits (128 levels). Velocity has only 128 steps; CC has only 128 steps. For a gesture

controller streaming a smoothly moving hand, that coarse quantization can produce audible "zipper" steps – a real design constraint we will smooth around, or escape with MIDI 2.0's higher resolution.

Application

Because MIDI is just events, the *same* performance can drive a grand piano, a string section, or a drum machine – you only change the synth on the other end, or the Program Change you send. Channel 10 is reserved by the General MIDI convention for percussion, where the note *number* chooses the drum (note 36 = kick, 38 = snare, 42 = closed hi-hat) instead of a pitch. This is why drums are our easiest first target: a gesture becomes a single Note On to channel 10 with the right number, and velocity becomes how hard you hit.

Continuous control versus discrete events

Notice the two flavors of message, because they map onto two flavors of gesture. *Note On/Off* are *discrete, timed events* – a tap, a strike, a trigger. *CC and Pitch Bend* are *continuous streams* – a tilt, a sweep, a squeeze, sampled and sent maybe 50–200 times a second. A good controller uses both: a sharp wrist flick fires a Note On (a drum hit, a plucked note); a slow forearm rotation rides a CC (a filter sweep, a swell in volume). Mapping which motions become events and which become streams is the heart of the capstone's design.

Listing 29.2: Turn a measured pitch and a gesture intensity into MIDI bytes.

```
def freq_to_note(f):
    return round(69 + 12 * math.log2(f / 440.0))

def note_on(note, velocity, channel=0):
    return bytes([0x90 | channel, note & 0x7F, velocity & 0x7F])

def note_off(note, channel=0):
    return bytes([0x80 | channel, note & 0x7F, 0])

def control_change(cc, value, channel=0):
    return bytes([0xB0 | channel, cc & 0x7F, value & 0x7F])

# Example: a flick of intensity 0..1 fires a snare hit on the drum channel (9 == "channel 10")
def flick_to_snare(intensity):
    vel = max(1, min(127, int(intensity * 127)))
    return note_on(38, vel, channel=9)
```

29.7 Latency: the unforgiving budget of a playable instrument

Here is the constraint that will discipline every design choice in the capstone. An instrument must respond to the player *fast enough that the response feels simultaneous with the action.* Not "fast." Fast enough to be invisible. The human perceptual threshold for action-to-sound delay is brutally

tight, and missing it does not make an instrument that feels slow – it makes an instrument that feels broken, because your hand and your ear disagree about when the note happened.

Focus

The numbers, roughly, from the perception and music-technology literature. Around **10 ms** of action-to-sound latency is the comfortable target – below it almost nobody notices. Around **20–30 ms** skilled players begin to feel a sluggishness, especially on percussive attacks. Past **40–50 ms** the delay is plainly audible to most people and rhythmic playing falls apart. For comparison, standing 3.4 m from your own amplifier already costs 10 ms in sound travel time, which is why drummers are so sensitive to it. The whole signal chain – sense, process, transmit, synthesize, output – must fit inside this budget.

Why is the attack so much less forgiving than a held note? Because, as Section 29.2 argued, the onset is where the ear's clock is set. A held swell can drift in by a few extra milliseconds and nobody minds; a drum hit that lands 30 ms after the flick feels rubbery and wrong. Percussive, transient-heavy gestures are simultaneously the *easiest* to detect (a sharp event is easy to spot) and the *least* forgiving of delay. That tension is the central engineering problem of the controller.

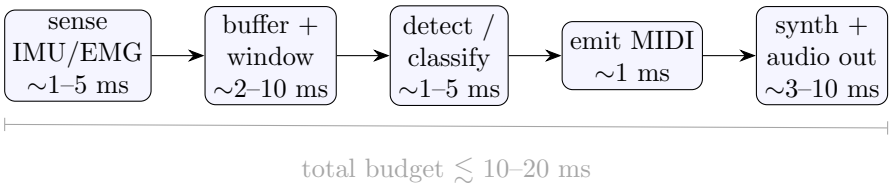


Figure 29.6: The latency budget, end to end. Every stage from the wrist to the speaker spends part of the ~10–20 ms total. The buffering/windowing stage is usually the biggest and the most under our control: a longer analysis window gives cleaner detection but eats latency directly. Choosing that window length is the central trade-off of the build.

Common trap

The latency villain is almost always *buffering*, not computation. To analyze a gesture you collect a window of samples; a 256-sample window at 1 kHz is already 256 ms – catastrophic. You cannot wait for a long window and also be fast. This is the same time–frequency uncertainty we have met throughout the book: a longer window buys cleaner frequency/feature estimates but costs you exactly that much delay. Onset detection that triggers on the *leading edge* of a gesture, rather than waiting to see the whole thing, is how playable controllers escape this trap – pay in occasional false triggers, not in latency.

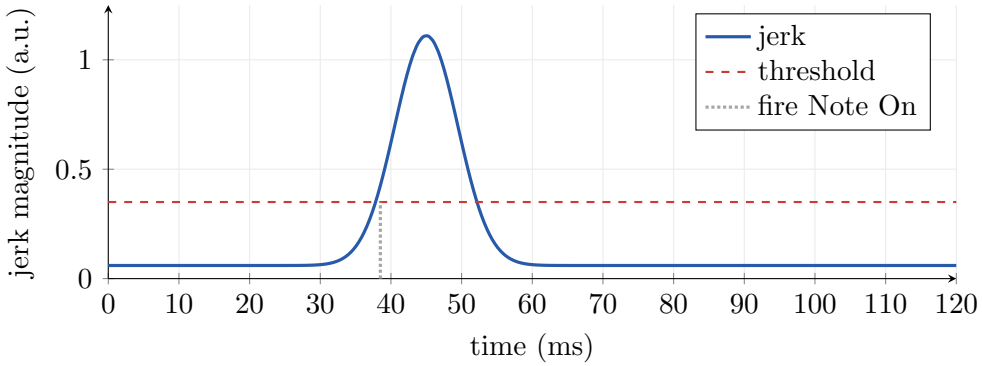


Figure 29.7: A wrist flick seen as jerk magnitude (derivative of acceleration) from the IMU. The Note On fires the instant the rising edge crosses the threshold, near $t = 38$ ms here – well before the peak at $t = 45$ ms. Waiting for the peak, or for a full analysis window, would add the difference straight to the latency budget. Triggering on the leading edge is what keeps a percussive gesture feeling instant.

Build it (hardware)

Concrete budget for a wrist controller streaming MIDI over BLE to a phone synth:

- **Sense.** Sample the IMU at ≥ 200 Hz (gyro/accel), EMG/FMG at ≥ 1 kHz. One sample period is 1–5 ms. Use the sensor's own FIFO/interrupt, do not poll lazily.
- **Detect.** Trigger Note On on the *rising edge* of a feature (jerk magnitude, EMG envelope crossing a threshold), as in Figure 29.7. Keep the decision window short – tens of samples – so detection adds only a few ms.
- **Transmit.** Classic serial MIDI puts a 3-byte Note On on the wire in ∼1 ms. **Bluetooth LE MIDI is the trap**: the BLE connection interval (often 7.5–15 ms, and on some phones much worse) can dominate your entire budget. Request the shortest connection interval the link allows, and prefer a wired/USB path when you need true tightness.
- **Synthesize.** A phone or laptop softsynth with a small audio buffer adds 3–10 ms; on Android use the low-latency audio path (AAudio), on a Pi use a small ALSA period.

Measure it, do not trust the spec sheet: tap the sensor against a mic, record both the tap transient and the synth output on one track, and read the gap. Anything under ∼15 ms feels alive.

Connection

The whole edifice of this section rests on facts from earlier chapters. The window-length-versus-delay trade-off is the time–frequency uncertainty principle of Chapters 7 and 10. "Trigger on the rising edge" is onset/edge detection – a high-pass or derivative filter (Chapter 16) followed by a threshold. And "the BLE interval dominates" is just sampling theory applied to the transport: your update rate is capped by how often the channel actually delivers, no matter how fast you compute.

29.8 Setting the musical target for the capstone

We can now state precisely what the gesture controller is trying to do, in the vocabulary of this chapter. It is a device that converts forearm and wrist motion into a stream of MIDI events – discrete Note On/Off for strikes and plucks, continuous CC and Pitch Bend for sweeps and bends – fast enough to feel simultaneous with the gesture, which a downstream synth turns into sound. That framing already tells us the shape of the build and, crucially, its real limits.

Toward the wrist tracker

The target, and the limits that go with it. Our controller's job is to emit musically useful MIDI from wrist/forearm sensing within the latency budget of Section 29.7. What that realistically allows, and what it does not:

- **Strongly within reach: discrete, percussive triggers.** A wrist-worn IMU detects sharp motion – taps, flicks, strikes – very well. Map each to a Note On (a drum on channel 10, a plucked note), with gesture intensity to velocity. This is the first thing to build because onset detection is robust and drums tolerate the most timbral slop.
- **Within reach: continuous macro-control.** Forearm orientation from sensor fusion (Chapter 36) drives CC and Pitch Bend smoothly – tilt for a filter sweep, rotation for volume, a held pose for a sustained swell. Watch the 7-bit "zipper" quantization of Section 29.6; smooth and rate-limit the stream.
- **Reachable only with the right sensor: coarse hand states.** A few *discrete* hand configurations – fist vs. open, a pinch – can be read from *EMG or force-myography* on the forearm, and mapped to mode switches or note selection. This needs muscle sensing, not just the IMU.
- **Out of reach, and we will say so plainly: continuous per-finger pitch.** A wrist-worn IMU *cannot* recover continuous individual finger joint angles – it senses the whole forearm, not the fingers. EMG/FMG can distinguish a handful of coarse gestures, not a precise continuous finger pose. Genuinely continuous finger tracking needs sensors *on the fingers* (rings, gloves) or external cameras. So a wrist device is a fine *drummer* and *macro-controller* and a poor *pianist*; we design for what the sensors can actually deliver, not for the demo we wish we could give.

Why this matters

The reason to lay out the musical target now, chapters before we wire anything, is that the music defines the spec. "Playable" is not a vibe; it is $\lesssim$ 15 ms of latency, 128 velocity levels, an integer note number plus a 14-bit bend, and an onset detector that fires on a leading edge. Every later chapter – IMUs, calibration, fusion, EMG/FMG, gesture recognition – is in service of hitting that spec. Knowing what a note *is*, as both a signal and a message, is what lets us judge whether the hardware we build is actually an instrument or merely a sensor that beeps.

29.9 Exercises

Exercise 29.1. A sustained oboe note has its strongest energy not in the fundamental but in the second and third harmonics. (a) Sketch a plausible harmonic comb for fundamental $f_0 = 440$ Hz with this property. (b) Explain, using Section 29.4's missing-fundamental idea, why you would still hear the pitch as 440 Hz even if you filtered the fundamental out entirely.

Exercise 29.2. Compute the frequencies of the first five harmonics of a piano note with $f_0 = 110$ Hz, first under the ideal integer ladder and then under the inharmonic model $f_k = k f_0 \sqrt{1 + Bk^2}$ with $B = 0.0004$. By how many cents is the fifth partial sharp of $5 f_0$? (Recall cents $= 1200 \log_2$ of the ratio.)

Exercise 29.3. Using Definition 29.3, find the MIDI note number nearest to (a) 261.6 Hz, (b) 1000 Hz, (c) 55 Hz. For each, report the residual mistuning in cents between the measured frequency and the exact equal-tempered frequency of that note.

Exercise 29.4. Write the exact byte sequence (in hex) to: play A4 at velocity 80 on channel 1, then while it is held send a modulation-wheel CC of value 64, then release the note. Use the message formats in Section 29.6.

Exercise 29.5. A controller samples a gesture feature at $f_s = 500$ Hz and detects an onset by waiting for a full N-sample window before deciding. (a) For $N = 8, 32, 128$, what latency does the buffering alone add? (b) Which of these fit inside a 15 ms total budget, leaving room for the other stages of Figure 29.6? (c) Explain why "trigger on the rising edge" beats "wait for the whole window."

Exercise 29.6. A C major chord plays C4 (≈ 262 Hz), E4 (≈ 330 Hz), G4 (≈ 392 Hz). List the first four harmonics of each and find every pair of harmonics from *different* notes that fall within 5 Hz of one another. Relate your answer to the `whybox` claim that consonance and hard-to-separate spectra are the same phenomenon.

Exercise 29.7. Classic MIDI velocity and CC are 7-bit (0–127). Suppose a forearm-tilt gesture sweeps a CC linearly over 2 seconds and you send updates at 100 Hz. (a) How many distinct CC values can the sweep pass through? (b) If the synth maps CC to a filter cutoff over 5 octaves, how many cents does one CC step correspond to, and is that audible? (c) Suggest two ways to hide the steps.

Exercise 29.8. The Risset/Mathews experiment (Section 29.2) showed that removing a note's attack transient destroys its identity. Design a small experiment with free software (e.g. record a single piano note, a single flute note) to test this: describe exactly which ∼50 ms you would delete, what you would play to listeners, and what result would confirm that timbre lives in the attack.

Exercise 29.9. Explain, in two or three sentences each and citing the relevant claim in the chapter, why each of the following capstone goals is realistic or not for a *wrist-only IMU*: (a) triggering a snare drum on a flick; (b) a continuous volume swell from forearm rotation; (c) recovering the continuous bend angle of the index finger; (d) distinguishing a closed fist from an open hand.

Exercise 29.10. (Open-ended build.) Sketch the block diagram of a minimal wrist drum-trigger: sensor and sample rate, the feature you threshold for onset, how you map intensity to velocity, the channel and note numbers for kick/snare/hi-hat, and the transport to the synth. Annotate each block with its latency budget so the total lands under 15 ms, and identify which block you expect to be the bottleneck.

Part VII

Image Signal Processing

Chapter 30

Images as Two-Dimensional Signals

For thirty chapters we have lived on a line. A signal was a function of one variable – almost always time – and everything we built, from sinusoids to the Fourier transform to convolution, was a story about how a single number wiggles as that one variable marches forward. Now we are going to add an axis. An image is what you get when the thing you measure depends not on *when* but on *where*: not $x(t)$ but $f(x, y)$, a brightness that varies as you move left and right *and* up and down across a plane.

This is a bigger leap than it sounds, and a smaller one than you fear. Bigger, because two dimensions buy you genuinely new phenomena – edges have orientation, textures have direction, and "frequency" splits into a vector that points somewhere. Smaller, because almost every tool you already own transfers across with one extra index bolted on. A pixel grid is just a 2-D array of samples. Brightness is just amplitude. The histogram you may have met for a random variable is exactly the right object for understanding contrast. And in the next chapters the 2-D Fourier transform will turn out to be the 1-D transform applied twice, once along each axis.

This chapter lays the foundation. We will see an image as a sampled function on a grid, understand what a pixel actually is and what intensity and color mean, think carefully about 2-D sampling and resolution (the same Nyquist story, now in a plane), meet brightness, contrast, and the all-important histogram, and learn the simplest family of image operations – *point operations*, which touch each pixel in isolation. By the end you will look at any image and see a signal.

30.1 From one axis to two

Start with what you know. A discrete-time signal is a list of numbers indexed by an integer: $x[n]$, where n runs $0, 1, 2, \ldots$ and each $x[n]$ is the value of the signal at that sample. Plot it and you get the familiar stem-and-line picture: time across, amplitude up.

A grayscale image is the same idea with *two* integer indices. We write $f[m, n]$, where m selects a row and n selects a column, and $f[m, n]$ is the brightness of the picture at that grid location. Instead of a list you have a *rectangular array* of numbers – a matrix, in the linear-algebra sense. Where a 1-D signal of length N is a point in $\mathbb{R}^N$, an image of height M and width N is a point in $\mathbb{R}^{M \times N}$: a single vector with MN coordinates, one per pixel.

Intuition

Picture the difference physically. A 1-D signal is a single microphone reporting one pressure value as time passes. A 2-D image is a *room full* of tiny light meters arranged in a grid, each reporting how bright it is at its own spot, all at the same instant. Time has been replaced by position. The signal no longer flows past you; it sits there, spread across a plane, and you read it by moving your eye instead of waiting.

It helps to keep both the *continuous* and the *sampled* pictures in mind, exactly as we did for time signals. The world presents a continuous image $f(x, y)$ – the real light field falling on a sensor, defined for every real position (x, y) in some rectangle. A camera cannot store a function of real numbers, so it *samples*: it measures the light only on a grid of points and *quantizes* each measurement to a finite number of levels. The result is the array $f[m, n]$. The continuous-to-discrete jump is the same one from the sampling chapter, just performed along two spatial axes instead of one temporal axis.

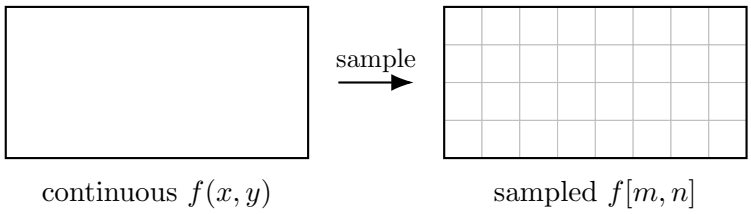


Figure 30.1: The continuous-to-discrete jump in 2-D. The real light field (left) is a function on a continuous rectangle; the camera samples it on a grid and stores one number per cell (right). Each cell is a pixel.

Focus

Two conventions will save you grief. **(1) Row, then column.** We index $f[m, n]$ with m as the row (vertical) and n as the column (horizontal), like a matrix. **(2) Rows count downward.** In image coordinates $m = 0$ is the *top* row and increases as you go down, because that is the order pixels stream out of a sensor and into a file. This flips the y-axis relative to a math plot, and is the source of countless upside-down images. We will be explicit whenever it matters.

30.2 What a pixel really is

The word "pixel" (picture element) gets used for two different things, and conflating them causes real bugs. A pixel is, first, a *sample*: a single number (or small group of numbers) measured at one grid location. A pixel is also, loosely, the little *square* you see when you zoom in on a screen. Those squares are a *rendering choice*, not the data. The data is a set of point samples; the squares are how we paint each sample so the eye can see it.

Common trap

"A pixel is a tiny square." This is the most persistent misconception in imaging. The samples are points; the square (or, on a sensor, the little light-collecting area) is the reconstruction. Believing pixels are squares leads people to expect that "zooming in" should show sharper

squares, when in truth a correct reconstruction should show a smoothly interpolated surface – the 2-D analog of connecting the dots in a 1-D plot. Alvy Ray Smith's classic 1995 note "A Pixel Is Not A Little Square" is the canonical takedown.

A grayscale pixel holds one number: its **intensity** or brightness. How that number is stored is the matter of *quantization*, which we treat properly in the chapter on quantization; here we only need the headline. Most photographs store intensity in 8 bits, giving integer levels $0, 1, \ldots, 255$, where 0 is black and 255 is white. Eight bits is not sacred – medical and scientific images often use 12, 14, or 16 bits, and internal processing is frequently done in floating point on the range $[0, 1]$ to avoid rounding. The key idea is that the stored value is a quantized estimate of a continuous physical brightness.

Connection

The image-as-vector view is the bridge to the linear-algebra companion. An $M \times N$ image *is* a vector in $\mathbb{R}^{MN}$; flattening it row by row (or column by column) is just choosing coordinates. Every operation we will study – brightening, blurring, edge detection – is a function on this vector space, and the nicest ones are *linear*, i.e. matrices. The inner product $\langle f, g \rangle = \sum_{m,n} f[m,n]\, g[m,n]$ measures how much two images overlap, exactly the $\langle \cdot, \cdot \rangle$ we used to project signals onto sinusoids in the signals-as-vectors chapter. Orthogonality, projection, and bases all carry over unchanged.

30.2.1 Color: three signals stacked

A color image is not one signal but several, sampled on the same grid. The human eye has three kinds of color-sensitive cone, roughly tuned to long, medium, and short wavelengths, so three numbers per location are enough to reproduce almost any color a person can see. The default representation is **RGB**: three arrays $R[m,n]$, $G[m,n]$, $B[m,n]$ giving the red, green, and blue intensities. Stacked, they form a *three-channel* image – think of it as three grayscale signals lying on top of one another, or as a single function whose value is a vector (R, G, B) rather than a scalar.

What is really happening

What is really happening with color is that we have moved from a scalar-valued signal to a *vector-valued* one. Each pixel carries a little vector in a 3-D "color space". Nothing about the geometry of the grid changes – the sampling, the resolution, the neighbor relationships are all identical to the grayscale case. We have simply attached a 3-vector to every grid point. Every single-channel technique in this book applies to a color image by running it on each channel (with the caveat that the channels are correlated, so independent processing can shift colors).

RGB is convenient for displays (which literally have red, green, and blue emitters) but awkward for analysis, because brightness and color are tangled across all three numbers. Make something twice as bright and all three of R, G, B change. So we often convert to a space that *separates* a single brightness channel from the color. The most useful for our purposes is some form of **luma plus chroma**. A standard luma (the brightness a grayscale viewer would see) is the weighted sum

$$Y = 0.299\, R + 0.587\, G + 0.114\, B, \tag{30.1}$$

with green weighted most because the eye is most sensitive to green. The unequal weights are not arbitrary: they are the eye's own sensitivity curve, baked into the standard. The remaining two numbers (the chroma) describe hue and saturation. Closely related spaces include **HSV/HSL** (hue, saturation, value/lightness), which are handy when you want to threshold "how red is this" independently of "how bright is this."

Table 30.1: Common color representations and what each is good for.

Space	Components	Good for
RGB	red, green, blue	displays, raw capture, storage
Grayscale	single luma Y	analysis where color is irrelevant
YCbCr	luma + 2 chroma	compression (JPEG), video
HSV/HSL	hue, saturation, value	color thresholding, painting tools

Why it works

Why bother converting to luma before analysis? Because most of the *structure* we care about – edges, shapes, motion – lives in brightness, not color. The eye agrees: it has far more brightness-sensitive rods than color-sensitive cones, and far sharper spatial acuity for luma than for chroma. This is exactly why JPEG and every video codec store luma at full resolution but *subsample* the chroma (the "4:2:0" you see in codec settings): you can halve the color resolution and almost nobody notices. Working in luma both matches the data's information content and cuts your computation by a factor of three.

30.3 Sampling and resolution in two dimensions

The number of samples is the image's **resolution**: an image that is 1920×1080 has 1920 columns and 1080 rows, about two million pixels. But raw pixel count is only half the story. What matters for fidelity is the *sample spacing* relative to how fast the scene's brightness changes – the exact 2-D version of Nyquist.

Recall the 1-D rule: to capture a signal without aliasing, you must sample at more than twice the highest frequency present. In 2-D the same rule applies *separately along each axis*, in terms of **spatial frequency** – how many brightness cycles fit per unit distance. A picket fence photographed straight on is a near-sinusoid in the horizontal direction; if its bars are finer than two samples per cycle, you get aliasing. Sample spacing Δx lets you represent horizontal spatial frequencies up to $1/(2\Delta x)$ cycles per unit length without aliasing; the same holds for Δy vertically.

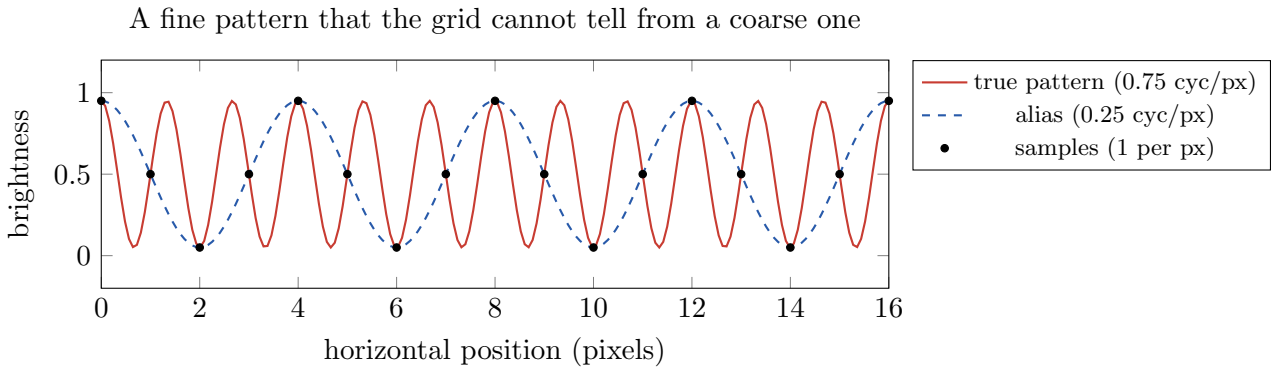


Figure 30.2: One row of pixels through a fine periodic pattern. The true pattern (red, 0.75 cycles per pixel) is finer than the Nyquist limit of 0.5 cycles per pixel, so the samples (dots, one per pixel) land in exactly the same places as a much coarser pattern (blue dashed, 0.25 cycles per pixel). The grid cannot distinguish the two: the fine pattern masquerades as the coarse one. This is aliasing, exactly as in 1-D, now along a spatial axis.

What is really happening

The famous *moiré* patterns you see when a camera photographs a striped shirt or a brick wall, and the jagged "stair-steps" on diagonal lines, are both spatial aliasing. High spatial frequencies in the scene – finer than two samples per cycle – fold down and appear as spurious low-frequency patterns that were never in the scene. Camera designers fight this with an *optical low-pass filter* (a slight deliberate blur in front of the sensor) – the optical version of the anti-aliasing filter you put before an ADC, and the first stage of the capture pipeline in Figure 30.3.

Resolution is therefore a contract between sample spacing and the finest detail you intend to capture. More pixels help only up to the point where the optics (and the anti-alias filter) stop delivering finer detail; past that you are sampling blur. This is why a 50-megapixel phone photo through a smudged plastic lens can carry less real information than a sharp 12-megapixel shot.

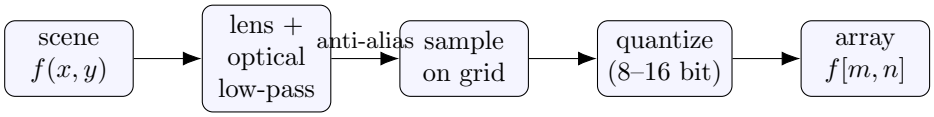


Figure 30.3: The capture pipeline, stage for stage the 2-D twin of the 1-D analog-to-digital chain. The optical low-pass filter band-limits the light field *before* the grid samples it – the spatial anti-alias filter – and quantization rounds each sample to a finite set of levels.

Connection

The same sampling theorem governs the time axis for our IMU and EMG signals (the sampling-and-Nyquist chapter) and the spatial axes here. "Sample fast enough relative to the fastest variation" is one theorem wearing two costumes. The difference is dimensionality: 1-D signals alias into a wrong tone, 2-D signals alias into a wrong *pattern with an orientation.*

30.4 Brightness and contrast

Two of the most basic image qualities have precise meanings as operations on the pixel values. **Brightness** is the overall level – the average intensity. Add a constant to every pixel and you raise the brightness, shifting the whole image lighter (or darker, for a negative constant):

$$g[m, n] = f[m, n] + b. \tag{30.2}$$

Contrast is the *spread* of intensities – how far the bright and dark pixels sit from the middle. Multiply every pixel's deviation from a midpoint by a gain $a > 1$ and you stretch the values apart, increasing contrast; $a < 1$ squeezes them together, washing the image out:

$$g[m, n] = a\,\big(f[m, n] - c\big) + c, \tag{30.3}$$

where c is the pivot (often the mid-gray 127.5 for 8-bit data). Combine the two and you have the general affine intensity map $g = a\,f + b'$.

Intuition

Think of the intensities as dots on a number line from black to white. *Brightness* slides all the dots together left or right. *Contrast* pulls them apart from (or pushes them toward) a center, like stretching a rubber band pinned at mid-gray. Neither operation cares *where* a pixel is in the image – only what value it holds. That locality-blindness is the defining feature of the point operations we study next.

Common trap

After an affine map some results fall outside the valid range – below 0 or above 255. You must *clip* (clamp) them back into range. Forgetting to clip, or letting an 8-bit integer wrap around (so 256 becomes 0), produces ugly artifacts: blown highlights turning black, or speckle where bright areas overflow. Always saturate, never wrap.

30.5 The histogram: a signal's value distribution

If you want to understand an image's brightness and contrast at a glance, do not stare at the pixels – look at the **histogram**. The histogram counts how many pixels hold each intensity value. For 8-bit data it is an array $h[k]$, $k = 0, \ldots, 255$, where $h[k]$ is the number of pixels with intensity exactly k. Normalize by the total pixel count MN and you get $p[k] = h[k]/(MN)$, the *fraction* of pixels at each level – a probability distribution over intensity.

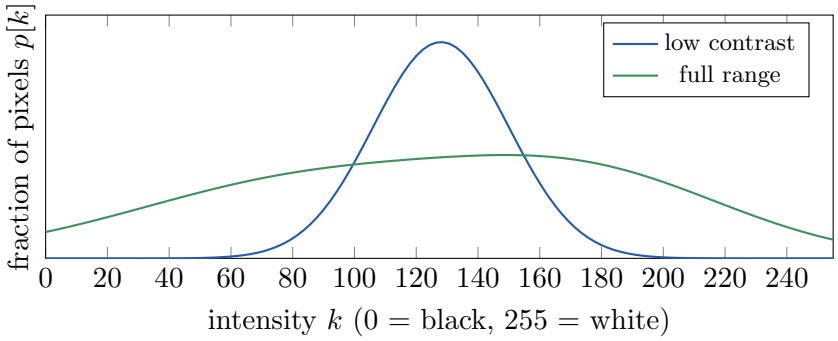


Figure 30.4: Two histograms. The narrow blue distribution uses only a sliver of the available range – a flat, low-contrast image. The wide green distribution spreads across the range and is bimodal (a dark subject on a light background, say) – a punchy, well-exposed image.

The shape of the histogram tells you nearly everything about exposure and contrast. A histogram bunched near 0 means an underexposed (dark) image; bunched near 255 means overexposed. A narrow spike anywhere means low contrast – the pixels barely differ. A distribution spread across the whole axis means good contrast. Two separated humps often mean a clear subject and background. Crucially, the histogram *throws away all spatial information*: shuffle the pixels into random positions and the histogram is unchanged. It is purely a statement about the distribution of *values*.

Connection

The normalized histogram $p[k]$ is a genuine probability mass function over the random variable "intensity of a pixel picked at random," which is exactly the object from the probability companion. Its mean $\bar{f} = \sum_k k\, p[k] = \mathbb{E}[\text{intensity}]$ is the image's average brightness, and its variance $\sum_k (k - \bar{f})^2\, p[k] = \text{Var}[\text{intensity}]$ is a clean measure of contrast. Brightness and contrast, defined informally above, are just the first two moments of this distribution. Histogram equalization (below) is the "make it uniform" transform from probability, applied to pixels.

30.5.1 Histogram stretching and equalization

If the histogram is narrow, you can improve contrast by **stretching** it to fill the range. Find the actual minimum and maximum intensities $f_{\min}$ and $f_{\max}$ in the image and apply the linear map that sends $f_{\min} \to 0$ and $f_{\max} \to 255$:

$$g[m,n] = 255 \cdot \frac{f[m,n] - f_{\min}}{f_{\max} - f_{\min}}. \tag{30.4}$$

This is just the affine map of (30.3) with the gain and pivot chosen automatically from the data.

Histogram equalization is cleverer. Instead of a linear stretch, it applies a nonlinear remapping that makes the *output* histogram as flat (uniform) as possible, allocating the most output range to the most populated input levels. The recipe is the cumulative distribution: compute the running total

$$C[k] = \sum_{i=0}^{k} p[i], \tag{30.5}$$

then map each intensity $k \mapsto \lfloor 255\, C[k] \rfloor$. Pixel values that are common get spread far apart (high contrast where it matters); rare values get squeezed together.

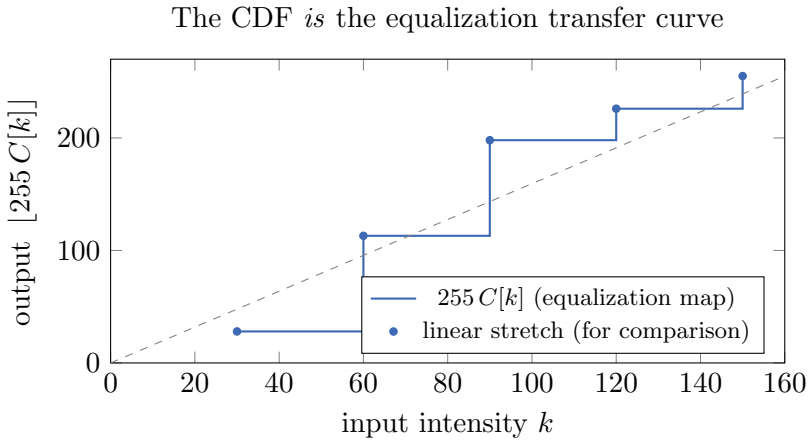


Figure 30.5: Histogram equalization for the worked-example patch of Section 30.8. The transfer curve is the scaled CDF: it climbs steeply where many pixels live (the crowded 60 and 90 levels) and gently where pixels are sparse. The steep segments stretch common intensities far apart, which is where the contrast goes. The dashed line is the linear stretch, which ignores how many pixels sit at each level.

Why it works

Why does mapping through the CDF flatten the histogram? Because the CDF is exactly the transform that turns *any* distribution into a uniform one – the "probability integral transform" from the probability book. If a random variable X has CDF C, then $C(X)$ is uniform on $[0, 1]$. Applying C to the pixel values therefore drives the value distribution toward flat, which means intensities are now spread evenly across the available range – the definition of maximum global contrast. (In practice it is approximate, because intensities are discrete and many pixels share a value, so the result is uniform-ish, not exactly uniform.)

Try it yourself

With any image library, load a photo, convert to grayscale, and plot its histogram. Then try the three contrast operations – a manual affine map, a min/max stretch, and equalization – and re-plot the histogram each time. Watch how stretching widens the support and equalization flattens it. Do it on a deliberately underexposed photo to see the dramatic version.

```
import numpy as np

def to_gray(rgb):                          # rgb: H x W x 3, uint8
    w = np.array([0.299, 0.587, 0.114])
    return rgb.astype(float) @ w           # H x W, float luma

def histogram(gray, levels=256):
    h, _ = np.histogram(gray, bins=levels, range=(0, levels))
    return h

```

```
def equalize(gray):
    g = np.clip(gray, 0, 255).astype(int)
    h = histogram(g)
    cdf = np.cumsum(h) / g.size             # normalized CDF in [0,1]
    lut = np.floor(255 * cdf).astype(np.uint8)   # 256-entry lookup table
    return lut[g]                           # remap every pixel via the LUT
```

Listing 30.1: Grayscale conversion, histogram, and equalization in NumPy.

30.6 Point operations: one pixel at a time

Brightness, contrast, stretching, and equalization are all examples of a **point operation** (also called a *pixel* or *intensity* transform): a rule that computes each output pixel from *only* the input pixel at the same location,

$$g[m, n] = T\big(f[m, n]\big), \tag{30.6}$$

where T is a single-input function $T : \mathbb{R} \to \mathbb{R}$ – the same T for every location. Because T ignores the neighbors entirely, a point operation can be implemented as a 256-entry **lookup table** (LUT) for 8-bit images: precompute $T(k)$ for $k = 0, \ldots, 255$ once, then replace each pixel by a table read. This is blisteringly fast and is how real-time brightness/contrast sliders work.

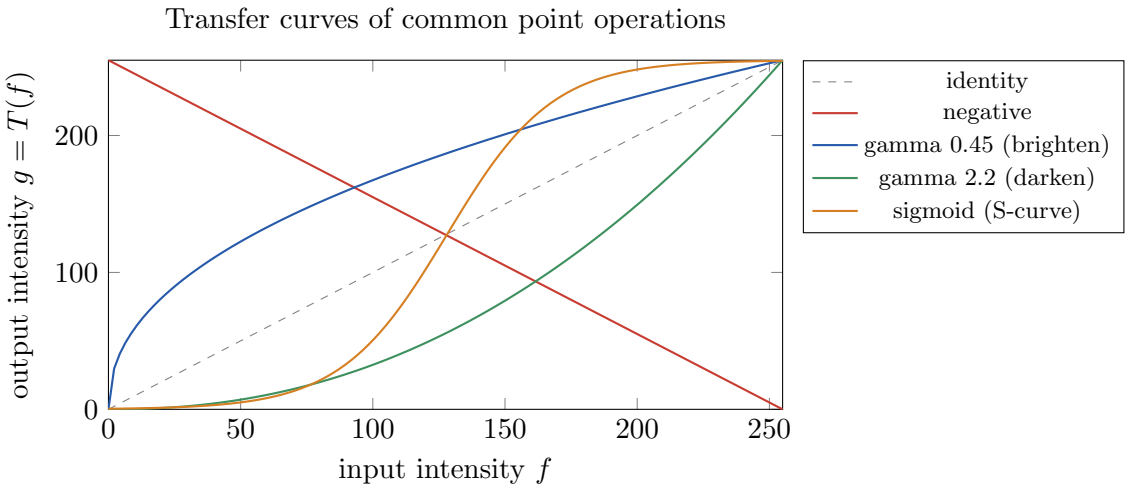


Figure 30.6: A point operation is fully described by its *transfer curve* $g = T(f)$: input intensity on the horizontal axis, output on the vertical. The diagonal is the do-nothing identity. Reflecting it gives the negative; bending it up brightens; bending it down darkens; an S-shape boosts mid-tone contrast.

The transfer curve is the single most useful way to think about point operations. Reading it is a skill: where the curve is *steep*, small input differences become large output differences – contrast is increased for that band of intensities. Where the curve is *flat*, input differences are crushed – contrast lost. A straight diagonal preserves everything. With that lens, every point operation becomes obvious from its shape.

Example 30.1 (The photographic negative)**.** The negative is the point operation $T(f) = 255 - f$ (for 8-bit data). It maps black to white and white to black, reflecting every value about mid-gray.

Its transfer curve is the descending diagonal in Figure 30.6. Because the slope has magnitude 1 everywhere, contrast is exactly preserved – only the sense of light and dark is flipped. This is why a film negative and its print carry the same detail.

30.6.1 Gamma: the most important nonlinear point operation

Not all point operations are linear, and the most consequential one is **gamma correction**:

$$g = 255 \left(\frac{f}{255} \right)^{\gamma}. \tag{30.7}$$

For $\gamma < 1$ the curve bows upward, brightening shadows and compressing highlights; for $\gamma > 1$ it bows downward, darkening. Gamma exists for two deep reasons. The first is that displays are physically nonlinear: the light a CRT or LCD emits is roughly the input value raised to a power near 2.2, so image data is *pre-distorted* by the inverse to come out right. The second reason is the one that makes gamma stick around: human brightness perception is itself roughly logarithmic (Weber-Fechner), so we perceive equal *ratios* of light as equal steps. Encoding images in a gamma-compressed space therefore spends bits where the eye can see differences, packing more perceptual resolution into the dark tones we are sensitive to.

History

Gamma is a fossil of cathode-ray-tube physics. The electron-gun brightness of a CRT followed a power law $L \propto V^{\gamma}$ with $\gamma \approx 2.2$ – not a choice, just how the hardware behaved. Early television engineers, rather than fix it at the display, pre-corrected the signal at the camera with the inverse power. By a happy accident this pre-correction also matched the eye's roughly logarithmic response, so when flat-panel displays arrived the entire imaging pipeline kept the 2.2 gamma on purpose. A hardware quirk became a perceptual standard, and the sRGB color space enshrines a curve very close to it.

Common trap

Do your linear math in linear light. Blurring, resizing, and averaging assume the pixel values add up the way light adds up – but stored values are usually *gamma-encoded*, not linear. Averaging two gamma-encoded pixels gives the wrong brightness (famously, a 50/50 blend of pure red and pure green comes out a muddy dark instead of a bright yellow). For correct results, undo gamma to linear, do the arithmetic, then re-apply gamma. Point operations like brightness are usually fine in encoded space; *neighbor* operations are not.

30.6.2 Thresholding

The most aggressive point operation is the **threshold**, which collapses the whole intensity axis to two values:

$$g[m,n] = \begin{cases} 255 & f[m,n] \geq \theta, \\ 0 & f[m,n] < \theta. \end{cases} \tag{30.8}$$

Its transfer curve is a single step at θ. Thresholding turns a grayscale image into a black-and-white *mask* – the foundation of segmentation, document scanning, and barcode reading. Choosing θ well

is the whole game; a classic automatic choice is *Otsu's method*, which picks the threshold that best separates the histogram into two clusters (minimizing within-class variance), once again treating the histogram as a probability distribution.

Application

Point operations are the front line of medical and scientific imaging. An X-ray or CT slice arrives as 12- or 16-bit data spanning a vast intensity range; no display and no eye can show it all at once. Radiologists apply a *window/level* point operation – an affine map that linearly stretches one narrow band of intensities (say, soft tissue, or bone, or lung) across the full display range and clips the rest. It is brightness-and-contrast (30.3) with clinically chosen a and c, and switching "windows" is how one scan reveals completely different anatomy.

30.7 The jump from 1-D to 2-D, made precise

We have been leaning on the slogan "an image is just a 2-D signal." It is worth spelling out exactly what carries over for free and what is genuinely new, because the next chapters depend on knowing the difference.

What transfers unchanged. Everything that treats the signal pointwise or as a vector: amplitude, sampling and Nyquist, quantization, the histogram, point operations, the inner product, energy ($\sum f^2$), and linearity. A 2-D LTI system (we will call it shift-invariant) still acts by convolution – now a *2-D* convolution that slides a small kernel over the grid. The Fourier transform still exists and still diagonalizes convolution; it just becomes a transform over two frequency variables, u (horizontal) and v (vertical).

What is genuinely new. Direction. On a line there are only two ways to move (left, right) and "frequency" is a single scalar. On a plane you can move in any direction, so spatial frequency becomes a *vector* (u, v): it has a magnitude (how fine the pattern) *and* an orientation (which way the stripes run). A pattern of vertical stripes and a pattern of horizontal stripes have the same fineness but live at different points in 2-D frequency space. Edges, too, acquire orientation. This is why the next chapter's 2-D spectra are pictures, not curves – and why image processing has a richness that 1-D signal processing lacks.

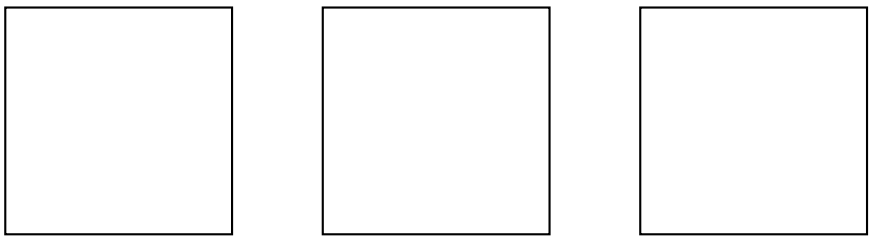


Figure 30.7: Three single-frequency 2-D patterns of identical fineness but different *orientation*. In 1-D these would be the same frequency; in 2-D they are distinct, because spatial frequency is a vector (u, v) carrying a direction. This extra degree of freedom is the heart of what is new in two dimensions.

Intuition

Here is the cleanest way to feel the new dimension. A 2-D sinusoid $\cos\big(2\pi(ux+vy)\big)$ is a corrugated sheet of light and dark stripes. The vector (u, v) points *perpendicular to the stripes*, in the direction the pattern changes fastest; its length $\sqrt{u^2+v^2}$ is how many cycles per unit length you cross moving that way. Rotate (u, v) and the stripes rotate; lengthen it and the stripes get finer. Every image is a sum of these corrugations at all orientations and finenesses – that sum is the 2-D Fourier transform, and it is where we go next.

Toward the wrist tracker

Where do images fit in a *wrist*-worn gesture controller, whose primary sensors are an IMU and (for finger-level intent) EMG or force myography? They earn their place in two ways. The first is as a *development and ground-truth tool*: during prototyping you point a camera at the hand and use computer vision (e.g. a hand-pose estimator) to label what the fingers were actually doing, then train the wrist sensors to predict those labels. The camera never ships on the wrist – it is the teacher, not the product. The second is as a *conceptual bridge*: a spectrogram (the chapter on the STFT) is literally a 2-D signal – time across, frequency up, magnitude as brightness – so every tool here (histograms, contrast, point operations, and soon 2-D filtering) applies directly to reading and cleaning spectrograms of IMU and EMG data. The image chapters quietly upgrade your ability to process the wrist signals themselves.

Focus

Be clear-eyed about the limit. A camera *can* see fingers and recover detailed hand pose, but a camera is not wrist-wearable in any practical, all-day, any-lighting sense. The wrist sensors that *are* wearable – IMU, EMG, FMG – cannot themselves form an image; they give time series, not pixels. So vision is a lab instrument here, not the deliverable. We will return to what the wearable sensors truly can and cannot do in the hand-sensing chapter; the takeaway for now is that "just put a camera on it" is not an answer to wrist-worn finger sensing.

30.8 A worked example

Let us run the numbers on a tiny image so the formulas stop being abstract. Take the 3×3 grayscale patch (8-bit)

$$f = \begin{bmatrix} 30 & 60 & 60 \\ 60 & 90 & 90 \\ 90 & 120 & 150 \end{bmatrix}.$$

There are $MN = 9$ pixels. The histogram counts: one pixel at 30, three at 60, three at 90, one at 120, one at 150. Everything else is zero.

Mean (brightness). $\bar{f} = \frac{1}{9}(30+60+60+60+90+90+90+120+150) = \frac{750}{9} \approx 83.3$.

Min/max stretch. Here $f_{\min} = 30$, $f_{\max} = 150$, so by (30.4) each pixel maps to $g = 255\,(f-30)/120$. For example $60 \mapsto 255\cdot 30/120 = 63.75 \to 64$, and $90 \mapsto 255\cdot 60/120 = 127.5 \to 128$. The darkest pixel becomes 0, the brightest 255: full contrast.

Equalization. Build the normalized histogram and its CDF over the five occupied levels:

k	30	60	90	120	150
$p[k]$	1/9	3/9	3/9	1/9	1/9
$C[k]$	1/9	4/9	7/9	8/9	9/9

Map each level by $k \mapsto \lfloor 255\, C[k] \rfloor$: the five levels become $\lfloor 255/9 \rfloor = 28$, $\lfloor 255{\cdot}4/9 \rfloor = 113$, $\lfloor 255{\cdot}7/9 \rfloor = 198$, $\lfloor 255 \cdot 8/9 \rfloor = 226$, and 255. Notice the crowded middle levels (60 and 90, three pixels each) are pushed far apart – from 30 apart to 85 apart – buying contrast exactly where most pixels live, which is the whole point of equalization.

Walking question

Why did equalization spread 60 and 90 so much further than the stretch did? Because the stretch is *linear* – it spaces output levels evenly with input value – while equalization spaces them by *population*. The jump from 60 to 90 crosses three pixels' worth of probability (3/9), so it earns three-ninths of the output range. Sparse regions of the histogram get little output range; dense regions get a lot. That is precisely "allocate contrast where the pixels are."

Build it (hardware)

On an embedded target (Arduino-class MCU or a Raspberry Pi) point operations are the cheapest image processing you can do, which matters when you are also running sensor DSP. Three notes. (1) Precompute the transfer curve into a `uint8_t lut[256]` once; the per-pixel cost is then a single array read – no float math in the inner loop. (2) Saturate, do not wrap: cast to a wider integer, clamp to $[0, 255]$, then store. (3) If you only need brightness for analysis (not display), skip the RGB-to-gray matrix multiply per pixel and use the cheap integer approximation $Y \approx (77R + 150G + 29B) \gg 8$, which is (30.1) scaled to 256 and bit-shifted – accurate to a level or two and far faster on a chip with no FPU.

```c
#include <stdint.h>
#include <math.h>

static uint8_t lut[256];

void build_gamma_lut(float gamma) {        /* gamma < 1 brightens */
    for (int k = 0; k < 256; k++) {
        float v = 255.0f * powf(k / 255.0f, gamma);
        int   q = (int)(v + 0.5f);         /* round */
        lut[k] = (q < 0) ? 0 : (q > 255 ? 255 : (uint8_t)q);  /* saturate */
    }
}

void apply_lut(uint8_t *img, int n) {      /* n = width*height pixels */
    for (int i = 0; i < n; i++)
        img[i] = lut[img[i]];              /* one table read per pixel */
}
```

Listing 30.2: A point operation as a precomputed LUT on an MCU. No floats in the inner loop.

30.9 Summary

An image is a signal that lives on a plane: a function $f(x, y)$ the world hands us continuously, which a camera samples on a grid and quantizes into an array $f[m, n]$. Each grid value is a pixel – a *sample*, not a square. A grayscale pixel is one intensity; a color pixel is a 3-vector in a color space, and pulling out the luma channel Y (30.1) separates brightness from color the way the eye does. Two-dimensional sampling obeys the same Nyquist law as 1-D, applied per axis, with aliasing showing up as moiré and jaggies. The histogram is the value distribution – a probability distribution over intensity – and its first two moments *are* brightness and contrast. Point operations remap each pixel through a single transfer curve T, implementable as a fast lookup table: brightness shifts, contrast stretches, gamma, negatives, and thresholds are all just curve shapes. The genuinely new ingredient in two dimensions is *direction*: spatial frequency is a vector with an orientation, which is what makes the next chapter's 2-D Fourier transform a picture rather than a plot.

30.10 Exercises

Exercise 30.1. An RGB image is 1280×720 with 8 bits per channel. (a) How many pixels does it have? (b) How many total bytes, uncompressed? (c) Viewed as a single vector, what is its dimension? (d) If you convert to single-channel luma, how many bytes and what dimension?

Exercise 30.2. A pixel has $(R, G, B) = (200, 100, 50)$. (a) Compute its luma Y from (30.1). (b) Compute the fast integer approximation $Y \approx (77R + 150G + 29B) \gg 8$ and compare. (c) Explain in one sentence why green dominates the weights.

Exercise 30.3. You want to map 8-bit intensities so that input 40 goes to output 0 and input 210 goes to output 255, linearly. (a) Find the gain a and offset b in $g = af + b$. (b) What output does input 125 produce? (c) What happens to inputs below 40 or above 210, and what must you do about it?

Exercise 30.4. Consider the 2×2 image $\left[\begin{smallmatrix} 10 & 10 \\ 200 & 200 \end{smallmatrix}\right]$. (a) Write its histogram. (b) Compute the mean and variance of intensity. (c) Is this high or low contrast, and how does the variance tell you?

Exercise 30.5. For the same image as Exercise 30.4, perform histogram equalization using $k \mapsto \lfloor 255\, C[k] \rfloor$. Report the new pixel values. Did contrast increase? Why is equalization nearly useless on a two-level image, and what kind of histogram makes it most effective?

Exercise 30.6. (a) Sketch the transfer curves for $\gamma = 0.5$ and $\gamma = 2.0$ on the same axes as the identity. (b) For an 8-bit input of 64, compute the output under each gamma using (30.7). (c) Which gamma brightens the shadows, and explain how the slope of the curve near $f = 0$ tells you so.

Exercise 30.7. Two pixels are gamma-encoded ($\gamma_{\text{enc}} = 1/2.2$) with values 0 and 255. (a) Naively average the encoded values. (b) Instead, decode each to linear light, average, then re-encode. (c) Compare the two results and explain why the naive average looks too dark.

Exercise 30.8. You photograph black text on white paper. (a) Sketch the expected histogram. (b) Where would you place a threshold θ and why? (c) Describe in words how Otsu's method would choose θ from the histogram automatically, and connect it to the idea of separating a distribution into two clusters.

Exercise 30.9. A camera samples at one pixel per millimetre and photographs a fabric with 0.7 mm-wide alternating stripes (period 1.4 mm). (a) What spatial frequency is the fabric, in cycles per mm? (b) What is the Nyquist limit for this sampling? (c) Will you see aliasing, and what does the moiré pattern's apparent frequency become? (d) What hardware fix do camera makers use?

Exercise 30.10. Write the 2-D sinusoid $\cos\big(2\pi(ux+vy)\big)$ for $(u,v) = (0,4)$, $(4,0)$, and $(3,3)$ cycles per unit. For each, state the stripe orientation, the direction of fastest change, and the fineness $\sqrt{u^2+v^2}$. Which two have the same fineness, and why are all three nonetheless distinct signals?

Exercise 30.11. A spectrogram of forearm-IMU gyro data is being treated as an image (time on the horizontal axis, frequency on the vertical, magnitude as brightness). (a) What would a histogram of this image tell you about the recording? (b) Which point operation would you apply to make faint high-frequency activity visible, and what shape transfer curve does that need? (c) Explain why this is the *same* operation a radiologist's window/level control performs.

Exercise 30.12. A teammate proposes mounting a small camera on the wristband so the gesture controller can see the fingers directly. Give two concrete reasons this is a poor choice for a wearable product, and explain the role a camera *should* play in developing the system (hint: ground truth). Name the sensor modalities that are actually wrist-wearable for finger-level intent.

Chapter 31

The 2D Fourier Transform and Image Filtering

In the last chapter we learned to see an image as a two-dimensional signal: a function $f(x, y)$ that assigns a brightness to every point on a grid. That is a complete picture, but it is a picture in the *spatial* domain – it tells you *where* the bright and dark patches sit. This chapter learns the other half of the story. Every image can also be written as a sum of two-dimensional waves, ripples of brightness that march across the picture in some direction at some spacing. The recipe of those waves is the *2D Fourier transform*, and once you can read it the toolbox opens up: blur and sharpen with fine control, remove periodic interference, detect orientation, compress a photo to a tenth of its size, and see why every camera and every eye is, at bottom, a frequency-limited device.

Everything we built in one dimension carries over. A 1D signal was a sum of sinusoids $e^{\mathrm{j}2\pi ft}$ that differ in *frequency* – how fast they wiggle. A 2D image is a sum of sinusoids $e^{\mathrm{j}2\pi(ux+vy)}$ that differ in two numbers: how fast they wiggle *and* in which *direction*. That extra degree of freedom – direction – is the only genuinely new idea in the whole chapter. Hold onto it.

Focus

By the end you should be able to look at an image and predict roughly where its energy sits in the 2D spectrum (smooth regions near the center, edges and texture out at the rim, repeating patterns as bright dots), design a low-pass or high-pass filter by reaching into that spectrum and reshaping it, and know exactly why a 3×3 blur kernel and a 1×3-then-3×1 pair of kernels can give the identical result for a fraction of the work.

31.1 Two-dimensional waves: the new basis

In one dimension the building block was a phasor $e^{\mathrm{j}2\pi ft}$: a unit vector spinning at frequency f. Its real part, $\cos(2\pi ft)$, is a wave that oscillates as you move along the single time axis.

In two dimensions we have two axes, x and y. The natural building block is

$$b_{u,v}(x, y) \;=\; e^{\mathrm{j}2\pi(ux+vy)} \;=\; \cos\big(2\pi(ux+vy)\big) \;+\; \mathrm{j}\,\sin\big(2\pi(ux+vy)\big). \tag{31.1}$$

Look at the real part, $\cos(2\pi(ux+vy))$. Its value depends only on the combination $ux+vy$. The set of points where $ux+vy$ is constant is a *straight line* – a level set – and as you cross those lines the cosine runs through its cycle. So $b_{u,v}$ is a corrugated sheet: parallel ridges and troughs of brightness, like a sheet of corrugated iron viewed from above.

Two numbers control that sheet.

- The **direction** of the ripples is set by the vector (u, v). The ridges run *perpendicular* to (u, v); the brightness changes fastest *along* (u, v).
- The **spatial frequency** – how tightly the ridges are packed – is the length $\sqrt{u^2+v^2}$. A large length means closely spaced ripples (fine detail); a small length means broad, lazy ripples (smooth shading). The spacing between ridges, the wavelength, is $1/\sqrt{u^2+v^2}$.

Intuition

A 1D frequency was a single number on a line. A 2D spatial frequency is a *vector* (u, v) in a plane. Its *length* is "how fine" and its *angle* is "which way." Everything strange about 2D Fourier – diagonal streaks in the spectrum, the way a vertical fence makes horizontal dots – is just this vector wearing a disguise. When in doubt, draw the arrow (u, v) and remember the ripples run across it.

Figure 31.1 shows three of these gratings. Notice that rotating (u, v) rotates the stripes, and lengthening (u, v) squeezes them together.

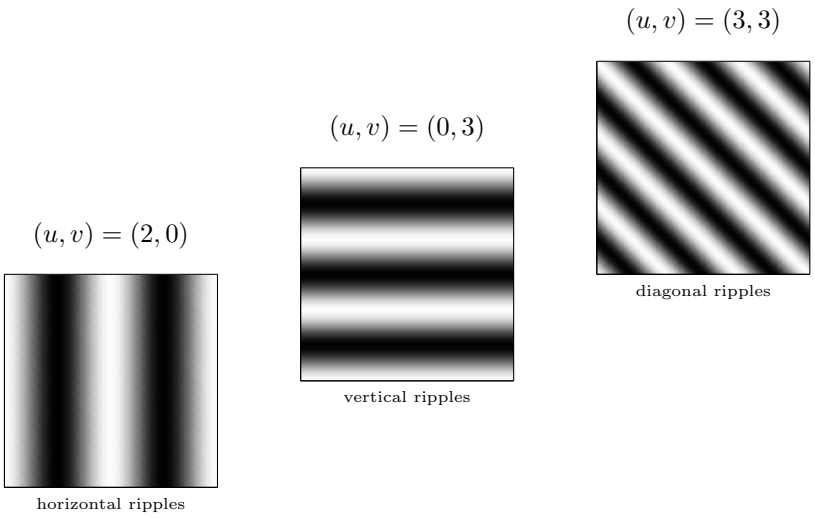


Figure 31.1: Three 2D cosine gratings $\cos(2\pi(ux+vy))$. The frequency vector (u, v) sets the direction (ripples run perpendicular to it) and the fineness (its length). These corrugated sheets are the basis from which every image is built.

Why it works

Why are these the right building blocks and not, say, little blobs or wavelets? Because shifting an image must do something *simple*. The deepest fact of Fourier analysis – in any number of dimensions – is that complex exponentials are the eigenfunctions of shift: slide $b_{u,v}$ by

(a, b) and you get the very same sheet back, multiplied by the constant phase $e^{\mathrm{j}2\pi(ua+vb)}$. Nothing else has that property. Because real-world imaging operations (lenses blurring, sensors averaging, translations) are shift-invariant, the gratings are the natural coordinates in which those operations become simple multiplications. We saw this in 1D for LTI systems; in 2D it is identical, just with two shift directions.

31.2 The 2D DFT

A digital image is sampled: an $M \times N$ grid of pixels $f[m, n]$, with row index $m = 0, \ldots, M-1$ and column index $n = 0, \ldots, N-1$. The two-dimensional discrete Fourier transform is the discrete cousin of the projection in Eq. (31.1):

$$F[k, \ell] \;=\; \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n]\, e^{-\mathrm{j}2\pi\left(\frac{km}{M} + \frac{\ell n}{N}\right)}, \tag{31.2}$$

with the inverse

$$f[m, n] \;=\; \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{\ell=0}^{N-1} F[k, \ell]\, e^{+\mathrm{j}2\pi\left(\frac{km}{M} + \frac{\ell n}{N}\right)}. \tag{31.3}$$

Here k indexes vertical frequency (along m, the rows) and ℓ indexes horizontal frequency (along n, the columns). Each output $F[k, \ell]$ is a complex number: a magnitude $|F[k, \ell]|$ saying how much of that grating is present, and a phase $\angle F[k, \ell]$ saying where its ridges sit.

What is really happening

$F[k, \ell]$ is the inner product of the image with the grating $b_{k,\ell}$. We are asking, for each direction and fineness, "how much does the image look like *this* corrugated sheet?" The whole transform is MN such questions, one per output pixel. The answer to each is a single complex number – a length and an angle – and the inverse transform just adds all those gratings back up, weighted by those numbers, to rebuild the picture exactly. Nothing is lost: the spectrum is the image, in different clothes.

31.2.1 The transform separates into rows then columns

Equation (31.2) looks like a double sum of size MN per output, which would be $O(M^2N^2)$ operations – ruinous. But the exponential *factors*:

$$e^{-\mathrm{j}2\pi\left(\frac{km}{M} + \frac{\ell n}{N}\right)} = e^{-\mathrm{j}2\pi \frac{km}{M}}\; e^{-\mathrm{j}2\pi \frac{\ell n}{N}}.$$

So we can split the work:

$$F[k, \ell] = \sum_{m=0}^{M-1} \underbrace{\left(\sum_{n=0}^{N-1} f[m, n]\, e^{-\mathrm{j}2\pi \frac{\ell n}{N}} \right)}_{\text{1D DFT of row } m} e^{-\mathrm{j}2\pi \frac{km}{M}}. \tag{31.4}$$

First take the ordinary 1D DFT of every *row*; then take the 1D DFT of every *column* of the result. (The order does not matter.) Each 1D DFT can use the FFT from Chapter 9, so the 2D DFT of an $N \times N$ image costs $O(N^2 \log N)$, not $O(N^4)$. This row–column trick is the workhorse behind every image-processing library.

Why it works

Why does "rows then columns" give the same answer as the full 2D sum? Because the 2D basis is a *product* of a row-basis and a column-basis. The 2D DFT is a *separable* transform: it is the 1D DFT applied along one axis composed with the 1D DFT applied along the other. Separability is not a numerical accident; it is the algebraic shadow of the fact that the plane is a product of two lines.

31.3 Reading a 2D spectrum

We almost always look at the *magnitude* spectrum $|F[k, \ell]|$, displayed as an image. Three conventions make it legible.

1. **Shift the DC term to the center.** As in 1D, the DFT's frequencies wrap around: index 0 is zero frequency, and the highest frequencies sit near the middle of the index range, with negatives folded to the top end. We `fftshift` so that $(0, 0)$ – the average brightness, the "DC" term – lands at the *center* of the displayed image. Then low frequencies cluster at the center and high frequencies spread to the corners.

2. **Take a log.** Spectral magnitudes span many orders of magnitude; the DC term alone dwarfs everything. We display $\log(1 + |F|)$ so the faint high-frequency structure becomes visible.

3. **Ignore the phase at first.** The magnitude tells you *which* gratings are present and how strongly; the phase tells you *where* they sit. For building intuition the magnitude is enough – but never forget the phase is doing the heavy lifting (see the box below).

Now the payoff – a vocabulary for reading the picture, summarized in Figure 31.2:

- **Center bright blob** = lots of low spatial frequency = the image is mostly smooth, slowly varying shading. Almost every natural photo has this; energy concentrates near DC.

- **Energy out at the rim** = high spatial frequency = sharp edges, fine texture, noise. An image of crisp text or gravel pushes a lot of energy outward.

- **A bright streak through the center at angle θ** = strong edges or stripes oriented *perpendicular* to θ. A picture full of vertical fence posts produces a bright *horizontal* line in the spectrum, because the ripples that build a vertical edge run horizontally.

- **Isolated bright dots off-center** = a periodic pattern in the image (a brick wall, a halftone screen, scan-line interference). The dot's position is exactly the frequency vector of that pattern. This is the key to removing periodic noise: find the dots, zero them, transform back.

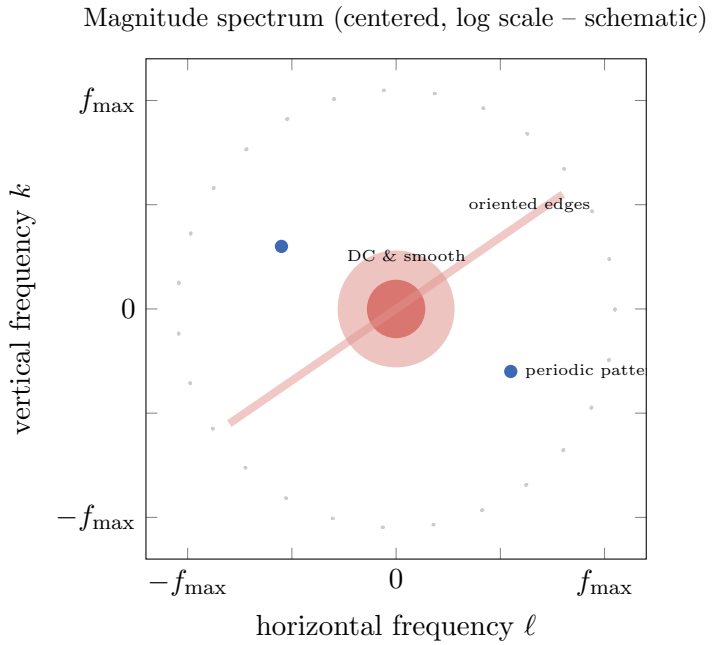


Figure 31.2: Schematic of a centered, log-scaled 2D magnitude spectrum. Smooth content piles up at the center; oriented edges make a streak *perpendicular* to their own direction; a repeating texture shows up as a symmetric pair of dots at its frequency vector. The faint ring marks the highest representable frequencies (the Nyquist rim).

Common trap

The spectrum is *point-symmetric*: $F[-k,-\ell] = \overline{F[k,\ell]}$ for a real image. That is why interference always shows up as a *pair* of dots placed symmetrically about the center, never a lone dot. If you zero one dot to kill a pattern you must zero its mirror twin too, or the inverse transform comes back complex and your "image" is nonsense.

Why it works

Phase carries the structure. A famous demonstration: take two photos, swap their phase spectra while keeping their magnitudes, and invert. The result looks like the *phase* donor, not the magnitude donor. Magnitude says *how much* of each grating; phase says *where each grating's ridges line up*, and it is that alignment that places edges and forms recognizable shapes. When we filter, we usually scale magnitudes and leave phase alone – precisely so we do not scramble *where* things are while changing *how sharp* they are.

31.4 2D convolution

Filtering an image in the spatial domain means *2D convolution* with a small array of numbers called a *kernel* (or mask, or point-spread function). For a kernel $h[i,j]$,

$$g[m,n] \;=\; (f * h)[m,n] \;=\; \sum_{i}\sum_{j} h[i,j]\, f[m-i,\, n-j]. \tag{31.5}$$

Geometrically this is exactly the 1D sliding-and-summing picture from Chapter 13, now on a grid: flip the kernel in both axes, slide it to position (m, n), multiply it pointwise onto the patch of image underneath, and sum. The output pixel is a weighted average of a neighborhood. Figure 31.3 shows one such step.

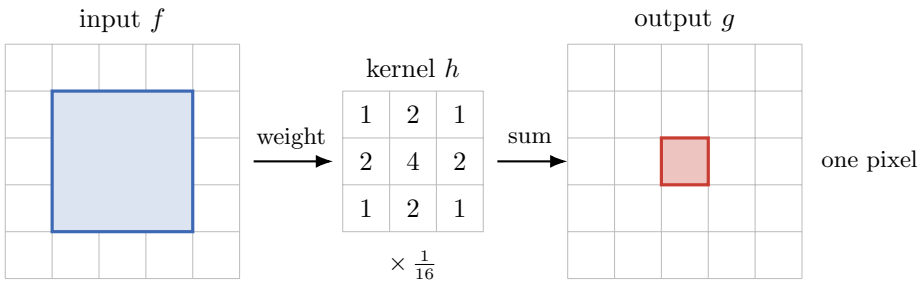


Figure 31.3: One step of 2D convolution. The 3×3 kernel is laid over a patch of the input, multiplied pointwise, and the products are summed into a single output pixel. Sliding the kernel over every position fills the whole output.

Walking question

Why "flip"? Convolution flips the kernel; *correlation* does not. For a *symmetric* kernel (a Gaussian blur, a box average) the flip changes nothing, so people speak loosely. But for an asymmetric kernel – a directional edge detector, a motion blur – the flip matters, and forgetting it mirrors your result. Many image libraries' "`convolve`" functions actually compute correlation; check the docs before you trust the orientation.

The same kernel makes very different things happen depending on its numbers. Here are the three archetypes we will build:

- **Weights all positive, summing to 1** $\Rightarrow$ a *low-pass* filter: a weighted local average that smooths and blurs.
- **Weights summing to 0, with a positive center and negative surround** $\Rightarrow$ a *high-pass* filter: it responds to change and ignores flat regions, so it finds edges.
- **Center positive and large, surround slightly negative, summing to 1** $\Rightarrow$ a *sharpening* filter: the original plus a dash of its own high-pass.

31.4.1 The convolution theorem, in 2D

The single most useful fact about filtering survives intact:

$$f * h \;\xleftrightarrow{\mathcal{F}}\; F \cdot H. \tag{31.6}$$

Convolving two images in space is the same as *multiplying* their spectra point by point. So a spatial kernel h has a frequency-domain alter ego $H[k, \ell]$ – its *frequency response* – and the kernel "passes" the frequencies where $|H|$ is near 1 and "stops" those where $|H|$ is near 0. This is the bridge between the two halves of the chapter: every blur kernel *is* a low-pass response in disguise, and every edge kernel *is* a high-pass response. To understand what a kernel does, look at the magnitude of its 2D DFT.

Connection

This is the identical theorem you proved for 1D signals in Chapter 11, applied twice – once per axis – because the 2D transform is separable. The inner product, orthogonality, and eigenfunction machinery from the linear algebra companion did all the real work; 2D just runs it on a product space. If the tensor-product language is unfamiliar, the takeaway is simply: *the plane is two lines, so 2D Fourier is 1D Fourier done twice.*

31.5 Low-pass filtering: blurring

A low-pass filter keeps the smooth, slowly varying content and discards the fine detail. In the spatial domain it is a weighted local average.

31.5.1 The box blur, and why it is mediocre

The simplest is the *box* (mean) filter: a $K \times K$ kernel of equal weights $1/K^2$. It is fast and it does smooth, but its frequency response is a 2D sinc – it has ripples and even *negative* lobes, meaning some mid-frequencies come back *inverted*. Visually this shows up as faint "ringing" and a boxy, directional character. We can do better.

31.5.2 The Gaussian blur

The well-behaved low-pass is the *Gaussian* kernel,

$$h_\sigma(i, j) \;=\; \frac{1}{2\pi\sigma^2}\,\exp\!\left(-\frac{i^2 + j^2}{2\sigma^2}\right), \tag{31.7}$$

normalized so its weights sum to 1. Its parameter σ (in pixels) sets the blur radius: bigger σ, smoother result.

Three facts make the Gaussian the default blur everywhere.

1. **Its Fourier transform is again a Gaussian** – it keeps its shape under the transform. So the frequency response is a smooth bump $H(u, v) = \exp(-2\pi^2\sigma^2(u^2 + v^2))$ centered at DC: a clean low-pass with *no* ripples, no negative lobes, no ringing. A wide Gaussian in space is a *narrow* bump in frequency (cuts more detail), and vice versa – the 2D version of the uncertainty trade-off from Chapter 7.
2. **It is isotropic** – circularly symmetric – so it blurs equally in all directions and never imprints a directional bias on the image.

3. **It is separable** (next section), so it is cheap.

Figure 31.4 shows the Gaussian's 1D profile and its frequency response side by side: smooth bump in, smooth bump out.

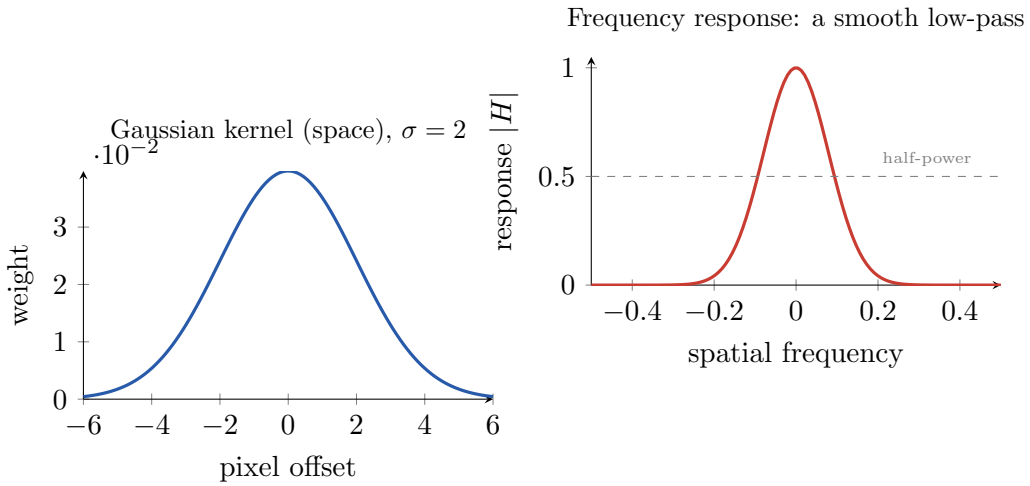


Figure 31.4: The Gaussian blur in both domains (1D slices shown). A smooth bump in space transforms to a smooth bump at the origin in frequency: a low-pass with no ringing. Wider in space ⇔ narrower in frequency ⇔ more aggressive smoothing.

Example 31.1 (A 3×3 Gaussian by hand)**.** A common integer approximation to a small Gaussian is

$$h = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

The weights sum to 16, so dividing by 16 makes them sum to 1 (preserving overall brightness). The center pixel counts four times as much as the corners, which fall off with distance just like a Gaussian. Apply it to a flat region and nothing changes ($16/16 = 1$); apply it across an edge and the edge softens. Notice the outer-product structure: this matrix is $\frac{1}{4}[1,2,1]^\top \, \frac{1}{4}[1,2,1]$. That is the seed of separability.

31.6 Separable filters: the same result, far cheaper

A 2D kernel $h[i,j]$ is *separable* if it is the outer product of a column vector and a row vector:

$$h[i,j] \;=\; c[i]\, r[j]. \tag{31.8}$$

When that holds, the 2D convolution splits into two 1D convolutions:

$$f * h \;=\; \big(f *_{\text{cols}} c\big) *_{\text{rows}} r, \tag{31.9}$$

that is, convolve every column with c, then convolve every row of the result with r (or vice versa); Figure 31.5 shows the two-pass pipeline.

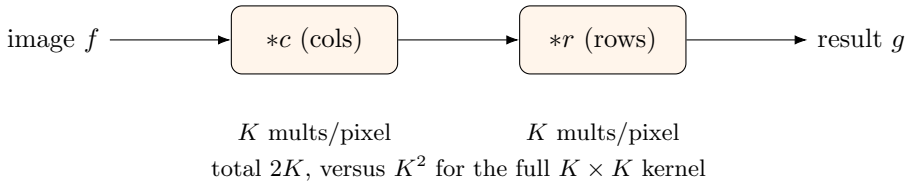


Figure 31.5: A separable filter runs as two cheap 1D passes: convolve down the columns with c, then across the rows with r. The cost drops from K^2 to $2K$ multiply–adds per pixel.

The Gaussian is separable because $e^{-(i^2+j^2)/2\sigma^2} = e^{-i^2/2\sigma^2}\, e^{-j^2/2\sigma^2}$; the $\frac{1}{16}[1,2,1] \otimes [1,2,1]$ kernel of Example 31.1 is separable by construction.

Why it works

Why is this such a big deal? A $K \times K$ kernel costs K^2 multiply–adds per output pixel. Doing it as two 1D passes costs $K + K = 2K$. For a modest $K = 15$ Gaussian that is 225 versus 30 operations per pixel – a 7.5× speedup, and the gap grows linearly with K. On a multi-megapixel image, or on a microcontroller, that is the difference between real time and not. The Gaussian's separability is *the* reason it is the default blur in every camera pipeline.

Try it yourself

Test separability yourself: a 2D kernel is separable exactly when its matrix has *rank 1*. Compute the singular value decomposition (see the linear-algebra companion); if only the first singular value is nonzero, the kernel factors as $c\,r^\top$ with c, r the leading singular vectors. Many "2D" kernels people invent are secretly separable, and the SVD will hand you the two 1D passes for free. Even a non-separable kernel can be *approximated* by the sum of a few rank-1 terms (its top few singular values) – a sum of separable passes.

```python
import numpy as np

def gaussian_1d(sigma, radius):
    x = np.arange(-radius, radius + 1)
    k = np.exp(-x**2 / (2 * sigma**2))
    return k / k.sum()                          # normalize to unit sum

def blur_separable(img, sigma):
    r = int(3 * sigma)                          # 3-sigma support is plenty
    k = gaussian_1d(sigma, r)
    # convolve along columns (axis 0), then rows (axis 1)
    tmp = np.apply_along_axis(lambda v: np.convolve(v, k, mode='same'), 0, img)
    out = np.apply_along_axis(lambda v: np.convolve(v, k, mode='same'), 1, tmp)
    return out
# Cost per pixel: 2*(2r+1) mults, vs (2r+1)^2 for the full 2D kernel.
```

Listing 31.1: Separable Gaussian blur in NumPy: two 1D passes.

31.7 High-pass filtering: edges and sharpening

A high-pass filter is the complement of a low-pass: it keeps the rapid changes and discards the smooth shading. The simplest way to build one is by *subtraction*:

$$\text{high-pass} \;=\; \text{original} \;-\; \text{low-pass}. \tag{31.10}$$

Whatever the blur threw away (the detail) is exactly what the high-pass keeps. In the frequency domain this reads $H_{\text{hp}} = 1 - H_{\text{lp}}$: a bump that is zero at DC and rises to 1 at high frequencies – the mirror image of Figure 31.4.

31.7.1 The Laplacian: a second-derivative edge finder

The classic high-pass kernel approximates the *Laplacian* $\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$, the sum of second derivatives. A discrete second derivative along one axis is $f[n-1] - 2f[n] + f[n+1]$, i.e. the stencil $[1, -2, 1]$. Summing the two axes:

$$h_{\nabla^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Its weights sum to 0, the signature of a high-pass: on a flat or smoothly graded region it outputs (near) zero, and it lights up only where brightness bends – at edges and corners. (We will study directional edge detectors – Sobel, Prewitt – in the next chapter; the Laplacian is the isotropic, undirected member of the family.)

31.7.2 Unsharp masking: sharpening

To *sharpen* rather than merely detect edges, add a scaled high-pass back onto the original. This is *unsharp masking*, and it is what every "sharpen" slider does:

$$g \;=\; f \;+\; \alpha\,(f - f_{\text{blur}}) \;=\; (1+\alpha)\,f \;-\; \alpha\, f_{\text{blur}}, \tag{31.11}$$

where f_{blur} is a Gaussian-blurred copy and $\alpha > 0$ controls the strength. The bracketed difference $f - f_{\text{blur}}$ is the high-pass detail; adding it back exaggerates edges. As a single 3×3 kernel (with a simple neighbor blur), sharpening becomes

$$h_{\text{sharp}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \qquad \text{sum} = 1,$$

which preserves overall brightness (sum 1) while boosting the center against its neighbors. Indeed $h_{\text{sharp}} = \delta - h_{\nabla^2}$: the identity (a single 1 at the center) minus the Laplacian, the original minus a high-pass-with- the-sign-flipped – which is the same as original plus high-pass.

Common trap

Sharpening amplifies high frequencies – and *noise lives at high frequencies*. Crank α too far and you do not get a crisper photo, you get a gritty, halo-ringed mess: edges sprout bright/dark fringes (overshoot) and grain explodes. The professional recipe is *blur a little first* (to tame noise), then sharpen gently. There is no free lunch: you cannot add detail that the optics never captured, only redistribute contrast around the edges that are already there.

Application

Where these filters earn their keep. *Gaussian low-pass:* the anti-alias / denoise stage before downsampling a photo, and the first step of nearly every feature detector (blur, then look for structure). *High-pass / unsharp:* the "clarity" and "sharpen" controls in every photo editor, and the edge-enhancement in medical and satellite imagery. *Band-pass* (a low-pass minus a smaller low-pass, the "difference of Gaussians"): a remarkably good model of the center–surround receptive fields in your own retina, and the basis of blob detection in computer vision.

31.8 Filtering in the frequency domain

For a small kernel, spatial convolution is fine. But when the filter is large – a heavy blur, or a custom shape like "remove this exact stripe" – it is far easier and often faster to work directly in the spectrum, using the convolution theorem in reverse. The recipe:

1. Compute $F = \mathrm{DFT2}(f)$.
2. Build a *mask* $H[k, \ell]$ in the frequency plane – 1 where you want to keep frequencies, 0 (or a smooth taper) where you want to remove them.
3. Multiply: $G[k, \ell] = F[k, \ell]\, H[k, \ell]$.
4. Invert: $g = \mathrm{IDFT2}(G)$, and take the real part.

An *ideal low-pass* keeps a disk of radius D_0 around the center and zeroes everything outside; an ideal high-pass does the opposite; a *band-pass* keeps an annulus; a *notch* filter zeroes a small spot (and its mirror) to kill one periodic pattern. Figure 31.6 shows the radial profiles.

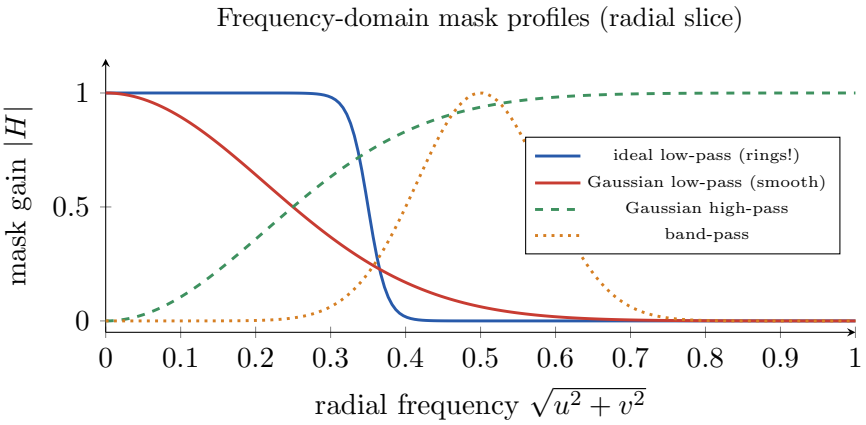


Figure 31.6: Radial profiles of frequency-domain masks. The ideal (hard-edged) low-pass has a cliff and causes ringing; the Gaussian-tapered low-pass rolls off smoothly and does not. High-pass is one-minus-low-pass; band-pass keeps an annulus of frequencies.

Common trap

Sharp masks ring. A perfectly hard-edged disk in the frequency domain is a sinc-like (Airy) ripple in the spatial domain. Multiply by a hard disk and your filtered image breaks out in concentric ripples around every edge – the 2D *Gibbs phenomenon*, the same ringing we met with truncated Fourier series in Chapter 6. The cure is the same: *taper* the mask. Use a smooth roll-off (a Gaussian or Butterworth profile) instead of a cliff, and the ringing vanishes. The Gaussian blur of Section 31.5 is exactly such a tapered low-pass, which is why it never rings.

Common trap

Circular wrap-around. The DFT treats the image as *periodic* – it tiles to infinity. So frequency-domain filtering convolves *circularly*: content can smear from the right edge onto the left, and a bright sky at the top can leak shadow onto the bottom. The fix is the same as in 1D: *zero-pad* the image before transforming (and/or window the borders) so the wrap-around region is harmless. Forgetting this produces the classic "why is there a glow on the opposite edge?" bug.

Example 31.2 (Removing scan-line interference with a notch)**.** Suppose an old scanned photo has a faint horizontal banding at a fixed period – the remains of a scanner's mechanical hum. In space it is nearly impossible to fix by averaging without blurring the whole image. In frequency it is a gift: a single period in space is a single frequency in the spectrum, so horizontal banding appears as a *bright pair of dots* on the vertical-frequency axis, symmetric about the center. Place a small smooth *notch* (a little Gaussian dip) at each dot, multiply, invert – and the banding is gone while the rest of the picture is untouched. No spatial kernel can do this so cleanly, because the pattern is global and periodic, exactly the thing the Fourier basis was built to name.

Why this matters

This is the deep reason the Fourier domain is worth the trip. Some structure is *local* (an edge, a speck of dust) and is easiest to touch in space. Other structure is *global and periodic* (banding, halftone screens, moiré, the hum of the world) and is a tangled mess in space but a few isolated dots in frequency. Choosing the domain is choosing the coordinates in which your problem is simple. A master of signal processing is, above all, fluent at switching between the two.

History

The fast algorithm that makes all of this practical is the Cooley–Tukey FFT (1965), and the row–column factorization of the 2D DFT was understood immediately: a 2D transform is just many 1D transforms. The push for cheap 2D spectra came partly from image transmission and reconnaissance, and within two decades the discrete cosine transform – a close relative of the DFT, real-valued and well-suited to the energy-compaction of natural images – became the engine of JPEG (1992). The lineage runs straight from Fourier's heat-flow series of 1822 to the photo on your phone.

31.9 A worked numeric example

Let us run one 4×4 image through a tiny low-pass by hand, just to make the machinery concrete. Take a single bright pixel on a dark field (a discrete 2D impulse):

$$f = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \qquad h = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

Convolving an impulse with a kernel simply *copies the kernel* to the impulse's location – that is the definition of the impulse response. So the output around the bright pixel becomes

$$g = \frac{1}{16}\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 2 & 4 & 2 \\ 0 & 1 & 2 & 1 \end{bmatrix},$$

i.e. the energy of the single pixel spreads into a little Gaussian hill: blurred, exactly as promised. And because the kernel summed to 1, the total brightness is preserved: $\frac{1}{16}(1+2+1+2+4+2+1+2+1) = \frac{16}{16} = 1$, the same total as the input. This impulse-response view – "a filter is what it does to a single point" – is the 2D version of the point-spread function and the heart of how lenses, telescopes, and your own eye are characterized.

Connection

That total-brightness check is conservation of the DC term: a unit-sum kernel has frequency response $H[0,0] = 1$, so it leaves the average untouched. A zero-sum kernel has $H[0,0] = 0$ and kills the average – which is why high-pass outputs hover around mid-gray. You can read a kernel's effect on overall brightness off a single number: the sum of its weights, which *is* its

DC response. (Recall the DC term is the $k = \ell = 0$ output of Eq. (31.2), a plain sum of all weights.)

31.10 Toward the wrist tracker

Toward the wrist tracker

Where does *image* processing touch a wrist-worn instrument controller? Two real places, and one warning.

1. The math transfers directly to your 1D sensor streams. A 2D Gaussian blur is two 1D Gaussian blurs; the 1D Gaussian smoother is precisely the gentle, ring-free low-pass you will run on noisy accelerometer and gyroscope traces before gesture detection (Chapters 34–38). Separability is why you will always prefer a short separable smoother to a fat 2D one. And the "blur-then-sharpen" discipline – denoise before you amplify detail – is exactly the right instinct for sensor data, where naive differentiation (a high-pass) screams with noise.

2. The spectrogram is a 2D image. When you take short-time Fourier transforms of an EMG or audio stream (Chapter 10) you get a time–frequency *image*, and the 2D filtering of this chapter applies to it: a 2D Gaussian smooths a jittery spectrogram, a notch removes a steady 50/60 Hz mains line, and 2D edge detection finds onsets (sharp vertical edges = sudden events). Treating the spectrogram as an image is a productive move for gesture onset detection.

The warning. A camera could, in principle, track a hand by image processing – and indeed optical hand-tracking is a real, mature technology. But our controller is *worn*, with no camera looking at the hand, so we cannot lean on image processing to recover finger pose. A wrist IMU sees only the gross motion of the forearm; it cannot image the fingers. Continuous finger pose needs sensors *on* the fingers or muscle sensing (EMG/FMG) at the forearm – a limit we will keep stating plainly. The lesson to carry forward from this chapter is the *mathematics* (2D = 1D twice, convolution = multiplication in frequency, taper to avoid ringing), not a fantasy that an arm-mounted camera-free device sees your hand.

Build it (hardware)

On a Raspberry Pi with a camera you can watch all of this happen live. OpenCV's `cv2.GaussianBlur` is separable under the hood; `cv2.Laplacian` gives you the high-pass edge map; and `numpy.fft.fft2` / `fftshift` let you display the live magnitude spectrum of the camera feed. Point the camera at a brick wall or a striped shirt and watch the symmetric dots appear – it is the fastest way to make "spatial frequency" stop being abstract. On a bare microcontroller (no camera), keep only the 1D separable smoother: it is a handful of multiply–adds per sample and runs comfortably inside a sampling interrupt.

31.11 Summary

- A 2D spatial frequency is a *vector* (u, v): its length is fineness, its angle is orientation. Images are sums of corrugated-sheet gratings $e^{j2\pi(ux+vy)}$, one per (u, v).

- The 2D DFT projects an image onto those gratings. It is *separable*: 1D DFTs of the rows, then of the columns – which is why it is fast.
- In a centered, log-scaled magnitude spectrum: smooth content sits at the center, edges and texture at the rim, oriented edges as a perpendicular streak, periodic patterns as symmetric dot pairs.
- Filtering is 2D convolution with a kernel; by the convolution theorem it is pointwise multiplication of spectra. Unit-sum kernels are low-pass (blur); zero-sum kernels are high-pass (edges); the Gaussian is the ring-free, isotropic, separable low-pass of choice.
- Frequency-domain filtering (build a mask, multiply, invert) shines for large or global/periodic operations – but *taper* the mask to avoid ringing and *zero-pad* to avoid circular wrap-around.

Exercises

Exercise 31.1. A grating has frequency vector $(u, v) = (0, 4)$ cycles per image width. In which direction do its ridges run, and how many ripples appear across the image? Now rotate the vector to $(4, 0)$: describe the new pattern. What does $(3, 4)$ look like, and what is its wavelength relative to $(0, 4)$?

Exercise 31.2. You photograph a picket fence – many tall vertical slats. Sketch where the energy concentrates in the centered magnitude spectrum, and explain (using the "ridges perpendicular to (u, v)" rule) why a *vertical* pattern produces a *horizontal* feature in the spectrum.

Exercise 31.3. Determine whether each kernel is separable; if so, give the two 1D factors. (a) $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$, (b) $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$ (the Sobel-x), (c) $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ (the Laplacian). *Hint: a kernel is separable iff its matrix has rank 1.*

Exercise 31.4. A 21×21 Gaussian blur is applied to a 4000×3000 image. Count the multiply–adds per output pixel (i) for the full 2D kernel and (ii) for the separable two-pass version. What is the speedup factor, and how does it scale with kernel size K?

Exercise 31.5. Show that a convolution kernel whose weights sum to S has frequency response $H[0, 0] = S$. Conclude that unit-sum kernels preserve average brightness and zero-sum kernels remove it. Use this to predict the mean gray value of a Laplacian-filtered image of a uniform gray card.

Exercise 31.6. Convolve the 5×5 impulse (a single 1 at the center, zeros elsewhere) with the sharpening kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$. Write out the resulting 5×5 array. Verify the weights still sum to 1 and explain, in terms of $H[0, 0]$, why brightness is preserved even though the kernel has negative entries.

Exercise 31.7. Explain in your own words why multiplying a spectrum by a hard-edged circular low-pass mask produces ringing in the filtered image, and why a Gaussian-tapered mask does not. Which 1D phenomenon from the Fourier-series chapter is this the 2D version of?

Exercise 31.8. An image is corrupted by a single diagonal sinusoidal interference pattern. (a) Describe its appearance in the centered magnitude spectrum. (b) Why must you zero *two* symmetric locations rather than one to remove it? (c) Why should the notch be a smooth dip rather than a hard hole?

Exercise 31.9. Starting from $g = f + \alpha(f - f_{\text{blur}})$ with $\alpha = 1$ and the four-neighbor average $f_{\text{blur}} = \frac{1}{4}\sum_{\text{N,S,E,W}}$ for the blur, derive the equivalent single 3×3 convolution kernel. Confirm its weights sum to 1. What goes wrong as $\alpha \to \infty$?

Exercise 31.10. A spectrogram (time on one axis, frequency on the other) of an EMG signal shows a horizontal bright line at 60 Hz across all time (mains hum). Treating the spectrogram as a 2D image, describe a frequency-domain (2D-DFT) operation that removes the line, and contrast it with simply notching 60 Hz in the original 1D signal. Which is cleaner, and why?

Exercise 31.11. The "difference of Gaussians" is $h = h_{\sigma_1} - h_{\sigma_2}$ with $\sigma_1 < \sigma_2$. (a) Sketch its frequency response and argue it is a band-pass. (b) Explain why this models the center–surround behavior of retinal cells. (c) Is a difference of two separable kernels itself separable? If not, how would you still compute it efficiently?

Chapter 32

Edges, Features, and the 2D Convolution

In the last two chapters we learned to read an image as a two-dimensional signal — a grid of brightness values $f[m, n]$ — and we learned what its 2D Fourier transform says about it: low frequencies are the broad washes of tone, high frequencies are the fine detail, the texture, and above all the *edges*. This chapter is about going after those edges directly, in the spatial domain, with a single tool: the **2D convolution**. We will see that an astonishing amount of computer vision — finding boundaries, detecting corners, blurring, sharpening, spotting blobs — is just convolution with a cleverly chosen little grid of numbers called a *kernel*.

And then we get to the punchline that makes this chapter matter far beyond images. The 2D convolution is the atomic operation of the convolutional neural network (CNN), the architecture behind essentially all modern image recognition. The difference between a 1980s edge detector and a 2020s vision model is not the operation — both convolve an image with small kernels. The difference is *where the kernels come from*: a human designed the old ones from geometry; gradient descent *learns* the new ones from data. By the end of this chapter you will see those two worlds as one.

Toward the wrist tracker

Our wrist tracker is about motion, not images — so why study image edges? Three reasons. First, the 2D convolution is the same flip-shift-multiply-sum machine from Chapter 13, now run over a grid, and seeing it twice cements it. Second, when we turn a window of IMU or EMG samples into a small 2D "time × channel" array (Chapter 38), the very same convolutional layers that read images will read our gesture data. Third, the idea of a *learned filter bank* replacing hand-designed features is exactly how modern gesture recognizers work. This chapter is where the vision world and the gesture world quietly merge.

32.1 An edge is a place where brightness changes fast

Look at any photograph and ask where the "edges" are. Your eye lands on the boundary between an object and its background, between a shadow and the lit floor, between the dark trunk of a tree and the bright sky. In every case the defining feature is the same: as you walk across that boundary, the

brightness *changes sharply over a short distance*. Inside a region of uniform tone, brightness barely changes; at an edge, it jumps.

That single observation hands us the whole strategy. "Changes fast" is the language of the *derivative*. In one dimension, a function changes fast where its derivative is large in magnitude. An image is a function of two variables, so the right object is the **gradient** — the pair of partial derivatives,

$$\nabla f(x, y) \;=\; \left(\frac{\partial f}{\partial x},\; \frac{\partial f}{\partial y} \right). \tag{32.1}$$

The gradient is a vector. Its magnitude $\|\nabla f\|$ tells you *how fast* brightness is changing at that point, and its direction tells you *which way* the brightness increases most steeply — which is always perpendicular to the edge itself.

Intuition

Think of the image as a landscape: bright pixels are hilltops, dark pixels are valleys. The gradient at a point is the direction a marble would roll if you flipped the landscape upside down — straight uphill — and its magnitude is how steep that slope is. A flat plateau (uniform region) has zero gradient. A cliff (an edge) has a huge gradient pointing over the edge. To find edges, find the cliffs: find where $\|\nabla f\|$ is large.

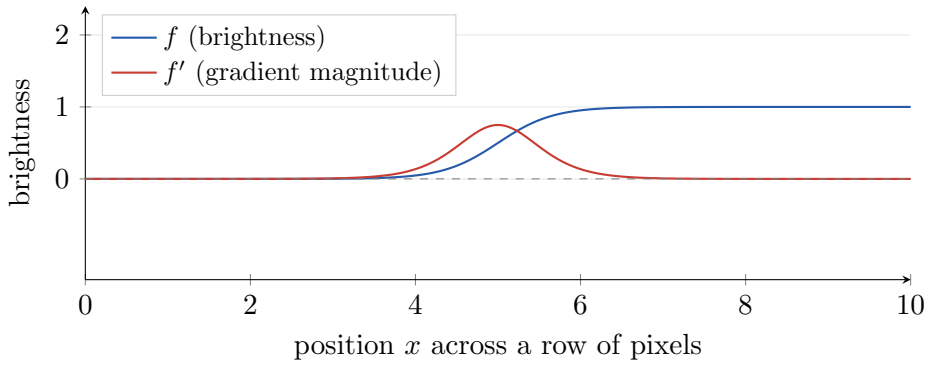


Figure 32.1: One row of pixels crossing an edge. The brightness f rises from dark to light over a short stretch (a smoothed step). Its derivative f' is near zero on the flat parts and *peaks exactly at the edge*. Detecting edges means finding the peaks of the derivative.

32.1.1 Discrete derivatives: differences between neighbors

Our image is sampled — a grid of integers, not a smooth surface — so we cannot take a true derivative. We approximate it the way Chapter 30 taught us: a derivative is a limit of differences, and on a grid we simply keep the difference. The cleanest estimate of $\partial f/\partial x$ at pixel $[m, n]$ is the **central difference**,

$$\left.\frac{\partial f}{\partial x}\right|_{[m,n]} \;\approx\; \frac{f[m, n+1] - f[m, n-1]}{2}, \tag{32.2}$$

the brightness of the pixel to the right minus the pixel to the left (the 2 in the denominator accounts for the two-pixel span; in practice it is often dropped because we only care about *relative* edge strength). The vertical derivative is the same idea down a column.

Now here is the move that ties this whole chapter together. Equation (32.2) can be written as a tiny convolution. Lay the coefficients $\frac{1}{2}(+1,\ 0,\ -1)$ over the three pixels $(n+1, n, n-1)$, multiply, and sum. That little row of numbers is a *kernel*. Taking a derivative *is* convolving with a kernel. Once we see that, every operation in this chapter becomes "pick a kernel and convolve."

32.2 The 2D convolution, concretely

Recall the 1D convolution from Chapter 13: flip one signal, slide it across the other, and at each shift multiply overlapping samples and add them up. The 2D version is identical in spirit — we just slide a small grid over a big grid.

Definition 32.1 (2D convolution)**.** Given an image $f[m, n]$ and a kernel $h[i, j]$, their convolution is

$$(f * h)[m, n] \;=\; \sum_{i} \sum_{j} f[m - i,\ n - j]\, h[i, j]. \tag{32.3}$$

The output pixel at $[m, n]$ is a weighted sum of the input pixels in a neighborhood, with the weights given by the (flipped) kernel.

What is really happening

Strip away the index gymnastics and here is what physically happens. Park the kernel — a small grid, usually 3×3 or 5×5 — on top of one pixel of the image, centered there. Multiply each kernel weight by the image pixel beneath it. Add up all those products. That single number is the output pixel. Now slide the kernel one step over and repeat, for every pixel in the image. The output is a brand-new image the same size as the input. Every output pixel is a little weighted opinion poll of its neighborhood, and the kernel is the set of votes.

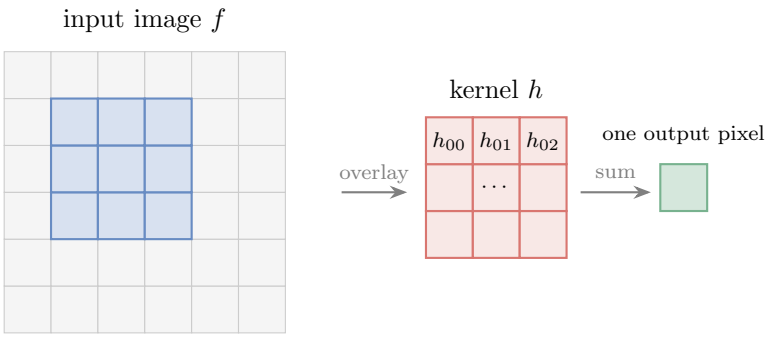


Figure 32.2: The 2D convolution as overlay-multiply-sum. The 3×3 kernel is laid over a patch of the image; each weight multiplies the pixel beneath it; the products are summed into a single output pixel. Sliding the kernel over every position produces the full output image.

Common trap

Convolution flips the kernel; correlation does not. The minus signs in Eq. (32.3) flip the kernel both horizontally and vertically before overlaying. Most image-processing code and *all* deep-learning frameworks actually implement *cross-correlation*, $\sum_{i,j} f[m+i, n+j]\, h[i, j]$, which

skips the flip. For a symmetric kernel (a blur) the two are identical. For an antisymmetric kernel (a derivative) the flip just negates the result — a sign you can absorb into the kernel. So in practice people say "convolution" and mean "correlation," and it almost never bites you. We will follow that convention but keep the distinction in mind; it matters when you compare a textbook kernel to a library's output.

Why it works

Why is convolution the *right* operation, and not some other neighborhood combination? Because it is the only operation that is both *linear* (the response to a sum of images is the sum of the responses) and *shift-invariant* (a feature looks the same wherever it appears in the frame). An edge in the top-left corner and the same edge in the bottom-right must produce the same response — a face is a face wherever it sits in the photo. Exactly as in Chapter 12, "linear + shift-invariant" forces the operation to be convolution with some kernel. The kernel *is* the system's impulse response, now a little 2D patch.

32.2.1 Boundaries, separability, and cost

Two practical points before we start choosing kernels.

Edges of the image. When the kernel hangs off the side of the image, some of its weights have no pixel beneath them. The standard fixes are to *zero-pad* (pretend the outside is black), *replicate* the border pixel outward, or *reflect* the image across its edge. Reflection usually looks best because it avoids inventing a fake dark border that would itself register as an edge.

Separability. A kernel is *separable* if it factors as an outer product of a column and a row, $h[i, j] = c[i]\, r[j]$. Then a 2D convolution splits into two cheap 1D convolutions: convolve every row with r, then every column with c. This drops the cost per output pixel from K^2 multiplies to $2K$ — a large saving for big kernels, and the reason the Gaussian blur (which is separable) is so fast. We will see that several of our edge kernels are separable too.

Example 32.1 (A 3×3 convolution by hand)**.** Take a tiny image patch and the horizontal-difference kernel:

$$f = \begin{pmatrix} 10 & 10 & 90 \\ 10 & 10 & 90 \\ 10 & 10 & 90 \end{pmatrix}, \qquad h = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}.$$

Centered on the middle pixel, cross-correlation gives

$$\sum h_{ij} f_{ij} = (-1)(10) + (0)(10) + (1)(90) + (-1)(10) + 0 + (1)(90) + (-1)(10) + 0 + (1)(90) = 240.$$

A large positive number — this kernel has detected the strong dark-to-light edge running down the right side of the patch. Slide it to a flat patch where all pixels equal 10 and the sum is 0: no edge, no response. The kernel *is* an edge detector, and the arithmetic is nothing more than a weighted sum of nine numbers.

32.3 From differences to the Sobel operator

The crude central-difference kernel $(+1,\ 0,\ -1)$ works, but it has a flaw: it is brutally sensitive to noise. A single noisy pixel produces a fake edge. The cure is to *average in the direction along the edge while differencing across it.* That is exactly what the famous **Sobel operator** does. Its horizontal-gradient kernel is

$$G_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix}, \qquad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{pmatrix}. \tag{32.4}$$

Read G_x as a product of two intentions. *Across* the kernel (left to right) it is the difference $(-1, 0, +1)$ — a derivative. *Down* the kernel it is the smoothing weights $(1, 2, 1)$ — a little blur. In fact G_x factors exactly:

$$G_x = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} -1 & 0 & +1 \end{pmatrix}, \tag{32.5}$$

so it is separable: smooth vertically with $(1, 2, 1)$, then difference horizontally with $(-1, 0, +1)$. *Differentiate across the edge, blur along it.* That is the whole design philosophy of a good gradient operator.

Why it works

Why blend blurring with differencing? Differentiation *amplifies* high frequencies — and noise is the highest-frequency content of all (Chapter 20). A pure difference kernel is a high-pass filter, so it boosts noise enormously. Blurring is low-pass; it suppresses noise. Sobel multiplies a low-pass along one axis by a high-pass along the other, giving a *band-pass* response that is tuned to real edges (which are spatially extended structures) while rejecting isolated noisy pixels (which are not). This is the same smooth-then-differentiate logic behind the derivative-of-Gaussian filter we meet in the next section.

Once we have G_x and G_y as two convolved images, we combine them into the gradient magnitude and direction at every pixel:

$$\|\nabla f\| = \sqrt{G_x^2 + G_y^2}, \qquad \theta = \operatorname{atan2}(G_y,\, G_x). \tag{32.6}$$

The magnitude image is bright exactly along edges and near-black everywhere else; the direction tells us each edge's orientation. Thresholding the magnitude gives a first, rough edge map.

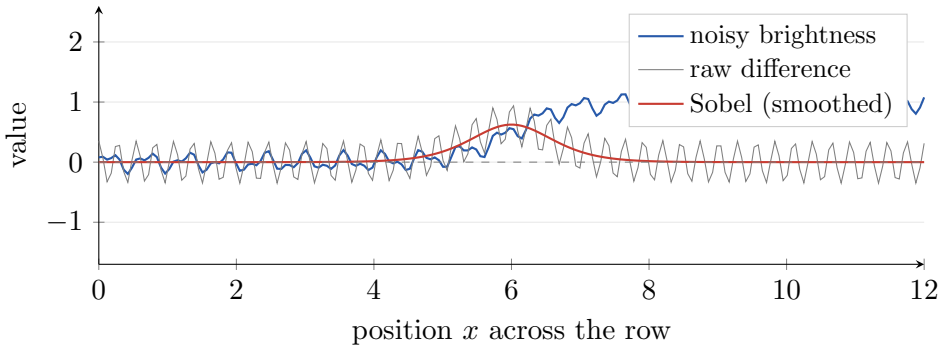


Figure 32.3: Why Sobel smooths along the edge. The raw difference of a noisy brightness profile (gray) is itself noisy — full of spurious bumps that would become fake edges. Sobel's built-in $(1, 2, 1)$ smoothing yields a clean bump (red) located right at the true edge.

Listing 32.1: Sobel edge magnitude in NumPy/SciPy.

```
import numpy as np
from scipy.ndimage import convolve

Gx = np.array([[-1, 0, 1],
               [-2, 0, 2],
               [-1, 0, 1]], dtype=float)
Gy = Gx.T  # transpose gives the vertical kernel

def sobel_edges(img):
    img = img.astype(float)
    gx = convolve(img, Gx, mode="reflect")    # reflect padding at borders
    gy = convolve(img, Gy, mode="reflect")
    mag = np.hypot(gx, gy)                     # sqrt(gx**2 + gy**2)
    return mag / mag.max()                     # normalize to [0, 1]
```

Application

Sobel and its cousins (Prewitt, with equal $(1, 1, 1)$ smoothing; Scharr, with optimized $(3, 10, 3)$ weights for better rotational accuracy) are everywhere in real systems: lane-line detection in cars, defect inspection on assembly lines, fingerprint ridge enhancement, and the first preprocessing step of countless medical-imaging pipelines. They run in real time on the cheapest hardware because they are just nine multiplies per pixel — which is precisely why they survived into the deep-learning era as the kind of filter a network's first layer reliably re-discovers on its own.

32.4 Canny: turning a gradient into clean edges

A thresholded Sobel magnitude is messy: edges come out as thick smears, faint edges flicker in and out depending on the threshold, and noise speckles the result. In 1986 John Canny asked what an *optimal* edge detector would look like and derived a multi-stage procedure that is still the default in practice. We will not redo his variational derivation, but the four stages are deeply intuitive once you have the gradient picture.

1. **Smooth with a Gaussian.** Convolve the image with a Gaussian blur first, to tame noise before differentiating. The blur width σ sets the scale of edges you care about: small σ finds fine detail, large σ finds only the big boundaries.

2. **Compute the gradient.** Apply Sobel to get $\|\nabla f\|$ and θ at every pixel. (Steps 1 and 2 together are a *derivative-of-Gaussian* filter — one convolution, by associativity.)

3. **Non-maximum suppression.** An edge should be one pixel thick, not a fat ridge. Walk along the gradient direction; keep a pixel only if its magnitude is a *local maximum* compared to its two neighbors across the edge. This thins the smear down to a crisp line at the ridge crest.

4. **Hysteresis thresholding.** Use *two* thresholds. Pixels above the high threshold are definitely edges. Pixels below the low threshold are definitely not. Pixels in between count as edges only if they connect to a strong edge. This links up broken contours while rejecting isolated noise — the trick that makes Canny output look like clean line drawings.

Intuition

Sobel answers "how steep is the cliff here?" Canny answers "draw me the exact ridgeline of the cliffs, one pixel wide, with no gaps and no false starts." The first is a measurement; the second is a decision. Non-maximum suppression is the "thin it" decision; hysteresis is the "connect it" decision. Both are heuristics layered on top of the same gradient convolution.

Connection

The derivative-of-Gaussian filter from step 2 is a beautiful meeting point with the calculus and linear-algebra companion books. By the convolution theorem (Chapter 11), *differentiating a blurred image equals convolving with the derivative of the blur*: $\frac{\mathrm{d}}{\mathrm{d}x}(g * f) = (\frac{\mathrm{d}g}{\mathrm{d}x}) * f$. So we never differentiate the noisy image at all — we differentiate the smooth Gaussian once, analytically, and convolve that fixed kernel with the image. Calculus does the hard part on paper; convolution does the cheap part on pixels.

32.5 Blurring versus differentiating: two faces of one operation

It is worth pausing on a duality that runs through everything above. Blurring and differentiating are both convolutions, but with opposite-looking kernels, and they do opposite things in the frequency domain.

A **blur** kernel has all-positive weights that sum to 1. The simplest is the 3×3 box average (every weight $\frac{1}{9}$); the better-behaved one is the Gaussian,

$$g[i, j] \;\propto\; \exp\!\left(-\frac{i^2 + j^2}{2\sigma^2}\right), \tag{32.7}$$

normalized so the weights sum to 1. Because the weights sum to one, a blur preserves the overall brightness and *attenuates high frequencies* — it is a low-pass filter. The output is a smoothed, softened image.

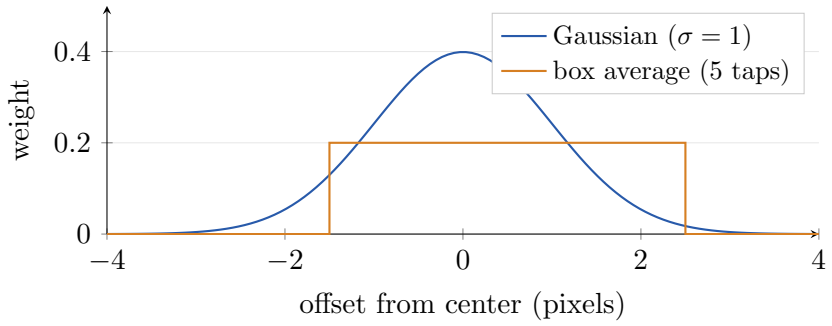


Figure 32.4: Two smoothing kernels in 1D. The box average gives every neighbor an equal vote (a hard window); the Gaussian weights nearby pixels more than distant ones (a soft window). Both sum to 1, so both preserve brightness, but the Gaussian's smooth taper avoids the ringing artifacts a sharp box edge produces.

A **derivative** kernel has weights that sum to *zero* (look back at Sobel: $-1-2-1+1+2+1=0$). A zero-sum kernel ignores constant brightness entirely — convolve it with a flat gray field and you get zero. It responds only to *changes*, which is why it is a high-pass filter and an edge detector. Differentiating boosts high frequencies; blurring suppresses them.

Focus

The single most useful rule of thumb for reading any kernel:

- **Weights sum to** 1 ⇒ it is a *smoother / low-pass* (blur, average). Preserves brightness, kills detail.
- **Weights sum to** 0 ⇒ it is a *differentiator / high-pass* (edge, derivative). Kills flat regions, keeps changes.
- **Center positive, surround negative, summing to** 1 ⇒ it is a *sharpener* — original plus a dash of high-pass.

Glance at the numbers, add them up, and you already know what a kernel does before you ever run it.

The two combine to give the second-derivative edge detector, the **Laplacian**, whose kernel is the discrete $\nabla^2 = \partial_{xx} + \partial_{yy}$:

$$L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}. \tag{32.8}$$

It sums to zero (a differentiator), and instead of *peaking* at an edge like the first derivative, it *crosses zero* there — the second derivative is positive on one side of the edge and negative on the other. Finding "zero crossings" of the Laplacian is another classic edge detector (Marr–Hildreth), and the Laplacian-of-Gaussian is the workhorse of the blob detector we meet next.

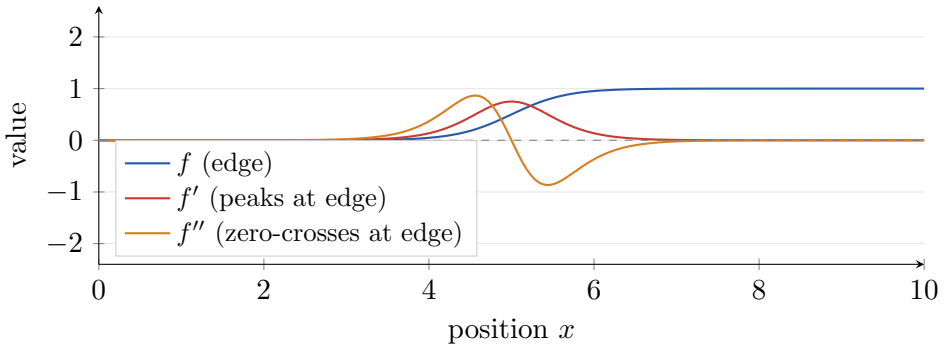


Figure 32.5: First versus second derivative at an edge. The first derivative f' *peaks* at the edge (find the maximum). The second derivative f'' *crosses zero* at the edge (find the sign change). Sobel uses the first; the Laplacian uses the second.

32.6 Beyond edges: corners and blobs

An edge is a 1D feature — it has a direction, and you can slide along it without anything changing. That makes a single edge a poor landmark: if you are trying to track a point from frame to frame (which is exactly what motion and gesture systems do), a point on an edge could be anywhere along that edge. What you want is a feature that is *distinctive in every direction* — a **corner**.

32.6.1 Corners: the structure tensor

The insight (Harris and Stephens, 1988; Moravec before them) is to ask: if I nudge a little window in any direction, how much does the patch underneath change? On a flat region, nudging changes nothing. Along an edge, nudging along the edge changes nothing, but nudging across it changes a lot — change in *one* direction. At a corner, nudging in *any* direction changes the patch a lot. Corners are where the brightness varies strongly along two independent directions at once.

We make this precise with the **structure tensor**, built entirely from the gradients we already have. Over a small window W around a pixel, form

$$M = \sum_{W} \begin{pmatrix} G_x^2 & G_x G_y \\ G_x G_y & G_y^2 \end{pmatrix}. \tag{32.9}$$

This 2×2 matrix summarizes how the gradient behaves in the window. Its two eigenvalues λ_1, λ_2 tell the whole story:

- Both small $\Rightarrow$ flat region (no structure).
- One large, one small $\Rightarrow$ an edge (strong variation in one direction only).
- Both large $\Rightarrow$ a corner (strong variation in two directions).

Computing eigenvalues per pixel is costly, so Harris uses the cheap *corner response* $R = \det M - k(\operatorname{tr} M)^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$, which is large and positive only when both eigenvalues are large. Peaks of R are corners.

Connection

The structure tensor is a gorgeous use of the linear-algebra companion book. The eigenvectors of M point along the directions of least and most change; the eigenvalues measure how much change. An edge is a near-singular M (one tiny eigenvalue); a corner is a well-conditioned M (two healthy eigenvalues). "Is this a corner?" literally becomes "is this 2×2 matrix well-conditioned?" — and every ingredient (G_x, G_y, the window sum) is a convolution.

32.6.2 Blobs: the Laplacian of Gaussian across scale

A **blob** is a roughly circular region that differs from its surroundings — a dot, a spot, a small object. The natural detector is the **Laplacian-of-Gaussian** (LoG): blur with a Gaussian of width σ, then take the Laplacian. The LoG kernel looks like a "Mexican hat": a positive bump in the middle ringed by a negative moat. Convolved with the image, it responds most strongly where a bright blob of size matched to σ sits on a darker background.

The crucial idea is *scale*. A blob has a size, and the LoG only responds strongly when its σ matches that size. So we run the LoG at a whole stack of scales and look for peaks in both space *and* scale — the (x, y, σ) where the response is maximal tells us *where* the blob is and *how big* it is. This scale-space search is the heart of classic feature detectors like SIFT (which approximates the LoG by a cheap *difference of Gaussians*, two blurs subtracted).

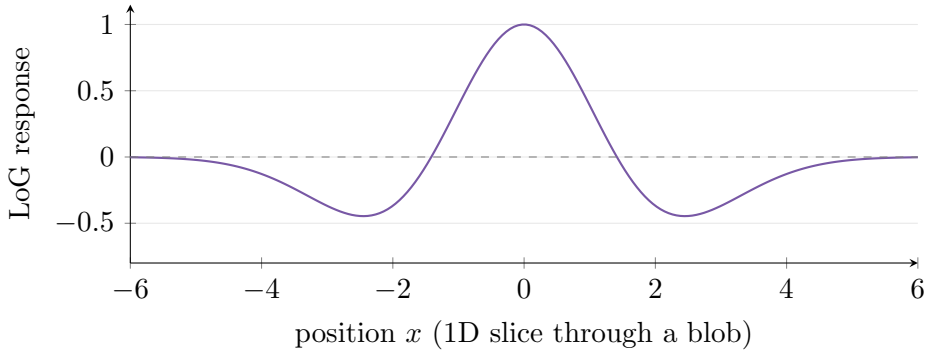


Figure 32.6: A 1D slice of the Laplacian-of-Gaussian ("Mexican hat") kernel: a positive center flanked by negative lobes. It fires hardest on a bright blob whose size matches the central lobe — the basis of multi-scale blob detection.

32.7 The 2D convolution as the heart of a CNN

Everything so far has been hand-designed: a human looked at the geometry of edges, corners, and blobs, and wrote down kernels by reasoning. The revolutionary idea of the convolutional neural network is to keep the operation — convolve the image with small kernels — but *stop choosing the kernels by hand*. Instead, treat the kernel weights as unknowns and *learn* them from labeled examples by gradient descent.

What is really happening

A convolutional layer is structurally identical to what we have been doing. It holds a bank of small kernels (say sixty-four 3×3 kernels). It convolves each one with the input image, producing sixty-four output images called *feature maps* — each map is bright wherever its kernel's pattern appears. A nonlinearity (usually ReLU: clamp negatives to zero) is applied, and the maps are downsampled (*pooling*). Then another convolutional layer does the same to those feature maps, and another, stacking dozens deep. The only thing the network "knows" at the start is that it will convolve; the *contents* of every kernel are learned.

The reason this works — and the reason convolution rather than a fully connected layer — comes straight from the geometry we built earlier.

Why it works

Why convolution instead of a generic neural-network layer? Two priors baked into the architecture, both of which we have already justified.

- **Locality.** A pixel's meaning is set by its neighbors, not by a pixel on the far side of the image. A small kernel looks only at a local patch.
- **Translation invariance / weight sharing.** The same kernel slides over the whole image, so a feature is detected identically wherever it appears — exactly the shift-invariance argument from Section 32.2. This also slashes the number of parameters: a 3×3 kernel has nine weights, no matter how big the image, versus millions for a fully connected layer.

These are the same two assumptions — linear and shift-invariant — that forced us to convolution in the first place. The CNN simply lets the data fill in the kernel weights that those assumptions left open.

32.7.1 Learned filters rediscover the classics

Here is the most satisfying fact in this chapter. When you train a CNN on natural images and then *look at* the kernels its first convolutional layer learned, you do not see random noise. You see oriented edge detectors at various angles, color-contrast detectors, small blobs, and little textures — the learned first-layer filters look strikingly like Sobel kernels, Gabor filters, and Laplacians of Gaussian. The network, given only a pile of photos and a classification goal, *rediscovers the very operators that Sobel, Canny, Harris, and Marr designed by hand.*

Why this matters

This convergence is not a coincidence; it is a statement about the world. Natural images really are built from oriented edges and blobs at multiple scales — that is the statistical structure of photographs of a world made of surfaces and objects under light. Any system that must efficiently represent such images, whether a human visual cortex (whose first-stage cells Hubel and Wiesel found to be oriented edge detectors in 1959), a hand-designed pipeline, or a trained CNN, lands on the same first-layer vocabulary. Three completely different design processes — evolution, engineering, optimization — arrive at the same answer because the answer is dictated by the data, not the designer.

Connection

This section is the bridge to the machine-learning and algorithms companion books. There you will see how the convolution's weights are trained: backpropagation computes the gradient of a loss with respect to each kernel weight, and gradient descent nudges them. The key fact that makes it work is that *convolution is differentiable in its weights* — and, pleasingly, the gradient of a convolution is itself a convolution. The forward pass, the backward pass, and the weight update are all the same flip-shift-multiply-sum machine we built in Chapter 13, run in three directions.

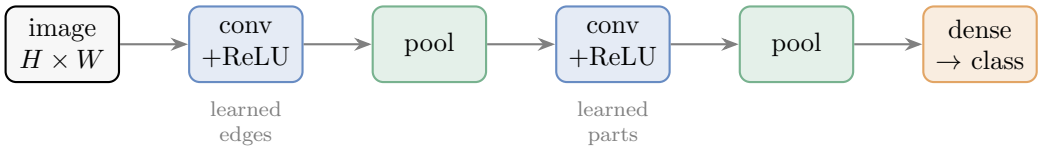


Figure 32.7: A small CNN as a stack of convolutions. Each `conv` block is a bank of learned 2D kernels — the same operation as a Sobel filter, but with weights set by training. Early layers learn edge-like filters; deeper layers combine them into parts and objects; a final dense layer reads out a class.

32.8 Carrying the idea to the wrist

Toward the wrist tracker

The wrist tracker never sees an image — so where does the 2D convolution land? In two concrete places.

1. Time × channel maps. Take a one-second window of our sensor data and stack the channels into a 2D array: rows are channels (3 accelerometer axes, 3 gyroscope axes, several EMG electrodes), columns are time samples. That array *is* a small image, and a CNN's 2D kernels can sweep across it, learning filters that detect, say, "a sharp flexion burst on EMG channel 2 coincident with a downward accelerometer spike." This is exactly how many published gesture and human-activity-recognition models work, and it is the same convolution from this chapter.

2. 1D convolution as the natural form. More often, each sensor channel is convolved on its own with a small 1D kernel (a *1D CNN*). That is nothing but the 1D special case of Definition 32.1: the same learned-filter idea, sliding a tiny window along the time axis to detect temporal motifs — the onset of a tap, the plateau of a held pose.

In both cases the lesson of this chapter carries over intact: a small, shared, learned kernel sliding over the signal, detecting a local pattern wherever it occurs.

Common trap

Know what convolution can and cannot recover. A learned convolutional filter is only as informative as the signal it sees. No amount of clever kernels on a *wrist-worn IMU alone* will reconstruct continuous individual finger-joint angles — the wrist simply does not carry that information, and a CNN cannot conjure it from data that is not there. What

2D/1D convolutional features genuinely deliver from the wrist is robust detection of *coarse, discrete* gestures (a flick, a twist, a tap, a hold) and, with forearm *EMG or force myography*, discrimination among a modest set of finger and grip patterns. Continuous finger *pose* needs sensors on the fingers themselves (instrumented rings or gloves). The convolution is a pattern matcher, not a source of information that the sensor never captured.

Build it (hardware)

A practical first-layer feature extractor for the wrist, runnable on a Pi or a beefy microcontroller, mirrors this chapter exactly. Treat a window of IMU samples as a $C \times T$ array (channels by time) and convolve with a handful of small learned 1D kernels, then pool. The skeleton:

Listing 32.2: A 1D conv feature stage for IMU windows (PyTorch sketch).

```
import torch.nn as nn

# input: (batch, C_channels=6, T_samples=128)  -- a window of accel+gyro
feature = nn.Sequential(
    nn.Conv1d(6, 16, kernel_size=7, padding=3),  # 16 learned temporal kernels
    nn.ReLU(),
    nn.MaxPool1d(2),                             # downsample in time
    nn.Conv1d(16, 32, kernel_size=5, padding=2), # combine into motifs
    nn.ReLU(),
    nn.AdaptiveAvgPool1d(1),                     # one vector per window
)
# feed the resulting vector to a small classifier over your gesture set
```

Each `Conv1d` kernel is the 1D sibling of a Sobel row. Train it on labeled gestures and inspect the learned kernels: you will find some that look like edge detectors (firing at motion *onsets*) and some that look like smoothers (averaging out tremor) — the same two faces of convolution from Section 32.5.

32.9 Summary

- An **edge** is where brightness changes fast; the **gradient** ∇f measures that change in magnitude and direction.
- Estimating derivatives on a grid is a small **convolution**: a derivative is just a kernel of differences.
- The **2D convolution** (Def. 32.1) is overlay- multiply-sum of a small kernel over the image — the unique linear, shift-invariant neighborhood operation.
- **Sobel** differentiates across an edge while smoothing along it (separable, noise-robust); **Canny** thins and links the gradient into clean one-pixel contours.
- A kernel summing to **1** blurs (low-pass); summing to **0** differentiates (high-pass). The **Laplacian** is a second-derivative detector whose zero crossings mark edges.
- **Corners** (structure-tensor eigenvalues both large) and **blobs** (Laplacian-of-Gaussian peaks across scale) are more distinctive features, all built from gradient convolutions.

- A **CNN** keeps the convolution but *learns* the kernels; its first layer reliably rediscovers edge, blob, and texture detectors — the same operators engineers designed by hand.
- For the wrist, the same idea appears as **1D/2D convolutions over time × channel windows** — a reliable detector of coarse gestures, but not a substitute for sensors that the wrist alone cannot replace.

32.10 Exercises

Exercise 32.1. Take the 3×3 patch $f = \left(\begin{smallmatrix} 5 & 5 & 5 \\ 5 & 5 & 5 \\ 80 & 80 & 80 \end{smallmatrix}\right)$ and apply the vertical Sobel kernel G_y from Eq. (32.4) by cross-correlation centered on the middle pixel. Compute the response. Does its sign tell you whether the edge runs dark-to-light going downward or upward? Now apply G_x; explain why you get zero.

Exercise 32.2. For each kernel below, sum the weights and state from the rule in Section 32.5 whether it blurs, differentiates, or sharpens: (a) $\frac{1}{16}\left(\begin{smallmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{smallmatrix}\right)$; (b) $\left(\begin{smallmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{smallmatrix}\right)$; (c) the Laplacian of Eq. (32.8). Verify your answers by predicting what each does to a flat gray image.

Exercise 32.3. Show that the Gaussian kernel of Eq. (32.7) is separable by writing it as an outer product of two 1D Gaussians. Count the multiplies per output pixel for a 7×7 Gaussian done as a single 2D convolution versus two 1D convolutions. By what factor is the separable version cheaper?

Exercise 32.4. The *difference of Gaussians* approximates the Laplacian-of-Gaussian: $\mathrm{DoG} = g_{\sigma_1} - g_{\sigma_2}$ with $\sigma_1 < \sigma_2$. Sketch the 1D profile of $g_{\sigma_1}(x) - g_{\sigma_2}(x)$ and argue that it has the Mexican-hat shape of Figure 32.6. Why is DoG so much cheaper to compute than a true LoG in a multi-scale pyramid?

Exercise 32.5. Explain, using the frequency-domain view, why differentiating an image amplifies noise and why pre-smoothing fixes it. Relate your answer to the band-pass nature of the Sobel operator described in Section 32.3.

Exercise 32.6. Consider the structure tensor of Eq. (32.9) computed over a window lying entirely on a straight vertical edge. Argue that one eigenvalue is large and the other near zero. What does this imply for the Harris response R? Why does this correctly say "edge, not corner"?

Exercise 32.7. A library reports that it computes "convolution," but feeding it the kernel $(-1, 0, 1)$ and a step that rises left-to-right gives a *negative* peak, the opposite of what Eq. (32.2) predicts. Using the trap box of Section 32.2, explain what happened and how to fix your kernel so the signs come out as expected.

Exercise 32.8. A convolutional layer with 64 kernels of size 3×3 takes a single- channel input. How many weights does it have (ignore biases)? Compare to a fully connected layer mapping a 256×256 image to 64 outputs. Use the ratio to explain the "weight sharing" argument of Section 32.7.

Exercise 32.9. You build a 6×128 "channel × time" array from one second of accel+gyro data and convolve it with a 3×3 learned kernel. Describe in words one spatiotemporal pattern such a kernel could learn to detect. Then state plainly one thing this representation *cannot* recover from the wrist alone, citing the limits-of-recovery trap of Section 32.8.

Exercise 32.10. It is observed that the first layer of a CNN trained on photos learns filters resembling oriented edge detectors. Give two independent reasons — one about the statistics of natural images, one about the architecture's built-in priors — for why this happens, connecting to the why box and aesthetic box of Section 32.7.

Chapter 33

Image Transforms and Compression

A photograph straight off a camera sensor is staggeringly redundant. A single twelve-megapixel image is roughly 12×10^6 pixels, each carrying three colour channels of eight bits, which comes to about 36 megabytes of raw data. Yet the JPEG file you actually keep is often under two megabytes – a twenty-to-one shrink – and you cannot see the difference. Something about images makes them *compressible*: most of the bits in the raw array are not telling you anything your eye would miss.

The remarkable thing is that the tool which makes this possible is the very same idea we have been building since the Fourier chapters. Compression does not come from cleverly packing pixels; it comes from *changing coordinates*. We rotate the image into a basis where its energy collapses into a few coefficients, throw away the coefficients that carry little energy or little visible detail, and store only what is left. The transform that JPEG uses – the Discrete Cosine Transform – is a close cousin of the DFT, and everything we know about projection onto sinusoids carries straight over.

This chapter builds the DCT from the geometry up, shows *why* a good transform enables compression, walks JPEG end to end (block DCT, quantization, entropy coding), and then says a word about the wavelet route that JPEG2000 takes. By the end you will understand why a transform turns "a lot of data" into "a little data plus error you agreed not to care about," which is exactly the trade we will make later when our wrist controller has to ship gesture features over a slow radio link.

33.1 Why a transform enables compression

Start with the central picture. A small image patch – say an 8×8 block of pixels – is just a vector in a high-dimensional space, here $\mathbb{R}^{64}$. In the standard pixel basis, each of the 64 coordinates is one pixel's brightness, and for a natural image *all* 64 *tend to be large and roughly equally important.* There is no obvious way to drop any of them: every pixel matters about as much as every other.

Now rotate to a different orthonormal basis – a different set of 64 perpendicular directions in the same space. The vector has not changed; only its coordinates have. If we pick the basis well, the energy of natural image patches piles up into just a handful of coordinates, leaving the rest near zero. *Those* we can discard cheaply.

Intuition

Compression by transform is a change of viewpoint, nothing more. Imagine a cloud of data points that forms a long thin cigar tilted at 45°. In the original x, y axes you need both coordinates to locate any point. Rotate the axes to lie *along* the cigar and one coordinate (the long one) carries almost all the variation while the other (the thin one) is tiny. Keep the long coordinate, round the short one to zero, and you have halved the storage while barely moving any point. The DCT does this in 64 dimensions instead of 2, and natural images really are "cigar-shaped": their energy concentrates along a few smooth directions.



Figure 33.1: Two correlated pixels (left) live along a tilted cigar; both coordinates are needed. After rotating the axes (right) almost all the spread is in u while v is confined to a thin band near zero. Rounding v to zero costs almost no error but halves the data. A transform that does this for whole blocks is the engine of image compression.

Why it works

Why does a transform let us drop coordinates at all? Because energy is conserved by an orthonormal change of basis (Parseval, which we proved for the DFT). If $\mathbf{X} = \mathbf{T}\mathbf{x}$ with $\mathbf{T}$ orthonormal, then $\|\mathbf{X}\|^2 = \|\mathbf{x}\|^2$: the total energy is the same in either basis. What changes is the *distribution* of that energy across coordinates. In the pixel basis it is spread out; in a good transform basis it is concentrated. And here is the key fact: if you zero out a transform coefficient $X[k]$, the resulting reconstruction error in the original domain is *exactly* $|X[k]|^2$ in energy, again by Parseval. So discarding the smallest coefficients does the least damage, measured in the same units the eye roughly cares about. The transform sorts the data into "important" and "ignorable" for you.

This is the whole philosophy, and it is worth saying plainly because it recurs everywhere in this book: **a transform is a lossless change of coordinates; the loss in "lossy compression" happens afterwards, when we round or drop the new coordinates.** The transform's only job is to make that rounding cheap in error per bit.

33.2 From DFT to DCT

We already have a transform that compacts energy for smooth signals: the DFT. Why invent a new one? The answer is about what happens at the *edges* of a block.

Recall that the DFT treats its input as one period of a periodic signal. When you take the DFT of an 8×8 block, you are implicitly tiling the plane with copies of that block. The right edge of one copy butts against the left edge of the next, and unless those edges happen to match, you get a sharp jump at every seam. Sharp jumps are made of high frequencies. So the DFT of an image block tends to have stubborn high-frequency coefficients that are *artifacts of the periodic wrap-around*, not real structure in the image. They refuse to be small, which ruins energy compaction and, worse, produces ringing when you discard them.

Intuition

The fix is a trick you can picture immediately: before going periodic, *mirror* the signal. Reflect the block about its right edge to make an even, symmetric signal of double length, then make *that* periodic. Now the seams match by construction – the value on one side of a seam equals the value on the other – so there is no artificial jump. A signal that is mirror-symmetric has a Fourier representation made of *cosines only* (cosines are the even basis functions; sines, being odd, cancel). That mirror-then-Fourier recipe *is* the Discrete Cosine Transform.

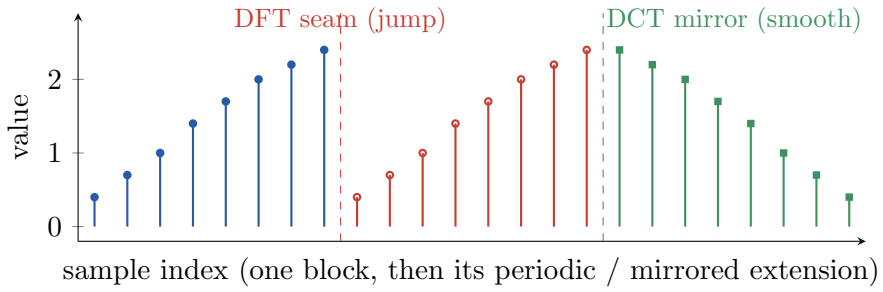


Figure 33.2: A ramp block (blue). The DFT periodically repeats it (red), creating a large jump at every seam – spurious high frequencies. The DCT first *mirrors* the block (green), so the extended signal is continuous across the seam and made of cosines only. This is why the DCT compacts image energy better than the DFT.

Let us write it down. For a length-N signal $x[0], \dots, x[N-1]$, the (type-II) **Discrete Cosine Transform** is

$$X[k] \;=\; \alpha_k \sum_{n=0}^{N-1} x[n] \cos\!\left[\frac{\pi}{N}\Big(n+\tfrac{1}{2}\Big)k\right], \qquad k = 0, \dots, N-1, \tag{33.1}$$

with normalization $\alpha_0 = \sqrt{1/N}$ and $\alpha_k = \sqrt{2/N}$ for $k \geq 1$, chosen so the transform is orthonormal. The inverse rebuilds x from the same cosines:

$$x[n] \;=\; \sum_{k=0}^{N-1} \alpha_k \, X[k] \cos\!\left[\frac{\pi}{N}\Big(n+\tfrac{1}{2}\Big)k\right]. \tag{33.2}$$

Read (33.1) exactly as you read the DFT: $X[k]$ is the inner product of the signal with the kth basis cosine, i.e. *how much of that cosine is in the signal*. The half-sample shift $n + \frac{1}{2}$ is the fingerprint of the mirror: it places the cosines so they are even about the block's edge rather than about a sample, which is what kills the seam.

Focus

The DCT is real, orthonormal, and energy-preserving. Unlike the DFT it maps real inputs to real outputs (no complex arithmetic, no phase to store), its basis vectors are mutually perpendicular unit vectors, and $\sum_k X[k]^2 = \sum_n x[n]^2$. Coordinate $k = 0$ is special: its cosine is constant, so $X[0]$ is proportional to the block's *average* brightness. It is called the DC coefficient, borrowing the electrical term for a constant; all the rest are AC coefficients.

Connection

Everything here is the inner-product / orthogonal-basis machinery from the linear algebra companion, reused verbatim. The DCT matrix $\mathbf{C}$ has orthonormal rows, so $\mathbf{C}^{-1} = \mathbf{C}^{\top}$ – inverting the transform is just transposing it. The basis cosines are the eigenvectors of a particular symmetric tridiagonal matrix, which is the deep reason the DCT, and not some arbitrary cosine set, is the "right" one for signals modelled as a smooth random process. We met the same eigenvector-of-a-shift story when the DFT diagonalized circular convolution.

33.2.1 The DCT basis functions, drawn

The eight one-dimensional DCT-II basis functions for $N = 8$ are sampled cosines of increasing frequency: a flat line ($k = 0$), then a half-cycle, a full cycle, and so on up to the fastest wiggle the block can hold. Figure 33.3 draws the first four. Reconstructing a block means scaling each of these by its coefficient $X[k]$ and adding them up.

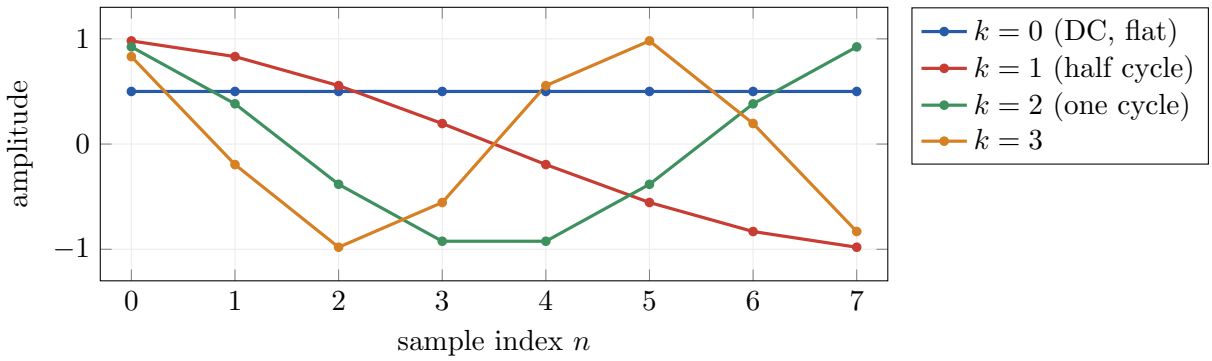


Figure 33.3: The first four DCT-II basis functions for a block of length 8. Index $k = 0$ is constant (the average), and higher k are progressively faster cosines. Any 8-sample signal is a weighted sum of these eight functions; the DCT computes the weights.

Example 33.1 (A tiny DCT by hand)**.** Take the smooth block $x = [10, 10, 10, 10, 12, 12, 12, 12]$ – flat, then a step up. Its average is 11, and the DC coefficient is $X[0] = \alpha_0 \sum_n x[n] = \sqrt{1/8} \cdot 88 \approx 31.1$. The signal is low-frequency, so the single step is captured mainly by the lowest AC cosine. Computing the rest, $X[1] \approx -2.56$, $X[2] = 0$, $X[3] \approx 0.90$, $X[4] = 0$, $X[5] \approx -0.60$, $X[6] = 0$, $X[7] \approx 0.51$. One coefficient, $X[1]$, carries essentially all the AC energy; everything above it is several times smaller and the even-indexed AC terms vanish outright. Eight numbers in, two that matter out – energy compaction in miniature.

33.3 Energy compaction: the property that matters

We keep using the phrase "energy compaction." Let us make it concrete and measurable, because it is the single number that decides whether a transform is good for compression.

Take many real image blocks, transform each, and look at where the energy lands across coefficient indices. For natural images the answer is dramatic: the DC and a few lowest-frequency AC coefficients hold the overwhelming majority of the energy, and the high-frequency coefficients are nearly zero almost all the time. Figure 33.4 sketches the typical falloff.

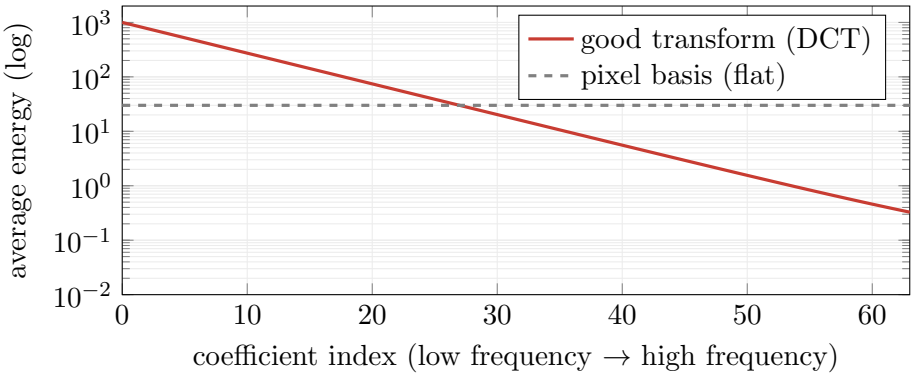


Figure 33.4: Average energy per coefficient for a 64-coefficient block. In the pixel basis (gray) energy is flat – every pixel matters equally. After the DCT (red) energy falls off steeply: the first handful of coefficients dominate, and the high-frequency tail is orders of magnitude smaller. Compression keeps the head of this curve and discards the tail.

What is really happening

What is really happening is that natural images are *spatially correlated*: neighbouring pixels are almost always similar. Correlation in the pixel domain becomes *concentration* in the transform domain – this is a general law. The more predictable a pixel is from its neighbours, the more the transform can pack into a few coefficients. Pure random noise, where no pixel predicts any other, has no correlation, so it does *not* compact – its energy stays flat in every orthonormal basis. That is precisely why noise is incompressible and why JPEG handles a clean portrait far better than a photo of static.

Why it works

Why the DCT, specifically? The theoretically optimal energy-compacting transform for a given correlation structure is the Karhunen–Loève Transform (KLT), whose basis is the eigenvectors of the data's covariance matrix. The KLT is optimal but *data-dependent*: you would have to compute and transmit a different basis for every image, which is hopeless. The DCT's gift is that for the simple, accurate model of images as a first-order Markov process (each pixel a slightly noisy copy of its neighbour with correlation near 1), the KLT basis converges to the fixed DCT cosines. So the DCT is a *universal* basis that gets you almost all of the KLT's compaction with none of its cost – a single transform that works for every image. That is why it won.

33.4 The 2D DCT

Images are two-dimensional, so we need a 2D transform. The trick is the same separability we used for the 2D Fourier transform: a 2D DCT is just a 1D DCT applied to every row, then a 1D DCT applied to every column of the result (or vice versa – the order does not matter). Formally, for an $N \times N$ block $x[m, n]$,

$$X[k, \ell] \;=\; \alpha_k \, \alpha_\ell \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} x[m, n] \, \cos\!\Big[\tfrac{\pi}{N}(m + \tfrac{1}{2})k\Big] \cos\!\Big[\tfrac{\pi}{N}(n + \tfrac{1}{2})\ell\Big]. \tag{33.3}$$

Each 2D basis function is the outer product of a horizontal cosine and a vertical cosine, giving a little checkerboard-like pattern. For an 8×8 block there are 64 such patterns, arranged in a grid by horizontal frequency k and vertical frequency ℓ. The top-left pattern ($k = \ell = 0$) is flat – pure average brightness. Move right and the pattern gains horizontal stripes; move down and it gains vertical stripes; the bottom-right pattern is the finest checkerboard the block can represent.

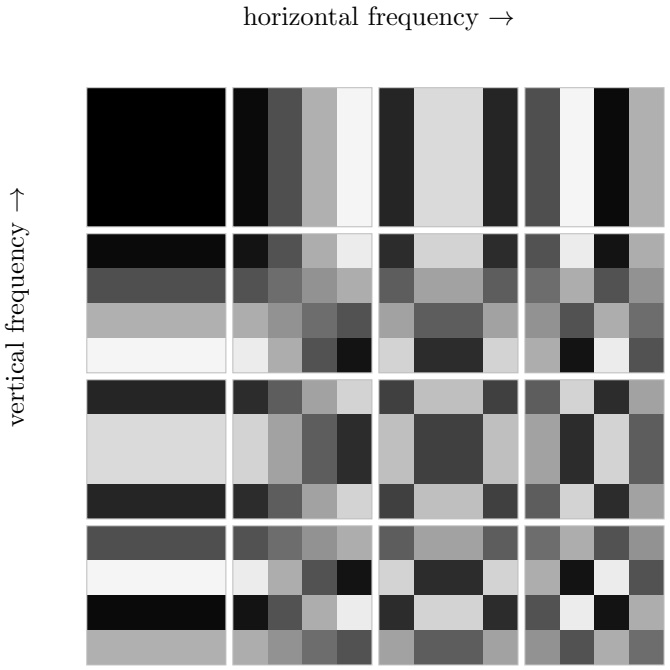


Figure 33.5: A 4×4 corner of the 2D DCT basis (the full set for an 8×8 block is 8×8). Top-left is constant brightness; rightward adds horizontal detail, downward adds vertical detail. Every image block is a weighted sum of these patterns, and the 2D DCT computes the 64 weights. Most weights are near zero for natural content.

Focus

Separability is a gift. A direct 2D DCT on an $N \times N$ block would cost $O(N^4)$. Doing $2N$ one-dimensional DCTs (rows then columns) costs $O(N^3)$, and each 1D DCT can itself be done with an FFT-style fast algorithm in $O(N \log N)$. For the 8×8 blocks JPEG uses, dedicated fast DCTs need only a few dozen multiplies per block – cheap enough that hardware JPEG encoders run on every phone camera.

33.5 JPEG, end to end

We now have every piece to understand the most widely deployed image codec in history. JPEG (named for the Joint Photographic Experts Group, 1992) is a pipeline of simple stages, each of which we can motivate. Figure 33.6 is the road map.



Figure 33.6: The JPEG encoding pipeline. Only the quantization stage loses information; everything before and after is reversible. Decoding runs the same stages in reverse (entropy decode, dequantize, inverse DCT, recombine blocks, inverse colour transform).

33.5.1 Stage 1: colour transform and subsampling

The eye is far more sensitive to brightness than to colour. JPEG exploits this by first converting RGB to $Y'C_BC_R$: a luma channel Y' (brightness) and two chroma channels C_B, C_R (blue-difference and red-difference). The conversion is a fixed linear map, e.g. $Y' = 0.299R + 0.587G + 0.114B$. Because the chroma channels carry detail the eye barely resolves, they are usually *subsampled* – stored at half resolution in each direction ("4:2:0"), immediately throwing away three-quarters of the colour samples with almost no visible cost. This is lossy, but it happens before the transform and is purely perceptual.

Common trap

The prime on Y' is not decoration: it marks *gamma-encoded* luma, not true linear luminance. JPEG operates on gamma-encoded values, which is one reason resizing or blending JPEGs naively (in the gamma domain) can darken edges. For our purposes the point is only that the colour transform is a deliberate, perceptual, lossy first step – separate from the DCT-based loss that follows.

33.5.2 Stage 2: block DCT

Each channel is sliced into non-overlapping 8×8 blocks, and each block gets the 2D DCT of (33.3). Before transforming, pixel values (originally 0 to 255) are "level shifted" by subtracting 128, centring them around zero so the coefficients are symmetric. The output is an 8×8 array of DCT coefficients: one DC term (block average) at the top-left and 63 AC terms of increasing frequency. As we saw, for natural blocks the energy clusters in the top-left.

Why it works

Why 8×8 blocks instead of one DCT of the whole image? Two reasons. First, images are not stationary – a smooth sky and a textured tree need different local treatment, and small blocks adapt locally. Second, cost and memory: a whole-image DCT is expensive and non-local, so a single edit or a single corrupted byte would smear across the entire picture.

Blocks localize both the computation and the damage. The price is the blocky artifact you see in over-compressed JPEGs – discontinuities at the 8×8 seams, the very seams the DCT's mirroring smooths *within* a block but cannot smooth *between* blocks.

33.5.3 Stage 3: quantization – where the bits are lost

This is the heart of JPEG and the only lossy step in the transform pipeline. Each DCT coefficient $X[k, \ell]$ is divided by a quantization step $Q[k, \ell]$ and rounded to the nearest integer:

$$\hat{X}[k, \ell] \;=\; \text{round}\left(\frac{X[k, \ell]}{Q[k, \ell]}\right). \tag{33.4}$$

The decoder later multiplies back: $\tilde{X}[k, \ell] = \hat{X}[k, \ell] \cdot Q[k, \ell]$, recovering the coefficient only up to a rounding error of at most $Q/2$. The genius is in the *quantization table* Q: it is not uniform. High-frequency coefficients get *large* step sizes, so they are rounded coarsely (often all the way to zero), while low-frequency coefficients get small steps and are preserved finely. The standard luma table looks like this:

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

What is really happening

The top-left entry (16) quantizes the DC term gently; the bottom-right entry (99) quantizes the finest detail brutally. This table *encodes a model of human vision* – it was tuned by experiment so that the quantization error in each frequency band is about equally invisible. We spend bits where the eye looks and starve the bands it ignores. That is the difference between mere data compression and *perceptual* compression: JPEG does not minimize numerical error, it minimizes *visible* error per bit. The "quality" slider in your image editor simply scales this entire table up or down: lower quality multiplies every Q, rounding more coefficients to zero.

Example 33.2 (Quantizing a block)**.** Suppose a block's DCT has a strong DC value $X[0, 0] = 240$ and a weak mid-frequency term $X[2, 3] = 18$, with the corresponding table entries $Q[0, 0] = 16$ and $Q[2, 3] = 24$. Then $\hat{X}[0, 0] = \text{round}(240/16) = 15$ and $\hat{X}[2, 3] = \text{round}(18/24) = \text{round}(0.75) = 1$. A slightly weaker coefficient, $X[2, 3] = 10$, would give $\text{round}(10/24) = 0$ and *vanish entirely*. After quantization most of the 63 AC entries in a typical block are exactly zero – and a block that is mostly zeros is exactly what the next stage was built to crush.

33.5.4 Stage 4: zigzag, run-length, and entropy coding

After quantization the 8×8 array is mostly zeros, clustered in the high-frequency bottom-right. To turn this into a short bitstream, JPEG reads the block in a **zigzag** order that snakes from the

top-left (DC) outward toward high frequencies. This groups the surviving non-zero low-frequency coefficients up front and sweeps all the trailing zeros into one long run at the end.

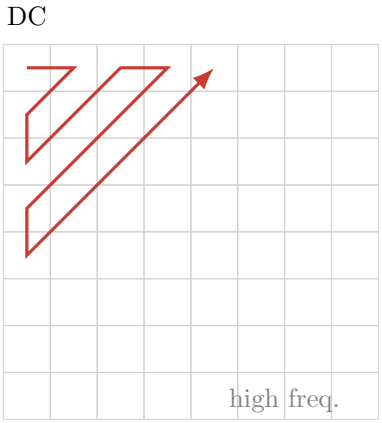


Figure 33.7: The JPEG zigzag scan order (first segment shown). Starting at the DC term, it traverses coefficients roughly in order of increasing frequency, so the typically-zero high-frequency terms end up bunched together at the tail – ideal for run-length coding.

The zigzagged sequence is then coded losslessly in two moves:

- **DC coefficients** are coded as *differences* from the previous block's DC. Adjacent blocks have similar average brightness, so these differences are small – another use of spatial correlation.
- **AC coefficients** are run-length encoded as (run of zeros, next nonzero value) pairs, with a special "end of block" symbol once only zeros remain. A block with five live coefficients and fifty-nine zeros becomes a handful of symbols.

Finally those symbols are **entropy coded** – classically with Huffman coding, optionally with arithmetic coding – which assigns short bit patterns to common symbols and long ones to rare symbols, squeezing out the last redundancy. This stage is fully lossless and reversible.

Connection

Entropy coding is information theory, the subject of the probability companion. Shannon's source coding theorem says you cannot, on average, code a source in fewer bits than its entropy $H = -\sum_i p_i \log_2 p_i$, and Huffman/arithmetic coding get within a fraction of a bit of that bound. The quantization stage's job, viewed this way, is to *lower the entropy* of the symbol stream – by mapping many coefficients to the same value (especially zero) it makes the distribution sharply peaked, so the entropy coder has little left to do. Transform, then quantize to reduce entropy, then code to the entropy limit: that three-step shape recurs in audio (MP3, AAC) and video (H.264) codecs too.

Walking question

Trace one block all the way through. Sixty-four pixels (512 bits raw) enter. The DCT (lossless) rotates them into 64 coefficients. Quantization (lossy) rounds those, typically leaving five or six nonzero and fifty-plus zeros. Zigzag plus run-length turns that into maybe a dozen symbols.

Huffman coding packs the dozen symbols into perhaps 40–80 bits. From 512 bits to ~ 60 bits – roughly 8:1 – with the only information lost being high-frequency detail the eye was never going to notice. Multiply across millions of blocks and you have the file on your phone.

33.5.5 Decoding and the artifacts

Decoding reverses every stage: entropy-decode the symbols, undo run-length and zigzag to rebuild the quantized array, multiply by Q to dequantize, inverse-DCT each block, undo the level shift, recombine blocks, and convert $Y'C_BC_R$ back to RGB. The reconstruction is not exact – the quantization rounding is gone forever – and its errors have a characteristic look:

- **Blocking**: visible 8×8 seams, because each block is quantized independently and its average drifts slightly from its neighbours'.
- **Ringing** (the Gibbs phenomenon, again): ghostly oscillations near sharp edges, because discarding high-frequency cosines is exactly truncating a Fourier series, which overshoots at discontinuities.
- **Colour bleeding**: smeared colour near edges, from chroma subsampling.

Common trap

Repeatedly opening, editing, and re-saving a JPEG re-quantizes it each time, and the errors compound – "generation loss." Worse, if you re-save with a different block alignment (after a crop by a non-multiple of 8) the new blocks straddle the old seams and quality collapses fast. Keep a lossless master (PNG or the raw file) and export to JPEG once, last.

History

The DCT was introduced by Nasir Ahmed, T. Natarajan, and K. R. Rao in 1974 – Ahmed has written that reviewers were initially skeptical the cosine transform could be so close to the optimal KLT, but the energy-compaction numbers were undeniable. The JPEG standard built on it was finalized in 1992 by a committee spanning industry and academia, and the ".jpg" file became one of the defining formats of the early web. The same DCT, in slightly different block sizes, sits inside MPEG and H.26x video, making the 1974 cosine transform arguably the most-computed numerical algorithm on Earth.

33.6 A word on wavelets: JPEG2000

JPEG's block structure is its weakness: independent 8×8 blocks cause blocking, and a fixed block size cannot be both local (for edges) and global (for smooth gradients) at once. The wavelet transform from the wavelets chapter solves both at once by analyzing the *whole image at multiple scales simultaneously*.

Intuition

Where the DCT asks "how much of each global cosine is in this block?", the wavelet transform asks "how much detail is there at each *scale* and each *location*?" It splits the image into a small blurry thumbnail (the coarse approximation) plus three detail images (horizontal, vertical, diagonal edges), then recursively splits the thumbnail again. The result is a pyramid: a tiny low-resolution image at the top and successively finer edge-detail bands below. Smooth regions produce near-zero detail coefficients at every scale; only true edges light up, and only in the bands and locations where they actually occur.

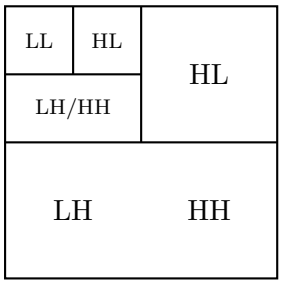


Figure 33.8: The multiresolution subband layout used by JPEG2000. The top-left corner is a recursively shrunk low-resolution version of the image (LL); the surrounding bands hold horizontal (HL), vertical (LH), and diagonal (HH) detail at each scale. Compression keeps the LL corner with fine precision and quantizes the detail bands, which are mostly zero.

JPEG2000 replaces the block DCT with this wavelet transform over the whole image, then quantizes and entropy-codes the subbands. The payoffs are real: *no blocking* (there are no blocks), graceful degradation (the same file can be decoded at low or high resolution by reading fewer or more subbands), and better quality at very high compression ratios. The costs are also real: it is computationally heavier, more complex to implement, and – partly for patent and momentum reasons – it never displaced plain JPEG for everyday photos. It found its niche in digital cinema, medical imaging, and archival, where its quality and scalability justify the cost.

Focus

The deep message survives the change of transform: **compression is transform plus quantization plus entropy coding, and the transform's only job is energy compaction.** DCT and wavelets are two different bases that both concentrate natural-image energy into few coefficients; the rest of the pipeline is shared. Choose the basis that compacts your particular signal best, and the same machinery does the rest.

33.7 Code: a DCT-based compressor in a few lines

To make it concrete, here is the entire idea – transform, threshold, inverse – in a few lines of NumPy. It is not JPEG (no blocking, no quantization tables, no entropy coder), but it demonstrates the load-bearing step: keep the largest DCT coefficients, zero the rest, and watch the image survive.

```python
import numpy as np
from scipy.fftpack import dctn, idctn   # n-dimensional DCT / IDCT

def compress(img, keep_fraction=0.05):
    """Keep only the largest |DCT| coefficients of a grayscale image."""
    X = dctn(img, norm='ortho')          # 2D DCT of the whole image
    flat = np.sort(np.abs(X), axis=None) # all magnitudes, ascending
    thresh = flat[int((1 - keep_fraction) * flat.size)]
    X_kept = X * (np.abs(X) >= thresh)   # zero out the small coefficients
    recon = idctn(X_kept, norm='ortho')  # inverse DCT -> image back
    nonzero = np.count_nonzero(X_kept)
    print(f"kept {nonzero} of {X.size} coeffs "
          f"({100 * nonzero / X.size:.1f}%)")
    return np.clip(recon, 0, 255)

# Keeping just 5% of coefficients typically yields a clearly
# recognizable image: that 20:1 drop is energy compaction at work.
```

Try it yourself

Run `compress` on a photo at `keep_fraction` of $0.50, 0.10, 0.05, 0.01$ and look at the results. Around 5% the image is still clearly readable; below about 1% ringing and smearing take over. Now feed it pure random noise (`np.random.rand`) and watch reconstruction quality collapse even at high keep fractions – noise does not compact, exactly as the energy argument predicted. Finally, replace the global DCT with an 8×8 block DCT and a real quantization table, and you have written the core of JPEG.

Figure 33.9 shows the shape you should expect from this experiment: discarding coefficients costs almost nothing at first, because the ones you drop are tiny, and only starts to bite once you reach into the coefficients that carry real structure.

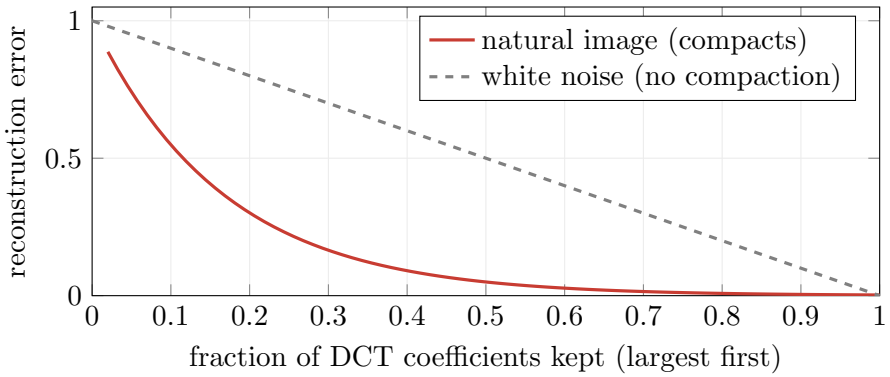


Figure 33.9: Reconstruction error against the fraction of largest coefficients retained. For a natural image (red) the error drops fast: keeping a few percent of the coefficients already captures nearly all the energy. For white noise (gray dashed) every coefficient is equally important, so error falls only linearly – you must keep almost everything. This is the energy-compaction argument turned into a curve.

33.8 Toward the wrist tracker

Toward the wrist tracker

Our gesture controller is not going to compress photographs – but it lives and dies by the same idea. A wrist-worn IMU streams accelerometer and gyroscope samples at, say, 200 Hz across several axes. Sending every raw sample over Bluetooth Low Energy to a host, hundreds of times a second, drains the battery and clogs the link. The fix is the lesson of this chapter applied to a 1D signal: transform a short window, keep the few coefficients that carry the gesture's energy, and send only those.

In practice we take a window of motion (the swing of a wrist flick, say 128 samples), run a DCT or FFT, and find – just as with images – that the energy compacts into a handful of low-frequency coefficients, because deliberate human motion is smooth and correlated in time, not random. Those few coefficients are simultaneously a *compressed* representation and an excellent *feature vector* for the classifier that decides which gesture happened. Transform-domain compaction and transform-domain feature extraction are the same act seen from two sides; we will lean on both in the recognition and embedded chapters.

A reality checkpoint. Compacting an IMU window does *not* buy us information that is not in the signal. A wrist IMU senses the gross motion and orientation of the forearm; no transform of its data can recover individual finger joint angles, because that information never reaches the sensor. Continuous finger pose needs sensors on the fingers (instrumented rings or gloves); finger-level intent without them needs EMG or force myography on the forearm. The DCT can shrink and sharpen what the IMU measures – it cannot conjure what the IMU cannot see.

Why this matters

Step back and notice how little new machinery this chapter required. We reused projection onto a basis, orthonormality, Parseval's energy identity, the periodic-extension picture, the Gibbs phenomenon, and separability – all of it from the Fourier chapters. The only genuinely new ideas were *which* basis (cosines, via mirroring) and *what to do after transforming* (quantize, then entropy-code). That economy is what a good viewpoint buys you: once you see signals as vectors and transforms as changes of coordinate, image compression, audio coding, and gesture features are one subject wearing three costumes.

Summary

- Compression is a *change of coordinates*: rotate the data into a basis where energy concentrates into few coefficients, then drop the rest. The transform is lossless; the loss happens in the rounding afterwards.
- The **DCT** is the DFT's real-valued cousin, built by mirroring a block before going periodic so the seams match. It compacts natural-image energy almost as well as the optimal but data-dependent KLT, while being a single fixed basis.
- Energy compaction works because natural images are spatially correlated; *uncorrelated noise does not compact in any basis* and is therefore incompressible.

- **JPEG** = colour transform and chroma subsampling, then per-block 8×8 2D DCT, then perceptual *quantization* (the only lossy transform step), then zigzag, run-length, and entropy coding. Its artifacts – blocking, ringing, colour bleed – come directly from those choices.
- **JPEG2000** swaps the block DCT for a whole-image wavelet transform: no blocking and resolution scalability, at higher cost and complexity. The pipeline shape – transform, quantize, entropy-code – is unchanged.
- The same transform-and-keep-the-big-coefficients trick compresses and *features* our IMU gesture windows – but no transform recovers information the sensor never captured.

33.9 Exercises

Exercise 33.1 (Parseval bookkeeping)**.** A length-4 signal has orthonormal-DCT coefficients $X = [10, 4, 0.5, 0.2]$. (a) What is the total energy $\sum_n x[n]^2$? (b) If you keep only $X[0]$ and $X[1]$ and zero the rest, what is the energy of the reconstruction error, and what fraction of the total energy is lost? (c) Comment on why this fraction is so small.

Exercise 33.2 (Why cosines, not the DFT)**.** Consider the block $x = [1, 2, 3, 4]$. (a) Sketch the periodic extension the DFT assumes and mark the jump at the seam. (b) Sketch the even (mirrored) extension the DCT assumes and confirm there is no jump. (c) In one sentence, explain why the DCT will therefore need less high-frequency energy to represent this ramp.

Exercise 33.3 (A 1D DCT by hand)**.** Using (33.1) with $N = 2$ (so $\alpha_0 = \sqrt{1/2}$, $\alpha_1 = \sqrt{1/2}$ up to the cosine factor), compute the DCT of $x = [3, 5]$. Verify that $X[0]$ is proportional to the average and that energy is conserved.

Exercise 33.4 (Quantization arithmetic)**.** A block has DCT values $X[0,0] = 204$, $X[0,1] = 23$, $X[1,0] = -9$, and $X[3,3] = 14$, with table entries Q of 16, 11, 12, and 29 respectively. (a) Compute the quantized indices $\hat{X}$. (b) Compute the dequantized values $\tilde{X}$ and the error $X - \tilde{X}$ for each. (c) Which coefficient vanished entirely, and why is that acceptable?

Exercise 33.5 (The quality knob)**.** Editors scale the quantization table by a quality factor. Suppose at quality $Q = 50$ the table is the standard one, and at lower quality every entry is multiplied by 3. (a) Using $X[2,3] = 18$ and base $Q[2,3] = 24$, find $\hat{X}$ at quality 50 and at the lower quality. (b) Explain in terms of entropy why the lower-quality file is smaller. (c) What visible artifact grows worst as the table is scaled up?

Exercise 33.6 (Energy compaction of noise)**.** Argue, using Parseval and the fact that an orthonormal transform of white noise is still white noise (uncorrelated, equal-variance), that no orthonormal transform can compact the energy of white noise into fewer coefficients. What does this predict about JPEG's file size for a photo of television static versus a photo of a clear sky?

Exercise 33.7 (Zigzag and run-length)**.** After quantization, a block in zigzag order reads [15, −2, 0, 1, 0, 0, 0, 3, 0, (all remaining entries zero). (a) Write the AC part as (run-of-zeros, value) pairs followed by end-of-block. (b) How many such symbols are needed, versus 63 raw AC numbers? (c) Why does the zigzag order, rather than row-by-row scanning, make these runs longer?

Exercise 33.8 (Block size trade-off)**.** JPEG uses 8×8 blocks. (a) Give one advantage of using larger blocks (16×16) and one disadvantage. (b) Give one advantage of 4×4 blocks and one disadvantage. (c) How does the wavelet transform sidestep having to choose a single block size at all?

Exercise 33.9 (Separable cost)**.** You must DCT a 512×512 image. (a) Estimate the multiply count for a direct, non-separable 2D DCT (treat it as a 262144×262144 matrix times a vector). (b) Estimate it for the separable row-then-column approach using a fast 1D DCT in $O(N \log N)$. (c) Roughly how many times faster is the separable fast approach?

Exercise 33.10 (From compression to features (toward the build))**.** You window your wrist IMU's z-axis gyro into 128-sample frames and take a DCT of each frame. (a) For a smooth, deliberate wrist rotation, predict roughly how many DCT coefficients carry most of the energy and where they sit. (b) Explain why those same coefficients make a good fixed-length feature vector for a gesture classifier. (c) State plainly one thing this DCT feature *cannot* tell you about the hand, and what sensor you would add to learn it.

Part VIII

Sensing Motion and the Hand

Chapter 34

Inertial Sensors: Accelerometers, Gyroscopes, Magnetometers

For thirty-three chapters we have treated signals as abstractions: a function $x(t)$, a sequence $x[n]$, a vector in some space, a spectrum. Now we plug into the physical world. The wrist tracker we are building toward does not receive a tidy waveform from the heavens – it has to *generate* its signals by pressing tiny pieces of silicon against the laws of mechanics and electromagnetism. This chapter is about the three sensors that almost every motion-sensing project leans on: the **accelerometer**, the **gyroscope**, and the **magnetometer**. Together they form what the industry calls an *IMU* (Inertial Measurement Unit), and when a magnetometer is added, a *MARG* array (Magnetic, Angular Rate, and Gravity).

The goal here is not to memorize datasheets. It is to *see*, geometrically, what each sensor actually measures, what its raw signal looks like when a real arm moves, and – crucially – what it cannot tell you no matter how cleverly you process it. By the end you should be able to wire an MPU-6050 or BNO055 to a microcontroller, stream its numbers, recognize the shape of a wrist flick in the trace, and understand exactly why a wrist IMU sees the bulk motion of the hand but leaves the fingers to a different sensor – one we add in the capstone, not because fingers are unreachable, but because they live in a different physical channel.

34.1 Three sensors, three different physical questions

It is tempting to lump the three sensors together as "the motion chip," because on a modern board they share one package and one I2C address. But they answer three genuinely different physical questions, and confusing them is the most common source of bugs in motion code.

- The **accelerometer** answers: *which way is the proof mass being pushed, and how hard?* It measures *specific force* – acceleration minus the effect of gravity, in a sense we will make precise – in units of $\mathrm{m/s^2}$ or g (where $1\,g \approx 9.81\,\mathrm{m/s^2}$).
- The **gyroscope** answers: *how fast is the chip rotating right now?* It measures *angular rate* about each of its three axes, in deg/s or rad/s.
- The **magnetometer** answers: *which way is the ambient magnetic field pointing, as seen from the chip?* It measures the magnetic flux density vector in microtesla (μT) or gauss ($1\,\mathrm{G} =$

$100\,\mu$T).

Intuition

A clean mnemonic for what each sensor "knows":

- Accelerometer: knows *down* (when you hold still) and *shoves* (when you move). It is a tilt sensor that is also a bump detector.
- Gyroscope: knows *turning*. It is blind to which way is down and blind to steady motion – it only feels change in orientation.
- Magnetometer: knows *north* (roughly). It is a compass, and like a compass it is easily fooled by nearby iron and motors.

Each sensor is excellent at one thing and useless at the others. Fusion (Chapter 36) is the art of combining their strengths; this chapter is about respecting what each one does and does not measure.

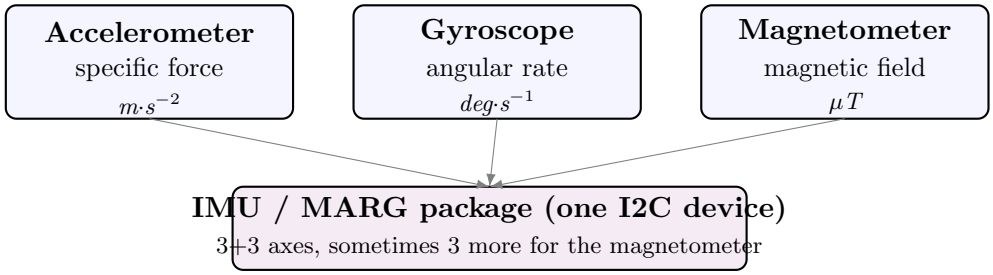


Figure 34.1: The three inertial/field sensors and what each physically measures. On a 9-axis board (e.g. MPU-9250, BNO055) all three live in one package and share a bus, but they answer independent physical questions.

34.2 The accelerometer: a mass on springs

34.2.1 The mechanical picture

Strip away the silicon and an accelerometer is embarrassingly simple: a small *proof mass* suspended by springs inside a frame. When the frame accelerates, the mass lags behind (inertia), the springs stretch, and the displacement of the mass relative to the frame tells you the force. Newton's second law for the mass, written in the frame of the chip, gives a spring displacement x obeying

$$m\,\ddot{x} + b\,\dot{x} + k\,x \;=\; -\,m\,a_{\text{frame}} + m\,g_{\text{axis}},$$

where a_{frame} is the frame's acceleration and g_{axis} is the component of gravity along the sensing axis. In steady state ($\ddot{x} = \dot{x} = 0$) the displacement is proportional to $g_{\text{axis}} - a_{\text{frame}}$. That combination is the whole story, and it has a name.

Definition 34.1 (Specific force)**.** An accelerometer measures *specific force* (also called proper acceleration):

$$\mathbf{a}_{\text{meas}} \;=\; \mathbf{a}_{\text{true}} \;-\; \mathbf{g},$$

where $\mathbf{a}_{\text{true}}$ is the true acceleration of the device in an inertial frame and $\mathbf{g}$ is the gravitational acceleration vector (pointing *down*, magnitude $\approx 9.81\,\text{m/s}^2$). All quantities are expressed in the chip's own (body) coordinate frame.

This minus sign is the single most misunderstood fact about accelerometers, so let us nail it with the two cases everyone gets wrong.

Common trap

A sitting accelerometer reads $+1\,g$ upward, not zero. A chip resting flat on a table is not accelerating ($\mathbf{a}_{\text{true}} = 0$), so it reports $\mathbf{a}_{\text{meas}} = -\mathbf{g}$, a vector of magnitude $9.81\,\text{m/s}^2$ pointing *up*. The table pushes the proof mass up against its springs exactly as if the whole thing were accelerating upward at $1\,g$ in deep space.

A freely falling accelerometer reads zero. Drop the chip and $\mathbf{a}_{\text{true}} = \mathbf{g}$, so $\mathbf{a}_{\text{meas}} = \mathbf{g} - \mathbf{g} = 0$. The proof mass and frame fall together; the springs relax. This is why an accelerometer detects free fall and why phones can sense a drop before impact.

Why it works

Why does it measure $\mathbf{a}_{\text{true}} - \mathbf{g}$ rather than plain acceleration? Because gravity acts on the proof mass and the frame *equally* (equivalence principle). Only forces transmitted *through the springs* stretch them, and gravity is not transmitted through the springs – it pulls the mass directly. So the springs report everything *except* the gravity-driven part of the motion, which is exactly $\mathbf{a}_{\text{true}} - \mathbf{g}$. The sensor cannot, even in principle, distinguish "sitting in gravity" from "accelerating upward in free space." That is not a defect; it is general relativity showing up in a cheap chip.

34.2.2 How MEMS makes it tiny

A modern accelerometer is a *MEMS* device (Micro-Electro-Mechanical System): the mass and springs are etched out of silicon, tens of microns across. The proof mass carries a comb of fingers interleaved with fixed fingers, forming a set of capacitors. When the mass moves, one gap shrinks and the other grows, so the differential capacitance changes by femtofarads. A capacitance-to-voltage converter turns this into a voltage, an on-chip ADC digitizes it, and you read the number over I2C.

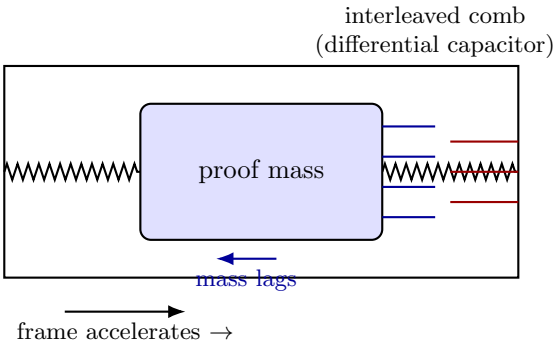


Figure 34.2: A MEMS accelerometer in cartoon form. The proof mass hangs on silicon springs; comb fingers form a differential capacitor whose imbalance reports the mass's displacement, hence the specific force along this axis. Three such structures, mutually perpendicular, give a 3-axis sensor.

34.2.3 Tilt sensing: getting angles from gravity

Because a still accelerometer reports $-\mathbf{g}$ in body coordinates, it is a very good inclinometer. If the chip is tilted, gravity's $9.81\,\mathrm{m/s^2}$ gets distributed across the three axes, and the split tells you the orientation relative to vertical. With (a_x, a_y, a_z) the measured components (in any consistent unit), the standard formulas are

$$\text{roll } \phi = \operatorname{atan2}(a_y,\, a_z), \qquad \text{pitch } \theta = \operatorname{atan2}\big(-a_x,\, \sqrt{a_y^2 + a_z^2}\big).$$

Example 34.1 (Reading tilt from a still chip)**.** Suppose a level board reads $(a_x, a_y, a_z) = (0, 0, +1)\,g$. Now tip it forward by 30° about the y-axis. Gravity now projects as $a_x = -\sin 30^\circ = -0.5\,g$ and $a_z = \cos 30^\circ = 0.866\,g$, so $\theta = \operatorname{atan2}(0.5,\, 0.866) = 30^\circ$. The accelerometer has become a protractor – *as long as the chip is not also being shaken*, because any real acceleration adds to gravity and corrupts the angle.

Common trap

Tilt-from-gravity works only when the device is (nearly) still. The instant the arm moves, $\mathbf{a}_{\text{true}} \neq 0$ and the measured vector is the *sum* of gravity and motion; you can no longer separate them from one sample. You also cannot get *yaw* (heading, rotation about vertical) from an accelerometer at all – rotating about the gravity vector leaves the gravity projection unchanged. Yaw needs the gyroscope or the magnetometer.

34.3 The gyroscope: rotation through the Coriolis effect

A MEMS gyroscope has no spinning wheel. Instead it exploits the *Coriolis force*: a mass driven to vibrate back and forth along one axis, when the whole frame rotates, feels a sideways force proportional to the rotation rate. Concretely, a proof mass is electrostatically driven to oscillate at velocity $\mathbf{v}$ along a "drive" axis. If the chip rotates with angular rate $\boldsymbol{\Omega}$, the mass experiences a Coriolis acceleration

$$\mathbf{a}_{\text{Cor}} \;=\; -\,2\,\boldsymbol{\Omega} \times \mathbf{v},$$

perpendicular to both. This tiny sideways motion is picked up by a second ("sense") capacitor, and its amplitude is proportional to $|\boldsymbol{\Omega}|$ (Figure 34.3). Three such resonators give angular rate about x, y, z.

Intuition

Picture walking straight out along a spoke of a spinning merry-go-round. Even though you walk in a "straight" line, you get pushed sideways – that push is Coriolis, and its strength grows with how fast the platform spins. The MEMS gyro replaces your walking with a mass vibrating thousands of times per second and measures the sideways push electronically. No spin rate, no push; that is why a gyro reads zero when the chip is held still, regardless of how it is oriented.

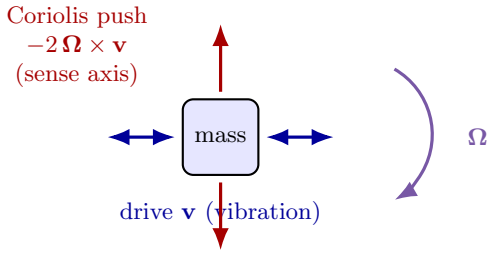


Figure 34.3: The Coriolis principle inside a MEMS gyroscope. The proof mass is driven to vibrate along the horizontal drive axis (blue). When the frame rotates at rate $\boldsymbol{\Omega}$ (purple), the mass feels a sideways Coriolis acceleration $-2\,\boldsymbol{\Omega} \times \mathbf{v}$ (red) along the perpendicular sense axis; a sense capacitor reports its amplitude, which is proportional to the rotation rate. No rotation means no sideways push, so a still gyro reads zero.

Focus

The gyro measures rate, not angle. To get an *orientation angle* you must integrate: $\theta(t) = \theta(0) + \int_0^t \Omega(\tau)\,\mathrm{d}\tau$. In discrete time, $\theta[n] = \theta[n-1] + \Omega[n]\,\Delta t$. This integration is both the gyro's superpower (smooth, fast, gravity-independent angle changes) and its curse (any constant error in Ω accumulates without bound). We return to this as *drift* below.

Connection

Integrating angular rate to get angle is the discrete antiderivative from the calculus companion: $\theta[n] = \theta[n-1] + \Omega[n]\Delta t$ is the rectangle rule applied sample by sample. The same operation as a system has transfer function $H(z) = \dfrac{\Delta t}{1 - z^{-1}}$ – a pure integrator, with a pole at $z = 1$. From Chapter 15 you already know a pole on the unit circle means infinite DC gain: the integrator amplifies any constant (bias) input without limit. That single pole at $z = 1$ *is* gyro drift, expressed in the language of the z-transform.

34.4 The magnetometer: a chip-scale compass

A magnetometer measures the local magnetic flux density vector $\mathbf{B}$ in the chip's frame. Most consumer parts use the *Hall effect* or, more sensitively, *anisotropic/giant magnetoresistance* (AMR/GMR), where the electrical resistance of a thin ferromagnetic film changes with the field threading it. Three orthogonal sensing elements give (B_x, B_y, B_z) in microtesla.

Outdoors and away from metal, the dominant field is the Earth's, magnitude roughly 25–65 μT depending on latitude, tilted into the ground by the *inclination angle*. Projected onto the horizontal plane it points toward magnetic north, which is why a magnetometer plus an accelerometer (to define horizontal) yields a tilt-compensated compass – the only one of the three sensors that gives an *absolute* heading rather than a relative change.

Common trap

The magnetometer is the diva of the three. Any nearby ferromagnetic material or current distorts the field: a *hard-iron* offset (a permanent magnet or magnetized screw shifts the whole reading by a constant vector) and a *soft-iron* distortion (nearby iron warps the field directionally, turning the locus of readings from a sphere into an ellipsoid). A wrist device sits centimeters from a watch motor, a metal buckle, maybe a phone – magnetometer heading on the wrist is notoriously unreliable and *must* be calibrated (Chapter 35) before it is worth anything.

34.5 Units, ranges, and what the raw numbers mean

Each sensor has a configurable *full-scale range* (FSR). A wider range covers more violent motion but spends fewer ADC codes per unit, so resolution drops. You read a raw signed integer (typically 16-bit, -32768 to $+32767$) and convert with a *sensitivity* (LSB per physical unit) from the datasheet.

Sensor	Typical FSR options	Example sensitivity	Unit
Accelerometer	$\pm 2, \pm 4, \pm 8, \pm 16\, g$	16384 LSB/g at $\pm 2g$	g
Gyroscope	$\pm 250, \pm 500, \pm 1000, \pm 2000$	131 LSB/(deg/s) at ± 250	deg/s
Magnetometer	$\pm 4800\,\mu$T (AK8963)	$0.15\,\mu$T/LSB	μT

Table 34.1: Representative full-scale ranges and sensitivities for the MPU-9250 family. The physical value is (raw integer)$\div$(sensitivity), or raw$\times$resolution for the magnetometer.

Example 34.2 (From raw counts to g)**.** At the $\pm 2\,g$ setting the accelerometer gives 16384 counts per g. A reading of $a_z^{\text{raw}} = 16500$ converts to $16500/16384 = 1.007\,g$ – a chip lying flat, reading the expected $+1\,g$ of supported gravity, with a 0.7% bias we would trim out in calibration. A gyro reading of $\Omega_x^{\text{raw}} = 262$ at ± 250 deg/s becomes $262/131 = 2.0\,$deg/s, a slow drift you might mistake for real rotation if you skipped bias removal.

Focus

Choosing a range for the wrist. Human limb motion rarely exceeds a few g at the wrist for ordinary gestures, but a sharp flick or a tap against a surface can spike to 8–16 g. Wrist angular rates in fast gestures reach several hundred deg/s. For a gesture controller, $\pm 8\,g$ and $\pm 1000\,\text{deg/s}$ are sensible defaults: wide enough not to clip on a snap, tight enough to keep resolution.

34.5.1 Noise, bias, and drift – the three error families

These three words get used loosely; keep them distinct, because each is fought with a different tool.

- **Noise** is random, zero-mean, sample-to-sample jitter. It is quoted as a *noise density*, e.g. $300\,\mu\text{g}/\sqrt{\text{Hz}}$ for an accelerometer or $0.01\,(\text{deg/s})/\sqrt{\text{Hz}}$ for a gyro. Multiply by $\sqrt{\text{bandwidth}}$ to get RMS noise. Noise is beaten by *averaging / low-pass filtering*, at the cost of bandwidth (Chapter 20's SNR tradeoff exactly).

- **Bias** (offset) is a constant added to every reading – a gyro that reads $1.8\,\text{deg/s}$ while perfectly still, or an accelerometer axis offset by $0.02\,g$. Bias is beaten by *calibration*: measure it while stationary and subtract.

- **Drift** is slow change *over time* – bias that wanders with temperature, or the unbounded angle error you get from integrating a biased gyro. Drift is the reason a gyro alone cannot hold orientation for more than seconds; it is beaten only by *fusion* with an absolute reference (gravity, magnetic north) in Chapter 36.

Connection

Gyro bias and noise interact viciously under integration. Treat the rate as $\Omega[n] = \Omega_{\text{true}} + b + w[n]$ with constant bias b and white noise w of per-sample variance σ^2. Integrating, the bias produces a *linear* angle error $b\,n\Delta t$, while the noise produces a random walk: the angle is the sum $\theta_{\text{noise}} = \sum_{k=1}^{n} w[k]\,\Delta t$ of independent steps, so its variance grows *linearly* with the number of samples, $\text{Var}[\theta_{\text{noise}}] = n\,\Delta t^2\,\sigma^2$ (the discrete Brownian motion of the probability companion). The standard deviation therefore grows like $\sqrt{n}$, i.e. like $\sqrt{t}$. So the deterministic bias term ($\propto t$) dominates the stochastic walk ($\propto \sqrt{t}$) at long times – which is exactly why removing bias buys you the most.

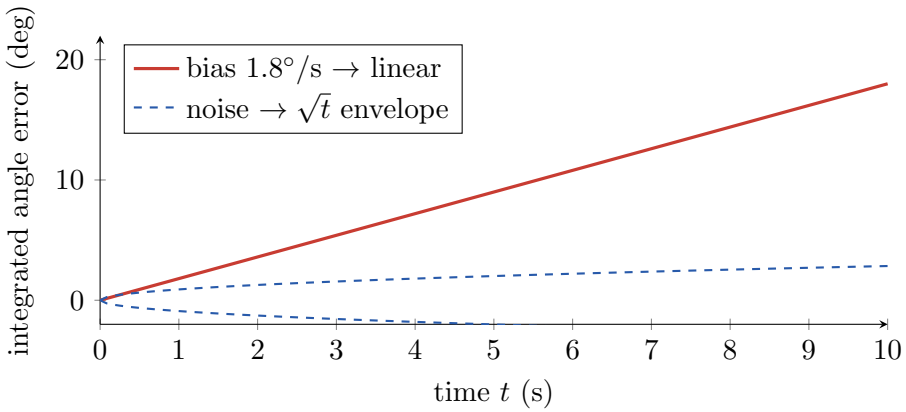


Figure 34.4: Why integrating a gyroscope drifts. A constant bias of 1.8°/s grows the angle error *linearly* (red), swamping the $\sqrt{t}$ random walk from white noise (blue, dashed envelope). After ten seconds the bias alone has produced an 18° error. Remove bias first; fuse for the rest.

34.6 Reading an IMU over I2C

The three sensors share a serial bus, almost always *I2C* (two wires: SDA data, SCL clock) or sometimes SPI for higher rates. You address the chip, point at a register, and read back bytes. The mechanics are identical across the popular boards; only the register map differs.

Build it (hardware)

Wiring an MPU-6050/9250 to an Arduino or Pi. Four wires: `VCC` (3.3 V – many breakouts have a regulator and tolerate 5 V, but the bare chip is 3.3 V), `GND`, `SDA`, `SCL`. On an Arduino Uno, `SDA`=A4, `SCL`=A5; on a Raspberry Pi, GPIO2/GPIO3 (pins 3/5). Add 4.7 kΩ pull-ups to 3.3 V on SDA and SCL if your breakout lacks them. The default 7-bit address is `0x68` (`0x69` if AD0 is tied high).

On power-up the MPU-6050 is asleep; you must clear the `PWR_MGMT_1` register (`0x6B`) to wake it. Accelerometer data starts at register `0x3B`, six bytes (X,Y,Z high/low), big-endian; gyro at `0x43`. On the 9250 the AK8963 magnetometer hangs off an auxiliary I2C bus and needs the chip put in "bypass/passthrough" mode (or master mode) before you can reach it – a classic gotcha that sends beginners hunting for a dead magnetometer that was never enabled.

Listing 34.1: Minimal MPU-6050 read over I2C on a Raspberry Pi.

```
import smbus, time
bus = smbus.SMBus(1)           # I2C bus 1 on a Pi
ADDR = 0x68
bus.write_byte_data(ADDR, 0x6B, 0)        # wake from sleep

def read_word(reg):
    hi = bus.read_byte_data(ADDR, reg)
    lo = bus.read_byte_data(ADDR, reg + 1)
    val = (hi << 8) | lo
    return val - 65536 if val >= 0x8000 else val   # signed 16-bit

```

```
ACC_LSB = 16384.0   # LSB per g at +/-2g
GYR_LSB = 131.0     # LSB per (deg/s) at +/-250 deg/s
while True:
    ax = read_word(0x3B) / ACC_LSB
    ay = read_word(0x3D) / ACC_LSB
    az = read_word(0x3F) / ACC_LSB
    gx = read_word(0x43) / GYR_LSB
    print(f"a=({ax:+.2f},{ay:+.2f},{az:+.2f}) g  gx={gx:+.1f} deg/s")
    time.sleep(0.02)   # ~50 Hz
```

Build it (hardware)

The BNO055 shortcut. Bosch's BNO055 bundles an accelerometer, gyro, magnetometer *and* a built-in ARM core running sensor fusion. You can ask it directly for an orientation quaternion or Euler angles over I2C – no integration math on your part. This is wonderful for a quick prototype and dangerous for understanding: the fusion is a black box you cannot tune, and its magnetometer-driven yaw still suffers all the wrist-proximity problems above. Prototype on a BNO055, but learn on a raw MPU so you know what the black box is hiding (Chapter 36 opens it).

Focus

Sampling rate and aliasing – the same Nyquist rule. An IMU is an ADC, so Chapter 18 applies without exception. If you read at 50 Hz, any vibration above 25 Hz aliases down into your gesture band and masquerades as slow motion. Good IMUs include a configurable analog/digital low-pass (the `CONFIG/DLPF` register on the MPU) – set its cutoff below your output Nyquist frequency. For wrist gestures, an internal DLPF around 20–40 Hz and an output rate of 100–200 Hz is a comfortable, alias-safe choice.

34.7 What wrist motion actually looks like

Enough abstraction – what do the numbers *do* when an arm moves? Three qualitative regimes dominate any wrist trace, and learning to read them by eye is half of gesture recognition (Chapter 38).

1. **Rest.** Accelerometer reads a steady vector of magnitude $\approx 1\,g$ whose direction encodes wrist orientation; gyro reads ≈ 0 plus bias and noise; magnetometer reads a steady (if distorted) field.

2. **Slow reorientation** (turning the wrist). Gyro shows a smooth bump (nonzero rate during the turn, back to zero after); the accelerometer's gravity split shifts to a new steady direction; integrating the gyro bump gives the angle turned.

3. **Sharp gesture** (a flick, tap, or snap). Both the accelerometer and gyro show a brief, large, oscillatory transient – a spike with ringing as the arm decelerates. This is the signature gesture recognizers key on.

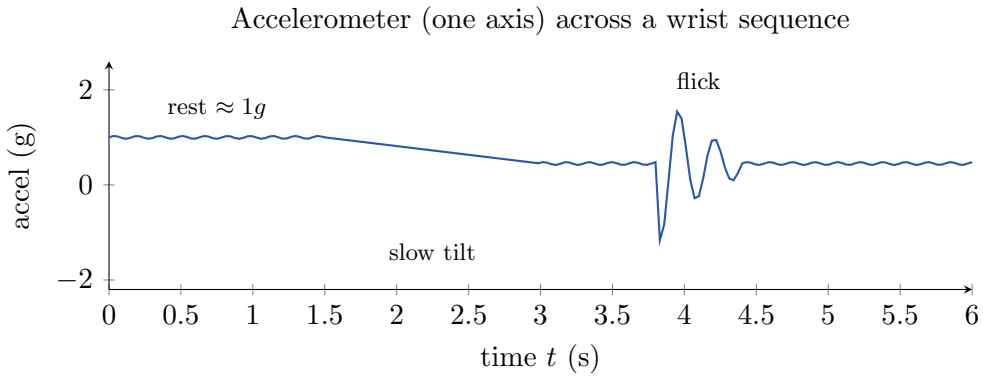


Figure 34.5: A stylized single-axis accelerometer trace through a wrist sequence: a steady $\approx 1\,g$ at rest, a smooth baseline shift as the wrist slowly tilts (gravity re-projecting onto this axis), then a sharp damped-oscillatory transient from a flick. Real traces are noisier but show these same three textures.

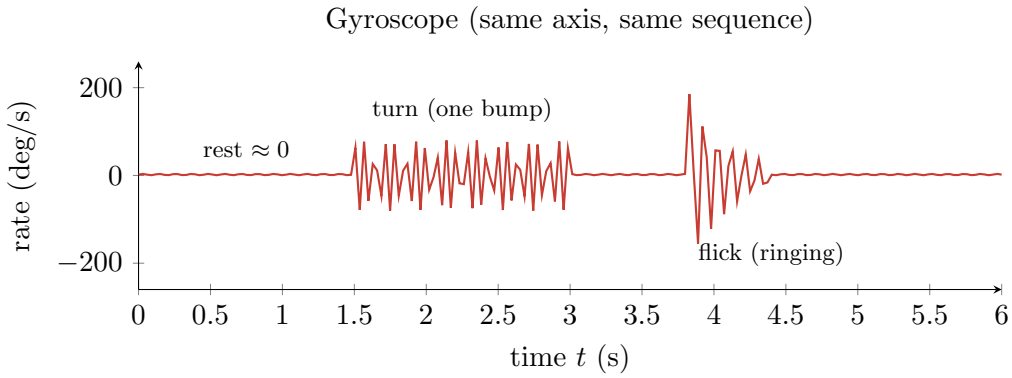


Figure 34.6: The gyroscope on the same sequence. At rest it sits near zero (a small bias visible). The slow turn is a single smooth bump whose *area* is the angle turned. The flick is a violent damped oscillation – the rate signature recognizers exploit.

What is really happening

Notice the division of labor across the two plots. During the slow tilt the accelerometer baseline *shifts* (gravity re-projects) while the gyro shows a *bump* (the rate of that shift). During rest the accelerometer holds a nonzero $1\,g$ value while the gyro sits at zero. The two sensors encode the *same* orientation change in complementary ways: the accelerometer sees it as a new static direction, the gyro sees it as a transient rate. That complementarity is precisely what makes fusion possible – and it is why neither sensor alone suffices.

34.8 What the wrist can and cannot read

This is the most important section in the chapter, and the one most often skipped in hobbyist tutorials. We are building an instrument controller driven by *finger movements*, and we need a precise map of which channel carries which information. The headline, stated up front so the rest

of the chapter does not read as a letdown: *finger movement is detectable from a forearm/wrist band* – just not from the IMU, and not as continuous joint angles. It lives in the tendon and electrical channels, which we read with the right sensor plus a per-finger calibration step.

Toward the wrist tracker

What the wrist band *can* do for the instrument controller:

- **IMU:** detect discrete gestures – flicks, taps, twists, shakes, knocks, and the broad direction of an arm sweep – which have large, distinctive accelerometer/gyro signatures (Figures 34.5–34.6). Track wrist orientation (roll/pitch reliably from gravity; yaw only with fusion and a trustworthy magnetometer, which the wrist often is not), driving continuous parameters such as pitch-bend, volume, or a filter sweep. Track gross arm rhythm: conducting beats, strums, tempo from periodicity in the trace.
- **FMG / EMG on the same band:** detect *which finger moved and how hard*. The flexor and extensor tendons that drive the fingers run through the forearm, right under a wrist/forearm band. When you curl the index finger, a specific bundle of tendons bulges (a force-myography, FMG, signal you can read with a small pressure or strain array) and a specific group of motor units fires (a surface-EMG signal). Each finger produces a repeatable spatial *signature* across the band (Figure 34.7). With a short per-user calibration, those signatures become reliable per-finger detections.

Why it works

Why fingers *are* reachable from the forearm, even though the IMU is blind to them. The muscles that move your fingers are not in your fingers – they are in your forearm, and they pull the fingers through long tendons. So the *control signal* for every finger physically passes under a forearm band twice over: mechanically, as tendon and muscle bulk that an FMG pressure array feels, and electrically, as the motor-unit action potentials that surface EMG picks up. The IMU misses all of this because finger muscles move almost no mass at the wrist – there is no inertial signature to see. Change the sensor to one that reads tendons or electrical activity and the finger information is right there, strong and repeatable. This is exactly what WristFlex (FMG) and the Myo / MyoWare sEMG bands demonstrated.

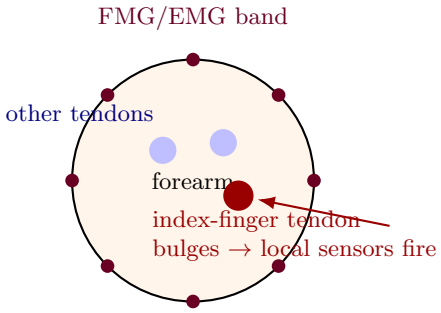


Figure 34.7: Why the fingers are readable from the forearm. A cross-section of the forearm under the band: the muscles and tendons that drive each finger pass right beneath the sensor ring. Moving one finger bulges a specific tendon bundle (red) and fires a specific group of motor units, lighting up a characteristic subset of the FMG/EMG sensors. Each finger leaves a repeatable spatial signature, which per-finger calibration turns into a reliable detection. The IMU, by contrast, sits rigidly with the band and feels none of this.

Common trap

The genuinely hard part, stated plainly: a wrist-worn band cannot recover *continuous per-joint finger angles*. Telling *that* the index finger flexed, and roughly how forcefully, is easy and repeatable. Reconstructing the exact angle of each of its three joints, continuously, is underdetermined: many distinct finger poses produce nearly the same forearm tendon bulge and EMG pattern, so the mapping from band signal to full hand pose is not invertible. The IMU is worse still – it sees only the bulk motion of the forearm and hand, so it carries *no* per-finger information at all. Continuous per-joint angles need sensors *on the fingers*: IMU rings, flex sensors, or instrumented gloves (see "Learning on the Rings" for finger-worn continuous tracking). We give that as an upgrade path in the capstone, not as a prerequisite – because nothing the user wants for playing an instrument requires it.

Why this matters

This framing is not a disappointment; it is the design brief. The IMU gives reliable discrete gestures and wrist orientation; FMG/EMG on the same band gives reliable per-finger detection after calibration; only continuous per-joint angles stay out of reach without finger-worn sensors. That tells us exactly how to architect the instrument: map wrist tilt and arm motion (IMU) to continuous expressive parameters, read note and finger *selection* from the muscle/tendon channel (FMG/EMG, with the per-finger calibration of Chapter 37 and the capstone), and treat full continuous finger pose as an optional ring-based upgrade. Every musical action the user actually wants – which finger, how hard, with what wrist gesture and tilt – sits on the doable side of this line.

History

The path from spinning-wheel gyros to the chip on your wrist is short and recent. Coriolis vibrating-structure gyros were proposed mid-20th century, but practical MEMS versions only arrived in the 1990s as silicon micromachining matured (Charles Stark Draper Laboratory and Bosch were early movers). The 2007 smartphone era drove the cost of a full 6-axis IMU

below a dollar, which is the only reason a hobbyist wrist tracker is feasible at all. The Earth's magnetic field they lean on for heading has itself been mapped for navigation since Gauss formalized geomagnetism in the 1830s – the compass in your IMU is the oldest idea in the package by two centuries.

34.9 A worked end-to-end example

Let us tie units, integration, and drift together on one concrete number, the kind you will actually compute in firmware.

Example 34.3 (Estimating a wrist twist angle)**.** You sample the gyro's roll axis at $f_s = 100\,\text{Hz}$ ($\Delta t = 0.01\,\text{s}$), $\pm 250\,\text{deg/s}$ range (131 LSB per deg/s). During a one-second twist you record raw values averaging 6550 counts, i.e. $6550/131 = 50.0\,\text{deg/s}$. Integrating over the second:

$$\Delta\theta = \sum_{n=1}^{100} \Omega[n]\,\Delta t \approx 50.0 \times 1.0 = 50^\circ.$$

Now suppose you forgot to remove a bias of $1.8\,\text{deg/s}$. Over the active second it adds only 1.8° – tolerable. But if you keep integrating during the following ten seconds of rest, the bias alone adds 18° of phantom rotation (Figure 34.4). *Lesson:* integrate the gyro only when motion is actually happening, remove bias first, and lean on the accelerometer (gravity) to re-anchor roll/pitch whenever the wrist is still – the seed of complementary fusion.

Try it yourself

Wire up any IMU you have, log $(a_x, a_y, a_z, g_x, g_y, g_z)$ at 100 Hz to a file, and perform four motions back to back: hold still 3 s, slowly rotate the wrist 90°, hold still 3 s, then a single sharp flick. Plot all six channels. Confirm by eye: (1) the accelerometer magnitude stays near $1\,g$ except during the flick; (2) the slow rotation is a smooth gyro bump whose integral is about 90°; (3) the flick is a brief, large transient in both sensors. You have now reproduced Figures 34.5 and 34.6 with your own arm.

34.10 Summary

- An accelerometer measures *specific force* $\mathbf{a}_{\text{true}} - \mathbf{g}$: it reads $+1\,g$ at rest, 0 in free fall, and serves as a tilt sensor only when motion is small. It cannot sense yaw.
- A gyroscope measures *angular rate* via the Coriolis effect: zero at rest regardless of orientation, excellent for fast smooth angle *changes*, but its integral drifts because constant bias accumulates linearly with time.
- A magnetometer measures the *magnetic field vector*, giving absolute heading – but it is severely corrupted by nearby iron and currents, a real problem on the wrist, and demands calibration.
- Raw readings are signed integers scaled by a datasheet sensitivity; choose a full-scale range wide enough not to clip and treat the IMU as an ADC bound by Nyquist (use the on-chip low-pass).

- Errors come in three flavors – noise (fight with filtering), bias (fight with calibration), drift (fight with fusion).
- For the wrist controller: the IMU delivers discrete gestures and orientation, but *not* individual fingers. Which finger moved, and how hard, is read instead from the tendon/electrical channel – FMG or EMG on the same band, made reliable by a short per-finger calibration (Chapter 37 and the capstone). Only continuous per-joint finger angles stay out of reach without finger-worn sensors, an optional upgrade.

Chapter 35 takes the raw numbers seriously and builds proper calibration and sensor models; Chapter 36 fuses the three sensors into a single drift-free orientation; and Chapter 37 turns to the FMG/EMG sensors that read the fingers directly.

34.11 Exercises

Exercise 34.1. A 3-axis accelerometer at rest reads $(a_x, a_y, a_z) = (0.50,\ 0.00,\ 0.866)\, g$. What is the chip's pitch and roll? Verify that the vector magnitude is consistent with the chip being stationary, and explain what a magnitude of $1.3\, g$ would have told you instead.

Exercise 34.2. Explain, using Definition 34.1, why an accelerometer in a satellite in orbit reads approximately zero on all axes even though the satellite is very much under the influence of Earth's gravity.

Exercise 34.3. A gyroscope is set to $\pm 2000\,\text{deg/s}$, with sensitivity 16.4 LSB per deg/s. A raw reading is -9840 counts. Convert to deg/s. If this rate is held constant for 0.5 s, what angle is swept?

Exercise 34.4. You integrate a gyro at 200 Hz. The bias is 0.6 deg/s and you fail to remove it. How much phantom angle accumulates after (a) 5 s, (b) 1 minute? Compare to the $\sqrt{t}$ growth of a noise random walk with per-sample standard deviation 0.05 deg/s and comment on which error dominates at one minute.

Exercise 34.5. At $\pm 2\, g$ an accelerometer has 16384 LSB/g; at $\pm 16\, g$ it has 2048 LSB/g. By what factor does the smallest resolvable acceleration change between the two ranges, and what is the engineering tradeoff you are making when you widen the range for the wrist controller?

Exercise 34.6. A magnetometer is rotated through all orientations while held far from metal; the cloud of (B_x, B_y, B_z) readings should trace a sphere. After mounting it on a wrist near a steel buckle, the cloud becomes an off-center ellipsoid. Identify which distortion shifts the center and which warps it into an ellipsoid, and state what each implies you must estimate during calibration.

Exercise 34.7. You sample an IMU at 40 Hz with the internal low-pass disabled, and a 55 Hz mechanical vibration (e.g. from a nearby motor) is present. To what apparent frequency does it alias? Using Chapter 18's rule, choose an internal low-pass cutoff and output rate that make a 30 Hz gesture band alias-free.

Exercise 34.8. Sketch (or describe) the accelerometer-magnitude and gyro traces you would expect from each of: (a) the wrist held perfectly still; (b) a slow 180° wrist rotation; (c) two sharp taps a quarter-second apart. For each, name the feature a recognizer would key on.

Exercise 34.9. A student claims their smartwatch app reads individual finger flexion "from the IMU alone." Using the physics of this chapter, give two concrete reasons the *IMU* cannot carry per-finger information, then name the sensor channel on the same band that *can* detect which finger moved, and explain in one sentence why the finger control signal is physically present under a forearm band even though the IMU misses it.

Exercise 34.10. Detecting *which* finger moved from a forearm band is achievable; reconstructing the *exact continuous angle* of every finger joint from the same band is not. Explain the difference in terms of invertibility: why can many distinct finger poses map to nearly the same FMG/EMG signature, and what kind of additional sensor removes the ambiguity?

Exercise 34.11. Derive the discrete integrator transfer function $H(z) = \Delta t/(1 - z^{-1})$ from the update $\theta[n] = \theta[n-1] + \Omega[n]\Delta t$, and explain in one sentence how its pole at $z = 1$ corresponds to gyroscope drift.

Chapter 35

Calibration and Sensor Models

In the last chapter we met the three workhorses of motion sensing — the accelerometer, the gyroscope, and the magnetometer — and saw, in principle, what each one measures. "In principle" is the load-bearing phrase. A datasheet promises that an accelerometer reports specific force in $\mathrm{m/s^2}$ and a magnetometer reports the field in microtesla. Wire one up, hold it dead still on your desk, and the numbers it gives you will be *wrong*. Not catastrophically wrong, usually — a few percent here, a fraction of a degree there, a slow creep over a minute — but wrong in ways that quietly poison everything downstream. Feed those readings into an orientation filter (next chapter) and the errors do not average out; they accumulate, and your virtual wrist drifts away from your real one.

The fix is *calibration*: building a small mathematical model of how the sensor lies, measuring the parameters of that model, and then undoing the lie in software before any other processing touches the data. This chapter is about that model and that measurement. It is one of the most practical chapters in the book, and also one of the most geometric — because the central trick, the ellipsoid fit, is a statement about a sphere that got squashed and shoved, and calibration is the act of un-squashing and un-shoving it.

We will build the standard sensor error model one term at a time, each with a picture of what it does to your data. We will see why an ideal accelerometer or magnetometer, swung through all orientations, should trace a perfect sphere — and why a real one traces a tilted, off-center ellipsoid instead. We will fit that ellipsoid and read the calibration off of it. We will estimate gyroscope bias and watch it drift. We will meet the Allan variance, the single most useful plot for understanding what a noisy sensor will do to you over time. And throughout, we will be clear about what calibration can and cannot rescue: no amount of clever fitting recovers individual finger joint angles from wrist bulk motion.

35.1 Why a raw reading is not the truth

Start with the cleanest possible mental model of a sensor. You have some physical quantity — call it the *true* value $\mathbf{a}$, a 3-vector for an accelerometer or magnetometer, a scalar per axis for a rate gyro. The sensor performs a measurement and hands you back a number $\mathbf{y}$. In a perfect world $\mathbf{y} = \mathbf{a}$ and we would never have written this chapter. In the real world there is a *transfer relationship* between them, and the whole art of calibration is to discover that relationship and invert it.

Intuition

Think of the sensor as a slightly drunk translator. You hand it the truth in one language (real acceleration in $\mathrm{m/s^2}$) and it hands you back a translation (counts, or volts, or scaled floats). A *good* translator gets it right. A drunk one consistently adds a fixed greeting you never asked for (bias), stretches every sentence by ten percent (scale factor), occasionally swaps two words (axis misalignment), gets sloppier as the room warms up (temperature drift), and slurs randomly from word to word (noise). Calibration is learning the translator's specific quirks well enough to mentally subtract them — so that even a drunk translator becomes reliable, as long as it is *consistently* drunk.

The crucial word there is *consistently*. Calibration only helps with the parts of the error that are *systematic* — repeatable, the same today as yesterday, a deterministic function of the true value. The part that is genuinely random — the slurring — cannot be calibrated away. It can only be *characterized* (so you know how much to trust a reading) and *filtered* (averaged down, as in Chapter 20). So our sensor model splits cleanly in two:

$$\underbrace{\text{systematic error}}_{\text{calibrate it out}} + \underbrace{\text{random error}}_{\text{characterize, then filter}}.$$

The rest of Section 35.2 builds the systematic part term by term; Sections 35.6 and onward handle the random part.

35.2 The standard sensor error model

Here is the model we are heading toward. For a 3-axis sensor, the measured vector $\mathbf{y}$ relates to the true vector $\mathbf{a}$ by

$$\mathbf{y} = \mathbf{S}\mathbf{M}\mathbf{R}\mathbf{a} + \mathbf{b} + \mathbf{n}, \tag{35.1}$$

where $\mathbf{b}$ is a constant *bias* (offset), $\mathbf{S}$ is a diagonal *scale-factor* matrix, $\mathbf{M}$ captures *misalignment* and cross-axis coupling, $\mathbf{R}$ is the rotation from the sensor's nominal axes to its true sensing axes, and $\mathbf{n}$ is random *noise*. The three matrices in front are almost always lumped together into one general 3×3 matrix,

$$\mathbf{y} = \mathbf{A}\mathbf{a} + \mathbf{b} + \mathbf{n}, \qquad \mathbf{A} = \mathbf{S}\mathbf{M}\mathbf{R}, \tag{35.2}$$

and calibration becomes: *find* $\mathbf{A}$ *and* $\mathbf{b}$*, then recover* $\hat{\mathbf{a}} = \mathbf{A}^{-1}(\mathbf{y} - \mathbf{b})$. That single equation is the spine of the whole chapter. Everything else is either a picture of one term, a method for measuring the terms, or a caveat worth stating plainly. Let us walk the terms.

35.2.1 Bias: a finger on the scale

The simplest error is a constant offset. The sensor adds (or subtracts) a fixed amount regardless of the true value:

$$y = a + b. \tag{35.3}$$

A gyroscope sitting perfectly still should read zero rotation rate; if it reads $0.6^\circ/\mathrm{s}$, that 0.6 is bias. An accelerometer lying flat should report exactly $1\,g$ on its vertical axis and 0 on the others; a few hundred mg on a horizontal axis is bias. Bias is geometrically the most innocent error — it just

shifts every reading by the same vector $\mathbf{b}$ — but for integrating sensors it is the most dangerous, because a constant offset integrates into a ramp. We return to that in Section 35.5.

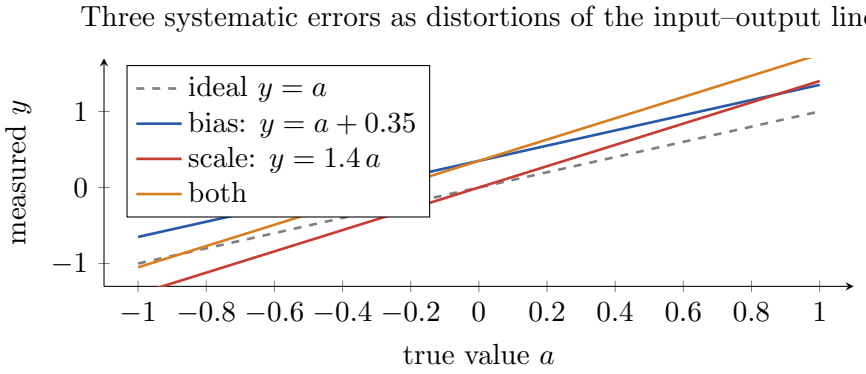


Figure 35.1: For a single axis, the ideal sensor is the line $y = a$ (gray dashed). Bias slides the line vertically; scale-factor error tilts it about the origin; real sensors do both. Calibration is the act of recovering the slope and intercept so the line can be straightened back to $y = a$.

35.2.2 Scale factor: the wrong gain

The next error multiplies rather than adds. The sensor's gain is off, so a true $1\,g$ reads as $1.03\,g$ or $0.97\,g$:

$$y = s\,a, \qquad s \approx 1. \tag{35.4}$$

Each axis can have its own scale factor, which is why $\mathbf{S}$ in (35.1) is diagonal, $\mathbf{S} = \mathrm{diag}(s_x, s_y, s_z)$. Scale error *tilts* the input–output line of Figure 35.1 instead of sliding it. Combined with bias you get the affine relationship $y = s\,a + b$ — a line with the wrong slope *and* the wrong intercept — and recovering that one line per axis is exactly the textbook "six-position" accelerometer calibration we will do by hand in Example 35.1.

Common trap

Bias and scale are easy to confuse when you only look at one orientation. If a flat accelerometer reads $1.05\,g$, is that $0.05\,g$ of bias or a 5% scale error? You *cannot tell from one measurement* — both predict the same reading at $+1\,g$. The two only separate when you look at *different* input levels: flip the sensor over so the true input is $-1\,g$. Bias makes both readings shift the same way ($+1g \to 1.05$, $-1g \to -0.95$); pure scale makes them shift oppositely ($+1g \to 1.05$, $-1g \to -1.05$). This is why any trustworthy calibration excites the sensor at *multiple* known inputs.

35.2.3 Misalignment and cross-axis coupling

Now the geometry gets interesting. A 3-axis sensor is supposed to have three *orthogonal* sensing axes lined up perfectly with the chip's package and, through it, with your board. Two things go wrong. First, the physical sensing axes are not perfectly orthogonal to each other — the MEMS structure was etched with finite precision. Second, the whole triad is rotated slightly relative to where you

think it points, because the die was bonded into the package at a tiny angle and you soldered the package onto the board at another tiny angle.

Both effects mean that motion along the true x axis leaks a little signal into the reported y and z channels. That leakage is exactly an off-diagonal entry in a matrix:

$$\begin{bmatrix} y_x \\ y_y \\ y_z \end{bmatrix} = \begin{bmatrix} 1 & m_{xy} & m_{xz} \\ m_{yx} & 1 & m_{yz} \\ m_{zx} & m_{zy} & 1 \end{bmatrix} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}. \tag{35.5}$$

The diagonal is the wanted response; the small off-diagonal terms m_{ij} (a few percent or a few tenths of a degree, in radians) are the cross-talk. A pure rotation $\mathbf{R}$ is one special, structured case of this (orthogonal, $\det = 1$); genuine non-orthogonality of the axes is another. In practice we do not bother to separate $\mathbf{S}$, $\mathbf{M}$, and $\mathbf{R}$ — they multiply into the single matrix $\mathbf{A}$ of (35.2), and the ellipsoid fit recovers $\mathbf{A}$ as a whole. The separation only matters when you care *why* the matrix is what it is.

Connection

Equation (35.5) is your linear-algebra book come to life. The matrix $\mathbf{A}$ is a linear map from the true field to the reported field; its diagonal is gain, its off-diagonals are coupling, and its inverse $\mathbf{A}^{-1}$ is precisely the change of basis that re-expresses your readings in the coordinate frame you actually wanted. When we later diagonalize the ellipsoid, we are finding the eigenvectors and eigenvalues of $\mathbf{A}^{\top}\mathbf{A}$ — the principal axes along which the squashing happened and by how much. The sphere-to-ellipsoid story is a singular-value-decomposition story wearing a lab coat.

35.2.4 Temperature drift: the model breathes

Here is the cruel part: $\mathbf{A}$ and $\mathbf{b}$ are not actually constants. They drift with temperature. MEMS structures are tiny silicon springs, and a spring's stiffness changes as it warms; the electronics' reference voltages drift too. So bias and scale are really functions of temperature T:

$$\mathbf{b}(T) \approx \mathbf{b}_0 + \mathbf{k}_b\,(T - T_0), \qquad \mathbf{A}(T) \approx \mathbf{A}_0 + \mathbf{K}_A\,(T - T_0), \tag{35.6}$$

to first order around a reference temperature T_0. Gyro bias is the notorious offender: a cheap MEMS gyro can shift its zero-rate output by several °/s across the temperature swing from a cold room to a self-heated, hand-warmed device. A wrist tracker turned on in a cold room and then warmed by body heat will see its gyro bias *walk* for the first minute or two of wear — a real, repeatable effect you can watch on a plot.

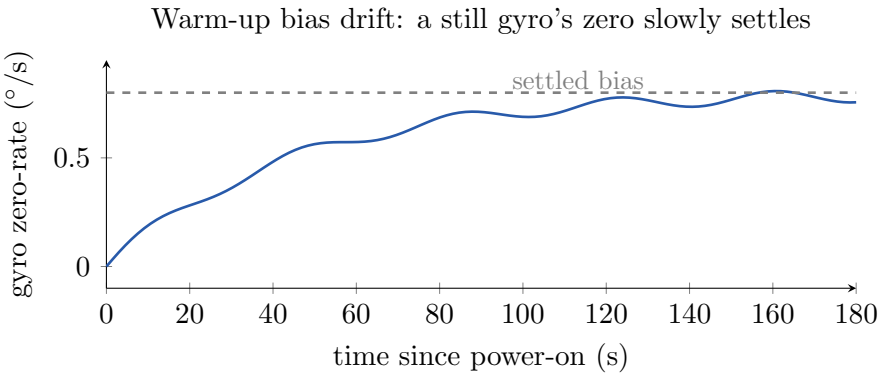


Figure 35.2: A stationary MEMS gyroscope after power-on. The zero-rate output is not flat — it ramps as the chip self-heats and the silicon's properties shift, then settles. Estimating bias in the first few seconds and trusting it for the next hour is a classic mistake; bias must be re-estimated, ideally continuously.

Common trap

The single most common field failure in cheap IMU projects is calibrating once, cold, on the bench, and baking those numbers into firmware forever. The device then warms up on the wrist, the bias walks off your stored value, and the orientation drifts. Either calibrate at the operating temperature, store a temperature curve as in (35.6), or — best for gyros — re-estimate bias on the fly whenever the device is detected to be still (see Section 35.5).

35.2.5 Random walk and the noise terms

Finally the genuinely random part, $\mathbf{n}$. It comes in two flavors that behave very differently under integration, and telling them apart is the whole reason the Allan variance exists.

The first is plain *white noise*: zero-mean, uncorrelated sample-to-sample jitter on top of every reading. For a gyro this is called *angle random walk* (ARW), because when you integrate white rate noise to get angle, the angle performs a random walk — it wanders away from the truth with a standard deviation that grows like $\sqrt{t}$ (Chapter 20's averaging law in reverse: summing N independent steps gives spread $\propto \sqrt{N}$).

The second is sneakier: the *bias itself* is not a fixed number but slowly random-walks over time. This is *bias instability* or *rate random walk*. It is what makes "calibrate the bias once" fail even at constant temperature — you nail the bias at $t = 0$, and ten minutes later it has quietly drifted to a different value through no fault of yours.

Intuition

White noise and bias drift pull in opposite directions when you average. Average *more* samples and white noise shrinks (good) — but average over a *longer time* and bias drift grows (bad), because the bias has had longer to wander. So there is a sweet-spot averaging time: short enough that the bias has not moved much, long enough that the white noise has been beaten down. The Allan variance is precisely the plot that reveals this sweet spot, and we draw it in

Section 35.6.

35.3 The big idea: a still sensor should trace a sphere

Now the geometric heart of the chapter. It applies to any sensor that measures a vector of *constant magnitude* regardless of orientation. Two of our three sensors qualify, and beautifully so:

- A **stationary accelerometer** feels only gravity. Gravity has a fixed magnitude ($\approx 9.81\,\mathrm{m/s^2}$, or $1\,g$) and points down. As you rotate the device through every orientation while holding it still, the gravity vector *in sensor coordinates* sweeps over all directions but always has the same length. The tip of that vector traces a **sphere of radius** $1\,g$.

- A **magnetometer** sitting in one location feels the local Earth field (plus any fixed nearby iron). That field has a fixed magnitude and direction in the room. Rotate the device through every orientation and, again, the field vector in sensor coordinates traces a **sphere**, this time of radius equal to the local field strength.

That is the reference truth: *a perfect sensor, swung through all orientations, returns points that lie exactly on a sphere centered at the origin.* We do not need to know which orientation produced which point. We do not need a turntable or a reference instrument. We only need the geometric fact that the magnitude is constant. This is what makes accelerometer and magnetometer calibration so wonderfully self-contained: *the calibration target is free.*

Why it works

Why does constant magnitude buy us a full calibration? Because the error model (35.2) maps that sphere to a predictable shape. Apply a matrix $\mathbf{A}$ to every point of a sphere and you get an *ellipsoid* (the matrix stretches different directions by different amounts and rotates the whole thing). Then add the bias $\mathbf{b}$ and the ellipsoid *shifts off the origin.* So a miscalibrated sensor's data does not lie on a centered sphere — it lies on a **tilted, stretched, off-center ellipsoid**. The entire calibration problem collapses to: *find the ellipsoid that best fits my data points, then find the transform that turns that ellipsoid back into the unit sphere.* That transform *is* $\mathbf{A}^{-1}$ and $\mathbf{b}$.

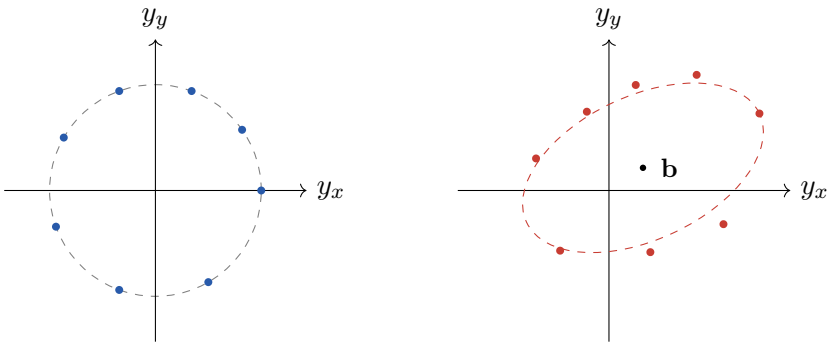


Figure 35.3: A 2D slice of the central idea. Left: a perfect sensor swung through all orientations returns points on a circle (sphere) centered at the origin. Right: scale and misalignment ($\mathbf{A}$) stretch and tilt it into an ellipse, and bias ($\mathbf{b}$) slides its center off the origin. Calibration finds the map that sends the red ellipse back to the gray circle.

35.4 The ellipsoid fit

Let us make the picture into arithmetic. Collect a cloud of measured points $\mathbf{y}_k$ by slowly tumbling the (stationary, for the accelerometer) sensor through as many orientations as you can — the more uniformly you cover the sphere, the better. We assert that, after we remove bias and undo $\mathbf{A}$, each point should have the known constant magnitude g (or field strength F). That is the constraint

$$\|\mathbf{A}^{-1}(\mathbf{y}_k - \mathbf{b})\|^2 = g^2 \quad \text{for all } k. \tag{35.7}$$

A general ellipsoid is the set of points satisfying a quadratic equation,

$$(\mathbf{y} - \mathbf{b})^{\top}\, \mathbf{Q}\, (\mathbf{y} - \mathbf{b}) = 1, \qquad \mathbf{Q} = \frac{1}{g^2}\, \mathbf{A}^{-\top}\mathbf{A}^{-1}, \tag{35.8}$$

where $\mathbf{Q}$ is a symmetric positive-definite matrix encoding the ellipsoid's shape and orientation, and $\mathbf{b}$ its center. Expanding (35.8) gives a *linear* equation in the unknown entries of $\mathbf{Q}$ and $\mathbf{b}$ (ten unknowns in 3D: six in symmetric $\mathbf{Q}$, three in $\mathbf{b}$, one scale). With many points $\mathbf{y}_k$ we get an overdetermined linear system and solve it by least squares — the same machinery from the linear-algebra book. Then we recover the calibration from $\mathbf{Q}$ and $\mathbf{b}$.

Why it works

Why does fitting an ellipsoid give us a *unique* $\mathbf{A}$? Strictly, it does not — and that subtlety matters. From $\mathbf{Q}$ we can recover $\mathbf{A}^{-1}$ only up to an arbitrary rotation, because rotating the whole sensor frame leaves all magnitudes unchanged and so leaves the ellipsoid identical. In symbols, if $\mathbf{A}^{-1}$ works then so does $\mathbf{R}\mathbf{A}^{-1}$ for any rotation $\mathbf{R}$. The standard fix is to demand the corrected matrix be *symmetric* positive-definite (the unique symmetric square root of $g^2\mathbf{Q}$). This pins down a definite answer that makes the magnitude correct in every direction; the leftover rotation ambiguity is harmless because it just means "which way is the sensor's true x axis" — and *that* we fix separately by physically defining the body frame, or by cross-referencing the accelerometer (see the next chapter on alignment).

What is really happening

What is really happening in an ellipsoid fit is a sphere being un-deformed. The matrix square root of $\mathbf{Q}$ is doing the work of an inverse SVD: it finds the three principal axes of your data cloud (the eigenvectors), notes how much each was stretched (the eigenvalues), un-stretches them back to equal length, and un-rotates them. Subtracting $\mathbf{b}$ recenters the cloud. After both steps your once-lopsided blob of points snaps onto a clean sphere of radius g. You can literally watch this happen if you plot the points before and after — it is the most satisfying "before/after" in all of sensor work.

35.4.1 The simplest version: the bounding box

You do not always need the full ten-parameter fit. The crudest useful method, beloved of hobbyist firmware, ignores misalignment and assumes $\mathbf{A}$ is *diagonal* (per-axis scale only). Then bias and scale come straight from the min and max reading on each axis as you tumble the sensor:

$$b_i = \frac{\max_i + \min_i}{2}, \qquad s_i = \frac{\max_i - \min_i}{2}. \tag{35.9}$$

The bias is the center of the swing; the scale is its half-range. This is the "hard-iron / soft-iron" min-max calibration you will see in five-line Arduino sketches. It captures bias and per-axis scale (the dominant errors) but *not* misalignment or non-orthogonality. For a hobby wrist controller it is often good enough; when it is not, you graduate to the full ellipsoid fit.

Example 35.1 (Six-position accelerometer calibration by hand)**.** Forget tumbling for a moment; here is the classic deterministic method. Place the accelerometer flat in six precise orientations: each of $\pm x$, $\pm y$, $\pm z$ pointing straight down. In each pose, exactly one axis should read $\pm 1\,g$ and the others 0. Suppose for the x-axis you measure $+1.04\,g$ with $+x$ down and $-0.98\,g$ with $-x$ down. Solve the two-point line $y = s_x a + b_x$ with $a = +1g \Rightarrow y = 1.04$ and $a = -1g \Rightarrow y = -0.98$:

$$s_x = \frac{1.04 - (-0.98)}{2} = 1.01, \qquad b_x = \frac{1.04 + (-0.98)}{2} = 0.03\,g.$$

So this axis has a 1% scale error and a 30 mg bias. Corrected reading: $\hat{a}_x = (y_x - 0.03)/1.01$. Repeat for y and z. Six poses give you the diagonal-$\mathbf{A}$ calibration without any curve fitting — the geometry of $\pm 1g$ does the work. Cross-axis terms need the ellipsoid fit or extra poses.

35.4.2 Magnetometer: hard iron and soft iron

For magnetometers the ellipsoid distortions have evocative names. *Hard-iron* distortion is a constant added field from permanent magnets and magnetized metal fixed to your board (a speaker magnet, a steel screw). It *shifts the sphere center* — it is exactly the bias $\mathbf{b}$. *Soft-iron* distortion comes from nearby ferromagnetic material that bends and concentrates the field direction-dependently; it *stretches and tilts the sphere* into an ellipsoid — it is exactly the matrix $\mathbf{A}$. So the abstract ellipsoid story and the folk magnetometer vocabulary are the same thing: hard iron = $\mathbf{b}$, soft iron = $\mathbf{A}$. The cure for both is one ellipsoid fit.

Common trap

Soft-iron sources that move *with* the sensor (screws on your own board) can be calibrated out, because they distort the field consistently in sensor coordinates. Iron in the *environment* (a steel desk, a passing car, the motor in a nearby drone) cannot — it changes as you move through the room, so it is not a fixed function of your orientation. Magnetometer calibration assumes all the iron rides along with the sensor. Out in a steel-framed building, even a perfectly calibrated magnetometer will lie. This is a deep reason a wrist tracker cannot lean too hard on the magnetometer for heading.

Listing 35.1: Hard/soft-iron-aware ellipsoid fit (least squares), then correction.

```python
import numpy as np

def fit_ellipsoid(Y):
    # Y: (N,3) magnetometer/accel samples taken while tumbling the sensor.
    # Fit  x^T Q x + p^T x = 1  (general quadric), then extract center b
    # and shape matrix; return corrector W and offset b so that
    #   a_hat = W @ (y - b)  has constant unit magnitude.
    x, y, z = Y[:, 0], Y[:, 1], Y[:, 2]
    # Design matrix for the 9 quadric coefficients (RHS folded to 1).
    D = np.column_stack([x*x, y*y, z*z, 2*x*y, 2*x*z, 2*y*z, 2*x, 2*y, 2*z])
    rhs = np.ones_like(x)
    coef, *_ = np.linalg.lstsq(D, rhs, rcond=None)    # least-squares solve
    A11, A22, A33, A12, A13, A23, gx, gy, gz = coef
    Q = np.array([[A11, A12, A13],
                  [A12, A22, A23],
                  [A13, A23, A33]])
    b = -0.5 * np.linalg.solve(Q, np.array([gx, gy, gz]))  # ellipsoid center
    # Shape: symmetric square root of Q, scaled so corrected radius = 1.
    evals, evecs = np.linalg.eigh(Q)
    evals = np.clip(evals, 1e-12, None)               # guard tiny negatives
    Qsqrt = evecs @ np.diag(np.sqrt(evals)) @ evecs.T
    scale = np.mean([np.linalg.norm(Qsqrt @ (yi - b)) for yi in Y])
    W = Qsqrt / scale                                 # so |W (y-b)| ~ 1
    return W, b

# usage:  a_hat = W @ (y - b)   -> lies on the unit sphere
```

Try it yourself

Log raw magnetometer readings for thirty seconds while you slowly rotate the sensor in a figure-eight, covering as many orientations as you can, then make a 3D scatter (or three 2D scatters: xy, xz, yz). *Before* calibration the points form a tilted, off-center ellipsoidal shell. Run Listing 35.1, apply $\hat{\mathbf{a}} = \mathbf{W}(\mathbf{y} - \mathbf{b})$, and scatter again: the shell snaps to a unit sphere centered on the origin. Now plot $\|\hat{\mathbf{a}}\|$ for every sample — it should hug 1 flat across all orientations. That flat line is the signature of a good calibration.

35.5 Gyroscope bias estimation

The sphere trick does not work for a gyroscope, and it is worth understanding exactly why. A gyro measures *rate of rotation*, not a constant-magnitude field. When the device is still, the true rate is zero — there is no sphere to trace, just a point at the origin. So gyros need a different, and in some ways simpler, calibration. The dominant systematic error is bias, and the dominant problem bias causes is *integration drift*.

35.5.1 Why gyro bias is the cardinal sin

To get orientation from a gyro you integrate rate over time: angle is the running sum of rate $\times\ \Delta t$. Now suppose the gyro has a tiny constant bias b. Integrating a constant gives a ramp:

$$\theta_{\text{error}}(t) = \int_0^t b \, \mathrm{d}\tau = b\,t. \tag{35.10}$$

A bias of just $0.5°/\text{s}$ — utterly invisible in the raw rate — becomes a $30°$ heading error after one minute and a full $180°$ after six. This is why, of all the error terms, gyro bias is the one you must hunt down most aggressively. It is also why a gyro alone can never hold orientation indefinitely: even after you subtract the best constant bias you can measure, the *leftover* drifting bias (the rate random walk of Section 35.2.5) keeps integrating into an ever-growing angle error. The accelerometer and magnetometer exist in the fusion filter precisely to nail this drift back down — which they can only do if *they* have been calibrated first.

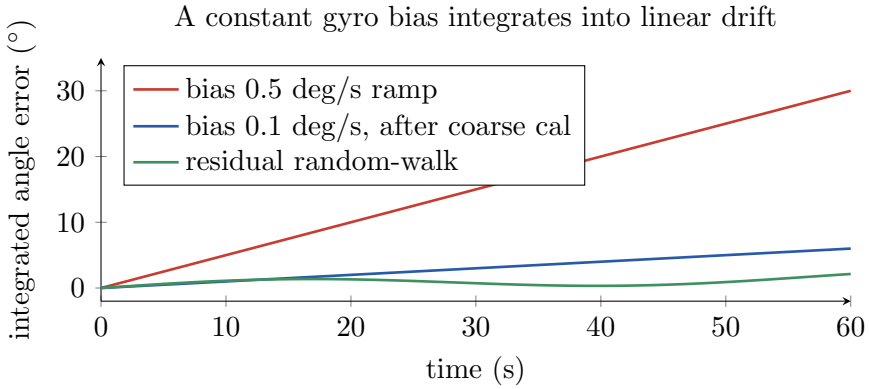


Figure 35.4: Integrating rate to get angle turns a constant bias into a linear ramp (red). Subtracting a measured bias flattens the ramp (blue), but the leftover slowly drifting bias still wanders (green). No static calibration removes the green term; that is the job of sensor fusion.

35.5.2 Estimating the bias: average while still

The good news: because the true rate is zero when the device is motionless, gyro bias is trivial to measure — *hold the device still and average*. With N samples ω_k taken at rest,

$$\hat{b} = \frac{1}{N} \sum_{k=1}^{N} \omega_k. \tag{35.11}$$

This is Chapter 20's averaging law doing exactly what it does best: the true signal is a known constant (zero), so the sample mean estimates the bias, and the white-noise part shrinks like $1/\sqrt{N}$. Average a few seconds at, say, 200 Hz and you have a clean bias estimate. The catch, as we keep insisting, is that $\hat{b}$ is only good *now*: it drifts with temperature and time. So good firmware re-runs (35.11) automatically every time it detects stillness.

Why it works

How does the device know it is still, so it can re-estimate bias? You watch two cheap signals. First, the accelerometer magnitude: at rest it reads almost exactly $1\,g$ with very little variance, so $\big|\,\|\mathbf{a}\| - g\,\big|$ small over a short window is strong evidence of stillness. Second, the gyro's own short-term variance: if the rate is barely wiggling, you are not turning. When both say "still" for, say, half a second, you trust that the true rate is zero and fold the current average into your running bias estimate. This "zero-rate update" (ZRU, or ZUPT for the velocity version) is the single most effective trick for taming gyro drift in a wearable, and it costs almost nothing.

35.6 Allan variance: reading a sensor's whole noise story

We have claimed twice now that white noise shrinks with longer averaging while bias drift grows. The Allan variance is the one plot that shows both effects at once and lets you read off the numbers. It was invented by David Allan in 1966 to characterize atomic-clock stability, and it migrated to inertial sensors because the problem is identical: how stable is a thing's output over different time scales?

History

David W. Allan, working at the U.S. National Bureau of Standards on the stability of atomic frequency standards, faced a problem ordinary variance could not answer. The standard variance of a drifting signal does not converge — it just grows as you record longer, because slow drift keeps adding spread. Allan's fix (1966) was to measure variance *between adjacent averaging windows* of length τ, and to plot it as a function of τ. The resulting curve cleanly separates noise processes that live at different time scales. Decades later, IEEE standard 952 adopted the Allan variance as the canonical way to specify fiber-optic and MEMS gyroscopes, which is why every serious IMU datasheet quotes an "angle random walk" and a "bias instability" number — both read straight off an Allan deviation plot.

35.6.1 What it measures

Take a long recording of a stationary sensor. Pick an averaging time τ. Chop the record into consecutive bins of length τ, compute the average value $\bar{y}_i(\tau)$ in each bin, and then measure how much *adjacent* bin averages differ from one another:

$$\sigma_A^2(\tau) = \frac{1}{2}\,\mathbb{E}\Big[\big(\bar{y}_{i+1}(\tau) - \bar{y}_i(\tau)\big)^2\Big]. \tag{35.12}$$

The factor of $\frac{1}{2}$ makes the answer match ordinary variance for white noise. We sweep τ from tiny to large and plot the *Allan deviation* $\sigma_A(\tau)$ (the square root) on log–log axes. Different physical noise processes show up as straight-line segments with characteristic slopes:

- **Angle/velocity random walk (white noise)** appears as a slope of $-\frac{1}{2}$: doubling the averaging time cuts the deviation by $\sqrt{2}$. This is the averaging-helps regime.
- **Bias instability** appears as a *flat* region (slope 0): the floor of the curve. Averaging longer than this stops helping — the bias is wandering as fast as you can average it out. The minimum of the curve is the sensor's best achievable stability.
- **Rate random walk** appears as a slope of $+\frac{1}{2}$: at long averaging times the deviation *grows*, because slow drift dominates.

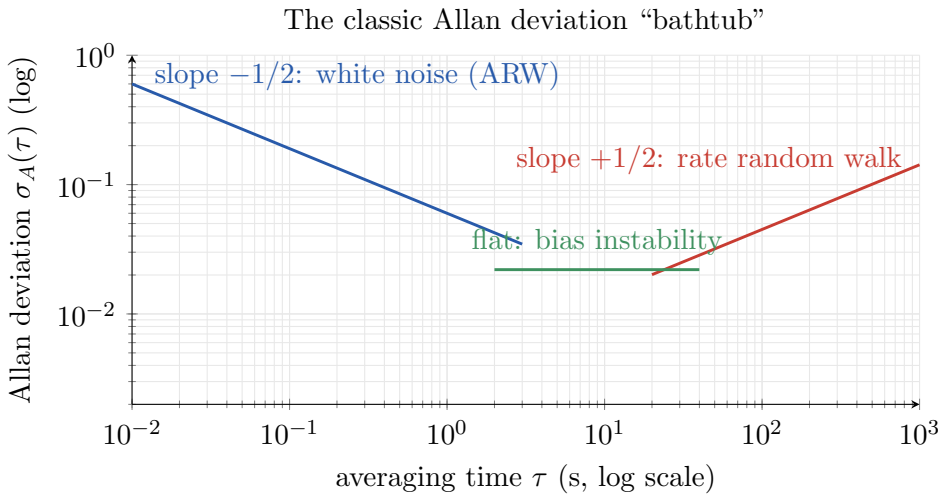


Figure 35.5: The Allan deviation of a real MEMS sensor traces a "bathtub." At short averaging times white noise dominates and falls with slope $-1/2$ (averaging helps). It bottoms out at the bias-instability floor (flat). At long times slow drift takes over and the curve climbs with slope $+1/2$. The bottom of the bathtub tells you the best averaging time and the best stability you can hope for.

Intuition

The Allan deviation plot is a sensor's autobiography. Read left to right it says: "For the first fraction of a second I am all hiss — average me and I get better fast. Around a few seconds I am as good as I will ever be; that flat floor is my true talent. Average me any longer and I start daydreaming — my bias wanders and I get worse again." The single most useful number on the plot is the τ at the bottom of the bathtub: that is the optimal averaging window for your zero-rate updates. Average exactly that long when you estimate gyro bias and you extract the most truth the sensor has to give.

Connection

The Allan variance is a cousin of the power spectral density from Chapter 20. Both describe how a random signal's energy is distributed across time scales / frequencies; in fact each Allan-slope corresponds to a power-law region of the PSD ($1/f^0$ white, $1/f$ flicker, $1/f^2$ random walk). The Allan variance is often preferred for sensors because the integrating, drifting low-frequency behavior that ruins a gyro is exactly where the ordinary PSD estimate is hardest to pin down — and where the Allan plot is clearest and most stable.

Listing 35.2: Computing the Allan deviation from a long stationary recording.

```python
import numpy as np

def allan_deviation(y, fs, taus):
    # y: 1D array of a stationary sensor axis; fs: sample rate (Hz)
    # taus: list of averaging times (s) to evaluate
    out = []
    for tau in taus:
        m = int(tau * fs)                  # samples per averaging bin
        if m < 1 or 2*m > len(y):
            continue
        n = len(y) // m
        bins = y[:n*m].reshape(n, m).mean(axis=1)   # bin averages
        d = np.diff(bins)                           # adjacent differences
        sigma2 = 0.5 * np.mean(d**2)                # Allan variance
        out.append((tau, np.sqrt(sigma2)))
    return np.array(out)
```

35.7 Why calibration must come before fusion

We have hinted at this throughout; now we state it as the chapter's thesis. The next chapter builds a *sensor fusion* filter that blends gyro, accelerometer, and magnetometer into one orientation estimate. The blend works by *trusting each sensor where it is strong and correcting it where it is weak*: the gyro for fast, short-term rotation; the accelerometer and magnetometer to pin down the slow drift. That correction is only as good as the sensors' agreement about "which way is down" and "which way is north." If the accelerometer's down is tilted 3° by uncorrected misalignment, or the magnetometer's north is bent 20° by uncorrected hard iron, the fusion filter will dutifully drag your orientation *toward the wrong answer* — and do it confidently, because it has no way to know the reference is lying.

Why this matters

Calibration is upstream of everything. A fusion filter, a gesture classifier, a machine-learning model — each assumes its inputs mean what they claim. Garbage in is not merely garbage out here; it is *confident, plausible-looking* garbage out, which is worse, because it survives a casual glance. The half-hour you spend calibrating buys correctness for every line of code downstream. It is the cheapest, highest-leverage step in the entire pipeline, and the one beginners skip first. Do it, and do it at operating temperature.

Toward the wrist tracker

For the wrist controller, here is the calibration discipline that makes the difference between a toy and an instrument. (1) **Accelerometer:** full ellipsoid fit once per build, so "down" is trustworthy to a fraction of a degree — this anchors pitch and roll. (2) **Magnetometer:** ellipsoid fit *with the final enclosure, battery, and any magnets in place*, because the hard/soft iron you must cancel is the iron that ships on the wrist; and accept that environmental iron will still corrupt heading, so the design must not depend on absolute magnetic north for playing notes. (3) **Gyro:** continuous zero-rate bias updates whenever the wrist is detected still between gestures — this is what keeps fast wrist flicks from accumulating drift over a

song. The same discipline carries over to the band's tendon and contact sensors, the channels that actually carry finger movement: the force-myography (FMG) array and the high-rate accelerometer used as a contact/bio-acoustic sensor each have their own baseline and gain, and each gets calibrated during the guided enrollment of the capstone — the user performs every finger movement a few times, the phone records the per-finger signal signature, and it sets the matching thresholds. With that per-finger calibration in hand, finger movements *are* detectable from a wrist or forearm band: tendon displacement (FMG) and the surface EMG pulse both carry which finger moved (Figure 35.6).

The one limit worth stating plainly: all of the IMU work above calibrates the *wrist's* bulk motion, and continuous per-joint finger angles are simply not present in that bulk motion — the problem is underdetermined, and no fit recovers what the signal does not contain. A continuous finger-pose curve needs sensors on the fingers themselves (instrumented rings). Treat that as an upgrade path, not a prerequisite: detecting which finger moved, and when, is enough to play notes, and that is squarely on the doable side.

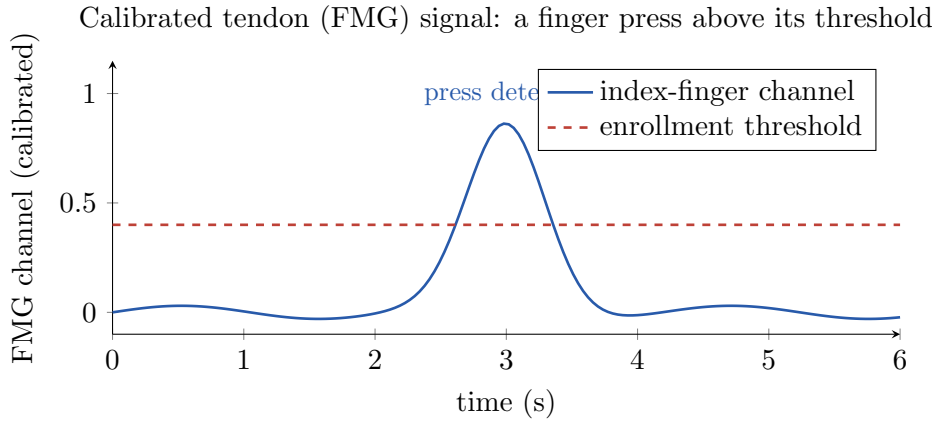


Figure 35.6: What per-finger calibration produces on a tendon (FMG) channel. The enrollment step subtracts each channel's resting baseline and normalizes its gain, so a still hand reads near zero. When the calibrated index finger presses, that channel rises into a clear bump; crossing the threshold set during enrollment (red dashed) fires a per-finger event. No machine learning is involved — the detector is a calibrated baseline, a gain, and a threshold per finger, matched live against the enrolled signature.

Build it (hardware)

A practical calibration routine you can ship in firmware and run on the bench.

Gyro bias (every boot, plus continuously).

1. At boot, ask the user to set the device down. Confirm stillness: $\big|\,\|\mathbf{a}\| - g\,\big| < 0.05\,g$ and gyro variance below a threshold over a 0.5 s window.
2. Average the gyro over 1–2 s (Eq. 35.11) and store $\hat{\mathbf{b}}_{\text{gyro}}$.
3. During use, re-run the average whenever stillness is detected; blend the new estimate into the stored one with a slow low-pass so temperature drift is tracked.

Accelerometer (once per build, at operating temperature).

1. Either do the six-position method (Example 35.1) for the diagonal model, or tumble slowly through all orientations and run the ellipsoid fit (Listing 35.1).
2. Store $\mathbf{W}_{\text{acc}}$ and $\mathbf{b}_{\text{acc}}$; apply $\hat{\mathbf{a}} = \mathbf{W}_{\text{acc}}(\mathbf{y} - \mathbf{b}_{\text{acc}})$ on every sample.
3. Sanity check: corrected $\|\hat{\mathbf{a}}\|$ should be $1\,g\pm$ a few mg in every pose.

Magnetometer (once, with final hardware assembled).

1. Power the board exactly as it will ship (battery in, enclosure closed).
2. Rotate in slow figure-eights for 20–30 s, logging raw mag.
3. Run the ellipsoid fit; store $\mathbf{W}_{\text{mag}}$ (soft iron) and $\mathbf{b}_{\text{mag}}$ (hard iron).
4. Verify corrected $\|\hat{\mathbf{m}}\|$ is flat across orientations.

Characterize (once, informative). Log a stationary gyro for 20–60 minutes and compute the Allan deviation (Listing 35.2). The bottom-of-bathtub τ tells you how long to average for zero-rate updates; the floor tells you the best heading stability to expect from the gyro alone.

Application

This exact ellipsoid machinery shows up far beyond wearables. Smartphone compass apps run a continuous background magnetometer ellipsoid fit — that "wave your phone in a figure eight" prompt is collecting points to refit the sphere after hard-iron conditions change. Drone flight controllers calibrate accelerometer and magnetometer with the same six-position and tumble routines before every serious flight, because a 2° heading error sends an autonomous waypoint mission off course. Even MRI and geophysical magnetometers use sphere/ellipsoid calibration to remove fixed instrument bias. The geometry of "constant magnitude implies a sphere" is one of the most reused ideas in all of sensing.

35.8 Worked example: separating bias from drift

Example 35.2 (Reading an Allan plot for averaging time)**.** Suppose your gyro's Allan deviation reaches its minimum (flat floor) at $\tau^\star \approx 8\,\text{s}$ with a floor value of $0.02°/\text{s}$, and at short times the white-noise branch passes through $\sigma_A(1\,\text{s}) = 0.06°/\text{s}$ with slope $-1/2$. Two practical readings follow. First, if you average a zero-rate update over 8 s you get the best possible bias estimate, good to about $0.02°/\text{s}$; averaging longer is wasted effort because the floor is flat and then climbs. Second, the angle random walk (the white-noise branch's value at $\tau = 1\,\text{s}$, by the standard convention) is $N \approx 0.06°/\sqrt{\text{s}}$, so after t seconds of free integration the random-walk angle uncertainty is roughly $0.06\sqrt{t}\ °$ — about 0.4° after 40 s. That tells you how often the accelerometer must step in to reset drift, which is exactly the tuning knob of the next chapter's fusion filter.

Example 35.3 (Bias still integrates faster than random walk)**.** Contrast Example 35.2 with an *uncalibrated* bias of $0.5°/\text{s}$. After 40 s that bias alone contributes $0.5 \times 40 = 20°$ of error — fifty times the $\sim 0.4°$ from random walk. The lesson is stark: a constant bias you failed to remove dwarfs the fundamental noise floor. Calibration (removing the deterministic 20°) is not a refinement on top of good noise performance; it is the difference between a usable device and an unusable one. Get the systematic part out first; only then does the random part even matter.

35.9 Exercises

Exercise 35.1. A flat accelerometer's z-axis reads $+1.06\,g$ pointing up and $-0.96\,g$ pointing down. Find the scale factor s_z and bias b_z for the affine model $y = s_z a + b_z$, and write the correction formula for $\hat{a}_z$. Is the bias positive or negative?

Exercise 35.2. Explain, using the input–output line of Figure 35.1, why a single measurement at $+1\,g$ cannot distinguish bias from scale error. What is the minimum number of distinct true input levels needed to separate them on one axis, and why?

Exercise 35.3. A magnetometer's raw data, plotted in 3D while tumbling, forms a sphere whose center is at $(12, -8, 30)$ μT rather than the origin. Which error term is this — hard iron or soft iron? What is the corresponding vector in the model of (35.2), and how do you correct it?

Exercise 35.4. Using the min–max method of (35.9), suppose the x-axis of a magnetometer swings between $\min_x = -55$ and $\max_x = +5$. Compute the hard-iron offset and per-axis scale for x. If the y-axis swings between -30 and $+30$, what does the difference in scale tell you about soft iron?

Exercise 35.5. A gyro has a constant bias of $0.2^\circ/\text{s}$ that you did not remove. You integrate to track heading. How long until the heading error reaches 90°? Now suppose you remove that bias but a residual rate random walk makes the angle uncertainty grow as $0.05\sqrt{t}\ ^\circ$. How long until *that* reaches 90°? Comment on the enormous ratio.

Exercise 35.6. Sketch (by hand or in code) the Allan deviation bathtub. Label the slope of each of the three regions and name the noise process responsible. Mark the point that gives the optimal averaging time, and explain in one sentence why averaging longer than that point makes the estimate *worse*.

Exercise 35.7. Why does the sphere-fit calibration work for accelerometers and magnetometers but *not* for gyroscopes? Identify the geometric property the first two share that the gyro lacks, and describe the alternative calibration the gyro requires instead.

Exercise 35.8. You calibrate a magnetometer perfectly on your wooden workbench, then carry the device to a steel-framed doorway. The calibration that gave a flat $\|\hat{\mathbf{m}}\| = 1$ on the bench now gives readings off by 15° in heading. Explain why, distinguishing iron that moves with the sensor from iron fixed in the environment. What does this imply for a wrist controller's use of magnetic heading?

Exercise 35.9. Write pseudocode for a stillness detector suitable for triggering a zero-rate gyro bias update. Use both the accelerometer magnitude and the short-window gyro variance. State the two thresholds you would tune and what failure mode each one guards against.

Exercise 35.10. (Scope-of-calibration essay.) A teammate hopes to recover *continuous per-joint finger angles* from a single well-calibrated wrist IMU. In two or three sentences, explain why no amount of IMU calibration can deliver that (what information is missing from wrist bulk motion), and what sensor change would be required. Separately, note why *detecting which finger moved* is a different, achievable problem given a calibrated tendon/contact sensor array.

Chapter 36

Orientation Estimation and Sensor Fusion

In the last two chapters we met the three little sensors that live inside an inertial measurement unit (IMU) – the accelerometer that feels gravity and motion, the gyroscope that feels rotation, the magnetometer that feels the Earth's field – and we learned to calibrate the raw numbers into trustworthy physical units. But raw numbers are not what we ultimately want. When you tilt your wrist to bend a note, or roll your forearm to sweep a filter, the thing that matters is not "what is the acceleration along the chip's x-axis right now" – it is *which way is the wrist pointing.* That single question – the device's *orientation* in space, also called its *attitude* or *pose* – is the subject of this chapter.

Orientation is a deceptively rich quantity. It is not three numbers you can simply add up; it lives on a curved space (the set of rotations), and the naive ways of writing it down hide nasty traps. Worse, none of our three sensors measures orientation directly. The gyro measures the *rate of change* of orientation, so to get orientation you must integrate – and integration quietly accumulates error until your estimate slowly walks away from the truth. The accelerometer and magnetometer, by contrast, give you *absolute* references (down, and roughly north) but they are noisy and, for the accelerometer, corrupted by every real motion. Neither sensor alone is trustworthy. The art of combining them so that each covers the other's weakness is called *sensor fusion*, and it is one of the most satisfying applications of the ideas in the companion linear-algebra and probability books: orientation as geometry, fusion as optimal estimation under noise.

We will build the picture from the ground up. First, how do you even *represent* a rotation – Euler angles and their gimbal-lock trap, rotation matrices, and the quaternion that practitioners actually use? Then, why does the gyro drift and why are the accelerometer and magnetometer "absolute but noisy"? Then the central trick: the *complementary filter*, which is just a high-pass on the gyro plus a low-pass on the accelerometer, fused in the frequency domain you already understand. We will derive the *Kalman filter* intuitively as the optimal version of that same idea, and meet the *Madgwick* and *Mahony* filters that run on real microcontrollers. We close, as always, with the wrist tracker: how to get a stable wrist orientation, and a clear accounting of what orientation can and cannot tell you about the hand.

36.1 What orientation is, geometrically

Fix two coordinate frames. One is glued to the world: call it the *navigation frame* or *Earth frame*, with axes pointing, say, North, East, and Down (the NED convention). The other is glued to the sensor chip and tumbles with it: the *body frame*, with axes along the chip's own x, y, z. *Orientation* is the answer to a single question: how is the body frame rotated relative to the world frame? Every way of writing down orientation is just a different bookkeeping for that one rotation.

Intuition

Hold your phone flat on the table, screen up. Body frame and world frame agree – zero rotation. Now tip the phone up to read it: you have rotated the body frame about a horizontal axis. Spin in your chair while holding it: another rotation, about vertical. Orientation is the *total* rotation that carries the world axes onto the body axes. The point of this chapter is that "a rotation" is a single geometric object, and Euler angles, matrices, and quaternions are three languages for naming the same object.

A crucial fact, easy to forget, is that rotations do *not commute*. Tip then spin lands you somewhere different from spin then tip. Try it with the phone: the final orientations genuinely differ. This non-commutativity is the deep reason rotations are subtle, and it is exactly what gimbal lock and quaternion algebra are wrestling with.

36.1.1 Euler angles: three turns in a row

The most human way to describe an orientation is as a sequence of three simple turns about axes: *roll* ϕ about the forward x-axis, *pitch* θ about the side y-axis, and *yaw* ψ about the down z-axis. Aircraft people have used these names for a century: roll dips a wingtip, pitch raises the nose, yaw swings the nose left or right. Any orientation can be reached by some (ϕ, θ, ψ).

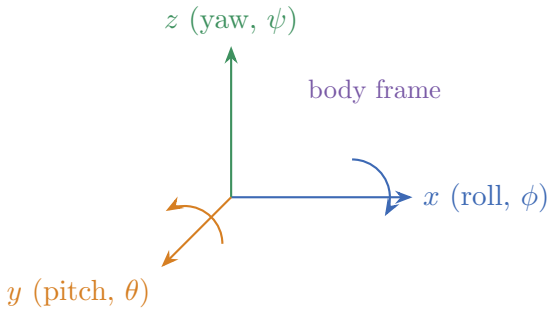


Figure 36.1: Euler angles describe orientation as three successive turns: roll about the forward axis, pitch about the side axis, yaw about the vertical axis. Intuitive, but the order matters and the scheme has a singularity (Section 36.1.2).

Euler angles are wonderful for talking to humans – "pitch up thirty degrees" is instantly clear – and they are how we will report the final answer. But three numbers hide three problems. First, the *order* matters (roll-pitch-yaw $\neq$ yaw-pitch-roll), so you must commit to a convention. Second, the angles *wrap*: 359° and -1° are the same heading, so naive averaging or differencing goes wrong near the wrap. Third, and most dangerous, is gimbal lock.

36.1.2 The gimbal-lock trap

Common trap

When pitch reaches $\pm 90°$, the roll axis and the yaw axis line up and become the *same* physical axis. You have lost a degree of freedom: two of your three numbers now do the same thing, and the third has become undefined. This is *gimbal lock*. Near it, tiny real motions demand huge, fast changes in the Euler angles, so any estimator that stores orientation as Euler angles becomes numerically violent there.

Intuition

Picture a camera on a mechanical gimbal with three nested rings: outer ring yaws, middle ring pitches, inner ring rolls. Pitch the middle ring up to $90°$ so the camera stares at the sky. Now the inner (roll) ring and the outer (yaw) ring spin about the *same* vertical line. Spinning either one just rotates the picture; you can no longer tip the camera sideways without moving the locked middle ring. A whole direction of motion has vanished from your control. The Apollo guidance computer had a real gimbal-locked inertial platform; astronauts were warned to avoid certain attitudes, and mission control joked grimly about "going to gimbal lock."

Gimbal lock is not a bug you can patch out of Euler angles; it is a topological fact. There is no way to cover the full space of 3-D rotations with three smooth angle coordinates without a singularity somewhere – the same reason every flat map of the globe distorts the poles. The fix is to stop forcing orientation into three numbers. We move to representations that are *global* and *smooth*: rotation matrices (nine numbers, six constraints) and quaternions (four numbers, one constraint).

36.1.3 Rotation matrices

A rotation in three dimensions is a linear map that preserves lengths and angles and keeps the handedness of space. In the language of the linear-algebra companion, it is an orthogonal matrix with determinant $+1$ – an element of the group $SO(3)$. Write $\mathbf{R}$ for the matrix that takes a vector's coordinates in the body frame and gives its coordinates in the world frame:

$$\mathbf{v}_{\text{world}} = \mathbf{R}\,\mathbf{v}_{\text{body}}, \qquad \mathbf{R}^{\top}\mathbf{R} = \mathbf{I}, \qquad \det \mathbf{R} = +1.$$

The columns of $\mathbf{R}$ are the body axes expressed in world coordinates – so $\mathbf{R}$ literally *is* a picture of where the chip's axes are pointing. The single elementary rotations are the ones you would guess; for example a rotation by θ about the y-axis is

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix},$$

and the full Euler orientation is a product like $\mathbf{R} = \mathbf{R}_z(\psi)\,\mathbf{R}_y(\theta)\,\mathbf{R}_x(\phi)$. The non-commutativity of rotations is now obvious: matrix multiplication does not commute.

Why it works

Rotation matrices never gimbal-lock and compose by plain multiplication, which makes them excellent for *computing*. Their drawback is redundancy: nine numbers for three degrees of freedom, tied by six constraints ($\mathbf{R}^\top\mathbf{R} = \mathbf{I}$). When you integrate a matrix forward in time with finite-precision arithmetic, those constraints slowly rot – the columns drift out of orthonormality – so you must periodically *re-orthonormalize*. Quaternions carry only one constraint and are cheaper to keep clean, which is why they win in practice.

36.1.4 Quaternions: the practitioner's choice

The quaternion is the representation almost every real attitude filter uses internally. It is a four-number object $q = (q_w, q_x, q_y, q_z)$, often written $q = q_w + q_x\mathrm{j}_1 + q_y\mathrm{j}_2 + q_z\mathrm{j}_3$ with three imaginary units. The geometry behind the algebra is clean and is the only part you must *feel*:

Intuition

Euler's rotation theorem. Any orientation, however complicated, equals a single rotation by some angle α about some fixed axis $\hat{\mathbf{n}}$ (a unit vector). A unit quaternion packs exactly that axis and angle into four numbers:

$$q = \Big(\cos\tfrac{\alpha}{2},\quad \hat{n}_x \sin\tfrac{\alpha}{2},\quad \hat{n}_y \sin\tfrac{\alpha}{2},\quad \hat{n}_z \sin\tfrac{\alpha}{2}\Big).$$

The scalar part remembers *how much* you turned; the vector part remembers *about what axis*. The half-angle is the famous quirk – a full 360° turn gives $-q$ and a 720° turn returns to the start. For our purposes you only need: unit quaternions name rotations, and there are no singularities.

Three facts let you use quaternions without drowning in algebra. (1) A *unit* quaternion, $\|q\| = 1$, represents a rotation; you renormalize after each update to keep it on the unit sphere – one constraint, cheap. (2) Rotations *compose* by the quaternion product $q_2 \otimes q_1$ (do q_1 then q_2), which is just a fixed bilinear formula. (3) To rotate a vector $\mathbf{v}$, sandwich it: $\mathbf{v}' = q \otimes \mathbf{v} \otimes q^{-1}$. The quaternion product is non-commutative, exactly mirroring the non-commutativity of rotations.

Formal

The Hamilton product of $a = (a_w, \mathbf{a})$ and $b = (b_w, \mathbf{b})$ (scalar part, vector part) is

$$a \otimes b = \big(a_w b_w - \mathbf{a}\cdot\mathbf{b},\quad a_w\mathbf{b} + b_w\mathbf{a} + \mathbf{a}\times\mathbf{b}\big).$$

For a unit quaternion the inverse is the conjugate $q^{-1} = q^* = (q_w, -\mathbf{q})$. The rotation matrix equivalent to q is

$$\mathbf{R}(q) = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_wq_z) & 2(q_xq_z + q_wq_y) \\ 2(q_xq_y + q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_wq_x) \\ 2(q_xq_z - q_wq_y) & 2(q_yq_z + q_wq_x) & 1-2(q_x^2+q_y^2) \end{bmatrix}.$$

You convert to Euler angles only at the very end, for display, and you keep the quaternion as the working state.

History

Quaternions were carved by William Rowan Hamilton into the stone of Broom Bridge in Dublin on 16 October 1843, in a flash of insight while walking with his wife: the relation $\mathrm{j}_1^2 = \mathrm{j}_2^2 = \mathrm{j}_3^2 = \mathrm{j}_1\mathrm{j}_2\mathrm{j}_3 = -1$. For decades they were a mathematical curiosity, eclipsed by vector calculus. Their revenge came with the space age and computer graphics, where their freedom from gimbal lock and their cheap, stable composition made them indispensable. Every spacecraft, drone, and VR headset today tracks attitude with quaternions.

Connection

This is the linear-algebra companion made physical. Rotation matrices are the orthogonal matrices ($\mathbf{R}^\top\mathbf{R} = \mathbf{I}$) from the chapter on orthonormal bases; their columns are an orthonormal basis – the body axes. Quaternions are the unit-norm elements of a 4-D algebra; "rotate by sandwiching" is a conjugation, the same pattern as a change of basis $\mathbf{PAP}^{-1}$. Orientation lives on a curved manifold ($SO(3)$), which is precisely why no flat three-number chart (Euler angles) can cover it without a singularity.

36.2 Why each sensor is not enough alone

We now have a language for orientation. Where do the numbers come from? Each sensor offers a partial, flawed view. Fusion is worth doing only because the flaws are *complementary*.

36.2.1 The gyroscope: smooth but drifting

A gyroscope measures *angular rate* $\boldsymbol{\omega}(t)$ – how fast the body is rotating, in rad/s, about each body axis. To get orientation you integrate. In quaternion form the kinematics are a clean differential equation,

$$\dot{q} = \tfrac{1}{2}\, q \otimes (0,\ \boldsymbol{\omega}),$$

which in a microcontroller becomes the discrete step $q_k = q_{k-1} \otimes \Delta q(\boldsymbol{\omega}_k, \Delta t)$, renormalized. Integration is the gyro's gift and its curse. Because it accumulates rate, the gyro is remarkably *smooth* and responsive over short times – it tracks fast wrist flicks accurately. But it has no idea where "down" is, and every small error in the rate accumulates.

What is really happening

Two things make integrated gyro orientation walk away from the truth. The big one is *bias*: a real gyro reads a small nonzero rate even when perfectly still (recall the calibration chapter). Integrating a constant bias b gives an angle error that grows *linearly*, $b \cdot t$ – a steady drift. The second is *random-walk*: broadband noise on the rate, once integrated, performs a random walk whose spread grows like $\sqrt{t}$. Bias dominates after a few seconds. A cheap MEMS gyro might drift a degree or more per second – tolerable for a one-second gesture, ruinous over a minute.

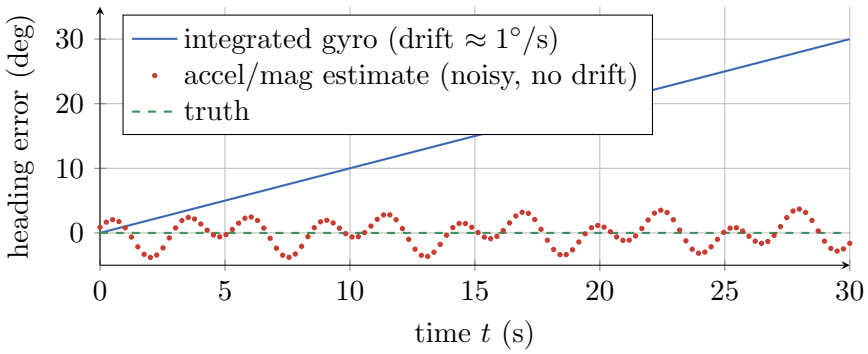


Figure 36.2: The complementary flaws. The integrated gyro (blue) is smooth but ramps away from truth as bias accumulates. The accelerometer/magnetometer estimate (red dots) is jittery but stays centered on truth – no drift. Fusion keeps the gyro's smoothness at short times and the accel/mag's long-term anchoring.

36.2.2 Accelerometer and magnetometer: noisy but absolute

A stationary accelerometer measures the reaction to gravity: it reports an upward $1g$ specific force, which points along world "up" regardless of how long you have been running. That gives you two of the three orientation angles directly – *roll* and *pitch* (the tilt of the body relative to gravity) – as

$$\phi = \operatorname{atan2}(a_y,\, a_z), \qquad \theta = \operatorname{atan2}\big(-a_x,\, \sqrt{a_y^2 + a_z^2}\big).$$

These are *absolute*: they do not drift, because gravity does not drift. The magnetometer plays the same role for the remaining angle, *yaw* (heading): the Earth's field points roughly toward magnetic north, giving an absolute compass bearing once you have tilt-compensated it using the accel-derived roll and pitch.

Common trap

The accelerometer measures gravity *plus all other acceleration.* The moment the wrist actually moves – and a gesture is nothing but motion – the accel vector is gravity contaminated by the motion you are trying to track. So the accel's "down" is only trustworthy when the device is nearly still, or on average over time. Likewise the magnetometer is wrecked by nearby steel, motors, speakers, and the very electronics on your board (hard- and soft-iron distortion from the calibration chapter). Both absolute references are *noisy and intermittently wrong*; only the gyro is reliable moment to moment.

The pattern is now sharp, and it is a frequency-domain pattern, which means we already own the tools to exploit it:

- The **gyro** is trustworthy at *high* frequencies (fast changes) and untrustworthy at *low* frequencies / DC (slow drift, bias).
- The **accel/mag** are trustworthy at *low* frequencies (the long-term average really is down/north) and untrustworthy at *high* frequencies (motion and noise).

Two signals that are each good in opposite halves of the spectrum – and we know exactly how to glue those: a high-pass on one, a low-pass on the other, added together so the filters sum to unity.

That is the complementary filter.

36.3 The complementary filter

Why it works

Let θ_g be the orientation you get by integrating the gyro and θ_a the orientation you get from the accelerometer (and magnetometer). Choose a high-pass $H(s)$ and a low-pass $L(s)$ that are *complementary*: $H(s) + L(s) = 1$ for every frequency. Then the fused estimate

$$\hat{\theta} = H(s)\,\theta_g + L(s)\,\theta_a$$

keeps the gyro's fast detail (high-pass kills the gyro's low-frequency drift) and the accel/mag's slow truth (low-pass kills their high-frequency motion noise), and because $H + L = 1$ a steady true orientation passes through with unit gain and no distortion. You are not throwing away either sensor; you are using each only where it is reliable.

The discrete-time form is one line of code and is what runs on a real board. With a single tuning constant α close to 1,

$$\boxed{\hat{\theta}_k = \alpha\left(\hat{\theta}_{k-1} + \omega_k\,\Delta t\right) \;+\; (1-\alpha)\,\theta_{a,k}}$$

Read it geometrically: *predict* the new angle by stepping the previous estimate forward with the gyro ($\hat{\theta}_{k-1} + \omega_k \Delta t$), then nudge that prediction a little ($1-\alpha$ of the way) toward what the accelerometer says. The α term is the high-pass on the gyro; the $(1-\alpha)$ term is the low-pass on the accel; they sum to 1, so it is exactly the complementary filter in disguise (Figure 36.3).

Intuition

The cutoff between "trust the gyro" and "trust the accel" is set by α and the sample time through the time constant $\tau = \dfrac{\alpha\,\Delta t}{1-\alpha}$. Above $1/\tau$ the gyro wins; below it the accel wins. With $\Delta t = 0.01\,\text{s}$ (a 100 Hz loop) and $\alpha = 0.98$, $\tau \approx 0.49\,\text{s}$: trust the gyro for changes faster than about half a second, trust the accel for anything slower. Want to lean harder on the gyro (steadier, but drift creeps in)? Push α toward 1.

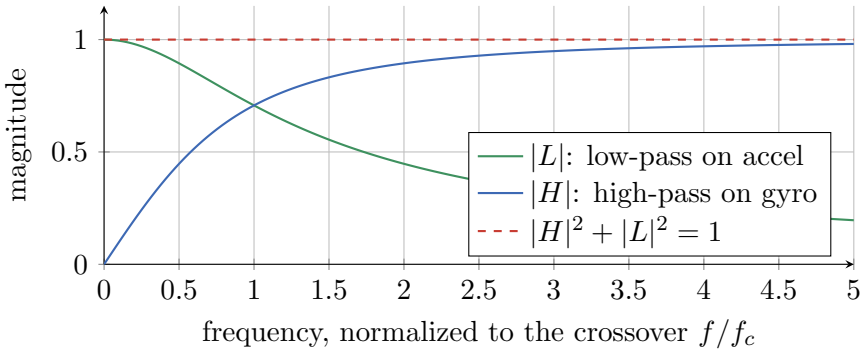


Figure 36.3: The complementary pair in the frequency domain. Below the crossover f_c the low-pass (green) lets the accelerometer's slow, drift-free truth through; above it the high-pass (blue) lets the gyro's fast detail through. The branches are designed so their powers sum to one at every frequency (red dashed), so a steady true orientation passes with unit gain and no notch. Sliding α slides f_c left or right.

Listing 36.1: Complementary filter for tilt from a 6-axis IMU.

```python
import numpy as np

def complementary_tilt(gyro, accel, dt, alpha=0.98):
    """gyro: (N,3) rad/s, accel: (N,3) g. Returns roll,pitch (deg)."""
    roll = pitch = 0.0
    out = []
    for w, a in zip(gyro, accel):
        # accel-only absolute tilt (good when nearly still)
        roll_a  = np.degrees(np.arctan2(a[1], a[2]))
        pitch_a = np.degrees(np.arctan2(-a[0], np.hypot(a[1], a[2])))
        # gyro integration (good for fast motion), then blend
        roll  = alpha*(roll  + np.degrees(w[0])*dt) + (1-alpha)*roll_a
        pitch = alpha*(pitch + np.degrees(w[1])*dt) + (1-alpha)*pitch_a
        out.append((roll, pitch))
    return np.array(out)
```

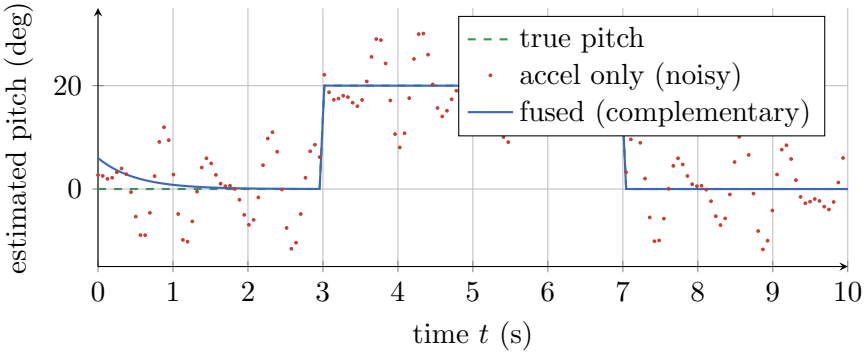


Figure 36.4: A pitch step. Accel alone (red) is centered but jittery; integrated gyro alone would track the step then drift. The complementary fusion (blue) follows the step crisply and settles onto the true value with little noise.

Application

The complementary filter is everywhere because it is so cheap: self-balancing robots and hoverboards, the early flight controllers of consumer drones (Multiwii, early ArduPilot), camera gimbals, and countless hobby projects. On an 8-bit Arduino it is a few multiplies per axis per loop. For two-axis tilt it is genuinely hard to beat for the effort, and it is the right first thing to put on your wrist board.

Connection

Look again at $\hat{\theta} = H(s)\theta_g + L(s)\theta_a$ with $H + L = 1$. This is exactly the complementary high-pass/low-pass pair from the filter-design chapters, and it is the *static* (constant-gain) ancestor of the Kalman filter, which lets the blend weights *vary in time* to be optimal given how noisy each sensor currently is.

36.4 The Kalman filter as optimal fusion

The complementary filter blends with a *fixed* weight α. But the right weight is not fixed: when the device is whipping around, the accel is mostly motion and should be trusted less; when it sits still, the accel is pure gravity and should be trusted more. The Kalman filter is the principled answer to "what is the *optimal*, time-varying blend, given everything I know about the noise?" It is the complementary filter with the weight recomputed every step from the statistics.

Intuition

The Kalman filter keeps two things at all times: a best *estimate* of the state (here, orientation, maybe plus gyro bias) and a *covariance* – a number, or matrix, saying how unsure it is. Each step has two beats. **Predict**: push the estimate forward with the gyro, and *grow* the uncertainty (we trusted a noisy gyro, so we know less now). **Update**: when an absolute measurement arrives (accel/mag), compare it to the prediction and step partway toward it – and *shrink* the uncertainty. How far you step is the *Kalman gain*, computed from the relative sizes of your current uncertainty and the measurement's noise. It is a self-tuning α.

Here is the whole idea in one dimension, which is enough to feel it. Let the state be a scalar x (an angle) with estimate $\hat{x}$ and variance P. The gyro gives a control input $u = \omega\Delta t$ with process noise variance Q. The accel gives a measurement z of the angle with measurement noise variance R.

Formal

Predict.

$$\hat{x}^- = \hat{x} + u, \qquad P^- = P + Q.$$

Update (Kalman gain K, then correct):

$$K = \frac{P^-}{P^- + R}, \qquad \hat{x} = \hat{x}^- + K\,(z - \hat{x}^-), \qquad P = (1 - K)\,P^-.$$

Stare at the gain. $K = P^-/(P^- + R)$ lives between 0 and 1. If the measurement is very noisy (R large), $K \to 0$: ignore the accel, coast on the gyro. If our prediction is very uncertain (P^-

large), $K \to 1$: snap to the accel. The correction $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$ is *identical in form* to the complementary filter's $(1-\alpha)$ nudge toward the measurement – but now the weight K breathes with the changing uncertainty.

Why it works

Why is this the *optimal* blend, not just a reasonable one? If both noises are Gaussian, "predict" and "measure" are two independent Gaussian opinions about the same angle. The product of two Gaussians is another Gaussian whose mean is the *inverse-variance-weighted average* of the two means – each opinion weighted by its certainty. Working that average out gives exactly $K = P^-/(P^- + R)$. The Kalman filter is nothing more than Bayes' rule for Gaussians, applied once per sample. That is the deep link to the probability companion: the gain is the precision-weighting of two beliefs.

Connection

Everything here is straight from the probability and linear-algebra books. The variance P is the spread of a belief; "predict grows P, update shrinks P" is uncertainty propagation. The product-of-Gaussians giving a precision-weighted mean is Bayesian updating with conjugate Gaussians. In full 3-D the scalars become vectors and matrices, $z - \hat{x}^-$ becomes the *innovation*, and K becomes a matrix – but the story is unchanged: optimal fusion is inverse-variance weighting.

History

Rudolf Kalman published the filter in 1960. Its first triumph was the Apollo program: the guidance computer used a Kalman filter to fuse inertial measurements with star and radar fixes, and it is no exaggeration that it helped get astronauts to the Moon and back. Today it runs in GPS receivers, missiles, weather models, robot localization, and the orientation chip in your phone. Stanley Schmidt at NASA Ames recognized its power and pushed it into Apollo; the nonlinear variant is sometimes called the Schmidt–Kalman filter.

Common trap

Orientation is nonlinear (the quaternion kinematics, the trig in the accel model), so a plain linear Kalman filter does not directly apply. Practitioners use the *extended* Kalman filter (EKF), which linearizes the model each step about the current estimate, or the *unscented* Kalman filter (UKF). These work well but are fiddly: you must supply Jacobians (EKF), tune the noise matrices Q and R, and guard against divergence if the linearization is poor during violent motion. This fiddliness is exactly why the lighter Madgwick and Mahony filters became popular.

36.5 The Madgwick and Mahony filters

A full quaternion EKF carries a 4×4 (or larger) covariance matrix and its matrix algebra – heavy for a tiny microcontroller. Around 2010 Sebastian Madgwick, then a PhD student at Bristol, published a filter that achieves EKF-grade accuracy with a fraction of the computation and *one* intuitive tuning

knob. It, and the closely related Mahony filter, are what most open-source IMU libraries actually ship.

Intuition

Both filters share one elegant idea, and it is a *gradient-descent* idea you already know from the calculus companion. Integrate the gyro to predict the quaternion. Now ask: given my predicted orientation, where *should* the accelerometer point if the only force were gravity? Compare that predicted gravity direction to the measured accel. The mismatch is an *error*, and you can compute which way to nudge the quaternion to shrink that error fastest – the gradient. Take a small step downhill. That small correction, blended with the gyro integration, is the whole filter: gyro for speed, a gradient nudge toward gravity (and north, with a magnetometer) for the absolute anchor.

Why it works

Madgwick frames fusion as an optimization: find the quaternion q that best aligns the *predicted* field directions (gravity, and magnetic north) with the *measured* accel and mag vectors. That is a least-squares problem,

$$\min_{q} \| f(q,\ \text{measured fields}) \|^2,$$

and one step of gradient descent on it is cheap to write in closed form. The single knob β sets the descent step size – the analog of the complementary α and the Kalman K. Small β: trust the gyro more (smooth, slight drift). Large β: pull harder toward accel/mag (drift-free, more noise). Mahony instead drives the same mismatch through a PI controller and, as a bonus, its integral term *estimates and removes the gyro bias* – a real practical win.

Focus

The unifying picture (Figure 36.5): *every* attitude filter in this chapter is the same skeleton.

1. **Predict** orientation by integrating the gyro (fast, drifts).
2. Measure how far the prediction's gravity/north disagree with the accel/mag.
3. **Correct** by nudging the prediction toward the absolute references.

They differ only in *how* they choose the correction strength:

- Complementary: fixed weight α.
- Kalman/EKF: optimal time-varying weight K from the noise statistics.
- Madgwick: gradient-descent step of size β.
- Mahony: PI feedback with gain pair, plus online bias estimation.

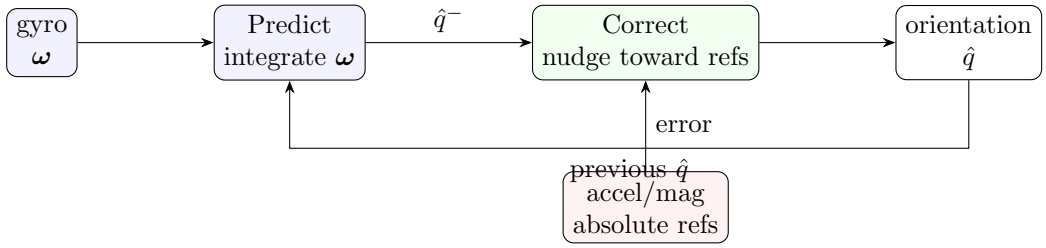


Figure 36.5: Every attitude filter in this chapter is this loop. The gyro drives a *predict* step (fast, drifts); the accel/mag supply an absolute reference that the *correct* step nudges toward. The filters differ only in the correction strength: fixed α (complementary), time-varying K (Kalman), gradient step β (Madgwick), or PI gains (Mahony).

Table 36.1: Choosing an attitude filter.

Filter	State	Cost	Tuning	Best for
Complementary	Euler/2 angles	tiny	α	tilt only, 8-bit MCUs
Madgwick	quaternion	low	β	full 9-DoF on MCU
Mahony	quaternion	low	K_p, K_i	like Madgwick, removes bias
EKF	quat (+ bias)	high	Q, R matrices	best accuracy, more compute

Example 36.1 (One Kalman step by hand)**.** A 1-D angle filter has $\hat{x} = 10.0°$, $P = 4.0$ (deg^2). The gyro says the angle rose by $u = 2.0°$ this step with process noise $Q = 1.0$. The accel measures $z = 13.5°$ with noise $R = 9.0$. *Predict:* $\hat{x}^- = 12.0$, $P^- = 5.0$. *Gain:* $K = 5.0/(5.0 + 9.0) = 0.357$. *Update:* $\hat{x} = 12.0 + 0.357\,(13.5 - 12.0) = 12.54°$, and $P = (1 - 0.357) \cdot 5.0 = 3.21$. The estimate moved only a third of the way toward the noisy accel (because $R > P^-$), and our uncertainty dropped from 5.0 to 3.21. Run it again with the device held still and P keeps shrinking; whip it around and Q keeps P large so the filter leans on the accel. That breathing weight is the whole point.

36.6 Getting a stable wrist orientation

Toward the wrist tracker

For the wrist controller, here is the recipe, in increasing order of effort.

1. **Tilt mapping (start here).** Run a complementary filter on the 6-axis IMU for roll and pitch only. Map wrist tilt to two continuous MIDI controls – e.g. pitch up/down to a pitch-bend, roll to a mod wheel. This is rock-solid, needs no magnetometer, and never drifts because gravity anchors it.
2. **Full orientation.** Add the magnetometer and run Madgwick or Mahony to get a drift-resistant quaternion, hence heading too. Now "which way the wrist points" becomes a usable control, e.g. left/right pointing selects an instrument.
3. **Bias and stillness.** Use Mahony's bias estimate (or a startup gyro-bias calibration while the arm rests) so the heading does not creep when the wrist is held still between notes.

Build it (hardware)

A workable loop on an ESP32 or Teensy with an MPU-9250 / ICM-20948:

- **Rate.** Run the filter at 100–200 Hz. Gyro integration error grows with Δt, so a steady, fast loop matters more than raw sensor quality.
- **Calibrate first.** Apply the gyro-bias, accel, and (especially) magnetometer hard/soft-iron calibration from the previous chapter *before* fusion. The strongest magnetic distortion is your own board – mount the IMU away from the battery, motors, and high-current traces.
- **Startup.** Hold the arm still for one second at boot; average the gyro to seed the bias and the accel to seed the initial quaternion. Tune β (or α) to taste: too low feels sluggish and drifts, too high feels twitchy.
- **Report.** Keep the quaternion as the working state; convert to Euler only at the moment you emit a MIDI value, and watch the $\pm 90°$ pitch singularity when you do.

Listing 36.2:).]Mahony-style correction core (per loop, quaternion q[4]).

```
// gx,gy,gz in rad/s; ax,ay,az normalized accel. Kp,Ki tuning gains.
// estimated direction of gravity from current quaternion:
float vx = 2*(q[1]*q[3] - q[0]*q[2]);
float vy = 2*(q[0]*q[1] + q[2]*q[3]);
float vz = q[0]*q[0] - q[1]*q[1] - q[2]*q[2] + q[3]*q[3];
// error = measured accel x estimated gravity (cross product):
float ex = (ay*vz - az*vy);
float ey = (az*vx - ax*vz);
float ez = (ax*vy - ay*vx);
ix += Ki*ex*dt; iy += Ki*ey*dt; iz += Ki*ez*dt;   // integral -> bias removal
gx += Kp*ex + ix;  gy += Kp*ey + iy;  gz += Kp*ez + iz;  // corrected rates
// integrate quaternion with corrected rates, then normalize q (omitted)
```

Common trap

What wrist orientation does, and where the fingers come from. A wrist- or forearm-worn IMU recovers the *orientation of the forearm/wrist segment* – tilt and heading – and discrete whole-hand gestures (a flick, a twist, a tap shock seen as an accel spike). That alone gives the instrument several expressive axes. What the IMU *cannot* give you is which *finger* moved: the IMU is bolted to the wrist, and finger motion barely jostles it. But the fingers are far from out of reach. Each finger is driven by tendons that run under the wrist and by forearm muscles whose electrical activity reaches the skin, so finger movement is genuinely detectable from a band on the wrist or forearm – through *force myography* (FMG, the tendon bulge under a band of force sensors) or *surface EMG* (the muscle's electrical pulses). With a short per-finger calibration for each user, those signatures separate cleanly enough to drive notes. That is the subject of the next chapter and the capstone. The one thing that stays out of reach from the wrist is the *continuous joint angle* of each finger – how far each knuckle is bent at every instant. That information is not present in wrist bulk motion and needs sensors *on the fingers* (instrumented rings); see the upgrade path in the capstone. Everything our instrument actually needs – which finger, how hard, when – is on the doable side.

Summary

Orientation is a single geometric object – a rotation carrying the world frame onto the body frame – and choosing how to write it down matters. Euler angles are intuitive but suffer gimbal lock; rotation matrices and quaternions are global and smooth, and the unit quaternion (axis-and-half-angle in four numbers) is what real filters carry. No single sensor gives orientation: the gyro is smooth but drifts (bias integrates to a linear error), while the accelerometer and magnetometer are absolute but noisy and motion-corrupted. Their flaws are complementary in frequency, so we fuse them. The complementary filter is a high-pass on the gyro plus a low-pass on the accel with a fixed weight; the Kalman filter makes that weight optimal and time-varying via inverse-variance (Bayesian) weighting; Madgwick and Mahony reach the same goal with a cheap gradient or PI correction toward gravity and north, Mahony also estimating gyro bias. For the wrist controller, start with complementary tilt, graduate to Madgwick/Mahony for full orientation – and remember that the wrist IMU sees forearm pose and whole-hand gestures. Which finger moved is recovered separately, from tendon (FMG) or muscle (EMG) sensing with per-user calibration; only the continuous per-joint finger angle stays out of reach of a wrist band.

36.7 Exercises

Exercise 36.1. Explain in your own words why three Euler angles cannot smoothly cover all orientations without a singularity, and identify the orientation(s) at which the roll-pitch-yaw convention gimbal-locks. What physical degree of freedom is lost there?

Exercise 36.2. A gyro has a constant bias of $0.5°/\text{s}$ and otherwise perfect output. Starting from a known orientation, compute the heading error after 1, 10, and 60 seconds of pure integration. Now suppose the bias is zero but there is white rate noise; explain why the error grows like $\sqrt{t}$ instead of t.

Exercise 36.3. Show that the discrete complementary filter $\hat{\theta}_k = \alpha(\hat{\theta}_{k-1} + \omega_k \Delta t) + (1-\alpha)\theta_{a,k}$ is a first-order high-pass on the gyro plus a first-order low-pass on the accel. Find the crossover time constant τ and evaluate it for $\alpha = 0.95$, $\Delta t = 0.005\,\text{s}$.

Exercise 36.4. Why is the accelerometer trustworthy for orientation only when the device is nearly still or averaged over time? Write the measured accel as gravity plus motion and explain which part the low-pass branch of the complementary filter is trying to keep.

Exercise 36.5. Repeat Example 36.1 but with the device held still, so you receive three accel measurements $z = 13.5, 13.4, 13.6$ with no gyro motion ($u = 0$). Run three predict/update cycles and report how $\hat{x}$, P, and K evolve. What happens to K as P shrinks, and what does that mean physically?

Exercise 36.6. Derive the Kalman gain $K = P^-/(P^- + R)$ from the product-of-two-Gaussians fact: if the prediction is $\mathcal{N}(\hat{x}^-, P^-)$ and the measurement is $\mathcal{N}(z, R)$, show the posterior mean is the inverse-variance-weighted average and rewrite it in the form $\hat{x}^- + K(z - \hat{x}^-)$.

Exercise 36.7. Convert the unit quaternion $q = (\cos 30°, 0, 0, \sin 30°)$ to a rotation matrix using the formula in Section 36.1.4. What axis and angle of rotation does it represent? Verify the matrix is orthogonal with determinant $+1$.

Exercise 36.8. Both Madgwick and Mahony correct the gyro prediction toward gravity. In the C sketch, the error is computed as the *cross product* of measured accel and estimated gravity. Explain geometrically why a cross product is the right error signal – what its direction and magnitude tell you about the misalignment, and why it vanishes when the two vectors agree.

Exercise 36.9. Tune by experiment: implement the complementary filter and try $\alpha \in \{0.90, 0.98, 0.999\}$ on the same recorded wrist motion. Describe qualitatively the trade-off you see between responsiveness, noise, and drift, and state which you would pick for (a) a fast percussive trigger and (b) a slow expressive filter sweep.

Exercise 36.10. Limits question. A friend proposes selling a single wristband that reconstructs every finger *joint angle* in real time from one 9-axis IMU. Explain precisely why a wrist IMU cannot recover continuous joint angles, and what the IMU *can* measure (forearm pose, whole-hand gestures). Then explain which sensing modality you would add to the same wrist band – and why placing it there works – to detect *which finger moved* without putting any hardware on the fingers themselves. Finally, state what you would have to add on the fingers if you genuinely needed per-joint angles.

Chapter 37

Sensing the Hand from the Wrist: EMG, Force Myography, and Bio-Acoustics

Everything so far in this part of the book has been about motion. An accelerometer feels the tug of gravity and the shove of acceleration; a gyroscope feels rotation; a fused orientation estimate tells us how the forearm is tilted and turned in space. That is genuinely useful, and we will lean on it hard. But the dream of a wrist-worn musical controller – playing notes by moving your fingers – needs more than bulk motion, and this chapter is where we find the channels that carry finger information to the wrist.

Focus

Two facts frame this chapter, stated plainly before any math.

Finger movement is detectable from a wrist or forearm band. Curl a finger and the tendon that drives it slides and the muscle that pulls it bulges, right there under the strap. A force-myography (FMG) band feels that bulge; surface EMG taps the electrical pulse that commands it. With a short per-finger calibration – you perform each movement a few times and the device records its signature – live detection becomes a matter of matching the current signal to your own calibrated templates. This is achievable on buildable hardware.

What the wrist cannot give you is continuous per-joint finger angles. A wrist-worn IMU alone cannot recover them, and neither can FMG or EMG: the full 20-dimensional hand pose is simply not present in wrist-surface signals. That target needs sensors on the fingers themselves (rings). We will be clear about which side of this line each goal falls on, because everything the instrument actually needs – triggering notes by finger movement – is on the doable side.

The reason to confront this now, rather than at the capstone, is that the choice of sensing modality reshapes *everything* downstream: the sampling rate you need, the filters you design, the features you extract, the per-finger calibration routine, even the geometry of the band you strap on. The chapter lays out what each channel senses, what it costs, how hard it is to build, and how a short calibration turns each one into a finger detector.

37.1 Why the wrist is a hard place to listen from

Hold your right wrist with your left hand and wiggle your fingers. Feel what moves. The tendons that flex and extend your fingers run through the wrist like cables through a conduit, and the muscles that pull those tendons sit mostly in the *forearm*, not the hand. The wrist is a privileged listening post: a great deal of finger information passes through it. The work is reading the *form* that information takes by the time it reaches the surface.

Intuition

Think of the forearm as a bundle of perhaps a dozen independent ropes (tendons), each pulled by its own winch (muscle), all sheathed inside a flexible tube (skin, fat, fascia). From outside the tube you are trying to figure out which ropes are being pulled and how hard. An IMU is a sensor bolted to the *outside of the tube* that only feels the tube swinging around in space. It feels the whole tube move; it is deaf to the ropes sliding inside. To hear the ropes you must press on the tube (force myography), tap the electrical chatter of the winches (EMG), or listen to the rumble the ropes make as they slide (bio-acoustics).

This single picture organizes the whole chapter. Every modality below is a different answer to "how do I sense the ropes from outside the tube," and each answer trades off cost, skill, robustness, and how much finger detail it actually recovers.

37.1.1 Degrees of freedom: what you are trying to estimate

The human hand has roughly 21 to 27 kinematic degrees of freedom (DOF), depending on how you count: four joints per finger reduced by tendon coupling, the thumb's extra mobility, wrist flexion/extension and radial/ulnar deviation. A full "hand pose" is therefore a point in a space of about two dozen dimensions. That is the target if you want to reconstruct the hand for, say, a virtual keyboard where each finger is independent.

Formal

Let $\mathbf{q}(t) \in \mathbb{R}^D$ be the joint-angle vector, $D \approx 20$. A continuous hand-tracking system estimates the full trajectory $\mathbf{q}(t)$. A gesture detector instead reports only a *label* $g(t) \in \{1, \dots, K\}$ from a small set of K movements – finger 1 curled, finger 2 curled, pinch, rest – plus the moment of onset. These are different problems. The first is regression onto a 20-dimensional manifold; the second is deciding which of a handful of calibrated patterns the current signal matches. The disappointment in this field comes from quietly hoping a sensor that can do the second can do the first.

Keep this distinction front of mind: **continuous pose** ($\mathbf{q}(t)$, hard, wrist cannot do it) versus **discrete finger movement** (g, achievable from the wrist with per-finger calibration). When we say "the IMU cannot sense fingers," we mean it cannot do either from bulk motion. When we say "FMG and EMG sense fingers," we mean they reliably recover the discrete movement g – which finger moved – once the band has been calibrated to you.

37.2 The IMU ceiling: what motion sensing alone can and cannot do

Let us be precise about the wrist IMU rather than hand-waving. An IMU on the wrist measures the linear acceleration and angular velocity of the wrist segment. By the rigid-body kinematics of the previous chapters, that is exactly six numbers describing the rigid motion of one bone-and-skin assembly. Finger flexion barely moves the wrist segment; the tendons slide but the wrist bone's pose changes by an amount swamped by ordinary tremor and the IMU's own noise floor.

Why it works

Why can't clever processing extract fingers from the IMU anyway? Because the information is not there to extract. Mutual information cannot be created by a filter. The map from finger pose to wrist-IMU output is many-to-one and nearly constant: a huge range of finger configurations produce essentially the same wrist-segment acceleration. No amount of FFT, wavelet, or signal conditioning can invert a function that has thrown the input away. The IMU's blindness to fingers is an information-theoretic fact about the sensor's placement, not a software shortcoming.

So what *can* a wrist IMU do for the hand? Three things, each solid:

1. **Detect that a motion occurred.** A finger tap or a wrist flick produces a transient acceleration spike. You can reliably detect the *event*, even time it to a millisecond, without knowing which finger.

2. **Classify a few coarse whole-hand or wrist gestures.** Wrist flexion/extension, radial/ulnar deviation, supination/pronation, a sharp "flick," a knock – these move the wrist segment itself and are separable. Counts in the literature are modest: a handful to a dozen gestures.

3. **Provide context and orientation** that other modalities lean on: gravity direction, arm posture, gating "is the hand even moving."

Application

The published wrist-IMU gesture literature backs up exactly these limits. A 2023 *Sensors* (MDPI) study used a wrist IMU to *count* finger and wrist movements and detect their occurrence – counting and detection, not pose recovery. A 2020 *PLOS One* study reached roughly 75–80% accuracy on 14 gestures, but only by going *multimodal*: an IMU *plus* a contact microphone (acoustic) on an Arduino DUE, not the IMU alone. The pattern across the field is consistent: IMU-only systems handle a small gesture vocabulary; adding a finger-driven channel – acoustic, FMG, or EMG – is what unlocks finger-level distinctions. That is the edge our instrument will take: not one clever sensor, but several modest ones fused.

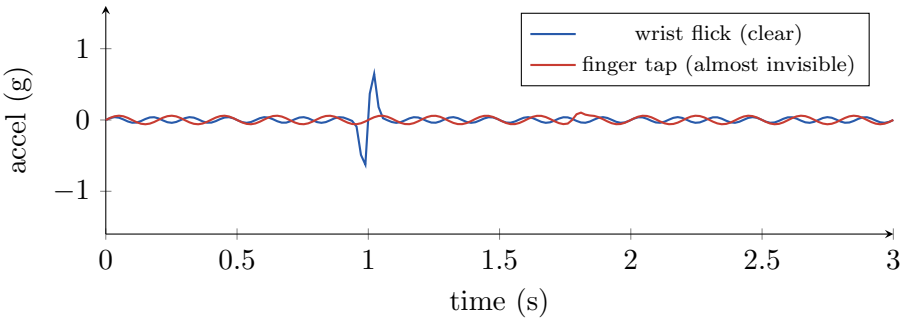


Figure 37.1: The IMU ceiling, drawn. A wrist flick (blue) drives a large, easily detected transient. A finger tap (red) barely lifts the wrist accelerometer above its own small-motion noise. The whole-hand event is legible; the finger event is not.

Toward the wrist tracker

For the wrist controller this fixes our architecture early. The IMU is the *conductor*, not the *pianist*. It will give us arm orientation (which way the hand points → pitch bend or instrument selection), motion gating (start/stop of a phrase), and a few robust wrist gestures (octave up/down, mode switch). The note-level finger triggering – if we want fingers at all – must come from a different sensor on the band. Designing as if the IMU could do fingers is the single most common way these projects fail.

37.3 Surface EMG: tapping the muscles' command lines

When your brain decides to flex a finger, motor neurons fire, and each firing makes a patch of muscle fibers (a *motor unit*) contract. Each contraction is preceded by a tiny electrical depolarization that propagates along the fibers: the motor unit action potential (MUAP). Surface electromyography (sEMG) places electrodes on the skin over the forearm muscles and measures the summed electrical field of many motor units firing – the *interference pattern*. It is, almost literally, an antenna held over the muscle's command bus.

What is really happening

What you are really recording in sEMG is not "muscle force" and certainly not "finger angle." You are recording the superposition of many spiky action potentials, each from a different motor unit, each firing at its own rate, low-pass filtered by the tissue between the muscle and the skin. The raw signal looks like noise – a zero-mean, roughly symmetric, rapidly fluctuating waveform – because it *is* a sum of many randomly-timed spikes. Its *amplitude envelope* grows with how hard and how many motor units are recruited. That envelope, not the raw wiggle, is what carries the useful information.

37.3.1 The signal and its spectrum

Surface EMG is small and fast. Amplitudes are on the order of tens of microvolts to a few millivolts. The useful energy spans roughly 20 to 450 Hz, peaking around 50–150 Hz. By the sampling theorem

of the Nyquist chapter, you must sample above twice the top frequency: 1000 Hz is a safe, common choice, 2000 Hz generous.

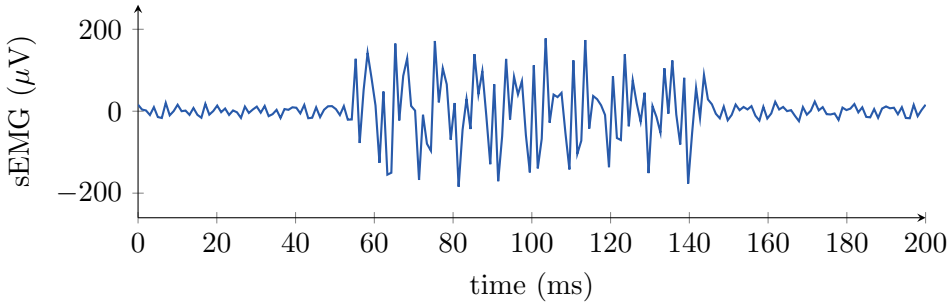


Figure 37.2: A schematic sEMG trace: near-baseline chatter, then a contraction burst (roughly 55–145 ms) where many motor units recruit and the amplitude envelope swells. The information is in the envelope, not in any single oscillation.

The signal-processing pipeline for sEMG is a textbook application of earlier chapters, which is why it makes such a satisfying capstone modality:

1. **Band-pass filter**, roughly 20–450 Hz, to remove motion-artifact drift below 20 Hz and high-frequency junk above the muscle band.
2. **Notch filter** at the mains frequency (50 or 60 Hz, plus harmonics) to kill power-line pickup – sEMG electrodes are excellent antennas for the wall.
3. **Rectify and smooth** to extract the amplitude envelope: take $|x[n]|$ or $x[n]^2$, then low-pass. This is the "linear envelope."
4. **Feature extraction** per short window (100–250 ms): mean absolute value (MAV), root-mean-square (RMS), waveform length, zero crossings, and the *median frequency* of the spectrum (which drops as muscles fatigue).
5. **Match** the per-window feature vector against the per-finger templates recorded during calibration – nearest template, or a correlation/threshold test – and emit the matching finger movement.

Formal

Two of the workhorse features in closed form. For a window of N samples,

$$\mathrm{MAV} = \frac{1}{N}\sum_{n=1}^{N}|x[n]|, \qquad \mathrm{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}x[n]^2}.$$

Both estimate the envelope amplitude. The *median frequency* f_{med} is the frequency that splits the power spectrum into two equal halves: $\sum_{f<f_{\mathrm{med}}} P(f) = \frac{1}{2}\sum_f P(f)$. It is prized because it tracks muscle fatigue (the spectrum shifts toward lower frequencies as fast fibers tire) while being insensitive to overall amplitude.

Connection

Notice how much of this is just the rest of the book reused. The band-pass and notch are filter design (the filters chapter). The envelope is rectification followed by low-pass – a nonlinearity plus convolution. Median frequency is read straight off the power spectral density from the spectral-estimation chapter. RMS is the ℓ_2 norm of the window divided by $\sqrt{N}$ – the vector length from the signals-as-vectors chapter. EMG is where the abstractions cash out.

37.3.2 What EMG achieves – and why calibration is the whole game

Within a single session on a single person, with electrodes that have not moved, sEMG finger detection is excellent: on a small set of finger movements (individual finger flexions, a fist, a pinch) within-subject accuracy reaches roughly 99%. That number is real, and it is exactly the regime our instrument runs in – one user, one strap-on, templates recorded minutes earlier. The catch is that the 99% lives on the *near* side of calibration, not the far side.

Common trap

The 99% figure is a per-user, per-session number. Three effects explain why, and each one points to the same fix – calibrate to the wearer:

- **Cross-subject collapse.** Take templates built on my arm and apply them to yours, and accuracy falls to 60–80% or worse. Forearm anatomy, fat thickness, and muscle geometry differ enough that one person's signatures do not fit another's. The cure is not a bigger model – it is a short enrollment on the actual wearer.
- **Electrode-shift sensitivity.** Rotate the band a centimeter, or put it on tomorrow, and the electrodes sit over different muscles. The templates from yesterday no longer line up. A fresh 30-second calibration on each donning restores it.
- **Discrete, not continuous.** Detecting *which* finger moved is the solved, reliable part. Continuous, simultaneous, proportional angle of multiple fingers from sEMG is still a research problem. Our instrument only needs the discrete part, so this limit costs us nothing.

The lesson is not "EMG is fragile." It is "EMG is a per-user instrument": measure the wearer once, and it performs.

History

The state of the art lives in two recognizable places. The *Myo armband* (Thalmic Labs, $\sim$ 2014) put eight dry sEMG electrodes plus an IMU on a consumer band; it was discontinued but seeded a generation of student projects and datasets. The *MyoWare* board (an open, single-channel sEMG front-end from Advancer Technologies / SparkFun) is the standard DIY entry point: it does the instrumentation-amp, rectification, and envelope in analog, handing your Arduino a clean envelope. At the high end, *CTRL-labs* – acquired by Meta in 2019 – pursued a high-density wrist sEMG band aiming at decoding individual motor-unit activity, with the explicit ambition of typing and fine control from the wrist. Meta has since publicly demonstrated a wrist sEMG wristband for neural input. The trajectory is real. The cross-user, no-calibration version is not yet a commodity – but the per-user, calibrate-then-play version,

which is all we need, is squarely within reach of a DIY build.

Build it (hardware)

A minimal DIY sEMG channel:

- One **MyoWare 2.0** sensor (or a classic MyoWare) over a forearm flexor belly, with the reference electrode on a bony, electrically-quiet spot (back of the wrist, elbow).
- Feed its `ENV` (envelope) output to an Arduino analog pin; or feed the `RAW` output if you want to do your own band-pass/notch/rectify in software.
- Sample RAW at $\geq 1000\,\text{Hz}$. The ENV pin is already low-frequency, so a few hundred Hz suffices.
- Expect to fight 50/60 Hz mains hum, motion artifacts when the wire tugs the electrode, and sweat changing skin impedance over a session.

A tiny software pipeline on the RAW pin:

```
import numpy as np
from scipy.signal import butter, sosfilt, iirnotch, tf2sos

fs = 1000.0
# 20-450 Hz band-pass (forearm sEMG band)
bp = butter(4, [20, 450], btype="band", fs=fs, output="sos")
# 60 Hz mains notch (use 50 Hz in Europe)
b, a = iirnotch(60.0, Q=30.0, fs=fs)
notch = tf2sos(b, a)

def emg_envelope(raw, win=200):
    x = sosfilt(bp, raw)           # band-pass
    x = sosfilt(notch, x)          # remove mains hum
    x = np.abs(x)                  # rectify
    k = np.ones(win) / win         # moving-average smoother
    return np.convolve(x, k, mode="same")   # linear envelope

def mav(window):  return np.mean(np.abs(window))
def rms(window):  return np.sqrt(np.mean(window**2))
```

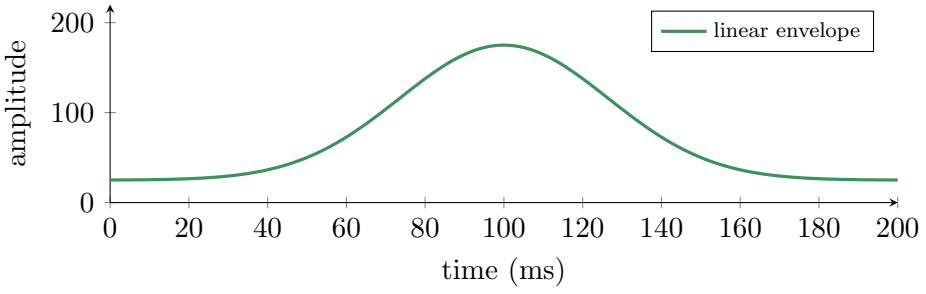


Figure 37.3: The same contraction after rectify-and-smooth. The noisy interference pattern of Figure 37.2 becomes a clean amplitude bump whose height and timing drive the template match. This is the curve a MyoWare ENV pin hands you in hardware.

37.4 Force myography: sensing the bulge, not the voltage

There is a humbler modality that, for a DIY builder, is often the *best* path to finger sensing: force myography (FMG), sometimes called pressure myography. When a forearm muscle contracts to pull a tendon, it changes shape – it bulges. The cross-section of the forearm physically deforms in a pattern that depends on which muscles are active. FMG places a ring of force-sensitive resistors (FSRs) around the wrist or forearm and measures how hard each one is being pushed.

Intuition

EMG eavesdrops on the *electrical command* to the muscle; FMG measures the *mechanical consequence*. Both ultimately report the same underlying event – a muscle contracting – but through completely different physics. The mechanical consequence is, refreshingly, easy to measure: a force-sensitive resistor is a few cents, its signal is large and slow, and it does not care about your skin's electrical impedance, your sweat, or the wall's 60 Hz hum. Where EMG is a delicate microvolt antenna, FMG is a robust "which buttons are being pushed" band.

37.4.1 The signal: slow, large, robust

Each FSR is a resistor whose resistance drops as you press on it. Put it in a voltage divider and you read a voltage that rises with force. Arrange N of them in a band and you get an N-dimensional vector $\mathbf{p}(t) = (p_1, \ldots, p_N)$ – a *pressure map* of the forearm's cross-section – updated maybe 50–200 times a second. That is far slower than EMG's kilohertz, because muscle *shape* changes at the speed of movement, not at the speed of action potentials. Each finger movement presses its own subset of the ring in its own pattern; per-finger calibration records those patterns, and live detection reads which one is happening now.

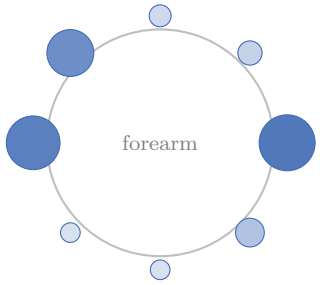


Figure 37.4: Force myography as a pressure map. A ring of force-sensitive resistors around the forearm reads where the cross-section bulges. Different finger movements light up different FSRs; calibration records each finger's pressure pattern, and live detection matches the current pattern to the nearest recorded one.

Why it works

Why does such a crude sensor work for fingers when the IMU cannot? Because, unlike the IMU, FMG *is* downstream of the finger muscles. The bulge of the flexor digitorum profundus when you curl your ring finger is a real, repeatable mechanical signature at the forearm surface,

and an FSR sitting over it sees a real change. The IMU sits over the same skin but measures rigid-body motion, to which the bulge contributes essentially nothing. Placement and physics, again, decide everything.

Application

The landmark DIY-scale result is *WristFlex* (Dementyev and Paradiso, MIT Media Lab, 2014): a wristband of FSRs that classified about five pinch gestures at around 80% accuracy, at very low power, on hardware you could plausibly build. It is the canonical demonstration that a cheap pressure band reads finger-level pinches the IMU cannot. FMG has since been studied extensively for prosthesis control, often matching or beating sEMG for robustness because it shrugs off sweat and electrode drift.

Toward the wrist tracker

For our wrist instrument, FMG is the most realistic finger path. A band of 4–8 FSRs distinguishes a set of pinches and finger curls – enough to trigger four or five different "strings" or drum pads – once you have run the short per-finger calibration that records each movement's pressure signature. Combined with the IMU (orientation for pitch, motion for dynamics) this gives a playable instrument without the microvolt fragility of EMG or the kernel-hacking of bio-acoustics. If you build only one finger-sensing channel, build this one first.

Build it (hardware)

A starter FMG band:

- 4 to 8 FSRs (e.g. Interlink 402) spaced evenly around an elastic wrist strap, each in a voltage divider with a $\sim 10\,\text{k}\Omega$ resistor.
- Read all channels on the microcontroller's ADC at 100 Hz; FMG bandwidth is low, so this is generous.
- Tighten the strap consistently – preload sets the operating point of every FSR, and a loose band is the FMG equivalent of EMG electrode shift.
- Calibrate per session: guide the user through each target finger movement, record a few seconds of each, and store the average pressure vector as that movement's template. Live detection compares the current vector to the templates and picks the nearest – no training of any model, just measure-and-match.

```python
import numpy as np

def read_fmg(adc, n=8):
    # raw 0..1 normalized FSR readings -> pressure vector
    return np.array([adc.read(ch) for ch in range(n)], dtype=float)

class NearestGesture:
    """Per-finger FMG matcher: store one template per movement, match nearest."""
    def __init__(self): self.templates = {}   # name -> mean pressure vector
    def enroll(self, name, samples):
        self.templates[name] = np.mean(samples, axis=0)
    def classify(self, p):
```

```
        best, bestd = None, np.inf
        for name, t in self.templates.items():
            d = np.linalg.norm(p - t)          # Euclidean distance in R^n
            if d < bestd: best, bestd = name, d
        return best
```

Connection

The `NearestGesture` matcher is nothing but the inner-product / distance geometry from the linear-algebra companion: each calibrated movement is a point in $\mathbb{R}^N$, and detection is "which template am I closest to." The robustness of FMG means those points stay put across a session, so this geometric rule is not a placeholder for something fancier – it *is* the method. Calibration records the templates; detection measures a distance. No model is trained, and none is needed.

37.5 Bio-acoustics: listening to the body's mechanical hum

There is a third, more exotic channel. When tendons slide, joints move, and fingers tap, they generate mechanical vibrations – structure-borne sound – that propagate through the arm. A sensitive accelerometer pressed to the skin, sampled fast enough, becomes a *contact microphone* for the body. This is bio-acoustic sensing.

What is really happening

The trick that makes bio-acoustics work is a re-use of the accelerometer you already have, but at a radically higher sample rate. Ordinary IMU use samples the accelerometer at a few hundred Hz to track gross motion. If instead you read it at a few kilohertz, it stops being a motion sensor and starts being a vibration sensor: it hears the rumble of tendons, the click of a tap, the resonance of the bones. Different finger actions excite the arm's mechanical structure differently, and those differences live in the audio-band spectrum, not in the slow motion signal.

History

The defining work is *ViBand* (Laput, Xiao, Harrison, CMU, UIST 2016). They sampled a smartwatch accelerometer at 4000 Hz – far above its normal use – turning it into a bio-acoustic microphone on the wrist. With it they recognized hand gestures, distinguished objects being touched and even held power tools by their vibration signature, and modulated data through the body. It shows what treating a motion sensor as a frequency instrument buys you: the entire spectral toolkit of this book, applied to a 4 kHz accelerometer stream.

Common trap

Bio-acoustics is powerful but the hardest of these modalities to DIY, for a concrete reason: getting a stock smartwatch or phone to deliver accelerometer samples at 4 kHz usually requires *modifying the device kernel or driver*, because standard OS sensor APIs cap the rate far below that. The signal processing is approachable; the plumbing to obtain the high-rate stream is

not. On a dedicated microcontroller wired directly to a high-bandwidth accelerometer (e.g. a ≥ 6 kHz-capable part read over SPI) you can hit the rate, but you are now doing embedded engineering, not app development.

Toward the wrist tracker

This is why our build puts a dedicated microcontroller on the wrist instead of leaning on a phone's sensors. A small wrist node wired to a high-bandwidth accelerometer over SPI can read it at a few kilohertz, treating it as a contact/bio-acoustic channel for taps and finger contacts – alongside the same node's ordinary low-rate IMU motion and the FMG ring. The node samples and lightly filters all three, then streams them over BLE. The phone, which never had to hit 4 kHz itself, runs the calibration routine and the deeper offline analysis. The hard ViBand plumbing problem dissolves once the high-rate sensor lives on a chip you control rather than behind a locked OS API.

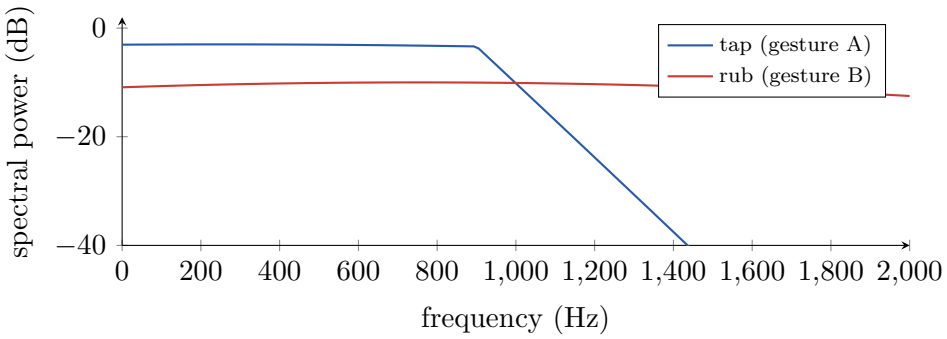


Figure 37.5: Why high-rate vibration sensing separates gestures. Two finger actions excite the arm's mechanics at different frequencies; their bio-acoustic spectra (schematic) peak in different places. At 300 Hz IMU sampling these spectra would be invisible; at 4 kHz they are easy to tell apart.

37.6 The decision table

Putting the five modalities side by side is the most useful single thing this chapter can leave you with. Read it as "what do I actually get, for what cost and skill, and what does calibration buy me."

Modality	What it senses	Finger detail	Cost / skill	Robustness
Wrist IMU	Rigid motion of the wrist segment	None (continuous); a few coarse wrist/whole-hand gestures	Very low / low	High (just motion)

Modality	**What it senses**	**Finger detail**	**Cost / skill**	**Robustness**
Surface EMG	Electrical command to forearm muscles	Good *discrete* finger gestures; $\sim$ 99% within-subject, much lower cross-subject	Medium / medium–high	Low: sweat, electrode shift, mains hum
Force myography	Mechanical bulge of muscles/tendons	Moderate: a few pinches/curls ($\sim$ 80% on 5, WristFlex)	Very low / low–medium	High: sweat-robust, no electrodes
Bio-acoustics	High-rate vibration (contact mic)	Good for taps/textures and some gestures (ViBand)	Medium / high (needs $\sim$ 4 kHz stream)	Medium: needs firm coupling, kernel access
Finger-worn IMUs (rings)	Motion/orientation of each finger directly	*Continuous* pose, $\sim$ 9° joint error	Medium / medium; not wrist-only	High, but intrusive (sensors on fingers)

Focus

The single most important row is the last one. **Continuous finger pose – the kind a virtual keyboard or full hand avatar needs – requires sensors on the fingers themselves**, typically small IMUs in rings, which report each finger's motion directly and reach roughly 9° of joint-angle error (the "Learning on the Rings" line of work on finger-worn continuous tracking). No wrist-only modality delivers this; the information is not in wrist-surface signals to begin with. What the wrist modalities *do* deliver – once calibrated to the wearer – is reliable detection of *which finger moved*, which is exactly what triggering notes requires. The line to hold is narrow and clear: discrete finger movement from the wrist, yes; continuous independent finger angles from the wrist, no.

37.6.1 A decision flow for the builder

1. **Do you only need a few triggers and coarse wrist gestures?** Wrist IMU alone. Cheap, robust, you already have it.

2. **Do you need to detect which finger moved, on a budget, surviving a sweaty performance?** Force myography with per-finger calibration. Best DIY finger path.

3. **Do you want the richest finger vocabulary and can run a 30-second calibration on each donning?** Surface EMG (start with MyoWare).
4. **Do you want tap/contact/texture sensing and can obtain a high-rate accelerometer stream?** Bio-acoustics (ViBand-style), best as a fused channel on a wrist node you control.
5. **Do you truly need continuous, independent finger angles?** Put sensors on the fingers (rings). Accept that this is no longer a wrist-only device.

Why this matters

The deeper lesson outlives the gesture controller. Sensing is choosing *where to stand* relative to the phenomenon, and signal processing only ever recovers what the sensor's vantage point preserves. The IMU, EMG, FMG, and accelerometer-as-microphone all watch the same hand; they differ entirely in which slice of reality reaches them. Mastering DSP is partly mastering filters and transforms, but it begins with this humbler, geometric question: from where I am standing, is the information I want even present in the signal? If it is not, no transform will conjure it.

37.7 Combining modalities: the realistic capstone

The literature's recurring answer – and ours – is *fusion*. The IMU and a finger-sensing channel are complementary: the IMU is robust and cheap but finger-blind; FMG and EMG see fingers but need per-finger calibration. Together they cover each other's weaknesses. Recall that the PLOS One 14-gesture result only reached its 75–80% by combining IMU with acoustic sensing; neither alone did as well. Our edge over the single-modality lab papers is exactly this: a high-rate IMU, an FMG/FSR ring, and a high-rate accelerometer used as a contact sensor, all fused, all calibrated to the wearer – not a cleverer model on one sensor.

Toward the wrist tracker

Our capstone architecture, which the final chapter will build, splits cleanly into a thin wrist node and a phone (Figure 37.6):

- **IMU** (the conductor): orientation → pitch / pitch-bend; motion energy → note velocity / dynamics; a few wrist gestures → mode and octave switches; motion gating → note-on/note-off windows.
- **FMG band** (the fingers): 4–8 FSRs → a set of pinches/curls that select which "string" or pad fires, resolved by matching the live pressure vector to the per-finger templates from calibration. Chosen over EMG for sweat-robustness and DIY simplicity; EMG is the upgrade path for a richer vocabulary, finger-worn rings the upgrade path for continuous angles.
- **Wrist node**: samples all sensors (including the high-rate accelerometer as a contact channel), applies light filtering, compresses, and BLE-streams. A lightweight on-device template-match handles the low-latency live-play path.
- **Phone**: runs the guided per-finger calibration, builds the templates and thresholds, conditions the streamed signals, and does deeper offline analysis from the logged data.

No model is trained – the method is measure-and-match throughout.

This is buildable, musically expressive, and clear about its limits – it triggers notes from detected finger movements and never pretends to read finger joint angles it cannot read.

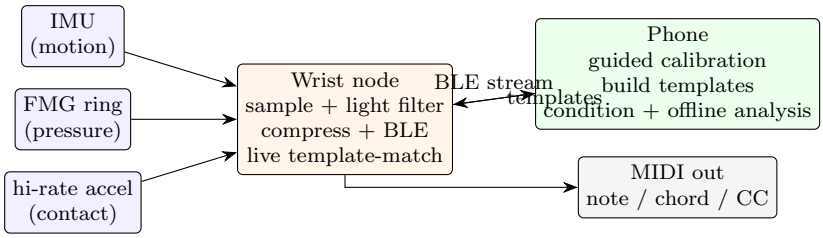


Figure 37.6: The capstone split. The thin wrist node samples three sensors, filters lightly, and BLE-streams to the phone; the phone runs the guided per-finger calibration and offline analysis and pushes templates back. A lightweight on-node template match drives the low-latency MIDI path. No model is trained anywhere – calibration measures templates, detection matches them.

Pause

If you remember one diagram from this chapter, make it the "ropes in a tube": the IMU feels the tube swing, FMG presses the tube, EMG taps the winches, bio-acoustics hears the ropes slide, and only sensors on the ropes themselves read continuous finger pose. Everything else here is detail hung on that picture.

37.8 Exercises

Exercise 37.1. Explain, in terms of mutual information and the map from finger pose to sensor output, why no amount of filtering or processing can make a wrist IMU recover individual finger joint angles. What property of the pose-to-output map is the real obstacle?

Exercise 37.2. sEMG has useful energy up to about 450 Hz. State the minimum sampling rate required by the Nyquist criterion, justify why 1000 Hz is a common safe choice, and explain what specifically goes wrong (in the spectrum) if you sample the RAW sEMG at 400 Hz.

Exercise 37.3. Given a one-second sEMG window sampled at 1000 Hz, write the formulas for MAV and RMS, and argue from the definition of the ℓ_2 norm why $\text{RMS} = \|\mathbf{x}\|_2/\sqrt{N}$. For a window of constant amplitude A (a square wave), compute both MAV and RMS.

Exercise 37.4. The median frequency f_{med} splits the power spectrum into equal halves. Suppose a muscle fatigues and its spectrum shifts uniformly downward by 15%. What happens to f_{med}, and why is it a better fatigue indicator than total power (RMS)?

Exercise 37.5. Force myography reads a pressure vector $\mathbf{p} \in \mathbb{R}^8$. During calibration you record one template per finger movement by averaging samples. Explain geometrically why nearest-template matching works, and describe one realistic situation (hint: strap tension) that shifts all eight readings and breaks it. How would you make the matcher robust to that shift?

Exercise 37.6. Contrast EMG and FMG at the level of physics: both ultimately report "a muscle contracted," yet one is a microvolt electrical signal and the other a slow large mechanical one. List two consequences of this difference for the sampling rate, and two for real-world robustness.

Exercise 37.7. ViBand samples a wrist accelerometer at 4000 Hz. Compute its Nyquist frequency. Explain why the *same* accelerometer sampled at 200 Hz cannot distinguish two gestures whose bio-acoustic signatures differ only above 300 Hz, referencing the spectra in Figure 37.5.

Exercise 37.8. Using the decision table of Section 37.6, choose a sensing modality for each of these targets and justify it in one sentence: (a) a fist/open-hand toggle for a simple game; (b) a sweat-proof five-pinch live-performance trigger; (c) a research prototype distinguishing 20 finger gestures within one session; (d) a full hand-avatar with independent finger motion.

Exercise 37.9. The PLOS One study reached 75–80% on 14 gestures only by fusing IMU with acoustic sensing. Propose a fused feature vector for an IMU+FMG template matcher (list at least three IMU-derived and three FMG-derived features), and explain which gestures each modality is expected to disambiguate.

Exercise 37.10. Sketch (in prose or a block diagram) the full capstone pipeline from the goalbox in Section 37.7: which sensor produces which MIDI message, where filtering happens, where windowing happens, where calibration builds the templates, and where the live template match sits. Mark which steps run on the wrist node and which on the phone. Identify the one place where an overclaim about sensor limits would most likely creep in, and how your design avoids it.

Chapter 38

Gesture and Activity Recognition

Everything we have built so far – signals as vectors, the Fourier transform as a projection onto sinusoids, spectrograms that localize frequency in time, filters that sculpt a signal's spectrum, and the inertial and muscle sensors of the last few chapters – has been working toward a single concrete question. A band sits on your forearm. A few sensors stream numbers many times a second. *What is the hand doing right now?* Is it making a fist, flicking left, tapping, rolling the wrist, or just resting? And can we answer fast enough, and reliably enough, to drive a musical instrument in real time?

That question is the subject of **gesture and activity recognition**. It is the bridge between raw, wiggly sensor traces and meaningful, discrete labels: *fist*, *wave*, *walking*, *typing*. The good news is that almost none of it is new physics. It is a *pipeline* built almost entirely from tools you already own. We cut the endless stream into manageable **windows**; from each window we compute a handful of **features** (many of them just the time-domain and frequency-domain quantities from earlier chapters); and we hand those features to a **classifier** that maps them to a label. The art is in choosing the windows and features so that the geometry of the problem – where different gestures land in feature space – becomes simple enough that even a tiny classifier can draw the boundaries.

This chapter is clear about what this pipeline can and cannot do. A wrist IMU on its own tells a flick from a fist from a roll. Add a force-myography (FMG) or surface-EMG band on the forearm, and – with a short per-finger calibration for the person wearing it – you can detect *which finger moved*, because the tendons and muscles that drive each finger leave a distinct mechanical and electrical signature just below the band. What stays out of reach from the wrist is the *continuous joint angle* of each finger; that is genuinely underdetermined and needs sensors on the fingers themselves. A recognizer tuned to one person can also drop in accuracy on a stranger – which is exactly why *calibration* sits at the center of our design rather than a one-size-fits-all model. We will see *why* each of these is true, and how to measure them so you are never fooled by your own optimism.

38.1 The recognition pipeline, end to end

Let us name the stages before we open any of them up. Sensor samples arrive as a stream – say a tri-axial accelerometer at $f_s = 100$ Hz, so every 10 ms we get a new triple (a_x, a_y, a_z). We never classify a single sample; one instant carries almost no information about a *motion*, which by definition unfolds over time. Instead:

1. **Segmentation / windowing.** Group the stream into chunks of N samples. Each chunk – a *window* – is the atomic unit we will label. (Section 38.2.)
2. **Feature extraction.** Reduce each window of (possibly many) raw samples to a short *feature vector* $\mathbf{x} \in \mathbb{R}^d$ of descriptive numbers: how big the motion is, how fast it wiggles, how the axes move together, where its energy sits in frequency. (Sections 38.3–38.4.)
3. **Classification.** Map the feature vector to one of K gesture labels, $\mathbf{x} \mapsto \hat{y} \in \{1, \ldots, K\}$. (Section 38.6.)
4. **Evaluation.** Measure how often $\hat{y}$ is right, on data the classifier has never seen, and ideally on *people* it has never seen. (Section 38.8.)

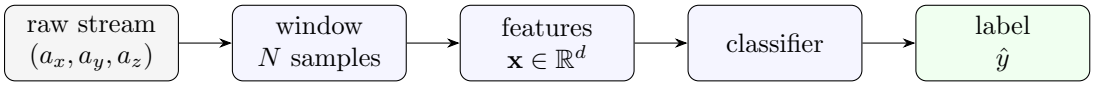


Figure 38.1: The recognition pipeline. A continuous sensor stream is chopped into windows; each window is reduced to a low-dimensional feature vector; a classifier maps that vector to a discrete gesture label. Every box reuses tools from earlier chapters.

Intuition

The whole pipeline is one long act of *dimensionality reduction with a purpose*. A two-second accelerometer window at 100 Hz is a point in $\mathbb{R}^{600}$ (200 samples $\times$ 3 axes) – a space far too big and too noisy to draw boundaries in. Feature extraction throws away the parts that do not matter (exactly *when* in the window a wiggle happened, the precise noise) and keeps the parts that do (how strong, how fast, how coupled), landing us in a friendly $\mathbb{R}^{10}$ or so. The classifier then just has to carve that small space into gesture-shaped regions. Good features make those regions large and well separated; bad features leave the classes tangled no matter how clever the classifier.

Connection

Feature extraction is a *lossy projection*, and you have met projections throughout this book and the companion linear-algebra reader. The Fourier transform projects onto sinusoids; PCA (Chapter on machine learning) projects onto directions of maximum variance; here we project onto a hand-chosen set of physically meaningful axes (energy, dominant frequency, inter-axis correlation). The choice of *what to project onto* is the entire game.

Toward the wrist tracker

For the wrist controller this pipeline is split across two devices. A thin wrist node samples the sensors, applies light filtering, compresses, and BLE-streams the result; the paired phone does the heavy lifting – it runs the guided per-finger calibration, builds the user's templates, conditions the signals, and keeps the streamed logs for deep offline analysis. For the low-latency live-play path a lightweight template match runs on the node itself: every $\sim$0.1 s it emits a verdict – is a new gesture starting? which finger? – and turns that into a MIDI message (note on, pitch bend, control change) fast enough that a musician does not feel the lag. That real-time constraint shapes every decision below: windows must be short, features

cheap, and the on-device match must fit in kilobytes. There is no machine-learned model anywhere in the loop. The method is direct: calibrate each user once, store their templates, and match the live signal against them.

38.2 Windowing and segmentation

A window is just N consecutive samples treated as one unit. Two numbers define the windowing scheme: the **length** N (or duration $T = N/f_s$) and the **hop** H, the number of samples we advance before starting the next window. The *overlap* is $N - H$ samples, often quoted as a fraction $1 - H/N$.

Definition 38.1 (Sliding window)**.** Given a stream $x[0], x[1], \ldots$, the m-th window of length N with hop H is the block

$$w_m = \big(x[mH],\, x[mH+1],\, \ldots,\, x[mH+N-1]\big), \qquad m = 0, 1, 2, \ldots$$

With $H = N$ the windows tile the stream without overlap; with $H < N$ they overlap and a given sample contributes to several windows.

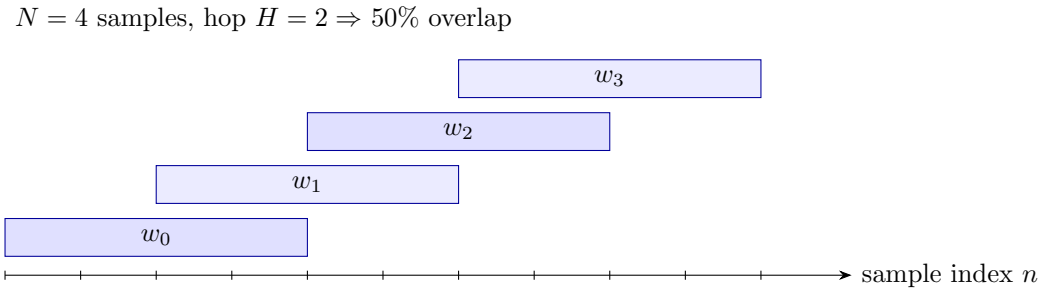


Figure 38.2: Sliding windows of length $N = 4$ with hop $H = 2$ (50% overlap). Each window is classified independently; overlap means a gesture is "looked at" from several alignments, which smooths the verdict and shortens latency.

38.2.1 How long should a window be?

This is the first real design tension, and it is the same time–frequency trade-off you met with the STFT (Chapter 10), wearing different clothes.

- **Too short** and the window cannot contain a whole gesture. A flick of the wrist lasts maybe 300–500 ms; a 50 ms window catches only a sliver and cannot tell a flick from the start of a punch. Short windows also give the frequency-domain features almost no resolution (recall the DFT bin spacing is f_s/N).
- **Too long** and you blur distinct actions together, add latency (you cannot label a 2 s window until 2 s have elapsed), and risk a single window straddling two different gestures.

A useful rule of thumb: make the window roughly as long as the *shortest gesture you must distinguish*, then choose the hop to meet your latency budget. For human activity and hand gestures, windows

of 1–2.5 s with 50% overlap are a common, defensible default; for the snappy, low-latency triggers a musician needs, you push down toward 200–400 ms and lean on detecting onsets rather than steady states.

Common trap

Beware analyzing a window long enough to contain a whole *repetitive* activity (walking, stirring) the same way you analyze a one-shot *gesture* (a flick). Repetitive activities have a clean periodicity that frequency-domain features capture beautifully. One-shot gestures are *transients*: their information is in the *shape* of a single burst, not in any steady frequency. Using a long window and a centroid feature on a transient throws away exactly what makes it recognizable. Match the analysis to the kind of motion.

38.2.2 Event segmentation: finding where a gesture is

Fixed sliding windows are simple and real-time friendly, but they are *dumb*: they chop on a clock, not on the action. Most windows of a wrist stream contain nothing – the hand is at rest – and we waste effort classifying emptiness, and worse, we risk slicing a gesture down the middle.

The fix is a cheap **onset detector** (or activity detector) that decides *when* something is happening, and only then fires the classifier. A robust and almost free detector watches a short-time energy or jerk signal and triggers when it crosses a threshold. Define the windowed energy of the (gravity-removed) acceleration magnitude,

$$E[m] = \frac{1}{N} \sum_{n=0}^{N-1} \left\| \mathbf{a}[mH + n] - \bar{\mathbf{a}} \right\|^2, \tag{38.1}$$

and call a window "active" when $E[m]$ exceeds a threshold θ. To avoid chattering at the threshold, use *hysteresis*: a high threshold θ_{on} to start an event and a lower θ_{off} to end it. This is exactly the Schmitt-trigger idea, and it is the difference between a controller that triggers once per gesture and one that machine-guns a dozen spurious notes as your hand crosses the threshold.

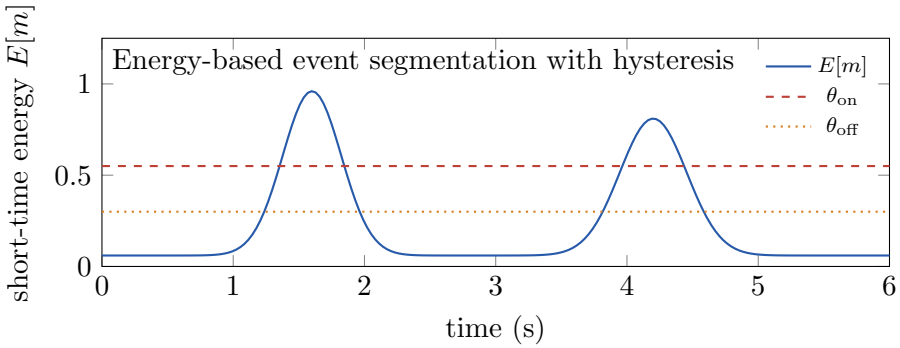


Figure 38.3: An onset detector with hysteresis. An event begins when energy rises above θ_{on} and ends only when it falls below the lower θ_{off}. The gap between thresholds prevents a single gesture from being split into many spurious triggers by noise near the boundary.

Build it (hardware)

A minimal real-time segmenter on a microcontroller. Keep a running window energy; emit a window for classification only on the rising edge through `TH_ON`, and refuse to re-trigger until energy has dropped below `TH_OFF`. This is a few lines and saves enormous CPU.

```
// streaming onset detector with hysteresis (one axis-magnitude sample at a time)
static int   armed = 1;          // ready to fire?
static float e = 0.0f;           // leaky energy estimate
const  float alpha = 0.05f;      // smoothing (approx 20-sample memory)
const  float TH_ON = 0.55f, TH_OFF = 0.30f;

int onset_step(float a_mag, float gravity) {
    float ac = a_mag - gravity;          // remove DC / gravity baseline
    e = (1.0f - alpha) * e + alpha * ac * ac;   // leaky short-time energy
    if (armed && e > TH_ON)  { armed = 0; return 1; }  // fire: classify now
    if (!armed && e < TH_OFF){ armed = 1; }            // re-arm
    return 0;
}
```

History

The threshold-with-hysteresis trick is the *Schmitt trigger*, invented by Otto Schmitt in 1934 while he was a graduate student studying nerve impulse propagation in squid axons – biology, not electronics. He noticed that a neuron fires cleanly despite noisy, slowly-varying inputs precisely because it has two different thresholds. The same idea now keeps your gesture controller from stuttering. Good ideas in signal processing are frequently borrowed from the nervous system, which has been doing real-time activity recognition for a very long time.

38.3 Time-domain features

Now we reduce a window to numbers. Time-domain features are computed directly from the samples without any transform – they are cheap, robust, and surprisingly powerful. Take a single-axis window $x[0], \ldots, x[N-1]$ with mean $\bar{x} = \frac{1}{N}\sum_n x[n]$.

Mean. $\bar{x}$ itself. For an accelerometer it captures the gravity component along that axis, hence *orientation*: a flat wrist and a vertical wrist have different per-axis means even when perfectly still. For a gyroscope or high-pass-filtered signal the mean is near zero and carries little.

Variance / standard deviation.

$$\sigma_x^2 = \frac{1}{N}\sum_{n=0}^{N-1}\big(x[n]-\bar{x}\big)^2. \tag{38.2}$$

The single most useful motion feature. It measures *how much* the signal moves regardless of orientation, separating "still" from "active" and gentle from vigorous. Standing and sitting both have low variance; walking and waving have high variance.

RMS (root mean square).

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x[n]^2}. \tag{38.3}$$

The signal's effective magnitude including its DC level; note $\mathrm{RMS}^2 = \bar{x}^2 + \sigma_x^2$, so RMS blends "how tilted" with "how much it moves." For zero-mean signals RMS equals the standard deviation and is a clean energy proxy.

Zero-crossing rate (ZCR). The number of times the signal crosses its mean per window,

$$\mathrm{ZCR} = \frac{1}{N-1}\sum_{n=1}^{N-1} \tfrac{1}{2}\,\big|\,\mathrm{sgn}(x[n]-\bar{x}) - \mathrm{sgn}(x[n-1]-\bar{x})\big|. \tag{38.4}$$

A poor-man's frequency estimate computed without a single multiply: a fast wiggle crosses its mean often, a slow one rarely. We met ZCR in the audio-features chapter; here it cheaply distinguishes a brisk tremor from a slow roll.

Min, max, range, and peak count. Order statistics of the window. The *range* $\max - \min$ captures the amplitude of a one-shot transient that variance smears out; the number of peaks above a threshold counts repetitions (steps, taps).

Correlation between axes. Real gestures move several axes *in a coordinated way*, and that coordination is a fingerprint. The Pearson correlation between axes x and y over the window,

$$\rho_{xy} = \frac{\sum_n (x[n]-\bar{x})(y[n]-\bar{y})}{\sqrt{\sum_n (x[n]-\bar{x})^2}\,\sqrt{\sum_n (y[n]-\bar{y})^2}} = \frac{\langle x-\bar{x},\ y-\bar{y}\rangle}{\|x-\bar{x}\|\,\|y-\bar{y}\|}, \tag{38.5}$$

is exactly the cosine of the angle between the two mean-centered window vectors – the inner-product geometry from Chapter 3. $\rho_{xy} \approx +1$ means the axes rise and fall together (a diagonal motion), -1 means they oppose, 0 means they are unrelated. A wrist *roll* couples two axes strongly; a straight *flick* drives mostly one. Correlation is what lets a classifier tell the *direction* of a motion, not just its size.

Why it works

Why is inter-axis correlation so informative when variance per axis is not? Because variance is *rotation-blind in a misleading way*: two completely different motions can deposit the same total variance, just distributed differently across axes and time. Correlation looks at the *joint* behavior – the *shape* of the cloud of $(x[n], y[n])$ points. A motion confined to a tilted line in the xy-plane has $|\rho| \approx 1$; an isotropic shake fills a disk and has $\rho \approx 0$. That shape is a geometric signature of the gesture that survives changes in how hard you perform it.

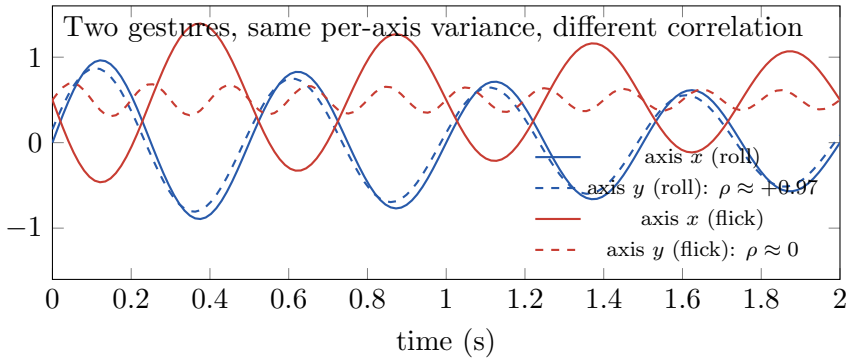


Figure 38.4: Per-axis variance alone cannot separate these two gestures, but their inter-axis correlation does: the roll drives x and y together ($\rho \approx +1$), while the flick drives x alone ($\rho \approx 0$). Correlation encodes the *direction* of motion.

Connection

The list above – variance, RMS, correlation – is the elementary statistics of the companion probability book applied to a window. Variance is $\mathbb{E}[(X - \mathbb{E}\,X)^2]$ estimated over the window; correlation is the normalized covariance $\mathrm{Cov}(X,Y)/(\sigma_X \sigma_Y)$. Treating a window as a small sample from a random process is not a metaphor; it is the literal bridge between Chapter 20 (random signals, noise) and feature engineering. A feature is an *estimator* of some property of the underlying motion, and like all estimators it has bias and variance of its own.

38.4 Frequency-domain features

For *periodic* or *textured* motions – walking, running, scrubbing, a sustained tremor – the most discriminating information lives in the spectrum. Take the DFT of the window, $X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}2\pi kn/N}$, and form the magnitude spectrum $|X[k]|$ over bins $k = 0, \ldots, N/2$, with bin k corresponding to frequency $f_k = k f_s / N$.

Band energies. Partition the frequency axis into a few bands and sum the power in each:

$$E_{\text{band}} = \sum_{k \in \text{band}} |X[k]|^2. \tag{38.6}$$

Human locomotion lives below ∼3 Hz (step cadence) with harmonics up to ∼10 Hz; voluntary hand gestures are slower still; physiological and pathological tremor sits in characteristic bands (rest tremor ≈ 4–6 Hz, essential tremor ≈ 6–12 Hz). A handful of band energies is often enough to separate walking from running from idle.

Dominant frequency. The bin with the most power, $f^\star = f_{\arg\max_k |X[k]|}$, e.g. the step rate while walking. Cheap and very informative for periodic activities; nearly meaningless for one-shot transients (whose energy is spread, not peaked).

Spectral centroid. The power-weighted average frequency – the spectrum's "center of mass",

$$f_c = \frac{\sum_k f_k \, |X[k]|^2}{\sum_k |X[k]|^2}. \tag{38.7}$$

We met the centroid as "brightness" in the audio-features chapter; for motion it says whether the energy is concentrated low (slow, sweeping) or high (jittery, fast). A gentle wave has a low centroid; a nervous shake has a high one.

Spectral entropy. How *spread out* the spectrum is. Normalize the power spectrum to a distribution $p_k = |X[k]|^2 / \sum_j |X[j]|^2$ and take

$$S = -\sum_k p_k \log p_k. \tag{38.8}$$

Low entropy means the energy is concentrated at one frequency – a clean periodic motion like steady walking. High entropy means the energy is smeared across many frequencies – broadband, irregular, noise-like motion. Entropy beautifully separates "rhythmic" from "erratic" even when the dominant frequency is the same.

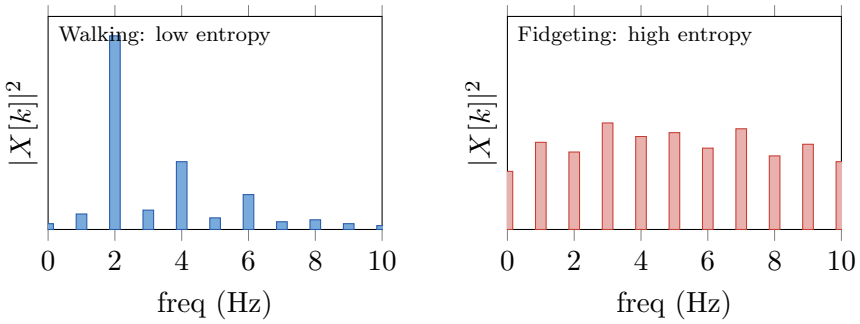


Figure 38.5: Two power spectra with similar total energy. Walking concentrates power at the step frequency and its harmonics (low spectral entropy); irregular fidgeting spreads power across the band (high entropy). The dominant-frequency feature would call the right plot's peak meaningless, but entropy cleanly flags it as broadband.

Common trap

Do not compute frequency features on a window without first removing the mean (DC) and ideally applying a taper (Hann window). The huge DC bin from gravity will dominate $\sum |X[k]|^2$ and drag the centroid toward 0 Hz, and spectral leakage from the rectangular cutoff (Chapter 10) will smear sharp peaks into neighbors, corrupting band energies. Mean-remove, taper, then transform.

38.5 A worked example: four wrist gestures

Let us make this concrete with small, hand-computable numbers. Suppose we capture short windows from four gestures on a wrist IMU and (after preprocessing) read off just three features per window:

the variance of the x-axis acceleration σ_x^2, the dominant frequency $f^\star$, and the xy correlation ρ_{xy}. Idealized values:

Gesture	σ_x^2 (g^2)	$f^\star$ (Hz)	ρ_{xy}
Rest	0.002	–	≈ 0
Flick left	0.45	2.5	$+0.1$
Wrist roll	0.30	2.0	$+0.9$
Tremor	0.05	8.0	$+0.2$

Look at how cleanly these separate, *one feature at a time*: variance alone peels Rest off the bottom; the dominant frequency isolates Tremor at the top; correlation distinguishes the otherwise-similar Flick and Roll. No single feature does the whole job, but together they place the four gestures at four corners of a small feature cube, and *any* reasonable classifier can separate corners.

Example 38.1 (Computing features from raw samples)**.** Take a tiny window of x-axis acceleration (gravity already removed), $N = 8$:

$$x = (0.0,\ 0.4,\ 0.6,\ 0.2,\ -0.3,\ -0.5,\ -0.2,\ 0.1)\ \text{g}.$$

Mean: $\bar{x} = \frac{1}{8}(0.3) = 0.0375 \approx 0$. Variance (about the mean),

$$\sigma_x^2 = \tfrac{1}{8}\sum (x[n] - \bar{x})^2 \approx \tfrac{1}{8}(0.939) \approx 0.117\ \text{g}^2,$$

so RMS $\approx \sigma_x \approx 0.34$ g. Zero crossings of $x - \bar{x}$: the sign pattern is $-, +, +, +, -, -, -, +$, which flips three times (between samples 0–1, 3–4 and 6–7), so ZCR $= 3/(N-1) = 3/7 \approx 0.43$ crossings per sample. At $f_s = 100$ Hz that suggests a fundamental of a few Hz – a quick sanity check against whatever the DFT reports. Every one of these is a few additions; this is why time-domain features are the backbone of an embedded recognizer.

Try it yourself

Record yourself (or just simulate) doing each of the four gestures ten times. Compute the three features per window and scatter-plot variance against correlation, coloring by gesture. You should *see* the four clusters with your own eyes – and where two clusters overlap is exactly where your classifier will make its mistakes. This plot, made before you train anything, is the single most valuable diagnostic in the whole project: if the clusters are not visually separable in your features, no classifier will rescue you, and you should go back and find better features.

38.6 Classification

Once each window is a feature vector $\mathbf{x} \in \mathbb{R}^d$, classification is the act of carving $\mathbb{R}^d$ into K labeled regions. You do not need a deep network. For the small, well-separated feature spaces of gesture recognition, simple classifiers are fast, interpretable, and frequently *better* (less prone to overfitting on the modest data you can realistically collect from a few people).

k-nearest neighbors (k-NN). Store all labeled training feature vectors. To classify a new $\mathbf{x}$, find the k stored points nearest to it (Euclidean distance, on *normalized* features) and take a majority vote. It is a one-line idea – "it is whatever its neighbors are" – with no training at all, and it makes the geometry of Section 38.5 literal: a gesture is recognized by landing near other examples of the same gesture. Its cost is memory and per-query distance computations, which can sting on a microcontroller.

Decision trees and random forests. A tree asks a sequence of threshold questions – "is $\sigma_x^2 > 0.1$? if so, is $f^\star > 5$?" – each splitting feature space with an axis-aligned cut, until a leaf assigns a label. This is *exactly* the one-feature-at-a-time reasoning we did by eye in Section 38.5, automated and made optimal. A *random forest* (many trees voting) is one of the most reliable off-the-shelf classifiers for tabular features like ours, and small trees run in microseconds on a microcontroller.

Linear and logistic classifiers. Draw a hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$ between classes; the sign of $\mathbf{w} \cdot \mathbf{x} + b$ is the decision. When classes are linearly separable in feature space (often the case after good feature engineering) this is the cheapest possible classifier – one dot product – and its weights $\mathbf{w}$ are interpretable as "how much each feature matters."

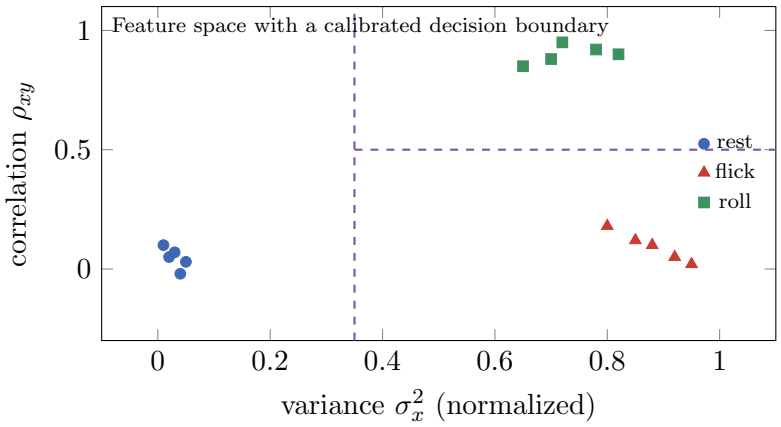


Figure 38.6: Three gestures in a two-feature space. Good features make the classes fall into separated clusters; a tiny decision tree – "high variance? then high correlation?" – draws the two axis-aligned cuts (dashed) that separate them. The classifier is doing exactly the by-eye reasoning of Section 38.5.

Focus

Always normalize your features before classifying. A k-NN or linear model measures distances in feature space; if variance ranges over $[0, 0.5]$ and dominant frequency over $[0, 50]$, the frequency axis silently dominates every distance and the variance feature is ignored. Standardize each feature to zero mean and unit standard deviation (using statistics computed *on the training set only*), then store those constants and apply the same transform on the device. This single step often matters more than the choice of classifier.

What is really happening

What is really happening when a classifier fits to data? It is drawing the *boundaries* between the clusters you already created with your features. The classifier adds almost nothing if the clusters are clean – a child could draw the lines in Figure 38.6. Conversely, no classifier can separate clusters that overlap. This is why experienced practitioners spend their time on windowing and features, not on swapping classifiers: **the features decide whether the problem is solvable; the classifier merely solves the solvable problem.** For our controller we lean on the simplest end of this spectrum – store the user's calibration examples and label a new window by the nearest stored template – precisely because clean features make the boundaries trivial and a direct match needs no training procedure at all.

38.7 Template matching and dynamic time warping

The feature-vector approach summarizes a window into static numbers, discarding the order in which things happened. For some gestures the *trajectory* – the precise temporal shape – is the point: a checkmark motion, a circle, a signature. For these, **template matching** keeps the whole trajectory and compares an incoming gesture to stored exemplars.

The naive comparison is the Euclidean distance between two sequences sample by sample. It fails badly because two performances of the same gesture are never the same length or speed: you do the circle a little faster this time, or pause in the middle. Aligning sample n of one to sample n of the other compares the top of one circle to the side of another and reports a huge, meaningless distance.

Dynamic time warping (DTW) fixes this by finding the best *nonlinear* alignment between two sequences before measuring their distance. Given sequences a (length M) and b (length N), it builds a warping path through the $M \times N$ grid of local distances $d(i,j) = (a_i - b_j)^2$ that stretches and compresses time to match the shapes. The optimal-path cost is found by dynamic programming:

$$D(i,j) = d(i,j) + \min\big\{\, D(i-1,j),\; D(i,j-1),\; D(i-1,j-1) \,\big\}, \tag{38.9}$$

with $D(0,0) = d(0,0)$ along the boundaries, and the DTW distance is $D(M-1, N-1)$. The three predecessors correspond to "advance in a only" (compress b), "advance in b only" (stretch b), and "advance both" (match). The cheapest path through the grid is the best way to line up the two gestures despite differences in tempo.

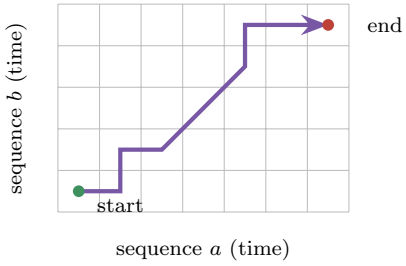


Figure 38.7: A dynamic time warping path through the cost grid. The path moves monotonically from corner to corner; horizontal/vertical steps stretch or compress one sequence in time to match the other, while diagonal steps match samples directly. DTW measures shape similarity while tolerating differences in speed.

Intuition

Picture two people humming the same tune, one slightly slower with a pause in the middle. Lay one performance along the bottom of a grid and the other up the side. A diagonal line would assume they are perfectly in sync; DTW instead finds a *wiggly monotone staircase* that pairs each note of one with the matching note of the other, pausing (going straight along a row) exactly where one singer paused. The total mismatch *along that best staircase* is the DTW distance. It answers "are these the same shape, played at possibly different speeds?"

Why it works

Why does the simple recurrence (38.9) find the *globally* optimal alignment? Because the best path to cell (i, j) must pass through one of its three valid predecessors, and the best path to *each* of those was already computed and stored. This is the optimal-substructure property that makes dynamic programming work: a globally optimal alignment is built from optimal sub-alignments. The cost is $O(MN)$ time and memory – fine for a stored library of short templates, but it grows with the number and length of templates, which is the practical limit on DTW.

Common trap

DTW is seductive but does not scale. Matching every incoming gesture against a large library is $O(MN)$ *per template*, and DTW distances are not a metric you can index easily. For a handful of distinctive whole-trajectory gestures (a few templates per class) it is excellent and needs no training. For many classes, many users, or high throughput, the window-feature-classifier path of Sections 38.3–38.6 is usually the better engineering choice. Use a constraint band (Sakoe–Chiba) to cap how far the path may stray from the diagonal: it speeds DTW up and, by forbidding absurd alignments, often improves accuracy too.

Application

DTW was born in 1970s speech recognition (Sakoe and Chiba, 1978), where it aligned a spoken word against templates despite people talking at different speeds. The same algorithm now powers gesture recognition on smartwatches, signature verification, gait analysis, and bioinformatics sequence alignment. Whenever you need "same shape, different speed," DTW is the classical answer – a single elegant idea that crossed from speech into motion sensing wholesale.

38.8 Evaluating a recognizer

A recognizer that has not been carefully evaluated is just a hope. This section is about not fooling yourself – the most important and most neglected skill in the whole pipeline.

38.8.1 Accuracy and its lies

Accuracy is the fraction of windows labeled correctly. It is the obvious metric and it is frequently misleading. If 95% of your windows are "rest" (as they will be – hands are still most of the time),

a classifier that *always* says "rest" scores 95% accuracy while detecting exactly zero gestures. With imbalanced classes, accuracy rewards the trivial.

38.8.2 The confusion matrix

The full summary is the **confusion matrix** C, where C_{ij} counts windows whose true label is i but were predicted j. The diagonal is correct predictions; every off-diagonal entry is a specific, named mistake. The confusion matrix tells you not just *how often* you are wrong but *how* – and "flick mistaken for roll" is an actionable clue (add a correlation feature) while a bare accuracy number is not.

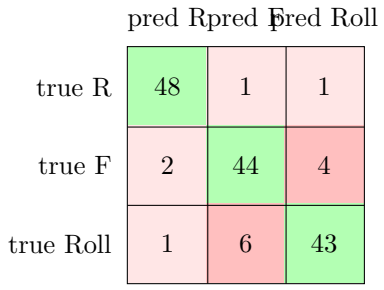


Figure 38.8: A confusion matrix for three gestures (Rest, Flick, Roll). The strong diagonal is good; the highlighted off-diagonal cells – Flick predicted as Roll (4) and Roll predicted as Flick (6) – reveal that these two are the classifier's main confusion, exactly as Figure 38.4 would predict if the correlation feature were weak or noisy.

From the confusion matrix come the per-class metrics that survive class imbalance. For a class c: **precision** $= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$ (of windows you *called* c, how many were?), **recall** $= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}$ (of windows that *were* c, how many did you catch?), and their harmonic mean the **F1 score** $= 2\,\frac{\mathrm{precision}\cdot\mathrm{recall}}{\mathrm{precision}+\mathrm{recall}}$. For a controller, precision and recall trade off in a way you can *feel*: low precision means spurious notes you did not intend; low recall means gestures you performed that the instrument missed. Which is worse depends on the instrument.

38.8.3 The cardinal sin: leaky evaluation

You must measure performance on data the classifier never saw in training. That much is well known. What is less appreciated – and what silently inflates most reported gesture-recognition numbers – is *how* you split.

Common trap

Never split overlapping windows randomly into train and test. With 50% overlap, adjacent windows share half their samples; a random split puts a window in training and its near-twin in test, so the classifier is graded on data it has effectively memorized. Accuracy looks spectacular and means nothing. Split by *recording session* or by *time block*, never by individual window.

38.8.4 Cross-subject generalization: the number that hurts

Here is the truth the glossy demos omit. Fit and test on the *same person* and you will see very high accuracy – often 95%+ – because the templates match *that person's* particular way of moving. Now test on a *different* person, using someone else's templates, and accuracy commonly falls hard, into the 70s or worse. The way to measure this gap is **leave-one-subject-out cross-validation (LOSO)**: fit on everyone except one person, test on that held-out person, and repeat for each person. The LOSO number is what a stranger would experience if you handed them a band with *someone else's* templates – and it is precisely the gap our per-user calibration step is designed to close. We do not ship a stranger's boundaries; we record the new user's own finger movements at enrollment, so the templates always belong to the hand wearing the band.

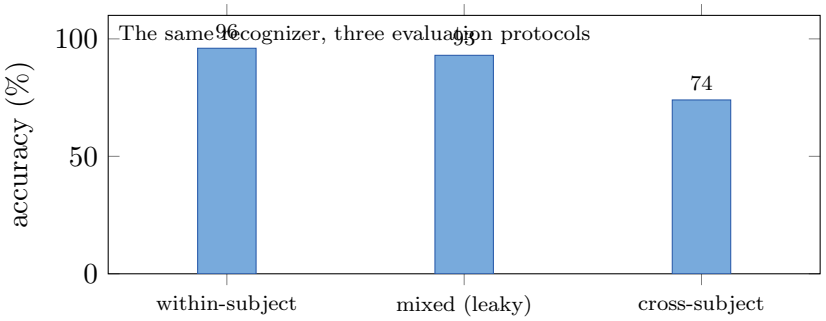


Figure 38.9: Illustrative (and typical) accuracies for one recognizer under three protocols. Within-subject and the leaky mixed split look great; the leave-one-subject-out (cross-subject) number is the one a new user feels with someone else's templates. Always report LOSO – the others flatter your model.

Why it works

Why does accuracy drop so much across people? Because the feature distributions *shift*. People differ in limb length, where the band sits, how they wear it (rotation!), baseline tremor, strength, and personal motion style. A boundary tuned to your variance and correlation values lands in the wrong place for someone whose "vigorous flick" is another person's "gentle one." This is *distribution shift*, and it is the central difficulty of cross-subject wearable recognition. The cleanest remedy is the one we build the whole product around: a brief per-user calibration so the templates and thresholds belong to *this* wearer, not to an average of strangers. Supporting tactics help too – orientation-invariant features (use acceleration *magnitude* and inter-axis correlation rather than raw axes), per-user normalization, and re-running the calibration whenever the band is re-seated. With calibration, the cross-subject gap stops being your problem: every user effectively runs a within-subject recognizer.

Connection

Cross-subject drop is a concrete instance of the train/test mismatch the machine learning chapter formalizes as the difference between empirical risk (error on your sample) and true risk (error on the real population). LOSO is your best cheap estimator of true risk for the population "people who are not in your dataset." It usually has high variance with few subjects, so report the mean *and* spread across the held-out folds, not a single rosy figure.

38.9 What the wrist can and cannot sense

We have built a pipeline that recognizes a vocabulary of distinct wrist-and-forearm motions. Now let us draw the line precisely between what is within reach and what is not – because the line is not where casual intuition puts it.

Focus

Which finger moved is detectable from the wrist; each finger's continuous joint angle is not. The accelerometer and gyroscope sense the wrist as a rigid segment, so on their own they give you whole-hand and wrist gestures (fist vs. open, flick, twist, tap-induced shock, double-tap). But the muscles and tendons that drive the fingers run *through the forearm, directly under a band*. Tap your index versus your ring finger and the tendon bulge and electrical activity just below the band differ in a repeatable way. A force-myography or surface-EMG sensor picks that up, and with a short per-finger calibration the band detects *which finger moved*. What remains genuinely out of reach from the wrist is the *continuous flexion angle* of each joint: that information is simply not present in wrist-frame bulk motion, and recovering it needs sensors on the fingers.

To detect *which finger* acted, add a forearm sensing modality alongside the IMU:

- **Force myography (FMG)** reads the mechanical bulging of forearm muscles with a ring of pressure sensors (FSRs). It is cheap, robust to electrical noise and sweat, and after a short per-finger enrollment it separates finger and grip patterns well (Figure 38.10). This is the principle behind WristFlex, which distinguished finger pinches from a wrist FMG band.

- **Surface EMG** on the forearm senses the electrical activity of the finger-driving muscles, the basis of myoelectric control (Myo, MyoWare). It catches *intended* movement slightly before motion appears, but is more sensitive to electrode shift and sweat than FMG, so it benefits most from per-user calibration and re-seating checks.

- **Continuous finger *pose*** – actual joint angles for a piano-like ten-voice instrument – requires sensors *on the fingers* (rings with IMUs, stretch sensors, instrumented gloves). Recent finger-worn work such as "Learning on the Rings" tracks continuous flexion this way. There is no way around the physics: to measure a joint you must observe something that the joint actually moves. We treat this as an optional upgrade path, not a requirement for the instrument we are building.

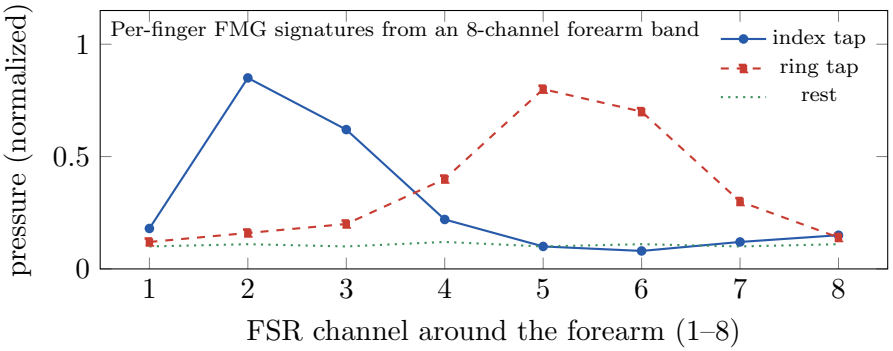


Figure 38.10: Force-myography snapshots across an eight-channel forearm band. Tapping the index finger bulges the tendons under channels 2–3; tapping the ring finger bulges channels 5–6; at rest the profile is flat. After a short per-finger enrollment, matching the live channel vector to these stored signatures tells *which finger* moved – something a wrist IMU alone cannot see. The signatures are person-specific, which is exactly why calibration is per user.

Toward the wrist tracker

This sets the design of our controller. From the wrist IMU we get a reliable *discrete gesture vocabulary* (taps, flicks, twists, holds) plus continuous wrist *orientation*, a superb expressive pitch-bend or modulation axis. Add a forearm FMG/FSR array (and optionally surface EMG) and, after the per-finger calibration, the band reports *which finger* acted – enough to play notes and chords by finger. The edge over single-sensor lab demos comes from *fusing* modalities: a high-rate IMU for gross motion and orientation, the FMG/FSR array for finger identity, and a separate high-rate accelerometer used as a contact / bio-acoustic pickup that hears the sharp transient of a fingertip tap (the ViBand idea). No one of these is enough alone; together, with per-user calibration, they cover finger *events* cleanly. The one thing we do not promise is ten continuously variable finger voices like a piano – that needs finger-worn sensors. For everything a player actually does on this instrument – triggering notes and chords with finger movements, bending and modulating with the wrist – the wrist band delivers.

History

Wearable activity recognition matured in the 2000s as MEMS accelerometers became cheap and tiny. Bao and Intille's 2004 study, recognizing everyday activities from multiple body-worn accelerometers, is a landmark that established the window-feature-classifier template still used today. The cross-subject gap they and successors documented has not been "solved" so much as managed; it remains the reason a fitness tracker counts *your* steps well after a few days and a stranger's less so. The thread that leads to our wrist instrument runs through specific systems: WristFlex (finger pinch detection from a wrist FMG band), the Myo and MyoWare surface-EMG modules, ViBand (UIST 2016, a high-rate wrist accelerometer used as a bio-acoustic pickup for tap and contact events), a PLOS One 2020 study fusing IMU with acoustic sensing, an MDPI Sensors 2023 study counting gestures from a wrist IMU, and "Learning on the Rings" for finger-worn continuous tracking. Our edge over any single one of these is to fuse modalities – IMU, FMG/FSR array, and a bio-acoustic accelerometer – and to calibrate per finger per user, rather than chase one sensor with a heavier model.

38.10 Putting it together: a reference recognizer

Here is the whole pipeline in a few lines of NumPy – windowing, a compact feature vector mixing time- and frequency-domain quantities, and a nearest-template match against the per-user enrollment. It is deliberately small and correct; it is the seed of what we will port to the microcontroller in the embedded chapter.

```
import numpy as np

def features(win, fs):
    """win: (N,3) tri-axial window (gravity removed). Returns a feature vector."""
    feats = []
    for ax in range(3):                        # per-axis time-domain features
        x = win[:, ax]
        feats += [x.mean(), x.var(), np.sqrt((x**2).mean())]   # mean, var, RMS
        zc = np.mean(np.abs(np.diff(np.sign(x - x.mean())))) / 2
        feats.append(zc)                                       # zero-crossing rate
    # inter-axis correlations (the coupling / direction signature)
    c = np.corrcoef(win.T)                     # 3x3 correlation matrix
    feats += [c[0,1], c[0,2], c[1,2]]
    # frequency-domain on the acceleration magnitude
    mag = np.linalg.norm(win, axis=1)
    mag = (mag - mag.mean()) * np.hanning(len(mag))   # detrend + taper
    P = np.abs(np.fft.rfft(mag))**2
    f = np.fft.rfftfreq(len(mag), 1/fs)
    P[0] = 0                                   # kill residual DC
    centroid = np.sum(f * P) / (np.sum(P) + 1e-12)
    dom = f[np.argmax(P)]
    p = P / (P.sum() + 1e-12)
    entropy = -np.sum(p * np.log(p + 1e-12))
    feats += [centroid, dom, entropy]
    return np.array(feats)

def windows(stream, N, H):
    """Yield (start, window) for each sliding window."""
    for m in range((len(stream) - N) // H + 1):
        s = m * H
        yield s, stream[s:s+N]

# usage sketch (direct template matching -- no learned model):
#   T = {g: features(cal_win, fs=100) for g, cal_win in enrollment}  # per-finger
    templates
#   x = (features(w, fs=100) - mu) / sigma     # normalize with enrollment mu, sigma
#   y_hat = min(T, key=lambda g: np.linalg.norm(x - (T[g]-mu)/sigma))  # nearest template
```

Why this matters

Step back and notice what we have. Twenty-odd numbers per window – means, variances, RMS, zero-crossings, three correlations, a centroid, a dominant frequency, an entropy – and almost every one is something you derived from first principles in an earlier chapter. The inner product became correlation; the DFT became band energy and centroid; the STFT's time–frequency trade-off became the window-length decision; the probability book's variance

and expectation became features. Gesture recognition is not a separate subject bolted on at the end. It is the *payoff*: the same handful of geometric ideas, pointed at your moving hand.

38.11 Summary

- Recognition is a pipeline: *window* the stream, extract a low-dim *feature vector*, *classify* it, *evaluate* without fooling yourself.
- Window length trades latency and gesture-containment against frequency resolution – the STFT trade-off in a new guise. An onset detector with hysteresis segments events cheaply and avoids spurious triggers.
- Time-domain features (mean, variance, RMS, ZCR, range, inter-axis correlation) are cheap and powerful; correlation, the cosine between mean-centered axis vectors, encodes *direction* of motion.
- Frequency-domain features (band energies, dominant frequency, spectral centroid, spectral entropy) shine for periodic/textured motion; always remove DC and taper first.
- Simple classifiers (k-NN, decision trees/forests, linear) suffice when features separate the classes; features matter more than the classifier. Always normalize features with train-set statistics.
- DTW matches whole trajectories despite speed differences via dynamic programming; great for a few template gestures, poor at scale.
- Evaluate with confusion matrices and per-class precision/recall/F1, never bare accuracy on imbalanced classes; never split overlapping windows randomly; report leave-one-subject-out accuracy as the number that counts.
- A wrist IMU yields whole-hand gestures plus orientation. Adding a forearm FMG/EMG band and a short per-finger calibration detects *which finger* moved. What the wrist cannot give is each finger's continuous joint angle; that needs sensors on the fingers.

38.12 Exercises

Exercise 38.1. A sensor streams at $f_s = 100$ Hz. You choose windows of $N = 256$ samples with 50% overlap. (a) What is the window duration in seconds? (b) What is the hop in samples and in seconds? (c) How many windows per second does the classifier process? (d) What is the DFT bin spacing f_s/N in Hz, and what is the lowest nonzero frequency you can resolve?

Exercise 38.2. For the 8-sample window in Example 38.1, verify the variance $\sigma_x^2 \approx 0.117$ by computing $\sum (x[n] - \bar{x})^2$ by hand. Then compute the range ($\max - \min$) and explain in one sentence which feature – variance or range – you would trust more to flag a sharp, one-shot tap, and why.

Exercise 38.3. Two accelerometer axes over a window are $x = (1, 2, 3, 2, 1)$ and $y = (2, 4, 6, 4, 2)$. (a) Compute the Pearson correlation ρ_{xy} from (38.5). (b) Without recomputing, state ρ for $y' = (-2, -4, -6, -4, -2)$ and explain geometrically why scaling and sign flips affect (or do not affect) the correlation.

Exercise 38.4. You build an onset detector with a single threshold θ (no hysteresis). The energy hovers right around θ during a slow gesture, dithering above and below it due to noise. Describe the failure you will observe at the controller's output, and explain quantitatively how splitting θ into $\theta_{\text{on}} > \theta_{\text{off}}$ fixes it. What is the cost of making the gap too wide?

Exercise 38.5. A dataset is 92% "rest" and 8% "gesture" windows. A lazy classifier always predicts "rest." (a) What is its accuracy? (b) What are its precision and recall *for the gesture class*? (c) What is its F1 for the gesture class? Use this to argue in two sentences why F1 (or a confusion matrix) is the right metric here.

Exercise 38.6. Compute a small DTW distance. Let $a = (0, 2, 0)$ and $b = (0, 0, 2, 2, 0)$ with local cost $d(i, j) = |a_i - b_j|$. Fill in the 3×5 cumulative cost grid using (38.9) and report D at the top-right corner. Sketch (or describe) the optimal warping path and say which samples of b get matched to the peak of a.

Exercise 38.7. You report 96% accuracy by randomly splitting 50%-overlapping windows into 80% train / 20% test. A colleague says the number is meaningless. (a) Explain the specific leak. (b) Describe two concrete changes to the split that would make the estimate trustworthy. (c) Predict qualitatively how the reported accuracy will change after each fix, and why.

Exercise 38.8. Design a five-feature vector to distinguish four wrist gestures: *rest*, *flick*, *wrist roll*, and *double-tap*. For each feature name it, say whether it is time- or frequency-domain, and state in one phrase which pair of gestures it primarily helps separate. Then identify which two gestures you expect to be hardest to tell apart and what extra feature or sensor would help.

Exercise 38.9. (Cross-subject reasoning.) You train on five people and test LOSO, getting per-fold accuracies $\{0.81, 0.62, 0.88, 0.55, 0.79\}$. (a) Report the mean and the spread. (b) The fold at 0.55 is one specific held-out person; give two physical reasons that one user might be much harder than the others. (c) Propose one change to the feature set that targets the most likely cause, and explain why it should help.

Exercise 38.10. (Real-time budget.) Your microcontroller must, every hop, compute the feature vector and run a small decision tree, all within the hop interval. With $N = 200$, which features in the reference code of Section 38.10 are $O(N)$ and which are $O(N \log N)$? If you must cut cost, which single feature would you drop first and which would you never drop, and why?

Exercise 38.11. (Sensing-limits check.) A teammate proposes that, using only the wrist IMU, the controller will continuously output each of the five fingers' joint angles to drive five independent synth voices. State plainly whether this is achievable with a wrist IMU alone, explain the physical reason in one or two sentences, and describe the minimal sensing change that would make a *discrete* per-finger scheme (which finger tapped) feasible. Then say which sensing change is required to recover the *continuous* joint angles, and why it cannot live at the wrist.

Chapter 39

Machine Learning on Signals

For most of this book we built filters and transforms by hand. We knew, from the physics and the geometry, exactly what to compute: a band-pass to isolate a rhythm, a spectrogram to watch frequencies move, an envelope to track muscle effort. That hand-design works beautifully when we understand the signal. But the wrist controller we are building has to answer a messier question: *which gesture is this?* The map from a window of accelerometer and EMG samples to the label "pinch" or "fist" or "rest" is not a clean formula. It is a tangled boundary in a high-dimensional space, different for every person's anatomy, drifting as the band slips on the arm. Nobody can write that boundary down by hand.

So we let a model *learn* it from examples. This chapter is about that move: from features we compute (the audio- and IMU-feature chapters) to a decision rule fit to data. We will keep the geometry front and center – a classifier is a way of carving feature space into regions – and we will be blunt about the two things that sink real gesture systems: *overfitting* (a model that memorizes its training people instead of learning the gesture) and the *cross-subject gap* (a model that works on you and fails on everyone else). We end on the constraints that matter when the model has to run inside a coin-cell-powered microcontroller strapped to a wrist.

This chapter is a tour, not a prerequisite. The wrist controller we build in the capstone does not rely on a learned model: it uses direct per-finger calibration and template matching, which are simpler to reason about and need no training set. Read this chapter to understand what machine learning buys you and what it costs, so you can decide when it is worth reaching for. For our build, it usually is not.

This chapter leans on the companion books. Probability gives us the language of distributions, expectation, and generalization; linear algebra gives us inner products and the geometry of decision boundaries. We will point to those connections as we go.

39.1 Classification as carving up feature space

Suppose the feature extractor of the previous chapter has already turned each one-second window of sensor data into a vector $\mathbf{x} \in \mathbb{R}^d$. A typical IMU window might give $d \approx 20$ numbers: mean and variance of each accel axis, dominant frequency, spectral energy in a few bands, the correlation between axes, and so on. Each window is now a single *point* in a d-dimensional space. A gesture, repeated many times, produces a *cloud* of points – never identical, because of jitter, noise, and the

fact that you never make a fist exactly the same way twice. Figure 39.1 shows where one such point comes from: a raw sensor trace, a window, a few numbers.

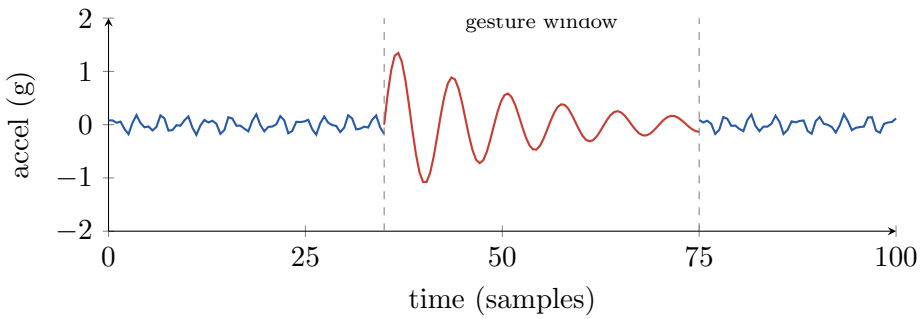


Figure 39.1: A one-axis accelerometer trace: quiet rest (blue) bracketing a "wave" burst (red). The feature extractor turns the boxed window into a handful of numbers – its energy, dominant frequency, and so on – which become one point in feature space.

Intuition

Forget twenty dimensions for a moment and picture two features on a sheet of paper: horizontal axis = signal energy, vertical axis = dominant frequency. "Rest" windows cluster in the bottom-left (low energy, low frequency). "Wave" windows scatter top-right (high energy, a clear oscillation). Classification is nothing more than *drawing a line (or curve) on this sheet* so that new points land on the correct side. Everything else in this chapter is about how to draw that line well, in spaces too high-dimensional to sketch.

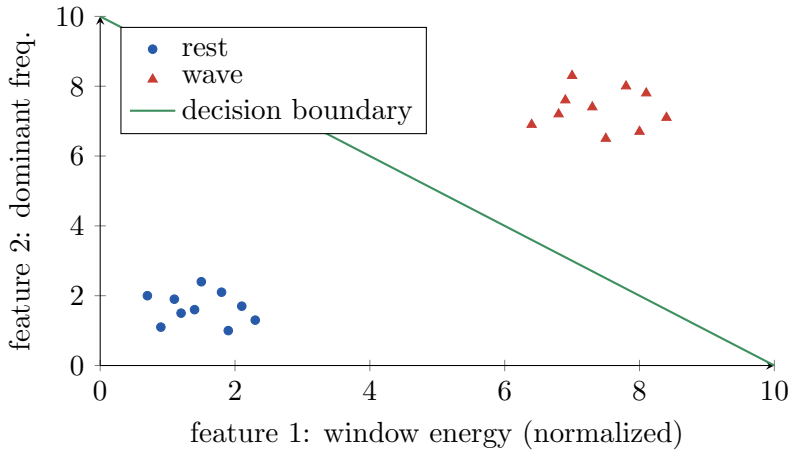


Figure 39.2: Two gesture classes as point clouds in a 2-D feature space. A classifier draws the green boundary; new windows are labeled by which side they fall on. Real problems live in $\mathbb{R}^{20}$ or higher, but the picture is the same.

Connection

A feature vector $\mathbf{x}$ is exactly the kind of object the linear-algebra book studies: a point in $\mathbb{R}^d$ with an inner product. A linear decision boundary is the set $\{\mathbf{x} : \langle \mathbf{w}, \mathbf{x} \rangle + b = 0\}$ – a hyperplane, the level set of a linear functional. The weight vector $\mathbf{w}$ is its normal direction. Classification geometry *is* the geometry of hyperplanes and projections you already met there.

39.2 Logistic regression: a soft hyperplane

The simplest useful classifier draws a single flat boundary – a hyperplane – and asks which side a point is on. But a hard "side" gives no sense of confidence: a window parked right on the boundary should not be reported with the same certainty as one deep in fist territory. *Logistic regression* fixes this by turning distance-from-the- boundary into a probability.

We compute a linear score, the signed distance (up to scale) from the hyperplane,

$$z \;=\; \langle \mathbf{w}, \mathbf{x} \rangle + b \;=\; \sum_{i=1}^{d} w_i x_i + b,$$

and squash it through the *logistic (sigmoid)* function

$$\sigma(z) \;=\; \frac{1}{1 + e^{-z}} \;\in\; (0, 1),$$

which we read as the model's estimated probability that the window is the positive class: $\hat{p} = P(y{=}1 \mid \mathbf{x}) = \sigma(z)$.

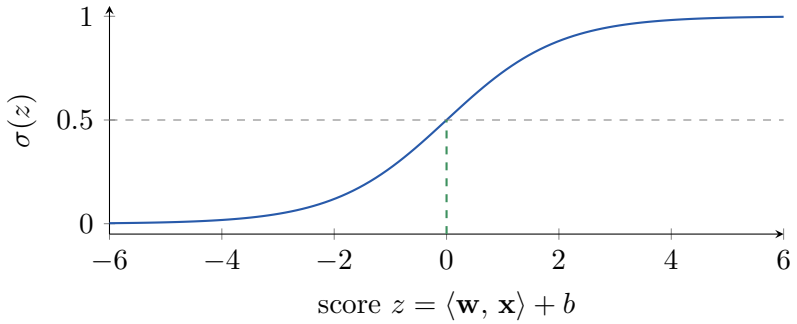


Figure 39.3: The logistic function maps any real score to a probability in $(0, 1)$. At the boundary ($z = 0$) the model is maximally unsure ($\hat{p} = 0.5$); far from it, the probability saturates toward 0 or 1.

Why it works

Why the sigmoid and not, say, a straight ramp? Two reasons. First, its output is always a valid probability, no clipping needed. Second, and deeper: if you assume each class is a Gaussian blob with the same covariance, Bayes' rule gives *exactly* a logistic form for $P(y{=}1 \mid \mathbf{x})$, linear in $\mathbf{x}$ inside the exponent. Logistic regression is therefore not an arbitrary squashing – it is the optimal posterior for a natural probabilistic model of two clouds. (See the probability book

on Gaussians and Bayes' rule.)

39.2.1 Training: maximize the probability of the data

We have parameters $\mathbf{w}, b$ and a labeled training set $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$ with $y^{(n)} \in \{0, 1\}$. We pick the parameters that make the observed labels most probable. The probability the model assigns to one example is $\hat{p}^{\,y}(1-\hat{p})^{1-y}$, and assuming independent examples, the log-likelihood of the whole set is a sum. Maximizing it is the same as minimizing the average *cross-entropy loss*

$$L(\mathbf{w}, b) \;=\; -\frac{1}{N}\sum_{n=1}^{N}\Big[\, y^{(n)} \log \hat{p}^{(n)} \;+\; (1 - y^{(n)}) \log\big(1 - \hat{p}^{(n)}\big) \,\Big].$$

There is no closed-form minimizer, so we use *gradient descent*: nudge the parameters downhill on L. The gradient is strikingly clean,

$$\frac{\partial L}{\partial \mathbf{w}} \;=\; \frac{1}{N}\sum_{n=1}^{N}\big(\hat{p}^{(n)} - y^{(n)}\big)\,\mathbf{x}^{(n)},$$

i.e. average the input vectors weighted by the prediction error. Each step moves the boundary to reduce its mistakes.

Connection

That gradient is just the chain rule from the calculus book applied to a composition $L \circ \sigma \circ$ (linear), and the loss is the negative log-likelihood the probability book derives from independence. Three books meet in one update rule.

Example 39.1 (Logistic boundary by hand)**.** Take one feature, EMG envelope energy x, and two classes: rest (y=0) and clench (y=1). Suppose after training $w = 1.5$, $b = -6$, so $z = 1.5x - 6$. The boundary $z = 0$ sits at $x = 4$. A window with $x = 5$ gives $z = 1.5$, so $\hat{p} = \sigma(1.5) \approx 0.82$ – "probably a clench." A window with $x = 4.2$ gives $z = 0.3$, $\hat{p} \approx 0.57$ – barely past the line, low confidence. The slope w controls how fast confidence rises as you move away from $x = 4$; a larger w makes a sharper, more decisive boundary.

39.2.2 Regularization: the first guard against overfitting

If a feature happens to separate the training set perfectly, gradient descent will push its weight toward infinity to make $\hat{p}$ hit exactly 0 or 1 – supreme confidence built on a coincidence. We discourage this by adding a penalty on large weights, typically the L_2 term $\frac{\lambda}{2}\|\mathbf{w}\|^2$, to the loss. This *regularization* keeps the boundary smooth and the confidence sober; λ is a knob we tune on validation data (Section 39.8). It is our first concrete defense against a model that takes its training data too literally.

39.3 Support vector machines: the widest street

Logistic regression will accept any separating hyperplane that lowers the loss. But if two classes are cleanly separated, infinitely many hyperplanes separate them – which is best? The *support vector*

machine (SVM) gives a crisp geometric answer: choose the boundary that leaves the *widest empty margin* between the classes, the one that stays as far as possible from the nearest points of both clouds.

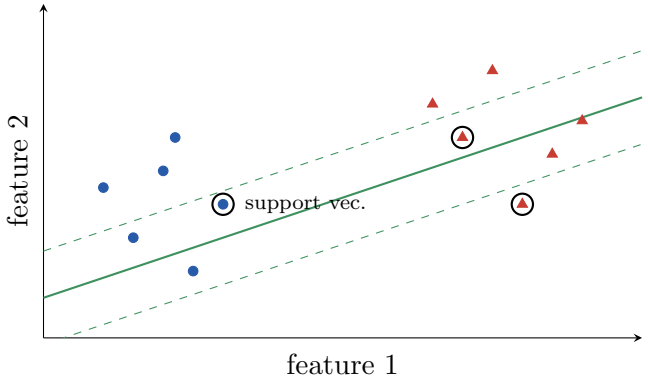


Figure 39.4: The SVM picks the boundary (solid) with the widest margin (dashed gutters). Only the closest points – the circled *support vectors* – determine the boundary; points far from the gutter could move without changing anything.

The distance from a point to the hyperplane $\langle \mathbf{w},\, \mathbf{x}\rangle + b = 0$ is $|\langle \mathbf{w},\, \mathbf{x}\rangle + b|/\|\mathbf{w}\|$. Fixing the scale so the closest points satisfy $|z| = 1$, the margin half-width is $1/\|\mathbf{w}\|$. Maximizing the margin therefore means *minimizing* $\|\mathbf{w}\|$ subject to every point being on the correct side by at least the margin:

$$\min_{\mathbf{w},b}\ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y^{(n)}\big(\langle \mathbf{w},\, \mathbf{x}^{(n)}\rangle + b\big) \geq 1 \quad \forall n,$$

where labels are written as $y^{(n)} \in \{-1, +1\}$. Real data are not cleanly separable, so we allow slack: points may sit inside the margin or on the wrong side, at a cost. This gives the *soft-margin* SVM, whose loss is the hinge loss $\max(0,\, 1 - y\, z)$ plus the same $\|\mathbf{w}\|^2$ penalty – again a margin-vs- violations trade-off controlled by a knob C.

What is really happening

Notice what the SVM ignores. Once the margin is set, only the support vectors (the few points touching the gutter) matter; the rest of each cloud is irrelevant. This is the opposite of logistic regression, which is nudged by *every* point. For gesture data with one dominant, well-separated rest class, the SVM's focus on the hard, borderline windows is often exactly what you want.

39.3.1 The kernel trick: curved boundaries for free

A flat hyperplane cannot separate clouds that wrap around each other – think "slow wrist roll" versus "fast wrist roll," where the deciding feature is a *ring* in feature space, not a half-plane. The kernel trick maps points into a higher-dimensional space where they *do* separate linearly, without ever computing the map explicitly. Because the SVM only ever needs inner products $\langle \mathbf{x},\, \mathbf{x}'\rangle$, we replace them with a *kernel* $k(\mathbf{x}, \mathbf{x}')$ – the inner product in the lifted space. The popular radial basis function (RBF) kernel

$$k(\mathbf{x}, \mathbf{x}') \;=\; \exp\!\big(-\gamma\, \|\mathbf{x} - \mathbf{x}'\|^2\big)$$

yields smooth, curved boundaries that bend around clusters. The price: an RBF-SVM must store the support vectors and compute a kernel against each one at inference time, which matters on a microcontroller (Section 39.11).

39.4 Random forests: many simple questions

Linear and kernel methods think in terms of distances and dot products. A *decision tree* thinks in terms of yes/no questions about one feature at a time: "Is the accel-z variance above 0.4? If yes, is the dominant frequency below 3 Hz?" Each question splits the data; following the branches leads to a leaf that votes for a class. A single tree carves feature space into axis-aligned boxes.

A single deep tree overfits wildly – it will keep asking questions until every training window sits alone in its own box. The *random forest* tames this by averaging many trees, each trained on a bootstrap resample of the data and allowed to consider only a random subset of features at each split. The trees disagree in their mistakes but agree on the real structure, so their majority vote is far more robust than any one tree.

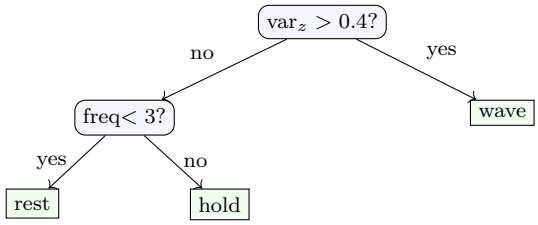


Figure 39.5: One small decision tree on IMU features. A random forest grows hundreds of such trees on resampled data and votes; the diversity is what prevents any single tree's overfitting from dominating.

Why it works

Why does averaging help? Each tree's prediction is a noisy estimate of the truth. If the trees' errors were perfectly correlated, averaging would do nothing. Bootstrap sampling and random feature subsets *decorrelate* the errors, and the variance of an average of T weakly-correlated estimators shrinks roughly like $1/T$ plus a residual correlation floor. This is the same variance-reduction-by-averaging argument the probability book makes for the sample mean – applied to predictors instead of numbers.

Why this matters

Random forests are often the smartest first thing to try on hand-crafted signal features. They need almost no feature scaling, ignore irrelevant features gracefully, handle nonlinear boundaries, rarely overfit catastrophically, and – crucially for us – hand back a *feature-importance* ranking that tells you which features earned their keep. That ranking is a guide to trimming your feature set down to what fits on an MCU.

39.5 When to hand-craft features and when to learn them

Everything above assumes someone already computed a good feature vector. Choosing those features – spectral energies, zero-crossing rates, axis correlations – is itself an act of design, and it bakes in your understanding of the signal. That is a strength (you inject physics) and a weakness (you might omit the very pattern that distinguishes two gestures). Neural networks offer the alternative: feed the model something close to raw samples and let it discover its own features.

Common trap

"Deep learning means I can skip feature engineering" is a tempting and expensive mistake on small sensor datasets. With a few thousand labeled windows from a dozen people, a random forest on well-chosen features will usually *beat* a raw-window neural net, train in seconds, and fit on the microcontroller. Raw-input nets pay off when you have a lot of data and patterns too subtle to name. Know which regime you are in before you reach for the heavy machinery.

39.6 Small neural networks

A neural network stacks the logistic idea. One *neuron* computes a weighted sum and a nonlinearity, $a = \phi(\langle \mathbf{w}, \mathbf{x} \rangle + b)$; a *layer* runs many neurons in parallel; stacking layers lets later neurons build features out of earlier ones. With just one hidden layer,

$$\mathbf{h} = \phi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \qquad \hat{\mathbf{p}} = \mathrm{softmax}(\mathbf{W}_2 \mathbf{h} + \mathbf{b}_2),$$

the network can represent boundaries no single hyperplane can. The nonlinearity ϕ is the engine: without it, stacking linear maps just gives another linear map. The modern default is the *ReLU*, $\phi(z) = \max(0, z)$ – cheap to compute and cheap to differentiate, which matters both for training speed and for MCU inference.

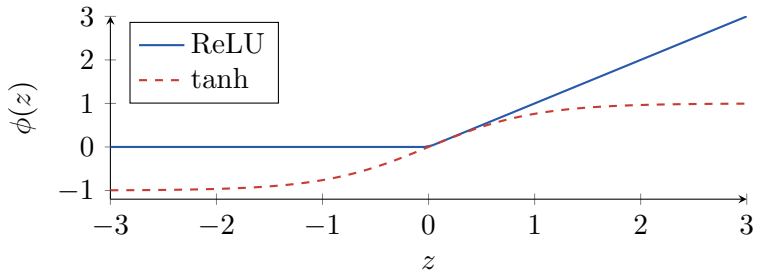


Figure 39.6: Two common nonlinearities. The ReLU's flat-then-linear shape is fast and avoids the saturation that slows learning; tanh is smooth but saturates at both ends.

39.6.1 Softmax for several gestures

Our controller has more than two gestures, so the output layer produces one score per class and normalizes them into a probability distribution with the *softmax*:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad \sum_k \hat{p}_k = 1.$$

Training minimizes multi-class cross-entropy, and the gradient is computed by *backpropagation* – the chain rule run backward through the layers, reusing each layer's stored output. It is exactly the calculus book's chain rule, organized to avoid recomputation.

Common trap

A small net has lots of parameters relative to a small gesture dataset, so it overfits even more eagerly than logistic regression. The standard countermeasures are *dropout* (randomly zero some hidden units during training, forcing redundancy), weight decay (the same L_2 penalty), and *early stopping* (halt when validation loss stops improving). Use them; an unregularized net on a few thousand windows will report wonderful training accuracy and fail in the field.

39.7 1-D convolutional networks on raw windows

There is a beautiful bridge between this chapter and the rest of the book. A convolutional neural network does not invent a new operation – its core layer is the *convolution* we spent whole chapters on. A 1-D CNN slides a short learnable filter (kernel) along the raw signal window and records the response at each position, exactly like an FIR filter, except the taps are *learned* rather than designed.

What is really happening

A hand-designed band-pass filter has fixed taps chosen so its frequency response passes 8–12 Hz. A learned conv kernel has taps chosen by gradient descent to maximize gesture-classification accuracy. The machinery is identical – multiply, shift, add (the convolution sum $\sum_k h[k]\,x[n-k]$) – but the design objective moved from "match this frequency response" to "be useful for the task." A CNN is a stack of FIR filters whose coefficients the data chooses.

A typical 1-D CNN for gestures looks like: a few conv layers (each a bank of short learned filters followed by a ReLU), interleaved with *pooling* that downsamples by keeping the max in each small window (cheap translation tolerance, so a gesture detected slightly early or late still fires), then a small dense head with a softmax. Because the same kernel is reused at every position (*weight sharing*), a CNN has far fewer parameters than a dense net reading the same raw window – a real advantage when data is scarce and memory is tight.

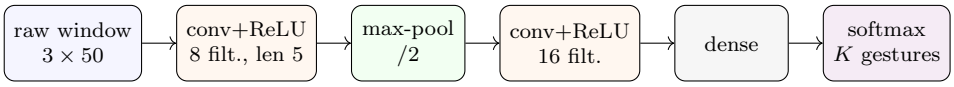


Figure 39.7: A small 1-D CNN for gesture windows. Input is three accel axes × 50 samples. Early conv layers learn FIR-like filters; pooling adds time-shift tolerance; the dense head maps learned features to gesture probabilities.

Toward the wrist tracker

Whatever classifier you choose, the sensor sets the ceiling. A wrist IMU sees *gross forearm motion*: a wave, a twist, a tap, a sharp flick. It does not see individual finger joints – those move with almost no signature at the wrist. If your gesture set is "open hand vs. closed fist held still," the IMU is nearly blind to it; you need EMG or force-myography for

finger-level discrimination (the hand-sensing chapter). A classifier can only be as good as the information the sensor captured. This is exactly why our capstone leans on per-finger sensing and calibration rather than on a learned model guessing gestures from coarse wrist motion: the better lever is the signal, not the algorithm.

39.8 Evaluating without fooling yourself: train, validation, test

A model's accuracy on the data it was trained on is almost meaningless – a lookup table that memorizes the training set scores 100% and learns nothing. What we care about is *generalization*: accuracy on windows the model has never seen. To estimate it we split our data into three disjoint parts.

- **Training set** – used to fit the parameters $\mathbf{w}, b, \mathbf{W}$.
- **Validation set** – used to choose hyperparameters (λ, C, γ, number of trees, network size, when to stop). The model never trains on it, but *you* tune against it, so it slowly leaks information.
- **Test set** – touched exactly once, at the very end, to report true performance. If you peek at it while developing, it stops being a test set.

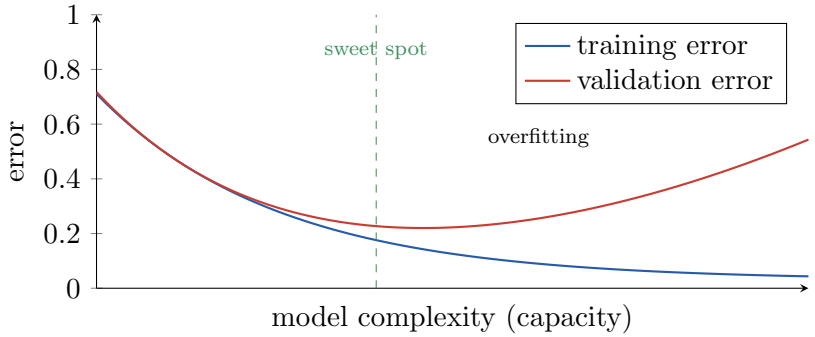


Figure 39.8: The overfitting picture. As capacity grows, training error keeps falling but validation error turns back up: the model starts memorizing noise. We pick the complexity at the validation minimum, not the training minimum.

Why it works

Overfitting is the central hazard, so it is worth saying precisely what it is. A flexible model can fit the *noise* in the training set – the random jitter that will be different next time – as if it were signal. The probability book frames this as the bias–variance trade-off: a too-simple model is biased (it cannot represent the truth), a too-complex one has high variance (it swings wildly with the particular training sample). Validation error is our empirical estimate of where that trade-off is best.

39.8.1 Cross-validation when data is scarce

Gesture data is expensive: every window is someone making a gesture on cue. A single fixed split then wastes data and gives a noisy estimate. *k-fold cross-validation* uses it all: partition into k folds, train on $k-1$ and validate on the held-out fold, rotate through all k choices, and average. You get a more stable estimate and a sense of its variance across folds.

39.9 The cross-subject gap (the one that bites)

Here is the single most important point in this chapter, and the one most often gotten wrong. **How you split matters more than the model.** If you pool all windows from all people and split randomly, windows from the *same person* – even the same recording session – land in both train and test. The model can then succeed by recognizing the *person* (their arm shape, band placement, skin impedance, personal style) rather than the *gesture*. Your reported accuracy looks superb and collapses the moment a new user straps on the band.

The fix is *cross-subject* (also called leave-one-subject-out) evaluation: hold out *all* data from some people, train on the rest, test on the held-out strangers. This measures what we actually care about – does the system work on a person it has never seen?

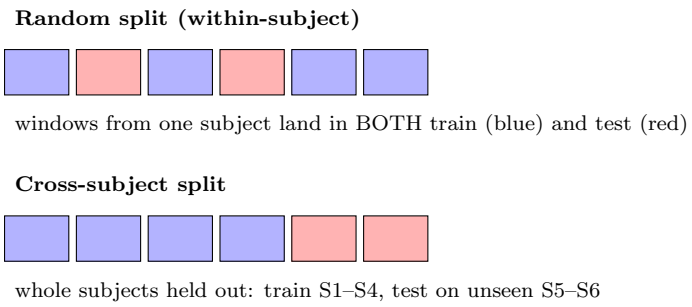


Figure 39.9: Two ways to split. Top: random split mixes each subject across train/test, so a model can cheat by identifying the person. Bottom: cross-subject split holds out whole people, measuring true generalization to new users.

Connection

This is the probability book's notion of the *i.i.d. assumption* failing. Windows from one person are not independent of each other – they share that person's quirks. The right unit of independence is the *subject*, not the window, so the right split is by subject. Get the unit of independence wrong and every confidence interval you compute is a fantasy.

Example 39.2 (How big the gap can be)**.** A 1-D CNN on a 6-gesture IMU set might report 97% accuracy under random splitting and only 74% under leave-one-subject-out – a 23-point drop that is *entirely* an artifact of how you evaluated, not of any change to the model. The 74% is the number that predicts field performance. Gesture papers that report only random-split numbers are, intentionally or not, overstating real-world accuracy. Always ask which split a number came from.

Toward the wrist tracker

The cross-subject gap is the reason our capstone skips the learned model entirely. Even a good cross-subject model usually needs a *tiny per-user calibration* – ask a new user to perform each gesture three or four times at setup – to close the gap. But once you accept a short enrollment, you may as well calibrate *directly*: record each user's own gesture templates and match against them, no training set required. That is the path the wrist controller takes. The lesson to carry forward is the enrollment step, not the classifier: expect a 30-second setup, not a one-size-fits-all model.

39.10 Beyond accuracy: confusion and thresholds

A single accuracy number hides which gestures get mixed up. The *confusion matrix* counts, for each true gesture (row), how often the model predicted each label (column). Its diagonal is correct calls; off-diagonal entries are the specific mistakes – and they are gold for design. If "pinch" and "rest" are confused but everything else is clean, you learn that your sensor barely sees the pinch, and the fix is better sensing, not a fancier model.

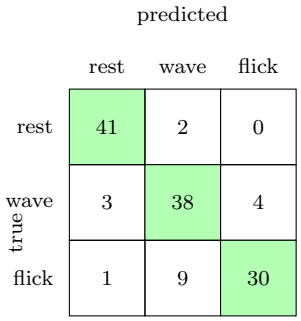


Figure 39.10: A confusion matrix for three gestures. The diagonal (shaded) is correct; here "flick" is most often mistaken for "wave" (9 windows), telling us exactly where the sensor or features fall short.

For a controller, you can also tune the *decision threshold*: only emit a MIDI event when $\max_k \hat{p}_k$ exceeds, say, 0.8, and otherwise stay silent. This trades a few missed gestures for far fewer false triggers – usually the right call for an instrument, where a spurious note is more jarring than a dropped one.

39.11 On-device inference on a microcontroller

Training happens on a laptop. *Inference* – running the trained model on each new window – has to happen on the wrist, on a microcontroller (MCU) with maybe 256 KB of RAM, no floating-point luxury to spare, and a battery you do not want to murder. Three budgets dominate.

- **Latency.** For a responsive instrument the path from motion to MIDI note should stay under roughly 30 ms; humans notice audio latency above that. With a windowed classifier the window length itself adds delay, so short hops and a fast model matter.

- **Memory.** The model's parameters plus activation buffers must fit in flash and RAM. A random forest of 100 deep trees can be larger than a small quantized CNN; an RBF-SVM must store every support vector. Count bytes early.
- **Energy.** Every multiply costs charge. Fewer parameters and integer arithmetic both extend battery life directly.

Build it (hardware)

Concrete tactics that make a model fit on an MCU class like an ARM Cortex-M4:

- **Quantize to int8.** Convert float weights/activations to 8-bit integers (4× smaller, and integer multiply-accumulate is fast). Frameworks like TensorFlow Lite for Microcontrollers and CMSIS-NN do this and ship the kernels.
- **Prefer small models.** A 2-layer 1-D CNN with a few thousand parameters, or a shallow random forest on ∼15 features, usually beats a big net once you account for the cross-subject gap anyway.
- **Compute features on the MCU.** If you classify hand-crafted features, the FFT/energy/zero-crossing code from earlier chapters runs on-device; only the tiny classifier needs porting.
- **Measure, don't guess.** Toggle a GPIO pin high at window-start and low when the label is ready; read the pulse width on a scope. That is your true end-to-end latency.

Listing 39.1: Laptop side: train, evaluate cross-subject (leave-one-subject-out, not random), and export.

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import LeaveOneGroupOut

X, y, subject = load_windows()          # features, labels, per-window subject id
logo = LeaveOneGroupOut()               # the unit of independence is the SUBJECT
accs = []
for tr, te in logo.split(X, y, groups=subject):
    clf = RandomForestClassifier(n_estimators=50, max_depth=8)
    clf.fit(X[tr], y[tr])
    accs.append(clf.score(X[te], y[te]))
print("cross-subject accuracy:", np.mean(accs), "+/-", np.std(accs))
export_thresholds_to_c(clf)             # emit a small C header of split rules
```

Listing 39.2: MCU side: the per-window inference loop. Read a window, compute a few integer features, run a tiny classifier, emit MIDI above threshold.

```
// runs once per hop (e.g., every 100 ms) inside the main loop
float feat[NF];
extract_features(window, feat);          // mean, var, dominant freq, band energy...
int   gesture; float conf;
classify(feat, &gesture, &conf);         // tiny ported model: tree votes or int8 net
if (conf > 0.80f && gesture != REST)
    midi_send_note(GESTURE_TO_NOTE[gesture]);
```

Application

The same pattern – window, feature, tiny model, threshold – powers a fitness band's step and activity classifier, a hearing aid's speech-vs-noise switch, a smartwatch's fall detector, and a keyword-spotting chip that wakes on "Hey." All run inference under a milliamp on an MCU; all face the same latency/memory/energy triangle. Learning it once for the wrist controller transfers everywhere embedded ML lives.

39.12 Data collection and labeling for a gesture set

Models are only as good as their data, and for gestures the data is something *you* must record – there is no public dataset for your exact band, electrodes, and gesture vocabulary. This section is the unglamorous, decisive part of the project. (Even the template-matching capstone needs this data discipline – it just feeds the recordings into calibration templates instead of into a model fit.)

Choose a gesture set the sensor can resolve. Before recording a single window, list the gestures and ask, for each pair, "what in the signal distinguishes these?" If the answer is "nothing my sensor can see," cut one. A good wrist-IMU starter set is *rest, wave, twist (pronate/supinate), tap, sharp flick* – all gross motions with distinct accel/gyro signatures. Resist adding "point with index finger," which the wrist IMU cannot resolve from a flat hand.

Record many repetitions, many people, many sessions. Variation is the whole point. Collect from at least 8–12 people so cross-subject evaluation has held-out strangers to test on. Record multiple sessions per person, taking the band off and re-donning it between sessions, because band placement is one of the biggest sources of drift – if the model only ever saw one placement it will fail when the band rotates 1 cm.

Label cleanly. Use a cued protocol: a screen prompts "WAVE" for two seconds, the subject performs, you tag those samples. Discard the transition regions at the start and end of each cue, where the hand is still moving into or out of the gesture and the label is genuinely ambiguous. Always keep a generous *rest / null* class – in deployment the hand is at rest the vast majority of the time, and a model that never learned "nothing is happening" will fire constantly.

Common trap

Two labeling traps that quietly inflate your accuracy: (1) recording all of one person's gestures back-to-back in one session, so the model latches onto that session's exact band placement; and (2) letting windows overlap across the train/test boundary, so a single gesture's first half trains and its second half tests – near-duplicate leakage. Split by subject and by recording, and keep overlapping windows entirely on one side of the split.

Toward the wrist tracker

A realistic first dataset for a learned gesture system: 10 people × 3 sessions × 5 gestures × 20 repetitions, windowed at 1 s with 50% overlap. That is a few thousand windows – plenty for

a random forest on hand features, marginal for a CNN on raw windows. If you go the learned route, start with features and a forest, get a clean cross-subject number, and only reach for a CNN if more data and a richer gesture set justify it. For the capstone we spend that same recording effort on per-user calibration templates instead, which sidesteps the cross-subject gap by never training a shared model in the first place (Section 39.9).

39.13 Exercises

Exercise 39.1. A logistic-regression gesture detector has $\mathbf{w} = (2, -1)$ and $b = -1$ over features (x_1, x_2). (a) Write the decision boundary as a line in the plane. (b) Classify the window $\mathbf{x} = (2, 1)$: compute z, then $\hat{p} = \sigma(z)$, and give the predicted label at threshold 0.5. (c) Which side of the boundary does $\mathbf{w}$ point toward, positive or negative class?

Exercise 39.2. Using the cross-entropy gradient $\partial L/\partial \mathbf{w} = \frac{1}{N}\sum(\hat{p}^{(n)} - y^{(n)})\mathbf{x}^{(n)}$, explain in one or two sentences why a correctly and confidently classified example ($\hat{p} \approx y$) contributes almost nothing to the gradient, while a confident *mistake* contributes a lot. What does this say about which examples drive learning?

Exercise 39.3. For a separable two-class problem, an SVM finds margin half-width $1/\|\mathbf{w}\|$. If retraining on more data yields a *larger* $\|\mathbf{w}\|$, did the margin get wider or narrower, and what does that suggest about how separable the new data is?

Exercise 39.4. You train a model and observe: training accuracy 99%, validation accuracy 71%, test accuracy 70%. (a) Name the phenomenon. (b) List three concrete changes that should narrow the train–validation gap. (c) Why is it reassuring that validation and test accuracy are close to each other?

Exercise 39.5. A colleague reports 96% gesture accuracy. You ask how they split the data and learn they pooled all subjects and split windows randomly. Explain precisely what the model may have learned instead of the gestures, and describe the experiment you would run to find the number that predicts field performance.

Exercise 39.6. From the confusion matrix in Figure 39.10: (a) compute the overall accuracy. (b) Compute the per-class recall for "flick" (fraction of true flicks correctly labeled). (c) Which single confusion would you attack first, and would you attack it with a better sensor, better features, or a bigger model? Justify.

Exercise 39.7. Explain in your own words why a 1-D convolution layer is "a stack of FIR filters whose taps are learned." Then state one thing a learned conv kernel can do that a fixed band-pass cannot, and one thing the fixed band-pass guarantees that the learned kernel does not.

Exercise 39.8. You must fit a classifier in 64 KB of flash with a 20 ms latency budget. Compare a random forest of 100 trees of depth 12, an RBF-SVM with 800 support vectors, and a quantized 2-layer 1-D CNN with 4,000 int8 weights. Estimate the storage of each (state your assumptions) and argue which is the safest choice and why.

Exercise 39.9. Design a cued data-collection protocol for the five-gesture set {rest, wave, twist, tap, flick}. Specify: number of subjects and sessions, repetitions per gesture, window length and overlap, how you handle the transition regions between cues, and how you will split the resulting data for a trustworthy cross-subject evaluation.

Exercise 39.10. A wrist-IMU gesture system works well in the lab but users complain it misfires while they walk. (a) Why would walking specifically confuse an accel/gyro gesture model? (b) Propose two fixes – one in data collection, one in the runtime decision logic – and explain what each one costs.

Exercise 39.11. (Sensor-limits check.) A product manager wants the wrist band to detect which individual finger the user is curling, using only the IMU. Write a short, clear explanation of why this is not possible with a wrist IMU alone, what sensing modality *would* be required, and what coarser capability you can deliver instead.

Exercise 39.12. (Programming.) Using the laptop-side code sketch as a starting point, write a script that loads windowed features, runs both a random-split and a leave-one-subject-out evaluation of a random forest, and prints both numbers with their standard deviations. Comment on the size of the gap you would expect to see and why the leave-one-subject-out number is the one to trust.

Part IX

Building It: Embedded DSP

Chapter 40

Embedded Signal Processing on Arduino and Raspberry Pi

Everything in this book so far has lived in a comfortable place: a notebook running NumPy, a plot that redraws in a quarter second, arrays of `float64` that never overflow and never run out. That is the right place to *learn* DSP. It is not where DSP *lives*. The spectrogram that watches your forearm, the filter that smooths an accelerometer trace, the peak-picker that decides a finger just tapped – in the finished wrist controller those run on a chip the size of a fingernail, powered by a coin cell, with no operating system, a few kilobytes of RAM, and a hard deadline arriving every millisecond whether you are ready or not. The math is identical. The engineering could not be more different.

This chapter is about that crossing – taking the ideas we built on graph paper and making them run, in real time, on a microcontroller (MCU) like an Arduino and on a small Linux computer like a Raspberry Pi. We will see how to sample without jitter, why `float` sometimes becomes your enemy and fixed-point your friend, how a ring buffer lets a finite chip process an infinite stream, what it actually costs to run an FFT on a Cortex-M, and how processed numbers leave the device as serial, BLE, or MIDI. The through-line goal – a wrist-worn finger-gesture instrument – forces every one of these choices to be concrete. There is a real-time budget, a real power budget, and a real latency a musician will feel. By the end you should be able to look at the project and say, with reasons, "this part runs on the Arduino, that part runs on the Pi, and here is why."

A word up front about the sensing, because it shapes every decision that follows. Finger movement *is* detectable from a wrist or forearm band, and we will treat it as a solved problem rather than a wish. Two physical channels make it work. When you move a finger, the tendons in your forearm slide and the muscle bellies change shape; a snug band of force sensors reads that mechanical bulge – this is **force myography** (FMG). At the same time the muscles fire electrical pulses that a pair of skin electrodes can pick up – **surface electromyography** (sEMG). Neither one needs a camera, a glove, or a machine-learning model. The method is direct: the user runs a short guided *enrollment*, performing each finger movement a few times while the phone records the signal signature; from those recordings we build a per-user template for each finger. Live detection is then a match – compare the current signal to the calibrated templates and fire the closest one. Calibration, not classification, is the heart of the instrument, and the whole chapter builds toward making that loop run on a chip you can wear.

40.1 The shape of the problem: streaming, not batch

On a laptop you think in *batches*. You have the whole signal in an array, you call `np.fft.fft(x)`, you get the answer. On an embedded device the signal does not exist yet. It is being born one sample at a time, forever, and you must produce outputs while later inputs are still arriving. The mental model is a pipe, not a bucket.

Intuition

Batch DSP is a photograph: the whole signal is laid out, and you operate on it at leisure. Streaming DSP is a river you stand in: water arrives at a fixed rate, you must scoop and act before it flows past, and you never get to hold all of it at once. Every embedded design decision – buffer sizes, fixed-point, interrupt timing – exists to keep up with the river without falling behind and without drowning in stored water.

This single shift reshapes everything. You cannot store an infinite stream, so you keep a short sliding window (a *ring buffer*). You cannot afford a sloppy sample clock, because a filter designed for $f_s = 1000$ Hz quietly lies if the true rate wobbles. You cannot pause to garbage- collect or wait on a disk. And you have a **real-time budget**: if your per-sample (or per-block) work takes longer than the time between samples (or blocks), the river overflows and you lose data. That budget is the spine of the whole chapter.

Focus

Three numbers govern every embedded DSP design, and you should be able to state all three for any project before writing code:

1. **Sample rate** f_s – how often a new sample arrives (the deadline clock).
2. **Work per sample/block** – how many cycles your processing costs.
3. **Cycle budget** – the MCU clock times the time between deadlines.

If work per block exceeds the cycle budget, no amount of clever code saves you; you must lower f_s, shrink the algorithm, or move to a faster processor.

40.2 Getting samples in: timers, ADCs, and the enemy called jitter

Sampling theory told us *how often* to sample. Embedded reality adds a brutal footnote: the samples must land on an *evenly spaced* time grid. The whole edifice of the DFT, every filter coefficient, every frequency you read off a spectrum – all of it assumes that sample n happened at exactly $t_n = n/f_s$. When the actual instants drift around that grid, we call the error **jitter**, and it is one of the most insidious problems in embedded DSP because the code looks correct and the numbers are merely wrong.

40.2.1 Where samples come from

A microcontroller has a built-in ADC. The naive way to use it is a loop:

Listing 40.1: The WRONG way to sample: timing set by loop speed.

```
void loop() {
  int x = analogRead(A0);   // read sensor
  process(x);               // do DSP
  delay(1);                 // "1 ms" -- but is it?
}
```

This is a trap. The interval between samples is now `delay(1)` *plus* however long `analogRead` took *plus* however long `process` took *plus* whatever interrupt fired in between. The sample spacing breathes with your code. If `process` sometimes branches into a slower path, your f_s changes mid-stream.

Common trap

`delay()`-based sampling produces a sample rate that depends on your processing time. The moment your DSP load varies (a conditional, a heavier block every N samples, an occasional log-print), the time grid warps. Spectra smear, filters detune, and the bug is nearly invisible because every individual sample looks fine. **Never set your sample clock from inside the work loop.**

The right way is to let a hardware **timer** fire an interrupt at a fixed rate and have the interrupt – and *only* the interrupt – take the sample. The timer counts the crystal oscillator directly; it does not know or care what your code is doing. This decouples the sample clock from the workload.

Listing 40.2: Timer-driven sampling on an AVR Arduino (Timer1, $f_s = 1$ kHz).

```
volatile int16_t latest;
volatile bool ready;

ISR(TIMER1_COMPA_vect) {        // fires exactly every 1 ms
  latest = analogRead(A0);      // sample on the grid
  ready = true;                 // flag the main loop
}

void setup() {
  cli();
  TCCR1A = 0; TCCR1B = 0; TCNT1 = 0;
  OCR1A  = 249;                 // 16 MHz / 64 / (249+1) = 1000 Hz
  TCCR1B |= (1 << WGM12);       // CTC mode: clear timer on compare
  TCCR1B |= (1 << CS11) | (1 << CS10);  // prescaler = 64
  TIMSK1 |= (1 << OCIE1A);      // enable compare interrupt
  sei();
}

void loop() {
  if (ready) { ready = false; process(latest); }
}
```

Now the sample instants are nailed to the crystal. The main loop can be late to *process* a sample, but the sample itself was taken on time. (Better still on richer chips: have the timer trigger the ADC in hardware and have the ADC's done-interrupt fetch the result, so even the `analogRead` latency leaves the timing path. On a Cortex-M you would wire a timer to the ADC trigger and the ADC to

DMA, and the CPU only wakes when a whole block is ready.)

40.2.2 What jitter does to a spectrum

Jitter is not just untidy – it injects a noise floor. Suppose each true instant $t_n = n/f_s$ is perturbed to $t_n + \delta_n$ with δ_n small and random. For a sinusoid $x(t) = A\sin(2\pi f t)$ the sampled value is

$$x(t_n + \delta_n) \approx A\sin(2\pi f t_n) + \underbrace{2\pi f A\cos(2\pi f t_n)}_{\text{slope}}\,\delta_n.$$

The error term scales with f: *the higher the frequency, the more a fixed timing error costs you.* A timing wobble of δ seconds turns into an amplitude error proportional to $2\pi f\delta$. This is why high-frequency, high-resolution sampling is so demanding on the clock, and why jitter shows up as a raised, roughly white noise floor that worsens toward the top of the band.

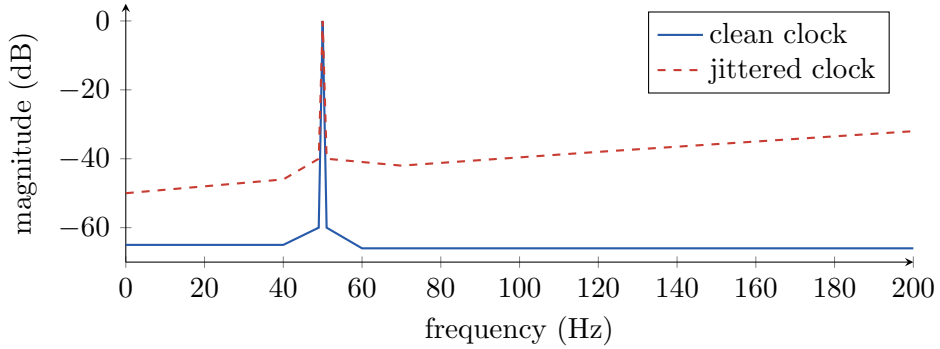


Figure 40.1: Sampling a single 50 Hz tone. A clean timer keeps the noise floor low and flat (solid). Clock jitter (dashed) raises the floor, and the rise grows toward higher frequencies because a fixed timing error perturbs fast-moving samples more.

Why it works

A hardware timer fires from a counter clocked by the crystal oscillator, completely independent of what the CPU is executing. Its edges are as regular as the crystal itself (tens of ppm). A software `delay()` loop, by contrast, measures time in instructions executed, so anything that changes the instruction count – a branch, an interrupt, a cache miss on bigger chips – moves the sample. Timers work because they move the timing decision out of software entirely and into dedicated silicon.

Toward the wrist tracker

The wrist controller streams several channels at once: a MEMS IMU (accelerometer and gyro), an FMG array of force sensors reading tendon bulge, and one or two sEMG channels reading muscle pulses. For gesture timing a musician can feel, you want low, *stable* latency, so every sample clock must be rock-steady. Practically: configure the IMU's own internal sample-rate generator (a good MEMS IMU has one, e.g. an ICM-20948 at a programmed output data rate) and read it over its FIFO, rather than polling `analogRead` in `loop()`. The IMU's clock becomes your time grid. For the analog FMG and EMG channels, drive the MCU

ADC from a timer as above so they land on the same grid and can be compared sample for sample. Either way, the rule holds: the clock comes from hardware, never from the work loop. Sample-aligned channels are what let the calibration step later treat one finger movement as a single multi-sensor signature.

40.3 Ring buffers: a finite window onto an infinite stream

You cannot store the whole stream, but most DSP only needs a short recent history: an FIR filter of length M needs the last M samples; a 256-point FFT needs the last 256; an envelope detector needs maybe a few dozen. The data structure that holds "the last N samples" efficiently is the **ring buffer** (circular buffer): a fixed array with a write index that wraps around to the start when it hits the end. Geometrically it is a loop of tape – you keep writing forward, and the oldest sample is overwritten by the newest, so the buffer always holds a window sliding along the signal.

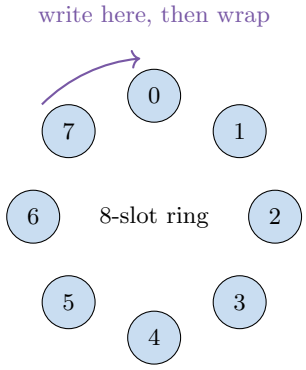


Figure 40.2: A ring buffer of 8 slots. The write index advances around the loop; when it passes slot 7 it returns to slot 0, overwriting the oldest sample. The array index is the sample number taken modulo the buffer length.

The key arithmetic is that the physical array index is the logical sample number *modulo* the buffer length N. If N is a power of two, that modulo is a single bitwise AND – one cycle – which is exactly why DSP buffer sizes are almost always powers of two.

Listing 40.3: A power-of-two ring buffer with a single-cycle wrap.

```
#define N 256                  // must be a power of two
int16_t buf[N];
uint16_t head = 0;             // next write position

static inline void ring_push(int16_t x) {
  buf[head] = x;
  head = (head + 1) & (N - 1);   // wrap via AND, no modulo divide
}

// read the sample k steps back (k=0 is newest)
static inline int16_t ring_get(uint16_t k) {
  return buf[(head - 1 - k) & (N - 1)];
}
```

What is really happening

A ring buffer turns "the last N samples" into a fixed footprint that never grows. No allocation, no shifting of array elements, no copying – pushing a sample is one store and one masked increment. The illusion of a smoothly sliding window is created by moving the *index* instead of moving the *data*. This is the single most important data structure in real-time DSP, and it is why an MCU with 4 KB of RAM can process a signal that has been running for hours.

Common trap

A ring buffer shared between an interrupt (writer) and the main loop (reader) is a classic concurrency hazard. The simplest safe pattern: the ISR only *pushes*, the main loop only *reads*, indices are a single aligned word (atomic to read/write on the MCU), and shared flags are `volatile`. For block processing, use *two* buffers (double buffering / ping- pong): the ISR fills buffer A while the main loop processes buffer B, then they swap. Never let the reader and writer touch the same buffer at the same time.

40.4 Numbers on a small chip: fixed-point vs floating-point

On your laptop every number is a 64-bit float and you never think about it. On a small MCU you must, because the way you represent numbers directly sets your speed, your memory, and your correctness. There are two camps: floating-point and fixed-point.

40.4.1 What floating-point costs

A floating-point number stores a sign, an exponent, and a mantissa – effectively scientific notation in binary, $\pm m \times 2^e$. Its glory is *dynamic range*: it represents both 0.0001 and 10000 with similar relative precision, so you rarely overflow and rarely lose all your bits. The catch on embedded hardware is the **FPU** (floating-point unit). A chip with a hardware FPU (Cortex-M4F, M7, the Raspberry Pi) does a float multiply in a cycle or few. A chip *without* one (the classic 8-bit AVR Arduino Uno, Cortex-M0/M0+) must emulate every float operation in software – dozens to hundreds of cycles each. On an Uno, a single `float` multiply can cost what an integer multiply costs many times over.

40.4.2 What fixed-point is

Fixed-point keeps the speed of integer arithmetic but reintroduces fractions by *agreeing on where the binary point sits*. In the common **Q-format** notation, $Q_{m.n}$ means an integer interpreted as having n fractional bits: the stored integer X represents the real value $X/2^n$. For example, in $Q_{1.15}$ (often just "Q15") a 16-bit signed integer represents a real number in $[-1, 1)$ with 2^{15} steps; the value 0.5 is stored as 16384.

Intuition

Floating-point lets the binary point *float* to wherever the number needs it – great range, hardware cost if there is no FPU. Fixed-point nails the binary point to one agreed place –

you trade away automatic range for the raw speed of plain integers. Choosing between them is choosing *who tracks the scale*: the silicon (float) or you, the programmer (fixed).

Arithmetic in Q-format follows simple rules once you track the point:

- **Add/subtract:** operands must share the same n; the result keeps that n. Watch for overflow – two Q15 numbers near 1 sum to near 2, which no longer fits.
- **Multiply:** $Q_a \times Q_b$ gives a product with $a+b$ fractional bits. Two Q15 values multiplied give a 32-bit Q30 result; to return to Q15 you shift right by 15. You must multiply into a *wider* accumulator ($16\times16 \to 32$) or you lose the top bits.

Listing 40.4: A Q15 fixed-point multiply with a 32-bit accumulator.

```
typedef int16_t q15_t;        // value v stored as round(v * 32768)

static inline q15_t q15_mul(q15_t a, q15_t b) {
  int32_t p = (int32_t)a * (int32_t)b;   // Q30 in 32 bits
  return (q15_t)(p >> 15);               // back to Q15 (truncating)
}
// example: 0.5 * 0.5 -> 16384 * 16384 = 268435456; >>15 = 8192 = 0.25
```

Common trap

Two fixed-point sins dominate embedded bug reports. **Overflow**: a Q15 accumulator quietly wraps from $+0.99$ to -1.0 when a filter's partial sums exceed the range; the fix is a wider accumulator (Q31) and *saturating* arithmetic that clamps instead of wrapping. **Loss of precision**: shifting right too early throws away low bits; keep full width through the whole inner sum and round/shift only at the very end. CMSIS-DSP's functions are written precisely to manage these for you, which is a strong reason to use them rather than rolling your own.

Connection

Quantization and fixed-point are two faces of the same coin from the ADC chapter. Quantization error is the gap between the true value and the nearest representable rung; in $Q_{m.n}$ that rung spacing is exactly 2^{-n}. The "6 dB per bit" rule is the same here: each extra fractional bit halves the step and buys about 6 dB of headroom against round-off noise. The noise you analyzed statistically in the noise chapter is the same round-off noise your fixed-point filter generates internally – which is why understanding random signals pays off the moment you go embedded.

40.4.3 A practical rule of thumb

If your chip has an FPU, write in `float` (or even `double` on a Pi) first. Floats are easier to get correct, and on an M4F/M7 they are fast. Reach for fixed-point only when you have no FPU (AVR, M0), when you need every last cycle or microamp, or when a vectorized integer path (CMSIS-DSP Q15) is meaningfully faster than the float path on your part. Premature fixed-point is a top source of subtle, hard-to-find DSP bugs.

40.5 Running real DSP: filters and FFTs on an MCU vs a Pi

Now the heart of it. We have samples on a clean grid, stored in a ring buffer, in a numeric format the chip likes. What does it cost to actually *do* the DSP?

40.5.1 FIR and IIR filters on the MCU

A direct-form FIR filter of length M is $y[n] = \sum_{k=0}^{M-1} h[k]\,x[n-k]$ – M multiply- accumulates (MACs) per output sample. This maps beautifully onto a ring buffer: the taps multiply the last M ring entries. A Cortex-M4 has a single-cycle MAC, so an $M = 64$ FIR costs roughly 64 cycles per sample; at $f_s = 1$ kHz that is 64,000 cycles/s, a rounding error against an 80 MHz clock. The same filter on an 8-bit AVR in floating point could cost a hundred times more per tap and might not keep up.

An IIR biquad is even cheaper – five MACs per sample per second-order section – and is the workhorse for the gentle low-pass and band-pass filters you will run on IMU and EMG signals. The price of that efficiency is sensitivity to coefficient quantization and the need to watch internal overflow, which is exactly why biquads are usually run as cascaded second-order sections in fixed-point.

Listing 40.5: A single biquad (Direct Form II transposed), float version.

```
typedef struct { float b0,b1,b2,a1,a2,z1,z2; } biquad_t;

static inline float biquad(biquad_t* s, float x) {
  float y = s->b0 * x + s->z1;
  s->z1   = s->b1 * x - s->a1 * y + s->z2;
  s->z2   = s->b2 * x - s->a2 * y;
  return y;                          // 5 mul, 4 add, 2 state updates
}
```

40.5.2 The FFT: CMSIS-DSP on the MCU

For an FFT on an ARM Cortex-M, do not write your own – use **CMSIS-DSP**, ARM's free, hand-optimized DSP library. It provides radix-based FFTs in float (`arm_rfft_fast_f32`) and fixed-point (`arm_rfft_q15`), plus FIR/IIR, statistics, and vector math, all tuned to use the chip's MAC and (on M4/M7) SIMD instructions.

Listing 40.6: A 256-point real FFT and magnitude with CMSIS-DSP.

```
#include "arm_math.h"
#define NFFT 256
arm_rfft_fast_instance_f32 fft;
float in[NFFT], spec[NFFT], mag[NFFT/2];

void fft_setup(void) { arm_rfft_fast_init_f32(&fft, NFFT); }

void fft_block(void) {
  arm_rfft_fast_f32(&fft, in, spec, 0);     // forward real FFT
  arm_cmplx_mag_f32(spec, mag, NFFT/2);     // |X[k]| for each bin
}
```

How long does that take? Order-of-magnitude: an $N = 256$ float FFT on an 80 MHz Cortex-M4F is on the order of tens of microseconds. If you want a new spectrum every 10 ms (a 100 Hz frame rate, plenty for gesture work), the FFT eats well under 1% of the budget. This is the headline result of the chapter: *modern MCUs are genuinely fast enough to do real frequency-domain DSP in real time*, as long as you respect the budget arithmetic.

Walking question

A 256-point FFT in ~ 30 μs. A frame every 10 ms (10,000 μs). What fraction of the processor does the FFT use, and how many such FFTs could you run per frame before you run out of time?

Answer: 30/10000 = 0.3% per FFT. The math alone would allow on the order of a few hundred such FFTs per frame – but in practice you would hit memory and I/O limits, and windowing/copying overhead, long before the FFT arithmetic itself becomes the bottleneck.

40.5.3 The Pi side: NumPy and SciPy

A Raspberry Pi is a different animal – a full Linux computer with a multi-core ARM CPU, hundreds of megabytes to gigabytes of RAM, and an FPU on every core. Here you write Python with NumPy and SciPy exactly as you did while learning. `np.fft.rfft`, `scipy.signal.sosfilt`, `scipy.signal.spectrogram` – all available, all fast (NumPy's FFT and filters are C/Fortran under the hood). The cost you trade for that comfort is real-time *determinism*: Linux is not a hard real-time OS. Most of the time your block runs in a millisecond; occasionally the scheduler, a page fault, or another process makes it late. For a feel-it-instantly instrument that occasional hiccup matters.

Listing 40.7: Streaming block processing on a Raspberry Pi.

```
import numpy as np
from scipy.signal import butter, sosfilt

fs = 1000
sos = butter(4, 20, 'low', fs=fs, output='sos')  # 20 Hz low-pass
zi  = np.zeros((sos.shape[0], 2))                 # filter state

def process_block(block):           # block: 256 new samples
    global zi
    y, zi = sosfilt(sos, block, zi=zi)            # stateful -> streaming
    spec  = np.abs(np.fft.rfft(y * np.hanning(len(y))))
    return y, spec
```

Why this matters

The same five-line biquad is, on the MCU, a hand-managed struct of floats you must budget cycles for, and on the Pi, a `butter()` call you barely think about. Neither is "better." The MCU gives you determinism and microamps at the cost of effort; the Pi gives you effortless horsepower at the cost of determinism and milliwatts. Mature embedded design is knowing which currency you are spending and choosing on purpose.

40.6 Getting answers out: serial, BLE, and MIDI

Processing is half the job; the result has to leave the device. Three channels matter for this project.

Serial (UART/USB). The workhorse for development and wired use. Simple, reliable, and on USB fast enough for raw sample streams. You will use it constantly for debugging – but printing inside your DSP loop is itself a timing hazard (see the trap below).

BLE (Bluetooth Low Energy). The right wireless link for a wearable: low power, ubiquitous on phones and laptops, and – crucially – it supports **BLE-MIDI**, a standard profile that carries MIDI messages over Bluetooth. A board like a Nordic nRF52 or an ESP32 can advertise as a BLE-MIDI device that a DAW or synth app sees as a normal MIDI controller, with no dongle.

MIDI. The destination format for our instrument. MIDI is delightfully thin: most messages are three bytes – a status byte (command + channel) and two data bytes. A Note-On is `0x90 | channel`, then note number (0–127), then velocity (0–127). A continuous controller (CC), perfect for mapping a gesture to a parameter like filter cutoff or vibrato, is `0xB0 | channel`, controller number, value.

Listing 40.8: Emitting MIDI: a note and a continuous-control message.

```
void midi_note_on(uint8_t ch, uint8_t note, uint8_t vel) {
  uint8_t m[3] = { (uint8_t)(0x90 | (ch & 0x0F)), note, vel };
  midi_send(m, 3);            // over USB-MIDI, serial-MIDI, or BLE-MIDI
}
void midi_cc(uint8_t ch, uint8_t cc, uint8_t val) {
  uint8_t m[3] = { (uint8_t)(0xB0 | (ch & 0x0F)), cc, val };
  midi_send(m, 3);
}
```

Common trap

`Serial.print()` for debugging inside a sensor- or audio-rate loop can block for milliseconds while the UART drains its buffer, blowing your real-time deadline and corrupting your sample timing. For diagnostics, either print from a *lower-priority* context (the slow main loop, not the ISR), throttle to a few messages per second, toggle a GPIO pin and watch it on a logic analyzer, or write to a ring buffer and flush it when idle. Logging is I/O, and I/O is the enemy of determinism.

Toward the wrist tracker

The wrist controller's output path is the whole point: finger gestures in, music out. The natural design is BLE-MIDI – the wearable advertises as a standard MIDI controller, and any phone, tablet, or laptop running a synth receives it with zero custom software. Map *discrete* finger movements (a per-finger tap or press, matched to its calibrated template) to Note-On/Note-Off, and *continuous* signals (finger pressure from FMG, wrist tilt) to CC messages so the player can bend, swell, and modulate. Keep total latency – sample to MIDI out – as low as you can; beyond about 20–30 ms a player begins to feel the lag, which is why the capstone leans on a fast onset detector (Section 40.9).

40.7 Power and real-time budgets

A wearable lives on a battery, so two budgets run in parallel: cycles (can I finish in time?) and joules (how long until it dies?). They pull in opposite directions, and good design balances them.

40.7.1 The real-time budget, made concrete

The rule is unforgiving: *work per deadline must be less than time per deadline*. Suppose you sample an IMU at $f_s = 200$ Hz and run the heavier feature/analysis block on 32 samples, so a block arrives every $32/200 = 160$ ms. (This block sets the analysis cadence, not the play latency – the fast onset path of Section 40.9 fires far sooner.) On an 80 MHz MCU that is $80{,}000{,}000 \times 0.160 = 12.8$ million cycles of budget per block – enormous. Your filtering and a small FFT might use a few tens of thousands. You are nowhere near the cliff, which is typical for motion-rate DSP and is exactly why an MCU suffices. The budget only gets tight at audio rates (f_s in the tens of kHz) or with big FFTs.

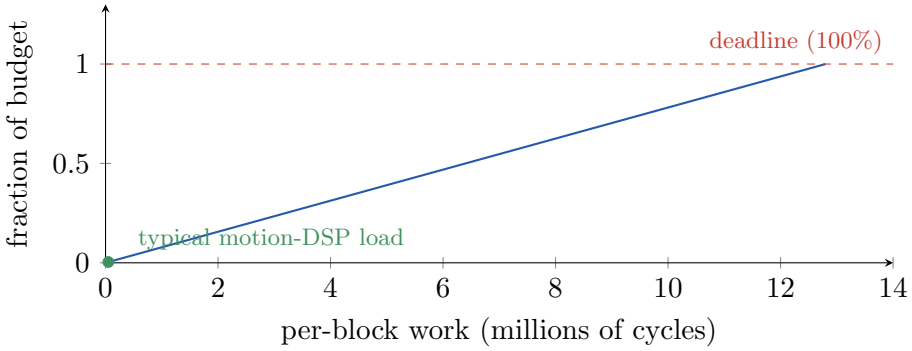


Figure 40.3: Real-time budget for a 200 Hz IMU processed in 32-sample blocks on an 80 MHz MCU: 12.8M cycles available per block. As work rises toward the dashed deadline you approach the cliff; motion-rate DSP (green dot) sits comfortably near the origin.

40.7.2 The power budget

Battery life is set by average current. The dominant lever on an MCU is *sleep*: a Cortex-M draws tens of milliamps running flat out but only microamps in a deep-sleep mode. Because motion DSP uses a tiny fraction of the cycle budget, the chip should compute its block, then *sleep* until the next timer or IMU interrupt. This "compute-and-sleep" duty cycling can cut average current by one to two orders of magnitude.

Listing 40.9: Compute-then-sleep duty cycling (pseudocode).

```
for (;;) {
  if (block_ready) {          // set by the IMU FIFO interrupt
    block_ready = false;
    process_block();          // a few tens of microseconds of work
    maybe_emit_midi();
  }
  enter_low_power_sleep();    // wake on next interrupt
}
```

Application

The power gap between Arduino-class and Pi-class hardware is stark. A sleeping nRF52 doing intermittent motion DSP and BLE can run for *days* on a small LiPo. A Raspberry Pi (even a Zero) idles in the hundreds of milliamps and has no microamp sleep – it is a poor wearable, often needing recharging within hours and a far bulkier battery. This single fact, more than raw compute, is why a battery-powered wrist device gravitates to an MCU and a tethered or desktop processor gravitates to a Pi.

Connection

The power story loops back to sampling theory. Lower f_s means fewer interrupts, fewer wake-ups, and less radio traffic – so the Nyquist choice is also a battery choice. Sample only as fast as the signal demands: human gesture bandwidth is a few tens of hertz, so a 100–200 Hz IMU rate is plenty, and pushing higher just burns joules for no extra information.

40.8 Arduino vs Raspberry Pi for the wrist controller

Time to decide. The two platforms are not competitors so much as different tools, and the right answer for this project is usually "use both" – a thin wrist node and a capable phone.

	Arduino / MCU (nRF52, ESP32, M4)	**Raspberry Pi** (Linux SBC)
Compute	MHz, single core, KB–MB RAM	GHz, multi-core, GB RAM
Numerics	fixed-point or FPU float; CMSIS-DSP	full `float/double`; NumPy/SciPy
Real time	hard, deterministic (ISR + timer)	soft; OS jitter, no hard deadlines
Power	mA active, μA asleep – days on a coin/LiPo	hundreds of mA – hours, bulky battery
I/O	native ADC, SPI/I2C, BLE-MIDI	USB, Wi-Fi/BT, needs ADC HAT for analog
Dev effort	C, manual budgets, flash-and-test	Python, fast iteration, easy logging
Best at	always-on sensing, wireless, low latency	calibration, conditioning, deep offline analysis

Build it (hardware)

Recommended split for the wrist controller. The architecture is deliberately lopsided: a *thin* wrist node and a *capable* phone.

- **On the wrist (MCU, e.g. nRF52840):** read the IMU FIFO and the timer-driven FMG/EMG channels on hardware clocks, apply only light conditioning – a couple of biquads and an envelope – compress, and BLE-stream the channels. For live play, run a lightweight on-device template match (a short correlation against the few calibrated per-finger templates) so the note fires with minimal latency even if the phone is busy. Compute-and-sleep between blocks for battery life. This is the always-on, low-latency, low-power core.

- **On the phone (or a Pi/laptop):** the heavy lifting lives here. The phone runs the guided calibration routine, builds and stores the per-finger templates and thresholds, conditions the streamed signals, and does deep offline analysis from the logged streams to refine the templates. There is no learned model – "analysis" here means filtering, alignment, averaging the enrollment repetitions into a clean template, and picking thresholds. The phone then ships the finished templates back down to the wrist node for the live path.

The general principle: keep the wrist node cheap in cycles and joules, and let the phone – which has power to spare and a screen to guide the user – own calibration and conditioning. The wrist sleeps; the phone thinks.

Toward the wrist tracker

What the wrist can and cannot sense. Be precise here, because the boundary is the whole design. *Which finger just moved* is observable from the forearm: tendon bulge (FMG) and muscle pulses (sEMG) carry a distinct per-finger signature, and after a short per-user calibration a template match reads it reliably. WristFlex demonstrated FMG-based finger-tap discrimination from a band of force sensors; Thalmic Labs' Myo and the open MyoWare front end read the same intent from sEMG. So per-finger *detection* – this finger tapped, that finger pressed, this one is held down – is firmly on the doable side, and it is plenty for playing an instrument.

The genuinely hard part, and the one limit worth stating plainly, is continuous per-*joint* finger *angles*. You cannot recover them from a wrist or forearm band: the mapping from finger pose to wrist-surface signal is underdetermined, and the bulk motion of the wrist simply does not contain that information. Recovering smooth joint angles needs sensors on the fingers themselves – instrumented rings, as in the finger-worn "Learning on the Rings" tracking work, or a glove. Treat that as an upgrade path, not a launch requirement. Everything the project actually needs – detect which finger moved, how hard, and when – sits comfortably on the achievable side.

History

The split we just drew is an old one in disguise. In the 1980s the Texas Instruments TMS320 fixed- point DSP chips earned their living doing exactly this – a small, deterministic processor crunching filters and FFTs in real time on a tight power budget – while general-purpose CPUs did the flexible, non-real-time work. ARM's CMSIS-DSP library and the Cortex-M's single-cycle MAC are the direct descendants of that fixed-point DSP lineage, now small and cheap enough to wear on your wrist. The Raspberry Pi, meanwhile, is the heir to the workstation. Embedded DSP has always been the art of putting the right work on the right side of that line.

40.9 The capstone: a buildable finger-gesture controller

Now assemble everything into one concrete, end-to-end project (Figure 40.4). The instrument detects which finger moved, how hard, and when, and emits MIDI – no camera, no machine learning, just multimodal sensing plus per-finger calibration. This section is meant to be *built*.

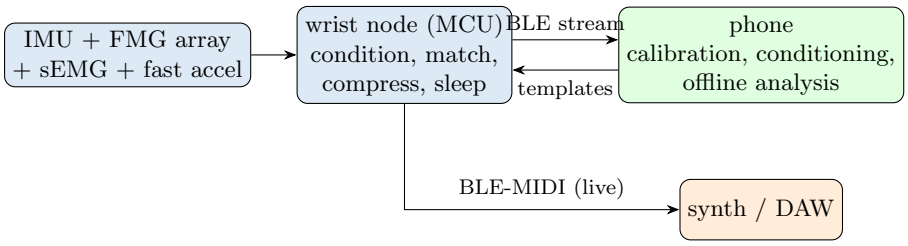


Figure 40.4: System architecture. A thin wrist node conditions and template-matches the fused sensor channels and emits BLE-MIDI for the low-latency live path. It BLE-streams raw channels to the phone, which runs calibration and deep offline analysis and ships finished per-finger templates back down.

40.9.1 Bill of materials and wiring

- **MCU:** Nordic nRF52840 (Cortex-M4F, FPU, BLE 5, deep-sleep in microamps). The wrist node.
- **IMU:** ICM-20948 (or LSM6DSx) over SPI, programmed to a 200 Hz output data rate with FIFO and a data-ready interrupt line.
- **High-rate accelerometer:** a second, fast accelerometer (a few kHz) used as a *contact / bio-acoustic* sensor – it hears the tendon snap and skin tap that a slow IMU misses, the trick ViBand (UIST '16) used to turn a watch into a bio-acoustic input device.
- **FMG array:** four to eight force-sensitive resistors (FSRs) in a band pressed against the underside of the forearm, each into an ADC channel through a voltage divider.
- **sEMG:** one or two MyoWare (or equivalent) sEMG front-end boards, each a single analog channel into the ADC.
- **Power:** a small LiPo (e.g. 150 mAh) and a charge/regulator board.

Wiring is straightforward: IMU on SPI with its interrupt to a GPIO; the high-rate accelerometer on a second SPI or I^2C bus with its own interrupt; the FSR dividers and sEMG outputs to ADC pins, all sampled from one hardware timer so they share a time grid.

40.9.2 The pipeline, channel by channel

1. **Acquire (hardware clocks).** The IMU FIFO interrupt and the ADC timer drive acquisition; the high-rate accelerometer interrupts separately. No `delay()`, no polling. Each channel lands on its own steady grid.
2. **Condition.** Run each slow channel (IMU, FMG) through a biquad low-pass near 20 Hz to kill motion noise; band-pass the sEMG ($\sim$ 20–450 Hz) and take its envelope (rectify then low-pass), since the useful sEMG signal is the *amplitude* of the muscle burst, not its waveform. Five MACs per biquad per sample.
3. **Feature.** Per short window, form a small feature vector: FMG channel levels, sEMG envelope levels, a high-rate-accelerometer onset/energy spike, and a couple of IMU stats. This fixed-length vector is the live "signature."

4. **Match.** Compare the live signature against the calibrated per-finger templates (next subsection) – nearest template by correlation or distance, gated by an onset threshold so silence stays silent.

5. **Emit.** Map the matched finger and its pressure to MIDI – Note-On for a tap, a held note while pressed, CC for continuous pressure or wrist tilt – and send over BLE-MIDI.

6. **Sleep.** Enter low-power mode until the next interrupt.

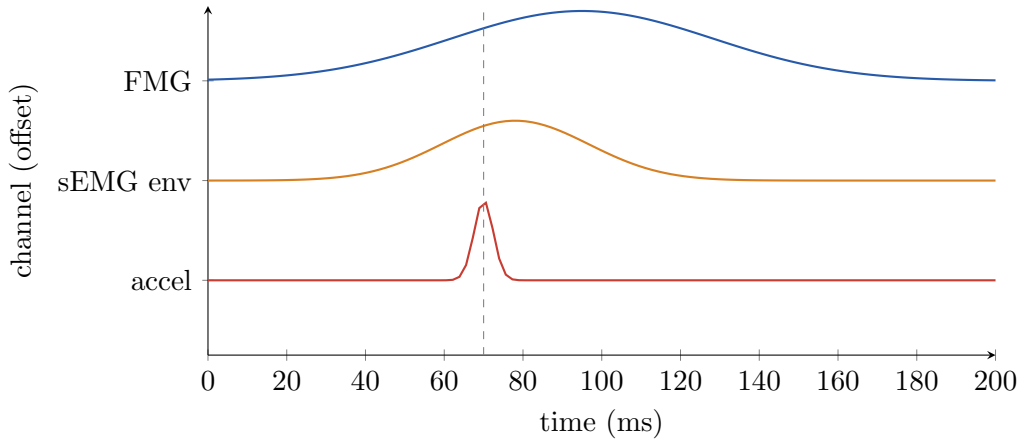


Figure 40.5: One finger tap across three channels, time-aligned. The high-rate accelerometer (bottom) spikes first at the contact onset; the sEMG envelope (middle) marks the muscle burst; the FMG channel (top) rises and falls more slowly as tendon force builds. The fast accelerometer onset is what lets the live path fire early, while the slower channels confirm which finger and how hard.

40.9.3 The per-finger calibration routine – the heart of it

Everything rests on enrollment. Multimodal fusion (high-rate IMU + FMG array + the bio-acoustic accelerometer + sEMG) gives a richer signature than any single-modality lab paper used – Figure 40.5 shows one tap across the channels – and per-finger calibration is what turns that signature into a decision. Combining motion with acoustic contact echoes the IMU-plus-acoustic gesture work (PLOS One, 2020), and wrist-IMU gesture counting (MDPI Sensors, 2023) shows how much one well-conditioned motion channel already carries; the edge here is fusing those channels and calibrating per finger rather than leaning on any one of them. The routine runs on the phone:

1. The phone prompts: "tap your index finger." The user does it, say, five times. The wrist streams the conditioned channels; the phone slices out each onset.

2. It *aligns* the repetitions on their onset and *averages* them into one clean template $\mathbf{t}_{\text{index}}$ – a short multi-channel snippet. Averaging suppresses the per-trial noise and leaves the shared signature.

3. It records the spread across repetitions to set a per-finger *threshold*: how close a live signature must be to count as a match, and how far from "idle" before anything fires.

4. Repeat for each finger (and for any chord or combined movement you want as its own gesture).

5. The phone ships the finished templates and thresholds back to the wrist node.

There is no learned model anywhere – "analysis" is filtering, onset alignment, averaging, and threshold-picking. Calibration is what makes a band that fits *this* arm read *this* user's fingers, and it is why the method generalizes across people without retraining anything.

40.9.4 Live template matching

On the wrist, live detection is a correlation. With templates $\mathbf{t}_f$ for fingers $f = 1 \dots F$ and a live window signature $\mathbf{s}$, pick the finger whose template is closest, provided the onset clears the idle threshold:

$$\hat{f} = \arg\max_f \frac{\langle \mathbf{s}, \mathbf{t}_f \rangle}{\|\mathbf{s}\| \, \|\mathbf{t}_f\|}, \qquad \text{fire only if onset energy} > \theta_{\text{idle}}.$$

The normalized inner product (cosine similarity) is robust to overall pressure scaling, and the onset gate keeps resting fingers quiet. For $F \approx 5$–10 templates of a few dozen samples each, this is a handful of dot products – microseconds on an M4F, trivially inside budget.

Listing 40.10: Live loop: condition, match against calibrated templates, emit MIDI.

```
void on_block_ready(void) {          // short window of aligned channels
  signature_t s;
  build_signature(&s);               // FMG + sEMG env + accel onset + IMU
  if (onset_energy(&s) > theta_idle) {
    int f = best_template(&s, templ, F);   // argmax cosine similarity
    emit_midi_for_finger(f, pressure(&s)); // Note-On / CC over BLE-MIDI
  }
  block_ready = false;
}

void main_loop(void) {
  for (;;) {
    if (block_ready) on_block_ready();
    enter_low_power_sleep();         // wake on next sensor interrupt
  }
}
```

40.9.5 Latency budget

Latency is sample-to-MIDI, and the player feels everything past roughly 20–30 ms. Budget it:

Stage	Typical
Sense / fill a short window (8 samples at 200 Hz)	~ 40 ms
Condition + build signature	< 0.1 ms
Template match (a few dot products)	< 0.05 ms
BLE-MIDI transmit	~ 5–10 ms
Total	~ 45–50 **ms**

The window-fill term dominates, so shorten it: detect on a fast *onset* from the high-rate accelerometer and sEMG envelope rather than waiting for a full slow-IMU block. Onset detection can fire within a sample or two of contact, pulling sense-to-decision down to a few milliseconds; the slower channels

then confirm and set pressure. With a short onset window the total lands near 10–15 ms – inside the playable range. Choosing the block length is choosing your latency, and the contact accelerometer is what buys the fast path.

40.9.6 MIDI mapping

- **Note:** each finger maps to a Note-On/Note-Off. A tap is a quick on/off; a hold sustains the note while pressure stays above threshold.
- **Velocity:** the onset energy (how hard the tap) maps to MIDI velocity (0–127).
- **Chords:** a calibrated multi-finger gesture is its own template, fired as several Note-Ons together.
- **CC:** continuous FMG pressure or wrist tilt maps to a continuous controller (e.g. CC 74 for filter cutoff, CC 1 for vibrato), so a held note can swell and bend.

40.9.7 Failure modes

- **Band shifts on the arm.** Templates assume the same sensor placement as enrollment; a big slip detunes them. Mitigation: a quick re-enrollment, and a snug, repeatable band fit.
- **Sweat / electrode drift.** sEMG baseline wanders; lean on FMG and the accelerometer, and high-pass the sEMG envelope's baseline.
- **Cross-talk between adjacent fingers.** Neighboring tendons co-activate; the multi-channel template and the onset gate separate them better than any single channel.
- **False fires at rest.** Set θ_{idle} from the enrollment spread, not by guessing.

40.9.8 The upgrade path

This build detects which finger moved, how hard, and when – enough to play. The one thing it *cannot* do is recover continuous per-joint finger angles from the wrist, because that information is not in wrist-surface signals (Section 40.8). If you later want smooth, continuous per-finger control – a finger acting as a fader, not a key – add finger-worn sensors (instrumented rings, in the spirit of the "Learning on the Rings" work). That is a clean addition, not a redesign: the rings stream another sample-aligned channel into the same fusion-and-match pipeline.

Every theme of the chapter is present in this build: hardware-clocked acquisition (no jitter), ring buffers holding the windows, conditioning in a chosen numeric format, CMSIS-DSP filters and FFTs well inside budget, a thin BLE-MIDI output, and compute-and-sleep to protect the battery. The same Fourier and filtering mathematics from the early chapters, now living inside a chip on your arm.

Pause

If this chapter felt like a gear change, that is because it is one. We left the frictionless notebook and met deadlines, milliamps, and bits that overflow. The consolation is that none of the *mathematics* changed – an FFT is an FFT. What changed is that you now also account

for *when* the work happens and *what it costs*. Carry these three questions into every embedded design: Is my clock from hardware? Does my work fit the budget? Will the battery survive? Answer those, and the math takes care of itself.

40.10 Exercises

Exercise 40.1. A timer-driven sampler uses an AVR with a 16 MHz clock, Timer1 in CTC mode, and a prescaler of 64. What value of `OCR1A` gives exactly $f_s = 2000$ Hz? (Recall $f_s = f_{\text{clk}}/(\text{prescaler} \times (\text{OCR1A} + 1))$.)

Exercise 40.2. Explain in one paragraph why `delay()`-based sampling can distort a spectrum even when the average sample rate is correct. Give a concrete example of a code change that would shift the rate mid-stream.

Exercise 40.3. A clock has RMS jitter of $\delta = 1$ μs. Using the linearization error $\approx 2\pi f A \delta$, compute the worst-case amplitude error (as a fraction of A) when sampling a 50 Hz tone and a 5 kHz tone. What does the ratio tell you about high-frequency sampling?

Exercise 40.4. You need a ring buffer that holds the last 200 samples. Why would you allocate 256 slots instead of exactly 200, and what single machine instruction does that choice enable for the index wrap?

Exercise 40.5. In $Q_{1.15}$ format, what integer represents 0.75? Multiply the Q15 representations of 0.75 and 0.5 using the wide-accumulator rule and confirm the result decodes to 0.375. Show each step.

Exercise 40.6. A Q15 FIR filter has 64 taps, each up to magnitude 1, and inputs up to magnitude 1. Argue why a 16-bit accumulator is unsafe and roughly how many guard bits a safe accumulator needs to hold the worst-case sum without overflow.

Exercise 40.7. An 80 MHz Cortex-M4F runs a 256-point float FFT in about 30 μs. If you want a new spectrum every 5 ms, what fraction of the CPU does the FFT use? Roughly how many such FFTs per frame could the math alone support, and name two practical limits you would hit first.

Exercise 40.8. You sample an IMU at $f_s = 200$ Hz, process in 32-sample blocks, on an 80 MHz MCU. (a) How many cycles of budget do you have per block? (b) If your processing costs 40,000 cycles per block, what fraction of the budget is used? (c) What happens to the budget if you raise f_s to 2000 Hz with the same block size?

Exercise 40.9. Explain why a Raspberry Pi is a poor choice for an always-on, battery-powered wrist device but an excellent choice for prototyping the gesture-detection algorithm. Frame your answer in terms of the two budgets (cycles and joules) from this chapter.

Exercise 40.10. Write the three MIDI bytes (in hex) for: (a) a Note-On of middle C (note 60) at velocity 100 on channel 1 (status nibble 0); (b) a continuous-controller message setting CC 74 to value 64 on channel 1.

Exercise 40.11. Sensing limits. A teammate proposes using only the wrist IMU to drive five independent synth voices, one per finger, by *continuous* finger flexion – each finger a smooth fader. Explain precisely why continuous per-joint angles cannot be recovered from a wrist band alone,

and propose what sensor(s) you would add to make continuous per-finger control possible. Separately, explain why discrete per-finger *detection* (which finger tapped) *is* achievable with the forearm FMG/sEMG channels plus calibration.

Exercise 40.12. Calibration. Describe the per-finger enrollment routine for the capstone: what the phone prompts, what it records, and how it turns several repetitions of one finger movement into a single template plus a threshold. Why does averaging aligned repetitions improve the template, and why is the threshold derived from the spread across repetitions rather than guessed?

Exercise 40.13. Template match. Live detection picks $\hat{f} = \arg\max_f \langle \mathbf{s}, \mathbf{t}_f \rangle / (\|\mathbf{s}\| \|\mathbf{t}_f\|)$ above an onset threshold. Why use the *normalized* inner product rather than the raw dot product, and what does the onset gate θ_{idle} prevent? For $F = 6$ templates of 24 samples each, estimate the multiply-accumulate count per match and confirm it is negligible against a 12.8M-cycle block budget.

Exercise 40.14. Design. Sketch (in prose or pseudocode) the compute-and-sleep loop for the wrist unit, and estimate the duty cycle if each block takes 50 μs of compute and a block arrives every 160 ms. What does that duty cycle imply for average current relative to running the CPU continuously?

Chapter 41

Capstone: A Wrist-Worn Gesture Controller for Instruments

Every chapter of this book has carried the same quiet promise: that the geometry of frequency — signals as vectors, the Fourier transform as a set of projections, convolution as a sliding overlap, filtering as turning knobs in the spectrum — was not abstract decoration but a toolkit for building something real. This is the chapter where we cash that promise. We are going to build a *wrist/forearm-worn finger-gesture controller*: a band you wear on your arm that detects which finger you just moved, reads how your wrist is oriented and moving, and turns all of that into MIDI — the language synthesizers and digital audio workstations speak. Play it like an instrument: lift the index finger to trigger a note, twist your wrist to bend the pitch, curl the ring finger for a chord, spread your fingers to open a filter.

Finger movement *is* detectable from the wrist, and this chapter shows how without a shred of machine learning. The trick is not a clever model; it is *calibration*. Each finger movement, sensed through the forearm tendons or muscle electrical activity, leaves a repeatable signature on a band of sensors. A short guided enrollment — the user wiggles each finger a few times — records those signatures as per-user *templates*. Live play is then a direct match: compare the current sensor reading to the templates and fire the nearest one. Calibration is the heart of the instrument; everything else is conditioning the signals so the match is clean.

We open by separating what the wrist genuinely reveals from the one thing it genuinely hides, then walk the entire pipeline end to end — sensor choice, wiring, the thin-node-plus-phone architecture, sampling, de-noising, the per-finger calibration routine, windowing, feature extraction, template matching, latency budgeting, and MIDI mapping — using the very techniques the earlier chapters built. We close with a real bill of materials, a frank accuracy-and-difficulty box, the failure modes that will actually bite you, and an upgrade path toward true continuous finger tracking.

41.1 What the wrist reveals, and the one thing it hides

Let us draw the line precisely, because the difference between a project that works and one that disappoints is almost always set in the first five minutes by an over-promise — or, just as bad, by an under-promise that throws away something real.

Here is the achievable part, stated plainly: **which finger moved is recoverable from a wrist or forearm band**, with per-user calibration. Your finger flexors and extensors sit in the forearm, right under the band. When you move a finger, the responsible muscle contracts and *bulges*, pressing on the skin (*force myography*, FMG), and it emits an electrical burst (*surface electromyography*, sEMG). Different fingers recruit different muscles, so each finger movement paints a different *spatial pattern* across a ring of sensors. Those patterns are repeatable enough, once calibrated to your arm, that a direct match identifies the finger. WristFlex did exactly this for finger pinches; the build here extends it with more sensing channels and a richer calibration.

Common trap

The one genuinely hard thing: continuous per-joint finger *angles* cannot be recovered from the wrist. Detecting *that* the index finger moved is easy; reconstructing the exact bend angle of each knuckle, continuously, is not. A wrist IMU measures the acceleration and angular velocity *of the wrist*, and finger motion produces almost no inertial signature there. Even the muscle sensors give a low-dimensional pressure/voltage pattern, not the dozen-odd joint angles of the hand — the mapping is underdetermined. So we read *discrete finger movements* (reliable) and *wrist orientation* (reliable), and we leave continuous joint angles to finger-worn sensors (Section 41.14). Everything a player actually needs to perform — trigger notes by finger, shape them by wrist — lives on the achievable side of that line.

So the instrument has two cooperating channels:

1. **Per-finger movement detection** — "index," "middle," "ring," "fist," "spread," "rest" — from the FMG or sEMG band, matched against per-user templates built during enrollment. This answers *which finger / which gesture*.

2. **Continuous expressive control from the wrist** — pitch bend from wrist tilt, modulation from roll, intensity from how hard and fast you moved — from the IMU, where it genuinely shines. This answers *how*.

Combine the two and you have a real instrument: the muscle/tendon band says *which note and articulation*, the IMU says *how high, how much vibrato, how loud*. That division of labor is the whole design.

Toward the wrist tracker

The wrist tracker we have been pointing at since Chapter 1 is a *finger-gesture-plus-expression* controller: a calibrated set of finger movements (from FMG and sEMG) selecting and gating notes, continuously modulated by wrist orientation and motion (from the IMU). There is **no machine-learning model** anywhere in it. The method is direct — a guided enrollment records each finger's signature, and live detection matches the current signal to those templates. Everything in this book — sampling, anti-alias filtering, windowing, the FFT, spectral and time-domain features, sensor fusion, correlation and distance in feature space, latency budgeting — is exactly what turns the raw sensor streams into expressive, low-latency MIDI. This chapter is the assembly.

Why this matters

More than a Fitbit, and not by adding a brain. A Fitbit-class wearable reads heart rate, steps, and sleep — bulk-body signals. It cannot tell which finger you moved, because it neither sits over the finger muscles nor samples fast enough to catch a contact event. Our band does three things a fitness tracker does not: it places a sensor array *over the forearm flexors* (FMG and sEMG, which carry per-finger information), it runs a high-rate accelerometer *as a contact/bio-acoustic sensor* (catching taps and snaps the slow sensors miss), and it *fuses* these with the IMU. The edge over the cited single-modality lab papers is exactly this *multimodal fusion* plus *per-finger calibration* — not a smarter algorithm and not a learned model. Several weak views of the hand, each calibrated and then combined, beat one clever view every time.

History

The wrist-as-input idea has a real research lineage, and reading it sets realistic expectations. *WristFlex* (Dementyev & Paradiso, UIST 2014) used a band of force-sensitive resistors around the wrist to tell apart a handful of finger pinches at around 80 % accuracy with very low power — the direct ancestor of the FMG build here. Thalmic Labs' *Myo* armband (and the open *MyoWare* sensor that survived it) brought 8-channel forearm sEMG to makers. *ViBand* (Laput, Xiao & Harrison, UIST 2016) repurposed a smartwatch accelerometer at 4 kHz to pick up bio-acoustic vibrations and recognize taps and grasps — the inspiration for our high-rate contact channel. A 2020 *PLOS ONE* study fused a wrist IMU with a contact microphone for hand-gesture recognition; a 2023 *MDPI Sensors* line of work uses a wrist IMU to count and segment gestures. Each of those is a single-modality or single-pair result; our edge is fusing several. And "Learning on the Rings" (2023) showed that to get *continuous* finger pose you put inertial sensors on the fingers, not the wrist — which is exactly our upgrade path in Section 41.14.

41.2 The pipeline, in one picture

Before details, hold the whole shape in your head. Several sensor streams flow in from the left, get conditioned and turned into features, the muscle/contact branch is *matched* against calibrated per-finger templates, the IMU branch is fused into a smooth orientation, and the two recombine on the right into MIDI events.

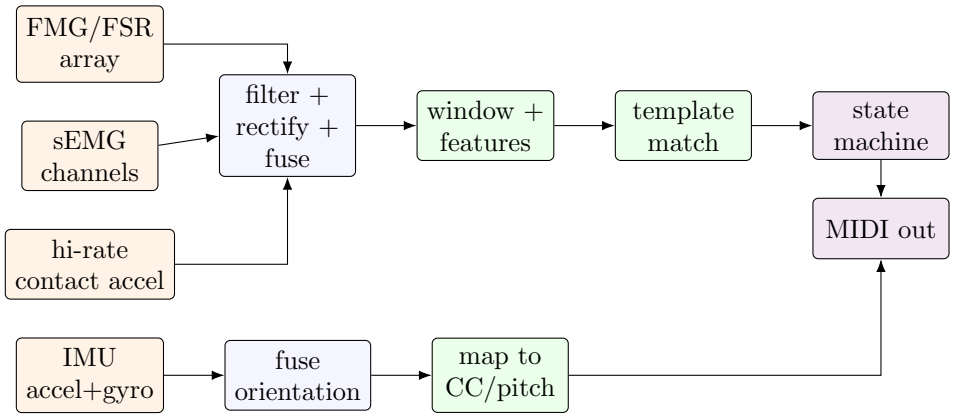


Figure 41.1: The end-to-end signal chain. Three hand-muscle/contact streams (top) are conditioned and fused, then matched against calibrated templates to answer *which finger*; the IMU (bottom) answers *how* (orientation and motion). They recombine into MIDI: finger events become note and articulation events, orientation becomes continuous controllers and pitch bend.

Every box in Figure 41.1 is a technique you already own. The rest of the chapter walks left to right and fills in the numbers.

41.2.1 Where the work runs: thin wrist node, phone does the heavy lifting

A wrist device must be small and sip power, so it cannot also be a workstation. Split the labor: the band is a *thin node* and the phone is the workhorse.

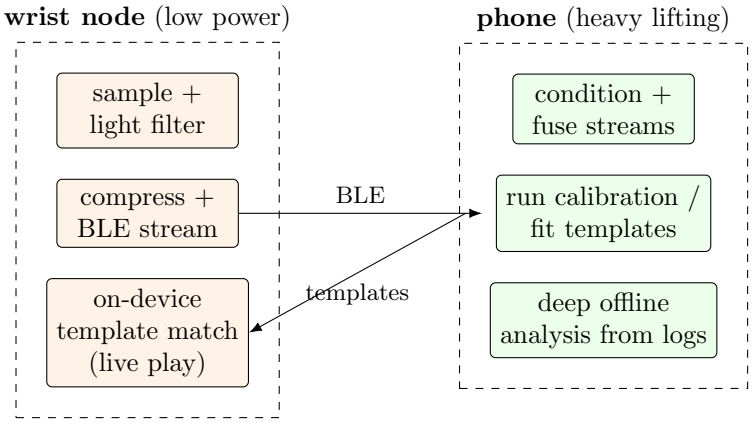


Figure 41.2: Division of labor. The wrist node samples, lightly filters, compresses, and streams over BLE; it keeps only a lightweight template-matcher for the low-latency live-play path. The phone runs the calibration routine, fits the per-finger templates, conditions and fuses the streams, and does deep offline analysis on the streamed logs. The fitted templates are pushed back to the node so live play does not depend on the phone round-trip.

This split is not an afterthought; it shapes every later choice. Calibration and deep analysis are heavy and infrequent, so they live on the phone. Live matching must beat a tight latency deadline (Section 41.10), so a stripped template-matcher runs on the node itself, using templates the phone

computed once and pushed down.

41.3 Choosing the hand sensors

How you sense the fingers sets your cost, your difficulty, and your accuracy ceiling. The recommended build uses three complementary streams — and that multimodal stack, not any single sensor, is what carries the per-finger information cleanly.

Force myography (FMG) — the backbone. When a forearm muscle contracts it *bulges*, and that bulge presses outward on the skin. Strap a ring of force-sensitive resistors (FSRs) snugly around the forearm and each FSR reads the local pressure. Different finger movements recruit different muscles, so they produce different *spatial patterns* of pressure around the band. FMG is cheap (a few dollars of FSRs), needs no skin prep, no gel, no high-gain analog front end, and its signal is a slow, well-behaved pressure voltage you can sample at a leisurely rate. This is the WristFlex approach, and it is the backbone of the array.

Surface EMG (sEMG) — faster, finer. Electrodes on the skin pick up the tiny ($\sim 10\,\mu$V to 1 mV) electrical potentials of firing motor units. sEMG reacts faster than FMG (electrical activity precedes the mechanical bulge by tens of milliseconds) and separates neighbouring fingers more finely, but it demands a careful analog front end (instrumentation amp, high gain, a 20–450 Hz bandpass, a 50/60 Hz notch for mains hum), and it is sensitive to electrode placement, skin moisture, and sweat. A MyoWare 2.0 sensor or a Myo-style 8-channel band makes it tractable. Add one to three channels over the flexor belly to sharpen the per-finger distinction the FMG array gets close to.

High-rate contact accelerometer — the bio-acoustic channel. A small accelerometer sampled fast (a few kHz) and pressed to the wrist hears the faint mechanical vibrations of finger taps, snaps, and surface contacts — the ViBand trick. It catches exactly the crisp *onset* events the slow FMG envelope smears out, which makes it the natural velocity and trigger sensor for percussive playing. Used *alongside* the slow sensors rather than alone, its motion-artifact sensitivity stops mattering: the FMG/sEMG pattern says which finger, the contact accelerometer says exactly *when* and how hard.

IMU — the expressive axis. The same accel-plus-gyro IMU used for contact, read at 100–200 Hz, supplies wrist orientation and gross arm motion for the expressive channel (Section 41.9).

Intuition

Think of FMG, sEMG, and the contact accelerometer as three different *projections* of the same hidden state (what your fingers are doing) onto measurable axes. FMG projects onto *mechanical bulge*; sEMG onto *electrical drive*; the contact channel onto *vibration onsets*. No single one recovers the full finger-pose vector — that information thins out on the way to the wrist — but each captures a different low-dimensional shadow, and the shadows are *complementary*: where FMG blurs two adjacent fingers, sEMG or the onset timing often pulls them apart. Stack the three and a calibrated per-finger signature becomes cleanly distinguishable. This is the same projection-and-separation thinking from the inner-product chapter, now across

several sensing modalities at once.

Build it (hardware)

FMG band wiring (the recommended build). Use eight FSRs (e.g. Interlink 402, round 0.5 in) sewn or taped evenly around an elastic forearm strap. Each FSR is one leg of a voltage divider:

```
//  3.3V ---[ FSR ]---+---[ R_fixed (10k) ]--- GND
//                    |
//                  ADC pin   (V rises as force rises)
```

Wire all eight dividers to eight ADC channels of a microcontroller (a Teensy 4.x, an ESP32, or an RP2040 all have plenty of inputs). Choose $R_{\text{fixed}} \approx 10\,\text{k}\Omega$ so the divider sits mid-range at typical contraction. No amplifier, no gel, no notch filter required. Total added cost over the microcontroller is roughly the price of a sandwich.

41.4 Sampling: how fast, and why

Chapter 18 gave us the Nyquist rule: sample at more than twice the highest frequency present, and band-limit *before* the ADC so nothing above $f_s/2$ folds back. Apply it to each stream and the answers differ by two orders of magnitude.

- **FMG** is mechanical bulge — its meaningful content lives below roughly 10–15 Hz. Sampling each channel at $f_s = 200\,\text{Hz}$ is luxurious and leaves enormous headroom. The simple RC of the FSR divider plus a software low-pass is ample anti-aliasing.
- **sEMG** has real spectral energy out to $\sim 450\,\text{Hz}$, so by Nyquist you need $f_s \geq 1\,\text{kHz}$ (1–2 kHz in practice), with a *hardware* bandpass (20–450 Hz) ahead of the ADC — here anti-aliasing is not optional, because muscle and mains energy above $f_s/2$ will alias straight onto your features.
- **Contact accelerometer** (the bio-acoustic channel) carries useful vibration content into the low kHz; sample it at $\sim 4\,\text{kHz}$, the ViBand rate, with its own anti-alias filter. The wrist node compresses this stream hard before streaming it, since you mostly need its onset timing and energy, not the full waveform.
- **IMU** for orientation needs only modest rates; 100–200 Hz captures all the wrist motion a human can produce and keeps the fusion filter (Chapter 36) well inside its stable region.

Common trap

Do not sample sEMG at, say, 500 Hz with no analog filter and assume a digital filter will save you. The 50/60 Hz mains hum and its harmonics, plus muscle energy above 250 Hz, will *alias* below 250 Hz *before* your code ever sees them — aliasing happens in the ADC, and no later filter can undo it (Chapter 18). The anti-alias filter must be in the analog domain, ahead of the converter.

41.5 Conditioning and de-noising

Raw sensor voltage is never the feature. Each stream needs the conditioning that earlier chapters justified. Figure 41.3 shows the move that matters most for the muscle sensor: turning a noisy bipolar activity burst into a clean, slowly-varying *envelope* the template matcher can read.

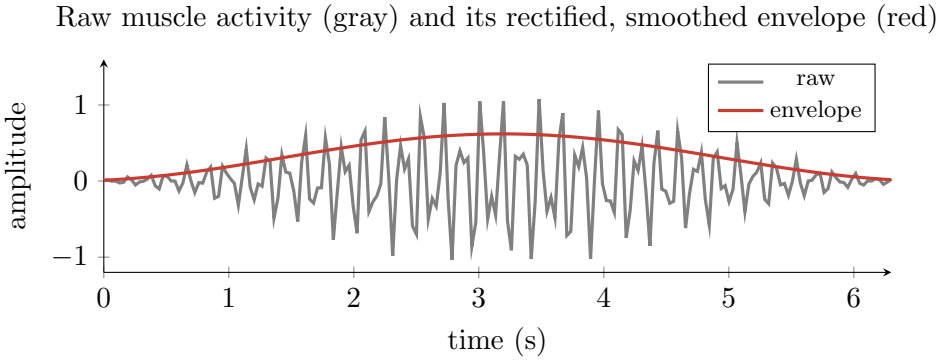


Figure 41.3: The key de-noising move for FMG/sEMG. The raw signal (gray) is a fast oscillation gated on and off as the muscle contracts. We do not care about the oscillation, only *that it is present and how strong*. Full-wave rectifying ($|x|$) and low-pass smoothing extracts the envelope (red) — a slow signal whose height is a clean proxy for contraction strength.

The recipe, stream by stream:

1. **sEMG:** bandpass 20–450 Hz (remove DC drift and out-of-band junk), notch at 50 or 60 Hz (mains), then *rectify* ($|x|$ or x^2) and *low-pass* at a few hertz to get the linear envelope. The rectify-then-smooth pair is exactly the magnitude-then-average idea from the analytic-signal chapter; it converts "how much muscle drive" into a slow controllable number.

2. **FMG:** the FSR voltage already *is* a slow envelope, so just low-pass it (a one-pole smoother or short moving average, Chapter 17) to kill ADC jitter and the odd spike. No rectification needed — pressure is one-sided.

3. **IMU:** a light low-pass on the accelerometer (remove vibration hash) and a gyro-bias estimate at rest; then hand both to the fusion filter. Do *not* aggressively low-pass the gyro — you would add lag to your orientation, which the player feels.

Why it works

Why rectify and smooth instead of, say, taking the spectrum? Because the *information you want is in the amplitude, not the waveform.* A contracting muscle produces a roughly stochastic burst whose exact zero-crossings are meaningless; what carries the gesture is the burst's *strength and which channels light up.* Rectify-and-smooth is the cheapest unbiased estimator of that strength — it is the matched move for an amplitude-modulated random carrier, and it runs in a handful of operations per sample, which matters when you are on a microcontroller meeting a deadline.

41.6 Calibration: the heart of the instrument

This is the section that makes the whole thing work without machine learning. No two arms, and no two strap-ons of the *same* arm, give the same raw numbers. Calibration (Chapter 35) makes the device work for a real person on a real morning rather than only on the bench where you tuned it — and, done as a *per-finger enrollment*, it is also what lets a direct match identify which finger moved.

41.6.1 Step one: per-channel baseline and scale

Before any finger means anything, put the channels on a common footing. At the start of a session, ask the user to rest the hand for one second; record each channel's mean, the *rest baseline* b_i. Then ask for one maximum voluntary contraction (a hard fist) for a second and record each channel's peak p_i. Normalize every later sample to

$$\tilde{x}_i[n] \;=\; \frac{x_i[n] - b_i}{p_i - b_i},$$

so each channel lands in roughly $[0, 1]$ regardless of skin, strap tension, or electrode contact. This single affine fix removes most cross-session variation.

41.6.2 Step two: per-finger enrollment and templates

Now the part that replaces a learned model. The phone runs a short, guided *enrollment*: it names a finger movement, the user performs it a few times, and the phone records the resulting normalized sensor pattern. There is no training of weights and no optimizer — just recording and averaging.

1. For each movement g in the vocabulary (rest, index, middle, ring, little, fist, spread, ...), the phone prompts "move your index finger," and the user repeats it, say, five times.

2. For each repetition the phone computes the per-frame feature vector (Section 41.7) at the moment of peak activity — the normalized envelope across all FMG, sEMG, and contact channels.

3. It *averages* those repetitions into one prototype vector, the **template** $\mathbf{t}_g$, and records the spread (a per-channel standard deviation) so it knows how tight each finger's signature is.

The result is a tiny table: one template vector per finger movement, plus a spread. That table *is* the user's personalized model. It takes under a minute to build, lives in a few hundred bytes, and is pushed down to the wrist node so live matching needs no phone.

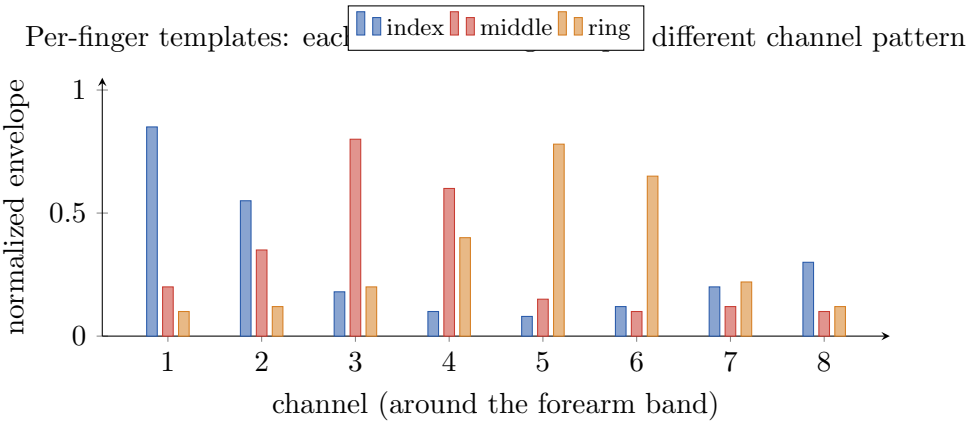


Figure 41.4: Three enrolled templates over an 8-channel FMG band. Each finger movement bulges a different group of forearm muscles, so it raises a different *spatial* pattern of normalized envelopes. Live detection compares the current 8-vector to these prototypes and picks the nearest. The patterns overlap in places — ring and middle share channel 4 here — which is why the extra sEMG and contact channels, and a tight calibration, matter.

41.6.3 Step three: IMU calibration

Gyros have a slowly-drifting bias: average a couple of seconds at rest and subtract it (re-estimate whenever the device is still). Accelerometers need the standard offset-and-scale-per-axis fit from Chapter 35 so that "1 g down" reads as exactly $1\,g$; otherwise your gravity-derived tilt, and thus your pitch-to-note mapping, is skewed.

Connection

The normalization in step one is the affine *whitening* you met in the linear algebra companion: subtract a mean, divide by a scale, to put every coordinate on comparable footing before measuring distances. Template matching compares feature vectors by distance, which implicitly assumes the axes are commensurate — calibration is what makes that assumption true. Skip it and a single dominant channel swamps the distance metric. The per-finger templates of step two then place one labelled landmark per movement in that whitened space, and detection is simply "which landmark is nearest."

41.7 Windowing and feature extraction

The matcher does not eat raw samples; it eats *feature vectors* computed over a short window. This is the short-time idea from Chapter 10: slice the stream into overlapping frames and summarize each frame with a few numbers. The same feature vector is computed during enrollment (to build templates) and during play (to match them), so the two stay comparable by construction.

Frame geometry. Use a window of $W = 128$–$256\,\text{ms}$ with 50–75 % overlap, recomputed every *hop* of $\sim 32\,\text{ms}$. A shorter window means lower latency but noisier features; a longer window is steadier

but laggier. For a played instrument, $W \approx 150\,\text{ms}$ with a 30 ms hop is a good corner of that trade-off — it updates the finger decision about 30 times a second, which feels instantaneous.

Features that work for muscle sensors. The literature (and a lot of hard-won practice) converges on cheap *time-domain* features per channel, because they are robust and microcontroller-friendly:

- **Mean absolute value (MAV):** $\frac{1}{W}\sum_n |x_i[n]|$ — the envelope height; the single most useful feature.
- **Root mean square (RMS):** $\sqrt{\frac{1}{W}\sum_n x_i^2[n]}$ — energy; closely related to perceived contraction strength.
- **Waveform length:** $\sum_n |x_i[n] - x_i[n-1]|$ — captures signal complexity/activity cheaply.
- **Zero crossings** and **slope-sign changes:** crude frequency proxies for sEMG; usually skip for slow FMG.

A handful of spectral features can help sEMG — the *mean* and *median frequency* of the power spectrum shift as a muscle fatigues, and a short FFT (Chapter 9) gives them for nearly free — but for a first build the time-domain features above carry most of the discriminating power.

Focus

The *spatial* pattern across channels is where the finger lives. A single channel's MAV tells you "some muscle fired"; the *vector* of MAVs across all channels tells you *which* muscles and thus *which* finger moved. For an 8-channel band the per-frame feature vector might be just the 8 MAVs plus 8 waveform-lengths — a 16-dimensional point. Detection is then: which enrolled template is this point closest to?

```python
import numpy as np

def frame_features(frame):
    """frame: shape (W, C), one window of C channels. Returns 1D feature vector."""
    x   = frame.astype(np.float32)
    mav = np.mean(np.abs(x), axis=0)                 # (C,) envelope height
    rms = np.sqrt(np.mean(x*x, axis=0))              # (C,) energy
    wl  = np.sum(np.abs(np.diff(x, axis=0)), axis=0) # (C,) waveform length
    return np.concatenate([mav, rms, wl])            # (3C,)
```

41.7.1 Why a small feature set beats raw samples

What is really happening

A 150 ms window at 1 kHz across 8 channels is 1200 raw numbers per frame. The 16 (or 24) features above compress that by nearly a hundredfold while *keeping* what separates the fingers and *throwing away* the stochastic detail that only muddies a match. This is dimensionality reduction in the service of separability: you are projecting a noisy high-dimensional frame onto the few directions that actually move between fingers. Matching then works in a space

small enough to describe with one template per finger and fast enough to evaluate every hop on a microcontroller.

41.8 Template matching: detection without a learned model

We have, per frame, a low-dimensional feature vector $\mathbf{x}$, and from enrollment (Section 41.6.2) one template vector $\mathbf{t}_g$ per finger movement. Detection is now embarrassingly direct: which template is the current frame closest to?

$$\hat{g}[n] \;=\; \arg\min_{g} \|\mathbf{x}[n] - \mathbf{t}_g\|^2.$$

That is the whole decision rule. No weights are fit, no optimizer runs, no network is trained — the only "model" is the table of per-user templates the phone recorded in under a minute. This is nearest-template matching, and it is the geometry of Chapter 39's inner product put to work: the squared distance $\|\mathbf{x} - \mathbf{t}_g\|^2 = \|\mathbf{x}\|^2 - 2\langle \mathbf{x}, \mathbf{t}_g \rangle + \|\mathbf{t}_g\|^2$ is small exactly when the frame *correlates* strongly with the finger's signature.

Distance or correlation. For amplitude-stable, calibrated channels, Euclidean distance to the template is the natural rule. If overall contraction *strength* varies but the *pattern* is what identifies the finger, use the normalized correlation (cosine similarity) instead,

$$s_g[n] \;=\; \frac{\langle \mathbf{x}[n], \mathbf{t}_g \rangle}{\|\mathbf{x}[n]\|\,\|\mathbf{t}_g\|},$$

and pick the largest. Cosine matching shrugs off a global "pressed harder this time" scaling, which the distance rule would penalize.

Reject when nothing matches. The **rest** template handles the common "no finger moved" case as a first-class entry — without it the matcher would always snap to the nearest *active* finger and hallucinate notes. Add a guard too: if even the best distance exceeds a threshold (the live frame is far from every template), declare rest rather than force a match. The per-finger spread recorded during enrollment sets that threshold per movement.

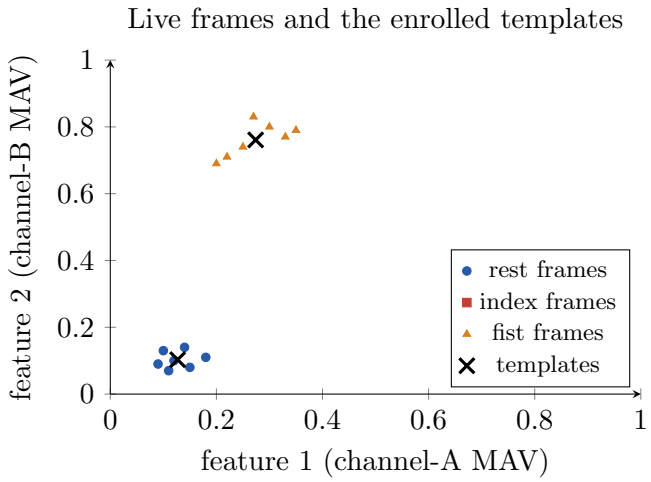


Figure 41.5: Why nearest-template matching suffices. Projected onto two channel MAVs, each finger movement's live frames form a tight cloud, and the enrolled template ($\times$) is just their average. A new frame is labelled by the nearest $\times$. Calibration put each channel in $[0, 1]$, which makes the clouds compact and the distances meaningful; the extra sEMG and contact channels separate clouds that would overlap in these two FMG dimensions alone.

```python
import numpy as np
# templates: (G, D) one row per finger movement (incl. "rest"); built at enrollment
# reject_dist: per-template distance threshold from the enrollment spread

def detect(x, templates, labels, reject_dist):
    d = np.sum((templates - x)**2, axis=1)   # squared distance to each template
    g = int(np.argmin(d))
    if d[g] > reject_dist[g]:                # too far from everything -> rest
        return "rest"
    return labels[g]
```

A handful of multiplies and adds per template, a few templates: this runs in microseconds on the wrist node, which is exactly why the live path does not need the phone.

Common trap

Check generalization on a fresh strap-on, not the enrollment run. It is tempting to declare victory because the enrolled frames match their own templates — of course they do, the templates are their average. The real test is re-donning the band and matching against the existing templates without re-enrolling. The day you re-don the band is the day you discover how much the signatures depend on exact placement. If accuracy sags, that is the cue to add a quick 3-second recalibration ritual on each don (Section 41.13), not to reach for a bigger model.

41.8.1 Smoothing the decision: from frame labels to finger events

A per-frame match flickers — one stray frame mid-movement matches "fist" for 30 ms. Raw, that becomes a spurious MIDI note. Tame it with a small *state machine* and *debouncing*, exactly the

hysteresis idea from the practical-filtering chapter:

- Require N consecutive frames (e.g. 3) agreeing before declaring a new finger event (majority vote over a short sliding window).
- Require a return to **rest** before the same movement can fire again — this gives you crisp note *onsets* instead of a stuttering retrigger.
- Confidence-gate: if the best match is barely closer than the runner-up (small margin), hold the previous state rather than switch.

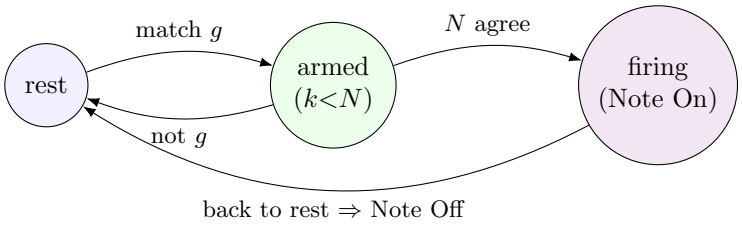


Figure 41.6: The debouncing state machine. A candidate match must survive N frames in the *armed* state before it fires a Note On; the same movement cannot fire again until the hand passes back through *rest*, which is also when the Note Off is sent. This turns a flickering per-frame label stream into crisp, non-repeating note events.

41.9 Sensor fusion: the IMU as the expressive axis

While the muscle branch decides *what*, the IMU branch decides *how*. From Chapter 36 we fuse accelerometer (good long-term tilt reference from gravity, noisy short-term) and gyroscope (clean short-term rotation rate, drifts long-term) into a stable orientation — a complementary filter is enough:

$$\theta[n] \;=\; \alpha\left(\theta[n-1] + \omega_{\text{gyro}}[n]\,T_s\right) \;+\; (1-\alpha)\,\theta_{\text{accel}}[n],$$

with α near 0.98. The gyro term gives responsiveness; the small accel term slowly pulls out drift. Out of this you get wrist *pitch* and *roll* (yaw needs a magnetometer and is the least reliable — avoid making it musically load-bearing).

Toward the wrist tracker

Map the fused orientation to expression:

- **Wrist pitch** → pitch bend or note selection (tilt up to go sharp).
- **Wrist roll** → a continuous controller (CC 1, modulation / vibrato depth).
- **Motion energy** (gyro magnitude) → note velocity or CC 11 (expression): a fast, snappy gesture hits harder than a lazy one.

These are *continuous* and feel alive precisely because the IMU genuinely measures them well — the opposite of trying (and failing) to read finger angles from the same sensor.

41.10 Latency budgeting

A musical controller lives or dies on latency. The well-known threshold for "feels instantaneous" on a played instrument is about 10 ms; up to ∼20–30 ms is usable; beyond that you feel the rubber band. Every stage in Figure 41.1 spends some of that budget. Add them up without flinching.

Stage	Typical latency
ADC + acquisition of one hop	∼ hop size (e.g. 30 ms)
Filtering group delay (envelope LPF, a few Hz)	5–15 ms
Feature window (centered cost ≈ $W/2$)	part of window
Template match + state-machine debounce (N frames)	$N\times$ hop
USB-MIDI transport + synth/DAW buffer	3–10 ms
Total (realistic)	**∼ 30–60 ms**

Table 41.1: A representative latency budget. The biggest spenders are the feature window, the debounce count, and the envelope low-pass. Every one is a knob you can trade against robustness.

Focus

The latency triangle from Chapter 17 reappears here in disguise: *quality* (clean envelope, a confident match) costs *latency* (longer window, slower LPF, more debounce frames). For a forgiving sustained-pad sound you can spend latency freely; for a percussive, snappy lead you must shrink the window, loosen the debounce, and accept more flicker. Tune the budget *to the sound*, not to a universal number.

Common trap

The sneakiest latency is the one you forget: the synth/DAW's own audio buffer. A 512-sample buffer at 48 kHz is already ∼11 ms before your MIDI even becomes sound. Measure end-to-end (tap the band, record the audio, count samples to the attack) rather than trusting the sum of your estimates.

41.11 MIDI mapping

The final box turns gesture state and orientation into MIDI bytes. The mapping *is* the instrument design — the same hardware feels utterly different under different maps.

Notes from gestures. On a confirmed gesture onset, send `Note On` (status `0x90`, note number, velocity); on the return to rest, send `Note Off` (`0x80`). Derive the note number from the gesture identity (index pinch → scale degree 1, middle pinch → degree 3, . . .) or from wrist pitch quantized to a scale. Derive velocity from motion energy at onset.

Chords. Map a gesture to a *set* of simultaneous Note Ons — "fist" could strike a triad. Honor the gesture vocabulary's discreteness here rather than fighting it; chords are a natural fit for a small, reliable gesture set.

Continuous control. Stream wrist roll as CC 1 (mod wheel) and pitch as Pitch Bend (14-bit, status `0xE0`) *between* note events. Smooth and rate-limit these (send at most every few ms) so you do not flood the bus.

```
import mido
port = mido.open_output()                          # a virtual MIDI port into your DAW

def on_gesture(gesture, velocity, channel=0):
    note = GESTURE_TO_NOTE[gesture]                # e.g. {"index_pinch": 60, ...}
    port.send(mido.Message('note_on', note=note, velocity=velocity, channel=channel))

def on_release(gesture, channel=0):
    note = GESTURE_TO_NOTE[gesture]
    port.send(mido.Message('note_off', note=note, channel=channel))

def on_orientation(roll, pitch_bend, channel=0):
    cc = int(max(0, min(127, roll * 127)))         # roll in [0,1] -> CC1
    port.send(mido.Message('control_change', control=1, value=cc, channel=channel))
    pb = int(max(-8192, min(8191, pitch_bend)))
    port.send(mido.Message('pitchwheel', pitch=pb, channel=channel))
```

Application

The same controller, three instruments, just by swapping the map:

- **Lead synth:** gestures pick scale degrees, wrist pitch bends, roll adds vibrato — an expressive monophonic solo voice.
- **Chord pad:** each gesture is a chord; motion energy sets velocity; roll opens a filter via CC — a one-hand harmony rig.
- **Drum trigger:** each gesture is a drum (kick, snare, hat); velocity from snap strength — finger-drumming in the air.

Nothing about the signal chain changed; only `GESTURE_TO_NOTE` and the CC routing did. That is the leverage of clean separation between sensing and mapping.

41.12 Bill of materials

Item	Notes	Rough cost
Microcontroller (Teensy 4.x / RP2040 / ESP32)	USB-MIDI capable, ≥ 8 ADC ch	\$10–\$30
8× FSR (Interlink 402 or similar)	FMG sensing band	\$25–\$40
8× 10 kΩ resistors	divider legs	\$1
IMU breakout (MPU-6050 / ICM-20948 / BMI270)	accel + gyro (+ mag opt.)	\$5–\$15
Elastic strap + hook-loop	forearm band	\$5
Wire, perfboard/flex PCB, connectors	assembly	\$10
FMG build total		**∼ \$60–\$100**
Alt: sEMG path MyoWare 2.0 (×1–3) or used Myo	adds gel/electrodes	+\$40–\$150

Table 41.2: A realistic bill of materials. The FMG path is deliberately the cheap, forgiving entry point; sEMG is an optional accuracy/latency upgrade at real cost and difficulty.

41.13 Accuracy, difficulty, and failure modes

Common trap

Accuracy and difficulty, told straight.

- A **5–6 finger-movement** vocabulary, well calibrated, on the *same* session realistically hits ∼85–95 % per-frame accuracy — excellent for music, where the state machine cleans up stragglers. WristFlex reported around 80 % from FMG alone; the extra sEMG and contact channels here push that up, which is the point of going multimodal.
- Accuracy **drops** when you re-don the band, when you sweat, when your arm posture changes, and as the movement count grows. Cross-session, un-recalibrated accuracy can fall 10–20 points. Re-running the 30-second per-finger enrollment on each don recovers it — which is why calibration, not a model, is the thing you maintain.
- **Distinguishing adjacent fingers** (index vs. ring, say) is the hard end of the achievable range. FMG alone can blur them where their muscle bulges overlap; the sEMG channel and the contact-onset timing are exactly what pull the templates apart. Enroll those pairs with extra care and keep them spatially distinct in your mapping.
- **Continuous per-joint finger angles: not from this hardware.** This is the genuinely underdetermined part — see Section 41.14. Everything a player needs (which finger, how hard, wrist shaping) sits on the achievable side.

Failure modes you *will* meet, and the fix for each:

- **Band shifts during play** → pressure pattern changes, matches misfire. Fix: snug strap, repeatable placement marks, quick re-enroll.
- **Mains hum (sEMG)** → 50/60 Hz buzz dominates. Fix: notch filter, shorter electrode leads, battery (not USB-wall) power, move away from the laptop charger.
- **Motion artifact** → a vigorous arm move slams the envelope and triggers a phantom note.

Fix: gate the matcher with the IMU — if gross arm motion is high, raise the match-distance threshold.

- **Drift / fatigue (sEMG)** → envelope baseline creeps over minutes. Fix: slow adaptive baseline (track and subtract a long-term mean per channel), which keeps the templates valid without re-enrolling.
- **Note stutter** → match flicker. Fix: the state-machine debounce of Section 41.8.1.
- **Stuck notes** → a missed Note Off. Fix: a watchdog that sends `All Notes Off` (CC 123) on any unexpected state, and on rest.

Try it yourself

Before wiring anything, prototype the *whole* pipeline in software on a recorded log: capture a few minutes of multi-channel FMG (or a public sEMG dataset), then run conditioning → features → enroll templates → nearest-template match → a printed finger label, offline. You will shake out most of the bugs without a soldering iron, and you will have a labelled dataset ready for the real-time port.

41.14 Upgrade path

When the finger controller delights you and you want *more*, here is the ladder — each rung adds real capability at real cost, and only the last one crosses into the genuinely hard territory.

1. **More sEMG channels** (8+ around the forearm) → larger, finer movement vocabularies and crisper templates for adjacent fingers; the Myo/MyoWare territory and the MDPI Sensors fusion benchmarks.
2. **Richer contact sensing** (a dedicated high-rate accelerometer or contact mic, ViBand-style) → sharper tap, scratch, and surface-contact detection for percussive triggers, beyond what the shared IMU catches.
3. **Finger-ring inertial sensors** → the route to *continuous individual finger pose*, the one thing the wrist genuinely cannot give. Put a tiny IMU on each finger ("Learning on the Rings"); now you measure each finger's motion directly instead of inferring it from the forearm. This is the real glove-replacement, and it is fundamentally a *different sensor placement*, not a smarter wrist algorithm. It is an upgrade, not a requirement: the wrist band already plays.

Why this matters

Notice what the whole book bought you. Sampling and Nyquist set the rates; anti-alias and notch filtering cleaned the streams; rectify-and-smooth (the analytic-signal envelope) made amplitude readable; windowing and the FFT (the short-time view) framed the features; inner-product geometry made nearest-template matching intelligible; complementary fusion turned two flawed motion sensors into one good orientation; per-finger calibration replaced a learned model with a table you build in under a minute; and the latency triangle disciplined every choice. The finger controller is not a new subject — it is every chapter, wearing a strap.

41.15 Exercises

Exercise 41.1. Explain, in two sentences and without equations, why a wrist IMU cannot distinguish a relaxed open hand from a careful chord shape while the wrist is held still. Then name the one sensor change (not algorithm change) that fixes it.

Exercise 41.2. For an sEMG stream with significant energy to 450 Hz, you sample at 800 Hz with no analog filter. A 60 Hz mains component and a 370 Hz muscle component are present. Compute the alias frequency each lands on, and explain why no digital filter applied after the ADC can separate the 370 Hz content from genuine low-frequency content.

Exercise 41.3. You have 8 FMG channels at $f_s = 200$ Hz. Using a $W = 150$ ms window and a 30 ms hop, how many samples are in one window, and how many windows per second does the matcher evaluate? How many raw numbers per window, and how many features if you use MAV + waveform length per channel?

Exercise 41.4. Implement the complementary filter $\theta[n] = \alpha(\theta[n-1] + \omega T_s) + (1-\alpha)\theta_{\text{acc}}[n]$ in code. With $T_s = 5$ ms and $\alpha = 0.98$, estimate the time constant over which the accelerometer correction pulls out gyro drift. Why would $\alpha = 0.999$ feel "floaty"?

Exercise 41.5. Your templates match their own enrollment frames near-perfectly, yet the device misfires constantly in real use. Give the most likely cause (hint: a template is the average of the very frames you are scoring it on), and describe the correct way to evaluate the matcher — including the test you must run after a fresh strap-on of the band.

Exercise 41.6. Design a state machine that converts noisy per-frame labels into clean MIDI Note On/Off events. Specify the debounce count, the rest-return requirement, and the confidence gate, and explain how each prevents a specific failure (stutter, double-trigger, low-confidence flicker).

Exercise 41.7. Build the latency budget for a *percussive* drum-trigger map and a *sustained* pad map. For each, choose window length, debounce count, and envelope-LPF cutoff, and justify why the two budgets differ. Where does the DAW's audio buffer enter?

Exercise 41.8. Write the MIDI byte sequence (status, data bytes) for: Note On of middle C at velocity 100 on channel 1; a Pitch Bend to the maximum-up position on channel 1; and an All Notes Off on channel 1. Why is All Notes Off a safety net worth sending on every return to rest?

Exercise 41.9. Your movement set is {rest, index, middle, fist, spread}. Sketch (in words or a 2-D plot) where you expect each template to land in (channel-A MAV, channel-B MAV) space, and explain which pair you expect to be hardest to separate and why. Which sensing channel (sEMG or contact onset) would you add to pull them apart?

Exercise 41.10. You want true continuous finger pose, not discrete gestures. Argue from the information available at each sensor location why finger-ring IMUs can deliver this while a wrist band (FMG, sEMG, or IMU) fundamentally cannot, and cite the class of work this belongs to.

Exercise 41.11. (Build.) Wire a single FSR voltage divider into one ADC pin, sample it at 200 Hz, and apply a one-pole low-pass smoother in code. Print the smoothed value and observe it rise as you press. Then add a second channel and verify the two move independently — you have just built the smallest possible FMG band.

Exercise 41.12. Write out the per-finger enrollment routine of Section 41.6.2 as pseudocode: prompting each movement, collecting a few repetitions, computing the peak-activity feature vector, and averaging to a template plus a per-channel spread. Then write the matching rule that uses those spreads to decide when to reject to **rest**. Note that no weights are fit anywhere.

Exercise 41.13. (Open-ended.) Pick one real paper from this chapter's history box (WristFlex, ViBand, the PLOS 2020 IMU+acoustic study, an MDPI Sensors wrist-IMU gesture study, or "Learning on the Rings"). Summarize its sensor, its movement set, and its reported accuracy, and state one limitation the authors acknowledge. How would its result change your build choices, and where would adding a second modality (fusion) help most?

Part X

Mathematical Background

Chapter 42

Appendix: Complex Numbers, Linear Algebra, and Calculus for DSP

The body of this book leans on three branches of mathematics so often, and so casually, that it is worth gathering the essential pieces into one place where you can find them quickly. This appendix is that place. It is not a substitute for the companion volumes – *Calculus From the Ground Up*, *Linear Algebra From the Ground Up*, and *Probability From the Ground Up* – each of which builds these ideas slowly and from scratch. The goal here is narrower and more practical: to give you, in one sitting, exactly the geometric pictures and exactly the formulas that signal processing actually uses, with enough intuition that the symbols stop being intimidating and start being *useful*.

We will work in the same spirit as the rest of the book: geometry first, formula second. A complex number is a point in a plane and, more importantly, a little machine for rotating. A vector is an arrow; a matrix is a way of moving arrows around; an inner product measures how much two arrows agree. An integral is a limit of sums that adds up a signal continuously. A probability distribution is a way of describing the cloud of values that noise can take. None of this is hard once you see it, and all of it is the connective tissue of DSP.

42.1 Why this toolkit, and how the pieces fit

Before the details, here is the map. Three threads run through every chapter of this book, and each one corresponds to a section below.

- **Complex numbers** (Section 42.2) are how we describe *rotation at a steady rate*, which is to say frequency. The single object $e^{\mathrm{j}\omega t}$ – a point marching around a circle – replaces the awkward pair $(\cos, \sin)$ and turns trigonometry into arithmetic. Without it, the Fourier transform is a mess of sum-to-product identities; with it, the whole theory becomes one clean idea.
- **Linear algebra** (Section 42.3) is how we treat a signal as a single geometric object – a vector – and how we measure, project, and decompose it. The Fourier transform is nothing but a change of basis; the DFT is literally a matrix; filtering is a linear operator. Inner products, orthonormal bases, and eigenvectors are the load-bearing concepts.
- **Calculus** (Section 42.4) is how we add up a continuous signal (integration, which powers the continuous-time Fourier and Laplace transforms) and how we handle the idealized spike called

the delta function. We need only a little, but we need it precisely.

- **Probability** (Section 42.5) is how we describe noise: the random part of every real measurement. Just enough – mean, variance, the Gaussian, white noise – to read the noise chapters and to reason about an SNR.

Focus

Read this appendix the way you would read a phrasebook before a trip: skim it once so you know what is here, then come back to the relevant page when the main text uses something you want nailed down. Every section ends by pointing to the companion book where the idea is built properly.

42.2 Complex numbers and Euler's formula

42.2.1 A number that is also a point

A real number lives on a line. A *complex number* lives in a plane. We write

$$z = a + \mathrm{j}\, b,$$

where a and b are ordinary real numbers and j is a new symbol with the single defining property

$$\mathrm{j}^2 = -1.$$

(Engineers write j rather than i because i is already taken by electric current; this book uses j throughout.) We call $a = \mathrm{Re}(z)$ the *real part* and $b = \mathrm{Im}(z)$ the *imaginary part*. The point is to stop thinking of j as mysterious and start thinking of z as the point (a, b) in a two-dimensional plane – the *complex plane*, with the real part along the horizontal axis and the imaginary part along the vertical.

Intuition

The word "imaginary" is a historical accident and an active obstacle to understanding. There is nothing imaginary about the vertical axis. A complex number is just an ordered pair of reals with a special multiplication rule – and that rule, as we are about to see, encodes rotation. Read j as "the quarter turn" and the fog lifts.

42.2.2 Arithmetic: addition slides, multiplication rotates

Addition is componentwise, exactly like adding vectors:

$$(a + \mathrm{j}b) + (c + \mathrm{j}d) = (a + c) + \mathrm{j}(b + d).$$

Geometrically, adding z_2 to z_1 slides z_1 over by the arrow z_2. Nothing surprising.

Multiplication is where the magic is. Using $\mathrm{j}^2 = -1$ and ordinary distribution,

$$(a + \mathrm{j}b)(c + \mathrm{j}d) = (ac - bd) + \mathrm{j}(ad + bc).$$

This looks like a mess of four products, but its *geometric* meaning is stunningly simple, and to see it we change coordinates.

42.2.3 Polar form: magnitude and angle

Every point in the plane can be named two ways: by its rectangular coordinates (a, b), or by how far it is from the origin and in what direction. The distance is the *magnitude* (or modulus),

$$r = |z| = \sqrt{a^2 + b^2},$$

and the direction is the *argument* (or phase angle),

$$\theta = \arg(z) = \operatorname{atan2}(b, a),$$

measured counterclockwise from the positive real axis. (Use `atan2`, not plain `atan`, so that the quadrant is correct.) Then

$$a = r\cos\theta, \qquad b = r\sin\theta, \qquad z = r(\cos\theta + \mathrm{j}\sin\theta).$$

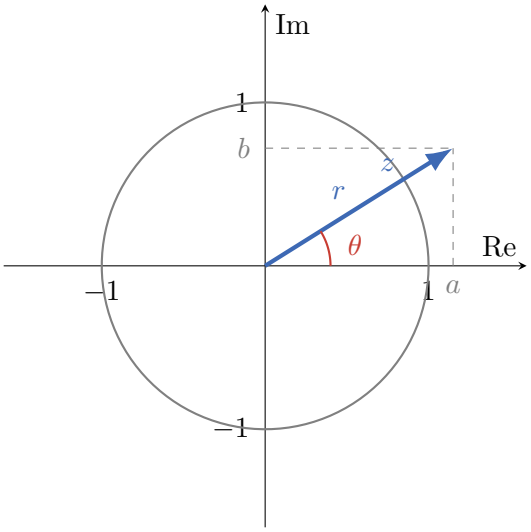


Figure 42.1: A complex number $z = a + \mathrm{j}b$ as a point in the plane. Its magnitude r is the length of the arrow from the origin; its argument θ is the angle from the real axis. The gray circle is the unit circle $|z| = 1$.

Now watch what multiplication does in polar form. If $z_1 = r_1(\cos\theta_1 + \mathrm{j}\sin\theta_1)$ and $z_2 = r_2(\cos\theta_2 + \mathrm{j}\sin\theta_2)$, then expanding the product and collecting terms using the angle-addition identities for sine and cosine gives

$$z_1 z_2 = r_1 r_2\big(\cos(\theta_1 + \theta_2) + \mathrm{j}\sin(\theta_1 + \theta_2)\big).$$

What is really happening

Multiplying two complex numbers **multiplies their magnitudes and adds their angles**. So multiplication is "scale and rotate." In particular, multiplying by a number of magnitude 1 is a *pure rotation* – it spins the plane without stretching it. And multiplying by j itself (magnitude 1, angle 90°) rotates everything a quarter turn counterclockwise. That is why $\mathrm{j}^2 = -1$: two quarter turns is a half turn, which sends 1 to -1. The defining equation was

geometry all along.

42.2.4 Euler's formula: the heart of the matter

The polar form begs for a shorthand, and there is one of breathtaking economy. Euler's formula states

$$\boxed{e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta.}$$

The left side is the exponential function evaluated at an imaginary argument; the right side is the point on the unit circle at angle θ. With it, the polar form collapses to

$$z = r\,e^{\mathrm{j}\theta},$$

and the multiplication rule becomes the ordinary law of exponents:

$$z_1 z_2 = r_1 e^{\mathrm{j}\theta_1}\, r_2 e^{\mathrm{j}\theta_2} = r_1 r_2\, e^{\mathrm{j}(\theta_1+\theta_2)}.$$

"Add the angles" is literally "add the exponents." This is why DSP is written in complex exponentials and not in cosines and sines: the algebra is trivial.

Why it works

Why should $e^{\mathrm{j}\theta}$ trace a circle? Think about velocity. Differentiate $f(\theta) = e^{\mathrm{j}\theta}$ with respect to θ: by the chain rule $f'(\theta) = \mathrm{j}\,e^{\mathrm{j}\theta} = \mathrm{j}\,f(\theta)$. Multiplying by j is a 90° rotation, so the velocity is always exactly perpendicular to the position vector, and (since $|\mathrm{j}| = 1$) always the same length. A point whose velocity is perpendicular to its radius and constant in size can only be going around a circle at constant speed. Euler's formula is the statement "rotation is exponential growth turned sideways."

Two consequences we use constantly. Setting $\theta = \pi$ gives *Euler's identity*, $e^{\mathrm{j}\pi} + 1 = 0$, linking the five most famous constants. Far more useful for us, adding and subtracting $e^{\mathrm{j}\theta}$ and $e^{-\mathrm{j}\theta}$ recovers the real trig functions:

$$\cos\theta = \frac{e^{\mathrm{j}\theta} + e^{-\mathrm{j}\theta}}{2}, \qquad \sin\theta = \frac{e^{\mathrm{j}\theta} - e^{-\mathrm{j}\theta}}{2\mathrm{j}}.$$

These *inverse Euler formulas* are how a real sinusoid splits into a pair of counter-rotating phasors, one at frequency $+\omega$ and one at $-\omega$ – the origin of the two-sided spectrum (Figure 42.2).

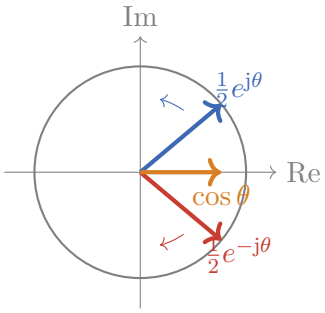


Figure 42.2: A cosine as the sum of two counter-rotating half-amplitude phasors. As θ grows, the blue phasor spins counterclockwise (frequency $+\omega$) and the red one clockwise (frequency $-\omega$); their imaginary parts always cancel, so the sum stays on the real axis and traces $\cos\theta$. This is why a real tone shows up at both $+\omega$ and $-\omega$ in a spectrum.

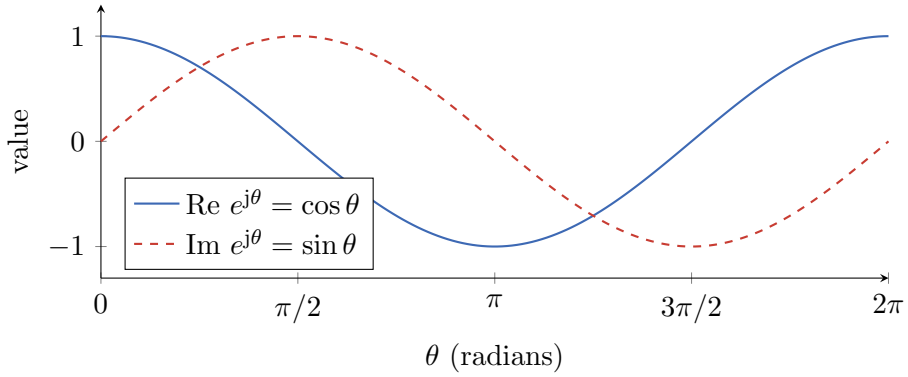


Figure 42.3: The real and imaginary parts of $e^{j\theta}$ as θ runs once around the circle. The shadow on the real axis is a cosine; the shadow on the imaginary axis is a sine. A sinusoid is the shadow of a spinning point.

42.2.5 Conjugate, magnitude squared, and division

The *complex conjugate* of $z = a + jb = re^{j\theta}$ is

$$\bar{z} = a - jb = re^{-j\theta},$$

its mirror image across the real axis – same magnitude, opposite angle. Conjugation is the single most useful operation for extracting real quantities from complex expressions, because

$$z\,\bar{z} = (a + jb)(a - jb) = a^2 + b^2 = |z|^2.$$

Multiplying a number by its conjugate kills the imaginary part and leaves the magnitude squared. We use this everywhere: a power spectrum is $|X(f)|^2 = X(f)\,\overline{X(f)}$, and the energy of a complex signal sample is $|x|^2 = x\bar{x}$. Division is then conjugate-times-reciprocal:

$$\frac{z_1}{z_2} = \frac{z_1\,\bar{z}_2}{|z_2|^2} = \frac{r_1}{r_2}\,e^{j(\theta_1 - \theta_2)}.$$

Example 42.1 (Computing with both forms)**.** Let $z = 1 + \mathrm{j}$. Rectangular gives $a = b = 1$. Polar: $r = \sqrt{1^2 + 1^2} = \sqrt{2}$ and $\theta = \operatorname{atan2}(1, 1) = \pi/4$, so $z = \sqrt{2}\, e^{\mathrm{j}\pi/4}$. Then squaring is trivial in polar form: $z^2 = (\sqrt{2})^2 e^{\mathrm{j}\pi/2} = 2\mathrm{j}$. Check in rectangular: $(1 + \mathrm{j})^2 = 1 + 2\mathrm{j} + \mathrm{j}^2 = 1 + 2\mathrm{j} - 1 = 2\mathrm{j}$. The conjugate is $\bar{z} = 1 - \mathrm{j}$, and $z\bar{z} = (1)^2 + (1)^2 = 2 = |z|^2$.

Example 42.2 (A phasor over time)**.** A signal $x(t) = Ae^{\mathrm{j}(\omega t + \phi)}$ is a phasor of fixed length A whose angle increases linearly at rate ω rad/s, starting at phase ϕ. Its real part $A\cos(\omega t + \phi)$ is the physical sinusoid you would measure. Sampling at intervals T_s replaces t by nT_s, giving $x[n] = Ae^{\mathrm{j}\phi}\,(e^{\mathrm{j}\omega T_s})^n$: a geometric sequence whose common ratio sits on the unit circle. That single fact – sampled sinusoids are powers of a unit-circle number – is the seed of the entire z-transform.

Connection

The full story of complex numbers as rotations, of $e^{\mathrm{j}\theta}$ derived from its power series, and of the complex plane as a field, lives in the early chapters of *Calculus From the Ground Up* (exponentials and the number e) and in the rotation chapter of *Linear Algebra From the Ground Up*. Section 42.2 of this appendix is the working summary; those books are where it is proven.

Toward the wrist tracker

On the wrist tracker, complex numbers show up the instant you compute a spectrum of the IMU or EMG stream. When you run an FFT on a window of accelerometer samples to find the dominant frequency of a tremor or a repetitive gesture, every output bin is a complex number $re^{\mathrm{j}\theta}$: r tells you how much energy sits at that frequency (useful), θ tells you the phase (often discarded for gesture features, but essential if you later want to align two sensors). Reading a magnitude spectrum is just taking $|z|$ of a list of complex numbers.

42.3 Vectors, matrices, inner products, bases, and eigenvectors

42.3.1 A signal is a vector

The central conceptual move of this whole book – made first in Chapter 3 of the main text – is to regard a signal as a single point in a high-dimensional space. A length-N discrete signal $x[0], x[1], \ldots, x[N-1]$ is the vector

$$\mathbf{x} = \big(x[0], x[1], \ldots, x[N-1]\big) \in \mathbb{C}^N.$$

Each *sample* is one coordinate. A two-second audio clip at 44.1 kHz is a point in a space of about 88,000 dimensions. We cannot picture that, but every operation we want – adding signals, scaling them, measuring their length, asking whether two are "perpendicular" – behaves exactly as it does for arrows in the plane. That is the whole reason to use the language of vectors: our two-dimensional intuition transfers, unchanged, to a million dimensions.

42.3.2 Inner product: how much two signals agree

For real vectors the inner product (dot product) is the sum of products of corresponding coordinates. For complex vectors – and DSP is full of them – we must conjugate one factor so that a vector's

inner product with itself comes out real and nonnegative:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{n=0}^{N-1} x[n]\,\overline{y[n]}.$$

The conjugate is not a nuisance; it is the rule that makes geometry work in $\mathbb{C}^N$. With it,

$$\|\mathbf{x}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle = \sum_{n=0}^{N-1} |x[n]|^2,$$

the total energy of the signal – a real, nonnegative number, as a squared length must be. The norm $\|\mathbf{x}\|$ is the signal's length.

Intuition

The inner product answers one question (Figure 42.4): *how much of* $\mathbf{y}$ *is present in* $\mathbf{x}$*?* If the two point the same way it is large and positive; if they are perpendicular it is zero; if opposite, negative. The Fourier transform asks exactly this question, with $\mathbf{y}$ ranging over pure sinusoids of every frequency: "how much of *this* frequency is in my signal?" Each Fourier coefficient is one inner product.

For continuous-time signals the sum becomes an integral (this is where the calculus of Section 42.4 enters):

$$\langle x, y \rangle = \int_{-\infty}^{\infty} x(t)\,\overline{y(t)}\,\mathrm{d}t.$$

Same idea, same geometry; we are just adding up infinitely many infinitesimal coordinates.

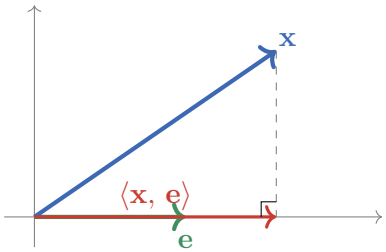


Figure 42.4: The inner product as a projection. With $\mathbf{e}$ a unit vector, $\langle \mathbf{x}, \mathbf{e} \rangle$ is the length of the shadow that $\mathbf{x}$ casts on the $\mathbf{e}$ direction – how much of $\mathbf{e}$ is present in $\mathbf{x}$. Every Fourier coefficient is one such shadow, taken onto a sinusoid.

42.3.3 Orthogonality and orthonormal bases

Two vectors are *orthogonal* when their inner product is zero – they share nothing, they are perpendicular. A set of vectors is *orthonormal* when each has length 1 and every pair is orthogonal. Orthonormal sets are the gold standard for a coordinate system because they make decomposition free of bookkeeping. If $\{\mathbf{e}_0, \mathbf{e}_1, \ldots, \mathbf{e}_{N-1}\}$ is an orthonormal basis of $\mathbb{C}^N$, then *any* vector expands as

$$\mathbf{x} = \sum_{k=0}^{N-1} c_k\,\mathbf{e}_k, \qquad c_k = \langle \mathbf{x}, \mathbf{e}_k \rangle.$$

The coefficient c_k is just the inner product of $\mathbf{x}$ with the k-th basis vector – the projection of $\mathbf{x}$ onto that direction. No matrix inversion, no system of equations: project and read off.

Why it works

Why does $c_k = \langle \mathbf{x}, \mathbf{e}_k \rangle$? Take the inner product of both sides of $\mathbf{x} = \sum_j c_j \mathbf{e}_j$ with $\mathbf{e}_k$. By linearity it becomes $\langle \mathbf{x}, \mathbf{e}_k \rangle = \sum_j c_j \langle \mathbf{e}_j, \mathbf{e}_k \rangle$. Orthonormality means $\langle \mathbf{e}_j, \mathbf{e}_k \rangle$ is 1 when $j = k$ and 0 otherwise, so every term in the sum vanishes except $j = k$, leaving c_k. The whole power of an orthonormal basis is that this collapse happens.

What is really happening

The Fourier transform *is* a change to an orthonormal basis. The DFT basis vectors are the complex exponentials $\mathbf{e}_k$ with entries $e_k[n] = \frac{1}{\sqrt{N}} e^{\mathrm{j}2\pi kn/N}$ – a phasor that completes exactly k whole turns over the N samples. They are mutually orthogonal (a fact you can verify with a geometric-series sum), and the k-th DFT coefficient is precisely $\langle \mathbf{x}, \mathbf{e}_k \rangle$: the amount of frequency k in the signal. "Analyzing a signal into frequencies" and "writing a vector in an orthonormal basis" are the same sentence in two dialects.

42.3.4 Matrices: linear maps on signals

A matrix is a rule for turning one vector into another that respects addition and scaling: $\mathbf{A}(\mathbf{x}+\mathbf{y}) = \mathbf{A}\mathbf{x} + \mathbf{A}\mathbf{y}$ and $\mathbf{A}(c\mathbf{x}) = c\,\mathbf{A}\mathbf{x}$. Such a rule is called *linear*, and a huge fraction of DSP operations are linear maps:

- The DFT itself is multiplication by the $N \times N$ matrix $\mathbf{W}$ with entries $W_{kn} = e^{-\mathrm{j}2\pi kn/N}$. The FFT is just a clever factorization of this one matrix.
- An LTI filter, applied to a finite signal, is multiplication by a special matrix (a circulant or Toeplitz matrix) built from the impulse response.
- Decimation, interpolation, the discrete cosine transform, and orientation rotations of an IMU reading are all matrix multiplications.

Reading $\mathbf{y} = \mathbf{A}\mathbf{x}$ as "each output sample is a weighted sum of input samples, with the weights given by a row of $\mathbf{A}$" is usually the most useful way to see what a matrix does.

42.3.5 Eigenvectors: the directions a system leaves alone

For a square matrix $\mathbf{A}$, a nonzero vector $\mathbf{v}$ is an *eigenvector* with *eigenvalue* λ if

$$\mathbf{A}\mathbf{v} = \lambda\,\mathbf{v}.$$

In words: the map $\mathbf{A}$ does not rotate $\mathbf{v}$ into a new direction; it only stretches it by the factor λ. Eigenvectors are the "natural axes" of a transformation – the special inputs that come out parallel to themselves.

Why this matters

Eigenvectors are the deep reason sinusoids matter so much. **Complex exponentials are the eigenvectors of every LTI system.** Feed a pure $e^{\mathrm{j}\omega n}$ into any linear time-invariant filter and you get back the *same* exponential, merely scaled by a complex number – the frequency response $H(e^{\mathrm{j}\omega})$ at that frequency. The filter cannot change the frequency, only the amplitude and phase. This is why we analyze signals frequency by frequency: in the frequency basis, every LTI system becomes diagonal – it acts independently on each component by simple multiplication. Convolution in time turns into multiplication in frequency precisely because the Fourier basis diagonalizes convolution. Eigenvectors are not an abstraction bolted on later; they are the engine.

Example 42.3 (The DFT basis is orthogonal)**.** Take $N = 4$ and the two basis phasors $e_1[n] = e^{\mathrm{j}2\pi n/4} = \mathrm{j}^n$ and $e_0[n] = 1$. Then $\langle \mathbf{e}_1, \mathbf{e}_0 \rangle = \sum_{n=0}^{3} \mathrm{j}^n \cdot \overline{1} = 1 + \mathrm{j} + \mathrm{j}^2 + \mathrm{j}^3 = 1 + \mathrm{j} - 1 - \mathrm{j} = 0$. The four powers of j sit at the four corners of the unit square and cancel in pairs. The same telescoping happens for any two distinct DFT frequencies: the partial sum of a full set of equally spaced points on the circle is zero. That cancellation is orthogonality, and orthogonality is why the inverse DFT is just another sum, not a matrix inversion.

Connection

Inner products, orthonormal bases, projection, matrices as linear maps, and eigen-decomposition are developed carefully – with proofs and pictures – in *Linear Algebra From the Ground Up*. The chapters on the spectral theorem and on orthogonal projection are the direct backbone of Fourier analysis. This appendix gives the operating manual; that book gives the theory.

Toward the wrist tracker

The wrist tracker uses linear algebra at two levels. Low level: every windowed FFT of the sensor stream is a matrix multiply (or its fast factorization), turning a block of samples into a spectrum. High level: gesture *classification* typically projects a high-dimensional feature vector (band powers, statistics across the IMU and EMG channels) down to a few discriminative directions – principal component analysis (an eigen-decomposition of the covariance matrix) or a learned linear layer – before a decision is made. Knowing that a classifier's first stage is "project onto a handful of well-chosen directions" demystifies most of the machine-learning chapter.

42.4 The calculus DSP needs

Discrete signal processing runs on sums, but the *continuous* theory that justifies it – the continuous-time Fourier transform, the Laplace transform, the sampling theorem, the analog filters we approximate – runs on integrals. You do not need the full machinery of calculus to follow this book, but you need three things solidly: what an integral really is, the two or three rules for manipulating the ones we meet, and the strange and wonderful delta function.

42.4.1 The integral as a continuous sum

An integral is what a sum becomes when the steps shrink to zero. Chop the time axis into slices of width Δt, sample the signal in each slice, multiply value by width, and add:

$$\int_a^b x(t)\,\mathrm{d}t \;=\; \lim_{\Delta t \to 0} \sum_n x(t_n)\,\Delta t.$$

Geometrically this is the signed area under the curve $x(t)$ between a and b. The symbol $\int$ is an elongated S for "sum," and $\mathrm{d}t$ is the infinitesimal width that used to be Δt. Every integral in this book is, at bottom, "add up the signal continuously."

Intuition

Whenever you see an integral in a transform, read it as a weighted average or a running total. The Fourier transform $X(f) = \int x(t)\, e^{-\mathrm{j}2\pi f t}\,\mathrm{d}t$ is the continuous version of the inner-product sum from Section 42.3: it multiplies the signal, point by point, by a probe sinusoid $e^{-\mathrm{j}2\pi f t}$ and adds up the result. A large total means the signal marches in step with that frequency; a total near zero means it does not. Same projection idea, infinitely fine.

42.4.2 The few rules we actually use

Three properties cover almost every integral manipulation in the main text.

Linearity. Integration adds and scales like any good averaging operation:

$$\int \big(a\,x(t) + b\,y(t)\big)\,\mathrm{d}t = a\int x(t)\,\mathrm{d}t + b\int y(t)\,\mathrm{d}t.$$

This is exactly why the Fourier transform is linear, which in turn is why we can analyze a signal one frequency component at a time.

The exponential integral. The single antiderivative we lean on hardest is that of the complex exponential, because $\frac{\mathrm{d}}{\mathrm{d}t}e^{\mathrm{j}\omega t} = \mathrm{j}\omega\, e^{\mathrm{j}\omega t}$ implies

$$\int e^{\mathrm{j}\omega t}\,\mathrm{d}t = \frac{1}{\mathrm{j}\omega}\, e^{\mathrm{j}\omega t} + C \quad (\omega \neq 0).$$

Evaluated over a finite window this gives the sinc that pervades sampling and windowing; over the whole line (in the limiting, distributional sense) it gives a delta in frequency. Either way, the integral of a complex exponential is the workhorse of transform calculus.

Integration by parts. For the occasional derivative-times-function integral,

$$\int u\,\mathrm{d}v = uv - \int v\,\mathrm{d}u,$$

which is just the product rule for derivatives, read backwards and integrated. It is how one proves the differentiation property of the Fourier and Laplace transforms (a derivative in time becomes multiplication by $\mathrm{j}\omega$ or s in the transform domain).

Example 42.4 (Fourier transform of a rectangular pulse)**.** Let $x(t) = 1$ for $|t| \le \frac{1}{2}$ and 0 otherwise – the unit rectangle $\mathrm{rect}(t)$. Its Fourier transform is a single application of the exponential integral:

$$X(f) = \int_{-1/2}^{1/2} e^{-\mathrm{j}2\pi ft}\,\mathrm{d}t = \frac{e^{-\mathrm{j}2\pi ft}}{-\mathrm{j}2\pi f}\Bigg|_{-1/2}^{1/2} = \frac{e^{\mathrm{j}\pi f} - e^{-\mathrm{j}\pi f}}{\mathrm{j}2\pi f} = \frac{\sin(\pi f)}{\pi f} = \mathrm{sinc}(f).$$

The last step used the inverse Euler formula for sine. A pulse in time is a sinc in frequency: the most important transform pair in sampling theory falls out of one integral and one Euler substitution.

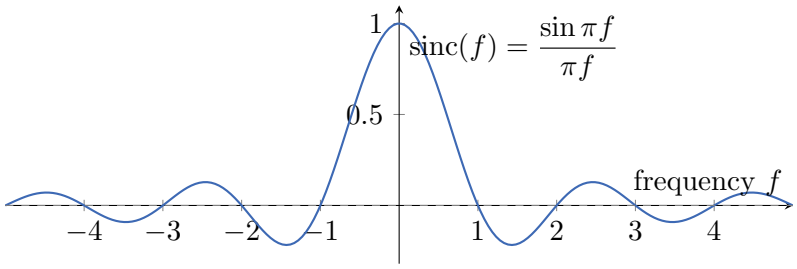


Figure 42.5: The sinc function, the Fourier transform of the unit rectangular pulse. It crosses zero at every nonzero integer and decays as $1/f$. This single curve governs spectral leakage, windowing, and ideal-filter ringing throughout the book.

42.4.3 The delta function: an infinitely thin spike

We need an idealized impulse: something that is zero everywhere except at one instant, yet has total area 1. No ordinary function can do this, so the *Dirac delta* $\delta(t)$ is not a function in the usual sense but a *distribution* – defined entirely by how it behaves inside an integral. Its one defining property is the *sifting property*:

$$\int_{-\infty}^{\infty} x(t)\,\delta(t - t_0)\,\mathrm{d}t = x(t_0).$$

Sliding a delta to time t_0 and integrating against x "sifts out" the single value $x(t_0)$. Picture δ as the limit of a tall narrow rectangle of area 1 as its width goes to zero and its height to infinity; the area is pinned at 1 no matter how thin it gets.

Common trap

Do not ask for the *value* of $\delta(0)$ – it is not a number, and chasing it leads nowhere. The delta only ever means something *under an integral*. Treat δ as a verb ("sample here"), not a noun ("a value"). Every legitimate use in this book either integrates against it or convolves with it.

What is really happening

The delta is the bridge between continuous and discrete worlds. Sampling a continuous signal is modeled as multiplying it by an infinite comb of deltas spaced T_s apart, $\sum_n \delta(t - nT_s)$: each delta sifts out one sample value. Convolving any signal with a single $\delta(t)$ leaves it unchanged – δ is the identity for convolution – which is why the impulse response (the output when the input is a delta) completely characterizes an LTI system. The spike that seems too pathological to be a function turns out to be the most useful test signal in the field.

Connection

Limits, the Riemann integral, the fundamental theorem of calculus, and a careful treatment of distributions are the subject of *Calculus From the Ground Up*. This appendix takes the integral as "a continuous sum" and the delta as "the sifting rule"; that book proves both from the foundations.

42.5 Just enough probability for noise

Every real measurement is corrupted. An IMU axis at rest does not read a clean 0 or $1g$; it jitters. An EMG electrode picks up thermal noise, motion artifacts, and mains hum. To reason about how much of a signal we can trust, we describe the random part with probability. We need only the basics.

42.5.1 Random variables, mean, and variance

A *random variable* is a quantity whose value is uncertain, described by a *distribution* that says how likely each value is. Two numbers summarize it for most DSP purposes. The *mean* (or expectation) is the long-run average value,

$$\mu = \mathbb{E}[X],$$

and the *variance* is the average squared deviation from the mean, a measure of spread,

$$\sigma^2 = \mathrm{Var}[X] = \mathbb{E}\left[(X-\mu)^2\right] = \mathbb{E}[X^2] - \mu^2.$$

Its square root σ, the *standard deviation*, has the same units as the signal and is the natural "size" of the noise. For a noise process we usually want $\mu = 0$ (no constant bias) and a small σ (little jitter).

Connection

Notice that the variance is an inner product in disguise: it is the expected value of a product, $\mathbb{E}[(X-\mu)^2]$, exactly as energy was $\langle \mathbf{x}, \mathbf{x} \rangle$ in Section 42.3. This is not a coincidence. *Probability From the Ground Up* develops expectation as an integral against a density, $\mathbb{E}[g(X)] = \int g(x)\,p(x)\,\mathrm{d}x$, which makes the whole subject a special case of the calculus and inner-product machinery you already have. Mean is a projection; variance is a squared length.

42.5.2 The Gaussian, and white noise

The single most important distribution for noise is the *Gaussian* (normal) distribution, with density

$$p(x) = \frac{1}{\sqrt{2\pi}\,\sigma}\,\exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Its bell shape is set entirely by μ (where it is centered) and σ (how wide). It earns its dominance through the *central limit theorem*: when many small independent disturbances add up – as they do in a real sensor – their sum is approximately Gaussian regardless of the details. That is why thermal and electronic noise is so often modeled as Gaussian.

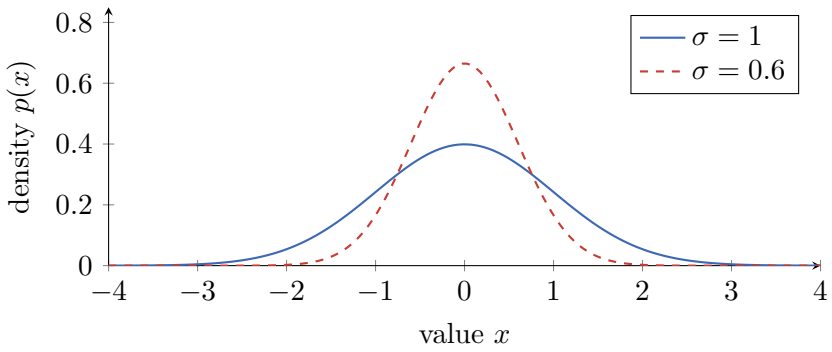


Figure 42.6: Gaussian densities with mean 0. Smaller σ means less spread (a sharper, taller bell) – less noise. About 68% of samples fall within $\pm\sigma$ of the mean and 95% within $\pm 2\sigma$.

White noise is a sequence of random samples that are uncorrelated from one to the next and share the same distribution – typically zero-mean Gaussian. "White" refers to its flat power spectrum: like white light, it contains every frequency in equal measure. It is the standard model for the irreducible jitter floor of a sensor, and the baseline against which we measure how well a filter cleans a signal.

42.5.3 Signal-to-noise ratio

The one figure of merit that ties probability back to engineering is the *signal-to-noise ratio*, the ratio of signal power to noise power, almost always quoted in decibels:

$$\mathrm{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}, \qquad \mathrm{SNR}_{\text{dB}} = 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}}.$$

Noise power is just the variance σ^2 of a zero-mean noise process. A higher SNR means a cleaner signal; averaging N independent measurements of a static quantity reduces noise power by a factor of N (improving SNR by $10 \log_{10} N$ dB), which is the quantitative reason averaging works.

Example 42.5 (Averaging beats down noise)**.** Suppose a stationary forearm gives an accelerometer reading $x[n] = g + w[n]$, where g is the true gravity component and $w[n]$ is zero-mean white noise with standard deviation $\sigma = 0.02g$. A single sample has noise power σ^2. Average $N = 100$ samples: the signal g is unchanged, but the noise variance drops to $\sigma^2/100$, so the standard deviation falls by $\sqrt{100} = 10$, to $0.002g$. The SNR improves by $10 \log_{10} 100 = 20$ dB. This is exactly the trick a stillness detector or a slow-tilt estimator uses on the wrist device – and it costs only a small averaging window.

Connection

Distributions, expectation as an integral, independence, the central limit theorem, and random processes are built from the ground up in *Probability From the Ground Up*. The noise and spectral-estimation chapters of this book (Chapters 20–21) use those ideas in earnest; this section is only the vocabulary needed to start.

Toward the wrist tracker

A clear-eyed view of the wrist tracker starts with noise. The reason a wrist-worn IMU *alone* cannot recover individual finger joint angles is partly geometric – the fingers move distally, far from the wrist sensor – and partly statistical: the tiny mechanical couplings that do reach the wrist are buried below the sensor's noise floor, and no amount of averaging recovers a signal that was never above σ in the first place. Coarse, discrete wrist and hand gestures *are* detectable because they produce large, repeatable signals well above the noise. Continuous finger pose needs sensors closer to the source – EMG or force myography over the forearm muscles, or rings on the fingers themselves. Probability is what lets you state that limit precisely instead of hoping: if the desired signal is below the noise floor, it is gone. See Chapters 34 and 37 for the sensor-by-sensor accounting.

42.6 Exercises

Exercise 42.1. Write $z = -1 + \mathrm{j}\sqrt{3}$ in polar form $re^{\mathrm{j}\theta}$ with $\theta \in (-\pi, \pi]$. Then compute z^3 in polar form and convert the result back to rectangular form. (Hint: atan2 and the quadrant matter.)

Exercise 42.2. Using the inverse Euler formulas, prove the identity $\cos^2\theta = \frac{1}{2} + \frac{1}{2}\cos 2\theta$ by writing $\cos\theta = (e^{\mathrm{j}\theta} + e^{-\mathrm{j}\theta})/2$, squaring, and simplifying. Explain in one sentence what this identity says about the spectrum of a squared sinusoid.

Exercise 42.3. Let $\mathbf{x} = (1, \mathrm{j}, -1, -\mathrm{j})$ in $\mathbb{C}^4$. Compute $\|\mathbf{x}\|^2$ using the complex inner product, being careful to conjugate. Then compute $\langle \mathbf{x}, \mathbf{e}_1 \rangle$ where $e_1[n] = e^{\mathrm{j}2\pi n/4} = \mathrm{j}^n$. What does the result tell you about the frequency content of $\mathbf{x}$?

Exercise 42.4. Show directly that the two DFT-basis vectors of length $N = 4$, $e_1[n] = \mathrm{j}^n$ and $e_3[n] = \mathrm{j}^{3n} = (-\mathrm{j})^n$, are orthogonal under the complex inner product. Why is conjugation essential to getting zero here?

Exercise 42.5. A complex exponential $\mathbf{v}$ with $v[n] = e^{\mathrm{j}\omega n}$ is claimed to be an eigenvector of any LTI filter. Take the simplest filter, a one-sample delay $y[n] = x[n-1]$. Verify that feeding $v[n]$ in produces $\lambda\, v[n]$ and find the eigenvalue λ in terms of ω. What are $|\lambda|$ and $\arg(\lambda)$, and what do they mean physically?

Exercise 42.6. Compute $\int_{-T/2}^{T/2} e^{-\mathrm{j}2\pi f t}\,\mathrm{d}t$ for a window of width T and show it equals $T\,\mathrm{sinc}(fT)$. At which frequencies f does it vanish? Relate this to why a finite observation window smears a single tone across nearby frequencies (spectral leakage).

Exercise 42.7. Use the sifting property to evaluate $\int_{-\infty}^{\infty} (t^2+3)\,\delta(t-2)\,\mathrm{d}t$ and $\int_{-\infty}^{\infty} \cos(\pi t)\,\delta(t-\frac{1}{2})\,\mathrm{d}t$. Then explain, in one or two sentences, why convolving an arbitrary signal $x(t)$ with $\delta(t - t_0)$ shifts it to $x(t - t_0)$.

Exercise 42.8. A zero-mean white-noise sensor channel has standard deviation $\sigma = 0.05g$. (a) What is its noise power? (b) If you average N consecutive samples of a stationary reading, by what factor does the noise standard deviation drop? (c) How many samples must you average to bring the noise standard deviation down to $0.01g$, and by how many decibels does that improve the SNR?

Exercise 42.9. For a Gaussian random variable with mean μ and variance σ^2, roughly what fraction of samples fall within $\pm 2\sigma$ of the mean? If a stillness detector flags "moving" whenever a sample

exceeds $\pm 2\sigma$ of the resting distribution, what false-alarm rate would you expect when the wrist is in fact still, and is that acceptable for triggering a MIDI note?

Exercise 42.10. (Synthesis.) Explain, in a short paragraph each, how each of the four topics in this appendix – complex numbers, inner products/orthonormal bases, integration/the delta, and probability – appears in the single act of computing one windowed FFT of an IMU stream and reading off the dominant motion frequency for gesture detection. Be specific about where each piece enters.